OpenText™ Service Manager

Add lookup functionality to Format Control
Version 9.82

PDF Generated on October 12, 2026

© Copyright 2026 Open Text

Table of Contents

Add lookup functionality to Format Control 23
1 Special processing considerations: Incident Management 23
1.1 Special processing considerations: Change Management 24
1.1.1 Format Control error messages 24
1.1.1.1 Common routines called from Format Control 25
1.2 Links 25
1.2.1 Understanding links 26
1.2.1.1 Advanced link editing features 26
1.2.1.2 Specifying a link query 27
1.2.1.3 Copying fields by name during fill operations 27
1.2.1.4 Scalar/non-scalar field links 27
1.2.1.5 Keeping changes 28
1.2.1.6 Link dependencies within Service Desk 28
1.2.1.7 Document Engine primary link record 29
1.2.1.8 Add a new link file 29
1.2.1.9 Test a link 29
1.2.1.10 Delete a link 30
1.2.1.11 Access advanced link maintenance 30
1.2.1.12 Copy information from structure fields to scalar fields 31
1.2.2 Link maintenance 31
1.2.2.1 Data relationships and the link file 31
1.2.2.2 Types of links 32
1.2.2.3 Find functionality 32
1.2.2.4 Fill functionality 33
1.2.2.5 Fill functionality with multi-select 33
1.2.2.6 Turn off multi-select functionality 34
1.2.2.7 Virtual joining functionality 34
1.2.2.8 Us.link 34
1.2.2.9 Variables used in links 35
1.2.2.10 Calling the us.link 37
1.2.2.11 Skipping query writing 38
1.2.2.12 Accessing $File/dates 39
1.2.2.13 Find from and fill to a $ variable 40
1.2.2.14 Find from and fill to an array structure 40
1.2.2.15 The $fill.display and $fill.display.add functionality 40
1.2.2.16 Access the link record 41
1.2.3 Virtual joins 41
1.2.3.1 Understanding subforms 42
1.2.3.2 Example: Creating a virtual join 42
1.2.3.2.1 Build the sales form 43
1.2.3.2.2 Create the sales file 44
1.2.3.2.3 Create the sales record list 44
1.2.3.2.4 Add data to the sales file 45
1.2.3.2.5 Create the sales1 subform 45
1.2.3.2.6 Create the orders form 46
1.2.3.2.7 Build the virtual join into the form 46
1.2.3.2.8 Build the link 47

1.2.3.2.9 Use the virtual join 48
1.2.3.2.10 Verify the sales1 form works 48
1.2.4 Entity Relationship Diagram creation utility 49
1.2.4.1 Checklist: Creating an ERD record for an application 49
1.2.4.2 What's the required input for ERD Create? 49
1.2.4.3 Create an ERD definition 50
1.2.4.4 Create an ERD record 50
1.2.4.5 Modify an ERD record 51
1.2.4.6 Create manual relationships 51
1.2.4.7 What are primary link records? 52
1.2.4.8 What are related link records? 52
1.2.4.9 Associate a data policy record with an application 52
1.2.4.10 Checklist: Generating a DDL file 53
1.2.4.11 DDL mode 53
1.2.4.12 Generate a DDL file 53
1.2.4.13 Import a DDL file into a database modeling tool 54
1.2.5 Posting 55
1.2.5.1 Posting link records 55
1.2.5.2 Posting link line definition file 56
1.2.5.3 Types of posting available in Format Control 56
1.2.5.4 Confirming the posting routine 57
1.2.5.5 Posting variables 57
1.2.5.6 Automatic updates using Format Control subroutines 58
1.2.5.6.1 Identify field input values for posting 58
1.2.5.6.2 Create the link record for posting 59
1.2.5.6.3 Create the Format Control record for automatic posting 60
1.2.5.6.4 Open a request for posting 61
1.2.5.7 Manual posting using Format Control additional options 62
1.2.5.7.1 Create the Format Control record for manual posting 62
1.2.5.7.2 Modify a hardware Change Request for posting 63
1.2.5.7.3 Use the confirmation function in the posting routine 64
1.2.5.7.4 Do manual posting with confirmation 65
1.3 Display functions 65
1.3.1 Menus 66
1.3.1.1 Menu record 66
1.3.1.2 Button properties 66
1.3.1.3 Access menu records 66
1.3.1.4 Example: Adding a URL link to a menu 67
1.3.1.5 Customize System Navigator menu icons 68
1.3.1.6 Adding and changing the image icon for menu and toolbar items 68
1.3.1.6.1 Add or change the image icon for menu and tool bar items 68
1.3.1.6.2 Common option icon names 69
1.3.2 Display application 74
1.3.2.1 Displayscreen definitions 74
1.3.2.2 Display application option definitions 76
1.3.2.3 Restricting access to display options 81
1.3.2.4 Selecting display options 81
1.3.2.5 Calling an application 81
1.3.2.6 Custom RAD 82
1.3.2.7 Creating displayscreen records 82

1.3.2.8 Displayoption Database Dictionary keys 83
1.3.2.9 Displayevent Database Dictionary keys 83
1.3.2.10 Access display records 83
1.3.2.11 Create a displayscreen record 85
1.3.2.12 Create a displayoption record 85
1.3.2.13 Define display conditions 86
1.3.2.14 Enable the Merge Conflicted Updates function for customized user operations 86
1.4 Advanced functions 88
1.4.1 Service Manager macros 88
1.4.1.1 Macro conditions 88
1.4.1.2 Accessing macro records 89
1.4.1.3 Create a macro 89
1.4.1.4 Definitions for macro forms 90
1.4.1.5 Macros provided with Service Manager 92
1.4.2 Publish and subscribe 94
1.4.2.1 Publish and subscribe workflow 94
1.4.2.2 Static messages 94
1.4.2.2.1 Publish the static message 95
1.4.2.2.2 Modify the form for static messages 95
1.4.2.2.3 Modify the form for a simple marquee 95
1.4.2.2.4 Publish the marquee 95
1.4.2.3 Stored queries for management view publishing 96
1.4.2.4 Agent records for publishing 96
1.4.2.4.1 Create an agent record for publishing 96
1.4.2.5 Scheduling background processes for publishing 96
1.4.2.5.1 Establish the publishing interval in the schedule file 97
1.4.2.5.2 Load agents into the startup record for my system 97
1.4.2.5.3 Start background processes 97
1.4.2.5.4 Stop background processes 97
1.4.2.5.5 Define which system processes manage message traffic 97
1.4.2.6 Marquee processor 98
1.4.2.7 Schedule file 99
1.4.2.8 Subscribing 99
1.4.2.9 Publishing in the RAD environment 99
1.4.3 Scripting 99
1.4.3.1 Scripts 100
1.4.3.2 Script forms 100
1.4.3.3 Script flow 100
1.4.3.4 Scripting processing flow 101
1.4.3.5 Diagramming the script flow 101
1.4.3.6 Using fill boxes in script forms 101
1.4.3.7 Executing scripts 102
1.4.3.8 Script reports 103
1.4.3.9 Access a script record from a menu 103
1.4.3.10 Access a script record from the Database Manager 103
1.4.3.11 Create a script 103
1.4.3.12 Define the scripts 104
1.4.3.13 Define an initial script in an Incident Management profile record 105
1.4.3.14 Execute a script from a displayoption 105
1.4.3.15 Execute a script from Format Control 105

1.4.3.16 Delete a script 106
1.4.3.17 Print a report on a script 106
1.5 Service Manager Web tier 106
1.5.1 Editing Web client Key bindings 107
1.5.2 Using the compact layout 108
1.5.3 Setting web client preferences 108
1.5.4 Branding the web client 110
1.5.5 Web client forms 114
1.5.6 Configuring color indicator 114
1.5.7 Configure the behavior when you right-click in the web client 116
1.5.8 Example: Running custom JavaScript from the web client 116
1.5.9 Example: Sending Web tier URLs through e-mail notifications 116
1.5.10 Example: Notifying specified operators upon an incident update 119
1.5.11 Generating Web tier URL queries 121
1.6 Windows client 122
1.6.1 Administration perspective 123
1.6.2 Preferences 127
1.6.2.1 Set Windows client preferences 127
1.6.2.1.1 Appearance preferences 128
1.6.2.1.2 Service Manager preferences 129
1.6.2.1.3 Log preferences 131
1.6.2.1.4 Security preferences 131
1.6.2.1.5 Spell Checker preferences 133
1.6.2.1.6 Help preferences 134
1.6.2.1.7 Help Server preferences 135
1.6.2.2 Setting Windows client preferences from the server 135
1.6.3 Views 135
1.6.3.1 Views 136
1.6.3.2 Related objects 137
1.6.3.3 Messages view 137
1.6.4 Perspectives 139
1.7 Implementing version control for your tailoring 141
1.7.1 SVN version control process 141
1.7.2 Git version control process 145
1.7.2.1 Prepare Git 146
1.7.2.2 Prepare Service Manager 148
1.7.2.3 Using Feature Tracker 149
1.7.2.4 Feature Tracker use case 156
1.7.3 Version control best practice 156
1.7.4 Applications data file represented in XML 157
1.7.5 Parameters to import and export files 162
1.8 Localization 165
1.8.1 Localize untranslated text strings in Service Manager resource files 166
1.8.2 Localize Service Manager into unsupported languages 169
1.8.3 Service Manager localization rules 171
2 Introduction 174
2.1 System language 175
2.1.1 Data types 175
2.1.1.1 Data type: Primitive 176
2.1.1.2 Data type: Compound 178

2.1.1.2.1 Data type: File/Record 178
2.1.1.2.2 Data type: Offset 179
2.1.1.2.3 Data type: Array 179
2.1.1.2.4 Data type: Structure 180
2.1.1.2.5 Data type: Operator 181
2.1.1.2.6 Data type: Expression 185
2.1.1.2.7 Data type: Pseudo field 186
2.1.1.2.8 Data type: Global variable 186
2.1.1.2.9 Data type: Local variable 186
2.1.2 Reserved words 186
2.1.3 Literals 187
2.1.4 Comments 187
2.1.5 Variables 188
2.1.6 Variable pools 191
2.1.7 Statements 192
2.1.8 List: Command line calls 194
2.1.9 Call a RAD application 205
2.1.10 Locate a RAD function 205
2.1.11 RAD Debugger 206
2.2 Introduction to JavaScript in Service Manager 208
2.2.1 Language overview 209
2.2.2 Learning JavaScript 209
2.2.3 Basic rules of JavaScript 209
2.2.4 Using JavaScript in Service Manager 212
2.2.4.1 JavaScript tailoring guideline 212
2.2.4.2 Avoiding the use of the for...in loop over an array 213
2.2.4.3 Avoiding variable declaration inside a loop 214
2.2.4.4 Counting records 215
2.2.4.5 Return bad validation message in Format Control 215
2.2.4.6 Avoiding the use of getLast() 216
2.2.4.7 Restricting fields selected by a query 216
2.2.4.8 Avoiding the use of assignment instead of comparison 216
2.2.4.9 Accessing system language array data 217
2.2.4.10 Working with system variables 217
2.2.4.11 Accessing a JavaScript class definition from another JavaScript 218
2.2.4.12 Implementing custom logging 219
2.3 Where can I use JavaScript in Service Manager? 221
2.3.1 What's the ScriptLibrary? 222
2.3.1.1 ScriptLibrary editing environment 222
2.3.1.2 Calling a function in the ScriptLibrary 222
2.3.1.3 Example: Function to control close button label in Change Management 223
2.3.1.4 The log4js script 224
2.3.2 Examples of calling JavaScript from different Service Manager tools 225
2.3.2.1 FormatControl 225
2.3.2.2 Links 226
2.3.2.3 Trigger scripts 227
2.3.3 Using RAD functions in JavaScript 230
2.3.3.1 JavaScript function: parse_evaluate() 230
2.3.3.2 RAD function: parse.evaluate 230
2.3.4 JavaScript global properties 231

2.3.5 Order of execution of JavaScript versus RAD language 233
2.3.6 Corresponding JavaScript syntax for RAD language tasks 233
2.4 System Language reference 234
2.4.1 Deleted RAD applications 234
2.4.2 List: RAD functions 236
2.4.2.1 RAD function: add.graphnodes(xml) 242
2.4.2.2 RAD function: cache.flush 242
2.4.2.3 RAD function: clean.display.value 243
2.4.2.4 RAD function: cleanup 243
2.4.2.5 RAD function: contents 244
2.4.2.6 RAD function: copyright 245
2.4.2.7 RAD function: currec 245
2.4.2.8 RAD function: current.device 245
2.4.2.9 RAD function: current.format 246
2.4.2.10 RAD function: cursor.field.contents 246
2.4.2.11 RAD function: cursor.field.display.content 246
2.4.2.12 RAD function: cursor.field.display.content.set 247
2.4.2.13 RAD function: cursor.field.name.set 247
2.4.2.14 RAD function: cursor.field.name 247
2.4.2.15 RAD function: cursor.field.readonly 248
2.4.2.16 RAD function: cursor.line 248
2.4.2.17 RAD function: date 248
2.4.2.18 RAD function: datecmp 249
2.4.2.19 RAD function: day 250
2.4.2.20 RAD function: dayofweek 250
2.4.2.21 RAD function: dayofyear 251
2.4.2.22 RAD function: dbdict.helper("db.type") 251
2.4.2.23 RAD function: dbdict.helper("field.type") 251
2.4.2.24 RAD function: dbdict.helper("is.alias") 253
2.4.2.25 RAD function: dbdict.helper("joinfile.names") 253
2.4.2.26 RAD function: dbdict.helper("unique.key") 254
2.4.2.27 RAD function: delete 254
2.4.2.28 RAD function: denull 255
2.4.2.29 RAD function: descriptor 255
2.4.2.30 RAD function: display.value.copy 255
2.4.2.31 RAD function: evaluate 256
2.4.2.32 RAD function: evaluate.query 256
2.4.2.33 RAD function: exists 257
2.4.2.34 RAD function: fduplicate 257
2.4.2.35 RAD function: filename 258
2.4.2.36 RAD function: filequeryex 258
2.4.2.37 RAD function: frestore 259
2.4.2.38 RAD function: genout 260
2.4.2.39 RAD function: get.base.form.name 260
2.4.2.40 RAD function: get.dateformat 261
2.4.2.41 RAD function: get.display.value 261
2.4.2.42 RAD function: get.graph.action 263
2.4.2.43 RAD function: get.graph.id 263
2.4.2.44 RAD function: get.graph.target 263
2.4.2.45 RAD function: get.graphnode.id 264

2.4.2.46 RAD function: get.lock.owner 264
2.4.2.47 RAD function: get.timezoneoffset 264
2.4.2.48 RAD function: get.uid 265
2.4.2.49 RAD function: get.web.url 265
2.4.2.50 RAD function: gui 265
2.4.2.51 RAD function: index 266
2.4.2.52 RAD function: insert 266
2.4.2.53 RAD function: iscurrent 268
2.4.2.54 RAD function: isExpressionValid 269
2.4.2.55 RAD function: isfileexist 269
2.4.2.56 RAD function: jscall 270
2.4.2.57 RAD function: lioption 271
2.4.2.58 RAD function: lng 273
2.4.2.59 RAD function: locks 273
2.4.2.60 RAD function: logoff 274
2.4.2.61 RAD function: logon 274
2.4.2.62 RAD function: mandant 274
2.4.2.63 RAD function: max 275
2.4.2.64 RAD function: messages 275
2.4.2.65 RAD function: min 277
2.4.2.66 RAD function: modtime 277
2.4.2.67 RAD function: month 278
2.4.2.68 RAD function: multiselect.selection(278
2.4.2.69 RAD function: multiselect.selection(279
2.4.2.70 RAD function: multiselect.selection(279
2.4.2.71 RAD function: multiselect.selection(280
2.4.2.72 RAD function: multiselect.selection(280
2.4.2.73 RAD function: multiselect.selection(280
2.4.2.74 RAD function: null 280
2.4.2.75 RAD function: nullsub 281
2.4.2.76 RAD function: operator 281
2.4.2.77 RAD function: option 282
2.4.2.78 RAD function: parse 282
2.4.2.79 RAD function: perf 282
2.4.2.80 RAD function: printer 283
2.4.2.81 RAD function: policyread 283
2.4.2.82 RAD function: processes 286
2.4.2.83 RAD function: prof 287
2.4.2.84 RAD function: recordcopy 288
2.4.2.85 RAD function: recordtostring 289
2.4.2.86 RAD function: round 289
2.4.2.87 RAD function: same 290
2.4.2.88 RAD function: scmsg 291
2.4.2.89 RAD function: set.display.value 292
2.4.2.90 RAD function: set.timezone 293
2.4.2.91 RAD function: setsort 293
2.4.2.92 RAD function: shutdown 294
2.4.2.93 RAD function: simple.file.load 294
2.4.2.94 RAD function: sleep 295
2.4.2.95 RAD function: str 295

2.4.2.96 RAD function: stradj 296
2.4.2.97 RAD function: strchrcp 296
2.4.2.98 RAD function: strchrin 297
2.4.2.99 RAD function: strclpl 298
2.4.2.100 RAD function: strclpr 298
2.4.2.101 RAD function: strcpy 299
2.4.2.102 RAD function: strdel 300
2.4.2.103 RAD function: strins 300
2.4.2.104 RAD function: strpadl 301
2.4.2.105 RAD function: strpadr 302
2.4.2.106 RAD function: strraw 302
2.4.2.107 RAD function: strrep 303
2.4.2.108 RAD function: strtml 303
2.4.2.109 RAD function: strtrmr 304
2.4.2.110 RAD function: substr 304
2.4.2.111 RAD function: substrb 305
2.4.2.112 RAD function: sysinfo.get(306
2.4.2.113 RAD function: sysinfo.get(307
2.4.2.114 RAD function: sysinfo.get(307
2.4.2.115 RAD function: sysinfo.get(307
2.4.2.116 RAD function: sysinfo.get(307
2.4.2.117 RAD function: sysinfo.get(307
2.4.2.118 RAD function: sysinfo.get(308
2.4.2.119 RAD function: sysinfo.get(308
2.4.2.120 RAD function: sysinfo.get(308
2.4.2.121 RAD function: sysinfo.get(308
2.4.2.122 RAD function: sysinfo.get(309
2.4.2.123 RAD function: sysinfo.get(309
2.4.2.124 RAD function: sysinfo.get(309
2.4.2.125 RAD function: sysinfo.get(309
2.4.2.126 RAD function: sysinfo.get(310
2.4.2.127 RAD Function: sysinfo.get(310
2.4.2.128 RAD function: sysinfo.get(310
2.4.2.129 RAD function: sysinfo.get(311
2.4.2.130 RAD function: sysinfo.get(311
2.4.2.131 RAD function: sysinfo.get(311
2.4.2.132 RAD function: sysinfo.get(311
2.4.2.133 RAD function: sysinfo.get(311
2.4.2.134 RAD function: sysinfo.get(312
2.4.2.135 RAD function: sysinfo.get(312
2.4.2.136 RAD function: sysinfo.get(312
2.4.2.137 RAD function: sysinfo.get(312
2.4.2.138 RAD function: sysinfo.get(312
2.4.2.139 RAD function: sysinfo.get(312
2.4.2.140 RAD function: sysinfo.get(313
2.4.2.141 RAD function: sysinfo.get(313
2.4.2.142 RAD function: time 313
2.4.2.143 RAD function: tod 314
2.4.2.144 RAD function: tolower 314
2.4.2.145 RAD function: toupper 314

2.4.2.146 RAD function: translate 315
2.4.2.147 RAD function: trunc 315
2.4.2.148 RAD function: type 316
2.4.2.149 RAD function: updatestatus 316
2.4.2.150 RAD function: val 317
2.4.2.151 RAD function: version 318
2.4.2.152 RAD function: year 319
2.4.3 List: rtecalls() 319
2.4.3.1 rtecall(321
2.4.3.2 rtecall(322
2.4.3.3 rtecall(322
2.4.3.4 rtecall(323
2.4.3.5 rtecall(324
2.4.3.6 rtecall(325
2.4.3.7 rtecall(326
2.4.3.8 rtecall(326
2.4.3.9 rtecall(327
2.4.3.10 rtecall(328
2.4.3.11 rtecall(329
2.4.3.12 rtecall(329
2.4.3.13 rtecall(330
2.4.3.14 rtecall(331
2.4.3.15 rtecall(332
2.4.3.16 rtecall(332
2.4.3.17 rtecall(333
2.4.3.18 rtecall(334
2.4.3.19 rtecall(334
2.4.3.20 rtecall(335
2.4.3.21 rtecall(336
2.4.3.22 rtecall(336
2.4.3.23 rtecall(337
2.4.3.24 rtecall(338
2.4.3.25 rtecall(338
2.4.3.26 rtecall(338
2.4.3.27 rtecall(339
2.4.3.28 rtecall(340
2.4.3.29 rtecall (341
2.4.3.30 rtecall (342
2.4.3.31 rtecall(343
2.4.3.32 rtecall(344
2.4.3.33 rtecall(345
2.4.3.34 rtecall(345
2.4.3.35 rtecall(346
2.4.4 List: RAD routines 347
2.4.4.1 RAD routine: as.copy 350
2.4.4.2 RAD routine: as.delete 352
2.4.4.3 RAD routine: as.get.name 354
2.4.4.4 RAD routine: as.insert 354
2.4.4.5 RAD routine: as.move 356
2.4.4.6 RAD routine: as.options 358

2.4.4.7 RAD routine: as.sort 359
2.4.4.8 RAD routine: axces.apm 361
2.4.4.9 RAD routine: axces.cm3.cts.write 362
2.4.4.10 RAD routine: axces.cm3 362
2.4.4.11 RAD routine: axces.database 363
2.4.4.12 RAD routine: axces.email 364
2.4.4.13 RAD routine: axces.move.intoout 365
2.4.4.14 RAD routine: axces.rm 365
2.4.4.15 RAD routine: axces.sap.hybrid.evin 366
2.4.4.16 RAD routine: axces.sm 367
2.4.4.17 RAD routine: axces.software 367
2.4.4.18 RAD routine: axces.write 368
2.4.4.19 RAD routine: database 369
2.4.4.20 RAD routine: es.approval 370
2.4.4.21 RAD routine: fill.fc 370
2.4.4.22 RAD routine: getnumb.fc 371
2.4.4.23 RAD routine: marquee.publish 373
2.4.4.24 RAD routine: marquee.send 374
2.4.4.25 RAD routine: message.fc 375
2.4.4.26 RAD routine: post.fc 377
2.4.4.27 RAD routine: publish 377
2.4.4.28 RAD routine: query.stored 378
2.4.4.29 RAD routine: scauto.inventory 378
2.4.4.30 RAD routine: sort.array 379
2.4.4.31 RAD routine: us.js.call 380
2.4.4.32 RAD routine: validate.fields 380
2.4.4.33 RAD routine: wmi.inventory.check 381
2.5 JavaScript and Service Manager reference 382
2.5.1 JavaScript global System objects 382
2.5.1.1 JavaScript object: system.files 383
2.5.1.2 JavaScript object: system.forms 384
2.5.1.3 JavaScript object: system.functions 384
2.5.1.4 JavaScript object: system.functions.scmsg 385
2.5.1.5 JavaScript object: system.library 386
2.5.1.6 JavaScript object: system.oldrecord 387
2.5.1.7 JavaScript object: system.record 388
2.5.1.8 JavaScript object: system.sysinfo 388
2.5.1.9 JavaScript object: system.threads 389
2.5.1.10 JavaScript object: system.user 389
2.5.1.11 JavaScript object: system.users 390
2.5.1.12 JavaScript object: system.vars 390
2.5.2 List: JavaScript global methods 391
2.5.2.1 JavaScript global method: base64Decode 392
2.5.2.2 JavaScript global method: base64Encode 392
2.5.2.3 JavaScript global method: compile 393
2.5.2.4 JavaScript global method: doHTTPRequest 394
2.5.2.5 JavaScript global method: doSOAPRequest 399
2.5.2.6 JavaScript global method: execute 401
2.5.2.7 JavaScript global method: getLog 402
2.5.2.8 JavaScript global method: help 403

2.5.2.9 JavaScript global method: insertDatum 404
2.5.2.10 JavaScript global method: makeSCWebURL 406
2.5.2.11 JavaScript global method: print 408
2.5.2.12 JavaScript global method: RCtoString 409
2.5.2.13 JavaScript global method: readFile 410
2.5.2.14 JavaScript global method: rteJSONParse 411
2.5.2.15 JavaScript global method: rteJSONStringify 411
2.5.2.16 JavaScript global method: setAppMessage 412
2.5.2.17 JavaScript global method: stripHtml 413
2.5.2.18 JavaScript global method: uncompressFile 413
2.5.2.19 JavaScript global method: writeAttachmentToFile 414
2.5.2.20 JavaScript global method: writeFile 416
2.5.2.21 JavaScript global method: xmlstring 417
2.5.3 Service Manager defined JavaScript objects 417
2.5.3.1 JavaScript object: Attachment 418
2.5.3.2 JavaScript object: Header 420
2.5.3.3 JavaScript object: SCDatum 421
2.5.3.3.1 JavaScript method: SCDatum.getSize() 422
2.5.3.3.2 JavaScript method: SCDatum.getText() 423
2.5.3.3.3 JavaScript method: SCDatum.getType() 424
2.5.3.3.4 JavaScript method: SCDatum.getXML() 425
2.5.3.3.5 JavaScript method: SCDatum.join() 426
2.5.3.3.6 JavaScript method: SCDatum.JSDate() 427
2.5.3.3.7 JavaScript method: SCDatum.length() 428
2.5.3.3.8 JavaScript method: SCDatum.pop() 429
2.5.3.3.9 JavaScript method: SCDatum.push() 430
2.5.3.3.10 JavaScript method: SCDatum.setType() 431
2.5.3.3.11 JavaScript method: SCDatum.setValue() 432
2.5.3.3.12 JavaScript method: SCDatum.shift() 433
2.5.3.3.13 JavaScript method: SCDatum.toArray() 434
2.5.3.3.14 JavaScript method: SCDatum.unshift() 435
2.5.3.4 JavaScript object: SCFile 436
2.5.3.4.1 JavaScript method: SCFile.deleteAttachment(attachID) 438
2.5.3.4.2 JavaScript method: SCFile.deleteAttachments() 439
2.5.3.4.3 JavaScript method: SCFile.doAction() 439
2.5.3.4.4 JavaScript method: SCFile.doClose() 441
2.5.3.4.5 JavaScript method: SCFile.doCount() 442
2.5.3.4.6 JavaScript method: SCFile.doDelete() 443
2.5.3.4.7 JavaScript method: SCFile.doInsert() 444
2.5.3.4.8 JavaScript method: SCFile.doPurge() 445
2.5.3.4.9 JavaScript method: SCFile.doRemove() 447
2.5.3.4.10 JavaScript method: SCFile.doSave() 448
2.5.3.4.11 JavaScript method: SCFile.doSelect() 448
2.5.3.4.12 JavaScript method: SCFile.doSelectEx() 450
2.5.3.4.13 JavaScript method: SCFile.doUpdate() 451
2.5.3.4.14 JavaScript method: SCFile.getAttachment(attachID) 452
2.5.3.4.15 JavaScript method: SCFile.getattachments() 452
2.5.3.4.16 JavaScript method: SCFile.getBinary() 453
2.5.3.4.17 JavaScript method: SCFile.getFirst() 453
2.5.3.4.18 JavaScript method: SCFile.getLast() 454

2.5.3.4.19 JavaScript method: SCFile.getLastRC() 455
2.5.3.4.20 JavaScript method: SCFile.getMessages() 456
2.5.3.4.21 JavaScript method: SCFile.getNext() 457
2.5.3.4.22 JavaScript method: SCFile.getPrev() 458
2.5.3.4.23 JavaScript method: SCFile.getPurgedRowCount() 459
2.5.3.4.24 JavaScript method: SCFile.insertattachment() 460
2.5.3.4.25 JavaScript method: SCFile.isRecord() 461
2.5.3.4.26 JavaScript method: SCFile.setBinary() 462
2.5.3.4.27 JavaScript method: SCFile.setFields() 463
2.5.3.4.28 JavaScript method: SCFile.setOrderBy() 464
2.5.3.4.29 JavaScript method: SCFile.setRecord() 466
2.5.3.4.30 JavaScript method: SCFile.updateAttachment(attachObj) 467
2.5.3.5 JavaScript object: SCRecordList 468
2.5.3.6 JavaScript object: XML 471
2.5.3.6.1 JavaScript method: XML.addAttribute() 473
2.5.3.6.2 JavaScript method: XML.addElement() 476
2.5.3.6.3 JavaScript method: XML.appendNode() 478
2.5.3.6.4 JavaScript method: XML.createNode() 479
2.5.3.6.5 JavaScript method: XML.getAttributeNode() 481
2.5.3.6.6 JavaScript method: XML.getAttributeValue() 483
2.5.3.6.7 JavaScript method: XML.getDocumentElement() 485
2.5.3.6.8 JavaScript method: XML.getFirstAttribute() 486
2.5.3.6.9 JavaScript method: XML.getFirstChildElement() 489
2.5.3.6.10 JavaScript method: XML.getName() 492
2.5.3.6.11 JavaScript method: XML.getNextAttribute() 493
2.5.3.6.12 JavaScript method: XML.getNextSiblingElement() 497
2.5.3.6.13 JavaScript method: XML.getNodeName() 499
2.5.3.6.14 JavaScript method: XML.getNodeType() 500
2.5.3.6.15 JavaScript method: XML.getNodeValue() 501
2.5.3.6.16 JavaScript method: XML.getParentNode() 504
2.5.3.6.17 JavaScript method: XML.getPrefix() 506
2.5.3.6.18 JavaScript method: XML.getQualifiedName() 510
2.5.3.6.19 JavaScript method: XML.getText() 511
2.5.3.6.20 JavaScript method: XML.getValue() 514
2.5.3.6.21 JavaScript method: XML.importNode() 516
2.5.3.6.22 JavaScript method: XML.isDocumentElement() 518
2.5.3.6.23 JavaScript method: XML.removeAttribute() 520
2.5.3.6.24 JavaScript method: XML.setAttributeValue() 520
2.5.3.6.25 JavaScript method: XML.setContent() 523
2.5.3.6.26 JavaScript method: XML.setNodeValue() 524
2.5.3.6.27 JavaScript method: XML.setText() 527
2.5.3.6.28 JavaScript method: XML.setValue() 529
2.5.3.6.29 JavaScript method: XML.toXMLString() 531
2.5.3.6.30 JavaScript method: XMLDoc.insertBefore() 531
2.5.3.7 JavaScript object: XMLDate 532
2.5.3.7.1 JavaScript method: XMLDate.addDuration() 534
2.5.3.7.2 JavaScript method: XMLDate.getDate() 534
2.5.3.7.3 JavaScript method: XMLDate.getDatum() 535
2.5.3.7.4 JavaScript method: XMLDate.getGMTSCDateTimeString() 535
2.5.3.7.5 JavaScript method: XMLDate.getISODate() 536

2.5.3.7.6 JavaScript method: XMLDate.getISODateTimeString() 536
2.5.3.7.7 JavaScript method: XMLDate.getISODateTimeWithSMTZString() 537
2.5.3.7.8 JavaScript method: XMLDate.getISODay() 537
2.5.3.7.9 JavaScript method: XMLDate.getISOMonth() 538
2.5.3.7.10 JavaScript method: XMLDate.getISOTime() 538
2.5.3.7.11 JavaScript method: XMLDate.getISOYear() 539
2.5.3.7.12 JavaScript method: XMLDate.getSCDateTimeString() 539
2.5.3.7.13 JavaScript method: XMLDate.JSDate() 539
2.5.3.7.14 JavaScript method: XMLDate.toSCDuration() 540
2.5.4 List: JavaScript functions 540
2.5.4.1 JavaScript function: add_graphnodes 541
2.5.4.2 JavaScript function: checkAttachmentLegal 541
2.5.4.3 JavaScript function: get_graph_action 542
2.5.4.4 JavaScript function: get_graph_id 542
2.5.4.5 JavaScript function: get_graph_node_id 542
2.5.4.6 JavaScript function: get_graph_target 542
2.5.5 List: JavaScript functions in the ScriptLibrary 542
2.5.5.1 JavaScript function: addToGroup 543
2.5.5.2 JavaScript function: checkAllowedOption 547
2.5.5.3 JavaScript function: checkVersionString 548
2.5.5.4 JavaScript function: isfileexist 552
2.5.5.5 JavaScript function: isInGroup 553
2.5.5.6 JavaScript function: removeFromGroup 554
2.5.5.7 JavaScript function: returnGroupMembers 558
2.5.5.8 JavaScript function: returnMyGroups 559
2.5.5.9 JavaScript function: skipApproval 561
2.5.6 List: JavaScript examples 561
2.5.6.1 Example: Getting a list of logged on users 563
2.5.6.2 Example: Iterating through a complex XML document 563
2.5.6.3 Example: Query a record using the SCFile object 564
2.5.6.4 Example: Testing a file name 565
2.5.6.5 Example: Updating a field with the current date and time 565
2.5.6.6 Example: Working with structured arrays 566
3 Document Engine 567
3.1 Document Engine overview 567
3.2 Objects 569
3.2.1 Create Objects 569
3.2.2 Object Definition form and fields 569
3.2.2.1 Object Info tab field descriptions 570
3.2.2.2 Locking tab field descriptions 572
3.2.2.3 Revisions tab field descriptions 572
3.2.2.4 Variable and Global Lists tab field descriptions 573
3.2.2.5 Activities tab field descriptions 573
3.2.2.6 Alerts tab field descriptions 574
3.2.2.7 Approvals tab field descriptions 575
3.2.2.8 Manage Queues tab field descriptions 577
3.2.2.9 Views and Templates tab field descriptions 579
3.2.2.10 Notifications tab field descriptions 579
3.2.2.11 Search Configuration tab field descriptions 579
3.2.2.12 Defined Queries tab 580

3.2.2.13 Ranges tab 580
3.3 States 581
3.4 Processes 583
3.4.1 Create and modify a Process 583
3.4.2 Process Definition form and field descriptions 583
3.5 Document Engine resources 586
3.5.1 Database Manager 586
3.5.2 RAD applications 586
3.5.3 RAD applications flow 586
3.5.4 Base functions in se.search.engine 588
3.5.5 Base functions in se.view.engine 590
3.5.6 Base functions in se.list.engine 592
3.5.7 Local variables 592
3.6 Detect issues when using the Document Engine 593
3.7 Work order example overview 594
3.7.1 Create a table 595
3.7.2 Add key fields to a table 596
3.7.3 Create a form 597
3.7.4 Create a copy of a form 598
3.7.5 Create a link for the WorkOrder form 598
3.7.6 Create a sequential number file 599
3.7.7 Create an Object definition 599
3.7.8 Create an Initialization Process definition 600
3.7.9 Create display application screen definition 601
3.7.10 Create display application option definitions 602
3.7.11 Create a State definition 607
3.7.12 Add a Close button 608
3.7.13 Create a wizard for the work order 610
3.7.14 Add a Process definition record 612
3.7.15 Create an input form for the wizard 613
3.7.16 Modify the close and update incident forms 614
3.7.17 Create alias in probsummary table for link 616
3.7.18 Modify im.set.close Process definition 617
3.7.19 Test the work order example 618
4 Web Services 619
5 Service Manager Integration Suite (SMIS) 619
5.1 Overview 619
5.2 Develop an Integration Template 623
5.3 Processing Logic of the Mapping Functionality 629
6 Introduction to tailoring best practices 630
6.1 Assumptions 630
6.2 What's tailoring? 630
6.3 Form tailoring 631
6.3.1 What's a form? 631
6.3.2 Form tailoring use cases 631
6.3.3 Selecting values from options in a control 632
6.3.3.1 Combo Box 632
6.3.3.2 Comfill 633
6.3.3.3 Auto complete for Comfill fields 634
6.3.4 Selecting values from options on a form 637

6.3.5 Changing the options available based on prior user input 639
6.3.6 Displaying related data 641
6.3.7 Grouping related controls 642
6.3.8 Typing text into a control 644
6.3.9 Managing global lists 646
6.3.10 Reducing upgrade conflicts 647
6.3.11 HTML formatting for Activities feature 647
6.4 Queries 656
6.4.1 How do I write a good query? 656
6.4.2 Where are queries used? 660
6.5 Database (RDBMS) mapping 661
6.5.1 How do I create good RDBMS mappings? 661
6.5.2 Database dictionary (DBDICT) utility 661
6.5.3 Mapping rights 661
6.5.4 Array mapping options 662
6.5.5 Example: Remapping an array 667
6.5.5.1 Existing array mapping 667
6.5.5.2 Existing query performance 668
6.5.5.3 Prepare to re-map the array 669
6.5.5.4 Remap array fields to an alias table 672
6.5.5.5 New array mapping 673
6.5.5.6 New array query performance 673
6.5.6 Valid data type changes 674
6.5.7 When are RDBMS mappings added or updated? 674
7 Process Designer Tailoring Best Practices 675
7.1 Best practices for configuring rule sets 676
7.1.1 Configure assignment rules 676
7.1.2 Run action rules 681
7.1.2.1 Configure Run Action rules on related records 681
7.1.2.2 Configure Run Action rules on other records 684
7.1.2.3 Configure Run Action rules on the current record 687
7.1.2.4 Configure Run Scheduled Action rules 689
7.1.2.5 Combine the Run Action rules and the Run Scheduled Action rules 691
7.2 Best practices for configuring task planner 693
7.3 Best practices for using the condition editor 696
7.4 Consider rule set execution sequence 699
7.5 Best practices for tailoring Service Manager Codeless module 707
7.5.1 Create a Security Module 707
7.5.2 Configure Security Areas for the module 708
7.5.3 Configure the Security Roles 710
7.5.4 Assign Security Roles to an operator 710
7.5.5 Configure the dbdict and the data policy 710
7.5.6 Configure a Process Designer object 712
7.5.7 Configure a workflow 713
7.5.7.1 Workflow Properties 713
7.5.7.2 Workflow-based rule sets and actions 714
7.5.7.3 Workflow backend transitions 714
7.5.7.4 Configure workflow phases 715
7.5.7.5 Configure transitions between workflow phases 717
7.5.8 Configure rule sets and actions 724

7.5.8.1 Configure rule sets at various levels 724
7.5.8.2 Configure rule sets at various triggering events 724
7.5.8.3 Configure actions at various levels 734
7.5.9 Migrate legacy code to Process Designer 734
7.5.9.1 Enable a workflow 734
7.5.9.2 Configure the workflow 735
7.5.9.3 Enable Process Designer role-based security 736
7.5.9.4 Migrate the Format Control to rule sets 736
7.5.9.4.1 Mandatory validation 737
7.5.9.4.2 Validation against a table 739
7.5.9.4.3 Set a field value 740
7.5.9.4.4 Run a Wizard 741
7.5.9.4.5 Run Javascript 742
7.5.9.4.6 Additional supported rule types 743
7.5.9.5 Migrate the display options to actions 743
8 Wizards 746
8.1 Wizards Overview 746
8.2 Wizard fields 749
8.2.1 Wizard order of execution 749
8.2.2 About variables 749
8.2.3 Variables to avoid using in wizards 749
8.2.4 Wizard Info tab field definitions 750
8.2.5 File Selection tab field definitions 753
8.2.6 Usage tab field definitions 755
8.2.7 Actions tab field definitions 759
8.2.8 Messaging tab field definitions 762
8.2.9 Variables tab field definitions 763
8.2.10 Next Wizard tab field definitions 763
8.2.11 Previous Wizard tab field definitions 763
8.2.12 Comments tab field definitions 763
8.2.13 Cancel Expressions tab field definitions 764
8.3 Calling Wizards 764
8.3.1 Call a wizard from menus 764
8.3.2 Call a wizard from display options 765
8.3.3 Call a wizard from Format Control 766
8.3.4 Call a wizard from process records 766
8.4 Example: Creating a Wizard that creates a new SLA 766
8.4.1 Adding Service Level Targets to the SLA 767
8.4.2 Designing the SLA wizard 767
8.4.3 Planning the SLA wizard panels 768
8.4.4 Building the Create SLA wizard 769
8.4.4.1 Step 1: Define the start node of the wizard 770
8.4.4.2 Step 2: Prompt the user to select SLA type 771
8.4.4.3 Step 3: Prompt for customer information 772
8.4.4.4 Step 4: Prompt for SLA title and description 777
8.4.4.5 Step 5: Prompt for effective dates 780
8.4.4.6 Step 6: Ask about agreement documents 783
8.4.4.7 Step 7: Prompt to add agreements 787
8.4.4.8 Step 8: Save new SLA and prompt for SLOs 789
8.4.4.9 Step 9: Call the Availability SLT wizard 795

8.4.4.10 Step 10: Prompt for another Availability SLT 797
8.4.4.11 Step 11: Call the Response SLT wizard 800
8.4.4.12 Step 12: Prompt for another Response SLT 801
8.4.4.13 Step 13: Display new SLA number and title 805
8.4.4.14 Step 14: Set menu to call the new SLA wizard 809
9 Service Manager processes and best practices 809
9.1 Service Desk overview 809
9.1.1 Service Desk within the ITIL framework 809
9.1.2 Service Desk application 810
9.1.3 Service Desk process overview 810
9.1.4 Input and output for Service Desk Interaction Management 813
9.1.5 Key performance indicators for Service Desk Interaction Management 813
9.1.6 RACI matrix for Service Desk Interaction Management 815
9.2 Service Desk workflows 815
9.2.1 Self-Service by User (process SO 0.1) 815
9.2.2 Interaction Handling (process SO 0.2) 818
9.2.3 Interaction Matching and Escalation (process SO 0.3) 821
9.2.4 Interaction Closure (process SO 0.4) 824
9.2.5 Withdraw Interaction (process SO 0.5) 825
9.3 Service Desk details 826
9.3.1 New interaction form 827
9.3.2 Interaction form after escalation 827
9.3.3 Service Desk Interaction Management form details 828
9.3.4 Interaction categories 838
9.3.5 Escalate Interaction solution matching 841
9.4 Incident Management overview 841
9.4.1 Incident Management within the ITIL framework 841
9.4.2 Incident Management application 842
9.4.3 Incident Management process overview 843
9.4.3.1 Incident Management categories 843
9.4.3.2 Incident Management phases 843
9.4.3.3 Incident Management tasks 843
9.4.3.4 Incident Task phases 844
9.4.3.5 Incident Management user roles 844
9.4.4 Input and output for Incident Management 845
9.4.5 Key performance indicators for Incident Management 846
9.4.6 RACI matrix for Incident Management 848
9.5 Incident Management workflows 848
9.5.1 Incident Logging and Categorization (process SO 2.1) 849
9.5.2 Incident Assignment (process SO 2.2) 851
9.5.3 Incident Investigation and Diagnosis (process SO 2.3) 854
9.5.4 Incident Resolution and Recovery (process SO 2.4) 857
9.5.5 Incident Review and Closure (process SO 2.5) 860
9.5.6 Incident Escalation (process SO 2.6) 862
9.5.7 SLA Monitoring (process SO 2.7) 865
9.5.8 OLA and UC Monitoring (process SO 2.8) 868
9.5.9 Complaint Handling (process SO 2.9) 871
9.6 Incident Management details 873
9.6.1 Incident form after escalation from Service Desk 873
9.6.2 Categorize incident form 875

9.6.3 Investigate incident form 875
9.6.4 Recover incident form 876
9.6.5 Review incident form 877
9.6.6 Incident Management form details 878
9.7 Problem Management overview 887
9.7.1 Problem Management within the ITIL framework 887
9.7.2 Problem Management application 888
9.7.3 Problem Management process overview 889
9.7.4 Input and output for Problem Management 891
9.7.5 Key performance indicators for Problem Management 892
9.7.6 RACI matrix for Problem Management 894
9.8 Problem Management workflows 894
9.8.1 Problem Detection, Logging, and Categorization (process SO 4.1) 894
9.8.2 Problem Investigation and Diagnosis (process SO 4.2) 899
9.8.3 Problem Resolution (process SO 4.3) 905
9.8.4 Problem Review and Closure (process SO 4.4) 911
9.8.5 Problem and Known Error Monitoring (process SO 4.5) 913
9.9 Problem Management details 916
9.9.1 Problem form after escalation from incident 916
9.9.2 Problem form details 922
9.10 Change Management overview 928
9.10.1 Change Management within the ITIL framework 929
9.10.2 Change Management application 929
9.10.3 Change Management process overview 930
9.10.3.1 Change categories and phases 930
9.10.3.2 Change Management categories 930
9.10.3.3 Change Management phases 931
9.10.3.4 Change Management tasks 931
9.10.3.5 Change Management user roles 932
9.10.4 Input and output for Change Management 934
9.10.5 RACI matrix for Change Management 937
9.11 Change Management workflows 940
9.11.1 Change Proposal (process ST 2.0) 940
9.11.2 Register RFC (process ST 2.1) 943
9.11.3 Standard Change (process ST 2.2) 946
9.11.4 Normal Change (process ST 2.3) 948
9.11.5 Emergency Change (process ST 2.4) 953
9.11.6 Review and Close Change (process ST 2.5) 955
9.12 Change Management details 957
9.12.1 Change Management form after escalation from a problem 957
9.12.2 Change Management form details 958
9.13 Knowledge Management overview 964
9.13.1 Knowledge Management Within The ITIL Framework 965
9.13.2 Knowledge Management Application 965
9.13.2.1 Types of Knowledge Documents in Service Manager 965
9.13.2.2 Knowledge Management Categories 965
9.13.2.3 Knowledge Groups 966
9.13.2.4 Knowledge Management Profiles 966
9.13.2.5 Generic OOB Knowledge Management Document Phases 967
9.13.3 Knowledge Management Process Overview 968

9.13.4 Key Interfaces with Other Processes 971
9.13.5 Key performance indicators for Knowledge Management 971
9.13.6 RACI Matrix For Knowledge Management 972
9.14 Knowledge Management workflows 972
9.14.1 Contribute and Approve Knowledge Document (process ST 7.1) 973
9.14.2 Search, View and Utilize Knowledge Document (Process ST 7.2) 976
9.14.3 Maintain and Continually Improve Knowledge (Process ST 7.3) 978
9.15 Knowledge Management details 980
9.16 Configuration Management overview 982
9.16.1 Configuration Management within the ITIL framework 983
9.16.2 Configuration Management application 983
9.16.3 Universal Configuration Management Database 984
9.16.3.1 Baselines 984
9.16.3.2 Managed state 986
9.16.3.3 Actual state 987
9.16.3.4 CI relationships 988
9.16.4 Configuration Management process overview 989
9.16.5 Input and output for Configuration Management 991
9.16.6 Key performance indicators for Configuration Management 992
9.16.7 RACI matrix for Configuration Management 994
9.17 Configuration Management workflows 994
9.17.1 Configuration Management Planning (process ST 3.1) 994
9.17.2 Configuration Identification (process ST 3.2) 998
9.17.3 Configuration Control (process ST 3.3) 1001
9.17.4 Configuration Status Accounting and Reporting (process ST 3.4) 1004
9.17.5 Configuration Verification and Audit (process ST 3.5) 1008
9.17.6 Master data management (process ST 3.6) 1012
9.18 Configuration Management Details 1014
9.18.1 MyDevices configuration item form 1014
9.18.2 Configuration Management form details 1015
9.18.2.1 Configuration Item types and subtypes 1021
9.18.2.2 Managed State subsections 1024
9.19 Request Fulfillment overview 1026
9.19.1 Request Fulfillment within the ITIL framework 1026
9.19.2 Request Fulfillment application 1027
9.19.2.1 Differences between Request Fulfillment and Change Management 1027
9.19.2.2 Key elements of Request Fulfillment 1027
9.19.3 Request Fulfillment process overview 1028
9.19.4 Input and output for Request Fulfillment 1031
9.19.5 Key performance indicators for Request Fulfillment 1031
9.19.6 RACI matrix for Request Fulfillment 1033
9.20 Request Fulfillment workflows 1033
9.20.1 Receive Request (process SO 3.1) 1033
9.20.2 Request Logging (process SO 3.2) 1035
9.20.3 Request Authorization (process SO 3.3) 1036
9.20.4 Request Fulfillment (process SO 3.4) 1037
9.20.5 Request Review (process SO 3.5) 1040
9.20.6 Request Closure (process SO 3.6) 1041
9.21 Request Fulfillment details 1042
9.21.1 Request Fulfillment process flow 1042

9.21.2 Request Fulfillment forms 1044
9.21.3 Request Fulfillment form details 1047
9.21.4 Request task forms 1052
9.21.5 Request task form details 1057
9.22 Service Level Management overview 1061
9.22.1 Service Level Management within the ITIL Framework 1062
9.22.2 Service Level Management Application 1062
9.22.3 Input and output for Service Level Management 1062
9.22.4 Key performance indicators for Service Level Management 1063
9.22.5 Service Level Management Roles 1065
9.22.6 RACI matrix for Service Level Management 1067
9.23 Service Level Management workflows 1067
9.23.1 Updating the Service Catalog (SD 2.1) 1069
9.23.2 Create or Amend IT Services (SD 2.2) 1071
9.23.3 SLRs and draft OLAs and UCs (SD 2.3) 1074
9.23.4 Finalize SLRs/OLAs/UCs (SD 2.4) 1077
9.23.5 Monitor Service Levels (SD 2.5) 1079
9.23.6 Service Level Reporting (SD 2.6) 1082
9.23.7 Perform Service Reviews (SD 2.7) 1084
9.23.8 Service Improvement Plan (SD 2.8) 1088
9.23.9 Review and Revise SLAs/OLAs/UCs (SD 2.9) 1092
9.24 Service Level Management details 1096
9.24.1 The Workflow for Agreements, Underpinning Contracts 1097
9.24.2 Creating a new SLA, OLA or UC 1098
9.24.3 The Draft phase 1099
9.24.4 The Review phase 1099
9.24.5 The Agreed phase 1100
9.24.6 The Expired phase 1102
9.24.7 Service Reviews and usages of SIPs/SQPs 1103
9.24.8 Service Level Management forms 1104
9.24.9 Service Level Management form details 1108
9.25 Release and Deployment Management 1113
9.25.1 Release and Deployment Management Overview 1113
9.25.1.1 Release and Deployment Management within the ITIL framework 1113
9.25.1.2 Release and Deployment Management application 1114
9.25.1.3 Release and Deployment Management process overview 1114
9.25.1.4 Key performance indicators for Release and Deployment Management 1117
9.25.2 Release and Deployment Management Workflows 1117
9.25.3 Release and Deployment Management Details 1119
9.26 Service Desk (Streamlined Interaction) 1125
9.26.1 Service Desk Overview (Streamlined Interaction) 1125
9.26.1.1 Service Desk within the ITIL framework 1125
9.26.1.2 Service Desk application 1126
9.26.1.3 Service Desk process overview 1126
9.26.1.4 Input and output for Service Desk Interaction Management 1128
9.26.1.5 Key performance indicators for Service Desk Interaction Management 1129
9.26.1.6 RACI matrix for Service Desk Interaction Management 1130
9.26.2 Service Desk Workflows (Streamlined Interaction) 1130
9.26.2.1 Self-Service by User (process SO 0.1) 1130
9.26.2.2 Interaction Handling (process SO 0.2) 1133

9.26.2.3 Interaction Escalation (process SO 0.3) 1136
9.26.2.4 Close Interaction (process SO 0.4) 1138
9.26.2.5 Cancel Interaction (process SO 0.5) 1139
9.26.3 Service Desk Details (Streamlined Interaction) 1140
9.26.3.1 New interaction form 1140
9.26.3.2 Interaction form after escalation 1141
9.26.3.3 Service Desk Interaction Management form details 1144
9.26.3.4 Interaction categories 1150
9.26.3.5 Escalate Interaction 1154
9.26.4 Changes in other modules 1154
10 Database reports 1158
11 Service Manager third-party components 1162

Add lookup functionality to Format Control

Applies to User roles: System Administrator

The topics below comprise an example of how to use Database record auditing. The steps in these topics must be followed in order. If you skip a step, Database record auditing may not work.

1. Open the audit specifications table.
2. Add an audit specifications record.
3. Define an audit specifications entry.
4. Invoke auditing.
5. Set up event triggers.
6. Add lookup functionality to Format Control.
7. Test audit lookup functionality.

Note: Before using Audit Lookup, be sure that the Audit Specifications include all fields that need auditing.

To add Lookup Functionality, perform the following steps:

1. Access the Format Control record associated with the form or file.
2. Select the **Save Copy**.
3. Click **Additional Options**.
4. Select **Additional Options**.
   The Additional Options panel opens.
5. Activate Lookup Functionality by adding the following specifications:

Field	Contents
Option	1
Desc	Audit Lookup
Condition	true
Application	audit.lookup
Comment	Audit Lookup
Message	Could not call Audit Lookup application.
Names	file
Values	$file

Related topics

Database record auditing
Audit log file
Audit specifications file
Audit processing

1. Special processing considerations: Incident Management

In Incident Management, each action taken on an incident record has the option of adding an additional page; therefore, update and delete operations aren't truly performed. Even though there is a Save button, the application is really adding a new record to the `problem` table. When you close an incident, a page can be added to the file with a status of closed. Consequently, when using Format Control on Incident Management, the add field must evaluate to true.

Format Control records are only executed on the initial, browse, open, update, and close forms for Incident Management.

Format Control can be placed on the following forms, or by using the master Format Control called `probsummary`.

- `IM.open.incident`

- IM.update.incident
- IM.close.incident
- IM.browse

Related topics

Format Control
Special processing considerations: Change Management

1.1. Special processing considerations: Change Management

Change Management enables you to define three types of Format Control records:

- Detail
- Primary
- Approval

The detail and primary records execute during standard add, update, and close processing. The approval record executes during Change Management approval processing.

Change Management uses the four processing functions as follows:

- Add

Processed at open time.

- Update

Processed at update time (update, reopen).

- Delete

Processed at close time.

- Display

Processed when an item is displayed.

The Format Control components executes after the process invokes but before the record permanently updates. For instance, the add function executes after the user clicks on the Open button in either the Request or Task structure, but before the Request/Task saves to the database. The display function executes after a record is selected from the record list but before the record opens.

You can execute both a primary Format Control record and a detail Format Control record for each Request/Task process. The primary Format Control record executes before the detail Format Control Record. If any of the Format Control components fail for any reason, the user always returns to the previously open screen with the appropriate error messages.

Special processing considerations: primary Format Control record

Change Management supports a primary Format Control record that enables you to define in one record the Format Control statements that apply to all Change Management Request Phases. The name of the primary Format Control record is cm3r (cm3t for Tasks). The options on this Format Control record execute during all Request/Task processing (including background processing) except approval processing. The primary format control record processes before the detail Format Control record.

Special processing considerations: detail Format Control record

To enforce processing rules that are unique to a Phase, define a format control record that's the same name as the default View of that Phase. The add, update, delete, and display features process just as they are with detail Format Control definitions.

Special processing considerations: approval Format Control record

The Change Management approval process checks for a Format Control record that associates with the approvals view of the Phase.

- The add features are processed when a Request is approved.
- The update features are processed when a Request is disapproved.
- The delete features are processed when a Request is unapproved.

The primary Format Control record doesn't execute during approval processing.

Related topics

Format Control
Special processing considerations: Incident Management

1.1.1. Format Control error messages

There are several error messages you receive if you enter an incorrect expression into a Format Control record. These errors take several forms:

- If any syntax errors are present when the system attempts to parse (evaluate) an expression field (for example, Calculation, Initialization) in a Format Control record, the following message opens in the Status Bar at the bottom of the screen when you click on the Back button or attempt to select another function from the More Actions menu:
  Field contains an invalid expression.
  In the case of incorrect syntax in a Calculation expression, the system doesn't let you exit the Format Control record until the error is resolved. If you can't resolve it, delete the line and re-enter the desired input when the correct syntax is determined.
  In the case of incorrect syntax in an Initialization Expression, the system doesn't let you access additional Format Control functions (Subroutines, Additional Options, etc.) from the More Actions menu.
- The system will accept a properly constructed Initialization Expression in a Format Control record that may not be accepted by the called application at run time. When an application can't recognize an Initialization Expression, the Format Control function fails, and Service Manager displays this message: *Cannot evaluate initialization expression #:1*
- Format Control accepts properly constructed but conflicting expressions within a record.
  For example:
  Initialization expression: $x="ibm"
  Calculation expression: $x+=1
  When Format Control attempts to perform this calculation on a non-numeric field at run time, the system displays the following error message:
  Wrong or mismatched type in increment or decrement

Related topics

Format Control

1.1.1.1. Common routines called from Format Control

The following is a list of RAD routines that have a specific function within Format Control:

- as.copy
- as.delete
- as.get.name
- as.insert
- as.move
- as.options
- as.sort
- axces.write
- database
- fill.fc
- fingerprint
- getnumb.fc
- marquee.publish
- marquee.send
- message.fc
- post.fc
- publish
- sort.array
- query.stored
- validate.fields

Related topics

Determine parameters without using the RAD Editor
Service Manager Programming

1.2. Links

One of the advantages of a relational database is the elimination of redundant information. You accomplish this by storing information about a particular subject in one place or file that has links to other subjects. Links are a combination of data and link definitions, sets of conditions containing the relationships for linked information. Use links in Incident Management and Configuration Management environments to relate information in one file to information in another.

Link records define the relationship among sets of data. They are used extensively throughout the system and the main topics include the following:

- Understanding links — general linking within the system
- Link maintenance — using links effectively
- Virtual joins — are used to display read-only data from another file.
- Posting — copy data from a source record to a target record.

Related topics

Tailoring
Understanding links
Link maintenance
Virtual joins
Posting

1.2.1. Understanding links

These topics discuss linking within the system. The topics demonstrate how to:

- Access link definitions
- Add a new link
- Modify an existing link
- Use advanced link editing features
- Link dependencies with the Service Desk environment

Related topics

Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.1. Advanced link editing features

Simple links define the relationships between a specific field in a source record and a specific field in a target record. The Find and Fill options perform in a straightforward manner. A link query is built based upon a search argument, which contains the value of a field in the source file and the name of a field in the target file. These fields need to be Unique key fields for simple linking.

The system provides extended flexibility to define complex link expressions. For example, you can:

- Define a specific query that uses more than one field to form the link query.
- Specify the record list format that should be used when the link query finds more than one record.
- Define a variable, rather than a specific name, to be used as the target file.
- Manipulate the value of fields by using link expressions.
- Specify that particular fields are copied from the target file to specific fields in the source file during the Fill operation without a requirement for identical field names.
- Include multiple non-keyed fields on the link line structure of a keyed field, enabling those fields to be copied to the source form along with the data retrieved from the keyed field query.



Note

Note Advanced link features aren't available when using Virtual Join.

Related topics

Specifying a link query
Copying fields by name during fill operations

Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.2. Specifying a link query

If you want to control the search criteria used by Find and Fill, define a specific link query.

There can be instances when the link is dependent on more than the simple value of fields in the records. For example, links from an incident document could be dependent on the device type. Link Maintenance allows specification of expressions that are evaluated by the Fill and Find operations before the link query is built.

Whenever the link relationship varies according to data in the source record, using expressions and a variable for the target file is the most efficient method for establishing a link.

Expressions process in the order they are listed on the form.

Note: Be aware that If the $query variable is used in link records, each $query value is valid only in the current thread. This means a $query value might be overwritten by executing a second link query. To avoid this issue, you can use $L.query instead of $query in these cases.

Related topics

Advanced link editing features
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.3. Copying fields by name during fill operations

Normally, the Fill operation copies fields of the same name from the target file to the source file using a function called project.

For example, if you open an incident document and place the cursor on the logical.name input field and then choose the Fill option, the system searches the device file for a record with the same logical.name. The system then copies fields with the same name from the device file into the incident document.

The incident document contains information about the device that copied from like-named fields in the device record.

As long as the field names are the same, and the information being copied is from an equal or higher level, the project function performs normally. When the field names are dissimilar, or if the information to be copied is from structure fields to scalar fields, you must use a different method.

Related topics

Advanced link editing features
Specifying a link query
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance
Copy information from structure fields to scalar fields

1.2.1.4. Scalar/non-scalar field links

A simple link can't copy information from a non-scalar field (composed of more than one data element of the same type) to a scalar field (composed of a single data element). For example, information from a record in the incident file can't be projected into a probsummary record. Even though the field names are the same, the fields in the incident record aren't scalar fields. They exist in one of the three structures that make up the `problem` table descriptor. The fully-qualified name of the assignment field in the incident record is really header,assignment.

The system can project the logical.name value in the incident file into the incident record's logical.name field. It can't project the incident file's non-scalar middle,logical.name field into the scalar logical.name field in the incident file. To copy non-scalar information to a scalar field, the Fill operation copies each field using instructions in the link record.

Related topics

Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.5. Keeping changes

There is no Update option when editing a link line entry. A copy of the link line entry is made when it is selected. That copy is compared to the current line entry when you exit using Back, Close Application, Next Entry or Previous Entry.

Remember that when you modify a link line entry, the change doesn't write to the database until you exit from the link record and confirm the update action.

Related topics

Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.6. Link dependencies within Service Desk

The Service Desk applications use two special link records. They perform the conversion of `problem` records to `probsummary` records and the conversion of inventory files to the entity and attribute files.

- The `build.inventory.files` link record converts information in the various inventory files to the Configuration Management device file and their associated attribute files. This link record can't be used for any other purpose since the $file variable has special meaning in the conversion application.
- The `build.problem.summary2` copies information from the problem record in the rootcause table to the incident record in the probsummary table whenever the appropriate problem record is updated. This link record uses the $pfile variable in place of the normal $File variable when building expressions. It processes expressions after copying the fields, rather than before as in normal link processing.

Related topics

Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Specifying a link query
Add a new link file

Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.7. Document Engine primary link record

For each object accessed through the Document Engine, a primary link record combines with the form-dependent link record. The primary link record is valid for the entire file, and always has the same name as the Object/filename. For example, if the filename is contacts, the primary link record is the contacts link record. This eliminates redundant links across many records for ease of maintenance and for consistency.

Related concepts

Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.8. Add a new link file

Applies to User roles: System Administrator

To add a new link file:

1. Click **Tailoring** > **Tailoring Tools** > **Links**.
2. In the Name field, type the name of the form or file. If desired, type a descriptive name or phrase in the Description field.
3. Click **New** to create the new link record.
   The record opens for editing.
4. Type the relationships in the appropriate columns.
5. Click **Save** to add the new link record.

Related topics

Understanding links
Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.1.9. Test a link

Applies to User roles: System Administrator

To test a link:

Note: The locations form is used for this example.

1. Click **Tailoring** > **Database Manager**.
2. Type **location** in the Form field, and then click **Search**.
   The location file opens.
3. Type **P** in the parent Location field.
4. Click **Find**. A list of records with a link definition for this file opens.

5. The Parent field is linked to the Location Full Name field in the Link file for *location*.

Note: Link records are case-sensitive.

Related topics

Understanding links
Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Delete a link
Access advanced link maintenance

1.2.1.10. Delete a link

Applies to User roles: System Administrator

To delete a link:

1. Click **Tailoring** > **Tailoring Tools** > **Links**.
2. In the **Name** field, type a link name, and then click **Search**..
   The link record opens.
3. Click **Delete** in the toolbar.
   You are prompted to confirm that you want to delete the link record.
4. Click **Yes** to confirm that you want to delete the link record.
   You are returned to the link record prompt screen.

Related topics

Understanding links
Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Access advanced link maintenance

1.2.1.11. Access advanced link maintenance

Applies to User roles: System Administrator

When you access a link record, the Source Field and Target Format/File Names copy to the link structure form. New fields are available and new option keys are enabled.

To access advanced link maintenance:

1. Click **Tailoring** > **Tailoring Tools** > **Links**.
2. Type an optional link name in the **Name** field, and then click **Search**.
   A list of all link records opens.
3. Click the link to edit.
   For example, click the locations link record.
4. Position the cursor on the line you want to edit.
5. Click **More** or the More Actions icon and then choose **Select Line**.

Related topics

Advanced link editing features
Specifying a link query

Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link

1.2.1.12. Copy information from structure fields to scalar fields

Applies to User roles: System Administrator

To copy information from structure fields to scalar fields (fields composed of a single data element):

1. Open a link record.
    1. Click **Tailoring** > **Tailoring Tools** > **Links**.
    2. In the Name field, type the name of the link record, and then click **Search**.
    3. Select a record from the list.
2. Place the cursor on the line containing the link.
3. Click **Select line** from the More Actions menu.
4. Switch the fields by typing the structure field in Source Field (Fill To) and the scalar field in Target Field (Fill From). The contents of the structure fields are copied to the scalar fields.

Related topics

Advanced link editing features
Specifying a link query
Copying fields by name during fill operations
Scalar/non-scalar field links
Keeping changes
Link dependencies within Service Desk
Specifying a link query
Add a new link file
Modify an existing link
Test a link
Delete a link
Access advanced link maintenance

1.2.2. Link maintenance

One of the advantages of a relational database is the elimination of redundant information. You accomplish this by storing information about a particular subject in one place or file that has links to other subjects. Links are a combination of data and link definitions, sets of conditions containing the relationships for linked information. Use links in Incident Management and Configuration Management environments to relate information in one file to information in another.

Related topics

Data relationships and the link file
Us.link
Variables used in links
Access the link record

1.2.2.1. Data relationships and the link file

Link maintenance involves establishing relationships between data so that information residing in one file can form the link query that selects, displays, or copies information from another file.

A separate link definition can occur for each form.

When a field is linked to more than one table, the order in which the entries appear in the link record determines the order in which Service Manager searches them. First, Service Manager searches the table at the top of the list. If no records are found to satisfy the link query, it searches the tables lower in the list, from top to bottom. It stops searching when the query is satisfied, or it finishes searching all tables in the list.

Related topics

Link maintenance
Us.link
Access the link record

1.2.2.2. Types of links

Find, Fill, and virtual join links are the basis for the most powerful features of Service Manager.

- The Find function uses the value of the field in which the cursor is placed to query for information based on data stored in existing records. Security parameters restrict a user's access to the requested data. If information is found, a record list of matching configuration item (CI) records opens. If no information is found, a negative message opens.
- The Fill function selects information from existing records and copies it into the current record. For example, incident documents are populated with device information when a logical name (logical.name) is specified.
- The virtual join combines information from many files and presents the results on a single form. At the service desk, a vendor's telephone number can display in an incident document without occupying any space in the actual record. The information presented doesn't reside in long-term memory and is another way of viewing Configuration Management data.

Linking ensures current and consistent data across all incident documents. A change to the original configuration item (CI) record automatically updates the incident document. Each time an incident document opens or updates, the applicable fields displaying linked data pick up the most recent Configuration Management data.

Important: Queries for Find, Fill and virtual joins are performed only when run on fields that have been set up as unique keys in the database dictionary record for the target files. These unique key definitions allow data to be stored using certain markers. Link queries need to locate these markers in order to retrieve the applicable data.

Search the Help for more information on setting up unique keys and other definitions in the database dictionary record.

Related topics

Link maintenance
Find functionality
Fill functionality
Virtual joining functionality
Us.link
Variables used in links
Calling us.link
Find from and Fill to a $ variable

1.2.2.3. Find functionality

Using the information in the link record, Find (us.link) locates and displays information in another record (or records) based on the contents of the current record.

In an Incident Management example, the source record could contain *pc* in the field labeled Affected CI (logical.name field). The link record relates the logical.name field in the SOURCE record to the logical.name field in the vendor (TARGET) file.

All device records containing logical.name fields beginning with *pc* are selected and displayed in a record list for the user to select.

The user selects the specific device sought and opens the applicable device record.

Find uses the field name of the text box where the cursor currently is, in order to determine what relationship to establish. To select the device records with names that begin with *pc*, the cursor must be in the CI Name field that contains *pc*.

It builds the following query:
`logical.name field in device file begins with "pc"`

That translates into the following link query that executes against the vendor file:
`device#"pc"`

Note: Using advanced features in Link Maintenance, you can apply rules to use a more complex link query for data selection. Refer to Using Advanced Link Editing Features Help for more information.

The Find option is available throughout the system. It is subject to security restrictions within the application. For example, Database Manager controls the Find option through the use of Format Control. In Incident Management, the profile determines whether Find is available for the current user and how it behaves.

The selected data records can be manipulated in accordance with security restrictions. Changes made to these records DO NOT modify the source record unless the user selects from an available list of options that allow updated information to be posted to the specific configuration item record. For more information on posting, refer to Format Control and Display Options Help.

Related topics

Link maintenance
Types of links
Fill functionality
Virtual joining functionality
Us.link
Variables used in links
Calling us.link
Find from and Fill to a $ variable

1.2.2.4. Fill functionality

Using information in the link file, Fill (us.link) locates information in another record and copies it into the current record.

Fill does the following:

- Starts on a source record.
- Accesses a record in a secondary (target) file based on data in the source record (usually the contents of the input field selected by the cursor).
- Copies (fills) data from that target record back to the source record.

The link record for the probsummary file is used in Incident Management.

In the following example, Fill populates fields in an incident document based on the value of the field labeled CI Name.

Rather than selecting the device record for display, fields in the `device` (TARGET) file that have the same name as the fields in the `problem` (SOURCE) file project into the source record.

Fields on the source form populate automatically with data directly from the applicable device record. For example, CI Name (logical.name) on the incident record fills with the value of Configuration Item (logical.name) from the device record. The Type field on the incident form fills with the value of Type from the device record, and the value for the Category field copies from the device record to the incident record.

Through this transfer, the TARGET record is unchanged but the SOURCE record modifies to reflect the values of fields in the TARGET record.

The resulting modified SOURCE record is not written to the database until some action instructs the system to do so. For example, open or update the incident.

The Fill option uses the value in the field where the cursor is positioned to determine which link relationship to use. This example assumes that the cursor is in the CI Name (logical.name) input field when you select the Fill option.

Use Fill when it is necessary to store information in the source record so that it can be changed or used as a link to other information. If you simply need to display the information in a form, use Virtual Join.

Related topics

Link maintenance
Types of links
Find functionality
Virtual joining functionality
Us.link
Variables used in links
Calling us.link
Find from and Fill to a $ variable
Find from and Fill to an array structure

1.2.2.5. Fill functionality with multi-select

The Fill Selected option is available from record lists resulting from a fill on an array field. When one or more records from a list are selected, the user can choose the Fill Selected option to fill the array with only the selected records.

- A multi-select fill will not overwrite the existing content of an array. The records selected will be inserted into the array from the element where the fill was initiated.
- The multi-select capability for a record list is determined by the field that the fill is being performed from.
  - If the field is a scalar the capability is disabled.
  - If the field is an array the capability is enabled.
- The multi-select functionality is disabled in system-generated record lists. The System Administrator needs to create a form with a table for the capability to be available.
- When a Fill All or Fill Selected is performed, a message indicating how many records were filled displays in the message area.

- The post expressions defined in a link line are executed as each record is filled into an array when performing a Fill All or Fill Selected.

Related topics

Fill functionality
Turn off multi-select functionality
Link maintenance
Types of links
Find functionality
Us.link
Variables used in links
Find from and Fill to a $ variable
Find from and Fill to an array structure

1.2.2.6. Turn off multi-select functionality

The Fill Selected option is available by default whenever a fill is performed on an array field. System Administrators can disable this option by setting the variable `$fill.mult` to false in the expressions of the link line for the field where the fill is being performed.

Note: The Multiple Selection property in Forms Designer has no effect on enabling or disabling the multi-select capability when performing a fill from an array field.

Related topics

Fill functionality
Fill functionality with multi-select
Link maintenance
Types of links
Find functionality
Us.link
Variables used in links
Find from and Fill to a $ variable
Find from and Fill to an array structure

1.2.2.7. Virtual joining functionality

The Virtual Join function allows a single form to display information from many files. You can't modify virtually joined information but it can link with other files using Find and Fill.

There are special requirements for virtual joins when specifying link information:

- The TARGET Format/File Name value in the link record must contain a file name not a form name.
- The target field must be a non-concatenated key in the target file.
- The target field must be the first instance of the key in the database dictionary's key array.
- The target field must be a scalar field (non-array).
- Virtual Join only executes simple links.

While the appearance of virtually joined information is the same as if it exists in the open record, the information is stored in one place and displayed on demand to another.

Note: Use virtual join in any form except record list forms. Use virtual join information as a source field to other information using Find or Fill. Nested virtual joins aren't supported.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Us.link
Variables used in links
Calling us.link
Find from and Fill to a $ variable
Find from and Fill to a $ variable

1.2.2.8. Us.link

This section describes the Universal Services- Link (us.link) application.

The us.link application replaces the old applications find, fill and fill.recurse for several reasons:

- To take advantage of the current Service Manager environment in order to speed up transactions.
- To have a common rule base behind each of these applications.
- To make the find/fill process easier to debug.

The us.link is responsible for selecting the correct records from the correct file and then passing control to either us.find or us.fill.

The find, fill, and fill.recurse applications are now single panels calling the us.link routine. No changes are necessary to existing code that calls one of these routines.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Virtual joining functionality
Variables used in links
Calling us.link
Find from and Fill to a $ variable

1.2.2.9. Variables used in links

The variables used in links are listed below. The application "us.link" is in charge of executing the link expressions using the values set in those variables. If values for the logical variables aren't specified, they default to false. Unless noted, the special variables used by prior applications are also used by us.link.

Variable	Value	Use
$fill.date.calendar	true or false	Enables a pop-up calendar window, and is used for date/time fields. **Note:** $fill.date.calendar is no longer needed when using 6.2 or later clients, which allow you to define a date/time field on a form as a "calendar"-type widget.
$fill.display.add	true or false	Determines whether you can add a record if the current link query returns no records.
$fill.display	true or false	Determines whether the record you are filling from opens before you copy data from it into your source record.
$fill.exact	true or false	Forces the project panel to adhere strictly to data types and index levels during the project.
$find.skip	true or false	Tells the application to skip the current entry whenever a "find" is performed on the field.
$fill.list	An array list of values. For example: $fill.list={"a", "b", "c"}	Enables you to fill from an array list of values. Target file and field names aren't required when using this option. For example, $fill.list={"a","b","c"} will execute a pop-up window where the user can select one of the array values, a, b, or c.
$fill.recurse.msg	A message. For example: $fill.recurse.msg="Hello world"	Defines the message to appear on a recursive link (when $fill.recurse=true).
$fill.option.copy	true or false	This option is no longer necessary when using us.link.
$fill.option.skip	true or false	Enables the skip option to skip the current link entry and move on when displaying the results of a recursive fill.
$fill.recurse	true or false	Determines whether to move to the next entry in the link record (perform a recursive fill).
$fill.replace	true or false	Governs whether a field that contains data should be overwritten with new data (including NULL). Only used for the project portion of a fill, not fields specified on the fill to/fill from section of the link line.
$fill.search.format	Form to use when performing a fill. For example: $fill.search.format="company"	Defines the search screen to use when performing a fill. If this variable is set in the link expression, and the field that you are filling is NULL, the indicated search screen opens. The search created here continues the standard fill process.

$fill.skip	true or false	Tells the application to skip the current entry and continue based upon the value of $fill.recurse for that entry.
$fill.structure	Specific link. For example: $fill.structure= {1,"rad"}	Enables you to perform a find from a field that's part of a structured array and to use fill to move information into that specific element of the structured array when: - The field name to fill from/post to is part of the structured array. - The field name to fill from/post to is not used in any other structure of the database dictionary record. - The name of the structure must be the same as the name of the array of which it is a part. To use find/fill on a structured array, you must set up the variable $fill.structure in the expressions of the specific link line. $fill.structure is an array of two elements. The first is the index of the field (that's used as the source field) in the structure and is of type number. The second is the name of the structure and is of type character. When using the fill function on a structured array, you can only modify the specific line of the structure being accessed. Instead of the actual field name, use the index number of the field within the structure. This includes the source field.
$project.first	true or false	Governs whether a project should be done before moving any of the fields defined in the list of fill to/fill from fields.
$fill.mult	true or false	Determines whether or not you can fill information from multiple records. True enables fill information from multiple records. The default value is true.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Virtual joining functionality
Us.link
Calling us.link
Find from and Fill to a $ variable
Find from and Fill to an array structure

1.2.2.10. Calling the us.link

Use the following parameters to call the us.link application:

Note

Note Fill and fill.recurse use different parameter names for the same fields.

Parameter Name	Description	Default
record	Source Record (required)	None
name	Field name to find from/link to	Current field
string1	Format Name	Current format
second.record	Link Record (optional)	None
prompt	Action (“find” or “fill”)	“find”
boolean1	Background flag	False
all.null	Exit when closing the current thread	None
index	Cursor line	None
cond.input	Skip Exact Find **Note** Note This parameter is used only for reference fields (that is, fields that have a referenced table defined in the data policy). By default, the Find function of a reference field performs an exact match search instead of using the query defined in the link record. If you want to use the query defined in the link record for a reference field, pass this parameter as true when calling the us.link application.	False

Note

Note The only required parameter for us.link is the source record.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Virtual joining functionality
Us.link
Variables used in links
Find from and Fill to a $ variable
Find from and Fill to an array structure

1.2.2.11. Skipping query writing

Service Manager performs the **Fill** action based on the query defined in the link line to filter data against the target table. Prior to Service Manager 9.41, you usually defined the query by ID.

As of Service Manager 9.41, a **Skip Query Rewriting** option is available on the Edit Link Line form. This option is added as part of the logical name solution, which adds the **display.name** field to the **device** table to store CI names.

> **Note**
>
> Note For more information about the logical name solution, see the Service Manager Logical Name Solution white paper.

Because CI names are now stored in the **display.name** field instead of the **logical.name** field, Service Manager by default automatically rewrites your existing link queries at runtime to address the impact of the logical name solution (the **Skip Query Rewriting** option is disabled by default). If the **Skip Query Rewriting** option is enabled, the system doesn't perform the additional operations that it does when the option is disabled.

To access this option, open a link record in Database Manager, highlight a link line, and then select the **Select Line** option from the **More** menu. By default, this option is disabled.

Guidelines on editing link queries for a reference field

When writing a link query for a reference field, you need to observe different guidelines depending on whether this option is enabled or disabled.

> **Note**
>
> Note The following descriptions use **affected.item** as an example reference field.

- If the **Skip Query Rewriting** option is disabled, update an expression such as `logical.name#affected.item in $File` to the following: `logical.name#\""+affected.item in $File+"\""`
- If the **Skip Query Rewriting** option is enabled, you must write a query like the following: `if (not (null(get.display.value($File, "affected.item")))) then $query+=(" and display.name#\""+get.display.value($File, "affected.item")+"\"")`

Additionally, as a system administrator, you need to understand how the Fill function works for a reference field:

- The `affected.item in $File` expression will be null when the user types something in the format, and the system gets what the user has typed by the **get.display.value** function.
- If the **Skip Query Rewriting** option is disabled, the system assigns what the user has typed to `affected.item in $File` when the user clicks the **Fill** button, and then rewrites at runtime the evaluated `$query` from, for example, `logical.name# DISPLAY VALUE` to `display.name# DISPLAY VALUE`. This makes the original link query still work for the reference field.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Virtual joining functionality
Us.link
Variables used in links
Find from and Fill to a $ variable
Find from and Fill to an array structure

1.2.2.12. Accessing $File/dates

$File is used in Fill and Find link line queries. You can modify the $File directly using the link expressions and the changes are automatically saved in the source record. When performing a fill on a date field, the system first checks to see if that date field is in the link record. If the date field is in the link record, the link expressions perform rather than just fill the current date and time. This way it is possible to fill with something other than tod() (such as date(tod()) or tod() + '7 00:00:00').

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Virtual joining functionality
Us.link
Variables used in links

Calling us.link
Find from and Fill to a $ variable

1.2.2.13. Find from and fill to a $ variable

The us.link application makes it possible to perform a find from a field that has a $ variable as an input and to use fill to move information into a $ variable field. To use a variable as a source field, place the variable name in the Source Field column of a link record. To fill to a specific variable, place that variable on the source field(fill to/post from) column on the specific link line.

Note: Use $file in virtual join link line queries and $File in fill and find link line queries.

Posting doesn't allow use of $ variables at this time.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Virtual joining functionality
Us.link
Variables used in links
Calling us.link
Find from and Fill to an array structure

1.2.2.14. Find from and fill to an array structure

The us.link application makes it possible to perform a find from a field that's part of an array structure and to use fill to move information into that specific element of the array structure. This can only be performed under the following circumstances:

1. The field name to fill from/post to is part of the array structure.
2. The field name to fill from/post to is not used in any other structure of the database dictionary record.
3. The name of the structure must be the same as the name of the array of which it is a part.

To use find/fill on an array structure, you must set up the variable $fill.structure in the expressions of the specific link line. $fill.structure is an array of two elements. The first is the index of the field (that's used as the source field) in the structure and is of type number. The second is the name of the structure and is of type character.

When using the fill function on an array structure, you can only modify the specific line of the structure being accessed. Instead of the actual field name, use the index number of the field within the structure. This includes the source field.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality
Virtual joining functionality
Us.link
Variables used in links
Calling us.link
Find from and Fill to a $ variable

1.2.2.15. The $fill.display and $fill.display.add functionality

When using $fill.display, you can modify the data using the standard rules that apply to that database using Format Control.

- Click **OK** to save the record and use the new version of the record to perform the Fill function, based on the associated link line.
- Click **Cancel** to cancel any updates. It will not fill information back into the source record.

When $fill.display.add is set to true and the us.link performs the standard link query and returns no records, the form defined in the link line opens and the user can add a record. Any information defined in the Fill to or Fill from fields copies into this record.

- Click **Add** to add the record while leaving the user on this form. At this point the rules for $fill.display are followed.
- Click **OK** to add the record and perform the Fill function back to the source record using the newly-added record.

Related topics

Link maintenance
Types of links
Find functionality
Fill functionality

Virtual joining functionality
Us.link
Variables used in links
Calling us.link
Find from and Fill to a $ variable

1.2.2.16. Access the link record

Applies to User roles: System Administrator

This examples uses the `problem` table.

Consider the case where there is a record against a user's pc. The incident record requires information regarding the serial number, location of the pc, and its vendor/supplier, as well as a description of the incident.

The required information for documenting an incident is often already in the database. To find and display the information with the incident document, you must tell the system:

- What value to use as a search argument
- Where to look for the matching information

At least two separate records store the required information for the incident document. Therefore, you must define at least two relationships in the link record: one to device, one to vendor/supplier, and one to any other applicable table.

The system stores this information about each configuration item in Configuration Management and references it according to several categories of information. The device record contains the logical name, contact, and location names. The vendor/supplier record contains the vendor/supplier ID, location, and vendor/supplier phone number.

There is an implied order within each link that defines these relationships. The device record contains the name of the vendor/supplier but not the phone number of the vendor/supplier, and the location record stores the contact name instead of the device record.

By defining the first relationship between the incident document and the device record, you can retrieve the data necessary to form the queries that retrieve data through the second and third relationships.

The link record defines the relationship between the incident document and information in the device, vendor/supplier, and other records. Although you can have different link records for each form in Incident Management, the `problem` and `probsummary` records always exist. These records provide a basis and example for other Incident Management link records.

Note: If you are using IR Expert, consider building links from one of the IR fields (like action) to itself. This allows a simple Find command to locate relevant incidents before a record opens.

1. Click **Tailoring** > **Tailoring Tools** > **Links**.
2. In the Name field, type **problem**, and then click **Search**.
3. Select a record from the list.

The column labeled **Source Field Name** contains the names of fields in the current record (in this case `problem`). The columns under the **Target File Name** contain the file names and corresponding field names that define the relationship. When the link runs, the contents of the source name in the `problem` record search the target table for other information to populate the form.

The configuration items link to the incident document by the field labeled CI Name (`logical.name`) field. The first link to the device record retrieves the serial number and location names. Subsequent links use that information to form their own relationships.

A link to the vendor/supplier record from the field labeled Service Provider (on any linked Incident Management form) provides the vendor/supplier phone number for the incident document.

Configuration Management data stored on Configuration Items (CIs), vendor, and other records can link to several forms in Incident Management in the same way multiple relationships for the same field exist. The link record specifies these link relationships. Links between data and forms are established in the order in which they appear in the list. A query for a specific field can link to more than one configuration item record in the process of searching for the necessary data.

Related topics

Link maintenance
Data relationships and the link file
Us.link
Forms Designer
Subform control

1.2.3. Virtual joins

Virtual joins allow you to display data fields from several different database files on a single form. Virtual joins don't allow data entry into the joined fields; they only display information from other files into a format for reference purposes.

Virtual joins can be used on any format. They are established when data from a record is displayed without the need to use the Find and Fill functions of the link utility.

You create a virtual join by adding a subform to any form, and displaying the desired data there.

Related topics

Understanding subforms
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works

1.2.3.1. Understanding subforms

Subforms allow the creation of multi-part forms, where sections of the form can be reproduced in other forms. These modular forms enable the simultaneous display of data from numerous database records via virtual joins. Subforms are also useful in constructing multiple views of the same subform data, for example, Incident Management views.

You construct subforms the same way as standard forms, with the exception that you create them to appear within another, larger, form. Subforms can be micro versions of larger forms, containing specific key information that's valuable for display on other forms. For example, Configuration Management subforms that appear on incident record forms.

You can create a subform on any form as long as it is properly placed using Forms Designer. Data from a record can open in a subform automatically or through the use of the Find and Fill function keys of the Link utility.

Related topics

Virtual joins
Forms Designer
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2. Example: Creating a virtual join

Applies to User roles: System Administrator

The tasks listed below illustrate the steps involved in retrieving data from a file called sales and displaying data in a form called orders.

The forms and subforms that you need to build to create a virtual join include:

- **sales**: You create the main sales form, and then you create a database dictionary file so data can be entered through the form and recorded to the database. The file in the database associated with this form is the target file from which data will be retrieved to display in the orders file.
- **sales1 subform**: When you create the sales1 subform, it is a copied and modified version of the original form (sales). It is already associated with the sales database dictionary record.
- **orders**: When you create the orders form, you will add fields for customer name, contact, phone number, and order amount.

Note: Code must be a single Unique key (not concatenated) in the target file. Also, the source and target fields must be scalar (non-array).

To complete building the necessary forms and creating the virtual join in order to retrieve data from the sales file and display data in the order forms, perform the following tasks:

1. Build the sales form
2. Create the sales file
3. Create the sales record list
4. Add data to the sales file
5. Create the sales1 subform
6. Create the orders form
7. Build the virtual join into the form
8. Build the link
9. Verify that the sales1 form works

Related topics

Virtual joins
Understanding subforms
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works

1.2.3.2.1. Build the sales form

Applies to User roles: System Administrator

When you create the main sales form, you need to create a database dictionary file so data can be entered through the form and recorded to the database.

1. Click **Tailoring** > **Forms Designer**.
2. Type **sales** in the Form field, and then click **New**.
3. Click **No** at the Forms Wizard prompt. A new blank form opens.
4. Using the Label tool, add a title across the top. For example, **Sales Personnel Information**.
5. Create two tabs by selecting the Notebook button. Name the front tab **Name/Code** and the back tab **Manager/Commission** by changing the Caption property.
6. Select the Name/Code tab and do the following:
   a. Add a Text box and place a Label called **Sales Code**.
   b. Place a Text box next to the Label with the input value **code**.
   c. Add a Text box and place a Label called **Seller Name**.
   d. Place a Text box next to the Label with the input value **name**.
   e. Add a Combo Box. Place a Label called **Location** and add a list of at least four city names in both the **Displaylist** and **Valuelist** fields.
   f. Place a Combo Box next to the Label with the input value **commission**.
   g. Add a Text box and place a Label called **Sales Commission %**.
   h. Place a Text box next to the Label with the input value **commission**.
7. Select the Manager/Commission tab. Add a Text box and place a Label called **Manager**. Next, place a Text box next to the Label with the input value **manager**.
8. Click **OK** to confirm the new form.
9. Click **OK** again to exit Forms Designer and save your new form.

Related topics

Virtual joins
Understanding subforms
Forms Designer
Example: Creating a virtual join
Create the sales file
Create the sales record list

Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2.2. Create the sales file

Applies to User roles: System Administrator

To create the sales file:

1. Sign in to the Service Manager Windows client.
2. Click **Tailoring** > **Forms Designer**.
3. In the Form field, type `sales`, and then click **Search**.
4. Go to the Detail menu to create a file. Either right-click in the sales form or select the Detail menu button in the top right corner, and then select **Create File**.
5. Type `sales` as the filename, and then click **OK**.
   If the file name is already prepopulated with `sales`, click **OK** to confirm.
   This creates a valid `sales` file. The database dictionary file (dbdict record) generates automatically and displays in the System Definition Utility, with the exception of the No Nulls key for the location field. It becomes the target file for your subform when it appears on other forms.
6. Click the Fields and Keys tab to see field definitions and keys. It contains one key on field "code," which is a type "unique."
7. In the Keys section, click **New**.
   Service Manager displays the General section.
8. In the Type field of the General section, select **No nulls**, and then click **Add**.
9. Select **location**, and then click **OK**.
10. Click **Save**.
    A warning appears, stating that the pending changes in the keys definitions for this dbdict record require a tableregen.
11. Click **OK**, and then close the System Definition Utility window.

The main sales data form and file are now complete.

Related topics

- Virtual joins
- Understanding subforms
- Forms Designer
- Example: Creating a virtual join
- Build the sales form
- Create the sales record list
- Add data to the sales file
- Create the sales1 subform
- Create the orders form
- Build the virtual join into the form
- Build the link
- Use the virtual join
- Verify that the sales1 form works
- Subform control

1.2.3.2.3. Create the sales record list

Applies to User roles: System Administrator

To create the sales record list:

1. Click **Tailoring** > **Forms Designer**.
2. In the Form field, type **sales.qbe**, and then click **New**.
3. Click **No** at the Forms Wizard prompt.
4. Draw a table on the design space.

5. Type **Code** and **Location** in the Columns field of the Table properties window. These are the two database dictionary keys for the sales file.
6. Type **code** in the Code column input field.
7. Type **location** in the Location column input field.
8. Click **OK** to finish.

Related topics

Virtual joins
Understanding subforms
Forms Designer
Example: Creating a virtual join
Build the sales form
Create the sales file
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2.4. Add data to the sales file

Applies to User roles: System Administrator

To add data to the sales file:

1. Click **Tailoring** > **Forms Designer**.
2. In the **Form** field, type **sales**, and then click **Search**.
3. Click **More** or the More Actions icon and then select **Database Manager**.
4. Create at least two new sales personnel records. Be sure to fill in the fields on both tabs.
5. Click **OK** to return to Forms Designer.

Related topics

Virtual joins
Understanding subforms
Forms Designer
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2.5. Create the sales1 subform

Applies to User roles: System Administrator

When you create the sales1 subform, it is a copied and modified version of the original form (sales). It is already associated with the sales database dictionary record.

To create the sales1 subform:

1. Click **Tailoring** > **Forms Designer**.
2. In the Form field, type **sales**, and then click **Search**.
3. Click **Copy/Rename** from the More Actions menu.
4. Make a copy of the sales form.
    1. In the New Name field, type **sales1**.
    2. Click **Copy** in the Format field.

3. Click **OK** to confirm the creation of the duplicate form.

5. Click **Design**.
6. Cut the Seller Name label and field, as well as the Sales Commission % label and field from the notebook tabs, and then paste them somewhere on the form.
7. Remove everything from the form except Seller Name and Sales Commission %.
8. Click **OK** to confirm changes to sales1.

Related topics

Virtual joins
Understanding subforms
Forms Designer
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2.6. Create the orders form

Applies to User roles: System Administrator

To create the orders form:

1. Click **Tailoring** > **Forms Designer**.
2. In the Form field, type **orders**, and then click **New**.
   If you have another form named orders, return to the Form field and rename this new form.
3. Click **No** at the Forms Wizard prompt.
4. Select the Frame tool and do the following:
   1. Draw a group on the screen and give it a label name **Customer Order**.
   2. Draw a frame on the screen for the Customer Order fields.
   3. Add fields and label the form. The inputs for the text box fields are as follows:
      - **customer.name**
      - **contact**
      - **phone.number**
      - **order.amount**
5. Click **OK**.

Related topics

Virtual joins
Understanding subforms
Forms Designer
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Build the virtual join into the form
Build the link
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2.7. Build the virtual join into the form

Applies to User roles: System Administrator

To build the virtual join into the form:

1. Open the orders form (created in Create the orders form) in Forms Designer.
2. Click **Design**.
3. Select the Group tool and Frame tool to create a frame on the screen where you want the seller data to appear. Give it the caption/title **Seller Data**.
4. Select the Label tool and create a label named **Seller Code**.
5. Select the Comfill tool and create a fillable input field. This enables you to choose from a list of seller codes.
6. In the Property box, type **code** in the input value and **9** (fill) in the ButtonID field. Click **Y** to confirm the addition.
7. Click **OK** to save changes to the form.

Subform placement

1. Select the Subform tool.
2. Position the cursor on the screen where you want to begin your virtual join, and then create a square where you want the virtually joined subform to appear.
3. In the Property box for the subform, type **code** in the Input field.
   The input field value (code) is the same as the previous input field value, because a link needs to be established with the target file. The Virtual Join tool uses the link with the code key on the sales file to pull in the information requested by the sales1 form, namely values for the name and commission fields.
4. Type **sales1** in the Form field to indicate which form should appear in the subform area.
5. Click **Yes** in the Virtual Join field of the Property box to activate the virtual join functions on the subform.
6. Click **OK** to save changes to the orders form.
   Note: You can't see the subform at this point. The subform data fields don't appear until you enter data through the Database Manager.
7. From the orders form, click **More** or the More Actions icon and then select **Create file**.
   The Database Dictionary (dbdict) utility opens.
8. Create an orders file in the database dictionary.
   a. Type **orders** as the filename, and then click **OK**. If the name is prepopulated with orders, click **OK** to confirm.
   This creates a valid "orders" file. The dbdict record for it is now displayed in the System Definition Utility. In the Fields and Keys tab, you can look at field definitions and keys. It contains one key on field "customer.name," which is type "unique."
   b. In the orders file, ensure there is a line for the code field at the bottom of the list. If there is not, in the Fields section click **New Field**, and then create a new character field called "code."
   c. Click **Save** to save your changes.
9. Close the System Navigator to leave the System Navigator utility and return to Forms Designer.

Related topics

Virtual joins
Understanding subforms
Forms Designer
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the link
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2.8. Build the link

Applies to User roles: System Administrator

To build the link record:

1. Click **Tailoring** > **Forms Designer**.
2. Type **orders** in the Form field, and then click **Search**.
   The orders form opens.
3. Click **More** or the More Actions icon and then select **Link**.
   The link record for the orders file opens.

4. Create a link for the code field. Link the (Seller Code) code field on the order file to the (Seller Code) code field on the sales file.
5. Click **Save**.
6. Click **Back** to exit the link record and return to Forms Designer.

Note: Use $file in virtual join link line queries and $File in fill and find link line queries.

Related topics

Virtual joins
Understanding subforms
Forms Designer
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Use the virtual join
Verify that the sales1 form works
Subform control

1.2.3.2.9. Use the virtual join

Applies to User roles: System Administrator

Once you save the record, you see the virtual join information whenever you select, add, or update the record.

To create a new orders record using the orders form.

1. Click **Tailoring** > **Database Manager**.
2. Type **orders** in the Form field, and then click **Search**.
   The "orders" form opens.
3. Type the name of a customer in the **Customer Name** field.
4. Type a contact for that customer in the **Contact** field.
5. Type a number for the **Phone** field.
6. Type any number for the **Order Amount** value.
7. In the **Seller Code** field, select a seller code from the record list of seller codes.
8. Click **Add** to save the new orders record.

The new orders record appears with the sales file data virtually joined in the subform at the bottom.

The Seller name and commission fields aren't stored with the record in the order file. They are only referenced and are noneditable. To make changes or updates to the seller data, use the sales form.

Note: If you delete this record, Seller name and commission don't delete because they are stored in the sales file.

Related topics

Virtual joins
Understanding subforms
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Verify that the sales1 form works

1.2.3.2.10. Verify the sales1 form works

Applies to User roles: System Administrator

To verify that the sales1 form works:

1. Click **Tailoring** > **Database Manager**.

2. Open the sales1 form.
3. Click **Search** and select a record from the record list.

The first *sales* form inputs sales personnel data that's retrieved through the *sales1* subform. The *sales1* subform is virtually joined to the orders form.

Related topics

Virtual joins
Understanding subforms
Example: Creating a virtual join
Build the sales form
Create the sales file
Create the sales record list
Add data to the sales file
Create the sales1 subform
Create the orders form
Build the virtual join into the form
Build the link
Use the virtual join

1.2.4. Entity Relationship Diagram creation utility

An Entity Relationship Diagram (ERD) is a graphic description of the logical structure of a database. Database administrators often rely on conceptual and physical data models to understand the relationships among tables, records, and data, and to produce reports.

The Entity Relationship Diagram (ERD) Creation utility enables you to generate Database Definition Language (DDL) statements that you can import into any tool that's capable of producing this type of conceptual data model. For example, Erwin Data Modeler can create a visual Service Manager data model when you import a DDL file created by the ERD Creation utility. If you have administrative rights, you can customize the files and links used to create the component DDL statements.

Related concepts

DDL mode
What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file
Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.1. Checklist: Creating an ERD record for an application

Ensure that you consider the following items prior to creating an ERD record for an application.

1. Identify all files that are related to the application name.
2. Identify the primary link records for all files listed on the Files tab. Primary link records are the link records with the same name as the application files.
3. Identify all link records that are associated with any file listed. The ERD Creation utility associates these link records by the format name.

Related concepts

DDL mode
Entity Relationship Diagram creation utility
What's the required input for ERD Create?
Checklist: Generating a DDL file
Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.2. What's the required input for ERD Create?

The ERD Create utility uses a variety of input to generate DDL statements. The following list describes the basic input requirements.

- The list of files specified in the erdcreate record and all fields in these files.
- The relationships among these files determined by the link records specified in the erdcreate record.
- If specified, the relationships among the files specified by joindef and erddef records.
- If specified, the relationships among the files specified by the Match Fields section of Data Policy.
- If specified, the relationships manually defined by the database administrator.

Related topics

DDL mode
Entity Relationship Diagram creation utility
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file
Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.3. Create an ERD definition

Applies to User roles: System Administrator

An Entity Relationship Diagram (ERD) is a graphic description of the logical structure of a database. Database administrators often rely on conceptual and physical data models to understand the relationships among tables, records, and data, and to produce reports.

To create an ERD definition:

1. From the System Navigator, click **Tailoring** > **Database Manager**.
2. In the Form field, type **erddef** and click **Search**.
3. From the Associated Forms section, double-click the **erddef.g** form.
4. Enter the **First Filename**, **Second Filename**, and **Relationship Type**.
5. In the **Field Names from First Filename** text box, enter the fields for which you want to create the join.
6. Enter the fields for which you want to create the join in the **Field Names from Second Filename** text box.
   In this example, the contact.name field in the probsummary table contains the same data as the contact.name field in the contacts table.
7. Click **Add**.

For additional information about relationship types, see **Create a join** in the related topics.

1.2.4.4. Create an ERD record

Applies to User Roles: System Administrator

An Entity Relationship Diagram (ERD) is a graphic description of the logical structure of a database. Database administrators often rely on conceptual and physical data models to understand the relationships among tables, records, and data, and to produce reports.

To create an ERD record:

1. Click **Tailoring** > **Tailoring Tools** > **ERD Create Records**.
2. Type the **Name** (usually the application name) for the ERD Create record.
3. Click **More** or the More Actions icon and then select **Add Files for Module**.
   Note: The Add Files process will fail if the name of the ERD record is not associated with one or more Data Policy records.
4. Click **More** or the More Actions icon and then select **Add Master Links**.
5. Click **More** or the More Actions icon and then select **Add all Related Links**.
6. Click **Add**.
7. Click **OK**.

Note: You can add relationships manually from the Manual Relationships tab. Use this tab to define relationships that don't occur in any of the automatically discovered categories. Each relationship that you create manually also appears in the ERD.

Related topics

DDL mode
Entity Relationship Diagram creation utility

What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file
Associate a data policy record with an application
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.5. Modify an ERD record

Applies to User Roles: System Administrator

An Entity Relationship Diagram (ERD) is a graphic description of the logical structure of a database. Database administrators often rely on conceptual and physical data models to understand the relationships among tables, records, and data, and to produce reports.

To modify an ERD record using the Entity Relationship Diagram (ERD) Creation utility:

1. Click **Tailoring** > **Tailoring Tools** > **ERD Create Records**.
2. To locate an existing ERD record, click **Search**.
3. Select a record from the record list.
4. Click **More** or the More Actions icon, and choose **Add Files for Module**.
5. From the More Actions menu, choose **Add Master Links**.
6. From the More Actions menu, and choose **Add all Related Links**.
7. Click **Save**.
8. Click **OK**.

Notes:

- You can add relationships manually from the Manual Relationships tab. Use this tab to define relationships that don't occur in any of the automatically discovered categories. Each relationship that you create manually also appears in the ERD record.
- The ERD record creation process will fail if the name of the ERD record is not associated with one or more Data Policy records.

Related topics

DDL mode
Entity Relationship Diagram creation utility
What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file

Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool

1.2.4.6. Create manual relationships

Applies to User Roles: System Administrator

An Entity Relationship Diagram (ERD) is a graphic description of the logical structure of a database. Database administrators often rely on conceptual and physical data models to understand the relationships among tables, records, and data, and to produce reports.

To create manual relationships in the ERD definition:

1. Click **Tailoring** > **Tailoring Tools** > **ERD Create Records**.
2. Click **Search** to display a list of existing records.
3. Select an application name, and then select the **Manual Relationships** tab.
   Note: The Manual Relationships tab enables you to define any relationship that's not automatically detected by Service Manager through link, erddef, and datadict records. You must identify the two files that are related and the specific fields that create the relationship.
4. Type a user-defined unique **Name** for the relationship.
5. Type the name of a source file where the relationship originates in the **From File** text box.
6. Type the name of a field in this file in the **From Field** text box.
7. Type the name of a target file in the **To File** text box.
8. Type the name of a field in this file in the **To Field** text box.

9. Click **Save**.
10. Click **OK**.

Related topics

DDL mode
Entity Relationship Diagram creation utility
What's the required input for ERD Create?
Associate a data policy record with an application
Create an ERD record
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.7. What are primary link records?

Primary link records are link records that have the same name as the associated file. For example, the primary link record for the contacts file is the contacts link record.

Related topics

DDL mode
Entity Relationship Diagram creation utility
What are related link records?
What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file
Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.8. What are related link records?

Related link records are form records that relate to the associated files and link records that relate to these form records. The ERD Create utility collects related link records by finding all forms associated with the selected files, and then locating any link records associated with these forms.

Related topics

DDL mode
Entity Relationship Diagram creation utility
What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file
Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.9. Associate a data policy record with an application

Applies to User Roles: System Administrator

Every data policy record doesn't automatically have associated applications. You may need to complete Step 4 to ensure there is a link between the data policy record and the appropriate application. When the ERD Creation Utility runs for a named application, the utility looks for all data policy records with links to that application to generate the list of associated files.

To associate a data policy record with an application:

1. Click **Tailoring** > **Data Policy**.
2. Click **Search** to generate a list of existing data policy records.
3. When you select a data policy record from the list, the fields in this record appear in the Data Policy form.
4. The General tab shows the current applications associated with the data policy record. To add another application, insert the cursor in a blank **Modules** row. Click the drop-down list to choose a Service Manager application.

5. Click **Save**.
6. Click **OK**.

Related topics

DDL mode
Entity Relationship Diagram creation utility
What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.10. Checklist: Generating a DDL file

Ensure that you consider the following items prior to generating a DDL file for an application.

1. Ensure that there is one or more data policy record associations for the application.
2. Create a new ERD record that lists all related files and fields.
3. Define any additional manual relationships.
4. Use the ERD Create utility to generate a DDL file that contains DDL statements.
5. Open the DDL file to import the DDL statements into a database modeling tool.

Related topics

DDL mode
Entity Relationship Diagram creation utility
What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.11. DDL mode

The DDL mode describes how the ERD Create utility creates foreign key definitions. You can select from one of two modes, Full mode or Simple mode.

- Full mode
  Full mode creates one foreign key definition for each unique field or file relationship. In Full mode, you can have multiple links between two files.
- Simple mode
  Simple mode creates one foreign key definition for each file-to-file relationship within each relationship type.

Related topics

Entity Relationship Diagram creation utility
What's the required input for ERD Create?
Checklist: Creating an ERD record for an application
Checklist: Generating a DDL file
Associate a data policy record with an application
Create an ERD record
Create manual relationships
Generate a DDL file
Import a DDL file into a database modeling tool
Modify an ERD record

1.2.4.12. Generate a DDL file

Applies to User roles: System Administrator

Database administrators often rely on conceptual and physical data models to understand the relationships among tables, records, and data, and to produce reports. If you have administrative rights, you can customize the files and links used to create the component Database Definition Language (DDL) statements.

To generate DDL statements in a file using the Entity Relationship Diagram (ERD) Creation utility:

1. Click **Tailoring** > **Tailoring Tools** > **ERD Create Records**.
2. Choose the correct export mode for the DDL statements.
    - **Full mode**: Creates one foreign key definition for each unique field or file relationship. You can have multiple links between two files.
    - **Simple mode**: Creates one foreign key definition for each relationship between the files within each relationship type.
3. From the ERD Creator Definition form, click the More Actions menu.
4. Select one of the following:
    - **Create/Export DDL** > **Full**.
    - **Create/Export DDL** > **Simple**.

The output of the ERD Create utility automatically resides at the root level of the Service Manager client installation folder. The file name is the DDL file name with a .sql file extension. For example:

`..\Program Files\OpenText\Service Manager x.xx\Client\`*ddl_name*`.sql`

Related topics

- DDL mode
- Entity Relationship Diagram creation utility
- What's the required input for ERD Create?
- Checklist: Creating an ERD record for an application
- Checklist: Generating a DDL file
- Associate a data policy record with an application
- Create an ERD record
- Create manual relationships
- Modify an ERD record
- Import a DDL file into a database modeling tool

1.2.4.13. Import a DDL file into a database modeling tool

To import a Database Definition Language (DDL) file into a database modeling tool, this example uses Erwin Data Modeler. Other tools may have different steps.

1. From the Erwin Data Modeler, click **File** > **Open**.
2. Change the file type to **SQL DDL**.
3. Browse to locate the DDL file.
   By default, Service Manager stores these files at the root level of the client installation folder. For example, **`..\Program Files\OpenText\Service Manager x.xx\Client\`**
4. Click the file to select, and then click **Open**.
   The Erwin Data Modeler displays the Reverse Engineer - Select Template wizard.
5. Select the appropriate Database and Version in the Target Database section.
6. Click **Next**.
7. Select the appropriate values.
8. Click **Next**.
   The resulting Entity Relationship Diagram (ERD) opens. You can use the Zoom tool to view the content of each entity in the diagram.

Related topics

- DDL mode
- Entity Relationship Diagram creation utility
- What's the required input for ERD Create?
- Checklist: Creating an ERD record for an application
- Checklist: Generating a DDL file
- Associate a data policy record with an application
- Create an ERD record
- Create manual relationships

- Generate a DDL file
- Modify an ERD record

1.2.5. Posting

Posting is the process of copying data from a source record to a target record. The purpose of posting is to update similar fields in other records without having to open those records to modify each field. The relationship between the two records is normally based on a value defined in one (or more) of the source record input fields. Posting is the opposite of the Fill function, which copies data from the target record to the source record. You can use posting to update Service Manager tables from another part of Service Manager.

Posting does the following:

- Starts on a source record.
- Accesses a record in a secondary (target) table, based on data in that record.
- Copies (posts) data from the source to the target, and then performs some action (for example, add, update, or open) on the target record.

The posting process enables you to control the process in several ways:

- Define the target table.
- Select the fields to copy.
- Determine whether to prompt the user for add/update confirmation.
- Decide which application to invoke for the posting process.
- Define manual or automatic posting.

Note: Do not post to the probsummary table or to incident records. The record being updated may be locked by another user and the post will not occur. In addition, the pm.eval.alerts application will create its own $file variable, which conflicts with the $file variable that post.fc is using. Updating the probsummary table triggers an application that cleans out the $file variable, the same variable that's used by the Format Control routines.

Related topics

Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.1. Posting link records

The posting process requires you to create a standard Service Manager link record. The link record establishes the connection between similar fields in the source and target files. When posting, it is best to create a new link record dedicated to the posting process, to avoid conflicts with link records defining common fill relationships.

To refer to fields in the source record, use the `$File` file variable. To refer to fields in the target record, use the `$Filet` file variable.

Important: The case must match the example, with the initial 'F' upper case and the rest lower case.

Related topics

Posting
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting

Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.2. Posting link line definition file

The link line definition file defines a single field as the reference field for the posting routine and lists the names of other fields linked for the posting process. When posting is invoked, the system compares the value for the reference field in the source file with the value for the linked field in the target file. If the values are the same, the system compares the listed fields for changes and posts the new data to the target file.

The fields used for the link line definition have the same definition or use for posting as they do for find and fill with the following exceptions:

- The source field array lists fields in the source record and the target field array lists fields in the target record.
- You can define parameters in the Expressions field to control the posting process flow.

Note: Reference or manipulate input fields with the $File file variable. Any modifications made to this record variable are passed back to the calling application.

Important: Processing statements may modify the contents of the original record.

Related topics

Posting
Posting link records
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.3. Types of posting available in Format Control

There are two types of posting available in Format Control:

- Automatic posting, set up via Format Control subroutines.
- User-controlled posting, set up via the Format Control additional options for use with Database Manager only.

Related topics

Posting
Posting link records
Confirming the posting routine
Automatic updates using Format Control subroutines
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.4. Confirming the posting routine

A confirmation function is available in Posting that posts the new contact information to the target file, but waits for user approval before updating the record. The target file is open to the user, who can edit any of the information before confirming the update. For example, when the Posting routine from Change Management creates a new Contact record, only the fields specified in the Link record populate. At some point, the record must be complete. When the confirmation feature is active, the new Contact record opens to the user when Posting occurs. The user can complete the contact information form, save the record, and then return to the request without leaving Change Management.

Note: This feature is available in both the automatic and the manual Posting modes.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Automatic updates using Format Control subroutines
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.5. Posting variables

The following variables are available during the posting process and can be used to control the Posting process flow:

$File
The source record file variable.

$Filet
The target record file variable.

$post.msg
The message issued to the operator when the posting routine successfully completes. The default message is: The posting routine is complete.

$post.confirm
A logical field that controls whether on not the user is prompted to confirm the Posting action. The default is false.

$post.confirm.fmt
The name of the form used to display the target record for Posting confirmation.

$post.appl
The name of the RAD routine used for posting. If an application is undefined to this variable, and the $post.confirm option is true, the confirmation is a simple add or update of the target record. No other processing is performed against the record.

$post.names
An array that defines the names of the parameter panel input fields to which data will be passed. This field is needed only when an application is defined to the $post.appl parameter.

$post.values
An array that defines the values passed to the called application. There is a one-to-one relationship between this array and the $post.names array.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Identify field input values for posting

Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Manual posting using Format Control additional options
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation

1.2.5.6. Automatic updates using Format Control subroutines

You can use posting to update Service Manager tables from any other part of Service Manager. For example, new contact information from a request can post directly to related fields in the contacts file without any user interaction. If you are opening a request for a new contact, posting creates a new Contact record and populates the common fields defined in the Link record. Posting from Request Management can occur when new requests are opened.

In this example, ocmq.open.newemployee.quote.g is our Source Format (accessing the ocmq file) and contacts is our Target form (accessing the contacts file). Both files contain such contact information as contact name, company, first name, last name, and department. The relationship between the two files is based on a single common field—the employee.id field in this example (Employee Number). When Posting initiates from a request, the system attempts to find a Contact record in which the value of the user.id field matches the value of the employee.id field in the request. If a match is found, the system updates the Contact record. If no match is found (as in the case of a request from a new person), the system creates a new Contact record based on the values in the request.

Posting should NOT be performed on the probsummary table when set up using a subroutine in Format Control as this causes the system to clear the variable, $file.

Use the procedures in the following help server topics to create automatic updates using Format Control subroutines:

1. Identify the field input values for posting
2. Create the link record for posting
3. Create the Format Control record for automatic posting
4. Open a request for posting

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Manual posting using Format Control additional options
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.6.1. Identify field input values for posting

Applies to User Roles: System Administrator

To create a Link record for Posting, you must identify the common fields in the Source and Target files. This can be accomplished from within Forms Designer. For this example, individual fields are selected in the design mode.

To identify common fields in the Source and Target files:

1. In Forms Designer open the form `ocmq.open.newemployee.quote`.
2. Click **Design**.
3. Select a field and check the Input value in the Property window.

Note: This method displays all input values, including those not shown in Fill boxes.

Important: Be certain not to alter the form when checking field input values. Click **Cancel** to exit the Design mode without changing the form.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Manual posting using Format Control additional options
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.6.2. Create the link record for posting

Applies to User Roles: System Administrator

Create a new Link record for the form you are going to use. For example, if a Link record doesn't already exist, create the Link record ocmq.post.contacts for the new form ocmq.open.newemployee.quote for the Posting process. This avoids any conflicts involving links established in a current record for the Fill function.

1. Click **Tailoring** > **Tailoring Tools** > **Links**.
2. In the Name field, type the name of the Link record you want to create. For this example, type **ocmq.post.contacts**.
3. Click **New**.
4. Type the following values in the first line of the new Link record.

Field	Value
Field Name (SOURCE)	employee.id
Format Name (TARGET)	contacts
Field Name (TARGET)	user.id
Comment	POST

   Important: You must type POST in upper case letters in the Comment field for the Posting process to work.
   The new Link record opens.
5. Place the cursor in the employee.id line and click **Select Line** from the More Actions menu.
   The Link line definition form opens.
6. Type the following values for the Source and Target fields.

Source Field	Target Field
employee.id	user.id
requested.for	contact.name
employee.id	full.name
request.phone	contact.phone
request.dept	corp.structure

7. Click **Back**. A prompt asks if you want to save the Link record you have created.
8. Click **Yes** to save the record.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Identify field input values for posting
Create the Format Control record for automatic posting
Open a request for posting
Manual posting using Format Control additional options
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.6.3. Create the Format Control record for automatic posting

The Subroutines process of Format Control defines Automatic Posting. The application called is post.fc. Posting takes place before the request is added or updated in the database.

Important: As posting may add a new record to the target file, you must fill in all fields that are part of a Unique or No Nulls key using the Source Field / Target Field table, otherwise posting fails.

1. Click **Tailoring** > **Format Control**.
2. In the **Name** field of the Format Control Initializations form, type **ocmq.open.newemployee.quote**.
3. Click **Search**.
4. If no record exists, click **New**.
   The Format Control record for ocmq.open.newemployee.quote opens.
5. Click **More** or the More Actions icon and then select Subroutines. You can also click **Subroutines** on the form.
6. Click **More** or the More Actions icon and then select Show Expanded Form.
   The long version of the Subroutines form opens.
7. Type the following values in the first available slot.

Names	Values
file	$file
name	ocmq.post.contacts
prompt	emoloyee.id
Error Message	Could not post to contacts file.
Application Name	post.fc
Comments	POST
Add	true
Before	true

   Note: The name parameter passes the name of the Link record you created, and the prompt parameter passes the source field for the Link relationship.

8. Click **Back**. A prompt asks if you want to save the changes to the Format Control record.
9. Click **OK** to save the record.

Related topics

Posting
Posting link records
Posting link line definition file

Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Identify field input values for posting
Create the link record for posting
Open a request for posting
Manual posting using Format Control additional options
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.6.4. Open a request for posting

Applies to User roles: System Administrator

To test the posting routine you have set up, you must create a new request. We will update the contacts file by changing the values in the First Name and SM Contact Name fields of the request, and then add a title.

1. Click **Request Management** > **Quotes** > **Create New Quote**.
2. Select **Human Resources** > **New Employee Setup**.
3. A summary form of the items you requested displays with total cost. You can choose to:
    - Cancel your request.
    - Add Items to your request.
    - Submit Request.
4. Click **Submit Request**. The Human Resources - New Employee Setup Request form opens.
5. Place the cursor in the **SM Contact Name** field of the Employee Information section, and then click **Fill**.
6. Place the cursor in the **Contact Name** field, and click **Search**. A record list of contacts opens. The contact information comes from the contacts file.
7. Select a name from the record list of contacts.
   Service Manager populates the Employee Information section with information from the contacts file.
8. Change the **First Name** and **SM Contact Name** in the Human Resources - New Employee Setup Request form.
9. Add a **Title**.
10. Fill in the required fields.
11. Copy the **Employee Number** to use it in the contacts file later.
12. Click **Save & Exit**.
13. If errors occur, read the posting messages in the Messages view.

To verify changes made to the contacts file during the posting process.

1. Click **System Administration** > **Base System Configuration** > **Contacts**.
2. Paste the Employee Number copied earlier into the **Employee ID** field, and then click **Search**.
   The Contact Information form opens. You can see the contact information you updated in the request, along with the title you added for the employee.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Manual posting using Format Control additional options
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.7. Manual posting using Format Control additional options

In some cases, you may want to give users the option to manually control posting. The additional options process of Format Control defines user options for posting data and require user interaction to complete the routine.

Manual posting is desirable whenever you don't want changes to post automatically. You might want to do this to avoid posting temporary changes to permanent records. For example, if the contact is calling from a different phone, you could add the temporary number to the change request without posting to it in the contacts record.

Additional options are only available for Database Manager. To set up manual posting within other Service Manager applications, set up a new display option definition to call the post.fc application.

In this example, we will reuse our posting link record from Database Manager and define additional options for a hardware change request form. The user will be able to click on a button to post updates to contact information. As in the previous example, the source file is cm3r, and the target file is contacts.

Note: We don't recommend accessing Change Management forms and records using Database Manager. We do so in this example only to demonstrate additional options capabilities for manual posting.

Perform the steps in the following help server topics to manually control posting using Format Control:

1. Create the link record for posting
2. Create the Format Control record for manual posting
3. Modify a hardware change request for posting

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.7.1. Create the Format Control record for manual posting

Applies to User Roles: System Administrator

Manual posting is user-controlled posting, set up via Format Control Additional Options, for use with Database Manager.

1. Click **Tailoring** > **Format Control**.
2. In the **Name** field of the blank Format Control record, type the name of the form. For this example, type cm3r.hardware. Click **Search**
   Note: If the Format Control record doesn't already exist, click **New** to create a new Format Control record.
3. Click **More** or the More Actions menu, and choose **Additional Options**. You can also click **Addl Options** in the form.
4. Click **More** or the More Actions menu, and then choose **Show Expanded Form**.
   The long version of the Additional Options form opens.
5. Type the following field values in the first available slot.

Field	Value
Option	1
Condition for Option	true
Description	Post to Contacts
Description ID	
Comments	POST
Application	post.fc
Error Message	Could not post to contacts file.
Reset on Return	true

Names	Values
file	$file
name	cm3r.hardware.post
prompt	requested.by

6. After entering the second parameter name and value (name/cm3r.hardware.post), click **Save** to save the updated Format Control record.
   A scroll bar appears next to the Names array, so you can scroll down and enter the third Names value (prompt).
7. To enter the third Values value (requested.by), position the cursor on the first Value entry($file) and press the Page Down key.
8. Click **OK** to save your changes.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Manual posting using Format Control additional options
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.7.2. Modify a hardware Change Request for posting

Applies to User Roles: System Administrator

To test the manual Posting routine you set up, you must access a hardware change request and modify one or more fields in the Initiator section that was defined in the link record. Then, you will invoke the Additional Options to verify that the contacts file data changes do reflect the changes in your change request.

We will do this in the Database Manager Administrator mode, because Format Control Additional Options aren't available in Change Management.

1. Click **Tailoring** > **Database Manager**.
2. Select the **Administration Mode** checkbox.
   Note: You must click Administration Mode or you will be using Change Management instead of Database Manager. Format Control Additional Options aren't available in Change Management.
3. Open the **cm3r.hardware** form.

4. Click **Search**.
   A list of change request records opens.
5. Retrieve any existing Hardware change request.
6. Verify that the existing Initiated By information matches an existing contacts file record. To do so, position the cursor on the **Name** field, and then click **Find**.
   The related Contact Information record opens.
7. If no contacts record match exists, do the following:
   1. Clear the existing **Name** field, and then use the Fill button to fill a valid contact value into the change request record.
   2. Update the **Department** and **Phone** fields.
   3. Click **Post to Contacts** from the More Actions menu to initiate posting to the contacts file.
      Note: The option name depends on the value entered in the Command field of the Additional Options definition. Therefore, you may see something other than Post to Contacts in your More Actions menu.
   4. Place the cursor in the **Change Initiator Name** field, and then click **Find**. The updated Contact record opens.
      The updated departmental data from the change request opens in the Corp. Structure field at the bottom of the Business Information tab, and the updated phone data opens in the Work phone field in the Contact Numbers tab.
8. Click **Save**.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Manual posting using Format Control additional options
Create the Format Control record for manual posting
Use the confirmation function in the posting routine
Do manual posting with confirmation
Posting variables

1.2.5.7.3. Use the confirmation function in the posting routine

Applies to User Roles: System Administrator

Activate the confirmation function by setting the flag $post.confirm to true in the Expressions field of the source file Link line definition record.

1. Click **Tailoring** > **Tailoring Tools** > **Links**.
2. Type the name of the source file Link record in the **Name** field of the Link Manager dialogue box. In this example, use the name of the Link record you created, cm3r.hardware.post.
3. Click **Search**. The source file Link record opens.
4. Place the cursor in the requested.by field of the Link record, and then click **Select Line** from the More Actions menu. The Link line definition form opens.
5. Type $post.confirm=true in the Expressions field.
6. Click **Back**. A prompt box opens, asking if you want to save the changes to the Link record.
7. Click **Yes**.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting

Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Do manual posting with confirmation
Posting variables

1.2.5.7.4. Do manual posting with confirmation

Applies to User Roles: System Administrator

Use the same procedures for evoking the confirmation process as for establishing User Options in manual posting. When you activate the Posting routine, the confirmation process opens the target form and file (contacts) and provides a new set of toolbar buttons.

1. Click **Tailoring** > **Database Manager**, and then select the **Administration Mode** checkbox.
   Note: You must click Administration Mode or you will be using Change Management instead of Database Manager. Format Control Additional Options aren't available in Change Management.
2. Open the cm3r.hardware form.
3. Retrieve any existing Hardware change request.
4. Verify that the existing Change Initiator matches an existing contacts file record. To do so, position the cursor on the Change Initiator Name field, and then click **Find**. The related Contact Information record opens on form contacts.g.
5. If no contacts record match exists, clear the existing Change Initiator Name field, and then use the Fill button to fill a valid contact value into the change request record.
6. Update the Department and Phone fields.
7. Click **Post to Contacts** from the More Actions menu to initiate posting to the contacts file.
   Note: The option name depends on the value entered in the Command field of the Additional Options definition. Therefore, you may see something other than Post to Contacts in your More Actions menu.
8. The Contact record (target file) for the contact specified in the change request form opens with a confirmation request.
   Edit fields other than those selected for the Posting routine.
9. Click **Save** to save ALL changes, from both the change request and also changes entered directly in the contact record, to the database.
10. Click **Cancel** to ignore any changes you made in the record. The Posting process is NOT complete and you return to the change request form.

Related topics

Posting
Posting link records
Posting link line definition file
Types of posting available in Format Control
Confirming the posting routine
Automatic updates using Format Control subroutines
Manual posting using Format Control additional options
Identify field input values for posting
Create the link record for posting
Create the Format Control record for automatic posting
Open a request for posting
Create the Format Control record for manual posting
Modify a hardware change request for posting
Use the confirmation function in the posting routine
Posting variables

1.3. Display functions

The system permits control of user interaction with the following utilities:

- Menus — enable you to control the functionality of the options that appear on the system navigator as well as the buttons that appear on the user's menu form. Different users access different menu forms based on different roles.
- Display application — enables you to control the functionality of buttons in the editor toolbar, on forms, and on items in the pull-down options menu.

Related topics

Tailoring
Menus

Display application
Displayoptions
Selecting display options

1.3.1. Menus

The menu of utilities and applications available to an individual user are based on the user's role in the organization.

You can create and customize menus to restrict user access.

A menu form:

- Opens when you log on to the system
- Controls the user's access to different areas of the system
- Is associated with a corresponding menu record
- Opens based on the user's role in an organization

Related topics

Button properties
Menu record
Example: Adding a URL link to a menu
Access menu records
Add or change the image icon for options menu and toolbar items

1.3.1.1. Menu record

Each option line in the menu record contains values that control the functionality of each link on the menu form. Based on the conditions specified, the menu record can call a RAD application to retrieve other menus, call stored queries, and launch external applications. The association between the form and the menu record is set up by entering the form name in the Format field of the menu record.

The numbers in the Option column correspond to the option on the referenced format. In general, there is a one-to-one relationship between menu and form. Each option calls a RAD application. Some RAD applications require parameters to be passed into the function while others don't.

The Command column permits you to call a menu option directly from the command line. For example, type `db` in the command line to go directly to the Database Manager.

The Condition column on the menu record controls the availability of the option. If the condition evaluates to false, the option displayed on the user's menu is unavailable.

Related topics

Menus
Button properties
Example: Adding a URL link to a menu
Access menu records
Add or change the image icon for options menu and toolbar items

1.3.1.2. Button properties

You can create menu buttons using the Button control in Forms Designer. When adding a button in Forms Designer, the following properties are important:

- ButtonID – A numeric value that matches an option number in the related menu record. (Required)
- Images – An image that represents the button's function. (Optional)
- Balloon help – A text string that appears when the cursor is held over the button to offer more information about the button's function. (Optional)

Related topics

Menus
Menu record
Example: Adding a URL link to a menu
Access menu records
Adding and changing the image icon for menu and toolbar items

1.3.1.3. Access menu records

Applies to User Roles: System Administrator

After you create a form, you must add a related menu record to specify the functionality of the buttons. Menu records are saved in the menu file. You can access menu records in the following ways:

- Click **Tailoring** > **Tailoring Tools** > **Menus**.
- In the Service Manager command line, type `menu`.

Related topics

Menus
Button properties
Menu record
Example: Adding a URL link to a menu

1.3.1.4. Example: Adding a URL link to a menu

Applies to User roles: System Administrator

Objective: Provide an example of how to add a link that navigates to a URL from the System Navigator tree or from a Service Manager menu form.

In this example, you need to find the menu file, modify the HOME menu record, and use Forms Designer to modify the menu.gui.home form.

To add a link that navigates to a URL from the System Navigator tree or from a menu form:

1. Click **Tailoring** > **Web Services** > **Database Manager**.
   The Database Manager search window (format.prompt.db.g) opens .
2. Type `menu` in the File text box and click **Search**.
   A record list of the files opens (format.qbe.g).
3. From the record list of menu files, double-click menu in the File Name column.
   The Menu form opens (menu.g).
4. Type `HOME` in the Menu Name field and click **Search** from the More Actions menu.
   The Menu form re-displays with a list of menu records for the HOME menu.
5. In the table, add a new menu entry with the following information:
   - Option #: 48
   - Description: <*Sample description*> (The label to give the URL link.)
   - Application: us.launch.external
   - Parameter: name
   - Parameter value: <*Sample URL*>
   - Thread: true
   - Condition: true
6. From the More Actions menu, click **Save**.
   Next you will use Forms Designer to add the menu item as a button on the form:f menu.gui.home.
7. Click **Tailoring** > **Tailoring Tools** > **Forms Designer**. Type `fd` in the Service Manager command line.
   The Forms Designer search form opens.
8. Type `menu.gui.home` in the Form text box and click **Search**.
   The menu.gui.home form opens in Forms Designer.
9. Click **Design**. The form opens in the edit window with the Design Tools and Properties panes open.
10. Drag-and-drop a button control from the Design Tools pane to the form.
11. Position the button control on the form. In this example, move the Exit Service Manager button, Service Manager Internal Mail button, and System Status button left to make room for the button you are adding.
12. Click the new button to display the sizing nodes on the button and then resize the button to match the other buttons.
13. In the Properties pane, update the following parameters:
    - Image File: Hewlett-Packard Enterprise
    - Button ID: Type the Option # (48) you entered for this record in the HOME file.
    - Caption: <*Sample caption*>
    - Name: <*Sample name*>
14. Click **OK**.
15. To verify the change, logout and then login and view the form you modified.

You should see the button you added on the menu.gui.home form. There should also be a new entry in the Menu navigator folder in the System Navigator.

Related topics

Forms Designer
Menus
Button properties
Menu record
More Actions menu
Access menu records

1.3.1.5. Customize System Navigator menu icons

Applies to User roles: System Administrator

Service Manager provides out-of-box icons that you can use to customize the icons in the System Navigator. To do this, you only need to specify the name of the icon in the **Icon** field of the corresponding menu record. You can perform this task from either the web client or the windows client. The customized icons apply to both the accordion mode and the sidebar mode of the System Navigator.

Note

Note

- You can only specify an icon for a first-level menu item (menu group).
- The System Navigator menu icons are available only when the **enableSidebarMenu** parameter in the web.xml file is configured as TRUE. For more information about the **enableSidebarMenu** parameter, see Web parameter: enableSidebarMenu.

For more information about how to access the menu record, see Access menu records.

Related topics

Menus
Access menu records
Button properties
Menu record
Example: Adding a URL link to a menu

1.3.1.6. Adding and changing the image icon for menu and toolbar items

You can edit or replace the out-of-box icon for existing menu items and add icons for new menu and toolbar items that you create. The More Actions menu and toolbar icons are 16x16 .gif files. You can use any graphics design tool to edit out-of-box icons or create new icons of the correct size.

When you create a menu or toolbar item, Service Manager generates an icon name for it based on the **Default Label** you give the menu or toolbar item in the Display Application Option Definition form.

Service Manager displays the image with that name if it is located in the Service Manager Client images directory. If there is no image with that name, the form displays the default icon. The default icon is blank (no icon defined) in the out-of-box system.

In a typical installation the icons are located in the following locations.

- Windows client:

```
...\plugins\com.hp.ov.sm.client.common_x.xx\src\resources\icons\obj16
```

- Web client on Tomcat (for example):

```
...\<Tomcat>\webapps\webtier-x.xx\images\obj16
```

Related topics

Menus
Menu record
Example: Adding a URL link to a menu
More Actions menu
Add or change the image icon for options menu and toolbar items
Common option icon names

1.3.1.6.1. Add or change the image icon for menu and tool bar items

Applies to User roles: System Administrator

You can edit or replace the out-of-box icon for existing menu items and add icons for the new menu and toolbar items that you create. The More Actions menu and toolbar icons are 16x16 .gif files. You can use any graphics design tool to edit out-of-box icons or create new icons of the correct size. These images are found in many forms.

To add or change the image icon for the options menu and toolbar items:

Important: If you change an image, Service Manager will replace the original image with the new image on all forms. If you don't want to change the image on all forms, you must change the **Default Label** in the Display Application Option Definition for the form you want to change, and create a new image.

1. Open the Display Application Option Definition for the More Actions menu or toolbar item.
2. Add the item, if necessary.
3. Make a note of text in the **Default Label**. Using Database Manager, open a form that the option is displayed on.
4. Open the Administrator view and go to the **Detail Data tab** for a single record form or the **List Data tab** for a record list. The list of icons in the toolbar and options menu is at the bottom of the form. The `img="<bmpname>"` tag gives you the name of the image.
5. Make a note of the icon image name.
6. Using your standard operating system search functionality, search your Service Manager directory for a .gif file with that name. If the item with that name exists in the Service Manager images directory, the form will display it.
    - `<installation location>\plugins\com.hp.ov.sm.client.eclipse.web_x.xx\webtier\images`
    - `<installation location>\plugins\com.hp.ov.sm.client.common_x.xx\src\resources\icons`
    - `<installation location>\plugins\com.hp.ov.sm.client.eclipse.admin_x.xx\src\resources\icons`
    - `<installation location>\plugins\com.hp.ov.sm.client.eclipse.user_x.xx\src\resources\icons`
7. Create a new image if none exists, or replace this existing image with the one you prefer to use.

1.3.1.6.2. Common option icon names

When looking for the image to display, Service Manager replaces spaces in the image tag with _ in the filename.

Detail/List Data entry	Windows Client		Web Client	
	Default Icon	File name	Default Icon	File Name
<option value="1" img="tadd"> <caption>Add</caption> <description>Add</description> </option>		tadd.gif		tadd.png
<option value="11" img="tprev"> <caption>Previous</caption> <description>Previous</description> </option>		tprev.gif		tprev.png
<option value="11" img="tnext"> <caption>Next</caption> <description>Next</description> </option>		tnext.gif		tnext.png
<option value="2" img="tok"> <caption>OK</caption> <description>OK</description> </option>		tok.gif		tok.png
<option value="205" img="tprint"> <caption>Print</caption> <description>Print</description> </option>		tprint.gif		tprint.png

<option value="211" img="texport/"> <caption>Export/Unload</caption> <description>Export/Unload</description> </option>		texport.gif		texport.png
<option value="212" img="tir quer"> <caption>IR Query</caption> <description>IR Query</description> </option>		tir_quer.gif		tir_quer.png
<option value="3" img="tcancel"> <caption>Cancel</caption> <description>Done Searching</description> </option>		tcancel.gif		tcancel.png
<option value="300" img="tcreate"> <caption>Create Operator</caption> <description>Create Operator</description> </option>		tcreate.gif		tcreate.png
<option value="4" img="tsave"> <caption>Save</caption> <description>Save</description> </option>		tsave.gif		tsave.png
<option value="5" img="tdelete"> <caption>Delete</caption> <description>Delete</description> </option>		tdelete.gif		tdelete.png

<option value="5000" img="tmass ad"> <caption>Mass Add</caption> <description>Mass Add</description> </option>		tmass_ad.gif		tmass_ad.png
<option value="5001" img="tmass up"> <caption>Mass Update</caption> <description>Mass Update</description> </option>		tmass_up.gif		tmass_up.png
<option value="5002" img="tmass de"> <caption>Mass Delete</caption> <description>Mass Delete</description> </option>		tmass_de.gif		tmass_de.png
<option value="5003" img="tmass un"> <caption>Mass Unload</caption> <description>Mass Unload</description> </option>		tmass_un.gif		tmass_un.png
<option value="5005" img="tmass cr"> <caption>Mass Create Operators</caption> <description>Mass Create Operators</description> </option>		tmass_cr.gif		tmass_cr.png
<option value="5100" img="tcount"> <caption>Count</caption> <description>Count Records in the List</description> </option>		tcount.gif		tcount.png

<option value="5110" img="tprint l"> <caption>Print List</caption> <description>Print List</description> </option>		tprint_l.gif		tprint_l.png
<option value="5120" img="trefresh"> <caption>Refresh</caption> <description>Refresh</description> </option>		trefresh.gif		trefresh.png
<option value="5130" img="tmodify"> <caption>Modify Columns</caption> <description>Modify Columns</description> </option>		tmodify.gif		tmodify.png
<option value="5140" img="texport"> <caption>Export to Excel</caption> <description>Export to Excel</description> </option>		texport.gif		texport.png
<option value="5160" img="tsave as"> <caption>Save As View</caption> <description>Save As View</description> </option>		tsave_as.gif		tsave_as.png
<option value="8" img="tfind"> <caption>Find</caption> <description>Find</description> </option>		tfind.gif		tfind.png

<option value="9" img="tfill"> <caption>Fill</caption> <description>Fill</description> </option>		tfill.gif		tfill.png

1.3.2. Display application

The display application allows System Administrators to customize certain features of their system without altering RAD code.

Data files within Display contain the individual records, or screens, in which options, events, and window controls are defined. Each screen is attached to a particular application and controls the features appearing on the forms associated with that application. The information from these screens is stored in data tables and is not embedded in the RAD code. A system administrator has ready access to all Display features and may edit them freely.

Note: Because custom features are independent of RAD programming, Display options are protected from conflict during the upgrade process.

Related topics

Displayoptions
Selecting display options
Access display records

1.3.2.1. Displayscreen definitions

The Display Application Screen Definition (displayscreen) record is the control point for applying options to Service Manager forms using the Display application.

To access the Display Application Screen Definition form, click **Tailoring** > **Tailoring Tools** > **Display Screens**.

The fields available to apply displayscreen options are described below.

Field Name (*Required)	Field ID	Description
*Screen ID	screen.id	The unique name, identifying the record whose display characteristics are being defined. The names of screen IDs generally reflect their function and closely resemble the names of the forms to which they are attached.
On option 0	on.enter.key	The action to take when the user presses Enter--Option 0 (zero). • Redraw screen: Refreshes the screen. The cursor remains in the same field, but if this is a multi-line field, the focus resets to the top of the field. • Do nothing: With this action, the display application will execute any statements and RAD applications defined in the display option, and then return to the original screen. • Return to appl: Exits the current screen to the calling application.
Title	title	The title of the form. The value in this field can be a variable or an expression.
Format	form	The form to open for this screen. The value in this field can be a variable or an expression.
I/O (If RIO)	io	Determines the user's privileges for modifying the displayed data. Service Manager uses this field only when the displayed panel is replacing an rio panel.
Time (if FDISP)	time	The date and time field determining the length of time a user may search the database. Service Manager uses this field only when the displayed panel is replacing an fdisp panel. The value in this field can be an expression.
Used only for Search?	search.only	This option is selected on screens that are used only for searches (such as db.search).
User Options	user.options	If selected (set to true), triggers functionality in the Display application. Do not change this setting in the production system.
Language	language	The language used on the form. The default is English (EN).
Main tab		
Initializations subtab	expressions	Expressions that set the initial values for variables and fields. Service Manager evaluates these expressions before the screen appears.

Javascript subtab	javascript	The JavaScript subtab is executed after the Initializations subtab. It is used to set and initialize variables, and run expressions on the screen being initialized.
Options tab		
Option #/Label/Action/Condition	gui.option	Click **Search** () to select a Display Application Option definition. • Detail options: These numbers also depend on the display screen and whether the option will appear on the detail options or list options. For example, the display options greater than 200 with Screen ID apm.edit.problem appear on the detail options of an incident. • List options: These numbers also depend on the display screen and whether the option will appear on the list options or detail options. For example, the display options greater than 200 with Screen ID apm.list.problem appear on the list options of the incident.
Events subtab		
Event/Action/Condition	event	Click **Search** () to select a Display Application Event definition.

Related topics

Display application

1.3.2.2. Display application option definitions

The display application allows System Administrators to customize certain features of their system without altering RAD code. You can select an option that performs an important function in your system within the Display application option definitions form. The following fields can be defined in the Display Application Option Definitions (`displayoption.g`) form.

Field Name (*Required)	Field ID	Description
*Screen ID	screen.id	The name of the displayscreen record that this option is associated with.
Modifies Record	modify.record	If selected (set to true), indicates that using this display option may make changes to the record. The display routine recognizes this and locks the record before executing the action defined in the Action field.

*Action	action	The display action, which is a direct link to the action in the states record. Predefined values include the following: • Do Nothing: With this action, the display application will execute any statements and RAD applications defined in the display option and then return to the original screen. • Redraw: Refreshes the screen. The cursor remains in the same field, but if this is a multi-line field, the focus resets to the top of the field. • Close: Exits the current screen to the calling application. • More: Selecting this action has the same effect as selecting Do Nothing. • NULL: If the Action field is blank, Service Manager assigns a value of NULL to the Action field. • UNKNOWN: If no option has a true condition, Service Manager assigns a value of UNKNOWN to the Action field. This only occurs if the condition for the option has changed to false by user input. • return: • ok: Save and exit. • cancel: Exit without save. • skip: Skip execution of a script. • views: Display a list of alternate viewing forms. • extend: • find: Find the related record detail. • fill: Fill in information from a related record. • validatefield: • useroptions: Execute user options defined in Format Control. • closeapplication:
*Unique ID	id	The unique ID of the option generated by the system and made up of the screen ID plus action. If multiple records for the same action are required, the GUI option is also appended.

*GUI option	gui.option	The following guidelines should be adhered to when entering a value into the GUI option. • Tool tray: Enter numbers less than 200. • Detail options: Enter numbers greater than 200. • List options: Enter numbers greater than 200. • **Note**: Option numbers between 4001 and 4099 and over 32,000 are reserved for system use. To look up the ID numbers for the GUI options for buttons on detail forms, go directly to the form, using Forms Designer, and look at the property Button ID on the buttons inside the form. To find out the GUI options for tool tray or menu options, find the default label or text alternative that matches the item.
Balloon Help (If Option greater than 200)	balloon	The text string that appears when the cursor is held over a button to offer more information about the button's function.
Text Option	txt.option	The number of this option in the Text mode. The Text Option was originally used for text-based clients. This field is no longer needed.
*Default Label	default	The default label for the option, which is required on GUI mode. You could use this as a way to identify display options, such as Create Template from Record.
Bank	txt.bank	Determines the set of function keys with which this option is grouped. This field is required only when 11 or more options appear on a form. Option 12 is reserved for the More button. This field is no longer needed.
Text Alternative	txt.alternative	If not NULL, determines the label used for the F-key in a Text form. This field is no longer needed.

*Condition	condition	This field has to either contain `true` or `false`, or an expression that when evaluated returns `true` or `false`. It defines the condition for the option. If the field is `true` or the expression evaluates to `true`, the option appears to the user. **Note** Note • The out-of-box records are read-only for this field. • It is possible to have two display option records with the same display screen and the same GUI option, but they have to have mutually exclusive conditions.
User Condition	user.condition	Same rules as the Condition field. This field has to either contain true or false, or an expression that when evaluated returns true or false. The User Condition expression overrides the Condition expression.
RAD tab		
PreRad Expressions subtab	post.expressions	RAD expressions evaluated when this option is selected, but before the RAD application is executed.
PreJavascript subtab	javascript.pre	JavaScript expressions evaluated when this option is selected, but before the RAD application is executed.
RAD subtab		
Rad Application	application	The RAD application to call when an operator selects this option. **Note:** The RAD application, us.launch.external, doesn't support launching a URL that contains a semi-colon.
Separate Thread?	new.thread	This field indicates whether or not the RAD application is executed in a new application thread.
Names	names	The parameters to pass to the RAD application. Optional, unless required by the RAD routine being executed.
Values	values	The value of each parameter to pass to the RAD application. Optional, unless required by the RAD routine being executed.

Post Rad Expressions subtab	post.rad.expressions	RAD expressions to be evaluated when this option is selected, but after the RAD application is executed.
Post Javascript subtab	javascript.post	JavaScript expressions evaluated when this option is selected, but after the RAD application is executed.
Comments tab	comments	Additional comments on this displayoption.

Related topics

Display application

1.3.2.3. Restricting access to display options

As a system administrator, you may wish to restrict users' access to certain display options based on their system rights. This is accomplished by creating an expression in a displayoption record that determines who may use the option. This expression is constructed with capability words found in a user's operator record. You may use existing capability words or create new ones that more narrowly define the restrictions.

To restrict access to a display option, perform the following steps:

- Identify the option and the group to whom you want to deny access.
- Define a capability word unique to the option.
- Add your capability word to the operator records of all users to whom you wish to grant access to the option.
- Create a condition expression in the displayoption record using your new capability word.

Related topics

Display application
Displayoptions
Selecting display options

1.3.2.4. Selecting display options

Select an option that performs an important function in your system.

- Is this option part of a broader functionality?
- Should all users with rights to this functionality have access to this option?

For example, should all users with access to Incident Records in Incident Management be able to close a record?

By using a **condition** expression in a **displayoption** record, a system administrator can limit a user's right to close record without restricting rights to the other features of Incident Management or incident record options.

Note: Users with SysAdmin capability will be able to view, update and save existing incident records, but will not be allowed to close records.

Related topics

Display application
Displayoptions
Example: Adding a URL link to a menu
Adding and changing the image icon for menu and toolbar items
Restricting access to display options
Access display records
Define display conditions
Access menu records
Create a displayscreen record
Create a displayoption record
Add or change the image icon for options menu and toolbar items
Define display conditions
Add a capability word
Displayevent database dictionary keys
Displayoption database dictionary keys
Common option icon names

1.3.2.5. Calling an application

Display panels allow the system administrator to set up a series of actions that otherwise would have to be hard-coded into each individual application. Among the added functionality is the ability to call separate applications from within the current application.

An example of this is calling the `build.list` application from a combo box on the `problem.initial` form.

The call to `build.list` recreates a list of viable subcategory, location, or other options, based on the information already appearing on the problem.initial form. When calling an application, several options are available, including threaded windows and scripting.

Related topics

Display application
Displayoptions
Selecting display options
Example: Adding a URL link to a menu
Adding and changing the image icon for menu and toolbar items
Restricting access to display options
Access display records
Define display conditions
Access menu records
Create a displayscreen record
Create a displayoption record
Add or change the image icon for options menu and toolbar items
Define display conditions
Add a capability word
Displayevent database dictionary keys
Displayoption database dictionary keys
Common option icon names

1.3.2.6. Custom RAD

Determine whether your display screen record should apply to a single form or be available to all the forms within your application. If applying to a single form, consider using Format Control. If you want to apply the display options to a single form, enter the name of that form in the **Format** field in the **displayscreen** record. If you want the same display options to be available on multiple forms, you must bind the Format field to the appropriate local variable. This is the variable you have defined in the **Array of local variables** field of the display panel in your RAD application.

When you are creating your displayoptions, the values you enter in the Action field of the **displayoption** record must be defined on a decision panel in your RAD application as **$L.action="<action type>"**

Related topics

Menus
Display application
Displayoptions
Selecting display options

1.3.2.7. Creating displayscreen records

Your system contains a number of **displayscreen** records that have been predefined in the RAD code. Some of these records bind their options to a specific form, while others bind the options to local variables for use with multiple forms. If you edit an option in a displayscreen record used by more than one form, every form that uses that Screen ID is affected. If you need option combinations that don't exist in the displayscreen records provided with the standard system and don't wish to edit any of the existing Screen IDs, you must create your own unique displayscreen record. You might choose to do this when:

- Writing custom RAD applications
- Creating script forms

Displayscreen records are created and hard-coded into RAD applications, which search for the appropriate **Screen ID** when called.

- RAD programmers who write custom applications for their enterprise can create new displayscreen records.
- Non-RAD licensee's can add display screens to support specific scripts because the display screen name is specified within the script record and is not hard coded in RAD.

Related topics

Menus
Display application
Displayoptions
Selecting display options
Create a displayscreen record

1.3.2.8. Displayoption Database Dictionary keys

These keys control how the data is placed and retrieved from the file. Use values based on these keys when issuing queries on the **displayoption** file.

Field	Key	Description
unique.id	unique	Ensures record numbers are unique
screen.id **language** **gui.option** **gui.sig**	unique	This key ensures that no two GUI options can have the same screen ID, language, or condition.
screen.id **language** **txt.bank** **txt.option** **txt.sig**	unique	This key ensures that no two text options can have the same screen ID, language, text bank, or condition.
action **screen.id** **language** **gui.option**	nulls&duplicates	Useful when querying by action.

Related topics

Display application
Displayoptions
Selecting display options

1.3.2.9. Displayevent Database Dictionary keys

Field	Key	Description
unique.id	unique	Ensures record numbers are unique.
screen.id **language** **event** **event.sig**	unique	This key ensures that no two Events records can have the same screen ID, language, event.name, object.name, or condition.
actions **screen.id** **language** **event**	nulls&duplicates	Useful when querying by action.

Related topics

Display application
Display functions
Selecting display options

1.3.2.10. Access display records

Applies to User Roles: System Administrator

The Display application reads data from the following data files:

File name	Description
displaymaster	The displaymaster file contains the general controls for **Display**. There is one record per supported language. The default is **ENG** for English.
displayscreen	A displayscreen record defines the attributes of a screen and provides the user with access to the individual records for options and events. Use the vertical scroll bar to view the entire form. All Service Manager applications using rio or fdisp panels have been converted with the **display.cv** utility before being shipped to you. The displayscreen records have been created and hard-coded into RAD applications, which search for the applicable Screen ID when called. RAD programmers who write custom applications for their enterprise will need to create new displayscreen records. Non-RAD licensee's can add display screens to support specific scripts because the display screen name is specified within the script record and is not hard coded in RAD. The existing displayscreen records are powerful tools for controlling the display attributes of certain forms within your system. Knowing which Screen ID is attached to a form is vital if you are to take full advantage of the features offered by the Display application.
displayoption	Set the various display options in the displayoption file. Display options can appear in the More Actions menu, as buttons in the toolbar, and as F–keys. Option numbers less than (<) 200 in GUI mode appear as buttons. Option numbers greater than(>) 200 appear in the More Actions menu. **Note**: Option numbers between 4001 and 4099 and over 32,000 are reserved for system use. RAD applications can be called from Options Definition records. Access the Options Definition record from any displayscreen record: • Click on the Ellipses button for a record list of all options. • Click on a numbered option button for individual displayoption records.
displayevent	The displayevent file defines the events a screen will handle. Access the form by clicking an **Event** button in a displayscreen record.

To access display records:

1. Click **Tailoring** > **Database Manager**.
2. Type `display` in the Form field of the Database Manager form.
3. Click **Search**. A record list of data file records opens.
4. To configure the Display application, open the **displaymaster** file first. Or select another file from the record list, and double–click the name to open it.

Related concepts

Display application
Displayoptions
Selecting display options

1.3.2.11. Create a displayscreen record

Applies to User roles: System Administrator

The displayscreen record is the control point for applying options to Service Manager forms using the **Display** application. You must create the **displayscreen** record first, then create the individual options in the **displayoption** file.

You can create a display screen record from scratch, or base your new record on an existing record.

To create a new displayscreen record:

1. Access the **displayscreen** file.
2. Type a unique name in the **Screen ID** field of the blank **displayscreen** record form. This name should identify the file involved and describe the process to which the options are attached. For example, a displayscreen record created for a script that opens incident records might be called pm.script.open.
3. In the **Format** field, type the name of the form on which the new options should appear.

   Note

   Note Type the appropriate format variable if you want the display options to appear on more than one form.

4. Complete any other applicable fields.
5. Click **Add**. The system creates the displayscreen record and displays the message "Record Added."

To create a displayscreen record based on an existing displayscreen record:

1. Access the **displayscreen** file.
2. Open the **displayscreen** you want to use as a base. For the example, select **approval.view**.
3. Change the **Screen ID** to a unique name. For the example, change the name to **new.approval.view** and then click **Add**. The Include displayoption/displayevent prompt opens.
4. Specify whether to add any associated displayoption or displayevent records when the new displayscreen record is added by checking the applicable box.
   - If you check **Include displayoption records**, new.approval.view records will be created from the approval.view records in the displayoption file.
   - If you check **Include displayevent records**, new.approval.view records will be created from the approval.view records in the displayevent file.
5. Click **Next**. The system creates the displayscreen record and any optional files you chose and displays the message "Record Added."

Related topics

Display application
Displayoptions
Selecting display options
Access display records

1.3.2.12. Create a displayoption record

Applies to User roles: System Administrator

Data files within Display contain the individual records, or screens, in which options, events, and window controls are defined. Each screen is attached to a particular application and controls the features appearing on the forms associated with that application. For example, you can create a Validity lookup option in the More Actions menu by creating a displayoption record.

To create a displayoption record:

1. In the Service Manager command line, type `do` and then click the Excecute Command icon.
2. In the **Screen ID** field of the blank displayoption record form, type the name of the screen ID you created for the displayscreen record.
3. Enter the applicable action for your option in the **Action** field.
4. Complete the displayoption record.

Important Notes:

- If you create a `displayoption` by copying an existing record, you must change the screen ID and leave the Unique ID field blank; the system will assign a unique ID to your option.
- If you are defining a Fill option, be sure that the value in the GUI option field matches the Button ID value of the field in the form.

5. Click **Add** to add the record to the database.
6. Open the displayscreen record for your screen ID and select the Options tab to make sure your new option appears there.

> **Tip**
> Tip The maximum number of displayoptions for a displayscreen is 300.

Related topics

Display application
Displayoptions
Selecting display options
Access display records

1.3.2.13. Define display conditions

Applies to User roles: System Administrator

Access to an option is controlled by the Condition field in a displayoption record. This field may evaluate to **true** or contain an expression defining a condition for display.

In the following example, you restrict access to the **Close** button in the toolbar of Incident Management incident records to only users with **SecClose** capabilities.

To define display conditions:

1. Select **Tailoring** > **Tailoring Tools** > **Display Options**.
2. Enter the screen ID of the update form, `apm.edit.problem`, and press Enter.
3. Locate the **Close** option from the record list. The **User Condition** field is empty by default. The User Condition is processed after the Conditions field and operates at the user-specific level.
4. Add the following condition statement to the User Condition field:
   `index("SecClose", $lo.ucapex)>0`

   Where *SecClose* is the name of the capability word you created.
5. Click **Save** to record the modification to the record.
6. Click **OK** to back out and return to the home menu.
7. Test the new display option, as follows:
   1. Open the **Incident Management Queue** and view an incident record. The **Close** button should appear.
   2. Open a new client as an operator who does *not* have the capability word defined in the operator record.
   3. Go to the **Incident Management Queue** and view the same record as you did above. The Close button should not appear for the operator whose record did not contain the capability word.

Related topics

Display application
Displayoptions
Selecting display options
Access display records
Access menu records
Define display conditions
Create a displayscreen record
Create a displayoption record
Add or change the image icon for options menu and toolbar items
Define display conditions
Add a capability word
Displayevent database dictionary keys
Displayoption database dictionary keys
Common option icon names

1.3.2.14. Enable the Merge Conflicted Updates function for customized user operations

Applies to User Roles:System Administrator

The Merge Conflicted Updates feature is out-of-box for Service Manager 9.61 default user operations. However, to apply the Merge Conflicted Updates function to a record with customized operations, you need to tailor your customized operations first.

The following example describes how to enable the Merge Conflicted Updates function for all operations on Service Desk. In this example, you need to:

1. Enable manual merge operation for Service Desk.
2. Enable auto merge operation for all operations.

Task 1: Add a display option for the Merge Conflicted Updates function

1. Click **Tailoring** > **Tailoring Tools** > **Display Options**.
2. As listed in the table below, fill in the required fields to add the display option:

Field	Value
Screen ID	The display screen you are using to display Service Desk records. For example, cc.edit.incident.
Unique ID	cc.edit.incident_merge
Action	merge
GUI option	Unused number under the display screen. For example, 5.
Text option	Unused number under the display screen. For example, 5.
Bank	1
Balloon Help	Merge
Default Label	Merge
Text Alternative	Merge
Condition	updatestatus($L.file)=$G.RC.MODIFIED and gui() and $L.mode~#"add"

3. Click **Add**.

Task 2: Add process call in the State record of a Service Desk Object.

1. Click **Tailoring** > **Document Engine** > **States**.
2. Search for the state that you use to view a Service Desk record. For example, sm.view.
3. As listed in the table below, add the values to the last line of the Non-base methods table:

Column	Value
Display Action	merge
Process Name	record.update.conflicts
Condition	true

4. Click **Save**.

By completing the first two tasks, you have enabled the manual merge function for Service Desk. If the record being updated has been modified since being read, the **Merge** button appears on the option toolbar.

Task 3: Enable auto merge operation for all operations.

1. Click **Tailoring** > **Tailoring Tools** > **Display Screens**.
2. Search for the **sd.view** record.
3. On the **Javascript** tab, append the following code in the text box:
   ```
   var mergeResult = lib.recordUpdateConflicts.autoMerge();
   ```
4. Click **Save**.

By completing the third task, you enable the auto merge function for all operations. If there are conflicted updates of the same field, the auto merge process doesn't merge this record. Instead, you must perform a manual merge operation.



Note

Note: In some scenarios, the auto merge operation won't be properly triggered. For example, if some background SLA operation is triggered to update the record, then the auto merge operation may not work.

1.4. Advanced functions

The system permits control of user interaction with the following utilities:

- Service Manager macros — invoke to do things like sending an email to a specific address when a specified event occurs.
- Publish and subscribe — display static or dynamic labels, marquees, and charts.
- Scripting — collect data by prompting users to enter specific information into their forms.

Related topics

Tailoring
Service Manager macros
Publish and subscribe
Scripting

1.4.1. Service Manager macros

Service Manager macros are discrete units of work a System Administrator invokes to do things like sending emails to a specific address. These macros are more similar to Microsoft Access macros than to Microsoft Word macros, which simply record and play back keystrokes.

Macros are distinct actions, driven by predefined conditions, that execute when saving a record in the database. Macro actions are associated with files and reflect certain states in the records of those files. If a macro's condition evaluates to true when saving a record, the macro's action executes. A typical condition is `priority.code in $L.new= "1"`, causing that macro's action to execute when a saved incident record has a priority code of 1.

As a Service Manager System Administrator, you can create macros to run processes automatically when specified events occur. For example, you can create a macro to send an email to a manager when an incident record hits a deadline alert.

Related topics

Macro conditions
Create a macro
Definitions for macro forms
Macros provided with Service Manager

1.4.1.1. Macro conditions

Macro conditions are expressions written in Service Manager Rapid Application Development (RAD) syntax that evaluate at run time. If the expression evaluates to true, the macro executes. If the expression doesn't evaluate to true, the macro can't proceed.

A macro's condition can be very simple. For example, true or tod()< = '17:00:00'>. Macro conditions often include a check against the record being saved. For example, a macro condition can be expressed for an action or group of actions when a specific set of incident records is saved.

The record currently being saved is identified to macro conditions as $L.new. This variable can be used as $file in Format Control expressions. Macro expressions also have $L.old available. This represents the state of the record before it was altered. All the following expressions are valid:

- priority.code in $L.new="1"

Executes whenever a priority 1 record is saved.

- assignment in $L.new~=assignment in $L.old

Executes whenever the assignment group changes.

- tod() <= '17:00:00'

Executes whenever a record is saved before 5:00 PM.

The variable $L.message can create evaluating expressions that gather certain information about incident records. This data is then sent as a message to specific users or groups defined in the Macro Parameters form. The $L.message is expressed as an array, using the following syntax:

```
$L.message={"Incident#" +number in $L.new, brief.description in $L.new}
```

The result is an array of the Incident ID and Incident Title in the record being saved ($L.new). The message might look like this:

```
Incident # IM1012
Phone is going dead intermittently.
```

Related topics

Service Manager macros
Create a macro
Definitions for macro forms
Macros provided with Service Manager

1.4.1.2. Accessing macro records

You can access macros from either of the following locations:

- Click **Tailoring** > **Tailoring Tools** > **Macros**.
- Incident Management Security Files.

Warning: Don't edit macros through the Database Manager. Certain processing doesn't occur and your edits may lose.

You can select individual macro records from a list or search for groups of related macros. For example, those associated with `probsummary`.

Note: You need System Administrator privileges to run the macro editor.

Access a macro record

1. Click **Tailoring** > **Tailoring Tools** > **Macros**.
   The list of available macros opens. This is the access point for all your macro activities. You must go through this form to add, edit, or delete macro records.
   The macro list form displays the results of macro queries. You can use the option buttons on this form to view and process macros.
2. To open an individual record, highlight an item in the list and then click **Edit**.
3. Use the following procedure to display a list of related records:
   a. Click **Search**. The macro search form opens.
   b. Add optional search criteria in any field. For example:
   • ID: the number identifying the macro. The system assigns this number.
   • Filename: the Service Manager file associated with the macros.
   • Name: the unique name of the macro.
   • Type: the macro type for the macro(s) that you are searching for.
   c. Click **OK**. The system opens the macro list form, showing all the macros matching your search criteria.
   d. Select a macro record to edit.
   You can edit existing macros and create new ones. You can also select macro types and set conditions for their execution.
   The values selected in this form decide which fields show in the parameter form.
4. If you make any changes, click **OK**.

Related topics

Service Manager macros
Macro conditions

1.4.1.3. Create a macro

When creating a macro, you must name and define the conditions of the macro before setting the parameters for its execution.

1. Click **Tailoring** > **Tailoring Tools** > **Macros**.
2. Click **Add**.
3. In the **Macro Name** field, type a name for the new macro.
4. In the **Applies When** field, select an event option from the list indicating when you want the macro to execute.
   For example, select **Incidents are Saved**.

5. In the **Macro Type** field, select an action you want the macro to execute. Options include mailing, starting and stopping clocks, executing a Rapid Application Development (RAD) function, or evaluating an expression.
6. Type a **Macro Condition** that triggers the macro to execute. When this condition evaluates to true, Service Manager executes the action as defined in the Macro Type field.
   For example, you can inform a specific person when a new incident record has a priority of 1. The condition would look like this: `priority in $L.new="1"`.
7. Click **Set Parameters** to establish the parameters for this macro.
   Note: The available fields in this form vary, depending on the value in the Macro Type field in the edit form for your new macro.
8. Specify additional information where needed. For example, Construct Message By.
9. Click **Save**.
   The macro edit form opens.
10. Click **OK**.
    The macro record saves and you are returned to the macro list form. Updating a macro record uses the same edit forms as the creating process.
11. Click **Search** from the macro list form to refresh the list of macros.
12. Click **OK** in the search dialog box.

Related topics

Service Manager macros

1.4.1.4. Definitions for macro forms

Macros are distinct actions, driven by predefined conditions that execute when a record is saved in the database. Definitions for macro forms are as follows:

- Macro list form
  - Column headers
  - Option buttons
- Macro editor
  - Fields

Macro list form definition[edit]

Column Label	Database field name	Description
Id	id	A unique number assigned to the macro to identify it.
Filename	filename	The Service Manager file that the macro is attached to. For example, *problem* or *device*.
Name	name	The name you give the macro.
Type	type	The type of action the macro takes when activated.

Button label	Description
Add	Opens a blank macro editor form for adding a new macro.
Edit	Accesses the macro editor to change the selected macro record.
Delete	Deletes the selected macro record. **Warning**: When you delete a record, no warning is given and the record is simply deleted.
Search	Accesses a query form.
Clear Filter	Removes the current filter used for searching the macros and returns the list to its previous state.
Back	Returns to the previous process.

Macro editor definition[edit]

Field label	Database field name	Description
Macro name	name	System administrator creates unique name for this macro.
Applies When	filename	Predefined event option for execution of this macro. Select an event from the drop-down list. For example, when a change request is saved.
Macro Type	type	Predefined macro type for this macro.
Macro Condition	condition	Define a condition for when this macro should execute. For example, when a priority 1 incident record is opened, email the members of an assignment group.

Contents

- 1 Macro list form definition
- 2 Macro editor definition

1.4.1.5. Macros provided with Service Manager

Macro Action	Description
Call A RAD Routine *	Executes a user-specified RAD routine and passes its parameters every time it executes. **Warning**: Macros don't work properly when calling a RAD application involving user interaction. For example, fill.recurse or validate.fields. Continue to use Format Control to call these types of applications.
Evaluate Expressions	Executes a number of user-defined expressions whenever it fires.
Get a Sequential Number	Fetches the next available sequential number of a specific class and stores it in a field in a file.
Mail 1 Person	Sends e-mail to one person only. This person can be defined as an operator, a contact, or a simple “raw” email address.
Mail a Change Request to 1 Person	Sends e-mail of a change request to one person only. This person can be defined as an operator, a contact, or a simple “raw” email address.
Mail a Change Request to a Message Group	Sends e-mail of a change request to all the members of a Change Management Message Group.
Mail a Change Request to Many People	Sends an e-mail of a change request to an arbitrarily defined list of people. These people can be defined as operators, contacts, or “raw” phone numbers.
Mail a Change Task to 1 Person	Sends e-mail of a change task to one person only. This person can be defined as an operator, a contact, or a simple “raw” email address.
Mail a Change Task to Many People	Sends an e-mail of a change task to an arbitrarily defined list of people. These people can be defined as operators, contacts, or “raw” phone numbers.
Mail 1 Assignment Group	Sends e-mail to an entire assignment group.
Mail Many People	Sends e-mail to an arbitrarily defined list of people. These people can be defined as operators, contacts, or “raw” email addresses.
Mail an Incident Ticket to 1 Person	Mails an incident record to one person. This person can be defined as an operator, a contact, or a simple “raw” email address.
Mail an Incident Ticket to an Assignment Group	Mails an incident record to all the members of an assignment group.
Mail an Incident Ticket to Many People	Mails an incident record to an arbitrarily defined list of people. These people can be defined as operators, contacts, or “raw” email addresses.
Mail an Incident Ticket to a Change Management Message Group	Mails an incident record to all the members of a Change Management Message Group.
Mail a Task to a Message Group	Mails a change task to all the members of a Change Management Message Group.
SC Mail 1 Person	Sends Service Manager mail to one person only. This person can be defined as an operator, a contact, or a simple “raw” phone number

SC Mail an Incident Ticket to Many People	Sends an incident record to an arbitrarily defined list of people with Service Manager mail. These people can be defined as operators, contacts, or “raw” email addresses.
SC Mail an Incident Ticket to a Change Management Message Group	Sends an incident record to all the members of a Change Management Message Group with Service Manager mail.
SC Mail Many People	Sends Service Manager mail to an arbitrarily defined list of people. These people can be defined as operators, contacts, or “raw” phone numbers.
Start a Clock	Starts a specified clock.
Stop a Clock	Stops a specified clock

1.4.2. Publish and subscribe

You can present virtually any data generated by Service Manager to users. End users, from first level support personnel to administrators, need current, updated information about changing conditions across the system. Publish and Subscribe allows a system administrator to select pertinent data and present it to user groups in a visual format.

Publish and Subscribe saves system resources by eliminating the need for individual users to query the system every time they want an update. Desired system data is gathered by automatic background processes at prescribed intervals and published to subscriber groups.

Display objects subscribing to Service Manager data are automatically updated with current information in intervals defined by the System Administrator. Rapidly changing features and slowly developing trends can be expressed with the same process.

Forms created for specific user groups (for example, Change Management personnel or Incident Management administrators) provide the end user with dynamic system data that requires no user interaction to extract data. The System Administrator chooses the data to be published and defines display attributes, such as chart values, marquee messages, and color.

Related topics

Publish and subscribe workflow

1.4.2.1. Publish and subscribe workflow

- Information is published in Service Manager when system data is associated with selected variables and stored in SYSPUB (Service Manager’s publishing house). The System Administrator designates what information stored here is available to subscribers.
- Forms designed by the System Administrator for user groups that subscribe to the data by using published variables to populate display objects, such as charts and marquees.
- Charts display data in bar graph formats. For example, record activity for different categories, assignment groups, time periods, and priorities. Charts can be added to a form and updated automatically at scheduled intervals; colors can be set to reflect priorities; and button bars can be added to provide drill-down capabilities for viewing underlying data, such as individual records within categories.
- Marquees display messages in horizontal, scrolling banners on a user’s screen. This is an effective method of keeping users informed of the status of the system. Marquees are added to a form in Forms Designer and set by variables to subscribe to a particular message. They may be updated automatically at scheduled intervals.

Related topics

Publish and subscribe

1.4.2.2. Static messages

Simple static messages and marquees can be published to any form in Service Manager. For example, you may want to attach a static message to a menu advising a certain user group of the average time for resolution of an open record. The text of a static message doesn't change until it is republished in Service Manager. This is the simplest method of publishing and subscribing.

Related topics

Publish and subscribe
Modify the form for static messages
Publish the static message
Modify the form for a simple marquee
Publish the marquee

1.4.2.2.1. Publish the static message

Follow these steps:

1. Click **System Administration** > **Base System Configuration** > **Publishing Utilities** > **Publish Messages**.
2. Type the name of the variable from the Input field of the Forms properties Box ($SYSPUB.xxx) and the text of your message in the field provided. In this case, type: The average time for the resolution of an incident record is 8 hours.
3. Click **OK** to publish your message to the Service Manager publishing house.
   The following message appears in the status bar:

   ```
   Your message has been published
   ```

Important: You can't delete SYSPUB messages. You must edit the label property on the form you modified in order to remove the message.

Related topics

Static messages
Modify the form for static messages

1.4.2.2.2. Modify the form for static messages

This process publishes a non-updating, text message to a user's form. The message remains unchanged until you edit it manually.

1. Click **Tailoring** > **Forms Designer**.
2. Type the name of the form (menu.gui.pm) in the Form field of the Forms Designer dialogue box, and then click **Search**.
   The Incident Management menu opens.
3. Click **Design** to open the Properties Box and the Tool Palette.
4. Use the Label tool to add a new label to the form. Ensure that the length of the label field is long enough to accommodate your message.
5. In the Properties Box, erase Caption. Ensure this field is blank.
6. Set Input to $SYSPUB.xxx, where xxx is your initials. The Label field is an empty box outlined with a dotted line in the Design mode.
7. Click **OK** to save your changes. The label field is invisible in this mode.

Related topics

Static messages

1.4.2.2.3. Modify the form for a simple marquee

This process publishes a non-updating marquee to a user's form. The marquee message remains unchanged until you edit it manually.

1. Click **Tailoring** > **Forms Designer**.
2. Type the name of the form (menu.gui.pm) in the Form field of the Forms Designer dialogue box.
3. Click **Design** to open the Properties Box and the Tool Palette.
4. Add a marquee to the form with the marquee tool.
5. In the Properties Box, erase Caption. Ensure that this field is blank.
6. Set Input to $MARQUEE.xxx, where xxx is your initials. The Label field appears as an empty box outlined with a dotted line in the Design mode.
7. Click **OK** to save your changes.

Related topics

Static messages

1.4.2.2.4. Publish the marquee

Follow these steps:

1. Click **System Administration** > **Base System Configuration** > **Publishing Utilities** > **Modify Marquees**.
2. Type the variable from the Input field of the Forms properties Box ($MARQUEE.xxx) in the Name field.
3. Select a display color from the drop-down list in the Color field.
4. Type in the text of your message in the Text field. For example: The average time for the resolution of an incident record is 8 hours.
5. Click **Add** to add your marquee to the Marquee Table. The marquee is published by the marquee background agent.
   Service Manager automatically adds the date and time of day to the Update Time field.
   The following message is displayed in the status bar: Record added to the marquee file.

6. Return to the system administrator's home menu.
7. Click **Incident Management** to check the marquee you created.

Important: You may have to wait a minute until the background agent publishes the data.

Related topics

Static messages
Modify the form for static messages
Modify the form for a simple marquee

1.4.2.3. Stored queries for management view publishing

Stored queries define the search parameters for data acquisition. The agents responsible for updating messages must query the database and report back with the current information. To publish a message that monitors alert status or counts records of a certain priority, queries must be created and stored.

The easiest method to generate a stored query is to select the information you want using normal search methods, and then open the Advanced Search window and store your query directly. By following these steps you can ensure that the query is optimized for best performance.

Select the queries you want to use for your published message from the stored query list.

Related topics

Agent records for publishing

1.4.2.4. Agent records for publishing

Agents are special background processes used to perform certain tasks for the user. In the case of Publish and Subscribe, they count information in your database, such as the number of open incident records in a specified alert status, or the number of records assigned to a certain group. Agent Records are the resources used by these background processes to reference stored queries, define messages and colors, and select display options. These records are published to the Service Manager publishing house and used by agents to update a subscriber's charts and marquees.

Related topics

Stored queries for management view publishing
Create an agent record for publishing

1.4.2.4.1. Create an agent record for publishing

Follow these steps:

1. Click **System Administration** > **Base System Configuration** > **Publishing Utilities** > **Define Agents**.
2. Click **Search** to open the agent records list in your database.
3. Select **pm.category** from the record list. This agent publishes records by category.
4. Select **pm.priority** to open the agent record for publishing priority status reports.
5. Refer to the field help if you have any questions about the main tab fields.
6. Click **Save** to save your changes.

Related topics

Agent records for publishing

1.4.2.5. Scheduling background processes for publishing

A schedule record is required for every agent that runs in Service Manager. In order for a subscriber to receive dynamic data, an agent background processor must be running to process agent records. Publish and Subscribe requires two background agents, marquee and agent, to be running in order to display data in marquees or charts.

These background agents are set to read records at specified intervals.

There are two methods for running these agents:

- Register them at startup.
- Start and stop them during a session from the system.status.list form.

Related topics

Load agents into the startup record for my system
Start background processes
Stop background processes

1.4.2.5.1. Establish the publishing interval in the schedule file

1. Click **System Administration** > **Base System Configuration** > **Publishing Utilities** > **Modify Schedule**.
2. From the record list, select a record for the type of display object you want to create. For example, select Agent-Priority, which retrieves priority status data.
   Tip: The entry in the Query field in the Description tab must relate to the agent record. For example, the Agent-Priority record's query entry is pm.pri.1.
3. Reset the counter in the Repeat Interval structure to the desired update interval.
4. Click **Save**.

1.4.2.5.2. Load agents into the startup record for my system

Follow these steps:

1. Click **System Administration** > **Ongoing Maintenance** > **System** > **Startup Information**.
2. Type `startup` in the Type field, and then click **Search**.
3. Using the scroll bar in the Processor Information structure and looking at the **Name** field for each process, make sure the **agent** and **marquee** names are listed here.
   Note: To run at startup, agent and marquee must be listed.
4. If necessary, you can edit the **Wakeup Interval** in which the agent background process reads the records.

Note: The startup registry functions separately from the registry for individual agents. To change the settings for an agent, be sure to change both records.

Important: Background processes must be started from the server.

Related topics

Scheduling background processes for publishing

1.4.2.5.3. Start background processes

Agent background processes can be started from the System Status form (system.status.list.g).

1. Click the **System Status** button in the system administrator's home menu.
   The System Status form opens, listing all current background processing agents.
2. Click **Start Scheduler** to open a list of background processes to start.
   A list of all the available processes opens.
3. Double-click on the agent you want to start.
   Note: Click **startup** to start all agent background processes.

1.4.2.5.4. Stop background processes

Agent background processes can be stopped from the System Status form (system.status.list.g).

1. Click the **System Status** button in the System Administrator's home menu.
   The System Status form opens, listing all current background processing agents.
2. Type `S` in the **Command** field beside the process.
3. Click **Execute Commands**. The Background Scheduler Status form opens.
4. Click **Stop schd**. The Schedule Stop window opens.
5. In the **Schedule Stop Time** field, type the date and time you want the background process to stop.
6. Select one of the following options:
   - Click **Schedule** to stop the process.
   - Click **Cancel** to return to the system status form without stopping the process.
   - Click **Menu** to return to the System Administrator's home menu without stopping the process.

1.4.2.5.5. Define which system processes manage message traffic

Applies to User roles: System Administrator

To define which system processes manage message traffic:

1. Click **System Administration** > **Base System Configuration** > **Miscellaneous** > **System Information Record**.
2. Click the **Message Processors** tab.
3. In the **Processor Name** field, type the names of the background or scheduler processes that are to manage message traffic. The processor name must match the class name specified for the message class processor.
4. Click **Save**.
5. Start the message processors.

Service Manager rotates messages among all the processes you define. If you don't define any message processors, then Service Manager uses the problem processor to manage all message traffic.

1.4.2.6. Marquee processor

If you have selected the **Add marquee to table** function in the agent record, your marquees aren't published immediately, but are added to the marquee table. To publish them from the table, you must schedule a publishing interval with the marquee processor.

Establish the publishing interval in the marquee processor[edit]

1. Click **System Administration** > **Base System Configuration** > **Publishing Utilities** > **Modify Schedule**.
2. Select **Marquee processor** from the record list.
3. Schedule the time interval for publishing your marquee in the Repeat Interval structure.
4. Click **Save** to save the record. The following message is displayed in the status bar: Record updated in the schedule file.

Contents

- 1 Establish the publishing interval in the marquee processor

1.4.2.7. Schedule file

Once you have created your Agent Record, you must schedule it to be run by the background agent. The Schedule File enables you to establish the interval in which your background processing agents refresh your charts and marquees.

1.4.2.8. Subscribing

Subscribing to Management View data published in Service Manager requires the use of display objects whose fields are identified by selected variables. These variables, stored in Service Manager's publishing house (SYSPUB), return the data published by the Agent Record.

View published data

There are several ways in which to view published data:

- **Startup screens for user groups**: Startup screens are created with each user's system rights in mind. Custom menus with automatically-updating display objects provide end users with applicable system tools.
- **Display objects added to another form**: Existing forms and menus can be modified to display Basic or Management View data.
- **Button access to a custom form**: Add a button in any Service Manager form to access a custom form containing automatically-updating display objects.

Related topics

Publish and subscribe

1.4.2.9. Publishing in the RAD environment

The following is a list of Rapid Application Development (RAD) routines that you can call from Format Control to display Publish and Subscribe messages:

- publish
- marquee.publish
- marquee.send

Related topics

Service Manager Programming

1.4.3. Scripting

Rapid Application Development (RAD) code controls the screen flow within Service Manager applications. Scripting enables you to interrupt the normal screen flow to display a series of forms or execute decision-tree processing without modifying the original RAD code. Scripting doesn't affect the RAD screen flow.

When scripting is useful

Scripting is useful for any process that requires an operator to supply prerequisite information. For example, during the incident determination cycle you can create a script flow for operator-entered data. Based on how a caller replies to questions regarding the incident, your script determines which screen the operator sees next. While the script is executing, the operator-entered data is accumulated in a file variable which is returned to the calling application when the script is complete.

When a script displays a form

During execution when a script displays a form, it has the look and feel of a customized RAD application and can be used in place of most RAD routines designed to gather data from a caller. Each script can display a standard form and execute its Format Control record. The instructions in the Format Control record where there is a condition on "display" that evaluates to true will be executed before the form opens, and the instructions where there is a condition on "add" that evaluates to true will be executed after the user clicks OK. If a Format Control definition fails (an error condition is detected), the user returns to the last script form.

When multiple complex decisions must be made

Scripting is also beneficial when multiple complex decisions must be made in order to reach a conclusion. For example, Change Management approval requirement conditions are normally based on the data contents of one field in the change record. Such a condition might be expressed as header,risk.assessment in $cm3r>4. However, there may be circumstances where the approval requirement condition is based on the values of several different fields. For instance, there may be three fields that affect the approval requirements: division, area, and department. Hard coding all the possible combinations of these field values into condition statements in RAD involves a great amount of work and would be nearly impossible to maintain. You can define these conditions in scripting records which don't display forms but that allow you to call a RAD subroutine or execute standard Service Manager processing statements

(similar to Format Control calculation statements). These options allow for the manipulation of record data. At run time, the script is a decision-tree which results in significant processing reductions over the original method of RAD coding.

Related topics

Script flow
Scripting processing flow
Diagramming the script flow
Using fill boxes in script forms
Executing scripts
Script reports

1.4.3.1. Scripts

A script is a series of instructions run from an *option*. These may include a progression of screens, requesting information or user-specific responses, or instructions to perform certain computations with existing record data. Since the script call is set up and performed at the **displayoptions** application level, it doesn't need to be hard-coded into the **RAD** of the application. This allows for easier maintenance and modification.

Script display calls often employ the **script.execute** application, specifying the progression of forms to appear for the user. An example of this form of display call is seen in **Service Manager Request Management**, where the **New** button option forces the user through a script at open time. The script calls a preliminary information screen which requests certain data that's later populated into a new Request Management record. The screen displayed serves as an organized and user-specific entrance point into the application.

Related topics

Stored queries
Using stored queries in display objects
Menu option searches

1.4.3.2. Script forms

You can create unique **displayoptions** for your script forms which are unrelated to those appearing in the application that executes the script. Changes to the **RAD** code are unnecessary when creating displayscreen records for this purpose; the appropriate variables have been hard coded into the script.execute application and need only be applied correctly.

Format variable

The format variable for script.execute is **$L.script.format** and is hard coded into the RAD layer of the application. Enter this variable into the Format field of any displayscreen record to bind its options to multiple script forms. This local variable is bound to the display form you entered in the Format field on the script definition record.

To apply unique options to a single form in a script, name the form in the **Format** field of the displayscreen record rather than use the local variable.

Display actions

Actions for displayoptions in scripts are defined on a decision panel in the RAD code of the script.execute application. Unless you possess a RAD license and are capable of programming in RAD, you are limited to the following, hard coded actions:

- do nothing
- back
- close
- more
- redraw

Related topics

Display application
Creating displayscreen records
Displayoptions

1.4.3.3. Script flow

Script flow defines the order in which script panels execute. Scripts can move in a straight line from start to finish, or branch into several possible processing flows.

Condition statements that must evaluate to true before certain scripts can execute, control more complex script flows. If none of the specified conditions evaluates to true, the script flow is considered complete, and control passes back to the calling application.

Related topics

Scripting
Scripting processing flow
Diagramming the script flow

1.4.3.4. Scripting processing flow

Each script can execute a Service Manager form, display options, a RAD subroutine, and condition statements. Note that each script can use all of these options. The sequence of this processing is:

1. Execute any display Format Control processing associated with the script's form.
2. Display the script form.
3. Execute any add Format Control processing associated with the script's form.
4. Execute statements defined in the script.
5. Execute the RAD application defined in the script.

Related topics

Scripting
Script flow
Diagramming the script flow

1.4.3.5. Diagramming the script flow

Develop a map of the entire flow showing the name of the form (if any) displayed by each panel and the conditions controlling the flow from panel to panel. Follow the map throughout the scripting process to avoid simple errors that could prevent your script from executing properly.

The first panel of this script provides an incident record number for your incident. The second panel displays a form requesting category and asset information. Depending upon the category selected, the third panel displays a category-specific form before exiting back into the common flow. Successive forms gather the remainder of the information needed to open the record: assignments, details, service agreements, and contacts. When the script exits, the new record containing all the data you accumulate opens and is added to the database.

Notice that the two conditional script panels for the hw.desktop and hw.mainframe categories display the same form. You can write a single condition statement to include all the categories using that form.

Related topics

Scripting
Script flow

1.4.3.6. Using fill boxes in script forms

For a Fill box to function properly on a script form, you must create the following additional records:

- Link record: Select Link from the Options menu in Forms Designer and establish a link between the field in your script form and the field in the source file.
- Format Control record: Select Format Control from the Options menu in Forms Designer and add a Format Control record granting Fill privileges.
- Displayscreen record: The displayscreen controls the options that are displayed with a form. Create a displayscreen record in which you define displayoptions for your script forms, including a Fill button. Enter the Screen ID of your new record in the Display Screen field of each script definition that displays a form in which a Fill box appears. You can create unique displayscreen records for each form in the script flow or associate a single record with all the forms.

Note: You can use a combination of displayscreen records and Format Control to activate Fill buttons in your script.

Displayscreen record options

If you create a displayscreen record and want to bind its options to more than one script form, you must enter the local variable $L.script.format in the Format field of the displayscreen record. This variable is hard coded into the script.execute application.

Defining displayoptions for script forms

When defining displayoptions for your script forms, refer to the following list of possible actions that are hard coded into the script.execute application. Your choice of options in the Action field is limited to the values in this list:

- ok
- back
- cancel
- skip

- views
- extend
- find
- fill
- validatefield
- redraw
- re draw
- closeapplication

Note: Use the ok action when defining a Continue option.

Related topics

Scripting

1.4.3.7. Executing scripts

After you have created your forms and the script definitions to display them, you must decide how you want to execute the script. You have several choices:

- Incident Management (using profile records)
- Display option (from within an application)
- Format Control
- Stored query

Access the scripts file

You can access the scripts file using one of the following methods:

- Menu button
- Database Manager

Executing a script from Incident Management

Profile records within Incident Management allow you to specify the script to execute when a user opens a new incident record. You can design a script that displays forms containing only those fields necessary for recording the particulars of an incident. For example, if the technician taking the call selects hw.network as the incident category, a condition statement in the script exits to a panel that displays a network hardware related form. When the script finishes, the record carrying all the necessary information is created and the technician is ready to work on another issue.

Executing a script from displayoption

Use a displayoption to create a button, the Options menu and the Context menu selections on a Service Manager form from which to execute your script. For information on the Display application, see the related topics.

Executing a script from Format Control

Scripting can be called from the routines process of Format Control. It is important to place the script call as the last item on the routine call list when executing a script from Format Control.

Executing a script from a stored query

A script executed from a stored query searches for records by allowing the user to select precise search parameters. The Service Manager standard system contains an inactive script that can execute from a stored query to display a list of key words relating to previous incidents. The stored query then uses the key word selected to search for similar incidents.

This stored query runs from an open incident record using either Format Control or a displayoption (Screen ID: `apm.edit.problem`). In this example, a displayoption has been created that adds an option called Probable Cause to the More Actions menu of an open incident record. This option runs the stored query called probcause.user.

The stored query, probcause.user, executes the script called probcause.user.1.

The script probcause.user.1 has a single panel that displays the probcause.user.1 form. The condition for exit (null($key.words)) requires the user to choose a key word before allowing the script to exit.

Important: Scripts executed from stored queries typically set values into the same fields used by the query to retrieve records. In this example, the common field is key.words.

The script is executed before the form from the query (probable.cause.user) opens. The form opened by the script is called probcause.user.1 and supplies the query form with a key word for searching.

When the user selects a key word and clicks **OK**, the script exits. The stored query then uses the key word to search the probcause file for matching entries. If more than one match is found, the system displays a record list of possible causes for your incident.

Click a selection to view the details with the probable.cause.user form. When you have finished viewing all the choices, click **Cancel** to return to the incident record.

1.4.3.8. Script reports

Script reports display the entire flow with different views. You can view the entire flow or just a part of it. You can display and print the details of each panel for comparison or troubleshooting. There are three script reports you can print:

- Script Flow — generates a summary report of all possible script flow paths based on the assumption that the currently displayed script panel is the starting point of the flow. This report enables you to isolate each separate branch within the script.
- Script Detail — lists all the fields in the script definition record and the values you have entered for the currently displayed script panel.
- Script Tree — generates a summary report showing the logical flow of the script panels in an outline form. The currently displayed script panel is used as the starting point. This report enables you to see the relationships of panels to one another at a glance.

Each report has the same header and prints in the default printer font.

Related topics

Scripting
Print a report on a script

1.4.3.9. Access a script record from a menu

Follow these steps:

1. Click **Tailoring** > **Scripts**.
2. Click **Search**.
   A record list of scripts opens.
3. Select the target record.

Related topics

Scripting
Executing scripts

1.4.3.10. Access a script record from the Database Manager

Accessing scripting from the Database Manager gives you additional options for manipulating the data. For example, you can perform a mass update to change the cluster name for each panel in your script flow or you can unload selected records to an external file.

1. Click **Tailoring** > **Database Manager**.
2. Type **scripts** in the Form field, and then click **Search**.
3. Click **Search** again.
   A record list of existing script records opens.
4. Select the target record.
5. Make any necessary changes, and then click **Save** to save your changes.

Related topics

Scripting
Executing scripts

1.4.3.11. Create a script

Rapid Application Development (RAD) code controls the screen flow within Service Manager applications. Scripting enables you to interrupt the normal screen flow to display a series of forms or execute decision-tree processing without modifying the original RAD code. Scripting doesn't affect the RAD screen flow. To learn more about scripting, see Scripting.

To create a script:

1. Design and diagram the script flow showing the names of all script panels and forms. For information, see Diagramming the script flow, Script reports, and Define the scripts.
2. If your script requires special forms, create them using the Forms Designer utility. Be sure to follow the naming conventions defined in your map. Create any support links and Format Control that are necessary for your forms. See Tailoring Best Practices for aids in creating forms.
3. Create the script definition records. For information, see Using fill boxes in script forms, and Script forms.
4. Add your script to the Service Manager application, Format Control record, or stored query record that will execute it. See Executing scripts.

Note: If you include expressions in your script, take care to use good naming conventions to ensure unique names for any local variables you define. This helps you to avoid potential naming conflicts and achieve desired tailoring results.

Related topics

Scripting
Creating and editing forms
Format Control

1.4.3.12. Define the scripts

After diagramming the flow, the next step is to create each panel in the script flow. Be sure to follow the naming conventions for scripts and forms you have established in your script map.

1. Access a blank script definition record. For more information, see Executing scripts.
2. Complete the definition for the first script in your flow.
   For example, enter the following values:

Field	Value	Parameter Values/Conditions
Name	pm.number	
Start	true	
Application	getnumb.fc	
Name	record prompt text name string1	$script number string Incident Management PM
Next Script	pm.open.1	true

   Note: The value for Start is true, indicating that this is the first panel in the script flow.
   This panel associates a properly incremented incident record number with the data you are going to accumulate from the caller.
   This panel calls a RAD application only and doesn't display a form.
3. Click **New** to add your panel to the scripts database.
4. Click **Back** to return to a blank script definition form.
5. Complete the definition for the second script in your flow.
   In this example, we would enter the following values:

Field	Value	Parameter Values/Conditions
Name	pm.number	
Format	pm.select.category	
Skip Display	false	
Bypass Cond	false	
Next Script	pm.hw1 pm.hw2	category in $script="hw.desktop" or category in $script="hw.mainframe" category in $script="hw.network"

6. Click **New** to add your panel to the scripts database.
7. Create the remainder of the script definitions in your flow.

In this example, we must create specific definitions for each category (pm.hw1 and pm.hw2) as well as the remainder of the scripts in the common flow (pm.open.2 through pm.open.5). Make sure that each panel except the last panel in the flow has the name of the next panel to execute in the Next Script field.

In this example, panel pm.open.5 displays a form and calls the RAD application apm.edit.problem. This application displays any information you have gathered in an incident record and enables you to abandon the incident or add it to the database.

Related topics

Scripting

1.4.3.13. Define an initial script in an Incident Management profile record

Follow these steps:

1. Click **System Administration** > **Ongoing Maintenance** > **Profiles** > **Incident Management Profiles**.
2. Click **Search**. A list of Incident Management profile records opens.
3. Select a profile record.
4. Select the **Forms** tab.
5. Select **Initial Script** and enter the name of the first panel of your script in the adjacent field.
6. Click **Save** to save your changes.

Related topics

Scripting

1.4.3.14. Execute a script from a displayoption

1. Determine the screen ID of the form for which you want to define the option.
2. Open a blank displayoption record using one of these methods:
   - Type `do` in the command line and press **Enter**.
   - Type `displayoption` in the Form field of the Database Manager dialog box.
   - Click **Tailoring** > **Tailoring Tools** > **Display Options**.
3. In the **Screen ID** field, type the screen ID for the form.
4. Click **Search**.
   The displayoption records for that form are listed.
5. Scan the list of options to determine an available number for your new option.
   Remember: Option numbers < 200 appear as buttons in the toolbar, and option numbers > 200 appear as Options menu items.
6. Create the record from scratch or modify an existing record.
7. Type `script.execute` in the RAD Application field and enter the following values:

Name	Value
file	$L.filed
name	Name of the first script panel in the flow

8. Click **Add**.

1.4.3.15. Execute a script from Format Control

1. Click **Tailoring** > **Format Control**.
2. In the **Name** field, type the name of the Format Control record you want to edit. For example, type `apm.quick`.
3. Click **Search**.
4. Click **Subroutines**, or click **Subroutines** from the Options menu.
5. In the **Application Name** field, type `script.execute` and enter the following values:

Name	Value
file	$file
name	Name of the first script panel in the flow

6. In the **Before** field, type true or type an expression for a condition that evaluates to true to execute the script before any other processing takes place.
7. In the **Dis** field (Display field), type true or type an expression for a condition that evaluates to true to execute the script before apm.quick opens.
8. Click **Save** to save your changes.
9. Click **OK**.

1.4.3.16. Delete a script

You can delete script panels and the related forms manually or allow the system to delete all the elements of a script flow automatically.

1. Select the first script in the flow. To access scripts, see Executing scripts.
   Note: If you select any other script panel in the flow, you may be given a partial listing only. Partial lists stop at any script panel in the flow whose display is controlled by a conditional statement. This feature enables you to isolate a single conditional branch of a script flow for deletion.
2. Click **Delete**.
   A confirmation prompt opens asking if you want to delete the related forms as well.
3. Select the **Delete associated forms** checkbox if you want to automatically delete all related forms.
4. Click **Delete**.
   All the script panels in the flow and all the associated forms are listed. Scripts or forms used elsewhere in the system aren't listed.
5. Click **Delete** to delete all the items listed in the form.
   If the deletion procedure is successful, the following message displays in the status bar: *The Script/Format delete process is complete. Check for error messages*.
6. Click **Cancel** to exit the delete routine and return to the last form.

Related topics

Scripting

1.4.3.17. Print a report on a script

Follow these steps:

1. Access a script definition record. For more information, see Executing scripts.
   You can report on any script panel in the flow. If you want to view the entire flow, run the report from the first panel.
2. Click **File** >**Print** or the Print icon.
   A prompt asks you to select a report type.
3. Select one of the radio buttons.
4. Click **Print**.

Related topics

Script reports

1.5. Service Manager Web tier

The Service Manager Web tier is an optional component that allows users to connect to the Service Manager server through a Web browser. The Web tier requires a Web application server and a free communications port to install. For more installation information, see Install the Service Manager Web Tier.

When users connect to the Web tier they see the Web client, which offers several different interface views:

Web client view	Description	URL to Web client view
Power user view	This Web client view most closely matches the Windows client and is intended for Service Desk technicians, managers, and administrators. This view can provide access to all licensed modules, such as Service Desk, Incident Management, Problem Management, Configuration Management, Change Management, Request Management, etc.	http://*server_name*:*port_number*/*app_name*/index.do
Accessible view	This Web client view is a modified version of the power user view with record list mode turned off, high contrast colors, and larger fonts. This view is intended for Service Desk technicians, managers, and administrators who need accessibility features.	http://*server_name*:*port_number*/*app_name*/accessible.do
Self-Service view	This Web client view provides a simplified Service Desk interface for users to create, view, and update service requests. This view is intended for end-users as an entry point to Service Desk.	http://*server_name*:*port_number*/*app_name*/ess.do
Accessible self-service view	This Web client view is a modified version of the self-service user view with high contrast colors and larger fonts. This view is intended for end-users who need accessibility features.	http://*server_name*:*port_number*/*app_name*/accessible_ess.do

Administrators can configure the behavior of the Web tier and set Web client preferences from the web.xml file. In addition, administrators may customize the Web client interface by creating custom style sheets and images.

Note: Do not use the buttons (such as Refresh, Back, and Forward) on your browser toolbar or their keyboard shortcuts (such as **Ctrl+R**, **Ctrl+Left**, and **Ctrl+Right**) to perform Service Manager actions. Instead, use the buttons on the Service Manager interface, as described in List and detail toolbars.

1.5.1. Editing Web client Key bindings

Key bindings for common operations for the Service Manager web client are determined by standard decimal ASCII character codes. ASCII character codes are commonly available on the World Wide Web. You can change the default settings for the bindings by editing the **key_val** parameter in the shortcut.xml file on the Service Manager Web tier server.



Note

Note Certain third-party browser toolbars trap ALT+<key> for their own use. Be sure to select keys that will not conflict with those toolbar key mappings.

Example:

To change the messages key from Alt+g to Alt+x, open shortcut.xml and make the following change.

Old

```
<shortcut ctrl="false" alt="true" key_val="71" key_ref="keyMessages" desc_ref="Misc.ToggleMsg"/>
```

New

```
<shortcut ctrl="false" alt="true" key_val="88" key_ref="keyMessages" desc_ref="Misc.ToggleMsg"/>
```

Caution

Caution It's not recommended to modify key_ref and desc_ref in `<shortcut ctrl="false" alt="true" key_val="82" key_ref="keyRefresh" desc_ref="Misc.Refresh"/>` in shortcut.xml.

Note

Note The keyboard shortcut for the Fill function is F9 on the Eclipse client, whereas the keyboard shortcut for Fill is ALT+F9 on the web client. The ALT key is compulsory in the ALT+F9 shortcut on the web client to avoid conflicts with other browser shortcut keys in some cases. In addition, you can't configure the shortcuts for the functions that are designed in the Form Designer in shortcuts.xml, including Fill, Find and so on.

1.5.2. Using the compact layout

The sizes of some user interface (UI) elements in Service Manager 9.61 are bigger than those of version 9.33 or earlier. There is a possibility that the new size may cause certain display issues in your tailored forms. In this situation, you can use the compact layout of the web client. For more information, see Web parameter: compactLayout.

Related topics

Web tier parameters
Web parameter: compactLayout

1.5.3. Setting web client preferences

You can define global web client preferences from the web configuration file (web.xml). You can find the web.xml file in the sm/WEB-INF folder of your application server installation. This file contains global preferences that you can change by editing the file with any text or XML editor. Any changes that you make to this file must conform to XML syntax requirements. You must have administrative access to the server operating system in order to set global web client preferences.

The settings that you define in the web.xml file determine the default client preferences for all web clients. However, users can override some of these preferences directly from the URL. For more information about which preferences users can set from the URL, see the comments in the web configuration file.

Set Web client preferences from the server[edit]

You can define global Web client preferences from the Web configuration file (web.xml). This file contains global preferences that you can change by editing the file with any text or XML editor. Any changes you make to this file must conform to XML syntax requirements.

The settings you define in the web.xml file determine the default client preferences for all Web clients. Users can override some preferences however, directly from the URL. See the comments in the web configuration file for information about which preferences users can set from the URL.

You must have administrative access to the server operating system to use this procedure.

1. Open the `web.xml` file in a text editor.
   This file is located in the `sm/WEB-INF` folder of your application server installation.
2. Add or edit the preferences in the file.
3. Save the file and restart the Web application server.
   All Web clients that log in to the Service Manager server use the new client preferences you defined.

Optimize web client cache control[edit]

Applies to User Roles: System Administrator

You can define the web client cache control from the following control file on the Web tier server: <webtier>/WEB-INF/classes/application-context.xml. The web tier uses the incoming request URL to determine whether to apply one of the following cache filtering rules:

- cacheFilter
- neverCacheFilter

When the neverCacheFilter rule applies, the response page or resource will never be cached in the user's browser. When the cacheFilter rule applies, the user's browser caches the response page or resource for a period of time specified in the max-age value (in seconds).

The cacheControlProxy bean defines which web resources the cacheFilter and neverCacheFilter rules apply to. For example, the Web tier uses the cacheFilter rule for any web resources whose path matches /images/**, and sets the Expires' or 'Cache-Control' HTTP header in the response. The Web tier uses the neverCacheFilter rule on any web resources whose path matches /**/*.jsp, and sets the corresponding HTTP header in the response to disable browser caching for the matched resources.



Note

Note We recommend that you don't change the default settings in the cacheControlProxy bean.

As of version 9.32, the directory structure of the Web tier has been modified to reflect the current build number of the Web tier. These changes were implemented because the SM Web tier is typically updated every few months. When these updates occur, it is sometimes necessary for users to clear their browser cache. For enterprise clients that have tens of thousands of users, getting all of those users to do this requirement is onerous. With this revised implementation, users will no longer need to clear their browser cache.

To change the max-age value:



Caution

Caution The default max-age value is 1555200 seconds (6 months). To better utilize browser cache, you are recommended to set the max-age property to a value no less than 1555200.

1. Open the file <webtier>/WEB-INF/classes/application-context.xml in a text editor.
2. In the cacheFilter bean section, set the max-age property to an appropriate value:

```
<bean id="cacheFilter"
class="com.hp.ov.cwc.web.CacheControlFilter">
<property name="headers">
<map>
<entry key="Expires"><value>Sat, 6 May 2017 12:00:00 GMT</value></entry>
<entry key="Cache-Control"><value>public, max-age=15552000</value></entry>
</map>
</property>
</bean>
```

   The cacheFilter bean defines when the cached web resources will expire (based on the Expires key) or how long the user's browser will cache web resources (based on the max-age value, if any). The max-age value supersedes the Expires value. For example, if you set the max-age to 1555200 seconds (default), the user's browser caches web resources for up to 180 days.
3. Restart the web application server for your change to take effect.

Contents

- 1 Set Web client preferences from the server
- 2 Optimize web client cache control

1.5.4. Branding the web client

Applies to User roles: System Administrator

The out-of-box Service Manager web client includes a preset **`OpenText`** theme, which can be modified to express your organization's own branding.

You can change your Service Manager logo icons, colors, and font style from the integrated user interface. You can also perform additional branding implementation options to change the splash screen, or to replace the images, including all the icons.

The following steps show how to implement web client branding changes.

> **Note**
>
> Note To perform branding operations, the operator should have the **SysAdmin** capability. To enable the branding rights for an operator, see *Update the operator records to enable the branding rights* below.

1. Specify the location for the branding files
2. Branding implementation options
3. Additional branding implementation options

> **Note**
>
> Note Currently, the branding feature doesn't support JAWS or color settings for the accessible.do web client.

Update the operator records to enable the branding rights

In order to prevent unintended changes or problems in the web client interface, it is recommended that the web client branding be performed by an administrator.

To enable the branding rights for an operator, follow these steps:

1. Click **System Administration** > **Ongoing Maintenance** > **Operators**.
2. Type or select optional search criteria.
3. Click **Search**.
4. Select the operator that you want to update from the record list.
5. Add the **SysAdmin** capability word to the **Execute Capabilities** table on the **Startup** tab.
6. Click **Save**.
7. Click **OK**.

This particular operator now has the right to do the branding operations.

Specify the location for the branding files

The branding feature is enabled only when you specify a location to store your branding files. If you don't do so, when you click **Tailoring** > **Branding**, an error message is displayed, which indicates that the branding feature is disabled.

To specify the location for the branding files, follow these steps:

1. Stop the web application server.
2. Specify the folder location in Web parameter: customize-folder.
   For example:

   ```
   <context-param>
           <param-name>customize-folder</param-name>
           <param-value>C:/customize</param-value>
   </context-param>
   ```

3. On the web tier host, create the <*customize-folder*> folder (for example, C:\customize). This folder is the root folder to store your branding image files, including your custom image files to replace the original ones and your branding settings in the branding menu. In this folder, Service Manager saves your color and font settings in a branding.xml file and custom images in a subfolder named **branding-images**.

   > **Note**
   >
   > Note You must have write access to this folder.

4. Restart the web application server.

Branding implementation options

The branding menu provides the following implementation options.

Note

Note The new settings are applied when you next sign in to Service Manager or when the system restarts.

Tip

Tip If you want to restore the default settings, click **Restore Defaults**, and then click **Save**.

Change the logo icons

The following table describes the format requirements for the logo files.

Logo name	Description	Format
Favorites Icon	This icon appears in the browser's address bar, the browser tab title, and the web client URL bookmark link.	.svg, .ico, .png, .jpg, .jpeg, .gif
Header Company Logo	This icon appears on the horizontal masthead at the top left corner of the web client screen. You may replace the default `OpenText` logo with your own organization's logo.	.svg, .ico, .png, .jpg, .jpeg, .gif
Login Company Logo	This icon appears on the Service Manager login page. You may replace the default `OpenText` logo with your own organization's logo.	.svg, .ico, .png, .jpg, .jpeg, .gif
About Company Logo	This image appears on the top left corner of the About information window.	.svg, .ico, .png, .jpg, .jpeg, .gif

To change the logo icons of the web tier, follow these steps:

1. Sign in to the web client.
2. Click **Tailoring** > **Branding**.
3. Click **Browse** to find the logo file and select it.
4. Click **Save**.
   Service Manager saves your images in the <customize-folder>\branding-images subfolder.

Change the colors

Service Manager saves your changes in the <customize-folder>\branding.xml file.

You can change the following colors.

Login Page

Title	Description
Body Background/Foreground	The background/foreground color of the login page
Button Background/Foreground	The background/foreground color of the login button on the login page
Button Disabled	The color of the login button when it's disabled on the login page
Button Hover Background/Foreground	The background/foreground color of the login button on the login page when you hover your mouse over the button

Main Page

Title	Description
Header Background/Foreground	The background/foreground color of the header on the main page
Header Command Background/Foreground	The background/foreground color of the command line box on the header of the main page
Header Inactive Switch Button	The color of the inactive switch button on the header of the main page
Header Split Line Color	The color of the split line between the header logo and the product logo
Body Background	The background color of the main page
Primary Color	The primary color of the main page. The primary color applies to the background of the button, the background of the highlighted menu item, the background of the highlighted combo option, the focus border, and the dialog title.

Navigation

Title	Description
Background/Foreground	The background/foreground color of the navigation accordion on the left side of the applications menu (only applies to the expanded mode)
Menu Item Hover Background/Foreground	The background/foreground color of the navigation menu items
Menu Group Background/Foreground	The background/foreground color of the navigation menu groups
Menu Group Icon	The color of the menu group icon

Tab

Title	Description
Active Tab Background/Foreground	The background/foreground color of the active tab

To change a color in the web tier, follow these steps:

1. Sign in to the web client.
2. Click **Tailoring** > **Branding**.
3. Type a desired hex color code in the color value field. A sample color, which automatically changes according to the hex color code you type, is displayed after the value field.
4. Click **Save**.

Change the font

By default, the web client uses Metric font in Latin and German versions of Service Manager, and Arial font in other languages. In the case of Asian language versions that don't support these two fonts, the operating system's default font is used instead.

To change the font of the web client, follow these steps:

Note

Note

- The new font takes effect only for users whose browser has the font installed. For users who don't have the font installed, the default font of the user's browser is used instead.
- In the login form container, the new font only takes effect on the login button.
- Your customized font doesn't apply to printed pages.

1. Sign in to the web client.
2. Click **Tailoring** > **Branding**.
3. Type the desired font name in the **Font Family** field.
4. Click **Save** to save the changes in the **<customize-folder>\branding.xml** file

Change the font size

To change the font size of the web client, follow these steps:

Note

Note

If you have upgraded from an earlier version of Service Manager in which you performed branding, the default font size is 12px. Otherwise, the default font size is 14px.

1. Sign in to the web client.
2. Click **Tailoring** > **Branding**.
3. Select the desired font size from the **Font Size** drop-down menu.
4. Click **Save** to save the changes in the **<customize-folder>\branding.xml** file.

Additional branding implementation options

Service Manager provides additional branding implementation options to change the name on the splash screen to reflect your organization's name, and to replace the images, including all the icons.

Change the applications name

To change the splash screen, follow these steps:

Your organization's name is now displayed on the login page and in the browser tab title.

1. Stop your web application server.
2. Navigate to the following directory:
   <SM web tier.war file>\WEB-INF\classes\com\hp\ov\cwc\web
3. Open the **app_labels_<*language version*>.properties** file in a text editor, for example, open the **app_labels_en.properties** file.
4. Modify the **App.Name** and **App.Title** field values according to your organization's needs. The default value of these two fields is **Service Manager**.
5. Save the properties file.
6. Restart your web application server.
7. Launch the web client in your browser.

Tailor the web client images

You can change the appearance of images in the web client by placing a new image of the same name in the ***<customize-folder>*/images** folder. Any images you place here override the existing out-of-box images. Using this images folder enables you to preserve the existing company-provided images, and ensures that any future upgrade process doesn't overwrite your tailoring changes. You can delete your specified images folder at any time to restore the original images of the company.

Each of your custom images must be in the .png, .gif, or .jpg format, and have the same name (but not necessarily the same format) as the out-of-box one. Service Manager looks for a custom image by name and then by format in the following order: .png, .gif, and then .jpg. For example, if the out-of-box image is back.gif, and your customization images folder contains back.png and back.gif, Service Manager uses back.png.

To change the images, follow these steps:

Now you can sign in to the web client to verify the new images.

1. Stop your web application server.

2. Create an **images** subfolder in the customize-folder you specified. For example, C:/customize/images.
3. Browse to the **images** folder of your web tier to find the context path of the image you want to replace. For example, if you want to replace the refresh icon on the toolbar, the path of this icon is ***<SM web tier.war file>*\images\toolbar\trefresh.png**.
4. Add the new image under the same web tier context path in your **images** folder. For example, you add a new image as ***<customize-folder>*\images\toolbar\trefresh.png** to replace the icon image mentioned in the previous step.
5. Restart your web application server.
6. Clear your browser's cache.

Branding synchronization

When you upgrade the web tier, you need to copy the <customize-folder> folder from the old web tier to the new web tier.

In addition, if you have multiple web tier instances load-balanced, your branding changes are saved to the <customize-folder> in one instance only when you perform branding implementation. In this case, you need to find out the web tier instance that has the latest branding changes and copy the changed files in the *<customize-folder>* folder to all other instances and restart the web application server.

1.5.5. Web client forms

The Service Manager Web tier offers automatic support for existing applications and their forms. The Web tier generates dynamic HTML that approximates the exact layout of forms as you define them with Forms Designer. The default application forms are portable from the Windows client to the Web client with no required modification.

If you upgrade an existing system, you will find that Service Manager clients automatically support and display your customized forms. However, there may be cases when further form modification is necessary to correct cosmetic issues that appear when you view the form with the Web client. The following table describes common form revisions that might be required.

Area	Correction
Overlapped objects	Ensure that form objects don't overlap each other. The Windows client makes slight adjustments to correct for overlapped objects, but these design issues are exposed on a web browser.
Dynamic resizing	The Windows client dynamically resizes objects such as text areas and notebooks when you resize the window. The Web client doesn't resize most objects and doesn't support elastic properties as you resize the browser window. Therefore, ensure that you assign a default initial size.
Graphics and images	Size any graphics and images to fit conservatively in the form. The Windows client supports scaled images used as buttons. However, the Web client displays these images in their native size. The button grows to accommodate the size of the image.
Dynamic View Dependency (DVD) conditions	Hidden data with DVD conditions may be exposed if the form permits the user to make changes.

1.5.6. Configuring color indicator

To add color indicator settings, follow these steps:



Note

Note

The color indicator doesn't support the Accessible view (accessible.do) and the Self-Service Accessible view (accessible_ess.do). It only assists users who have normal vision. Therefore, the content in color indicator settings doesn't support JAWS.

1. Go to **Tailoring** > **Color Indicator Setting**.
2. Specify the following fields and then click **Add**.
   File: Select the table that contains the field on which you want to apply the color indicator.

Field Caption: Select the field on which you want to apply the color indicator. This field is optional at this step; you can specify this field later.

3. Set the following fields and options:

Field name	Description
File	Select the table that contains the field on which you want to apply the color indicator.
Field Caption	Select the field on which you want to apply the color indicator.
Apply to list	Select this option if you want to apply the color indicator setting on this field in the record list.
Apply to Combo & Comfill	Select this option if you want to apply the color indicator setting on this field in the record detail.  **Note** Note In the detail form, the color indicator only applies the background color setting to the Combo and Comfill widgets.
Apply to chart	Select this option if you want to apply the color indicator settings on this field in charts of the Reporting module.  **Note** Note • The color indicator only applies the background color settings to charts in the Reporting module. • The color indicator settings are only applied to the charts in the Reporting module. For example, color indicator settings don't affect the chart that you create by using **More** > **Chart by field count**.

4. Click the **Add** button in the form. The Color Indicator Setting Wizard opens.
5. Set the following fields and then click **OK**.

Field name	Description
Value	Specify a possible value of the field. The following color settings apply when the field contains the specified value.
Foreground Color	Specify the foreground color. You can directly type in the hex code of the color, or you can first click in the text field and then select a color from the color palette.
Background Color	Specify the background color. You can directly type in the hex code of the color, or you can first click in the text field and then select a color from the color palette.

6. Review the **Preview** column for the color indicator setting of each value. Click **Edit** to edit the color indicator setting for the value. Click **Delete** to delete the color indicator setting for the value.
7. Repeat Step 4 to Step 6 if you need to add color indicator settings for more values.

8. Click **OK**.

The new setting takes effect in the next login.

1.5.7. Configure the behavior when you right-click in the web client

By default, when you right-click lists in the details pane of a record in the Service Manager web client, the browser's contextual menu is displayed.

However, you can use the *enableMoreContextMenu* parameter to enable a Service Manager contextual "More" menu. When this parameter is enabled, the contextual "More" menu is displayed instead of the browser menu when you right-click lists on the details pane of a record in the web client.

> **Note**
>
> Note When the *enableMoreContextMenu* parameter is enabled, the browser menu is still displayed if you right-click in an area outside of a list, such as a text entry field. This enables you to use the copy and paste functionality that's included in the browser's menu.

Related topics

Service Manager Web tier
Web parameter: enableMoreContextMenu

1.5.8. Example: Running custom JavaScript from the web client

This example runs a JavaScript during loading the web client.

- The script opens an alert window before loading the Web client
- The script returns to the web client when the user clicks **OK**.

Follow these steps:

1. Sign in to the web tier.
2. Stop the web application server.
3. Browse to the `ext` folder of your Service Manager web tier's context root. For example, `sc/ext`.
4. Open the Java Server Pages (JSP) file named **appheader.jsp** in a text editor.
5. Search for the following text:
   `<script type="text/javascript">`
6. Add the following entries to the `appheader.jsp` file just after the script element:

```
document.onload = init();
function init()
{
alert("Testing document.onload from appheader extension./n Press Ok to continue with navmenu application.");
}
```

7. Save the `appheader.jsp` file.
8. Clear your web application server's cache.
9. Restart your web application server.
   The web client now runs your custom JavaScript when the web client starts.

Related topics

Branding the web client

1.5.9. Example: Sending Web tier URLs through e-mail notifications

The following example creates a notification record with the following properties.

- Creates an e-mail message whenever someone updates a Service Desk interaction record.
- The e-mail message contains a direct URL to the updated interaction record.
- The e-mail message contains a URL to the self-service interface if the record was opened from self-service or a URL to the Web client interface if it was opened from any other interface.

Note: This example assumes you have installed the Service Manager Web tier and e-mail service.

Update the Web configuration file

1. Sign in to the Service Manager web tier.
2. Stop the Web tier Web application server.
3. Open the Service Manager Web tier's Web configuration file (web.xml) in a text editor.

   Note
   Note The web.xml file is WEB-INF folder of your Web tier's context root.

4. Search for the following entry:

   ```
   <init-param>
     <param-name>serverHost</param-name>
     <param-value>localhost</param-value>
   </init-param>
   ```

5. Change the `localhost` param-value to the fully-qualified domain name of your Service Manager server, for example, myserver.mydomain.com.
   Replace the values *myserver* and *mydomain.com* with the server host name and domain name of the Service Manager server.

   Important
   Important You must specify a fully qualified domain name for the **serverHost** parameter or the server will generate invalid URLs.

6. Save the Web configuration file.

Update the system information record:

1. Restart your Web application server.
2. Sign in to the Service Manager server as a system administrator.
3. Click **System Administration** > **Base System Configuration** > **Miscellaneous** > **System Information Record**.
4. Click the **Active Integrations** tab.
5. In the **WebServer URL** field, type the fully qualified URL to your Web tier. For example:

   ```
   http://myserver.mydomain.com:myport/SM/index.do
   ```

   Replace the values *myserver* and *mydomain.com* with the server host name and domain name of the server running the Service Manager Web tier. Replace *myport* with the communications port your Service Manager Web tier Web server listens for HTTP requests.
   The server stores the value of this field in the **$L.web.url** global variable.
6. In the **ESS URL** field, type the fully qualified URL to your Web tier. For example:

   ```
   http://myserver.mydomain.com:myport/SM/ess.do
   ```

   Replace the values *myserver* and *mydomain.com* with the server host name and domain name of the server running the Service Manager Web tier. Replace *myport* with the communications port your Service Manager Web tier Web server listens for HTTP requests.
   The server stores the value of this field in the **$L.ess.url** global variable.
7. Save the system information record.

Update the SD.incident.mail form:

1. Sign out and restart the Service Manager server.
2. Sign in to Service Manager again as a system administrator.
3. Click **Tailoring** > **Forms Designer**.
4. Search for the form SD.incident.mail.
5. Click **Design**.
6. Add a new text area field with the following properties.

Property	Value
Input	$L.web.url
Width	400

This form will display the URL to the Web client interface.

7. Make any other changes to the form.
   You will use this form as the template for your e-mail messages.
8. Click **OK** twice to exit Forms Designer and save the form.
9. Search for and select the **SD.incident.mail** form .
10. Click **More** or the More Actions icon, and then click **Copy/Rename**.
11. In the **New Name** field, type `SD.incident.mail.ess` .
12. Select the **Copy** option.
13. Click **OK**.
14. Click **Design**.
15. Select the text area field you created to display the Web tier URL and update it to have the following properties.

Property	Value
Input	$L.ess.url
Width	400

This form will display the URL to the self-service interface.

16. Click **OK** twice to exit Forms Designer and save the form.

Update the SM Update notification record:

1. Click **Tailoring** > **Notifications** > **Notifications**.
2. Search for the **SM Update** notification record.
3. Add the following new rows to the notification record.

URL sent	Msg class	Msg No.	Arguments	Condition	Format	Notify Method	Recipient(s)
Self-service	sm	10	incident.id in $L.file	ess.entry in $L.file=true	SD.incident.mail.ess	email	contact.name in $L.file
Web client	sm	10	incident.id in $L.file	nullsub(ess.entry in $L.file,false)=false	SD.incident.mail	email	contact.name in $L.file

4. Click **OK** to save the notification record.

Create and update a self-service interaction record:

1. Open a web browser and sign in to the self-service interface.
2. Create a new self-service interaction record and note the interaction record ID.
3. From a Windows client, click **Service Desk > Search Interaction Records**.
4. Use the interaction record ID of the self-service interaction record to retrieve the interaction record details.
5. Add a category, sub category, and update to the self-service interaction record.
6. Exit out of the self-service interaction record and return to the Service Desk search form.
7. Search for SD1001.
8. Open the **Activities** section.
9. In the **Update** field, type an update.
10. Click **Save**.

Review the output event

1. Click **Tailoring** > **Event Services** > **Output Events**.

2. Click **Search**.
3. Look for an output event with an **Event Code** of email and the ID number of the self-service interaction record that you created in the **External Information String** field.
4. If you are working in a Windows client, right-click in the **External Information String** field and click **Magnify** to display the field text in a new window.
5. Search the form for a URL.
   The URL should contain the path to your self-service interface, for example:

   ```
   http://myserver.mydomain.com:myport/SM/ess.do?
   ctx=docEngine&file=incidents&query=incident.id=%22SD10005%22
   &queryHash=eb12d3d8&action=&title=Interaction%20;SD10005
   ```

6. Look for an output event with an **Event Code** of email and SD1001 in the **External Information String** field.
7. If you are working in a Windows client, right-click in the **External Information String** field and click **Magnify** to display the field text in a new window..
8. Search the form for a URL.
   The URL should contain the path to your Web client interface, for example:

   ```
   http://myserver.mydomain.com:myport/SM/index.do?
   ctx=docEngine&file=incidents&query=incident.id=%22SD1001%22
   &queryHash=89472347&action=&title=Interaction%20;SD1001
   ```

Related topics

Example: Running custom JavaScript from the Web client
Generating Web tier URL queries
Add URLs for Service Manager clients

1.5.10. Example: Notifying specified operators upon an incident update

This example specifies a group of operators that will receive a notification when an incident is updated.

> **Note**
>
> Note The notification will not be sent to a specified operator if the incident is created or updated when the operator is offline.

Add a Script Library record[edit]

1. Execute the `sl` command in the command line.
2. In the **Name** field, enter IncidentMsg1.
3. Select a package in the **Package** field.
4. Enter the following script in the text box below:

   ```
   system.functions.msg("Incident "+vars.$file.number+" updated successfully by: "+vars.$lo_operator.name , 1,
   ["falcon","Incident.Manager"], "popup", 1);
   ```

   You need to specify each of the operators in the script. The example script above specifies that falcon and Incident Manager will receive the notification of an incident update.

   > **Note**
   >
   > Note The script above is an example for Service Manager 9.32. If you use a previous version of Service Manager, change `vars.$file.number` to `vars.$L_file.number`.

5. Click **Add**.
6. Click **Compile**.

Update the `im.save` process[edit]

1. Click **Tailoring** > **Document Engine** > **Processes**
2. Search the `im.save` process.
3. Select the **RAD** tab of the `im.save` process.
4. Scroll down until you find the first empty RAD Application.
5. Add the following script in the **Post RAD Expression** field of the empty RAD Application:

```
if $L.continue then ($L.void=jscall("IncidentMsg1"))
```

6. Click **Save**.

Contents

- 1 Add a Script Library record
- 2 Update the im.save process

1.5.11. Generating Web tier URL queries

If you have installed a Web tier, you can have Service Manager server generate URL queries to your Web tier that display specific records in your applications. You can use these generated URL queries to programmatically send links to your users when records are opened, updated, or closed. Service Manager offers the following out-of-box methods for generating valid Web tier URL queries.

- Create custom JavaScript that uses the **makeSCWebURL** method. You can invoke your custom JavaScript from a macro or RAD expression.
- Create custom notification records that use the Web tier URLs defined in the system information record.

When a user clicks on a generated URL query (link), Service Manager requires the user to log in before displaying the requested data, although the user can bypass the login screen if you have enable trusted sign-on on the Web tier.

By default, the Service Manager server secures Web tier URL queries by including a unique hash key in the URL. This unique key prevents people from modifying the URL and attempting to access restricted parts of your system. If the URL query doesn't match the unique hash key, Service Manager displays a warning message in the log file.

If you want to generate URL queries outside of Service Manager you can disable the unique hash key using the **querysecurity** parameter. Disabling this parameter in the Service Manager initialization file allows the server to accept URL queries without a unique hash key. If you disable the security hash, access to tables is controlled by the Document Engine. To restrict access to tables through the Document Engine, you must enable security features such as format control or application profiles that restrict access at the operator level.

Web tier URL format

Service Manager requires generated web tier URLs to use the following format.

```
http://myserver.mydomain.com:myport/SM/index.do?ctx=docEngine
&file=incidents&query=incident.id=%22SD1001%22
&queryHash=89472347&action=&title=Interaction%20;SD1001
```

URL portion	Required?	Description
http://myserver.mydomain.com:myport/SM/*index.do*	Yes	This portion of the URL specifies the web tier host name, port, and web client connection to use. Use *index.do* to have users connect to the standard web client. Use *ess.do* to have users connect to the self-service web client.
ctx=docEngine	Yes	This portion of the URL specifies that Service Manager use the Document Engine to fulfill the query.
&file=*incidents*	Yes	This portion of the URL specifies the table you want to query for records. You must use the file name for the table as specified in the database dictionary.
&query=*incident.id=%22SD1001%22*	Yes	This portion of the URL specifies the Service Manager query you want to use to search for records. You must URI encode the query string to prevent special characters from invalidating the URL. For example, use %22 instead of quotation marks.
&queryHash=*89472347*	No	This portion of the URL specifies the optional hash key used to encode the URL query. The hash key prevents users from modifying the URL query to view other portions of Service Manager. You can only include a hash key if you generate the URL from the makeSCWebURL method.
&action=	No	This portion of the URL specifies the Document Engine action you want the URL query to perform. By default, the URL query performs a search operation.
&title=*Interaction%20;SD1001*	No	This portion of the URL specifies the optional title of the query. You must URI encode the query string to prevent special characters from invalidating the URL. For example, use %20; instead of a space.

Related topics

Service Manager macros
Notifications
Add URLs for Service Manager clients
Service Manager Programming
Parameter: querysecurity

1.6. Windows client

Service Manager includes both Windows and Web clients. The Windows client is intended for administrators and implementers, while the Web client is suitable for most end users. The Windows client enables you to design forms and work with the dbdict; capabilities that aren't available to the Web client user.

The Windows client includes the following features:

Interface

- A dynamic user interface that you can customize with preferences to change the way you interact with perspectives and views
- Windows client update capability

Client/server connection

- A Connection dialog that enables you to define multiple connections to different servers and saves them from one session to another
- Automatic connection for connection profiles with the automatic logon option enabled

System Navigator

- Favorites folder to save shortcuts to frequently used applications, forms, and queries
- Simplified access to forms, files, and records through the System Navigator

Applications

- Application-specific user interface elements
- A graphical workflow used by the Request and Change Management applications
- Charts and dashboards to show data graphically

Help

- Field help
- Help for end users and administrators

Other features

- UTF-8 (Unicode) support to display data in any language
- SOAP-based client/server communication
- Automatic reconnection of client/server sessions after brief network connection disruptions

Related topics

Clients
Web client
Differences between the Windows and web clients
Accessing Service Manager
Sign in to the Windows client

1.6.1. Administration perspective

The Service Manager Administration perspective contains several Service Manager administrative views. It is easy to show or hide administrative views by selecting or closing a single perspective instead of each individual view. You can use the Administration perspective to troubleshoot the user interface or monitor client/server traffic.

Within the Administration perspective, there are eight available views arranged in tab format. Each view displays different information.

- Messages
- Console
- Detail Data
- Detail Form
- List Data
- List Form
- Last Request
- Last Response



Note

Note

Views and perspectives are available in the Windows client only.

Console view

Note: Views are available in the Windows client only.

The console view displays the information sent to the system log, depending on the logging level. If you enable SOAP tracing from the Connection Dialog, SOAP messages also appear. If an error occurs, you may also see error messages or a stack trace from the Java Runtime Environment (JRE). If you are an administrator, you might want to monitor the console during normal activities instead of locating the sm.log file and opening it with a text editor. Ensure that you do the following:

- Specify the sm.log file in the **Window** > **Preferences** > **Service Manager** > **Logs** preferences.
- Ensure that the log file is accessible from your client.

Stopwatches view

The Stopwatches view is an administrative tool that lists information about the number of milliseconds required for some Service Manager client operations. This view shows the number of individual service desk interactions, the shortest time (Min), the longest (Max) time, the average, and the total time. It is helpful for troubleshooting.

The number of displayed operations varies. They are neither configurable or predictable because they depend on your Service Manager installation and current actions. The following table lists some operations that might appear in this view.

Operation	Purpose
Form.generate	The elapsed time to render the form using information sent by the server.
Generator.generate	The elapsed time required to retrieve a System Navigator branch from the server.
Navigator.getChildren	When you click a closed node in the System Navigator, the elapsed time required to expand that node and render content.
SOAP.transact	the elapsed time to exchange one message between the client and server. The stopwatch starts when the client sends the message and stops when it receives the server response. This time doesn't include message processing.
XSL.transform.jdom	The elapsed time for XSL transformation of forms and menus.

Click the appropriate icon to modify the Stopwatches view.

Icon	Function
trefresh.gif	Refresh all stopwatches.
	Reset all stopwatches.
	Save current stopwatches state in the log file.

XML views

The Administrator perspective contains these views that display data exchanged between the client and server.

View name	What's in this view?	Use this view to	Related capability word
Detail Data	Displays all data in the records (in XML format) that populate the current form. For example, this view shows related query information, as well as RAD-defined variable values, associated displayoptions, and messages.	Trace or debug Service Manager transactions.	Requires the Debug capability word.
Detail Form	Displays the form fields (in XML format) in the current form.	Trace or debug Service Manager transactions, Dynamic View Dependencies (DVD), or display applications.	Requires the Debug capability word.
List Data	Displays all records (in XML format) that appear in the current record list.	Trace or debug Service Manager queries.	Requires the Debug capability word.
List Form	Displays the form fields (in XML format) for the current record list form.	Trace or debug Service Manager queries.	Requires the Debug capability word.
Last Request	Displays the XML-formatted action, modified data, and cursor position of the last request sent from the client to the server.	Trace or debug client/server communication.	Requires the Debug capability word.
Last Response	Displays the XML-formatted data of the last response sent from the server to the client.	Trace or debug client/server communication.	Requires the Debug capability word.

Administrative views

You can select Service Manager views individually to create a customized workbench or saved perspective.

Administrative and RAD Debugger views appear only if you have the Admin plug-in installed. Each view displays different information.

View name	Included in this perspective	What's in this view?	Use this view to
Console	Service Manager Administrator	Displays log messages for the client, depending on your log preferences.	Review log messages.
Detail Data	Service Manager Administrator	Displays all data in the records (in XML format) that populate the current form. This view shows related query and SLA information, as well as RAD-defined variable values, associated display options, and messages.	Trace or debug Service Manager transactions.
Detail Form	Service Manager Administrator	Displays the form fields (in XML format) that appear in the current form.	Trace or debug Service Manager transactions, Dynamic View Dependencies (DVD), or display applications.
Last Request	Service Manager Administrator	Displays the XML-formatted action sent from the client to the server.	Trace or debug client/server communication.
Last Response	Service Manager Administrator	Displays the XML-formatted response sent from the server to the client.	Trace or debug client/server communication.
List Data	Service Manager Administrator	Displays all records (in XML format) that appear in the current record list.	Trace or debug Service Manager queries.
List Form	Service Manager Administrator	Displays the form fields (in XML format) for the current record list form.	Trace or debug Service Manager queries.
Properties	Service Manager Administrator	Displays the properties of the current form.	Troubleshoot form problems.
RAD Debugger	Service Manager Administrator	Isolated RAD processes.	Troubleshoot RAD processes.
Stopwatches	Service Manager Administrator	Displays timing information about Service Manager processes.	Identify slow processes or opportunities for system tuning.
Script Library Explorer	Service Manager Administrator	Displays a tree view of Service Manager Script Library records.	Access Script Library.
Dashboard	Service Manager Interface	Displays one or more charts.	Compare data visually.
Messages	Service Manager Interface	Displays client messages, depending on your log preferences.	Review log messages.
Secondary System Navigator	Service Manager Interface	Displays another view of the System Navigator at the bottom of the workbench. You can move this second view to another location.	Add another navigation view to access all Service Manager components.

System Navigator	Service Manager Interface	Displays a tree view of Service Manager applications, utilities, and data.	Access all Service Manager components.

Access an administrative view

Applies to User roles: Administrator

The administrative view provides additional panes to display different types of information that may be useful in a debug or problem solving situation for an administrator. In the out-of-box system, the messages and system navigator panes are standard views.

To access an administrative view:

1. Click **Window** > **Show View** > **Other**.
2. Expand the **Service Manager Administrator** folder. Choose one of these views:
   - Console
   - Detail Data
   - Detail Form
   - Last Request
   - Last Response
   - List Data
   - List Form
   - Properties
   - RAD Debugger
   - Stopwatches
   - Web Preview

1.6.2. Preferences

You can customize the way you view the Windows client by changing settings in Preferences dialog.

If you are an administrator, you can configure a common set of Windows client preferences. For example, you can ensure all users have the same log file, security, and help server settings.

Administrators also have the option to set Windows client preferences in the sm.ini file. These preferences override any conflicting settings that appear in the Windows client Preferences dialog.

Administrators can configure a common set of Web client preferences in the ../WEB-INF/web.xml file. Thereafter, these preference settings apply to all Web client sessions.

If you are an end user, you can change default settings to customize your Windows client environment. For example, you can change some appearance, chart, and spell checking properties.

There is a Preferences filter available that enables you to filter preferences so that only a few preferences are displayed or a specific preference set (for example Appearance) is displayed. By typing the beginning letters or complete name of the preference setting in the Preferences filter text box, you can filter the list so only those preferences that match what you type appear in the list.

Related topics

Appearance preferences
Log preferences
Security preferences
Spell Checker preferences
Client parameters for web clients
Set Windows client preferences

1.6.2.1. Set Windows client preferences

Applies to User roles: All roles

To set Windows client preferences:

1. From the Service Manager menu bar, click **Window** > **Preferences**.
2. Expand the Service Manager Preferences filter:
3. Highlight the following filter settings to update your preferences:
   - **Service Manager**
   - **Appearance**
   - **Charts**
   - **Logs**

- **Security**
- **Spell Checker**

4. Expand the Help Preferences filter.
5. Highlight the following filter settings to update your Help preferences:
   - **Help**
   - **Help Server**
6. Restart the client for these preferences to take effect.

Related topics

Preferences
Appearance preferences
Log preferences
Security preferences
Spell Checker preferences
Client parameters for web clients

1.6.2.1.1. Appearance preferences

This Preferences dialog enables you to customize the Windows client.

To access appearance preferences, from the Service Manager menu bar click **Window** > **Preferences** > **Service Manager** > **Appearance**.

Preference	Description	Default setting
Use server provided tab order	The default Windows tab order is from left to right, and top to bottom. When you specify a custom tab order with Forms Designer, the server follows this order.	False
Show Messages View when new messages occur	Enable a view to display messages.	False
Skip readonly controls when tabbing	Enable so that read-only fields will be skipped when tabbing through a form.	False
Prompt for save	Prompt to save information before you leave a form.	True
Show detail pane with record list	Display the record list at the top of a form.	True
Sort menu items alphabetically in the System Navigator	Display System Navigator menu items alphabetically.	True
Show context-sensitive help debug information	Display context-sensitive help test information.	True
Font	Specifies the primary font used by Service Manager. Hewlett-Packard delivers Service Manager with a default font for each language. For example, in English, Tahoma Regular is the default font. In Japanese, MS UI Gothic is the default font. Although you can change the font face and size, we recommend the default font for the best results. Font sizes larger than 8 points may produce unpredictable results.	Varies depending on the language pack installation
Percentage to increase form width	Specify a percentage to increase the width of the forms. Useful to display localized forms that may require more space to display translated text.	0

Related topics

Preferences
Service Manager preferences
Log preferences
Security preferences
Spell Checker preferences
Client parameters for Windows clients
Set Windows client preferences

1.6.2.1.2. Service Manager preferences

This Preferences dialog enables you to customize the Windows client.

To access Service Manager preferences, click **Window** > **Preferences** > **Service Manager**.

Preference	Description	Default setting
Client printing	This preference allows users to access the client-side printing feature by using the Print icon in the Windows client. Web and Windows clients can also use the Print commands supplied by the web browser or Windows operating system.	True
Client side load/unload	This preference determines the destination of unload files. When enabled, Service Manager saves unload files on the client system. When disabled, Service Manager saves unload files on the server. Requires a client restart to take effect.	True
Use form caching	Form caching improves server responsiveness by reducing the need to load forms.	True
Record list request count	This preference specifies the number of records the client requests from the server when displaying a record list. The user can view additional records in the record list by advancing to the next page of records or by using the go to page option. The default value is set to maximize the server's response time to client record requests. The fewer records the client requests of the server the faster the server can answer the query.	32
Image path	This preference specifies the path to any custom images you want the Windows client to use. System administrators may use this preference to customize the Windows client to use custom branded images.	None
Display names of missing images	This preference determines if the Windows client displays the name and of any missing images. This preference is intended to make customizing the Windows client easier.	False
DDE server name	This preference determines the name the Windows client uses when acting as a DDE server for integrations with DDE applications such as computer telephony integration (CTI).	HP_Service_Manager
Show line number	This preference displays line numbers in the comparison window when you perform application upgrades or apply application patches.	True

Related topics

Preferences
Appearance preferences
Log preferences
Security preferences

Spell Checker preferences
Client parameters for Windows clients
Set Windows client preferences

1.6.2.1.3. Log preferences

You can set log preferences to control how Service Manager logs and displays messages.

To access log preferences in the Windows client, click **Window** > **Preferences** > **Service Manager** > **Logs**.

Preference	Description	Default setting
Use a specific file	If you want to use a specific log file, type the path and name of the file or browse to its location.	The default log file is C:\Users\<username>\ServiceManager\workspace\.metadata\.log.
Message level for console view	Choose the severity of messages that appear in the Console view. Select these logging options: None Debug, Info, Warning, and Error messages Info, Warning, and Error messages Warning and Error messages Error only messages	None
Message level for log file	Choose the severity of messages that appear in the log file. Error only Debug, Info, Warning, and Error messages Info, Warning, and Error messages Warning and Error messages None	Error only
Append exception trace to log option	Add the stack trace of the associated exception to the log.	False

Related topics

Preferences
Appearance preferences
Security preferences
Spell Checker preferences
Client parameters for web clients
Set Windows client preferences

1.6.2.1.4. Security preferences

Security preferences point to the location of the CA certificate keystore, client certificates, and private keys.

To access Security preferences in the Windows client, click **Window** > **Preferences** > **Service Manager** > **Security**.

Preference	Description	Default value
FIPS Mode	Enables FIPS mode in the Windows Client. When FIPS mode is enabled on the server side (that is, fipsmode:2 is specified in the server's sm.ini file), you have the option to enable FIPS mode in the Windows Client. For more information, see Configuring FIPS mode in Service Manager. **Caution** Caution Do not enable this option when the Service Manager Server is using the `fipsmode:1` legacy configuration in the server's sm.ini file.	Disabled
JCE provider name	**Note** Note This option is provided only for backward compatibility. Use it only when the Service Manager Server is using the `fipsmode:1` legacy configuration in the server's sm.ini file. Specify the name of a FIPS-certified third-party JCE provider you use for the Windows client. **Note** Note This option becomes read-only when the **FIPS Mode** checkbox is selected.	JsafeJCE

JCE provider class name	**Note** Note This option is provided only for backward compatibility. Use it only when the Service Manager Server is using the `fipsmode:1` legacy configuration in the server's sm.ini file. Specify the class name of a FIPS-certified third-party JCE provider you use for the Windows client. **Note** Note This option becomes read-only when the **FIPS Mode** checkbox is selected.	com.rsa.jsafe.Provider.JsafeJCE **Note** Note This option is provided only for backward compatibility. Use it only when the Service Manager Server is using the `fipsmode:1` legacy configuration in the server's sm.ini file.
CA certificates file	Specify the path to a keystore of Certification Authorities (CA) used to certify the client and server certificates.	Service Manager has an out–of–box example of this file in \<Web Tier>\WEB-INF\cacerts.
CA certificates password	Specify the password to access the CA certificates file.	None.
Client keystore file	Specify the path to the keystore containing the client certificate and associated private keys.	None. You must create this file.
Client keystore password	Specify the password to access the client keystore.	None.

Related topics

Preferences
Appearance preferences
Log preferences
Spell Checker preferences
Client parameters for web clients
Configuring FIPS mode in Service Manager
Set Windows client preferences

1.6.2.1.5. Spell Checker preferences

These preferences specify how to customize the spell checker. To access Spell Checker preferences in the Windows client, click **Window** > **Preferences** > **Service Manager** > **Spell Checker**.

You can select or clear General Options, customize the dictionary, and choose one of these language dictionaries:

- American English
- British English
- French
- German
- Italian
- Spanish

Note: You must select a custom dictionary if you want to add new terms.

Spell checker custom dictionaries

You can configure the Service Manager Windows client to use a custom dictionary when running the spell checking utility. You can save a custom dictionary file on a network share and then use the Client Packaging Utility to direct the Windows clients to the network path.

Change the spell checker dictionary

Applies to User roles: Administrator

You must have administrative access to the server operating system to use this procedure.

To change the spell checker dictionary:

1. Sign in to the Windows Client.
2. Click **Windows** > **Preferences**.
   The Preferences window opens.
3. Click **Spell Checker Options**.
   The Spelling Preferences window opens.
4. From the **Language** box, select the language dictionary you want to use.
5. To define a custom dictionary, click **Browse**.
   - Type or browse to the path of your custom dictionary.
     You can either create your own file or use the default custom dictionary file `customdict.tlx` provided in the `plugins\com.hp.ov.sm.client.eclipse.user_version_number\spellchecker` folder.
   - Click **OK**.
6. To add correction entries to the custom dictionary, click **Edit**.
   The Edit user dictionary window opens.
   - To add a new correction entry to the dictionary, type the *misspelled* word in the **Words** field.
   - Type the *correction* in the **Other word** field.
   - Select an **Action** from the available list.
   - Click **Add word**.
   - Repeat these steps for each custom correction entry you want to add.
7. Select any other spelling preferences you want to use.
8. Click **OK**.

1.6.2.1.6. Help preferences

This Preferences dialog enables you to customize the way Service Manager help information is displayed.

To access Help preferences, click **Window** > **Preferences** > **Help**.

Preference	Description	Default setting
Use external browser	Opens the Service Manager help in your default web browser. By default, Service Manager opens the help in a Help View window.	False
Open window context help in a dynamic help view	Opens window context help information in a dynamic help view.	True
Open window context help in an infopop	Opens window context help information in a pop-up information window.	False
Open dialog context help in dialog tray	Opens dialog context help information in the dialog tray.	True
Open dialog context help in an infopop	Opens dialog context help information in a pop-up information window.	False
Show all potential hits (faster)	Shows all records that fully or partially match your search criteria.	True
Show only actual hits	Shows only records that fully match your search criteria.	False

Related topics

Preferences
Appearance preferences
Log preferences
Security preferences
Spell Checker preferences
Help Server preferences
Client parameters for Windows clients
Set Windows client preferences

1.6.2.1.7. Help Server preferences

This Preferences dialog enables you to specify your Help Server settings so that you can access the online help documentation by clicking **Help** > **Help Contents** from the Windows client. Service Manager launches the help contents from the URL as you specified in the Help Server URL field.

To access Help Server preferences, click **Window** > **Preferences** > **Help**> **Help Server**.

Preference	Description	Default setting
Use a Help Server to access documentation	Enables you to access the online help from Service Manager.	False
Help Server URL	Specifies the URL of the help server. For example, `/doc/Service_Manager/9.xx/Home`.	None

Related concepts

Preferences
Appearance preferences
Log preferences
Security preferences
Spell Checker preferences
Help preferences
Client parameters for Windows clients
Set Windows client preferences

1.6.2.2. Setting Windows client preferences from the server

You can define global Windows client preferences from the *sm.ini* file available in the *RUN* folder of your Service Manager server installation. The client parameters you define in this file override any individual client preferences. You must have administrative access to the server operating system to set Windows client preferences using client parameters.

Set Windows client preferences from the server

You must have administrative access to the server operating system to use this procedure.

To set Windows client preferences from the server:

1. Review the list of client parameters for Windows clients and determine which parameters you want to define from the server.
2. Open the *sm.ini* file in a text editor.
   This file is located in the *RUN* folder of your Service Manager server installation.
3. Add or edit the client parameters for Windows clients.
4. Save the file.

All Windows clients that log in to the Service Manager server use the new client parameters you defined.

Related topics

Client parameters for Windows clients

1.6.3. Views

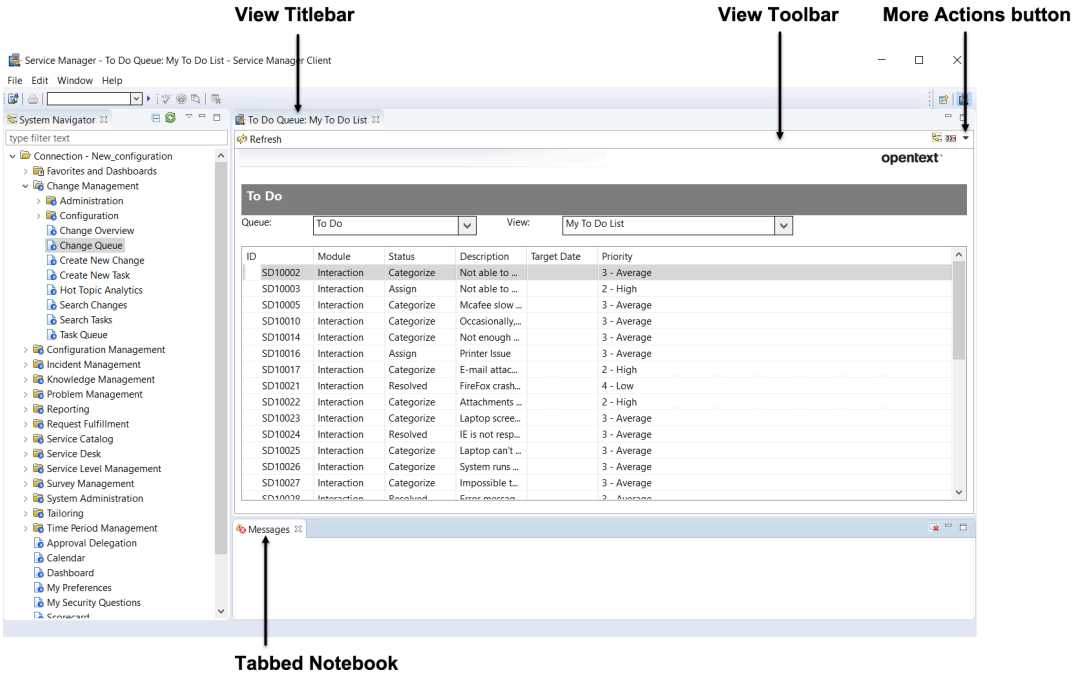
Views are a Windows-client feature that enable you to display editors and system information outside the editor, and provide alternate ways to access forms. For example, you can navigate the file system hierarchy with the System Navigator, and access and edit information about a form using the Properties view. Service Manager saves any changes that you make in a view immediately.

A Perspective can contain one or more views.

Views have menus and toolbars that affect only the items in the view. For example, if you click **Expand All** in the System Navigator, the tree expands but you see no change in the editor.

Most views also have a *More Actions menu* that displays a list of related Service Manager actions and a *context menu* that displays a list of related actions for the view. Right-click the view's titlebar to display the view's context menu; right-click the form or record list to display Service Manager's options menu.

You can save screen space and customize your perspective by stacking views. Drag one view over another to create a *Tabbed notebook*. You can also change the view sizes. You can't open more than one of each view at one time. If necessary, use the Secondary System Navigator to create a second instance of a view.



Component	Description
View Toolbar	Each view has an individual toolbar with buttons that support available functions.
View Titlebar	Click the X to remove the view from the workspace. Right-click the view's title bar to display the view's context menu.
Tabbed Notebook	Tabbed notebooks support stacked views. Click any tab to move that view to the top of the stack.
More Actions Menu	Click the down arrow to display the More Actions menu. Right-click the form or record list to display the Service Manager's More Actions menu.

Related topics

Perspectives
Views

1.6.3.1. Views

The following information identifies all of the views available in Service Manager.

Note: Views are available in the Windows client only.

Service Manager Administrator views

View	Description
Console	Displays real-time logging and debugger messages.
Detail Data	Displays all data in the records (in XML format) that populate the current form.
Detail Form	Displays the form fields (in XML format) that appear in the current form.
Last Request	Displays the XML-formatted action sent from the client to the server.
Last Response	Displays the XML-formatted response sent from the server to the client.
List Data	Displays all records (in XML format) that appear in the current record list.
List Form	Displays the form fields (in XML format) for the current record list form.
Properties	Displays the properties of the current form.
RAD Debugger	Displays isolated RAD processes for troubleshooting.
Stopwatches	Displays timing information about Service Manager processes.

Service Manager Interface views

View	Description
Dashboard	Displays one or more charts.
Messages	Displays client messages, depending on your log preferences.
System Navigator	Displays a tree view of Service Manager applications, utilities, and data.
Secondary System Navigator	Displays another view of the System Navigator at the bottom of the workbench. You can move this second view to another location.

1.6.3.2. Related objects

Related objects are data-model relationships created by a System Administrator for users of a specific database. Administrators can customize System views for users that specify the relationships between database objects and fields. These relationships can define one-to-one, many-to-one, or many-to-many relationships. One-to-one and many-to-one relationships are defined by a database JOIN between two corresponding tables for the data relationship. Many-to-many relationships are modeled by a relational database JOIN between the two corresponding tables and the cross-link table defining the relationship.

1.6.3.3. Messages view

Service Manager notifies you when you have activity messages to review. For example, each time you add or change a record, open or close an incident, complete a Service Manager task, or an error occurs, Service Manager generates a message. These are the choices you have to view messages:

- Configure the Preference dialog to display the Message view by default.

Click **Window** > **Preferences** > **Service Manager** > **Appearance** and choose this preference:
Show message window when new messages arrive.
The Message view opens automatically when messages occur.

- Show the Messages view on demand.

Click **Window** > **Show View** > **Messages**.

- Show the Messages view as needed. When Service Manager sends a new message, an information icon appears.

Click the icon or the toolbar.
The Message window displays the new message and any others that appear during your Service Manager session.

The server can process different threads concurrently and each thread can generate a separate stream of messages. If you have a single client running, all messages generated by that client connection appear in the same Messages view. If you have more than one client connected, each client window has its own Messages view that might display only the messages affecting that connection.

>

The Service Manager active notes feature notifies you when new activity messages appear in the Messages view. For example, each time you add or change a record, open or close an incident, complete a Service Manager task, or an error occurs, the active notes icon notifies you that a new message is available. To activate the active notes feature, you can set preferences to enable the Messages view to appear each time a new message occurs.

The Active Notes icon is flat when there is only one message and stacked when there are multiple messages.

View messages[edit]

Applies to User roles: All roles

Service Manager messages appear at the top of the detail window.

Icon	Icon name	Meaning
	Information	Information about the most recent action
	Warning	A warning about the most recent action
	Error	The most recent action caused an error to occur

To view all messages in a Service Manager session:

1. Click the message icon to open the Messages window.
2. To copy a message, select the message and use any standard Windows copy operation, such as Ctrl+C, to copy it to the clipboard.
3. Click the Close button to close the Messages view.
4. To erase all messages, click **Erase**.

Note: If you enable the Messages view from the Service Manager Preferences dialog, it appears automatically when messages occur.

Contents

- 1 View messages

1.6.4. Perspectives

Note: Perspectives are available in the Windows client only.

A perspective contains one or more views, and perhaps an editor, that provides all of the data and tools you need to complete a Service Manager task. Service Manager has a default perspective that's standard for accessing forms and data. The Administration perspective contains several Service Manager administrative views that help troubleshoot the user interface or monitor client/server traffic.

You can customize these standard perspectives, or create new ones according to your needs. For example, one user might want to add another view to all of the default Service Manager views. If you save this with a name, you can reuse it. Another user might want to see the same views, but docked or stacked differently within the Workbench. Building a customized perspective is similar to creating a personal Windows desktop.

The default Service Manager perspective opens the first time you log in to Service Manager. Click **Window** > **Open Perspective** to choose a default or customized perspective. You can also access perspectives from the icons on the left shortcut bar. When you choose a perspective for the first time, its icon appears on the shortcut bar and remains there until you end the session.

The following table lists available perspectives in Service Manager. You can define your own perspectives to customize the way you work with the Windows client.

Icon	Perspective name	Purpose	User
	Administration	Troubleshoot the user interface or monitor client/server traffic. Within the Administration perspective, there are eight available views arranged in tab format. Each view displays different information. For more information, see the Application Administration help.	Administrators
	Service Manager	The default perspective when you first log in. This perspective is for any Service Manager user and displays view and editor panes that help you access Service Manager applications.	Service desk operators and administrators

Create a new perspective[edit]

Applies to User roles: All roles

Note: Perspectives are available in the Windows client only.

To create a new perspective:

1. Start in an out–of–box perspective. The default perspective is the Service Manager perspective.
2. Arrange the views and editor according to where you want them to be.
3. Click **Window** > **Save Perspective As**.
4. Name the new perspective, then click **OK**.
5. Click the perspective icon on the shortcut bar. The new perspective is in the list of perspectives.

Open a list of perspectives[edit]

Applies to User roles: All roles

Note: Perspectives are available in the Windows client only.

To find a list of available perspectives:

1. From the Window menu click **Open Perspectives**. (You can also find the Open Perspectives shortcut on the Shortcut bar located on the left side of the Workbench.)
2. Click **Other**.

The out–of–box list of perspectives includes the following:

- Administration
- Service Manager (default)

Contents

- 1 Create a new perspective
- 2 Open a list of perspectives

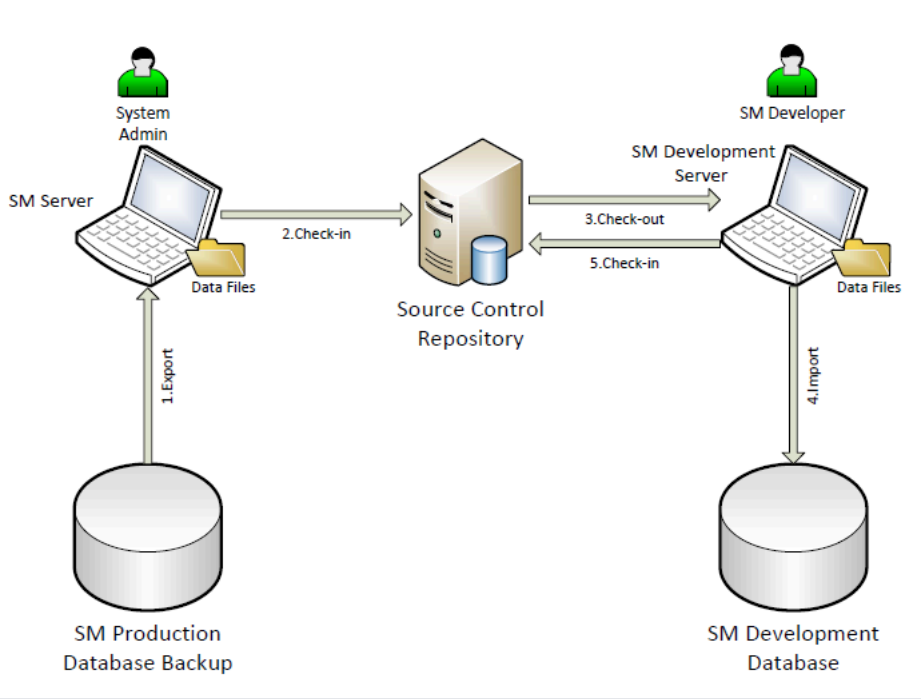
1.7. Implementing version control for your tailoring

Service Manager provides comprehensive and fully-integrated IT service management through variously customized applications. Although Service Manager already provides an in-tool revision control based on the individual record, it has always been a challenge to implement version control for customization stacks. Service Manager can integrate with Apache Subversion (SVN) or Git to provide a version control solution for your customized applications.

> **Note**
> Note
> This topic doesn't cover the basic concept of version control and the usage of Apache Subversion or Git.

In short, it takes three steps to build up your version control process by using SVN or Git:

1. A system administrator exports all the applications records from Service Manager and check in the exported files to an SVN or Git server to create a baseline repository.
2. Developers check out the latest version of the applications records and import them into a local Service Manager development environment.
3. Developers customize the applications according to site-specific requirements, and check in the changes to the SVN or Git server. The following flow chart illustrates your version control process:



Related topics

SVN version control process
Version control best practice
Applications data file represented in XML

1.7.1. SVN version control process

The process in this section is provided as an example assuming that you integrate your Service Manager with SVN for tailoring version control.

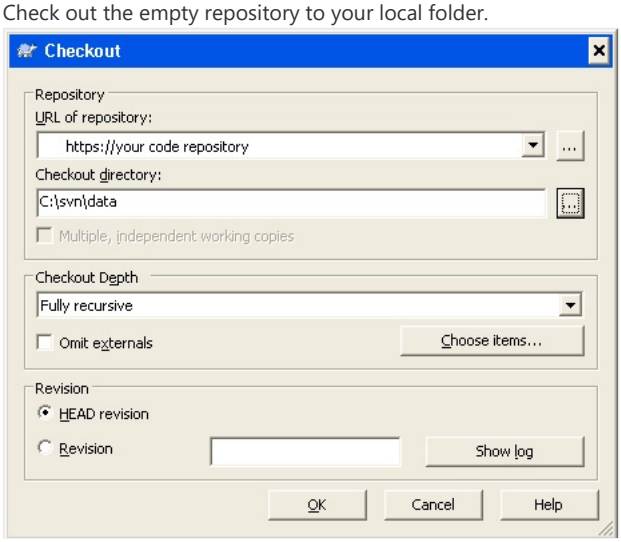
The SVN version control process includes the following:

Code repository preparation by a system administrator

The first step of your version control process is that a system administrator creates a code repository on an SVN server. This enables you to use the repository to track all changes made to applications objects as you create, develop, or extend the Service Manager applications.

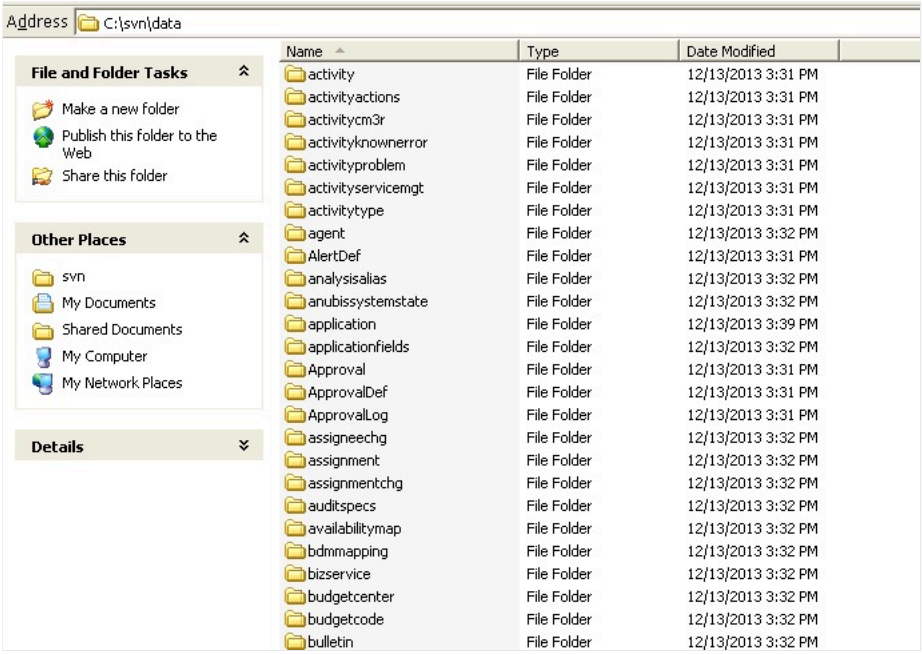
To leverage the SVN version control system, follow these steps:

1. Set up the SVN server and create the repository folder for the Service Manager applications data.
2. Set up the SVN client. To do this, follow these steps:
    1. Create a folder in your local drive to check the SVN code repository in and out. For example, you create "`c:\svn\data`."
    2. Check out the empty repository to your local folder.

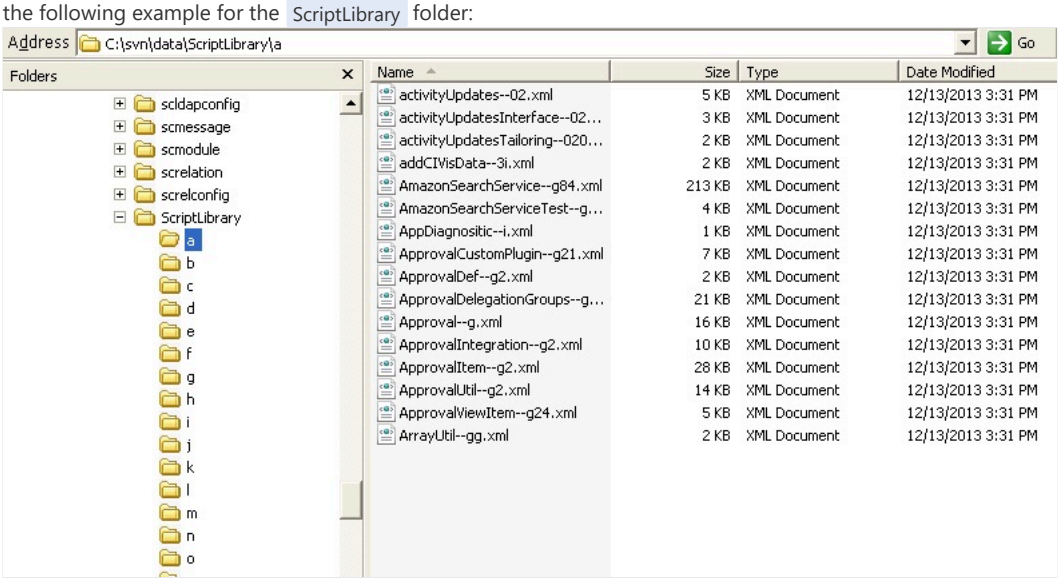


3. Configure Service Manager to use the repository. To do this, add the following two new lines to the `sm.ini` file in the `<Server Dir>/RUN/` directory:
   `svc_rootdir:` *<Your check-out Service Manager applications path, in this example,* `C:\svn\data` *>*
   `svc_mode:2`
   Make sure that your Service Manager server is installed on the computer that has the SVN client installed.

4. Run the **sm -svc_export** command to export the Service Manager applications data as XML files to the `svc_rootdir` folder. For information about more parameters to export the applications data, see Parameters to import and export files.
   Your output folder should resemble the following:



   The XML files are exported to subfolders that are alphabetically-ordered by the unique key names of the applications, as shown in the following example for the `ScriptLibrary` folder:



5. Commit all the files to the SVN repository.

Note

Note You should exclude any business-related records when you commit the files, as these types of records require a great deal of capacity and decrease efficiency. These types of records include the following, but your organization's records may differ:

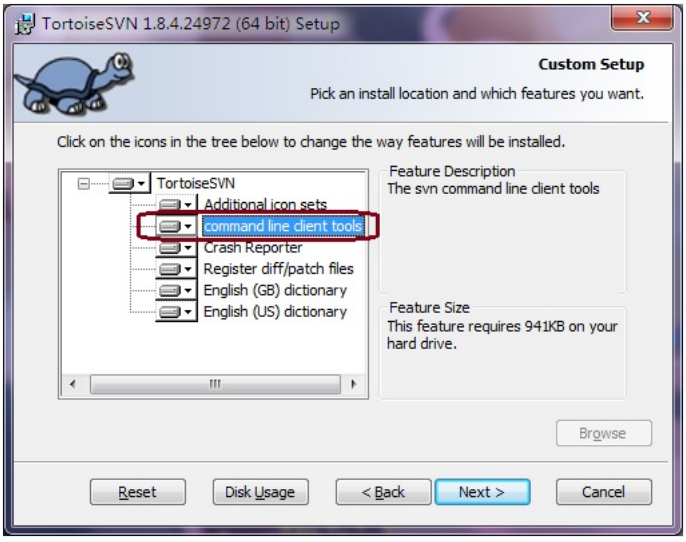
Object	Table
Interaction	incidents
Incident	probsummary
Problem	rootcause
Problem Task	rootcausetask
Known Error	knownerror
Known Error Task	knownerrortask
Change	cm3r
Change Task	cm3t

Development environment preparation by developers

Note

Note Environment preparation is only required when a developer starts applications development for the first time or starts to use a new computer.

1. Set up your SVN client. To do this, choose the command line client tools in SVN. The following example displays the command line client tools feature in TortoiseSVN:



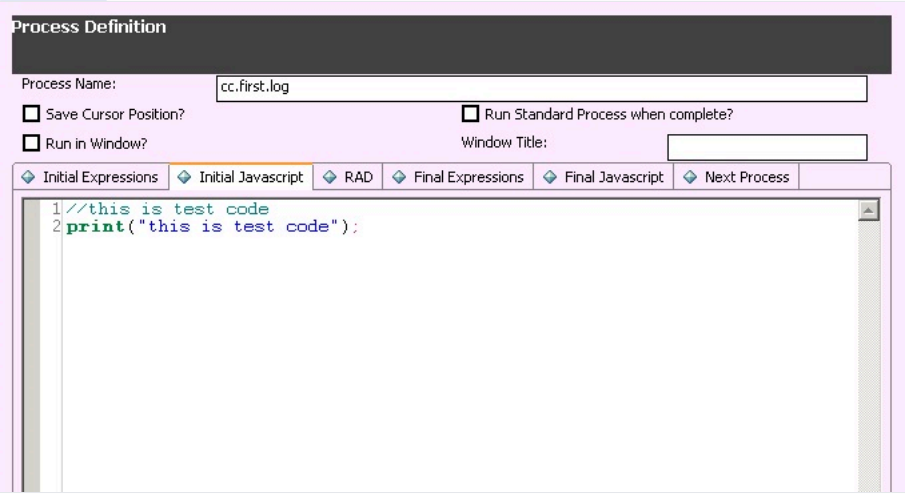
2. Create a folder in your local drive to check the SVN code repository in and out. For example, you create "`c:\svn\data`."
3. Set up Service Manager by adding the following two new lines to the `sm.ini` file in the `<Server Dir>/RUN/` directory:
   `svc_rootdir:` *<Your check-out Service Manager applications path, in this example,* `C:\svn\data` *>*
   `svc_mode:2`
   Make sure that the Service Manager server is already installed on the same computer, and then restart the server to enable the added svc command.
4. Check out the applications data files from the SVN repository to your local folder.
5. Import the applications data in the XML files by copying the production Service Manager database to the development system or by running the following command to a clean empty development database:
   `sm -svc_import -svc_cleanbuild:1`

Note

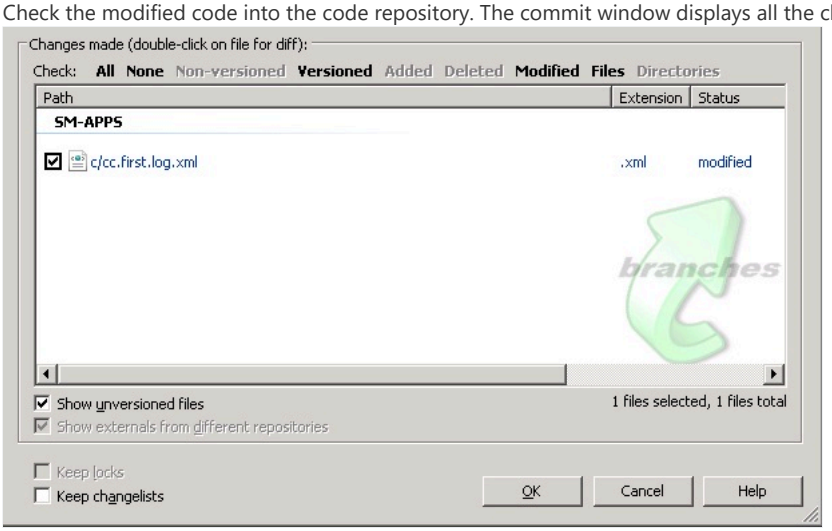
Note For information about more parameters to import the XML files, see Parameters to import and export files.

Development process by developers

1. Update all the source code to the latest version from the code repository.
2. Modify the records by the Service Manager client and then save the changes to automatically synchronize the XML files in the `svc_rootdir` directory, as shown in the following example for the process record:



3. Check the modified code into the code repository. The commit window displays all the changed files.



Note

Note The commit window may not show all the changed files in the **SM-APPS** change list, and this problem occurs when the tables you have modified aren't present in the `svc_include.props` property file in your `<Server Dir>/RUN` directory. To solve the problem, manually add the names of the tables to this file.

In this way, SVN retains a clear history of any changes made to a specific application.

Related topics

Implementing version control for your tailoring
Version control best practice
Parameters to import and export files Applications data file represented in XML

1.7.2. Git version control process

We provide the information contained in this section regarding Git configurations to our customers and partners as a courtesy. This section doesn't replace a Git reference, and you may need to conduct additional research regarding Git technology by consulting with sources outside of this section. Our company hereby disclaims all liability associated with the use and accuracy of this information. As Git technology evolves, we may or may not update this reference.

Prior to proceeding with the Git configuration tasks, make sure you have prepared the following resources:

- Possess the basic Git knowledge and hands-on skills to install Git Bash, set up the Git server, create a repository, folk/clone the repository to a local drive, pull/commit/push code changes, and so on.
  For more instructions, see https://help.Github.com/en.
- Install Git Bash and the version must be 2.10.x or above.

> **Note**
> Note
> During the Git Bash setup process, make sure to select the **Use Git and optional Unix tools from the Windows Command Prompt** option on the **Adjusting your PATH environment** screen.

Related topics

Prepare Git
Prepare to use Feature Tracker
Using Feature Tracker
Feature Tracker use case

1.7.2.1. Prepare Git

After you perform a system upgrade, you need to follow the instructions in this topic and prepare Git again. Due to the data table structure changes in the upgrade process, the original feature records and Watchdog records become invalid. These records are for reference only and you can no longer use these records to commit and deploy codes. Visit the Git server if you need to view the original submission history.

For more information about the feature record, the release record, and Watchdog, see the Using Feature Tracker topic.

Prepare a code repository

User role: System administrator

The first step of your version control process is that a system administrator creates a code repository on a Git server. This enables you to use the repository to track all changes made to applications objects as you create, develop, or extend the Service Manager applications.

> **Note**
> Note
> Make sure that your Service Manager server is installed on the computer that has the Git client installed.

To leverage Git version control, follow these steps:

1. Set up the Git server and create the repository folder for the Service Manager applications data.
2. Set up the Git client.
3. Create a folder in your local drive to check the Git code repository in and out. For example, c:\Git\data.
4. Use the SSH URL or the HTTPS protocol to clone the empty repository to your local folder. See the following command as an example:
   Git clone Git@<Github.com>:<organization>/<repo>.Git

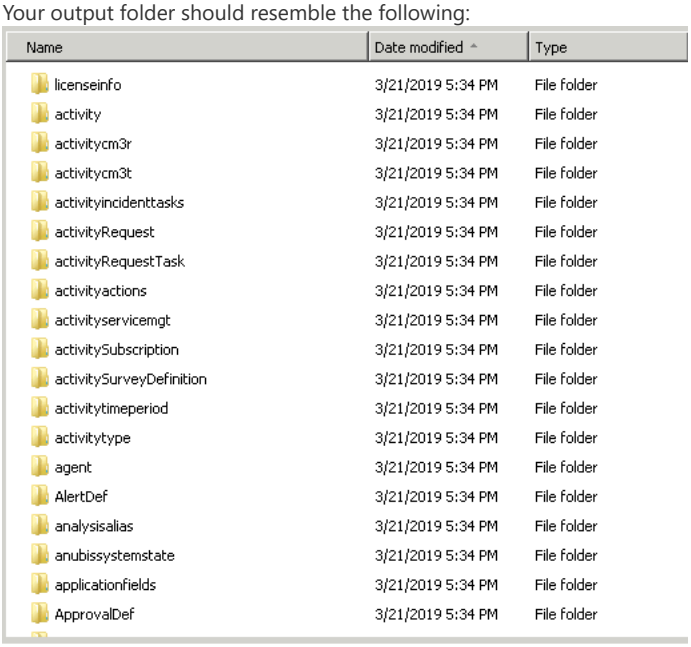
   > **Note**
   > Note
   > You need to configure Credential Storage for Git when using the HTTPS protocol. As a result, you don't need to repeatedly provide your username and password for every Git operation such as pull or push. For more instructions about Credential Storage, see https://Git-scm.com/book/en/v2/Git-Tools-Credential-Storage.

5. Configure to use the repository. To do this, add the following two new lines to the sm.ini file in the <Server Dir>/RUN/ directory:
   `svc_rootdir:` *<Your check-out Service Manager applications path, in this example, C:\Git\data>*
   `svc_mode:99`
   Restart the Service Manager server to enable the added svc commands.

6. In the <Server Dir>/RUN/ directory, run the **sm -svc_export** command to export all Service Manager applications data as XML files to the svc_rootdir folder. You may also refer to Parameters to import and export files for more parameters to export some of the applications data.

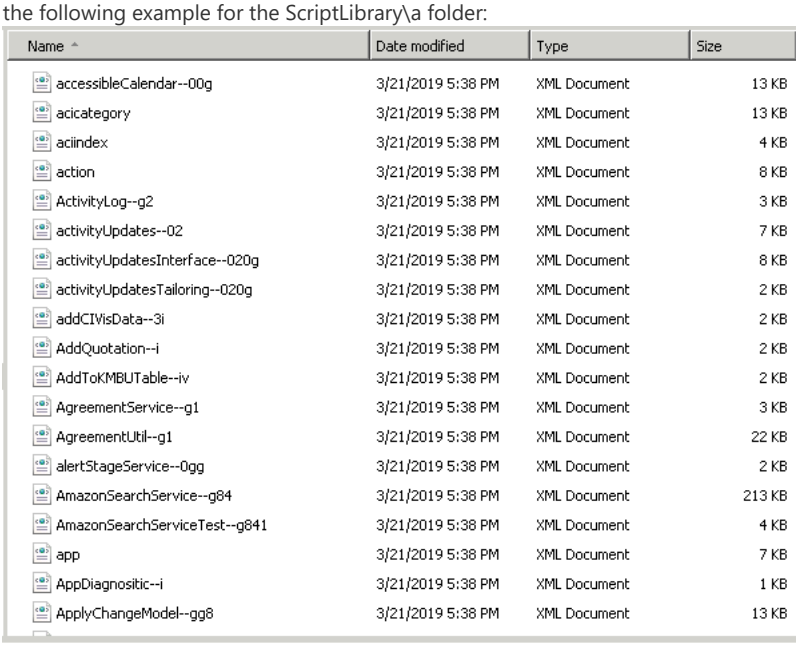
> **Note**
> Note
> Before you export the applications data records, be sure to stop the Service Manager server.

Your output folder should resemble the following:



Name	Date modified	Type
licenseinfo	3/21/2019 5:34 PM	File folder
activity	3/21/2019 5:34 PM	File folder
activitycm3r	3/21/2019 5:34 PM	File folder
activitycm3t	3/21/2019 5:34 PM	File folder
activityincidenttasks	3/21/2019 5:34 PM	File folder
activityRequest	3/21/2019 5:34 PM	File folder
activityRequestTask	3/21/2019 5:34 PM	File folder
activityactions	3/21/2019 5:34 PM	File folder
activityservicemgt	3/21/2019 5:34 PM	File folder
activitySubscription	3/21/2019 5:34 PM	File folder
activitySurveyDefinition	3/21/2019 5:34 PM	File folder
activitytimeperiod	3/21/2019 5:34 PM	File folder
activitytype	3/21/2019 5:34 PM	File folder
agent	3/21/2019 5:34 PM	File folder
AlertDef	3/21/2019 5:34 PM	File folder
analysisalias	3/21/2019 5:34 PM	File folder
anubissystemstate	3/21/2019 5:34 PM	File folder
applicationfields	3/21/2019 5:34 PM	File folder
ApprovalDef	3/21/2019 5:34 PM	File folder

The XML files are exported to subfolders that are alphabetically-ordered by the unique key names of the applications, as shown in the following example for the ScriptLibrary\a folder:



Name	Date modified	Type	Size
accessibleCalendar--00g	3/21/2019 5:38 PM	XML Document	13 KB
acicategory	3/21/2019 5:38 PM	XML Document	13 KB
aciindex	3/21/2019 5:38 PM	XML Document	4 KB
action	3/21/2019 5:38 PM	XML Document	8 KB
ActivityLog--g2	3/21/2019 5:38 PM	XML Document	3 KB
activityUpdates--02	3/21/2019 5:38 PM	XML Document	7 KB
activityUpdatesInterface--020g	3/21/2019 5:38 PM	XML Document	8 KB
activityUpdatesTailoring--020g	3/21/2019 5:38 PM	XML Document	2 KB
addCIVisData--3i	3/21/2019 5:38 PM	XML Document	2 KB
AddQuotation--i	3/21/2019 5:38 PM	XML Document	2 KB
AddToKMBUTable--iv	3/21/2019 5:38 PM	XML Document	2 KB
AgreementService--g1	3/21/2019 5:38 PM	XML Document	3 KB
AgreementUtil--g1	3/21/2019 5:38 PM	XML Document	22 KB
alertStageService--0gg	3/21/2019 5:38 PM	XML Document	2 KB
AmazonSearchService--g84	3/21/2019 5:38 PM	XML Document	213 KB
AmazonSearchServiceTest--g841	3/21/2019 5:38 PM	XML Document	4 KB
app	3/21/2019 5:38 PM	XML Document	7 KB
AppDiagnositic--i	3/21/2019 5:38 PM	XML Document	1 KB
ApplyChangeModel--gg8	3/21/2019 5:38 PM	XML Document	13 KB

7. Commit and push all the files to the Git repository.

Note

Note

You should exclude any business-related records when you commit the files, as these types of records require a great deal of capacity and decrease efficiency. These types of records include the following, but your organization's records may differ:

Object	Table
Interaction	incidents
Incident	probsummary
Problem	rootcause
Problem Task	rootcausetask
Known Error	knownerror
Known Error Task	knownerrortask
Change	cm3r
Change Task	cm3t

Prepare a development environment

User role: Developer

Note

Note

Environment preparation is only required when a developer starts applications development for the first time or starts to use a new computer.

1. Set up your Git client.
2. Create a folder in your local drive to check the Git code repository in and out. For example, you create `c:\Git\data`.
3. Set up Service Manager by adding the following two new lines to the sm.ini file in the <Server Dir>/RUN/ directory:
   `svc_rootdir:` *<Your check-out Service Manager applications path, in this example, C:\Git\data>*
   `svc_mode:99`
   Restart the Service Manager server to enable the added svc commands.
4. Use the SSH URL or the HTTPS protocol to clone the applications data files from the Git repository to your local folder. See the following command as an example:
   Git clone Git@<Github.com>:<organization>/<repo>.Git

   Note

   Note

   You need to configure Credential Storage for Git when using the HTTPS protocol. As a result, you don't need to repeatedly provide your username and password for every Git operation such as pull or push. For more instructions about Credential Storage, see https://Git-scm.com/book/en/v2/Git-Tools-Credential-Storage.
5. Import the applications data in the XML files by copying the production Service Manager database to the development system.

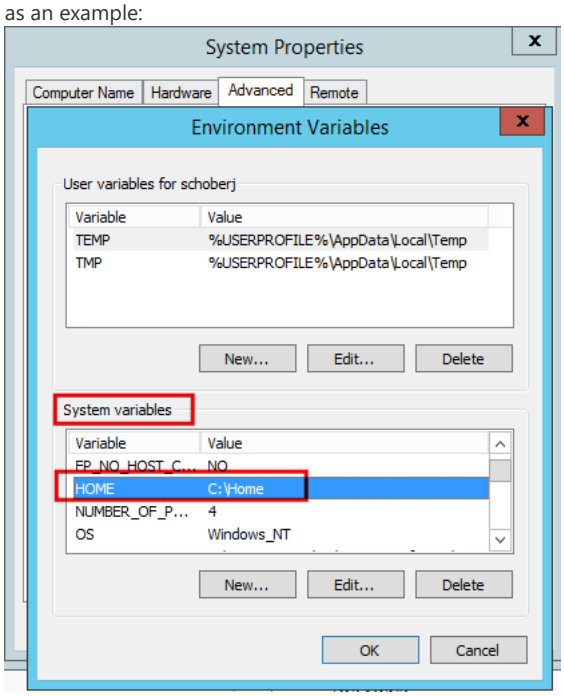
1.7.2.2. Prepare Service Manager

In Windows, you can either start Service Manager from the command line or from Windows Services.

- **Command line (Recommended)**
  We recommend to start Service Manager from the command line. In this way, Service Manager uses the GIT configurations the current login user set up for GIT servers. You don't need perform further GIT configurations.
- **Windows Services**
  When you start Service Manager from Windows Services, Service Manager searches for GIT configurations for the background service user, generally the Local System account. You need to configure the GIT environment for the background service user so that you can commit changes using Feature Tracker.

To configure the GIT environment for the background service user, follow these steps:

1. Create a home folder. For example, C:\Home.
2. Add a Windows System Environment `HOME`, with the value set as the full path of the HOME folder. See the following screenshot as an example:



3. RUN the **git config** command again to set up GIT configurations such as "user.email" and "user.name" in the HOME folder.

> **Note**
> Note
> These two variables are required by GIT to do commit operations.

4. Copy the .ssh folder to the HOME folder in both the development and deployment environments.

> **Tip**
> Tip
> The .ssh folder contains the id_rsa, id_rsa_pub, and known hosts files. The known hosts file is generated once you connect the remote GIT repository successfully.

For more information, see Connecting to GitHub with SSH.

1.7.2.3. Using Feature Tracker

Important: To perform configurations and use Feature Tracker, an operator must have the following capability words in the user profile record:

- FeatureTrackerCommit
- FeatureTrackerReleaseManager

Configure settings to track features

To configure global settings for Feature Tracker, follow these steps:

1. Go to the System Navigator and then click **Tailoring** > **Feature Tracker** > **Global Settings**.

2. Modify the settings as described in the table below:

Field	Description
Release Repository URL	Specify the URL of the remote repository for release.
Temp Folder	Specify the folder to store temporary files.
Remote SM End Point	Specify the URL of the Service Manager server with HTTP protocol and port number. For example, `http://<IP address>:13080`. Using the endpoint set here, Feature Tracker communicates with the target Service Manager server on which the release is to be deployed.
Remote SM User Name	Specify a user name of the target Service Manager on which the release is to be deployed.
Remote SM Password	Specify the login password of the user that's set in **Remote SM User Name**.
Watched Tables	Display the table names for which the system identifies changed objects. You can define the tables in either of the following methods: ○ Access <*Service Manager installation directory*>\Server\RUN\sm.ini, add the *svc_files* parameter, and then add a list of tables. ○ Access <*Service Manager installation directory*>\Server\RUN\svc_include.props, and then add a list of tables. **Note:** The tables defined in the *svc_files* parameter take precedence over those defined in svc_include.props. If you defined tables in both places, the Watched Tables field lists those defined in *svc_files* only. **Note:** Feature Tracker can't track the tables without the sysmodtime, sysmoduser, and sysmodcount fields. For example, the `codes` table doesn't have these three fields by default and Feature Tracker can't track this table. To track the `codes` table, you can access Database Manager and manually add the sysmodtime, sysmoduser, and sysmodcount fields.

3. Click **Save**.

Create a feature or release record

To create a feature or release record, follow these steps:

1. From the System Navigator, click **Tailoring** > **Feature Tracker** > **Search Feature Tracker**. The Search Feature Tracker Records page opens.
2. Click **New** on the toolbar.
3. In the **Type** drop-down list, select **Feature** or **Release** as needed. If you select **Release**, fill a release number in the **Release Number** field.

   Note
   Note
   If you are working with Feature Tracker for the first time, you must create a release first.

4. Type a title in the **Title** field, and then click **Save**.

5. Fill in the fields as shown below:

Field	Description
Internal ID	The unique ID of the feature or release. The system populates this field automatically when creating a new feature or release.
External ID	Specify the external ID for this feature in third-party integration. For example, ALM or Octane.
Type	The type of this Feature Tracker record.
Area	Specify the module that the feature or release relates to. For example, Incident Management.
Development Owner	Specify the development owner of the feature or release.
Development Team	Specify the development team responsible for the feature or release.
Status	Specify one of the following statuses the feature or release is in: ○ New ○ Work in progress ○ Testing ○ Done ○ Cancelled ○ Ready to release
Release	This field relates to the feature type of a record. Specify which release the feature belongs to.
Release Date	Specify the date of the release.
Release Manager	Specify the release manager that's responsible for the release.
Test Owner	Specify the test owner of the feature or release.
Test Team	Specify the test team responsible for the feature or release.

6. Click **Save**.

View detailed information of a feature record

To view detailed information of a feature record, follow these steps:

1. From the System Navigator, click **Tailoring** > **Feature Tracker** > **Search Feature Tracker**. The Search Feature Tracker Records page opens.
2. Search for the desired feature record.
3. Open the feature record by clicking the ID.
4. You will find detailed information about the feature regarding internal/external ID, type, area, current phase, status, and related release.

You can also click different tabs on the record page to see more detailed information as shown below:

Tab	Description
General	This section contains the following two subsections: ○ **Details** In this subsection, you can view the information about the feature title, development owner, development team, test owner, and test team. ○ **Comments and Instructions** This subsection includes comments and instructions relate to the feature. For example, manual steps before or after feature deployment.
Objects	This section contains the following two subsections that show different information about changed objects. ○ **All modified objects** This subsection displays a list of all modified objects with information about the ID, date, flag, table, user, tag, feature, and release. You can filter out objects in the list by selecting criteria from the drop-down boxes of **Date > =**, **Date < =**, **Table**, **User**, **Feature**, and **Release**. Click the **Filter Objects** icon to apply the filter criteria, or the **Clear Filter Criteria** icon to clear. To update the list, click **Refresh Objects**. To have Feature Tracker update the list automatically, see *Update the changed object list automatically*. To link objects with the feature, select the objects of your interest from the list and then click **Link selected objects with the feature**. ○ **Related objects** This subsection displays a list of modified objects which are related to the feature, with information about the ID, Date, Flag, Table, User, Tag, Feature, Release, and Status. Click **Unlink selected objects from the feature** to remove the relationship between a modified object and the feature. To commit the related objects, click **Commit All Related Objects**.
Activities	On this tab, you can enter new activity updates, view Journal Updates, and view history activities for the record. **Journal Updates** tab ○ **New Update Type**: Specify or categorize the activity update. If you leave this field blank, the system sets this value to **Developer update** by default. ○ **New Update**: Enter notes to explain and describe updates made for the record. ○ **Journal Updates**: View text entered in the New Update field along with the operator name and a timestamp for the update. This field displays information when Journaling is enabled. **Activity Logs** tab View the record's history activities, including information about date/time, type, operator, and description. You can use the **Activity Type** and **User** filters to sort the history activities.

Tab	Description
Workflow	This section displays a figure of feature workflow. It also indicates the current phase that the feature is in, and traces the phase transition history.

View detailed information of a release record

To view detailed information of a release record, follow these steps:

1. From the System Navigator, click **Tailoring** > **Feature Tracker** > **Search Feature Tracker**. The Search Feature Tracker Records page opens.
2. Search for the desired release record.
3. Open the release record by clicking the ID.
4. You will find detailed information about the release regarding internal/external ID, type, current phase/status, release manager, and release date.
   You can also click different tabs on the record page to see more detailed information as shown below:

Section	Description
General	This tab displays the release title and description.
Related Features	This tab displays a list of features related to the release, with brief information about ID, external ID, owner, current phase, status, and title. To unlink a feature from the release, open the feature record and then unlink it by selecting another release or **Unknown** from the **Related Release** drop-down box. In addition, you can build a release or schedule a release by using the **Build Release** and **Schedule Release** buttons in this section.
Activities	On this tab, you can enter new activity updates, view Journal Updates, and view history activities for the record. **Journal Updates** tab ○ **New Update Type**: Specify or categorize the activity update. If you leave this field blank, the system sets this value to **Developer update** by default. ○ **New Update**: Enter notes to explain and describe updates made for the record. ○ **Journal Updates**: View text entered in the New Update field along with the operator name and a timestamp for the update. This field displays information when Journaling is enabled. **Activity Logs** tab View the record's history activities, including information about date/time, type, operator, and description. You can use the **Activity Type** and **User** filters to sort the history activities.
Workflow	This tab displays a figure for the release workflow. It also indicates the current phase that the release is in and traces the phase transition history.
Documentation	This tab allows you to create a document that lists the features and the modified objects for the release. Click **Create Documentation** and then add some descriptions in the Documentation column for the related features. After you build a release, this documentation is saved as an HTML file in the specific release folder. For example, if your temporary files are saved in C:\<*temp folder*>, you can find the C:\<*temp folder*>\<*release name*>\<*release name*>.html document after you build a release.

Update the changed object list automatically

To ensure all modified objects are properly listed and automatically updated in Feature Tracker, follow these steps.

1. Make sure you have started the **problem** processor in the background scheduler.
2. From the System Navigator, click **Tailoring** > **Feature Tracker** > **Search Feature Tracker**. The Search Feature Tracker Records page opens.
3. Search for any feature record and then open it.
4. From the record details toolbar, click **More Actions** > **Watchdog Schedule**. The system opens the **Watchdog Schedule** page.
5. Click **Start Schedule** to start the schedule. The system updates the changed object list related to this feature automatically in an interval of 60 minutes.
   To stop the schedule, click **Stop Schedule**.

Commit changes using Feature Tracker

To commit changes using Feature Tracker, follow these steps:

1. Open the feature record of your interest.
2. Go to the **Related Objects** tab.
3. (Optional) From the **All modified objects** section, select changed objects as needed and then click **Link selected objects with the feature**.
4. From the **Related objects** section, click **Commit all related objects**.
   After this process completes successfully, the system displays a message that may resemble the following:
   `Commit <number> item(s) ...`



Note

Note

Feature Tracker dose doesn't provide any revert code functions. To revert code changes, you must manually modify the code back as required and then commit the object to the source control repository.

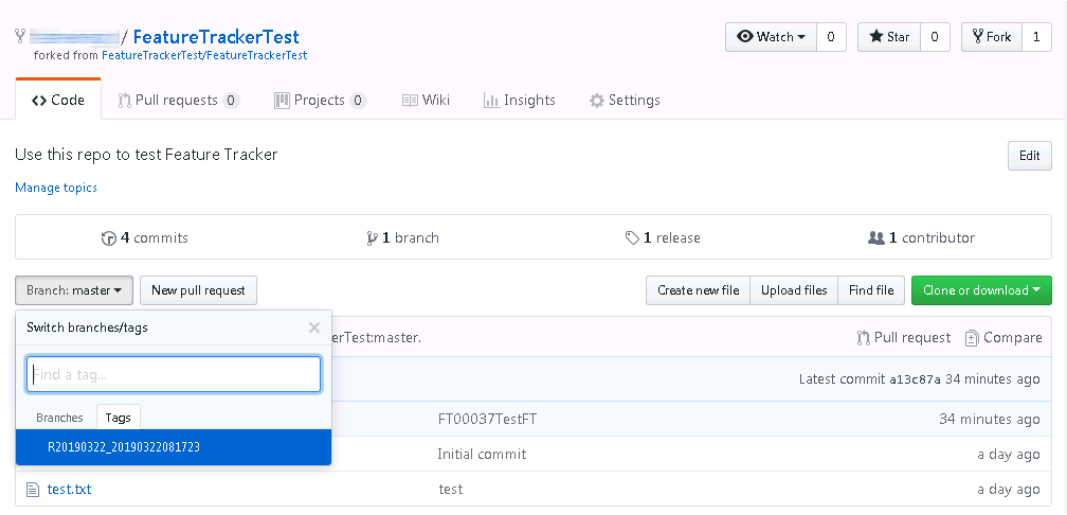
Build releases using Feature Tracker

Feature Tracker allows you to build releases in real-time or a scheduled time.

To build a release using Feature Tracker, follow these steps:

1. Open the related feature record, and then update the status to **Ready to release** or **Done**.
2. Open the release record of your interest, and then update the status to **Ready to release** or **Done**.
3. Click the **Related Features** tab.
4. Click **Build Release** to generate a release immediately or **Schedule Release** to generate a release on a scheduled time.
   Build Release
   If you click **Build Release**, the system displays the following messages:
   `prepare release files...`
   `generate documentation...`
   `Tag "R<Release Number>_<Timestamp>"created on git@<github.com>:<orgnization>/<repo>.git`

The last message indicates that a new release is generated with the tag on your remote GIT repository. You can check the release build by browsing to the **Tag** section of the remote repository that's defined for **Release Repository URL** on the Globe Settings page. See the following screenshot as an example:



Schedule Release

When you click **Schedule Release**, the Schedule Release Build wizard opens. Follow the instructions on the screen to finish the schedule settings including start time, end time, and repeat interval.

> **Tip**
>
> Tip
>
> Make sure you have started the **problem** processor in the background scheduler.

Deploy releases using Feature Tracker

Feature Tracker allows you to deploy a release on a remote Service Manager. However, in case you have complex database dictionary records updates, we recommend you manually perform these updates on the remote Service Manager instead of using Feature Tracker.

> **Caution**
>
> Caution
>
> You may need to deploy complex database dictionary records updates during a maintenance window as users may get affected and need a restart.

To deploy a release using Feature Tracker, follow these steps:

1. Sign in to your development or staging environment of Service Manager where the final release locates. Then, open the respective release record.
2. Go to the **Related Features** tab.
3. Click **Deploy Release** to deploy the release.
4. (Optional) In the remote Service Manager system, manually delete the files that are deleted from the development Service Manager system using the source control code repository. The deletion of files from a source control repository isn't applicable to Service Manager.

You can also deploy your releases with external tools, outside of Service Manager. To do this, follow these steps:

> **Caution**
>
> Caution
>
> The following procedure overwrites the source codes on the target Service Manager server. Before proceeding, you must make sure that the target Service Manager server has the same code as the development environment.

1. Clone changed files from the source control repository to the Service Manager server where you want to deploy the release, using the command shown below:
   git clone --depth 1 -b <tag> <repo> <temp-folder>

2. Go to the <Server Dir>/RUN/ directory and then run the following command to import the changed files to the remote Service Manager:
   sm -svc_import -svc_rootdir:<directory path> -svc_mode:99 -svc_cleanbuild:1
   In case you have database dictionary records updates, run the following command to import the changed files to the remote Service Manager:
   sm -svc_import -svc_rootdir:<directory path> -svc_mode:99 -svc_cleanbuild:1 -svc_updatedbdict
   In this command, `<directory path>` refers to the path of the directory where you clone the changed files in *step 1*.



Note

Note

If the content you deploy contains either the **displayoption** or **displayscreen** records, you need to manually run the `us.compile.screens` RAD function to refresh the **displaycache** records.

1.7.2.4. Feature Tracker use case

This topic describes a typical use case to leverage Feature Tracker for version control of Service Manager customization:

1. Prepare Git.
   1. The system administrator exports all the applications records from Service Manager and check in the exported files to a Git server to create a baseline repository. For details, see the *Prepare a code repository* section in the Prepare Git topic.
   2. Developers check out the latest version of the applications records and import them into a local Service Manager development environment. For details, see the *Prepare a development environment* section in the Prepare Git topic.
2. Prepare Service Manager.
3. Before performing configurations and using Feature Tracker, make sure an operator have the **FeatureTrackerCommit** and **FeatureTrackerReleaseManager** capability words in the user profile record.
4. Configure Feature Tracker settings. See the *Configure settings to track features* section in the Using Feature Tracker topic.
5. Create a release, and then create a feature by using Feature Tracker. For details, see the *Create a feature or release record* section in the Using Feature Tracker topic.
6. Open the feature record, and then relate the feature to a release by selecting the release in the **Related Release** drop-down list.
7. Developers customize application data records of Service Manager.
8. Open the feature record and then click the **Objects** tab to view the modified objects. Click the **Refresh Objects** button to refresh the object list. Select the objects of your interest from the list and then click **Link selected objects with the feature**. For details, see the *View detailed information of a feature record* section in the Using Feature Tracker topic.
9. Committing customization changes to your source control repositories by using Feature Tracker. For details, see the *Commit changes using Feature Tracker* section in the Using Feature Tracker topic.
   Operators with commit right to Feature Tracker can check in or discard the changes.
10. Developers check the commit in the Git repository and then merge it to the main repository.
11. When the feature is ready to release, the release manager can go to the related Release record and then build a release by using Feature Tracker. For details, see the *Build releases using Feature Tracker* section in the Using Feature Tracker topic.
12. The release manager deploys the desired release on the test Service Manager system by using Feature Tracker. For details, see the *Deploy releases using Feature Tracker* section in the Using Feature Tracker topic.
13. The release manager deploys the desired release on the production Service Manager system by using Feature Tracker.

1.7.3. Version control best practice

Create an operator record for each developer

Each developer should have their own operator record to log in to Service Manager rather than using the default System Administrator operator record. This helps to track code changes, because the value of the **sysmoduser** field of the applicatios object is set as the currently logged-in user.

Streamline the code repository versions

Prior to any development, export the Service Manager OOB database as the initial repository for future comparison and analysis. Then, export your organization's current, customized database as the second-version repository.



Note

Note When you upgrade Service Manager to a later version and resolve all conflicts, you must export the complete upgraded database again as the new code repository.

Turn off this function when it's not needed

The source version control function decreases system performance. Therefore, you should comment out the source version control configuration lines in the `sm.ini` file when it's not needed.

Synchronize number/counters records after merging updated code to another environment

When you merge updated code to another environment, sometimes the maximum ID number in number/counters records may not synchronize with the related records. Run the following script to synchronize data:

```
run ScriptLibrary
lib.BaseUtil.function synchronizeCountersAndNumbers()
```

Related topics

Implementing version control for your tailoring
SVN version control process
Applications data file represented in XML

1.7.4. Applications data file represented in XML

Exporting applications from Service Manager converts the applications records to data files represented in XML, with the applications object name in the `<recordset table>` element and the unique id in the `<record id>` element. The following code is an example of the `Process` record:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<recordset table="Process">
<record id="process=&amp;quot;cc.first.log&amp;quot;" recordid="cc.first.log">
<process type="string">cc.first.log</process>
<pre.expressions sctype="array">
<pre.expressions sctype="operator">$L.oldinc=$L.lastinc</pre.expressions>
<pre.expressions sctype="operator">if ($L.action="lastincident") then ($L.golast="true";$L.action="log") else
($L.golast="false")</pre.expressions>
<pre.expressions sctype="operator">$L.continue=true</pre.expressions>
<pre.expressions sctype="operator">if ($L.action="logcatalog" and null(title in $L.file)) then (title in $L.file=1
in description in $L.file)</pre.expressions>
</pre.expressions>
<post.expressions sctype="array">
<post.expressions sctype="operator">if ($G.ess and $L.continue) then ($L.exit.when.done=true)</post.expressions>
</post.expressions>
<rad sctype="array">
<rad sctype="structure">
<application type="string">sc.get.sla</application>
<names sctype="array">
<names sctype="string">file</names>
<names sctype="string">number1</names>
<names sctype="string">numbers</names>
</names>
<values sctype="array">
<values sctype="string">$L.file</values>
<values sctype="string">contract.id in $L.file</values>
<values sctype="string">agreement.ids in $L.file</values>
</values>
<rad.condition type="operator">enable in $G.sla.environment and ($L.action="complete" or $L.action="log" or
$L.action="logcatalog") and $L.mode="add"</rad.condition>
<pre.rad.expressions sctype="array">
<pre.rad.expressions sctype="operator">if ($affected.ci~="Other – See Description") then (affected.item in
$L.file=nullsub(affected.item in $L.file, $affected.ci))</pre.rad.expressions>
</pre.rad.expressions>
<post.rad.expressions sctype="string"/>
</rad>
<rad sctype="structure">
<application type="string">sla.check.sh.wrapper</application>
<names sctype="array">
<names sctype="string">number1</names>
<names sctype="string">text</names>
</names>
<values sctype="array">
<values sctype="string">1 in agreement.ids in $L.file</values>
<values sctype="string">$L.exit</values>
</values>
<rad.condition type="operator">not  (null(callback.contact in $L.file)) and not  (null(agreement.ids in $L.file))
and check.sh in $G.sla.environment and (nullsub($L.overridden, false)=false or not  (same($L.or.id, 1 in
agreement.ids in $L.file)))</rad.condition>
<pre.rad.expressions sctype="array">
<pre.rad.expressions sctype="operator">if (null(agreement.ids in $L.file) and not  (null(agreement.id in $L.file)))
then (1 in agreement.ids in $L.file=agreement.id in $L.file)</pre.rad.expressions>
</pre.rad.expressions>
<post.rad.expressions sctype="array">
<post.rad.expressions sctype="operator">if ($L.exit="normal") then ($L.overridden=true;$L.or.id=1 in agreement.ids
in $L.file)</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.exit="blocked") then ($L.string=scmsg(292, "pm",
{nullsub(callback.contact in $L.file, "NULL"), nullsub(1 in agreement.ids in $L.file,
"NULL")});$L.overridden=false;$L.or.id=NULL;$L.action="complete";resolution in $L.file=resolution in $L.file+
{$L.string};resolution.code in $L.file="Out of Scope")</post.rad.expressions>
</post.rad.expressions>
```

```
</rad>
<rad sctype="structure">
<application type="string">getnumb</application>
<names sctype="array">
<names sctype="string">name</names>
<names sctype="string">index</names>
<names sctype="string">text</names>
</names>
<values sctype="array">
<values sctype="string">number.record in $L.object</values>
<values sctype="string">incident.id in $L.file</values>
<values sctype="string">"string"</values>
</values>
<rad.condition type="operator">null(incident.id in $L.file)</rad.condition>
<pre.rad.expressions sctype="array">
<pre.rad.expressions sctype="operator">if ($L.action={"log", "logexit"} and open in $L.file~="Closed") then
(first.call in $L.file=false)</pre.rad.expressions>
<pre.rad.expressions sctype="operator">if ($L.action={"associate", "incident.change", "incident.request",
"createchange", "createrequest", "create"}) then (first.call in $L.file=false;$L.notify.name="SM Add")
</pre.rad.expressions>
<pre.rad.expressions sctype="operator">if ($L.action="complete") then ($L.oldstatus=open in $L.file;open in
$L.file="Closed";$L.notify.name="SM Close")</pre.rad.expressions>
<pre.rad.expressions sctype="operator">if ($L.action={"log", "logexit", "save", "newcall"}) then (open in
$L.file=nullsub(open in $L.file, "Open - Callback");if ($L.bg=false) then ($L.notify.name="SM Add"))
</pre.rad.expressions>
</pre.rad.expressions>
<post.rad.expressions sctype="string"/>
</rad>
<rad sctype="structure">
<application type="string">us.consume.wrapper</application>
<names sctype="array">
<names sctype="string">index</names>
<names sctype="string">name</names>
<names sctype="string">boolean1</names>
<names sctype="string">text</names>
</names>
<values sctype="array">
<values sctype="string">contract.id in $L.file</values>
<values sctype="string">"i"</values>
<values sctype="string">$L.bg</values>
<values sctype="string">$L.exit</values>
</values>
<rad.condition type="operator">enable in $G.cm.control and not  (nullsub(contract.consumed in $L.file, false)) and
not  (null(contract.id in $L.file))</rad.condition>
<pre.rad.expressions sctype="array">
<pre.rad.expressions sctype="operator">if (null($G.src) or $G.src=false) then ($L.exit="normal")
</pre.rad.expressions>
</pre.rad.expressions>
<post.rad.expressions sctype="array">
<post.rad.expressions sctype="operator">if ($L.exit="badval") then if ($L.action="complete") then (open in
$L.file=$L.oldstatus)</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.exit="normal") then (contract.consumed in $L.file=true)
</post.rad.expressions>
</post.rad.expressions>
</rad>
<rad sctype="structure">
<application type="string">cc.save</application>
<names sctype="array">
<names sctype="string">file</names>
<names sctype="string">name</names>
<names sctype="string">record</names>
<names sctype="string">boolean1</names>
<names sctype="string">text</names>
</names>
<values sctype="array">
```

```
<values sctype="string">$L.file</values>
<values sctype="string">$L.init.action</values>
<values sctype="string">$L.fc</values>
<values sctype="string">$L.bg</values>
<values sctype="string">$L.exit</values>
</values>
<rad.condition type="operator">$L.exit="normal"</rad.condition>
<pre.rad.expressions sctype="array">
<pre.rad.expressions sctype="operator">$L.handle.save=$L.handle.start</pre.rad.expressions>
<pre.rad.expressions sctype="operator">if ($L.handle.start~=0) then (handle.time in $L.file=nullsub(handle.time in
$L.file, '00:00:00');handle.time in $L.file=(handle.time in $L.file+tod()) - $L.handle.start;$L.handle.start=0)
</pre.rad.expressions>
<pre.rad.expressions sctype="operator">$L.init.action=nullsub($L.init.action, "add")</pre.rad.expressions>
<pre.rad.expressions sctype="operator">$L.pri.calc=jscall("PriorityCalc.getPriorityCalc")</pre.rad.expressions>
</pre.rad.expressions>
<post.rad.expressions sctype="array">
<post.rad.expressions sctype="operator">if ($L.exit="normal") then if ($L.golast="false") then
($L.lastinc=incident.id in $L.file)</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.exit="badval") then
($L.handle.start=$L.handle.save;$L.continue=false;if ($L.action="complete") then (open in $L.file=$L.oldstatus))
</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.exit="normal" and $G.ess) then ($L.file.save=$L.file)
</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.exit="normal" and not  ($G.ess) and blank.call in
$G.sm.global.environment) then ($L.format=nullsub(edit.format in $G.sm.environment, "SD.update.interaction"))
</post.rad.expressions>
</post.rad.expressions>
</rad>
<rad sctype="structure">
<application type="string">us.save.relation</application>
<names sctype="array">
<names sctype="string">name</names>
<names sctype="string">prompt</names>
<names sctype="string">query</names>
<names sctype="string">string1</names>
</names>
<values sctype="array">
<values sctype="string">$L.related.id</values>
<values sctype="string">$L.related.filename</values>
<values sctype="string">$L.depend.id</values>
<values sctype="string">$L.depend.filename</values>
</values>
<rad.condition type="operator">not  (null($L.related.id)) and not  (null($L.related.filename)) and
$L.continue</rad.condition>
<pre.rad.expressions sctype="array">
<pre.rad.expressions sctype="operator">$L.depend.filename="incidents"</pre.rad.expressions>
<pre.rad.expressions sctype="operator">$L.depend.id=str(incident.id in $L.file)</pre.rad.expressions>
</pre.rad.expressions>
<post.rad.expressions sctype="array">
<post.rad.expressions sctype="operator">if $L.continue then if (not  (null($L.related.id)) and not
(null($L.related.filename))) then ($L.exit="normal";$L.exit.when.done=true)</post.rad.expressions>
<post.rad.expressions sctype="operator">if $L.continue then ($L.related.id=NULL;$L.related.filename=NULL)
</post.rad.expressions>
</post.rad.expressions>
</rad>
<rad sctype="structure">
<application type="string">us.notify</application>
<names sctype="array">
<names sctype="string">name</names>
<names sctype="string">record</names>
<names sctype="string">second.file</names>
</names>
<values sctype="array">
<values sctype="string">$L.notify.name</values>
<values sctype="string">$L.file</values>
```

```
<values sctype="string">$L.file.save</values>
</values>
<rad.condition type="operator">$L.exit="normal"</rad.condition>
<pre.rad.expressions sctype="string"/>
<post.rad.expressions sctype="array">
<post.rad.expressions sctype="operator">if ($L.exit="normal" and not  ($L.exit.when.done)) then if blank.call in
$G.sm.global.environment then ($L.exit="restart";$L.void=rtecall("resetnotebook", $L.rc)) else
($L.exit.when.done=true)</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.continue and $L.action="save" and not  ($L.exit.when.done)) then
($L.exit="";$L.mode="browse")</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.action="addcloseme") then ($L.exit="";$L.mode="browse")
</post.rad.expressions>
<post.rad.expressions sctype="operator">if ($L.continue and $L.pmtapi=true) then ($L.exit="exit")
</post.rad.expressions>
</post.rad.expressions>
</rad>
<rad sctype="structure">
<application type="string">cc.set.approval</application>
<names sctype="array">
<names sctype="string">file</names>
<names sctype="string">record</names>
</names>
<values sctype="array">
<values sctype="string">$L.file</values>
<values sctype="string">$L.object</values>
</values>
<rad.condition type="boolean">false</rad.condition>
<pre.rad.expressions sctype="string"/>
<post.rad.expressions sctype="string"/>
</rad>
</rad>
<next.process sctype="array">
<next.process sctype="string">sm.close</next.process>
<next.process sctype="string">cc.clone.relation</next.process>
<next.process sctype="string">cc.screlate</next.process>
<next.process sctype="string">sm.save</next.process>
</next.process>
<process.condition sctype="array">
<process.condition sctype="operator">$L.continue and $L.action="addcloseme"</process.condition>
<process.condition sctype="operator">$L.continue and ($L.action="save" or $L.mode="addclone") and not  ($G.ess)
</process.condition>
<process.condition sctype="operator">$L.continue and $L.action="complete"</process.condition>
<process.condition sctype="operator">$G.ess and $L.mode="addclone"</process.condition>
</process.condition>
<run.standard NullValue="1" type="boolean"/>
<sysmodcount type="decimal">9</sysmodcount>
<sysmoduser type="string">zhangqi</sysmoduser>
<sysmodtime type="dateTime">11/22/13 00:48:44</sysmodtime>
<save.cursor.position NullValue="1" type="boolean"/>
<run.in.window NullValue="1" type="boolean"/>
<window.name NullValue="1" type="string"/>
<javascript.pre NullValue="1" type="string"/>
<javascript.post type="string">if (vars.$G_src &amp;amp;&amp;amp; vars.$L_exit=="blocked")
{
system.functions.msg(vars.$L_string, 3);
}</javascript.post>
<baseMethod NullValue="1" type="string"/>
</record>
</recordset>
```

Related topics

Implementing version control for your tailoring
Version control best practice
SVN version control process

1.7.5. Parameters to import and export files

The version control command line tool provides you the following parameters to import and export specific files. In Service Manager, a file is equivalent to a Relational Database Management System (RDBMS) table. For more information, see File.

Parameters to export files

To export Service Manager files, you can run the *sm -svc_export* command with the following parameters, or add the following parameters in the *sm.ini* file.

Parameter	Description
svc_rootdir	Specifies the folder to which you export the Service Manager applications data as XML files. To run the **svc_export** command, you must specify a valid directory with the read/write permission.
svc_files	Specifies a list of files to export. Use commas to separate multiple files (no space before or after a comma). Service Manager exports all files in the system if the parameter is not set.
svc_excludefiles	Specifies a list of files that you want to exclude from the export operation. Use commas to separate multiple files (no space before or after a comma).
svc_unicode	Determines whether the exported XML file names support UTF-8 symbols or not. The default value is 0. If this parameter is set to 1, exported XML file names can contain UTF-8 symbols in Windows.
svc_language	Specifies the language of the files that you want to export. • To export files for a specific language, use `svc_language:<language code>` (for exmaple, `svc_language:de`). • To export files for all languages, use `svc_language:all`. • If you don't specify the svc_language parameter, the system only exports files in English. The parameter only works for the files with the SYSLANGUAGE field, and the language is the value of SYSLANGUAGE.

For example, if you want to export ScriptLibrary and formatctrl data only , add the following line in *sm.ini*:

```
svc_files:ScriptLibrary,formatctrl
```

Parameters to import files

To import Service Manager files, you can run the *sm -svc_import* command with the following parameters, or add the following parameters in the *sm.ini* file.

Parameter	Description
svc_rootdir	Specifies the folder from which you import the Service Manager applications data. To run the **svc_import** command, you must specify a valid directory with the read/write permission. For example, the ScriptLibrary--g8.xml file locates in the C:\src\data\dbdict\s directory. When importing this file, you need to set the value of the svc_rootdir parameter to C:\src\data.
svc_cleanbuild	If the value of this parameter is set to **1**, all the files in the *svc_rootdir* directory are imported; if the value of this parameter is set to **0** or not set, the list of files to import is determined by the **svc_files** parameter.
svc_files	Specifies a list of files to import. Use commas to separate multiple files (no space before or after a comma). **Note** Note This parameter works only if the **svc_cleanbuild** parameter is set to **0** or not set.
svc_excludefiles	Specifies the list of files that's excluded in the importing operation. Use commas to separate multiple files (no space before or after a comma).
svc_updatedbdict	Imports dbdict changes. Run the following command to import the updated dbdict xml files (such as probsummary.xml) to database: sm -svc_import -svc_updatedbdict **Note** Note If there are dbdict changes without this parameter, the system displays some log information in the sm.log file according to different dbdict file names. For example: `dbdict 'probsummary' already exists, ignore loading! To update the dbdict, please add option 'svc_updatedbdict'`

svc_postimport	Specifies a list of JavaScript interfaces to be invoked after the import operation. Follow these rules when using this parameter: • Use semicolons to separate multiple JavaScript interfaces (no space before or after a semicolon). • Use parentheses to include parameters of a JavaScript interface (no space before or after parentheses). • Use commas to separate multiple parameters of a JavaScript interface (no space before or after a comma). **Note** Note • Omit the void parameter of JavaScript. Do not omit the parentheses even if the JavaScript has no parameters. • Do not include parentheses, semicolon, and commas in the string parameter of JavaScript. • The system will not exclude the double quotation marks or the single quotation marks of the string parameter. For example, if you want to print `Hello world`, the system will print it with double quotation marks. The principle is verbatim. For example: 1. When running the **svc_import** command between different environments, sometimes the data in the number table and the counter table may not be synchronized. You can add **svc_postimport:BaseUtil.synchronizeCountersAndNumbers()** to the **svc_import** command. 2. To implement multiple JavaScript interfaces: `svc_postimport:BaseUtil.synchronizeCountersAndNumbers();CustomerLib.customerFunction(Hello world, 1234)`

For example, if you want to import only records in the ScriptLibrary and formatctrl folders under the svc_root directory, add the following lines in *sm.ini*:

```
svc_cleanbuild:0
svc_files:ScriptLibrary,formatctrl
```

Other parameters

Parameter	Description
svc_mode	Specifies the type of Source Code Management (SCM) tool. Possible values: • 1: Perforce • 2 : Subversion • 3 : Git • 99 : Null (default)
svc_dateformat	Determines the date format of the xml files that are generated when you run the **svc_export** or **svc_import** command together with the *svc_mode:2* parameter. Valid if set from • Server's operating system command prompt • Initialization file (sm.ini) Requires restart of the Service Manager server? Yes Default value MM/DD/YY Possible values YYYY/MM/DD YY/MM/DD MM/DD/YYYY MM/DD/YY DD/MM/YYYY DD/MM/YY Example usage Command line: **sm -httpPort:13080 -sqldb:dbserver -svc_dateformat:MM/DD/YYYY** Initialization file: `svc_dateformat:MM/DD/YYYY`

Related topics

Implementing version control for your tailoring
SVN version control process

1.8. Localization

Service Manager provides official localization support for a number of languages. To view a list of the supported languages for the current release, see Support matrix.

Here are some typical Service Manager localization user scenarios for your reference:

User scenarios		Procedures
1	You need to localize the content developed by yourself.	See Localize untranslated text strings in Service Manager resource files for detailed instructions.
2	You have introduced a new language that's not officially supported in the out-of-the-box SM system.	1. Localize the content in the SM database. See Localize Service Manager into unsupported languages for detailed instructions. 2. Localize the content in SM UI. See Localize untranslated text strings in Service Manager resource files for detailed instructions.
3	You have introduced a new language that's not officially supported in the out-of-the-box SM system. You have worked with this system for a while and now you have upgraded SM to a higher version.	1. Localize the content in the SM database. See Localize Service Manager into unsupported languages for detailed instructions. 2. Localize the content in SM UI. See Localize untranslated text strings in Service Manager resource files for detailed instructions.
4	You haven't introduced any new language to SM. Now you have upgraded SM to a higher version.	The Service Manager Applications Upgrade Utility upgrades the existing language packs during the upgrade process. The content in the SM database gets localized automatically. Note that a new version of SM may introduce new strings in the UI. You need to manually localize these new strings. See Localize untranslated text strings in Service Manager resource files for detailed instructions.

1.8.1. Localize untranslated text strings in Service Manager resource files

If you want to localize untranslated text strings in Service Manager, follow the instructions in this section.

Note

Note For the Windows Client, it is hard to distribute text translations by customers; additionally, since the company has been focusing on defect fixings instead of new feature development since Windows Client 9.3x, a very limited number of new text strings have been introduced. For these reasons, the Windows Client doesn't really needs self localization and is hence not discussed in this section.

Applications

The localization of the applications may include the following types of records: Format, Message, Help, HTML Template, and Unit of Measure.

Format

To localize a Format record into a target language (for example, Korean), follow these steps:

1. Sign in to the Service Manager Windows client as a system administrator with the target language.
2. From the System Navigator, click **Tailoring** > **Forms Designer**.
3. In the Form field, enter the name of the format.

4. Click **Search** to display the record.
5. Click **Design**.
6. Localize the format record to the target language.
7. Click **OK**.
8. Click **OK**.

For more information, see Forms Designer.

Message

To localize a Message record into a target language, follow these steps:

1. Sign in to Service Manager as a system administrator.
2. From the System Navigator, click **Tailoring** > **Notifications** > **Messages**.
3. Enter other information as appropriate, and then click **Search** to display the record.
4. Update the **Text** field to a localized version for the target language.
5. Click **Save**.

For more information, see Add a message record.

Help

To localize a Help record into a target language, follow these steps:

1. Sign in to the Service Manager Windows client as a system administrator with the target language.
2. From the System Navigator, click **Tailoring** > **Database Manager**.
3. Type **help** in the **Form** field, and then click **Search**.
4. Double-click the help (help.g) form.
5. Click **Search** to display the record.
6. Update the help record.
7. Click **Save**.
8. Click **Accept**.

For more information, see Field help editor.

HTML Template

To localize an HTML template into a target language, follow these steps:

1. Sign in to the Service Manager Windows client as a system administrator with the target language.
2. From the System Navigator, click **Tailoring** > **Notifications** > **HTML Templates**.
3. Enter information as appropriate, and then click **Search** to display the record.
4. Update the template record to a localized version for the target language.
5. Click **Save**.

Teams Message Template

To localize a Teams message template into a target language, follow these steps:

1. Sign in to the Service Manager Windows client as a system administrator with the target language.
2. From the System Navigator, click **Tailoring** > **Notifications** > **Teams Message Templates**.
3. Enter information as appropriate, and then click **Search** to display the record.
4. Update the template record to a localized version for the target language.
5. Click **Save**.

Unit of Measure

To localize a unit of measure into a target language, follow these steps:

1. Sign in to the Service Manager Windows client as a system administrator with the target language.
2. From the System Navigator, click **Tailoring** > **Database Manager**.
3. Type **unitofmeasure** in the **Table** field, and then click **Search**.
4. Enter information as appropriate, and then click **Search** to display the record.
5. Update the Unit of Measure record to a localized version for the target language.
6. Click **Save**.

Web Tier

The Service Manager Web Tier is a standard web application developed with Java, which uses the standard internalization technique introduced in Java Standard Edition.

The text strings that need to be localized are located in the following files:

```
<webtier-9.xx.war>/WEB-INF/classes/alert_en.properties
<webtier-9.xx.war>/WEB-INF/classes/calendarBundle_en.properties
<webtier-9.xx.war>/WEB-INF/classes/chMBundle_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/cwc/web/app_labels_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/branding/brandingBundle_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/cac/cac_logon_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/cpe_web_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/diagram/diagram_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/L10N/recordList/recordList_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/report/pivot/pivot_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/smc/smc_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/view/cpew_view_en.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/xslstrings_en.properties
<webtier-9.xx.war>/WEB-INF/classes/conditionEditorBundle_en.properties
<webtier-9.xx.war>/WEB-INF/classes/dashboardBundle_en.properties
<webtier-9.xx.war>/WEB-INF/classes/globalSearchBundle_en.properties
<webtier-9.xx.war>/WEB-INF/classes/workflowBundle_en.properties
```

The originally localized cwc_labels file in languages other than English are as follows:

```
<webtier-9.xx.war>/WEB-INF/classes/alert_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/calendarBundle_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/chMBundle_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/cwc/web/app_labels_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/branding/brandingBundle_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/cac/cac_logon_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/cpe_web_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/diagram/diagram_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/L10N/recordList/recordList_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/report/pivot/pivot_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/smc/smc_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/telephony/telephonyBundle_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/view/cpew_view_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/com/hp/ov/sm/client/webtier/xslstrings_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/conditionEditorBundle_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/dashboardBundle_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/globalSearchBundle_xx.properties
<webtier-9.xx.war>/WEB-INF/classes/workflowBundle_xx.properties
```

Each resource file follows a naming rule, which is <basename>_<language>.properties or <basename>_<language>_<country>.properties. Take the cpe_web_en.properties file for example, cpe_web is the basename, and en is the language code. For cpe_web_zh_CN.properties, CN is the country code. The file names are case-sensitive.

The following table lists the language codes of the supported languages in Service Manager. This table uses cpe_web as an example resource file base name (other resource files follow the same rule).

Resource base name	Language Code	Country Code	Description
cpe_web			Default
cpe_web	en		English
cpe_web	ar		Arabic
cpe_web	cs		Czech
cpe_web	de		German (Standard)
cpe_web	es		Spanish(Spain)
cpe_web	fr		French(Standard)
cpe_web	hu		Hungarian
cpe_web	it		Italian (Standard)
cpe_web	ja		Japanese
cpe_web	ko		Korean
cpe_web	zh	CN	Chinese (PRC)
cpe_web	nl		Dutch (Standard)
cpe_web	pt_BR		Portuguese (Brazil)
cpe_web	ru		Russia
cpe_web	iw		Hebrew
cpe_web	pl		Polish
cpe_web	tr		Turkish

As of Java 9, UTF-8 is the default encoding specifically for property resource bundles. So, you can directly add your newly introduced string in native language to the **target .properties** file.

For example, you can add the below string in Turkish to the **cpe_web_tr.properties** file.

```
WebController.Main_Menu_1=Ana Menü
```

Mobility Client

Text strings in Mobility Client that should be localized are located in the following folder: <webapp-9.xx.xxxx.war>\WEB-INF\classes. There are three base names: appLoginBundle, appMainBundle, and getIT.

The steps are similar to those for the web tier. For details, see the *Web Tier* section above.

Chat

To localize the Chat UI into a target language, follow these steps:

1. Go to the <chat-ui-xxxx.war>\assets\locales folder.
2. Open the folder for the target language. For example, open the `tr` folder for Turkish.
3. Open the i18n.json file in a text editor.
4. Update the untranslated text strings in this file to the target language.
5. Save the file with the UTF-8 encoding.

> **Caution**
> Caution Do not save it with the ANSI encoding.

1.8.2. Localize Service Manager into unsupported languages

This section aids the company's partners to localize Service Manager into an unofficially supported language. The partners can export translatable content from Service Manager, translate these content, and then import the localized content back to Service Manager.

Export the content for translation

This section describes how to extract the content to be translated. Later, you can import the translated content back to Service Manager.

Navigate to the `<RTE installation folder>/RUN` directory and then run the following command:

sm -bg -str_export:all

The system generates the following files and folder in the RUN directory:

- htmltemplates (folder)
- format.str
- help.str
- htmltemplates.str
- msMessageTemplate.str
- message.str
- unitofmeasure.st

The htmltemplates folder contains html files.

Translate the exported content

You can translate the exported content to the target language(s). This section provides instructions about how to manage the content for translation.

> **Note**
> Note
> This section is for software localization professionals only. We assume that you have a profound understanding of software product localization, knowledge of which items require translation, and knowledge of the relevant translation tools.

Service Manager doesn't provide any translation or translation memory features. We recommend you either use a translation tool (such as Passolo or Trados) or use the services of professionals that use such kinds of tools.

Follow these steps to access, configure, and translate the exported content:

1. Ensure that you have read the Service Manager Localization Rules. Make sure that your translations meet the guidelines and requirements described in this section.
2. Create a folder named<*new_lang*> according to the target language, and then copy all the export content to this folder.
3. Translate each file in the <*new_lang*> folder.

Refer to the following example for the Vietnamese language:

1. From the System Navigator, click **Tailoring** > **Database Manager**.
2. Type `language` in the Table field, type `language` in the Form field, and then click **Search**.
3. Double-click the language format to open the **Language Identification** page.
4. Type `Vietnamese` in the language field, and then click **Search**.
   The identifier for the Vietnamese language record is `vi`.
5. Create a folder named `vi` and copy all the export content to this folder.
6. Translate each file in the `vi` folder.

> **Note**
> Note
> You need to add the target language to the language table in the Service Manager environment if it doesn't already exist.

Import the localized content and active the language

This section describes how to import the translated content back to Service Manager.

Copy the <*new_lang*> folder to the <RTE installation folder>/RUN directory, and then run the following command in the RUN directory:

sm -bg -str_import:all -str_language:<language name>

Note
Note
The system will first remove all files for this language.

Note
Note
You must enclose the language name in double quotation marks if the language name contains a space. For example, **sm -bg -str_import:all -str_language:"Chinese Simplified"**.

Refer to the following example for the Vietnamese language:

Copy the `vi` folder to the RUN directory, and then run the following command in the RUN directory:

sm -bg -str_import:all -str_language:Vietnamese

Follow these steps to active the Vietnamese language record after import:

1. From the System Navigator, click **Tailoring** > **Database Manager**.
2. Type language in the Table field, type language in the Form field, and then click **Search**.
3. Double-click the language format to open the **Language Identification** page.
4. Type `Vietnamese` in the language field, and then click **Search**.
5. Select the **Active for logins** checkbox.
6. Click **Save**.

1.8.3. Service Manager localization rules

Preface

This section describes the localization rules for Service Manager. Be sure to adhere to these rules and localize language and words only when allowed.

The product name Service Manager should always be in English in all cases. Module names such as Incident Management, and Problem Management should be localized according to ITIL standards or left in English, depending on market requirements.

Localization rules for software

Properties files for format, message and help on field

The strings extracted from format, message and help on field are placed in the format.properties, message.properties, and help.properties files, respectively. When you translate strings from format, message, and help on field, you must follow these rules:

- Files that return from translation and need to be imported must be encoded in ANSI (not Unicode).
  For example, the English string "Properties" should appear as: "Propri\u00e9t\u00e9s" (which is encoded in ANSI) rather than the native string in French: "Propriétés" (which is encoded in Unicode).

  Note
  Note
  To convert a file with native strings to ASCII encoding (in the /u#### representation) and vice versa, you can use the "native2ascii" tool provided in the Java JDK. More information on this tool is available at https://docs.oracle.com/javase/8/docs/technotes/tools/windows/native2ascii.html.

- Use hyphens instead of n-dashes.
  Use hyphens instead of n-dashes when localizing strings such as: `FORMAT.STR/AlertDef.g\13\0\0\0\Caption=$L.slo.expiration - SLO expiration date/time`.
  If you use an n-dash (for example, `$L.slo.expiration - SLO expiration date/time`), this character will display as a symbol (something like a vertical rectangle).
- Use a backslash before double quotes.
  If you need to insert double quotes within a string to identify GUI terms, use an escaping backslash before the double quotes.

  Caution
  Caution
  If we don't have backslashes preceding the double quotes, the string will be truncated, giving us error while re-importing the files into Translation Editor.

The following is an example:
When you write:

```
<span class=FormTitle>Recherche $$(_DocumentLabel)</span><br> Entrez
autant de valeurs que vous le souhaitez pour affiner la recherche, puis
appuyez sur le bouton "Rechercher".
```

The string will be truncated and will display as:

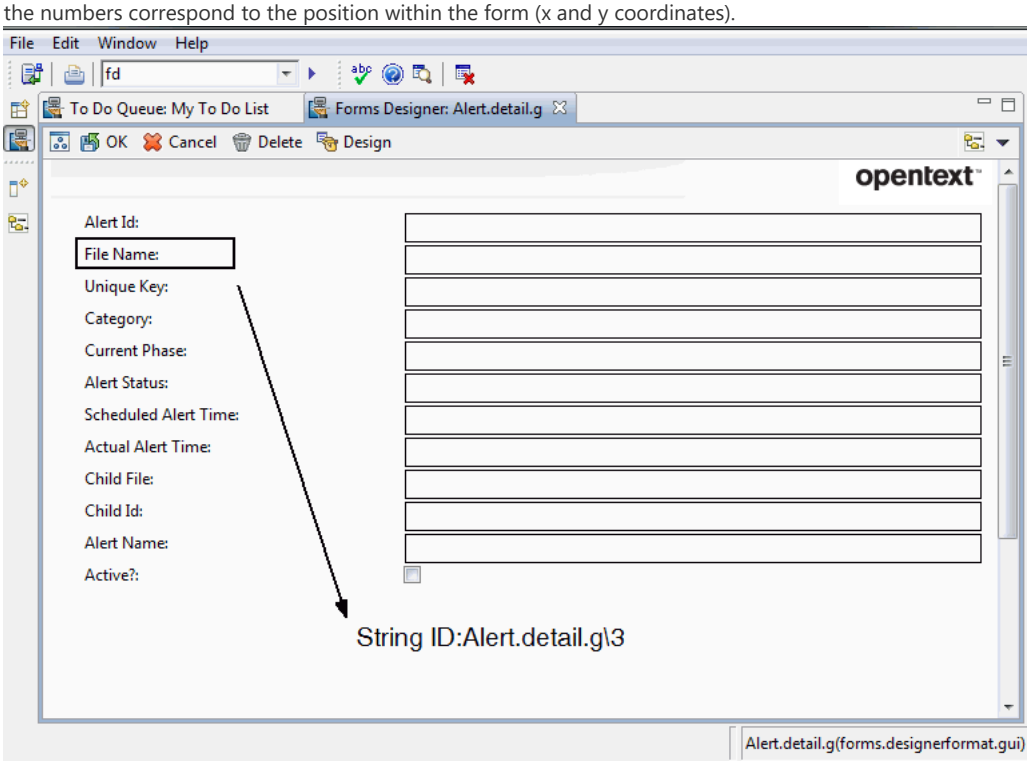
```
<span class=FormTitle>Recherche $$(_DocumentLabel)</span><br> Entrez
autant de valeurs que vous le souhaitez pour affiner la recherche, puis
appuyez sur le bouton
```

In order to avoid this, you need to write the string like this:

```
<span class=FormTitle>Recherche $$(_DocumentLabel)</span><br> Entrez
autant de valeurs que vous le souhaitez pour affiner la recherche, puis
appuyez sur le bouton \"Rechercher\".
```

Note
Note
If double quotes are already included in the strings, don't delete them.

- If the strings contain logic or variables, the logic should not be broken and the variables should not be translated.
  See the following examples:
  - `<word>;<word>:\:\<word>` should be translated as `<trans>;<trans>:\:\<trans>`.
  - `&<char>` or `$<word>` should remain untranslated.
  - Strings that contain parameters like {0} can be translated, but the variable representation must remain unchanged - {0}.
    For example: `{0} files were found in folder {1}` will be translated to Chinese as `{1} 文件夹中发现了{0} 个文件`.
- Do NOT add special characters unless they appear in the English strings.
- Translate the output and caption strings for each label exactly the same way.
  For each label in a form, there are two strings: output and caption. The two strings must be translated exactly the same way. The following screenshots illustrate an example string ID: Alert.detail.g\3. The first part of the string corresponds to the actual form, the numbers correspond to the position within the form (x and y coordinates).



```
"Alert.detail.g\1\1\0\0\output", "Alert Id"
"Alert.detail.g\1\1\0\0\Caption", "Alert Id"
"Alert.detail.g\3\1\0\0\output", "File Name"
"Alert.detail.g\3\1\0\0\Caption", "File Name"
"Alert.detail.g\5\1\0\0\output", "Unique Key"
"Alert.detail.g\5\1\0\0\Caption", "Unique Key"
"Alert.detail.g\7\1\0\0\output", "Category"
"Alert.detail.g\7\1\0\0\Caption", "Category"
"Alert.detail.g\9\1\0\0\output", "Current Phase"
"Alert.detail.g\9\1\0\0\Caption", "Current Phase"
"Alert.detail.g\11\1\0\0\output", "Alert Status"
"Alert.detail.g\11\1\0\0\Caption", "Alert Status"
"Alert.detail.g\13\1\0\0\output", "Scheduled Alert Time"
"Alert.detail.g\13\1\0\0\Caption", "Scheduled Alert Time"
"Alert.detail.g\15\1\0\0\output", "Actual Alert Time"
"Alert.detail.g\15\1\0\0\Caption", "Actual Alert Time"
"Alert.detail.g\17\1\0\0\output", "Child File"
"Alert.detail.g\17\1\0\0\Caption", "Child File"
"Alert.detail.g\19\1\0\0\output", "Child Id"
"Alert.detail.g\19\1\0\0\Caption", "Child Id"
"Alert.detail.g\21\1\0\0\output", "Alert Name"
"Alert.detail.g\21\1\0\0\Caption", "Alert Name"
"Alert.detail.g\23\1\0\0\output", "Active?"
"Alert.detail.g\23\1\0\0\Caption", "Active?"
"Alert.g\1\0\0\0\output", "Alert"
"Alert.g\1\0\0\0\Caption", "Alert"
"Alert.g\2\1\0\0\output", "Alert Name"
"Alert.g\2\1\0\0\Caption", "Alert Name"
"Alert.g\4\1\0\0\output", "Alert Status"
"Alert.g\4\1\0\0\Caption", "Alert Status"
"Alert.g\6\1\0\0\output", "Scheduled Alert Time"
"Alert.g\6\1\0\0\Caption", "Scheduled Alert Time"
"Alert.g\8\1\0\0\output", "Actual Alert Time"
"Alert.g\8\1\0\0\Caption", "Actual Alert Time"
"Alert.g\10\1\0\0\output", "Active?"
"Alert.g\10\1\0\0\Caption", "Active?"
"Alert.qbe\1\0\0\0\output", "Alert Name"
```

Properties files for the clients

Do not remove \n where in use, otherwise the string will be truncated because sentences don't wrap automatically with the Eclipse client.

JS and HTML files

Follow the standard rules in the localization industry.

How to find strings in the application

The string ID consists of the format name and a numbering scheme for the position within the format. The format name can be entered in Service Manager Forms Designer. Follow these steps to find strings:

1. Start the Service Manager server.
2. Start the Service Manager client (don't close the Command Prompt window).
3. Select your login language.
4. Sign in to Service Manager as a system administrator.
5. Type `fd` in the Service Manager command line field, and then press Enter. The Forms Designer window opens.
6. Type the full format name and click **OK** to search for a specific format.
7. Type `*.g` to search for all formats with the ending `.g` or type any letter to search for all formats starting with that letter.

How to find Help on Field messages in the application

Field Help enables the user to learn more about an individual field in a form and the value to specify in the field.

Follow these steps to view Field Help:

1. Navigate to any Service Manager form.
2. Insert the cursor in any field or table cell.
3. Do one of the following:
   - Press CTRL+H.
   - Click **Help** > **Help on Field**.
   - Click **Help** on the toolbar.

Strings that must remain in English

See the following table for strings that must remain in English.

Type	Examples
Event names	rmqin rmlin rmoin
Predefined system events	alerts recalc alerts reset approval approval reeval
boolean expressions	UNKNOWN TRUE FALSE
parameter names	analyzelog lockanalysis lockslog
RAD debugger commands	d (display) t (trace) ta tar tt b (breakpoint) ba (ba menu.manager) bt (bt rinit) c (continue) s (step) gl (globals) v (variables) l (locals) sta (stack) re (relations) m (memory) h (help)
Display list variables	$G.init.impact.disp $G.urgency.disp

2. Introduction

This document is intended for Service Manager system administrators and implementers. It assumes that the reader is familiar with object-oriented programming, the Service Manager System language (RAD), and JavaScript. It provides a brief introduction to RAD, and it provides an overview of JavaScript functionality in Service Manager, using both examples and descriptions of JavaScript's objects, properties, methods, and functions.

System administrators and implementers can use the standard JavaScript scripting language and RAD to customize their Service Manager applications. RAD is the Service Manager system language that enables administrators and implementers to manipulate Service Manager processes and workflows. JavaScript is a widely supported, industry-standard programming language that eliminates the need for a separate programming skill set, and reduces user reliance on proprietary tools. JavaScript in Service Manager doesn't invalidate the current Service Manager RAD tool set; instead, the JavaScript interface is an alternative to the RAD language for tailoring.The main components of this document are:

- System language (RAD)
- JavaScript and Service Manager
- Where to use JavaScript in Service Manager
- System language reference that describes the RAD functions, rtecalls, and RAD routines
- JavaScript and Service Manager reference that describes the JavaScript global system objects, JavaScript global methods, Service Manager defined JavaScript objects, JavaScript functions, JavaScript functions in the ScriptLibrary, and JavaScript examples

2.1. System language

The System language (RAD) provides the ability for administrators to manipulate the Service Manager processes and workflows using a programming language with a syntax that's similar to the C programming language. Administrators and implementers use this language daily to apply Service Manager to a specific purpose or enterprise. The key to Service Manager's remarkable flexibility can be found in the proper use of the system language.

The System language creates RAD applications that contain application panels (records) linked together in a logical flow. Each panel performs a specific function. When the function is complete, the application exits to another panel. The field value within the exit is a label name of another panel.

The parameter panel defines local variables passed to it by a calling application. In general, two kinds of local variables exist: those used within an application, and those used as exits. Normally, the exit variables are $normal (in the normal exit) and $error (in the error exit).

The first command panel, where execution begins, is always labeled start. Execution continues at a panel's normal exit (or another exit depending on conditions). When that exit is defined as $normal, execution of the panel is complete. If an error is encountered, then the $error exit is taken.



Note

Note You must have a RAD license to view and modify the RAD applications with the RAD editor.

2.1.1. Data types

The Service Manager data type specifies the expected format of a field's data. The data type determines the possible range of values of the set, the operations that can be performed on the values, and the way values are stored in memory. Defining a data type enables the system to manipulate the data appropriately. The values are denoted either by literals or variables. They also can be obtained as a result of operations. Data types can be either primitive or compound.



Note

Note

System Administrators and implementers should only use types 1 through 4 or 8 to 10.

List: Data types

Data type	Numeric representation	Compound / Primitive	Purpose
number	1	Primitive	Used for all numeric type data.
character	2	Primitive	Used for character strings.
time (date/time)	3	Primitive	Used for absolute dates and times as well as relative time intervals.
Boolean (logical)	4	Primitive	Used to represent true or false.
label	5	Primitive	Used to define exits.
File/record	6	Compound	used to define a table or record.
Offset	7	Compound	This is the compiled version of a label. It is used within the code for the go to statements.
Array	8	Compound	Used to represent multiple fields of the same data type.
Structure	9	Compound	Used to represent a collection of fields of the same or different data types.
Operator	10	Primitive	Used in expressions.
Expression	11	Primitive	Used to represent parsed Rapid Application Development (RAD) language expressions.
Pseudo field	12	Compound	Used similarly to functions, it converts input data into output data.
Global variable	13	Compound	Used for variables visible to the whole system.
Local variable	14	Compound	Used for variables only visible to the RAD application

2.1.1.1. Data type: Primitive

Primitive data types consist of one element. Service Manager internally represents each data type by its numeric representation. Certain functions (**val()** and **type()**) allow the user to determine or modify the numeric representation of a data type, thus changing the data type.

Data type: Number

The number data type is a primitive data type represented by the number 1 in the database dictionary. A number indicates an amount and can be either a whole number, a positive or negative number, or a floating point number. Enter numbers with an optional sign (+ or -) followed by digits which are optionally followed by a decimal point, then more digits which are optionally followed by the letter E and an exponent.

The following number formats display valid numbers accepted by RAD.

Description	Valid literal examples
Integer	`1`
Negative Integer	`-1`
Fixed Point	`32.1`
Floating Point	`1.257E05`

Furthermore, each bullet shows different representations of the same number:

- 1 = 1.0 = 1E0 = 1.0E0
- -1 = -1.0 = -1E0 = -1.0E0
- 32.1 = 32.10 = 3.21E1
- -32.1 = -32.10 = -3.21E1
- 1.257E05 = 125700 = 1.25700E5

Data type: Character

The character data type is a primitive data type represented by the number 2 in the database. A character data type is typically expressed as a character string. A character string has the following characteristics:

- It must be enclosed in double quotation marks.
- It can be any combination of letters, digits spaces, or special characters.
- It can contain any special characters including those used as operators.
- It can contain a double quotation mark, but it must be preceded by a backslash (\) to enable the system to distinguish between double quotation marks as part of the string and a double quote at the beginning or end of the string. For example, `"Vendor=\"473\""` denotes the string: `Vendor="473"`.
- It can contain non-characters, such as two hexadecimal digits preceded by a backslash.
- It can treat any special or reserved character as a literal character by preceding it with a backslash.
- It can present a literal backslash (\) by containing a double backslash (\\).
- It can be any length, including zero.
- An empty (for example, length of zero) character string is not the same as a character string with a NULL value.

The following table presents some examples.

Description	Literal examples
Alphabetic	`"abcde"`
Numeric	`"12379"`
Alphanumeric	`"a1276b45"`
Special	`"$%=+*#@/"`
Mixed	`"$vendor"`
Mixed	`"$12379@6:54"`
Mixed	`"A1276%#B"`
Hexadecimal	`"\01\ff\c2"`
Containing backslash	`"C:\\Program Files\\OpenText\\Service Manager 9.80\\Server\\RUN\\sm.ini"`
Containing double quote	`"Enter the \"uid\" in the Unique ID field"`

Data type: Time

The Time data type is a primitive data type represented by the number 3 in the database dictionary. In all of the examples in this topic, the following acronyms apply: MM=month; DD=day; DDD=day; YY=year; HH=hours; MM=minutes; SS=seconds. Furthermore, MM/DD/YY is collectively referred to as date and HH:MM:SS is collectively referred to as time. The following rules govern the use of the time data type:

- Time must be enclosed in single quotation marks.
- Time is recorded on a 24-hour clock. '00:00:00' is midnight, '12:00:00' is noon, '18:00:00' is 6 p.m.
- Enter absolute time in the following format: 'MM/DD/YY HH:MM:SS'. The format's sequence is affected by the set.timezone() function. It determines the order of date (MM/DD/YY). Note that absolute time is stored as GMT notation.
- Enter relative time in the following format: 'DDD HH:MM:SS'
- The use of seconds is optional. If you don't use seconds, the default is :00.
- The use of time is optional in absolute time. If you omit time, it defaults to 00:00:00. For example, 'MM/DD/YY' is 'MM/DD/YY 00:00:00'.
- The use of days is optional in relative time. If you omit days, the default is 0. For example '12:00' is zero days and 12 hours, '345 03:00' is 345 days and 3 hours.
- Use $time = '00:00' to set a time variable to a 0 (zero) value.

The following date/time formats show valid times/dates accepted by RAD:

Date/time format	Description	Valid literal examples
Absolute time	Indicates a specific date and time	'07/16/83 04:30:00'
Relative time	Indicates an amount of time	'197 05:00' (means 197 days and 5 hours)
Relative time	Indicates an amount of time	'1 00:00:00' (means 1 day)

Data type: Boolean

The Boolean or logical data type is a primitive data type represented by the number 4 in the database dictionary. A Boolean indicates true, false or unknown (of undetermined truth value). The rules for forming a Boolean are as follows:

- Enter the Boolean directly as true, false, NULL, unknown (t, f or u).
- Enter the Boolean in either upper or lowercase characters.

The following formats are accepted.

Description	Valid literal examples
True	t = T= true = TRUE = y = Y = yes = YES
False	f = F = false = FALSE = n = N = no = NO
NULL	Null = NULL
Unknown	u = U = unknown = UNKNOWN

Data type: Label

The label data type is a primitive data type represented by the number 5 in the database dictionary. Labels are used to define exits. The following examples are valid labels.

- $exit
- $normal
- <panel name>

2.1.1.2. Data type: Compound

Compound data types consist of more than one element. You may combine primitive data types to form compound data types. Any compound data type can have elements of any other valid data types.

2.1.1.2.1. Data type: File/Record

The File/Record data type is a compound data type represented by the number 6 in the database dictionary. A file is a set of related records treated as a single unit and is stored on disk. A file is a particular kind of structure, but a structure is not a kind of file.

The term file is synonymous with the terms relation or table and is sometimes used interchangeably with the term table because of the legacy file system (P4) in Service Manager.

Every table must have a table name and other descriptive data in the database dictionary. For example, the name of the contacts table is contacts. The contacts table has keys, fields with index numbers, and a descriptor. Table variables are the mechanism you use to make

changes to tables. Always create table variables on the rinit command panel. The table variable stores one record at any given point in time. The record resides in memory for display or modification. For additional information, see Database administration.

File variables

A file variable is an instance of a single record in a file. It's not an array, but it is conceptually similar. A file variable contains the dbdict name, the data, and the structure of an entire record as it would be stored in the database according to the dbdict. A file variable has four parts:

- **File name in the database dictionary** The RAD Database Manager requires this to be a single word without spaces or periods. The rinit command panel binds the file variable to a file. The file name of a file variable can be accessed either by using the pseudo field or using the **filename()** RAD function.
- **List: records selected from the file** When executed, a select command panel retrieves the file's records from the database and sets up the list. When executed, a **fdisp** (File Display) command panel displays summary information from records in the list.
- **Current record** A file variable can only contain one record from a file at a time. You can move to the next record in the list of records by using the next command panel (Read Next Record). You can move to the previous record in the list by using the previous command panel (Read Previous Record). The select command panel always sets the current record to the first record that satisfies the selection query. The fdisp (File Display) command panel sets the current record to the record selected by the cursor when the user presses **Enter**.
- **Descriptor** The descriptor enables fields to be extracted by name. Each record is a structure and has the structure delimiter {[ ]}. The field names in the structure are listed in the database dictionary.

For example:

- $file is a file variable.
- lastname is a field name listed in the database dictionary for the file that $file has been bound to using the rinit command panel.
- lastname in $file is a valid expression whose value is the lastname field in the current record in $file, or the nth field, based on the index of the lastname field in the descriptor.



Note

Note

Service Manager is case-sensitive. The case for field names, file names, and other elements must match. For example, STATUS.CUST is not the same as status.cust.

2.1.1.2.2. Data type: Offset

The Offset data type is a legacy compound data type represented by the number 7 in the Database Dictionary. Offset fields aren't supported by RDBMS tables.

2.1.1.2.3. Data type: Array

The array data type is a compound data type represented by the number 8 in the database dictionary. Arrays store a list of elements of the same data type accessed by an index (element) number. The term array is synonymous with the terms list, vector, and sequence. Elements in arrays can be of any data types (including arrays or structures). A fully qualified array name (array field in $file) can be used in place of an array variable. The number of items in an array can vary and doesn't have to be allocated in advance. Arrays are delimited by curly braces ({ }). The following table shows some examples.

Description	Literal examples
Numeric	{1,2,3}
Character	{"a","b","c","f","e","h"}
Boolean	{true, true, unknown}
Time	{'12/7/42 00:00', '1/3/62 00:00'}
Nested	{{1,2},{3,4}}
Structures	{{[1,"a"]},{[2,"b"]}} {{[1,"a",true]},{[2,"b",false]},{[3,"c",unknown]}}
Empty	{}

You can use either of the following equivalent syntaxes to access an element in an array:

- $array[element_number]
- element_number in $array

For example, to extract the value of the first customer number (2753) in the array $customer with value {2753, 2842, 2963}, use any of the following equivalent syntaxes:

- 1 in $customer
- $customer[1]

If the accessed array element doesn't exist, the element is created and set to NULL. This effectively extends the array. To insert a value into an array use the insert RAD function. To delete an element from an array, use the delete RAD function.



Tip

Tip You can use the denull RAD function to remove any null values from the end of an array before assigning the array to a record or adding it to the database.

2.1.1.2.4. Data type: Structure

The Structure data type is a compound data type represented by the number 9 in the database dictionary. A structure is a group of named elements of (possibly) differing data types accessed by an index (element) number. Structures are delimited with both curly braces and square brackets ({[ ]}). Note the following examples:

- {[1,"a",true]}
- {[true,'10/16/90 00:00',false]}
- {[1,1,"b",0]}

You can use either of the following syntaxes to extract an element from a structure:

- $structure[index of field]
- index of field in $structure

For example, to extract the value of the part number (672) in the part.no field within the structure $order.line1 with the value of {[672,10,"ball.bearings"]} with the field names part.no, quantity and description, use either of the following syntaxes:

- $order.line1[1]
- 1 in $order.line1

In addition, elements of structures may be extracted using their numeric order in the structure . For example, the 1 in $order.line1 is the same as part.no in $order.line1.

Field names are contained in the database dictionary and can only be used when associated with a file variable.

Assuming that the table contains a structure called part and within the structure and there is field called part.no, this command works because it is associated with a file variable.

```
$x=part,part.no in $file
```

These commands don't work because the second command is not associated with a file variable.

```
$y=part in $file
```

```
$x=part.no in $y
```

2.1.1.2.5. Data type: Operator

The Operator data type is any expression that's parsed and returns a value, such as `tod()`, `gui()`, `rtecall()`, `=` (comparison). An operator is a special symbol or function commonly used in expressions. Service Manager uses several different operators:

- arithmetic operators
- logical operators
- relational operators
- special operators
- string operators

The following table presents a list of operators used in Service Manager.

Type	Name	Character	Description
Arithmetic	Addition	+	Indicates that two numbers are to be added together. Example: `49 + 51 = 100`
Arithmetic	Subtraction	-	Indicates that one number is to be subtracted from another number. To distinguish subtraction from a minus sign, the subtraction operator must be followed by a space. Example: `40 - 20 = 20`
Arithmetic	Multiplication	*	Indicates that one number is to be multiplied by another number. Example: `5 * 5 = 25`
Arithmetic	Division	/	Indicates that one number is to be divided by another number. Example: `300 / 10 = 30`
Arithmetic	Exponentiation	**	Indicates that the exponential value of a number is to be calculated. Example: `2 **5 = 32`
Arithmetic	Modulus	mod or %	The modulus is the remainder of a division operation. You may specifically want the remainder for a division operation, or you may want to generate a circular number sequence within a given range. Example: `5 mod 2 = 1 or 5 % 2 = 1`

String	Concatenate	+	Indicates that two strings or two arrays are to be combined (concatenated) into a single string.
			Example:
			`"a" + "b" = "ab"`
			`{"a", "b", "c",} + {"d", "e"} = {"a", "b", "c", "d", "e"}`

Logical	not	!	Inverts the Boolean value of the Boolean expression. If the expression is true, the system returns FALSE. If the expression is false, the system returns TRUE.
			Example:
			`not TRUE = FALSE`
			`~ FALSE = TRUE (UNIX only)`
			`UNKNOWN = UNKNOWN`
			`~ UNKNOWN = UNKNOWN`

Logical	and	AND and &	Evaluates two expressions and returns a value of `TRUE` if both expressions are true. If one or both of the expressions is false, the system returns `FALSE`.
			Example:
			`TRUE and TRUE = TRUE`
			`TRUE and FALSE = FALSE`
			`TRUE and UNKNOWN = UNKNOWN`
			`FALSE and UNKNOWN = FALSE`

Logical	or		Evaluates two expressions and returns a TRUE if either or both of the expressions is true. If both expressions are false, it returns a FALSE.
			Example:
			`TRUE or FALSE = TRUE`
			`FALSE | FALSE = FALSE`
			`FALSE OR UNKNOWN = UNKNOWN`
			`TRUE OR UNKNOWN = TRUE`

Relational	Less Than	<	Indicates that the value of one item is less than the value of another item. Example: `400 < 500 is TRUE`

Relational	Less Than or Equal To	<= or =<	Indicates that the value of one item is less than or equal to the value of another item.
			Example:
			`400 < = 500 is TRUE`
			`400 = < 500 is TRUE`

Relational	Equal To	=	Indicates that the value of one item is equal to the value of another item. Example: `1 = 1 is TRUE`
Relational	Greater than	>	Indicates that the value of one item is greater than the value of another item. Example: `'08/01/83 00:00' > '07/20/83 00:00' is TRUE`

Relational	Greater Than or Equal To	>= or =>	Indicates that the value of one item is greater than or equal to the value of another item.
			Example:
			`600> =300 is TRUE`
			`600= >300 is TRUE`

Relational	Not Equal To	(¬ =) or (~=) or (<>) or (><)	Indicates that the value of one item is not equal to the value of another item. Example: 1 ~=2 is TRUE (UNIX only)
Relational	Starts With (Truncated Equals)	#	Indicates that the value of the first string starts with the value of the second string. The order of the operands affects this operation. Example: `"abc"#"ab" is TRUE`
Relational	Does Not Start With (Truncated Not Equal To)	¬# or ~#	Indicates that the value of the first string doesn't start with the value of the second string. The order of the operands affects this operation. Example: `"ab" ~ #"abc" is TRUE (UNIX only)`

Special	Statement Separation	;	This operator separates two or more statements on the same line. Example: `$A=$B+$C;$B=$C+$D`

Special	Parentheses	()	This operator groups together expressions or statements. Service Manager follows the standard order of operations: operators inside the parentheses are evaluated first. Parentheses themselves are evaluated from left to right.
			Example:
			`3*($x + $y)`
			`IF ($x=1) THEN ($y="z")`
			`ELSE ($y=3)`
			`IF ($x=1) THEN`
			`($x=2;$z=1) ELSE ($y=3)`

Arithmetic operators

An arithmetic operator indicates actions to be performed under the terms of an arithmetic expression. Arithmetic operators have the following precedence: exponentiation, followed by multiplication, division and modulus (all equal); then addition and subtraction (both equal). Operators with higher precedence are evaluated first. When the operators have equal precedence, they run from left to right.

Logical operators

A logical operator evaluates one or two Boolean expressions. It determines whether the expression is true, false, or unknown.

The logical operators are as follows (executed in the order shown):

- not
- and
- or

The unknown truth value is treated according to the Substitution Principle, which dictates:

- If `UNKNOWN` occurs as a logical operand, then the result of the operation is `TRUE` if substituting `TRUE` or `FALSE` for `UNKNOWN` always yields `TRUE`.
- The result is `FALSE` if substituting `TRUE` or `FALSE` for `UNKNOWN` always yields `FALSE`.
- The result is `UNKNOWN` if substituting `TRUE` or `FALSE` for `UNKNOWN` sometimes yields `TRUE` and sometimes yields `FALSE`.

Relational operators

A relational operator makes a comparison, then generates logical results on whether the comparison is true or false. An operand can be checked for null using the special value `NULL`. `$x=NULL` is true if `$x` is null. Relational operators treat null operands according to the `NULL` substitution principle:

- If the relation is true regardless of the value of the null operand, the result is true.
- If the relation is false, the result is false.
- Otherwise, the result is unknown.

2.1.1.2.6. Data type: Expression

Expressions are sequences of operators and operands used to compute a value. An expression contains literals or variables. An expression can also be used with a function call. Here are examples of expressions:

- $X * 25
- $Y > 32

2.1.1.2.7. Data type: Pseudo field

The pseudo field data type is a legacy compound data type represented by the number 12 in the database dictionary. A pseudo field is similar to a function, because it converts input data to output data and can sometimes be assigned. It can be placed on the left-hand side of the equals (=) sign in an assignment statement. Service Manager has pre-defined pseudo-fields (Month and Name). Pseudo fields were supported for legacy ServiceCenter data but aren't supported by SQL tables. The following tables describe the pseudo fields.

Month	
Explanation	Accesses the number of months from the beginning of the time base in a date. Use it to increment a date by a period of months.
Format	Month in $time +=1 This increments the current month by one.
Factors	The month pseudo field is useful for scheduling monthly reports, since it calculates the number of days to add to each month.

Name	
Explanation	Accesses the file name in a specific file variable.
Format	$filename=Name in $file This sets $filename to the name of the file bound to $file.
Factors	The name pseudo field is analogous to the filename function

2.1.1.2.8. Data type: Global variable

The global variable data type is a compound data type represented by the number 13 in the database dictionary. Global variables begin with `$G.` or `$lo.` and persist throughout a user session (`$lo.` is an abbreviation for logged on. These variables are set when the operator logs on). The server automatically cleans up global variables when a user logs off the system.

2.1.1.2.9. Data type: Local variable

The local variable data type is a compound data type represented by the number 14 in the database dictionary. Local variables begin with `$L.` and persist only within the currently executing RAD application. The server automatically cleans up local variables when it exits a RAD application.

2.1.2. Reserved words

Reserved words have special meaning in Service Manager. This meaning is defined in RAD. They can only be used for that purpose. Do not use them for any other purpose such as field names. Here is a alphabetical list of reserved words:

- AND
- BEGIN
- DO
- ELSE
- END
- FALSE
- FOR
- IF
- IN
- ISIN
- NOT
- NULL

- OR
- STEP
- WHILE
- THEN
- TRUE
- UNKNOWN

2.1.3. Literals

A literal is an explicit value. It is expressed as itself rather than as a variable's value or as the result of an expression. Some simple examples are “23”, “2.5”, and “John.” You can use a literal wherever you can use a variable, with one exception. A literal can't be on the left side of an assignment statement.

2.1.4. Comments

The RAD programming language supports full line, C++ style comments in RAD statements. Comments may be indented, and are prefixed by a double slash ("//").

> **Note**
>
> Note Currently, comments in RAD expressions aren't supported.

Examples[edit]

Valid comments

```
//comment                    //comment
```

Invalid comment

```
$L.n = 1//comment
```

Contents

- 1 Examples

2.1.5. Variables

A variable is a location in memory where a value can be stored. It is a named entity that refers to data to which you can assign values. The data type and value of a variable can be different at different times and can have a primitive or compound data type as its value. Service Manager contains three types of variables:

- Local
- Global
- Thread

Local variables begin with `$L.` and persist only within the currently executing RAD application. The server automatically cleans up local variables when it exits a RAD application.

Global variables begin with `$G.` or `$lo.` and persist throughout a user session (`$lo.` is an abbreviation for logged on. These variables are set when the operator logs on). The server automatically cleans up global variables when a user logs off the system.

Thread variables don't have a consistent naming scheme other than beginning with a dollar sign ($). Thread variables are only valid for the current RAD thread. If the RAD thread terminates, then the server automatically cleans up all thread variables.

Creating variables

The rules for creating variables are as follows:

- A variable name must begin with a dollar sign (`$`) and an alphabetic character followed by zero or more alphanumeric characters that may include periods, but may not include blanks.
- A variable name can be any length.
- A variable's value is initialized to a null string.
- Data assigned to a variable has a data type. This data type becomes the variable's data type.
- Any variable can be set to a null value by assigning the value NULL to it. For example, `$variable=NULL` .
- Global variables are treated as global among all applications executed within the process or task with two exceptions:
    - When the variable is designated local (or something other than global) by an initial identifier. For example, when the variable is created within a function.
    - When the variable is a parameter variable.

Global variables

The following global variables are available when defining registration events.

Variable	Description	Comment
$axces	Represents the eventin record.	
$axces.fields	Represents the evlist field in the eventin record.	
$axces.register	Represents the event registration record.	
$axces.lock.interval	An interval of time before a retry occurs if Service Manager denies the attempt to update an incident because the record is locked. For example: '00:02:00' for two minutes.	
$axces.debug	If true, the evlist array in the eventin record is not removed before attempting to update the record. If the size of the record exceeds 32KB, Event Services generates an error, the eventin record is not updated, and the event reprocesses because the evtime field is not removed.	Use this feature with discretion.
$axces.bypass.failed.validation	Used in events calling the application axces.apm. The default value is NOT true. If true, the application ignores any failed formatctrl validations. If false, the event status is error-fc.	

List: Default variables

It is likely that the list of default variables varies from release to release. The out-of-box system includes the following global and local variables.

Variable	Definition
$G.profiles	List of personal profiles.
$G.pm.global.environment	Incident Management's global environment record .
$G.assignment	List of assignment groups for Service Desk, Incident Management, Problem Management and Change Management.
$G.categories	List of categories.
$G.problem.inboxes	List of Incident Management views available to all the users as well as the views available to the logged on operator.
$G.open.lists	List of all global lists built at login.
$G.icm.status	List of Configuration Management statuses.
$G.assignment.groups	List of assignment groups for Service Desk, Incident Management, Problem Management and Change Management.
$G.prompt.for.save	Global flag (Boolean) that determines whether or not to prompt you to save when you press OK or Cancel after you update a record.
$G.availability.maps	List of availability maps.
$G.pm.environment	The current user's Incident Management personal profile record.
$G.pm.status	List of Incident Management statuses.
$G.technicians	List of technicians (operator table).
$G.incident.inboxes	List of Service Desk views available to all the users as well as the views available to the logged on operator.
$L.filed	This is the current file opened by the display application.
$L.new	The current file being updated when evaluating macro conditions.
$L.old	The pre-updated state of a file undergoing an update while evaluating macro conditions.
$lo.appl.name	The name of the application responsible for managing the user's menu (menu.manager)
$lo.appl.names	Application parameter names passed.
$lo.appl.values	Application parameter values passed.
$lo.cm.limit	Partial-key time threshold for Change Management.
$lo.company.name	The company name as defined in the System Information Record as Menu Title under the Menu Information tab.
$lo.copyright	The company copyright (copyright())
$lo.date.order	The number representing date order (mdy, dmy, ymd) in the operator record.
$lo.db.limit	Partial-key time threshold for Database Manager.
$lo.device	Device ID (current.device())
$lo.groups	List of query group names in operator record.

$lo.home	Name of Home menu for GUI.
$lo.i	Temporary variable that functions as a loop counter (variable is cleaned up in the login application).
$lo.main	Name of main menu for text.
$lo.month.abv	List of abbreviated months (“Jan”, “Feb”, “Mar”).
$lo.month.ext	List of months (“January”, “February”, “March”).
$lo.msglog.lvl	Level of messages to store in the message queue for the operator Level of messages to store in the message queue for the operator logged on. Message levels affect on-screen messages only.
$lo.pm.limit	Partial-key time threshold for Incident Management .
$lo.system.startup	Set at login to value of the field called system.startup.time in Company Information record.
$lo.time.zone	User’s time zone.
$lo.uallow.syslog	Determines whether syslog table is updated when processes start and end in the system.
$lo.uallow.timezone	Allows user to modify time zone based on capability.
$lo.uapprovals	List of change approval groups.
$lo.ucapex	List of user’s capability words.
$lo.uchgrps	List of change groups.
$lo.ufname	User’s full name.
$lo.ulogin.time	User login time stamp.
$lo.user.name	operator()

2.1.6. Variable pools

Variable pools are functional groupings of variables within Service Manager. There are currently four variable pools:

- Global
  A global variable is visible to the entire system. It begins with `$G.` or `$lo.` .
- Local
  A local variable is only visible to the RAD application in which it was defined. It begins with `$L.` .
- Parameter
  A parameter variable is defined on a parameter command panel. It may contain a value passed in from another application. By convention, parameter variables are written in uppercase letters, such as `$PHASE` or `$GROUP.LIST` . Parameter variables are invisible to the debugger.
- Thread
  A thread variable is only visible to the thread in which it was defined. The same variable in different threads has different values, even when the threads are spawned by the same parent.

Variable pool	Variable	Comments
Global	$G.	
Global	$lo.	
Global	$MARQUEE.	
Global	$SYSPUB.	
Local	$L.	There is only one local variable. It can contain many pieces of data, similar to the way a structure contains many pieces of information. For example, `$L.env` and `$L.file` are two pieces of data that `$L.` can contain.
Parameter	$PHASE $GROUP.LIST	These are just a few examples, there are many parameter variables.
Thread	$file $array $post	These are just a few examples, there are many thread variables.

2.1.7. Statements

Statements are the smallest executable entities within Service Manager. A statement is governed by the following rules.

- A statement doesn't have a value.
- A statement is comprised of expressions combined with key words.
- The following BASIC language statements perform processing and looping: IF, WHILE and FOR. They can be nested. For example, for...to...if...then...else...if...then.
- The following assignment statements are used: assign, increment, and decrement.
- RAD functions can be used in statements to further define selection criteria and/or execute commands, initialize values, and perform calculations. Service Manager syntax rules must be followed. The result of a RAD function can be assigned to a variable using the assign (=) assignment statement. For example, `$string.length=lng($string)`.

The following table lists all of the statements available in Service Manager.

Type	Name	Character	Description and Examples
Assignment	Assign	=	Assigns the value of the right hand operand to the left hand operand. Example: `$x=1 assigns 1 to $x`

Assignment	Increment	+ =	Increment the left hand operand by the right hand operand.
			Example:
			`traditional: $x=$x+1 increments $x by 1`
			`shortcut: $x += 1 increments $x by 1`

Assignment	Decrement	- =	Decrement the left hand operand by the right hand operand.
			Example:
			`traditional: $x=$x-1 decrements $x by 1`
			`shortcut: $x -= 1 decrements $x by 1`

Processing and looping	FOR		Allows you to perform a loop. You can set a variable, perform a statement, and increment the variable until the variable is greater than a maximum value. Format this statement as follows:
			`FOR variable name = initial value TO maximum value [DO] statement`
			The brackets ([ ]) indicate that the DO keyword is optional. Example:
			`for $I = 1 to 10 do ($J=$I*$I+$J)`

Processing and looping	IF		Specifies a condition to be tested and a statement to be executed if the condition is satisfied and a statement to be executed if the condition is not satisfied. Format these statements as follows:
			`IF Boolean condition THEN statements [ELSE statements]`
			Example:
			`if ($location="Seattle") then ($x=$x+1) else ($x=$x - 1)`
			If Boolean condition evaluates to UNKNOWN, then neither statement is executed; therefore, we recommend using a default value when the condition evaluates to UNKNOWN.
			Example:
			`if (nullsub($location,"Chicago")="Seattle") then ($x=$x+1) else ($x=$x - 1)`

Processing and looping	WHILE		Specifies a condition to be tested and a statement to be executed when the condition is TRUE. Format these statements as follows:
			`WHILE (expression) [DO] statement`
			Example:
			`while ($x>6) do ($x=$x - 1;$y=$y - 1)`
			The brackets ([ ]) indicate that the DO keyword is optional.

An assignment statement assigns a value to a variable. The order for the assignment statements when read left to right is the destination variable first, then the assignment operator, then the expression. It usually consists of three elements:

- An assignment operator (typically an =).
- An expression to be assigned.
- A destination variable.

On execution, Service Manager evaluates the expression and stores the resulting value in the variable.

2.1.8. List: Command line calls

You can use the Service Manager command line to navigate through the Service Manager system and to run applications, wizards, and macros.

Generic command line calls[edit]

Command	RAD application called	Function	Example
*a<application name>	as specified	Runs the specified RAD routine.	*adatabase accesses Database Manager.
*f<file name>	database	Accesses a specified file.	*fcontacts accesses the contacts file.
*q<query name>	query.stored	Runs a stored query.	*qIM.open.ALL displays a list of all open incidents.
*s<script name>	script.execute	Runs the specified script.	*socmq.open.hr.quote opens an HR Request record.
#<shortcut name>	non-applicable	Queries for a list of shortcut commands.	#do presents a list of commands that start with "do."

Direct command line calls[edit]

Command	RAD application called	Function
about	bulletin.key	Displays the system bulletin and any active "hot" incidents.
ag	encl.appl	Displays the RAD Editor prompt (formerly known as Application Generator).
agcompare	compare.applications	Displays the RAD Application Comparison form.
agentrev	query.stored	Executes the "agentrev" query which displays a list of the schedule records for all SCAuto agents.
agentstat	scauto.check.setup	Displays a currently-available SCAuto agents. Form includes options for stopping agents
agmap	agmap.sched	Displays the Application Mapping form.
alert	database	Displays the Alert Definition form.
alertdefs	se.search.engine	Displays the Alert Definition form.
alertlogs	se.search.engine	Displays the Alert Log form.
appr	database	Displays the Approval Definition form.
approvaldefs	se.search.engine	Displays the Approval Definition form.
approvallogs	se.search.engine	Displays the Approval Log form.
auditdelta	audit.unload.front	Displays the form for unloading an Audit Delta.
audithist	database	Displays the Audit History form.
auditonoff	database	Displays the Audit Control form.
audlog	database	Displays the Audit Log form.
audspec	database	Displays the Audit Specification Table form.
autoshut	database	Displays the Automatic Shutdown Information form.
bulletin	database	Displays the System Bulletin form.
calduty	database	Displays the Normal Working Hours form.
calholiday	database	Displays the Holiday Names form.
call	exercise	Displays the Application Exerciser form for testing/running a RAD application directly.
callqueue	sc.setup.manage	Displays the Interaction) queue.
chgqueue	sc.setup.manage	Displays the Change queue.
cls	pm.access	Displays the Incident queue.
cm3grp	database	Displays the Change Management Groups Definition form.

cm3msg	database	Displays Change Management Event Definition form.
cm3profile	database	Displays the Change Management (ChM) Security Profile form.
cm3r	se.search.engine	Displays the search form for Change Management requests.
cm3rcat	cm3r.category.maint	Displays the Change Management Request Category form.
cm3renv	se.search.engine	Displays the Change Management Application Environment form.
cm3ropen	cmc.open.from.menu	Initiates the script for opening a new change request record.
cm3t	se.search.engine	Displays the search form for Change Management tasks.
cm3tcat	cm3t.category.maint	Displays the Change Management Task Category form.
cm3tenv	se.search.engine	Displays the Change Management Application Environment - Tasks form.
cm3topen	cmt.open.from.menu	Initiates the script for opening a new change task.
cmcontrol	cm.edit.config	Displays the Contract Management Configuration form.
cmdlist	report.exerciser	Executes the Menu Command List report via Report Exerciser.
comp	se.search.engine	Displays the System Information Definition form (system Company record).
compare	compare.applications	Displays the RAD Application Comparison form.
configure sla	sla.edit.config	Displays the Service Level Agreement Control form.
contacts	database	Displays the Contacts search form.
copyfile	copy.database.file	Initiates the script for copying a database file.
createschd	database	Displays the Schedule form.
curalerts	se.search.engine	Displays the Alert search form.
curapprovals	se.search.engine	Displays the Approval search form.
curconvert	database	Displays the Currency Conversion search form.
currency	database	Displays the Currency form.
datamap	database	Displays the Data Map form.
db	database	Displays the Database Manager form.
dbdict	dbdict.utility database	Displays the Database Dictionary prompt.

de	database	Displays the Display Application Event Definition form.
delmail	database	Displays the form for sending internal Service Manager broadcast mail.
dist	database	Displays the Distribution Group form.
do	database	Displays the Display Application Option Definition form.
doc	menu.manager	Displays the menu with options for maintaining the document engine. **Note** Note This command line call will only work if the menu forms are being displayed.
ds	database	Displays the Display Application Screen Definition form.
edit gkn	database	Displays the form for editing core knowledge.
evemail	sca.window	Displays the form for sending an email event.
eventbld	axces.build.maps	Initiates the wizard for building event mappings.
eventfltr	database	Displays the Event Filters form.
eventin	database	Displays the Event Services input queue.
eventmap	database	Displays the Event Map form.
eventout	database	Displays the Event Services output queue.
eventreg	database	Displays the Event Registration form.
evicmbld	scauto.build.maps	Initiates a script for building Configuration Management event mappings.
evwrite	sca.window	Displays the form for sending an Incident Management, Configuration Management, or generic event.
expline	database	Displays the Expense Line Information form.
fc	format.ctrl.maint	Displays the Format Control form.
fd	forms.designer	Displays the Forms Designer form.
files	database	Displays the Configuration File form for database unloads.
formatctrl	format.ctrl.maint	Displays the Format Control form.
forms	forms.designer	Displays the Forms Designer prompt.

gen core	ke.gencore.verify	Displays the prompt for generating the core Knowledge Base.
gl	database	Displays the global list form.
hd	pm.access	Displays the Incident queue.
htopic	database	Displays the Help topic search form.
imapplenv	database	Displays the Incident Management Environment Profile form.
imassign	database	Displays the Assignment Groups form.
imbuild	pm.build.probsum	Initiates the script for the building of probsummary records.
imconvert	pm.convert.format	Displays the Convert Incident Format form.
imgroups	database	Displays the Incident Management Groups form.
improfile	database	Displays the Incident Management Security Profile form.
imreset	pm.clear.downtime	Allows for the clearing of availability information.
inact	kill.inactive.setup	Displays the Kill Inactive Users Setup Parameters form.
incidentqueue	sc.setup.manage	Displays the Incident queue.
info	database	Displays the Background Processor Initialization Registry form.
irq	ir.query.window	Displays the form for issuing an IR Query File form.
knowledge	get.search.application	Displays the search form for Knowledgebase information.
link	edit.link	Displays the Link File form.
linklist	report.exerciser	Executes the Link File Listing report via Report Exerciser.
load transfer	apm.upgrade.load.transfer	Initiates the Service Manager Upgrade Utility Load Transfer form.
location	database	Displays the Location form.
logmsg	database	Displays the “log” type Message Class form.
mailmsg	database	Displays the “mail” type Message Class form.
mcf	database	Displays the Capability Word form.
menu	database	Displays the Menu form.
message	database	Displays the Message form.

model	database	Displays the Configuration Item (CI) Model search form.
modelicm	database	Displays the Configuration Item (CI) Model search form.
modelven	database	Displays the Model Vendor Information form.
monitor	system.monitor	Displays the system status form.
msg	database	Displays the generic Message Class form.
msglog	database	Displays the Message Log form.
msgtype	database	Displays the Message Type form.
new call	cc.first	Displays a blank form to create a Service Desk interaction.
new incident	apm.first	Displays a blank incident form for record entry.
note	database	Displays the Notification Definition form.
number	database	Displays the Sequential Number form.
ocmbldlvl	ocm.co.bld.lvls	Displays the prompt for updating all of the level numbers for all components in the model file.
ocmco	database	Displays the Configuration Item (CI) Model search form.
ocmdeliv	report.exerciser	Executes the Delivery Forecast for Next 7 Days report via Report Exerciser.
ocmdreqj	report.exerciser	Executes the Daily Quote Log report via Report Exerciser.
ocmdrj	report.exerciser	Executes the Daily Receipt Journal report via Report Exerciser.
ocmevents	database	Displays the Request Management Events form.
ocmgroups	database	Displays the Request Management Group Definition form.
ocml	ocml.access	Displays the search form for Request Management line items
ocmlcat	database	Displays the Request Management Line Item Category form.
ocmlcreate	ocm.category.create	Displays the Request Management Line Item Category form.
ocmlenvir	query.stored	Displays the Request Management Line Items Application Environment form.
ocmlmast	database	Displays the Request Management Catalog Select Categories form.

ocmlphase	database	Displays the Request Management Line Item Phase form.
ocmlrec	database	Displays the Request Management Receiving Log form.
ocmo	ocmo.access	Displays the search form for Request Management orders.
ocmoappr	ocmo.access	Displays a list of orders that have been approved.
ocmocat	database	Displays the Request Management Order Category form.
ocmocavail	database	Displays the Request Management Check Availability Order Generation Schedule Record.
ocmocreate	ocm.category.create	Displays the Request Management Order Category form.
ocmodemand	database	Displays the Request Management Background Order Generation Schedule Record form.
ocmoenvir	query.stored	Displays the Request Management Orders Application Environment form.
ocmoopen	ocmo.access	Initiates the script for creating a new Request Management order.
ocmophase	database	Displays the Request Management Order Phase form.
ocmpo	report.exerciser	Executes the Print Purchase Orders report via Report Exerciser.
ocmprofile	database	Displays the Request Management Profile form.
ocmq	ocmq.access	Displays the search form for Request Management quotes.
ocmqcat	database	Displays the Request Management Quote Category form.
ocmqcreate	ocm.category.create	Displays the Request Management Quote Category form.
ocmqopen	ocmq.access	Initiates the script for creating a new Request Management quote.
ocmqphase	database	Displays the Request Management Quote Phase form.
ocmvenper	report.exerciser	Executes the Vendor Performance Based on Line Item Dates report via Report Exerciser.
oncall	database	Displays the On Call Schedule contact form.
oncallsched	database	Displays the On Call Schedule calendar form.

operator	database	Displays the Operator form.
opn	pm.access	Displays the Incident queue.
overspent	query.stored	Executes the "overspent contracts" query which displays a list of all service contracts that have exceeded their financial budget.
password	password.change	Displays the prompt for changing the login password.
printappl	report.exerciser	Executes the Print of Application report via Report Exerciser.
prtmsg	database	Displays the "print" type Message Class form.
purgarch	pa.main.appl	Displays the Purge/Archive form.
purgeaudit	audit.purge	Displays the form for purging audit history.
RAD	encl.appl	Displays the RAD Editor prompt.
rad	encl.appl	Displays the RAD Editor prompt.
recalc partials	sla.recalc.totals	Displays the prompt for recalculating SLA information over a given time frame.
review cc	cc.setup.manage	Displays the Service Desk Interaction queue.
review im	apm.setup.manage	Displays the Incident queue.
rmail	read mail	Displays the prompt for selecting which type of internal Service Manager mail to read.
run	exercise	Displays the Application Exerciser form for testing/running a RAD application directly.
rv	pm.access	Displays the Incident queue.
sch	database	Displays the Schedule File form.
schedule	database	Displays the Schedule File form.
schmail	mail.set.schedule	Displays the form for scheduling background mail.
scknowledge	get.search.application	Displays the Knowledgebase search form.
scmsg	database	Displays the Service Manager Message form.
scripts	script.maint	Displays the Script Definition form.
search cal	cc.search.incidents	Displays the search form for Service Desk interactions.
search calls	cc.search.incidents	Displays the search form for Service Desk interactions.
search incident	apm.search.problems	Displays the search form for Incident Management incidents.

search sla	sla.search.objects	Displays the search form for Service Level Agreements.
servcont	database	Displays the Service Contract form.
shutdown	system.shutdown	Initiates the process for shutting down the Service Manager server.
smail	mail.send.appl	Displays the prompt for confirming the sending of internal Service Manager mail.
smapplenv	database	Displays the Service Desk Environment form.
smgroups	database	Displays the Service Desk Groups form.
smprofile	database	Displays the Service Desk Security Profile form.
software	database	Displays the Software Management form.
sol cans	se.search.engine	Displays the Pending Knowledge Items form.
spool	spool.scheduler	Displays the Spoolheader form.
sq	database	Displays the Stored Query Maintenance form.
start	apm.upgrade.main	Initiates the Upgrade Utility wizard.
status	system.status	Displays the system status form.
stdptrs	database	Displays the Configuration File form for standard printers.
subtotals	database	Displays the Subtotal Definition form.
syslog	database	Displays the System Log form.
task	query.stored	Displays a list of the user's change tasks.
terminals	database	Displays the Terminal Configuration form.
triggers	database	Displays the triggers form.
tskqueue	sc.setup.manage	Displays the Task queue.
unload	us.unload	Displays a list of unload scripts.
uoperprof	query.stored	Displays the Operator Status Display form.
upd	pm.access	Displays the Incident queue.
userlevel	us.userlevel	Displays the current user access level and the options for changing this access.
userstats	report.userstats	Displays the prompt for executing the Usage Statistics report.
usrcmr	cm3r.main	Initiates the script for opening a new change request record.
usrdir	database	Displays the Contacts form.

usricm	icm.access	Displays the Configuration Item Search form.
usrknow	database	Displays the search form for Knowledge Base information.
usrocma	ocmo.access	Displays a list of the current Request Management orders.
usrocmo	ocmo.access	Displays the search form for Request Management orders.
validity	se.search.engine	Displays the Validity Table Specifications form.
vendor	database	Displays the Vendor/Manufacturer search form.

Contents

- 1 Generic command line calls
- 2 Direct command line calls

2.1.9. Call a RAD application

You can call a RAD application from the following places in Service Manager:

- Display Options
- Format Control
- Scripts
- Processes
- Triggers

When you call a RAD application, you may need to pass the required parameters. These parameters consist of a parameter name and a value.

Note

Note Service Manager doesn't currently support calls from JavaScript on any RAD application that uses the rio/fdisp panels. Avoid calls from JavaScript on RAD applications that use the rio or fdisp panels. If you can't avoid doing so, always set the RunInNewThread parameter to true. This doesn't work for all cases.

To find view the parameters, follow these steps:

1. Locate the RAD you want to use. See Locate a RAD function.
2. Click **View** when you have the RAD application panel displayed. This displays a list of parameter to pass.
3. Position the cursor on each parameter and then click Help to view the name of the field used as the parameter.

Note

Note To display the field help, make sure either of the following conditions is fulfilled:

- In the Windows client, click **Preferences** > **Service Manager** > **Appearance** and then select the **Show context-sensitive help debug information** option.
- In the Web client, viewcontexthelp=true is appended to the URL before you log in.

2.1.10. Locate a RAD function

To locate a Rapid Application Development (RAD) function, follow these steps:

1. Click **Tailoring** > **Database Manager**.
2. Type `=application` in the **Form** field, and then click **Search**.
3. Click **More** or the **More Actions** icon and then select **Expert Search**. Service Manager displays a query window.

4. Type one of the following in the **Query** field:

Desired search	Syntax

Search for a function that's not an `rtecall` function.	`index("<function name>",`
	`str(contents(currec()))>0`
	Example:
	`index("jscall", str(contents(currec()))>0`
	Returns all the RAD application panels that contain the `jscall` function (to call a JavaScript function).

Search for all instances of a particular `rtecall` function. For this query to run normally, a backslash must precede each double quotation mark within the `rtecall` parentheses.	`index("rtecall(\"<function name>\"",`
	`str(contents(currec()))>0`
	Example:
	`index("rtecall(\"rinit\"", str(contents(currec()))>0`
	Returns all the RAD application panels that contain the `rinit` function (to initialize a file).

Use this syntax to search for all instances of all `rtecall` functions beginning with a particular letter (for example, the letter `r`). For this query to run normally, a backslash must precede the first set of double quotation marks. The closed parenthesis is unnecessary in this syntax.	`index("rtecall(\"<first letter>",`
	`str(contents(currec()))>0`
	Example:
	`index("rtecall(\"s", str(contents(currec()))>0`
	Returns all the RAD application panels that contain a function that starts with the letter `s` such as `sort`.

5. Click **Search**. Service Manager displays a record list of RAD panels matching the search criteria.
6. Double-click on a panel to display that instance of the function.

2.1.11. RAD Debugger

The RAD Debugger enables you to view and isolate RAD processes for the purpose of troubleshooting. Information about the RAD flow opens in a separate window containing a scrollable text field. The RAD Debugger uses its own command line language to trace, display values, and set break points.

Start RAD Debugger

To start the RAD Debugger, follow these steps:

1. Click **Window** > **Show View** > **Other** > **Service Manager Administrator** > **RAD Debugger**.
   The RAD Debugger command window opens.
2. Type the desired command followed by a parameter.
3. Press **Enter**.

RAD Debugger commands

The following is a list of the commands used in the RAD Debugger to troubleshoot an application:

Command	Description
d (display)	Displays the contents of a variable. A common use of this command is to display the name of the Display application Screen ID attached to the current form. `d $L.screen`
dd	Displays the contents of a variable in the structured format. `dd $L.file`
dq	Displays the tag query of a variable. `dq $L.file`

t (trace)	Turns tracing on or off. Tracing allows you to see every panel that the RAD flow encounters. Use the command by itself to display the status of the trace function.
	`t on`
	`t off`
	`t`

ta	Turns panel tracing on or off for a specific RAD application. To turn application tracing on, include the name of a RAD application. To turn application tracing off, repeat the same command combination a second time. You may have multiple traces engaged at one time. Use the command by itself to display a list of the RAD applications being traced.
	`ta script.execute`
	`ta`

tar	Removes all RAD specific panel traces. If you are tracing multiple applications, you can save time by using tar rather than using ta to turn off each trace individually.

b (breakpoint)	Sets a panel breakpoint. When the RAD flow encounters a panel with this label, it halts before executing the named panel and gives you an opportunity to perform debugging procedures such as displaying variables or executing statements. Repeat the command combination a second time to turn off the breakpoint. Execute the command by itself to display a list of all current breakpoints.
	`b init.operator`
	`b`

ba	Sets a breakpoint on a specific RAD application. A breakpoint is applied each time the RAD flow enters the named application, either when it runs for the first time or when the flow returns to it from a subroutine. `ba menu.manager`

bt	Sets a breakpoint on a specific type of RAD panel. When the RAD flow encounters a RAD command panel with this name, the application stops running and returns you to the RAD Debugger. Enter the command combined with the name of a RAD panel to set the breakpoint. Re-enter the same command combination to turn off the breakpoint. `bt rinit`
bv	Set a breakpoint when a variable changes.
rb	Removes all breakpoints of any type.
go	Resume execution after a breakpoint.
c (continue)	Resumes the execution of the RAD flow after a breakpoint occurs.
s (step)	Steps through the RAD flow one panel at a time.
gl (globals)	Displays all the global variables: those beginning with `$G.` , `$lo.` , `$CHART.` , `$SYSPUB.` , `$MARQUEE` .
v (variables)	Displays all the thread variables: those not beginning with a special code.
l (locals)	Displays all the local RAD variables: those beginning with `$L` .
sta (stack)	Displays the current RAD stack.
re (relations)	Displays only the variables which are table relations (file handles initialized by the rinit command panel).
m (memory)	Displays all the variables in memory. This is equivalent to issuing the globals (**g**), variables (**v**), and locals (**l**) commands.
x	Execute a statement.
display	Display name - view a variable.
h (help)	Displays a brief summary of the RAD Debugger commands.

debugadhocsql	Switch debug adhocsql
	Switch on :
	`debugadhocsql 1`
	Switch off :
	`debugadhocsql 0`

sqldebug	Set sqldebug level [0-3]
rtm	Set response time monitor value [0-5]

2.2. Introduction to JavaScript in Service Manager

Similar to RAD, JavaScript in Service Manager provides the ability to query, insert, update, and delete database records, and to make use of existing RAD variables and functions. The principal benefit of using JavaScript is that it uses an industry-standard scripting language to execute commands instead of a proprietary language. JavaScript within Service Manager is implemented using the Mozilla® SpiderMonkey (JavaScript–C) Engine.

For detailed information about the syntax of Mozilla SpiderMonkey JavaScript, refer to:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

For information about the SpiderMonkey project, refer to:

https://developer.mozilla.org/en-US/docs/Mozilla/Projects/SpiderMonkey

2.2.1. Language overview

JavaScript is a scripting language which is different from programming languages like Java™, C++, or Visual Basic®. It is a smaller, dynamically typed, scripting language that offers programming tools with easy syntax and built-in functionality.

Unlike programming languages, scripting languages don't use compilers to create executable program code. Because JavaScript is an interpreted language, each time the script is executed, it is loaded into an interpreter that runs the code. The Compile button in the Script Library doesn't create an executable load module; it runs a syntax check to verify that the script is executable.

JavaScript can be either client-side, server-side, or core language. Core JavaScript is the base JavaScript language with client-side and server-side JavaScript as extensions. Server-side JavaScript is used for accessing and manipulating data and is the only one available within Service Manager.

2.2.2. Learning JavaScript

We don't endorse these references but provides them as a courtesy to our customers and partners. These are helpful resources that contain information about JavaScript.

- For JavaScript basics, refer to the JavaScript tutorial(s) at W3 Schools.
- The Mozilla SpiderMonkey (JavaScript-C) Engine project.
- *JavaScript: The Definitive Guide, 5th Edition* by David Flanagan (ISBN: 0596101996).

For best results, fully understand the programming language prior to attempting to use it with Service Manager.



Note

Note

There is a difference between server-side and client-side JavaScript. Service Manager only supports server-side JavaScript. Therefore, any JavaScript that addresses output is unavailable in the Service Manager environment.

2.2.3. Basic rules of JavaScript

The following rules apply to the basic syntax of JavaScript:

- JavaScript is case-sensitive.
- Statements should end in a semicolon (;).
- Variables:
  • Must be defined before being used. The variable name can contain A-Z, a-z, underscore, or digits and must start with a letter or an underscore ("_").
  • Assume the type of data that's put into the variable. The data type doesn't have to be explicitly defined
  • Are global variables when defined outside of a function, and are available anywhere in the current script context. Variables created within a function are local variables, and can be used only within that function.
- Strings have to be enclosed in quotation marks, either a single or double. For example: `print("Hello" + 'world' + Country.name)` produces the following: Hello world US.
- Special characters that are displayed literally must be preceded by a backslash character (\). Quotes within a string can be entered preceded by a backslash as well. See the *Core Language Features - Literals - "String literals"* section for more information in https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide.
- To increment a variable, such as a = a + 1, you can use a++. You can decrement a variable in the same way, as in a--.
- To enter comments in the script, use "//" to start a single line comment or the combination of "/*" and "*/" to enclose a multi-line comment.
- Values that aren't defined as a data type (string, number, Boolean) may be defined as an object, such as Date, Array, Boolean, String, and Number. As an example, you could define: `var ArrayList=new Array("test", "this", "list");`.
- Dots in Service Manager field names must be replaced by an underscore (_) in JavaScript. For example, contact.name becomes contact_name.
- Service Manager field names that are reserved words in JavaScript have to be preceded by an underscore, such as "_class" for the "class" field.

Note

Note

Starting from Service Manager 9.70, the following enhancements are introduced on JS engine integration:

- All SM extended JS classes, for example, SCFile/SCDatum/XML/Script/Query/Record/Vars/Record, now fully support the prototype-chain-based inheritance mechanism in JavaScript.
- The 'Reflect' and 'Debugger' extensions of SpiderMonkey JS are now enabled on the global object.
- When a Javascript error stops the code execution, SM server will display file name and line number of the JS source code in the error message. If there being any JS exception in the code execution, SM server will further return the detailed JS source code call stack and log the information in the sm.log file.
- Jasmine v3.x is now fully supported as the JavaScript testing framework. The `spyOn()` function provided by Jasmine is supported for 'SCFile/SCDatum' SM extension classes.

Structure of a JavaScript application

Refer to the following example for the structure of a JavaScript application.

```
< Global variable declarations; >
function < function name > ( < parameters > ) {
    <
    local variable declarations; >
    <
    Statements; >
} <
function calls; >
```

Expressions and operators

For more information on this topic, refer to the *Expressions and Operators* section in:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

The list of operators includes assignment operators, comparison operators, arithmetic operators, bitwise operators, logical operators, string operators, and special operators.

JavaScript objects and properties

The key to understanding JavaScript is learning the structure of its objects. Most JavaScript functionality is contained in objects. The objects contain methods, parameters, and events. Each object may contain a set of properties that more closely define the object. The syntax you use to call methods and properties is: Object.method or Object.property.

For more information about JavaScript objects and properties refer to the *Details of the Object Model* section in :

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Built-in objects and functions

Built-in objects described in this section are Array, Boolean, Date, Function, Math, Number, and String.

Each of these is described in detail in the *Working With Objects - Predefined Core Objects* section of:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Conditional statements

The following example returns true if the date entered is within the number of days in a month (here, 30 days); otherwise, it returns false.

Note

Note Pay attention to the use of the conditional statement to shorten the "if" expressions. The `var return = (parm.Date.getDate <= 30) ? true : false;` statement works in the same way as its much longer equivalent:

```
var result = false;
if (parm.getDate() <= 30) {
    result = true;
} else {
    result = false;
}
```

While and loop statements

Note

Note Whenever programming a loop statement, ensure that the exit condition will be met; otherwise, an infinite loop will occur.

For detailed information about the for and while loop statements, refer to the *Statements - Loop Statements* section in:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

The following example shows how to check how many times a given number can be divided by 2, first using a `for` loop, and then using a `while` loop:

```
function DivisableByTwoFOR(ParmNumber) {
    for (var i = 0; ParmNumber > 1; i++) {
        ParmNumber = ParmNumber / 2;
    }
    return i;
}

function DivisableByTwoWHILE(ParmNumber) {
    var i = 0;
    while (ParmNumber > 1) {
        ParmNumber = ParmNumber / 2;
        i++;
    }
    return i;
}
```

Note

Note A `Do..While` loop is available for those instances when you want to execute a statement (or block of statements) at least once. In a `Do..While` loop, the exit condition is checked after the statements are executed.

Caution

Caution The `for...in` loop over an array may return unexpected results. For details, see Avoiding the use of the for...in loop over an array.

Break and continue statements

The break statement terminates a `while` or `for` loop completely. The continue statement terminates execution of the statements within a `while` or `for` loop and continues the loop in the next iteration. The following two examples demonstrate how these statements are used.

Note

Note Both these statements violate the rules of structured programming but are nonetheless widely used in special cases.

```
function break4error(parmArray, x) {
    var i = 0;
    var n = 0;
    while (i < 10) {
        if (isNaN(parmArray[x])) {
            print(parmArray[i].toString + "is not numeric.Please repair and run again");
            break;
        }
        n = n + parmArray[i];
        i++;
    }
    return n;
}
```

Modifying the above example, the continue statement would work as follows:

```
function break4error(parmArray, x) {
    var i = 0;
    var n = 0;
    while (i < 10) {
        if (isNaN(parmArray[x])) {
            print(parmArray[i].toString + "is not numeric.Continuing with next number");
            i++;
            continue;
        }
        n = n + parmArray[i];
        i++;
    }
    return n;
}
```

Exception handling statements

As a best practice, exception handling should always be included in any JavaScript program. The following code provides an example of using exception handling statements for JavaScript.

```
function SetFullName(FirstName, LastName) {
    if (FirstName != null || LastName != null) {
        FullName = FirstName + "" +LastName;
        return FullName;
    } else {
        throw "InvalidName";
    }
}
try {
    // statements to try
    var MyName = SetFullName(First, Last); // function could throw exception
} catch (e) {
    MyName = "unknown";
    logMyErrors(e); // pass exception object to error handler
}
```

For detailed information about exception handling, refer to the *Statements - Exception Handling Statements* section in:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

2.2.4. Using JavaScript in Service Manager

JavaScript implementation in Service Manager enables complex tailoring. However, JavaScript has specialities that should be respected. The following sections discuss a few of them.

2.2.4.1. JavaScript tailoring guideline

The embedded Mozilla SpiderMonkey JavaScript engine was upgraded to a higher version in the 9.70 server. In order to avoid any potential problems, you need to check your existing scripts and pay attention when you add new scripts.

To check your existing custom scripts:

1. Go through your existing scripts and make sure that all custom scripts comply with the following guideline.
2. Run a Functional Test and a Performance Test and check the sm logs. Fix the remaining issues as required.

JavaScript native object prototype tailoring

The upgraded JavaScript engine supports more methods on JavaScript native objects. If you have added additional prototypes on JavaScript native objects before, some of these scripts may conflict with the newly supported methods and cause errors.

Note

To check your existing custom scripts, type `fa` in the command line, then input `String.prototype.` or `Array.prototype.` in the search field and click Search. Check all search results for tailoring code.

For example, the following scripts may cause errors because the `startsWith()` and `indexOf()` methods are already available in the upgraded JavaScript engine.

✗ `String.prototype.startsWith = function(str){...}`
✗ `Array.prototype.indexOf = function(obj){...}`

String replace() function

The upgraded JavaScript engine supports 2 parameters for regexp instead of 3 parameters.

Note

To check your existing custom scripts, type `fa` in the command line, then input `.replace(` in the search field and click **Search**. Check all search results for tailoring code.

For example:

✗
```
cond = cond.replace("", "\\"", "g");
```
✓
```
cond = cond.replace(/\"/g, '\\"');
```

✗
```
strTarget = strTarget.replace(strOld, strNew, "g");
```
✓
```
var regexp = new RegExp(strOld, "g");
strTarget = strTarget.replace(regexp, strNew);
```

Apart from the changes brought by the JavaScript engine upgrade, you also need to pay attention to the following JavaScript changes:

Global Method: base64Decode

If the value of the `data` parameter defined in `base64Decode()` is not a valid base64 string, the `base64Decode()` method will throw an exception with the "Exception occurred when calling JSBase64Decode" error message.

Note

To check your existing custom scripts, type fa in the command line, then input "base64Decode" in the search field and click **Search**. Check all search results for tailoring code.

To avoid any potential problems, you need to use the try-catch block to catch such exceptions for the base64Decode() method.

For example:

```
try {
   var str = base64Decode(data);
} catch (e) {
   // Take some actions to handle such exceptions
}
```

2.2.4.2. Avoiding the use of the for...in loop over an array

When tailoring Service Manager, you often need to iterate over an array. However, using a `for...in` loop to iterate over an array may return unexpected results, particularly, in the two scenarios illustrated below. We recommend that you use the `for ([initialization]; [condition]; [final-expression])` loop instead when iterating over an array object.

Enhancing an array object with custom methods

Service Manager currently uses a JavaScript Engine that complies with JavaScript 1.5, in which some `Array.prototype` functions are implemented as built-in functions. When enhancing an array object with a custom method, the `for...in` loop iterates on all items in the array including the custom method. Similar issue also occurs to customized `Array.prototype` functions, as demonstrated in the following example code and its output. This behavior might lead to errors that are difficult to debug.

```
Array.prototype._prototypeFunction = function() {
    // prototype function will be iterated as non built-in property
    print('prototype element');
}
print('built-in  property: ' + array.length);
// length is built-in property of array object, it will not be iterated within for…in statement
for (var item in [1]) {
    print(' non built-in  properties of Array type:' + item);
    //iterate the non built-in properties
}
```

Output:

```
non built-in  property of Array type: _prototypeFunction
// prototype function added as non built-in
non built-in  property of Array type: 0
built-in  property: 4
```

In this situation, you can use a guarding `if` statement to help debug the code. For example:

```
if (<array> .hasOwnProperty(<item index>))
```

or

```
if (typeof(<array>[<item index>]!="function")
```

Iterating out of order

The `for...in` loop iterates objects in an order according to the ASCII values of the property names. Do not use the `for...in` loop to iterate over an array when the index order is important. See the following example code and its output.

```
var array = [1,2,3];
array['_'] = 'test';
// interrupting element
array[10] = 10;
for(var item in array) {
      print(' non built-in  properties of Array type:' + item);
      //iterate the non built-in properties
}
```

Output:

```
non built-in  property of Array type : 10
// Missing index 3~9!
non built-in  property of Array type: _
//interrupt the element index order
non built-in  property of Array type: 2
non built-in  property of Array type: 1
non built-in  property of Array type: 0
```

For more information about using the `for...in` statement, refer to https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for...in.

2.2.4.3. Avoiding variable declaration inside a loop

JavaScript is a language with a dynamic garbage collection. Memory allocated even if inaccessible, will not be given free before the garbage collector deallocates it.

If a variable is declared inside a loop, JavaScript will allocate fresh memory for it in each iteration, even if older allocations will still consume memory. While this might be acceptable for variables of scalar data types, it may allocate a lot of memory for SCFile variables, or huge arrays or objects.

Unrecommended implementation

The following implementation, in which a variable is declared inside a loop, is not recommended.

```
var arr = ["IM10001", "IM10002", "IM10003", "IM10004"];
var i;
for (i = 0; i < arr.length; i++) {
    var file = new SCFile("probsummary");
    file.doSelect('name="' + arr[i] + '"');
    // do something with file
}
```

Recommended implementation

The following implementation, in which a variable is declared before a loop, is recommended.

```
var arr = ["IM10001", "IM10002", "IM10003", "IM10004"];
var file = new SCFile("probsummary");
var i;
for (i = 0; i < arr.length; i++) {
    file.doSelect('name="' + arr[i] + '"');
    // do something with file
}
```

2.2.4.4. Counting records

Counting the record set returned by a query is a frequent task in tailoring. Instead of fetching each record in a loop, you can use the Service Manager `rtecall("count")` more efficiently.

Recommended implementation using the doCount() method

```
function countFile(filename, query) {
    var file = new SCFile(filename);
    var nCount = file.doCount(query);
    var rc = getLastRC();
    if (RC_SUCCESS == rc) {
        return nCount;
    } else {
        print("Could not count records. " + RCtoString(rc));
        return -1;
    }
}
```

Recommended implementation using system.functions.rtecall("count")

```
function countFile(file) {
    var iTotal = new SCDatum();
    var returnCode = 1;

    funcs.rtecall("count", returnCode, file, "", iTotal);

    return iTotal.getText();
}
```

2.2.4.5. Return bad validation message in Format Control

When you run Format Control through JavaScript, you can use the following JavaScript statements to display the detailed error messages if data validation fails.

`funcs.rtecall("msgonoff", ret, 1); // turn on the Message switch`

`funcs.rtecall("msgonoff", ret, 0); // turn off the Message switch`

Example

```
var ret = new SCDatum();
funcs.rtecall("msgonoff", ret, 1);

var radResults = lib.RAD.run("se.get.object", argNames, argVals, argTypes, results);

funcs.rtecall("msgonoff", ret, 0);
```

2.2.4.6. Avoiding the use of getLast()

The `getLast()` method for an SCFile object will select the last record of the list of results to the currently selected records. However, accessing the last record in a long list of records is not really a good idea. Since Service Manager selects always a block of unique keys, this method requires Service Manager to blockwise step through the complete result list, until it gets to the last one and then fetches its contents.

Unrecommended implementation

The following information is not recommended:

```
var file = new SCFile("probsummary");
file.doSelect('true');
file.getLast();
```

Alternatively, you can set the result order by the `setOrderBy()` method at the time of the select. You can change the sort order that the record you are looking for is no more the last in the list, but the first.

Recommended implementation

The following implementation is recommended:

```
var file = new SCFile("probsummary");
file.setOrderBy(["number"],[SCFILE_DSC]);
file.doSelect('true');
```

2.2.4.7. Restricting fields selected by a query

When looping on a larger set of records by a query, a huge amount of memory may be allocated for each record. At the same time, a dbdict might be mapped to multiple tables on the RDBMS. Service Manager by default fetches the complete records and therefore selects against each of these tables.

In JavaScript implementations, you can use read-only file variables and restrict them to a set of fields only. Using this option will significantly reduce memory consumption, as well as select against tables not containing any of these fields.

Recommended implementation

```
var file = new SCFile(filename, SCFILE_READONLY);
var query = "true";
var fields = ["number", "status"];
file.setFields(fields);

var rc = file.doSelect(query);

while (RC_SUCCESS == rc) {
    // Do something with file here
    rc = file.getNext();
}
```

When using the `setFields()` function on a file variable that's not read-only, the query is executed against all fields and this message is written to the log file:

`JS W SCFile.setFields called on non readonly SCFile("number", "status")- ignored`

2.2.4.8. Avoiding the use of assignment instead of comparison

A common error in JavaScript code is the use of `=` (assignment), when a `==` (comparison) is actually intended.

Example of this issue

```
while (rc = RC_SUCCESS) {
    // Do something with file here
    rc = file.getNext();
}
```

A good practice to avoid this issue proactively, is to position the unchangeable value at the left side.

Recommended implementation

```
while (RC_SUCCESS == rc) {
    // Do something with file here
    rc = file.getNext();
}
```

2.2.4.9. Accessing system language array data

The Service Manager system language and JavaScript support the array-data type. However, the implementation of this data type is different. For that reason, the access to the elements inside the array is different as well.

While Service Manager allows `array[<index>]` access to system language arrays from JavaScript, it doesn't allow to access sub items in an array of array like this: `array[<index1>][<index2>]`.

However, it is possible to access the sub items in an array of array using this syntax: `array["<index1>.<index2>"]` instead.

Example

```
// Create a nested system language array
var array1 = new SCDatum();
array1.setType(8);
array1.push("1_1");
array1.push("1_2");
array1.push("1_3");

var array2 = new SCDatum();
array2.setType(8);
array2.push("2_1");
array2.push("2_2");
array2.push("2_3");

var nestedArray = new SCDatum();
nestedArray.setType(8);
nestedArray.push(array1);
nestedArray.push(array2);

print("nestedArray: " + nestedArray);
print("Now printing the nested Arrays sub-arrays: ");
print("nestedArray[0]: " + nestedArray[0]);
print("nestedArray[1]: " + nestedArray[1]);

print("Now printing the elements of the first sub-array: ");
print('nestedArray["0.0"]: ' + nestedArray["0.0"]);
print('nestedArray["0.1"]: ' + nestedArray["0.1"]);
print('nestedArray["1.2"]: ' + nestedArray["1.2"]);
```

2.2.4.10. Working with system variables

The Service Manager System Language provides different types of variables identified by their names. Variables of each type have different behaviors and scopes, for example, local to RAD application, global inside RAD thread, and global inside session. For more information, see Variable pools. This may affect the following JavaScript implementation:

- Function calls in JavaScript use call-by-reference. That is, when calling a function with a variable as parameter, the value of the variable is NOT copied, but only referenced.
- Calling **doAction()** function from JavaScript will call the Document Engine, and by this MAY CHANGE non-local variables such as `$file`. That is, when calling a JavaScript function from Format Control, and calling **doAction()** inside this function might corrupt the `$file` variable when control is returned to Format Control.

Example of the issue

The following is an example of this issue:

```
function test() {
    var file = new SCFile("probsummary");
    var rc = file.doSelect('number="IM10001"');

    if (rc == RC_SUCCESS) {
        file.job_name = "test";
        file.doAction("save");
    } else {
        print("JS testFC.test():Select of incident IM10001 failed! ");
    }

}
```

Format Control "device", tab page "Javascript" to be executed on display:

```
print("1. Filename:" + funcs.filename(vars.$file));
lib.testFC.test();
print("2. Filename:" + funcs.filename(vars.$file));
```

To

reproduce the issue, follow these steps:

1. In the System Navigator, navigate to **Configuration Management** > **Resources** > **Search CIs**.
2. The following messages appear:

   ```
   2. Filename:probsummary
   US/Mountain 01/31/14 09:35:29: Incident IM10001 has been updated by falcon
   1. Filename:device
   ```

Recommended implementation

Back up the `$file` variable before the function call, and restore afterwards. Change the Format Control "device" implementation in the example above like this:

```
print("1. Filename:" + funcs.filename(vars.$file));
var backup = new SCFile(funcs.filename(vars.$file));
var rc = funcs.fduplicate(backup, vars.$file);
lib.testFC.test();
vars.$file = backup;
print("2. Filename:" + funcs.filename(vars.$file));
```

2.2.4.11. Accessing a JavaScript class definition from another JavaScript

Suppose you have two different JavaScript libraries and you don't want to copy the code from one library to the other just to make a JavaScript class definition available to the other one. For example, you define a MyClass class in Test2, then you instantiate MyClass in Test3, and then you call a function in Test4 passing the instantiated object in Test3 to that function; additionally, you may want the function in Test4 to be able to check if the received object is actually an instantiation of the class defined in Test2. You then need to make Test3 to recognize MyClass.

To achieve this behavior, in the Test2 file, you can add a function that returns the class itself, instead of an instantiated object of the class. If you call this function from Test4, you get a pointer to the class definition.

See the following for the sample scripts.

Test2

```
var MyClass = function() {
    this.var1 = 'variable 1';
    this.var2 = 'variable 2';
};
MyClass.prototype.getVariable1 = function() {
    return this.var1;
};
MyClass.prototype.getVariable2 = function() {
    return this.var2;
};

function require() {
    return MyClass;
};
```

Test3

```
var MyClass = lib.Test2.require();
var objMyClass = new MyClass();
print('Test3>', objMyClass instanceof lib.Test2.require());
lib.Test4.checkit(objMyClass);
```

Test4

```
var MyClass = lib.Test2.require();

function checkit(anObject) {
    print('Test4>', anObject instanceof MyClass);
    print(anObject.getVariable1());
    print(anObject.getVariable2());
    print(anObject.var1);
    print(anObject.var2);
};
```

2.2.4.12. Implementing custom logging

Instructions in this topic can be used for integrating logging aspect into custom implementations as well as for temporarily adding logging statements for issue analysis.

Log file

The default log file is the log file for the servlet. Writing to this file can be done like this:

- System language:

  ```
  $success=rtecall("log", $L.rc, "My message")
  ```

- SM JavaScript:

  ```
  var rc = new SCDatum();
  var success = funcs.rtecall("log", rc, "My message");
  ```

  SM JavaScript also implements the **print("My message");** function to write to the message log in the current session. This information appears in the message view of the client. These messages are printed into the servlet log as well, when the servlet was started with parameter `msglog:1`.

Sometimes you may want to write information into a log file separated from the servlet log file. This can be done using the **writeFile()** function:

```
writeFile("c:\\SM\\logs\\mylog.log", "a", "My message");
```

Note In a JavaScript string, the backslash is used as an escape character, and therefore the backslash character has to appear twice.

Converting variables to a string to print to a log file

You can convert variables into a string for logging:

- System language:

```
$s=str('1 00:00:00')
```

- SM JavaScript:

```
// converting a JavaScript value into string
var s = 1.toString();
// JavaScript implictly uses toString() method to convert a value to string
print(1);
// converting a value by System Language function into string
var s = funcs.str(1);
```

For compound data types, JavaScript function toString() returns frequently just the string representation of the data type, which is not really helpful. You can use the following statements to convert complex objects into a string:

- Converting SCFile object to XML string:

```
var f = new SCFile("dbdict");
var rc = f.doSelect('name="pmstatus"');
// like system language str() function - doesn't include fieldnames
var s1 = f.getText();
// output as XML document - this includes tags with fieldnames
var s2 = f.getXML();
```

- Converting a JS object to JSON string:

```
var s = JSON.stringify(["a", 1, {field1: "abc", field2: 2}]);
```

Writing a custom logging function in SM JavaScript

When designing a logging function, you may want to implement a log level: some messages should only be logged when debugging, but not during normal operations.

A typical log function design is as follows

```
var loglevel = "INFO";

function logDebug(s) {
    if (loglevel == "DEBUG") {
        print(s);
    }
}
```

This design of the function call has an issue:

```
logDebug("Record: " + record);
```

At the time of the call, the object "record" is converted into a string. However, when the global variables "loglevel" is not "DEBUG", the message is not printed at all. Therefore, depending on the size of "record", and the frequency of this call (that is, in a loop), this may cause significant memory consumption and execution time.

Therefore, consider the following design:

```
var loglevel = "INFO";

function logDebug() {
    if (loglevel == "DEBUG") {
        print(Array.prototype.join.call(arguments));
    }
}
logDebug("Record: ", record);
```

Now, the reference to object "record" is handed to the logDebug() function and the conversion to a string takes only place if the variable "loglevel" is equal to "DEBUG".

Out-of-Box loggers for JavaScript

There are two out-of-box loggers.

- ScriptLibrary log4js:
  The ScriptLibrary "log4js" supports logging in Log4J style to a message log.

  ```
  // Create a logger object set to log level "INFO"
  var myLogger = new lib.log4js.Log(lib.log4js.Log.INFO);
  myLogger.info("an info");
  ```

  For more information, see The log4js script.
- RTE-supported logger
  Using the getLog() global JavaScript function, the Service Manager RTE provides logger support.

  ```
  // Get a logger for name "MyLog"
  var mylog = getLog("MyLog");
  // Set the log level
  mylog.setLevel('trace');
  // Log a message for log level "trace"
  mylog.trace('trace message');
  ```

  For more information, see JavaScript global method: getLog.
  Tip There is GUI-support for this kind of loggers (using ScriptLibrary "jlog"): On the search form for ScriptLibrary, you select the "JavaScript Logger" option and you get to a dialog where you can set the log level for loggers by name.

2.3. Where can I use JavaScript in Service Manager?

JavaScript can be used in several places throughout Service Manager. These areas focus on providing maximum flexibility for users to tailor their environment through the use of JavaScript in Service Manager.



Note

Note It is a best practice that you store JavaScript in the ScriptLibrary and refer to it from other places.

You can use JavaScript in these places in Service Manager.

- Cascade updates
- Display Screens
- Display Options
- Format Control
- Interoperability (the ioaction table)
- Links
- Processes (Document Engine)
- ScriptLibrary
- Scripts
- Triggers
- Wizards

2.3.1. What's the ScriptLibrary?

The ScriptLibrary is a file within Service Manager used to create, edit, compile, test, and store scripts and functions. Service Manager can call and execute these scripts and functions from anywhere within the system. The ScriptLibrary contains JavaScript created by users. It's a central place for you to create, edit, compile, test, and store scripts and functions.

You can use some of these stored JavaScript programs to interface with third-party Web Services providers, such as real-time currency conversions. All these JavaScript are identified as SOAP packages within the ScriptLibrary. Integrations are typically performed through JavaScript stored in the Script Library as well.



Note

Note

Do not use Administrative Mode when entering the ScriptLibrary because functions such as compile and execute aren't available in that mode.

You can access any script or function in the ScriptLibrary from any of the locations in Service Manager that support JavaScript. To call one of these functions, use one of the following statements:

`system.library.<scriptname>.<functionname>( parameters )`

or

`lib.<scriptname>.<functionname>( parameters )`

2.3.1.1. ScriptLibrary editing environment

The maximum size of a script allowed in the ScriptLibrary is 256K. The color codes that the script editor uses are:

- Red: Keywords.
- Green: Objects, methods, and properties as well as function names.
- Teal: Comments.
- Magenta: Curly braces, operators, punctuation characters.
- Black: Default color.

2.3.1.2. Calling a function in the ScriptLibrary

ScriptLibrary records contain functions that can be called from anywhere you can call JavaScript. Using this technique allows for more effective management of user JavaScript functions. Functions can use parameters as part of the calling structure, and they can also return values.

First, you must establish a record in the ScriptLibrary that contains the function to be called. Within the record you must know the name of the script as well as create a properly-constructed function.



Note

Note

A record can contain more than one function.

This is an example of using more than one function.

```
/* Sample script lib.functionTest */
function square(value)
{
        var ret_val = value * value;
        return ret_val;
}
function cube(value)
{
        var ret_val = value * value * value;
        return ret_val;
}
```

To call the function use the following syntax:

```
lib.<script name>.<function name>(<Comma separated list of parameters>)
```

The following is an example of calling the functions of **lib.functionTest**:

```
var x = 3;
var y = lib.functionTest.square(x);
print(x, "  squared =  ", y);
x = -2;
y = lib.functionTest.square(x);
print(x, "  squared =  ", y);
x = 2;
y = lib.functionTest.cube(x);
print(x, "  cubed   =  ", y);
/*
The function calls produce the following output:
3  squared =  9
-2  squared =  4
2  cubed   =  8
*/
```

When working with the ScriptLibrary, you can compile the JavaScript code to check for syntax errors. Executing the script will compile the code and then run it. Before executing the code, ensure that all inputs are available.

You can access any script or function in the ScriptLibrary from any tool within Service Manager that supports JavaScript, by using the following syntax:

```
lib.<scriptname>.<functionname>( parameters )
```

Within a ScriptLibrary record, you can call any function that's defined previously within the same record by simply typing:

```
<functionname>( parameters )
```

To call JavaScript from any RAD process panel, you can use the **jscall** RAD function. The syntax for this function is:

```
jscall( "script.function" , arg1 , arg2, ... , argN )
```

The following statements illustrate a JavaScript call to the system library and a call to the script library by using **jscall()**, respectively:

```
lib.tzfunctions.getTZforOperator(“falcon”);
jscall("tzfunctions.getTZforOperator", “falcon”);
```

To call a JavaScript function from the operating system's command prompt, you can use the **us.js.call** RAD routine. The syntax for the command is:

```
sm us.js.call script.function arg1 arg2
```

As an example, the following command calls the **UpdateAllInteractions.updateInteractionFields** function, which needs the `{"abc", "def"}` arguments:

```
sm us.js.call UpdateAllInteractions.updateInteractionFields {\\\"abc\\\",\\\"def\\\"} NULL
```

2.3.1.3. Example: Function to control close button label in Change Management

Change Management uses the **getCloseLabel()** function to control what label to display on the Close button. The labels are Close Task, Close Change, or Close Phase. Included in this function is code to process a special case, Unplanned Changes, in the out-of-box system. This code can serve as an example for Administrators who need to customize the Close button label for other special cases.

The function is **getCloseLabel(filename, currentPhase, file)** in the ScriptLibrary file changeManagement. The details are as follows:

- function **getCloseLabel(filename, currentPhase, file)**
- filename: name of the table we are currently on, cm3r for Change record, cm3t for Task
- currentPhase: current phase of the change or task

- file: the file record

This function is currently called from the cm.view.display display screen.

Example:

```
$L.close.label=jscall("changeManagement.getCloseLabel", filename($L.filed), current.phase in $L.file, $L.filed)
```

2.3.1.4. The log4js script

The log4js is a logger type script similar to log4j. It provides a lightweight but useful script to log events in a Service Manager script that you want to debug . It provides a means to trace information in the Message view while controlling the level of detail being logged. It allows developers to turn in-line debugging on and off.

The log4js script allows developers to set the following logging levels.

log4js log level	Description
OFF	Nothing is logged.
ALL	Everything is logged.
DEBUG	Debug information is logged.
INFO	Information messages are logged (default).
WARN	Warning messages are logged.
ERROR	Error messages are logged.
FATAL	Fatal error messages are logged.

API Details

The log4js should be initialized before it is used.

Constructor syntax

```
var samplelog = new lib.log4js($loglevel, $prefix);
```

You can also use the following method to create an instance of the logger:

```
var myLogger=new lib log4js.Log(lib.log4js.Log.INFO);
```

Sample code snippets

```
myLogger.info("an info");

myLogger.warn("a warning");

myLogger.error("an error");
```

You can change the log level or turn off logging with:

```
myLogger.setLevel(lib.log4js.Log.WARN);

myLogger.setLevel(lib.log4js.Log.OFF);
```

You can also change the log level or turn off logging with this format:

```
myLogger.setLevel("WARN");

myLogger.setLevel("OFF");
```

The log4js script supports appending a prefix to each message. For example:

```
var myLogger=new lib.log4js.Log("INFO","my prefix");
```

Type of Logger

Logger	Description
alertLogger	Output information to Service Manager Message view or the server log file (default: <SM server installation directory>\logs\sm.log), depending on whether a background or foreground process calls the log4j functions. background: server log foreground: Message view

API methods

The following table lists the API methods for log4js.

Method	Description
$log.debug	Write a debug message.
$log.info	Write an information message.
$log.warn	Write a warning message.
$log.error	Write an error message.
$log.fatal	Write a fatal message.
$log.setLevel	Set a log level.
$log.getLevel	Retrieve the log level.
$log.setPrefix	Set the prefix information.
$log.getPrefix	Get the prefix information.
$log.getLogger	Get logger information.
$log.setLogger	Set the logger information.

Script name: log4js

Package name: null

Performance impact

The log4js uses the standard Service Manager print method to write the Service Manager server log file or the Messages view, which is useful for debugging your JavaScript, but keep in mind that when logging is turned on there is a performance impact so log4js should be used carefully.

2.3.2. Examples of calling JavaScript from different Service Manager tools

The following examples offer suggestions about using JavaScript in some of the tailoring tools that support this functionality. Additional JavaScript examples can be found in section JavaScript and Service Manager reference.

2.3.2.1. FormatControl

The following example can be used to verify whether or not the operator who was just entered in the assignee.name field in an incident record is currently logged in. Otherwise, validation will fail.

1. In the **Calculations** section of Format Control, enter a condition of true in the initial column. Then add the following codes in the calculation column.

```
$assignee.old = nullsub(assignee.name in $file, “no assignee”)
```

2. Click the **JavaScript** tab, enter true for **Add** and **Update**, and then enter the following statements.

```
var usersXML = system.users;
var usersString = usersXML.toXMLString();
var file = system.vars.$file;
var newAssignee = file.assignee_name;
system.vars.$IsUser=usersString.indexOf(newAssignee);
```

3. In the **Validation** section, enter the following codes in the **Add** and **Update** column respectively.

```
assignee.name in $file~=$assignee.old;
```

4. And the following expression in the **Validation** column.

```
$IsUser > 0
```

5. Enter the following in the **Validation** message field.
   The new assignee is not logged in at this time. Please try another assignee.

This example uses the standard **indexOf** method for the String Object, and the **toXMLString()** function, to search for the user who was entered as the new assignee in the XML object containing all logged-in users.

Any RAD variable can be set and used by preceding it with system.vars. This lets variable values be exchanged between JavaScript and RAD expressions.

2.3.2.2. Links

This example shows how to query a record from within Service Manager, and how to update fields in that record. Additionally, it shows how to transfer an array from Service Manager to JavaScript. An array from Service Manager can't be directly copied into a JavaScript array, because the syntax of Service Manager {“IM1001”, “IM1002”} doesn't work in JavaScript. Properties such as the length property assume that the value is a function if it is enclosed in curly braces {}. To successfully convert a Service Manager array into a JavaScript array, you can either use the **SCFile.toArray** method, or create a **while** loop or a **for** loop to move through the elements and do an element-by-element transfer, as shown above.



Note

Note In JavaScript, it is very important that you handle null values. Functions such as length may not work on null values.

In this example, the JavaScript code performs the following functions:

- Queries the configuration item that was previously selected from a fill.
- Increases a counter in that configuration item called num.issues.on.CI (this new field has to be added in the device file).
- Adds the new record number to an array of issues in the device file called array.incidents.
- Updates the device record.
- Issues an error message if the configuration item doesn't exist.
- Sends an alert to the person who is opening the record if the item is currently down.

```
var CI_Record=new SCFile("device"); /* opens the device file */
var CIName=system.vars.$File.logical_name; /* passes in the logical name from $File */
/* Selects the configuration item to update */
function getConfigurationItem( CIName )
{
var findCI_RC = CI_Record.doSelect( "logical.name=\""+ CIName + "\"" );
if (findCI_RC==RC_SUCCESS)
{
return CI_Record;
}
else
{
print( "Could not find configuration Item. " +
RCtoString( findCI_RC ) );
return null;
}
}
/* Increases the use counter that counts how often this item was affected by an incident ticket and enters the
incident ticket number into the array of incidents */
function increaseUseCounter(CI)
{
if (CI.is_down == true)
{
print("The configuration item is down at the moment. Please check
existing tickets if the cause for this issue is related to the down
system.");
}
CI.num_issues_on_CI++; /* increase counter */
var incNumber = system.vars.$File.number; /* get the incident ticket
number */
var array_incidents = new Array();
var ind = 0;
while(CI.array_incidents[ind] != null) /* convert the SC array into a
JS array (see paragraph below) */
{
array_incidents[ind]=CI.array_incidents[ind];
ind++;
}
var j=0;
var temp_string = array_incidents.toString();
var i = temp_string.indexOf(incNumber);
if (i < 0) /* Check if incident is already in the list */
{ /* If not, enter the incident number to the list */
if (array_incidents == null)
{
j=0;
}
else
{
j=array_incidents.length;
}
array_incidents[j] = incNumber;
CI.array_incidents = array_incidents;
}
CI.doUpdate();
}
/* Call previously defined functions to execute code */
CI=getConfigurationItem(CIName);
increaseUseCounter(CI);
```

2.3.2.3. Trigger scripts

The triggers.g form contains a field for entering JavaScript (Script). We recommend that the **Script** field contain single statements that invoke functions in the ScriptLibrary. For example, `system.library.myTriggers.triggerXYZ( );` .

Click the Compile icon in the ScriptLibrary to verify the syntax of your trigger script function. A trigger script only needs to return a value if the trigger fails. The user can indicate the failure of a trigger by returning the value of -1. Trigger scripts can access the value of the current record in memory using the global object called system.record, and they can access the value of the same record on disk using the global object called system.oldrecord. For example, `system.library.myTriggers.triggerXYZ( system.record, system.oldrecord );` .

Triggers example

In this example, a trigger sends a message to the assignment group identified in an incident record that the record has been updated and to whom it is assigned. It then adds an activity note that the message has been sent.

```
var NewRecord=system.vars.$L_new; /* This is the $L.new record passed in from the trigger */
var incNumber=NewRecord.number;
var incAssignee=NewRecord.assignee_name;
if (incAssignee==null)
{
        incAssignee="no one at the moment";
}
/* Insert a message schedule record into the schedule file */
function insertSchedule( incNumber, incAssignee, Adressees)
{
        var newSchedule = new SCFile( "schedule" );
        var today=new Date();
        newSchedule.application= "message.bg";
        newSchedule._class = "problem";
        newSchedule.name= "message processor record";
        newSchedule.expiration= today;
        newSchedule.strings[1]="1";
        newSchedule.strings[2]="problem update";
        newSchedule.strings[3]="Problem " + incNumber + " has been
        updated. It is assigned to " + incAssignee + ".";
        newSchedule.strings[5]="pm.main";
        newSchedule.strings[6]=" Soap-Windows XP";
        newSchedule.strings1=Adressees;
        var rc = newSchedule.doInsert();
}
/* Selects the operators that are members of the assignment group to return to the calling function */
function selectAddressees(assignmentGroup)
{
        var assignment = new SCFile( "assignment" );
        var findAssignee = assignment.doSelect( " name=\""+ assignmentGroup +
        "\"" );
        if ( findAssignee == RC_SUCCESS )
        {
                addressList=assignment.operators;
                return addressList;
        }
        else
        {
                print( "Could not find assignment group. " +
                RCtoString( findAssignment ) );
                return null;
        }
}
/* Inserts an activity with the information that the email notification
to the assignment group members has been sent */
function addActivity(incNumber)
{
        var newActivity = new SCFile( "activity" );
        var today=new Date();
        newActivity.number= incNumber;
        newActivity.datestamp= today;
        newActivity.type= "Update";
        newActivity.description[0]= "Notification email on the update has been
        sent to all members of the assignment group.";
        var result = newActivity.doAction("add");
        if (result == RC_CANT_HAVE || result == 51 )
        {
                system.library.activityUpdates.scheduleActivityUpdate
                ( newActivity, newActivity.type, newActivity.description );
        }
}
/* Selects the operators to send the message to */
Addressees = selectAddressees( NewRecord.assignment );
if (Addressees != null)
{
```

```
/* adds the message to the schedule file for sending */
insertSchedule( incNumber, incAssignee, Addressees);
/* adds an activity to inform the user of the email being sent*/
addActivity(incNumber);
}
```

The example also shows (among other things) how to call a Document Engine Action, how to call a function from the Script Library, and how to insert a new record. In addition, it shows how to use several functions within a single JavaScript and how to call them.

Note

Note

Because the class field name in the schedule file is in conflict with the class keyword used in JavaScript, it must be preceded by an underscore (_).

2.3.3. Using RAD functions in JavaScript

You can use most RAD functions in JavaScript. To use a RAD function in a JavaScript, simply prefix the function with **system.functions** and replace any periods (.) in the function name with an underscore (_).

For an example of how to do this, compare the following functions:

- RAD function: parse
- RAD function: evaluate
- JavaScript function: parse_evaluate()

Refer to List: RAD functions for a list of other RAD functions.

2.3.3.1. JavaScript function: parse_evaluate()

Returns the value of a given expression.

Syntax

```
system.functions.parse_evaluate( exp , 10 );
```

Note

Note The data is the character string to parse and the parse type is the numeric value normally passed into the parse function.

Example

If there is an `$lo.username="Sys.Admin"` expression in the useroptions table, this example returns the value of that expression.

```
if (system.functions.parse_evaluate(expression in $l.file, 10 ))
userIsSysAdmin = true;
```

Note

Note The expression works when the statement being evaluated and parsed contains thread or global variables. It doesn't work for local variables. However, you can transfer local variables to thread variables before calling the `parse_evaluate` expressions.

2.3.3.2. RAD function: parse.evaluate

Returns the value of a given expression.

Syntax

```
evaluate(parse(exp, 10 )
```

Note

Note The data is the character string to parse and the parse type is the numeric value normally passed into the parse function.

Example

If there is an expression `$lo.username="Sys.Admin"` in the useroptions table, this example returns the value of that expression.

```
if (evaluate(parse(expression in $l.file, 10)))
$userIs.Sys.Admin = true
```

Note

Note The expression works when the statement being evaluated and parsed contains thread or global variables. It doesn't work for local variables. However, you can transfer local variables to thread variables before calling the parse and evaluate expressions.

2.3.4. JavaScript global properties

Global properties are predefined properties that aren't part of any object. They are names representing values that are globally defined. The core JavaScript language has several global properties, such as NaN (Not a Number), and Service Manager has a number of additional global properties.

There are two categories of Service Manager-defined global properties:

- Query operators
- JavaScript return code properties

Query operators[edit]

These operators are used solely for constructing query expressions using the Query object.

Note

Note The Query object has been deprecated, but older JavaScript code within the ScriptLibrary still uses the object.

Query operator	Description
EQ	Equals to
LIKE	Starts with
NEQ	Not equal to
GT	Greater than
GE	Greater or equal to
LT	Lower than
LE	Lower or equal to
ISIN	Is an element in the array

Example:

```
var q=new SCFile("probsummary")
q.doSelect(new QueryCond("number", EQ, "IM10002"));
```

JavaScript return code properties[edit]

Use these properties (which are represented internally as integer values) to test the values returned by object methods such as SCFile. To print the error message rather than the return code, use the **RCtoString** function.

Return code	Description
RC_ERROR	Some other error occurred. Examine the contents of the Messages view or the sm.log file for more information.
RC_SUCCESS	The operation succeeded.
RC_CANT_HAVE	The operation failed because the resource is unavailable because some other user or process has the resource locked.
RC_NO_MORE	No more records are available in the result set.
RC_DUPLICATE_KEY	The insert operation failed because the file already contains a record with this unique key value.
RC_MODIFIED	The update operation failed because the record was modified by another user or process since you read it.
RC_DELETED	The operation failed because the record was deleted by another user or process.
RC_BAD_QUERY	The query failed due to incorrect query syntax.
RC_NOT_AUTHORIZED	The request operation was not performed due to an authorization failure. Check the permissions associated with the user who submitted the request.
RC_VALIDATION_FAILED	The operation failed because the data supplied in a field or the record did not pass validity checks performed by the application.
RC_UNABLE_TO_WRITE_TO_FILE	The operation failed. Check to make sure the file name is correct, that the file exists, and that it's not read-only.
RC_UNABLE_TO_CLOSE_FILE	The operation failed. Check to make sure the file name is correct, that the file exists, and that it's not read-only.
RC_UNABLE_TO_DELETE_FILE	The operation failed. Check to make sure the file name is correct, that the file exists, and that it's not read-only.
RC_INVALID_FILENAME	The operation failed because the file name is invalid.

Example:

```
var rc = mySCFile.doInsert();
if ( rc == RC_SUCCESS )
{
        print( "Insert succeeded" );
}
else
        throw( "Error " + RCtoString(rc) + " occurred trying to insert a
        record" );
```

Contents

- 1 Query operators
- 2 JavaScript return code properties

2.3.5. Order of execution of JavaScript versus RAD language

This section lists all tailoring tools in which JavaScript can be directly entered and describes at what point in the process the JavaScript code will be executed.

- Format Control: The JavaScript is executed after calculations and before validations. JavaScript code in Format Control is executed via the format.cjavascript RAD.
- Links: JavaScript in links is executed after the (initial) expressions, but before the fill or find is executed. This is done in us.link on evaluate.expressions.
- The post JavaScript is executed after the fill or find was performed, immediately after the post expressions. This happens in us.fill or us.find, respectively, on the eval.post.expressions panel.
- Triggers: The JavaScript in the triggers file is executed after the RAD application. This is controlled by the server binaries and not by RAD code.
- Cascade updates: The JavaScript in cascade updates is executed from the RAD application process.update.config.record. The evaluate expressions panel is executed before the evaluate.javascript panel, so that expressions take precedence over JavaScript.
- Scripts: In script.execute on panel exec.statements, RAD expressions are processed before JavaScript expressions.
- Wizards: Wizards use JavaScript in three different areas: During File Selection, Actions, and Cancel Expressions.
    - File Selection: The Select $L.file by information is processed first, followed by the Initial Expressions and then the JavaScript.
    - Actions: In the Actions tab, the Expressions are run first, then the JavaScript, followed by the information on the Format Control / Process Name tab.
    - Cancel Expressions: First the Expressions Executed on Cancel are processed, followed by Javascript Executed on Cancel.
- Interoperability (ioaction file) : The ioaction file executes only JavaScript and not RAD expressions. The JavaScript code is executed in the ioevents.process.action application on the process.action panel .
- Display screens: In the display application on the panel prep.screen, the JavaScript expressions are executed after the RAD expressions.
- Display options: Display options have pre- and post-JavaScript and RAD expressions that are executed in the following order:
    1. Pre-RAD expression
    2. Pre-JavaScript
    3. RAD application
    4. Post-RAD expression
    5. Post-JavaScript
    6. Display Action
- Processes: Process records have initial and final JavaScript and RAD expressions executed in the following order:
    1. Initial RAD expression
    2. Initial JavaScript
    3. Pre-RAD expressions
    4. RAD application(s)
    5. Post-RAD expressions
    6. Final RAD expression
    7. Final JavaScript
    8. Next Process (if applicable)
- Schedule: JavaScript in the schedule record is executed after the RAD application is called and before the check for deleting or rescheduling the schedule record.

2.3.6. Corresponding JavaScript syntax for RAD language tasks

Most Service Manager tasks can be translated from RAD language to JavaScript by using the JavaScript expressions identified in the following table. In the JavaScript expression, the ObjectName is the name of a Service Manager object such as a table.

Service Manager task	RAD panel / expression	JavaScript expression
Create a file variable	rinit	new SCFile(“filename”)
Select a record	select	RC=ObjectName.doSelect(“Query”)
Update a record	rupdate	RC=ObjectName.doUpdate()
Add a record	radd	RC=ObjectName.doInsert()
Delete a record	rdelete	RC=ObjectName.doRemove()
Refer to a field in a file	result=fieldname in filevariable	result=ObjectName.fieldname
Access an element of an array	result=rownumer in arrayname in filevariable	result=ObjectName.arrayname[rownumber]
Send a message to screen	call message application	print(“message”)
Convert a field / Object to string	str()	RC=ObjectName.getText()

2.4. System Language reference

This is the reference section for the system language. In this section you will find a list of the RAD functions, a list of the rtecalls, and a list of RAD routines.

2.4.1. Deleted RAD applications

The following RAD applications have been deleted from Service Manager because the functionality they provided is no longer supported by Service Manager, or it has been replaced with updated capabilities.

add.bigroup	apm.get.incident.query	cm.open.scm7
adl.initial.setup	apm.get.initial.query	cm3r.group.assign.totals
ag.editor.backup	apm.get.parts	cm3r.group.request.totals
agmap.flow	apm.get.probassign.query	cm3t.group.assign.totals
agmap.flow.panels	apm.get.problem.query	cm3t.group.request.totals
agmap.panel.xref	apm.get.values	convert.db2mvs
agmap.print.arrays	apm.inbox	convert.oracle
agmap.print.condense	apm.initial	convert.sqlserver
agmap.print.condense.only	apm.launch.update	convert.sybase
agmap.print.fields	apm.monthly.downtime.job	create category
agmap.process.reports	apm.open.inbox	db2mvs.template.to.sqlbasename
am.cigroup.bulkupdate2	apm.post.cost.save	display.backup
am.search.related.SAV	apm.remove.membership	display.pass.file.position
apm.add.assign.group	apm.remove.record	encl.appl.fla
apm.add.group	apm.rename.group	find.alias.fields
apm.add.parts	apm.repair.problem	find.illegal.fc
apm.add.profile	apm.set.assignment	fix.form.graph
apm.add.user	apm.tapi.header	get.serno
apm.admin	apm.thread.stub	isg.check.pm.contract
apm.agi.main	apm.user.to.group	isg.decide.contract
apm.assign.clean	append.to.related	isg.delete.sm.warnings
apm.build.child.list	appl.copy.rename	isg.set.warnings
apm.build.screlate	big.green.arrow	label.display
apm.build.screlate.sub	category.delete	login.saf.msgs
apm.cascade.problem.close	cau.add.profile	pm.close.pending
apm.cp.send	cau.add.profile2	pm.log
apm.create.group	cau.add.user	pmc.sql.repair
apm.create.record	cau.add.user.backup	rca.add.group
apm.delete.group	cau.check.group	rca.add.profile
apm.delete.membership	cau.edit.profile	rca.add.user
apm.delete.profile	cau.get.rm.profiles	rca.admin
apm.delete.record	cau.main.2	rca.create.group
apm.delete.user	cau.remove.from.groups	rca.delete.group
apm.done.link	cau.remove.from.msg.groups	rca.delete.profile

apm.dt.process.demon	cau.remove.profile	rca.delete.user
apm.edit.assign.group	cc.add.group	rca.edit.group
apm.edit.group	cc.add.profile	rca.edit.profile
apm.edit.operator	cc.add.user	rca.edit.user
apm.edit.profile	cc.admin	rca.fill.profile
apm.edit.record	cc.create.group	system.nls
apm.edit.user	cc.delete.group	trigger.counter
apm.email.problem	cc.delete.profile	trigger.problem.homesite
apm.end.user	cc.delete.user	trigger.problem.shadowsite
apm.expand.downtime	cc.edit.group	unconvert.qbe.lists
apm.fill.profile	cc.edit.profile	us.calc.delta
apm.find.next.event	cc.edit.user	us.gen.message
apm.fix.call	cc.end.user	us.load.errmsg
apm.fix.shell	cc.remove.membership	us.make.text
apm.fix.stored.queries	cc.required.text	xanadu.timezone.set
apm.get.assignments	cc.user.to.group	

2.4.2. List: RAD functions

A RAD function is an operand, which means that it can be used in any expression on the right hand side of an equals sign. Functions provide a method of performing certain commands that will return a value when executed. For example:

- operator()--returns the logged-on operator ID.
- tod()--returns the current date and time.
- option()--returns GUI option number of the last displayoption selected.

RAD functions can be used in statements to further define selection criteria and/or execute commands, initialize values, and perform calculations. Service Manager syntax rules must be followed. For example, the result of a RAD function can be assigned to a variable using the assign (=) assignment statement: `$string.length=lng($string)`

In most cases, there is a JavaScript function that can do the same operation as a RAD function. If you prefer to work with JavaScript rather than RAD, see Using RAD functions in JavaScript for more information.

Function	Description
add.graphnodes(xml)	Returns XML to the Run-Time Environment (RTE) for the expand action on a node in the graph diagram.
cache.flush	Removes the specified item from the memory cache.
clean.display.value	Cleans up (clears) the display value of a field in a file.
cleanup	Frees the storage associated with a variable.
contents	Returns a structure containing the current record in a file variable.
copyright	Returns a string with the company's copyright notice.
currec	Represents the current file handle in queries.
current.device	Returns a string with the type of system the user is using to log in to Service Manager, such as "SOAP-Windows Vista."
current.format	Returns a string containing the name of the current format.
cursor.field.contents	Returns a string containing the contents of the input field in which the cursor was positioned when the last interrupt key was pressed.
cursor.field.display.content	Returns a string that contains the display value of the field where the cursor is currently positioned.
cursor.field.display.content.set	Sets the display value of the field where the cursor is currently positioned.
cursor.field.name	Returns a string containing the name of the field in which the cursor was positioned when the last interrupt key was pressed.
cursor.field.name.set	Moves the cursor to a field.
cursor.field.readonly	Returns true if the selected field is read-only. Returns false if the selected field is not read only.
cursor.line	Returns the number of the line, relative to the screen, in which the cursor was positioned when the last interrupt key was pressed.
date	Returns the date portion of a date/time variable and defaults time portion to '00:00:00'.
datecmp	Translates the date/time fields to the correct SQL statement dialect.
day	Returns the day of the month for a date regardless of the date format.
dayofweek	Returns a number from 1 to 7 representing the day of the week for a specific date.
dayofyear	Returns the day of the year for a date regardless of the date format.
dbdict.helper("db.type")	Helps retrieve the current RDBMS type.
dbdict.helper("field.type")	Helps retrieve the field type property efficiently.
dbdict.helper("is.alias")	Helps determine whether a field is an alias or not.

dbdict.helper("joinfile.names")	Helps retrieve an array of names of the sub files that belong to a join file.
dbdict.helper("unique.key")	Helps retrieve the first unique key or the primary key of a file efficiently.
delete	Returns an array with specific elements deleted.
denull	Returns an array with all trailing NULL entries deleted.
descriptor	Returns the database dictionary descriptor record for a file variable.
display.value.copy	Copies all of the display values from one file to another.
evaluate	Executes a specific string as though it were a processing statement.
evaluate.query	Evaluates a query against a record.
exists	Checks for the existence of a field in a file.
fduplicate	Copies an entire file variable from one record to another.
filename	Returns a string containing the name of the file for a specified file variable.
filequeryex	Returns the query parameters of a file variable.
frestore	Restores all fields in a file variable to their original database values.
genout	Generates a formatted string based on a record and format name passed as parameters.
get.base.form.name	Returns the base name of a form, stripping off the form extension for GUI or Web forms.
get.dateformat	Returns the date format of the current operator ID.
get.display.value	Returns the display value of a field in a Service Manager file.
get.graph.action	Returns the activated action on a node in the graph diagram.
get.graph.id	Returns the value of the graph ID for the expand action on a node in the graph diagram.
get.graphnode.id	Returns the value of the node ID for the expand action on a node in the graph diagram.
get.graph.target	Returns the target for an activated action on a node in the graph diagram.
get.lock.owner	Returns the name of the owner of a resource lock.
get.timezoneoffset	Returns the absolute time difference between GMT and the time zone of the operator.
get.uid	Returns a 24-character identifier that's unique to an installation of Service Manager
get.web.url	Returns the URL of the web client.
gui	Determines whether or not Service Manager is running in GUI mode.

index	Returns the element number of an array or the position in a string that matches a specified string.
insert	Returns a modified array with a specified number of new elements inserted at a specified location.
iscurrent	Determines if the record with which you are working is identical to the version stored in the database.
isExpressionValid	Determines in a RAD expression is a valid expression of a specified data type.
isfileexist	Determines whether a file exists or not.
jscall	Enables you to invoke JavaScript from within RAD.
lioption	Indicates whether a given input value is a licensed option on the current system.
lng	Returns the number of elements in an array or characters in a string.
locks	Returns array of current outstanding locks.
logoff	Logs a user off.
logon	Logs a user on.
mandant	Allows an application to create a subset of the current Mandanten values.
max	Returns the largest value in a list of values or in an array.
messages	Provides logging functions for use in the memory message log. The log is implemented as a wrap-around cache of the x most recent messages (error, informational, and action).
min	Returns the smallest value in a list of values or in an array.
modtime	Returns the time a record was last modified.
month	Returns the month of year for a date regardless of the date format.
multiselect.selection("fieldcontents")	Returns the contents of the fields associated with the multiselection.
multiselect.selection("fieldname")	Returns the field name of the column associated with the multiselection. The associated column name is set using the Forms Designer property "Selection Field."
multiselect.selection("rows")	Returns an array of row numbers that were selected.
multiselect.selection("selected")	Returns true or false if the provided file variable has multiple records selected.
multiselect.selection("selections")	Returns the number of rows selected.
multiselect.selection("tablename")	Returns the Table ID associated with the multiselection.
null	Returns true if the value of a variable is NULL. Returns false if the value of a variable or field is not NULL.
nullsub	Substitutes a null field with the value given.

operator	Returns the string of the name of the currently logged on operator.
option	Returns the number of the last option key pressed.
parse	Returns an evaluative expression from a given string value.
perf	Evaluates system performance and writes the information to disk.
policyread	Reads the data policy from a datadict table.
processes	Returns array of current logged on processes.
prof	Returns the value requested on various system performance profiles.
recordcopy	Copies a set of fields from one record to another record.
recordtostring	Takes the value of a field from an array and appends the value to a string.
round	Returns a number rounded to a specified number of digits.
same	Compares two compound or null values. Returns true if two values are identical, otherwise returns false.
scmsg	Returns a text string matching the format of the message ID number and message class specified. The function replaces any variables in the message with the text specified in an array of values.
set.display.value	Sets the display value for a field in a file.
set.timezone	Sets time zone and other system parameters.
setsort	Sorts the fields of an array in a file.
shutdown	Shuts down Service Manager from inside the system. This function doesn't return any values.
simple file load	Loads an unload file containing only data records of one file into a target file.
sleep	Temporarily ceases the execution of an executing thread for the specified number of seconds.
str	Returns a string of a string or non-string data variable.
stradj	Makes a string a specific length by either clipping or adding trailing blanks.
strchrcp	Replaces part of a string with copies of another string.
strchrin	Inserts copies of a string into another string.
strclpl	Clips a number of characters from the beginning of a string.
strclpr	Clips a number of characters from the ending of a string.
strcpy	Replaces part of a string with a substring.
strdel	Deletes a number of characters from a specific spot in a string.
strins	Inserts a substring into another string.
strpadl	Pads a string with leading blanks until the string is a certain size.

strpadr	Pads a string with trailing blanks until the string is a certain size.
strraw	Exports array data to a delimiter-separated string.
strrep	Returns a string, replacing a specified character sequence with another string.
strtrml	Removes all leading blanks from a string.
strtrmr	Removes all trailing blanks from a string.
substr	Returns a string from a portion of a string.
substrb	Extracts a substring from a multibyte character or binary string.
sysinfo.get("ActiveFloatUsers")	Returns the total number of currently active floating users.
sysinfo.get("ActiveLicenses")	Returns the total number of currently active licenses.
sysinfo.get("ActiveNamedUsers")	Returns the total number of currently active named users.
sysinfo.get("AuthMode")	Returns the authentication mode of the current user.
sysinfo.get("ClientNetAddress")	Returns the Service Manager client network IP address as a character string.
sysinfo.get("ClientOSName")	Returns the name of the operating system that Service Manager is running on.
sysinfo.get("ClientPID")	Returns the PID for RAD.
sysinfo.get("ClientVersion")	Returns the version number of the client software as a string.
sysinfo.get("Display")	Returns "GUI" or "text", depending on the type of client that's running.
sysinfo.get("Environment")	Returns the name of the environment in which the client is operating.
sysinfo.get("languagecode")	Returns the server session language code value that's used for localized forms and messages.
sysinfo.get("MaxFloatUsers")	Returns the total number of floating users logged on since the server started.
sysinfo.get("MaxLicenses")	Returns the maximum number of licenses used since the server started.
sysinfo.get("Mode")	Returns a value that states the type of client being used.
sysinfo.get("PrevLabel")	Returns the label name of the last RAD panel that was executed.
sysinfo.get("PrintOption")	Returns printing option information, "server" or "client" depending on how the system is set.
sysinfo.get("Quiesce")	Returns "true" if system is quiesced and "false" if system is not quiesced.
sysinfo.get("RecList")	Returns "true" if record list is enabled and "false" if record list is disabled.
sysinfo.get("ServerNetAddress")	Returns the Service Manager server network IP address as a character string.
sysinfo.get("ServerNetPort")	Returns the Service Manager server port used by the connected client or the system name when called by a background process.

sysinfo.get("ServerPID")	Returns the PID of the server as a number.
sysinfo.get("ThreadID")	Returns the number of the current RAD thread.
sysinfo.get("TotalFloatUsers")	Returns the total number of floating users allowed.
sysinfo.get("TotalLicenses")	Returns the total number licenses allowed.
sysinfo.get("TotalNamedUsers")	Returns the total number of named users allowed.
sysinfo.get("TotalProcs")	Returns the total number of executing system and user tasks.
sysinfo.get("TotalSystemProcs")	Returns the total number of system tasks.
sysinfo.get("TotalUserProcs")	Returns the total user tasks.
time	Returns the time portion of a date/time variable.
tod	Returns the current date and time.
tolower	Returns a string, replacing upper-case letters with lower-case letters.
toupper	Returns a string, replacing lower-case letters with upper-case letters.
translate	Returns a string translated from one character set to another.
trunc	Truncates the decimal digit number to a specified number of digits.
type	Returns the numeric type of its argument.
updatestatus	Returns the result of the last update operation on a Service Manager file.
val	Returns a non-string data type converted from a string.
version	Returns a list of version information.
year	Returns the full year for a date regardless of the date format.

2.4.2.1. RAD function: add.graphnodes(xml)

Description

A RAD function that returns XML to the Run-Time Environment (RTE) for the expand action on a node in the graph diagram.

Function

add.graphnodes()

Example

```
add.graphnodes(xml)
```

2.4.2.2. RAD function: cache.flush

A RAD function that removes the specified item from the memory cache.

Function

cache.flush

Format

```
cache.flush( nType, pItem )
```

Parameters

`nType` is a number from 1 to 4 representing the type of flush operation. Flush types 1, 2, and 4 remove the item from the global shared memory cache. Flush type 3 removes the item from the local format cache.

- 1 flushes application/code records
- 2 flushes dbdicts
- 3 flushes formats
- 4 flushes SQL maps

`pItem` is the name of the item to flush enclosed in quotation marks. Note that `pItem` is not used if flush type is 3.

Factors

This function returns no value.

Example

To flush the dbdict record for the probsummary table:

```
cache.flush( 2, "probsummary" )
```

To flush the local format cache:

```
$L.type = 3
cache.flush( $L.type )
```

2.4.2.3. RAD function: clean.display.value

A RAD function that cleans up the internal display value cache associated with a file variable.

Function

clean.display.value

Format

```
clean.display.value($file, [$fieldname])
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$file	The file variable whose internal display value cache is to be cleaned up.
$fieldname	The field whose display value is to be cleaned up. If this parameter is not specified, the entire internal display value cache of $file is cleaned up.

Return values

Null or the old display value.

Examples

```
clean.display.value($file)
```

```
clean.display.value($file, “logical.name”)
```

2.4.2.4. RAD function: cleanup

A RAD function that frees the storage associated with a variable.

Function

cleanup

Format

```
cleanup($filename, [$fieldname], [index])
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$filename	The variable that you want to clean up. If only $filename is specified, this variable can be a file variable or a field.
[$fieldname]	Optional. When specified, indicates the name of a field whose field value and display value (if any) are to be cleaned up. **Note** Note Fields that have a referenced table defined have a display value.
[index]	Optional. If the field specified in $fieldname is of the array type, the value and display value (if any) of the element that has the specified index or position number in that array are cleaned up.

Factors

If only $filename is specified when calling this function, this function returns the same result as it would in versions earlier than 9.41.

This function doesn't return a value. It cleans up (empties) the variable or field specified by setting the value to NULL.

You can use this function on any variable type (local, thread, or global), but it is generally used on local and thread variables to clean up values before a user logs out.

This is the preferred way to clean up both variables and fields in records as opposed to $x=NULL or field in $file=NULL.

Unless you run this function, variables remain in memory until their defining environment ends.

For local variables, this is the parent RAD program. For thread variables, this is the parent RAD thread. For global variables, this is the user session.

Note

Note This RAD function is unavailable as a JavaScript system.functions call.

Return values

None

Examples

```
cleanup($file)
```

```
cleanup($file, “logical.name”)
```

```
cleanup($file, “assets”, 0)
```

2.4.2.5. RAD function: contents

A RAD function that returns a structure containing the contents of the current record in a file variable.

Function

contents

Format

```
contents(file variable)
```

`file variable` is the contents you want to obtain.

Factors

If you use the **contents()** function to compare two records, always use the denull function on both records first to remove null values. If one record is displayed and the other is not, the arrays in one record may contain extra null values.

Using the denull function on an empty array of structures converts the array of structures to a simple array.



Note

Note

Always test for null values first, as shown in the following example:

```
denull(contents($save.rec))=denull(contents($rec))
```

Example

```
contents($file.variable)
```

2.4.2.6. RAD function: copyright

A RAD function that returns the company's copyright notice in a string.

Function

copyright

Format

```
copyright()
```

2.4.2.7. RAD function: currec

A RAD function that represents the current file handle on queries. When used as a select statement, it selects all records modified since the date you specify from the current file.

Function

currec

Format

```
currec()
```

Example

```
modtime(currec())>'1/1/05'
```

2.4.2.8. RAD function: current.device

A RAD function that returns a string containing the name of the device or Configuration item (CI) associated with the user that's currently logged on.

Function

current.device

Format

```
current.device()
```

Factors

The value of `$lo.device` is set to the value of current.device upon login.

Example

`current.device()` returns `/dev/tty00` (on Linux)

2.4.2.9. RAD function: current.format

A RAD function that returns a string containing the name of the most recently displayed format.

Function

current.format

Format

```
current.format()
```

Example

`current.format( )` returns `problem.open`

2.4.2.10. RAD function: cursor.field.contents

A RAD function that returns a string containing the contents of the field where the cursor was last positioned when the user performed an action such as clicking **Fill** or **Find**, or double-clicking on a QBE list. If the action opens a new form, **cursor.field.contents** will return the focus on the pop-up form.



Note

Note The cursor.field.contents function doesn't support record lists in the Web client. For example, when you double-click a row in a record list from the Web client, no value is returned by the cursor.field.contents function.

Function

cursor.field.contents

Format

```
cursor.field.contents()
```

Factors

If the field is an array, the content of the current element is returned.

Example

`cursor.field.contents()` returns `a101a01`.

2.4.2.11. RAD function: cursor.field.display.content

A RAD function that returns a string containing the display value of the field where the cursor was last positioned when the user performed an action such as clicking Fill or Find, or double-clicking on a QBE list. If the action opens a new form, cursor.field.display.content will return the focus on the pop up form.

Function

cursor.field.display.content

Format

```
cursor.field.display.content()
```

Parameters

There are no parameters for this function.

Factors

If the field is an array, the content of the current element is returned.

Return values

The display value of the current field. For example, "Adobe Reader".

2.4.2.12. RAD function: cursor.field.display.content.set

A Rapid Application Development (RAD) function that sets the display value of the field where the cursor is currently positioned.

Function

cursor.field.display.content.set

Format

```
cursor.field.display.content.set($display.value)
```

Parameters

The following parameter is valid for this function.

Parameter	Description
$display.value	The display value that you want to specify for the current field

Factors

If the field is of the array type, the display value is the content of the element where the cursor is currently positioned.

Return values

The display value that you specify.

Example

`cursor.field.display.content.set("Adobe Reader")`

2.4.2.13. RAD function: cursor.field.name.set

A RAD function that sets the cursor to a specific field on a form.

Function

cursor.field.name.set

Format

```
cursor.field.name.set($field.name, $row.number)
```

`$field name` is the field name

`$row.number` is for arrays (optional)

Factors

- This function returns no value. It is unnecessary to use it in an assignment statement.
- If the field is an array, you can position the cursor on a specific row by specifying *$row.number.*

Example

```
cursor.field.name.set("address", 2)
```

After execution, the cursor appears in the second row of the **Address** field on the next form displayed.

2.4.2.14. RAD function: cursor.field.name

A RAD function that returns a string containing the name of the input field where the cursor was located when the user pressed an interrupt key, such as the Enter or option keys. To position the cursor in a specific field, use the **cursor.field.name.set** function. The function returns the field name as it is defined in the current form.

Function

cursor.field.name

Format

```
cursor.field.name(n)
```

Factors

Returns the fully-qualified field name if the function is called with an argument.

Example

`cursor.field.name()` returns `number`.

`cursor.field.name(1)` returns `header,number`.

2.4.2.15. RAD function: cursor.field.readonly

A RAD function that returns true if the cursor is in a read-only field and false if the cursor is in a field that's not read-only.

Function

cursor.field.readonly

Format

```
cursor.field.readonly()
```

2.4.2.16. RAD function: cursor.line

A RAD function that returns the line number where the cursor was positioned when the user performed an action such as clicking Fill or Find, or double-clicking an item in a QBE list. If the action opens a new form, cursor.line returns the focus on the pop-up form.

Function

cursor.line

Format

```
cursor.line(n)
```

Factors

- `$l=cursor.line()` = the numeric value of the line where the cursor was positioned.
- `$l=cursor.line(1)` = the numeric value of the index of the array or structure where the cursor was last located, including consideration for vertical scrolling. If the cursor is in an array input field, `$l` contains the numeric value of the index of the array in which the cursor is positioned. If the cursor is not in an array input field, `$l` contains the numeric value, relative to the format in which the cursor is positioned, including accounting for vertical scrolling, which may be greater than the client's screen length.

2.4.2.17. RAD function: date

A RAD function that returns the date portion of a date and time variable.

Function

date

Format

```
date($date.time.variable)
```

Factors

- The date portion of a date and time value is midnight of that day. For example, 00:00:00. The function uses the local time zone to determine the date.
- The argument can be any absolute date and time value, or an absolute time, including tod.

Example

```
$date.opened=date($date.time.variable)
```

Where `$date.time.variable` contains the current date and time or another valid date and time value. For example, if `$date.time.variable = '08/01/08 10:10:10'`, then `$date.opened = '08/01/08 00:00:00'`

.

2.4.2.18. RAD function: datecmp

A RAD function that translates the date/time fields to the correct SQL statement dialect. You can use this function in expert search of incidents, as well as in JavaScript programming.

Function

datecmp

Format

```
datecmp("DateTimeField1","LogicOperator","DateTimeField2","+/-", "TimeInterval")
```

Parameters

This function uses the following arguments.

Argument	Description	Example Value (s)
DateTimeField1	A date time field in an Service Manager table.	close.time
LogicOperator	A logic operator.	>, >=, =, <=, <
DateTimeField2	Another date time field in the same Service Manager table.	open.time
+/-	Arithmetic operator: +or -.	+, -
TimeInterval	A string that represents the time interval to be added to or subtracted from the second date time field. The format of time interval can be: d, d hh:mm:ss, d h:m:s, hh:mm:ss, h:m:s, or hh:m:ss (1 digit mixed with 2 digits). Days can be omitted, or at most 9 digits. Hours, minutes, and seconds can be 1 or 2 digits (from 0 to 99), and hour:minute:second as a whole can be omitted if you enter only days.	10 02:03:04 (This string represents 10 days, 2 hours, 3 minutes and 4 seconds.)

Pay attention to the following items:

- All arguments must be enclosed in a pair of double quotes; otherwise, the query parsing will fail.
- This function supports `AND`, `OR`, and `NOT` to concatenate multiple **datecmp()** calls in one query.
  The following are two examples:
  `datecmp("close.time", "<", "open.time","+", "1") or datecmp("close.time",">=", "open.time","+", "5:0:0")`
  `problem.status="Closed" and (not datecmp("close.time",">", "open.time", "+","31 04:02:30"))`
- You can combine the result of this function with other query conditions to construct a complete query. For example, you can execute one of the following queries when performing an expert search of incidents:
  - `problem.status="Closed" and datecmp("close.time","<","open.time","+","04:02:30") and datecmp("close.time",">=","open.time","+","02:02:30" )`
  - `problem.status="Closed" and datecmp("open.time",">","close.time", "-", "04:02:30" ) and datecmp("close.time",">=","open.time","+","02:02:30" )`
  - `problem.status="Closed" and datecmp("close.time",">", "open.time", "+","31 04:02:30")`



Note

Note The first two queries should return the same results, which are incidents whose closed time is between 2 hours and 4 hours from their open time; the third query should return incidents that were closed more than 31 days after their open time.

Example

An example of a JavaScript program that uses this function is as follows:

```
var f = new SCFile('probsummary', SCFILE_READONLY);
var query = 'problem.status="Closed" and datecmp("close.time", "<", "open.time","+", "04:02:30")
and datecmp("close.time",">=", "open.time","+", "02:02:30")';
if (RC_SUCCESS == f.doSelect(query))
{
do
{
print(f);
}
while (RC_SUCCESS == f.getNext());
};
```

2.4.2.19. RAD function: day

A RAD function that returns the day of month for a date, regardless of the date format.

Function

day

Format

```
day=day($date)
```

Where the value of *$date* is a date and time value.

Factors

Although the **set.timezone** function affects the way a date is presented, the day function always extracts the proper day value.

Example

```
$mday=day('2/15/08')
```

After execution, the value of `$mday` is `15`.

2.4.2.20. RAD function: dayofweek

A RAD function that returns the day of the week for a date, regardless of the date format. The **dayofweek** function returns 1-7, where 1 is Monday, 2 is Tuesday, 3 is Wednesday, 4 is Thursday, 5 is Friday, 6 is Saturday, and 7 is Sunday.

Function

dayofweek

Format

```
day=dayofweek($date)
```

Where the value of *$date* is a date and time value.

Factors

The **set.timezone** function affects the way a date is presented, whereas the **dayofweek** function always extracts the proper day of week value.

Examples

From the RAD debugger type:

```
d dayofweek('11/06/09')
```

This expression displays the day of the week of a specific date. (The command **d** stands for 'display.') In this example, the value of `dayofweek` is `5` or Friday.

From the RAD debugger type:

```
x $Ltoday=dayofweek('11/06/09')
```

This expression executes the function. (The command **x** stands for 'execute.') In this example, you want to assign the value to a variable.

2.4.2.21. RAD function: dayofyear

A RAD function that returns the day of year for a date, regardless of the date format.

Function

dayofyear

Format

```
$yday=dayofyear($date)
```

The *$date* variable is a date and time value.

Example

```
$yday=dayofyear('2/15/08')
```

After execution, the value of `$yday` is `46`.

2.4.2.22. RAD function: dbdict.helper("db.type")

A RAD utility function that helps retrieve the current RDBMS type.

Function

dbdict.helper("db.type")

Format

```
dbdict.helper("db.type")
```

Parameters

None

Return values

A string type value that represents the current RDBMS type, which can be "sqlserver", "oracle", or "db2universal".

2.4.2.23. RAD function: dbdict.helper("field.type")

A RAD utility function that helps retrieve the field type property efficiently.

Function

dbdict.helper("field.type")

Format

```
dbdict.helper("field.type",$L.file/$filename,fieldname)
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$L.file/$filename	The variable or name of the file to which the field belongs
fieldname	The name of the field whose field type property is to be retrieved.

Factors

Always consider using a file variable first for better performance.

Return values

A number value that represents the field type. For information about the numeric representations of data types, see Data types.

Example

```
function testDBDictHelper()
{
        var file = new SCFile( 'cm3r' );
        var fieldType = system.functions.dbdict_helper("field.type", file, "assets" );
        print( "fieldType of assets in cm3r=", fieldType );
        var fieldType = system.functions.dbdict_helper("field.type", file, "type" );
        print( "fieldType of type in cm3r=", fieldType );
        fieldType = system.functions.dbdict_helper("field.type", "cm3r", "assets" );
        print( "fieldType of assets in cm3r by filename=", fieldType );
        fieldType = system.functions.dbdict_helper("field.type", "cm3r", "type" );
        print( "fieldType of type in cm3r by filename=", fieldType );

        print( "db type=", system.functions.dbdict_helper("db.type" ) );

        var isAlias = system.functions.dbdict_helper("is.alias", file, "assets" );
        print( "assets is an alias:", isAlias );

        isAlias = system.functions.dbdict_helper("is.alias", file, "number.vj" );
        print( "number.vj is an alias:", isAlias );

        isAlias = system.functions.dbdict_helper("is.alias", file, "number.vj.ooflow" );
        print( "number.vj.ooflow is an alias:", isAlias );
        isAlias = system.functions.dbdict_helper("is.alias", "cm3r", "number.vj.ooflow" );
        print( "number.vj.ooflow is an alias by filename:", isAlias );

        isAlias = system.functions.dbdict_helper("is.alias", file, "number.string" );
        print( "number.string is an alias:", isAlias );

        isAlias = system.functions.dbdict_helper("is.alias", "cm3r", "middle,type" );
        print( "middle,type is an alias by filename:", isAlias );
        isAlias = system.functions.dbdict_helper("is.alias", "cm3r", "number.string" );
        print( "number.string is an alias by filename:", isAlias );

        isAlias = system.functions.dbdict_helper("is.alias", file, "middle,type" );
        print( "middle,type is an alias:", isAlias );

        var fileArray = system.functions.dbdict_helper( "joinfile.names", "joincomputer" );
        print( "sub file array of joincomputer is: ", fileArray );

        var uniquekey = system.functions.dbdict_helper( "unique.key", file );
        print( "uniquekey of cm3r is: ", uniquekey );
}
```

This example returns the following results:

```
    fieldType of assets in cm3r= 8
fieldType of type in cm3r= 2
fieldType of assets in cm3r by filename= 8
fieldType of type in cm3r by filename= 2
db type= sqlserver
assets is an alias: false
number.vj is an alias: true
number.vj.ooflow is an alias: true
number.vj.ooflow is an alias by filename: true
number.string is an alias: false
middle,type is an alias by filename: false
number.string is an alias by filename: false
middle,type is an alias: false
sub file array of joincomputer is:  [C++ object Datum] - {"device", "computer"}
uniquekey of cm3r is:  [C++ object Datum] - {"number"}
uniquekey of cm3r by filename is:  [C++ object Datum] - {"number"}
```

2.4.2.24. RAD function: dbdict.helper("is.alias")

A RAD utility function that helps determine whether a field is an alias or not.

Function

dbdict.helper("is.alias")

Format

```
dbdict.helper("is.alias",$L.file/$filename,fieldname)
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$L.file/$filename	The variable or name of the file to which the field belongs
fieldname	The name of the field to be checked

Factors

Always consider using a file variable first for better performance.

Return values

True: The field is an alias.

False: The field is not an alias.

2.4.2.25. RAD function: dbdict.helper("joinfile.names")

A RAD utility function that helps retrieve an array of names of the sub files that belong to a join file.

Function

dbdict.helper("joinfile.names")

Format

```
dbdict.helper("joinfile.names",$joinfilename)
```

Parameters

The following parameter is valid for this function.

Parameter	Description
$joinfilename	The name of the join file whose sub file names array is to be retrieved.

Factors

The join file must be already defined in joindefs.

Return values

Null or an array of strings containing the sub file names.

2.4.2.26. RAD function: dbdict.helper("unique.key")

A RAD utility function that helps retrieve the first unique key or the primary key of a file efficiently.

Function

dbdict.helper("unique.key")

Format

```
dbdict.helper("unique.key",$L.file/$filename)
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$L.file/$filename	The variable or name of the file whose first unique key or primary key is to be retrieved.

Factors

Only the normal file type is supported. Other types of tables aren't supported.

This function returns the first unique key or the primary key from the datadict file instead of the dbdict file; the application layer will keep consistency between datadict and dbdict for the first unique key or the primary key.

Return values

An array of strings containing the names of all fields that constitute the first unique key or the primary key.

2.4.2.27. RAD function: delete

A RAD function that deletes one or more elements in an array and returns the updated array.

Function

delete

Format

```
delete(array, element number, [number of elements])
```

Where the `array` variable is the array, the `element number` is the index number of the first element to delete, and `[number of elements]` is the number of elements to delete. The number of elements to delete is optional. The default value is one (`1`).

Example

```
delete({1,2,3},2)
```

Returns `{1,3}`

```
delete({1,2,3},2,2)
```

Returns `{1}`

2.4.2.28. RAD function: denull

A RAD function that compresses an array by removing all trailing NULL entries and returns the compressed array.

Function

denull

Format

```
denull(array)
```

Factors

- If the array contains a NULL entry in the middle of the array, that entry is not removed.
- Using **denull** on an empty array of structures converts the array of structures to a simple array.



Note

Note Always test for NULL first.

- Displaying arrays extends the length of the array to accommodate window size.



Tip

Tip It is a good practice to denull arrays before records are added or updated. You can use `denull(contents())` to accomplish this, but you should first ensure that the contents don't include any empty array of structures or they will be converted to simple arrays.

Example

```
denull({1,2,3,,})
```

Returns `{1,2,3}`.

```
denull({1,,2,,})
```

Returns `{1,,2}`.

2.4.2.29. RAD function: descriptor

A RAD function that returns the database dictionary descriptor structure for the table specified.

Function

descriptor

Format

```
descriptor($filename)
```

Parameters

The `$filename` variable is used for the table whose descriptor structure you are requesting.

2.4.2.30. RAD function: display.value.copy

A RAD function that copies the internal display value cache from one file variable to another.

Function

display.value.copy

Format

```
display.value.copy($targetfile, $sourcefile)
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$targetfile	The file to which the internal display value cache is copied.
$sourcefile	The file from which the internal display value cache is copied.

Factors

The source or target file variable must be a valid file variable.

The old internal display value cache on the target file variable (if any) is released and a new internal display value cache is copied from the source file variable.

Return values

0: success

-1: failure

Example

See RAD function: same.

2.4.2.31. RAD function: evaluate

Evaluates an operator and may return a value.

Function

evaluate

Format

```
$e = evaluate($x)
```

Factors

- An operator is created using the parse function.
- The evaluate function is also valuable when the statement to be evaluated is contained in a Service Manager field that's not qualified once the application is compiled.
- The parse flag is used to parse data in a form, which in turn converts the data to an operator (data type 10).
- Although an *lvalue* is always required, a *value* is not always assigned to the *lvalue*.

Example

`$x=parse("1+1",1)` returns two (`2`).

`$x=evaluate($x)`

2.4.2.32. RAD function: evaluate.query

A RAD function that evaluates a query against a record. Both are passed as function parameters.

Function

evaluate.query()

Format

```
$L.rc=evaluate.query( <query>, <file variable> )
```

Function returns

`true` : if the record matches the query.

`false` : if it doesn't match the query.

`unkown` : if a comparison in the query encountered a NULL value or a field was referenced in the query that doesn't exist in the record.

`null` : if the function was called incorrectly.

Parameters

The following parameters are valid for the **evaluate.query** function:

Parameter	Data type	Description
query"<query>"	operator or character	Can be a previously parsed query or a string containing a query
record"<file variable>"	record	Must be a valid file variable

Examples

```
$x = evaluate.query( "true", $L.file )
```

This example returns `TRUE` for any valid file variable.

```
$query = parse( "contact.name # \"S\" ", 4 )
$y = evaluate.query( $query, $L.filed )
```

This example returns `TRUE` only for contacts records that start with "S" and `FALSE` for every other contacts record.

2.4.2.33. RAD function: exists

A Rapid Application Development (RAD) function that checks for the existence of a field in a table.

Function

exists

Format

```
exists(<field name>, $file)
```

Factors

- Returns `true` or `false`.
- Replaces the `index(<field.name>, descriptor($file))>0` expression.
- The *$file* variable must be of data type **6**. Any other data type returns false.
- The first parameter can be either a character type variable or a quoted string.

Example

```
$L.return=exists("schedule.id", $L.schedule)
```

The value of `$L.return` is `true` if `$L.schedule` is a file variable containing a schedule record.

2.4.2.34. RAD function: fduplicate

A RAD function that copies an entire file variable from one record to another.

Function

fduplicate

Format

```
fduplicate($target, $source)
```

Factors

- The system doesn't recognize **fduplicate** alone.
- Returns a Boolean value: `true` if it is successful, and `false` if it fails.
- Used primarily in RAD process panels and in Format Control.

Example

```
$L.void=fduplicate($file0, $file)
```

Creates `$file0`, an independent copy of `$file`.

2.4.2.35. RAD function: filename

A RAD function that returns the name of the file for the specified file variable.

Function

filename

Format

```
filename($filename)
```

Parameters

The `$filename` variable has been bound to a database file with the rinit command panel.

Factors

This function is useful when executing a common subroutine that has been passed a file variable. This function allows you to determine the file name because the local variable contains data from an unknown file.

Example

`filename($file)` returns the dbdict name.

2.4.2.36. RAD function: filequeryex

A RAD function that returns the query parameters of a file variable.

Function

filequeryex

Format

```
filequeryex(file)
```

Parameters

`file` is an Service Manager file variable, which can be either `$L.file` in RAD or `SCFile` in JavaScript.

Factors

This function returns an array with the following elements:

- No.0 is the query string, which supports both Service Manager queries and ADHOC SQL queries.
- No.1 is the sort-by fields, separated by a comma. Note that the group-by fields will be merged with a higher priority.
- No.2 is the sort-by order: 0 is ascending, 1 is descending, and an empty value is NULL.
- No.3 is the group-by fields, separated by a comma.
- No.4 is the group order: 0 is ascending, 1 is descending, and an empty value is NULL.



Note

Note To set the sort-by and group-by arrays, use " `var sortOrder=[SCFILE_ASC];` " instead of " `var sortOrder=new Array(SCFILE_ASC);` ".

Examples

Achoc query example:

```
var file = new SCFile("probsummary");
```

```
var sql = 'select affected.item, number from probsummary where affected.item isin (select logical.name from device
where owner=\""Administration\"")';
```

```
var success=file.doSelect( sql ) ;
```

```
var fields = new Array("affected.item");
```

```
var sortOrder=[SCFILE_ASC];
```

```
var rc = system.functions.setsort(file,fields,sortOrder);
```

```
var q = system.functions.filequeryex(file);
```

```
print("rc: " + success + ", q[0]: " + q[0] + ", q[1]:" + q[1] + ", q[2]:" + q[2]+", q[3]:" + q[3] + ", q[4]:" +
q[4]);
```

Non-adhoc query example:

```
var sql='number#"IM1"';
```

```
var f1 = new SCFile("probsummary");
```

```
var rc =f1.doSelect(sql);
```

```
var fields = new Array("affected.item");
```

```
var sortOrder=[SCFILE_ASC];
```

```
var rc = system.functions.setsort(f1,fields,sortOrder);
```

```
var q = system.functions.filequeryex(f1);
```

```
print("rc: " + success + ", q[0]: " + q[0] + ", q[1]:" + q[1] + ", q[2]:" + q[2]+", q[3]:" + q[3] + ", q[4]:" +
q[4]);
```

2.4.2.37. RAD function: frestore

A RAD function that restores all the fields in a file to their original values from the database.

Function

frestore

Format

```
frestore($file)
```

The `$file` variable is the file for which you want to restore the fields to their original values.

Factors

- Use only with a Windows client.
- This function returns no value, it is unnecessary to use it in an assignment statement.

Example

```
frestore($file)
```

If the `$file` variable is initialized and a record is selected, changes are made to the contents of the `$file` variable in memory, but the updates aren't yet written to the database. After execution, the `$file` variable is restored to the original value, which is the state stored in the database.

2.4.2.38. RAD function: genout

A RAD function that generates a string containing the contents of a record using a specified form. The output produced is either fixed or variable length.

Function

genout

Format

```
genout($file.variable, $format.name)
```

Parameters

The `$file.variable` is a variable containing a record or records to process.

The `$format.name` variable is the name of format to be used.

Factors

- Spaces are substituted for NULL in fixed length output.
- All normal format processing occurs, including input and output routines. Data can therefore be automatically reformatted or manipulated as part of the function. For example, a date can be converted from *02/28/05 00:00:00*to *February 28, 2005.*

Examples

```
$output=genout($operator, “operator.view”)
```

```
$output={“Name:joes Full Name:Joe Smith”,
```

```
“Printer:sysprint Email:joes@microfocus.com”}
```

2.4.2.39. RAD function: get.base.form.name

A RAD function that returns the base name of a form, without the .g (GUI) or .w (Web) file extension.

Function

get.base.form.name

Format

```
$base.form.name=get.base.form.name($form.name)
```

Parameters

$form.name

Examples

In each of the following examples, the result is the same after execution: `$base.form.name` is `operator`.

Example 1:

```
$form.name=“operator.g”
```

```
$base.form.name=get.base.form.name($form.name)
```

Example 2:

```
$form.name=“operator.w”
```

```
$base.form.name=get.base.form.name($form.name)
```

Example 3:

```
$form.name=“operator”
```

```
$base.form.name=get.base.form.name($form.name)
```

Example 4:

```
$form.name=“operator.a”
```

```
$base.form.name=get.base.form.name($form.name)
```

2.4.2.40. RAD function: get.dateformat

A RAD function that returns the date format, represented by a number, used by your operator ID.

Function

get.dateformat

Format

```
$date.fmt=get.dateformat()
```

Factors

Date formats represented, by number:

- 1 = *mm/dd/yy*
- 2 = *dd/mm/yy*
- 3 = *yy/mm/dd*
- 4 = *mm/dd/yyyy*
- 5 = *dd/mm/yyyy*
- 6 = *yyyy/mm/dd*

Example

```
$date.fmt=get.dateformat()
```

After execution for a typical user in the United States, the value of `$date.fmt` is one (`1`).

2.4.2.41. RAD function: get.display.value

This RAD function returns the display value of a field in a Service Manager file.

Function

get.display.value

Format

```
get.display.value($file, $fieldname, [index], [bForceFromDB])
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$file	The $file variable is the file from which you want to retrieve the display value.
$fieldname	The name of a field in the file. **Caution** Caution You can't use a variable for this parameter. For example, `$L.display=get.display.value($L.file, "$affected.cis", -1, true)` is unsupported.
[index]	This parameter is optional. If the field specified in $fieldname is of the array type, the index is the position number of an element in the array where the element is located. If the index is not specified or is -1, the entire array is retrieved instead.
[bForceFromDB]	This parameter is optional. This is a Boolean type variable, which can be true or false. If it is true, this function will bypass the internal display value cache associated with $file, and retrieve the display value directly from the RDBMS.

Factors

If an internal display value cache doesn't already exist, a null value is returned; however, if bForceFromDB is set to true, the Service Manager server looks for the display value directly from the RDBMS according to the current field value.

For an array type field, if bForceFromDB is set to true, this function combines the value in the display value cache (if any) and the value fetched directly from the RDBMS, and returns the combined value.

For a string type field, if bForceFromDB is set to true, while the Service Manager server can't find any value from the RDBMS, the value in the display value cache (if any) is returned instead.

There are two ways to create an internal display value cache associated with $file:

- Use the set.display.value function explicitly.
- Any time when users enter or change the display value from the client side and the client submits the request to the server side, the internal display value cache is updated. This is automatic and not under the user's control.

Return values

Can be null, a string type display value, or an array of string type display values.

Examples

```
function testDisplayValue()
{
        var sql = 'select * from cm3r where number="C10008"';
        var file = new SCFile('cm3r');
        var rc = file.doSelect(sql);
        print( "Field value of assets=" + file.assets );
        print( "Field value of logical.name=" + file.logical_name );
        var darray = new SCDatum();
        darray.push( "aaa" );
        darray.push( "bbb" );
        print( "Value of darray=" + darray );
        system.functions.set_display_value(file, "assets", darray, -1 );
        var value = system.functions.get_display_value(file, "assets" );
        print( "get.display.value for assets from internal display value cache=" + value );
        value = system.functions.get_display_value(file, "assets", -1, true );
        print( "get.display.value for assets from RDBMS=" + value );
        value = system.functions.get_display_value(file, "logical.name" );
        print( "get.display.value for logical.name from internal display value cache=" + value );
        value = system.functions.get_display_value(file, "logical.name", -1, true );
        print( "get.display.value for logical.name from RDBMS=" + value );
}
testDisplayValue();
```

This example returns the following results:

Field value of assets=[C++ object Datum] - {"CI1000984", "CI1000988"}

Field value of logical.name=CI1000984

Value of darray=[C++ object SCDatum] - {"aaa", "bbb"}

get.display.value for assets from the internal display value cache=[C++ object Datum] - {"aaa", "bbb"}

get.display.value for assets from the RDBMS=[C++ object Datum] - {"adv-Windows-101", "adv-Windows-102", "aaa", "bbb"}

get.display.value for logical.name from the internal display value cache=null

get.display.value for logical.name from the RDBMS=adv-Windows-101

2.4.2.42. RAD function: get.graph.action

Description

A RAD function that returns the activated action on a node in the graph diagram.

Function

get.graph.action()

Example

`$L.graphaction = get.graph.action()`

2.4.2.43. RAD function: get.graph.id

Description

A RAD function that returns the value of the graph ID for the expand action on a node in the graph diagram.

Function

get.graph.id()

Example

```
$L.graphid = get.graph.id()
```

2.4.2.44. RAD function: get.graph.target

Description

A RAD function that returns the target for an activated action on a node in the graph diagram.

Function

get.graph.target()

Example

```
$L.graphtarget = get.graph.target()
```

2.4.2.45. RAD function: get.graphnode.id

Description

A RAD function that returns the value of the node ID for the expand action on a node in the graph diagram.

Function

get.graphnode.id()

Example

```
$L.graphnodeid = get.graphnode.id()
```

2.4.2.46. RAD function: get.lock.owner

A RAD function that returns the owner of a resource (lock). The function returns the name of the user who currently has the resource locked.

Function

get.lock.owner

Format

```
$owner= get.lock.owner("locked resource")
```

Example

Assume the user "falcon" is actively updating Incident IM1017. By viewing the Lock Resource Status display, one can determine that the name of the associated resource is `probsummary;IM1017`. To verify the owner from any other application:

```
$lock="probsummary;IM1017"
$owner=get.lock.owner($lock)
```

After execution, the value of `$owner` is `falcon`.



Note

Note This function is primarily intended for internal use as it requires specific knowledge of the underlying resource names associated with a specific action.

2.4.2.47. RAD function: get.timezoneoffset

A RAD function that returns the absolute time difference between Greenwich Mean Time (GMT) and the time zone of the operator.

Function

get.timezoneoffset

Format

```
$tzoffset=get.timezoneoffset()
```

Example

```
$tzoffset=get.timezoneoffset()
```

After execution for a Pacific Standard Time (PST) time zone user, the value of the `$tzoffset` variable is `-08:00:00`.

2.4.2.48. RAD function: get.uid

A RAD function that returns a 32-character identifier that's unique to an installation of Service Manager. Each successive call returns a value greater than the previous value.

Function

get.uid

Format

```
$uid=get.uid()
```

Example

```
$uid=get.uid()
```

After execution, `$uid` contains a unique value such as 608AE3ED276562980000283400002F44.

2.4.2.49. RAD function: get.web.url

A RAD function that returns the URL of the web client.

If there's no parameter added, or if the value is set as false or 0, the URL address will be the Web Tier URL.

If the the value is set as true or 1, the URL address will be the original URL when the enableGetOriginalUrl parameter is enabled.

Function

get_web_url()

Format

```
system.functions.get_web_url()
```

Example

When the enableGetOriginalUrl parameter is enabled and the web tier full URL is http://12x.x.x.x/webtier-x.xx.xx/index.do?originalUrl=http://servicemanager.example.com/index.do

If you execute the following commands:

`print(system.functions.get_web_url());`

`print(system.functions.get_web_url(false));`

`print(system.functions.get_web_url("false"));`

`print(system.functions.get_web_url(0));`

After execution, the URL http://12x.x.x.x/webtier-x.xx.xx/index.do will be returned.

If you execute the following commands:

`print(system.functions.get_web_url(true));`

`print(system.functions.get_web_url("true"));`

`print(system.functions.get_web_url(1));`

After execution, the URL http://servicemanager.example.com/index.do will be returned.

Note that if the enableGetOriginalUrl parameter is disabled, then Unknown will be returned.

2.4.2.50. RAD function: gui

A RAD function that determines whether or not a process is running in GUI mode.

Function

gui

Format

```
$bool=gui()
```

Example

```
$bool=gui()
```

After execution in GUI mode, the value of `$bool` is `true`. After execution in text mode, value of `$bool` is `false`.

2.4.2.51. RAD function: index

A RAD function that returns the index or position number for the value of a specific element in an array or a character in a string. If the target value is not in the array or string, NULL is the return.

Function

index

Format

```
index(target value, $variable, starting position #)
```

Parameters

The `target value` variable is the value you are searching for in the array or string.

The `$variable` variable is the name of the variable to be searched.

The `starting position #` variable is the index in the array or position in the string where the search will begin. The default value is `1`).

Factors

- The **index()** function operates identically for arrays, regardless of the data type in the array.
- If the array is composed of structures or of arrays, the target variable must exactly match the structure or array.
  For example, the function doesn't allow selecting a field from a structure.
- You can use the index function to search for any value by converting both the value and the search variable to the same type.
  For example, if you want to search for the `$problem.number` variable on all forms, you could pass the following query to the select panel:

  ```
  index("$problem.number", str(field))
  ```

  The index function searches for the `$problem.number` variable in the `field` field of the format table.
- When you use the index() function in a query, Service Manager evaluates the function based on database case sensitivity.
  For example, if you use the Expert Search option on the Incident Management search screen, and search for `index("Windows Server", brief.description) > 0`:
  - On a case-insensitive database, the query returns all records where the `brief.description` field contains `windows server` or `WINDOWS SERVER` or `Windows SERVER`, and so on.
  - On a case-sensitive database, the query only returns records where the `brief.description` field contains exactly `Windows Server`.
- When you use the **index()** function in a RAD or JavaScript expression and not within a query, Service Manager evaluates it case-sensitively (regardless of the database setting).
  For an example, consider the following RAD or JavaScript expressions:
  - The expression `index("Windows Server", {"WINDOWS SERVER", "windows server"})`, returns 0.
  - The expression `index("Windows Server", {"WINDOWS SERVER", "Windows Server", "windows Server"})`, returns 2.

Example

`index(1,{1,2,3})` returns `1`

`index(2,{1,2,3})` returns `2`

`index(1,{1,2,3},2)` returns `0`

`index(2,{1,2,3},2)` returns `2`

2.4.2.52. RAD function: insert

A RAD function that returns an array with one or more inserted elements.

Function

insert

Format

```
insert( $array[, $position[, $number[, $value[, $denull]]]])
```

Parameters

- The `$array` parameter is the array into which an element is inserted.
- The `$position` parameter is the position of the first inserted element. If the value is zero (*0*), the element is inserted at the end of the array. This is an optional parameter. The default value is zero (*0*).
- The `$number` parameter is the number of elements to insert. This is an optional parameter. The default value is one (*1*). If the value is zero (*0*), this function will return without changing the array.
- The `$value` parameter is a value to insert into the new elements. This is an optional parameter. The default value is *NULL*.
- The `$denull` parameter is a logical value that determines whether or not the array should be denulled before inserting elements. This is an optional parameter. The default value is *true*.

Factors

If the source array has NULL elements at the end, this function automatically denulls the array before inserting new elements in any position. To avoid this, set the value of the `$denull` parameter to *false*.

Examples

The following examples show a desired result and the syntax used to achieve that result.

Action desired	Example and result

Insert an element at the beginning of an array.	`$a={"a", "b", "c", "d"}`
	`$a=insert($a, 1, 1, "z")`
	Where:
	The value of *$a* is *{"z", "a", "b", "c", "d"}*

Insert an element in the middle of an array.	To insert an element in the middle of an array:
	`$a={"a", "b", "c", "d"}`
	`$a=insert($a, 3, 1, "z")`
	Where:
	The value of *$a* is *{"a", "b", "z", "c", "d"}*

Insert an element at the end of an array.	`$a={"a", "b", "c", "d"}`
	`$a=insert($a, 0, 1, "z")`
	Where:
	The value of *$a* is *{"a", "b", "c", "d", "z"}*

The array is denulled.	`$a={"a", "b",  ,  , "e", "f",  ,  ,  , }`
	`$a=insert( $a, 0, 1, "z")`
	Where:
	The value of *$a* is *{"a", "b", , , "e", "f", "z"}*

To avoid denulling the array.	`$a={"a", "b",  ,  , "e", "f",  ,  ,  , }`
	`$a=insert( $a, 0, 1, "z", 0)`
	Where:
	The value of *$a* is *{"a", "b", , , "e", "f", , , , , "z"}*

The array is (again) denulled.	`$a={"a", "b",  ,  , "e", "f",  ,  ,  , }`
	`$a=insert( $a, 2, 1, "z")`
	Where:
	The value of *$a* is *{"a", "z", "b", , , "e", "f"}*

2.4.2.53. RAD function: iscurrent

A RAD function that determines if the record that you are working with is identical to the version in the database. A value of true or false is returned.

Function

iscurrent

Format

```
$Boolean=iscurrent($file.variable)
```

Factors

This function only determines if another user has changed the current record since you open it.

Examples

Example 1:

1. User1 logs in and opens record IM10005.
2. User1 modifies IM10005, but doesn't save it.
3. User1 calls `iscurrent($L.file)` with `$L.file` pointing to IM10005.
   A value of `true` is returned. This is because the record user1 gets in step 1 is identical to the version in the database.

Example 2:

1. User2 logs in and opens record IM10005.
2. User2 logins and opens record IM10005. (User2 gets the same version of record IM10005 as user1.)
3. User1 modifies IM10005 and saves it. (IM10005 in the database is changed.)
4. User2 calls `iscurrent($L.file)` with `$L.file` pointing to IM10005.
   A value of `false` is returned. This is because the record user2 gets in step 2 is not identical to the version in the database.

2.4.2.54. RAD function: isExpressionValid

A RAD function that determines if a RAD expression is a valid expression of a specified data type.

Function

isExpressionValid

Format

```
$e = isExpressionValid(expr_string, type)
```

Parameters

expr_string is a RAD expression, and *type* is a basic data type of Service Manager.

Factors

- It returns `true` or `false`.
- It works the same way as the **parse.evaluate()** function， except that it doesn't send error messages to the client when the input string is invalid.

Examples

```
$x=isExpressionValid(“abc”,3) returns false
```

```
$x=isExpressionValid("date(tod()) + 5*24*60", 3) returns true
```

2.4.2.55. RAD function: isfileexist

A RAD function that determines whether a file or a folder exists or not. If the file exists, this function returns `true`. Otherwise, this function returns `false`.

Function

isfileexist

Format

```
isfileexist("path",type)
```

Parameters

The `path` variable is the full path of a file or a folder.

The `type` variable is optional. If omitted, the value is set to 0 by default.

- 0: determines whether a file or a folder exists
- 1: determines whether a file exists
- 2: determines whether a folder exists

Example

```
$L.path="c:\\note.txt"
L.ret=isfileexist($L.path)
if ($L.ret=false) then ($L.void=rtecall("msg", $L.code, "File doesn't exist"))
else ($L.void=rtecall("msg", $L.code, "File exists"))
```

2.4.2.56. RAD function: jscall

A RAD function that enables you to invoke JavaScript from within RAD.

Function

jscall

Format

```
jscall( "script.function" , arg1 , arg2, ... , argN )
```



Note

Note

If the `"script.function"` returns const strings rather than fields, you need to wrap the return value with quotes.

For example, if the `"script.function"` returns the const strings, the following usage may cause some problems:

```
function Test()

{

return "test-test";

}
```

Instead, you need to use the following standard format:

```
function Test()

{

return "\"test-test\"";

}
```

Parameters

The following parameters are valid for the **jscall** function:

Parameter	Data type	Required	Description
"script.function"	String	Yes	Refers to a script in the ScriptLibrary and a particular function within the script. This parameter must be enclosed in double quotes.

arg1, arg2, – , argN	Varies with each argument	The required arguments depend upon the requirements of the script you call.	The arguments passed to JavaScript. You can pass an infinite number of arguments. Arguments aren't enclosed in quotes or double quotes and can be RAD expressions, as shown in the following examples:
			`city in $L.OLD`
			Or:
			`level in $L.NEW=0`

2.4.2.57. RAD function: lioption

This RAD function indicates whether a given input value is a licensed option on the current system.

Function

lioption

Format

```
lioption(input value)
```

Parameters

Parameters for this function are as follows:

Input value	Description
Asset Contract Management	Authority to use Asset Contract Management application
Change Management	Authority to use Change Management application
Configuration Management	Authority to use Configuration Management application
Contract Management	Authority to use Contract Management application
Incident Management	Authority to use Incident Management application
IR Expert	Authority to use IR Expert
Knowledge Management	Authority to use Knowledge Management application
Knowledge Management ESS	Authority to use Employee Self-Service with the Knowledge Management application
Problem Management	Authority to use Problem Management application
RAD Compiler	Authority to use RAD Compiler
Request Management	Authority to use Request Management application
SCAuto/SDK Unix	Authority to use SC Automate Adapters Unix SDK
SCAuto/SDK Windows	Authority to use SC Automate Adapters Windows SDK
SCAuto/HP Network Node Manager	Authority to use SC Automate Adapters OpenView
SCAuto/Netview (AIX/NT)	Authority to use SC Automate Adapters AIX Netview
SCAuto/SunNet	Authority to use SC Automate Adapters SunNet
SCAuto/SPECTRUM	Authority to use SC Automate Adapters SPECTRUM
SCAuto/HP Operations (ITO/VPO)	Authority to use SC Automate Adapters IT Ops
SCAuto/UNIXmail	Authority to use SC Automate Adapters SMTP or UNIX mail
SCAuto/MAPImail	Authority to use SC Automate Adapters MAPI mail
SCAuto/Lotus Notes	Authority to use SC Automate Adapters Lotus Notes
SCAuto/Netview OS/390	Authority to use SC Automate Adapters OS/390 Netview
SCAuto/TBSM	Authority to use SC Automate Adapters TBSM
SCAuto/Unicenter AMO	Authority to use SC Automate Adapters Unicenter
SCAuto/Unicenter TNG	Authority to use SC Automate Adapters CA TNG
SCAuto/Tivoli	Authority to use SC Automate Adapters Tivoli
SCAuto/Tally NetCensus	Authority to use SC Automate Adapters Tally NetCensus
SCAuto/Remedy ARS	Authority to use SC Automate Adapters Remedy ARS
SCAuto/ERP Connection	Authority to use SC Automate Adapters ERP Connection
Scheduled Maintenance	Authority to use Scheduled Maintenance application
Self-Service Ticketing	Authority to use Self-Service Ticketing
Service Catalog	Authority to use Service Catalog application

Service Desk	Authority to use Service Desk application
SLA	Authority to use Service Level Agreements application
SOAP API	Authority to publish Service Manager data as Web services

Factors

The function returns a true value if the input value is part of the system license.

The function returns a false value if the input value is not part of the system license.

Examples

The following table describes a desired action and the syntax used to initiate the action.

Action desired	Syntax
Check for authorization to use Change Management	`$licensecheck=lioption("Change Management")`
Check for authorization to publish Web services	`$licensecheck=lioption("SOAP API")`

2.4.2.58. RAD function: lng

A RAD function that returns the number of elements in an array or structure and the number of characters in a string. Remember to denull arrays to get the correct number of elements, but be careful when denulling arrays of structures.

Function

lng

Format

```
lng($variable)
```

Where `$variable` is the array, structure, or string.

Factors

- The `$variable` must be an array, structure or a character string.
- Arrays return the number of elements, `NULL` or otherwise.
- Strings return the number of characters.
- Structures return the number of fields.

Examples

`lng("1234567890")` returns `10`.

`lng({1,2,3})` returns `3`.

`lng({1,2,3,,,,})` returns `7`.

`lng(denull({1,2,3,,,,}))` returns `3`.

2.4.2.59. RAD function: locks

A RAD function that returns an array of structures containing information about the locks used to lock a database for an update or the locks set by RAD applications using a locked command panel.

Function

locks

Format

```
locks()
```

Factors

Each element of the array contains:

- lock time
- process id

- terminal id
- operator name
- resource name (name of the lock)
- number
- exclusive
- locked
- breakable

Example

```
locks()
```

Returns the following:

```
{{['12/26/96 16:01:07', 20212, "SYSTEM", "marquee", "agent:marquee",
0, false, true, false]}
```

```
{["01/02/97 11:26:08", 22699, "Windows 32", "falcon", "AG/pm.main", 0, true, true,
false]}}
```

2.4.2.60. RAD function: logoff

A RAD function that logs off the current user and terminates the session.

Function

logoff

Format

```
logoff()
```

Factors

This function doesn't return a value. It executes a function.

2.4.2.61. RAD function: logon

A RAD function that logs a user on.

Function

logon

Format

```
logon()
```

Factors

This function doesn't return a value. It executes a function.

2.4.2.62. RAD function: mandant

The Service Manager mandanten feature protects records of multiple clients by logically separating a shared database. Clients see only the data they are allowed to share. The mandant RAD function allows an application to create a subset of the current mandanten values. For example, if a new record is being opened and the client (customer) for the record is known, then a subset of the mandanten values can be created to show only the assignments that are valid for that client.

Function

mandant

Format

```
mandant(n,string)
```

Where `n` equals `0`, `1`, `2`, or `3`.

Parameters

- `0` represents a request for a subset of the current mandanten values.
- `1` represents a request to restore the original mandanten values.
- `2` links the current user to the set of records in the scsecuritygroup file that matches the array of security IDs.
- `3` sets the security roles for the tables defined in the scFolderAccess file.

The value of the `string` variable is the mandanten value.

Example usage

In Service Manager the out-of-box system has three scsecurity group records: "Company A", "Company B" and "Company C". To establish the mandanten security for the user, the RAD could call the following:

```
mandant(2, {"Company A"}) or mandant(2,{"Company A", "Company B"})
```

Security roles are implemented in the scFolderAccess table. The RAD calls the mandant function with 3 to set the security roles for the given tables. The following example sets the security role of ADMIN for two tables, as defined in the scFolderAccess file.

```
mandant(3, "ADMIN",{"cm3", "device"})
```



Note

Note

This fails if the string value doesn't match the values currently established.

2.4.2.63. RAD function: max

A RAD function that returns the largest value in a list of values or arrays.

Function

max

Format

```
max(element1, element2, element3, ...)
```

Or:

```
max($variable)
```

Where the `$variable` variable is an array or a list of values.

Factors

- The max function returns the maximum value of either times or numbers.
- If you include an array in the list of elements, the system evaluates all of the elements in the array as if they were individual values.
- An attempt to use an array of strings as a parameter results in a segmentation fault, which causes the application to exit in error.

Example

`max(17,24,35,73,10)` returns `73`.

`max(1,{2,3,4},3)` returns `4`.

2.4.2.64. RAD function: messages

A RAD function that provides logging functions for use in the memory message log. The log is implemented as a wraparound cache of the *n* most recent messages.

Function

messages

Format

```
messages($function_number, $function_parameter)
```

Parameters

Parameters for this function are as follows:

Function	Parameters	Action
0	0-500	Open message log with 0 to 500 entries.
1	None	Start or resume logging.
2	None	Stop logging.
3	None	Stop logging and clear the log.
4	None	Close the log and recover memory.
5	0	Retrieve array of all log entries in most recently displayed order. The log is not cleared.
5	1	Retrieve array of logged informational messages in last displayed order; excludes error and action messages. The log is not cleared.
5	2	Retrieve array of logged action messages in last displayed order; excludes error and informational messages. The log is not cleared.
5	3	Retrieves array of error messages in last displayed order; excludes informational and action messages. The log is not cleared.

Factors

The following table outlines the proper order for each action related to this function and the corresponding function number.

Action	Function
Open log.	Function zero (0)
Start logging.	Function one (1)
...run application	N/A
Stop logging.	Function two (2)
Retrieve log.	Function five (5)
Clear log.	Function three (3)
start logging	Function one (1)
...run application	N/A
...run application	N/A
...run application	N/A
...run application	N/A
Stop logging.	Function two (2)
Retrieve log.	Function five (5)
Close log.	Function four (4)

Note

Note

- A user task (the sm process) can have only one private message log at a time.
- You can't directly access message logs that are using other tasks.
- A maximum of 500 logged messages is supported.

Examples

The following table describes a desired action and the syntax used to initiate the action.

Action desired	Syntax
Open a log for 200 messages.	`$throwaway=messages(0,200)`
Start logging	`$throwaway=messages(1)`
Stop logging.	`$throwaway=messages(2)`
Clear log.	`$throwaway=messages(3)`
Close log.	`$throwaway=messages(4)`
Retrieve all logged messages.	`$array=messages(5,0)`
Retrieve all logged informational messages.	`$array=messages(5,1)`
Retrieve all logged action messages.	`$array=messages(5,2)`
Retrieve all logged error messages.	`$array=messages(5,3)`

2.4.2.65. RAD function: min

A RAD function that returns the smallest element in a list of values or arrays.

Function

min

Format

```
min(element1, element2, element3, ...)
```

Or:

```
min($variable)
```

Where the `$variable` variable is an array or a list of values.

Factors

- The min function returns the minimum value of either times or numbers.
- If you include an array in the list of elements, Service Manager evaluates all elements in the array as individual values.
- An attempt to use an array of strings as a parameter results in a segmentation fault, which will cause the application to exit in error.

Example

`min(17,24,35,73,10)` returns `10`.

`min(4,{1,2,3,},2)` returns `1`.

2.4.2.66. RAD function: modtime

A RAD function that returns the last modified date and time of a record.

Function

modtime

Format

```
modtime($file)
```

Factors

- Use this function to determine if a record has been modified before or after a specified time.
- Use `modtime(currec())` to return the last modified date and time for any record. You can also use this variation in queries to retrieve all records modified after a specified point in time.
- The **modtime** function is not updated on clients when the records are updated on the server until after the records are selected again.

Examples

`modtime($file)` returns `1/10/96 12:15:17`.

`modtime(currec)>tod( )- '08:00:00'` selects all records in a file that were added or updated in the last eight hours.

2.4.2.67. RAD function: month

A RAD function that returns the month of year for a specified date, regardless of the date format.

Function

month

Format

```
$ymonth=month($date)
```

Parameters

Where the `$date` variable is the date and time value.

Example

```
$ymonth=month(`2/15/96')
```

After execution, the value of `$ymonth` is `2`.

2.4.2.68. RAD function: multiselect.selection("fieldcontents")

A RAD function that returns the contents of the field associated with the multiselection. Field contents are only returned if the associated column name is set using the Selection Field property in Forms Designer.

To set the associated column name using the Selection Field property in Forms Designer, follow these steps:

1. Sign in to Service Manager as a System Administrator.
2. Click **Tailoring** > **Forms Designer**.
3. In the **Form** field, type the form name. For example, type probsummary.qbe.g and click **Search**. The form opens.
4. Click **Design** to open design mode.
5. Click the table object.
6. In the **Selection Field** property, type the name of the column on the table (for example, type number).
7. Click **OK** to see what the form will look like in the Windows client.
8. Click **OK** to save your changes.

Function

multiselect.selection("fieldcontents")

Format

```
$info=multiselect.selection("fieldcontents")
```

Example

```
$x=multiselect.selection("fieldcontents")
```

After execution, the value of `$x` is set to the selected field contents. For example, `{"IM10003", "IM10006", "IM10007", "IM10008", "IM10010", "IM10012"}`.



Note

Note

Field contents are only returned if the associated column name is set using the Selection Field property in Forms Designer.

2.4.2.69. RAD function: multiselect.selection("fieldname")

A RAD function that returns the field name of the column associated with the multiselection. The associated column name is set using the Selection Field property in Forms Designer.

To set the associated column name using the Selection Field property in Forms Designer, follow these steps:

1. Sign in to Service Manager as a System Administrator.
2. Click **Tailoring** > **Forms Designer**.
3. In the **Form** field, type the form name. For example, type probsummary.qbe.g and click **Search**. The form opens.
4. Click **Design** to open design mode.
5. Click the table object.
6. In the **Selection Field** property, type the name of the column on the table (for example, type number).
7. Click **OK** to see what the form will look like in the Windows client.
8. Click **OK** to save your changes.

Function

multiselect.selection("fieldname")

Format

```
$info=multiselect.selection("fieldname")
```

Example

```
$x=multiselect.selection("fieldname")
```

After execution, the value of `$x` is set to the column name (for example,"number").



Note

Note Field contents are only returned if the associated column name is set using the Selection Field property in Forms Designer.

2.4.2.70. RAD function: multiselect.selection("rows")

A RAD function that returns an array of row numbers that were selected.

Function

multiselect.selection("rows")

Format

```
$info=multiselect.selection("rows")
```

Example

```
$x=multiselect.selection("rows")
```

After execution, the value of `$x` is set to the selected rows. For example `{2, 3, 7, 22, 23, 30}`.

2.4.2.71. RAD function: multiselect.selection("selected")

A RAD function that returns true or false if the provided variable has multiple records selected.

Function

multiselect.selection("selected",$file.variable)

Format

```
$info=multiselect.selection("selected",file.variable)
```

Example

```
$x=multiselect.selection("selected",$L.file)
```

After execution, the value of `$x` is set to `true` or `false`.

2.4.2.72. RAD function: multiselect.selection("selections")

A RAD function that returns the number of rows selected.

Function

multiselect.selection("selections")

Format

```
$info=multiselect.selection("selections")
```

Example

```
$x=multiselect.selection("selections")
```

After execution, the value of `$x` is set to the number of selected rows. For example, `6`.

2.4.2.73. RAD function: multiselect.selection("tablename")

A RAD function that returns the Table ID associated with the multiselection.

Function

multiselect.selection("tablename")

Format

```
$info=multiselect.selection("tablename")
```

Example

```
$x=multiselect.selection("tablename")
```

After execution, the value of `$x` is set to the selected Table ID. For example, `Table0`.

2.4.2.74. RAD function: null

A RAD function that returns a value of true if the parameter is NULL or is comprised of a compound data type consisting of all NULL values. Use the null function to determine whether or not the value of a field or variable is null.



Note

Note The **null** function is different that the reserved word, *NULL*.

Function

null

Format

```
null(value)
```

Factors

- The null function can handle compound data types.
- The reserved word NULL only handles primitive data types.
- An array or structure is null if it contains all null elements. This function applies itself recursively to nested elements.
- An empty string, such as `(" ")` is not null.
- A zero (0) is not null.
- The value `00:00` is not null.
- A value of `unknown` in a Boolean field is not null.

Examples

`null(1)` returns `false`.

`null(NULL)` returns `true`.

`null({})` returns `true`.

`null({1,})` returns `false`.

`null({{},})` returns `true`.

2.4.2.75. RAD function: nullsub

A RAD function that substitutes a null value with the second value specified.

Function

nullsub

Format

```
nullsub(value1,value2)
```

Where the `value2` variable is the value to return if `value1` is null.

Examples

`nullsub(1,2)` returns `1`.

`nullsub(NULL,2)` returns `2`.

`$a=NULL`

`$a=nullsub($a,"test")` returns `test`.

2.4.2.76. RAD function: operator

A RAD function that sets and returns the name of the current operator ID. The operator function is also used to set the operator name.

Function

operator

Format

```
operator()
```

Factors

The `operator()` value is set to the value of the login name in the operator record at the time of login.

Examples

```
$a = operator()
```

```
operator() = $a
```

2.4.2.77. RAD function: option

A RAD function that returns the number of the last interrupt key. For example, 0 = Enter, 1 = first option key, 2 = second option key.

Function

option

Format

```
option()
```

Factors

- The Enter key returns a value of zero (0).
- Use caution when option() is a criteria for a decision or process panel that can be accessed from several paths. To avoid conflicts, set a variable to option() when the first panel is accessed.
- Clicking a button returns the number defined in the Button ID property.
- Clicking an item in the More menu returns the GUI option number associated with this option in the displayoption table.

Example

`option()` returns `10`.

2.4.2.78. RAD function: parse

Parses a string into an operator.

Function

parse

Format

```
parse(string,type)
```

Factors

- An operator can be an expression, such as *type<11*, or a statement, such as *type=11*.
- When enabled on forms, the parse property automatically parses the data entered.

Examples

```
$x=parse("1+1",1)
```

Evaluate($x) returns `2`.

```
$x=parse("$x=1",11)
```

Evaluate($x) causes the value of `$x` to be set to `1`.

2.4.2.79. RAD function: perf

A RAD function that evaluates system performance and then logs this information in Service Manager's systemperform and systemtotals tables. A value of zero (`0`) indicates success, and a value of negative one (`-1`) indicates failure.

Function

perf

Format

```
perf(n)
```

Where the value of `n` is the number that corresponds to the option you select.

Parameters

The following table shows the parameters used with the **perf** function:

Option	Result
1	Delete records in the systemtotals and systemperform files where capture=false before writing new information to disk.
2	Add records and don't delete. Nothing is removed before new records are written to the systemtotals and systemperform files. Maintenance of these files is typically the responsibility of the RAD programmer.
3	Delete records in the systemtotals and systemperform files where capture=false and don't write information to disk.
4	Delete all records in the systemtotals and systemperform files, regardless of capture status.

Example

`perf(1)` Returns zero (`0`).

2.4.2.80. RAD function: printer

A RAD function that sets and returns the value of the currently logged-on printer.

Function

printer

Format

```
printer()
```

Factors

This function is most often used to set the current printer. When an operator logs on, the login application is executed, assigning printer() to be either the printer specified by the user at login, or the default printer specified in the operator record for the user.

Example

```
$a = printer()

printer() = $a
```

2.4.2.81. RAD function: policyread

A RAD function that reads the data policy from a datadict table.

Function

policyread

Format

```
$L.returnvalue = policyread( $L.file/$filename, fieldname, fieldsetting)
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$L.returnvalue	The policy value returned after it is read from the database. It can be a string, a boolean, or null.
$L.file/$filename	This can be either a file variable or a file name: $L.file - The file handle of the table for which the data policy information is to be read from the database. It can be a join table, a normal table or an adhoc table. Note: Other kinds of tables (merge files, etc.) aren't supported. $filename â€“ The name of the table for which the data policy information is to be read from the database. It can be a normal table only. Note: All other kinds of tables aren't supported. The table name is the value in the Name field in the datadict record.
fieldname	The name of the field in the specified table for which the data policy information is to be read from the database. The field name is the value in the Field Name field in the datadict record.
fieldsetting	The field setting on the specified field for which the data policy information is to be read from the database. If no field setting is passed in when calling policyread function, the function will return true if the field exists in data policy, otherwise will return false. This is one of the fields on Field Settings tab of the datadict record. Valid field settings are: "invisible", "readonly", "mandatory", "captions", "avail","encrypt", "defaults", "globallist", "matchfields", "matchfiles", and "validations", â€?typesâ€?, â€œreference.tableâ€?. In addition, there two special settings: â€œdisplay.fieldâ€?, â€œautocomplete.show.fieldâ€?, if one of these settings is specified, the field name will be ignored.

Factors

If the fieldsetting is not in the list of Field Settings in the datadict table, the server will throw an error message. In any other case, this function returns a valid value or null.

Return values

A String, Boolean or null value.

Examples

Example 1

This example demonstrates how you could use a JavaScript to access the data policy on a simple file, such as the operator table.

```
// Access DataPolicy definition on operator file.

var operatorFile = new SCFile( "operator" );
var rteReturn;

rteReturn = system.functions.policyread( operatorFile , "profile.change", "invisible" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:invisible) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "readonly" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:readonly) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "mandatory" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:mandatory) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "captions" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:captions) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "avail" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:avail) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "encrypt" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:encrypt) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "defaults" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:defaults) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "globallist" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:globallist) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "matchfields" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:matchfields) "
+ rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "matchfiles" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:matchfiles) " +
rteReturn );

rteReturn = system.functions.policyread( operatorFile , "profile.change", "validations" );
print( "DataPolicy on (file:operator, fieldname:profile.change), as defined in (file:datadict, field:validations) "
+ rteReturn );
```

Example 2

This example demonstrates how you could use a JavaScript to access the data policy on a complex file, such as a join of the incidents and contacts tables.

```
// Access DataPolicy definitions on join file inccontacts (incidents and contacts)

var incident_contacts_file = new SCFile( "inccontacts" );
var rteReturn;

// Access policy definition on file(contacts) and field(city) and datadict field defaults
rteReturn = system.functions.policyread( incident_contacts_file, "file.contacts,city", "defaults" );
print( "DataPolicy on (joinfile:inccontacts, subfile:contacts, field:city), as defined in(file:datadict,
field:defaults)" + rteReturn );

// Access policy definition on file(incidents) and field(phone) and datadict field defaults
rteReturn = system.functions.policyread( incident_contacts_file, "file.incidents,phone", "defaults" );
print( "DataPolicy on (joinfile:inccontacts, subfile:incidents, field:phone), as defined in(file:datadict,
field:defaults)" + rteReturn );

// Access policy definition on file(incidents) and field(phone), and datadata field readonly
rteReturn = system.functions.policyread( incident_contacts_file, "file.incidents,phone", "readonly" );
print( "DataPolicy on (joinfile:inccontacts, subfile:incidents, field:phone), as defined in(file:datadict,
field:readonly)" + rteReturn );
```

Example 3

```
function testPolicy()
{
        var file = new SCFile( 'cm3r' );

        var ret = system.functions.policyread(file, "assets", "reference.table" );
        print( "The reference.table attribute for assets field in cm3r file is: ", ret );

        ret = system.functions.policyread(file, "assets", "" );
        print( "assets field in cm3r exists in data policy: ", ret );
        ret = system.functions.policyread(file, "nofield", "" );
        print( "nofield field in cm3r exists in data policy: ", ret );

        ret = system.functions.policyread(file, "", "display.field" );
        print( "display.field for file cm3r is: ", ret );

        ret = system.functions.policyread("device", "", "display.field" );
        print( "display.field for file device is: ", ret );
}
```

Example 3 returns the following results:

```
The reference.table attribute for assets field in cm3r file is:  device
```

```
assets field in cm3r exists in data policy:  true
```

```
nofield field in cm3r exists in data policy:  false
```

```
display.field for file cm3r is:  null
```

```
display.field for file device is:  display.name
```

2.4.2.82. RAD function: processes

A RAD function that returns an array describing all processes.

Function

processes

Format

```
processes(s)
```

Factors

Each element of the array contains the following information:

- login time
- session id
- device name
- user name
- idle time
- process id
- thread id
- hostname
- IP address

The argument of *s* determines the types of processes to be returned:

S	Type of process
null	All
ALL	All
SYSTEM	System (terminal="SYSTEM")
USER	User (not System)
ACTIVE	Active
INACTIVE	Inactive (not active)

Example

`processes("ACTIVE")` returns `{{['04/06/09 11:02:22', 11, "Soap-Windows Vista", "System.Admin", '00:00:00', 1692, 5164, "HPSD9667", "15.80.163.173"]}}`.

2.4.2.83. RAD function: prof

A RAD function that returns system performance statistics while checking the system resources used by an application.

Function

prof

Format

```
prof(n)
```

Where:

The value of `n` is the value for the parameter corresponding to the option you want returned.

Parameters

The following parameters are valid for the prof function:

Parameter	Description
1	Returns the user CPU time.
2	This parameter is obsolete.
3	Returns the memory allocated.
4	This parameter is obsolete.
5	This parameter is obsolete.
6	This parameter is obsolete.
7	This parameter is obsolete.
8	This parameter is obsolete.
9	This parameter is obsolete.
10	This parameter is obsolete.
11	This parameter is obsolete.
12	This parameter is obsolete.
13	This parameter is obsolete.
20	Returns the symbol table size.
21	Returns the stack size.

Example

`prof(5)` returns `7752`.

2.4.2.84. RAD function: recordcopy

A RAD function that copies a set of fields from one record to another record.

Function

recordcopy

Format

```
$L.voide=recordcopy($source.file, $source.fields, $target.file, $target.fields)
```

Parameters

The following parameters are valid for the **recordcopy** function:

Parameter	Description
$L.void=	Syntax requirement only, doesn't affect the outcome of the function.
$source.file	The source record from which fields are copied.
$source.fields	Array of field names in $source.file.
$target.file	The target record to which fields are copied.
$target.fields	Array of field names in $target.file.

Factors

The recordcopy function doesn't check the data type. Ensure that you don't copy a number field to a string, or a structure to a number.

Example

`$source.file` is an operator record, and `$target.file` is a contacts record.

```
name in $source.file="falcon"
phone in $source.file="858-481-5000"
$source.fields={"name", "phone"}
$target.fields={"contact.name", "contact.phone"}
$L.void=recordcopy($source.file, $source.fields, $target.file, $target.fields)
```

After execution, the value of `contact.name` in `$target.file` is `falcon` and `contact.phone` in `$target.file` is `858-481-5000`.

2.4.2.85. RAD function: recordtostring

A RAD function that uses the value of a field from an array and appends that value to a string.

Function
: recordtostring

Format
: recordtostring($string, $file, $arrayofnames, $sep)

Parameters
: The following values are valid for the recordtostring function:

Parameter	Description
$string	The string to which the field value is appended.
$file	The current file.
$arrayofnames	The array from which the field is taken.
$sep	A separator character.

Factors

Event Services uses the caret character (^) as a default separator. You can use the recordtostring function to build an eventout string.

Example

```
recordtostring("", $file, {"location", "location.name"}, "^")
```

Where:

The value of `$file` is a location record in which `location=Advantage HQ`, and `location.name=downtown`. After execution, the value of `$string` will contain `Advantage HQ^downtown`.

2.4.2.86. RAD function: round

A RAD function that rounds up, in the positive direction, to the nearest specified number of decimal places.

Function

round

Format

```
round(number,n)
```

Parameters

The following parameters are valid for the round function:

Parameter	Description
number	The number to round.
n	The number of decimal digits to use when rounding. This number can be any input that equates to a numeric value, such as an expression, a variable, or a number.

Example

```
$a = round(3.47,1)
$a = 3.5

$a = round(3.53,1)
$a = 3.5

$a = round(3.45,1)
$a = 3.5

$a = round(-3.45,1)
$a = -3.4

$a = round(100/30,2)
$a = 3.33

$a = round('1/19/96 06:19:34',0)
$a = '1/19/96 06:20'
```

2.4.2.87. RAD function: same

A RAD function that compares two values. If the values are equal or both values are null, the result is true; otherwise, the result is false.

Function

same

Format

```
same(value1,value2)
```

Parameters

The following parameters are valid for this function.

Parameter	Description
value1	The first value to be compared.
value2	The second value to be compared.

Factors

Use this function to compare arrays and structures, nulls, files, or empty strings.

When using the same function to compare arrays or structures, preface the variable names with denull, as shown in the following example:

```
not same(denull($filea), denull($fileb))
```



Note

Note

When using the **same** function to compare two files, the files must have the same schema, data, display value, and so on to return a result of True.

Return values

- True: same
- False: not same

Examples

`same(1,NULL)` returns `false`.

`same(NULL,NULL)` returns `true`.

`same({},{,})` returns `true`.

`same({1},{1,})` returns `true`.

`same("",NULL)` returns `false`.

The following is another example:

```
function testdisplayvaluecopy()
{
        var sql = 'select * from cm3r where number="C10001"';
        var file = new SCFile('cm3r');
        var file1 = new SCFile('cm3r');
        var rc = system.functions.fduplicate( file, file1 );

        rc = file.doSelect(sql);
        rc = system.functions.same( file, file1 );
        print("file is same as file1 after file.doSelect: ", rc );

        rc = file1.doSelect(sql);
        rc = system.functions.same( file, file1 );
        print("file is same as file1 after file1.doSelect: ", rc );
        var vRet = system.functions.set_display_value(file1, "assets", "testbystone", 1);

        rc = system.functions.same( file, file1 );
        print("file is same as file1 after set.display.value: ", rc );
        vRet = system.functions.display_value_copy(file, file1);

        rc = system.functions.same( file, file1 );
        print("file is same as file1 after display.value.copy: ", rc );
}
testdisplayvaluecopy();
```

This example returns these results:

```
file is same as file1 after file.doSelect:  false
```

```
file is same as file1 after file1.doSelect:  true
```

```
file is same as file1 after set.display.value:  false
```

```
file is same as file1 after display.value.copy:  true
```

2.4.2.88. RAD function: scmsg

Returns a text string matching the format of the message ID number and message class specified. The function replaces any variables in the message with the text specified in an array of values.

Function

scmsg

Format

```
scmsg($id, $class, $arrayof values)
```

Parameters

The following parameters are valid for the scmsg function:

Parameter	Data type	Description
$id	Number	The ID number of the message (message.id) from the scmessage table.
$class	String	The Message class from the scmessage table.
$arrayofvalues	Array	An array of substitution text for the `%S` variables in the message text from the scmessage table.

Factors

- All substitution text must be `%S`, which indicates that a string is used to provide the data. Each entry in the array of substitution text is examined for the type. If it's not a string type, it is converted to a string.
- You must create a separate record in the scmessage table for the message in each language you want to display.
- The $arrayofvalues defined in the scmsg function are applied to the message in the scmessage table corresponding to the language defined for the client login.
- When adding a scmessage record to the scbutton class, the scballoon class, the sccaption class, and the sctitle class, the message ID must end with `_<number>`. `<number>` is a string of decimal positions of the uppercase characters in the message ID. For example, `My Affected Example_1413`. In this example, `1413` symbolizes the positions of the uppercase characters of `M` (1), `A` (4), and `E` (13) in the message ID, respectively.
- When calling the `scmsg()` function, the message ID must be used without the `_<number>` suffix.

Example

```
scmsg("21", "cib", {"mail", "middle,caller.id", "open"})
```

Resulting message from the scmessage table:

```
Built macro to %S to %S on %S.
```

When the %S array values from the scmsg function in the example are applied, the message is displayed as:

```
Built macro to mail to middle,caller.id on open
```

2.4.2.89. RAD function: set.display.value

A RAD function that sets the display value for a specified field of a file variable. The display value can then be retrieved by using the get.display.value($file,$fieldname) syntax.

Function

set.display.value

Format

```
set.display.value($file,$fieldname,$dispvalue,[index])
```

Parameters

The following parameters are valid for this function.

Parameter	Description
$file	The file variable for which the display value is set
$fieldname	The field name in the file
$dispvalue	The display value to be set
[index]	This parameter is optional. If the field specified in $fieldname is of the array type, the index is the position number of an element in the array where the element is located; if the index is not specified or is -1, $dispvalue must be an array type value as well.

Factors

This function sets the display value in an internal display value cache associated with the $file variable. If an internal display value cache doesn't already exist, this function creates a cache.

Return values

Null or the old display value, which can be an array of strings or a string.

Example

See RAD function: get.display.value

2.4.2.90. RAD function: set.timezone

A RAD function that sets the time zone, the translation from local to internal characters, and the date format. Time zone is established at either user login or process startup time. Note that not all processes that are started are user processes. At login, Service Manager determines the time zone first by the company record and then the operator record.

Function

set.timezone

Format

```
set.timezone(servertz)
```

Parameters

The following parameter is valid for the set.timezone function:

Parameter	Data type	Description
servertz	File/record	A record in a Service Manager tzfile table.

Example

```
$void=rtecall("rinit",$r,$L.tz, "tzfile")
$void=rtecall("select",$r,$L.tz, "name=\"Australia/Queensland\"")
set.timezone($L.tz)
```

2.4.2.91. RAD function: setsort

A RAD function that sorts the fields in a table. The setsort function allows for a third parameter so that it could be an array of numbers, zero or one (`0` or `1`). The entries in this array correspond to the entries in the name array. A `0` indicates the field will be used to sort in ascending order and a `1` indicates the field will be used to sort in descending order. A `0` or `1` can also be used to sort on all fields in ascending or descending order.

Function

setsort

Format

p

```
 setsort($L.file, $L.arrayofnames, 0): ascending sort
setsort($L.file, $L.arrayofnames, 1): descending sort
$L.void=setsort($L.file, $L.arrayofnames, $L.arraysortorder)
```

Example

If you wanted to sort a list of contacts by name (ascending), age (ascending), and state (descending).

```
$L.arrayofnames={"name","age","state"}
$L.arraysortorder={0,0,1}
$L.void=setsort{$L.contacts, $L.arrayofnames, $L.arraysortorder}
```

2.4.2.92. RAD function: shutdown

A RAD function that shuts down Service Manager from within the system. No values are returned, this function only performs an operation.

Function

shutdown

Format

```
shutdown()
```

Factors

Place this function on the last panel to be executed in a shutdown application. Once the function is executed, the form from which Service Manager was started is displayed.

2.4.2.93. RAD function: simple.file.load

A RAD function that loads an unload file containing only data records of one file into a target file. If the target file already exists it will either drop the target file first, or return an error based on a flag passed as a parameter.

This function only works with binary unload files. Text unload files aren't supported.



Caution

Caution Due to the limited capabilities of this routine and the potential loss of data, this function should not be used except by system RAD routines such as the upgrade.

Function

simple.file.load()

Format

```
$L.rc=simple.file.load( <unload file name>, <target file name>, <drop if already exists flag> )
```

Function returns

`true` - if successful.

`false` - if there was a failure.

Parameters

The following parameters are valid for the **simple.file.load** function:

Parameter	Data type	Description
unload file name	character	Specifies where the unload file is located
target file name	character	Specifies the target file for this function, for example, signatures. As a safety precaution, this file name has to match the dbdict record internally stored in the unload file, or the function returns an error.
drop if already exists	logical	Controls the behavior of the function if the target file already exists. If set to FALSE, the function will fail and return an error if the target file already exists in the system. If set to TRUE, the function will drop the existing target file first, before recreating it based on the dbdict record stored in the unload file.

Example

```
$L.result=simple.file.load( "C:\\Temp\\my_signatures.unl", "signatures", false )
```

This example attempts to load the unload file C:\Temp\my_signatures.unl into the signatures file. It will fail if the signatures file already exists (third parameter is false).



Note

Note The escape characters have to be escaped.

2.4.2.94. RAD function: sleep

A RAD function that temporarily ceases the execution of an executing thread for the specified number of seconds.

Function

sleep

Format

```
sleep($seconds)
```

Parameters

The following parameter is valid for the **sleep** function:

Parameter	Data type	Description
$seconds	Integer	Specifies the positive integer of seconds to sleep.

Example

```
sleep(10)
```

The thread will sleep 10 seconds.

2.4.2.95. RAD function: str

A RAD function that converts a data type that's not a string into a string.

Function

str

Format

```
str(any valid expression)
```

Parameters

Any valid expression passed to the str function can be converted to a string.

Examples

`str(1+1)` returns `2`.

`"report run at "+ str(tod())` returns `"report run at 12/10/90 10:10:00"`.

2.4.2.96. RAD function: stradj

A RAD function that ensures a string is a specific length by removing or adding trailing blanks.

Function

stradj

Format

```
$L.void=stradj($string, $size)
```

Parameters

The following parameters are valid for the **stradj** function:

Parameter	Data type	Description
$L.void=	Not applicable	A syntax requirement only, which doesn't affect the outcome of the function.
$string	String	The string being modified.
$size	Number	The desired size of $string.

Examples

```
$string="Hewlett-Packard"
$size=15
$L.void=stradj($string, $size)
```

After execution, the value of `$string` is `Hewlett-Packard`.

```
$string="Hewlett-Packard"
$size=10
$L.void=stradj($string, $size)
```

After execution, the value of `$string` is `Hewlett-Pa`.

2.4.2.97. RAD function: strchrcp

A RAD function that replaces part of a string with copies of another string.

Function

strchrcp

Format

```
$L.void=strchrcp($target, $index, $source, $copies)
```

Parameters

The following values are valid for the **strchrcp** function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$target	String	The string being modified.
$index	Number	The position in `$target` to begin copying `$source`.
$source	String	The string to be copied into `$target`.
$copies	Number	The number of copies of `$source` to replicate within `$target`.

Factors

This function never extends the length of `$target`.

Example

```
$target="Hewlett-Packard"
$index=4
$source="falcon"
$copies=2
$L.void=strchrcp($target, $index, $source, $copies)
```

After execution, the value of `$target` is `Hewfalconfalcon`.

2.4.2.98. RAD function: strchrin

A RAD function that inserts copies of a string into another string.

Function

strchrin

Format

```
$L.void=strchrin($target, $index, $source, $copies)
```

Parameters

The following parameters are valid for the **strchrin** function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$target	String	The string being modified.
$index	Number	The position in `$target` to begin inserting copies of `$source`.
$source	String	The string to be inserted into `$target`.
$copies	Number	The number of copies of `$source` to insert into `$target`.

Example

```
$target="Hewlett-Packard"
$index=4
$source="falcon"
$copies=2
$L.void=strchrin($target, $index, $source, $copies)
```

After execution, the value of `$target` is `Hewfalconfalconlett-Packard`.

2.4.2.99. RAD function: strclpl

A RAD function that removes a number of characters from the beginning of a string.

Function

strclpl

Format

```
$L.void=strclpl($string, $number)
```

Parameters

The following parameters are valid for the strclpl function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$string	String	The string being modified.
$number	Number	The number of characters to remove.

Example

```
$string="Hewlett-Packard"
$number=6
$L.void=strclpl($string, $number)
```

After execution, the value of `$string` is `t-Packard`.

2.4.2.100. RAD function: strclpr

A RAD function that removes a specified number of characters from the end of a string.

Function

strclpr

Format

```
$L.void=strclpr($string, $number)
```

Parameters

The following parameters are valid for the strclpr function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only. It doesn't affect the outcome of the function.
$string	String	The string being modified.
$number	Number	The number of characters to remove.

Example

```
$string="Hewlett-Packard"
$number=6
$L.void=strclpr($string, $number)
```

After execution, the value of `$string` is `Hewlett-P`.

2.4.2.101. RAD function: strcpy

A RAD function that replaces a part of a string with a substring.

Function

strcpy

Format

```
$L.void=strcpy($target, $tindex, $source, $sindex, $number)
```

Parameters

The following parameters are valid for the **strcpy** function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$target	String	The string being modified.
$tindex	Number	The position in `$target` to begin copying a substring from `$source`.
$source	String	The string supplying a substring.
$sindex	String	The beginning position of the substring within `$source`.
$number	Number	The number of characters to copy from `$source`.

Example

```
$target="Hewlett-Packard"
$tindex=4
$source="falcon"
$sindex=2
$number=3
$L.void=strcpy($target, $tindex, $source, $sindex, $number)
```

After execution, value of `$target` is `Hewalct-Packard`.

2.4.2.102. RAD function: strdel

A RAD function that deletes a number of characters from a specific location in a string.

Function

strdel

Format

```
$L.void=strdel($string, $index, $number)
```

Parameters

The following parameters are valid for the strdel function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$string	String	The string being modified.
$index	Number	The location in `$string` from which characters are to be deleted.
$number	Number	The number of characters to delete from `$string`.

Example

```
$string="Hewlett-Packard"
$index=6
$number=8
$L.void=strdel($string, $index, $number)
```

After execution, the value of `$string` is `Hewletd`.

2.4.2.103. RAD function: strins

A RAD function that inserts a substring into another string.

Function

strins

Format

```
$L.void=strins($target, $tindex, $source, $sindex, $number)
```

Parameters

The following parameters are valid for the **strins** function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$target	String	The string being modified.
$tindex	Number	The position in `$target` to begin inserting a substring from `$source`.
$source	String	The string supplying a substring.
$sindex	String	The beginning position of the substring within `$source`.
$number	Number	The number of characters to insert from `$source`.

Example

```
$target="Hewlett-Packard"
$tindex=4
$source="falcon"
$sindex=2
$number=3
$L.void=strins($target, $tindex, $source, $sindex, $number)
```

After execution, value of `$target` is `Hewalclett-Packard`.

2.4.2.104. RAD function: strpadl

A RAD function that pads a string with leading blanks until the string is a specified size.

Function

strpadl

Format

```
$L.void=strpadl($string, $size)
```

Parameters

The following parameters are valid for the **strpadl** function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$string	String	The string being modified.
$size	Number	The desired size of `$string`.

Example

```
$string="Hewlett-Packard"
$size=20
$L.void=strpadl($string, $size)
```

After execution, value of `$string` is `(space)(space)(space)(space)(space)Hewlett-Packard`.

2.4.2.105. RAD function: strpadr

A RAD function that pads a string with trailing blanks until the string is a specified size. If the string is longer than the size desired, it is truncated.

Function

strpadr

Format

```
$L.void=strpadr($string, $size)
```

Parameters

The following parameters are valid for the strpadr function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only. It doesn't affect the outcome of the function.
$string	String	The string being modified.
$size	Number	The desired size of string.

Examples

```
$string="Hewlett-Packard"
$size=20
$L.void=strpadr($string, $size)
```

After execution, the value of `$string` is `Hewlett-Packard(space)(space)(space)(space)(space)`.

```
 $string="Hewlett-Packard"
$size=10
$L.void=strpadr($string, $size)
```

After execution, the value of `$string` is `Hewlett-Pa`.

2.4.2.106. RAD function: strraw

A RAD function that exports array-type data to a delimiter-separated string.

Function

strraw

Format

```
$result = strraw($array [,$delim])
```

Parameters

The following parameters are valid for the **strraw** function.

Parameter	Data type	Description
$result	Character	Returns a Character type value.
$array	Array	The array from which the delimited string is created.
$delim	Character	The delimiter that separates the elements of the array in the result string. The default value is a space.

Note

Note Null elements are handled as an empty string.

Example

```
$result=strraw({“a”,2,,true},”;”)
```

After execution, $result contains **a;2;;true**.

2.4.2.107. RAD function: strrep

A RAD function that returns a string after replacing a specified character sequence with another string.

Function

strrep

Format

```
strrep($target,$string1,$string2)
```

Parameters

The following parameters are valid for the **strrep** function:

Parameter	Data type	Description
$target	String	The string containing the characters specified to be replaced.
$string1	String	The string within `$target` to be replaced.
$string2	String	The string replacing *$string1*.

Example

```
$s=strrep("Personnel","sonnel","formance")
```

After execution, the value of `$s` is `Performance`.

2.4.2.108. RAD function: strtml

A RAD function that removes all leading blanks from a string.

Function

strtrml

Format

```
$L.void=strtrml($string)
```

Parameters

The following parameters are valid for the **strtrml** function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only. It doesn't affect the outcome of the function.
$string	String	The string being modified.

Note

Note This RAD function is unavailable as a JavaScript system.functions call.

Example

```
$string="Hewlett-Packard"
$L.void=strtrml($string)
```

After execution, the value of `$string` is `Hewlett-Packard`.

2.4.2.109. RAD function: strtrmr

A RAD function that removes all trailing blanks from a string.

Function

strtrmr

Format

```
$L.void=strtrmr($string)
```

Parameters

The following parameters are valid for the **strtrmr** function:

Parameter	Data type	Description
$L.void=	Not applicable.	A syntax requirement only, doesn't affect the outcome of the function.
$string	String	The string being modified.

Note

Note This RAD function is unavailable as a JavaScript system.functions call.

Example

```
$string="Hewlett-Packard  "
$L.void=strtrmr($string)
```

After execution, the value of `$string` is `Hewlett-Packard`.

2.4.2.110. RAD function: substr

A RAD function that extracts a substring from a string.

Function

substr

Format

```
substr(string, beginning position, length)
```

Parameters

The following parameters are valid for the **substr** function:

Parameter	Data type	Description
$string	String	The string being modified.
beginning position	Number	The position in the string where the substring is to begin.
length	Number	The total number of characters you want the substring to contain. The default behavior is to include all remaining characters in the string from the starting position. You can use the length parameter to specify a shorter substr length.

Example

```
$a=substr(" Service Manager", 9)
$b=substr(" Service Manager", 1, 7)
```

After execution, the value of `$a` is `Manager` and the value of `$b` is `Service`.

2.4.2.111. RAD function: substrb

A RAD function that extracts a substring from a multibyte character or binary string.

Function

substrb

Format

```
substrb(string, beginning position, length)
```

Parameters

The following parameters are valid for the **substr** function:

Parameter	Data type	Description
string	String	The string being modified.
beginning position	Number	The position in the string where the substring is to begin.
length	Number	The total number of bytes you want the substring to contain. The default behavior is to include all remaining bytes in the string from the starting position. You can use the length parameter to specify a shorter byte length. **Caution** Caution You should choose a length value that takes into account the number of bytes available in the source string. If you choose too low a value the substituted string may contain corrupted data because the function did not have enough bytes to substitute a multibyte character. Refer to a UTF-8 or UTF-16 character encoding map to see how many bytes a particular multibyte character requires. It is recommended that you only use this function on strings of a fixed byte length to avoid possible data corruption. If you want to replace a fixed number of characters use the **substr** function instead.

Example

```
$a=substrb("abcÄÖÜdef", 5)
$b=substrb("abcÄÖÜdef", 1, 7)
```

After execution, the value of `$a` is `ÖÜdef` and the value of `$b` is `abcÄÖ`.

For the string $b, setting the length value to 4 or 6 would lead to data corruption because there aren't enough bytes available to contain the multi-byte characters Ä and Ö respectively.

2.4.2.112. RAD function: sysinfo.get("ActiveFloatUsers")

A RAD function that returns the total number of floating users currently active.

Function

sysinfo.get("ActiveFloatUsers")

Format

```
$info=sysinfo.get("ActiveFloatUsers")
```

2.4.2.113. RAD function: sysinfo.get("Active Licenses")

A RAD function that returns the total number of licenses currently active.

Function

sysinfo.get("ActiveLicenses")

Format

```
$info=sysinfo.get("ActiveLicenses")
```

2.4.2.114. RAD function: sysinfo.get("ActiveNamedUsers")

A RAD function that returns the total number of named users currently active.

Function

sysinfo.get("ActiveNamedUsers")

Format

```
$info=sysinfo.get("ActiveNamedUsers")
```

2.4.2.115. RAD function: sysinfo.get("AuthMode")

A RAD function that returns the authentication mode of the current user:

- Returns `db` if using password authentication and the password exits in database.
- Returns `ldap` if using password authentication and the password exits in LDAP.
- Returns `tso` if using TSO.
- Returns `lwsso` if using LWSSO.

Function

sysinfo.get("AuthMode")

Format

```
$info=sysinfo.get("AuthMode")
```

Example

```
$info=sysinfo.get("AuthMode")
```

After execution, the value of `$info` is set to the authentication mode of the current user.

2.4.2.116. RAD function: sysinfo.get("ClientNetAddress")

A RAD function that returns the Service Manager client network IP address as a character string.

Function

sysinfo.get("ClientNetAddress")

Format

```
$info=sysinfo.get("ClientNetAddress")
```

Example

```
$x.clientNetAddress=sysinfo.get("ClientNetAddress")
```

2.4.2.117. RAD function: sysinfo.get("ClientOSName")

A RAD function that returns the name of the operating system (OS) that Service Manager is running on, such as Windows XP or Mac OS.

Function

sysinfo.get("ClientOSName")

Format

```
$info=sysinfo.get("ClientOSName")
```

2.4.2.118. RAD function: sysinfo.get("ClientPID")

A RAD function that returns the Process ID (PID) for RAD. The client PID is the same as the server PID.

Function

sysinfo.get("ClientPID")

Format

```
$info=sysinfo.get("ClientPID")
```

Example

```
$x.clientPid=sysinfo.get("ClientPID")
```

2.4.2.119. RAD function: sysinfo.get("ClientVersion")

A RAD function that returns the version number of the client software as a string in the release.version.level form. For example, `7.0.1`.

Function

sysinfo.get("ClientVersion")

Format

```
$info=sysinfo.get("ClientVersion")
```

2.4.2.120. RAD function: sysinfo.get("Display")

A RAD function that returns the following:

- Returns `gui` if the user is running a GUI client.
- Returns `text` if the process is not running through a GUI client, such as a background process.

Function

sysinfo.get("Display")

Format

```
$info=sysinfo.get("Display")
```

Example

```
$display=sysinfo.get("display")
```

After execution, the `$display` variable is `gui` if the user is running a Windows session.

```
If sysinfo.get("Display")="gui" then (...)

If index(sysinfo.get("Display"), "gui")>0 then (...)

If sysinfo.get("Display")~="text" then (...)
```

2.4.2.121. RAD function: sysinfo.get("Environment")

A RAD function that returns the following:

- Returns `scserver` if your session is a client application connection, for example, ConnectIT.
- Returns `sm` if your session is a background process such as a scheduler.
- Returns `scguiwswt` if your session is a Windows client connection.
- Returns `scguiwweb` if your session is a Web client connection.

Function

sysinfo.get("Environment")

Format

```
$info=sysinfo.get("Environment")
```

2.4.2.122. RAD function: sysinfo.get("languagecode")

A RAD function that returns the server session language code value used for localized forms and messages.

Function

sysinfo.get("languagecode")

Format

```
$info=sysinfo.get("languagecode")
```

> **Note**
>
> Note If no language has been set, `en` is returned.

2.4.2.123. RAD function: sysinfo.get("MaxFloatUsers")

A RAD function that returns the total number of floating users logged on since startup.

Function

sysinfo.get("MaxFloatUsers")

Format

```
$info=sysinfo.get("MaxFloatUsers")
```

2.4.2.124. RAD function: sysinfo.get("MaxLicenses")

A RAD function that returns the maximum number of licenses used since startup.

Function

sysinfo.get("MaxLicenses")

Format

```
$info=sysinfo.get("MaxLicenses")
```

2.4.2.125. RAD function: sysinfo.get("Mode")

A RAD function that returns the following:

- Returns `server` if RAD is running in server mode.
- Returns `sm` if RAD is running on a sm process.
- Returns `SOAP` if RAD is running on a Windows or Web client.
- Returns `express` if RAD is running in express mode.

Function

sysinfo.get("Mode")

Format

```
$info=sysinfo.get("Mode")
```

Examples

```
$mode=sysinfo.get("mode")
```

Returns *SOAP* if the user is running on a Windows or Web client.

```
If sysinfo.get("mode")="client" then (...)
```

```
If (index(sysinfo.get("mode"),{"sm","server",
"express"})>0 then (...)
```

2.4.2.126. RAD function: sysinfo.get("PrevLabel")

A RAD function that returns the label name of the last RAD panel executed, thus helping determine where errors occur. A RAD program may also require the label of the last RAD panel that was executed.

Function

sysinfo.get("PrevLabel")

Format

```
$info=sysinfo.get("PrevLabel")
```

Example

```
$L.lastlabel = sysinfo.get("prevlabel")
```

The panel name is the last panel executed in the RAD program called. At the start panel for any application, the name is the *calling* panel name from the parent application.

2.4.2.127. RAD Function: sysinfo.get("PKMode")

A RAD function that returns the primary key mode for the system. Returned values are of the following types:

- `true` : indicates that the system is running in primary key mode.

  Note
  Note In Service Manager 9.4x, the primary key feature is always enabled and therefore a value of `true` is always returned.

- `false` : indicates that the system is not running in primary key mode (`primary_key_mode=0` in sm.ini).

  Note
  Note Starting with Service Manager 9.34p4 (for 9.3x) and 9.40 (for 9.4x), the primary_key_mode parameter is obsolete and removed.

Function

sysinfo.get("PKMode")

Format

```
$info=sysinfo.get("PKMode")
```

Example

```
<variable> = sysinfo.get("pkmode")
```

2.4.2.128. RAD function: sysinfo.get("PrintOption")

A RAD function that returns printing option information. The return is either `client` or `server`, depending on how the system is set.

Function

sysinfo.get("PrintOption")

Format

```
$info=sysinfo.get("PrintOption")
```

2.4.2.129. RAD function: sysinfo.get("Quiesce")

A RAD function that returns `true` if the system is quiesced and `false` if the system is not quiesced. When the system is quiesced, it is readonly.

Function

sysinfo.get("Quiesce")

Format

```
$info=sysinfo.get("Quiesce")
```

2.4.2.130. RAD function: sysinfo.get("RecList")

A RAD function that returns `true` if record list is enabled and `false` if record list is disabled.

Function

sysinfo.get("RecList")

Format

```
$info=sysinfo.get("RecList")
```

2.4.2.131. RAD function: sysinfo.get("ServerNetAddress")

A RAD function that returns the Service Manager server network IP address in the form of a character string.

Function

sysinfo.get("ServerNetAddress")

Format

```
$info=sysinfo.get("ServerNetAddress")
```

2.4.2.132. RAD function: sysinfo.get("ServerNetPort")

A RAD function that returns the Service Manager server port used by the connected client or the system name when called by a background process.

Function

sysinfo.get("ServerNetPort")

Format

```
$info=sysinfo.get("ServerNetPort")
```

Factors

Returns either the port number or the service name, as specified .

Examples

The system started with:

```
sm -httpPort:12932
```

A connected client issuing this RAD function returns `12932`.

The system started with:

```
sm -httpPort:scport1
```

A connected client issuing this RAD function returns `scport1`.

2.4.2.133. RAD function: sysinfo.get("ServerPID")

A RAD function that returns the Process ID (PID) of the server, in the form of a number.

Function

sysinfo.get("ServerPID")

Format

```
$info=sysinfo.get("ServerPID")
```

2.4.2.134. RAD function: sysinfo.get("ThreadID")

A RAD function that returns the number of the current RAD thread.

Function

sysinfo.get("ThreadID")

Format

```
$info=sysinfo.get("ThreadID")
```

Example

```
$x=sysinfo.get("ThreadID")
```

After execution, the value of *$x* is set to the number ID of the current RAD thread.

2.4.2.135. RAD function: sysinfo.get("TotalFloatUsers")

A RAD function that returns the total number of floating users allowed.

Function

sysinfo.get("TotalFloatUsers")

Format

```
$info=sysinfo.get("TotalFloatUsers")
```

2.4.2.136. RAD function: sysinfo.get("TotalLicenses")

A RAD function that returns the total number of licenses allowed.

Function

sysinfo.get("TotalLicenses")

Format

```
$info=sysinfo.get("TotalLicenses")
```

2.4.2.137. RAD function: sysinfo.get("TotalNamedUsers")

A RAD function that returns the total number of named users allowed.

Function

sysinfo.get("TotalNamedUsers")

Format

```
$info=sysinfo.get("TotalNamedUsers")
```

2.4.2.138. RAD function: sysinfo.get("TotalProcs")

A RAD function that returns the total number of executing processes.

Function

sysinfo.get("TotalProcs")

Format

```
$info=sysinfo.get("TotalProcs")
```

2.4.2.139. RAD function: sysinfo.get("TotalSystemProcs")

A RAD function that returns the total number of system processes.

Function

sysinfo.get("TotalSystemProcs")

Format

```
$info=sysinfo.get("TotalSystemProcs")
```

2.4.2.140. RAD function: sysinfo.get("TotalUserProcs")

A RAD function that returns the total user processes.

Function

sysinfo.get("TotalUserProcs")

Format

```
$info=sysinfo.get("TotalUserProcs")
```

2.4.2.141. RAD function: sysinfo.get("ess")

A RAD function that checks if the existing session is running in the context of employee self-service (ESS) session. If the user is accessing service manager through the ESS portal, it suppose to return true. Otherwise, it returns false.

Function

sysinfo.get("ess")

Format

```
$info=sysinfo.get("ess")
```

Example

```
if (sysinfo.get("ess")=true) then ($G.ess=true)

$G.ess is set in "login" RAD application under following panel check.entry.point & init.name
```

2.4.2.142. RAD function: time

A RAD function that returns the time portion of a date and time value.

Function

time

Format

```
time($date.time.variable)
```

Parameters

The following parameter is valid for the time function:

Parameter	Data type	Description
$string	String	The string variable containing upper-case characters.

Factors

The time of a given date and time may be different in different time zones.

Example

```
time($date.time)
```

After execution, the value of `$date.time` is `12:10:15`.

2.4.2.143. RAD function: tod

A RAD function that returns the current system date and time in the logged on user's timezone and date format.

Function

tod

Format

```
tod()
```

Factors

The tod function returns `mm` / `dd` / `yy` `hh` : `mm` : `ss` .

Examples

```
tod()=`4/20/96 12:15:16'

$curr.time=tod()
```

2.4.2.144. RAD function: tolower

A RAD function that converts all uppercase characters in a string variable to lowercase characters. Characters with no lowercase equivalent remain the same.

Function

tolower

Format

```
tolower($string)
```

Parameters

The following parameter is valid for the **tolower** function:

Parameter	Data type	Description
$string	String	The string variable containing uppercase characters.

Factors

Character conversions are based on the value of the language parameter.

Example

```
$s=tolower("Copyright 2011")
```

After execution, the value of `$s` is `copyright 2011` .

2.4.2.145. RAD function: toupper

A RAD function that converts all lower–case characters in a string variable to upper–case characters. Characters with no upper–case equivalent remain the same.

Function

toupper

Format

```
toupper($string)
```

Parameters

The following parameter is valid for the toupper function:

Parameter	Data type	Description
$string	String	The string variable containing lower-case characters.

Factors

Character conversions are based on the value of the language parameter.

Example

```
$s=toupper("Copyright 2004")
```

After execution, the value of `$s` is `COPYRIGHT 2004`.

2.4.2.146. RAD function: translate

A RAD function that translates from one character set to another.

Programming considerations: This function has been deprecated and is maintained for backward compatibility only. This function can only be used on simple single byte character strings and is not UTF-8 safe. Case conversion on strings should be done with the **toupper()** and **tolower()** functions, which are UTF-8 safe.

Function

translate

Format

```
translate(source, old character set, new character set)
```

Parameters

The following parameters are valid for the translate function:

Parameter	Data type	Description
source	String	The string to translate.
old character set	String	The existing character set.
new character set	String	The character set to translate to.

Factors

- Old and new character sets must be strings of the same length.
- Old and new character sets may be literals, variables, or fields.
- A one–for–one correlation must exist between the old and new character sets. For example, the third character in the new set replaces the third character in the old set.

Example

```
translate("123abcdef","abc","ABC")
```

After execution, the value is `123ABCdef`.

2.4.2.147. RAD function: trunc

A RAD function that truncates the decimal digits of a number to a specified number of digits.

Function

trunc

Format

```
trunc(number,n)
```

Parameters

Parameter	Data type	Description
number	Number	The number to truncate.
n	Number	The number of decimal digits desired. The default value of *n* is zero (0).

Factors

- The second parameter is not required. By default, the number passed is truncated to a whole number.
- Negative and positive numbers are treated the same.
- Character conversions are based on the value of the language parameter.

Examples

```
trunc(3.45,1)
```

After execution, `3.4` is returned.

```
trunc(3.44)
```

After execution, `3` is returned.

```
trunc('13:19:34')
```

After execution, `13:19:00` is returned.

2.4.2.148. RAD function: type

A RAD function returns the numeric type of a datum. See Data types for a numeric representation of each data type.

Function

type

Format

```
$ttype=type($any.value)
```

Parameters

The following parameter is valid for the type function:

Parameter	Data type	Description
$any.value	String	Any literal or variable.

Example

```
$ttype=type('2/15/96')
```

After execution, the value of `$ttype` is three (`3`).

2.4.2.149. RAD function: updatestatus

A RAD function that returns the result of the last update operation on a Service Manager file.

Function

updatestatus

Format

```
updatestatus(file_variable)
```

Parameters

`file_variable` is the variable you want to retrieve.

Factors

- If the last update operation succeeded, this function returns 0 (zero), otherwise it returns one of the following values:
  - 48: The update failed because it contained an invalid duplicate key.
  - 49: The update failed because it contained an invalid null key.
  - 50: The update failed because it contained all null keys.
  - 51: The update failed because the record had already been modified.
  - 52: The update failed because the record had already been deleted.
  - 53: The update failed because of a trigger error.
  - -1: Other errors.
- Every time the file variable is moved or changed (for example, a new record is inserted), the return value of this function is reset to 0.

Example

```
$L.updateresult=updatestatus($L.file)
```

2.4.2.150. RAD function: val

A RAD function that converts a datum to a different data type.

Function

val

Format

```
$new.data=val($data, $new.type)
```

Parameters

The following parameters are valid for the val function:

Parameter	Data type	Description
$data	Any data type	Datum to be converted.
$new.type	Number	Data type to convert the data to. The default value is one (`1`). For a list of the numbers that correspond to the data types, see Data types.

Factors

- A number can be converted to date and time, a string, or logical.
- A string can be converted to a number, date and time, or logical.
- A date and time can be converted to a number or a string.
- Logical can be converted to a number or a string.

Examples

The following examples show the value after execution:

Variable	Value of `$x` after execution
$x=val(100)	100
$x=val(100, 2)	"100"
$x=val(100, 3)	'00:01:40'
$x=val(0, 4)	false
$x=val(1, 4)	true
$x=val(2, 4)	unknown
$x=val(3, 4)	NULL
$x=val("MICROFOCUS", 1)	NULL
$x=val("100.3", 1)	100.3
$x=val("MICROFOCUS", 2)	"MICROFOCUS"
$x=val("MICROFOCS", 3)	NULL
$x=val("2/15/96", 3)	'02/15/96 00:00:00'
$x=val("true", 4)	true
$x=val("0", 4)	NULL
$x=val('01:00:23', 1)	3623
$x=val('02/16/96 08:00:00')	62992915200
$x=val('01/01/96', 2)	"01/01/96 00:00:00"
$x=val('02/09/96 08:00:00', 3)	'02/09/96 08:00:00'
$x=val('01/01/96', 4)	NULL
$x=val('12:34:56', 4)	NULL
$x=val(false)	0
$x=val(true, 1)	1
$x=val(unknown, 1)	2
$x=val(false, 2)	"false"
b	NULL
$x=val(unknown, 4)	unknown

2.4.2.151. RAD function: version

A RAD function that returns an array containing the release number of the Service Manager executable.

Function

version

Format

```
version()
```

Factors

Character conversions are based on the value of the language parameter.

Example

```
$a=version()
```

After execution, the value of `$a` is `{"unix",9.0,"9.0.1"}`.

- The first element is a string naming the platform: unix, or winnt.
- The second element is `9.0`, the RAD release level.
- The third element is a string containing the Service Manager version number and the Service Pack number, such as `9.0.1` indicates `Service Manager version 9.0, Service Pack 1`.

2.4.2.152. RAD function: year

A RAD function that returns the full year for a date, regardless of the date format.

Function

year

Format

```
$fyear=year($date)
```

Parameters

The following parameter is valid for the year function:

Parameter	Data type	Description
$date	Date/time	The date and time value.

Example

```
$fyear=year('2/15/05')
```

After execution, the value of $ `fyear` is `2005`.

2.4.3. List: rtecalls()

The rtecalls are based on one RAD function, the **rtecall( )** function. They are in-line statements that are useful in the Service Manager RAD Debugger. The **rtecall** function is available anywhere in Service Manager that supports expressions, statements, or calculations. For example, rtecalls can be used in Format Control calculations, scripts, and displayoptions. The **rtecall()** function is extensible. New capabilities may appear in every release of Service Manager.

Function	Description
rtecall("alalnum")	Checks to make sure a string contains only alphanumeric characters, or only alphanumeric characters and the provided non-alphanumeric characters.
rtecall("counter")	Turns counters on or off for the current session. Other Service Manager users are unaffected.
rtecall("datemake")	Returns a date, in the proper form, based upon a series of numbers passed to it.
rtecall("escstr")	Precedes special characters in a string with an escape character.
rtecall("filldate")	Places the current date and time in a field in the current record.
rtecall("filecopy")	Copies all of the data in a collection to another file variable.
rtecall("fileinit")	Initializes a new file (rinit) in $targetfile.
rtecall("getnumber")	Replaces the getnumb RAD application.
rtecall("getrecord")	Retrieves the record identified by Unique key values in $L.array.
rtecall("getunique")	Returns into $L.array the values for the Unique key from the current record in $L.file.
rtecall("isalnum")	Checks to make sure a string contains only numeric characters, or only numeric characters and the provided non-numeric characters.
rtecall("isalpha")	Checks to make sure a string contains only alphabetic characters, or only alphabetic characters and the provided non-alphabetic characters.
rtecall("islower")	Checks to make sure a string contains only lowercase characters, or only lowercase characters and the provided non-lowercase characters.
rtecall("isnumeric")	Checks to make sure a string contains only numeric characters, or only numeric characters and the provided non-numeric characters.
rtecall("isupper")	Checks to make sure a string contains only uppercase characters, or only uppercase characters and the provided non-uppercase characters.
rtecall("log")	Sends a message to the external sm.log file.
rtecall("passchange")	Changes this user's password.
rtecall("policycheck")	Imposes data policy as defined in the datadict table.
rtecall("qbeform")	Returns a record list form, which can be passed into an rio or fdisp panel.
rtecall("recdupl")	Copies the contents of the current record into the contents of another record.
rtecall("recordexists")	Specifies whether any records exist that match a particular query regardless of any Mandanten data restrictions.
rtecall("refresh")	Retrieves the most current value of a record from the database and returns it in the file variable.

rtecall("resetnotebook")	Adds an XML attribute "resetnotebook=true" to the "execute" response of the current RAD thread.
rtecall("rfirst")	Places the pointer at the first record in a record collection (a record list).
rtecall("rgoto")	Places the pointer at the indicated record.id in a record collection (a record list).
rtecall("rid")	Returns the record number of the current record (represented by $L.file).
rtecall("scantable")	Scans the specified table for certain types of invalid records and fixes them.
rtecall ("select")	Retrieves a list of records for a file in ascending or descending order according to the specified sort fields and group fields.
rtecall("setdebug")	Sets the debug parameters.
rtecall("sort")	Sorts a list or a list of lists in ascending or descending order.
rtecall("statusupdate")	Publishes a message to a specific field on a form.
rtecall("transtart")	Measures the amount of data transferred, elapsed time and CPU usage of any transaction.
rtecall("transtop")	Measures the amount of data transferred, elapsed time and CPU usage of any transaction. Transtop is commonly invoked from the RAD debugger and is used in conjunction with transtart.
rtecall("trigger")	Turns triggers on or off for the current session.

2.4.3.1. rtecall("alalnum") function

A RAD function that verifies that a string contains only alphanumeric characters. The first character must be alphabetic. The remaining characters must be alphabetic, numeric, or one of the non-alphabetic characters provided.

Function

rtecall("alalnum")

Format

```
$L.success.flag= rtecall ($L.fnc.name, $L.return.code, $L.str, $L.chars)
```

Parameters

The following parameters are valid for the rtecall("alalnum") function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful
$L.fnc.name	String	Name of the sub-function to call, in this case "alalnum"
$L.return.code	Number	Provides a more detailed return code
$L.str	String	The string to be checked
$L.chars	String	An optional comma delimited string of non-alphanumeric characters to allow

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flag=true
$L.return.code=0
$L.str=name in $L.file
$L.chars="., _"
$L.success.flag=rtecall("alalnum", $L.return.code, $L.str, $L.chars)
```

Results:

- `$L.success.flag` is true when name in `$L.file="my.name"`.
- `$L.success.flag` is true when name in `$L.file="my_name"`.
- `$L.success.flag` is false when name in `$L.file="my name"`.

2.4.3.2. rtecall("counter") function

A RAD function that turns counters on or off for the current session. Other users aren't affected.

Function

rtecall("counter")

Format

```
$L.success.flag= rtecall($L.fnc.name, $L.return.code, $L.switch)
```

Parameters

The following parameters are valid for the rtecall("counter") function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	The name of the sub-function to call, in this case "counter".
$L.return.code	Number	Provides a more detailed return code.
$L.switch	Number	The value is either one (`1`), indicating counters on, or zero (`0`), indicating counters off.

Factors

If the `$L.success.flg` is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
 $L.success.flg=rtecall("counter", $L.return.code, 1) to turn on counters
$L.success.flg=rtecall("counter", $L.return.code, 0) to turn off counters
```

2.4.3.3. rtecall("datemake") function

A RAD function that returns a date, in the proper form, based upon a series of numbers passed to it.

Function

rtecall("datemake")

Format

`$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.date, $L.yr, $L.mo, $L.da, $L.hr, $L.mn, $L.se)`

Parameters

The following parameters are valid for the **rtecall("datemake")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	The name of the sub-function to call. In this case, "datemake".
$L.return.code	Number	Provides a more detailed return code.
$L.date	Date/Time	The variable in which the date will be returned.
$L.yr	Number	The year. A 2-digit year uses the prefix `20` for years up to 50, and `19` for years after. For example, `48` returns `2048`, `99` returns `1999`.
$L.mo	Number	The month.
$L.da	Number	The day. Valid values are from one (1) through 31.
$L.hr	Number	The hour.
$L.mn	Number	The minutes.
$L.se	Number	The seconds.

Factors

The $L.success.flag and $L.return.code always return `true`, even when the function is unsuccessful.

Examples

```
$L.cal.date='01/01/05 00:00:00'
$L.success.flg=rtecall("datemake",$L.return.code, $L.cal.date, 0, 5, 31, 17, 15, 22)
```

Results:

- `$L.success.flag=true`
- `$L.cal.date='05/31/2005 17:15:22'`

```
$L.cal.date='01/01/05 00:00:00'
$L.success.flag= rtecall("datemake",$L.return.code, $L.cal.date, 0, 2, 31, 17, 15,
22)
```

Note that `2/31/05` is an invalid date.

Results:

- `$L.success.flag=true`
- `$L.cal.date='01/01/2005 00:00:00'`

The result is invalid; therefore the date variable is unchanged. Regardless of the success or failure, the `$L.success.flag` returns a value of `true`.

2.4.3.4. rtecall("escstr") function

A RAD function that precedes special characters in a string with an escape character. This function is rarely needed. The only situation in which a string needs the escape characters added is when that string is to be fed back into Service Manager. Service Manager treats any data between quotes as a string. If the data contains a quote, then that quote must be *escaped* with a backslash escape character (\) to ensure that the quote is treated as data instead of the end of the string. Since the backslash is used as the escape character, any occurrence of a backslash in the data must also be *escaped*.

An example of when this function might be needed is when a RAD program is constructing a query to retrieve a record based on data from another record. In this case, the contact name in an incident retrieves the contact information from the contacts table. The RAD program might construct a query as follows:

```
$L.query = "name="+contact.name in $file
```

This query will not function properly if the `contact.name` from `$file` contains a quote or a backslash character because these characters aren't properly *escaped* and will cause the parse of the query to end prematurely. The correct example is as follows:

```
$L.temp = contact.name in $file
$L.ret = rtecall("escstr",$L.rc,$L.temp)
$L.query="name="+$L.temp
```

Function

rtecall("escstr")

Format

```
rtecall($L.fnc.name, $L.rc, $L.str)
```

Parameters

The following parameters are valid for the rtecall("escstr") function:

Parameter	Data type	Description
$L.fnc.name	String	Name of the sub-function to call, in this case "escstr".
$L.rc	Number	Provides a more detailed return code.
$L.str	String	The string to be modified.

Example

Before the call to `"escstr"`, `$L.str` contains **c:\dir\sub**.

```
$L.rc=0
$L.ret=rtecall("escstr", $L.rc, $L.str)
```

After the rtecall, `$L.str` contains the following string: **c:\\dir\\sub**. The backslash escape character (\) is now inserted in front of the existing backslash.

2.4.3.5. rtecall("filecopy") function

A RAD function that copies all of the data in a collection to another file variable. The database dictionary record for both the source and target files must exist. Records are only added.

Function

rtecall("filecopy")

Format

```
$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.dbdict.source.name,
$L.dbdict.target.name, $L.count, $L.bad)
```

Parameters

The following parameters are valid for the **rtecall("filecopy")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "filecopy".
$L.return.code	Number	Provides a more detailed return code.
$L.dbdict.source.name	String	The name of the source database dictionary record.
$L.dbdict.target.name	String	The name of the target database dictionary record.
$L.count	Number	A count of the number of records successfully moved.
$L.bad	Number	A count of the number of errors encountered.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.dbdict.source.name="location"
$L.dbdict.target.name="locationbak"
$L.success.flg=rtecall("filecopy", $L.return.code, $L.dbdict.source.name,
$L.dbdict.target.name, $L.count, $L.bad)
```

Returns the locationbak file as an exact copy of the location file.

2.4.3.6. rtecall("fileinit") function

A RAD function that initializes a file variable in `$targetfile` from `$sourcefile`.

Function

rtecall("fileinit")

Format

```
rtecall($L.fnc.name, $errcode, $targetfile, $sourcefile)
```

Parameters

The following parameters are valid for the **rtecall("fileinit")** function:

Parameter	Data type	Description
$L.fnc.name	String	Name of the sub-function to call, in this case "fileinit".
$L.errcode	Number	The number of the error code encountered.
$L.targetfile	String	The target file in which to initialize the variable.
$L.sourcefile	String	The source file that contains the variable.

Factors

- The file initialized is identified by the `$sourcefile` variable.
- The current record for `$targetfile` is the same as the current record in `$sourcefile`.
- Update and delete operations can't be performed against `$targetfile`.

Example

```
$L.void=rtecall("fileinit", $L.errcode, $file.old, $file)
```

2.4.3.7. rtecall("filldate") function

A RAD function that places the current date and time in a field in the current record.

Function

rtecall("filldate")

Format

```
$L.success.flg= rtecall($L.fnc.name, $L.return.code, $L.file, $L.field.name)
```

Parameters

The following parameters are valid for the rtecall("filldate") function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	The name of the sub-function to call, in this case "filldate".
$L.return.code	Number	Provides a more detailed return code.
$L.file	File	The file handle.
$L.field	String	The name of the field to be updated.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.file={[, , {}, , , {}, , , {}, , , {}, , , {}, {}, {}, {{[, , , ]}}, , , ]} where the first field is
named "date"
$L.field.name="date"
$L.success.flg= rtecall("filldate", $L.return.code, $L.file, $L.field.name)
$L.file returns ={[‘04/14/2000 08:27:19’, , {}, , , {}, , , {}, , , {}, , , {}, {}, {}, {{[, , , ]}}, , ,
]}
```

2.4.3.8. rtecall("getnumber") function

A RAD function that replaces the getnumb RAD application.

Function

rtecall("getnumber")

Format

```
$L.flg=rtecall($L.fnc.name, $L.return.code, $L.number, $L.class, $L.field)
```

Parameters

The following parameters are valid for the **rtecall("getnumber")** function:

Parameter	Data type	Description
$L.flg	Logical	The value is `true` for success and `false` if an error occurs.
$L.fnc.name	String	The name of the sub-function to call, in this case "getnumber".
$L.return.code	Number	The value is zero (`0`) if everything is correct, negative one (`-1`) if an error occurs. This is related directly to $L.flg.
$L.number	String	The number or string returned
$L.class	String	The class of number for which you are searching (from the number file).
$L.field	String	Reserved for future use.

Factors

A sequential overview:

1. The function establishes a lock called "getnumb"+$L.class+$L.field and waits until the lock is established.
2. If the number class is not found, or if there are duplicates, the following error displays: `($L.flg=false)`
3. The function reads the current number.
4. The function increments or decrements by the step value. The step defaults to one (`1`).
5. If incrementing and the number is greater than the reset value, it uses the start number. Start defaults to zero (`0`).
6. A new field called string.flg.
    - If string.flg is `false`, it remains a numeric type.
    - If string.flg is `true`, the number is converted to a string.
    - If string.flg is `unknown`, or the field isn't in the database dictionary (not added during an upgrade), the function checks for a prefix, a suffix, or length. If any one of these exists, the function converts the value to a string. If none of these exist the function leaves the value as a number.
7. To convert the value to a string, the function uses the prefix, corrects the length of the number (padded with zeros on the left), and then adds the suffix.
8. The function eventually saves the new number to the file and returns it to the user, converted to a string if necessary.
9. Finally, it unlocks and returns.

2.4.3.9. rtecall("getprimary") function

A RAD function that returns an array containing the primary key values for a current record. The keys can later be used to retrieve the record using the rtecall("getrecord") function. This function can only be used against tables that have a primary key. Otherwise, error code `2` is returned.

Function

rtecall("getprimary")

Format

```
$L.void=rtecall($L.fnc.name, $L.errcode, $L.array, $L.file)
```

Parameters

The following parameters are valid for the **rtecall("getprimary")** function:

Variable/Value	Description
$L.void = true	If key values are returned in $L.keyvalues. For example, if $L.file represents a current database record.
$L.void= false	If no key values are returned. For example, if $L.file doesn't represent a current database record.
$L.errcode = 0	The function succeeded.
$L.errcode = 1	The $L.file doesn't represent a file variable.
$L.errcode = 2	The file identified by $L.file doesn't have a primary key defined.
$L.file	The file variable initialized by the **rinit** command panel and containing a current record.

Factors

`$L.keyvalues` is an output variable. The output returns an array containing the primary key values for the record in $L.file. For example, if $L.file is for the problem table, an example of the key values returned in `$L.keyvalues` is `{"IM1001",1}`.

Example

```
$L.void = rtecall( "getprimary", $L.errcode, $L.keyvalues, $L.file )
```

2.4.3.10. rtecall("getrecord") function

A RAD function that retrieves the record identified by Unique key values in $L.array.

Function

rtecall("getrecord")

Format

```
$L.void=rtecall($L.fnc.name, $L.errcode, $L.array, $L.file)
```

Parameters

The following parameters are valid for the **rtecall("getrecord")** function:

Parameter	Data type	Description
$L.fnc.name	String	The name of the sub-function to call, in this case "getrecord".
$L.errcode	Number	The number of the error code encountered.
$L.array	String	The name of the record to retrieve.
$L.file	String	The name of the table to retrieve the record from.

Factors

$L.array is an `input` variable, typically the key values returned by $L.array in the **rtecall("getunique")** function.

Example

```
$L.void=rtecall("getrecord", $L.errcode, {"IM1001"}, $L.file)
```

Where:

The value of `$L.file` is initialized for the probsummary table.

2.4.3.11. rtecall("getunique") function

A RAD function that returns an array containing the Unique key values for a current record. The keys can later be used to retrieve the record using the rtecall("getrecord") function.

Function

rtecall("getunique")

Format

```
$L.void=rtecall($L.fnc.name, $L.errcode, $L.array, $L.file)
```

Parameters

The following parameters are valid for the **rtecall("getunique")** function:

Variable/Value	Description
$L.void = true	If key values are returned in $L.keyvalues. For example, if $L.file represents a current database record.
$L.void = false	If no key values are returned. For example, if $L.file doesn't represent a current database record.
$L.errcode = 0	The function succeeded.
$L.errcode = 1	The $L.file doesn't represent a file variable.
$L.errcode = 2	The file identified by $L.file doesn't have a Unique key defined.
$L.file	The file variable initialized by the **rinit** command panel and containing a current record.

Factors

$L.keyvalues is an output variable. The output returns an array containing the unique key values for the record in $L.file. For example, if $L.file is for the problem table, an example of the key values returned in $L.keyvalues is `{"IM1001",1}`.

Example

```
$L.void = rtecall( "getunique", $L.errcode, $L.keyvalues, $L.file )
```

2.4.3.12. rtecall("isalnum") function

A RAD function that verifies that a string contains only alphabetic, numeric, or the non-alphabetic characters provided.

Function

rtecall("isalnum")

Format

```
$L.success.flag= rtecall ($L.fnc.name, $L.return.code, $L.str, $L.chars)
```

Parameters

The following parameters are valid for the **rtecall("isalnum")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "isalnum".
$L.return.code	Number	Provides a more detailed return code.
$L.str	String	The string to be checked.
$L.chars	String	An optional comma-delimited string of non-numeric characters to allow.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
 $L.success.flag=true
$L.return.code=0
$L.str=phone in $L.file
$L.chars=" , (, ), -"
$L.success.flag=rtecall("isalnum", $L.return.code, $L.str, $L.chars)
```

Results:

- When `phone in $L.file="(508) 362.2701"`, $L.success.flag is `false`.
- When `phone in $L.file="(508) 362-2701"`, $L.success.flag is `true`.

2.4.3.13. rtecall("isalpha") function

A RAD function that verifies that a string contains only alphabetic characters or only alphabetic characters and the non-alphabetic characters provided.

Function

rtecall("isalpha")

Format

```
$L.success.flag= rtecall ($L.fnc.name, $L.return.code, $L.str, $L.chars)
```

Parameters

The following parameters are valid for the **rtecall("isalpha")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	The name of the sub-function to call, in this case "isalpha".
$L.return.code	Number	Provides a more detailed return code.
$L.str	String	The string to be checked.
$L.chars	String	An optional comma-delimited string of non-alphabetic characters to allow.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
 $L.success.flag=true
$L.return.code=0
$L.str=name in $L.file
$L.chars=" "
$L.success.flag=rtecall("isalpha", $L.return.code, $L.str, $L.chars)
```

Results:

- When name in $L.file="my name" $L.success.flag is `true`.
- When name in $L.file="my first name" $L.success.flag is `true`.
- When name in $L.file="my 1st name" $L.success.flag is `false`.

2.4.3.14. rtecall("islower") function

A RAD function that verifies that a string contains only lowercase characters, or only lowercase characters and the provided non-lowercase characters.

Function

rtecall("islower")

Format

```
$L.success.flag= rtecall ($L.fnc.name, $L.return.code, $L.str, $L.chars)
```

Parameters

The following parameters are valid for the **rtecall("islower")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "islower".
$L.return.code	Number	Provides a more detailed return code.
$L.str	String	The string to be checked.
$L.chars	String	An optional comma-delimited string of non-lowercase characters to allow.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
 $L.success.flag=true
$L.return.code=0
$L.str=name in $L.file
$L.chars=" "
$L.success.flag=rtecall("islower", $L.return.code, $L.str, $L.chars)
```

Results:

- When `name in $L.file="b1b"`, $L.success.flag is `false`.
- When `name in $L.file="Bob"`, $L.success.flag is `false`.
- When `name in $L.file="bob"`, $L.success.flag is `true`.

2.4.3.15. rtecall("isnumeric") function

A RAD function that verifies that a string contains only numeric characters, or only numeric characters and the provided non-numeric characters.

Function

rtecall("isnumeric")

Format

```
$L.success.flag= rtecall ($L.fnc.name, $L.return.code, $L.str, $L.chars)
```

Parameters

The following parameters are valid for the **rtecall("isnumeric")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "isnumeric".
$L.return.code	Number	Provides a more detailed return code.
$L.str	String	The string to be checked.
$L.chars	String	An optional comma-delimited string of non-numeric characters to allow.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
 $L.success.flag=true
$L.return.code=0
$L.str=name in $L.file
$L.chars="-,+,*,/,%,$"
$L.success.flag=rtecall("isnumeric", $L.return.code, $L.str, $L.chars)
```

Results:

- When `name in $L.file="B1B"`, $L.success.flag is `false`.
- When `name in $L.file="Bob"`, $L.success.flag is `false`.
- When `name in $L.file="36"`, $L.success.flag is `true`.

2.4.3.16. rtecall("isupper") function

A RAD function that verifies that a string contains only uppercase characters, or only uppercase characters and the provided non-uppercase characters.

Function

tecall("isupper")

Format

```
$L.success.flag= rtecall ($L.fnc.name, $L.return.code, $L.str, $L.chars)
```

Parameters

The following parameters are valid for the **rtecall("isupper")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "isupper".
$L.return.code	Number	Provides a more detailed return code.
$L.str	String	The string to be checked.
$L.chars	String	An optional comma-delimited string of non-upper case characters to allow.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
 $L.success.flag=true
$L.return.code=0
$L.str=name in $L.file
$L.chars=" "
$L.success.flag=rtecall("isupper", $L.return.code, $L.str, $L.chars)
```

Results:

- When `name in $L.file="B1B"`, $L.success.flag is `false`.
- When `name in $L.file="Bob"`, $L.success.flag is `false`.
- When `name in $L.file="BOB"`, $L.success.flag is `true`.

2.4.3.17. rtecall("log") function

A RAD function that sends a message to the external sm.log file.

Function

rtecall("log")

Format

```
$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.message)
```

Parameters

The following parameters are valid for the rtecall("log") function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	The name of the sub-function to call, in this case "log".
$L.return.code	Number	Provides a more detailed return code.
$L.message	String	The message to be sent.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg=rtecall("log", $L.return.code, "This is a new message")
```

Generates the last line in the following example in sm.log:

```
   137 04/12/2000 07:03:44 System background scheduler: event started at:
04/11/2000 23:02:48.
   137 04/12/2000 07:03:45 System background scheduler: availability started at:
04/11/2000 23:02:49.
   137 04/12/2000 07:03:45 System background scheduler: contract started at:
04/11/2000 23:02:50.
   137 04/12/2000 07:03:45 System background scheduler: ocm started at:
04/11/2000 23:02:51.
   137 04/12/2000 07:03:45 System startup completed successfully, elapsed time:
00:00:18.
   137 04/12/2000 07:32:48 24 records from location unloaded to: locations.save,
00:00:01 elapsed.
   306 04/12/2000 09:05:09 This is a new message
```

2.4.3.18. rtecall("passchange") function

A RAD function that changes a user password.

Function

rtecall("passchange")

Format

```
$L.success.flag=rtecall($L.fnc.name, $L.return.code, $L.old.pass, $L.new.pass,
$L.confirm.pass)
```

Parameters

The following parameters are valid for the **rtecall("passchange")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call. In this case, the sub-function called is passchange.
$L.return.code	Number	Provides a more detailed return code.
$L.old.pass	String	The old password.
$L.new.pass	String	The new password.
$L.confirm.pass	String	A confirmation of the new password.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg= rtecall("passchange", $L.return.code, "oldpassword",
"newpassword", "newpassword")
```

2.4.3.19. rtecall("policycheck") function

A RAD function that imposes data policy as defined in the System Definition.

Function

rtecall("policycheck")

Format

```
$L.success.flag= rtecall($L.fnc.name, $L.return.code, $L.file)
```

Parameters

The following parameters are valid for the **rtecall("policycheck")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "policycheck".
$L.return.code	Number	Provides a more detailed return code.
$L.file	String	The file handle.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.rc=rtecall("policycheck", $L.errcode, $L.filed)
```

2.4.3.20. rtecall("qbeform") function

A RAD function that returns a record list form, which can be passed into a rio (Record Display/Input) Command panel or fdisp panel. You can insert this function into a format file handle using the contents() function. The following example shows the syntax to use in the contents function: `contents($L.format) = $L.qbe.form`.

Function

rtecall("qbeform")

Format

```
$L.success.flg= rtecall($L.fnc.name,$L.return.code,$L.file,$L.qbe.form)
```

Parameters

The following parameters are valid for the rtecall("qbeform") function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	The name of the subfunction to call; in this case "qbeform".
$L.return.code	Number	Provides a more detailed return code.
$L.file	File	The file handle used to generate the record list. For example, obtain the contacts table by using the "rinit" panel.
$L.qbe.form	Structure	The resulting record list form returned as a structure.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg= rtecall("qbeform",$L.return.code,$L.file,$L.qbe.form)
```

2.4.3.21. rtecall("recdupl") function

A RAD function that copies the contents of the current record into the contents of another record.

Function

rtecall("recdupl")

Format

```
rtecall($L.fnc.name, $return.code, $targetfile, $sourcefile)
```

Parameters

The following parameters are valid for the **rtecall("recdupl")** function:

Parameter	Data type	Description
$L.fnc.name	String	Name of the sub-function to call, in this case "recdupl".
$L.return.code	Number	Provides a more detailed return code.
$targetfile	String	The target record to which fields are copied.
$sourcefile	String	The source file from which fields are copied.

Factors

- Both `$targetfile` and `$sourcefile` must have identical descriptors. Data is copied by position, not field name.
- Both `$targetfile` and `$sourcefile` must be initialized file variables.
- The update and delete operations can't be performed against `$targetfile`.

Example

```
$L.flg=rtecall("recdupl", $L.return.code, $L.temp, $L.file)
```

2.4.3.22. rtecall("recordexists") function

Specifies whether any records exist that match a particular query regardless of any Mandanten data restrictions. This call doesn't return any records from the query.

Function

rtecall("recordexists")

Format

```
$Lexists=rtecall("recordexists", $L.rc, $L.filename, $L.query)
```

Parameters

The following parameters are valid for the **rtecall("recordexists")** function:

Parameter	Data type	Description
"recordexists"	String	Determines the name of the RTE call, in this case "recordexists".
$L.rc	String	Provides a more detailed return code. This parameter is not currently used but is reserved for future use.
$L.filename	String	The name of the Service Manager table to query. For example, contacts or probsummary.
$L.query	String	The query to be run. The query can be true, false, a string, or a parsed expression. This query ignores all Mandanten restrictions.

Factors

The call returns `true` if at least one record matches the query.

The call returns `false` if there are no records that match the query.

Example

```
rtecall( "recordexists", $L.rc, "contacts", true )
```

Returns `true` if there are any records in the contacts table.

```
rtecall( "recordexists", $L.rc, "device", "logical.name=\"pc001\"" )
```

Returns `true` if there is a record in the device table with a logical name of pc001 regardless of whether the current user has access to the record or not.

2.4.3.23. rtecall("refresh") function

This function retrieves the most current value of a record from the database and returns it in the file variable.

Function

rtecall("refresh")

Format

```
$L.success.flg=rtecall("refresh", $L.errcode, $L.file)
```

Parameters

The following parameters are valid for the **rtecall("refresh")** function:

Variable/Value	Description
$L.errcode = 0	The function succeeded.
$L.errcode = 1	The $L.file doesn't represent a file variable.
$L.file	The file variable initialized by the **rinit** command panel. It contains the current record.

Factors

If the $L.success.flg is `false`, the function failed. It is `true`, the function succeeded.

Example

```
$L.success.flg = rtecall( "refresh", $L.errcode, $L.file )
```

2.4.3.24. rtecall("resetnotebook") function

A RAD function that adds an XML attribute "resetnotebook=true" to the "execute" response of the current RAD thread. The Service Manager client will set the active page of a notebook to its first page when this XML attribute is present.

Function

rtecall("resetnotebook");

Format

```
system.functions.rtecall("resetnotebook", ret, 0);
```

Parameters

This function has no parameters.

Factors

This function always returns RC_SUCCESS.

Example

```
var ret = RC_SUCCESS;
```

```
system.functions.rtecall("resetnotebook", ret, 0);
```

2.4.3.25. rtecall("rfirst") function

A RAD function that places the pointer at the first record in a record list.

Function

rtecall("rfirst")

Format

```
$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.file)
```

Parameters

The following parameters are valid for the **rtecall("rfirst")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	The name of the sub-function to call, in this case "rfirst".
$L.return.code	Number	Provides a more detailed return code.
$L.file	File	A file handle that represents the collection.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg=rtecall("rfirst",$L.return.code, $L.file)
```

2.4.3.26. rtecall("rgoto") function

A RAD function that places the pointer at the indicated record.id in a record list.

Function

rtecall("rgoto")

Format

```
$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.file, $L.record.id)
```

Parameters

The following parameters are valid for the **rtecall("rgoto")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	The name of the sub-function to call, in this case "rgoto".
$L.return.code	Number	Provides a more detailed return code.
$L.file	File	A file handle that represents the collection.
$L.record.id	Number	A record number.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg=rtecall("rgoto",$L.return.code, $L.file, $L.record.id)
```

2.4.3.27. rtecall("rid") function

A RAD function that returns the record number of the current record, as represented by $L.file.

Function

rtecall("rid")

Format

```
$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.file, $L.record.id)
```

Parameters

The following parameters are valid for the **rtecall("rid")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	The name of the sub-function to call, in this case "rid".
$L.return.code	Number	Provides a more detailed return code.
$L.file	File	A file handle that represents the collection.
$L.record.id	Number	A record number.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg=rtecall("rid",$L.return.code, $L.file, $L.record.id)
```

2.4.3.28. rtecall("scantable") function

A RAD function that scans the specified table for certain types of invalid records and fixes the records.

Function

rtecall("scantable")

Format

`$L.success.flg=rtecall($L.fnc.name, $return.code, $scantype, $tablename, $bfix, $columns)`

`$L.success.flg=rtecall($L.fnc.name, $return.code, $scantype, $tablename, $bfix, $totalnum, $fixed, $deleted)`

Parameters

The following parameters are valid for the **rtecall("scantable")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call. In this case, "scantable".
$L.return.code	Number	Provides a more detailed return code.
$scantype	Number	Specifies the type of scan the function will perform, which can be one of the following: • 1 - to scan for duplicate records on specified columns • 2 - to scan for null values on non-nullable fields, such as fields with a "Unique" or "No Nulls" key applied • 3 - to scan for values whose data types don't match the data type defined on the field
$L.tablename	String	The name of the table to be scanned.
$bfix	Bool	Specifies whether to automatically fix the invalid data.
$columns	String array	Only available when $scantype is 1, specifies the columns on which the function will scan for duplicate records.
$totalnum	Number	Indicates the number of the invalid records that were found.
$fixed	Number	Indicates the number of the invalid records that were fixed.
$deleted	Number	Indicates the number of the records that were removed.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

$totalnum, $fixed, and $deleted are available when $scantype is 2 or 3. This total number should include $fixed, $deleted, and the number of invalid records that failed to be fixed.

Examples

The following function call scans the **ioaction** table for duplicate records on the **field1** and **field2** fields and fixes the duplicate records.

```
$L.flg=rtecall("scantable", $L.return.code, 1, "ioaction", true, {"field1","field2"})
```

The following function call scans the **ioaction** table for null values that are disallowed by keys.

```
$L.flg=rtecall("scantable", $L.return.code, 2, "ioaction", true, $all, $fixed, $deleted)
```

Additional information

If you choose to fix invalid data by setting `$bfix` to 1:

- In a "type-1" scan, the function differentiates duplicate items by appending suffixes such as -dup1, -dup2...-dupx.
- In a "type-2" scan, the following behaviors occur:
  - The function removes the record if null values violate a "Unique" key.
  - Or the function replaces the first of the null values with a default value if null values violate a "No Nulls" key.
- In a "type-3" scan, the following behaviors occur:
  - The function converts the value into the correct data type if the value data makes sense in the target data type.
  - The function replaces the value with a default value in the correct data type.

2.4.3.29. rtecall ("select") function

A RAD function that retrieves a list of records for a file in ascending or descending order according to the specified sort fields and group fields.

Function

rtecall("select")

Format

```
$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.file, $L.query, $L.sortfields, $L.sorttype, $L.groupfields,
$L.groupsorttype, $L.columns)
```

Parameters

The following parameters are valid for the **rtecall("select")** function:

Parameter	Data Type	Description
$L.success.flg	Logical	Indicates whether the function is successful or not.
$L.fnc.name	String	Name of the sub-function to call. This parameter is "select" in this case.
$L.return.code	Number	Provides a more detailed return code: • 1: Only one record found • 2: Multiple records found • 3: No records were found • 4: Query reached a timeout
$L.file	File	A file handle that represents the record list.
$L.query	String	The query condition for retrieving records.
$L.sortfields	Array	The array of fields which the record list is sorted by.
$L.sorttype	Logical	The sorting order. `true` is descending, and `false` is ascending. By default, this value is `false`.
$L.groupfields	Array	The array of fields which the record list is grouped by.
$L.groupsorttype	Logical	The sorting order for group fields. `true` is descending, and `false` is ascending. By default, this value is `false`.
$L.columns	Array	The array of the selected fields of the file.

Factors

- If `$L.success.flg` is `false`, the function fails. If `$L.success.flg` is `true`, the function succeeds.
- The parameters after `$L.query` are optional.

Example

`$L.success.flg=rtecall("rinit", $L.return.code, $L.operator, "operator");`

`$L.success.flg=rtecall("select", $L.return.code, $L.operator, "contact.name=contact.name in $L.file", {"last.login"}, false, {"company"}, false, {"name", "contact.name", "company"});`

2.4.3.30. rtecall ("setdebug") function

A RAD function that sets the debug parameters.

Function

rtecall("setdebug")

Format

```
$L.success.flag= rtecall($L.fnc.name, $L.return.code, $L.debug.parameter)
```

Parameters

The following parameters are valid for the **rtecall("setdebug")** function:

Parameter	Data Type	Description
$L.success.flag	Logical	Indicates whether the function is successful or not.
$L.fnc.name	String	The name of the sub-function to call. This parameter is "select" in this case.
$L.return.code	Number	Provides a more detailed return code.
$L.debug.parameter	Number	The debug parameters to set.

Factors

If `$L.success.flg` is `false`, the function fails. If `$L.success.flg` is `true`, the function succeeds.

Example

```
$L.success.flg=rtecall("setdebug", $L.return.code, "sqldebug:1 rtm:3")
```

This example sets sqldebug to 1 and rtm to 3 in current session.

2.4.3.31. rtecall("sort") function

A RAD function that sorts a list or a list of lists in ascending or descending order.

Function

rtecall("sort")

Format

```
$L.success.flg=rtecall($L.fnc.name, $L.return.code, $L.grid, $L.index, $L.type, $L.nulls)
```

Parameters

The following parameters are valid for the **rtecall("sort")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "sort".
$L.return.code	Number	Provides a more detailed return code.
$L.grid	Array	The list or list of lists to be sorted.
$L.index	Number	The index of the array in the grid to use as the sort field. A zero (`0`) is the first element of the array.
$L.type	Number	The type of sort to perform. Use zero (`0`) for an ascending sort or one (`1`) for a descending sort.
$L.nulls	Number	Specifies how the function should sort null values. Use one (`1`) to sort null values at the end of the list or zero (`0`) to sort values at the start of the list.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Examples

```
$L.list={29, 2, 72, 42}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 1, 0)
Returns: $L.list={2, 29, 42, 72}
```

```
$L.list={29, 2, 72, 42}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 1, 1)
Returns: $L.list={72, 42, 29, 2}
```

```
$L.list={29, 2, , 72, 42}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 1, 0, 1)
Returns: $L.list={2, 29, 42, 72, }
```

```
$L.list={29, 2, , 72, 42}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 1, 0, 0)
Returns: $L.list={, 2, 29, 42, 72}
```

```
$L.list={{"a", "b", "d", "c"},{1, 3, 4, 2}}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 1, 0)
Returns: $L.list= {{"a", "c", "b", "d"}, {1, 2, 3, 4}}
```

```
$L.list={{"a", "b", "d", "c"},{1, 3, 4, 2}}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 1, 1)
Returns: $L.list={{"d", "b", "c", "a"}, {4, 3, 2, 1}}
```

```
$L.list={{"a", "b", "d", "c"},{1, 3, 4, 2}}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 0, 0)
Returns: $L.list={{"a", "b", "c", "d"}, {1, 3, 2, 4}}
```

```
$L.list={{"a", "b", "d", "c"},{1, 3, 4, 2}}
$L.success.flg=rtecall("sort", $L.return.code, $L.list, 0, 1)
Returns: $L.list={{"d", "c", "b", "a"}, {4, 2, 3, 1}}
```

2.4.3.32. rtecall("statusupdate") function

A RAD function that dynamically changes the caption of a widget on the current format. The widget must have a name property specified. This function is commonly used to update the format during a long-running transaction to inform the user of the status of the current operation. For example, converting a file to an RDBMS.

Function

rtecall("statusupdate")

Format

`$L.success.flag = rtecall($L.func.name, $L.return.code, $L.widget.name, $L.widget.caption)`

Parameters

The following parameters are valid for the **rtecall("statusupdate")** function:

Parameter	Data type	Description
$L.widget.name	String	The name of the form widget, as specified in the name property of the widget.
$L.widget.caption	String	A string to be displayed in the widget.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

`$L.success.flg = rtecall("statusupdate", $L.return, "MyScrollingMarqueeWidget", "The end user will see this text in the marquee" )`

2.4.3.33. rtecall("transtart") function

A RAD function that measures the amount of data transferred, elapsed time, and CPU usage of any transaction. This function is commonly invoked from the RAD debugger and used in conjunction with the rtecall ("transtop") function.

Function
    rtecall("transtart")
Format
    $L.success.flag= rtecall($L.fnc.name, $L.return.code, $L.transaction)

Parameters

The following parameters are valid for the rtecall("transtart") function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "transtart".
$L.return.code	Number	Provides a more detailed return code.
$L.transaction	String	Any name selected by the user for the transaction to measure.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg= rtecall("transtart", $L.return, "problemopen")
```

2.4.3.34. rtecall("transtop") function

A RAD function that measures the amount of data transferred, elapsed time, and CPU usage of any transaction. The function is commonly invoked from the RAD debugger and used in conjunction with the **rtecall("transtart")** function.

- The CPU time is retrieved from the server machine.
- You can have multiple transaction timings active at the same time.

Function

rtecall("transtop")

Format

```
$L.success.flag= rtecall($L.fnc.name, $L.return.code, $L.transaction, $results)
```

Parameters

The following parameters are valid for the **rtecall("transtop")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates whether or not the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "transtop".
$L.return.code	Number	Provides a more detailed return code.
$L.transaction	String	Any name selected by the user for the transaction to measure.
$L.$results	Array	The results gathered, in the form of an array: `{elapsed seconds, cpu seconds, 0, 0, 0}`. Service Manager no longer keeps track of the third, fourth, and fifth value in the array and it always reports as zero.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg=rtecall("transtop", $L.return.code, "problemopen", $results)
$results={58.164, 0.961, 0, 26712, 2477}
```

This example shows that the problemopen transaction took 58.164 seconds of real time and 0.961 CPU seconds.

2.4.3.35. rtecall("trigger") function

A RAD function that turns triggers on or off for the current session. Other Service Manager users aren't affected.

Function

rtecall("trigger")

Format

```
$L.success.flag= rtecall($L.fnc.name, $L.return.code, $L.switch)
```

Parameters

The following parameters are valid for the **rtecall("trigger")** function:

Parameter	Data type	Description
$L.success.flg	Logical	Indicates if the function was successful.
$L.fnc.name	String	Name of the sub-function to call, in this case "trigger".
$L.return.code	Number	Provides a more detailed return code.
$L.switch	Number	The value is either one (`1`) to turn triggers on. or zero (`0`) to turn triggers off.

Factors

If the $L.success.flg is `false`, the function failed. If it is `true`, the function succeeded.

Example

```
$L.success.flg=rtecall("trigger", $L.return.code, 1) to turn triggers on
$L.success.flg=rtecall("trigger", $L.return.code, 0) to turn triggers off
```

2.4.4. List: RAD routines

This section provides a list of RAD routines, a brief definition of each, and a link to a detailed description of their use.

Routine	Description
as.copy	Copies an element from one position to another, leaving the original in place.
as.delete	Deletes an element from an array.
as.get.name	Returns the structure name of the arrayed structure in which the cursor is located.
as.insert	Inserts a NULL element.
as.move	Moves an element from one position in an array to another by creating a copy and then deleting the original
as.options	Displays all available arrayed structure options on the rio (Record Display/Input) Command panel.
as.sort	Sorts an arrayed structure based on the contents of a particular field within the arrayed structure.
axces.apm	Creates or updates an incident based on an Event Services eventin record.
axces.cm3	Creates or updates a change request based on an Event Services eventin record.
axces.cm3.cts.write	Creates a schedule record that processes alerts for Change Management records.
axces.database	Adds, updates or deletes records from Service Manager files using Event Services.
axces.email	Creates an eventout record based on the Service Manager mail record that was passed in.
axces.move.intoout	Copies an eventin record to the eventout file and changes the evtype.
axces.rm	Creates or updates Request Management quotes via Event Services eventin record.
axces.sap.hybrid.evin	Breaks events into appropriate constituent parts.
axces.software	Adds, updates, or deletes a software configuration item that an external agent (other than ServerView or StationView) discovers to the pcfiles file if the filter criteria are satisfied.
axces.sm	Creates or updates Service Desk interactions based on an eventin record using Event Services.
axces.write	Build the eventout record used by the SCAutomate interface.
database	Provides access to other tables from a particular record.
es.approval	Processes approvals for Request Management and Change Management.
fill.fc	Copies data from a target file to the current file, using predefined links under user control.
getnumb.fc	Establishes the numbering sequence in sequential numbering.
marquee.publish	Sends a marquee message.

marquee.send	Reads the schedule record and initiate the marquee.publish routine.
message.fc	Sends messages under user control.
post.fc	Executes the posting routine from the subroutines process (automatic posting) and from the additional options process (manual posting).
publish	Publishes a variable into SYSPUB.
query.stored	Accesses all queries for a specific file or to execute a specific stored query.
scauto.inventory	Provides access to configuration items.
sort.array	Sorts simple arrays (number, character, date/time) in either ascending or descending order.
us.js.call	A RAD routine that can invoke any JavaScript function in the ScriptLibrary. This routine enables you to run a JavaScript function without connecting to the Service Manager server via a client.
validate.fields	Validates fields in a form.
wmi.inventory.checks	Checks the eventin record for status information from a type of scan. It also processes the information from the scan into the Configuration Management application using the scauto.inventory RAD routine.

RAD input and output conversion

Input conversion

Routine	Description
input.accum.types	Validates the entry of accumulator types in the Report Writer.
input.backslash	Looks for ending backslash on query, appends another backslash.
input.date	Allows entry of dates in various formats.
input.edate	Allows entry of European dates in the format DD/MM/YY(YY) HH:MM:SS.
input.fdate	Allows entry of European dates in the format YY(YY)/MM/DD HH:MM:SS.
input.number	Input Number from character field ($4.33=4.33).
input.percent	Edits numbers into valid percent values.
input.time	Append a time entered by a user to the current date.
input.underlines	Convert Embedded Spaces to Underlines.

Output conversion

Routine	Description
output.all.caps	Converts to all caps.
output.asterisk	Boolean Data Output Conversion Routine (Converts 'true' to *).
output.backslash	Looks for ending backslash on query, appends another backslash.
output.date	Converts a standard SM date to one of a number of specified formats.
output.edate	Converts a standard SM date to the following European date: DD/MM/YY(YY) HH:MM:SS.
output.exdate	Converts a standard SM date to: DD MMM YYYY.
output.fdate	Converts a standard SM date to the following French date: YY(YY)/MM/DD HH:MM:SS.
output.fm	Displays numbers as money fields using specified mask.
output.init.caps	Converts to initial caps.
output.money	Displays numbers as money fields using specified mask.
output.money.save	Displays numbers as money fields using specified mask.
output.money3	Displays numbers as money fields using specified mask.
output.no.caps	Converts to all lowercase.
output.percent	Edits numbers entered as percentages and then displays them.
output.privilege	Converts all except true, false, unknown and NULL to *.
output.time.adj	Converts a standard SM date to one of a number of specified formats.
output.yn	Converts 'true/false' to 'yes/no' and leaves 'unknown' alone.
output.zf	Edits strings entered with specified mask.
output.strip.htmltags	Strips away HTML tags from the text.

2.4.4.1. RAD routine: as.copy

A RAD routine that copies an element within an array or structure from one position to another, leaving the original in place. This routine is used in conjunction with as.delete, as.insert, and as.move routines.

Routine

as.copy

Parameters

The following parameters are valid for the as.copy routine:

Name	Data type	Required	Description
file	record	No	The file variable that you are currently editing. The value is always `$file` when called from Format Control.
name	character	No	Data is not normally passed to this parameter. The default is the result of executing the **cursor.field.name(1)** function. The application as.get.name then executes against this result. **Note** Note You can pass the name of a specific arrayed structure. If the data passed to this parameter doesn't contain commas, it is considered to be an arrayed structure name. If a comma is found, the application as.get.name is executed against the data.

index	number	No	The number of lines (height property) in the subform that displays the arrayed structure. For example, if the arrayed structure subform contains data on lines 1 and 2, then this parameter is 2. The default value is 1. **Note** Note This is not the window size that displays the arrayed structure. When calling the application from Format Control, the value must be passed as a number by using the following syntax: `val("n", 1)` Where: The value of `n` is the number of lines in the subform, and `1` indicates a data type of number.

2.4.4.2. RAD routine: as.delete

A RAD routine that deletes an element from an array. This routine is used in conjunction with as.copy, as.insert, and as.move.

Routine

as.delete

Parameters

The following parameters are valid for the **as.delete** routine:

Name	Data type	Required	Description
file	record	No	The file variable currently being edited. The value is always `$file` when called from Format Control.
name	character	No	Data is not normally passed to this parameter. The default is the result of executing the **cursor.field.name(1)** function . The application as.get.name then executes against this result. **Note** Note You can pass the name of a specific arrayed structure. If the data passed to this parameter doesn't contain commas, it is considered to be an arrayed structure name. If a comma is found, the application as.get.name executes against the data.

index	number	No	The number of lines (height property) in the subform that displays the arrayed structure. For example, if the arrayed structure subform contains data on lines 1 and 2, then this parameter is 2. The default value is 1. **Note** Note This is not the window size that displays the arrayed structure. When calling the application from Format Control, the value must be passed as a number by using the following syntax: `val("n", 1)` Where: The value of `n` is the number of lines in the subform, and `1` indicates a data type of number.

2.4.4.3. RAD routine: as.get.name

A RAD routine that returns the structure name of the arrayed structure where the cursor is located.

Routine

as.get.name

Parameters

The following parameters are valid for the **as.get.name** routine:

Name	Data type	Required	Description
name	character	No	The structure name of the arrayed structure. The default is the result of executing the **cursor.field.name(1)** function .
prompt	character	No	The returned structure name of the arrayed structure. This is a data return parameter, data should not be passed to it.

2.4.4.4. RAD routine: as.insert

A RAD routine used to insert a NULL element. It is used in conjunction with as.copy, as.delete, and as.move.

Routine

as.insert

Parameters

The following parameters are valid for the **as.insert** routine:

Name	Data type	Required	Description
file	record	No	The file variable currently being edited. The value is always `$file` when called from Format Control.
name	character	No	Data is not normally passed to this parameter. The default is the result of executing the **cursor.field.name(1)** function. The application as.get.name then executes against this result. **Note** Note You can pass the name of a specific arrayed structure. If the data passed to this parameter doesn't contain commas, it is considered an arrayed structure name. If a comma is found, the application as.get.name executes against the data.

index	number	No	The number of lines (height property) in the subform that displays the arrayed structure. For example, if the arrayed structure subform has data on lines 1 and 2, then this parameter is `2`. The default value is `1`. **Note** Note This is not the window size that displays the arrayed structure. When calling the application from Format Control, the value must be passed as a number by using the following syntax: `val("n", 1)` Where: The value of `n` = the number of lines in the subform, and `1` indicates a data type of number.

2.4.4.5. RAD routine: as.move

A RAD routine used to move an element from one position in an array to another by creating a copy and then deleting the original. This routine is used in conjunction with **as.copy**, **as.insert**, and **as.delete**.

Routine

as.move

Parameters

The following parameters are valid for the **as.move** routine:

Name	Data type	Required	Description
file	record	No	The file variable currently being edited. The value is always `$file` when called from Format Control.
name	character	No	Data is not normally passed to this parameter. The default is the result of executing the cursor.field.name(1) function. The application as.get.name then executes against this result. **Note** Note You can pass the name of a specific arrayed structure. If the data passed to this parameter doesn't contain commas, it is considered an arrayed structure name. If a comma is found, the as.get.name application executes against the data.

index	number	No	The number of lines (height property) in the subform that displays the arrayed structure. For example, if the arrayed structure subform has data on lines 1 and 2, then this parameter is `2`. The default value is `1`. **Note** Note This is not the window size that displays the arrayed structure. When calling the application from Format Control, the value must be passed as a number by using the following syntax: `val("n", 1)` Where the value of `n` = the number of lines in the subform, and `1` indicates a data type of number.

2.4.4.6. RAD routine: as.options

A RAD routine used to display all available arrayed structure options one *rio* (Record Display/Input) Command panel.

Note

Note The as.options routine provides the control needed to copy, insert, delete, and move fields. It's not necessary to run **as.copy**, **as.move**, **as.insert**, and **as.delete** in conjunction with this routine.

Routine

as.options

Parameters

The following parameters are valid for the **as.options** routine:

Name	Data type	Required	Description
file	record	No	The file variable currently being edited. The value is always `$file` when called from Format Control.
name	character	No	Passes the name of the current form. If left blank, the default is the result of executing the **current.format()** function.
cond.input	Boolean	No	Allows the user to edit the data record when the Arrayed structure maintenance options are displayed. The default value is `true`. **Note** Note When calling the application from Format Control, the value must be passed as a data type of 4 (logical) by using the following syntax: `val("false", 4)`
index	number	No	The number of lines in the subform that displays the arrayed structure. For example, if the arrayed structure subform has data on lines 1 and 2, then the parameter is 2. The default value is `1`. **Note** Note This is not the window size that displays the arrayed structure. When calling the application from Format Control, the value must be passed as a number by using the following syntax: `val("n", 1)` Where: The value of `n` is the number of lines in the subform, and one (`1`) indicates a data type of number.

2.4.4.7. RAD routine: as.sort

A RAD routine used to sort a structure that includes an array. It sorts based on the contents of a particular field within the array of the structure. This routine supports major, minor, and intermediate sorting by allowing you to sort multiple array fields in the structure with one pass of the routine. You can also specify an ascending or descending sort order. You must call this routine from Format Control or RAD on an as-needed basis.

Routine

as.sort

Parameters

The following parameters are valid for the **as.sort** routine:

Name	Data type	Required	Description
file	record	No	The file variable currently being edited. The value is always `$file` when called from Format Control.
name	character	No	The name of the structure you want to sort. This name corresponds to the input field of the subform object in the form to which the Format Control record is attached. **Note** Note If the data passed to this parameter doesn't contain commas, it is considered an arrayed structure name. If a comma is found, the **as.get.name** application executes against the data.
numbers	array (numbers)	Yes	Specifies the field numbers on which to sort within the arrayed structure. In Format Control, you must first define a `$variable` in the Initializations form for the array of numbers. Next, pass that variable to the numbers input field in the Additional Options definition. **Note** Note Field numbers correspond to the index numbers listed in the database dictionary and not to the sequence in which they appear on the form.

condition	array (Boolean)	No	Specifies either ascending or descending order for each field being sorted in the arrayed structure, as specified in the numbers parameter. To specify an ascending order, pass a value of `true`. To specify descending order, pass a value of `false`. The default value is `true`. Each element in this array has a direct correlation with the elements of the numbers parameter. In Format Control, you must first define a `$variable` in the Initializations screen that defines the Boolean array. Next, pass the variable to this input field in the Additional Options definition.

2.4.4.8. RAD routine: axces.apm

A RAD routine that creates or updates an incident based on an Event Services eventin record.

Routine

axces.apm

Parameters

The following parameters are valid for the **axces.apm** routine:

Name	Data type	Required	Description
record	record	Yes	Identifies the eventin record to pass.
text	character	No	Provides the specific action to take.
prompt	character	No	Indicates the Map Name used in event registration.
query	character	No	Indicates the query to use to find the pre-existing record updated by this Input event.
string1	character	No	Identifies the Service Manager file where the data is recorded.
boolean1	Boolean	No	An error occurs if this is not set. If the value is set to `NULL`, change to `close`. If evstatus begins with error, the value is `false`. If evstatus doesn't begin with error, the value is `true`.

2.4.4.9. RAD routine: axces.cm3.cts.write

A RAD routine that creates a schedule record that processes alerts for Change Management records.

Routine

axces.cm3.cts.write

Parameters

The following parameters are valid for the **axces.cm3.cts.write** routine:

Name	Data type	Required	Description
record	record	No	Indicates what file variable passes to the application.
name	character	No	Indicates the System ID of the system to send the message to and where to calculate the schedule time.
prompt	character	No	Indicates the Client of the system to send the message to and where to calculate the schedule time.
time1	time	No	What's the target time for this event? Use the **tod()** function to indicate to the system that the message sends at the next acceptable time.
query	character	No	Once you find the next acceptable time for this System ID and client combination, a new schedule record generates for a message of this type.
boolean1	Boolean	No	When set to `true`, the rescheduling portion of the code is bypassed and all messages occur as soon as possible.

2.4.4.10. RAD routine: axces.cm3

A RAD routine that creates or updates a change request based on an Event Services eventin record.

Routine

axces.cm3

Parameters

The following parameters are valid for the **axces.cm3** routine:

Name	Data type	Required	Description
string1	character	No	Identifies the area of CM3 to work in and where to record data for branching or changing tasks (cm3r or cm3t).
record	record	No	Indicates eventin record to process.
prompt	character	No	What map interprets the incoming message?
text	character	No	What map interprets the incoming message?
boolean1	Boolean	No	Are Output events created?
string1	character	No	Identifies the area of CM3 to work in and where to record data for branching or changing tasks (cm3r or cm3t).
query	character	No	The query for existing change record.

2.4.4.11. RAD routine: axces.database

A RAD routine called by many Event Services events to add, update or delete records from Service Manager tables using Event Services.

Routine

axces.database

Parameters

The following parameters are valid for the **axces.database** routine:

Parameter Name	Data type	Required	Description
record	file record	Yes	Identifies the eventin record to pass.
prompt	character	Yes	Identifies the Map name used in Event Registration. The event map name must end with **a** for add events and must end with **u** for update events.
string1	character	Yes	Identifies the Service Manager table to populate with data.
text	character	Yes	Provides the specific action to take, such as the following: • add • delete • update
query	character	No	Indicates the query to use to find the existing record to update.
boolean1	Boolean	No	When flagged, the value is `true` and Service Manager writes an eventout record. When not flagged, the value is `false` and no eventout record is written.
cond.input	Boolean	No	Specifies the behavior to follow when more than one record matches the query. When flagged `true`, Service Manager processes more than one record. When not flagged (value is `false`), Service Manager processes only the first record found.
name	character	No	Indicates the Format Control record to use.

2.4.4.12. RAD routine: axces.email

A RAD routine that creates an eventout record based on the Service Manager mail record that was passed in.

Routine

axces.email

Parameters

The following parameters are valid for the **axces.email** routine:

Name	Data type	Required	Description
record	record	No	Writes the unique record identifier to the eventin record. Set up the `$axces` variable using Format Control to create the intended result.
text	character	No	Indicates the delimiter character.

2.4.4.13. RAD routine: axces.move.intoout

A RAD routine that copies an eventin record to the eventout file and changes the evtype.

Routine

axces.move.intoout

Parameters

The following parameters are valid for the **axces.move.intoout** routine:

Name	Data type	Required	Description
record	record	Yes	Identifies the eventin record to pass.
name	character	No	Identifies the eventout type.

2.4.4.14. RAD routine: axces.rm

A RAD routine that creates or updates Request Management quotes via Event Services eventin record.

Routine

axces.rm

Parameters

The following parameters are valid for the **axces.rm** routine:

Name	Data type	Required	Description
record	record	No	This parameter identifies which eventin record to pass.
text	character	No	Provides the specific action to take, such as add, open, update, close, approve, disapprove or unapprove.
prompt	character	No	Indicates the Map name in event registration.
query	character	No	The query string used to select a pre-existing record to update.
string1	character	No	Identifies the Service Manager table (ocmq, ocmo, or ocml) where data is recorded.
boolean1	Boolean	No	The logical parameter flag, `true` or `false`, that indicates whether to write the eventout record on transaction completion.
name	character	No	Identifies the operator (security profile) in Service Manager to use and which userID to stamp on records that are processed using this event.

2.4.4.15. RAD routine: axces.sap.hybrid.evin

A RAD routine that splits events out into the appropriate constituent parts.

Routine

axces.sap.hybrid.evin

Parameters

The following parameters are valid for the **axces.sap.hybrid.evin** routine:

Name	Data type	Required	Description
record	record	No	A hybrid event that calls the appropriate routines for header and detail events. This parameter expects one header and any number of properly-formed detail events.
prompt	character	No	Identifies the eventregister that interprets the detail portion of the event message.
name	character	No	Identifies the eventregister that interprets the header portion of the event stream.

2.4.4.16. RAD routine: axces.sm

A RAD routine that creates or updates Service Desk interactions based on an eventin record using Event Services.

Routine

axces.sm

Parameters

The following parameters are valid for the **axces.sm** routine:

Name	Data type	Required	Description
record	record	Yes	Identifies the eventin record to pass.
prompt	character	No	Indicates the Map name used in event registration.
string1	character	No	Identifies the Service Manager file where data is recorded.
query	character	No	Indicates the query to use to find the pre-existing record that this Input event updates.
boolean1	Boolean	No	A logical parameter flag, `true` or `false`, that indicates whether the eventout record writes on transaction completion.
text	character	No	Designates the output event type.

2.4.4.17. RAD routine: axces.software

A RAD routine that adds, updates, or deletes a software configuration item (CI) that an external agent discovers to the pcfiles file once the filter criteria are satisfied.

Routine

axces.software

Parameters

The following parameters are valid for the **axces.software** routine:

Name	Data type	Required	Description
record	record	No	Writes the unique record identifier to the eventin record. Set up the `$axces` variable using Format Control to create the intended result.
prompt	character	No	Indicates the Map name used in event registration.
string1	character	No	Identifies the software file in Service Manager where the configuration item (CI) data is recorded.
query	character	No	In this case, `logical.name` = `1` and `description` = `9` in $axces.fields.
boolean1	Boolean	No	A logical parameter flag, `true` or `false`, that indicates whether the eventout record writes on transaction completion.
name	character	No	Specifies the Format Control record to use.

2.4.4.18. RAD routine: axces.write

A RAD routine used to build an eventout record used by the SCAutomate (SCAuto) interface.

Routine

axces.write

Parameters

The following parameters are valid for the **axces.write** routine:

Name	Data type	Required	Description
record	record	Yes	Names the record to be written. In Format Control the record is `$file`.
name	character	No	Names the registration type as it appears in the eventregister file. For example, to write an eventout record when an incident is opened, use the value `pmo` for this parameter.
string1	character	No	Defines the separation character. If you define your own character, ensure that it doesn't occur naturally in fields in the event. The default is value is the caret character (`^`).
text	character	No	Defines a system sequence ID with a maximum length of 16 characters. Service Manager generates the system sequence ID unless you supply one.
prompt	character	No	Defines a user sequence ID with a maximum length of 16 characters.
query	character	No	Defines the user name. The default value is `operator()`.

2.4.4.19. RAD routine: database

A RAD routine used to access other tables from a particular record. For example, to enable an option button to appear on your users' screens, allowing them to call the location table from the contacts table.



Note

Note This routine is only called from Additional Options in Format Control.

Routine

database

Parameters

The following parameter is valid for the database routine:

Name	Data type	Required	Description
name	character	No	The name of the form you want to open.
record	record	No	Specifies the Format Control record to use.

2.4.4.20. RAD routine: es.approval

A RAD routine that processes approvals for the Request Management and Change Management applications.

Routine

es.approval

Parameters

The following parameters are valid for the **es.approval** routine:

Name	Data type	Required	Description
record	record	Yes	Identifies the eventin record to pass.
text	character	No	Identifies the Service Manager table where data is recorded.
name	character	No	Indicates the Map name used in event registration.
cond.input	Boolean	No	A logical parameter, `true` or `false`, that specifies whether the eventout record writes on transaction completion.

2.4.4.21. RAD routine: fill.fc

A RAD routine that copies data from a target file to the current file using predefined links under user control.

Routine

fill.fc

Parameters

The following parameters are valid for the **fill.fc** routine:

Name	Data type	Required	Description
record	record	Yes	The value is `$file` when calling fill.fc through the Additional Options process.
text	character	No	The name of the field in the target file to fill. The default is the current field (the field where the cursor is located). If the field is an array, the fill operation attempts to copy data back to that item in the selected array.
string1	character	No	The form name that determines which link record to use. The default is the link record for the current form.

Variables supported

`$fill.replace`

`$project.first`

`$fill.exact`

2.4.4.22. RAD routine: getnumb.fc

A RAD routine used to establish the numbering sequence in sequential numbering. You can create simple identification numbers for database records, or complex numbers composed of prefixes and suffixes.

Routine

getnumb.fc

Parameters

The following parameters are valid for the **getnumb.fc** routine:

Name	Data type	Required	Description
record	record	Yes	The data record where the number will be placed. The value of this parameter must be `$file`.
name	character	Yes	The sequential number Class (category) of the record that the Format Control record is attached. This value appears in the Class field of the Sequential Numbering File.
prompt	character	Yes	Names the field in the data record where the sequential number appears.
text	character	No	Defines the data type of the sequential number (number or string).
number1	number	No	Defines the length of the sequential number. The subroutine prefixes as many zeroes to the number as necessary to match the required number of digits. Pass the **val()** RAD function to the *number1* parameter. For example, a four-digit number has a value of `val("4", 1)`. Where: The number `4` is the length of the number and the number `1` is the data type (a number representation).
string1	character	No	Defines the prefix. This is the identifying character string that appears before the number.
numbers,1	array (numbers)	No	Determines the reset point for the sequential number. When the value passed to the application is reached, the sequential number resets to the starting value.
numbers,2	array (numbers)	No	Establishes a new starting value for the sequential number when the reset point is reached.

numbers,3	array (numbers)	No	Sets the increment or decrement value. For example, if you want to increment by 2, displaying odd or even numbers only.
numbers,4	array (numbers)	No	Allows the user to increment or decrement the sequential number independent of the number file. Use this parameter to attach a specified number to your records without updating the number file.

2.4.4.23. RAD routine: marquee.publish

A RAD routine used to send a marquee message. This routine is always called from within marquee.send.

Routine

marquee.publish

Parameters

The following parameters are valid for the **marquee.publish** routine:

Name	Data type	Required	Description
name	character	Yes	The published variable used by the agent to search the database. **Note** Note The system automatically adds `$MARQUEE` to the beginning your variable name before publishing it.
text	character	No	A variable that contains the message to be published.
prompt	character	No	The display color for your marquee or the valid system number in the Color field. The default value is Red.
boolean1	Boolean	No	A logical field. Set the flag to `true` to add this message to the marquee table. The marquee agent then reads the record and publishes it. If the flag is set to `false`, the routine immediately publishes your message without adding it to the marquee table (recommended method).
cond.input	Boolean	No	A logical field. Set the flag to `true` to add the name to the database if that record doesn't exist.

2.4.4.24. RAD routine: marquee.send

A RAD routine used to read the schedule record and initiate the **marquee.publish** application.

Routine

marquee.send

Parameters

The following parameters are valid for the **marquee.send** routine:

Name	Data type	Required	Description
record	character	Yes	Schedule record for the marquee process
boolean1	Boolean	No	Logical field. Set the flag to `true` to send all marquee messages defined in the schedule record. If set to `false`, or the field is `NULL`, the application publishes only the specific marquee message defined in the schedule record (recommended method).

2.4.4.25. RAD routine: message.fc

A RAD routine used to send messages under user control.

Routine

message.fc

Parameters

The following parameters are valid for the **message.fc** routine:

Name	Data type	Required	Description
index	number	No	Select one of three message levels: • `1` for information, this is the default. • `2` for action. • `3` for error. In Text mode, message levels may appear in different colors.
prompt	character	No	Enter a message class that matches one of the records in the msgclass table, such as problemclose. To send email messages, there must be a msgclass record with a type of email for the message class name specified. The default value is `msg`.
text	character	Yes	The text of the message can be a string or an array. For example, generate an array of the screen contents using the **genout()** function and then insert lines of text at the top of the array.
name	character	No	This value can contain either a list of operator names or a single name. For internal Service Manager messages, the names must be operator ID's defined in the operator table. For email, the names can either be operator ID's or contact names (contact.name) defined in the contacts table; an email address must be specified in the relevant table. The default is `operator()`.
string1	character	No	The message name parameter identifies the message. Use the name of the Service Manager application or application area that generates the message.
number1	number	No	The message number parameter identifies a message within the area specified by the *string1* parameter.

query	character	No	The mail class parameter is used within Incident Management applications to identify the record number so that mail already sent can be selected and updated. It must contain the string pm.main and a Mail Target must be defined.
names,1	array (characters)	No	The mail target parameter must contain the incident record number.

2.4.4.26. RAD routine: post.fc

A RAD routine used to execute the posting routine from the subroutines process (automatic posting) and from the additional options process (manual posting).

Routine

post.fc

Parameters

The following parameters are valid for the **post.fc** routine:

Name	Data type	Required	Description
file	record	Yes	The value is always `$file` when calling post.fc through the Additional Options process. In most cases, the variable is `$file` when calling post.fc through the Subroutine process; however, it could be `$file1`, or another file variable established by Format Control or custom RAD processing.
name	character	No	Defaults to the name of the currently open form. You can override this default by passing a string of the specific Link record you want to use.
prompt	character	No	Defaults to the name of the input field that the cursor is in when the you select the option. You can override this default by passing a string of a specific field name you want to use.

2.4.4.27. RAD routine: publish

A RAD routine used to publish a variable into SYSPUB.

Routine

publish

Parameters

The following parameters are valid for the **publish** routine:

Name	Data type	Required	Description
name	character	No	The published variable used by the agent to search the database.
text	character	No	The variable that identifies the message to be published.

2.4.4.28. RAD routine: query.stored

A RAD routine used to access all queries for a specific table or to execute a specific stored query.

Note: This routine is called only from Additional Options in Format Control.

Routine

query.stored

Parameters

The following parameters are valid for the **query.stored** routine:

Name	Data type	Required	Description
name	character	No	The name of the table you want to enable the user to query, for example, `device.workstation`.
text	character	No	The name of the stored query you want to execute, for example, `name=operator()`.

2.4.4.29. RAD routine: scauto.inventory

A RAD routine that provides access to Configuration items (CIs).

Routine

scauto.inventory

Parameters

The following parameters are valid for the **scauto.inventory** routine:

Name	Data type	Required	Description
record	record	Yes	Writes the unique record identifier to the eventin record. If you use a variable such as `$axces`, set it up using Format Control to create the intended result.
prompt	character	No	Indicates the Map name in event registration.
string1	character	No	Identifies the Service Manager table name to write to.
text	character	No	Provides the specific action to take, such as add, update, or delete.
query	character	No	Indicates the query to use to find the pre-existing record that this Input event updates.
boolean1	Boolean	No	The logical parameter flag, `true` or `false`, that indicates whether to write the eventout record on transaction completion.
name	character	No	Specifies the event registration name.

2.4.4.30. RAD routine: sort.array

A RAD routine called from subroutines to sort simple arrays, such as number, character, or date and time, in either ascending or descending order.

Routine

sort.array

Parameters

The following parameters are valid for the **sort.array** routine:

Name	Data type	Required	Description
file	record	Yes	When calling **sort.array** from Format Control, always pass `$file` to this parameter.
name	character	Yes	The name of the input field (array) in `$file` you want to sort.
boolean1	Boolean	No	Controls the sort order. A value of `true` sorts in ascending order, and a value of `false` sorts in descending order. The value is expressed as `val()`, for example, `val("false", 4)`.

2.4.4.31. RAD routine: us.js.call

A RAD routine that can invoke any JavaScript function in the ScriptLibrary. This routine enables you to run a JavaScript function without connecting to the Service Manager server via a client.

Routine

us.js.call

Parameters

The following parameters are valid for the **us.js.call** routine:

Parameter	Data type	Required	Description
script.function	string	Yes	Refers to a script in the ScriptLibrary and a particular function within the script.
arg,1	array (characters)	The required arguments depend upon the requirements of the script you call.	The arguments passed to JavaScript. You can pass an infinite number of arguments. If an argument need be enclosed in double quotes in JavaScript, use `\\\"` to enclose the argument when you use this routine. For example, if a JavaScript function has arguments: `{"abc", "def"}`, specify these arguments as `{\\\"abc\\\", \\\"def\\\"}` in this parameter.
arg,2		No	This parameter is for the returned value of the JavaScript function. When you use this routine in the operating system's command prompt, set this parameter to `NULL`.

2.4.4.32. RAD routine: validate.fields

A RAD routine used to validate fields in a form.

Routine

validate.fields

Parameters

The following parameters are valid for the **validate.fields** routine:

Name	Data type	Required	Description
name	character	No	Names the field in the current form to validate, such as logical.name. To name the field in which the cursor was last located when an add or update was performed, use the **cursor.field.name()** RAD function .
names	array (of characters)	No	Name of the array in the current form containing the field names to validate. The line of the array to validate must be defined in the Initializations process.
second.file	record	Yes	The data record in which the field is being validated. The value of this parameter must be `$file` .
cond.input	Boolean	No	The required value of the field defined by the *val()* parameter . For example, to validate a Boolean field to `true` , use the value `val("true", 4)` . When set to true, the value can be looked up online. This means a list of valid values is presented for selection.

2.4.4.33. RAD routine: wmi.inventory.check

A RAD routine that checks the eventin record for status information from a type of scan. This routine also processes the information from the scan into the Configuration Management application using the **scauto.inventory** RAD routine.

Routine

wmi.inventory.check

Parameters

The following parameters are valid for the **wmi.inventory.check** routine:

Name	Data type	Required	Description
record	record	Yes	Identifies the eventin record to pass.
prompt	character	No	Indicates the Map name used in event registration.
string1	character	No	Identifies the Service Manager table where data is recorded.
text	character	No	Provides the specific action to take, such as add, update, or delete.
query	character	No	Indicates the query to use to find the pre-existing record that this Input event updates.
boolean1	Boolean	No	A logical parameter flag, `true` or `false`, that indicates whether the eventout record writes on transaction completion.
name	character	No	Identifies the event registration name.
cond.input	Boolean	No	A logical parameter, `true` or `false`, that specifies whether or not to process additional records if you select more than one record.

2.5. JavaScript and Service Manager reference

This chapter provide the reference details for the JavaScript global System objects, JavaScript global methods, Service Manager defined JavaScript objects, JavaScript functions available in Service Manager, JavaScript functions available in the ScriptLibrary of Service Manager, and some JavaScript examples.

2.5.1. JavaScript global System objects

Global objects are predefined objects that are always instantiated and can't be created using the “new” operator. Objects have methods and/or subordinate properties that you refer to by using the dot operator.

The primary global object is the **System** object. It acts as a container for an entire hierarchy of objects. These other objects are properties of the System object.



Note

Note None of the Service Manager global System objects have constructors.

This is a complete list of global objects exposed as properties of the System object.

Global System objects	Description
system.files	Allows to call a particular Service Manager table into memory.
system.forms	Allows to call a particular Service Manager form into memory.
system.functions	Calls a particular Service Manager RAD function from JavaScript.
system.library	Calls a particular Service Manager script library function.
system.oldrecord	Calls the last Service Manager record into memory.
system.record	Calls the current Service Manager record into memory.
system.sysinfo	Calls an XML list of properties about the Service Manager server.
system.threads	Calls an XML list of the current Service Manager threads into memory.
system.user	Calls an XML list of properties of the currently logged on Service Manager user.
system.users	Calls an XML list of the currently logged on Service Manager users into memory.
system.vars	Calls a particular Service Manager variable into memory.



Note

Note All of the alias or shortcut global objects available in ServiceCenter 6.0 are deprecated in Service Manager. They are deprecated because the alias or shortcut is likely to be used as a script variable. The scripts that attempt to use these names as variables will not function as expected. Therefore, you must type out the entire global system object name. For example, type system.user instead of user.

2.5.1.1. JavaScript object: system.files

The following JavaScript object is unique to Service Manager.

The **system.files** object allows you to call a particular Service Manager table into memory.

Usage

```
system.files.table name
```



Note

Note Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

The following properties are valid for this object:

Property	Required	Description
table name	Yes	This argument contains the table name you want to bring into memory.

Methods

None

Example

This example does the following:

- Sets a variable equal to system.files.contacts
- Displays the contents of the contacts table

This example requires the following sample data:

- A valid Service Manager table name (for example, contacts)

```
var f = system.files.contacts;
print( "The value of system.files.contacts is:\n" + f );
```

2.5.1.2. JavaScript object: system.forms

The following JavaScript object is unique to Service Manager.

The system.forms object allows you to call a particular Service Manager form into memory.

Usage

```
system.forms.form name
```



Note

Note Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

The following properties are valid for this object:

Property	Required	Description
form name	Yes	This argument contains the form name you want to bring into memory.

Methods

None

Example

This example does the following:

- Sets a variable equal to system.forms.ScriptLibrary.
- Displays the contents of the ScriptLibrary form.

This example requires the following sample data:

- A valid Service Manager form name (for example, ScriptLibrary)

```
var fm = system.forms.ScriptLibrary;
print( "The value of system.forms.ScriptLibrary is:\n" + fm );
```

2.5.1.3. JavaScript object: system.functions

This JavaScript object is unique to Service Manager. The **system.functions** object allows you to call a particular Service Manager RAD function from JavaScript.

Usage

```
system.functions.RAD function name( RAD function arguments )
```

Note

Note There are two exceptions to syntax used for RAD functions: `delete` and `null`.

- `system.functions._delete( RAD function arguments )`
- `system.functions._null( RAD function arguments )`

Note

Note Service Manager global system objects don't have constructors.

Arguments

The following arguments are valid for this object:

Argument	Data type	Required	Description
RAD function arguments	String	Yes	This argument contains any arguments required to execute the RAD function.

Properties

The following properties are valid for this object:

Property	Required	Description
RAD function name	Yes	This argument contains the RAD function name you want to bring into memory.

Methods

None

Example

This example does the following:

- Sets a variable equal to **system.functions.operator()**.
- Displays the results of the RAD operator() function.

This example requires the following sample data:

- A valid Service Manager RAD function name (for example, operator())

```
var operator = system.functions.operator();
print( "The value of system.functions.operator() is:\n" + operator );
```

2.5.1.4. JavaScript object: system.functions.scmsg

The following JavaScript object is unique to Service Manager.

The **system.functions.scmsg** object allows you to call a particular Service Manager message record from JavaScript.

Usage

```
system.functions.scmsg( message number, message class, message elements )
```

Note

Note Service Manager global system objects don't have constructors.

Arguments

The following arguments are valid for this object:

Argument	Data type	Required	Description
message number	String	Yes	This argument contains the Service Manager message number you want to call into memory.
message class	String	Yes	This argument contains the Service Manager message class you want to call into memory.
message elements	Array	No	This argument contains the variable message elements indicated by %S in the message record.

Properties

None

Methods

None

Return values

A string containing the complete message text indicated by the arguments.

Example

This example does the following:

- Sets variables to define the message record
- Calls the RAD function scmsg()
- Displays the results of the RAD function scmsg()

This example requires the following sample data:

- A valid Service Manager message record (for example, "km" class record 14 "Group %S and Profile %S are already in category %S")

```
var msgrecord = "14";
var msgclass = "km";
var arguments = new Array();
arguments.push( "New Group" );
arguments.push( "New Profile Name" );
arguments.push( "Category Name" );
print( system.functions.scmsg( msgrecord, msgclass, arguments ) );
```

2.5.1.5. JavaScript object: system.library

The following JavaScript object is unique to Service Manager.

The **system.library** object allows you to call a particular Service Manager script library function.

Usage

```
system.library.script name.function name( function arguments )
```



Note

Note Service Manager global system objects don't have constructors.

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
`function arguments`	String	Yes	This argument contains the function arguments you want to use.

Properties

The following properties are valid for this object:

Property	Required	Description
`script name`	Yes	This argument contains the script name you want to run.
`function name`	No	This argument contains the function name you want to run.

Methods

None

Example

This example does the following:

- Creates an array.
- Calls a script to check if a value is in the array.

```
var testArray = new Array();
testArray.push("Blue");
testArray.push("Pink");
var result = system.library.ArrayUtil.contains(testArray, "Blue");
print("The value you specified is in the array:  " + result);
```

2.5.1.6. JavaScript object: system.oldrecord

The following JavaScript object is unique to Service Manager.

The **system.oldrecord** object allows you to call the last Service Manager record into memory.

Usage

```
system.oldrecord
```



Note

Note

Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

There are no properties for this object.

Methods

None

Example

This example does the following:

- Sets a variable equal to `system.oldrecord`.
- Displays the results of `system.oldrecord`.

```
var o = system.oldrecord;
print( "The value of system.record is:\n" + o );
```

2.5.1.7. JavaScript object: system.record

The following JavaScript object is unique to Service Manager.

The **system.record** object allows you to call the current Service Manager record into memory.

Usage

```
system.record
```



Note

Note

Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

There are no properties for this object.

Methods

None

Example

This example does the following:

- Sets a variable equal to `system.record`.
- Displays the results of `system.record`.

```
r = system.record;
print( "The value of system.record is:\n" + r );
```

2.5.1.8. JavaScript object: system.sysinfo

The following JavaScript object is unique to Service Manager.

The **system.sysinfo** object allows you to call an XML list of properties about the Service Manager server.

Usage

```
system.sysinfo
```



Note

Note

Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

There are no properties for this object.

Methods

None

Example

This example does the following:

- Sets a variable equal to `system.sysinfo`.
- Displays the results of `system.sysinfo`.

```
var s = system.sysinfo;
print( "The value of system.threads is:\n" + s );
```

2.5.1.9. JavaScript object: system.threads

The following JavaScript object is unique to Service Manager.

The **system.threads** object allows you to call an XML list of the current Service Manager threads into memory.

Usage

```
system.threads
```

> **Note**
> Note Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

There are no properties for this object.

Methods

None

Example

This example does the following:

- Sets a variable equal to `system.threads`.
- Displays the results of `system.threads`.

```
var th = system.threads;
print( "The value of system.threads is:\n" + th );
```

2.5.1.10. JavaScript object: system.user

The following JavaScript object is unique to Service Manager.

The **system.user** object allows you to call an XML list of properties of the currently logged on Service Manager user.

Usage

```
system.user
```

> **Note**
> Note Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

There are no properties for this object.

Methods

None

Example

This example does the following:

- Sets a variable equal to `system.user`.
- Displays the results of `system.user`.

```
var u = system.user;
print( "The value of system.user is:\n" + u );
```

2.5.1.11. JavaScript object: system.users

The following JavaScript object is unique to Service Manager.

The **system.users** object allows you to call an XML list of the currently logged on Service Manager users into memory.

Usage

```
system.users
```

> **Note**
> Note Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

There are no properties for this object.

Methods

None

Example

This example does the following:

- Sets a variable equal to `system.users`.
- Displays the results of `system.users`.

```
u = system.users;
print( "The value of system.users is:\n" + u );
```

2.5.1.12. JavaScript object: system.vars

The **system.vars** object is unique to Service Manager.

The **system.vars** object allows you to call a particular Service Manager variable into memory.

Usage

```
system.vars.variable name
```

> **Note**
> Note Service Manager global system objects don't have constructors.

Arguments

There are no arguments for this object.

Properties

The following properties are valid for this object:

Property	Required	Description
`variable name`	Yes	This argument contains the variable name you want to bring into memory.

Methods

None

Example

This example does the following:

- Sets a variable equal to `system.vars.$L_file`.
- Displays the contents of the `$L_file` variable.

This example requires the following sample data:

- A valid Service Manager variable name (for example, `$L_file`)

```
var v = system.vars.$L_file;
print( "The value of system.vars.$L_file is:\n" + v );
```

2.5.2. List: JavaScript global methods

Global methods are functions that are available to any script as they're not methods of any specific object. You can invoke global methods directly just as you would do with any core JavaScript global functions such as **parseInt()** or **eval()**.

This is a complete list of all the global methods available in Service Manager.

Global method	Description
base64Decode	Converts base 64 string data to its original format.
base64Encode	Converts binary data to a base 64 string format.
compile	Validates the syntax of the specified JavaScript.
doHTTPRequest	Issues an HTTP request to a specified URL.
doSOAPRequest	Issues an SOAP request to a specified URL.
execute	This method performs the specified process or program.
getLog	This method retrieves a logger named according to the value of the name parameter.
help	Displays a brief description of a Service Manager-defined JavaScript object.
insertDatum	Inserts data into an array datum directly.
makeSCWebURL	Creates a URL query to the Service Manager Web tier.
print	Displays a message in the client Messages view.
RCtoString	Converts a Service Manager global return code value into a localized text string.
readFile	Reads data from the local file system.
rteJSONStringify	Converts a value to JSON.
rteJSONParse	Parses a string as JSON.
setAppMessage	Defines the message returned in the "message" attribute in the soap response.
stripHtml	Takes HTML content and strips out the HTML tags and returns the content as text without the HTML tags.
uncompressFile	Expands a .zip file into a specified location.
writeAttachmentToFile	Writes a requested attachment record to the local file system.
writeFile	Writes data to the local file system.
xmlstring	Converts a JavaScript string to an XML string.

2.5.2.1. JavaScript global method: base64Decode

Converts base 64 string data to its original format.

> **Note**
> Note
> If the value of the `data` parameter defined in `base64Decode()` is not a valid base64 string, the `base64Decode()` method will throw an exception with the "Exception occurred when calling JSBase64Decode" error message. See JavaScript tailoring guideline for details.

Syntax

```
base64Decode( data );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
`data`	Binary	Yes	This argument contains the data you want the script to decode to its original format.

Return values

A binary or string object containing the original data.

Description

This method converts data in base 64 string format to its original format.

Example

This example does the following:

- Reads the contents of a binary file
- Converts the file to a base 64 string format
- Writes the base 64 string data to a local file
- Prints the number of bytes written to the local file system
- Reads the contents of a base 64 string format file
- Converts the data into a binary file
- Writes the binary data to a local file
- Prints the number of bytes written to the local file system

This example requires the following sample data:

- A binary file on the local file system (for example, an image file)

```
var source = readFile( "C:\\header_left.gif", "b" );
var encode = base64Encode( source );
var encodedFile = writeFile( "C:\\base64EncodedImage.txt", "b", encode );
print( "Wrote " + encodedFile + " bytes to C:\\base64EncodedImage.txt" );
source = readFile( "C:\\base64EncodedImage.txt", "t" );
var decode = base64Decode( source );
var decodedFile = writeFile( "C:\\newImage.gif", "b", decode );
print( "Wrote " + decodedFile + " bytes to C:\\newImage.gif" );
```

2.5.2.2. JavaScript global method: base64Encode

Converts binary data to a base 64 string format.

Syntax

```
base64Encode( data );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
data	Binary	Yes	This argument contains the data you want the script to encode in base 64 format.

Return values

A string object containing the data in base 64 format.

Description

This method converts binary data to a base 64 string format. You can also use this method to encode plain text data in base 64 format.

Example

This example does the following:

- Reads the contents of a binary file
- Converts the file to a base 64 string format
- Prints the base 64 string

This example requires the following sample data:

- A binary file on the local file system (for example, an image file)

```
var source = readFile( "C:\\header_left.gif", "b" );
var encode = base64Encode( source );
print( "The value of encode is:\n" + encode );
```

2.5.2.3. JavaScript global method: compile

Validates the syntax of the specified JavaScript.

Syntax

```
compile( script, description );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
script	String	Yes	This argument contains the name of the script you want to validate.
description	String	No	Use this argument to store any comments you want.

Return values

RC_SUCCESS if there is no compilation error or -1 if any compilation error.

This method also returns a success or error message. It displays the following message if the JavaScript passes validation:

`Successful compilation of JavaScript function or expression.`

Throws

None

Description

This method validates the syntax of a JavaScript and displays a message as to the success or failure of the validation. This method is an alias of the **execute()** method.

Example

This example does the following:

- Creates a variable to store a JavaScript name
- Validates the contents of the script

This example requires the following sample data:

- A valid JavaScript

```
var s = "lib.SCFILEgetTextTest"
compile( s, "Testing execute" );
```

2.5.2.4. JavaScript global method: doHTTPRequest

Issues an HTTP request to a specified URL.

Syntax

doHTTPRequest(HTTP command, URL, Headers, POST body, Connect Timeout, Read Timeout, Obsolete, Response headers, Binary request data, Binary response data, Follow redirects, HTTP proxy host, HTTP proxy port, HTTP proxy authentication user, HTTP proxy authentication password, Client Keystore Name);

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
HTTP command	String	Yes	This argument specifies a command verb. We support GET, POST, PUT, HEAD, PATCH and DELETE.
URL	String	Yes	This argument specifies the URL to which the script should send an HTTP request. If the HTTP request requires SSL then the URL must start with https.
Headers	Array	Yes	This argument contains a JavaScript array of Service Manager-defined Header objects or an empty array. See JavaScript object: Header for more information.
POST body	String	No	This argument contains the body of a POST command request.
Connect Timeout	Number	No	This argument specifies the number of seconds an HTTP request has to establish a connection before it times out. **Note** The default value is 120 seconds, and the maximum value is 600 seconds. That is, if you try to set the value to a number larger than 600 seconds, the actual timeout period will be 120 seconds. In addition, setting the value to 0 also makes the actual timeout period 120 seconds.

Read Timeout	Number	No	This argument specifies the number of seconds an HTTP request has to read data before it times out. **Note** The default value is 120 seconds, and the maximum value is 600 seconds. That is, if you try to set the value to a number larger than 600 seconds, the actual timeout period will be 120 seconds. In addition, setting the value to 0 also makes the actual timeout period 120 seconds.
Receive timeout (Obsolete)	Number	No	This argument is obsolete.
Response headers	Object	No	This argument receives the headers of an HTTP response.
Binary request data	Boolean	No	This argument specifies if the content of a post body is binary data.
Binary response data	Boolean	No	This argument specifies if the content of an HTTP response is binary data.
Follow redirects	Boolean	No	This argument specifies if the HTTP client automatically follows the HTTP redirects (requests with the response code 3xx). The default value is true.
Http Proxy Host	String	No	This argument specifies the HTTP proxy host.
Http Proxy Port	Number	No	This argument specifies the HTTP proxy port.
Http Proxy Authentication user	String	No	This argument specifies the authentication user of the HTTP proxy.
Http Proxy Authentication password	String	No	This argument specifies the authentication password of the HTTP proxy.

Client Keystore Name	String	No	This argument specifies the client keystore for external Web Services. When you execute `doHTTPRequest` to send an HTTPS request to the Web Services, if you omit this parameter, the keystore of Service Manager server will be used. If you want the Service Manager server to act as a client while the external Web Services acts as a server, you need to follow these steps: 1. Get the following information from the Web Services server: ○ The client keystore and the client keystore password ○ The root certificate or the certificate authority's (CA) certificate **Note:** The client keystore should only contain one keystore and its associated certificate (chain) 2. Convert the format of the client keystore according to your SM setup. ○ For FIPS mode: Covert the keystore to BCFKS format ○ For non-FIPS mode: Covert the keystore to PKCS12 format 3. Update the client keystore password with the value of the keystorePass parameter in sm.ini. For example: `> keytool -keystore clientkeystore.p12 -alias clientkey -keypasswd` 4. Copy the client keystore to the <SM Server intall Path>\RUN directory.

			5. Import the root certificate to Service Manager server's truststore.
			For example:
			`> keytool -import -keystore <SM Server intall Path>\RUN\ <truststoreFile> -trustcacerts -file clientcert.crt -alias clientkey`
			6. Set the following parameter values in the **sm.ini** file:
			○ truststoreFile
			○ truststorePass
			○ keystorePass
			7. For horizontal scaled (HS) environments, repeat step #4, #5 and #6 for each host.
			8. Use the doHTTPRequest JavaScript method in Service Manager to send an HTTP request to the Web services API.
			For example:
			`doHTTPRequest("GET", url, headers, null, 10, 10, 10, null,false, false, true, null,'', '','', 'clientkeystore.p12');`

Return values

A string containing an HTTP response or an error message.

Description

This method sends an HTTP request to a remote server.

Note

If the charset is undefined in the `Content-Type` header in the `Headers` parameter, the charset for the message body will be UTF-8 by default.

Example

This example does the following:

- Sends an HTTP request to the Service Manager Web services API
- If successful, writes the response to a local file
- If unsuccessful, displays the HTTP error message

Note

To see a working example of the doHTTPRequest method that incorporates multiple error-checking conditions, open the SOAP script in the script library. The WSDL2js function uses the doHTTPRequest method at line 114 as part of a try/catch statement.

This example requires the following sample data:

- The URL to the Service Manager Web services API

```
// Use the following url value to produce an HTTP 400 error.
//var url = "http://localhost:13080/IncidentManagement.wsdl";

// Use the following url value to produce valid WSDL.
var url = "https://fqdn:13443/SM/7/IncidentManagement.wsdl";

var headers = new Array();
var WSDLrequest;
var reply;
var respHeaders = new Object();

try
      {
      // request without proxy
      WSDLrequest = doHTTPRequest( "GET", url, headers, null, 10, 10, null, respHeaders );
      // request with proxy which no need authentication
      // WSDLrequest = doHTTPRequest( "GET", url, headers, null, 10, 10, null, respHeaders, "web-
proxy.us.softwaregrp.net", 8080);
      // request with proxy which need authentication
      // WSDLrequest = doHTTPRequest( "GET", url, headers, null, 10, 10, null, respHeaders,
"smcfrdrte305.hpeswlab.net", 3128, "smdev", "!QAZ2wsx");
      // request specified with the client keystore for external Web Services
      // WSDLrequest = doHTTPRequest("GET", url, headers, null, 10, 10, 10, null, false, false, true, null,'',
'','', 'clientkeystore.p12');
      print( "The response headers of the doHTTPRequest is:" );
      for( var name in respHeaders )
      {
          print( name + "=" + respHeaders[name] );
      }
      print( "The result of the doHTTPRequest is \n" + WSDLrequest );
      reply = writeFile( "C:\\IncidentManagement.wsdl", "text", WSDLrequest
);
      }
catch ( e )
      {
      print( "WSDL request failed with exception \n" + e );
      }
```

2.5.2.5. JavaScript global method: doSOAPRequest

Issues a SOAP request to a specified URL.

Syntax

`doSOAPRequest( URL, SOAPAction, XML, SOAP user ID, SOAP password, Connect Timeout, Read Timeout, Obsolete, Attachment object );`

Parameters

The following arguments are valid for this function:

Argument	Data type	Required	Description
URL	String	Yes	This argument specifies the URL to which the script should send an HTTP request. If the HTTP request requires SSL then the URL must start with https.
SOAPAction	String	Yes	This argument specifies the SOAPAction value.
XML	String	Yes	This argument contains the XML for an entire SOAP request including the SOAP envelope. Refer to the Web Service's WSDL to determine the proper format of the SOAP request.
SOAP user ID	String	No	This argument contains the user ID to be used for the Basic Authorization HTTP header.
SOAP password	String	No	This argument contains the password value to be used for the Basic Authorization HTTP header.
Connect Timeout	Number	No	This argument specifies the number of seconds a SOAP request has to establish a connection before it times out.
Read Timeout	Number	No	This argument specifies the number of seconds a SOAP request has to read data before it times out.
Receive timeout (Obsolete)	Number	No	This argument is obsolete.
Attachment object	Array	No	This argument contains a JavaScript array of Service Manager-defined Attachment objects or an empty array. See JavaScript object: Attachment for more information.
AcceptFastInfoset	Number	No	To set the SOAP Message flag "acceptfastinfoset". Default value is 1. To turn "acceptfastinfoset" off: 0.

Return values

A string containing a SOAP response or an error message.

Description

This method sends a SOAP request to a remote server and returns the SOAP response.

Examples

Example 1 :

- Sends a SOAP request to a delayed stock quote Web service and indicates SM accept the "fastinfoset" formatted SOAP message
- Prints the SOAP response to the screen

This example requires the following sample data:

- The URL to a delayed stock quote Web service
- Stock symbol for company

```
var url = "http://ws.cdyne.com/delayedstockquote/delayedstockquote.asmx";
var action = "http://ws.cdyne.com/GetQuote";
var xml = "<soap:Envelope xmlns:soap=\"http://schemas.xmlsoap.org/soap/envelope/\"

   xmlns:xsi=\"http://www.w3.org/2001/XMLSchema-instance\"
   xmlns:xsd=\"http://www.w3.org/2001/XMLSchema\"
   xmlns:s0=\"http://ws.cdyne.com/\">
   <soap:Body>
             <GetQuote xmlns=\"http://ws.cdyne.com/\">
             <StockSymbol xsi:type=\"s:string\">GOOG</StockSymbol>
             <LicenseKey xsi:type=\"s:string\">0</LicenseKey>
             </GetQuote>

    </soap:Body></soap:Envelope>";
var uid = null;
var pass = null;
var attachments = new Array();
var quote = doSOAPRequest( url, action, xml, uid, pass, 60, 60, null, attachments );
print( "The return value is: " + quote );
```

Example 2:

- Sends a SOAP request to a delayed stock quote Web service and indicates SM doesn't accept the "fastinfoset" formatted SOAP message.
- Prints the SOAP response to the screen

This example requires the following sample data:

- The URL to a delayed stock quote Web service
- Stock symbol for company

```
var url = "http://ws.cdyne.com/delayedstockquote/delayedstockquote.asmx";
var action = "http://ws.cdyne.com/GetQuote";
var xml = "<soap:Envelope xmlns:soap=\"http://schemas.xmlsoap.org/soap/envelope/\"

   xmlns:xsi=\"http://www.w3.org/2001/XMLSchema-instance\"
   xmlns:xsd=\"http://www.w3.org/2001/XMLSchema\"
   xmlns:s0=\"http://ws.cdyne.com/\">
   <soap:Body>
          <GetQuote xmlns=\"http://ws.cdyne.com/\">
          <StockSymbol xsi:type=\"s:string\">GOOG</StockSymbol>
          <LicenseKey xsi:type=\"s:string\">0</LicenseKey>
          </GetQuote>

    </soap:Body></soap:Envelope>";
var uid = null;
var pass = null;
var attachments = new Array();
var quote = doSOAPRequest( url, action, xml, uid, pass, 60, 60, null, attachments, 0 );
print( "The return value is: " + quote );
```

2.5.2.6. JavaScript global method: execute

Validates the syntax of the specified JavaScript, and then runs it.

Syntax

```
execute( script, description );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
`script`	String	Yes	This argument contains the name of the script you want to validate.
`description`	String	No	Use this argument to store any comments you want.

Return values

A success or error message.

This method displays the `Successful compilation of JavaScript function or expression` message if the JavaScript passes validation.

Throws

None

Description

This method validates the syntax of a JavaScript, and then runs it. The method also displays a message as to the success or failure of the validation.

Example

This example does the following:

- Creates a variable to store a JavaScript name
- Validates the contents of the script
- Runs the script

This example requires the following sample data:

- A valid JavaScript

```
var s = "lib.SCFILEgetTextTest"
execute( s, "Testing execute" );
```

2.5.2.7. JavaScript global method: getLog

This method retrieves a logger named according to the value of the name parameter.

Syntax

`getLog (name)`

Arguments

The following argument is valid for this function.

Argument	Data type	Required	Description
`name`	String	Yes	The name of the logger to retrieve

Return values

A logger named according to the value of the name parameter.

Description

This method retrieves a logger named according to the value of the name parameter. Loggers are normally named entities, using dot-separated names such as "lib.test01.func01." The logger object in JavaScript is an adapter of log4j in the Service Manager server. Logger names and levels follow the rules of log4j.

Example

This example does the following:

1. Retrieve a logger with the given name.
2. Set the logger level to **trace**.
3. Print different level messages.
4. Set the log levels by the code.

```
var log = getLog('lib.test01.func01');
log.setLevel('trace');

if (log.isTraceEnabled()){
log.trace('trace message');
}

if (log.isDebugEnabled()){
log.debug('debug message');
}

if (log.isInfoEnabled()){
log.info('informatin message');
}

if (log.isWarnEnabled()){
log.warn('warning message');
}

if (log.isErrorEnabled()){
log.error('error message');
}

if (log.isFatalEnabled()){
log.fatal('fatal message');
}
// Set level by code
log.setLevel("trace");
log.setLevel("debug");
log.setLevel("info");
log.setLevel("warn");
log.setLevel("error");
log.setLevel("fatal");

log.setLevel(0); //trace
log.setLevel(1); //debug
log.setLevel(2); // info
log.setLevel(3); // warn
log.setLevel(4); // error
log.setLevel(5); // fatal
```

To set a Javascript logger level, follow these steps:

1. Go to **Tailoring** > **Script Library**.
2. Click **More** > **Javascript Logger**.
3. In the **Name** field, type the name of the logger.
4. In the **Level** field, select a value from the following values: **Trace**, **Debug**, **Info**, **Warn, Error**, and **Fatal**.
5. Click **Save**.



Note

Note The logger level will be cleared after the Service Manager server is restarted.

2.5.2.8. JavaScript global method: help

Displays a brief description of a Service Manager-defined JavaScript object.

Syntax

```
help( object );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
object	Service Manager-defined JavaScript object	Yes	This argument contains the Service Manager-defined JavaScript object for which you want a brief description.

Return values

A string or null.

This method returns a brief text string describing the Service Manager-defined JavaScript object or null if there is no help available.

Throws

None

Description

This method displays a brief description of a Service Manager-defined JavaScript object. You must use the print() method to see the help text string.

Example

This example displays the help contents of several Service Manager-defined JavaScript objects.

```
var f = new SCFile();
print( help( f ) );
var x = new XML();
print( help( x ) );
var d = new XMLDate();
print( help( d ) );
```

2.5.2.9. JavaScript global method: insertDatum

Inserts data into an array datum directly.

Syntax

`insertDatum( array[, position[, number[, value[, denull]]]])`

Arguments

The following parameters are valid for this function.

Parameter	Data type	Description
array	Array datum	The *array* parameter is the array into which an element is inserted.
position	Number	The *position* parameter is the position of the first inserted element. If the value is zero (0), the element is inserted at the end of the array. This is an optional parameter. The default value is zero (0).
number	Number	The *number* parameter is the number of elements to insert. This is an optional parameter. The default value is one (1). If the value is zero (0), this function will return without changing the array.
value	Any data type supported by RAD	The *value* parameter is a value to insert into the new elements. This is an optional parameter. The default value is NULL.
denull	Boolean	The *denull* parameter is a logical value that determines whether or not the array should be denulled before inserting elements. This is an optional parameter. The default value is true.

Return values

The array is returned with the new data inserted.

Throws

None

Description

This method insert data directly into an existing array datum.

Examples

Action desired	Example and result
Insert an element at the beginning of an array datum	`vars.$a=["a", "b", "c", "d"]; insertDatum(vars.$a, 1, 1, "z");` Returns: `vars.$a = {"z", "a", "b", "c", "d"}`
Insert an element in the middle of an array datum	`vars.$a=["a", "b", "c", "d"]; insertDatum(vars.$a, 3, 1, "z")` Returns: `vars.$a = {"a", "b", "z", "c", "d"}`
Insert an element at the end of an array datum	`vars.$a=["a", "b", "c", "d"]; insertDatum(vars.$a, 0, 1, "z");` Returns: `vars.$a = {"a", "b", "c", "d", "z"}`
To denull the array datum	`vars.$a=["a", "b", null, null, "e", "f", null, null, null, null]; insertDatum(vars.$a, 0, 1, "z");` Returns: `vars.$a = {"a", "b", , , "e", "f", "z"}`
To avoid denulling the array datum	`vars.$a=["a", "b", null, null, "e", "f", null, null, null, null]; insertDatum(vars.$a, 0, 1, "z", 0);` Returns: `vars.$a = {"a", "b", , , "e", "f", , , , , "z"}`
To denull the array datum again	vars.$a=["a", "b", null, null, "e", "f", null, null, null, null]; insertDatum(vars.$a, 2, 1, "z"); Returns: `vars.$a = {"a", "z", "b", , , "e", "f"}`

2.5.2.10. JavaScript global method: makeSCWebURL

Creates a URL query to the Service Manager Web tier.

Syntax

`makeSCWebURL( Web tier URL, docEngine, table name, query, hash key seed, action);`

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
Web tier URL	String	Yes	This argument specifies the fully qualified URL to the Service Manager Web tier. This URL must include the http:// protocol syntax as well as the server name or IP address and communications port number.
docEngine	String	Yes	This argument specifies that the query should be handled by the Service Manager Document Engine. This argument doesn't accept any other string value.
table name	String	Yes	This argument specifies the table where the Document Engine should query for records.
query	String	Yes	This argument specifies the Service Manager query you want to use to search for records. You must URI encode the query string to prevent special characters from invalidating the URL.
hash key seed	String	No	This argument specifies an optional string you want to use to create a unique hash key. The hash key seed doesn't appear in the final URL produced.
action	String	Not applicable	This argument can't be used. By default, the URL query always performs a search operation.

Return values

A string containing a URL query to the Service Manager web tier.

Description

This method creates a URL query to the Service Manager web tier. It doesn't convert any special characters in the URL (such as spaces or quotation marks) to the character code equivalent in URI format. You can use the **encodeURIComponent** method to convert any special characters in the URL.

Example

This example does the following:

- Creates a URL query to the Service Manager Web tier
- Converts any special characters in the URL to valid URI format

This example requires the following sample data:

- The URL to Service Manager Web tier
- A valid Service Manager table name (for example, incidents)
- A valid Service Manager query against the table (for example, incident.id="SD1001")

```
function createURLquery( table, query)
{

 var url;
 var webtier;
 var webserver;
 var doceng;
 var hashkey;
 var action;

 print( "Parameters: table="
        + table +
        "query='"
        + query + "'");
 print( "Converting special characters in query to valid URI format..." );
 queryURI = encodeURIComponent( query );
 print( "Converted: old query='"
        + query +
        "' new query='"
        + queryURI +
        "'" );

 webserver = "http://pdoref02/sc/index.do";
 doceng = "docEngine";
 hashkey = "";
 action = "";
 webtier = makeSCWebURL( webserver,
           doceng,
           table,
           queryURI,
           hashkey,
           action, "abc");
 url=encodeURIComponent(webtier);
 print( "Creating URL from information provided..." );
 print( "The value of webtier is:\n" + webtier );
 print( "The value of url is:\n" + url );
 return url;
}

var tablename = "incidents"
var SCquery = "incident.id=\"SD1001\""

createURLquery( tablename, SCquery);
```

2.5.2.11. JavaScript global method: print

Displays a message in the client Messages view and prints it in the Service Manager log file.



Note

Note Messages only print to the Service Manager log file when JavaScript is running in the background.

Syntax

```
print( output );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
output	String	Yes	This argument contains the text string or variable value you want the script to print to the Message View and the sm.log file. You must enclose literal text strings in quotation marks.

Return values

None

Throws

None

Description

This method displays the contents of the method argument in the Messages view and also prints it in the Service Manager log file.

Example

This example does the following:

- Creates a variable with a string value
- Displays the contents of the variable

This example requires the following sample data:

- A text string

```
var example = "abc123";
print("The value of example is: " + example);
```

2.5.2.12. JavaScript global method: RCtoString

Converts a Service Manager global return code value into a localized text string.

Syntax

```
RCtoString( return code );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
return code	Integer	Yes	This argument contains the Service Manager global return code value you want to convert to a localized string.

Return values

A localized text string

Throws

None

Description

This method converts a Service Manager global return code value into a localized text string.

Example

This example does the following:

- Searches the contacts table for any contact name you define in the search variable

- Displays the contact record as a text string

This example requires the following sample data:

- A valid contact name (for example, "ADMINISTRATOR, SYSTEM")
- A invalid contact name (for example, "NOT A, CONTACT")

```
var contactName;

function findContactName( name )
{
 print( "Searching for contact: " + name + "..." );
 var contactList = new SCFile( "contacts" );
 var findContact = contactList.doSelect( "contact.name=\""+ name + "\"" )
 if ( findContact == RC_SUCCESS )
 {
  print( "Success. found " + name + " in contact record:\n" + contactList.getText() );
  return contactList
 }
 else
 {
  print( "Could not find contact. " + RCtoString( findContact ) );
  return null
 }
}

contactName = "ADMINISTRATOR, SYSTEM";
findContactName( contactName );

contactName = "NOT A, CONTACT";
findContactName( contactName );
```

2.5.2.13. JavaScript global method: readFile

Reads data from the file system on the Service Manager server host.

Syntax

```
readFile( path, binary );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
path	String	Yes	This parameter contains the text string or variable value containing the fully qualified path of the file to be read. You must enclose literal text strings in quotation marks.
binary	String	Yes	This parameter determines if the file format is binary or text. Use the string "b" or "B" to read a binary file. Use any other value to read a text file.

Return values

A string object containing the contents of the file.

Description

This method reads the contents of a local file and returns a string object. The file can be in binary format or text format as determined by the binary argument.

Example

This example does the following:

- Reads the contents of a text file
- Reads the contents of a binary file

This example requires the following sample data:

- A text file on the local file system
- A binary file on the local file system

```
var textFile = readFile( "C:\\test.xml", "t" );
print( "The value of textFile is: " + textFile );

var binFile = readFile( "C:\\header_left.gif", "b" );
print( "The value of binFile is: " + binFile );
```

2.5.2.14. JavaScript global method: rteJSONParse

Parses a string as JSON.

Syntax

```
rteJSONParse(text);
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
text	Boolean, Number, or String	Yes	This argument contains the string to parse as JSON.

Return values

Returns the object corresponding to the given JSON text.

Throws

Throws an exception if the string to parse is not a valid JSON.

Description

This method parses a string as JSON.

Example

```
rteJSONParse('{}');              // {}

rteJSONParse('true');            // true

rteJSONParse('"foo"');           // "foo"

rteJSONParse('[1, 5, "false"]'); // [1, 5, "false"]

rteJSONParse('null');            // null
```

2.5.2.15. JavaScript global method: rteJSONStringify

Converts a value to JSON.

Syntax

```
rteJSONStringify(value);
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
value	Boolean, Number, or String	Yes	This argument contains the value to convert to a JSON string.

Return values

Returns the JSON string corresponding to the given object.

Throws

None

Description

This method converts a value to JSON notation that represents it.

- This method doesn't guarantee to convert the properties of non-array objects to strings in any particular order. Don't rely on the ordering of properties in the same object in the converted string.
- This method converts Boolean, Number, and String objects to the corresponding primitive values, in accord with the traditional conversion semantics.
- If undefined or this method encounters a function during conversion, this method either omits it (when finding it in an object) or censors it to null (when finding it in an array).

Example

```
rteJSONStringify({});                      // '{}'

rteJSONStringify(true);                    // 'true'

rteJSONStringify("foo");                   // '"foo"'

rteJSONStringify([1, "false", false]);     // '[1,"false",false]'

rteJSONStringify([1, undefined, function(){}, "false", false]); // '[1,null,null,"false",false]'

rteJSONStringify({ x: 5 });                // '{"x":5}'

rteJSONStringify({x: 5, y: 6});            // '{"x":5,"y":6}' or '{"y":6,"x":5}'

rteJSONStringify({a:undefined, b:function(){}, x: 5, y: 6}); // '{"x":5,"y":6}' or '{"y":6,"x":5}'

rteJSONStringify([new Number(1), new String("false"), new Boolean(false)]); // '[1,"false",false]'
```

2.5.2.16. JavaScript global method: setAppMessage

Defines the message returned in the "message" attribute in the soap response. For example, in the following soap response: `<CreateRelationshipResponse message="Here you define your message" returnCode=....>`, where the `message` attribute is shown in italic.

Syntax

```
setAppMessage(message);
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
message	String	Yes	This argument contains the text string you want to show in the "message" attribute in the soap response. You must enclose literal text strings in quotation marks.

Return values

None

Throws

None

Description

This method defines the message returned in the "message" attribute in the soap response.

Example

```
inputValue = "CompanyA";
setAppMessage("The input value "+inputValue+" is invalid");
```

2.5.2.17. JavaScript global method: stripHtml

Takes HTML content and strips out the HTML tags and returns the content as text without the HTML tags.

Syntax

```
stripHtml();
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
htmlstr	String	Yes	This argument contains the JavaScript String with HTML tags that you want to strip out of the string.

Returns

JavaScript String without HTML tags.

Throws

None

Description

Takes HTML content and strips out the HTML tags and returns the content as text without HTML tags.

Example

```
var htmlstr = "<html><body>Hello</body></html>";
var text = stripHtml(htmlstr);
print("before stripping HTML tags: " + htmlstr);
print("after stripping HTML tags: " + text);
```

2.5.2.18. JavaScript global method: uncompressFile

This method expands a .zip file into a specified location.

Syntax

```
uncompressFile(file name, target directory);
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
file name	String	Yes	This argument specifies the path and file name of the .zip file to uncompress.
target directory	String	No	This argument specifies the target directory. If not specified, the location of the .zip file is used.

Return values

True indicates that the file uncompressed successfully, and any other return value indicates failure.

Throws

None

Description

This method expands a .zip file into a specified location.

Example

This example unzips the upgtest.zip file to c:/test.

```
var rc = uncompressFile("c:/test/upgtest.zip");
if( rc == true )
{
  print("uncompress file with one file successfully");
  // this will unzip the file to C:/test/test
  rc = uncompressFile("c:/test/upgtest.zip", "c:/test/test");
}
else
{
print("Failed to uncompress file.");
}
if(rc == true)
{
print("All tasks are finished.");
}
```

2.5.2.19. JavaScript global method: writeAttachmentToFile

Writes a requested attachment record to the local file system.

Syntax

```
writeAttachmentToFile( File, Application, Topic, UID );
```

Arguments

The following arguments are valid for this method:

Name	Data type	Required	Description
File	String	Yes	This argument specifies the file name of the attachment you want to write out to the file system. You may also use this argument to specify the destination path of the attachment. If this argument doesn't specify a path, Service Manager creates the attachment in the system working directory.
Application	String	Yes	This argument specifies the application name portion of the key that Service Manager uses to identify the attachment in the SYSATTACHMENTS table.
Topic	String	Yes	This argument specifies the topic name portion of the key that Service Manager uses to identify the attachment in the SYSATTACHMENTS table.
UID	String	Yes	This argument specifies the unique identifier portion of the key that Service Manager uses to uniquely identify the attachment in the SYSATTACHMENTS table.

Return values

RC_SUCCESS or one of the other global return code values.

The method writes the attachment record specified in the method arguments to the local file system and returns a global return code value of RC_SUCCESS or returns one of the failure global return code values if the method can't return an attachment record.

Description

This method extracts the specified attachment from the SYSATTACHMENTS table and writes it to the specified file and path on the local file system.



Note

Note If you specify a path, you must escape out any JavaScript reserved-characters in the path name such as slashes and backslashes. Consult a JavaScript reference for a full list of JavaScript reserved-characters and escape methods.

Example

This example does the following:

- Requests an attachment from the SYSATTACHMENTS table
- Writes the attachment to the file c:\kmdocument.pdf
- Prints the return code to the screen

This example requires the following sample data:

- The Knowledge Management record KM0018
- An attachment in KM0018

```
var rc;
rc = writeAttachmentToFile( "c:\\kmdocument.pdf", "kmdocument", "KM0018", "4462590a062ee0c2101d85c8" );
if ( rc == RC_SUCCESS )
{
 print( "kmdocument.pdf successfully created" );
}
else
{
 print( "kmdocument.pdf failed " , RCtoString(rc) );
}
```

2.5.2.20. JavaScript global method: writeFile

Writes data to the local file system.

Syntax

```
writeFile( path, mode, object );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
path	String	Yes	This argument contains the text string or variable value containing the fully qualified path of the file to be written. You must enclose literal text strings in quotation marks.
mode	String	No	Use the string "b" to write a binary file. Use the string "ba" to append to a binary file. Use the string "a" to append to a text file. Use any other string or omit the parameter to write to a text file. This is the default. The values are case sensitive and only the values specified are valid.
object	String	Yes	This parameter contains the text string or variable value containing the data you want to write to a file.

Return values

A number representing the number of bytes written to a file.

Description

This method writes the contents of the object parameter to the file specified in the path parameter. The file can be in binary format or text format as determined by the binary parameter.

Example

This example does the following:

- Writes the list of currently logged on users to a file

This example requires the following sample data:

- The list of currently logged on users stored in system.users

```
var userList = system.users;
var filePath;
var isMode;
var fileObject;

function writeToFile( path, binary, object )
{
 print( "Writing " + path + " to file..." );
 var output = writeFile( path, mode, object );
 print( "The number of bytes written to file was: " + output );
 return output
}

filePath = "C:\\users.xml";
isMode = null;
fileObject = userList;
writeToFile( filePath, isMode, fileObject );
```

2.5.2.21. JavaScript global method: xmlstring

This method converts a JavaScript string to an XML-formatted string.

Syntax

```
xmlstring( string );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
`string`	String	No	This argument contains the text string or variable value you want the script to convert to an XML string.

Return values

None

Throws

None

Example

```
var xmldoc = new SCFile( system.users );
xmlstring( xmldoc );
print( xmldoc );
```

2.5.3. Service Manager defined JavaScript objects

The defined objects are unique to the implementation of JavaScript within Service Manager. This is a complete list of the Service Manager defined objects.

Defined object	Description
Attachment	This object allows you to create attachments for use with **doSOAPRequest** methods.
Datum	This object extracts aggregate types of data (structures or arrays) from a record in the file object. In Service Manager, a file record is a datum and so is each component of the record.
SCDatum	This object converts a Datum to a SCDatum so that you can test the results by using the return codes. When an object is initialized in RAD and then passed to JavaScript, the object becomes a Datum object and all the methods return true/false values. If an error occurs and you want a more specific error message, you can create a SCDatum object from the Datum object.
SCFile	This object constructs a new file object. You can use the SCFile object to access, update, and move between Service Manager files. This object exposes a small set of frequently used return code constants that make it easy to test values returned by methods without having to hard code return code values in the script.
SCRecordList	This object is an array of Datums that qualify with the query. SCRecordList has two available functions: getPosition() and getCount().
Query	This object returns an array of records. The Query object can't be used to write data back to Service Manager.
QueryCond	This object defines the conditions of the query. It is used in conjunction with the Query object to define the parameters for returning data from Service Manager.
XML	This object represents an XML document or a node in an XML document. Underlying the XML object is the Document Object Model (DOM) object. You may find that in certain instances the XML object in Service Manager is not compliant with the DOM binding for JavaScript.
XMLDate	This object makes it possible to do conversions between three kinds of datetime values. These include the following constructors: • **XMLDate(dateobject)**: passes a JavaScript Date object to the constructor. • **XMLDate(iso8601datetimeordurationstring)**: passes an XML schema date, time, datetime, or duration string to the constructor. • **XMLDate(datetimedatum)**: a Service Manager datum object of type TIME to the constructor.

Note

Note In ServiceCenter version 6.1, the File and SCDoc objects are deprecated. We strongly recommend that you don't use these objects since they may be removed in a future release.

2.5.3.1. JavaScript object: Attachment

The following JavaScript object is unique to Service Manager. This object allows you to create attachments for use with doSOAPRequest methods. You have to manually define the value property of each attachment object. For binary data attachments, you can use the readFile global method to assign the value property. If you need to create multiple attachments for a doSOAPRequest request, you can use the push method to add each Attachment object to a JavaScript array.

Constructor

```
new Attachment();
```

Arguments

None

Properties

The following properties are valid for this object:

Property	Data type	Description
value	Binary or String	Use this property to store the binary or string data of the attachment.
len	Integer	This property contains the file size of the attachment in bytes.
href	String	Use this property to store the unique identifier for the attachment in the SOAP request message.
action	String	Use this property to store the name of action to take with the attachment in the Service Manager Web Services API. The following options are available: • **add** – adds the specified attachment • **remove** – removes the specified attachment • **get** – retrieves the specified attachment • **update** – updates the specified attachment
name	String	Use this property to store the file name of the attachment.
type	String	Use this property to store the MIME type of the attachment.
attachmentType	String	Use this property to store the attachment type as defined in Service Manager. If present and the property has a value of "img," then the attachment must also have a MIME type of "image/gif".
stringValue	String	This is a read only property. When the attachment object contains a text value (utf-8 or ascii characters), user can use this property to get the string data of the attachment. The obtained string date then can be printed, concatenated, etc.

Methods

None

Example

- This example does the following:
- Assume there are two attachments in the incident IM10005
- Get the existing attachments from the IM10005
- Concatenate these two attachments to a string
- Create a new attachment which contains the concatenated string value
- Insert MyAttachment.txt into IM10005 as a new attachment

This example requires the following sample data:

- A binary file (for example, an image)

```
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10005"' );
var attachmentObj = f.getAttachments();
var str = "";
for ( var attachment in attachmentObj )
{
print("Attachment Name:  "+ attachmentObj[ attachment ].name );
print("Attachment Value:  "+ attachmentObj[ attachment ].stringValue );
print("Attachment Value Length:  "+ attachmentObj[ attachment ].stringValue.length );
print("Attachment HREF:  "+ attachmentObj[ attachment ].href );
print("Attachment Type:  "+ attachmentObj[ attachment ].type);
str+= attachmentObj[ attachment ].stringValue ;
}
//Create new attachment with a concatenated value
print("String Length:  "+str.length);
var attachObj = new Attachment();
attachObj.type  = "text/plain";
attachObj.name  = "MyAttachment.txt";
attachObj.value = str;
var attachmentID = f.insertAttachment( attachObj );
print("AttachmentID:   "+attachmentID);
```

2.5.3.2. JavaScript object: Header

The following JavaScript object is unique to Service Manager.

With no arguments, the **Header()** constructor creates an empty Header object.

With the HTTP header arguments, the **Header()** constructor creates a Header object containing the provided header types and values. You must enclose header types and header values in quotation marks.

Constructor

```
new Header();
new Header(Header type, Header value);
```

Arguments

The following arguments are valid for this object:

Argument	Description
Header type	The HTTP header type such as Accept-Encoding or Content-Type.
Header value	The HTTP header value such as gzip,deflate or textl.

Properties

None

Methods

None

Example

This example does the following:

- Creates a Header object using the `Header type` and `Header value` arguments
- Creates a second empty Header object
- Defines the properties of the second Header object
- Creates an empty JavaScript array
- Adds the two Header objects to the JavaScript array

This example requires the following sample data:

- A valid header

```
var h = new Header( "Accept-Encoding", "gzip,deflate" );
var hd = new Header();
hd.name = "Content-Type";
hd.value = "text/html";
var headers = new Array();
headers.push( h );
headers.push( hd );
```

2.5.3.3. JavaScript object: SCDatum

The following JavaScript object is unique to Service Manager.

With no arguments, the **SCDatum()** constructor creates an empty **SCDatum** object.

With an object argument, the **SCDatum()** constructor creates an **SCDatum** object containing the provided record, structure, or array. You must enclose table names and arrays in quotation marks.

This object's methods use the Service Manager-defined global return codes.

Constructor

```
new SCDatum()
new SCDatum(object)
```

Arguments

The following arguments are valid for this object:

Argument	Data type	Description
`object`	String	The table name, record, structure, or array you want to query or update.

Properties

None

Methods

The following methods are valid for this object:

Defined method	Description
getSize()	This method returns the size of the object.
getText()	This method returns a text representation of the object.
getType()	This method returns the object type.
getXML()	This method returns an XML representation of the object.
join()	This method returns a string of all the entries in the array.
JSDate()	This method returns a JavaScript Date object.
length()	This method returns the length of the object.
pop()	This method removes an item from an array.
push()	This method adds an item to an array.
setType()	This method sets the object type.
setValue()	This method sets the object to a given value.
shift()	This method returns a string of all the entries in the array.
toArray()	This method converts an object to a JavaScript array.
unshift()	This method returns a string of all the entries in the array.

2.5.3.3.1. JavaScript method: SCDatum.getSize()

This method returns the number of array elements in an array stored in a Service Manager Datum object. You can use this method to return the length of Service Manager array fields, which are always stored as Datum objects. To query a particular array field in an SCFile object, you can use the following format: `SCFile_object.array_field_name.getSize()`

Replace any periods in the name of `array_field_name` with underscores. For example, you must convert the Service Manager array field `update.action` to `update_action` when querying it from JavaScript.

Syntax

```
SCDatum_object.getSize();
```

Arguments

There are no arguments for this method.

Return values

A number and `RC_SUCCESS` or one of the other global return code values.

The method returns the number of array elements in an array stored in a Service Manager Datum object and a global return code value of `RC_SUCCESS`, or returns one of the error global return code values if the method can't return the number of array elements.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Attempts to get the size of the returned SCFile object (the result is always zero)
- Displays the contents of the action field (the action field contains Service Manager array data)
- Displays the size of the action field using the Service Manager **getSize()** method
- Converts the SCFile object into a JavaScript array using the Service Manager **toArray()** method
- Displays the size of the new JavaScript array using the core JavaScript length property

This example requires the following sample data:

- A valid incident record number (for example, "IM1005")

```
var incidentQuery;

function findIncidents( query )
{
 print( "Searching for incident records starting with " + query + "..." );
 var incidentFile = new SCFile( "probsummary" );
 var findIncident = incidentFile.doSelect( "number#\""+ query + "\"" );
 if ( findIncident == RC_SUCCESS )
 {
 print( "Success. Found incident records starting with " + query + ". The first record is:\n" + incidentFile );
 var objectSize = incidentFile.getSize();
 print( "Running the getSize() method on the returned SCFile object produces the following result: " + objectSize );
 var actionText = incidentFile.action;
 print( "The contents of the action field is:\n" + actionText );
 var actionSize = incidentFile.action.getSize();
 print( "The size of the action field is: " + actionSize );
 print( "Converting the SCFile object to a JavaScript array..." );
 var incidentArray = incidentFile.toArray();
 print( "The new JavaScript array is:\n" + incidentArray );
 var arraySize = incidentArray.length;
 print( "The size of the converted array is: " + arraySize );
        return incidentFile;
 }
 else
 {
  print( "Could not find incident records starting with " + query + ". " + RCtoString( findIncident ) );
  return null
 }
}

incidentQuery = "IM1005";
findIncidents( incidentQuery );
```

2.5.3.3.2. JavaScript method: SCDatum.getText()

This method returns a text string representation of the current SCFile or SCDatum object. The text string has the following format:

```
table={["field value",{"array value","array value"},...]}
```

Syntax

```
SCFile_object.getText();
```

```
SCDatum_object.getText();
```

Arguments

There are no arguments for this method.

Return values

A string and `RC_SUCCESS` or one of the other global return code values.

The method returns a text string containing the current SCFile or SCDatum object and a global return code value of `RC_SUCCESS`, or returns one of the error global return code values if the method can't return a string.

Example

This example does the following:

- Searches the contacts table for any contact name you define in the search variable
- Displays the contact record as a text string

This example requires the following sample data:

- A valid contact name (for example, "ADMINISTRATOR, SYSTEM")
- A invalid contact name (for example, "NOT A, CONTACT")

```
var contactName;

function findContactName( name )
{
 print( "Searching for contact: " + name + "..." );
 var contactList = new SCFile( "contacts" );
 var findContact = contactList.doSelect( "contact.name=\""+ name + "\"" )
 if ( findContact == RC_SUCCESS )
 {
  print( "Success. found " + name + " in contact record:\n" + contactList.getText() );
  return contactList
 }
 else
 {
  print( "Could not find contact. " + RCtoString( findContact ) );
  return null
  }
}

contactName = "ADMINISTRATOR, SYSTEM";
findContactName( contactName );

contactName = "NOT A, CONTACT";
findContactName( contactName );
```

2.5.3.3.3. JavaScript method: SCDatum.getType()

This method returns the data type of Service Manager datum objects, such as arrays and structured arrays. This method fails on any scalar fields within file records, such as number fields, date fields, string fields, and boolean fields. To determine the type of scalar fields you can use, see the core JavaScript .type property in the *JavaScript Guide* at http://developer.mozilla.org/.

Syntax

```
SCFile_object.getType();
```

```
SCDatum_object.getType();
```

Arguments

There are no arguments for this method.

Return values

A data type and `RC_SUCCESS` or one of the other global return code values.

The method returns a data type and a global return code value of `RC_SUCCESS` or returns one of the error global return code values, if the method can't determine the data type.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the contents of the incident record object
- Returns the data type of the incident record object using the **getType()** method
- Displays the contents of the action field object
- Returns the data type of the action field object using the **getType()** method
- Displays the contents of the category field
- Returns the data type of the category field object using the core JavaScript type property

This example requires the following sample data:

- A valid incident record (for example, "IM1001")

```
var incidentQuery;

function findTypes( query )
{
 print( "Searching for incident records starting with " + query + "..." );
 var incidentFile = new SCFile( "probsummary" );
 var findIncident = incidentFile.doSelect( "number#\""+ query + "\"" );
 if ( findIncident == RC_SUCCESS )
 {
  print( "Success. Found incident records starting with " + query + ". The first record is:\n" + incidentFile );
  var objectType = incidentFile.getType();
  print( "The object incidentFile is of type: " + objectType );
  var arrayField = incidentFile.action;
  print( "The contents of the action field are:\n" + arrayField );
  fieldType = arrayField.getType();
  print( "The type of the action field is: " + fieldType );
  var textField = incidentFile.category;
  print( "The contents of the category field are:\n" + textField );
  fieldType = textField.getType;
  print( "The type of the category field is: " + fieldType );
 }
 else
 {
  print( "Could not find incident records starting with " + query + ". " + RCtoString( findIncident ) );
  return null
 }
}

incidentQuery = "IM1001";
findTypes( incidentQuery );
```

2.5.3.3.4. JavaScript method: SCDatum.getXML()

This method returns an XML object representation of the current SCFile or SCDatum object. The XML object has the following format, which is compatible with the Service Manager Web services API:

`<model name="table" query="query"><keys><fieldname>key value</fieldname>...</keys><instance><fieldname>instance value</fieldname>...</instance></model>`

Syntax

```
SCFile_object.getXML();
```

```
SCDatum_object.getXML();
```

Arguments

There are no arguments for this method.

Return values

An XML object and `RC_SUCCESS` or one of the other global return code values.

The method returns an XML object containing the current SCFile or SCDatum object and a global return code value of `RC_SUCCESS` or returns one of the failure global return code values if the method can't return an XML object.

Example

This example does the following:

- Searches the contacts table for any contact name you define in the search variable
- Displays the contact record as an XML object

This example requires the following sample data:

- A valid contact name (for example, "ADMINISTRATOR, SYSTEM")
- A invalid contact name (for example, "NOT A, CONTACT")

```
var contactName;

function findContact( name )
{
 print( "Searching for contact: " + name + "..." );
 var contactList = new SCFile( "contacts" );
 var isContact = contactList.doSelect( "contact.name=\""+ name + "\"" )
 if ( isContact == RC_SUCCESS )
 {
  print( "Success. found " + name + " in contact record:\n" + contactList.getXML() );
  return contactList
 }
 else
 {
  print( "Could not find contact. " + RCtoString( isContact ) );
  return null
 }
}

contactName = "ADMINISTRATOR, SYSTEM";
findContact( contactName );

contactName = "NOT A, CONTACT";
findContact( contactName );
```

2.5.3.3.5. JavaScript method: SCDatum.join()

This method returns the elements of either a Service Manager Datum array object or a JavaScript array. This method doesn't use the Service Manager global return code values.

Syntax

```
SCDatum_object.join();
```

Arguments

There are no arguments for this method.

Return values

A string.

The method returns the elements of an array.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the new incident record as a text string
- Displays the contents of action field using the **join()** method

This example requires the following sample data:

- A valid value for the number field (for example, "IM1010")
- An incident record with an action field value (for example, "IM1010")

```
var incidentID;

function findIncident( id )
{
 print( "Searching for Incident record: " + id + "..." );
 var incidentFile = new SCFile( "probsummary" );
 var rc = incidentFile.doSelect( "number=\"" + id + "\"" )
 if ( rc == RC_SUCCESS )
 {
  print( "Success. found Incident record:\n" + incidentFile.getText() );
  print( "Displaying the contents of the action field as an object: " + incidentFile.action );
  print( "Displaying the contents of the action field with join() method: " + incidentFile.action.join() );
  return incidentFile
 }
 else
 {
  print( "Could not find Incident record. " + RCtoString( rc ) );
  return null
 }
}

incidentID = "IM1010";
findIncident( incidentID );
```

2.5.3.3.6. JavaScript method: SCDatum.JSDate()

This method is deprecated as of version 6.1 with the introduction of the SCFile object. This method converted Service Manager date/time data into JavaScript date/time objects. The SCFile object now automatically converts Service Manager date/time data into JavaScript date/time objects. Alternatively, you can now use the XMLDate object and its methods to convert between date/time formats.

Syntax

```
SCDatum_object.JSDate( dataType );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
dataType	String	This argument specifies the data type of a field in an SCDatum object.

Return values

None

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the incident record and the value of the open.time field
- Creates an SCDatum object containing the number of milliseconds you want to convert
- Converts the open.time field into an SCDatum object (by default the open.time field data is in Service Manager's date/time data type)
- Converts the SCDatum object into a JavaScript date/time

This example requires the following sample data:

- A valid incident record with a value in the open.time field (for example, "IM1010")

```
var incidentID;

function findIncident( id )
{
 print( "Searching for incident: " + id + "..." );
 var incidentFile = new SCFile( "probsummary" );
 var f = incidentFile.doSelect( "number=\""+ id + "\"" );
 if ( f == RC_SUCCESS )
 {
  print( "Success. found " + id + " in incident record:\n" + incidentFile.getText() );
  print( "The value of the open.time field is: " + incidentFile.open_time );
  print( "Converting open.time to an SCDatum object..." );
  var dat = new SCDatum( incidentFile.open_time );
  print( "The new SCDatum object is: " + dat );
  print( "Converting the SCDatum object to a JSDate..." );
  var j = dat.JSDate();
  print( "The converted SCDatum object is: " + j );
  return incidentFile
 }
 else
 {
  print( "Could not find incident record. " + RCtoString( f ) );
  return null
 }
}

incidentID = "IM1010";
findIncident( incidentID );
```

2.5.3.3.7. JavaScript method: SCDatum.length()

This method returns the number of array elements in an array stored in a Service Manager Datum object. You can use this method to return the length of Service Manager array fields, which are always stored as Datum objects. To query a particular array field in an SCFile object, use the following format: `SCFile_object.array_field_name.length()`.

> **Note**
>
> Note Replace any periods in the name of `array_field_name` with underscores. For example, you must convert the Service Manager array field `update.action` to `update_action` when querying it from JavaScript.

Syntax

```
SCDatum_object.length();
```

Arguments

There are no arguments for this method.

Return value

A number and `RC_SUCCESS` or one of the other global return code values.

The method returns the number of array elements in an array stored in a Service Manager Datum object and a global return code value of RC_SUCCESS or returns one of the error global return code values if the method can't return the number of array elements.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Attempts to get the size of the returned SCFile object (the result is always zero)
- Displays the contents of the action field (the action field contains Service Manager array data)
- Displays the size of the action field using the Service Manager **length()** method
- Converts the SCFile object into a JavaScript array using the Service Manager **toArray()** method
- Displays the size of the new JavaScript array using the core JavaScript length property

```
var incidentQuery;

function findIncidents( query )
{
 print( "Searching for incident records starting with " + query + "..." );
 var incidentFile = new SCFile( "probsummary" );
 var findIncident = incidentFile.doSelect( "number#\""+ query + "\"" );
 if ( findIncident == RC_SUCCESS )
 {
  print( "Success. Found incident records starting with " + query + ". The first record is:\n" + incidentFile );
  var objectSize = incidentFile.getSize();
  print( "Running the getSize() method on the returned SCFile object produces the following result: " + objectSize
);
  var actionText = incidentFile.action;
  print( "The contents of the action field is:\n" + actionText );
  var actionSize = incidentFile.action.getSize();
  print( "The size of the action field is: " + actionSize );
  print( "Converting the SCFile object to a JavaScript array..." );
  var incidentArray = incidentFile.toArray();
  print( "The new JavaScript array is:\n" + incidentArray );
  var arraySize = incidentArray.length;
  print( "The size of the converted array is: " + arraySize );
        return incidentFile;
 }
 else
 {
  print( "Could not find incident records starting with " + query + ". " + RCtoString( findIncident ) );
  return null
 }
}

incidentQuery = "IM1005";
findIncidents( incidentQuery );
```

2.5.3.3.8. JavaScript method: SCDatum.pop()

This method removes the last element from a JavaScript array and returns the element removed as a string. This method doesn't use the Service Manager global return code values.

Syntax

```
SCDatum_object.pop();
```

Arguments

There are no arguments for this method.

Return values

A string.

The method returns the element removed from the array.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the new incident record as a text string
- Converts the action field into a JavaScript array
- Adds an item to the end of the array and displays the total number of elements in the array
- Removes an item from the end of the array and displays the array item removed

This example requires the following sample data:

- A valid value for the number field (for example, "IM1010")
- An incident record with an action field value (for example, "IM1010")

```
var incidentID;
function findIncident( id )
{
print( "Searching for Incident record: " + id + "..." );
var incidentFile = new SCFile( "probsummary" );
var rc = incidentFile.doSelect( "number=\"" + id + "\"" )
if ( rc == RC_SUCCESS )
{
print( "Success. found Incident record:\n" + incidentFile.getText() );
print( "Converting the action field to a JavaScript array..." );
var a = incidentFile.action.toArray();
print( "The action field array contains the following: " + a );
print( "Pushing new array entry..." );
var addOne = a.push( "First array item" );
print( "The number of items in the array are: " + addOne );
print( "The action field array contains the following: " + a );
print( "Popping new array entry..." );
var removeOne = a.pop();
print( "The array item removed was: " + removeOne );
print( "The action field array contains the following: " + a );
return incidentFile;
}
else
{
print( "Could not find Incident record. " + RCtoString( rc ) );
return null
}
}
incidentID = "IM1010";
findIncident( incidentID );
```

2.5.3.3.9. JavaScript method: SCDatum.push()

This method adds an entry to the end of a JavaScript array and returns the total number of entries in the array. If you apply this method to a Service Manager array object, then it adds the item but doesn't return the number of entries in the array. This method doesn't use the Service Manager global return code values.

Syntax

```
SCDatum_object.push( arrayItem );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
arrayItem	String	This argument specifies the array item you want to add to a JavaScript array.

Return values

A number.

The method returns the number of items in the array.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the new incident record as a text string
- Converts the contents of the action field into a JavaScript array
- Adds an item to the end of the JavaScript array and prints the total number of items in the array

This example requires the following sample data:

- A valid value for the number field (for example, "IM1010")

- An incident record with an action field value (for example, "IM1010")

```
var incidentID;
function findIncident( id )
{
print( "Searching for Incident record: " + id + "..." );
var incidentFile = new SCFile( "probsummary" );
var rc = incidentFile.doSelect( "number=\"" + id + "\"" )
if ( rc == RC_SUCCESS )
{
print( "Success. found Incident record:\n" + incidentFile.getText() );
print( "Converting the action field to a JavaScript array..." );
var a = incidentFile.action.toArray();
print( "The action field array contains the following: " + a );
print( "Pushing new array entry..." );
var addOne = a.push( "First array item" );
print( "The number of items in the array are: " + addOne );
print( "The action field array contains the following: " + a );
print( "Pushing new array entry..." );
var addTwo = a.push( "Second array item" );
print( "The number of items in the array are: " + addTwo );
print( "The action field array contains the following: " + a );
return incidentFile;
}
else
{
print( "Could not find Incident record. " + RCtoString( rc ) );
return null
}
}
incidentID = "IM1010";
findIncident( incidentID );
```

2.5.3.3.10. JavaScript method: SCDatum.setType()

This method sets the data type of Service Manager datum objects. The SCDatum object also automatically converts Service Manager data types into corresponding JavaScript data types. You can also use the XMLDate object and its methods to convert between date/time formats.

Syntax

```
SCDatum_object.setType( dataType );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
dataType	Number	This argument specifies the Service Manager data type for the object. See Data types.

Return values

None

Example

This example does the following:

- Creates an SCDatum object containing the number of milliseconds you want to convert
- Sets the data type of the SCDatum object to a Service Manager date/time type
- Converts the number of milliseconds into a JavaScript date/time

This example requires the following sample data:

- A number of milliseconds (for example, "1234567890")

```
var millisecs;
function findDateTime( num )
{
var t = new SCDatum( num );
t.setType( 3 );
print( num + " milliseconds is equal to the date: " + t.JSDate() );
}
millisecs = 1234567890;
findDateTime( millisecs );
```

2.5.3.3.11. JavaScript method: SCDatum.setValue()

This method updates an Service Manager record with the new field value defined in the `newValue` argument. This method doesn't use the Service Manager global return codes.

Syntax

```
SCDatum_object.setValue( newValue );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`newValue`	String	This argument specifies the new value you want the Service Manager object to have. You must use the slash character to escape out quotation marks and any special characters restricted from JavaScript.

Return values

None

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the current value of the action field
- Sets the value of the action field to any value you define in the input variable

This example requires the following sample data:

- A valid incident record with a value in the action field (for example, "IM1010")
- A valid action field value update (for example, set the action field to "New description")

```
var incidentID;
var newValue;
function findIncident( id, value )
{
print( "Searching for Incident record: " + id + "..." );
var incidentFile = new SCFile( "probsummary" );
var rc = incidentFile.doSelect( "number=\"" + id + "\"" )
if ( rc == RC_SUCCESS )
{
print( "Success. found Incident record " + id );
print( "Displaying the contents of the action field as an object: " + incidentFile.action );
print( "Setting the value of the action field..." );
incidentFile.action.setValue( value );
print( "Displaying the contents of the action field as an object: " + incidentFile.action );
print( "Displaying the Incident record: " + incidentFile );
return incidentFile
}
else
{
print( "Could not find Incident record. " + RCtoString( rc ) );
return null
}
}
incidentID = "IM1010";
newValue = "New description";
findIncident( incidentID, newValue );
```

2.5.3.3.12. JavaScript method: SCDatum.shift()

This method removes the first element from a JavaScript array and returns the element removed as a string. This method doesn't use the Service Manager global return code values.

Syntax

```
SCDatum_object.shift();
```

Arguments

There are no arguments for this method.

Return values

A string.

The method returns the element removed from the array.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the new incident record as a text string
- Converts the action field into a JavaScript array
- Adds an item to the end of the array and displays the total number of elements in the array
- Removes an item from the beginning of the array and displays the array item removed

This example requires the following sample data:

- A valid value for the number field (for example, "IM1010")
- An incident record with an action field value (for example, "IM1010")

```
var incidentID;
function findIncident( id )
{
print( "Searching for Incident record: " + id + "..." );
var incidentFile = new SCFile( "probsummary" );
var rc = incidentFile.doSelect( "number=\"" + id + "\"" )
if ( rc == RC_SUCCESS )
{
print( "Success. found Incident record:\n" + incidentFile.getText() );
print( "Converting the action field to a JavaScript array..." );
var a = incidentFile.action.toArray();
print( "The action field array contains the following: " + a );
print( "Pushing new array entry..." );
var addOne = a.push( "First array item" );
print( "The number of items in the array are: " + addOne );
print( "The action field array contains the following: " + a );
print( "Shifting array one entry..." );
var removeOne = a.shift();
print( "The array item removed was: " + removeOne );
print( "The action field array contains the following: " + a );
return incidentFile;
}
else
{
print( "Could not find Incident record. " + RCtoString( rc ) );
return null
}
}
incidentID = "IM1010";
findIncident( incidentID );
```

2.5.3.3.13. JavaScript method: SCDatum.toArray()

This method converts an Service Manager Datum object into a JavaScript array.

Syntax

```
SCDatum_object.toArray();
```

Arguments

There are no arguments for this method.

Return values

A JavaScript array and `RC_SUCCESS` or one of the other global return code values.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the incident record as a text string
- Converts the contents of the action field into a JavaScript array

This example requires the following sample data:

- A valid incident record number (for example, "IM1010")
- An incident record with an action field value (for example, "IM1010")

```
var incidentID;
function findIncident( id )
{
print( "Searching for Incident record: " + id + "..." );
var incidentFile = new SCFile( "probsummary" );
var rc = incidentFile.doSelect( "number=\"" + id + "\"" )
if ( rc == RC_SUCCESS )
{
print( "Success. found Incident record:\n" + incidentFile.getText() );
print( "Converting the action field to a JavaScript array..." );
var a = incidentFile.action.toArray();
print( "The action field contains the following " + a );
return incidentFile
}
else
{
print( "Could not find Incident record. " + RCtoString( rc ) );
return null
}
}
incidentID = "IM1010";
findIncident( incidentID );
```

2.5.3.3.14. JavaScript method: SCDatum.unshift()

This method adds an entry to the beginning of a JavaScript array and returns the total number of entries in the array. If you apply this method to a Service Manager array object, then it adds the item but doesn't return the number of entries in the array. This method doesn't use the Service Manager global return code values.

Syntax

```
SCDatum_object.unshift( arrayItem );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
arrayItem	String	This argument specifies the array item you want to add to a JavaScript array.

Return values

A number.

The method returns the number of items in the array.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the new incident record as a text string
- Converts the contents of the action field into a JavaScript array
- Adds an item to the beginning of the JavaScript array and prints the total number of items in the array

This example requires the following sample data:

- A valid value for the number field (for example, "IM1010")
- An incident record with an action field value (for example, "IM1010")

```
var incidentID;
function findIncident( id )
{
print( "Searching for Incident record: " + id + "..." );
var incidentFile = new SCFile( "probsummary" );
var rc = incidentFile.doSelect( "number=\"" + id + "\"" )
if ( rc == RC_SUCCESS )
{
print( "Success. found Incident record:\n" + incidentFile.getText() );
print( "Converting the action field to a JavaScript array..." );
var a = incidentFile.action.toArray();
print( "The action field array contains the following: " + a );
print( "Unshifting array one entry..." );
var unshifting = a.unshift( "First array item" );
print( "The number of items in the array are: " + unshifting );
print( "The action field array contains the following: " + a );
return incidentFile;
}
else
{
print( "Could not find Incident record. " + RCtoString( rc ) );
return null
}
}
incidentID = "IM1010";
findIncident( incidentID );
```

2.5.3.4. JavaScript object: SCFile

The following JavaScript object is unique to Service Manager.

With no parameters, the **SCFile()** constructor creates an empty **SCFile** object.

With an object parameter, the **SCFile()** constructor creates an **SCFile** object containing the provided record, structure, or array. You must enclose table names and arrays in quotation marks.

This object's methods use the Service Manager-defined global return codes.

Constructor

```
new SCFile()
new SCFile(object)
new SCFile( object, SCFILE_READONLY )
```

Arguments

The following arguments are valid for this object:

Argument	Data type	Description
`object`	String	The table name, record, structure, or array you want to query or update.
`SCFILE_READONLY`	Number	Causes the file object to become read-only. This constant is only supported as a second parameter, with the file name specified as the first parameter.

Properties

None

Methods

The following methods are valid for this object:

Defined method	Description
deleteAttachment(attachID)	This method deletes a specific attachment associated with the current file record.
deleteAttachments()	This method deletes all of the attachments associated with the current file record.
doAction()	This method executes a method defined on the object.
doClose()	This method closes the cursor that the SCFile Object stands for.
doCount()	This method returns the number of records for the provided query.
doDelete()	This method is synonymous with the doRemove() method. This method simply has a more intuitive name.
doInsert()	This method inserts an object into the database.
doPurge()	This method purges a set of records directly in the back-end RDBMS.
doRemove()	This method removes an object from the database.
doSave()	This method inserts a record if it is new and updates the record if it already exists.
doSelect()	This method makes a query. It no longer requires that the first argument be the name of the table if the object is constructed with the name of the table.
doUpdate()	This method updates the database.
getAttachment(attachID)	This method gets a specific attachment associated with the current file record.
getAttachments()	This method gets all of the attachments associated with the current file record.
getBinary()	This method returns the binary representation of a field in a Service Manager file object.
getFirst()	This method moves to the first record if the object represents a result set.
getLast()	This method moves to the last record if the object represents a result set.
getLastRC()	This method returns the last return code generated by the **SCFile** and **SCDatum** objects.
getMessages()	This method returns an array of messages generated by the Document Engine from a previous **doAction()**.
getNext()	This method moves to the next record if the object represents a result set.
getPrev()	This method moves to the previous record if the object represents a result set.
getPurgedRowCount()	This method retrieves the number of records deleted from a table by the **doPurge()**.

getText()	This method returns a text representation of the object.
getType()	This method returns the object type.
getXML()	This method returns an XML representation of the object.
insertAttachment()	This method creates a new attachment associated with the current file record.
isRecord()	This method checks if an object represents a current table and has a record.
setBinary()	This method saves binary data to a field in a Service Manager file object.
setFields()	This method causes any subsequent **doSelect()** calls to fetch only the specified fields.
setRecord()	This method sets the field values from a given XML string.
setOrderBy()	This method causes any subsequent **doSelect()** call on a SCFile object to fetch only the specified fields by the specified sort sequences of the SCFile Object.
updateAttachment(attachObj)	This method updates a specific attachment associated with the current file record.

Example

This example does the following:

- Creates an SCFile object from the contacts table
- Selects records from this contacts table starting with the letter B
- Displays the contacts matching the select query

This example requires the following sample data:

- A valid query against the contacts table (for example, contact.name # "B")

```
var q = new SCFile("contacts");
if ( ( q.doSelect( "contact.name # \"B\"" ) ) == RC_SUCCESS )
{
  print(q.getText());
}
else
  print( "Return code is: " + q.RCtoString );
```

2.5.3.4.1. JavaScript method: SCFile.deleteAttachment(attachID)

This method deletes a specific attachment associated with the current file record.

Values passed to **deleteAttachment()** will normally be ones obtained from a previous **insertAttachment()**, **updateAttachment()**, or **getAttachments()** operation.

Before calling **deleteAttachment**, you must establish a current record using one of the positioning methods, such as **doSelect()**, **getNext()**, etc.

Syntax

```
SCFile_object.deleteAttachment( attachID );
```

Arguments

deleteAttachment() takes one argument, which is a string containing an attachment ID of the cid:xxxxx form.

Return values

If successful, **deleteAttachment()** returns `RC_SUCCESS`. Otherwise. it returns an error return code. For example, it returns `RC_NO_MORE` if the attachment was not found.

Example

This example does the following:

- Selects one record from the probsummary file
- Deletes one attachment associated with the current record

This example requires the following sample data:

- Record number IM10001 in the probsummary table

```
function createTestAttachment()
{
var attachmentObj = new Attachment();
attachmentObj.type  = "text/plain";
attachmentObj.name  = "MyAttachment.txt";
attachmentObj.value = "My text attachment";
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
var attachmentID = f.insertAttachment( attachmentObj );
return attachmentID;
}
var attachmentID = createTestAttachment();
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
var rc = f.deleteAttachment( attachmentID );
```

2.5.3.4.2. JavaScript method: SCFile.deleteAttachments()

Deletes all of the attachments associated with the current file record.

Before calling **deleteAttachments**, you must establish a current record using one of the positioning methods, such as **doSelect()**, **getNext()**, etc.

Syntax

```
SCFile_object.deleteAttachments()
```

Arguments

deleteAttachments() takes no arguments and returns no arguments.

Return values

If successful, **deleteAttachments()** deletes all of the attachments associated with the current record.

Example

This example does the following:

- Selects one record from the probsummary file
- Deletes all attachments associated with the current record
- Displays the number of attachments associated with the current record

This example requires the following sample data:

- Record number IM10001 in the probsummary file

```

var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
f.deleteAttachments();
var attachments = f.getAttachments();
print( attachments.length );  // will be zero
```

2.5.3.4.3. JavaScript method: SCFile.doAction()

This method executes a Document Engine action using any new field values defined in a Service Manager file object. You must define a Service Manager file object and new values for required fields.

Syntax

```
SCFile_object.doAction( docEngineAction );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
docEngineAction	String	This argument specifies the Document Engine action you want the script to execute on the object. This argument must contain a valid Service Manager Document Engine action appropriate for the Service Manager object.

Return values

RC_SUCCESS or one of the other global return code values.

The method returns RC_SUCCESS if the method successfully runs the Document Engine action or returns one of the error global return code values if the method can't run the Document Engine action.

Sample global return code values:

```
RC_ERROR (-1) : Some other error occurred. Examine the contents of the Messages view or the sm.log file for more
information.
RC_SUCCESS (0) : The operation succeeded.
RC_CANT_HAVE (3) : The operation failed because the resource is unavailable because some other user or process has
the resource locked.
RC_NO_MORE (9) : No more records are available in the result set.
RC_DUPLICATE_KEY (48) : The insert operation failed because the file already contains a record with this unique key
value.
RC_MODIFIED (51) : The update operation failed because the record was modified by another user or process since you
read it.
RC_DELETED (52) : The operation failed because the record was deleted by another user or process.
RC_BAD_QUERY (66) : The query failed due to incorrect query syntax.
RC_NOT_AUTHORIZED (28) : The request operation was not performed due to an authorization failure. Check the
permissions associated with the user who submitted the request.
RC_VALIDATION_FAILED (71) : The operation failed because the data supplied in a field or record did not pass
validity checks performed by the application.
RC_UNABLE_TO_WRITE_TO_FILE (20) : The operation failed. Check to make sure the file name is correct, that the file
exists, and that it is not read-only.
RC_UNABLE_TO_CLOSE_FILE (47) : The operation failed. Check to make sure the file name is correct, that the file
exists, and that it is not read-only.
RC_UNABLE_TO_DELETE_FILE (57) : The operation failed. Check to make sure the file name is correct, that the file
exists, and that it is not read-only.
```

Example

This example does the following:

- Creates a new incident record from variable values
- Resolves the incident record

This example requires the following sample data:

- The "Use Resolved Status?" option enabled in the Incident Management Environment record
- A valid value for the number field (for example, "IM11112")

```
var numberValue;
var callbackContact;
var categoryValue;
var assignmentValue;
var descriptionValue;
var actionType;
var resultionCodeValue;
var resolutionValue;
function insertIncident( num, name, cat, group, desc )
{
print( "Creating new incident record..." );
var newIncident = new SCFile( "probsummary" );
newIncident.number = num;
newIncident.callback_contact = name;
newIncident.category = cat;
newIncident.assignment = group;
newIncident.description = desc;
var rc = newIncident.doInsert();
if ( rc == RC_SUCCESS )
{
print( "Success. Created new record " + newIncident.getText() );
return newIncident.number
}
else
{
print( "Could not create record. " + RCtoString( rc ) );
return null
}
}
function resolveIncident( id, action, rescode, res )
{
print( "Resolving incident " + id + "..." );
var findIncident = new SCFile ( "probsummary" );
var f = findIncident.doSelect( "number=\"" + id + "\"" );
if ( f == RC_SUCCESS )
{
print( "Success. Found interaction record..." );
findIncident.resolution_code = rescode;
findIncident.resolution = res;
var a = findIncident.doAction( action );
print( "The resolve Incident return value is " + RCtoString( a ) );
print( "The incident record is " + findIncident.getText() )
print( "The value of the status field is: " + findIncident.problem_status );
return findIncident
}
else
{
print( "Could not find record. " + RCtoString( f ) );
return null
}
}
numberValue = "IM11121"
callbackContact = "ADMINISTRATOR, SYSTEM";
categoryValue = "network";
assignmentValue = "HELPDESK";
descriptionValue = null;
interactionID = insertIncident( numberValue, callbackContact, categoryValue, assignmentValue, descriptionValue );
actionType = "resolve";
resultionCodeValue = "Advice & Guidance";
resolutionValue = null;
resolveIncident( interactionID, actionType, resultionCodeValue, resolutionValue );
```

2.5.3.4.4. JavaScript method: SCFile.doClose()

This method closes the cursor that the SCFile Object stands for.

Syntax

```
SCFile_object.doClose();
```

Arguments

There are no arguments for this method.

Return values

RC_SUCCESS or one of the other global return code values.

The method returns RC_SUCCESS if the method successfully closes an object or returns one of the error global return code values if the method can't close an object.

Example

This example does the following:

- Determines the number of contact records.
- Displays the contact record as a text string.
- Close the SCfile Object.

This example requires the following sample data:

- A contact name.

```
var contactName = "ADMINISTRATOR, SYSTEM";
var contactList = new SCFile( "contacts" );
var findContact = contactList.doSelect( "contact.name=\""+ contactName + "\"" )
if ( findContact == RC_SUCCESS )
{
  print( "Success. found " + contactName + " in contact record:\n" + contactList.getText() );
}
else
{
  print( "Could not find contact. " + RCtoString( findContact ) );
}
try{
  var rc = contactList.doClose();
  if (rc == RC_SUCCESS)
    print("Success. close the SCFile Object");
  else
    print("Could not close the SCFile Object");
}
catch(e){
  print(e.toString());
}
```

2.5.3.4.5. JavaScript method: SCFile.doCount()

This method returns the number of records specified by the query.



Note

Note You should not use this method on an SCFile object for which you have performed a previous **doSelect()** if you intend to iterate over the results of the **doSelect()**.

Syntax

```
SCFile_object.doCount( "query" );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
query	String	This argument specifies the query you want to use to search for Service Manager records. You must use the slash character to escape out quotation marks and any special characters restricted from JavaScript.

Return values

A number representing the count of records.

Example

This example does the following:

- Determines the number of contact records.
- Displays the contact record as a text string.

This example requires the following sample data:

- A valid contact name (for example, "ADMINISTRATOR, SYSTEM").
- An invalid contact name (for example, "NOT A, CONTACT").

```
var contactName;
function countContacts( location )
{
print( "Counting contacts at location: " + location + "..." );
var contactList = new SCFile( "contacts" );
var nCount = contactList.doCount( "location=\""+ location +
                  "\"" );
var rc = getLastRC();
if ( rc == RC_SUCCESS )
{
return nCount;
}
else
{
print( "Could not count records. " + RCtoString( rc ) );
return -1;
}
}
```

2.5.3.4.6. JavaScript method: SCFile.doDelete()

This method removes a Service Manager record with any new field values defined in a Service Manager file object. You must define a Service Manager file object and new values for fields. This method is a synonym of the **doRemove()** method, and related to the more extensive **doPurge()** method.

Syntax

```
SCFile_object.doDelete();
```

Arguments

There are no arguments for this method.

Return values

RC_SUCCESS or one of the other global return code values.

The method returns RC_SUCCESS if the method successfully removes a record or returns one of the error global return code values if the method can't remove the record.

Example

This example does the following:

- Searches the probsummary table for any interaction record you define in the search variable
- Removes the interaction record

This example requires the following sample data:

- A valid value for the number field (for example, "IM11111")

```
var numberValue;
function removeInteraction( num )
{
print( "Removing interaction record..." );
var interactionRecord = new SCFile( "probsummary" );
var findRecord = interactionRecord.doSelect( "number=\"" + num + "\"");
print( "The interaction record is " + interactionRecord );
var rc = interactionRecord.doDelete();
if ( rc == RC_SUCCESS )
{
print( "Success. Removed record " + interactionRecord.getText() );
return interactionRecord
}
else
{
print( "Could not remove record. " + RCtoString( rc ) );
return null
}
}
numberValue = "IM11111"
removeInteraction( numberValue );
```

Related topics

JavaScript object: SCFile
JavaScript method: SCFile.doRemove()
JavaScript method: SCFile.doPurge()

2.5.3.4.7. JavaScript method: SCFile.doInsert()

This method creates a Service Manager record with any new field values defined in a file object. You must define a Service Manager file object and new values for the fields.

Syntax

```
SCFile_object.doInsert();
```

Arguments

There are no arguments for this method.

Return values

`RC_SUCCESS` or one of the other global return code values.

The method returns `RC_SUCCESS` if the method successfully creates a record or returns one of the error global return code values if the method can't create the record.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the new incident record as a text string

This example requires the following sample data:

- A valid value for the number field (for example, "IM11111")
- A valid value for the callback.contact field (for example, "ADMINISTRATOR, SYSTEM")
- A valid value for the category field (for example, "network")

```
var numberValue;
var callbackContact;
var categoryValue;
var descriptionValue;
function insertContact( num, name, cat )
{
print( "Creating new interaction record..." );
var newInteraction = new SCFile( "probsummary" );
newInteraction.number = num;
newInteraction.callback_contact = name;
newInteraction.category = cat;
var rc = newInteraction.doInsert();
if ( rc == RC_SUCCESS )
{
print( "Success. Created new record " + newInteraction.getText() );
return newInteraction
}
else
{
print( "Could not create record. " + RCtoString( rc ) );
return null
}
}
numberValue = "IM11112"
callbackContact = "ADMINISTRATOR, SYSTEM";
categoryValue = "network";
insertContact( numberValue, callbackContact, categoryValue, descriptionValue );
```

2.5.3.4.8. JavaScript method: SCFile.doPurge()

This method enables you to purge a set of Service Manager records directly in the back-end RDBMS while bypassing record level processing. The **doPurge()** method is an alternative to **doDelete()** or **doRemove()**. It enables the back-end RDBMS to do most of the processing, thereby improving execution times when deleting a large amount of data from a single table.

The **doDelete()** method requires you to select a list of records from the database, iterate over each record in the list and perform a "delete" on a record by record basis. When the delete is performed, Service Manager respects workflow actions as defined in record level triggers. The **doPurge()** method reduces the overhead of much of this by sending one SQL statement to the RDBMS to purge the data. As a result of the RDBMS processing the record set purge in one transaction, Service Manager doesn't have control to process record level triggers on the data being purged. Standard Service Manager trigger processing will not occur on these records.

You should use **doPurge()** only when you need to quickly delete a large record set from a single Service Manager table, and it doesn't matter whether Service Manager triggers are processed.



Caution

Caution Purging data has some risks. Use doPurge() with caution. It is the equivalent of issuing a SQL statement to the RDBMS such as, `DELETE FROM "SM_TABLE_NAME" WHERE "COLOUMN_NAME" = 'value'` .

Some issues to understand and consider before using this method:

- Data referential integrity could be an issue by purging data that's related to other types of Service Manager data. This method doesn't execute normal record level workflow processing in the applications layer. When you turn off triggers and run **doPurge()**, you will bypass normal workflow processing in addition to record level triggers.
- No Service Manager rollback or undo is available after you run **doPurge()**.
- No Service Manager audit or other tracking is available to indicate changes made by **doPurge()**. Consider coding your own audit mechanism.

You can retrieve the number of records deleted from a table by **doPurge()** by using the JavaScript method: getPurgedRowCount()

Note

Note When you use **doPurge()**, Service Manager deletes a set of records. This causes a delete to the back-end database. When Service Manager attempts to delete a large set of records, the database could run out of space for this activity and cause an error. Each supported database platform provides some type of transaction or undo log to back up data changes and allow a rollback in case of errors. If you encounter such an error, please contact your database administrator and ask them to increase the size available for this purpose.

Some examples of the errors you could encounter are:

Oracle: `SQL code=30036 message=ORA-30036: unable to extend segment by % in undo tablespace`

SQL Server SQL State: `42000-9002 Message: [Microsoft][SQL Native Client][SQL Server]The transaction log for database '%' is full`

Syntax

```
SCFile_object.doPurge( query );
```

Arguments

The following arguments are valid for this method:

Argument	Description
`query`	String, Boolean, or QueryCond object This argument specifies the query you want to use to search for Service Manager records. You must use the slash character to escape out quotation marks and any special characters restricted from JavaScript. Boolean values (true or false) or a QueryCond object are also acceptable.

Return values

Returns `RC_SUCCESS` if successful and `RC_ERROR` on failure.

The following conditions will cause a **doPurge()** failure:

- Calling **doPurge()** on an uninitialized SCFile or a read-only SCFile
- Calling **doPurge()** with an empty query argument
- Calling **doPurge()** with an invalid argument type (not a string)
- Calling **doPurge()** with one (or more) invalid (or unmapped) fields.

A basic validation is performed on the field names to ensure they exist in the DBDICT. This validation will not cause a failure of the method; it will return `RC_SUCCESS`. However, in this case, it will default to a "false" query and no records will be deleted.

- Calling **doPurge()** with an unsupported complex type SCFile relation.

See the limitations below for supported types.

- Calling **doPurge()** with an SCFile object which has "delete" triggers defined and enabled.

When the method fails, it has no effect.

Limitations

Only simple SCFile relations are supported:

- Must not be a join file or a merge file
- Must have only 1 main table alias (m1)
- Must not have any alias tables (a1,a2,....)
- No LDAP mapped fields
- The table must not have "delete" triggers defined and enabled

Example

This example does the following:

- Searches the kmknowledgebaseupdates table for records where docid="docid1"
- Deletes whatever it finds.

This example requires the following sample data:

- Records in kmknowledgebaseupdates table where docid = "docid1"

```
var fileName = "kmknowledgebaseupdates";
var recordList = new SCFile(fileName);
var deleteRC = recordList.doPurge("docid = \"docid1\"");
```

Related topics

JavaScript object: SCFile
JavaScript method: SCFile.getPurgedRowCount()
JavaScript method: SCFile.doDelete()
JavaScript method: SCFile.doRemove()

2.5.3.4.9. JavaScript method: SCFile.doRemove()

This method removes a single Service Manager record matching the field values defined in the Service Manager file object. To remove multiple records use the doSelect method to store a record list as a JavaScript object, and then use the **getNext**, **getPrev**, **getFirst**, or **GetLast** methods to bring a particular record into memory for removal. It requires defining a Service Manager file object. This method is a synonym of the **doDelete()** method, and related to the more extensive **doPurge()** method.

Syntax

```
SCFile_object.doRemove();
```

Arguments

There are no arguments for this method.

Return values

`RC_SUCCESS` or one of the other global return code values.

The method returns `RC_SUCCESS` if the method successfully removes a record or returns one of the error global return code values if the method can't remove the record.

Example

This example does the following:

- Searches the probsummary table for any interaction record you define in the search variable
- Removes the interaction record

This example requires the following sample data:

- A valid value for the number field (for example, "IM11111")

```
var numberValue;
function removeInteraction( num )
{
print( "Removing interaction record..." );
var interactionRecord = new SCFile( "probsummary" );
var findRecord = interactionRecord.doSelect( "number=\"" + num + "\"");
print( "The interaction record is " + interactionRecord );
var rc = interactionRecord.doRemove();
if ( rc == RC_SUCCESS )
{
print( "Success. Removed record " + interactionRecord.getText() );
return interactionRecord
}
else
{
print( "Could not remove record. " + RCtoString( rc ) );
return null
}
}
numberValue = "IM11111"
removeInteraction( numberValue );
```

Related topics

JavaScript object: SCFile
JavaScript method: SCFile.doDelete()
JavaScript method: SCFile.doPurge()

2.5.3.4.10. JavaScript method: SCFile.doSave()

This method adds to or updates the Service Manager database with any new field values defined in a Service Manager file object. It requires defining a Service Manager file object and defining new values for fields.

Syntax

```
SCFile_object.doSave();
```

Arguments

There are no arguments for this method.

Return values

`RC_SUCCESS` or one of the other global return code values.

The method returns `RC_SUCCESS` if the method successfully updates a record or returns one of the error global return code values if the method can't update the record.

Example

This example does the following:

- Searches the contacts table for any contact name you define in the search variable
- Displays the contact record as a text string
- Updates the value of the user.type field
- Saves the Service Manager record

This example requires the following sample data:

- A valid contact name (for example, "ADMINISTRATOR, SYSTEM")
- A valid field value update (for example, set the "user.type" field to "site")

```
var contactName;
function updateContact( name )
{
print( "Searching for contact: " + name + "..." );
var contactList = new SCFile( "contacts" );
var isContact = contactList.doSelect( "contact.name=\""+ name + "\"" );
if ( isContact == RC_SUCCESS )
{
print( "Success. found " + name + " in contact record:\n" + contactList.getText() );
print( "The current contents of the user.type field are: " + contactList.user_type );
print( "Updating the user.type field..." );
contactList.user_type = "site";
print( "Saving record..." );
var rc = contactList.doSave();
print( "The return code value for the doSave() method is: " + RCtoString( rc ) );
print( "The contents of the user.type field are now: " + contactList.user_type );
return contactList
}
else
{
print( "Could not find contact. " + RCtoString( isContact ) );
return null
}
}
contactName = "ADMINISTRATOR, SYSTEM";
updateContact( contactName );
```

2.5.3.4.11. JavaScript method: SCFile.doSelect()

This method queries for records in an Service Manager object. The SCFile object has the following format: `table={["field value",{"array value","array value"},...]}`

For performance improvement, you can use **SCFile.setFields** method to limit the fields that **doSelect** returns from the database.



Note

Note This method is used for retrieving normal data, not for retrieving attachments. If you use this method to retrieve attachments, you could corrupt the SYSATTACHMENTS table. If you need to retrieve attachments, use one of the following functions to get the attachments associated with the current file record: **SCFile.getAttachment(attachID)** or **SCFile.getAtttachments()**.

Syntax

```
SCFile_object.doSelect( query );
```

or

```
SCFile_object.doSelect( query, breferencesort );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`query`	String	This argument specifies the query you want to use to search for Service Manager records. You must use the slash character to escape out quotation marks and any special characters restricted from JavaScript.
`breferencesort`	Boolean	When this parameter is set to `true`, doSelect will check the specified sort sequences of reference field. When the parameter is set to `false`, it is equal to `SCFile_object.doSelect( query );`. **Note:** If you want to sort a reference field, set this parameter to true.

Return values

An SCFile object containing records and `RC_SUCCESS` or one of the other global return code values.

The method returns an SCFile object containing the records matching the query and a global return code value of `RC_SUCCESS` or returns one of the error global return code values if the method can't return any records.

Example

This example does the following:

- Searches the contacts table for any contact name you define in the search variable
- Displays the contact record as a text string

This example requires the following sample data:

- A valid contact name (for example, "ADMINISTRATOR, SYSTEM")
- A invalid contact name (for example, "NOT A, CONTACT")

```
var contactName;
function findContactName( name )
{
print( "Searching for contact: " + name + "..." );
var contactList = new SCFile( "contacts" );
var findContact = contactList.doSelect( "contact.name=\""+ name + "\"" )
if ( findContact == RC_SUCCESS )
{
print( "Success. found " + name + " in contact record:\n" + contactList.getText() );
return contactList
}
else
{
print( "Could not find contact. " + RCtoString( findContact ) );
return null
}
}
contactName = "ADMINISTRATOR, SYSTEM";
findContactName( contactName );
contactName = "NOT A, CONTACT";
findContactName( contactName );
```

Related topics

JavaScript object: SCFile
JavaScript method: SCFile.setFields()

2.5.3.4.12. JavaScript method: SCFile.doSelectEx()

The **SCFile.doSelectEx()** method is designed to improve the system performance. You can restrict the data retrieval by specifying the start and count parameters. In comparison, the legacy **SCFile.doSelect()** method retrieves data from the beginning and may cause potential performance issue on large tables.

Syntax

```
SCFile_object.doSelectEx( query, start, count );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`query`	String	This argument specifies the query you want to use to search for Service Manager records. You must use the slash character to escape out quotation marks and any special characters restricted from JavaScript.
`start`	Number	This argument specifies the start record of query results.
`count`	Number	This argument specifies the get count from start.

Return values

An SCFile object containing records and `RC_SUCCESS` or one of the other global return code values.

The method returns an SCFile object containing the records matching the query and a global return code value of `RC_SUCCESS` or returns one of the error global return code values if the method can't return any records.

Example

This example does the following:

- Searches the device table from 1000 for 1 record

- Displays the device record as a text string

This example requires the following sample data:

- A device table records >=1001

```
var deviceFile = new SCFile( "device" );
var rc = deviceFile.doSelectEx( "true", 1000, 1 )
if ( rc == RC_SUCCESS )
{
print( "Success. found 1 record in device table:" + deviceFile);
}
else
{
print( "Could not find any record from device table" );
}
```

Related topics

JavaScript object: SCFile

2.5.3.4.13. JavaScript method: SCFile.doUpdate()

This method updates a Service Manager record with any new field values defined in a Service Manager file object. It requires defining a Service Manager file object and defining new values for fields.

Syntax

```
SCFile_object.doUpdate();
```

Arguments

There are no arguments for this method.

Return values

`RC_SUCCESS` or one of the other global return code values.

The method returns `RC_SUCCESS` if the method successfully updates a record or returns one of the error global return code values if the method can't update the record.

Example

This example does the following:

- Searches the contacts table for any contact name you define in the search variable
- Displays the contact record as a text string

This example requires the following sample data:

- A valid contact name (for example, "ADMINISTRATOR, SYSTEM")
- A valid field value update (for example, set the "user.type" field to "site")

```
var contactName;
function updateContact( name )
{
print( "Searching for contact: " + name + "..." );
var contactList = new SCFile( "contacts" );
var isContact = contactList.doSelect( "contact.name=\""+ name + "\"" );
if ( isContact == RC_SUCCESS )
{
print( "Success. found " + name + " in contact record:\n" + contactList.getText() );
print( "The current contents of the user.type field are " + contactList.user_type );
print( "Updating the user.type field..." );
contactList.user_type = "site";
contactList.doUpdate();
print( "The contents of the user.type field are now " + contactList.user_type );
return contactList
}
else
{
print( "Could not find contact. " + RCtoString( isContact ) );
return null
}
}
contactName = "ADMINISTRATOR, SYSTEM";
updateContact( contactName );
```

2.5.3.4.14. JavaScript method: SCFile.getAttachment(attachID)

Gets a specific attachment associated with the current file record.

Values passed to **getAttachment()** will normally be ones obtained from a previous **insertAttachment()**, **updateAttachment()**, or **getAttachments()** operation.

Before calling **getAttachment**, you must establish a current record using one of the positioning methods, such as **doSelect()**, **getNext()**, etc.

Syntax

```
SCFile_object.getAttachment( attachID );
```

Arguments

getAttachment() takes one argument, which is a string containing an attachment ID of the cid:xxxxx form.

Return values

If successful, **getAttachment()** returns an Attachment object. Otherwise, it returns `null`.

Example

This example does the following:

- Selects one record from the probsummary table.
- Gets a specific attachment associated with the current record

This example requires the following sample data:

- Record number IM10001 in the probsummary table

```
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
var attachmentObj = f.getAttachment( attachmentID );
```

2.5.3.4.15. JavaScript method: SCFile.getattachments()

This method gets all of the attachments associated with the current file record.

Before calling **getAttachments**, you must establish a current record using one of the positioning methods, such as **doSelect()**, **getNext()**, etc.

Syntax

```
SCFile_object.getAttachments()
```

Arguments

The **getAttachments** method takes no arguments.

Return values

If successful, **getAttachments()** returns an array of attachment objects. The array will be empty if there are no attachments for the record. The attachment objects retrieve here will only contain metadata. The Attachment.value will be `NULL`. To get each attachment's binary data, you will need to use **SCFile.getAttachment(attachID)** method where attachID contains the href property of the attachment object.

Example

This example does the following:

- Selects one record from the probsummary table
- Gets all of the attachments associated with the current record

This example requires the following sample data:

- Record number IM10001 in the probsummary table

```
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
var attachments = f.getAttachments();
print( attachments.length );
for ( var attachment in attachments )
{
print( attachments[ attachment ].name );
}
```

2.5.3.4.16. JavaScript method: SCFile.getBinary()

This JavaScript method returns the binary representation of a field in a Service Manager file object.

Syntax

```
SCFile_object.getBinary (fieldName);
```

Arguments

The following argument is valid for this method:

Argument	Data type	Description
`fieldName`	String	This argument specifies the name of a field from which you want to extract the binary data.

Return values

The method returns a JS object that contains the binary data of the field.

Example

```
var bb = new SCFile("bbtosysattachments");
var rc = bb.doSelect("true");
var attachmentObj= new Attachment();
attachmentObj.value = bb.getBinary("att_attachment")
attachmentObj.name = bb.att_filename;
attachmentObj.type = "excel";
```

2.5.3.4.17. JavaScript method: SCFile.getFirst()

This method returns the first record in a record list stored in an SCFile object.You must use one of the other SCFile methods to create a record list.

Syntax

```
SCFile_object.getFirst();
```

Arguments

There are no arguments for this method.

Return values

An SCFile object containing the first record and `RC_SUCCESS` or one of the other global return code values.

The method returns an SCFile object containing the first record in a record list and a global return code value of `RC_SUCCESS` or returns one of the error global return code values if the method can't find the first record.

Example

This example does the following:

- Queries the contacts table for any contact name you define in the search variable
- Displays the last record in the record list
- Displays the previous record in the record list
- Displays the first contact in the record list

This example requires the following sample data:

- A contact query that produces a record list (for example, "B")
- A contact query that produces a single record (for example, "F")

```
var contactQuery;
function findFirstContact( query )
{
print( "Searching for contacts starting with " + query + "..." );
var contactFile = new SCFile( "contacts" );
var findContact = contactFile.doSelect( "contact.name#\""+ query + "\"" );
if ( findContact == RC_SUCCESS )
{
print( "Success. Found contacts starting with " + query + "." );
var findLastRecord = contactFile.getLast();
print( "The last record is:\n" + contactFile );
var findPrevRecord = contactFile.getPrev();
print( "The previous record is:\n" + contactFile );
var findFirstRecord = contactFile.getFirst();
print( "The first record is:\n" + contactFile );
}
else
{
print( "Could not find contacts starting with " + query + ". " + RCtoString( findContact ) );
return null
}
}
contactQuery = "H";
findFirstContact( contactQuery );
```

2.5.3.4.18. JavaScript method: SCFile.getLast()

This method returns the last record in a record list stored in an SCFile object. You must use the other SCFile methods to create a record list.

Syntax

```
SCFile_object.getLast();
```

Arguments

There are no arguments for this method.

Return values

An SCFile object containing the last record and `RC_SUCCESS` or one of the other global return code values.

The method returns an SCFile object containing the last record in a record list and a global return code value of `RC_SUCCESS` or returns one of the error global return code values if the method can't find the last record.

Example

This example does the following:

- Queries the contacts table for any contact name you define in the search variable
- Displays the first contact in the record list
- Displays the last record in the record list

This example requires the following sample data:

- A contact query that produces a record list (for example, "B")
- A contact query that produces a single record (for example, "F")

```
var contactQuery;
function findLastContact( query )
{
print( "Searching for contacts starting with " + query + "..." );
var contactFile = new SCFile( "contacts" );
var findContact = contactFile.doSelect( "contact.name#\""+ query + "\"" );
if ( findContact == RC_SUCCESS )
{
print( "Success. Found contacts starting with " + query + ". The first record is:\n" + contactFile );
var findLastRecord = contactFile.getLast();
print( "The last record is:\n" + contactFile );
return contactFile
}
else
{
print( "Could not find contacts starting with " + query + ". " + RCtoString( findContact ) );
return null
}
}
contactQuery = "H";
findLastContact( contactQuery );
contactQuery = "F";
findLastContact( contactQuery );
```

2.5.3.4.19. JavaScript method: SCFile.getLastRC()

This method returns the most recent global return code value generated by the SCFile object. You can then use the **RCtoString** global method to convert the return code into a string.

Syntax

```
SCFile_object.getLastRC();
```

Arguments

There are no arguments for this method.

Return values

One of the global return code values.

The method returns the most recent global return code value generated by the SCFile object.

Example

This example does the following:

- Inserts a new record into a local variable
- Displays the return code of the insert operation

This example requires the following sample data:

- A valid record in a local variable (for example, `$L.file`)

```
system.vars.$L_file.doInsert();
var rc = getLastRC();
if ( rc == RC_SUCCESS )
{
print( "Insert of document succeeded" );
}
else if ( getLastRC() == false )
print( "Error " + RCtoString(rc) + " occurred trying to insert the record" );
```

2.5.3.4.20. JavaScript method: SCFile.getMessages()

Returns an array of messages generated by the Document Engine from a previous **SCFile.doAction()**.

Syntax

```
SCFile_object.getMessages();
```

Arguments

There are no arguments allowed for this method.

Return values

Returns an array of messages from previous **SCFile.doAction()** call.

Example

This example does the following:

- Uses the Document Engine add to create a new incident record from variable values.
- Shows a validation failed due to invalid assignment group.
- **SCFile.doAction()** returns 71, "validation failure".
- **SCFile.getMessages()** returns messages generated from the Document Engine.

```
var numberValue;
var callbackContact;
var categoryValue;
var assignmentValue;
var descriptionValue;
var actionType;
var resultionCodeValue;
var resolutionValue;
function insertIncident( num, name, cat, group, desc )
{
print( "Creating new incident record..." );
var newIncident = new SCFile( "probsummary" );
newIncident.number = num;
newIncident.callback_contact = name;
newIncident.category = cat;
newIncident.assignment = group;
newIncident.description = desc;
var rc = newIncident.doAction('add');
if ( rc == RC_SUCCESS )
{
print( "Success. Created new record " + newIncident.getText() );
return newIncident.number
}
else
{
print( "Could not create record. " + RCtoString( rc ) );
print( "Reason: "+ newIncident.getMessages() );
return null
}
}
numberValue = "IM11121"
callbackContact = "ADMINISTRATOR, SYSTEM";
categoryValue = "network";
assignmentValue = "HELPDESK";
descriptionValue = null;
interactionID = insertIncident( numberValue, callbackContact, categoryValue, assignmentValue, descriptionValue );
Output
         Category specified is invalid. Reason: Category specified is invalid. Could not create record. Validation
failed creating new incident record...
```

2.5.3.4.21. JavaScript method: SCFile.getNext()

This method returns the next record in a record list stored in an SCFile object. It requires the use of the other SCFile methods to create a record list.

Syntax

```
SCFile_object.getNext();
```

Arguments

There are no arguments for this method.

Return values

An SCFile object containing the next record and `RC_SUCCESS` or one of the other global return code values.

The method returns an SCFile object containing the next record in a record list and a global return code value of `RC_SUCCESS` or returns one of the error global return code values if the method can't find the next record.

Example

This example does the following:

- Queries the contacts table for any contact name you define in the search variable
- Displays the first contact in the record list
- Displays the next record in the record list

This example requires the following sample data:

- A contact query that produces a record list (for example, "B")
- A contact query that produces a single record (for example, "F")

```
var contactQuery;
function findNextContact( query )
{
print( "Searching for contacts starting with " + query + "..." );
var contactFile = new SCFile( "contacts" );
var findContact = contactFile.doSelect( "contact.name#\""+ query + "\"" );
if ( findContact == RC_SUCCESS )
{
print( "Success. Found contacts starting with " + query + ". The first record is:\n" + contactFile.getText() );
var findNextRecord = contactFile.getNext();
print( "The next record is:\n" + contactFile.getText() );
return contactFile
}
else
{
print( "Could not find contact " + query + ". " + RCtoString( findContact ) );
return null
}
}
contactQuery = "B";
findNextContact( contactQuery );
contactQuery = "F";
findNextContact( contactQuery );
```

2.5.3.4.22. JavaScript method: SCFile.getPrev()

This method returns the previous record in a record list stored in an SCFile object. You must use of the other SCFile methods to create a record list.

Syntax

```
SCFile_object.getPrev();
```

Arguments

There are no arguments for this method.

Return values

An SCFile object containing the previous record and `RC_SUCCESS` or one of the other global return code values.

The method returns an SCFile object containing the previous record in a record list and a global return code value of `RC_SUCCESS` or returns one of the error global return code values if the method can't find the previous record.

Example

This example does the following:

- Queries the contacts table for any contact name you define in the search variable
- Displays the first contact in the record list
- Displays the last record in the record list
- Displays the previous record in the record list

This example requires the following sample data:

- A contact query that produces a record list (for example, "H")

```
var contactQuery;
function findPrevContact( query )
{
print( "Searching for contacts starting with " + query + "..." );
var contactFile = new SCFile( "contacts" );
var findContact = contactFile.doSelect( "contact.name#\""+ query + "\"" );
if ( findContact == RC_SUCCESS )
{
print( "Success. Found contacts starting with " + query + ". The first record is:\n" + contactFile );
var findLastRecord = contactFile.getLast();
print( "The last record is:\n" + contactFile );
var findPrevRecord = contactFile.getPrev();
print( "The previous record is:\n" + contactFile );
}
else
{
print( "Could not find contacts starting with " + query + ". " + RCtoString( findContact ) );
return null
}
}
contactQuery = "H";
findPrevContact( contactQuery );
```

2.5.3.4.23. JavaScript method: SCFile.getPurgedRowCount()

This method enables you to retrieve the number of Service Manager records that were deleted from a table after invoking the **doPurge()** method.

Syntax

```
SCFile_object.getPurgedRowCount();
```

Arguments

There are no arguments for this method.

Return values

- If successful, **getPurgedRowCount()** returns the number of records deleted by the last successful **doPurge()** call (which returned `RC_SUCCESS`). This value can be 0 if no records were deleted.
- If **getPurgedRowCount()** is not able to retrieve the record count it will return `-1`.

The value returned by **getPurgedRowCount** is unaffected by any SCFile method call other than **doPurge**. It is also unaffected by a failed call to **doPurge** (returned `RC_ERROR`).

Example

This example does the following:

- Searches the kmknowledgebaseupdates table for records where docid="docid1"
- Deletes whatever it finds.

This example requires the following sample data:

- Records in kmknowledgebaseupdates table where docid = "docid1"

```
var fileName = "kmknowledgebaseupdates";
var recordList = new SCFile(fileName);
var deleteRC = recordList.doPurge("docid = \"docid1\"");
if ( deleteRC == RC_SUCCESS )
{
print ("doPurge succeeded");
var purgedCount = recordList.getPurgedRowCount();
if ( purgedCount >= 0 )
{
print(purgedCount + " records have been purged");
}
else
{
print("Could not retrieve purged record count");
}
}
else
{
print ("doPurge failed");
}
```

Related topics

JavaScript object: SCFile
JavaScript method: SCFile.doPurge()

2.5.3.4.24. JavaScript method: SCFile.insertattachment()

This method creates a new attachment associated with the current file record. Before calling **insertAttachment**, you must establish a current record using one of the positioning methods, such as **doSelect()**, **getNext()**, etc.

Syntax

```
SCFile_object.insertAttachment( attachObj );
```

Arguments

The **insertAttachment** method takes one argument, which is an asttachment object.

The Attachment object must be properly populated prior to calling **insertAttachment**.

Attachment objects can be obtained via Web Services calls or constructed by JavaScript code.

Required properties

Attachment.name: This is a string containing the name of the attachment, for example, foo.txt or foo.pdf.

Attachment.type: This is a string containing the MIME type of the attachment, such as text/plain or application/pdf.

Attachment.value: This is a binary string containing the actual attachment data. One way to obtain such data is by using the **readFile()** global method with the binary file format (**readFile(path,"b")**).

The "href" property of the attachment object should not be set for an insert operation. Any value supplied will be ignored. Attachment ID values are assigned by the server when attachments are inserted or updated.

Return values

If successful, insertAttachment() returns a string of the form cid:xxxxxx containing the attachment ID for the inserted attachment.

Example

This example does the following:

- Creates a new attachment object
- Selects one record from the probsummary table
- Inserts the attachment object to the current record

This example requires the following sample data:

- Record number IM10001 in the probsummary table

```
var attachmentObj = new Attachment();
attachmentObj.type   = "text/plain";
attachmentObj.name  = "MyAttachment.txt";
attachmentObj.value = "This is a test text attachment";
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
var attachmentID = f.insertAttachment( attachmentObj );
```

2.5.3.4.25. JavaScript method: SCFile.isRecord()

This is a deprecated method as of version 6.1 because it doesn't use the Service Manager-defined global return codes. It requires using the deprecated **File()** object with a query or query condition. The **isRecord()** method verifies that the query or query condition returns a Service Manager record.

Syntax

```
SCFile_object.isRecord();
```

Arguments

There are no arguments for this method.

Return values

A Boolean value: `1` (true) or `0` (false).

The method returns a Boolean value of 1 if the File object contains a Service Manager record or returns 0 if the method doesn't find a Service Manager record.

This method doesn't use the Service Manager-defined global return codes.

Example

This example does the following:

- Searches the probsummary table for any incident record you define in the search variable
- Displays the incident record object and the results of the **isRecord()** method

This example requires the following sample data:

- A valid incident record (for example, "IM1005")
- An invalid incident record (for example "incident")

```
var incidentQuery;
function findTypes( query )
{
print( "Searching for incident records starting with " + query + "..." );
var incidentFile = new File( "probsummary", "number#\""+ query + "\"" );
if ( incidentFile != null )
{
print( "Successfully allocated a File object");
if ( incidentFile.isRecord() )
{
print( "Success. Found incident records starting with " + query + ". The first record is:\n" + incidentFile );
return incidentFile
}
else
{
print( "Could not find incident records starting with " + query + "." );
return null
}
}
else
{
print( "Could not allocate a File object." );
return null
}
}
incidentQuery = "IM1005";
findTypes( incidentQuery );
incidentQuery = "incident";
findTypes( incidentQuery );
```

2.5.3.4.26. JavaScript method: SCFile.setBinary()

This method saves binary data to a field in a Service Manager file object.

Syntax

```
SCFile_object.setBinary (fieldName, binaryObj);
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`fieldName`	String	The name of a field in which you want to save the binary data.
`binaryObj`	String	The name of a JS object that contains the binary data.

Return values

`RC_SUCCES` S or one of the other global return code values. The method returns `RC_SUCCESS` if the method successfully saves binary data to a Service Manager file object or returns one of the error global return code values if the method can't save binary data to a Service Manager file object.

Example

```
var f = new SCFile("contacts");
rc = f.doSelect('contact.name#"AARON"');
var attachmentObj = f.getAttachments();
var b = new SCFile("blob1");
b.name = attachmentObj[0].name;
b.setBinary ("data",attachmentObj[0].value );
rc = b.doSave();
```

2.5.3.4.27. JavaScript method: SCFile.setFields()

This method causes any subsequent doSelect() calls to fetch only the specified fields. If setFields has not been called on a table, all fields are selected and a "SELECT *" is performed.

This method is supported for read-only objects. You can make an object read-only specifying the `SCFILE_READONLY` argument in the SCFile constructor.

Calling **setFields** with no parameters will reset any previously specified fields. Any subsequent calls to **doSelect()** will fetch all fields (a SELECT * will be performed).

> **Note**
>
> Note Do not reference fields in the object that have not been specified in the **setFields** method. If you reference field in the object that has not been specified in the **setFields** method, the field appears as "null" whether the database has data in that column or not.

Syntax

```
SCFile_object.setFields( field-array );
SCFile_object.setFields( field-string );
SCFile_object.setFields( );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`field-array`	Array	A javascript array of field name strings; for example, `setFields(new Array("contact.name", "user.id"))`.
`field-string`	String	A string of space-separated field names; for example, `setFields("contact.name user.id")`.

Return values

The method returns `RC_SUCCESS` if successful and `RC_ERROR` on failure.

The following conditions cause a setFields failure:

- Calling setFields on an uninitialized SCFile or a non read-only SCFile
- Calling setFields with an invalid argument type (not a string or an array of strings)
- Calling setFields with one (or more) invalid fields. (The method performs a basic validation on the field names to ensure they exist in the DBDICT.)

When the method fails, it has no effect, and the last successful call to setFields stays in effect. It issues a warning in the sm.log file.

Limitations

- The main unique key fields of the dbdict are always selected from the table (and for join tables, from each underlying table) even if not explicitly specified in setFields.
- Since only basic field validation is performed in the setFields method, there are certain situations when the call can be successful (returns `RC_SUCCESS`) but a "SELECT *" is still performed.
  This would be the case if:
  - All specified fields exist in the DBDICT but at least one field is an alias field, a structure name or an unmapped field (which are unsupported).
  - LDAP is the primary data source for the table. (LDAP mapped fields are supported as long as LDAP is not the primary data source.)

Example usage

This example does the following:

- Takes the prefix of a contact name and returns a list of contacts that start with the specified prefix.
- On failure, returns the message "Could not find contact."

This example requires the following sample data:

- a contacts table with some records

```
function findPrefix( prefix )
{
var contactList = new SCFile( "contacts", SCFILE_READONLY ); //added new parameter SCFILE_READONLY
var fields = new Array("contact.name", "user.id"); // build array of fields to be returned from DB
var rc = contactList.setFields( fields ); // call new setFields method to identify fields to query from DB
if ( rc != RC_SUCCESS )
{
print( "setFields() failed, will revert to SELECT * (see log for more info)" );
return null;
}
var findContact = contactList.doSelect( "contact.name # \"" + prefix + "\"" );
if ( findContact == RC_SUCCESS )
{
return contactList;
}
else
{
print( "Could not find contact. " + RCtoString( findContact ) );
return null;
}
}
```

Related topics

JavaScript object: SCFile
JavaScript method: SCFile.doSelect()

2.5.3.4.28. JavaScript method: SCFile.setOrderBy()

This method causes any subsequent **doSelect()** call on a SCFile object to fetch only the specified fields by the specified sort sequences of the SCFileObject. If **setOrderBy()** has not been called, a subsequent **doSelect()** will use the default sort sequence.

Syntax

```
SCFile_object.setOrderBy(sortFields, sortSeqs);
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`sortFields`	Array	A javascript array of field name string, for example, `newArray ("name","company")`.
`sortSeqs`	Array	A javascript array of sort sequenence specification. It is either a `SCFILE_ASC`, or `SCFILE_DSC`. For example, `new Array(SCFILE_ASC, SCFILE_DSC); SCFILE_ASC` is a constant for specifying sort by ascending order, and `SCFILE_DSC` for specifying sort by descending order.

Required properties

This method requires two parameters of the data type of array. The two arrays must be of the same length. Calling **setOrderBy()** with parameters of uneven length of arrays or non-array data results in failure return status.

Return values

Returns `RC_SUCCESS` if successful and `RC_ERROR` on failure.

The following conditions cause a **setOrderBy()** failure:

- Calling **setOrderBy()** on an uninitialized SCFile
- Calling **setOrderBy()** with an invalid argument type (not an array) for either argument.
- Calling **setOrderBy()** with wrong number of arguments.
- Calling **setOrderBy()** with two arrays of different length.
- Calling **setOrderBy()** with one (or more) invalid fields. (The method performs a basic validation on the field names to ensure they exist in the DBDICT.

A warning is written to SM.log file.

If the sort sequence specifications in the `sortSeq` array is of non-recognizable constant, `SCFILE_ASC` is assumed.

Limitations:

- Fields used in **setOrderBy()** should not be within an array in the dbdict.
- Fields from a join file should not be within a structure in the dbdict.

Example



Note

Note If you are running **setOrderBy()** on a single array, these samples show how to define a single sort sequence for descending sort order.

1. `var sortOrder = [SCFILE_DSC];`
2. `var sortOrder = new Array(1);`

`sortOrder[0] = SCFILE_DSC;`

This example does the following:

- Takes two fields, name and company, from the operator table, and sorts the company field by descending order, then the name field by ascending order.
- On failure returns a `RC_ERROR` status code.

This example requires the following sample data:

- Records in the operator table that have various company and name fields.

```
function runOrderByTest( table, fields, sortOrder)
{
var timeBefore = new Date();
var fCount = 0;
print( "doSelect select column OrderBy test : Start time is: " +  timeBefore );
var fRec = new SCFile( table);
var setOrderByRC = fRec.setOrderBy( fields, sortOrder);
if ( setOrderByRC != RC_SUCCESS )
{
print( "ERROR: setOrderBy failed, RC=" + RCtoString( setOrderByRC ) );
return RC_ERROR;
}
var selectRC = fRec.doSelect( "true" );
if ( selectRC != RC_SUCCESS )
{
print( "ERROR: doSelect OrderBy failed, RC=" + RCtoString( selectRC ) );
return RC_ERROR;
}
do
{
fCount++;
}
while (fRec.getNext() == RC_SUCCESS);
var timeAfter = new Date();
print( "doSelect select column Orderby test: table processed = " + table );
print( "doSelect select column Orderby test: fields processed = " + fields );
print( "doSelect select column Orderby test: sort order = " + sortOrder);
print( "doSelect select column Orderby test: count = " + fCount );
print( "doSelect select column Orderby test: End time is " + timeAfter );
print( "doSelect select column Orderby test: Elapsed time is " + (timeAfter - timeBefore) + " milliseconds" );
return RC_SUCCESS;
}
var table = "operator";
var fields = new Array("company", "name");
var sortOrder = new Array(SCFILE_DSC, SCFILE_ASC);
var testRC = runOrderByTest( table, fields, sortOrder);
```

Related topics

JavaScript object: SCFile
JavaScript method: SCFile.setFields()

2.5.3.4.29. JavaScript method: SCFile.setRecord()

Sets the SCFile field values from a given XML string.

Syntax

```
SCFile_object.setRecord();
```

Argument

setRecord("xmlstring");

Example

```
function updateContactPhoneFromXML( name, xmlstr )
{
print("Searching for contact: "+ name + "...");
var contactRecord = new SCFile("contacts");
var rc = contactRecord.doSelect( "contact.name=\""+ name + "\"" );
if ( rc == RC_SUCCESS)
{
print("found contact " + name + ", phone number:\n" + contactRecord.contact_phone.toString() );
contactRecord.setRecord(xmlstr);
contactRecord.doUpdate();
print("updated phone number in contact record:\n" + contactRecord.contact_phone.toString());
}
else
{
print("Could not find contact. " + RCtoString( rc ) );
}
}
var contactName = "ADMINISTRATOR, SYSTEM";
var xmlString = "<model name=\"contacts\" query=\"contact.name="ADMINISTRATOR, SYSTEM"\"><keys><contact.name
sctype=\"string\">ADMINISTRATOR, SYSTEM</contact.name></keys><instance recordid=\"ADMINISTRATOR, SYSTEM\"
uniquequery=\"contact.name="ADMINISTRATOR, SYSTEM"\"><contact.name mandatory=\"true\" type=\"string\">ADMINISTRATOR,
SYSTEM</contact.name><contact.phone type=\"string\">(335) 123-4567</contact.phone></instance></model>";
updateContactPhoneFromXML(contactName, xmlString);
```

Note

Note

- The input XML string should follow the Service Manager File schema.
- You can't use **setRecord()** to set values for files that contain binary data. For example,you can't use **setRecord()** for displaycache, code, SYSATTACHMENTS files.
- You can't use **setRecord()** to set dbdict records.

2.5.3.4.30. JavaScript method: SCFile.updateAttachment(attachObj)

This method updates a specific attachment associated with the current file record. The attachment object must contain a valid attachment ID in the "href" property of the attachment object, as well as valid values for the other required properties (name, type, and value). Before calling **updateAttachment**, you must establish a current record using one of the positioning methods, such as **doSelect()**, **getNext()**, etc.

Syntax

```
SCFile_object.updateAttachment( attachObj );
```

Arguments

updateAttachment() takes one argument, which is an Attachment object.

Return values

If successful, **updateAttachment()** returns a string of the form cid:xxxxxx containing the new attachment ID for the attachment. Otherwise, it returns `NULL`.

Example

This example does the following:

- Selects one record from the probsummary table
- Gets a specific attachment associated with the current record
- Updates the attachment

This example requires the following sample data:

- "IM10001" record in "probsummary" table
- An attachment associated with "IM10001" record

```
function createTestAttachment()
{
var attachmentObj = new Attachment();
attachmentObj.type  = "text/plain";
attachmentObj.name  = "MyAttachment.txt";
attachmentObj.value = "My text attachment";
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
var attachmentID = f.insertAttachment( attachmentObj );
return attachmentID;
}
var attachmentID = createTestAttachment();
var f = new SCFile( 'probsummary' );
var rc = f.doSelect( 'number = "IM10001"' );
var attachmentObj = f.getAttachment( attachmentID );
// Now update the attachment object
var newName = "NewName.text";
var newValue = "Updated attachment text value";
attachmentObj.name = newName;
attachmentObj.value = newValue;
// Do the update. This should result in a new attachment ID
var newID =  f.updateAttachment( attachmentObj );
```

2.5.3.5. JavaScript object: SCRecordList

The following JavaScript object is unique to Service Manager.

SCRecordList is an array of Datums that qualify with the query.

Constructor

```
new SCRecordList( <table name>, <QueryCond object>);
```

Arguments

The following arguments are valid for this object:

Argument	Data type	Description
table name	String	The name of the Service Manager table. For example, probsummary.
QueryCond	object	Service Manager JavaScript object QueryCond which defines the SC query to be executed.

Properties

None.

Methods

The following methods are valid for this object:

Defined method	Description
getCount()	This method returns the total number of records in the SCRecordList.
getPosition()	This method returns the last position in the record list that has been accessed. .

Example

This example does the following:

- lists the location's position in a record list

- counts the records in the list

This example requires the following sample data:

- valid location(s)
- valid record list with at least one record

```
var x = new SCRecordList( 'location', new QueryCond('location.name', LIKE, 'a'));
print("Total record in the list: " + x.getCount());
for (i in x ) {
print(x[i].location_name + " is the " + x.getPosition() + " record in the record list." );
}
```

JavaScript method: SCRecordList.getCount()[edit]

This method counts the total number of records in the SCRecordList.

Syntax

```
SCRecordList.getCount();
```

Arguments

There are no arguments for this method.

Return values

This method returns the total number of records in a record list.

Example

This example does the following:

- counts the records in the list

This example requires the following sample data:

- valid location(s)
- valid record list with at least one record

```
var x = new SCRecordList( 'location', new QueryCond('location.name', LIKE, 'a'));
print("Total record in the list: " + x.getCount());
```

JavaScript method: SCRecordList.getPosition()[edit]

This method returns the position of the last accessed record in the record list. If there are no records are in the SCRecordList, it returns -1. If there is at least one record and it has not been accessed, the position is set to the first element, which is position 0.

Syntax

```
SCRecordList.getPosition();
```

Arguments

There are no arguments for this method.

Return values

Last position of Datum accessed in record list.

Returns `-1` if there are no records in the list.

Returns `0` if list is not yet or the first Datum is the last Datum accessed and the list contains at least one record.

Example

This example does the following:

- lists the location's position in a record list
- counts the records in the list

This example requires the following sample data:

- valid location(s)
- valid record list with at least one record

```
var x = new SCRecordList( 'location', new QueryCond('location.name', LIKE, 'a'));
print("Total record in the list: " + x.getCount());
for (i in x ) {
print(x[i].location_name + " is the " + x.getPosition() + " record in the record list." );
}
```

Contents

- 1 JavaScript method: SCRecordList.getCount()
- 2 JavaScript method: SCRecordList.getPosition()

2.5.3.6. JavaScript object: XML

The following JavaScript object is unique to Service Manager.

The XML() constructor creates an empty XML object where you can store and manipulate XML documents.

This object's methods don't use the Service Manager-defined global return codes.

Constructor

```
new XML();
```

Arguments

None

Properties

None

Methods

The following methods are valid for this object:

Defined method	Description
setContent	Creates an XML document from a string or file.
getParentNode	Returns an XML object representing the parent node of the current node.
getFirstChildElement	Returns an XML object representing the first child node of the current node.
getNextSiblingElement	Returns an XML object representing the next node at the same level in the Document Object Model (DOM) tree as the current node.
getFirstAttribute	Returns an XML object representing the first attribute of the current node.
getNextAttribute	Returns an XML object representing the next attribute of the current node.
getNodeName	Returns a string representing the name of the current element or attribute.
getName	Returns a string representing the name of the current element or attribute.
getQualifiedName	Returns a string representing the name of the current element or attribute including any namespace value.
getPrefix	Returns a string representing the namespace value of the current element or attribute.
getNodeValue	Returns a string representing the value of the current element.
getValue	Returns a string representing the value of the current element.
setNodeValue	Adds or updates the value of the current element.
setValue	Adds or updates the value of the current element.
getNodeType	Evaluates an XML object and returns an integer representing the XML DOM type of the current element or attribute.
isDocumentElement	Returns true if the current element is the DOM document element.
getDocumentElement	Creates an XML object containing the document element and all child elements from an XML object or XML string.
createNode	Creates an XML node from an XML object.
appendNode	Inserts an XML node into an XML object.
importNode	Copies an XML node from one XML document to another.
addAttribute	Inserts an XML attribute and attribute value into the current element.
getAttributeNode	Returns an XML object containing an attribute node.
getAttributeValue	Returns the value of the target attribute.
setAttributeValue	Adds or updates an attribute value.

addElement	Inserts an XML element as a child of the current element.
setText	Adds or updates the value of the current element.
getText	Returns a string representing the value of the current element.
toXMLString	Converts an XML object into a valid XML string.
removeAttribute	emoves an XML attribute from the current element.

Example

This example does the following:

- Creates an XML object
- Sets the content of the XML object to a local XML file

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)

```
/* Instantiate an empty XML object */
var xmlObj = new XML();
/* Instantiate a variable containing the path to an XML file */
var xmlFile = "C:\\myxmldoc.xml";
/* Use setContent to make the XML object read and parse the XML file
*  If the setContent method fails, print an error message */
if ( ! xmlObj.setContent( xmlFile, true ) )
{
  print( "setContent failed. Unable to parse xml: " + xmlFile );
}
```

2.5.3.6.1. JavaScript method: XML.addAttribute()

This method inserts an XML attribute and attribute value into the current element. It requires you use the other XML get methods to navigate through an XML document.

Syntax

```
XML.addAttribute( AttributeName, AttributeValue );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`AttributeName`	String	This argument specifies the text string you want the script to use as the XML attribute name. The string argument must contain characters valid for an XML element (for example, the string can't include the characters < or >).
`AttributeValue`	String	This argument specifies the text string you want the script to use as the XML attribute value. This argument must contain characters valid for an XML element (for example, the string can't include the characters < or >).

Return values

An XML object or `null`.

If successful, the method returns the updated XML element. Otherwise, it returns `null`.

Example

This example adds a new attribute and attribute value to any element you select from an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test02.xml which contains `<document><parent1>value1</parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test02.xml", true );
var XMLSource;
var TargetNode;
var searchResults;
var printResults;
var AttributeName;
var AttributeValue;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function createNewAttribute( searchResult, targetNode, attrName, attrValue )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
print( "Adding new attribute " + attrName + " to element " + elementName );
searchResult.addAttribute( attrName, attrValue );
print( "The new object is:\n" + searchResult );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Set variable values for XMLFile and TargetNode. Run functions.  */
```

```
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent1"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
AttributeName = "id";
AttributeValue = "test";
createNewAttribute( searchResults, TargetNode, AttributeName, AttributeValue );
```

2.5.3.6.2. JavaScript method: XML.addElement()

This method inserts an XML element as a child of the current element. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.addElement( String );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
String	String	This argument specifies the text string you want the script to use as the XML element name. The string argument must contain characters valid for an XML element (for example, the string can't include the characters < or >).

Return values

An XML object or `null`.

The method returns an XML object containing the new element or returns null if the method can't add the element.

Example

This example adds a child element to any element you select in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test02.xml which contains `<document><parent1>value1</parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test02.xml", true );
var XMLSource;
var TargetNode;
var searchResults;
var printResults;
var NewNodeName;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function createNewNode( searchResult, targetNode, nodeName )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
print( "Adding new node " + nodeName + " to element " + elementName );
var newNode = searchResult.addElement( nodeName );
print( "The new object is:\n" + searchResult );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
```

```
TargetNode = "parent1"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
NewNodeName = "child8";
createNewNode( searchResults, TargetNode, NewNodeName );
```

2.5.3.6.3. JavaScript method: XML.appendNode()

This method inserts an XML node into an XML object. There must be an existing XML object with a document element. This method doesn't update the existing XML object.

Syntax

```
XML.appendNode( name );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`name`	String	This argument specifies the XML node name you want to append to the target XML object. The string argument must contain characters valid for XML (for example, the string can't include the characters < or > except as XML entities such as < and >).

Return values

An XML object or `null`.

The method returns an XML object containing the new node or returns `null` if the method can't append the node to the current target node.

Example

This example appends an XML node to an XML source object.

This example requires the following sample data:

- An XML object (for example, an object containing "<document> </document>")

```
var XMLObject = new XML;
var NewNodeName;
var NewNodeValue;
var NewNodeType;
var TargetNode;
XMLObject = XMLObject.setContent( "<document> </document>" );
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
function appendNewNode( type, name, value, xmlSource, target )
{
print( "The XML source object is: " + xmlSource);
print( "Creating node of type: " + type + " named: " + name + " with value: " + value );
var NewNode = xmlSource.createNode( type, name, value );
print( "Appending " + NewNode.toXMLString() + " to: " + target );
var FoundNode = findTargetElement( xmlSource, target );
var appendedNode = FoundNode.appendNode( NewNode );
print( "The new XML object is: " +  xmlSource );
}
NewNodeType = 1;
NewNodeName = "node";
NewNodeValue = "test";
TargetNode = "document";
appendNewNode( NewNodeType, NewNodeName, NewNodeValue, XMLObject, TargetNode );
```

2.5.3.6.4. JavaScript method: XML.createNode()

This method creates an XML node from an XML object. It requires an existing XML object with a document element, but it doesn't update the existing XML object. The new method requires a rootname.

Syntax

```
XML.createNode( type, name, value );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`type`	Integer	This argument specifies the XML DOM node type for the node you want the script to create. This method only accepts XML DOM types 1, 2, and 3. See the W3C Web site for a complete listing of XML DOM node-types by integer.
`name`	String	This argument specifies the text string you want the script to add as the XML node name. The string argument must contain characters valid for XML (for example, the string can't include the characters < or > except as XML entities such as < and >).
`value`	String	This argument specifies the text string you want the script to add as the XML node value. The string argument must contain characters valid for XML (for example, the string can't include the characters < or > except as XML entities such as < and >).

Return values

An XML object or `null`.

The method returns an XML object containing the new node or returns `null` if the method can't create the node.

Example

This example creates an XML node from an XML source object.

This example requires the following sample data:

- An XML object (for example, an object containing "<document> </document>")

```
var XMLObject = new XML("document");
var NewNodeName;
var NewNodeValue;
var NewNodeType;
XMLObject = XMLObject.setContent( "<document> </document>" );
function createNewNode( type, name, value, xmlSource )
{
print( "The XML source object is: " + xmlSource);
print( "Creating node of type: " + type + " named: " + name + " with value: " + value );
var NewNode = xmlSource.createNode( type, name, value );
print( "The XML source object remains unchanged: " + XMLObject );
print( "The new node converted to an XML string is: " + NewNode.toXMLString() );
print( "Applying the getName method to the new node produces: " + NewNode.getName() );
print( "Applying the getValue method on the new node produces: " + NewNode.getValue() );
}
NewNodeType = 1;
NewNodeName = "node";
NewNodeValue = "test";
createNewNode( NewNodeType, NewNodeName, NewNodeValue, XMLObject );
NewNodeType = 2;
NewNodeName = "id";
NewNodeValue = "test";
createNewNode( NewNodeType, NewNodeName, NewNodeValue, XMLObject );
NewNodeType = 3;
NewNodeName = null;
NewNodeValue = "Text node";
createNewNode( NewNodeType, NewNodeName, NewNodeValue, XMLObject );
```

2.5.3.6.5. JavaScript method: XML.getAttributeNode()

This method returns an XML object containing the target attribute, or returns null if the method can't find the target attribute in the current element. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.getAttributeNode( Attribute );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`Attribute`	String	This argument specifies the XML attribute you want the script to return.

Return values

An XML object or `null`.

The method returns an XML object containing the target attribute, or returns `null` if the method can't find the target attribute in the current element.

Example

This example searches for a particular attribute of any given element in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which containings `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLFile to store a valid XML file.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var XMLSource;
var TargetNode;
var TargetAttribute;
var searchResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult, attribute )
{
print( "The target element we are looking for is: " + target );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
var attributeFound = findAttributeNode( searchResult, attribute );
return attributeFound
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the attribute node of the target element */
function findAttributeNode( sourceObject, attribute )
{
print( "Looking for attribute node " + attribute );
var attributeNode = sourceObject.getAttributeNode( attribute );
if ( attributeNode != null )
{
```

```
print( "Success. Found attribute " + attributeNode );
return attributeNode
}
else
{
print( "Fail. Did not find attribute " + attribute );
return null
}
}
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "child1";
XMLSource = XMLFile;
TargetAttribute = "attribute1";
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults, TargetAttribute );
```

2.5.3.6.6. JavaScript method: XML.getAttributeValue()

This method returns the value of the target attribute. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.getAttributeValue( Attribute );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`Attribute`	String	This argument specifies the XML attribute whose value you want the script to return.

Return values

A string or `null`.

The method returns the string value of the target attribute or returns `null` if the method can't find an attribute value.

Example

This example searches for the attribute value of any given element in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLFile to store a valid XML file.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var XMLSource;
var TargetNode;
var TargetAttribute;
var searchResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult, attribute )
{
print( "The target element we are looking for is: " + target );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
var attributeFound = findAttributeValue( searchResult, attribute );
return attributeFound
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the attribute node of the target element */
function findAttributeValue( sourceObject, attribute )
{
print( "Looking for attribute value of " + attribute );
var attributeValue = sourceObject.getAttributeValue( attribute );
if ( attributeValue != null )
{
```

```
print( "Success. The value of " + attribute + " is " + attributeValue );
return attributeValue
}
else
{
print( "Fail. Did not find attribute " + attribute );
return null
}
}
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "child1";
XMLSource = XMLFile;
TargetAttribute = "attribute1";
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults, TargetAttribute );
```

2.5.3.6.7. JavaScript method: XML.getDocumentElement()

This method searches an XML object for the DOM document element (or "root" node). It creates an XML object containing the document element and all child elements from an XML object or XML string.



Note

Note You can't use this method on objects that you have converted using the **toXMLString** method.

Syntax

```
XML.getDocumentElement();
```

Arguments

There are no arguments for this method.

Return values

An XML object or null.

If successful, this method returns an XML object containing the document element and all of its child elements. This method returns null if applied to an empty or invalid XML object.

Example

This example displays the XML document element from four different input sources.

This example requires the following sample data:

- A valid XML object (for example, the list of currently logged on users stored in system.users)
- A valid XML string (for example, `<document><parent1><child1 attribute1=\"1\" /></parent1><parent2 /></document>`)
- A noncompliant XML string (for example, `<document1><parent1 /></document1><document2><parent2 /></document2>`)
- An empty XML object

```
/* Create variables to store XML objects and strings.
*  Use goodXML to store a valid XML string.
*  Use badXML to store any erroneous XML string, in this case a second document element.
*  Use xmlObject to store a valid XML object such as the system.users list
*  The script sets the content of xmlString to goodXML.
*  The script sets the content of xmlBad to badXML.
*  The script sets the content of xmlEmpty to an XML object. */
var goodXML = "<document><parent1><child1 attribute1=\"1\" /></parent1><parent2 /></document>"
var badXML = "<document1><parent1 /></document1><document2><parent2 /></document2>"
var xmlObject = system.users
var xmlString = new XML();
xmlString = xmlString.setContent( goodXML );
var xmlBad = new XML();
xmlBad = xmlBad.setContent( badXML );
print( "The value of xmlBad is:\n" + xmlBad );
var xmlEmpty = new XML();
/* Create a function to execute the getDocumentElement method on the document argument. */
function getDocElem( document )
{
var DocElem = document.getDocumentElement();
if ( DocElem != null )
{
var DocElemName = DocElem.getNodeName();
print( "Success. Found the document element " + DocElemName + " in the following object: \n" + document );
return DocElem
}
else
{
print( "Error. Did not find document element in the following object: \n" + document );
return null
}
}
/* Run the getDocElem and getDocElemName functions on four input sources.
*  Print the objects returned by each function. */
print( "Now searching for a document element in a valid XML object...\n" );
getDocElem( xmlObject );
print( "Now searching for a document element in a valid XML string...\n" );
getDocElem( xmlString );
print( "Now searching for a document element in a non-compliant XML string...\n" );
getDocElem( xmlBad );
print( "Now searching for a document element in an empty XML object...\n" );
getDocElem( xmlEmpty );
```

2.5.3.6.8. JavaScript method: XML.getFirstAttribute()

This method returns an XML object representing the first attribute of the current node.

Syntax

```
XML.getFirstAttribute();
getFirstAttribute(Element)
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`Element`	String	This argument contains the name of the element you want the method to use as the starting position when searching for the first attribute.

Return values

An XML object or `null`.

The method returns an object representing the first attribute of the current node or returns `null` if the XML object has no attributes.

Example

This example displays the first attribute of any element you select. The example allows you to select an element from two different input sources.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- A valid XML string (for example, `<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var sourceXMLString = "<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>";
var XMLString = new XML();
XMLString = XMLString.setContent( sourceXMLString );
var XMLSource;
var TargetNode;
var searchResults;
var attributeResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult )
{
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the first attribute of the target element */
function findFirstAttribute( sourceObject, targetNode )
{
var startingNode = findTargetElement( sourceObject, targetNode )
```

```
var attribute = startingNode.getFirstAttribute();
if ( attribute != null )
{
return attribute
}
else
{
return null
}
}
/* Create a function to print the first attribute of the target element */
function printAttributeResults( target, searchResult )
{
if ( searchResult != null )
{
var attributeName = searchResult.getNodeName();
print( "The first attribute of " + target + " is: " + attributeName );
}
else
{
print( "There are no attributes of " + target );
}
}
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "child1"
XMLSource = XMLFile
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
attributeResults = findFirstAttribute( XMLSource, TargetNode );
printAttributeResults( TargetNode, attributeResults );
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLString... \n" );
TargetNode = "child2"
XMLSource = XMLString
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
attributeResults = findFirstAttribute( XMLSource, TargetNode );
printAttributeResults( TargetNode, attributeResults );
```

2.5.3.6.9. JavaScript method: XML.getFirstChildElement()

This method returns an XML object representing the first child node of the current node.

Syntax

```
XML.getFirstChildElement();
getFirstChildElement(Element)
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`Element`	String	This argument contains the name of the element you want the method to use as the starting position when searching for the first child element.

Return values

An XML object or `null`.

The method returns an object representing the first child node of the current node or returns `null` if the XML object has no child node.

Example

This example displays the first child element of any element you select. The example allows you to select an element from two different input sources.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- A valid XML string (for example, `<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var sourceXMLString = "<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>";
var XMLString = new XML();
XMLString = XMLString.setContent( sourceXMLString );
var XMLSource;
var TargetNode;
var searchResults;
var childResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult )
{
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the first child element of the target element */
function findFirstChild( sourceObject, targetNode )
{
var startingNode = findTargetElement( sourceObject, targetNode )
```

```
var firstChild = startingNode.getFirstChildElement();
if ( firstChild != null )
{
return firstChild
}
else
{
return null
}
}
/* Create a function to print the first child element of the target element */
function printChildResults( target, searchResult )
{
if ( searchResult != null )
{
var childName = searchResult.getNodeName();
print( "The first child element of " + target + " is: " + childName );
}
else
{
print( "There are no child elements of " + target );
}
}
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent1"
XMLSource = XMLFile
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
childResults = findFirstChild( XMLSource, TargetNode );
printChildResults( TargetNode, childResults );
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLString... \n" );
TargetNode = "parent2"
XMLSource = XMLString
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
childResults = findFirstChild( XMLSource, TargetNode );
printChildResults( TargetNode, childResults );
```

2.5.3.6.10. JavaScript method: XML.getName()

This method returns a string representing the name of the current element or attribute. You must use the other XML get methods to navigate through an XML document. This method is a synonym for **getNodeName();**.

Syntax

```
XML.getName();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string representing the name of the current element or attribute. It returns `null` if the current node has no name.

Example

This example displays the names of the elements and attributes in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)

- A valid XML string (for example, `<document><parent1><child1>0123</child1></parent1></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var sourceXMLString = "<document><parent1><child1>0123</child1></parent1></document>";
var XMLString = new XML();
XMLString = XMLString.setContent( sourceXMLString );
var XMLSource;
function walkXML( elem )
{
print( "Element " + elem.getName() );
var node = elem.getFirstAttribute();
while( node != null )
{
var attrName  = node.getName();
var attrValue = node.getNodeValue();
print( "Attribute " + attrName + " = " + attrValue );
node = elem.getNextAttribute();
}
var nodeValue = elem.getNodeValue();
if ( typeof nodeValue == "string" && nodeValue.length > 0 )
{
print( "Element value of " + elem.getName() + " is: " + nodeValue );
}
var child = elem.getFirstChildElement();
while( child != null )
{
var nodeName  = child.getName();
var nodeValue = child.getNodeValue();
walkXML( child );
child = child.getNextSiblingElement();
}
}
print( "Now printing the structure of an XML file...\n" );
XMLSource = XMLFile
walkXML( XMLSource );
print( "Now printing the structure of an XML string...\n" );
XMLSource = XMLString
walkXML( XMLSource );
```

2.5.3.6.11. JavaScript method: XML.getNextAttribute()

This method returns an XML object representing the next attribute of the current node.

Syntax

```
XML.getNextAttribute();
getNextAttribute(Element)
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`Element`	String	This argument contains the name of the element you want the method to use as the starting position when searching for the first attribute.

Return values

An XML object or `null`.

The method returns an object representing the first attribute of the current node or returns `null` if the XML object has no attributes.

Example

This example displays the first attribute of any element you select. The example allows you to select an element from two different input sources.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- A valid XML string (for example, `<document><parent1><child1 /><child2 attribute2=\"b\" attribute3=\"c\" /></parent1><parent2 /></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var sourceXMLString = "<document><parent1><child1 /><child2 attribute2=\"b\" attribute3=\"c\" /></parent1><parent2
/></document>";
var XMLString = new XML();
XMLString = XMLString.setContent( sourceXMLString );
var XMLSource;
var TargetNode;
var searchResults;
var attributeResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult )
{
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the first attribute of the target element */
function findFirstAttribute( sourceObject, targetNode )
{
```

```
var startingNode = findTargetElement( sourceObject, targetNode )
var firstAttribute = startingNode.getFirstAttribute();
if ( firstAttribute != null )
{
return firstAttribute
}
else
{
return null
}
}
/* Create a function to print the first attribute of the target element */
function printAttributeResults( target, searchResult )
{
if ( searchResult != null )
{
var attributeName = searchResult.getNodeName();
print( "The first attribute of " + target + " is: " + attributeName );
return searchResult
}
else
{
print( "There are no attributes of " + target );
return null
}
}
/* Create a function to find the next attribute of the target element */
function findNextAttribute( sourceObject, targetNode )
{
var startingNode = findTargetElement( sourceObject, targetNode )
var firstAttribute = startingNode.getFirstAttribute();
if ( firstAttribute != null )
{
var nextAttribute = startingNode.getNextAttribute();
while ( nextAttribute != null )
{
var attributeName = nextAttribute.getNodeName();
print( "The next attribute of " + targetNode + " is: " + attributeName );
nextAttribute = nextAttribute.getNextAttribute();
}
print( "There are no more attributes of " + targetNode );
return targetNode
}
else
{
return null
}
}
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "child1"
XMLSource = XMLFile
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
attributeResults = findFirstAttribute( XMLSource, TargetNode );
printAttributeResults( TargetNode, attributeResults );
findNextAttribute( XMLSource, TargetNode );
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLString... \n" );
TargetNode = "child2"
XMLSource = XMLString
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
```

```
attributeResults = findFirstAttribute( XMLSource, TargetNode );
printAttributeResults( TargetNode, attributeResults );
findNextAttribute( XMLSource, TargetNode );
```

2.5.3.6.12. JavaScript method: XML.getNextSiblingElement()

This method returns an XML object representing the next node at the same level in the Document Object Model (DOM) tree as the current node.

Syntax

```
XML.getNextSiblingElement();
getNextSiblingElement(Element)
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`Element`	String	This argument contains the name of the element you want the method to use as the starting position when searching for the next sibling element.

Return values

An XML object or `null`.

The method returns an object representing the next node at the same level in the DOM tree as the current node or returns `null` if the XML object has no sibling node.

Example

This example displays the next sibling element of any element you select. The example allows you to select an element from two different input sources.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- A valid XML string (for example, `<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var sourceXMLString = "<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>";
var XMLString = new XML();
XMLString = XMLString.setContent( sourceXMLString );
var XMLSource;
var TargetNode;
var searchResults;
var siblingResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult )
{
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the next sibling element of the target element */
function findNextSibling( sourceObject, targetNode )
{
var startingNode = findTargetElement( sourceObject, targetNode )
```

```
var nextSibling = startingNode.getNextSiblingElement();
if ( nextSibling != null )
{
return nextSibling
}
else
{
return null
}
}
/* Create a function to print the first attribute of the target element */
function printSiblingResults( target, searchResult )
{
if ( searchResult != null )
{
var childName = searchResult.getNodeName();
print( "The next sibling element of " + target + " is: " + childName );
}
else
{
print( "There are no sibling elements of " + target );
}
}
/* Set variables to search the XMLFile for the TargetNode.
*  Run functions findTargetElement, printFindResults, findAttribute, and
*  printAttributeResults on the XMLFile and TargetNode */
print( "Processing XML from XMLFile... \n" );
TargetNode = "child3"
XMLSource = XMLFile
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
siblingResults = findNextSibling( XMLSource, TargetNode );
printSiblingResults( TargetNode, siblingResults );
/* Set variables to search the XMLString for the TargetNode.
*  Run functions findTargetElement, printFindResults, findAttribute, and
*  printAttributeResults on the XMLString and TargetNode */
print( "Processing XML from XMLString... \n" );
TargetNode = "child1"
XMLSource = XMLString
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
siblingResults = findNextSibling( XMLSource, TargetNode );
printSiblingResults( TargetNode, siblingResults );
```

2.5.3.6.13. JavaScript method: XML.getNodeName()

This method returns a string representing the name of the current element or attribute. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.getNodeName();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string representing the name of the current element or attribute or returns `null` if the current node has no name.

Example

This example displays the names of the elements and attributes in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- A valid XML string (for example, `<document><parent1><child1>0123</child1></parent1></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var sourceXMLString = "<document><parent1><child1>0123</child1></parent1></document>";
var XMLString = new XML();
XMLString = XMLString.setContent( sourceXMLString );
var XMLSource;
function walkXML( elem )
{
print( "Element " + elem.getNodeName() );
var node = elem.getFirstAttribute();
while( node != null )
{
var attrName  = node.getNodeName();
var attrValue = node.getNodeValue();
print( "Attribute " + attrName + " = " + attrValue );
node = elem.getNextAttribute();
}
var nodeValue = elem.getNodeValue();
if ( typeof nodeValue == "string" && nodeValue.length > 0 )
{
print( "Element value of " + elem.getNodeName() + " is: " + nodeValue );
}
var child = elem.getFirstChildElement();
while( child != null )
{
var nodeName  = child.getNodeName();
var nodeValue = child.getNodeValue();
walkXML( child );
child = child.getNextSiblingElement();
}
}
print( "Now printing the structure of an XML file...\n" );
XMLSource = XMLFile
walkXML( XMLSource );
print( "Now printing the structure of an XML string...\n" );
XMLSource = XMLString
walkXML( XMLSource );
```

2.5.3.6.14. JavaScript method: XML.getNodeType()

This method evaluates an XML object and returns an integer representing the XML DOM type of the current element or attribute. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.getNodeType();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns an integer representing the XML DOM type of the current element or attribute or returns null if the method can't determine the XML DOM type. See the W3C Web site for a complete listing of XML DOM node types by integer. It returns a TypeError if applied to anything other than an XML object. This limits this method to elements and attributes since the Service Manager XML object can't represent element values as an XML object.

Example

This example displays the XML DOM types of the elements and attributes in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1>value1<child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)

```
function visitElement( elem )
{
print( "Element " + elem.getQualifiedName() + " is of type " + elem.getNodeType() );
/* Print attributes for this element */
var node = elem.getFirstAttribute();
while( node != null )
{
var attrName  = node.getQualifiedName();
var attrValue = node.getNodeValue();
print( "Attribute " + attrName + "=" + attrValue + " and is of type " + node.getNodeType() );
node = elem.getNextAttribute();
}
/* Print possible element value */
var nodeValue = elem.getNodeValue();
if ( typeof nodeValue == "string" && nodeValue.length > 0 )
{
print( "Element value is " + nodeValue + " and is of type " + elem.getNodeType() );
}
/* Now print child elements */
var child = elem.getFirstChildElement();
while( child != null )
{
var nodeName  = child.getNodeName();
var nodeValue = child.getNodeValue();
visitElement( child );
child = child.getNextSiblingElement();
}
}
var xmlObj =  new XML();
xmlObj.setContent( "c:\\test.xml", true );
visitElement( xmlObj.getDocumentElement() );
```

2.5.3.6.15. JavaScript method: XML.getNodeValue()

This method returns a string representing the value of the current element. It requires you use the other XML get methods to navigate through an XML document.

Syntax

```
XML.getNodeValue();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string representing the value of the current XML element or returns `null` if the current node has no value.

Example

This example displays the value of the elements in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test02.xml which contains `<document><parent1>value1</parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test02.xml", true );
var XMLSource;
var TargetNode;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( searchResult, targetNode )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
findValue( searchResult, targetNode );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the parent node element of the target element */
function findValue( sourceObject, targetNode )
{
var valueFound = sourceObject.getNodeValue();
if ( valueFound.length >= 1 )
{
print( "The value of " + targetNode + " is: " + valueFound );
```

```
return valueFound
}
else
{
print( "There is no value for " + targetNode );
valueFound = null
return valueFound
}
}
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent1"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent2"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
```

2.5.3.6.16. JavaScript method: XML.getParentNode()

This method returns an XML object representing the parent node of the current node.

Syntax

```
XML.getParentNode();
getParentNode(Element)
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`Node`	String	This argument contains the name of the element you want the method to use as the starting position when searching for the parent node element.

Return values

An XML object or `null`.

The method returns an object representing the parent node of the current node or returns `null` if the XML object has no parent node.

Example

This example displays the parent node element of any element you select. The example allows you to select an element from two different input sources.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- A valid XML string (for example, `<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var sourceXMLString = "<document><parent1><child1 /><child2 attribute2=\"b\" /></parent1><parent2 /></document>";
var XMLString = new XML();
XMLString = XMLString.setContent( sourceXMLString );
var XMLSource;
var TargetNode;
var searchResults;
var parentResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult )
{
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the parent node of the target element */
function findParent( sourceObject, targetNode )
{
var startingNode = findTargetElement( sourceObject, targetNode )
```

```
var parentNode = startingNode.getParentNode();
if ( parentNode != null )
{
return parentNode
}
else
{
return null
}
}
/* Create a function to print the first attribute of the target element */
function printParentResults( target, searchResult )
{
if ( searchResult != null )
{
var parentName = searchResult.getNodeName();
print( "The parent node element of " + target + " is: " + parentName );
}
else
{
print( "There is no parent node elements of " + target );
}
}
/* Set variables to search the XMLFile for the TargetNode.
*  Run functions findTargetElement, printFindResults, findAttribute, and
*  printAttributeResults on the XMLFile and TargetNode */
print( "Processing XML from XMLFile... \n" );
TargetNode = "child3"
XMLSource = XMLFile
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
parentResults = findParent( XMLSource, TargetNode );
printParentResults( TargetNode, parentResults );
/* Set variables to search the XMLString for the TargetNode.
*  Run functions findTargetElement, printFindResults, findAttribute, and
*  printAttributeResults on the XMLString and TargetNode */
print( "Processing XML from XMLString... \n" );
TargetNode = "child1"
XMLSource = XMLString
print( "The target element we are looking for is: " + TargetNode );
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults );
parentResults = findParent( XMLSource, TargetNode );
printParentResults( TargetNode, parentResults );
```

2.5.3.6.17. JavaScript method: XML.getPrefix()

This method returns a string representing the namespace value of the current element or attribute. It requires you use the other XML get methods to navigate through an XML document.

Syntax

```
XML.getPrefix();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string representing the namespace value for the current node (element or attribute) or returns `null` if the current node has no namespace value.

Example

This example displays the namespace value of the elements and attributes in an XML document.

This example requires the following sample data:

- A local XML file (for example, a file called "C:\namespace.xml" containing <?xml version="1.0" encoding="UTF-8" standalone="yes" ?><document xmlns:http="http://schemas.xmlsoap.org/wsdl/http/%22 xmlns:ns="http://servicemanager.hp.com/PWS%22><http:element1 /><ns:element2 /></document>

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\namespace.xml", true );
var XMLSource;
var TargetNode;
var searchResults;
var prefixResults;
var firstAttribute;
var attributeResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( searchResult, targetNode )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
findPrefix( searchResult, targetNode );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the parent node element of the target element */
function findPrefix( sourceObject, targetNode )
{
```

```
var prefixFound = sourceObject.getPrefix();
if ( prefixFound.length >= 1 )
{
print( "The namespace prefix of " + targetNode + " is: " + prefixFound );
return prefixFound
}
else
{
print( "There is no namespace prefix of " + targetNode );
prefixFound = null
return prefixFound
}
}
/* Create a function to find the first attribute of the target element */
function findFirstAttribute( sourceObject, targetNode )
{
var startingNode = findTargetElement( sourceObject, targetNode )
var firstAttribute = startingNode.getFirstAttribute();
if ( firstAttribute != null )
{
var attributeName = firstAttribute.getNodeName();
print( "The first attribute of " + targetNode + " is: " + attributeName );
findPrefix( firstAttribute, targetNode );
return firstAttribute
}
else
{
print( "There are no attributes of " + targetNode );
return null
}
}
/* Create a function to find the next attribute of the target element */
function findNextAttribute( sourceObject, targetNode )
{
var startingNode = findTargetElement( sourceObject, targetNode )
var firstAttribute = startingNode.getFirstAttribute();
if ( firstAttribute != null )
{
var nextAttribute = startingNode.getNextAttribute();
while ( nextAttribute != null )
{
var attributeName = nextAttribute.getNodeName();
print( "The next attribute of " + targetNode + " is: " + attributeName );
findPrefix( nextAttribute, targetNode );
nextAttribute = nextAttribute.getNextAttribute();
}
print( "There are no more attributes of " + targetNode );
return targetNode
}
else
{
return null
}
}
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "element1"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "document"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
```

```
printFindResults( searchResults, TargetNode );
findFirstAttribute( XMLSource, TargetNode );
findNextAttribute( XMLSource, TargetNode );
```

2.5.3.6.18. JavaScript method: XML.getQualifiedName()

This method returns a string representing the name of the current element or attribute, including any namespace value. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.getQualifiedName();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string representing the name of the current element or attribute. It returns `null` if the current node has no name.

Example

This example displays the qualified names of the elements and attributes in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\namespace.xml which contains `<?xml version="1.0" encoding="UTF-8" standalone="yes" ?><document xmlns:http="http://schemas.xmlsoap.org/wsdl/http/%22 xmlns:ns="http://servicemanager.hp.com/PWS%22><http:element1 /><ns:element2 /></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\namespace.xml", true );
var XMLSource;
function nsXMLWalk( elem )
{
print( "Element " + elem.getQualifiedName() );
var node = elem.getFirstAttribute();
while( node != null )
{
var attrName  = node.getQualifiedName();
var attrValue = node.getNodeValue();
print( "Attribute " + attrName + " = " + attrValue );
node = elem.getNextAttribute();
}
var nodeValue = elem.getNodeValue();
if ( typeof nodeValue == "string" && nodeValue.length > 0 )
{
print( "Element value of " + elem.getQualifiedName() + " is: " + nodeValue );
}
var child = elem.getFirstChildElement();
while( child != null )
{
var nodeName  = child.getQualifiedName();
var nodeValue = child.getNodeValue();
nsXMLWalk( child );
child = child.getNextSiblingElement();
}
}
function walkXML( elem )
{
print( "Element " + elem.getName() );
var node = elem.getFirstAttribute();
while( node != null )
{
var attrName  = node.getName();
var attrValue = node.getNodeValue();
print( "Attribute " + attrName + " = " + attrValue );
node = elem.getNextAttribute();
}
var nodeValue = elem.getNodeValue();
if ( typeof nodeValue == "string" && nodeValue.length > 0 )
{
print( "Element value of " + elem.getName() + " is: " + nodeValue );
}
var child = elem.getFirstChildElement();
while( child != null )
{
var nodeName  = child.getName();
var nodeValue = child.getNodeValue();
walkXML( child );
child = child.getNextSiblingElement();
}
}
print( "Now printing the structure of an XML file with qualified names...\n" );
XMLSource = XMLFile
nsXMLWalk( XMLSource );
print( "Now printing the structure of an XML file without qualified names...\n" );
XMLSource = XMLFile
walkXML( XMLSource );
```

2.5.3.6.19. JavaScript method: XML.getText()

This method returns a string representing the value of the current element. You must use the other XML get methods to navigate through an XML document. This method is a synonym for **getNodeValue();**.

Syntax

```
XML.getText();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string representing the value of the current XML element or returns `null` if the current node has no value.

Example

This example displays the value of the elements in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test02.xml which contains `<document><parent1>value1</parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test02.xml", true );
var XMLSource;
var TargetNode;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( searchResult, targetNode )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
findValue( searchResult, targetNode );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the parent node element of the target element */
function findValue( sourceObject, targetNode )
{
var valueFound = sourceObject.getText();
if ( valueFound.length >= 1 )
{
print( "The value of " + targetNode + " is: " + valueFound );
```

```
return valueFound
}
else
{
print( "There is no value for " + targetNode );
valueFound = null
return valueFound
}
}
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent1"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent2"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
```

2.5.3.6.20. JavaScript method: XML.getValue()

This method returns a string representing the value of the current element. You must use the other XML get methods to navigate through an XML document. This method is a synonym for **getNodeValue();**.

Syntax

```
XML.getValue();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string representing the value of the current XML element or returns `null` if the current node has no value.

Example

This example displays the value of the elements in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test02.xml which contains `<document><parent1>value1</parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test02.xml", true );
var XMLSource;
var TargetNode;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( searchResult, targetNode )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
findValue( searchResult, targetNode );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the parent node element of the target element */
function findValue( sourceObject, targetNode )
{
var valueFound = sourceObject.getValue();
if ( valueFound.length >= 1 )
{
print( "The value of " + targetNode + " is: " + valueFound );
```

```
return valueFound
}
else
{
print( "There is no value for " + targetNode );
valueFound = null
return valueFound
}
}
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent1"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent2"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
```

2.5.3.6.21. JavaScript method: XML.importNode()

This method copies an XML node from one XML document to another. It requires two existing XML objects with document elements: one is the source XML object and the other is the target XML document. This method doesn't update either the source or the target XML object. Instead, you must use the appendNode method to add the imported node into the target XML document.

Syntax

```
XML.importNode( name );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`name`	String	This argument specifies the XML node name you want to copy from the source XML object. The string argument must contain characters valid for XML (for example, the string can't include the characters < or > except as XML entities such as < and >).

Return values

An XML object or `null`.

The method returns an XML object containing the copied node or returns `null` if the method can't import a copy of the node.

Example

This example imports an XML node from one XML object to another.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- An XML object (for example, an object containing `<document> </document>`)

```
var XMLFile = new XML();
var XMLObject = new XML();
var TargetNode;
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult )
{
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
function importNewNode( xmlSource, xmlTarget, node )
{
print( "The XML source object is:\n" + xmlSource );
print( "The target element we are looking for is: " + node );
var FoundNode = findTargetElement( xmlSource, node );
printFindResults( node, FoundNode );
print( "The XML target object is:\n" + xmlTarget );
print( "Importing " + FoundNode + " into XML target object..." );
var NewNode = xmlTarget.importNode( FoundNode );
print( "The XML target object remains unchanged: " +  xmlTarget );
print( "Appending " + FoundNode + " into XML target object...");
var appendedNode = xmlTarget.appendNode( NewNode );
print( "The new XML target object is: " +  xmlTarget );
}
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
XMLObject = XMLObject.setContent( "<document> </document>" );
TargetNode = "parent1";
importNewNode( XMLFile, XMLObject, TargetNode );
```

2.5.3.6.22. JavaScript method: XML.isDocumentElement()

This method returns true if the current element is the Document Object Model (DOM) document element (the "root" node of the XML document). You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.isDocumentElement();
```

Arguments

There are no arguments for this method.

Return values

A Boolean value: `true` or `false`.

The method returns `true` if the current XML element is the DOM document element or `false` if the current XML element is not the DOM document.

Example

This example checks to see if the selected element is the DOM document element.

This example requires the following sample data:

- A local XML file (for example, C:\test02.xml which contains `<document><parent1>value1</parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test02.xml", true );
var XMLSource;
var TargetNode;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( searchResult, targetNode )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
testDocElem( searchResult);
return searchResult
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to find the parent node element of the target element */
function testDocElem( sourceObject )
{
var docElement = sourceObject.isDocumentElement();
if ( docElement == true )
{
var nodeName = sourceObject.getName();
```

```
print( "Success. The element " + nodeName + " is the DOM document element." );
return valueFound
}
else
{
var nodeName = sourceObject.getName();
print( "Fail. The element " + nodeName + " is not the DOM document element." );
valueFound = null
return valueFound
}
}
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "parent1"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
/* Set variable values for XMLFile and TargetNode. Run functions.  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "document"
XMLSource = XMLFile
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( searchResults, TargetNode );
```

2.5.3.6.23. JavaScript method: XML.removeAttribute()

This method removes an XML attribute from the current element. It requires that you use other XML get methods to navigate through an XML document.

Syntax

```
XML.removeAttribute( AttributeName );
```

Arguments

The following argument is valid for this method.

Argument	Data type	Description
AttributeName	String	This argument specifies the text string you want the script to use as the XML attribute name. The string argument must contain characters valid for an XML element(for example, the string can't include the characters< or >).

Return values

An XML object or null.

If successful, the method returns the updated XML element. Otherwise, it returns null.

2.5.3.6.24. JavaScript method: XML.setAttributeValue()

This method adds or updates the attribute value of the target attribute. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.setAttributeValue( AttributeName, AttributeValue );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`AttributeName`	String	This argument specifies the XML attribute whose value you want the script to add or update.
AttributeValue	String	This argument specifies the text string you want the script to use as the XML attribute value. This argument must contain characters valid for an XML element (for example, the string can't include the characters < or >).

Return values

This method doesn't return any values.

Example

This example adds or updates the attribute value of any given element in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLFile to store a valid XML file.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test.xml", true );
var XMLSource;
var TargetNode;
var TargetAttribute;
var TargetValue;
var searchResults;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function printFindResults( target, searchResult, attributeName, attributeValue )
{
print( "The target element we are looking for is: " + target );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
var attributeSet = setAttributeValue( searchResult, attributeName, attributeValue );
print( "The XML object is " + searchResult );
return attributeSet
}
else
{
print( "Cannot find " + target );
return null
}
}
/* Create a function to set the attribute value of the target attribute */
function setAttributeValue( sourceObject, attributeName, attributeValue )
{
print( "Setting attribute value of " + attributeName );
var setAttributeValue = sourceObject.setAttributeValue( attributeName, attributeValue );
```

```
var newAttributeValue = sourceObject.getAttributeValue( attributeName );
print( "The value of " + attributeName + " is now " + newAttributeValue );
return newAttributeValue
}
/* Set variable values for XMLFile and TargetNode. Run functions  */
print( "Processing XML from XMLFile... \n" );
TargetNode = "child1";
XMLSource = XMLFile;
TargetAttribute = "attribute1";
TargetValue = "z";
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults, TargetAttribute, TargetValue );
print( "Processing XML from XMLFile... \n" );
TargetNode = "child2";
XMLSource = XMLFile;
TargetAttribute = "attribute1";
TargetValue = "z";
searchResults = findTargetElement( XMLSource, TargetNode );
printFindResults( TargetNode, searchResults, TargetAttribute, TargetValue );
```

2.5.3.6.25. JavaScript method: XML.setContent()

This method creates an XML object from a string or the contents of an external file. It always creates a valid XML document with a document element. The method removes any values outside the document element.



Note

Note This method returns null if applied to an existing XML object because the header information ([C++ object XML] @nnnnnnnnn -) is invalid.

Syntax

```
XML.setContent( String ); setContent( "FilePath", IsFile );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
String	String	This argument specifies the text string you want the script to parse as on XML object. The string argument must contain a valid XML document with a document element.
FilePath	String	This argument specifies the path to the XML document you want the script to parse an XML object. You must enclose the FilePath argument in quotation marks and use the true argument to indicate that the first argument is a file path and not a string value.
IsFile	Boolean	This argument specifies whether the first argument is a string or a file path. A true value indicates the first argument is a file path. Any other value, including omitting the second argument, indicates that the first argument is a string.

Return values

An XML object or `null`.

The method returns an XML object if it parses the XML input and returns `null` if it can't parse the input.

Arguments

Use the **toXMLString** method to convert an XML object to an XML string.

Example

This example attempts to create an XML object from five different sources.

This example requires the following sample data:

- A valid XML string (for example, `<document><parent1><child1 attribute1=\"1\" /></parent1><parent2 /></document>`)
- An XML string with more than one document element (for example, `<document1><parent1 /></document1><document2><parent2 /></document2>`)
- A local XML file (for example, C:\test.xml which contains `<document><parent1><child1 attribute1="a" /><child2 /></parent1><parent2><child3 /><child4 /></parent2></document>`)
- An existing XML object (for example, the list of currently logged on users stored in system.users)
- An XML object converted to a string (for example, the list of currently logged on users stored in system.users)

```
/* Create a function to execute the setContent method.
*  The function creates a new XML object xmlObj.
*  It sets the content of the XML object to the source argument.
*  The isFile argument determines if the source is an XML string or file path.
*  If setContent returns true, it prints an XML string and returns the XML object.
*  If setContent returns false, it prints the source and returns null. */
function setXML( source, isFile )
{
var xmlObj = new XML();
if ( xmlObj.setContent( source, isFile ) )
{
var xmlDoc = xmlObj.toXMLString();
print( "setContent succeeded! \n The XML document is: \n" + xmlDoc );
return xmlObj
}
else
{
print( "setContent failed. \n Could not parse the following input as XML: \n" + source );
return null
}
}
/* Create variables to store your XML source objects.
*  Use xmlObject to test setting content to an XML object.
*  xmlObjectConverted converts xmlObject to an XML string.
*  Note: The toXMLstring method removes the object header information.
*  Use xmlString to test setting content to an XML string.
*  Note: You must use the backslash character to escape out quotation marks in your XML string.
*  Use xmlFile to test setting content to an XML file. */
var xmlString = "<document><parent1><child1 attribute1=\"1\" /></parent1><parent2 /></document>";
var xmlBad = "<document1><parent1 /></document1><document2><parent2 /></document2>"
var xmlFile = "C:\\test.xml";
var xmlObject = system.users
var xmlObjectConverted = xmlObject.toXMLString();
/* Note: You can omit the isFile argument when the content source is an XML string.
*  See the xmlString example below. */
print( "Testing setting content to a valid XML string...\n" );
setXML( xmlString );
print( "Testing setting content to an XML string with more than one document element...\n" );
print( "The non-compliant XML string before conversion is: \n" +  xmlBad );
setXML( xmlBad );
print( "Testing setting content to an XML file...\n" );
setXML( xmlFile, true );
print( "Testing setting content to an existing XML object...\n" );
setXML( xmlObject, false );
print( "Testing setting content to an existing XML object converted to an XML string...\n" );
setXML( xmlObjectConverted, false );
```

2.5.3.6.26. JavaScript method: XML.setNodeValue()

This method adds or updates the value of the current element. It requires you use the other XML get methods to navigate through an XML document.

Syntax

```
XML.setNodeValue( String );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`String`	String	This argument specifies the text string you want the script to add as the XML element value. The string argument must contain characters valid for XML (for example, the string can't include the characters < or > except as XML entities such as < and >).

Return values

An XML object or `null`.

The method returns an XML object containing the new value of the current XML element or returns `null` if the method can't set a value for the element.

Example

This example sets the value of the elements in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test03.xml which contains `<document><parent1></parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test03.xml", true );
var TargetNode;
var TargetValue;
var findElement;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function setElementValue( searchResult, targetNode, newValue )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
searchResult.setNodeValue( newValue );
print( "The new value of " + targetNode + " is: " + searchResult );
return searchResult
}
else
{
print( "Cannot find " + targetNode );
return null
}
}
TargetNode = "parent1";
TargetValue = "1234";
findElement = findTargetElement( XMLFile, TargetNode );
setElementValue( findElement, TargetNode, TargetValue );
```

2.5.3.6.27. JavaScript method: XML.setText()

This method adds or updates the value of the current element. You must use the other XML get methods to navigate through an XML document. This method is a synonym for **setNodeValue();**.

Syntax

```
XML.setText( String );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`String`	String	This argument specifies the text string you want the script to add as the XML element value. This argument must contain characters valid for XML (for example, the string can't include the characters < or > except as XML entities such as < and >).

Return values

An XML object or `null`.

The method returns an XML object containing the new value of the current XML element or returns `null` if the method can't set a value for the element.

Example

This example sets the value of the elements in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test03.xml which contains `<document><parent1></parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test03.xml", true );
var TargetNode;
var TargetValue;
var findElement;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function setElementValue( searchResult, targetNode, newValue )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
searchResult.setText( newValue );
print( "The new value of " + targetNode + " is: " + searchResult );
return searchResult
}
else
{
print( "Cannot find " + targetNode );
return null
}
}
TargetNode = "parent1";
TargetValue = "1234";
findElement = findTargetElement( XMLFile, TargetNode );
setElementValue( findElement, TargetNode, TargetValue );
```

2.5.3.6.28. JavaScript method: XML.setValue()

This method adds or updates the value of the current element. You must use the other XML get methods to navigate through an XML document. This method is a synonym for **setNodeValue();**.

Syntax

```
XML.setValue( String );
```

Arguments

The following arguments are valid for this method:

Argument	Data type	Description
`String`	String	This argument specifies the text string you want the script to add as the XML element value. The string argument must contain characters valid for XML (for example, the string can't include the characters < or > except as XML entities such as < and >).

Return values

An XML object or `null`.

The method returns an XML object containing the new value of the current XML element or returns `null` if the method can't set a value for the element.

Example

This example sets the value of the elements in an XML document.

This example requires the following sample data:

- A local XML file (for example, C:\test03.xml which contains `<document><parent1></parent1><parent2></parent2></document>`)

```
/* Create variables to store XML objects and strings.
*  Use XMLObject to store a valid XML object such as an XML file.
*  Use sourceXMLString to store a valid XML string.
*  The script sets the content of XMLString to sourceXMLString.
*  The script assigns values to the empty variables later on */
var XMLFile = new XML();
XMLFile = XMLFile.setContent( "C:\\test03.xml", true );
var TargetNode;
var TargetValue;
var findElement;
/* Create a function that searches an XML object (node) for a particular
*  element (targetElem) and returns an XML object containing the element.
*  This function assumes that there is only one instance of the target
*  element in the source data. If it finds more than one element in the
*  data it only returns the first instance of the element. */
function findTargetElement( node, targetElem )
{
var topNodeName = node.getNodeName();
while (node != null && node.getNodeName() != targetElem )
{
var childNode = node.getFirstChildElement();
if (childNode == null)
{
childNode = node.getNextSiblingElement();
while (childNode == null)
{
node = node.getParentNode();
if ( node == null || topNodeName == node.getNodeName() )
{
return null;
}
childNode = node.getNextSiblingElement();
}
node = childNode;
}
else
{
node = childNode;
}
}
return node;
}
/* Create a function to print the target element and search results */
function setElementValue( searchResult, targetNode, newValue )
{
print( "The target element we are looking for is: " + targetNode );
if ( searchResult != null )
{
var elementName = searchResult.getNodeName();
print( "Found element " + elementName + " in: \n" + searchResult + "\n" );
searchResult.setValue( newValue );
print( "The new value of " + targetNode + " is: " + searchResult );
return searchResult
}
else
{
print( "Cannot find " + targetNode );
return null
}
}
TargetNode = "parent1";
TargetValue = "1234";
findElement = findTargetElement( XMLFile, TargetNode );
setElementValue( findElement, TargetNode, TargetValue );
```

2.5.3.6.29. JavaScript method: XML.toXMLString()

This method converts an XML object into a valid XML string. You must use the other XML get methods to navigate through an XML document.

Syntax

```
XML.toXMLString();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string containing valid XML or returns `null` if the current node is not an XML object.

Example

This example converts an XML object into a valid XML document.

This example requires the following sample data:

- An existing XML object (for example, the list of currently logged on users stored in system.users)

```
var xmlObject = system.users
print( "This is the original XML object:\n" + xmlObject );
var xmlString = xmlObject.toXMLString();
print( "This is the XML object after conversion to a string:\n" + xmlString );
```

2.5.3.6.30. JavaScript method: XMLDoc.insertBefore()

Inserts the specified node before a reference element as a child of the current node.

Syntax

```
var insertedElement = parentElement.insertBefore(newElement, referenceElement)
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
insertedElement	XML	Yes	The node being inserted, that's `newElement`.
parentElement	XML	Yes	The parent of the newly inserted node.
newElement	XML	Yes	The node to insert.
referenceElement	XML	Yes	The parent of the newly inserted node.

Usage Notes

1. The `referenceElement` may NOT be null. If `referenceElement` is null, `newElement` the function will fail.
2. The `newElement` must be a newly created element within the same document (created using importNode or createNode) which has not yet been attached to the tree, such as a newly created element for which appendNode has not been called. The result of invoking `insertBefore` using references to nodes already in the same DOM tree as `newElement` is undefined.

Example

```
var xmlDoc = new XML();
var someXML = "<doc> \
<one/> \
<two/> \
</doc>";
// Create an XML document from the literal XML string
// we pass "false" as the second argument to setContent() to indicate
// that the supplied string does NOT represent a path to an XML file
xmlDoc.setContent( someXML, false );
// Now add create and add new element <four> in one step.
// It will be added at the end, i.e. after <two>
var nodeFour = xmlDoc.addElement( "four" );
// Now insert a new element <three> in front of <four>
// We do this in two steps:
// a) create a new element node called <three>
// b) insert the new element node before the <four> element
// Note: Passing "1" as the first argument to createNode means create an element
var nodeThree = xmlDoc.createNode( 1, "three" );
// The newly created node has not yet been given a position in the tree.
// Just as in the case of a node created with "importNode", we must
// attach the new node somewhere. We can do that with appendNode or insertBefore
// To get the new node in between two sibling nodes (as in this example)
// we must use insertBefore. appendChild can only be used on a parent node,
// and if we were to append to the parent element <doc>, it would end up following <four>
xmlDoc.insertBefore( nodeThree, nodeFour );
print( xmlDoc );
```

2.5.3.7. JavaScript object: XMLDate

The following JavaScript object is unique to Service Manager.

With no arguments, the **XMLDate()** constructor creates an empty XMLDate object.

With a *DateObject* argument, the **XMLDate()** constructor converts a JavaScript date object to an XMLDate object.

With an *ISO8601DateTimeOrDurationString* argument, the **XMLDate()** constructor converts an XML schema date object to an XMLDate object.

With a *DateTimeDatum* argument, the **XMLDate()** constructor converts a Service Manager date object to an XMLDate object.

This object's methods use the Service Manager-defined global return codes.

Constructor

```
new XMLDate();
new XMLDate( DateObject );
new XMLDate( ISO8601DateTimeOrDurationString );
new XMLDate( DateTimeDatum );
```

Arguments

The following arguments are valid for this object:

Argument	Data type	Description
`DateObject`	JavaScript date object	Passes a JavaScript date object to the XMLDate constructor.
`ISO8601DateTimeOrDurationString`	XML schema date	Passes an XML schema date, time, datetime, or duration string to the XMLDate constructor.
`DateTimeDatum`	Service Manager Datum object	Passes a Service Manager datum object formatted as a TIME type to the XMLDate constructor.

Properties

None

Methods

The following methods are valid for this object:

Defined method	Description
getISODateTimeString	This method passes a JavaScript Date object to the constructor.
getISODate	This method returns the indicated portions of the XMLDate value.
getISOTime	This method returns the indicated portions of the XMLDate value.
getISOYear	This method returns the indicated portions of the XMLDate value.
getISOMonth	This method returns the indicated portions of the XMLDate value.
getISODay	This method returns the indicated portions of the XMLDate value.
getISODateTimeWithSMTZString	This method passes a JavaScript Date object to the constructor.
getDatum	This method returns an SCDatum object.
getSCDateTimeString	This method returns a datetime string in Service Manager format.
getGMTSCDateTimeString	This method returns the Greenwich Mean Time (GMT) datetime in Service Manager format.
toSCDuration	This method converts an ISO duration string to a Service Manager duration.
addDuration(iso8601durationstring)	This method adds the indicated duration to the contained value in the XMLDate object.
JSDate	This method returns a JavaScript date object.
getDate	This method returns a JavaScript date object (same as the JSDate method).

Example

This example does the following:

- Creates an SCFile object from the cm3r table
- Creates an XMLDate object
- Sets the value of two fields to the XMLdate

This example requires the following sample data:

- A valid duration

```
/* Instantiate a new SCFile object with records of the cm3r table */
var changeRequest = new SCFile("cm3r");
/* Set the Service Manager fields date.entered and planned.start to today's date.
*  The dots in Service Manager field names must be converted to underscores */
var theXMLDate = new XMLDate( new Date() );
var todaysDate = theXMLDate.getDatum();
changeRequest.header.date_entered = todaysDate;
changeRequest.header.planned_start = todaysDate;
/* Increment the value of the planned.end field to be a week later.
*  P is the addDuration parameter for duration
*  7D is the addDuration parameter for 7 days
*  See the addDuration method for additional information */
theXMLDate.addDuration('P7D');
var nextWeeksDate = theXMLDate.getDatum();
changeRequest.header.planned_end = nextWeeksDate;
changeRequest.doInsert();
```

2.5.3.7.1. JavaScript method: XMLDate.addDuration()

This method returns an XMLDate object containing the new date/time after adding the duration from the duration argument.

Syntax

```
XMLDate.addDuration( duration );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
duration	ISO 8601 duration string	Yes	This argument contains the ISO 8601 duration string or variable value you want the script to add to an XMLDate object.

Return values

An XMLDate object or `null`.

The method returns an XMLDate object or `null` if conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Adds 1 year, 2 months, 3 days, and 4 hours to the XMLDate object then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Adding 1 year, 2 months, 3 days, and 4 hours to date..." );
var dur = d.addDuration( "P1Y2M3DT4H" );
print( "The value of the new XMLDate object is: " + dur );
```

2.5.3.7.2. JavaScript method: XMLDate.getDate()

This method returns a JavaScript date/time object.

Syntax

```
XMLDate.getDate();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a JavaScript date/time object or `null` if conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to JavaScript date/time object then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to a JavaScript object..." );
var js = d.getDate();
print( "The JavaScript date/time object is: " + js );
```

2.5.3.7.3. JavaScript method: XMLDate.getDatum()

This method returns an Service Manager-formatted date/time Datum object.

Syntax

```
XMLDate.getDatum();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a Service Manager-formatted date/time Datum object or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to Service Manager-formatted date/time Datum object then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to a Service Manager Datum..." );
var sm = d.getDatum();
print( "The Service Manager Datum object is: " + sm );
```

2.5.3.7.4. JavaScript method: XMLDate.getGMTSCDateTimeString()

This method returns Service Manager-formatted date/time string in Greenwich Mean Time (GMT).

Syntax

```
XMLDate.getSCDateTimeString();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a Service Manager-formatted date/time string or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date

- Converts the XMLDate object to Service Manager-formatted date/time string then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to a Service Manager date/time string..." );
var sc = d.getGMTSCDateTimeString();
print( "The Service Manager GMT date/time string is: " + sc );
```

2.5.3.7.5. JavaScript method: XMLDate.getISODate()

This method returns a string containing the date portion of date/time object in ISO format.

Syntax

```
XMLDate.getISODate();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string containing the ISO date of a date/time object or `null` if conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to an ISO date then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to ISO format..." );
var ISO = d.getISODate();
print( "The ISO date is: " + ISO );
```

2.5.3.7.6. JavaScript method: XMLDate.getISODateTimeString()

This method returns a string containing the date and time portion of date/time object in ISO format.

Syntax

```
XMLDate.getISODateTimeString();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string containing the ISO date and time of a date/time object or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to an ISO date and time then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to ISO format..." );
var ISO = d.getISODateTimeString();
print( "The ISO date and time is: " + ISO );
```

2.5.3.7.7. JavaScript method: XMLDate.getISODateTimeWithSMTZString()

This method returns a string containing the date and time portion of date/time object in ISO format with the operator's timezone.

Syntax

```
XMLDate.getISODateTimeWithSMTZString();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string containing the ISO date and time of a date/time object or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to an ISO date and time then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to ISO format..." );
var ISO = d.getISODateTimeWithSMTZString();
print( "The ISO date and time is: " + ISO );
```

2.5.3.7.8. JavaScript method: XMLDate.getISODay()

This method returns a string containing the day portion of date/time object in ISO format.

Syntax

```
XMLDate.getISODay();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string containing the ISO day of a date/time object or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to an ISO day then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to ISO format..." );
var ISO = d.getISODay();
print( "The ISO day is: " + ISO );
```

2.5.3.7.9. JavaScript method: XMLDate.getISOMonth()

This method returns a string containing the month portion of date/time object in ISO format.

Syntax

```
XMLDate.getISOMonth();
```

Arguments

There are no arguments for this method.

Return values

A string or null.

The method returns a string containing the ISO month of a date/time object or null if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to an ISO month then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to ISO format..." );
var ISO = d.getISOMonth();
print( "The ISO month is: " + ISO );
```

2.5.3.7.10. JavaScript method: XMLDate.getISOTime()

This method returns a string containing the time portion of date/time object in ISO format.

Syntax

```
XMLDate.getISOTime();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string containing the ISO time of a date/time object or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to an ISO time then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to ISO format..." );
var ISO = d.getISOTime();
print( "The ISO time is: " + ISO );
```

2.5.3.7.11. JavaScript method: XMLDate.getISOYear()

This method returns a string containing the year portion of date/time object in ISO format.

Syntax

```
XMLDate.getISOYear();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a string containing the ISO year of a date/time object or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to an ISO year then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to ISO format..." );
var ISO = d.getISOYear();
print( "The ISO year is: " + ISO );
```

2.5.3.7.12. JavaScript method: XMLDate.getSCDateTimeString()

This method returns an Service Manager-formatted date/time string.

Syntax

```
XMLDate.getSCDateTimeString();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a Service Manager-formatted date/time string or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to Service Manager-formatted date/time string then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to a Service Manager date/time string..." );
var sc = d.getSCDateTimeString();
print( "The Service Manager date/time string is: " + sc );
```

2.5.3.7.13. JavaScript method: XMLDate.JSDate()

This method returns a JavaScript date/time object. This method is an alias of the **getDate()** method.

Syntax

```
XMLDate.JSDate();
```

Arguments

There are no arguments for this method.

Return values

A string or `null`.

The method returns a JavaScript date/time object or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing today's date
- Converts the XMLDate object to a JavaScript date/time object then displays the result

```
print( "Creating XMLDate object..." );
var d = new XMLDate( new Date() );
print( "The value of the new XMLDate object is: " + d );
print( "Converting the XMLDate to a JavaScript object..." );
var js = d.JSDate();
print( "The JavaScript date/time object is: " + js );
```

2.5.3.7.14. JavaScript method: XMLDate.toSCDuration()

This method converts an ISO 8601 duration string to an Service Manager duration string.

Syntax

```
XMLDate.toSCDuration( duration );
```

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
`duration`	ISO 8601 duration string	Yes	This argument contains the ISO 8601 duration string or variable value you want the script to convert to a Service Manager duration string.

Return values

A string or `null`.

The method returns a Service Manager duration string or `null` if the conversion fails.

Example

This example does the following:

- Creates a new XMLDate object containing a hard-coded ISO date
- Converts the ISO date/time to a Service Manager duration string

```
print( "Creating XMLDate object..." );
var d = new XMLDate( "P01DT12H01M01S" );
print( "The value of the new XMLDate object is: " + d );
print( "Converting " + d.toString() + " to a SC duration..." );
var smDur = d.toSCDuration();
print( "The Service Manager duration is: " + smDur );
```

2.5.4. List: JavaScript functions

The following table lists some of the JavaScript functions available in Service Manager. These functions provide a method of performing certain commands that will return a value when executed.

Refer to List: RAD functions for more information on using RAD functions to perform these commands instead of JavaScript.

JavaScript function	Description
add_graphnodes	Returns XML to the Run-Time Environment (RTE) for the expand action on a node in the graph diagram.
checkAttachmentLegal	Checks if an attachment is legal.
get_graph_action	Returns the activated action on a node in the graph diagram.
get_graph_id	Returns the value of the graph ID for the expand action on a node in the graph diagram.
get_graph_node_id	Returns the value of the node ID for the expand action on a node in the graph diagram.
get_graph_target	Returns the target for an activated action on a node in the graph diagram.

2.5.4.1. JavaScript function: add_graphnodes

This function returns an XML file to the Run-Time Environment (RTE) for the expand action on a node in the graph diagram.

Function

add_graphnodes()

Example

```
system.functions.add.graphnodes(String strXML)
```

2.5.4.2. JavaScript function: checkAttachmentLegal

Description

A JavaScript function that checks if an attachment is legal before added.

The legal standard is same as the system attachment validation methods such as the attachment allowlist and Tika.
See Support for allowing attachments with certain file extensions for more information.

Function

checkAttachmentLegal(attachmentobj);

Arguments

The following arguments are valid for this function:

Argument	Data type	Required	Description
attachmentobj	Attachment	Yes	The attachment object to check

Return values

The return code indicates whether the attachment is legal or not.

- RC_SUCCESS, legal
- RC_ERROR, illegal

Example

```
var attachObj = new Attachment();
attachObj.type  = "text/plain";
attachObj.name  = "MyAttachment.doc";
attachObj.value = "text";

results = checkAttachmentLegal(attachObj);
print(results)

output:
-1
```

2.5.4.3. JavaScript function: get_graph_action

Description

A JavaScript function that returns the activated action on a node in the graph diagram.

Function

get_graph_action()

Example

```
system.functions.get_graph_action()
```

2.5.4.4. JavaScript function: get_graph_id

Description

A JavaScript function that returns the value of the graph ID for the expand action on a node in the graph diagram.

Function

get_graph_id.htm()

Example

```
system.functions.get_graph_id.htm()
```

2.5.4.5. JavaScript function: get_graph_node_id

Description

A JavaScript function that returns the value of the node ID for the expand action on a node in the graph diagram.

Function

get_graph_node_id()

Example

```
system.functions.get_graph_node_id()
```

2.5.4.6. JavaScript function: get_graph_target

Description

A JavaScript function that returns the target for an activated action on a node in the graph diagram.

Function

get_graph_target

Example

```
system.functions.get_graph_target()
```

2.5.5. List: JavaScript functions in the ScriptLibrary

The following table lists some of the JavaScript functions available in the ScriptLibrary of Service Manager. These functions are part of system JavaScript files and should not be directly modified.

JavaScript functions in the ScriptLibrary	Description
addToGroup	Adds one or more configuration items (CIs) to a Configuration Management list group and updates the list group record if successful.
checkAllowedOption	Returns the next valid version number of a Configuration Management baseline group if there is only one valid version number available.
checkVersionString	Validates a version string to ensure it adheres to Configuration Management baseline group version number requirements.
isfileexist	Determines whether a file exists or not.
isInGroup	Searches for one configuration item (CI) in a Configuration Management list or query group.
removeFromGroup	Removes one or more configuration items (CI) from a Configuration Management list group.
returnGroupMembers	Returns an array containing the group members of a Configuration Management group.
returnMyGroups	Returns an array containing all the groups to which a configuration item (CI) belongs.
skipApproval	Prevents an item's approval status from being reset to "pending."

2.5.5.1. JavaScript function: addToGroup

This function adds one or more configuration items (CIs) to a Configuration Management list group and updates the list group record if successful. It is part of a system JavaScript and should not be directly modified.

Syntax

```
lib.ciGrouping.addToGroup( ci, groupName );
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
`ci`	Datum object or string	Yes	This argument contains the CI records you want to add. To add an array of CIs, the `ci` argument must be an SCFile or SCDatum object. To add a single CI, you can specify it as a string.
`groupName`	String	Yes	This argument contains the name of the configuration group to which you want to add CIs.

Return values

A numerical value: `0` or `1`.

The function returns `0` if it is unable to add CIs to the list group or if the *groupName* argument specifies a query group. The function returns `1` if the function successfully adds the CIs to the list group.

Example

This example attempts to add CIs to a list group from the following sources:

- The logical.name value from a hard-coded string
- The logical.name value from a configuration group record
- The logical.name values from an array of CI records
- The logical.name values from a record list of one or more CIs

This example requires the following sample data:

- Create a list group containing a single configuration item (for example, a list group called "test01" containing the configuration item "DEFAULT Phone 0001")
- Create a list group containing one or more configuration items (for example, a list group called "test02" containing the configuration item "DEFAULT Phone 0003")
- Create a list group containing one or more configuration items (for example, a list group called "test03" containing the configuration items "HH-000001" and "HH-000002")

```
/* Create variables to store CI group names, strings, and datum objects.
*  Use targetCIGroup to store the name of CI group you want to add records to.
*  Use sourceCIGroup to store the name of CI group you want to create an array from.
*  Use oneLogicalName to store a string containing a logical.name you want to add.
*  Use deviceType to store a device type you want to query for.
*  Use startsWith to store a alphabetical value you want to query for.
*  The variable recordTest stores the output of the findCIGroup function.
*  The variable arrayTest stores the output of the createArray function.
*  The variable recordListTest stores the output of the findRecordList function. */
var targetCIGroup;
var sourceCIGroup;
var oneLogicalName;
var deviceType;
var startsWith;
var recordTest;
var arrayTest;
var recordListTest;
/* Create a function to store a particular CI group in memory. */
function findCIGroup( ciGroup )
{
print( "Looking for CI Group " + ciGroup + "..." );
var members = lib.ciGrouping.returnGroupMembers( ciGroup )
var ciGroupRecord = new SCFile( "cigroup" );
var queryTest = ciGroupRecord.doSelect( "logical.name=\"" + ciGroup + "\"" )
if ( queryTest == RC_SUCCESS )
{
system.vars.$L_file = ciGroupRecord
print( "Found CI group record for " + ciGroup + "\nThe record is:\n" + ciGroupRecord );
print( "The current members of " + ciGroup + " are:\n" + members );
return ciGroupRecord
}
else
{
print( "Cannot find record for " + ciGroup + ". " +  RCtoString( queryTest ) + "." );
return null;
}
}
/* Create function to find a record list based on a device type and starting query */
function findRecordList( deviceType, query )
{
var ciRecordList = new SCFile( "device" );
if ( query != null )
{
print( "Looking for CIs matching type=\"" + deviceType + "\"\&" + "logical.name#\"" + query + "\"" );
var queryTest = ciRecordList.doSelect( "type=\"" + deviceType + "\"\&" + "logical.name#\"" + query + "\"" )
}
else
{
print( "Looking for CIs matching type=\"" + deviceType + "\"" );
var queryTest = ciRecordList.doSelect( "type=\"" + deviceType + "\"" )
}
if ( queryTest == RC_SUCCESS )
{
print( "Found the following CIs:\n" + ciRecordList );
system.vars.$L_file = ciRecordList
return ciRecordList;
}
else
{
if ( query != null )
{
print( "Cannot find records matching query: device=\""+ deviceType + "\"&logical.name#\"" + query + "\""
+  RCtoString( queryTest ) + "." );
return null;
}
```

```
else
{
print( "Cannot find records matching query: device=\""+ deviceType + "\"" +  RCtoString( queryTest ) + "." );
return null;
}
}
}
/* Create a function to store the logical.names of the record. */
function createArray( record )
{
var arrayOfLogicalNames = record.group_members;
if (arrayOfLogicalNames == null )
{
print( "Cannot create array from " + record + ". Group not in the cigroup table." );
return null
}
else
{
print( "Created array from " + record + ".\nThe array is:\n" + arrayOfLogicalNames );
return arrayOfLogicalNames
}
}
/* Create a function to call the addToGroup function and print results. */
function testAdd( ci, group )
{
var groupMembers = lib.ciGrouping.returnGroupMembers( group )
var AddCI = lib.ciGrouping.addToGroup( ci, group );
var afterAdd = lib.ciGrouping.returnGroupMembers( group );
print( "Now trying to add configuration item(s) to " + group + "..." );
print( "The current members of " + group + " are:\n" + groupMembers );
if ( AddCI == 1 )
{
print( "Successfully added configuration item(s) to " + group );
print( "The members of " + group + " are now: \n" + afterAdd );
}
else
{
print( "Could not add configuration item(s) to " + group );
print( "The members of " + group + "are : \n" + groupMembers );
}
}
/* Test adding a CI to a group from a string */
print( "Now testing adding a single CI from a string...\n" );
targetCIGroup = "test01";
oneLogicalName = "DEFAULT Phone 0002";
members = lib.ciGrouping.returnGroupMembers( targetCIGroup )
testAdd( oneLogicalName, targetCIGroup );
/* Test adding a CI to a group from an existing CI record */
print( "Now testing adding a CI member from a CI group record...\n" );
targetCIGroup = "test01";
sourceCIGroup = "test02";
recordTest = findCIGroup( sourceCIGroup );
if ( recordTest != null )
{
testAdd( system.vars.$L_file, targetCIGroup );
}
/* Test adding one or more CIs from an array of logical,names */
print( "Now testing adding one or more CIs from an array of logical names...\n" );
targetCIGroup = "test01";
sourceCIGroup = "test03";
recordTest = findCIGroup( sourceCIGroup );
if ( recordTest != null )
{
arrayTest = createArray( recordTest );
testAdd( arrayTest, targetCIGroup );
```

```
}
/* Test adding one or more CIs to a group from a CI record list */
print( "Now testing adding one or more CIs from a CI record list...\n" );
targetCIGroup = "test01";
deviceType = "furnishings"
startsWith = null;
recordListTest = findRecordList( deviceType, startsWith );
if ( recordListTest != null )
{
testAdd( system.vars.$L_file, targetCIGroup );
}
```

2.5.5.2. JavaScript function: checkAllowedOption

Returns the next valid version number of a Configuration Management baseline group if there is only one valid version number available.

Syntax

```
lib.versionControl.checkAllowedOption( oldVersion, maxIncrement );
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
`oldVersion`	String	Yes	This argument contains the previous version number.
`maxIncrement`	Array	Yes	This argument contains an array of numbers representing the decimal value of the maximum number increments possible. For example, if *maxIncrement* equals [0,1,0] then there are ten possible increments permissible from versions 0.0.1 to 0.1.0.

Return values

A valid version number or the string `more`.

The function returns a valid version number if there is one and only one valid next version number. The function returns the string `more` if there is more than one valid next version number.

Description

This function is part of a system JavaScript and should not be directly modified.

This function returns the next valid version number of a Configuration Management baseline group if there is one and only one valid version number available. If there is more than one valid next version number then the function returns the string `more`.

Example

This example attempts to create the next valid version from two variables.

This example requires the following sample data:

- A variable storing the old version number
- A variable storing the maximum number of increments allowed

```
/* Use oldVersion to store the old version number as a string.
*  Use maxIncrement to store the maximum number of increments as an array. */
var oldVersion;
var maxIncrement = new Array();
/* Create a function to test the checkAllowedOption function */
function nextVersionTest( versionOld, increment )
{
var nextVersion = lib.versionControl.checkAllowedOption( versionOld, increment );
if ( nextVersion == "more" )
{
print( "Error. There is more than one valid next version number for " + versionOld
               + ".\n");
return versionOld
}
else
{
print( "Success. The next valid version number is " + nextVersion + ".\n" );
return nextVersion
}
}
/* Test a valid version number update */
print( "Testing checkAllowedOption...\n" );
oldVersion = "1.0";
maxIncrement = [0,1];
print( "The current version number is " + oldVersion + "." );
print( "The maximum allowed increment is " + maxIncrement + ".");
nextVersionTest( oldVersion, maxIncrement );
/* Test a valid version number update */
print( "Testing checkAllowedOption...\n" );
oldVersion = "0.9.9";
maxIncrement = [0,0,1];
print( "The current version number is " + oldVersion + "." );
print( "The maximum allowed increment is " + maxIncrement + ".");
nextVersionTest( oldVersion, maxIncrement );
/* Test an invalid version number update */
print( "Testing error condition...\n" );
oldVersion = "1.0";
maxIncrement = [0,2];
print( "The current version number is " + oldVersion + "." );
print( "The maximum allowed increment is " + maxIncrement + ".");
nextVersionTest( oldVersion, maxIncrement );
```

2.5.5.3. JavaScript function: checkVersionString

Validates a version string to ensure it adheres to Configuration Management baseline group version number requirements.

Syntax

```
lib.versionControl.checkVersionString( oldVersion, newVersion, maxIncrement );
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
`oldVersion`	String	Yes	This argument contains the previous version number.
`newVersion`	String	Yes	This argument contains the new version number.
`maxIncrement`	Array	Yes	This argument contains an array of numbers representing the decimal value of the maximum number increments possible. For example, if `maxIncrement` equals [0,1,0] then there are ten possible increments permissible from versions 0.0.1 to 0.1.0.

Return values

An integer value: `0`, `1`, `-1`, `-2`, `-3`, or `-4`.

Return value	Condition
`0`	The function returns `0` if any version level value in the `newVersion` argument exceeds the value in the `maxIncrement` argument. For example, if `oldVersion` equals "1.0". and `maxIncrement` equals "2", then the function returns a value of `0` if `newVersion` equals "1.3" since going from a value of zero to three exceeds the `maxIncrement` value of 2.
`1`	The function returns `1` if it successfully validates the proposed change from the `oldVersion` to the `newVersion`.
`-1`	The function returns `-1` if the `newVersion` argument contains greater than five version levels. For example, the function returns `-1` if the `newVersion` argument equals "1.0.0.0.0.0" as this is a version with six version levels.
`-2`	The function returns `-2` if a version level value in the `newVersion` argument contains anything other than the digits 0 to 9. For example the function returns `-2` if the `newVersion` argument equals "gold" or "1.0a" because the version number contains alphabetical characters.
`-3`	The function returns `-3` if the number of version levels in the `oldVersion` argument don't match the number of version levels in the `newVersion` argument. For example, the function returns `-3` if `oldVersion` equals "1.1" and `newVersion` equals "1.1.1" since the `oldVersion` has two version levels and the `newVersion` has three.
`-4`	The function returns `-4` if the value of the `newVersion` argument is less than the value of the `oldVersion` argument. For example, the function returns `-4` if `oldVersion` equals "1.1" and `newVersion` equals "1.0" because the `newVersion` value is prior to the `oldVersion` value.

Description

This function is part of a system JavaScript and should not be directly modified.

This function validates a new Configuration Management baseline group version string against an old version string to ensure it adheres to version number requirements.

Example

This example attempts to validate any given version number update from three variables.

This example requires the following sample data:

- A variable storing the old version number
- A variable storing the desired new version number
- A variable storing the maximum number of increments allowed

```
/* Use oldVersion to store the old version number as a string.
*  Use newVersion to store the new version number as a string.
*  Use maxIncrement to store the maximum number of increments as an array. */
var oldVersion;
var newVersion;
var maxIncrement = new Array();
/* Create a function to test the checkVersionString function */
function versionTest( versionOld, versionNew, increment )
{
var validate = lib.versionControl.checkVersionString( versionOld, versionNew, increment );
print( "The checkVersionString return value is " +  validate );
if ( validate == 1 )
{
print( "Success. Updating " + versionOld + " to " + versionNew + " is a valid change.\n");
return versionNew
}
if ( validate == 0 )
{
print( "Error. Could not update " + versionOld + " to " + versionNew + "." );
print( "The new version " + versionNew + " exceeds the maximum version increment value of "
               + increment + "." );
return versionOld
}
if ( validate == "-1" )
{
print( "Error. Could not update " + versionOld + " to " + versionNew + ".");
print( "The new version " + versionNew + " contains more than five version levels." );
return versionOld
}
if ( validate == "-2" )
{
print( "Error. Could not update " + versionOld + " to " + versionNew + ".");
print( "The new version " + versionNew + " contains characters other than 0 to 9." );
return versionOld
}
if ( validate == "-3" )
{
print( "Error. Could not update " + versionOld + " to " + versionNew + ".");
print( "The new version " + versionNew
               + " contains a different number of version levels than the old version " + versionOld );
return versionOld
}
if ( validate == "-4" )
{
print( "Error. Could not update " + versionOld + " to " + versionNew + ".");
print( "The new version " + versionNew + " is an earlier version than the old version "
               + versionOld );
return versionOld
}
else
{
return versionOld
}
}
/* Test a valid version number update */
print( "Testing checkVersionString...\n" );
oldVersion = "1.0";
newVersion = "1.1";
maxIncrement = [0,2];
versionTest( oldVersion, newVersion, maxIncrement );
/* Test an invalid version number update */
print( "Testing error condition 0...\n" );
oldVersion = "1.0";
newVersion = "1.9";
maxIncrement = [0,2];
```

```
versionTest( oldVersion, newVersion, maxIncrement );
/* Test an invalid version number update */
print( "Testing error condition -1...\n" );
oldVersion = "1.0.0.0.0.0";
newVersion = "1.0.0.0.0.1";
maxIncrement = [0,0,1,1,2];
versionTest( oldVersion, newVersion, maxIncrement );
/* Test an invalid version number update */
print( "Testing error condition -2...\n" );
oldVersion = "1.0.0.0.0";
newVersion = "1.a.0.0.0";
maxIncrement = [0,0,1,1,2];
versionTest( oldVersion, newVersion, maxIncrement );
/* Test an invalid version number update */
print( "Testing error condition -3...\n" );
oldVersion = "1.0.0";
newVersion = "1.1";
maxIncrement = [0,2,2];
versionTest( oldVersion, newVersion, maxIncrement );
/* Test an invalid version number update */
print( "Testing error condition -4...\n" );
oldVersion = "1.2";
newVersion = "1.1";
maxIncrement = [0,2];
versionTest( oldVersion, newVersion, maxIncrement );
```

2.5.5.4. JavaScript function: isfileexist

Determines whether a file or a folder exists or not.

Syntax

```
isfileexist(“path”,type)
```

Arguments

The following argument is valid for this function.

Name	Data type	Required	Description
`path`	String	Yes	This argument is the full file path.
`type`	Boolean	Optional	If this argument is omitted, the value is set to 0 by default. • 0: determines whether a file or a folder exists • 1: determines whether a file exists • 2: determines whether a folder exists

Return values

A Boolean value: `true` or `false`.

The function returns `true` if a file or a folder exists. Otherwise, this function returns `false`.

Description

This function determines whether a file or a folder exists or not and returns a Boolean value of `true` if the file exists.

Example

This example attempts to determine whether the note.txt file exists or not in c:\\.

```
if ( system.functions.isfileexist(“c:\\note.txt”))
print ("file exist")
else
print ("file doesn't exist");
```

2.5.5.5. JavaScript function: isInGroup

Searches for one configuration item (CI) in a Configuration Management list or query group.

Syntax

```
lib.ciGrouping.isInGroup( ci, groupName );
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
`ci`	String	Yes	This argument contains the CI record you want to search for in the group.
`groupName`	String	Yes	This argument contains the name of the configuration group where you want to search for the CI.

Return values

A Boolean value: `true` or `false`.

The function returns `true` if the function successfully finds the CI in the list or query group. The function returns `false` if it is unable to find the CI in the list or query group.

Description

This function is part of a system JavaScript and should not be directly modified.

This function searches for a CI in list or query groups and returns a Boolean value of true if successful.

Example

This example attempts to search for a CI in the following groups:

- A list group
- A query group

This example requires the following sample data:

- Create a list group containing one or more configuration items (for example, a list group called "test01" containing the configuration item "DEFAULT Phone 0001")
- Create a query group containing one or more configuration items (for example, a query group called "test04" containing the configuration items where type equals "telecom")

```
/* Create variables to store CI group name and the results of returnGroupMembers */
var targetCIGroup;
var targetCI;
var members;
function findInGroup( ci, group )
{
inGroupTest = lib.ciGrouping.isInGroup( ci, group );
if ( inGroupTest == true )
{
print( "Success. Found " + ci + " in " + group );
}
else
{
print( "Error. Could not find " + ci + " in " + group );
}
}
/* Test returnGroupMembers on a list group. */
print( "Now testing isInGroup on list group...\n" );
targetCIGroup = "test01";
targetCI = "DEFAULT Phone 0001";
members = lib.ciGrouping.returnGroupMembers( targetCIGroup )
print( "Looking for " + targetCI + " in " + targetCIGroup + "..." );
findInGroup( targetCI, targetCIGroup );
print( "The current members of " + targetCIGroup + " are:\n" + members );
/* Test returnGroupMembers on a query group. */
print( "Now testing isInGroup on query group...\n" );
targetCIGroup = "test04";
targetCI = "DEFAULT Phone 0006";
members = lib.ciGrouping.returnGroupMembers( targetCIGroup )
print( "Looking for " + targetCI + " in " + targetCIGroup + "..." );
findInGroup( targetCI, targetCIGroup );
print( "The current members of " + targetCIGroup + " are:\n" + members );
```

2.5.5.6. JavaScript function: removeFromGroup

Removes one or more configuration items (CI) from a Configuration Management list group.

Syntax

```
lib.ciGrouping.removeFromGroup( ci, groupName );
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
`ci`	Datum object or string	Yes	This argument contains the CI records you want to remove. To remove an array of CIs, the `ci` argument must be an SCFile or SCDatum object. To remove a single CI, you can specify it as a string.
`groupName`	String	Yes	This argument contains the name of the configuration group from which you want to remove CIs.

Return values

A numerical value: `0`, `1`, or `-1`.

The function returns `0` if it is unable to remove CIs from the list group or if the `groupName` argument specifies a query group. The function returns `1` if the function successfully removes CIs from the list group. The function returns `-1` if it can't remove CIs from the

list group because the action would result in an empty configuration list group.

Description

This function is part of a system JavaScript and should not be directly modified.

This function removes one or more CIs from a configuration list group and updates the list group record if successful.

Example

This example attempts to remove the following CIs from a list group:

- A string containing a single CI record
- A configuration group record containing one or more CIs
- A Datum object containing an array of CI records

This example requires the following sample data:

- Create a list group containing a single configuration item (for example, a list group called "test01" containing the configuration item "DEFAULT Phone 0001")
- Create a list group containing one or more configuration items (for example, a list group called "test02" containing the configuration item "DEFAULT Phone 0003")
- Create a list group containing one or more configuration items (for example, a list group called "test03" containing the configuration items "HH-000001" and "HH-000002")

```
/* Create variables to store CI group names, strings, and datum objects.
*  Use targetCIGroup to store the name of CI group you want to remove records from.
*  Use sourceCIGroup to store the name of CI group you want to create an array from.
*  Use oneLogicalName to store a string containing a logical.name you want to remove.
*  Use deviceType to store a device type you want to query for.
*  Use startsWith to store a alphabetical value you want to query for.
*  The variable recordTest stores the output of the findCIGroup function.
*  The variable arrayTest stores the output of the createArray function.
*  The variable recordListTest stores the output of the findRecordList function. */
var targetCIGroup;
var sourceCIGroup;
var oneLogicalName;
var deviceType;
var startsWith;
var recordTest;
var arrayTest;
var recordListTest;
/* Create a function to store a particular CI group in memory. */
function findCIGroup( ciGroup )
{
print( "Looking for CI Group " + ciGroup + "..." );
var members = lib.ciGrouping.returnGroupMembers( ciGroup )
var ciGroupRecord = new SCFile( "cigroup" );
var queryTest = ciGroupRecord.doSelect( "logical.name=\"" + ciGroup + "\"" )
if ( queryTest == RC_SUCCESS )
{
system.vars.$L_file = ciGroupRecord
print( "Found CI group record for " + ciGroup + "\nThe record is:\n" + ciGroupRecord );
print( "The current members of " + ciGroup + " are:\n" + members );
return ciGroupRecord
}
else
{
print( "Cannot find record for " + ciGroup + ". " +  RCtoString( queryTest ) + "." );
return null;
}
}
/* Create function to find a record list based on a device type and starting query */
function findRecordList( deviceType, query )
{
var ciRecordList = new SCFile( "device" );
if ( query != null )
{
print( "Looking for CIs matching type=\"" + deviceType + "\"\&" + "logical.name#\"" + query + "\"" );
var queryTest = ciRecordList.doSelect( "type=\"" + deviceType + "\"\&" + "logical.name#\"" + query + "\"" )
}
else
{
print( "Looking for CIs matching type=\"" + deviceType + "\"" );
var queryTest = ciRecordList.doSelect( "type=\"" + deviceType + "\"" )
}
if ( queryTest == RC_SUCCESS )
{
print( "Found the following CIs:\n" + ciRecordList );
system.vars.$L_file = ciRecordList
return ciRecordList;
}
else
{
if ( query != null )
{
print( "Cannot find records matching query: device=\""+ deviceType + "\"&logical.name#\"" + query + "\""
+  RCtoString( queryTest ) + "." );
return null;
}
```

```
else
{
print( "Cannot find records matching query: device=\""+ deviceType + "\"" +  RCtoString( queryTest ) + "." );
return null;
}
}
}
/* Create a function to store the logical.names of the record. */
function createArray( record )
{
var arrayOfLogicalNames = record.group_members;
if (arrayOfLogicalNames == null )
{
print( "Cannot create array from " + record + ". Group not in the cigroup table." );
return null
}
else
{
print( "Created array from " + record + ".\nThe array is:\n" + arrayOfLogicalNames );
return arrayOfLogicalNames
}
}
/* Create a function to call the addToGroup function and print results. */
function testRemove( ci, group )
{
var groupMembers = lib.ciGrouping.returnGroupMembers( group )
var removeCI = lib.ciGrouping.removeFromGroup( ci, group );
var afterRemove = lib.ciGrouping.returnGroupMembers( group );
print( "Now trying to remove configuration item(s) from " + group + "..." );
print( "The current members of " + group + " are:\n" + groupMembers );
if ( removeCI == 1 )
{
print( "Successfully removed configuration item(s) to " + group );
print( "The members of " + group + " are now: \n" + afterRemove );
}
else
{
if ( removeCI == "-1" )
{
print( "Could not remove configuration item from " + group + " as "
+ ci + " is the last member of the group." );
print( "The members of " + group + "are : \n" + groupMembers );
}
else
{
print( "Could not remove configuration item from " + group + ".\n"
+ ci + " or " + group + " doesn't exist.");
}
}
}
/* Test removing a CI to a group from a string */
print( "Now testing removing a single CI from a string...\n" );
targetCIGroup = "test01";
oneLogicalName = "DEFAULT Phone 0002";
members = lib.ciGrouping.returnGroupMembers( targetCIGroup )
testRemove( oneLogicalName, targetCIGroup );
/* Test removing a CI to a group from an existing CI record */
print( "Now testing removing a CI member from a CI group record...\n" );
targetCIGroup = "test01";
sourceCIGroup = "test02";
recordTest = findCIGroup( sourceCIGroup );
if ( recordTest != null )
{
testRemove( system.vars.$L_file, targetCIGroup );
}
```

```
/* Test removing one or more CIs from an array of logical,names */
print( "Now testing removing one or more CIs from an array of logical names...\n" );
targetCIGroup = "test01";
sourceCIGroup = "test03";
recordTest = findCIGroup( sourceCIGroup );
if ( recordTest != null )
{
arrayTest = createArray( recordTest );
testRemove( arrayTest, targetCIGroup );
}
/* Test removing one or more CIs to a group from a CI record list */
print( "Now testing removing one or more CIs from a CI record list...\n" );
targetCIGroup = "test01";
deviceType = "furnishings"
startsWith = null;
recordListTest = findRecordList( deviceType, startsWith );
if ( recordListTest != null )
{
testRemove( system.vars.$L_file, targetCIGroup );
}
```

2.5.5.7. JavaScript function: returnGroupMembers

Returns an array containing the group members of a Configuration Management group.

Syntax

```
lib.ciGrouping.returnGroupMembers( groupName );
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
groupName	String	Yes	This argument contains the name of the configuration group whose members you want to return as an array.

Return values

An array or null.

The function returns an array containing the logical.name values of the group members if there are group members. The function returns null if it can't find a matching group name or if the group doesn't have any members.

Description

This function is part of a system JavaScript and should not be directly modified.

This function returns the logical.name values of the members of a Configuration Management group.

- If the group is a list type, the function returns the array from the group configuration item record.
- If the group is a query type, the function returns an array of all the CIs it finds.

Example

This example returns the members of the group listed in the variable.

This example requires the following sample data:

- Create a list group containing one or more configuration items (for example, a list group called "test01" containing the configuration item "DEFAULT Phone 0001")
- Create a query group containing one or more configuration items (for example, a list group called "test04" containing the configuration items where type equals "telecom")

```
/* Create variables to store CI group name and the results of returnGroupMembers */
var targetCIGroup;
var members;
/* Test returnGroupMembers on a list group */
print( "Now testing returnGroupMembers on a list group...\n" );
targetCIGroup = "test01";
members = lib.ciGrouping.returnGroupMembers( targetCIGroup );
print( "The current members of " + targetCIGroup + " are:\n" + members );
/* Test returnGroupMembers on a query group */
print( "Now testing returnGroupMembers on a query group...\n" );
targetCIGroup = "test04";
members = lib.ciGrouping.returnGroupMembers( targetCIGroup );
print( "The current members of " + targetCIGroup + " are:\n" + members );
```

2.5.5.8. JavaScript function: returnMyGroups

Returns an array containing all the groups to which a configuration item (CI) belongs.

Syntax

```
lib.ciGrouping.returnMyGroups( ci, listType );
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
ci	String	Yes	This argument contains the CI record you want to search for in your list and query groups.
listType	String	No	This argument contains a string describing which types of configuration groups you want the function to query. The string can have one of four values: • list: Add this argument to only search list groups. • query: Add this argument to only search query groups. • both: Add this argument to search both list and query groups. • null: If there is a null entry for this argument, the function searches both list and query groups.

Return values

An array of group names.

The function returns an array containing the logical.name values of the list and query groups in which the CI is a member. The function returns an empty array if it can't find the CI in any list or query group.

Description

This function is part of a system JavaScript and should not be directly modified.

This function returns an array containing the logical.name values of the groups in which the CI is a member. The function queries all group records and does the following:

- If the group is a list type, the function indexes the group.members and if the CI exists in the list group, adds it to the groupArray.
- If the group is a query type, the function uses the RTE function to determine whether the values in the CI record satisfy the search criteria defined by the query.

Example

This example returns the groups containing the CI listed in a variable.

This example requires the following sample data:

- Create a list group containing a single configuration item (for example, a list group called "test01" containing the configuration item "DEFAULT Phone 0001")
- Create a list group containing one or more configuration items (for example, a list group called "test02" containing the configuration item "DEFAULT Phone 0003")
- Create a list group containing one or more configuration items (for example, a list group called "test03" containing the configuration items "HH-000001" and "HH-000002")
- Create a query group containing one or more configuration items (for example, a query group called "test04" containing the configuration items where type equals "telecom")

```
/* Create variables to store CI group name and the results of returnMyGroups */
var targetCIGroup;
var listTypes;
var groups;
/* Search for a CI in a list group only */
print( "Now testing search for configuration items in list groups only...\n" );
targetCI = "DEFAULT Phone 0001";
listTypes = "list";
groups = lib.ciGrouping.returnMyGroups( targetCI, listTypes );
if ( groups.length >= 1 )
{
print( "The list groups containing " + targetCI + " are:\n" + groups );
}
else
{
print( "There are no list groups containing " + targetCI );
}
/* Search for a CI in a query group only */
print( "Now testing search for configuration items in query groups only...\n" );
targetCI = "DEFAULT Phone 0001";
listTypes = "query";
groups = lib.ciGrouping.returnMyGroups( targetCI, listTypes );
if ( groups.length >= 1 )
{
print( "The query groups containing " + targetCI + " are:\n" + groups );
}
else
{
print( "There are no query groups containing " + targetCI );
}
/* Search for a CI in both list and query groups */
print( "Now testing search for configuration items in both list and query groups...\n" );
targetCI = "DEFAULT Phone 0001";
listTypes = "both";
groups = lib.ciGrouping.returnMyGroups( targetCI, listTypes );
if ( groups.length >= 1 )
{
print( "The groups containing " + targetCI + " are:\n" + groups );
}
else
{
print( "There are no groups containing " + targetCI );
}
```

2.5.5.9. JavaScript function: skipApproval

Prevents an item's approval status from being reset to "pending."

Syntax

```
ApprovalItem.skipApproval (isChanged, finalstatus, lCartItemFile);
```

Arguments

The following arguments are valid for this function:

Name	Data type	Required	Description
`isChanged`	Boolean	Yes	This argument indicates whether the cart Item gets changed.
`finalstatus`	String	Yes	This argument contains the latest cart item's approval status
`lCartItemFile`	Datum object	No	This argument is the cart Item object.

Return values

A Boolean value: `true` or `false`

If the function returns true, the item approval status isn't reset to "pending."

Description

You can modify the implementation of this function base on your organization's own business logic.

Example

Out-of-the-box, if the item isn't changed, and the approval status isn't denied, you can skip resetting the approval status to pending.

```
function skipApproval(isChanged, finalstatus, lCartItemFile){
if (isChanged !=null && finalstatus!= null && isChanged==false && finalstatus !="denied") {
return true;
}else {
return false;
}
}
```

2.5.6. List: JavaScript examples

The following table lists the JavaScript examples available for review. These examples include sample JavaScript code for the Service Manager-defined JavaScript objects, methods, and properties, as well as code samples that call the system JavaScript functions provided in the Script Library. These JavaScript are intended to work with an out-of-box Service Manager system and sample data.

JavaScript example	Service Manager object, method, or property demonstrated	System functions called
Example: Getting a list of logged on users	• system.users (global property) • print(); (global method) • getName(); (method of the XML object) • getValue(); (method of the XML object)	None
Example: Iterating through a complex XML document	• XML(); (XML object) • setContent(); (method of the XML object) • getDocumentElement(); (method of the XML object) • getQualifiedName(); (method of the XML object) • getFirstAttribute(); (method of the XML object) • getFirstChildElement(); (method of the XML object) • getNextSiblingElement(); (method of the XML object)	None
Example: Query a record using the SCFile object	• SCFile(); (SCFile object) • doSelect(); (method of the SCFile object) • RCtoString(); (method of the SCFile object) • getFirstChildElement(); (method of the XML object) • getName(); (method of the XML object) • getValue(); (method of the XML object)	None
Example: Testing a file name	• SCFile(); (SCFile object) • doSelect(); (method of the SCFile object) • print(); (global method)	None
Example: Updating a field with the current date and time	• SCFile(); (SCFile object) • doSelect(); (method of the SCFile object) • print(); (global method) • system.functions (global property) • Date(); (Date object) • SCDatum(); (SCDatum object) • getType(); (method of the SCDatum object) • doUpdate(); (method of the SCDatum object) • RCtoString(); (method of the SCDatum object)	None

2.5.6.1. Example: Getting a list of logged on users

This JavaScript example accesses the built in system.users object to get an XML list of all the currently logged-on users on the system. It then loops through to access each individual user, and then prints the name (XML tag) and value of each field.

```
var usersXML = system.users;
```

If you uncomment the line of code, recompile, and then run the script again, you can view the suppressed output. It prints all logged-on users as a large XML document.

```
//print( usersXML );
```

This part of the script loops through to process each <user> element within <users>.

```
for ( userIndex in usersXML )
{
var userXML = usersXML[userIndex];
```

If you uncomment the line of code, recompile, and then run the script again, you can view the suppressed output. It prints an individual user as an XML document.

```
//print( userXML );
```

This part of the script loops through to process each XML element node within the <user> element.

```
for ( fldIndex in userXML )
{
var node = userXML[fldIndex];
print( node.getName() + ": " + node.getValue() );
}
}
```

2.5.6.2. Example: Iterating through a complex XML document

This JavaScript example demonstrates how to iterate through a complex XML document that includes any number of levels of nesting and contains attributes. The script visits each node, be it an attribute node or an element node, and prints the name of the node and its value.

```
var xmlObj =  new XML();
xmlObj.setContent( "c:\\test.xml", true );
visitElement( xmlObj.getDocumentElement() );  // getDocumentElement returns the root node
function visitElement( elem )
{
print( "Element " + elem.getQualifiedName() );
// Print attributes for this element
var node = elem.getFirstAttribute();
while( node != null )
{
var attrName  = node.getQualifiedName();
var attrValue = node.getNodeValue();
print( "Attribute " + attrName + "=" + attrValue );
node = elem.getNextAttribute();
}
// Print possible element value
var nodeValue = elem.getNodeValue();
if ( typeof nodeValue == "string" && nodeValue.length > 0 )
{
print( "Element value is " + nodeValue );
}
// Now print child elements
var child = elem.getFirstChildElement();
while( child != null )
{
var nodeName  = child.getNodeName();
var nodeValue = child.getNodeValue();
visitElement( child );
child = child.getNextSiblingElement();
}
}
```

2.5.6.3. Example: Query a record using the SCFile object

This JavaScript example demonstrates how to perform the following objectives.

1. Query a record from a table using the SCFile object.
2. Obtain the resulting record in XML format by using the getXML method that gives you field names in addition to values.
3. Iterate through the resulting XML and print the field names and values. For a more robust approach, see Example: Iterating through a complex XML document.

```
var fContacts = new SCFile( "contacts" );
var rc = fContacts.doSelect( "contact.name = \"BROWN, NICHOLAS\" " );
if ( rc != RC_SUCCESS )  // doSelect can also return RC_BAD_QUERY or RC_NO_MORE
{
print( "Error - doSelect returned " + RCtoString(rc) );
}
else
{
var xmlContactsRecord = fContacts.getXML().instance;
var fldIndex;
for ( fldIndex in xmlContactsRecord )
{
var node  = xmlContactsRecord[fldIndex];
if ( node.getFirstChildElement() != null ) // this node has child elements
{
var i;
for ( i in node ) // print the child elements
{
var childNode = node[i];
print( childNode.getName(), "=", childNode.getValue() );
}
}
else
print( node.getName(), "=", node.getValue() );
}
}
```

2.5.6.4. Example: Testing a file name

This JavaScript example demonstrates how to test whether or not a Service Manager file name in a string is valid. The script uses a string containing the file name to create a new SCFile object, and then tests the resulting object to see if it is valid. If the file name is valid, it contains the expected properties and methods (such as **doSelect**).

```
function testFile( fname )
{
var undefined;
var f = new SCFile( fname );
if ( typeof f == "object" && f.doSelect != undefined )
{
print( fname + " is a real file!" );
}
else
print( fname + " is NOT a real file!" );
}
testFile( "nonexistentfile" );
testFile( "contacts" );
```

2.5.6.5. Example: Updating a field with the current date and time

This JavaScript example demonstrates how to update a field with the current date and time. The script takes a file object (such as **system.vars.$L_file** or an SCFile object) and a string containing the name of a datetime field in that file, and updates the indicated field with the current date and time.

This script illustrates several key concepts of using JavaScript in Service Manager.

- How to call the RAD **filename** function from JavaScript.
- How to access a field in a record using an expression of the form file[`fieldName`] where `fieldName` is a string.
- How to test the field's data type.
- How to return JavaScript date objects when you reference Service Manager datetime fields.
- How to assign JavaScript date objects to Service Manager datetime fields.
- How to use SCFile method return codes to test against global properties like RC_SUCCESS.
- How to obtain a localized error message from a return code value using the RCtoString method.

- How to pass traditional query strings that are in Service Manager syntax directly to the doSelect method.
- How to use underscores in JavaScript to reference Service Manager field names containing dots (such as contact.name).

This script demonstrates calling the datetimestamp function. It selects a record from the contacts table and updates the sysmodtime field.

```
var f = new SCFile( "contacts" );
f.doSelect( "contact.name = \"BROWN, NICHOLAS\"" );
print( "sysmodtime was: " + f.sysmodtime );
datetimestamp( f, "sysmodtime" );
print( "sysmodtime is now: " + f.sysmodtime );
```

This script demonstrates how passing the wrong kind of field to the datetimestamp function generates an error message.

```
datetimestamp( f, "contact_name" );
function datetimestamp( file, fieldName )
{
var fileName;
var today;
var fieldType;
var rc;
fileName = system.functions.filename( file );
today    = new Date();
print( "datetimestamp called for file " + fileName + " and field " + fieldName );
var d = new SCDatum( file[fieldName] );
fieldType = d.getType();
if ( fieldType != "TIME" )
{
print( "Error! Field " + fieldName + " is a " + fieldType + " field!
                This function can only be called with date/time fields!" );
return false;
}
file[fieldName] = today;  // Timestamp the field
rc = file.doUpdate();
if ( rc != RC_SUCCESS )
{
print( "Error " + RCtoString(rc) + " trying to update " + fileName + " record" );
return false;
}
return true;
}
```

2.5.6.6. Example: Working with structured arrays

This JavaScript example shows you how to work with structured arrays.



Note

Note The sample "teststructarray" has the following Database Dictionary format fields:

- id character
- arr array
- arr structure
- a number
- b character

This table has an array of type structure called "arr."

Note

Note : To populate the elements of the field, the elements have to be populated in order.

```
var fvar = SCFile("teststructarray");
```

```
fvar.id="abc";
```

```
fvar.arr[2].a=10;      // Add field 'a' first
fvar.arr[2].b="Test";   // Then add field 'b'
```

```
print(fvar);
```

```
[C++ object SCFile] - teststructarray={[["abc",{{[, ]},, {[10, "Test"]}}]}
```

Caution

Caution Populating the fields out of order will cause an error.

For example, this will break the structure:

```
fvar.arr[2].b="Test";   // This will break the structure!
fvar.arr[2].a=10;
```

3. Document Engine

The Document Engine comprises a set of tools and methodologies for developing and modifying Service Manager workflows. The Document Engine was originally tasked to develop an underlying set of base functionality that would support multiple modules inside Service Manager, improve consistency of the user interface between modules and reduce the amount of code needed for any new module.

The Document Engine extends the display application capabilities with simplified and more extensive actions especially those involving multiple application calls. In addition, the Document Engine supports the use of joined tables and master format control calls. It is designed to meet the needs of most customers out of the box, yet retain flexibility. The relationship between Objects, States, and Processes is hierarchical.

The Document Engine controls behavior with Objects. An Object is referenced whenever a form opens, and the Object determines the behavior for the state of the form (open, list, search, etc.). Objects define overall table behavior. Within the Object, a State describes where a record is in its lifecycle (open, list, search, etc.). Within the State, different Processes are executed depending on the actions initiated by a user on the record. States also define how the system displays a record and what options (actions) are available at specific times or circumstances. For instance, States can determine an action, such as Save, given a user's access privileges.

Processes are called from States based on the user's action. The Process uses RAD expressions, JavaScript, and calls to existing RAD applications to perform actions against the current record.

3.1. Document Engine overview

The Document Engine is a tailoring tool that includes the possibility to customize the system without the need of RAD changes. It provides a centralized method for setting privileges and behavior for standard actions, such as list, view, and search. It increases consistency across modules. The three main components of the Document Engine are Objects, States, and Processes. Processes can be reused for more modular programming and the integration is seamless, which can reduce development time.

Benefits of the Modular Approach[edit]

The advantages of the modular design include consistency in development, reduced development time, and flexibility.

Consistency[edit]

The fact that the engine allows all applications to run using the same base RAD applications brings consistency to the Service Manager application suite. Core functionality, such as locking, alerts, approvals, and use of record list functionality, will work the same for any module, as they are all using the same code base.

Reduced Development Times[edit]

The modularity of the Document Engine allows for reuse of existing code and Processes.

Flexibility[edit]

The Document Engine uses Process records as a mechanism to modify the behavior of modules inside of Service Manager applications. You can create a new Process that has a different behavior than the base system, without the need to change or remove the original Process from the system. Additionally, the system's base Processes can be overridden with your own Processes to give greater flexibility to the system developer when tailoring Service Manager to meet an organization's specific requirements.

Accessing the Document Engine[edit]

To access the Document Engine, follow these steps:

1. Start a Service Manager Client and log on as an administrator.
2. In the Service Manager System Navigator, click **Tailoring**.
3. Click **Document Engine**. From here, you can access the main three areas to define Objects, Processes or States. You can also set up Alerts and Approvals to be called from Objects or create Search Configuration Records, which are used by Objects.

Contents

- 1 Benefits of the Modular Approach
  - 1.1 Consistency
  - 1.2 Reduced Development Times
  - 1.3 Flexibility
- 2 Accessing the Document Engine

3.2. Objects

Objects are a base set of definitions that determine behavior of records and set the definitions and governing rules. Objects correspond one-to-one with database dictionary (dbdict) records in Service Manager. If a table doesn't have a dedicated Object record, the Document Engine applies the settings in the DEFAULT Object. All Objects should have list and default States defined. If not otherwise supplied, the default States are db.browse, db.list, db.search, and db.view.



Note

Note Do not modify or delete the DEFAULT Object Record as doing so will cause unpredictable results.

The Object record sets up the definitions and governing rules for the behavior of the table within the Document Engine. For example these may include:

- The application used to create the users profile within this Object that determines what actions this user may take against any record of the table.
- The State records used in specific circumstances (see States for more details).
- The category, phase, and paging file names for the Object.
- The name of the number record to be used for this Object.
- How locking is to be used by this Object.
- Setup of revisions for records in the table.
- Which variables should be available to processes that run against this Object.
- Which global lists should always be available when using this Object.
- Use of activity records.
- How alerts are processed against this Object.
- How approvals are processed for this Object.
- Settings for the work queues.
- Ability to set up personal or global views and default templates.
- Notifications on add / update / delete of a record in this Object.
- Ability to configure additional search choices against this Object.

To view a list of the out-of-box Objects, click **Search** from the Object Definition form.

To view a list of the fields and field descriptions for the Object definitions see Object Definition form and fields.

3.2.1. Create Objects

To create an Object, follow these steps:

1. Navigate to the Document Engine. See the **Accessing the Document Engine** section in Document Engine overview.
2. Double click **Objects**. The Objects form opens.
3. Use the tabs on the Objects form, to fill in the fields required to create an Object that will perform the functions you desire. See the field descriptions for additional information.

To update an Object definition, follow these steps:

1. Access the Document Engine. See the **Accessing the Document Engine** section in Document Engine overview
2. Enter the name of the Object you want to update in the Object field or click **Search** to search for the Object.

3.2.2. Object Definition form and fields

The field descriptions for the Object Definition form are:

Field Name	Description
File name file.name	Enter the dbdict name for the Object, that is, use the dbdict name that corresponds to this Object's name. (required)
Common name message	The system fills in this information from Data Policy. It is a common name for the Object. The common name can be a simple name, such as Work Order.
Unique key unique.field	The system fills in this information from the dbdict. This is the unique key for the Object.

This form also includes the following tabs. The fields on these tabs are described in the field descriptions tables for the respective tab.

- Object Info tab field descriptions - specifies the general properties and behavior of the Object.
- Locking tab field descriptions - determines the locking behavior for the Object.
- Revisions tab field descriptions - tracks revisions for the Object.
- Variable and Global Lists tab field descriptions - describes local variables and global variables used by the Object.
- Activities tab field descriptions - defines logging (activities).
- Alerts tab field descriptions - defines where to set alerts and the conditions for generating alerts.
- Approvals tab field descriptions - sets approval options for the Object.
- Manage Queues tab field descriptions - controls how queues display as well as threading, and who can create views (inboxes).
- Views and Templates tab field descriptions - defines whether or not a user can create global and personal views and templates on the Object.
- Notifications tab field descriptions - identifies notifications sent automatically for add, delete, or update activity for any record in the Object.
- Search Configuration tab field descriptions - controls the available options on the More Choices tab of the search screen.

3.2.2.1. Object Info tab field descriptions

This tab specifies the general properties and behavior of the Object.

The field descriptions for the Object Info tab are:

Field Name	Description
Description field desc.field	Specifies a short description of the Object.
Profile application profile.appl	Specifies the RAD application that creates the profile that determines if a user can perform certain functions, such as add and delete. For example, db.environment. (required)
Profile variable profile.variable	Specifies a variable that can be accessed any time this Object is called without accessing the environment record.For example, $L.env. (required)
Number record name number.record	Defines a number class for the Object, which can be accessed from a call to the getnumb application either via a Process or FormatControl or RAD call. It can be used to retrieve a sequential number as a unique key for the record. For example, EXWorkOrder.
Category table name category.file.name	Specifies the table name that links with the category table associated with this Object's category value. When displaying a record of this type, if a field called category exists, then the Object will search the category table name and for the record with the corresponding name. If found, the system stores the Category File Name as a variable: $L.category.
Phase table name phase.file.name	Specifies the table name that links with the phase table associated with this Object, if applicable. When the system displays a record of this type, if a field called phase exists, then the Object goes to the phase table name and selects a record with a corresponding name. If found, the system will store the phase table name as a variable: $L.phase.
Paging table nam paging.file	Specifies the name of the table used for storing pages. Pages are created as full copies of the current record prior to the latest updates being applied. This is done every time a record is updated, which creates a detailed audit trail.
Primary format control master.fc	Specifies the name of the Primary Format Control record, if one exists for the record Object. Primary Format Control allows you to define in one record the Format Control statements that apply to all phases in an area, for example Change Management Request Phases. Typically, the name of the primary format control is the name of the dbdict or all categories in Incident Management, Problem Management, or Service Desk.
Joindef joindef	Specifies the name of the joindef record used to join multiple tables, for example joincomputer, or an expression that evaluates to a valid joindef name, such as joindef in $L.category
Status field statusField	Specifies the field name that contains the record's status information.
Assigned to fields assignedToFields	Specifies the field name that contains the assignee name field for this Object. This field will be referred to when Folder Entitlement verifies that the record is assigned to the logged in operator.
Workgroup fields workgroupFields	Specifies the field name that contains the assignment group field for this Object. This field will be referred to when Folder Entitlement verifies that the record is assigned to any of the logged in operator's work groups.

Open state open.state	Specifies the State definition record to use upon opening a new record.
Close state close.state	Specifies the State definition record to use for close processing of an existing record.
List state list.state	Specifies the State definition record used to display a list of records.
Default state default.state	Specifies the name of the State used as default for the Object. To edit a record in this Object, the default State is used.
Search state search.state	Specifies the State definition record to use for searching.
Browse state browse.state	Defines the State definition record to use when records use locking. Essentially, this field defines a read-only State when a record is currently locked by another user.
Manual states manual.states	Specifies the array of States that may be used with this Object other than the life-cycle states of open, close, list, view, search or browse.

3.2.2.2. Locking tab field descriptions

The Locking tab determines the locking behavior for the Object. This means that when someone is actively updating a record the system doesn't allow another user to update the same record.

The field descriptions for the Locking tab are:

Field	Description
Use locking use.locking	Select this checkbox to enable locking.
Lock on display lock.on.display	If true or a condition that evaluates to true, Service Manager attempts to lock the record as soon as it is displayed. Use locking must be enabled for this field to work.
Lock parent record lock.parent	Locks the current record and the record's parent. For example, when this field is checked and someone is updating a change task, the change request for the task is also locked.
Parent Id Field parent.id	Enter a field name in the current record that contains the ID of the parent to lock.
Parent Filename/Object parent.object	The name of the table that contains the parent record.
Watch Variables watch.variables	Watch Variables are used when the Document Engine checks to see if a record has been changed or not. Watch Variables must be NULL when the record is first displayed.

3.2.2.3. Revisions tab field descriptions

The Revisions tab determines the revision behavior for the Object

The field descriptions for the tab are:

Field Name	Description
Revision table name revision.file	Specifies the name of the table in which you want to store revisions to this Object's records.
Max # of revisions max.revisions	Specifies the total number of revisions allowed for this Object. If left blank, then there is no upper limit.

3.2.2.4. Variable and Global Lists tab field descriptions

The Variables/Global List tab describes local variables and global variables used by the Object.



Note

Note Global variables are built and stored in memory.

The field descriptions for the Variable/Global List tab are:

Field Name	Description
Local variables local.variables	Enter a list of local variables that can be used in Processes. Local variables are defined in the application. Local variables are assigned to the Object you are creating and are available to all Processes and States associated with the Object. Local variables aren't available to other Objects.
Global lists global.lists	Global lists, once created, are available to all Processes. Global lists can be built on login, if they are listed in the Startup list. Global lists are available every time you access the Object.

3.2.2.5. Activities tab field descriptions

The Activities tab defines update (Activity) logging for the Object.

The field descriptions for the Activities tab are:

Field Name	Description
Activity log table activitylog.file.name	The name of the table to hold the activity log entries for the Object.
Selection list variable activity.selection.var	The variable to use on an update form to display the types of activities the operator can select when performing an update on a record for a specific Object.
Posting link activity.post.link	The name of the link used to post information.
Require update if an activity record is NOT generated? activity.mandatory	A checkbox to indicate that an activity update is required when the selected.
Update field update.field.var	The field or variable that contains the activity update on the form. This field appears only when **Require update if an activity record is NOT generated?** is present.
Display message activity.mandatory.msg	The message indicating that an activity update is required. This field appears only when **Require update if an activity record is NOT generated?** is present.

3.2.2.6. Alerts tab field descriptions

The Alerts tab defines where to set alerts and the conditions for generating alerts. Any Object that has a unique key can use these alerts.

The field descriptions for the Alerts tab are:

Field Name	Description
Alert location alert.location	The Alert location defines where the name of the Alert Definition to run is found or stored. Specify where the Alert Definition is stored: Record: Store alerts in the record itself. Category: Store alerts in the category file defined on the Object Info tab. Phase: Store alerts in the Phase record defined on the Object Info tab. Object: Store alerts in the Object record. When selected an Alerts array table will be displayed to enable you to fill in alert(s) to be used. Note: The Object's table needs to have a unique key for the Alerts to relate to. A table that contains a no nulls key rather than a unique key can't use Alerts.
Alert condition alert.condition	Specify a condition to determine whether or not to process the alert. For example, open in $L.file~=false.
Alert field name alert.field.name	Specify the field name that contains the actual alert name, as defined by Alert location.
Alert status field alert.status.field	Specifies the field in the current record in which to put the alert status, after the alert is processed.
Alert update process alert.update.process	Specifies the name of the Process record for additional functions that the system performs after the alert runs.
Log alerts? log.alerts	If selected, the alerts are moved after processing to the Alertlog file to keep an Alert history.
Process alerts on parent? alerts.against.parent	If you have selected the Locking Parent Record field on the Locking/Revisions tab and you select this checkbox, then the alert will register against the parent record as well, when it is activated.
Recalculate alerts if alert.recalc	Specifies conditions that determine whether to recalculate conditions on existing alerts.
Reset alerts if alert.reset	Determines when to delete existing alerts and recalculate all conditions.

3.2.2.7. Approvals tab field descriptions

The Approvals tab sets approval options and their associated notifications for the Object. Approvals are defined in the ApprovalDef file.

The field descriptions for the Approvals tab are:

Field Name	Description
Approval condition approval.condition	If the approval condition evaluates to true, approvals are used on the Object's records.
Approval location approval.location	Indicates where the approval information is stored: record, phase, object, or category.
Approval field name approval.field.name	The field name that contains the actual Approval name within the table that's defined in the Approval location.
Approval status field approval.status.field	The field in the current record in which to store the approval status.
Approval groups approval.groups	Stores a variable to contain the groups the current user must belong to in order to issue approvals for this Object.
Approval type appr.cond.type	For a list of available approval types and their descriptions, see What's an approval type?.
Approval notification *single.notify.approva*l	Select the notification to run if the request is approved.
Denial notification single.notify.denial	Select the notification to run if one approver denies the request.
Retraction notification single.notify.retraction	Select the notification to run when retracting a previous action.
Final approval notification final.notify.approval	Select the notification to send once the final approval is granted.
Final denial notification final.notify.denial	Select the notification to send when the request is denied.
Approval FC appr.fc	Specifies the name of the Format Control record to run upon approval.
Approval process approval.process	Select the Process to run when the record is approved.
Denial process denial.process	Select the Process that runs when the record is denied.

Preapprove on open preapprove.cond	Determines whether the record should be automatically approved. If the condition is true and the user belongs to one of the pending approval groups, the approval is processed automatically. If the user doesn't belong to one of the pending approval groups, the approval doesn't occur automatically and must then go through the regular approval process. Defaults to true.
Log approvals? log.approvals	Select this checkbox to log the history of approvals in the ApprovalLog table.
Require appr. comments approval.comments	If checked, approval comments are requested from the approver.
Aggregate approvals? aggregate.approvals	If checked, approvals are cumulative.
Recalculate approvals if approval.recalc	Specifies the conditions that determine whether or not to recalculate the conditions on the existing approvals.
Reset approvals if approval.reset	Determines when to delete existing approvals and recalculate all conditions.

3.2.2.8. Manage Queues tab field descriptions

The Manage Queues tab controls how queues and views display as well as threading. These same fields are available in Data Policy for files that don't have an associated Object record, or they are virtual joined into the datadict record from the Object record if it exists.

The field descriptions for the Manage Queues tab are:

Field Name	Description
Manage condition scm.condition	Specifies a condition that allows only certain users to view the queues that display records of this Object. For example, browse in $G.pm.environment.
Manage display format scm.manage.screen	Select the format to use to display the view. The out-of-box Service Manager has a default display format: sc.manage.generic that's used if no other form is chosen. We recommend that you don't change the sc.manage.generic format.
Manage default view scm.inbox	Select the default view for this queue. By specifying a user view for a particular user, a specific list of views can be set up for the Service Manager Manage Queues. If a user doesn't have a specific view defined , the default user view is used.
Manage default query scm.query	Specifies a default query to run if no default view is selected.
Default query description scm.query.name	Specifies a name for the above field. You can associate a message with this field. For example, scmsg(491, "us").
Thread view -> search? scm.thread.list.edit	Specifies true or an expression that evaluates to true to open a new thread when conducting a search.
Search format (if necessary) *scm.search.format*	Select a default search format.
Thread search -> list? scm.thread.search.list	Specifies true or an expression that evaluates to true for a new thread when a user finds a list of records to view.
Thread list -> edit? scm.thread.list.edit	Specifies true or an expression that evaluates to true for a new thread when a user selects a record to view out of a list of records.
Thread view -> edit? scm.thread.inbox.edit	Specifies true or an expression that evaluates to true to open a new thread when the user views an existing record out of the queue.
Allow add condition scm.add.condition	Specifies an expression that evaluates an operator's ability to add a record.
Add/open application scm.add.appl	Specifies the name of the application to call when a record is added or opened.
Parameter Names scm.add.names	Specifies the parameter names to pass to the application specified in the Add/open application field.
Parameter Values scm.add.values	Specifies the parameter values to pass to the application specified in the Add/open application field

3.2.2.9. Views and Templates tab field descriptions

The Views/Templates tab defines whether or not a user can create global and personal views as well as template support.

The field descriptions for the tab are:

Field Name	Description
Can create personal views personal.inbox	Specifies a condition that evaluates to true or false. True allows the user to create personal views.
Can create system views global.inbox	Specifies a condition that evaluates to true or false to determine if the user can create global views.
Default Template default.template	Specifies the name of the default template to use for records in this table.
Supports Templates? supportTemplates	Select this checkbox to enable support for templates for the Object.

3.2.2.10. Notifications tab field descriptions

The Notification tab identifies notifications sent automatically for add, update, or delete activity for the Object.

The field descriptions for the Notifications tab are:

Field Name	Description
Add notification.add	Select the notification that's sent automatically when a record is added to the table.
Update notification.update	Select the notification that's sent automatically when a record is updated in the table.
Delete notification.delete	Select the notification that's sent automatically when a record is deleted from the table.

3.2.2.11. Search Configuration tab field descriptions

The Search Configuration tab controls the available choices on the More Choices tab of the search screen.

The field descriptions for the Search Configuration tab are:

Field Name	Description
Table Name tablename	The name of the table that will be queried.
Search Format searchFormat	The name of the sub-format used for the More Choices tab.
Initialization Process init.process	The name of a Process that will be run before displaying the search form.
Allow Advanced Find allowAdvAccess	Defines the conditions that determine whether or not to allow Advanced Find.

3.2.2.12. Defined Queries tab

The Defined Queries tab defines the query statements and labels to use on the More Choices tab of the search screens (for example, More Choices on the Search Incidents form). The fields are defined in the SearchConfig table.

The field descriptions for the Defined Queries tab are:

Field Name	Description
Id id	Provides the unique ID for the query and can't include special characters including spaces.
Query query	A query expression using system language syntax. For example, `assignee.name=operator()`.
Description description	Provides the label for the checkbox on the More Choices tab.

3.2.2.13. Ranges tab

The Ranges tab allows you to easily set up a search for begin and end date ranges. To do so click the **Modify Configuration** link, to define a variable each for begin and end to use as input in the form. Enter this variable in the Variable 1 column and use the Field and Operator 1 columns to define the query that will be executed. Modify Configuration is also how modifications are made on the Defined Queries tab as well.

The field descriptions for the Ranges tab are:

Field Name	Description
Field fieldName	Specifies the field in the table to use in the query.
Operator 1 operator1	The comparison operator in the query. (for example: >=)
Variable 1 variable1	Specifies the variable used as input on the form.
Special Type specialType	Not used at this time.

3.3. States

States are called by Objects and are defined by Processes. The State record contains information about how a record looks and acts at a specific period in time.

A State record definition can include the following:

- The State name.
- The display screen to use in order to display the record or records.
- The initialization process used when a list is first displayed.
- The format to use to display the data.
- The input condition to indicate whether or not a user can modify the record.
- The Process to run when a user triggers a specific display option (Display Action).

The State record used depends on how many records the user views. There are out-of-box State records for searching for records, viewing a list, viewing a single record, and browsing a record.

Searching

When there are no records in the current file variable, it is assumed that the user is in “search” mode. The State that's used is the “Search State” defined in the table’s Object record. The default search state is “db.search”.

Record Lists

When viewing a list of records, the “List State” defined in the table’s Object record is used. The default list state is “db.list”.



Note

Note In some earlier versions of Service Manager, record lists are referred to as QBE lists.

Viewing a single record

When viewing a single record, the record is first checked to see if it includes a “State” field in the dbdict. If the field exists and is populated, the contents of that field will be used as the current State of the record. If this field doesn't exist or is NULL, the “Default State” defined in the table’s Object record is used. The default value for the default state is “db.view”.

Browsing a record

When browsing a record in read-only mode, the “Browse State” defined in the table’s Object record is used. Note that only files that use locking need a browse State. Although viewing a record without having update rights looks similar to the browse screen, it is using the default State, not the browse State. There is no default browse state.

Integration tips for display application

If a display screen doesn't have any display events associated with it, the system automatically uses the se.default display event that performs se.lock.object in case of “OnFormModified”. If a display option record has the “Modifies Record” checkbox activated, the record is locked when the button is pressed.

Create and update States

To create a new State, follow these steps:

1. Access the Document Engine. See the **Accessing the Document Engine** section in Document Engine overview
2. On the State Definition form, fill in the fields required to create a State to perform the functions you desire.

To modify an existing State, follow these steps:

1. Access the Document Engine. See the **Accessing the Document Engine** section in Document Engine overview
2. Enter the name of the State you want to modify in the State field or click **Search** to search for the State.

State Definition field descriptions

The State definition record defines the behavior and display specifications for a record at a specific period in time.

In the State record non-base methods can be defined or the behavior of base methods can be modified. Base methods are described in the sections Base functions in se.search.engine, Base functions in se.view.engine, and Base functions in se.list.engine. They include functions such as save, find, fill, OK, Cancel, and Search.

See the following tables for the field descriptions of the State Definition form.

Field Name	Description
State state	Specifies the name of the State. (required)
Display Screen display.screen	The display screen to associate with the State.
Initialization Process init.process	The name of a Process to run prior to entering the State.
Format format.name	Specifies the format in which the State record opens. This form is stored in a record, and it can be a variable or hard-coded. The name of the format can be hard-coded or be contained in a variable or retrieved from a record via a system language expression.
Input Condition input.condition	The input condition is evaluated for view States and determines whether or not the record is read-only. Specify false for read-only or true for editable. You may also enter an expression that evaluates to true or false.otherwise, enter true or enter an expression that evaluates to true or false.

Non-base methods

Field Name	Description
Display Action process.label	Specifies the action parameter that comes from a display action after input from a user. The display action comes from the action field in the display option of a button or menu item clicked by a user.
Process Name valid.process	Specifies the name of the Process run as a result of the action.
Condition process.condition	Specifies an expression that evaluates to true for the Process specified in the Process Name to be called.
Save First run.save.before	Choose to run the save Process before you run this Process. Type true to save first; otherwise, enter false. The default is false.

3.4. Processes

Processes are the smallest discrete units of work available to the Document Engine and are the level where the data is manipulated. Users can create their own Process or use one of the over 700 Processes that ship with Service Manager. The Process definition consists of initial expressions, RAD, and final expressions, entered on the Initial Expressions, Initial Javascript, RAD, Final Expressions, Final Javascript, and Next Process tabs, as well as an optional call to a next Process. Expressions are written using standard RAD expressions and/or JavaScript.

3.4.1. Create and modify a Process

To create a Process, follow these steps:

1. Access the Document Engine. See the **Accessing the Document Engine** section in Document Engine overview
2. Using the tabs on the Process panel, fill in the fields required to create a Process that will perform the functions you desire. See Process Definition form and field descriptions for more information.

To modify a Process, follow these steps:

1. Access the Document Engine. See the **Accessing the Document Engine** section in Document Engine overview
2. Enter the name of the Process you want to modify in the Process Name field or click **Search** to search for the Process.

3.4.2. Process Definition form and field descriptions

On the Process Definition form, you define new Processes or edit existing Processes. Processes run code or expressions to perform the user selected actions

The field descriptions for the Process Definition form are:

Field Name	Description
Process Name process	Specifies the name of the process. (required)
Save Cursor Position save.cursor.position	Select this box if you want to return to the same cursor position after the action (for example on a fill).
Run Standard Process when complete? run.standard	When selected, then the system runs a standard process after completing the current action or Process. A standard Process such as save, for example. If you have created a save Process and want to run the save Process that comes with the Document Engine after completing the save process you defined, check this box. A standard process is defined in the base functions sections of this document.
Run in Window? run.in.window	When selected, the process runs in a separate window.
Window Title window.name	If the **Run in Window?** checkbox is selected, specify a title for the window. An scmsg expression can be used for localized window titles, such as scmsg(1980, "us").

In addition to the fields described above, there are also tabs on this form that allow you to further define the Process. These tabs are:

- Initial Expressions tab
- Initial Javascript tab
- RAD tab
- Final Expressions tab
- Final Javascript tab
- Next Process tab

Initial Expressions tab[edit]

The Initial Expressions tab defines the initial expressions that run prior to the initial Javascript and prior to the RAD calls defined on the RAD tab. The initial expressions are written using standard Service Manager expressions.

Initial Javascript tab[edit]

The Initial Javascript tab defines the initial javaScript expressions that run before the RAD defined on the RAD tab.

RAD tab[edit]

The RAD tab defines the pre-RAD expressions, RAD calls, and post-RAD expressions that run as part of the Process.

The field descriptions for the RAD tab are:



Note

Note These fields are repeated for each RAD application defined on this tab.

Field Name	Description
Expressions evaluated before RAD call pre.rad.expressions	Specify the expression to run prior to the RAD application defined in the RAD Application field. All parameter values passed to the RAD application must be in the form of variables or expressions. The variables must be assigned their values in the pre-RAD expressions; for example, `$L.value.name="Test"`.
RAD Application application	Specifies the name of the RAD application to run.
Condition rad.condition	Specify under which circumstances the RAD application should be executed (condition evaluates to true) or skipped (condition evaluates to false).
Parameter Names names	Specifies the parameter names passed to the RAD application
Parameter Values values	Specifies the parameter values passed to the RAD application. These values must be in the form of variables or expressions. String values can be passed when enclosed in double quotes ("Wizard Name").
Post RAD Expressions post.rad.expressions	Specifies any RAD expression that will run after the RAD application completes.

Final Expressions tab[edit]

The Final Expressions tab defines the final expressions that run after the RAD tab processing completes. The final expressions are written using standard Service Manager expressions.

Final Javascript tab[edit]

The Javascript defined on this tab runs after the final expressions and after the RAD applications on the RAD tab.

Next Process tab[edit]

This tab specifies the next Process or Processes to run when the current Process completes.

The field descriptions for the Next Process tab are:

Field Name	Description
Next Process next.process	Specifies the name of the next Process to run.
Condition process.condition	Specifies a condition associated with the Process in the Next Process field that evaluates to true or false. For example, `true` for the sm.close Process.

Contents

- 1 Initial Expressions tab
- 2 Initial Javascript tab
- 3 RAD tab
- 4 Final Expressions tab
- 5 Final Javascript tab
- 6 Next Process tab

3.5. Document Engine resources

The Document Engine includes local variables and base functions that can be used by any Object. The base functions are defined out-of-box to execute standard actions that are available to the user. For example, when a user clicks the save button after updating a record, the Document Engine processes this request and writes the changes to the database. A base function can be overwritten in the State record to run a different Process.

3.5.1. Database Manager

Any file accessed by the Database Manager automatically uses the rules and processes defined for the corresponding Object. If no Object has been defined for the file, it uses the DEFAULT Object. The DEFAULT Object duplicates the functionality of the earlier versions of the Database Manager.

A System Administrator may also access a file that has a corresponding Object via the DEFAULT Object by accessing the file through Database Manager with the "Administration Mode" checkbox marked. This checkbox is available for System Administrators only.

The Document Engine uses environment profiles. In previous versions of the system, a System Administrator was granted all rights (add, update, delete.), regardless of the format control settings. With the Document Engine, the rights granted match those defined in the format control record. Administrators that prefer the old method of granting rights can simply modify the "Profile Application" setting in the DEFAULT Object record from "db.environment" to "db.environment.sysadmin".

3.5.2. RAD applications

With a database driven application, a user may be working with any of the following three basic record sets:

- Zero records – when searching for information.
- Many records – when looking at a list of information.
- One record – when making changes to a single record.

From a user perspective when working with data:

- you either view a blank form, where you can enter search information (or create a new record),
- or you are working on updating / viewing a single record,
- or you did a search and now have a set of multiple records displayed as a list.

The Document Engine uses three main RAD applications to mirror this idea. The RAD routine used is determined by the number of records displayed at any time.

se.search.engine

The search engine is used when there are no records being viewed. The main purpose of this routine is to formulate a query and select records from the correct table. This routine may also be used for the initial entry of information into a blank record for the purpose of adding a new record to the database. The Default State is db.search.

se.list.engine

The list engine displays multiple records. Using the list engine a user may select a specific record from the list, or perform actions on the entire list of records. Default State is db.list.

se.view.engine

The view engine displays a single record. This application is used to perform actions against a specific record, such as updates or deletes. Default State is db.view.



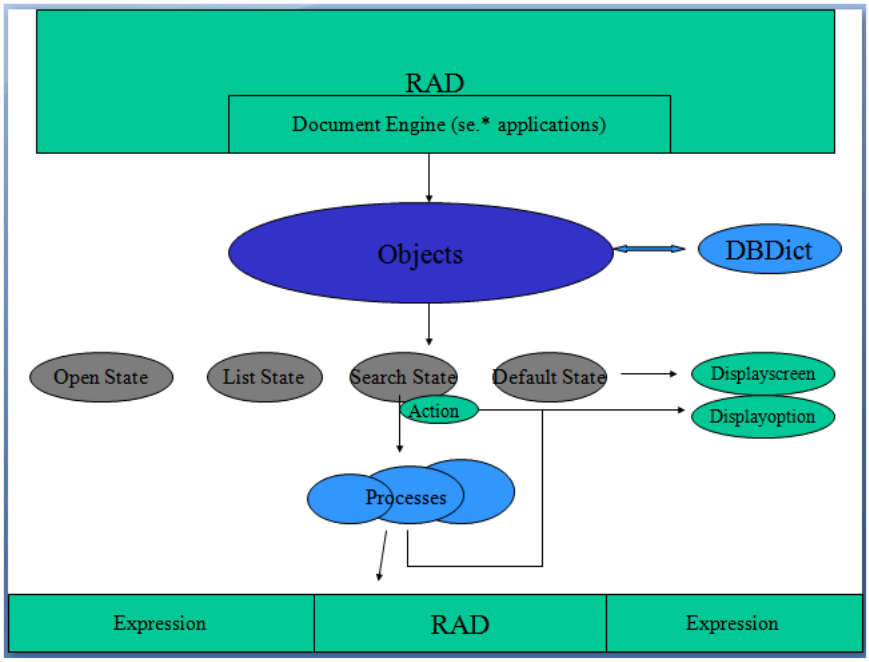
Note

Note

When using Service Manager's record list functionality, the view engine is used for both the list and single record information.

3.5.3. RAD applications flow

The following figure provides an overview of how the other tailoring tools interact with the Document Engine.



Note

Note All RAD applications related to the Document Engine start with "se", such as se.view.engine. The document engine treats every record as a document made available to the user.

When a file is displayed using the Document Engine it will always be using one of the three applications specified earlier in this document. The display screen, display options, and format are determined by the current State of the record. When a display option is triggered, after the standard display functionality is performed, the engine checks the “action” of that option against the available processes defined in the current State record. If there is a process defined that has a condition that evaluates to true, the engine then performs that process against the current record (or record set). If the action is undefined in the State record, the engine will then check to see if that action is defined as a “Base Process” in the current application. If it is, then the system will perform that base process against the record. If not, no action is taken.

Integration tips for display application[edit]

If a display screen doesn't have any display events associated to it, the system automatically uses the se.default display event that performs se.lock.object in case of “OnFormModified”.

If a display option record has the “Modifies Record” checkbox activated, the record is locked when the button is pressed.

Contents

- 1 Integration tips for display application

3.5.4. Base functions in se.search.engine

Display Action	Description
back	Exits the record.
find	Displays detailed information from the record linked to the field.
fill	Fills information from a linked record into a target record. Puts data from a source record field into a target field.
advanced	Starts the Advanced Search process that gives the user more search options. Typically, this is only available for system administrators, but this option can be made available to other users.
clear	Clears the current screen.
openinbox / inbox	Prompts the user to open a view associated with the current file. This is a predefined query that displays results as a view.
search	Performs a standard search by creating a search query using the information provided and then displays the results in a record list.
add	Performs “add” format control and attempts to add a record to the database.
restore	Restores the contents of the screen (after a “clear” is performed).
irquery	Runs an IR Query text search.
validitylookup	Performs Service Manager validity lookup. Validity lookup verifies via an entry in the validity table that a specific value is valid for the defined field. Using validity lookup, the system verifies that a user-entered value complies with the validation rules.
expandarray	Expand array provides additional functionality for editing arrays, such as insert line and delete line.
reset	Resets the current file. This action deletes **ALL** records in the database for the table.
regen	Regens the indexes of the current file. This action applies only to tables that have an IR Key, and it will perform only an IR regen.
export/unload	Starts the “no records” export/unload process. Used for unloading empty DBDICT definitions without data only.
views	When selected, the system presents to the user a list of alternate forms in which to display the current record.
findrevision	Displays revisions for records in this object.
initrevision	Creates a revision of a record in this object.
newsite	
newview	
newTable	
addFilter	Triggered by customer performing an Advanced Filter search in the standard search screen.
editFilter	Triggered by customer performing an Advanced Filter search in the standard search screen.

addCompound	Triggered by customer performing an Advanced Filter search in the standard search screen.
removeSelection	Triggered by customer performing an Advanced Filter search in the standard search screen.

3.5.5. Base functions in se.view.engine

Display Action	Description
save	Performs update in Format Control and updates the record in the database if the update is valid.
add	Performs add in Format Control and adds a new record to the database if the new record is valid.
ok	If the record has changed, perform a save, otherwise exit.
reselect	Re-selects the current record from the database (in case it was changed).
fill	Fills information from a linked record into a target record. Puts information from a source field into a target field.
find	Displays detailed information from the record linked to the field.
next	Move to next record in the list (after checking for changes).
previous	Move to previous record in the list (after checking for changes).
back	Exit to list or search form.
menu	Exits to the calling menu.
delete	Performs delete Format Control and attempts to delete record from the database after validating the request.
views	When selected, the system presents to the user a list of alternate forms in which to display the current record.
print	Prints the current record.
printlist*	Prints the current list of records.
validitylookup	Performs Service Manager validity lookup. Validity lookup verifies via an entry in the validity table that a specific value is valid for the defined field. Using validity lookup, the system verifies that a user entered value complies with the validation rules.
export/unload	Standard Service Manager export/unload functionality (multiple record).
massunload*	Provides unload functionality for a list of records.
massadd*	Adds a new set of records that exactly duplicates an existing set of records except that new set of records will have a unique key value updated with new data. At the end of the processing there will be twice as many records in the table.
massupdate*	Provides the same updates to a set of specified fields in a list of records.
massdelete*	Deletes a specified set of records from a table.
irquery	Runs an IR Query text search.
expandarray	Expand array provides additional functionality for editing arrays, such as insert line and delete line.
count*	Counts records in the record list and displays the total count.
audithistory	Calls the standard audit history routine. The audit history is determined by the fields to audit as defined in the auditdef table.

inbox.save / inbox*	Saves the current query as a view.
approval.log	Views the approval history for the current record.
alert.log	Views the alert history for the current record.
alerts	Views the current and scheduled alerts for the current record.
pagelist / listpages	The page list shows a complete history of a record. When paging is enabled, every time an update is performed, a copy of the record is written to the page file. The page list shows the complete history of a record based on the paging file.
clocks	Displays clock records associated with this record.
xmlfill	Handles XML fields, such as the user options in Service Catalog.
createTemplate	Creates a Template record out of the current record.
applyTemplate	Applies an existing Template to the current record.



Note

Note

* These commands only apply if the record list functionality is being used.

3.5.6. Base functions in se.list.engine

Display Action	Description
exit (or back)	Returns to the search (or calling) form.
inbox.save / inbox	Prompts the user to save the current query as a new view.
count	Performs standard count functionality and displays the total record count.
refresh	Refreshes the list using the current query.
big.green	Completely exits the current module. “Big Green Arrow”
print	Prints the list of records displayed.
views	When selected, the system presents to the user a list of alternate forms in which to display the current record.
export/unload	Standard Service Manager export/unload functionality (multiple records).
massadd	Adds a new set of records that exactly duplicates an existing set of records except that new set of records will have a unique key value updated with new data. At the end of the processing there will be twice as many records in the table.
massupdate	Provides the same updates to a set of specified fields in a list of records.
massdelete	Deletes a specified set of records from a table.

3.5.7. Local variables

Local variables begin with $L. and persist only within the currently executing RAD application. The server cleans up local variables automatically when it exits a RAD application.

The following is a list of Standard Variables used with the Document Engine:

$L.action - the display action value from the display option

$L.bg - Background flag

$L.category - The category record (if available)

$L.env - The current environment record

$L.exit - internal exit parameter

$L.file - The current file variable

$L.file.save - A copy of the record in its original state

$L.format – name of the format used to display the record

$irspread – determines the IR discovery options: 0=shallow search, 2=deep search, 4=complete match

$L.mode – the mode the viewed record is in, typically add to create a new record, update to modify an existing record or close to finish processing of an existing record

$L.mult - Flag that's true if there are multiple records in the $L.file variable

$L.object - The object record

$L.phase - The phase record (if available)

$L.sql or $L.query- the current query

$L.sort - the current sort order

$L.state – The state record the system is using (=the state the record is in).

Variables that are available in View mode (when viewing a single record)

$L.fc – copy of the detail FormatControl record

$L.fc.master – copy of the master FormatControl record

3.6. Detect issues when using the Document Engine

To successfully detect the issues when using the Document Engine, you need to gather the following information:

- What dbdict and Object is being used
- What State is the record in
- What's the Process being called
- Steps followed to reproduce (STR) the problem

Research application path through the Document Engine

To detect the issues when using the Document Engine, enter `RTM:3` and `debugdbquery:999` in the Service Manager sm.ini file and then start a new client connection. Unless this user process is the very first to invoke the Document Engine processes to debug, it may not show the selection of the State or Process records in the sm.log file when doing this trace, but it will give helpful hints as to which Process was being invoked.

Find dbdict or Object used

To determine which dbdicts or Object is being used, search the log file.

Log sample:

```
1320 07/18/2006 11:00:36 RADTRACE 20 [ 1] se.get.object get.object select CPU( 0 1411 )
1320 07/18/2006 11:00:36 (0x0129AC08) DBACCESS - Cache Find against file Object found 1 record, query: file.name="pcsoftware"
1320 07/18/2006 11:00:36 RADTRACE 20 [ 1] se.get.object set.access process CPU( 0 1411 )
```

Find State for the records

The next question to look at is which State the record is in. To find that information, search for the following in the trace sm.log:

Log sample:

```
1320 07/18/2006 11:00:48 RADTRACE 10 [ 1] se.get.state select.state select CPU( 0 1491 )
```

`1320 07/18/2006 11:00:48 (0x01292FB0) DBACCESS - Cache Find against file States found 1 record, query: state="pcs.list"`

`1320 07/18/2006 11:00:48 RADTRACE 10 [ 1] se.get.state exit.normal process CPU( 0 1491 )`

Find the name of the Process

The name of the Process can be found as well, by searching in the sm.log containing the trace shown in the following example.

Log sample:

`1320 07/18/2006 11:00:50 RADTRACE 20 [ 1] se.call.process select.process select CPU( 0 1542 )`

`1320 07/18/2006 11:00:50 (0x00B56810) DBACCESS - Cache Find against file Process found 1 record, query: process="upgrade.pcs"`

`1320 07/18/2006 11:00:50 RADTRACE 20 [ 1] se.call.process run.pre.exp process CPU( 0 1542 )`

Research application errors

Processes call a number of RAD applications and execute a number of expressions, with a possibility of invoking more Processes afterwards. If any of the applications or expressions caused an error exit due to wrong syntax or wrong logic, this information can be found in the sm.log file.

Log sample:

```
Process panel run.pre.exp in RAD se.call.process encountered error in line 1 (se.call.process,run.pre.exp)
Cannot evaluate expression (se.call.process,run.pre.exp)
Cannot evaluate expression (se.call.process,run.pre.exp)
Bad arg(2) oper = (se.call.process,run.pre.exp)
Cannot evaluate expression (se.call.process,run.pre.exp)
Cannot evaluate expression (se.call.process,run.pre.exp)
Bad arg(2) oper = (se.call.process,run.pre.exp)
Cannot evaluate expression (se.call.process,run.pre.exp)
Bad arg(2) oper nullsub (se.call.process,run.pre.exp)
Cannot evaluate expression (se.call.process,run.pre.exp)
Bad arg(2) oper  in  (se.call.process,run.pre.exp)
Cannot evaluate expression (se.call.process,run.pre.exp)
Unrecoverable error in application:  se.search.objects on panel call.list.engine
Unrecoverable error in application:  se.list.engine on panel call.process.1
Unrecoverable error in application:  se.call.process on panel run.pre.exp
```

In this example (not out-of-box), the error occurred in se.call.process, run.pre.exp, or in other words, while evaluating the initial expressions of the Process. To find out which Process was causing the issue, go through the steps outlined above and note the process from the line:

`DBACCESS - Cache Find against file Process found 1 record, query: process="upgrade.pcs"`

Go to the Process record by the name of **upgrade.pcs** and check for any statements on the initial expressions tab. In this specific case, the expression will include the word nullsub. For example, the expression in question for this test may be

`$L.icount=nullsub($L.icount, anynumberIwant)`

The variable, anynumberIwant, is not a valid field, literal or variable, so it will have to be changed to prevent this issue.

Print values of variables or results of expressions

In the Document Engine, the path through a work flow is often determined by the value assign to a field or variable. To determine the value assigned to a field or variable that influences the work flow, use the **JavaScript print()** function or use the `log rtecall ($L.void=rtecall("log", $L.rc, "message"`) in the RAD expressions. The message can be a concatenated string such as

```
$L.message="The value of $L.test is " + $L.test
```

where `$L.test` is a variable that was assigned a character value.

3.7. Work order example overview

What's a work order? A work order is a specific task assigned to a single engineer that identifies some activity necessary to resolve an incident or possibly any other module if the work order system were extended to include changes, problems, or known errors. The following example is based onService Manager version 7.11 and was written to enable users to create, update, and close work orders for Incident Management. It can be modified easily for another Service Manager application.

This example creates a work order system to demonstrate how to use the Document Engine. This work order system will enable users to create work orders for an incident and view the status of these work orders. The work order system also enables users to view and update a work order from Incident Management. All work orders are associated with a particular incident in the system, and the incident can't be closed until all work orders for the incident are closed.

This example is intended for users who tailor or customize the system. You should have a good understanding of the following tailoring functions:

- Database Dictionary to create a new table
- Forms Designer to modify out-of-box forms and create new forms.
- Wizard creation tool

The work order example walks you through the following steps:

- Create a new database dictionary (dbdict) using the dbdict utility
- Specify the key fields for the table
- Create a EXWorkOrder form for the EXWorkOrder table using Forms Designer
- Create a Sequential Number File
- Modify the form to include drop-down lists
- Create a link for the EXWorkOrder form
- Create a form for the wizard to use to collect information for the work order.
- Modify the close and update incident forms
- Create aliases to link work orders to incidents
- Create State definitions
- Create Display Application Screen Definitions for open, close, and view
- Create Display Application Option Definition of Add, Cancel, Fill and Find for open, close and view.
- Modify the Process definition record im.set.close
- Test the work order example

3.7.1. Create a table

Use the dbdict utility to create a new table. In this example you will create a table named EXWorkOrder. Before you can create this table, you need to know what fields are needed and the attributes of the fields.In this example, the table stores the data for work orders associated with incidents.

To create a table, follow these steps:

1. From the System Navigator, click **Tailoring** > **Database Dictionary** to open the Database Dictionary.
2. Type EXWorkOrder in the **File Name** field.
3. Click **New**.
4. On the Fields tab, enter the following information after clicking **NewField/Key**.

Field	Description
ID	Will be filled from the number file. Type: character
RelatedID	Will be filled by the unique ID of the related record. For example: incident number. Type: character
status	Used to store the status of the work order. Type: character
initiator	The operator who opened the work order. Type: character
assignee.name	The operator assigned to the work order. Type: character
description	Used to hold the description of the work to do for the work order. Type:array of character
category	Filled from the category of the related record. Type: character
RelatedCIs	Used for the list of Configuration Items (CIs) associated with the work order. Type: array of character
impact	Type: character
urgency	Type: character
priority	Type: character
closure.code	Type: character
deadline	Date by which the work order must be completed and filled on open. Type: date/time
est.finish	The assignee's estimate for completion. Type: date/time
update.action	Type: array of character
closure.comments	Type: array of character

5. After you have added the fields to the table you need to add keys to the table. You should not exit Database Dictionary utility until you have done this for the **EXWorkOrder** table. See Add key fields to table for the detailed information.

3.7.2. Add key fields to a table

Use the dbdict utility to add keys to a new table. After you create a table named EXWorkOrder, you need to add keys to the table. Keys are used to enable indexed searches as well as ensure data consistency.

To add keys to a table, follow these steps:

1. From the Database Dictionary, click the **Keys** tab.
2. Select the first available entry for a new key field.
3. Click the **New Field/Key**.
4. Enter the following information for each key you create or edit.

Field	Description
ID	Type: unique
RelatedID	Type: no nulls
RelatedCIs	Type: nulls & duplicates
assignee.name	Type: nulls & duplicates

5. Click **OK**.

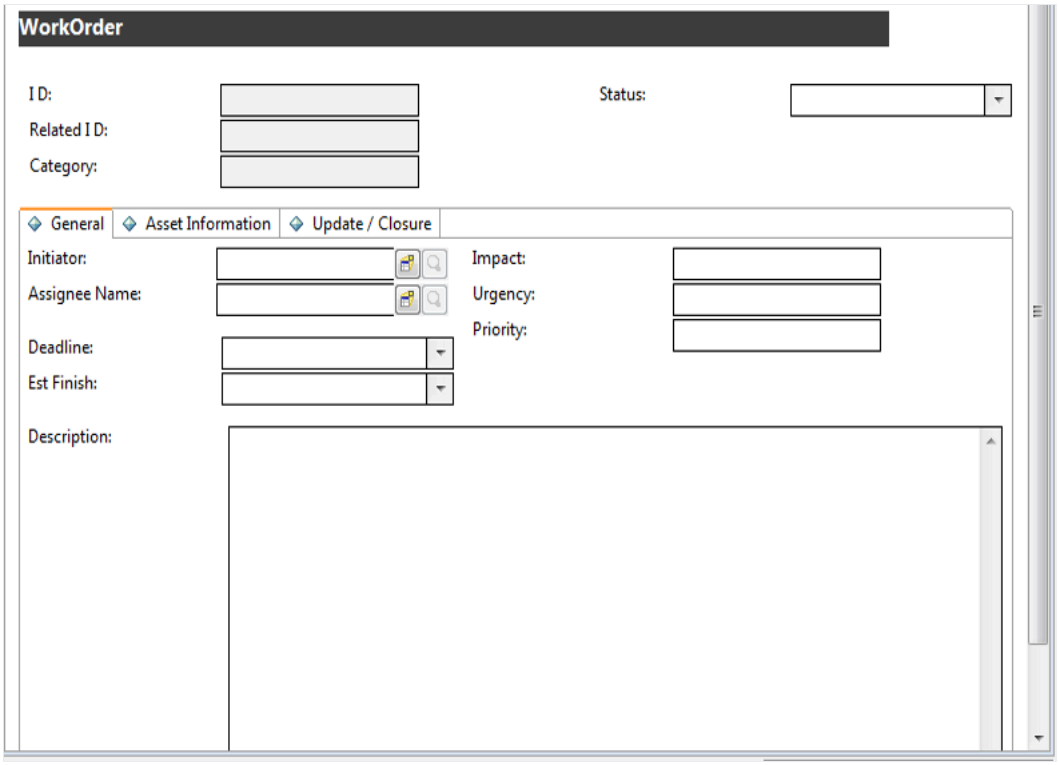
3.7.3. Create a form

Go to Forms Designer to create a form for the EXWorkOrder table. Utilizing the wizard to create the form select Detail of a single record and proceed. This creates a basic form with all of the fields which can be modified as needed. Below is an example for the EXWorkOrder form.

To create a form, follow these steps:

1. From the System Navigator, click **Tailoring** >**Forms Designer** to open Forms Designer.
2. Type EXWorkOrder in the **Form Name** field.
3. Click **New**.
4. Click **Yes** to use the Form Wizard.
5. Type EXWorkOrder for the name of the table for which to create a form.
6. Select **Detail of a Single Record** for the type of form you want to create.
7. Click **OK**.
8. Click **Proceed** to accept the defaults for the fields to include on the form.
9. Use the Forms Designer tool to modify the form layout. Below is an example of the **EXWorkOrder** form.
10. The values for Status are:
    - New (default status)
    - Open
    - Ready
    - Closed
11. The Values for the Closure Code are:
    - Implemented
    - Canceled
    - Rollback

The following is a sample EXWorkOrder form.



3.7.4. Create a copy of a form

In Forms Designer create the sc.manage.WorkOrder form by making a copy of sc.manage.problem. Modify the table column inputs to use the fields from the EXWorkOrder table.

To create a new form from a copy of a form, follow these steps:

1. From the System Navigator, click **Tailoring** > **Forms Designer** to open Forms Designer.
2. Type sc.manage.problem in the **Form** field.
3. Click **Search**.
4. Select **sc.manage.problem.g**.
5. Click **Copy/Rename** in the detail Options menu.
6. Type sc.manage.WorkOrder in the **New Name** field.
7. Click **OK**.
8. In Forms Designer, update the input fields for the columns using the EXWorkOrder table.
    - Incident ID - ID
    - Category - category
    - Related Id - RelatedID
    - Status - status
    - Assignee - assignee.name
    - Description - description,1
    - Priority - priority
    - Impact - impact
    - Urgency - urgency
9. Click **OK**.

3.7.5. Create a link for the WorkOrder form

One of the advantages of a relational database is the elimination of redundant information. You accomplish this by storing information about a particular subject in one place or table that has links to other subjects. Links are a combination of data and link definitions with

sets of conditions containing the relationships for linked information.

To create a link for the work order form, follow these steps:

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Links** to open the Link File form.
2. Type EXWorkOrder in the **Name** field.
3. Add a description in Description.
4. Click **New**.
5. Enter the following information.

Source Field Name	Target File Name	Target Field Name
initiator	operator	name
assignee.name	operator	name
RelatedCIs	device	logical.name

6. Select the initiator row (highlight) then right click **initiator** and choose **Select Line**.
7. In the link.structure.g form enter the following information for initiator. Then repeat Step 6 for **assignee.name** and **RelatedCIs** and use the following information for those fields, respectively.

Source Field (Fill To/Post From)	Target Field Name (Fill To/Post From)
initiator	name
assignee.name	name
RelatedCIs	logical.name

8. Click **Save**.
9. Click **OK**.

3.7.6. Create a sequential number file

Create sequential number file to generate sequence numbers for the records in the EXWorkOrder table.

To create a sequential number file, follow these steps:

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Sequential Numbers** to open the Sequential Number File form.
2. Type EXWorkOrder in the **Class** field.
3. Type 1 in the **Last Number** field.
4. Type Number for WorkOrder in the **Description** field.
5. Type 5 in the **Length** field.
6. Type WO in the **Prefix** field.
7. Click **Add**.

3.7.7. Create an Object definition

The purpose of this Object is to define the characteristics and behavior of the EXWorkOrder Object that determines what data needs to be included in a work order record and how the system will process work orders.

Note

Note See Object Definition form and fields for the Object definition field descriptions.

To create an Object definition, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **Objects** to open the Object Definition form.
2. Type EXWorkOrder in the **File Name** field.
3. Click **Add** to create the Object record.

4. On the Object Info tab, enter the following information.

Field	Value
Unique key	Automatically filled (ID)
Common name	Automatically filled (EXWorkOrder)
Description field	Table to hold work orders that can relate to any of the modules
Profile application	db.environment
Profile variable	$L.env
Number record name	EXWorkOrder
Category table name	category
Primary format control	EXWorkOrder
Status field	status
Assigned to fields	assignee.name
Open state	EXWorkOrder.open
Close state	EXWorkOrder.close
Default state	EXWorkOrder.view
Search state	EXWorkOrder.search

5. Click **Save**.
6. Click the **Manage Queues** tab, and then enter the following information.

Field	Value
Manage condition	true
Manage display format	sc.manage.WorkOrder
Manage default query	assignee.name=operator()
Allow add condition	false
Default query description	My WorkOrders

7. Click **Save**.
8. Click **OK**.

3.7.8. Create an Initialization Process definition

These Process definitions for the work order example specifies what initial expressions and RAD applications to use when a user opens a work order record.

To create a Process definition for initialization, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **Processes** to open the Process Definition form.
2. Type EXWorkOrder.open.initial in the **Process Name** field.
3. Click **Add**.
4. Type $L.format="EXWorkOrder" in the **Initial Expressions** tab.
5. Click **Save**.
6. Enter the following information in the RAD tab.

Field	Value
Expressions evaluated before RAD call	$L.number.record="EXWorkOrder";$L.number.type="string"
RAD Application	getnumb
Parameter Names	**Parameter Values**
name	$L.number.record
index	ID in $L.file
text	$L.number.type

7. Click **Save**.

3.7.9. Create display application screen definition

You create the display application screen definitions for open, close, and view screens for the work order forms that allow users to open, close, and view work order records.

To create a screen application definition for open, close, and view, follow these steps:

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Display Screens** to open the Display Application Screen Definition form.
2. Type EXWorkOrder.open in Screen ID.
3. Enter the following information:

Field	Value
Screen ID	EXWorkOrder.open
Title	Create New Work Order
Format	$L.format
I/O (If RIO)	true
On option 0:	redraw screen
Language	ENG

4. Click **Add** and then click **OK**.

For this example you also need to create display application screen definitions for a close screen.

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Display Screens** to open the Display Application Screen Definition form.
2. Use the following information:

Field	Value
Screen ID	EXWorkOrder.close
Title	Close Work Order
Format	$L.format
I/O (If RIO)	true
On option 0:	redraw screen
Language	ENG

3. Click **ADD** and then click **OK**.

For this example you need to create display application screen definitions for view screen also.

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Display Screens** to open the Display Application Screen Definition form.
2. Use the following information:

Field	Value
Screen ID	EXWorkOrder.view
Title	View New Work Order
Format	$L.format
I/O (If RIO)	true
On option 0:	redraw screen
Language	ENG

3. Click **Add** and then click**OK**.

3.7.10. Create display application option definitions

This procedure provides step-by-step instructions for creating add, cancel, fill, and find display application option definitions for the following WorkOrder definitions:

- open
- close
- view

In doing this, you will repeat the same steps four times for each of the WorkOrder screen definitions (open, close, view). However, each time you enter different values in the fields on the Display Application Option Definition form. The tables below the steps provide the values you need to configure the display application options.

For additional information about the display application options, see the online help topics for the display application on the Service Manager online help server.

After you create the display application screen definitions for open, close, and view, you need to create display application options for the open, close, and view screens.

To create a display application option definition, follow these steps:

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Display Options** to open the Display Application Screen Definition form.
2. Type EXWorkOrder.open in Screen ID.
3. Enter the following information.

Field	Value
Unique ID	Automatically generated (EXWorkOrder.open_add)
Action	add
GUI option	4
Text Option	4
Bank	1
Condition	true
Default Label	Add

4. Click **Add**.
5. Click **OK**.
6. Continue to follow *steps 1* through *step 6* for each of the following display option definitions using the values provided in the table.

Note

Note In some cases the value required for the Action field is unavailable in the drop-down list, but you can type the applicable value in the field.

Open - Cancel

Field	Value
Screen ID	EXWorkOrder.open
Unique ID	Generated by the system. (EXWorkOrder.open_cancel)
Action	back
GUI option	3
Text Option	3
Bank	1
Condition	true
Default Label	Cancel

Open - Fill

Field	Value
Screen ID	EXWorkOrder.open
Unique ID	Generated by the system. (EXWorkOrder.open_fill)
Action	fill
GUI option	9
Text Option	9
Bank	1
Condition	true
Default Label	Fill

Open - Find

Field	Value
Screen ID	EXWorkOrder.open
Unique ID	Generated by the system. (EXWorkOrder.open_find)
Action	find
GUI option	8
Text Option	8
Bank	1
Condition	true
Default Label	Find

Close - Cancel

Field	Value
Screen ID	EXWorkOrder.close
Unique ID	Generated by the system. (EXWorkOrder.close_cancel)
Action	back
GUI option	3
Text Option	3
Bank	1
Condition	true
Default Label	Cancel

Close - Close

Field	Value
Screen ID	EXWorkOrder.close
Unique ID	Generated by the system. (EXWorkOrder.close_close)
Action	close
GUI option	5
Text Option	5
Bank	1
Condition	true
Default Label	Close

Close - Fill

Field	Value
Screen ID	EXWorkOrder.close
Unique ID	Generated by the system. (EXWorkOrder.close_fill)
Action	fill
GUI option	9
Text Option	9
Bank	1
Condition	true
Default Label	Fill

Close - Find

Field	Value
Screen ID	EXWorkOrder.close
Unique ID	Generated by the system. (EXWorkOrder.close_find)
Action	find
GUI option	8
Text Option	8
Bank	1
Condition	true
Default Label	Find

View - Cancel

Field	Value
Screen ID	EXWorkOrder.view
Unique ID	Generated by the system. (EXWorkOrder.view_cancel)
Action	back
GUI option	3
Text Option	3
Bank	1
Condition	true
Default Label	Cancel

View - Fill

Field	Value
Screen ID	EXWorkOrder.view
Unique ID	Generated by the system. (EXWorkOrder.view_fill)
Action	fill
GUI option	9
Text Option	9
Bank	1
Condition	true
Default Label	Fill

View - Next

Field	Value
Screen ID	EXWorkOrder.view
Unique ID	Generated by the system. (EXWorkOrder.view_next)
Action	next
GUI option	10
Text Option	10
Bank	1
Condition	true
Default Label	Next

View - Find

Field	Value
Screen ID	EXWorkOrder.view
Unique ID	Generated by the system. (EXWorkOrder.view_find)
Action	find
GUI option	8
Text Option	8
Bank	1
Condition	true
Default Label	Find

View - Save

Field	Value
Screen ID	EXWorkOrder.view
Unique ID	Generated by the system. (EXWorkOrder.view_save)
Action	save
GUI option	4
Text Option	4
Bank	1
Condition	true
Default Label	Save

3.7.11. Create a State definition

These State definitions for the work order example specifies what process to use and what actions are allowed when a user opens, closes, or views a work order record.

> **Note**
>
> Note See the **State Definition field descriptions** section in States for the State definition field descriptions.

To create a State definition for open, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **States** to open the State Definition form.
2. Enter the following information:

Field	Value
State	EXWorkOrder.open
Display Screen	EXWorkOrder.open
Initialization Process	EXWorkOrder.open.initial
Format	$L.format
Input Condition	true

3. Click **ADD**.

To create a State definition for close, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **States** to open the State Definition form.
2. Enter the following information:

Field	Value
State	EXWorkOrder.close
Display Screen	EXWorkOrder.close
Initialization Process	EXWorkOrder.close.initial
Format	$L.format
Input Condition	true

3. Click **ADD**.

To create a State definition for view, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **States** to open the State Definition form.
2. Enter the following information:

Field	Value
State	EXWorkOrder.view
Display Screen	EXWorkOrder.view
Initialization Process	EXWorkOrder.view.initial
Format	$L.format
Input Condition	true

3. Click **ADD**.

To create a State definition for search, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **States** to open the State Definition form.
2. Enter the following information:

Field	Value
State	EXWorkOrder.search
Display Screen	db.search
Initialization Process	EXWorkOrder.view.search
Format	$L.format
Input Condition	true

3. Click **OK**.

3.7.12. Add a Close button

When a user completes the task or tasks, the work order should be closed. This procedure describes how to put a close button on the EXWorkOrder form and update the status of the work order to close. This procedure adds a display application option definition, adds a State definition for EXWorkOrder.view, and adds a Process definition for EXWorkOrder.close.



Note

Note This procedure doesn't explain how to include validation for this activity. You would use Format Control to add validation.

To create a display application option definition for EXWorkOrder.view, follow these steps:

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Display Options** top open the Display Application Screen Definition form.
2. Type EXWorkOrder.view in the **Screen ID** field.
3. Enter the following information.

Field	Value
Unique ID	Automatically generated (EXWorkOrder.view_close)
Action	close
GUI option	5
Text Option	5
Bank	1
Condition	true
Default Label	Close

4. Click **Add**.
5. Click **OK**.

To update the EXWorkOrder.view state definition, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **States** to open the State Definition form.
2. In the State field, type EXWorkOrder.view and then click **Search**. The EXWorkOrder.view state definition form opens.
3. Add the following to update the state definition form.

Field	Value
Display Action	close
Process Name	EXWorkOrder.close
Condition	true

4. Click **Save** and then click **OK**.

To add a process definition record for EXWorkOrder.close, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **Processes** to open the Process Definition form.
2. Type EXWorkOrder.close in Process Name.
3. Click **Add**.
4. On the Initial Expressions tab, type the following expression:
    - status in $L.file="Closed"
    - $L.mode="closed"
5. Click **Save**.

6. On the RAD tab, enter the following information:

Field	Value
Expressions evaluated before RAD call	$L.EXaction="update"
RAD Application	se.base.method
Condition	true
Parameter Names	Parameter Values
file	$L.file
prompt	$L.EXaction
second file	$L.file.save
record	$L.fc
second.record	$L.object
boolean1	false

7. Click **Save** and then **OK**.

3.7.13. Create a wizard for the work order

This examples uses a wizard to create a work order because a wizard simplifies the task of opening a work order for the user and the functionality is already available in the system.

This example uses the wizard creation tool to create a wizard to allow users to create a work order from the Incident Management module. When a user creates a work order from an incident, the wizard prompts the user for information and auto-fills some of the fields in the work order.

To create a wizard for the work order form, follow these steps:

1. From the System Navigator, click **Tailoring** > **Wizards** to open the Wizard Information form.
2. Type Create Workorder - 1 in the **Wizard Name** field.
3. Click **Add** to create the Wizard record.
4. On the Wizard Info tab, enter the following information:

Field	Value
Brief Description	Create a new work order.
Window Title	Create Workorder
Title	Create Workorder
Start Node	Select (set to true) to indicate that this is the first wizard in a series of wizards in cases where you have a series of wizard records.

5. Click **Save**.
6. In the File Selection tab of the Wizard Information form, select the **Select $L.file by** tab.
7. On the Select $L.file by tab, enter the following information:

Field	Value
Create a record	true
of type	EXWorkOrder

8. Click **Save**.
9. On the Usage tab, enter the following information:

Field	Value
Wizard Usage	Click **Request user input** in the Wizard Usage section.
Sub Format to Display	Type createWO.assigneeAndCIs
Display Screen	Type wizard.display
Activate "Finish" option	Select (set to true) to have a Finish button on the wizard form.

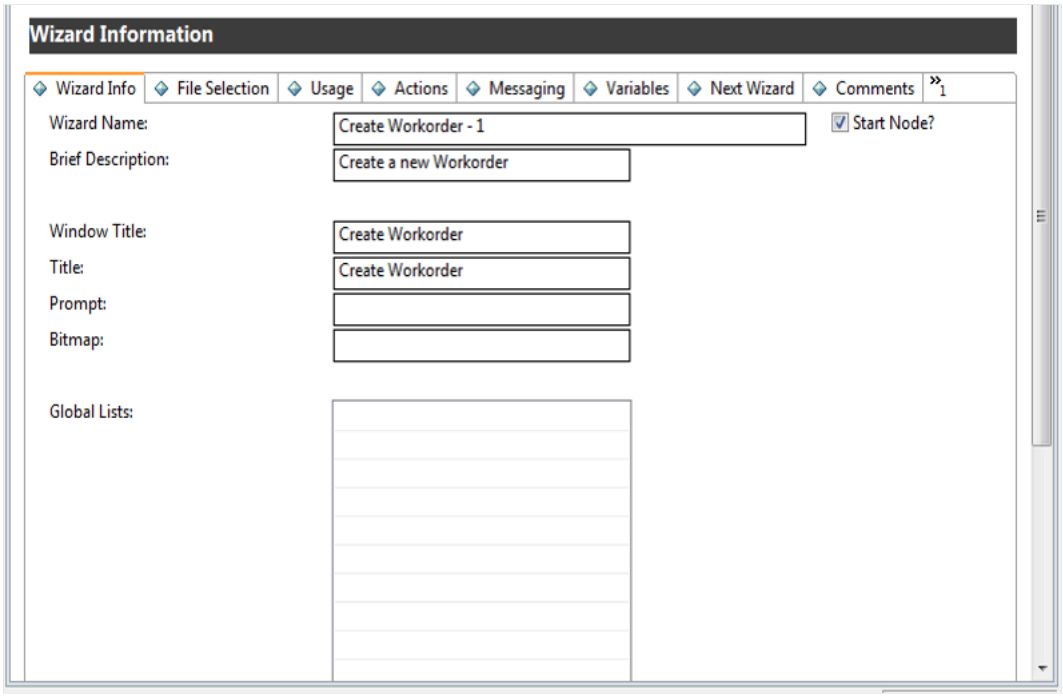
10. Click **Save**.
11. On the Actions tab, enter the following information:

Field	Value
Perform Actions On	Select Current File ($L.file).
Actions to Perform > Expressions	initiator in $L.file=operator() status in $L.file="New" RelatedID in $L.file=number in $relatedRec category in $L.file=category in $relatedRec impact in $L.file=initial.impact in $relatedRec urgency in $Lfile=severity in $relatedRec priority in $L.file=priority.code in $relatedRec
Display Record(s) when complete?	Click **Display Record(s) when complete?**
Mode	Select Add.

12. Click **Save**.
13. On the Cancel Expressions tab, enter the following information:

Field	Value
Expressions Executed on Cancel	cleanup($relatedRec)

14. Click **Save** and then **OK**.



3.7.14. Add a Process definition record

The Process definition record defines how the system responds to a user's action. The Process definition uses RAD expressions, JavaScript, and calls to existing RAD applications to perform actions against the current record, in this case, a work order record.

To add a process definition record, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **Processes** to open the Process Definition form.
2. Type create.WorkOrder in the **Process Name** field.
3. Click **Add**.
4. On the Initial Expressions tab, type the following expression:
   - $L.void=fduplicate($relatedRec, $L.file)
   - $relatedCIs={}
5. Click **Save**.
6. On the Initial Javascript tab, type the following expression:
   system.vars.$relatedCIs=system.library.BSGFunctions.getMembers(system.vars.$L_file.affected_item, false, 3

   Note
   Note This expression must be entered on one line.

7. Click **Save**.

8. On the RAD tab, enter the following information:

Field	Value
Expressions evaluated before RAD call	Type the following two expressions: $L.wiz.name="Create Workorder-1" if (not null(logical.name in $L.file)) then ($relatedCIs=insert ($RelatedCIs, 1, 1, logical.name in $L.file)) **Note** Note Be sure to enter this expression on one line. Also note that when entering this expression, there is no space after the word insert. For example, the expression above continues at the word 'insert' as follows: insert($RelatedCIs, 1, 1, logical.name in $L.file))
RAD Application	wizard.run
Parameter Names	name
Parameter Values	$L.wiz.name
Parameter Names	text
Parameter Values	$L.exit
Condition	true

9. Click **Save**.
10. On the Final Expressions tab, type cleanup($relatedRec).
11. Click **Save** and then **OK**.

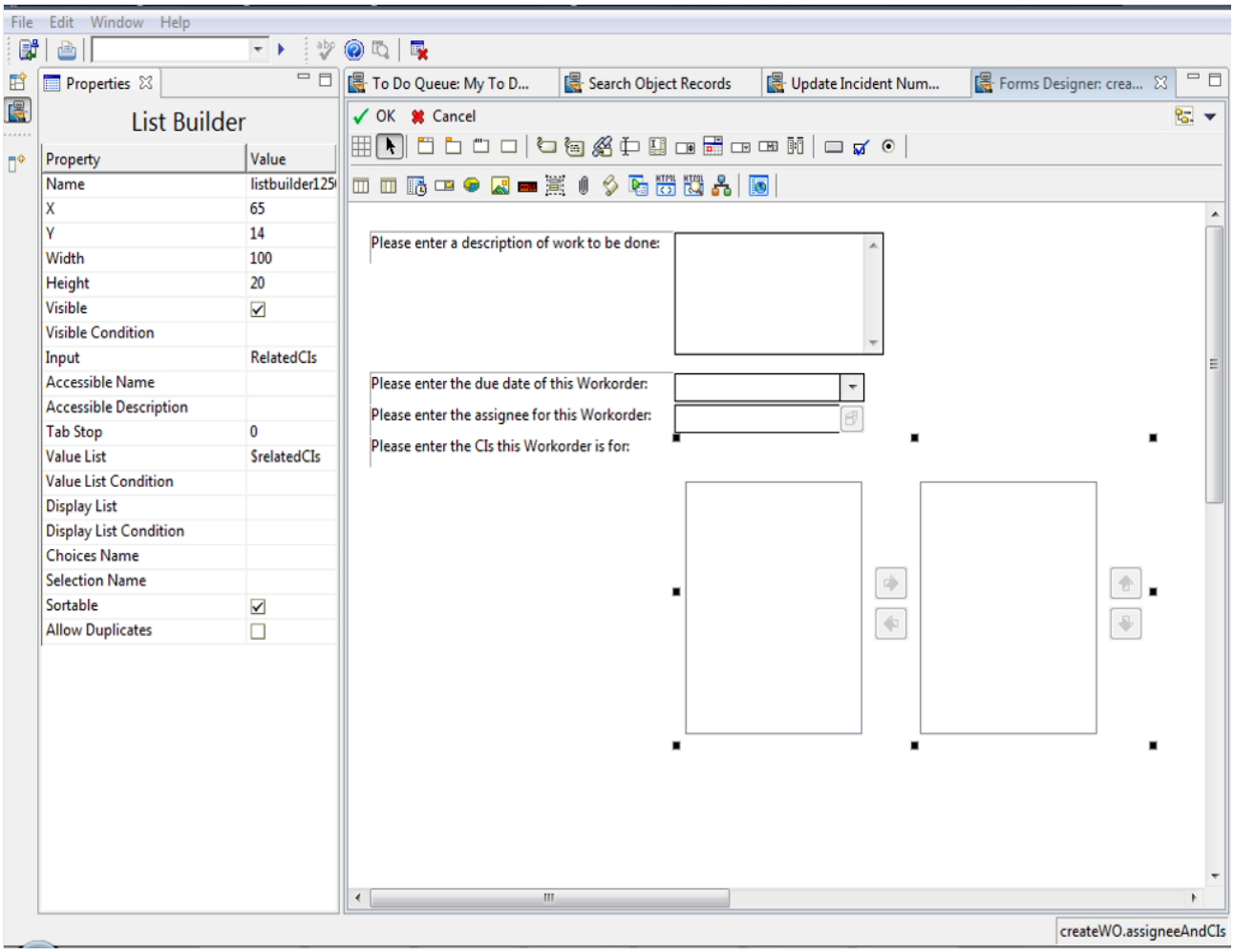
3.7.15. Create an input form for the wizard

This form is the first form to display in the work order wizard. The user enters the information the user needs to provide for the wizard to create the work order record. Use Forms Designer to create this form. For this example, the form name is createWO.assigneeAndCIs.

To create an input form for the wizard, follow these steps:

1. From the System Navigator, click **Tailoring** > **Forms Designer** to open Forms Designer.
2. Type createWO.assigneeAndCIs in the Form for the form name.
3. Click **New**. You don't need to use the Forms Designer wizard for this form.
4. Create the following input fields on the form:
   - Description (Please enter a description of the work to be done)
   - Due date (Please enter the due date for this work order)
   - Assignee (Please enter the assignee for this work order)
   - CIs (Please enter the CIs this work order is for)
5. The Properties for this form should include:
   - Input: **RelatedCIs**
   - Value List: **$relatedCIs**
   - Sortable: checked
6. Click **Save**.

The following figure shows a sample createWO.assigneeAndCIs form for the EXWorkorder wizard input form.



3.7.16. Modify the close and update incident forms

For the work order example, you need to modify the close incident and update incident form so that you can view the work orders assigned to the incident and then double click and read or edit the work order from the incident. Use Forms Designer to update the *IM.update.incident* and *IM.close.incident* form. To do this you must add a tab and then a form on the tab on both the IM.update.incident and IM.close.incident forms.

To modify update incident form ,follow these steps:

> **Note**
>
> Note Use these same general procedures to update the close incident form.

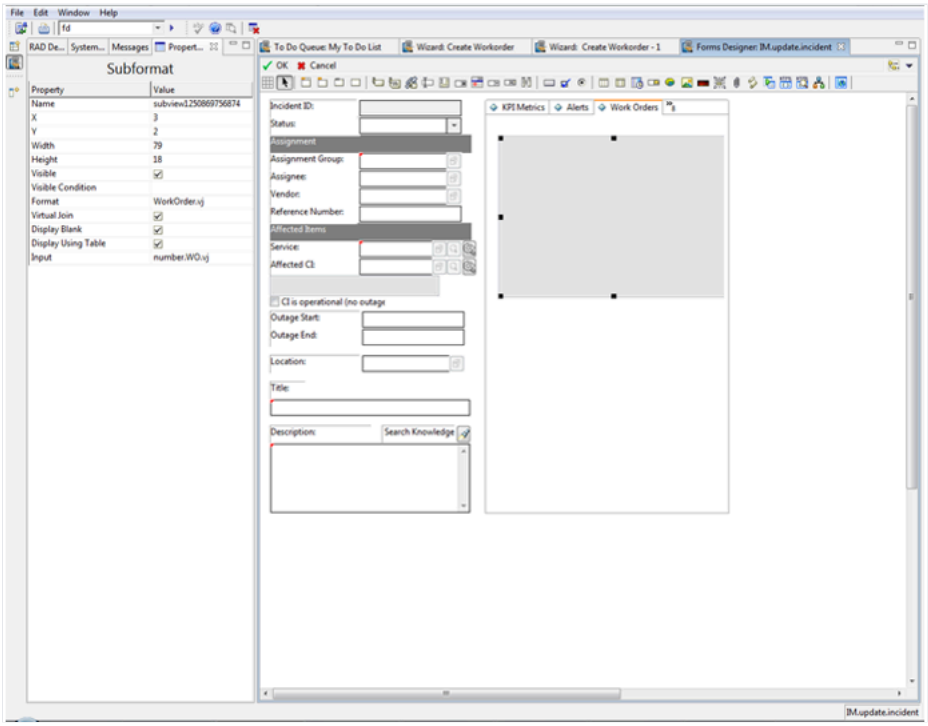
1. From the System Navigator, click **Tailoring** > **Forms Designer** to open the Forms Designer search form.
2. Type IM.update.incident in the Form for the form name.
3. Click **Search**.
4. Click **Design**.
5. Add a new tab to the existing notebook.
6. Set the caption of the tab to WorkOrder.
7. Add a subformat with the following properties:
   - Visible: checked
   - Format: WorkOrder.vj
   - Virtual Join: checked
   - Display Blank: checked

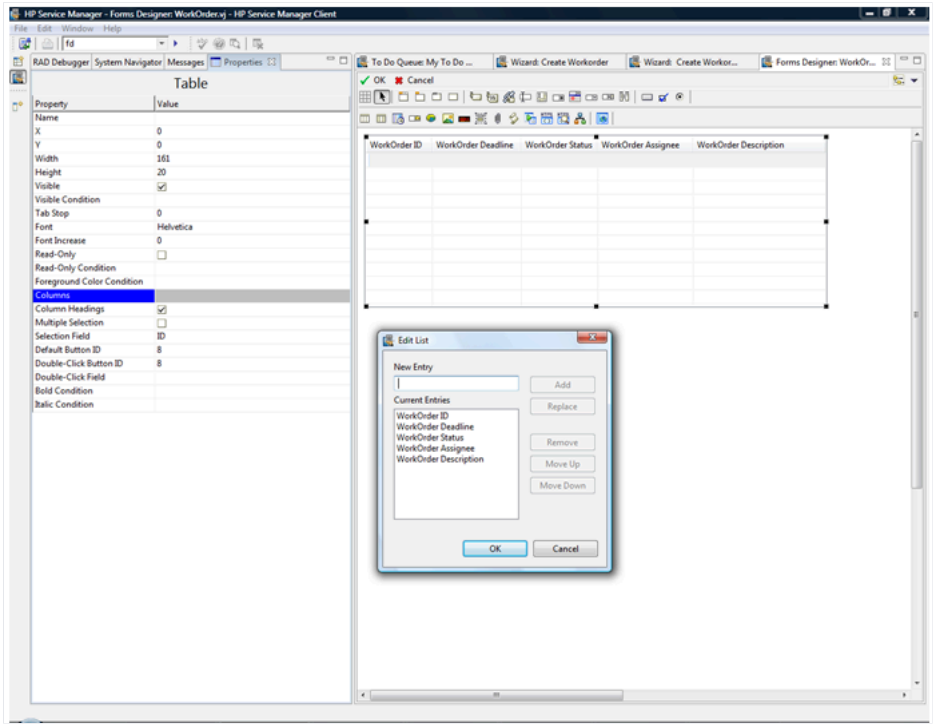
- Display Using Table: checked
- Input: number.WO.vj

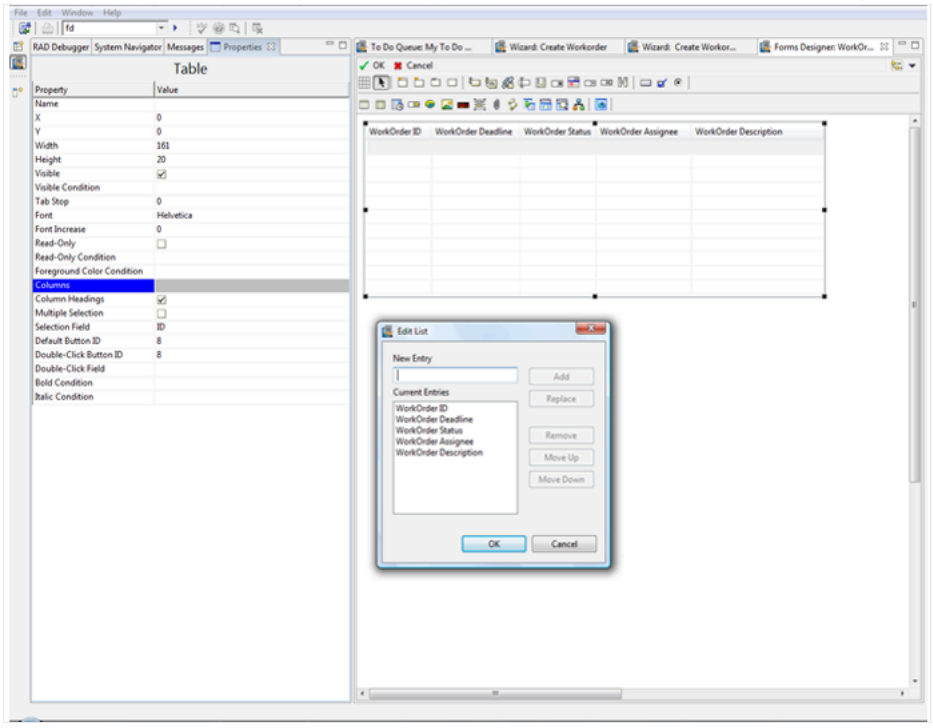
8. Click **Save**.
9. Use the Forms Designer tool to create the WorkOrder.vj form:
   - WorkOrderID: ID
   - WorkOrder Deadline: deadline
   - WorkOrder Status: status
   - WorkOrder Assignee: assign.name
   - WorkOrder Description: description
10. Continue to follow *steps 1* through *step 8* for **IM.close.incident**.

> **Note**
> Note You can use the WorkOrder.vj form created in *step 9* for the virtual join.

The following figures show samples of the tabs to add.







3.7.17. Create alias in probsummary table for link

In the work order example, users need to be able to access a work order from an incident record when the incident has a work order associated with it. In order to do this, you must create an alias in the probsummary table and then use that alias to create a link between the probsummary table and the EXWorkOrder table.

To add an alias to the probsummary table, follow these steps:

1. From the System Navigator, click **Tailoring** > **Database Dictionary** to open the Database Dictionary form.
2. Type probsummary in the **File Name** field.
3. Click **Search**.

4. On the Fields tab, select the number field and click **Edit Field/Key**.
5. Click **Create Alias** and type number.WO.vj for name and character for type.
6. Click **OK**.

To link the EXWorkOrder table with the probsummary table, follow these steps:

1. From the System Navigator, click **Tailoring** > **Tailoring Tools** > **Links** to open the Link File form.
2. Type probsummary in the **Name** field.
3. Click **Search**.
4. Click after the last entry to create a blank line for a new entry.
5. In the Source field, type number.WO.vj.
6. Select the entire new line and then click **Select Line**.
7. Enter the following information:

Field	Value
Field (From/Source)	number.WO.vj
File (To/Target)	EXWorkOrder
Field (To/Target)	RelatedID
Query	$query
Expressions	$query="RelatedID=\""+number in $File+"\""

8. Click **Save** and then **Back**.
9. Repeat steps to create a link line for the ID field.
10. Enter the following information.

Field	Value
Field (From/Source)	ID
File (To/Target)	EXWorkOrder
Field (To/Target)	ID
Query	$query
Expression	$query="ID=\""+nullsub(cursor.field.contents(), "xxx")+"\""

11. Click **Save** and then **Back**.

3.7.18. Modify im.set.close Process definition

This process needs to be modified so that a user can't close an incident if there are work orders still open for the incident.

To modify a process definition record, follow these steps:

1. From the System Navigator, click **Tailoring** > **Document Engine** > **Processes** to open the Process Definition form.
2. Type im.set.close in the **Process Name** field.
3. Click **Search**.

4. On the Initial Javascript tab, type the following JavaScript:

```
var WO=new SCFile ("EXWorkOrder")
var FoundOpenWO=WO.doSelect ("RelatedID=\""+system.vars.$L_
file.number + "\""+" and status ~=\"" + "Closed" + "\""
if (FoundOpenWO == RC_SUCCESS)
{
system.vars.$openWO=true;
}
else
{
system .vars.$openWO=false;
}
```

5. Click **Save**.
6. On the RAD tab, enter the following information:

Field	Value
Note: Some of the fields on the RAD tab are pre-filled with values that you don't need to modify	
RAD Application - Select the empty section below the section calling the RAD Application us.consume.wrapper and then enter the following information:	
Expressions evaluated before RAD call	$L.text="There are still open work orders. This incident can't be closed yet."
RAD Application	apm.mb.ok
Condition	$openWO=true
Parameter Names	text
Parameter Values	$L.text

7. Click **Save**.
8. On the Final Expressions tab, enter the following information:
    - if ($openWO=true) then ($L.exit="badval")
    - $L.exit="closestate"
9. Click **Save** and then **OK**.

3.7.19. Test the work order example

After you complete the tasks to create a work order system, you need to verify that it works correctly. To verify the basic functionality of the work order example, do the following:

- In Incident Manager, find an open incident or create one.
- Use the Options menu or button on the tool bar to create a work order. Create Workorder should display on the Options menu.
- In the Workorder wizard, enter the data to create a work order for the incident.
- Add the work order to the incident and save the changes to the incident.
- Open the incident for editing, and from the WorkOrder tab edit the work order.
- Save your changes.
- Open the incident again, and this time edit the work order and close it.
- You should now be able to close the incident.
- Repeat these steps, but this time create two work orders for an incident.
- Close only one work order and then attempt to close the incident. The system should generate an error message indicating that there are still open work orders for the incident, and therefore it can't be closed.
- Close all work orders for the incident. You should now be able to close the incident.

4. Web Services

You can write Web Services and their clients in any programming language and for any platform. Service Manager Web Services ships with examples using both the Java™ and Visual C++® programming languages.

For information on how to publish or consume Web Services using Service Manager, see the Web Services section. You can use the examples in this section as templates.

5. Service Manager Integration Suite (SMIS)

The *Service Manager Integration Suite (SMIS)* document aids developers in using SMIS to develop integrations for Service Manager. This document describes the Service Provider Interface (SPI) of SMIS, and provides instructions on how to develop a template for a schedule-based or UI-based integration, as well as on how to register an integration template in SMIS.

Overview

Develop an integration template

Processing logic of the mapping functionality

Related topics

Integrations
Integration Manager
Integration Instance Mapping
Integration Instance Information fields
Add or delete an integration instance
Edit an integration instance
Add or delete field mappings
Edit field mappings
Configure value mappings
Configure a callback

5.1. Overview

Service Manager Integration Suite (SMIS) is a platform that provides centralized management of integration instances, which fall into two categories:

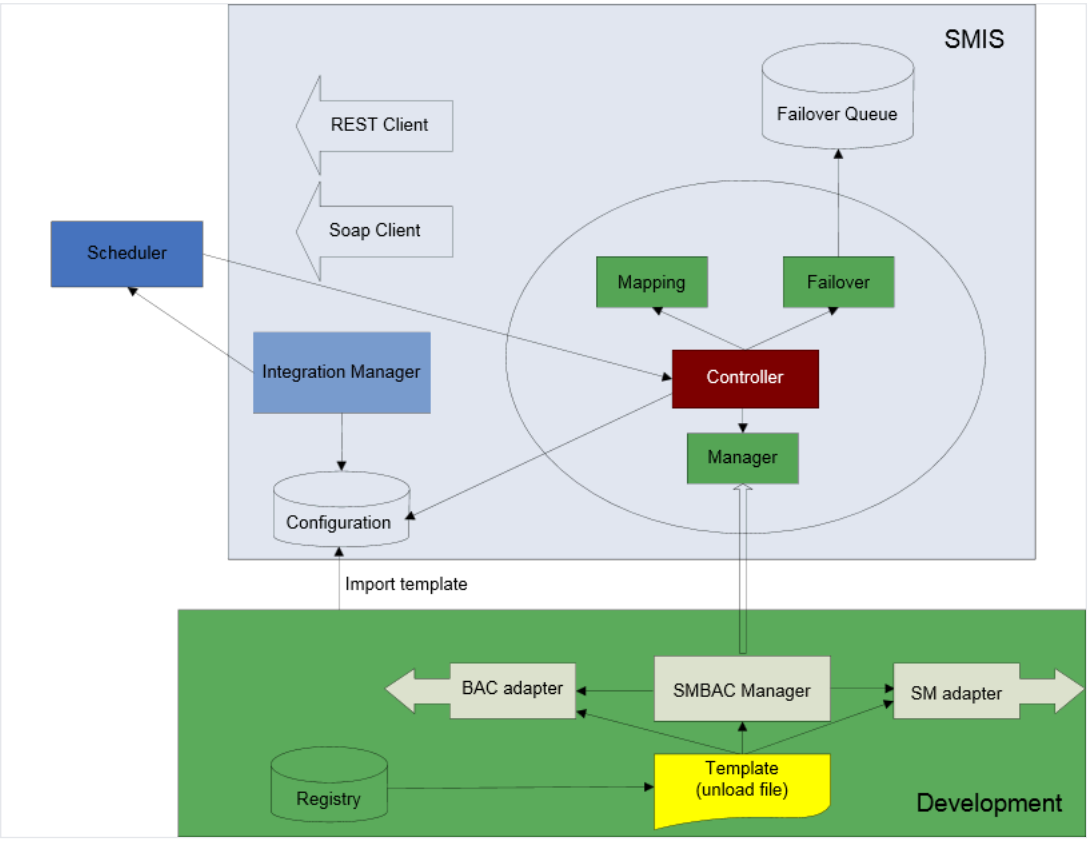
- **Schedule-based**: runs as a schedule in the background.
- **UI-based**: Can be only invoked in the user interface (UI).

SMIS is also a plug-in-based development platform that enables you to complete the following tasks:

- Reuse common functions and resolves conflicts across integrations
- Customize a default controller to suit the workflow of your integrations
- Use JavaScript to develop integration templates

An integration must be registered as a template so that it can be added to SMIS. To register an integration as a template, the integration must be developed by following the Service Provider Interface (SPI) provided by SMIS.

The functional block diagram of SMIS and a sample integration are shown in the following figure.



SMIS SPI

SMIS provides the following SPI components.

Manager SPI

The following table describes the functions used to develop a manager.

Function	Parameter	Return	Description
appendTasks			Directs the source adapter to retrieve records, converts these records into tasks, and calls the task manager to push these tasks into the task queue.
preProcess	task	Boolean	Prepares the destObj and actions. If it returns false, the task will be ignored and removed.
process	task	Boolean	Returns true or false when the task is or is not processed successfully.
postProcess	task		Performs postprocessing after the task is processed.
getDestObj		Object	SCFile or Object.
getAction		String	Returns a customized action defined by the manager. Can be any value (for example: Insert/Update/Delete).
isScheduleBased		Boolean	Indicates whether the integration is schedule-based or not.
finalize			Performs finalization actions.
initParams			Initializes parameters when the instance is first added. Use `this.configItem.setConfigParameterValue` (<parameter_name>, <value>) to set parameter values.

Adapter SPI

The following table describes the functions used to develop source and destination adapters.

Function	Parameter	Return	Description
getRecords		Array	Retrieves records from external sources, and returns them to the manager.
sendRecord			Sends out the record according to the action. For example, `sendRecord (data, "delete")`.
sendRecord	Record	Object	JavaScript object that contains the result data to send.
sendRecord	Action	String	Customized action. For example, `insert`, `update`, or `delete`.
getFields		Array	Returns an array of field names, types, and descriptions. The field information will be used for mapping.

Mapping SPI

The following table describes the functions provided by the mapping function in SMIS.

Function	Parameter	Return	Description
validate		boolean	Validates the input values.
validate	inRecord	Object	Input field values.
validate	direction	tring	Mapping direction.
getOutRecord		Object	Gets outRecord by inRecord according to field mapping and value mapping (not including callback).
getOutRecord	inRecord	Object	Input field values.
getOutRecord	direction	String	Mapping direction.
setFieldValues			The final processing step of mapping, which sets the mapped value to destObject.
setFieldValues	inRecord	Object	Input field values.
setFieldValues	outRecord	Object	The object of getOutRecord.
setFieldValues	destObject	Object	The final object of the mapping result.
setFieldValues	smisContext	Object	The container that contains context values.
setFieldValues	direction	String	Mapping direction.

TaskManager SPI

The following table describes the functions provided by the task manager in SMIS.

TaskManager SPI

Function	Parameter	Return	description
readTasks		Array	Reads tasks by instance ID.
readTasks	intId	String	Instance ID.
removeTask+		Object	Removes the task if it is processed successfully.
removeTask+	task	Object	Task object.
setFieldValues			The final processing step of mapping, which sets the mapped value to destObject.
updateTask		Object	Updates the task if it is processed unsuccessfully.
updateTask	task	Object	Task object.

Development procedure

Developing integrations with the SMIS SPI involves different steps for schedule-based and UI-based integrations.

Schedule-based integrations

To develop a schedule-based integration, perform the following tasks:

1. Initialize global variables.
2. Develop an endpoint adapter and a Service Manager adapter.
3. Develop a manager.
4. Develop a custom controller.
5. SMIS provides a default controller. Determine if the default workflow suits the integration. If not, develop a custom controller.
6. Register the integration as a template.
7. Export the integration template into an unload file.

UI-based integrations

To develop a UI-based integration, perform the following tasks:

1. Initialize global variables.
2. Develop a manager.
3. Implement UI controls.
4. Register the integration as a template.
5. Export the integration template into an unload file.

5.2. Develop an Integration Template

This section describes the tasks to develop an integration template.

Initialize global variables

Each integration has global variables, which you must initialize. To initialize the global variables of an integration, follow these steps:

1. Add a field to the `info` table, using the integration template name as its name.

   Note

   Note If the integration has global parameters, the field must be a structure field.

2. If the integration has global parameters, add all these global parameter names as fields to the newly added structure.
   For example, an integration template named `SMBSM` has two global parameters: `PI` and `BIR`. You need to add a structure named `SMBSM` under the SMIS structure, and then add two fields, `PI` and `BIR`, to the `SMBSM` structure. If the template `SMBSM` has no global parameters, you only need to add one field named `SMBSM` to the SMIS structure.

Develop a source adapter and a destination adapter

To develop a source adapter, use `readRecords` to return records to the manager.

To develop a destination adapter, use `sendRecord` to process records from the manager.

Note

Note The `getFields` function defines the fields used in an integration. See `smis_TestSrcAdapter` and `smis_TestDestAdapter` in the Service Manager script library for examples.

Develop a manager

A manager is required for both schedule-based and UI-based integrations.

To develop a manager, follow these steps:

1. Develop `appendTasks` to prepare tasks in the task queue.
2. Put the logic in either the `preprocess` or `process` method.
3. Do cleaning in the `postProcess` or `finalize` method.
4. Prepare the destination object in the `getDestObj` method for the mapping function to set mapped values.
5. Prepare the action in the `getAction` method to use in the mapping callback(s).
6. Implement `isScheduleBased` to indicate whether the integration is schedule-based or not.

Tip

Tip See **smis_TestManager** in the Service Manager script library for an example.

Note

Note For a UI-based integration, the manager only needs to implement `isScheduleBased` and add the integration-specific methods to it. See the following script for an example.
An example manager for a UI-based integration

```
var Class = lib.smis_Prototype.getClass();
var PIManagerClass = Class.create(lib.smis_Manager.getClass(),
{
getUrl: function(vFile) {
var baseUrl = this.configItem.getConfigParameterValue("baseurl");
var url = baseUrl + "&IsmEntityId="+vFile["number"];
url += "&IsmSubject=";
var device = new SCFile("device");
var RC = device.doSelect("logical.name=\"" + vFile["logical.name"] +
"\"");
if ( RC == RC_SUCCESS && device["ucmdb.id"] != null){
url += device["ucmdb.id"];
} else {
url += vFile["logical.name"];
}
return url;
},
isScheduleBased: function() {
return false;
}
});
function getClass() {return PIManagerClass;}
```

Develop a custom controller

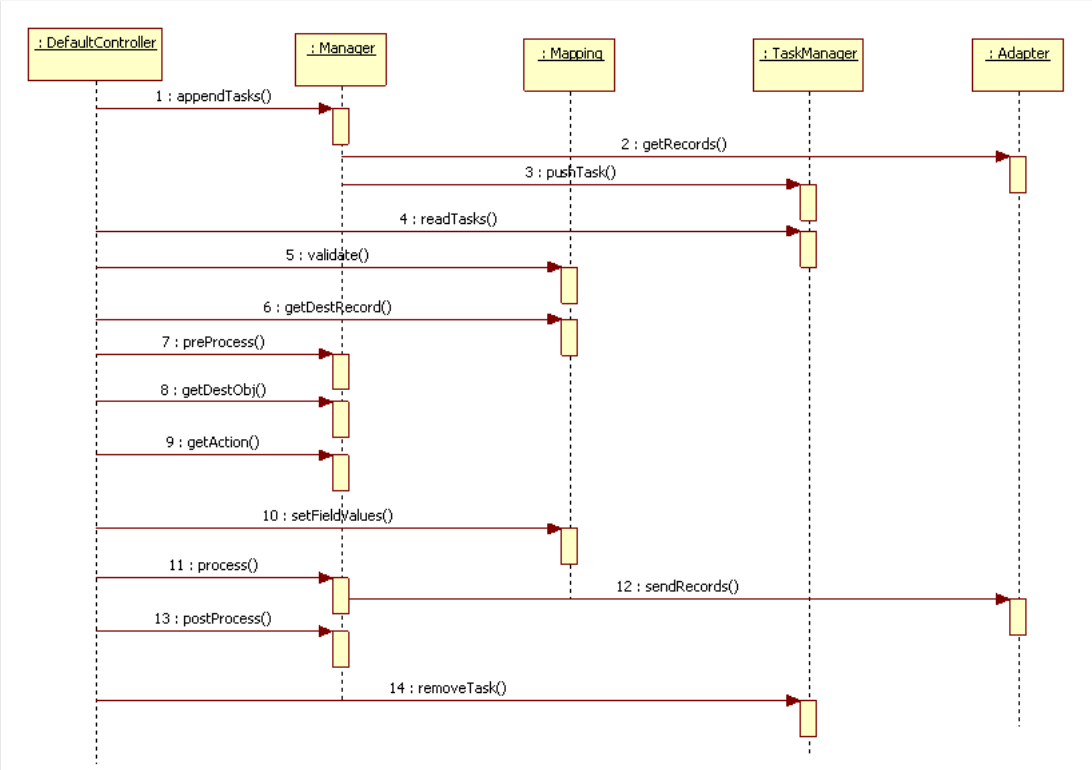
A controller controls interactions between the manager, source and destination adapters, mapping function, and task manager of a schedule-based integration.

Note

Note UI-based integrations don't need a controller.

SMIS provides a default controller. The following figure shows the workflow of the default controller. If this default workflow doesn't suit your integration, develop a custom controller.

Workflow of the Default Controller



When developing a custom controller, you can refer to **smis_Controller** in the Service Manager script library.

The typical workflow of a controller is as follows:

1. Call the manager to do the following to prepare tasks:
    1. Delegate its source adapter to retrieve data from the endpoint;
    2. Wrap the data as tasks;
    3. Pass the tasks to the taskManager to save to the failover queue.
2. Read all the tasks from the failover queue.
3. Pass the data in the task to the mapping function for validation.
4. If the data passes validation, get the destRecord (which is the result of value mapping) from the mapping function.
5. Call the manager to preprocess and prepare the following:
    - The destObject (which is the target to save or update) ;
    - The action (for example, add/save/delete) that the manager should perform to process the destObject.
6. Call the mapping function to process the destObject. All the final mapped values by the final mapping are set to the destObject.
7. Call the manager to process the final destObject, and to return the result (success or failure).
8. If the result is success, remove the task from the failover queue; If the result is failure, increase the retry count and update it to the failover queue.
9. Call the manager's `postprocess` and `finalize` functions to do cleaning.

Normally, you don't need to create a custom controller. You can leave some processes empty if you don'thing in them. In some cases, you may not need a mapping function and failover queue, so you can create a simple controller.

Implement UI controls

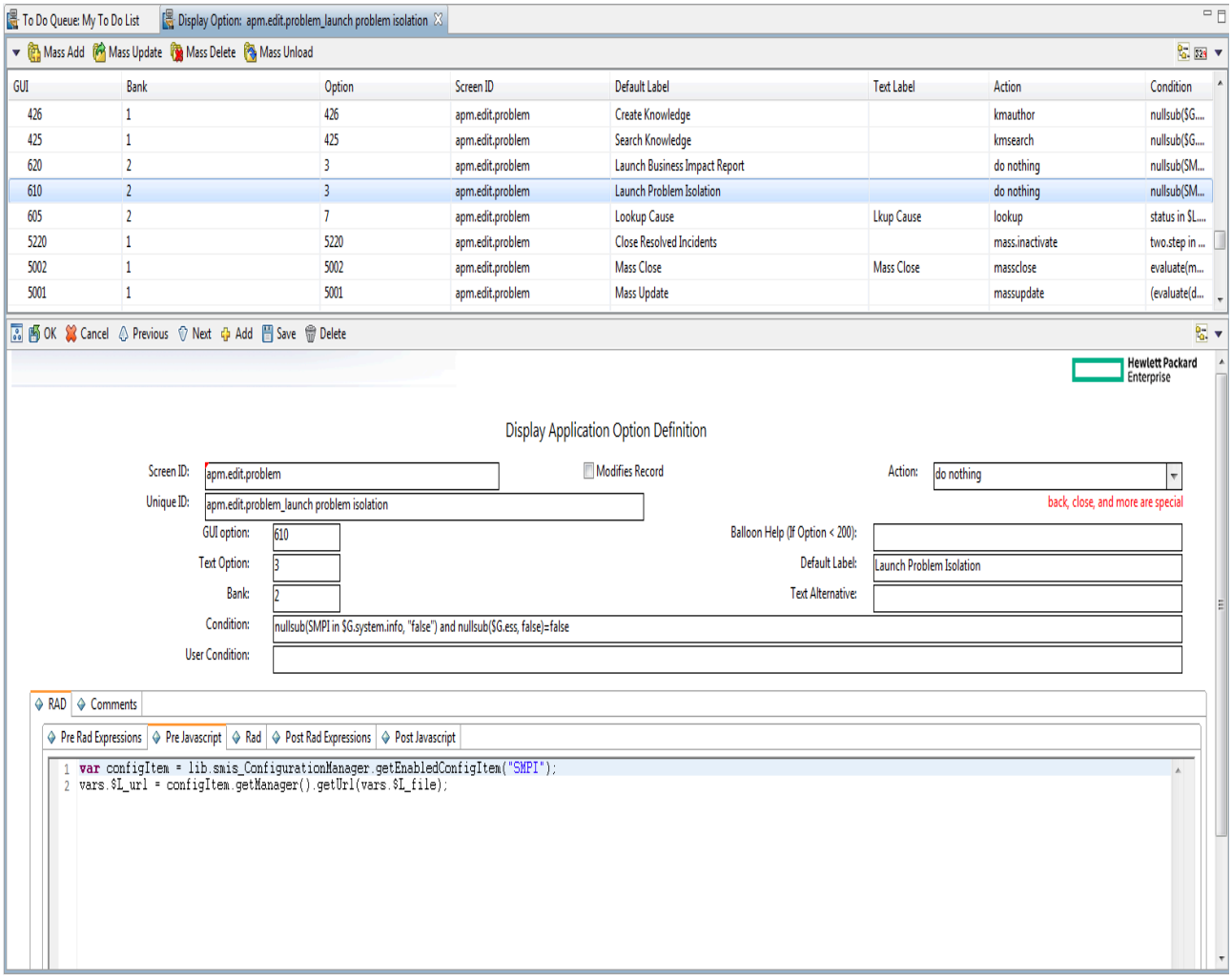
An integration normally comes with new UI controls, such as new menu options or buttons. To control the behavior of these UI controls, the integration needs to interact with SMIS. The following are examples:

- Getting parameter values specified in SMIS;
- Calling some business logic in the integration's manager (for example, to prepare a URL);
- Deciding if a UI element should be visible/enabled by checking the integration's global variables specified in SMIS.

To illustrate new UI controls, consider the Service Manager to Business Availability Center (BAC) Problem Isolation integration. This integration includes a new menu option to the Incident form: **Launch Problem Isolation**. When you select this menu option, a new

browser window opens. The URL of this window is based on two things: the value of "baseurl" specified in SMIS and the affected CI field of the incident.

To implement this feature, you can use the Display Options tailoring tool to create a display option record, in which the script on the **Pre Javascript** tab controls the above described behavior. See the following figure.



The following examples show you some of the values you can retrieve or set. This is not an exhaustive list:

- To get the enabled instance of the integration:
  `var configItem =lib.smis_ConfigurationManager.getEnabledConfigItem(<templateName>)` ;
  For example: `var configItem =lib.smis_ConfigurationManager.getEnabledConfigItem(SMPI)`
- To get a URL from the manager:
  `vars.$L_url = configItem.getManager().getUrl(vars.$L_file)` ;
- To get a parameter value:
  `var paramValue = configItem.getConfParamaterValue(paramName)`
- To set the property of a UI element:
  Each integration has global variables, which can be used in RAD expressions like `SMBSM in vars.$G.system.info` . If an integration has global parameters, they can be used the same way, for example: `PI in vars.$G.system.info` . You can use these global variables to set the property of a UI element (for example, to set a menu item to be visible/invisible, or to set a button to be enabled/disabled).

Register an integration template in SMIS

You need to register an integration as a template in SMIS before it is available in the template list in SMIS. You can view or edit an integration template after it is registered in SMIS.

Register an integration template

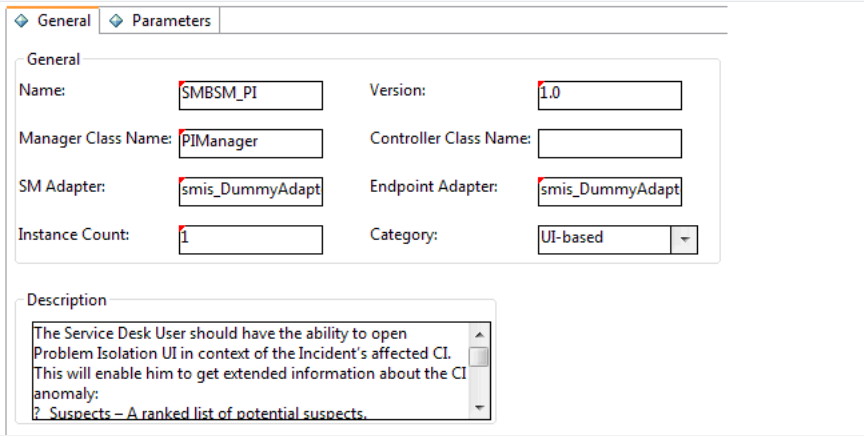
To register an integration as a template, follow these steps:

1. Register the general information of the integration template.

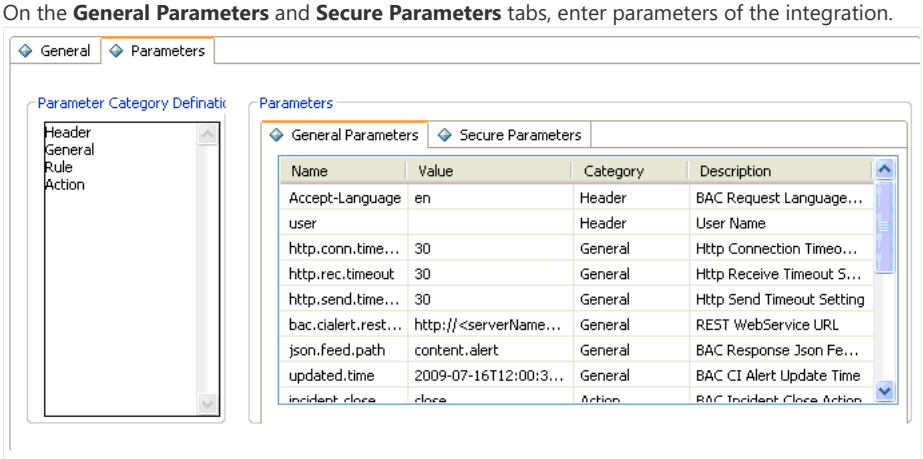
1. Sign in to Service Manager as a system administrator.
2. From **Database Manager**, open the **SMISRegistry.g** form.
3. On the **General** tab, enter the following information of the integration:

Field	Description
Name	Name of the integration template.
Version	Version of the integration template.
Manager Class Name	Script name of the manager of the integration.
Controller Class Name	Script name of the controller of the integration. If this field is left blank, a default controller is used.
SM Adapter	Name of the Service Manager adapter.
Endpoint Adapter	Name of the endpoint adapter.
Instance Count	Maximum allowed number of instances of the integration.
Category	Category of the integration (Schedule-based or UI-based).

See the following screenshot for an example.



2. Enter parameters required for the integration.
    1. Select the **Parameters** tab.
    2. If necessary, enter parameter categories for the integration.
    3. On the **General Parameters** and **Secure Parameters** tabs, enter parameters of the integration.



Note

Note Newly added parameter categories aren't available in the `Category` list until the form is saved and then reopened.
There is a default `Global` category. SMIS will initialize all parameters of this category to global variables, which can be used in Service Manager. These parameters can only be set to true or false.

3. Edit the out-of-box mappings.
    1. On the **General** tab, click the **Go to Configure Field Mapping** link. The Registry Field Mapping page opens.
    2. On the **Field Mapping** tab, add or edit field mappings.
    3. On the **Field Mapping** tab, click **Edit Callback** to edit callbacks, or click **Clear Callback** to delete callbacks.
    4. On the **Value Mapping** tab, add or edit value mappings.
    5. Click **Finish**.

Note

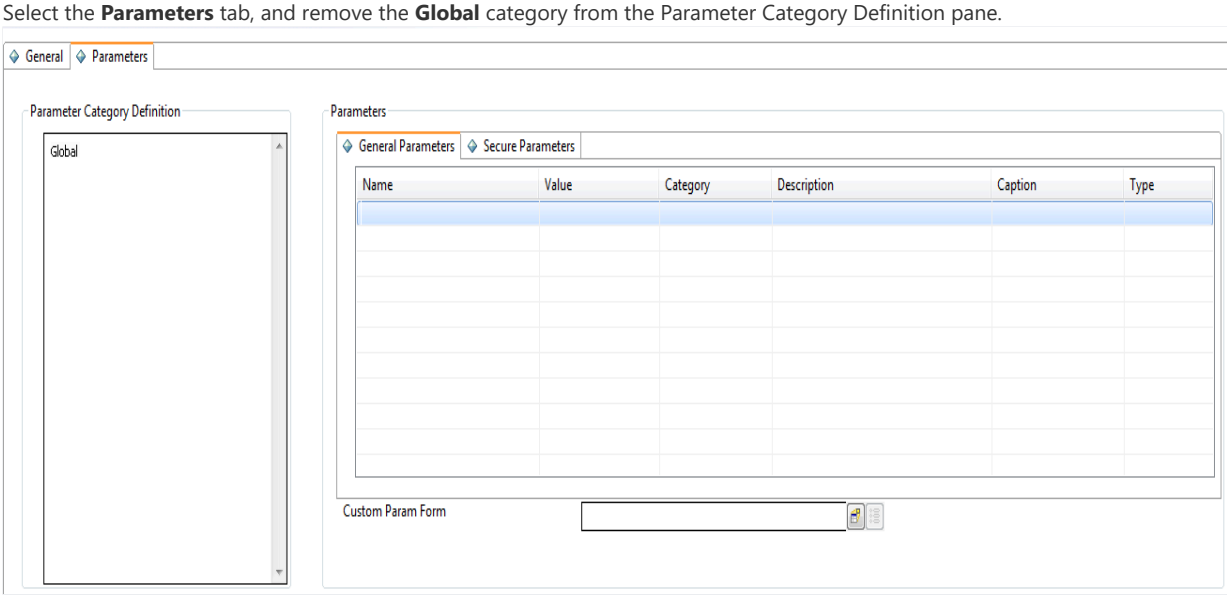
Note The default fields in the drop-down list are defined in the `getFields` method of the Service Manager adapter and endpoint adapter. Each mapping value can be calculated by combining the direct mapping value, default value, value mapping table, and callback value. For information about the processing logic of mapping, see Processing Logic of the Mapping Functionality.

View or edit a registered integration template

Once you have registered an integration template in SMIS, a record is created in the **SMISRegistry.g** form. You can view or edit the integration template.

To view or edit a registered integration template, perform the following steps:

1. Sign in to Service Manager as a system administrator.
2. From **Database Manager,** open the **SMISRegistry.g** form.
3. Select the **Parameters** tab, and remove the **Global** category from the Parameter Category Definition pane.



Note

Note The **Global** category is a default category, which displays on the **Parameters** tab by default. If you don't clear it before performing a search, the search returns only those records with global parameters.

4. Click **Search**. A list of integration templates displays.
5. Select a record to view or edit the details.

Export an integration template into an unload file

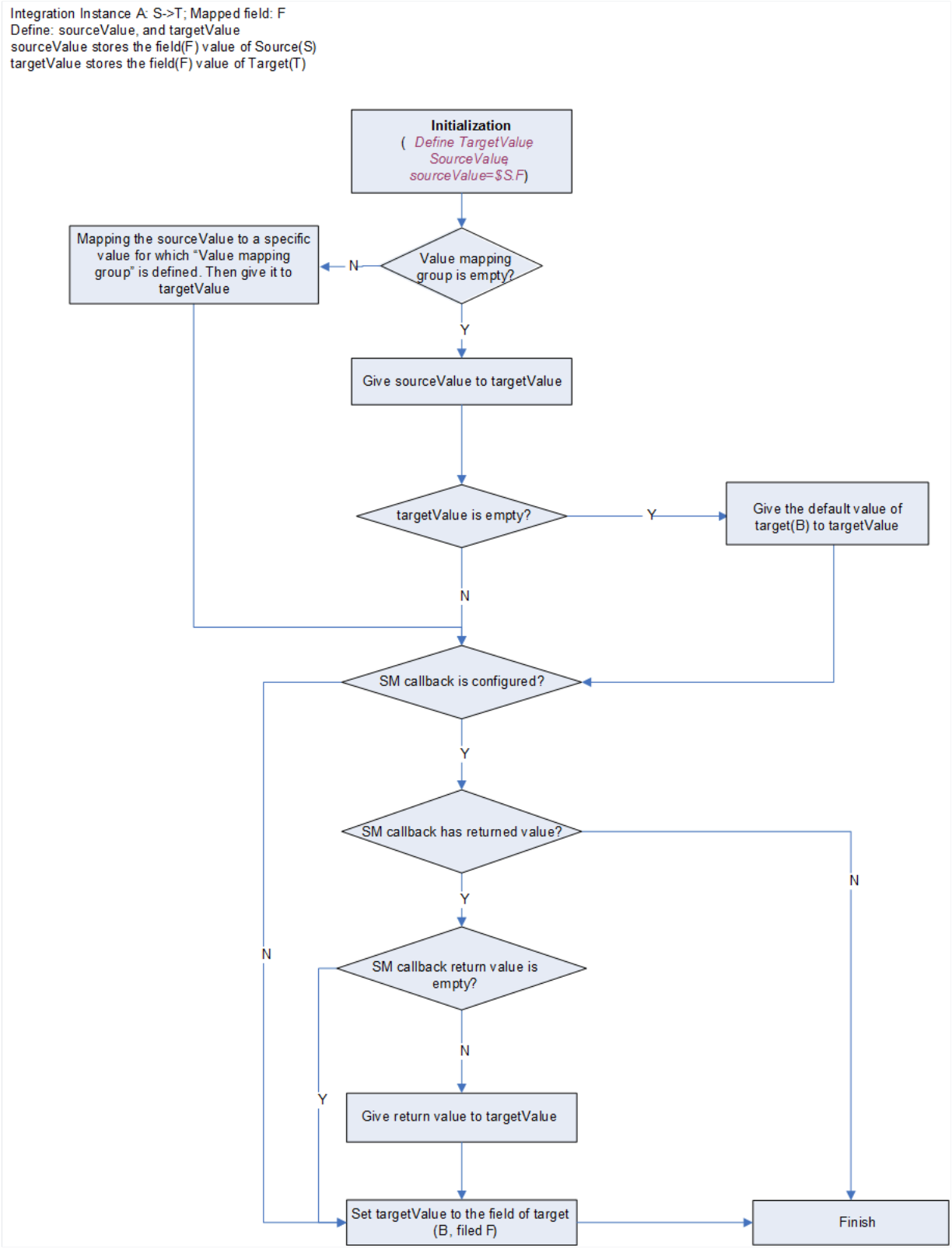
After you have registered an integration template in SMIS, you need to export it into an unload file, which you can then import into a testing system or production system.

To export an integration template into an unload file, follow these steps:

1. Create an unload script for the integration template. This unload script must include all the changes you made to the system when developing the integration template.
2. Go to **Tailoring** > **Unload Script Utility**. Select the unload script you created and export it into an unload file.

5.3. Processing Logic of the Mapping Functionality

The following diagram illustrates the processing logic of the mapping functionality.



6. Introduction to tailoring best practices

The purpose of this document is to provide customers with best practices to successfully tailor Service Manager. This document describes the most common tailoring activities and how to best accomplish them to meet particular use case goals. These best practices represent the collective experience of both our organization and other customers. By following these best practices you should realize the following benefits.

- Less need for support to complete a Service Manager implementation
- Improved system performance and efficiency
- Improved customer experience with your implementation
- Easier upgrades with more opportunity to review new features and functions

The following chapters provide information on how to accomplish common tailoring activities, tips and tricks from experts, and examples of how to meet particular use case goals. These best practices aren't the only ways to implement Service Manager, nor are all tailoring scenarios covered.

Related topics

Assumptions
Mapping rights
Array mapping options
Valid data type changes
When are RDBMS mappings added or updated?

6.1. Assumptions

The information in this document is most useful if all of the following criteria are met.

- You are implementing Service Manager version 9.20 or greater
- You are a system implementer with general knowledge of Service Manager tailoring tools and techniques
- You have a design based on a clearly-defined and agreed-upon business plan or system model to direct your tailoring efforts
- You have an understanding of Service Manager's out-of-box data model and ITIL work flows

This document assumes you have access to features available since the Service Manager version 9.20 release. Some of these best practices can also apply to earlier versions, but some performance benefits and user experience features require you have access to newer application versions.In general, best practices that rely on a particular tool or control will have the most benefit when implemented with the newest version of the applications.

System implementers will gain the most benefit from this document as it assumes you are already familiar with basic tailoring activities and have access to all the tools referred to within. At a minimum, you will need administrative access to all of Service Manager's tailoring tools. It is also recommended that you have earned an Accredited Integration Specialist certificate.

Having a business plan or system model will help you determine which parts of Service Manager you will need to tailor. Furthermore, having a list of objectives will help you decide between competing goals. For example, if your overall tailoring goal is to provide the best user experience for your Service Manager users, then you may choose options that maximize usability rather than the best practices that are easiest to implement or maintain. Lastly, a business plan allows you to get prior approval for your implementation from your business process owners. The more agreement there is on the business plan, the more likely your business process owners will accept the tailored implementation.

Many of the suggestions in this document assume you either know or have access to the Service Manager data model and work flows. This document doesn't describe the out-of-box data model or workflow in detail. If you need to change the out-of-box data model or workflow, see the Service Manager help.

Related topics

Introduction to tailoring best practices
What's tailoring?

6.2. What's tailoring?

Tailoring is any change to the out-of-box system data. Tailoring never changes the actual Service Manager code or program. Service Manager allows you to change almost any record value. The following are examples of tailoring changes you can make by changing the underlying record values.

- Change the look and operation of forms by changing the form definition in the format table.
- Change how fields are mapped in the back-end database by changing the value of the field mapping in the data dictionary.
- Change the workflow of a particular change by changing the values of the underlying phase and task records.

- Change what records a favorite or view displays by changing the value of the underlying query.



Tip

Tip Since tailoring involves changing record values, you should create a back-up of your system data before tailoring your system. Having a backup available gives you the option to revert to a working system should your tailoring changes cause a system outage.

Related topics

Assumptions
Mapping rights
Array mapping options
Valid data type changes
When are RDBMS mappings added or updated?

6.3. Form tailoring

You will typically use one or more of the following tools and features to tailor forms to meet your business needs.

- Forms Designer controls
  - Combo Box
  - Comfill
  - Radio Button
  - Checkbox
  - Subform
  - Group
  - Frame
  - Text
  - Text Area
  - HTML Editor
- Dynamic View Dependencies (DVD)
- Link Records
- Global Lists

The following sections describe some of the use cases where you can get the most performance and benefit from these controls and tools.

Related topics

Form tailoring use cases
Mapping rights

6.3.1. What's a form?

A form is a graphical input used to access, add, change, and view records in a Service Manager table. Forms serve as the interface between the user and the underlying table. A form can be associated with only one Service Manager table; however, a table may have many associated forms. If you need a form to access data from multiple tables, you can create one or more subordinate forms and link to them using the Subform control. Service Manager stores form records in the format table.

You can create and modify forms with the Forms Designer (FD) utility. You can add or edit rules for processing form data with the Format Control (FC) utility.

Related topics

Form tailoring use cases

6.3.2. Form tailoring use cases

The following use cases will help you decide the best form control to use given a particular situation.

- You want users to select values from a control with a list of values. See Selecting values from options in a control.
- You want users to select values from a list of values on the form. See Selecting values from options on a form.
- You want a user's prior selection to decide what other values are available. See Changing the options available based on prior user input.
- You want to display values from another table or record. See Displaying related data.

- You want to group logically associated items together. See Grouping related controls.
- You want users to type text into a field. See Typing text into a control.
- You want to adopt the activities feature improvement. See Adopt activities feature improvement.

The following use cases will help you improve your system performance and maintainability.

- You want to reduce the amount of system resources required during user sign in. See Managing global lists.
- You want to reduce the effort required to upgrade your tailored system. See Reducing upgrade conflicts.

Related topics

- Selecting values from options in a control
- Selecting values from options on a form
- Changing the options available based on prior user input
- Displaying related data
- Typing text into a control
- Managing global lists
- Reducing upgrade conflicts

6.3.3. Selecting values from options in a control

Forms Designer offers two controls that allow users to select a value from a list of values.

- Combo Box
- Comfill

You can determine which of these two controls to use by the number of options there are to choose from.

If there are a small number of options to choose from (for example, 32 or less options), then a Combo Box control is typically the most efficient control to use. See Combo Box.

If there are large number of options to choose from (for example, 100 or more options), then a Comfill control is typically the most efficient control to use. See Comfill.

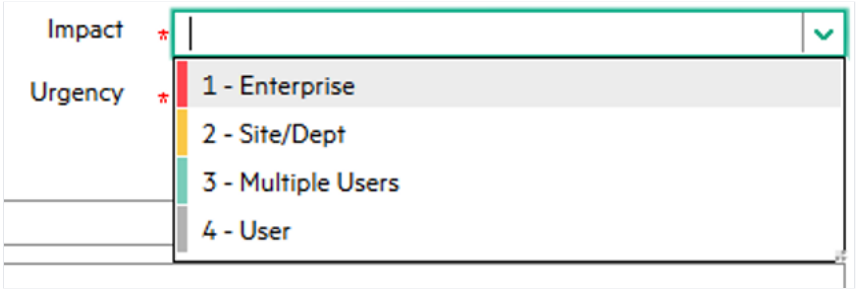
Related topics

Form tailoring use cases
Combo Box
Comfill

6.3.3.1. Combo Box

The Combo Box control lists a set of values for a user to choose from in a drop down box. You determine the list of possible values by either manually entering the values for the control or providing a global list variable for the Value List attribute.

Example Combo Box control



The benefits of a Combo Box control are listed below.

- The user sees a complete list of values to choose from on the same form as the control.
- You can control both the labels users see for each option (the Display List attribute) and the actual value each option has when stored in the database (the Value List attribute).

The costs of a Combo Box control are listed below.

- The more options there are to display, the more system resources are required to display them.
- If the list of values comes from a global list, then the server must send the entire list to the client each time the form is displayed. The more list items there are, the more network bandwidth it takes to transmit the list to the client. Furthermore, the more users there are who can access the global list, the more often the server has to transmit the list to clients.

- If the list of values comes from a global list, then the server must periodically rebuild the list of values. The more items there are in the list, the more system resources it takes to rebuild the list.
- If the list of values comes from a hard-coded list, then each list item increases the size of the form. The larger a form is, the longer it takes for the client to initially display it.

The more options there are to display, the more costly a Combo Box is to your system's performance. There is no set number of values that determines when a Combo Box is less efficient than a Comfill, but in general a Combo Box is best used in the following circumstances.

- There are few enough options to display that the control can display them all without the user needing to scroll down to see more options.
- The list of options either doesn't change at all or changes infrequently.

If you have a lot of options to display, consider using a Comfill control instead where you can control how many options are displayed per page. Also, if you expect to frequently add or change the options available to the control, you may want to use a Comfill control linked to the data of another field. Using a linked Comfill control allows you to add or update the options as records and guarantees your users always see the most current list of options.

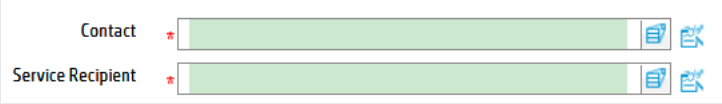
Related topics

Selecting values from options in a control
Comfill

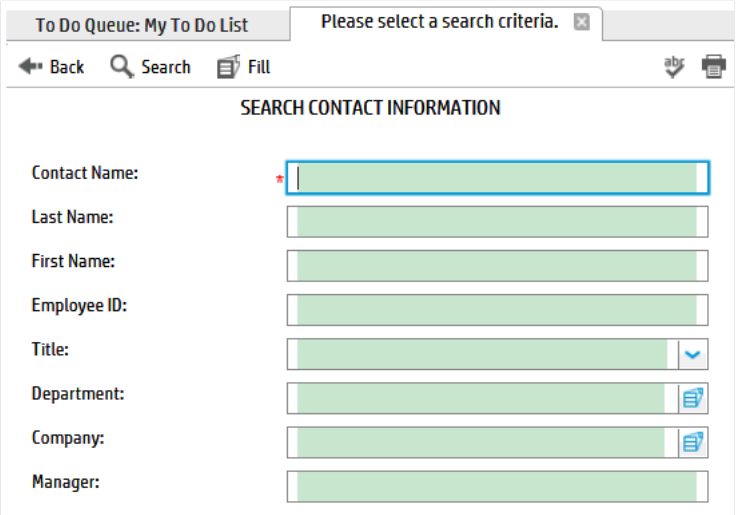
6.3.3.2. Comfill

The Comfill control lists a set of related records for a user to choose a value from. You determine the list of possible values by creating a link record entry for the Comfill control's Input field and specifying the related database field whose values you want to list in a link record line entry. You can also use the link record to associate a Comfill control with a particular search form if you want to allow users to search for related records.

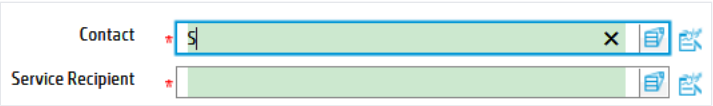
Example Comfill controls



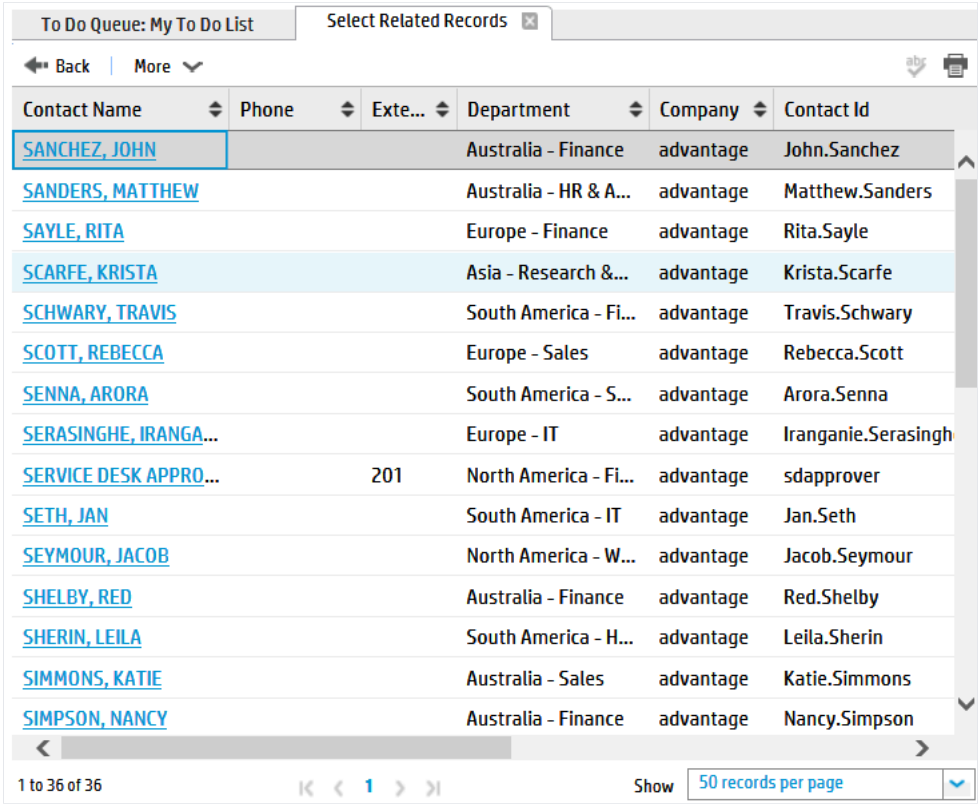
Example Comfill control associated with a search form



Example partial query in a Comfill control



Example query results from a partial query in a Comfill control



The benefits of a Comfill control are listed below.

- The system only has to display one page's worth of options at a time. It takes less system resources to display a partial list than the full list of options. The longer the list of options is, the more efficient it is to display the list in pages.
- Users can type in partial option values to search for options that match the partial value. For example, typing an "S" in the Contact field and clicking the Fill button shows a list of contacts that begin with the letter S.
- You manage the list of possible values by updating the related records linked to by the fill. For example, as you add, update, or remove contact records any fill linking to contact records will automatically pick up these changes.

The costs of a Comfill control are listed below.

- You must create a link line entry and a query for each Comfill control.
- You must create a search form if you want the fill functionality to use it.

Since Comfills depend upon link queries, any inefficiencies in the underlying query will negatively affect your system performance. Following the best practices in How do I write a good query? will reduce this performance impact. In addition, keep in mind that it takes more effort to design and maintain a search form for each fill. You may want to limit search forms to circumstances where you want to provide users with more than one way to search for related records. If one field is sufficient to find the related records, you don't need to add a search form to the Comfill control.

In general, a Comfill is best used in the following circumstances.

- There are enough options to display that the control can display them on multiple pages.
- The list of options changes frequently and you want the control to always have access to the most recent changes.

If you have a limited number of options to display, consider using a Combo Box control instead where you can display all the options at once in a drop-down box. Also, if you don't expect the data to ever change or to change infrequently, consider using a global list to make the data available to any application that needs it.

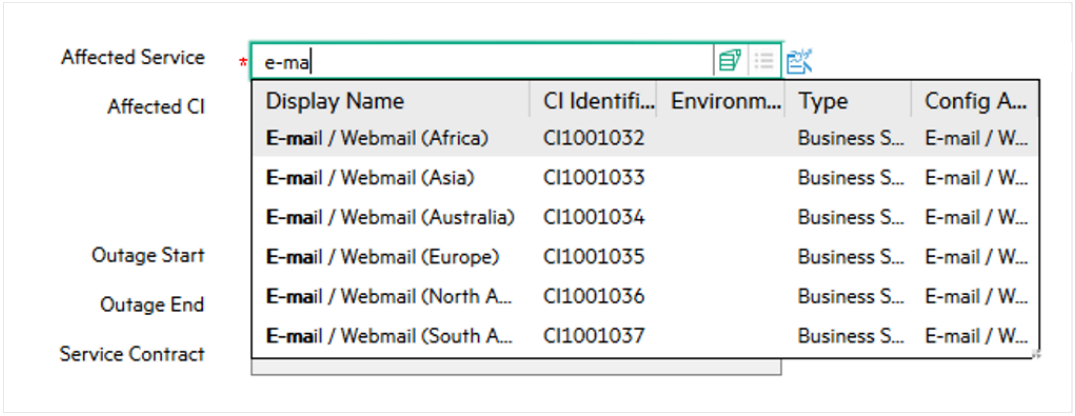
Related topics

Selecting values from options in a control
Combo Box
Web parameter: comfillAutoComplete

6.3.3.3. Auto complete for Comfill fields

Out-of-box, the auto complete feature is available for many Comfill fields in Service Manager. This feature has similar functionality as the Fill option, which locates information from a link file. For details about how the Fill option works, see Fill functionality.

When the auto complete feature is enabled for a Comfill widget, you can type just a few characters in this Comfill widget and select a matching result from the auto complete list.



Note

Note

- Auto Complete is always enabled for a field that has a referenced table defined, regardless of the value of the comfillAutoComplete parameter and the Auto Complete property setting defined in Forms Designer for that field. For more information, see Specify a referenced table for a field.
- The auto complete feature supports only "start with" link queries, which return a list of records that start with the user input. It doesn't support other kinds of link queries, such as "end with" or "not start with".
- If a Comfill widget has a defined **Data Change Event**, and the link for this field has defined **Post Expression**/**Post JavaScript**, we recommend that you don't enable auto complete for this Comfill widget. Otherwise, the **Data Change Event** may not work.
- If the input for a Comfill widget is a numeric value, auto complete is unsupported.
- If the auto complete results aren't returned in five seconds, the system displays a message, which directs you to the Fill option, or you can open the Fill option directly.

Out-of-box Comfill fields with auto complete enabled

The following table lists the Comfill fields with auto complete enabled in the out-of-box data of Service Manager.

Module	Tables	Forms	Fields
IM	probsummary	advFind.incident.search IM.open.incident IM.update.incident	assignee.name assignment category vendor contact.name subcategory product.type affected.item logical.name location.full.name
SD	incidents	advFind.SD.search SD.open.interaction SD.update.interaction	category subcategory product.type callback.contact contact.name affected.item logical.name kpf.id
CM	cm3r	advFind.search.change CM.change.approval CM.change.approvalprep CM.change.implementation CM.change.logging CM.change.planning CM.change.review cm3r.cigroup.g cm3r.discover cm3r.release cm3r.subscription	assign.dept coordinator requested.by category affected.item subcategory assets location.full.name logical.name corp.structure location assigned.to
PM	rootcause	advFind.search.problem PM.problem.closure PM.problem.investigation PM.problem.logging PM.problem.planning PM.problem.resolution	assignment assignee.name subcategory affected.item product.type logical.name closure.code

Service Manager provides the following parameters that control this feature. Their default values are intended to provide optimized user experience; however you may need to optimize these parameters according to your system performance.

- Web parameter: autoCompleteListSize

- Web parameter: autoCompleteDelayTime
- Web parameter: autoCompleteMinChars
- Web parameter: autoCompleteSkipCachingChar

Enabling auto complete for a new Comfill widget

When creating or tailoring a form, if you add a new Comfill widget and want to enable auto complete for the widget, perform the following steps.

> **Note**
>
> Note Make sure this Comfill widget has a link definition before you enable the auto complete feature for it. Also, be aware that the auto complete feature doesn't work in the following situations:
>
> - In Comfill fields that have a value list or display list defined
> - If the **Combo Button Visible** option is selected

1. Make sure auto complete is enabled globally, that is, the comfillAutoComplete parameter is set to **true**.
2. Update the form to enable auto compete for the Comfill widget.
   a. In the Windows client, go to **Tailoring** > **Forms Designer**, and open the form in **Design** mode.
   b. Select the Comfill widget, and enable the **Auto Complete** property.
   c. Click **OK**.
   d. Click **Save**.
3. The auto complete feature will consume additional system resources (memory and CPU). After you enable auto complete for the Comfill widget, you need to validate the form performance. To do so, perform the following steps.
   a. Open the form in **Forms Designer**, and click **Validate Format** from **More** or the More Actions menu.
   b. Click **Continue**.
   c. Specify the file name and click **OK**.

   > **Note**
   >
   > Note Service Manager skips this step if a file is already specified for your form. To see if your form has a file specified, click **More** on the form and select **Format Detail**.

   A message displays if the system finds any performance issues. You can check the message for details.

6.3.4. Selecting values from options on a form

Forms Designer offers two controls that allow users to select a value from a list of options on the form.

- Radio Button
- Checkbox

You can determine which of these two controls to use by determining if the options on the form are mutually exclusive.

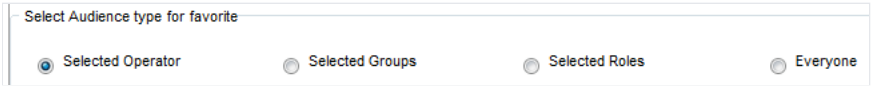
If you want users to select one value from a list of mutually exclusive values, then list each option as a Radio Button control.

If you want users to select a true or false value for one particular option, then use a Checkbox control for the option.

Radio Button[edit]

The Radio Button control lists one possible value for a set of mutually exclusive values that a user can choose from. You determine the list of possible values by creating multiple Radio button controls and giving them all the same Input attribute.

Example Radio Button control



The benefits of a Radio Button control are listed below.

- The user sees a complete list of values to choose from on the same form as the control.
- The user has a visual indicator for the currently selected value.
- You can control both the label users see for the option (the Caption attribute) and the actual value the option has when stored in the database (the Value attribute).

The costs of a Radio Button control are listed below.

- The more options there are to display, the more of the form's screen space is required to display them.
- You have to use a Frame or Group control to indicate that a list of Radio Button options are related.
- The list of values is only stored on the form and not in a global list or related record. This means you have to keep track of which forms use a particular list if you want to re-use the list in another form.
- The only way to change the list of values is to change the value of each individual Radio Button control.
- Each Radio Button option increases the size of the form. The larger a form is, the longer it takes for the client to initially display it.

A list of Radio Button options is very easy for your system to display since there are no link queries run or global lists to update at start-up. However, a list of Radio Button options requires more form space than using a single Combo box or Comfill control.

In general, a Radio Button control is best used in the following circumstances.

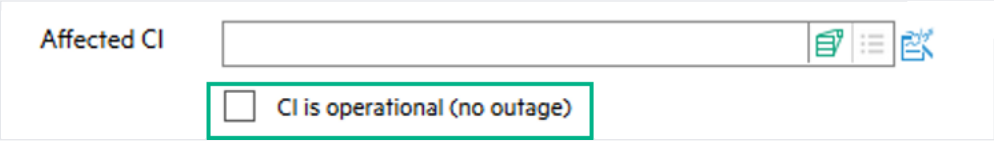
- There are few enough options to display that the user can see them all without needing to scroll to see more options.
- The list of options either doesn't change at all or changes infrequently.
- You want a visual indicator to highlight the currently selected value.
- You can't select more than one value at a time with this control.

If you have more than a few options to display, consider using a Combo Box control instead to reduce the amount of screen space required to display the options. Also, if you expect to re-use the list of options in another form or in other circumstances, consider creating a global list so the options will be available for multiple forms. If you want users to be able to select more than one value from a list of values, consider creating a list of checkboxes.

Check Box[edit]

The Check Box control lists a true or false value for a single option. The list of possible values is always true or false for a Check Box control.

Example Check Box control



The benefits of a Check Box control are listed below.

- The value of each Check Box control is independent of other Check Box controls.
- The user has a visual indicator for the currently selected value.
- The actual value of the control is always either true or false.
- You can control the label users see for the control (the Caption attribute)

The costs of a Check Box control are listed below.

- You can't use this control to provide a list of mutually exclusive options.
- You may have to use a Frame or Group control to indicate that a list of Check Box options are related.

A Check Box control is very easy for your system to display since there are no link queries run or global lists to update at start-up. The longer the Check Box label is the more form space the control requires.

In general, a Check Box control is best used in the following circumstances.

- The only options the user needs to choose between are true and false values.
- The values of one control are independent of the values in other controls.
- You want a visual indicator to highlight the currently selected value.

If you want users to select one value from a list of mutually exclusive values, use a set of Radio Button controls instead to ensure that only one value can be selected at a time.

Contents

- 1 Radio Button
- 2 Check Box

6.3.5. Changing the options available based on prior user input

Forms Designer offers two methods for you to change what options are available based on prior user input.

- Use DVD select statements to change what options appear on a form
- Use recursive fills to change what records the user can select as options

You can determine which of these two methods to use by how you want to change the list of available options.

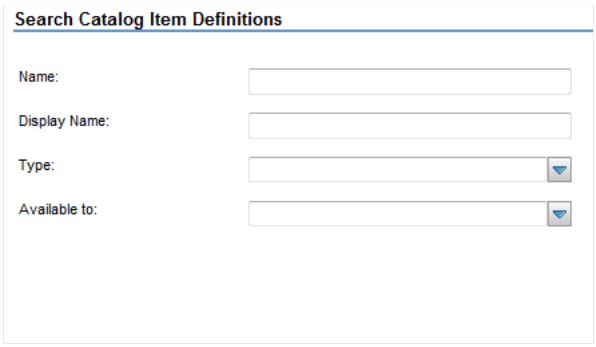
If you want to change what options are visible on form, then use one or more DVD select statements.

If you want to change the list of records that a user can choose from, then use one or more recursive fills.

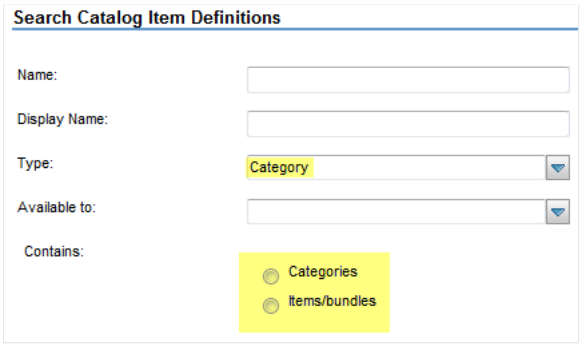
DVD select statements

A DVD select statement changes what controls or values appear on a form based a triggering condition. You determine what controls or values the DVD select statement displays by assigning a condition to each control or value. When the condition is true, the system displays the controls and values. When the condition is false, the system hides the controls and values.

Example DVD select statement where the display condition is false



Example DVD select statement where the display condition is true



The key benefits of a DVD select statement are listed below.

- The user only sees the controls and values when the appropriate condition applies.
- The user has a visual indicator that new options or controls are available.

The costs of a DVD select statement are listed below.

- The more conditions there are to evaluate, the more system resources if takes to display the form.
- You have to reserve part of the form's screen space to display hidden items.
- You must create a query for the DVD select statement condition.
- The only way to change the list of values is to change the value of each individual control of the DVD select statement.

Since DVD select statements depend upon queries for the condition, any inefficiencies in the underlying query will negatively affect your system performance. Following the best practices in How do I write a good query? will reduce this performance impact. In addition, the more conditions you have, the more of the form's screen space you will have to reserve for the controls hidden by the DVD statement.

In general, a DVD select statement is best used in the following circumstances.

- There are only a few conditions that change the options available on the form.
- The list of options either doesn't change at all or changes infrequently.

If you have a lot of conditional data to manage, consider using multiple Comfill controls with recursive fills instead. This allows you to restrict the options displayed in one Comfill control based on the selections in a previous Comfill control. Also, if you expect to

frequently add or change the options available to the DVD select statement, you may want to use a Comfill control so that you can add or update the options as records.

Note

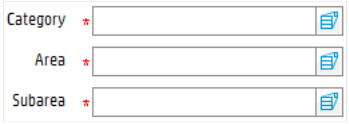
Note

The system may have performance issue that leads to significant increase of form loading time if there are too many DVD select statements added to a form.

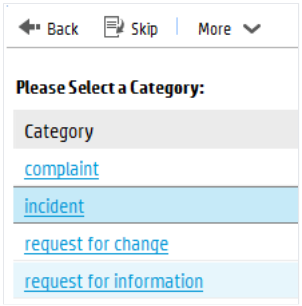
Recursive fills

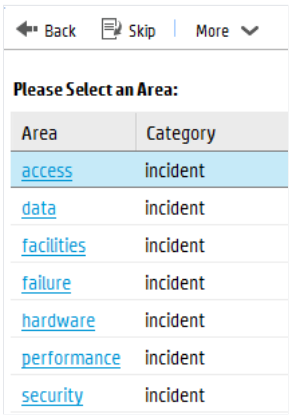
A recursive fill uses the value of one Comfill control to determine the available values of another Comfill control. You determine what values the recursive fills display by adding a link record line entry for each Comfill control on the form. When a link record includes the expression `$fill.recurse=true` , then the system evaluates each line of associated link record. You may also want to associate a search form or QBE format for each Comfill control in the recursive fill to make easier for users to select multiple values.

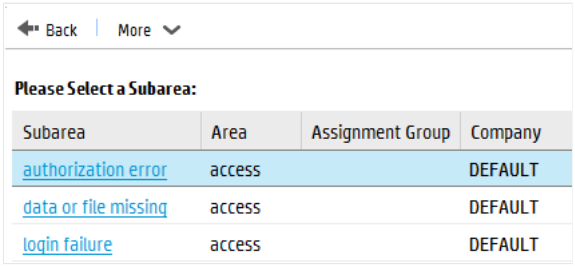
Example Comfill controls linked by recursive fills



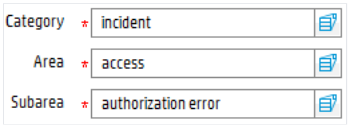
Example QBE formats allowing users to select a Category, Area, and Sub Area.







Example completed recursive fill



The benefits of recursive fills are listed below.

- You can use a query to ensure that users only select valid combinations of values.
- Users can select values for related data in one process.
- The system only has to display one page's worth of options at a time. It takes less system resources to display a partial list than the full list of options. The longer the list of options is, the more efficient it is to display the list in pages.
- You manage the list of possible values by updating the related records linked to by the fill. For example, as you add, update, or remove contact records any fill linking to contact records will automatically pick up these changes.

The costs of recursive fills are listed below.

- You must create a link record entry and a query for each Comfill control.
- You must create a search form if you want the fill functionality to use it.
- Users can only select values that match the query for each Comfill control

Since recursive fills depend upon link queries, any inefficiencies in the underlying query will negatively affect your system performance. Following the best practices in How do I write a good query? will reduce this performance impact. In addition, you must be very familiar with the Service Manager data model to create the necessary queries.

In general, a recursive fill is best used in the following circumstances.

- The value of one field determines the value another field can have. Typically, all of the fields come from a set of related records.
- The list of options changes frequently, and you want your Comfill controls to always have access to the most recent changes.

If you want to hide or display data on a form, use Combo Box controls and DVD select statements instead.

6.3.6. Displaying related data

Forms Designer offers two controls that allow users to display data from related tables and records.

- Combo Box
- Subform

You can determine which of these two controls to use by how you want users to interact with the related data.

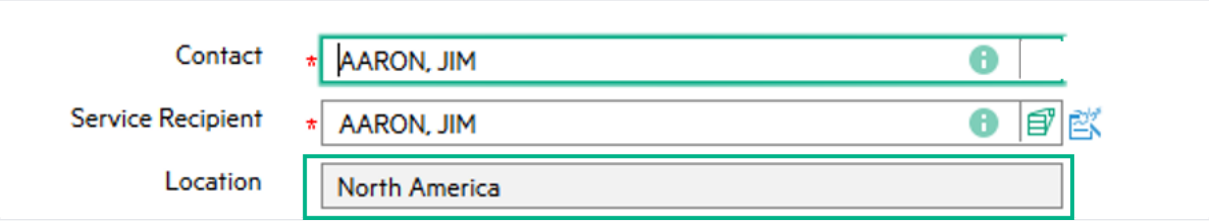
If you want to use a control that allows users to select and interact with related data, then use a Combo Box control. See Combo Box.

If you want use a control that protects any virtual-joined data by making it read-only, then use a Subform control. See *Subform* below.

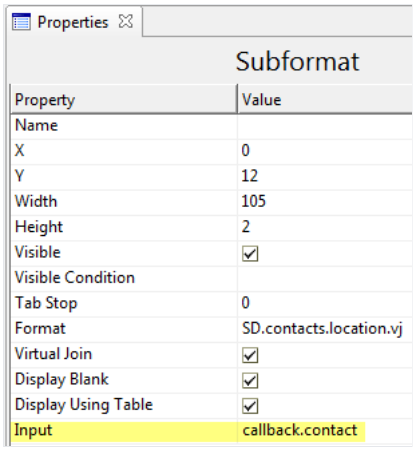
Subform

The Subform control displays a separate form containing data from a different table than the table associated with the main form. A Subform control allows any controls on the subordinate form to use the link record data to perform a virtual join. For example, a text control or table control on the subordinate form can display related data from another table. You determine what related data to display by creating a link record entry for the Subform control's Input field and specifying the related database field whose values you want to display in link record line entry.

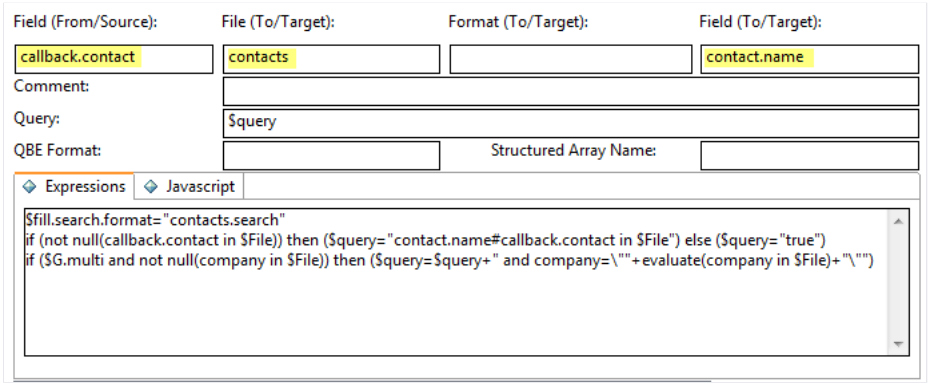
Example Subform control on the open interaction form



Example Subform control Input value



Example link record line entry for the callback.contact field



The benefits of a Subform control are listed below.

- You can display related record data from any other source in the system.
- You can display related records that have one to many relationships. For example, from one input, the contact name, you can use a Subform control to display many related fields such as the contact's location, address, or telephone number.
- You can provide link record data to controls that don't normally have link records such as Text or Table controls.
- Any information on a Subform control is automatically read-only when it is part of a virtual join.

The costs of a Subform control are listed below.

- You must create a separate link line entry and a query for each Subform control.
- You must create and maintain a subordinate form for the linked controls.
- You can't use the virtual join option if you want users to interact with the controls on the subordinate form. This typically requires using either global lists or hard-coded values for any Combo Box controls on the subordinate form.

Since Subform controls depend upon link queries, any inefficiencies in the underlying query will negatively affect your system performance. Following the best practices in How do I write a good query? will reduce this performance impact. In addition, you must be very familiar with the Service Manager data model to create the necessary queries.

In general, a Subform control is best used in the following circumstances.

- You need to display data from a different table than the one associated with the main form
- You want the related information to be displayed in read only controls

If you want users to be able to select or interact with the related data, either use a Comfill control or don't enable the Virtual Join option for the Subform control. See the Comfill section inSelecting values from options on a form for a description of Comfill costs and benefits.

6.3.7. Grouping related controls

Forms Designer offers two controls that allow users to group related sets of controls in a container.

- Group
- Frame

You can determine which of these two controls to use by which client you want the container to appear in.

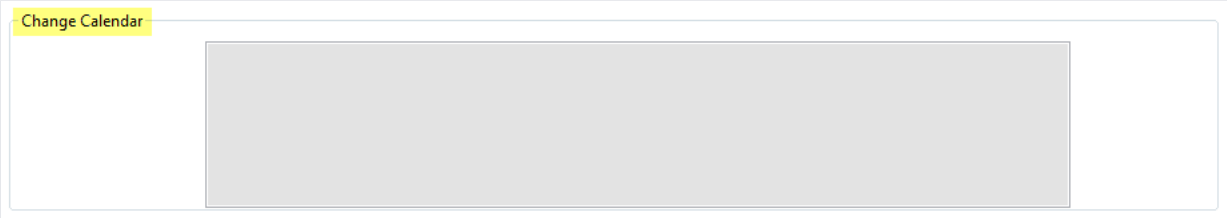
If you want Web client users to be able to collapse and expand the container, then use a Group control. See **Group** below.

If you only want Windows client users to see a container with line borders, then use a Frame control. See **Frame** below.

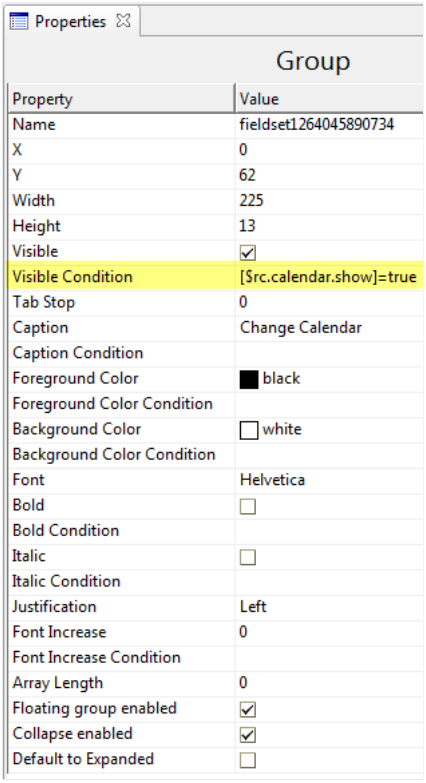
Group

The Group control provides a container with a text label to logically group items and controls. Any control you place within a Group control inherits the visibility condition of the Group.

Example Group control



Example Visible Condition for a Group control



The benefits of a Group control are listed below.

- You can make the container a section that collapses and expands in the Web client.
- The container has a text label to clearly identify the purpose of the controls within the container.
- The controls within the container inherit the Visible Condition property of the Group. The controls within the group are only visible when the group itself is visible.
- If you move or delete a Group control, all the controls within the container are also moved or deleted.

The costs of a Group control are listed below.

- You have to update the text label should you ever change the controls within the container.
- You have to localize the label if you want to make the form available in other languages.

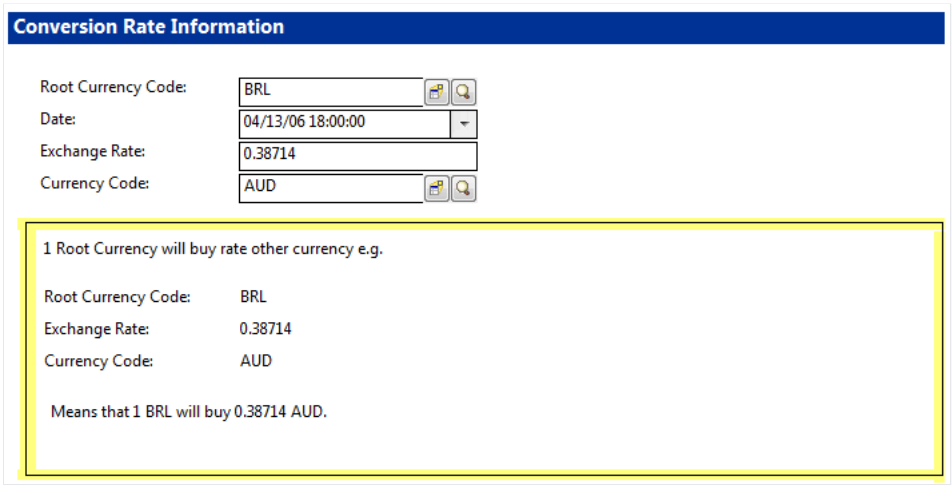
In general, a Group control is best used in the following circumstances.

- You want a visual indicator to highlight that a group of controls are related.
- You want to allow users to efficiently manage the available form space in the Web client.
- You want to manage the visibility settings of a group of controls from one place.

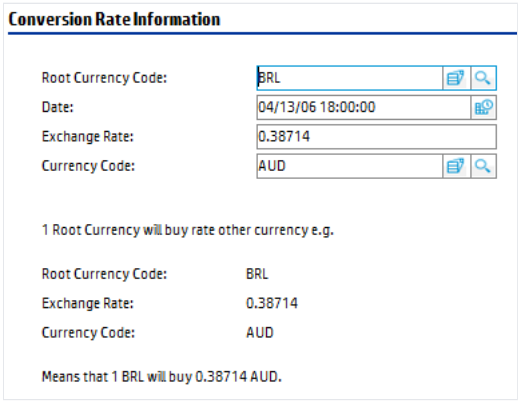
If you want to change the appearance of the line around the container, then use a Frame control instead. However a Frame control doesn't allow users to collapse and expand the section in the Web client.

The Frame control provides a container with a beveled line surrounding the items and controls in the container. Any control you place within a Frame control inherits the visibility condition of the Group. Since the Web client doesn't display the line surrounding the Frame control, it almost always better to use a Group control rather than a Frame control.

Example Frame control in Windows client



Example Frame control in Web client



The benefits of a Frame control are listed below.

- The container has a border to indicate that the controls are within a container.
- The controls within the container inherit the Visible Condition property of the frame. The controls within the frame are only visible when the frame itself is visible.

The costs of a Frame control are listed below.

- The line surrounding the Frame control is not visible in the Web client
- Users can't collapse and expand the container in the Web client

The Frame control has less functionality than the Group control. In most cases, you should use a Group control rather than a Frame control.

Related topics

Form tailoring use cases

6.3.8. Typing text into a control

Forms Designer offers three controls that allow users to type text into a control.

- Text
- Text Area
- HTML Editor

You can determine which of these controls to use by the amount and formatting of the text you want users to type.

If you want users to type a single line of unformatted text, then use a Text control. See Combo Box.

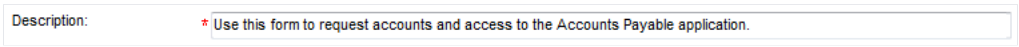
If you want users to type in multiple lines of unformatted text, then use a Text Area control. See Comfill.

If you want users to type in any amount of HTML-formatted text, then use an HTML Editor control. See **HTML Editor** below.

Text

The Text control allows users to type in or view a single line of unformatted text. You can also make a Text control read-only to display the value of a particular field or if the control is part of a Subform control virtual join.

Example Text control



The benefits of a Text control are listed below.

- You can make the control read-only to display data you don't want users to change, such as a record ID number.
- RDBMS tools can typically query Text control fields as long as you map the field to a simple character data type in the back-end RDBMS.
- Third-party reporting tools can typically read Text control fields as long as you map the field to a simple character data type in the back-end RDBMS.
- The user doesn't have to apply any text formatting to the value typed.
- If the control is editable, users can update the text value at any time.

The costs of a Text control are listed below.

- The content of the control is displayed on one line of text. The database definition mapping of the fields determines the width of the line. For example, if the input field is mapped to a VARCHAR(255) column, then a user can only type 255 characters of text.
- The control uses the character encoding of the back-end RDBMS to determine how to store character data. If users type in text using a character encoding that your system doesn't support, the data will likely become corrupted.
- It is difficult to validate that users type in meaningful values into the control.
- There is no way for users to format the text they type into the control.
- The longer the width of the Text control the more form space the control requires.

In general, a Text control is best used in the following circumstances.

- The user only needs to type in or view a single line of text.
- You want the option to make the text read-only. For example, the text is a unique ID or part of a virtual join in a Subform control.
- You don't need to format the text.

If you want users to type multiple lines of text, use a Text Area controls instead. If you want users to be able to format the text they type in, use an HTML Editor control instead.

Text Area

The Text Area control allows users to type in or view multiple lines of unformatted text. You can also make a Text Area control read-only to display the value of a particular field or if the control is part of a Subform control virtual join.

Example Text Area control



The benefits of a Text Area control are listed below.

- Users have multiple lines in which to type or view text.
- You can make the control read-only to display data you don't want users to change, such as the description of an open interaction.
- RDBMS tools can typically query Text Area control fields as long as you map the Text Area's input field to a multi-row array table. See the **Multi-row array table** section in Array mapping options.
- Third-party reporting tools can typically read Text Area control fields as long as you map the Text Area's input field to a multi-row array table. See the **Multi-row array table** section in Array mapping options.
- The user doesn't have to apply any text formatting to the value typed.
- If the control is editable, users can update the text value at any time.

The costs of a Text control are listed below.

- You must choose an array mapping strategy in which to store the Text Area control data. See Array mapping options.
- The control uses the character encoding of the back-end RDBMS to determine how to store character data. If users type in text using a character encoding that your system doesn't support, the data will likely become corrupted.
- It is difficult to validate that users type in meaningful values into the control.

- There is no way for users to format the text they type into the control.
- The longer the width of the Text Area control the more form space the control requires.

In general, a Text Area control is best used in the following circumstances.

- The user needs to type in or view multiple lines of text.
- You want the option to make the text read-only. For example, the text is a part of an closed record or part of a virtual join in a Subform control.
- You don't need to format the text.

If you want users to type a single line of text, use a Text controls instead. If you want users to be able to format the text they type in, use an HTML Editor control instead.

HTML Editor

The HTML Editor control allows users to type in and format multiple lines of text for display in an HTML Viewer control. Unlike other text input controls, if you don't want users to change the HTML text, you must display it in an HTML Viewer control.

The benefits of an HTML Editor control are listed below.

- Users can format multiple lines of text for display with the built-in editing tools or with HTML tags.
- Users can spell check the text.
- RDBMS tools can typically query HTML Editor control fields as long as you map the HTML Editor's input field to a multi-row array table. See the **Multi-row array table** section in Array mapping options.
- Third-party reporting tools can typically read HTML Editor control fields as long as you map the HTML Editor's input field to a multi-row array table. See the **Multi-row array table** section in Array mapping options.
- Users can update the text value at any time.

The costs of an HTML Editor are listed below.

- You must use a separate HTML Viewer control to display a read-only version of data you don't want users to change, such as an approved knowledge article.
- You must choose RDBMS mapping that has sufficient space for potentially large HTML input. Typically, an array mapping strategy is best for storing HTML Editor control data as there is no risk of data truncation when users type large amounts of text. See Array mapping options.
- The control uses the character encoding of the back-end RDBMS to determine how to store character data. If users type in text using a character encoding that your system doesn't support, the data will likely become corrupted.
- It is difficult to validate that users type in meaningful values into the control.
- You must use a separate HTML Viewer control to provide a read-only version of the text.
- The longer the width of the HTML Viewer control the more form space the control requires.

In general, an HTML Editor control is best used in the following circumstances.

- The user needs to type in and format multiple lines of text. For example, the text is a knowledge document.
- You want the option to spell check the text.

If you don't want users to format the text, use either a Text or Text Area control instead. Likewise, if you need to display the value of another text field in the system, use either a Text or Text Area control.

6.3.9. Managing global lists

Global lists provide an efficient way to display frequently-used list values by storing the list in system memory as a global variable. All applications can access the global list variable as needed without needing to query the database each time a control needs to display the list. However, the more items there are in a global list, the more costly the list is to system performance.

A global list has the following performance benefits.

- Since a global list is stored in system memory, there is no need to query the back-end database for a list of values. Using a global list therefore reduces the amount of input and output traffic to the RDBMS.
- Since a global list is available to all users logged into the system, there is no need to create conditional queries to grant access to the information. Using a global list therefore potentially reduces the number of queries needed to access a particular set of information.

When managing global list, keep in mind the following performance costs.

- The server must send the entire list to the client each time the form is displayed. The more list items there are, the more network bandwidth it takes to transmit the list to the client.
- The more users there are who can access the global list, the more often the server has to transmit the list to clients.

- The server must periodically rebuild the list of values. The more items there are in the list, the more system resources it takes to rebuild the list

If you want to maximize your system performance, use the following guidelines to determine when to use global lists.

If there are a small number of options to choose from (for example, 32 or less options), then displaying a global list in a Combo Box control is typically the most efficient strategy. Since the list of options is small, you gain a performance improvement by skipping a query to the back-end database that a Comfill control would require. See Combo Box.

If there are large number of options to choose from (for example, 100 or more options), then displaying a list of values from a Comfill control linked to other records in the database is typically more efficient than a global list. Since a global list must always send the entire list of options to the client, the Comfill control has better performance because it can send one page worth of options to the client at a time. In addition, it is more efficient to add, update, or remove the related records a Comfill displays than it is to rebuild an entire global list each time you want to modify the list of values. See Comfill.

Removing unused global lists

You should periodically review your global lists to determine if any of them are unused and can be deleted. Since the server periodically rebuilds each global list, you will see a performance improvement if you remove any unused global lists from your system. Removing unused lists frees up system resources and memory that your system can use for other purposes.



Tip

Tip You can archive and purge any unused global lists for later use. Should you find you require the global list you can simply import the global list back into the system from the archive unload.

6.3.10. Reducing upgrade conflicts

The Service Manager application upgrade process allows you to add new features and functions to your existing system. An upgrade conflict occurs when the upgrade utility determines there are multiple versions of the same out-of-box component. For every conflict the upgrade utility identifies, you must manually reconcile between the out-of-box component version and the customer-modified version of the same component. This reconciliation process is time-consuming and often delays the adoption of new features and functions because you can't use the new features and functions until after you have reconciled the conflicts.

You can use the following guidelines to reduce the number of conflicts you have to resolve during an application upgrade.

- Do not directly modify the out-of-box forms. Instead, create your own copies of the forms, give them unique names (such as adding a company abbreviation to the front of each form name), and make any modifications to the copied forms.
- Do not directly modify the out-of-box scripts in the script library. Instead, create your own script library with copies of any out-of-box functions, give them unique names (such as adding a company abbreviation to the front of each script name), and make any modifications to the copied scripts.
- Keep an audit log of any changes you make to the out-of-box Document Engine components such as processes, states, and objects. You can then use your audit log to determine how to merge any changes to Document Engine components.

Making your own copies of forms and scripts prevents you from having any conflicts on these items and allows you to review any new functionality the upgrade adds without changing your existing system. For example, if you create a copy of the out-of-box form **SD.open.interaction** called **OpenText.SD.open.interaction**, then any upgrade to the out-of-box form doesn't create a conflict with your custom form. Likewise, you can compare the functionality of the new out-of-box form to your custom form and determine if it has any features or functions you want to include in your custom form.

While keeping an audit log will not prevent upgrade conflicts, it will reduce their impact if you can quickly determine what functionality your changes support and whether the company-version of the Document Engine components also support this functionality. At a minimum, your audit log should list what Document Engine components you changed and what the out-of-box state was. You may also want to list why you changed the component or what applications, forms, workflow your change supports.

Related topics

Form tailoring use cases

6.3.11. HTML formatting for Activities feature

As an improvement to the activities feature in previous releases, Service Manager embeds the **Journal Updates Viewer** and **Activities Viewer** widgets to the Activities tab. The Journal Updates section is now a Journal Updates Viewer and the Activities Viewer widget replaces the original table control in Activity logs. Hence, the notes and logs on the Activities tab now have HTML formatting with significantly improved readability.

The activities feature is available after you Upgrade the applications.

- When you perform the Resolve conflicts step during the applications upgrade, if you skip the conflicts for the activity-related format records, and you decide to work with the activities feature after upgrade, you need to follow the instructions in **Use the**

new Activities feature.

- If you still hope to use your previous customized forms with the legacy activities feature after the applications upgrade, you may follow the instructions in **Use the legacy Activities feature**.

Use the new Activities feature

For those customized forms with the activities feature, the **New Update** and **Journal Updates** fields still remain with Text Area controls, and the activity logs section remains with Table control. Hence, to use the new Activities feature after the applications upgrade, you need to do some tailoring to adopt the latest changes for the activities feature.

The tailoring process covers the following tasks:

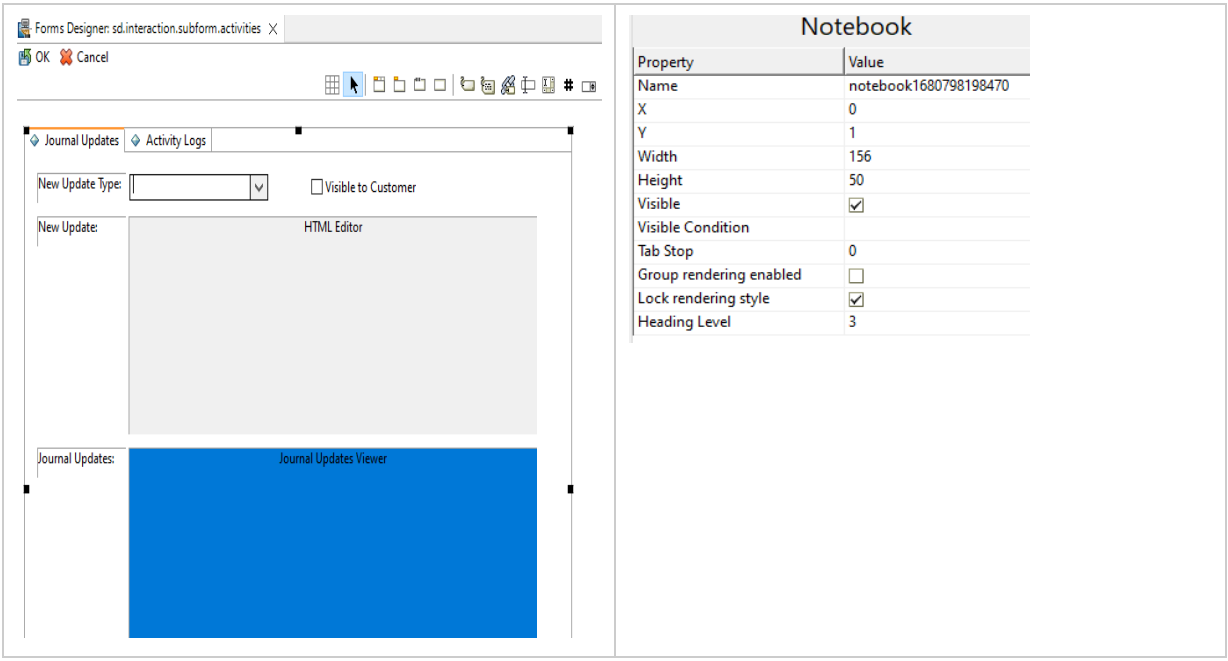
1. Redesign the corresponding forms.
2. Perform a series of verifications or updates to make sure that the corresponding logical files, fields, keys, rulesets, and variables exist for the latest activities feature.

The following tailoring tasks are examples based on the activity feature on the Categorized phase form in the Service Desk module. If there are more forms for other phases that use the activities feature, you need to perform similar configurations to these forms as well.

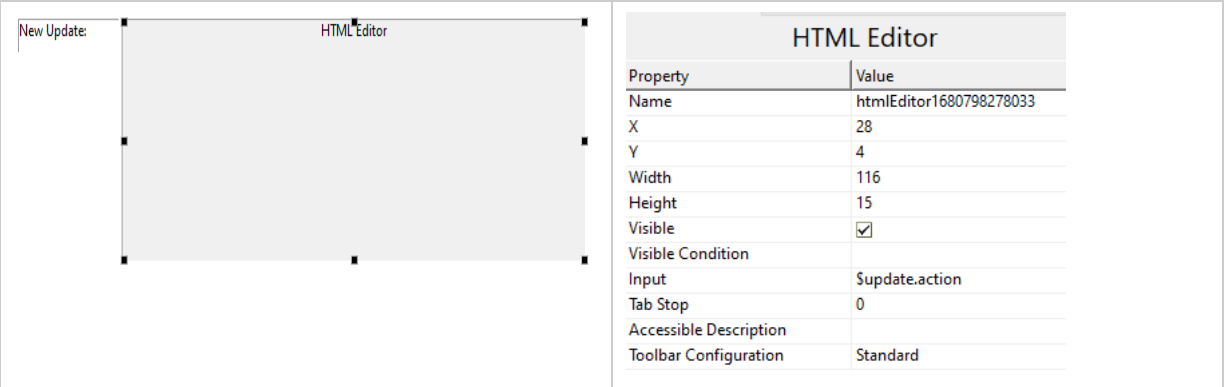
Redesign the corresponding forms

Refer to the following steps for some guidelines. Here we take the interaction form as an example.

1. Sign in to the Windows client as a system administrator.
2. Open Forms Designer. Search for the **sd.interaction.subform.activities** form that is embedded in the **sd.interaction.categorization** form.
3. Back up the **sd.interaction.subform.activities** form:
   Select **More > Copy/Rename**, and then back up it as **sd.interaction.subform.activities.bak** with **Copy** Format.
4. Remove all controls from the **sd.interaction.subform.activities** form.
5. Add a notebook with two notebook tabs: **Journal Updates** and **Activity Logs**. Set the **Heading Level** property of the notebook to **3**.

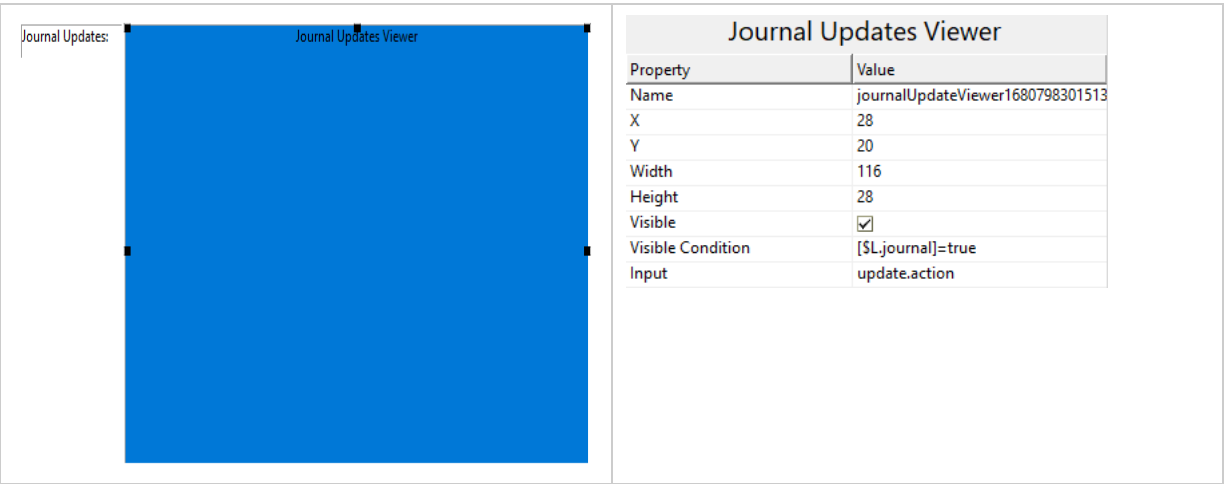


6. Copy the original updates-related fields such as **New Update Type** and **Visible to Customer** from the **sd.interaction.subform.activities.bak** form to the **Journal Updates** notebook tab.
7. The original control of the **New Updates** field is a Text Area. Replace this control with HTML Editor, and then set the value of the **Toolbar Configuration** property to **Standard**. Keep the value for **Input** and **Visible Condition** properties the same as the original design.

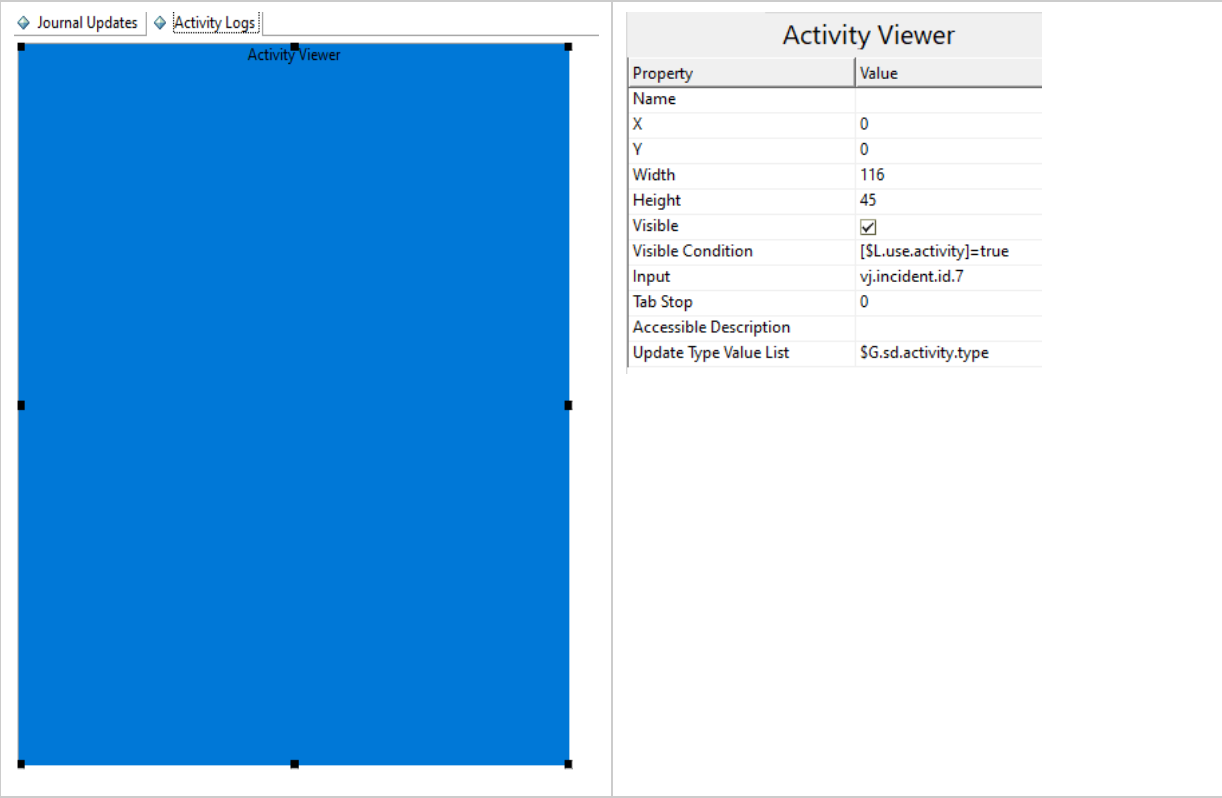


Note: It's not advisable to set the **Toolbar Configuration** property to other values than **Standard**. Otherwise, issues may occur.

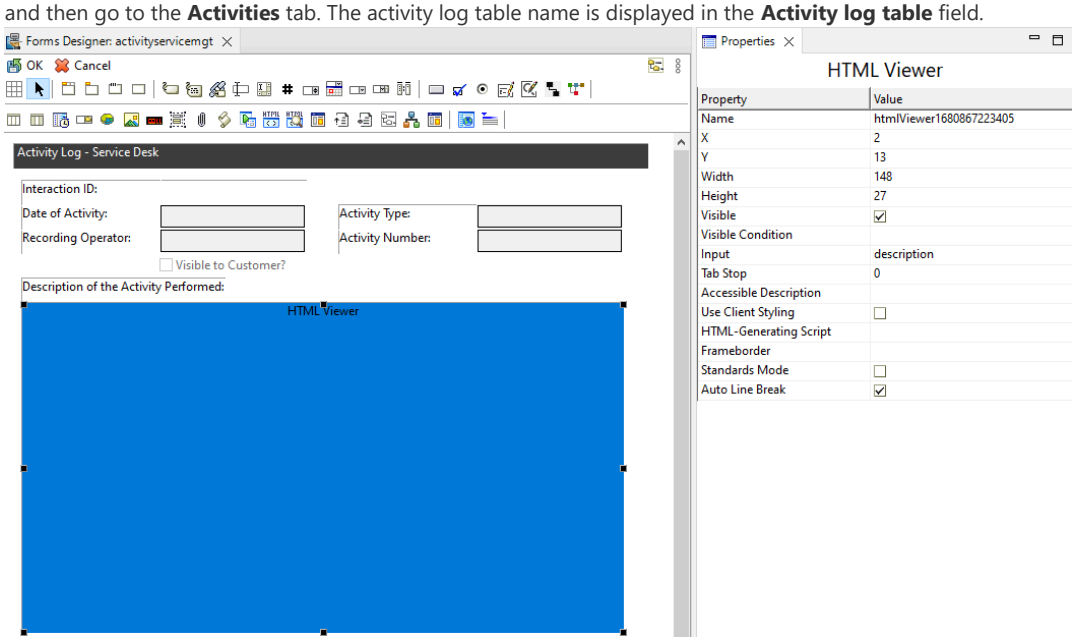
8. The original control of the **Journal Updates** field is a Text Area. Replace this control with Journal Updates Viewer. Keep the **Input** and **Visible Condition** properties the same as the original design.



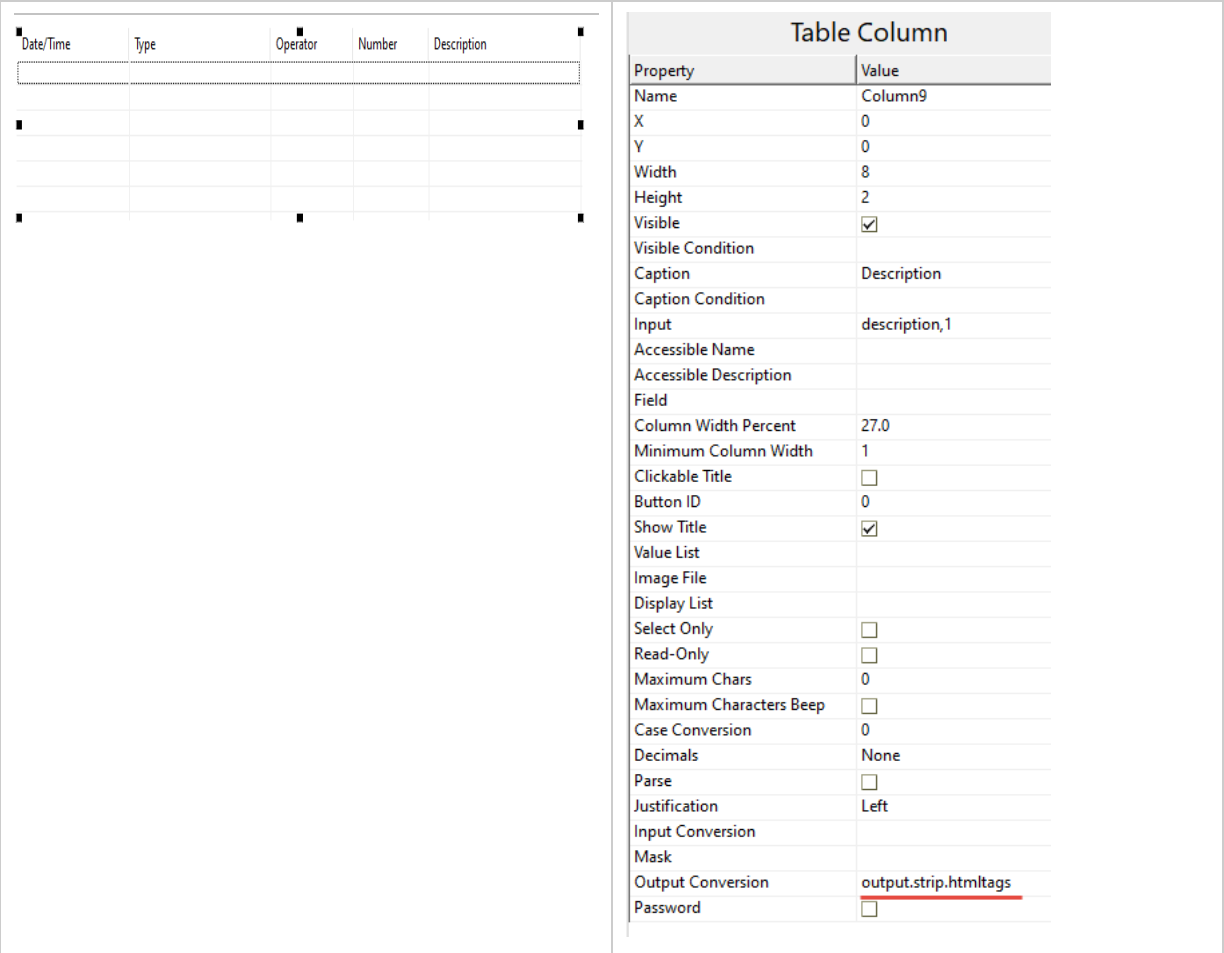
9. The original control of activity logs is an activity table, and the activity table is in a subform with virtual join input. Add **Activity Viewer** to the **Activity Logs** notebook tab, and then set the value of the **Visible Condition** property to the same as that in the backup (for example, **[$L.use.activity]=true**). Set the value of the **Input** property to the same as that in the virtual join field of the activity table in the backup (for example, **vj.incident.id.7**). Set the value of the **Update Type Value List** property to the same as Value List for Update Types (for example, **$G.sd.activity.type**).
Note: For some other modules which don't have Update Types, you may skip setting the value of the **Update Type Value List** property .



10. Design the activity logs detail page. Open the **activityservicemgt** form and replace the Text Area control with HTML Viewer. Keep the value for the **Input** property the same as the original design.
    To locate the table name of the activity logs detail page, you may search for the object name(for example, search for **incidents**), and then go to the **Activities** tab. The activity log table name is displayed in the **Activity log table** field.



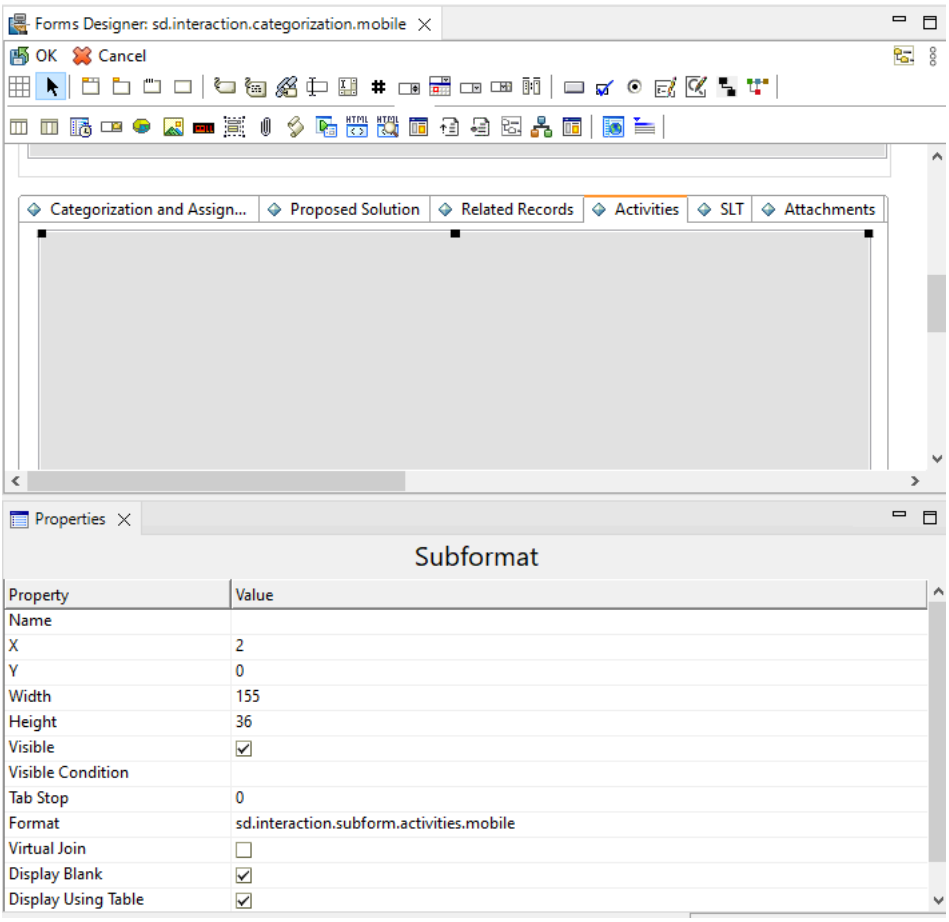
11. Search for the **activityservicedesk.list** format of the activity table. Ensure that the **Output Conversion** property of the **Description** table column is set to **output.strip.htmltags**.



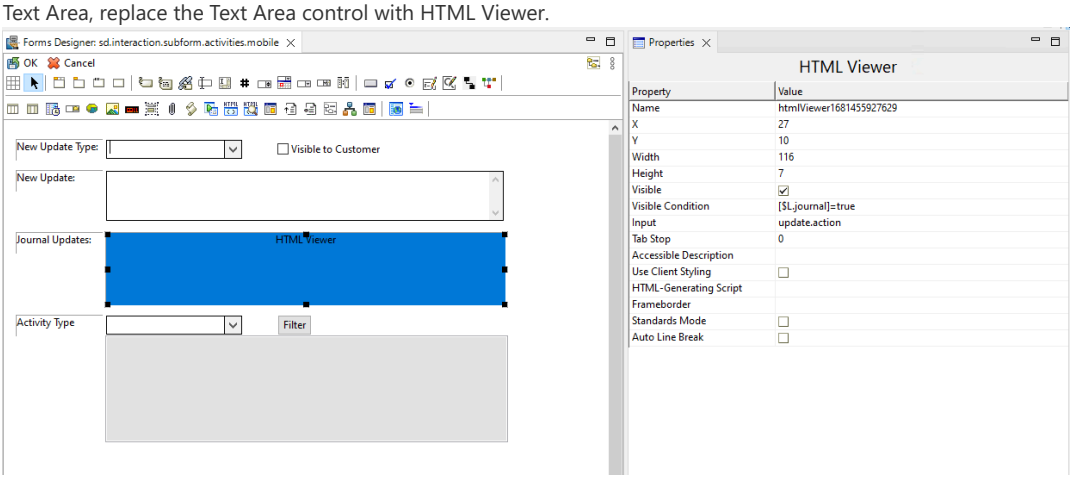
After completing the above tailoring procedures, access an Interaction whose phase is Categorized. The Activities tab is the same as designed just now. Enter some input in the New Updates field, which is now an HTML editor control and you can set the format for your input. After saving the changes, the Journal Updates and Activity Logs tabs display the updates in HTML format.

If you are working with the Mobility client, we also suggest updating the forms for the Mobility client. Perform the steps on all Mobility Interaction forms.

1. Open Forms Designer. Search for the **sd.interaction.categorization.mobile** form. Ensure that the value of the **Format** property on the Activities notebook tab is set to **sd.interaction.subform.activities.mobile**.



2. Open Forms Designer. Search for the **sd.interaction.subform.activities.mobile** form. Check the **Journal Updates** field. If it's a Text Area, replace the Text Area control with HTML Viewer.



After completing the above tailoring procedures, you can view journal updates in HTML format on the Activity tab in the Mobility client. Note that by design, the activity logs section still displays plain text without HTML format.

Perform a series of verifications

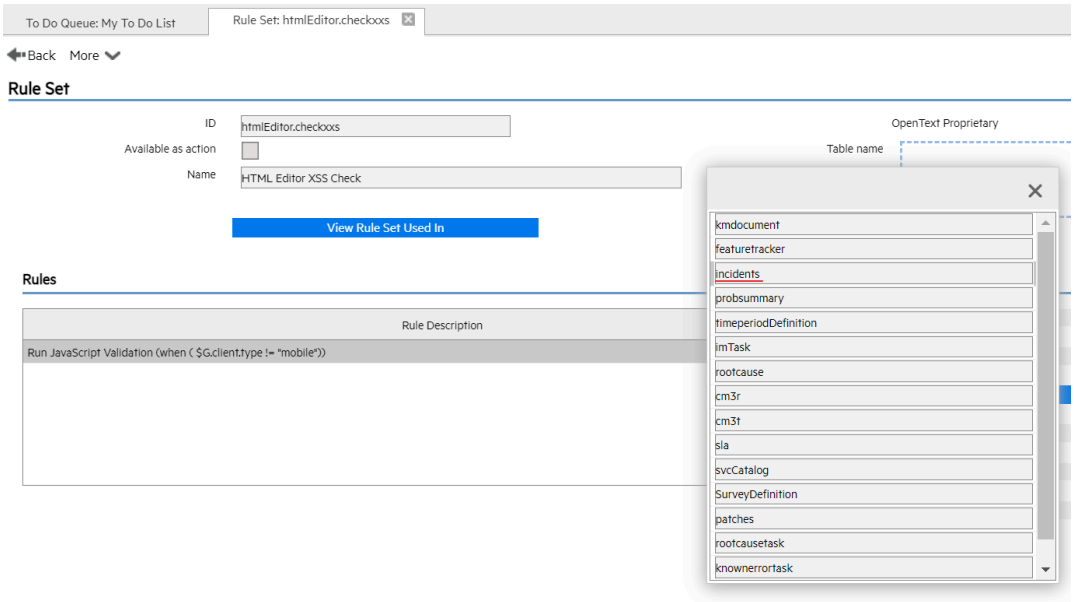
Perform the following verifications or updates to make sure that the corresponding logical files, fields, keys, rulesets, and variables exist for the latest activities feature.

1. Open Database Dictionary and then search for the incidents file. Make sure that the **update.action** field already exists.

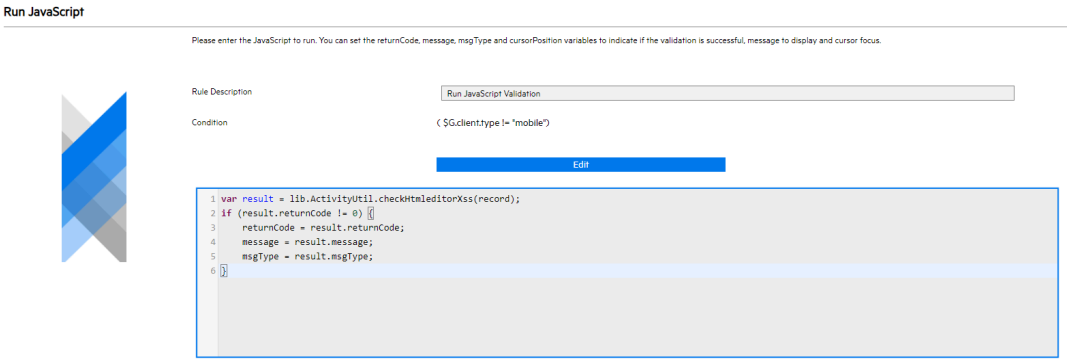
update.action	array	1	153			
update.action	character	2	1	UPDATE_ACTION	TEXT	m1

   If an old form doesn't display journal updates, to adopt the latest activities feature, you need to manually add the **update.action** field.

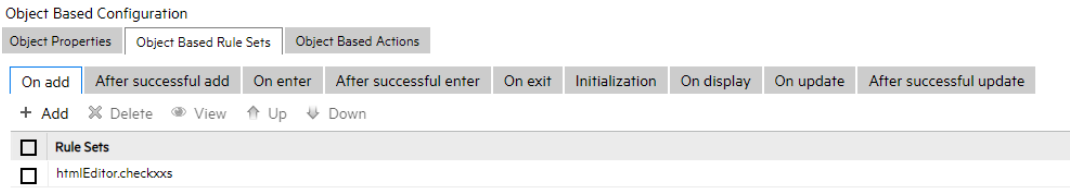
2. Make sure the **htmlEditor.checkxxs** rule set exists. Ensure that the related table name **incidents** is in the Table name list.



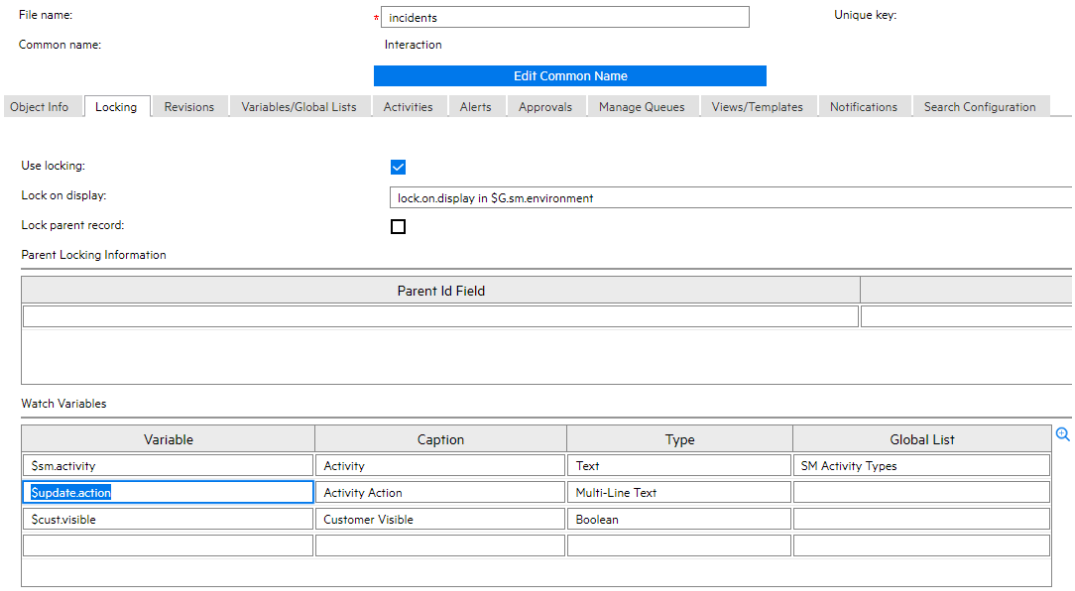
3. Make sure that the scripts below are in the rule.



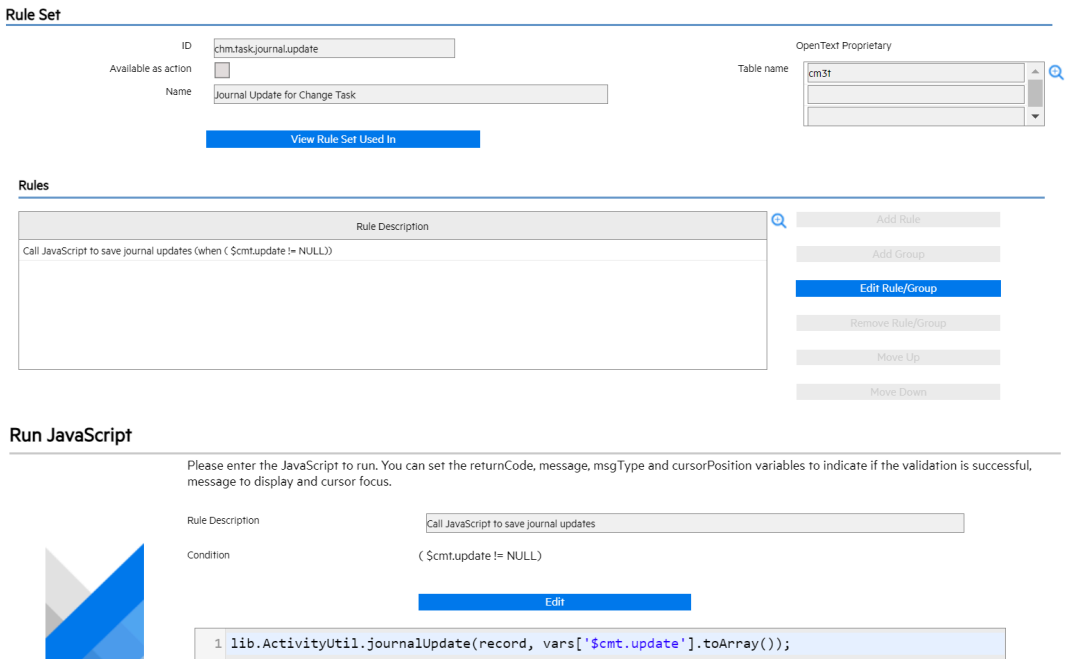
4. Open the workflows for incidents. Click **Edit Object Properties**, and then click the **Object Base Rule Sets** tab. Make sure that the **htmlEditor.checkxxs** rule is added to **On add** and **On update** rule sets.
   Note that for the Knowleget Management module, you also need to make sure that the **htmlEditor.checkxxs** rule is added in the following rule sets:
   kmdocument_republishExternal
   kmdocument_republishInternal
   kmdocument_retire
   kmdocument_submit
   kmdocument_publish



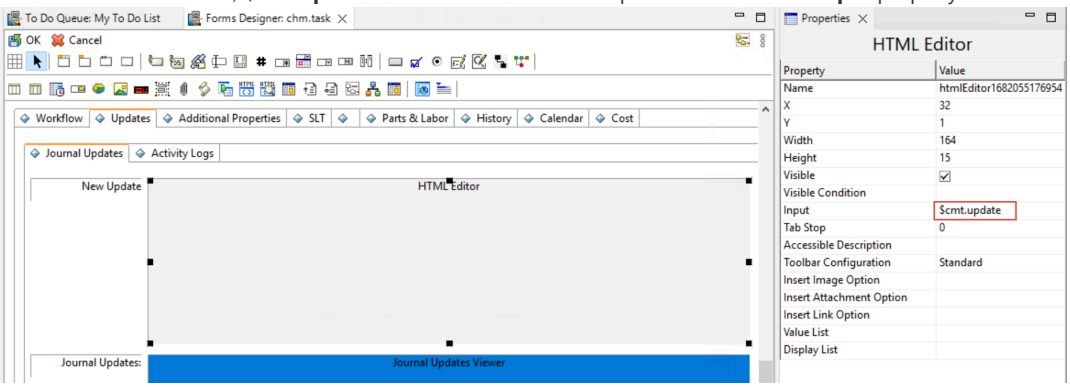
5. Click **Tailoring** > **Document Engine** > **Objects**, and then search for the **incidents** file. Click the **Locking** tab, and then make sure the value for New Update HTML editor's **Input** property (for example, **$update.action**) exists in Watch Variables.
   If those modules which don't display journal updates before upgrade, to adopt the latest activities feature, you need to manually add this variable.



6. If an module doesn't display journal updates before upgrade, to adopt the latest activities feature, you need to manually add rule sets. For example, add the **chm.task.journal.update** rule set to Change tasks. You may search for the ***.journal.update** rule set to check the full list and then add the rule sets as required.

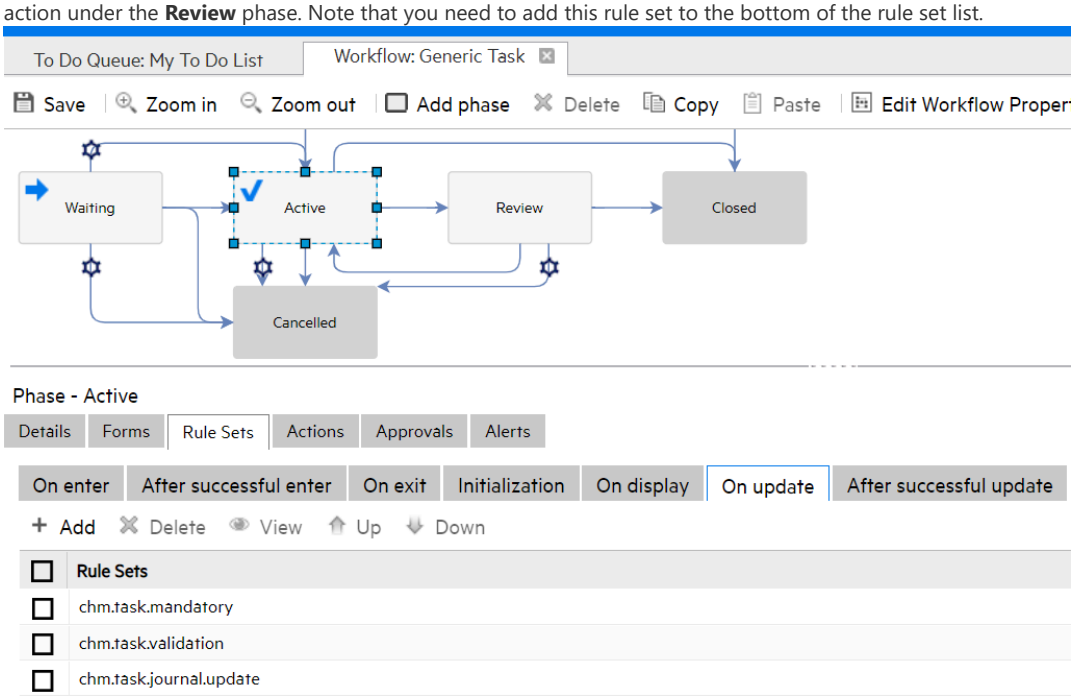


In the screenshots above, **$cmt.update** is the value for the New Update HTML editor's **Input** property.

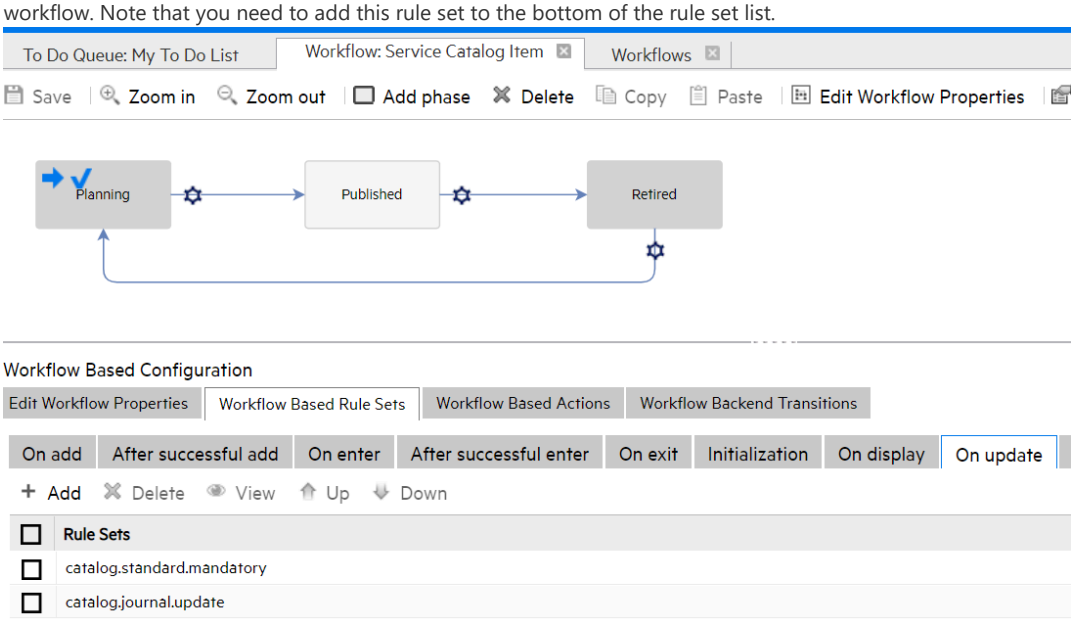


Next, add the rule set to the corresponding workflow or the corresponding phase of a workflow. Refer to the following screenshot as an example. Open the **Generic Task** workflow, add **chm.task.journal.update** to both the **Active** phase and the **On update**

action under the **Review** phase. Note that you need to add this rule set to the bottom of the rule set list.



If the ***xxx*.mandatory** and ***xxx*.validation** rule sets are on a workflow instead of individual phases, you can click **Edit workflow Properties** to add the ***xxx*.journal.update** rule set. See the following screenshot as an example of the **Service Catalog Item** workflow. Note that you need to add this rule set to the bottom of the rule set list.

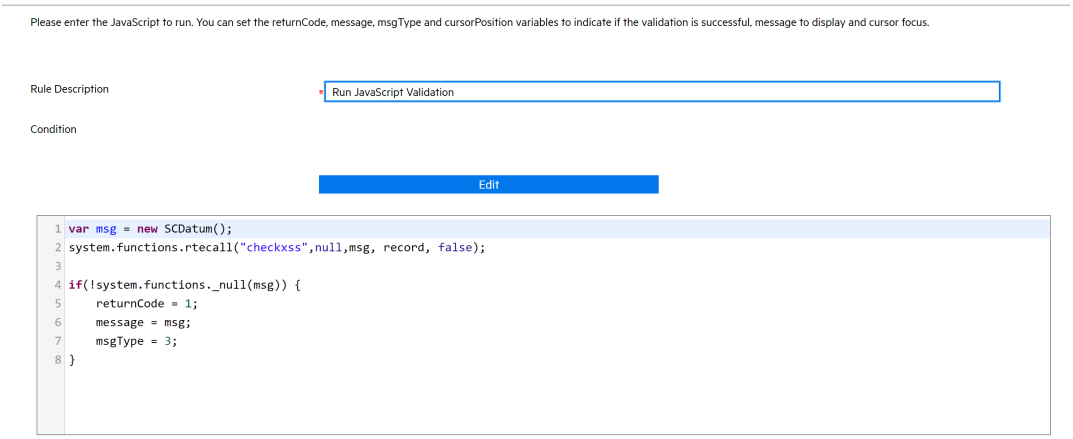


Use the legacy Activities feature

After the applications upgrade, if you still hope to use your previous customized forms with the legacy activities feature, you need to remove some rule sets or scripts as introduced by the applications upgrade. The new activities feature uses these rule sets or scripts to check the HTML Editor Allowlist. Since the **New Update** and **Journal Updates** fields still remain with Text Area controls, and the activity logs section remains with Table control, these rule sets or scripts are unnecessary.

Refer to the following steps for some guidelines.

1. Create a backup of the **htmlEditor.checkxxs** rule set. For example, backup to **htmlEditor.checkxxs.bak**.
2. Revert the scripts to the version before upgrade.



3. Update the related workflows to use **htmlEditor.checkxxs.bak** rule set.
4. For the Knowleget Management module, remove the **htmlEditor.checkxxs** rule from the following rule sets:
   kmdocument_republishExternal
   kmdocument_republishInternal
   kmdocument_retire
   kmdocument_submit
   kmdocument_publish

6.4. Queries

A query is a request to the database (RDBMS) that generates a reply. You can author queries by using RAD (the Service Manager internal system language) or JavaScript. In either case, the query must be translated to the native language of the RDBMS and the reply translated back into the original requesting language. See the Service Manager Programming Guide for information on the Service Manager system language or JavaScript.

In order to write an effective query you must be familiar with the logical structure of the Service Manager tables and fields included in the query as they are defined in the system's database dictionary. It is sometimes also helpful to know the architecture of the tables and fields as they are defined in the database; however this information is often restricted to database administrators (DBAs) in enterprise organizations.

Related topics

How do I write a good query?
Where are queries used?

6.4.1. How do I write a good query?

An ideal Service Manager query has the following characteristics.

- Runs against indexed columns to avoid full table scans. See **Indexing** below.
- Uses operators like equal to (=) that map well to SQL to avoid full table scans. See **Query operators** below.
- Avoids RAD functions like index() that are likely to cause full table scans. See **Function usage** below.
- Ensures that variables always have a non-null value. See **Variable usage** below.

Indexing[edit]

An ideal query runs against indexed columns. Querying against indexed columns increases the speed at which Service Manager can receive data. However, adding too many indexes adds a performance cost to inserting (adding) new records as well as updating or deleting existing ones. Indexes also require additional storage space.

Your database administrator will weigh the potential benefit of indexing your queries against the cost of increased write times (insertions and updates). Typically, your database administrator may want to index any query that many people access or that gets run frequently. You may want to index tables that are part of the following Service Manager frequently-used queries:

- Public favorites
- Commonly-used forms containing Format controls
- Fills
- Drop-downs (combo boxes)
- Global lists
- Background processes

Query operators[edit]

Since Service Manager must translate queries into native RDBMS language, you should try to use operators that map well to native RDBMS languages. In general, queries that use the EQUAL TO operator (=) are the most efficient and easiest to translate.

Operator	Example query	Sample SQL statement
EQUAL TO (=)	`contact.name="AARON, JIM"`	`SELECT "CONTACT_NAME" FROM CONTCTSM1 WHERE "CONTACT_NAME"='AARON, JIM';`

As the example illustrates, an EQUAL TO query often directly translates into a SQL query as part of a WHERE clause. In addition, queries with EQUAL TO operations are also the most likely to use and benefit from table indexes.

If you can't use an EQUAL TO operator in a query, the following operators will also in many cases facilitate direct translation into SQL queries.

Operator	Example query	Sample SQL statement
Starts with (#)	`contact.name#"AA"`	`SELECT "CONTACT_NAME" FROM CONTCTSM1 WHERE "CONTACT_NAME" LIKE 'AA%';`
LIKE	`contact.name like "A*RON*"`	`SELECT "CONTACT_NAME" FROM CONTCTSM1 WHERE "CONTACT_NAME" LIKE 'A%RON%';`
ISIN	`contact.name isin {"AARON, JIM","ARMSTRONG, TRACY"}`	`SELECT "CONTACT_NAME" FROM CONTCTSM1 WHERE "CONTACT_NAME" IN ('AARON,JIM','ARMSTRONG,TRACY');`

In these examples, the LIKE operator directly translates into a SQL query as part of a WHERE clause. The ISIN operator translates into an IN predicate. The STARTS WITH operator (#) translates into a LIKE predicate using a wildcard character.

Such kind of queries now translate into a SQL statement like the following:

`'1 in assets="adv-Unix-101"'`

`'a3.record_number = 1 AND a3.asset = ?'`

Supported query operators

The following operators are supported in a query:

- All logical operators
- TR, and NTR
- Like



Caution

Caution If other operators are used in a query, the query translates into "1=1" and may cause performance issues.

For example, the following queries are supported:

`'1 in assets ="adv-Unix-101"'`

`'7 in assets >"adv-Unix-101"'`

`'7 in assets#"adv-Desktop"'`

`'7 in assets like "adv-*-104"'`



Caution

Caution Only one array member can be used in the same query. For example, the following query is unsupported:

`1 in assets ="adv-Unix-101" and 2 in assets ="adv-Unix-102"`

This is because in the final SQL statement there would be `"a3.record_number = 1 AND a3.record_number = 2"`, which is always `false` for the RDBMS.

Function usage[edit]

Any query that contains a RAD system language function such as index() can potentially cause a full table scan because it may not translate to SQL. Generally, you want to limit the use of functions in queries to avoid degrading your system's performance every time the query is run. If you anticipate a query will run frequently, consider rewriting the query to use operators instead of functions. The most typical high-use queries are those in the following areas:

- Favorites
- Views
- Links
- Stored queries.

For example, suppose you want to create a view that displays closed high priority incidents with activity from a particular operator named Manager. You could create this query by using the index() function.

Inefficient query example: index() function

View definition field	View definition value
Table	`probsummary`
Query	`(flag=false) and (priority.code<="2") and index("Manager", assignee.name)>0`

As written this is an inefficient query. However, you can rewrite this query to use simple operators and prevent unnecessary table scans. For example, the following query accomplishes the same objective.

Efficient query example: using simple operators

View definition field	View definition value
Table	`probsummary`
Query	`(flag=false) and (priority.code<="2") and (assignee.name like "*Manager*")`

If you can't rewrite the query (for example, if you are using an exclusive feature of the function), you can minimize its performance impact by limiting access to the query.

Variable usage and null values[edit]

Anytime you use a custom variable in query you should ensure that it doesn't have a null value. Null values in query variables can cause full table scans which lower your system's performance.You can avoid null values in custom variables by using the following methods:

- Use the nullsub() function to replace the null value with a default value.
- Replace a custom variable with system variable. System variables obtain a value when a user logs in to the system or accesses a particular application.

For example,suppose you create a custom variable $name to store a contact's name. You could provide a default value for the variable with the following expression.

`$name=nullsub($name, contact.name in $L.file)`

In this case, the default value will become the contact name listed in any file that uses this contact.name field. If query doesn't have access to the contact.name field value, you may consider hard-coding the default value, but the utility of such a hard-coded value may be limited.

Alternatively, you may want to use an existing system variable such as `$lo.user.name` or `$lo.ufname` to provide the name of current operator.

Miscellaneous queries[edit]

Service Manager handles miscellaneous queries as described in the following.

Queries with numbers in an array field

Queries with numbers in an array field, such as `'1 in assets="adv-Unix-101"'` , translate into a SQL statement like the following:

`'a3.record_number = 1 AND a3.asset = ?'`

The Service Manager server supports the following operators in such kind of queries:

- All logical operators
- TR, and NTR

- Like

Caution

Caution If other operators are used in a query, the query translates into `"1=1"` and may cause performance issues.

Contents

- 1 Indexing
- 2 Query operators
- 3 Function usage
- 4 Variable usage and null values
- 5 Miscellaneous queries

6.4.2. Where are queries used?

There are several places within Service Manager where you can tailor queries. You should review any custom queries in these areas to ensure they are as efficient as possible:

- Background processes
- Global lists
- Links
- Format controls

Service Manager runs these queries frequently because they are either part of a scheduled task or common user actions trigger them.

Queries in background processes[edit]

Most background processes issue queries to your RDBMS at regularly scheduled intervals.While these queries already have proper indexes in an out-of-box system, you may consider limiting how often these queries run or eliminating them entirely if your system doesn't use them. You should review the following background processes to determine how often your system needs to run them:

- Agent (not started in an out-of-box system)
- Marquee (not started in an out-of-box system)
- Lister

You can control how often these background processes run from the Service Manager schedule record. You can set these processes to run infrequently or never run at all if they're not needed. See the Service Manager help for more information.

Queries in global lists[edit]

Because Service Manager often builds global lists at startup, the total number of global lists you have determines how long your system spends generating these lists. If you have custom global lists, you should verify that they are still in use and if not clear the Build List at Startup option. Reducing the number of global lists you build reduces the number of queries your system has to run and will result in improved login performance.

In addition, you should verify that your custom global list query returns a useful number of results. The more items there are in a global list the more memory a Service Manager client requires to display them. Consider tuning your global list queries to return a manageable number of results to improve your system's performance.

Queries in links[edit]

Because link queries are run frequently, it is essential that you index them correctly and write the queries with the most restrictive fields first in the search order. You can also reduce the amount of computation needed to create links results by minimizing the use of pre- and post-link expressions.

Queries in Format Control[edit]

You can use Format Control to specify the conditions that trigger a query to run. It is most efficient if you only run queries when they are needed. For example, if you plan to use query results during the addition or update of a record, then set the Format Control conditions so that the query only during add or update operations. The same suggestion applies to any custom Format Control calculations and subroutines you use. Verify that you need the custom query and that it runs only when it is needed. By restricting the conditions under which a Format Control query runs you minimize unnecessary calls to the RDBMS and keep system resources free for other operations.

Contents

- 1 Queries in background processes
- 2 Queries in global lists
- 3 Queries in links
- 4 Queries in Format Control

6.5. Database (RDBMS) mapping

The Service Manager database dictionary (DBDICT) contains a logical definition of the tables and columns that make up your relational database management system (RDBMS). Each DBDICT record maps a Service Manager logical file to one or more RDBMS table and a Service Manager logical field to an RDBMS column. In addition, database dictionary records specify what data type the system uses to store data in the RDBMS.

Related topics

How do I create good RDBMS mappings?
Mapping rights

6.5.1. How do I create good RDBMS mappings?

The following tips will help you create good mappings between your Service Manager DBDICT records and your RDBMS.

- Use the DBDICT utility to create and update RDBMS mappings. See Database dictionary (DBDICT) utility.
- Provide Service Manager with create, alter, and drop table rights. See Mapping rights.
- Choose an array mapping that meets your data access needs. See Array mapping options.
- Only change data type mappings to increase the column width and between character data types. See Valid data type changes.
- Know when your database dictionary changes will cause Service Manager to add, update, or delete RDBMS mappings. See When are RDBMS mappings added or updated?.

Related topics

Database dictionary (DBDICT) utility
Mapping rights
Array mapping options
Valid data type changes
When are RDBMS mappings added or updated?

6.5.2. Database dictionary (DBDICT) utility

The database dictionary (DBDICT) utility allows you to view, create, and update the Service Manager logical files, fields, and keys that your system maps to tables, columns, and indexes in your relational database management system (RDBMS).



Important

When modifying a dbdict record, ensure that there are no other users updating existing records or inserting new records in the same file; otherwise errors will occur. For example, when updating the probsummary dbdict, make sure that there are no other users updating existing incidents or opening new incidents.

Do not change (add, modify, or delete) any of the Service Manager system tables, such as the scmessage table. These tables are used by Service Manager to provide the basic functionality that allows the system to function. In addition, use extreme caution when modifying the records in a system table.

The key benefits of the DBDICT utility over other database management utilities are:

- You can define or change how Service Manager maps array data
- You can create database definition language (DDL) for any additions and changes in environments where Service Manager doesn't have access rights to your RDBMS
- You can view and create mappings to null tables in environments where Service Manager doesn't have access rights to your RDBMS
- You can create mappings by importing columns from your RDBMS



Caution

Do not change the `NLS_LENGTH_SEMANTICS` parameter value from `Byte` to `CHAR` in **Oracle** databases.
For details, see NLS_LENGTH_SEMANTICS in Oracle Database Reference.

See the Service Manager help for more information about using the DBDICT utility to create database dictionary records.

6.5.3. Mapping rights

As of version 7.11, your Service Manager system automatically determines if it has the rights to create, alter, and drop tables. If it has such rights, then Service Manager will automatically update your RDBMS anytime you make changes in a DBDICT record.

If it doesn't have these rights, Service Manager creates a null table mapping to act as a placeholder until you can update the DBDICT record with the actual table and column mappings. In addition, Service Manager offers the option to export DBDICT record changes as

database definition language (DDL) so that a database administrator (DBA) can review and manually make the desired changes to the RDBMS. After the DBA has added the needed tables and columns, you can import the actual table and column mappings into the DBDICT record.

6.5.4. Array mapping options

In a database dictionary record, an arrays contain one or more elements of a single data type or a structure field. If an array contains a structure element it is referred to as an array of structure. Arrays of structure contain one or more fields that can be of any data type.

You can map your array data in the following five formats.

- Field in main table
- Field in alias table (Deprecated)
- BLOB in main table
- BLOB in alias table (Deprecated)
- Multi-row array table

The array mapping format you choose determines the following factors about your array data.

- Can your RDBMS directly query your array data?
- Can third-party tools report against your array data?
- Is your array data stored with other fields in one or more main tables?
- Is your array data stored separately in one or more dedicated alias tables?
- Can your array support complex arrays of structure?
- How efficiently does your system retrieve and write data to your array?

Field in main table[edit]

The field in main table format maps a Service Manager array field as a character large object (CLOB) within a main table. This array format is good for situations where the following conditions apply:

- You want to be able to use third-party tools to run reports against your array data
- You want the option to store very large array elements
- You don't want to remap an existing array

The following table summarize the advantages of using this array mapping format.

Advantages

Advantage	Description
Third-party tools can report against the array	Service Manager stores array data as a character string that third-party reporting tools can read. The system separates each element of the array with a \n character delimiter.
The array can store very large array elements	Since the array is mapped to a character large object column, the column can occupy a large amount of space. Most RDBMS vendors support at least 2 gigabytes of large object data in such a column.
The array has access to other fields in the main table	There is a greater probability that other fields reside in the same main table as the array. Having all or most of your fields in a main table reduces the number of SELECT and JOIN statements your system has to make to fetch a record. The less RDBMS traffic required, the better performance your queries and reports will have.
There is no need to remap if you use the default array mapping format	The field in main table format is Service Manager's out-of-box array mapping format. Unless someone has changed the default array mapping format, all your array data will automatically be in this format.

The following table summarize the disadvantages of using this array mapping format.

Disadvantages

Disadvantage	Description
The RDBMS can't directly query the array	Most RDBMS vendors can't query a column containing a reference to character large object data.
Any query against the array is inefficient	The RDBMS can't directly query this array format because the column is mapped to a large object data type. Instead, the RDBMS must scan the table to return all rows of the large object column, and then forward the results to Service Manager for evaluation.
The array shares table space with other fields in the table	The array doesn't have a dedicated table in which to store data. Each RDBMS vendor has its own rules to determine the maximum number of columns allowed in a table, the maximum size of any row, and the maximum table size. Any array in a main table will consume some of these RDBMS resources. If an array requires more resources than any one main table can provide, Service Managerwill move the array to another main table.

To set up an array in a field in main table mapping, you must create the following mappings in a database dictionary record.

Database dictionary element type	RDBMS mapping required
Array data type	Not mapped
Sub-field of the array	Mapped as a CLOB column in a main table
RC flag	False

Field in alias table (Deprecated)[edit]

The field in alias table format maps a Service Manager array field as a character large object (CLOB) within an alias table. **This array format is deprecated**. In earlier versions, this format was intended to support the following features:

- Your RDBMS vendor could only support one "long" data type per table (This is still a restriction on Oracle databases but can be worked around by using CLOB and BLOB data types instead.)
- You wanted the option to store very large array elements
- You wanted to be able to use third-party tools to run reports against your array data

You should use one of the following array formats instead of the field in alias table format.

- Field in main table
- BLOB in main table
- Multi-row array table

BLOB in main table[edit]

The BLOB in main table format maps a Service Manager array field as a binary large object (BLOB) within a main table. This array format is good for situations where the following conditions apply:

- Your array data is in a binary format (such as attachment data or RAD language expressions)
- You want the option to store very large array elements
- You want to support complex arrays of structure
- You don't need to use third-party tools to run reports against your array data

The following table summarize the advantages of using this array mapping format.

Advantages

Advantage	Description
Preservation of any native binary data	The RDBMS doesn't have to do any code page translations to store binary data in the array. Since there is no translation from a binary format to a character-based format, there is no chance that the translation will result in data corruption such as high-bit characters becoming question marks.
The array can store very large array elements	Since the array is mapped to a binary large object column, the column can occupy a large amount of space. Most RDBMS vendors support at least 2 gigabytes of large object data in such a column.
The array has access to other fields in the main table	There is a greater probability that other fields reside in the same main table as the array. Having all or most of your fields in a main table reduces the number of SELECT and JOIN statements your system has to make to fetch a record. The less RDBMS traffic required, the better performance your queries and reports will have.
Support for complex arrays of structure	This array format supports complex arrays of structure such as an array of structure within an array of structure.

The following table summarize the disadvantages of using this array mapping format.

Disadvantages

Disadvantage	Description
Third-party tools can't report against the array	Service Manager stores array data in a proprietary binary format that third-party reporting tools can't read.
The RDBMS can't directly query the array	Most RDBMS vendors can't query a column containing a reference to character large object data.
Any query against the array is inefficient	The RDBMS can't directly query this array format because the column is mapped to a large object data type. Instead, the RDBMS must scan the table to return all rows of the large object column, and then forward the results to Service Manager for evaluation.

To set up an array in a field in main table mapping, you must create the following mappings in a database dictionary record.

Database dictionary element type	RDBMS mapping required
Array data type	Mapped as a BLOB column in a main table
Sub-field of the array	Not mapped
RC flag	True

BLOB in alias table (Deprecated)[edit]

The BLOB in alias table format maps a Service Manager array field as a binary large object (BLOB) within an alias table. **This array format is deprecated**. In earlier versions, this format was intended to support the following features:

- Your RDBMS vendor could only support one "long" data type per table (This is still a restriction on Oracle databases but can be worked around by using CLOB and BLOB data types instead.)
- You want the option to store very large array elements
- You want to support complex arrays of structure
- You don't need to use third-party tools to run reports against your array data

You should use one of the following array formats instead of the BLOB in alias table format.

- Field in main table

- BLOB in main table
- Multi-row array table

Multi-row array table[edit]

The multi-row array table format maps a Service Manager array field as a character string within one or more rows in an alias table. This array format is good for situations where the following conditions apply:

- You want to be able to run RDBMS queries on your array data
- You want to be able to use third-party tools to run reports against your array data
- You want the option to store very large array elements
- You want to support simple arrays of structure

The following table summarize the advantages of using this array mapping format.

Advantages

Advantage	Description
The RDBMS can directly query the array	In this array format, Service Manager stores each array element as a simple character string in one or more rows. This format allows RDBMS queries to read each individual array element without needing to forward the query to Service Manager for evaluation.
Third-party tools can report against the array	Service Manager stores each element of the array in its own separate row. Third-party reporting tools can read the row and extract the element value.
There is no risk of data truncation for new array elements	Each array element occupies as many rows as necessary to store the full element. This means that the length of the array element is not limited by the length of the column. Any array element that exceeds the column length for one row continues onto the next row. This feature prevents Service Manager from truncating array element data.
Support for simple arrays of structure	This array format supports simple arrays of structure such as an array of structure within an array of structure.

The following table summarize the disadvantages of using this array mapping format.

Disadvantages

Disadvantage	Description
Service Manager must maintain a separate table	In order to fetch a record, the system must make SELECT and JOIN statements between the main and alias tables. These statements require more RDBMS traffic and can lower your RDBMS performance.
The array uses more table rows	Each array element will require one or more table rows. Each time you fetch an array record, the system must select multiple rows from an alias table. The more rows the system has to select, the more costly your record retrieval is in terms of RDBMS performance.

To set up an array in a field in main table mapping, you must create the following mappings in a database dictionary record.

Database dictionary element type	RDBMS mapping required
Array data type	Mapped to an alias table
Sub-field of the array	Mapped as a character column in a main table
RC flag	False

Contents

- 1 Field in main table
- 2 Field in alias table (Deprecated)
- 3 BLOB in main table
- 4 BLOB in alias table (Deprecated)
- 5 Multi-row array table

6.5.5. Example: Remapping an array

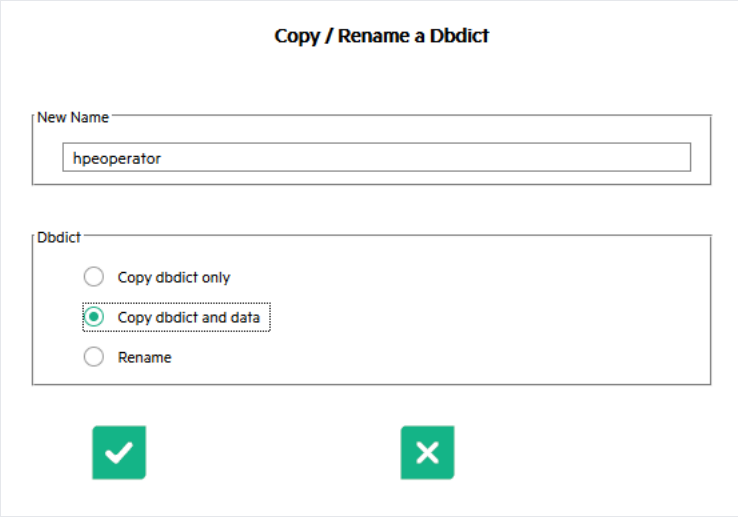
The following example illustrates remapping an array from a field in main table format to a multi-row array table format. Typically, you would remap a field in main table array in order to improve your query performance. Using a multi-row array table array format allows the RDBMS to directly query the array rather than having to forward the query to Service Manager for evaluation.

Related topics

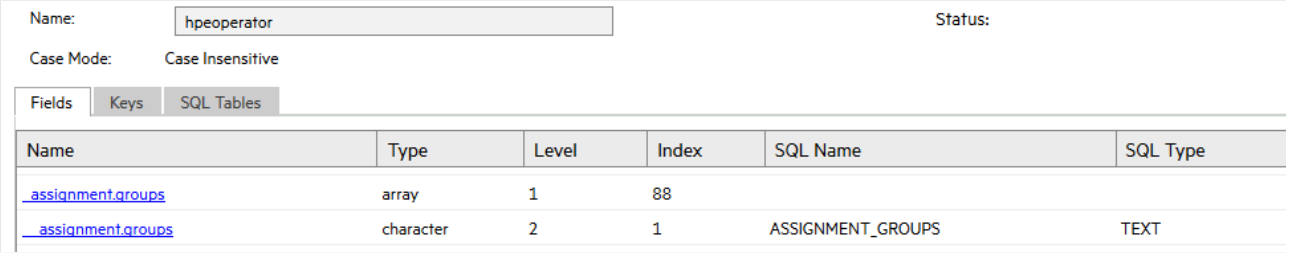
Existing array mapping
Existing array query performance
Prepare to remap the array
New array mapping
New array query performance

6.5.5.1. Existing array mapping

Suppose you want to remap the `assignment.groups` array field in the `operator` table from a field in a main table array to a multi-row array table array. To ensure that your tailoring efforts don't negatively impact your system, you should first make a copy of the `operator` table. For example, create a new table called `hpeoperator`.



The existing array might be mapped as follows on a Microsoft SQL Server RDBMS.



Name	Type	Level	Index	SQL Name	SQL Type
assignment.groups	array	1	88		
assignment.groups	character	2	1	ASSIGNMENT_GROUPS	TEXT

Existing array field mapping: field in main

Database dictionary field	Value
Name	assignment.groups
Type	array
Level	1
Index	88
SQL Name	
SQL Type	
SQL Table	
SQL RC	

Since this is a field in main table mapping, the array itself has no direct mapping on the RDBMS. Only the array's sub-field has a SQL mapping.

Existing sub-field mapping: field in main

Database dictionary field	Value
Name	assignment.groups
Type	character
Level	2
Index	1
SQL Name	ASSIGNMENT_GROUPS
SQL Type	TEXT
SQL Table	m1
SQL RC	False

The sub-field of the array is mapped to a character large object data type. For example, TEXT is a character large object data type in Microsoft SQL Server.

Related topics

Existing array query performance
Prepare to remap the array
New array mapping
New array query performance

6.5.5.2. Existing query performance

Suppose you make the following Service Manager expert search from the Search Interaction Records form.

```
assignment.groups="Application"
```

If you enable the sqldebug:1 parameter, you can see that Service Manager converts this query in the following SQL statement:

```
SELECT NAME, ASSIGNMENT_GROUPS FROM HPOPERATORM1 WHERE ASSIGNMENT_GROUPS='Application';
```

However, since the ASSIGNMENT_GROUPS column is in a character large object data type, the RDBMS can't query the field directly. Instead, the RDBMS must forward each row to Service Manager for it to evaluate the contents of the array. Using the out-of-box data, this query returns all 18 sample incident rows.

18 rows forwarded to Service Manager for evaluation

Operator ID	Assignment Groups
CMS.Process	Application, Hardware, Network
Catherine.Campbell	Application, Network, Office Support (North America) SAP Support (North America)
...	Application, ...
Rachel.Boudreau	Application, E-mail / Webmail (North America), Network, SAP Support (North America)

Because of the inefficiency of this query, the system produces performance alert messages.

```
Partial File Scan for query involving fields {assignment.groups} (se.search.engine,select.records)
```

```
SQL Query incomplete because field (assignment.groups) in file (hpoperator) can't be used in an SQL query (se.search.engine,select.records)
```

```
Hit Ratio not achieved on file hpoperator and query (assignment.groups="Application"): Of 101 records checked so far, 94 did not match the query (display,show.rio)
```

A partial file scan has two adverse performance impacts on your system:

- The RDBMS uses network traffic to send rows to Service Manager
- Service Manager must search within the array of each row the RDBMS forwards to it
- The hit ratio can be low for such queries

Related topics

Existing array mapping
Prepare to remap the array
New array mapping
New array query performance

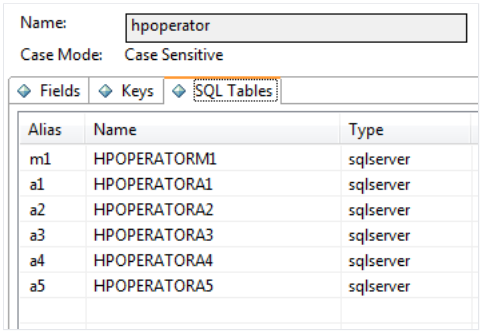
6.5.5.3. Prepare to re-map the array

Before you remap your array, you need to determine the answers to following questions.

- What's the next free alias table for the database dictionary record?
- What SQL data type will the array sub-element require?
- What length should your character fields have?

Determine the next free alias table[edit]

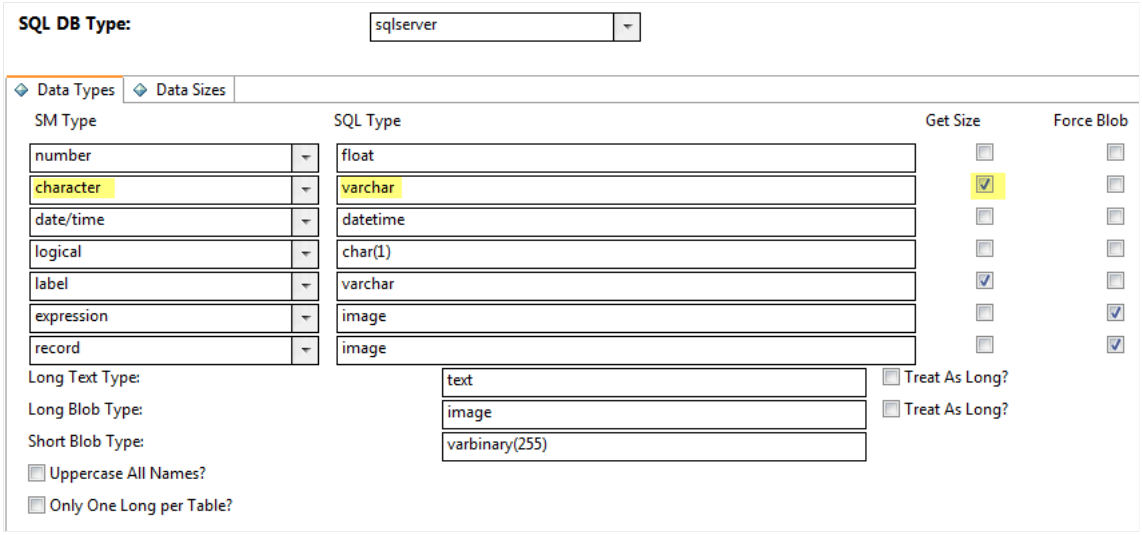
To remap the array as a multi-row array table, you must first determine what the next free alias table is for the database dictionary record. You can see what alias tables are already in use from the **SQL Tables** tab of the database dictionary record.



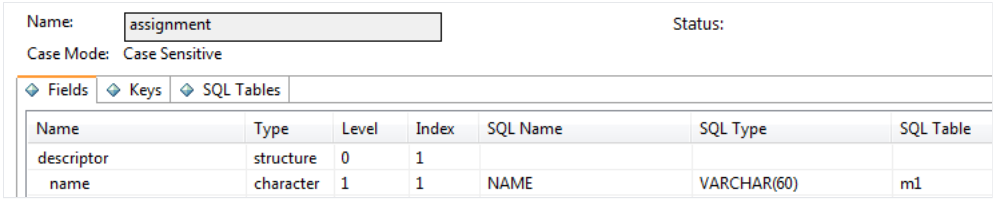
In this case, the incidents database dictionary already has five existing alias tables, so the next free alias table is a6. We shall create a new alias table called a6 that will store our remapped array data.

Determine the data type of the array sub-element[edit]

Next, determine what data type the array sub-element will require in SQL terms. You can use the Service Manager**sqldbinfo** table as a guideline for mapping a Service Manager data type to a SQL data type. For example, the **assignment.groups** sub-element uses the character data type. According to the **sqldbinfo** table, a character data type maps to a varchar data type on Microsoft SQL server.



Verify that your SQL data type mapping meets your data needs. Ideally, you want to identify the source of your data and match its SQL mapping. For example, the source of the **assignment.groups** field in the **operator** table is the **name** field in the **assignment** table. When you look at the database dictionary for the source data, you see that the source data is mapped as a varchar(60) data type.



In this example, we shall remap the array sub-element to use the varchar(60) data type to match the source data type.

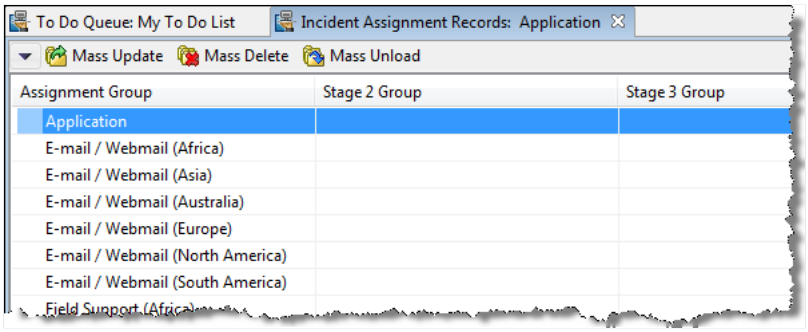
Determine the length of character fields[edit]

If you want to remap your array sub-element with a simple character data type, you must choose a length for the character field. Remember that multi-row array tables can't use character large object data types. There are two methods to determine an appropriate length for your character data.

- Match the length of the data source (preferred)
- Review your existing data and add a padding

Ideally, you want to identify the source of your data and match its SQL mapping. For example, the source of the `assignment.groups` field is the `name` field of the `assignment` table. Since the `name` field is mapped as a varchar(60) data type, we will map our array to the same length of varchar(60).

You can also use your existing data to determine the proper size of your character fields. For example, if you review the source data of the assignment.groups field, you will see text strings from 20 to 40 character long.



You could therefore remap this data as a varchar(40) data type and likely not lose any data. However, you may want to provide the option to store assignment groups more than 40 characters long without having to store the array elements in separate rows. In this case, you may want to use a larger value such as varchar(50) or varchar(60).

When choosing a length value, you want to avoid selecting a SQL length shorter than the maximum length of your existing data. The system will wrap any existing data that exceeds the SQL length of the character field into a new row. Wrapping data into a new row can

result in data corruption and broken queries. For example, if you select a length of varchar(10), then you any existing assignment group longer than ten characters will wrap into two or more rows. Using this mapping, the system would change an existing assignment group of "Application" to have one array element row with the value "Applicatio" and another array element row with the value "n". Any RDBMS query searching for the value "Application" would fail because there is no longer one element with this value.

Contents

- 1 Determine the next free alias table
- 2 Determine the data type of the array sub-element
- 3 Determine the length of character fields

6.5.5.4. Remap array fields to an alias table

To improve your query performance, you can move an existing array field or an array of structure field to an alias table (for example, a1).

Important

- The " field in alias" and "BLOB in alias" mapping formats are deprecated. The multi-row array mapping format is recommended.
- You can revert your remapping changes by moving the field back to the main table, removing the alias table, and restoring the original SQL configuration.
- It may take some time if the table includes large amounts of records.

Array

You can remap an array field to an alias table using the multi-row array mapping format.

To remap an array field using the multi-row array mapping format, follow these steps:

1. Sign in to the Service Manager Windows client as a system administrator.
2. Type **dbdict** in the command line, and press Enter.
3. Open the dbdict that you want to update by performing a search.
4. Select the **Fields** tab.
5. Select the field of the **array** type, and click **Edit Field/Key**.
6. Enter a value in the fields as described in the following table.

   Note

   As an example, the following table lists the SQL Type values allowed for an array of characters. For other types of arrays, the allowed SQL Type values may differ.
   Note that both lines of the field should be mapped to the alias table.

Mapping format	SQL Name	SQL Type	SQL Table	SQL RC
Multi-row array	MULTI_ROW_ARRAY	TEXT, varchar, char, nvarchar, or nchar	a1	False

7. Click **OK**.

Array of structure

To remap an array of structure using the multi-row array mapping format, follow these steps:

1. Sign in to the Service Manager Windows client as a system administrator.
2. Type **dbdict** in the command line, and press Enter.
3. Open the dbdict that you want to update by performing a search.
4. Select the **Fields** tab.
5. In the array of structure, select the top level field (whose type is **array**), change its SQL Table field value to **a1** and leave the rest of the fields empty.
6. Select the middle level field (whose type is **structure**), and make sure its SQL Name, SQL Type, and SQL Table fields are empty.
7. Select each element under the structure, and enter a value in the fields as described in the following table.

   Note

   As an example, the following table lists the SQL Type values allowed for a structure element of the character data type. For elements of other data types, the allowed SQL Type values may differ.

SQL Name	SQL Type	SQL Table	SQL RC
<FIELD_NAME_1>	TEXT, varchar, char, nvarchar, or nchar	a1	False

8. Click **OK**.

Related concepts

Existing array mapping
Existing array query performance
Prepare to remap the array

6.5.5.5. New array mapping

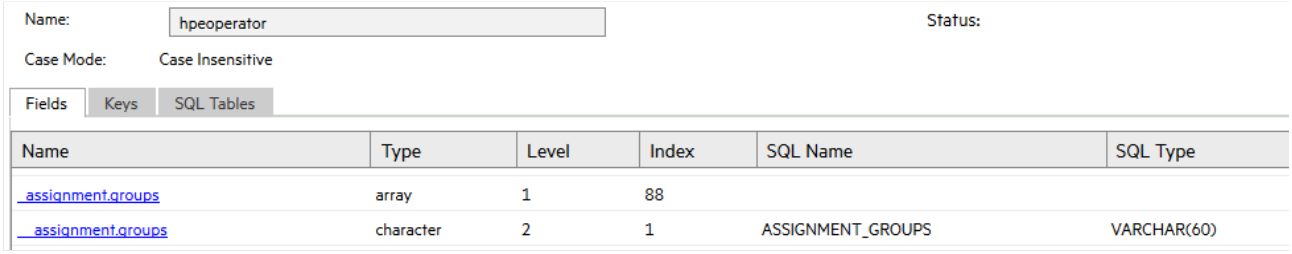
Suppose we have chosen to remap our assignment.groups array to a multi-row array table with the following values.

Multi-row array table remapping options

Mapping option	Value
Next available alias table	a6
Array field remapped to alias table	a6
Array sub-element remapped to alias table	a6
Array sub-element remapped to simple data type	VARCHAR
Array sub-element length remapped to appropriate value	VARCHAR(60)

You can use the following steps to remap the agreement.ids field from a field in the main table array to a multi-row array table.

1. Open the `hpeoperator` file in the database dictionary utility.
2. Double-click on the `assignment.groups` array field.
3. In the SQL Table field, type `a6`.
4. Click **Next** to change the mapping of the array sub-element.
5. In the SQL Name field, verify that the column name meets the naming conventions for your RDBMS. For example, AGREEMENT_IDS is a valid name for a Microsoft SQL Server column.
6. In the SQL Type field, type `VARCHAR(60)`.
7. In the SQL Table field, type `a6`.
8. Click **OK** to update the mapping
9. Click **OK** to save the incident database dictionary record. Service Manager displays remapping status messages in the top right of the form.



Related topics

Existing array mapping
Existing array query performance
Prepare to remap the array
New array query performance

6.5.5.6. New array query performance

Suppose we re-use the same Service Manager expert search from the Search Interaction Records form.

```
assignment.groups="Application"
```

Service Manager generates a new SQL statement that joins the main and alias tables:

```
SELECT m1."NAME" FROM HPOPERATORM1 m1 JOIN HPOPERATORA6 a6 ON (m1."NAME" = a6."NAME") WHERE ((a6."ASSIGNMENT_GROUPS"='Application'));
```

The join statement requires some additional system resources that the old query did not, however since the ASSIGNMENT_GROUPS column is now a simple character data type, the RDBMS can query the field directly. While the query still returns the same 18 sample operator records, this time it doesn't have to forward any rows to Service Manager for evaluation. Nor does running this query generate performance warning messages. This means that the new query is more efficient despite the added overhead of a join statement.

Related topics

Existing array mapping
Existing array query performance
Prepare to remap the array
New array mapping

6.5.6. Valid data type changes

You can only change existing database dictionary mappings under certain conditions. If your database dictionary change doesn't fall within one of the following conditions, you will have to create a new database dictionary mapping to replace the existing mapping.

Valid data type changes

Starting data type	New data type	Example
Fixed-length character field	Variable-length character field with the same column length	Change a CHAR(10) mapping to a VARCHAR(10) mapping
Variable-length character field	Fixed-length character field with the same column length	Change a VARCHAR(10) mapping to a CHAR(10) mapping
Fixed-length character field	Fixed-length character field with a longer column width	Change a CHAR(10) mapping to a CHAR(15) mapping
Fixed-length character field	Variable-length character field with a longer column width	Change a CHAR(10) mapping to a VARCHAR(15) mapping
Variable-length character field	Variable-length character field with a longer column width	Change a VARCHAR(20) mapping to a VARCHAR(40) mapping
Variable-length character field	Fixed-length character field with a longer column width	Change a VARCHAR(20) mapping to a CHAR(40) mapping

In addition to changing a column data type, you can also rename a column or a table.

Valid name changes

Database dictionary item	Equivalent RDBMS item	Example
File name	Table name	Change a table name from MY_TABLE to SAMPLE_TABLE
Field name	Column name	Change a column name from MY_COLUMN to SAMPLE_COLUMN

Related topics

Database dictionary (DBDICT) utility
Mapping rights
Array mapping options

6.5.7. When are RDBMS mappings added or updated?

Service Manager must have proper RDBMS rights to directly change RDBMS mappings. If the system doesn't have these rights, it will not add or update any RDBMS mappings.

Service Manager issues CREATE TABLE statements to create RDBMS mapping in the following circumstances:

- You create a new file (a logical view of a table) in a database dictionary record
- You create new fields (a logical view of a column) in a database dictionary record

Service Manager issues ALTER TABLE statements to update existing RDBMS mappings in the following circumstances.

- You change a fixed-width character field mapping to a variable-width character field mapping
- You change a variable-width character field mapping to a fixed-width character field mapping
- You widen a column
- You rename a column
- You rename a table

Service Manager issues DROP TABLE statements to delete existing RDBMS mappings in the following circumstances.

- You remove a table entry the SQL Table field in a database dictionary record
- You remove a table from the SQL Tables file

Service Manager does a full table copy of an existing table in the following circumstances.

- You change the RC flag for any field in a database dictionary record
- You change a field alias that's not mapped to a NULLTABLE alias
- You associate a new main or alias table to an existing database dictionary record
- You change the RDBMS connection information to use a new type of RDBMS

Related topics

How do I create good RDBMS mappings?
Valid data type changes

7. Process Designer Tailoring Best Practices

Basic concepts

Service Manager Process Designer is a graphical interface that enables you to develop the workflows that control the flow of a single record throughout its lifecycle within Service Manager.

Process Designer enables you to perform the following tasks without being an expert in RAD programming:

- Graphically create and update a workflow
- Define a workflow process in a graphical editor
- Create business rules to drive the workflow logic in a user-friendly interface
- Model complex processes that require branching and looping
- Lower the Total Cost of Ownership of Service Manager by enabling an intuitive and simplified configuration
- Simplify future upgrades by enabling standardization and clarity of business logic

Process Designer includes the following components:

- **Workflow designer**
  A graphic interface used to design and update a workflow
- **Rules editor**
  Used to create rules to enforce business logic in workflows and in forms
- **Security model**
  Provides a common role-based security model
- **Task Planner**
  A graphical interface used to design and update planned tasks

A workflow is a collection of phases, and the transitions between those phases. Each phase represents the current state of the workflow and is linked to a form for data capture.

Transitions

Transitions can connect phases in any manner to create a workflow that includes many branches and loops back to previous phases. Transitions can be triggered manually, automatically, or by default. A manual transition requires the workflow user to make a manual decision to move to the next workflow phase. An automatic transition moves the workflow to another phase based on data in the workflow record. A default transition is a special case that moves the workflow automatically, when no other automatic transition condition applies.

Rule sets

The rules editor enables administrators (such as business process owners, implementers, and developers) to add or remove the out-of-box rules and actions of a workflow to match particular business requirements. You can group individual rules into a rule set to allow their re-use within the process. You can configure a rule set to run when certain events occur, such as when a record is saved or updated. Rules can also be set up to run when a workflow transition occurs.

Actions

You can also configure rules as actions, which allows operators to run rules on demand rather than wait for specific events in the workflow. When an operator initiates an action, the system runs the rule sets that are associated with that phase of the workflow.

The Condition Editor

The Condition Editor enables you to apply conditions to rules, form conditions, actions, alert and approval in phases, and the conditional Task Planner. When a condition is met, the rule or action runs. You can also edit conditions by using the Condition Editor.

The Process Designer security model

The Process Designer security model provides a consistent method to assign permissions to users across all areas of the system. You can use the security model to configure out-of-box rights for a specific role within an area. The security model also provides standardized methods to manage user rights.

Service Manager Codeless modules

Process Designer features are implemented in the Knowledge Management, Change Management, Request Fulfillment, Service Level Management, and Help Desk (comprised of Service Desk, Incident Management, and Problem Management) modules, as well as Time Period and Survey Integration.

To access the Process Designer features from the system navigator, click **Tailoring** > **Process Designer**.

A typical Process Designer flow broadly comprises the following major tasks:

- Enforce business logic on traditional Service Manager forms by creating rule sets with rules that use out-of-box rule types
- Build a process flow by creating a workflow with phases and transitions
- Implementing your business process by associating forms, rule sets, approvals, and alerts in workflow phases and transitions.

7.1. Best practices for configuring rule sets

This section describes the best practices and recommendations for configuring some of the rule sets. Rules sets can enable Service Manager to perform tasks automatically. For example, Service Manager can automatically assign a record to the most applicable group based on a preset rule set.

7.1.1. Configure assignment rules

Service Manager uses assignment rules to automatically distribute records, such as tasks, to the most appropriate groups or individuals for processing. You can configure the assignment rules with different conditions. For example, you can set a rule so that a record is automatically assigned to a group that handles the associated service, or you can set a rule so that a record is automatically assigned to an individual who has the lightest workload.

The following sections provide detailed information about the best practices of configuring assignment rules:

Configure assignment rules for groups[edit]

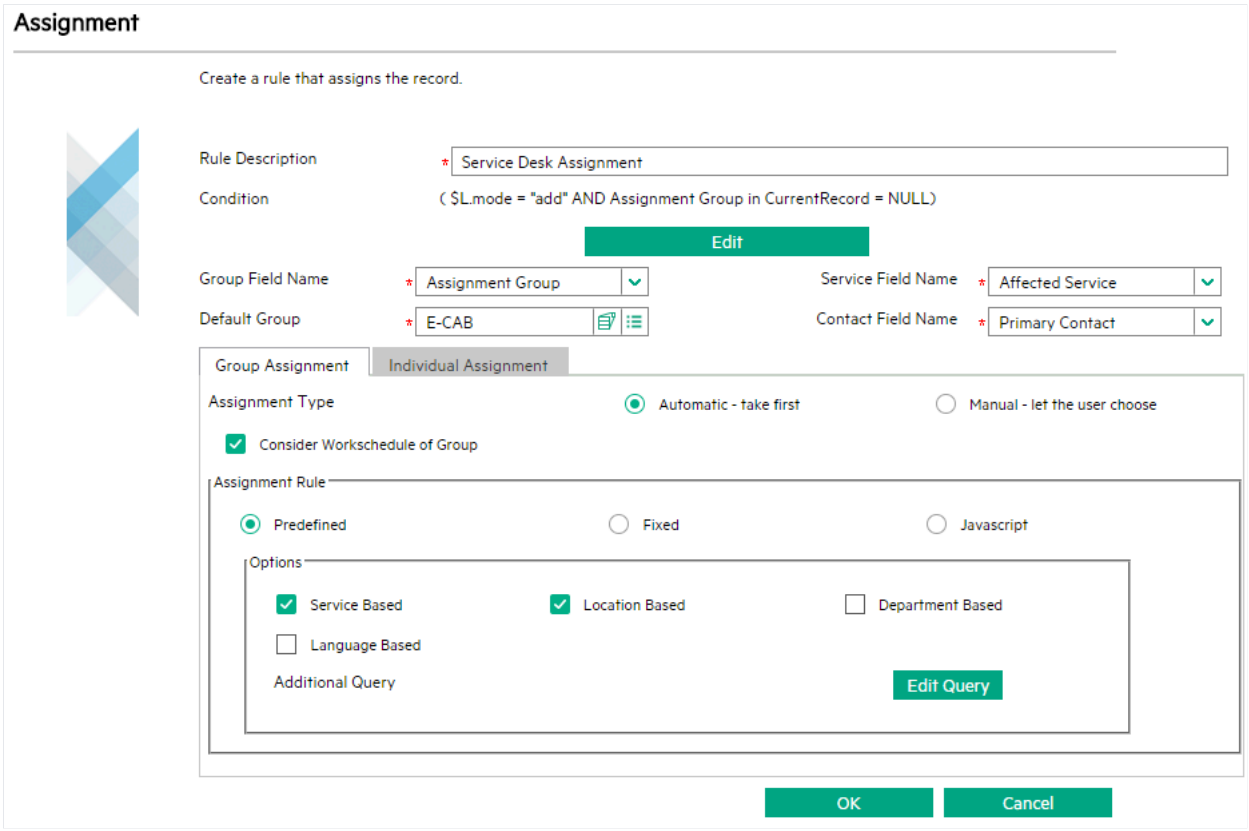
You can configure group assignment rules so that a record can be automatically assigned to a specific group or a group that handles the associated service, or based on location, department, language, or company. The assignment rules can be configured either by using the Assignment form or by using JavaScript.

Assign to a predefined or fixed group[edit]

You can use the **Predefined** or **Fixed** assignment rule to enable Service Manager to automatically assign a record to a group.

If you use a predefined assignment rule, you can specify whether the assignment is based on service, location, department, language, or company. You can also add additional query to filter the assignment groups.

For example, in the out-of-box configuration, an assignment rule is configured in the "sd.init.assignment.set" rule set of the Service Desk workflow. Based on the predefined assignment rule, the system assigns the records based service and location, as shown in the following screenshot.



Assign to a group that handles the associated service[edit]

In Service Manager, a record is usually associated with a service, so you can usually assign the record to the Config Admin group or the Support Groups of the service.

Service Manager follows the following logic to decide the assignment group:

- If the service has only the Config Admin group, and no Support Groups are set, the Config Admin group of the service is used.
- If the service has both the Config Admin group and the Support Groups, then the assignment depends on the assignment type you specify for the rule:
  - If the assignment type is **Automatic - take first**, the Config Admin group of the service is used.
  - If the assignment type is **Manual - let the user choose**, a list that combines the Config Admin group and the Support Groups of the service is shown to the operator for selection.

Note

Note: If this rule is triggered by the system, the assignment type **Automatic - take first** is used even when you have selected **Manual - let the user choose** as the assignment type. This means that the Config Admin group of the system is used.

Use JavaScript to configure assignment rules for groups[edit]

In addition to using the configuration forms to configure assignment rules for groups, you have the flexibility to use JavaScript to implement your assignment rules. In JavaScript, you can use the `groupValue` variable to specify an assignment group or a combination of groups.

In addition, when you assignment rules for groups, you use the **Consider Workschedule of Groups** checkbox to specify whether the work schedule is considered or not for the assignment.

Configure assignment rules for individuals[edit]

You can configure assignment rules for individuals so that a record can be automatically assigned to a specific individual for processing. The assignment rules can be configured either by using the Assignment form or by using JavaScript.

You can configure the assignment rules so that a record can be automatically assigned to the following individuals:

- None: Do not assign to any individual

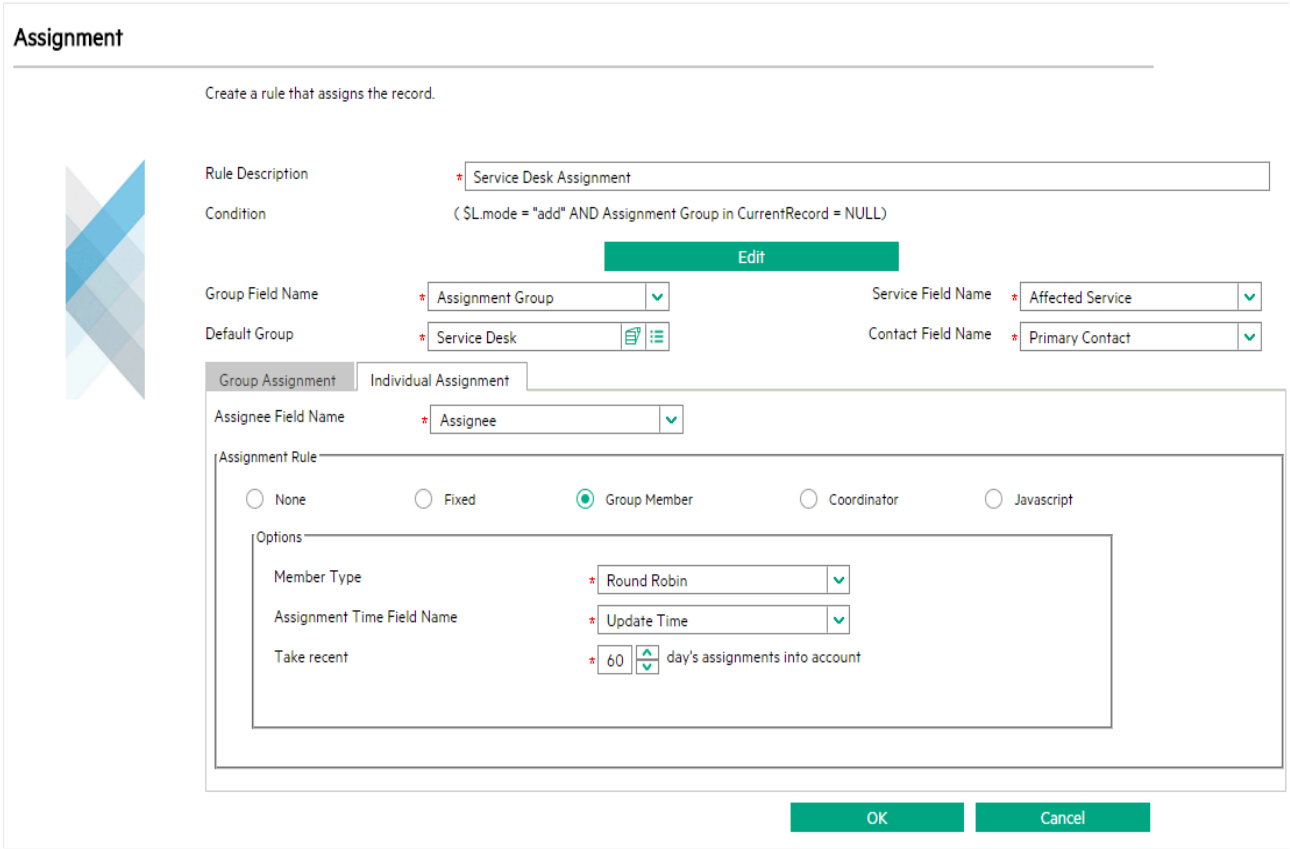
- A member of the assigned group: The member can be decided in a round robin manner or based on the workload of the group members
- Coordinator of the assigned group
- A specific assignee

Assign in a round robin manner[edit]

In the round robin manner, the system checks the latest assignment time of the members in a group, and then assigns the record to the member who has the earliest assignment time.

If you have a huge number of records to be assigned in Service Manager, follow these guidelines to avoid possible performance issues:

- **Only take recent assignments into account.** By default, the value of 60 days is set while you add assignment rules in the round robin manner. You can adjust the value based on your needs. If this value is set to 60 days, it means that only assignments within the most recent 60 days are considered; assignments before 60 days are ignored. If the value is set to 0, it means there is no assignment time restriction; all the assignments of the members are considered.



- **Create index on the assignment time field and the assignee field of the corresponding table.** This can avoid full table scans and thus can increase the query speed. At run time, the calculation SQL is as follows:

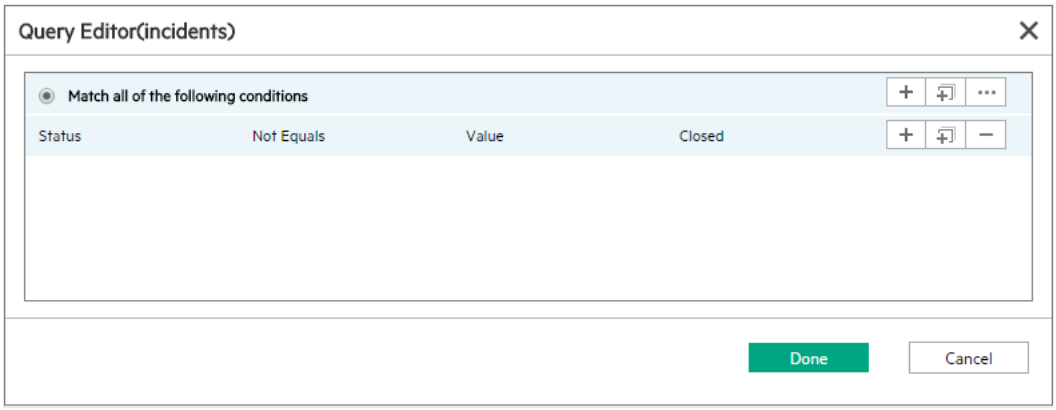
```
select [assignee field], max([assign time field]) time from [ticket file] where [assign time field] >= [assigned time restriction] and [assignee field] isin [members in group] group by [assignee field] order by time
```

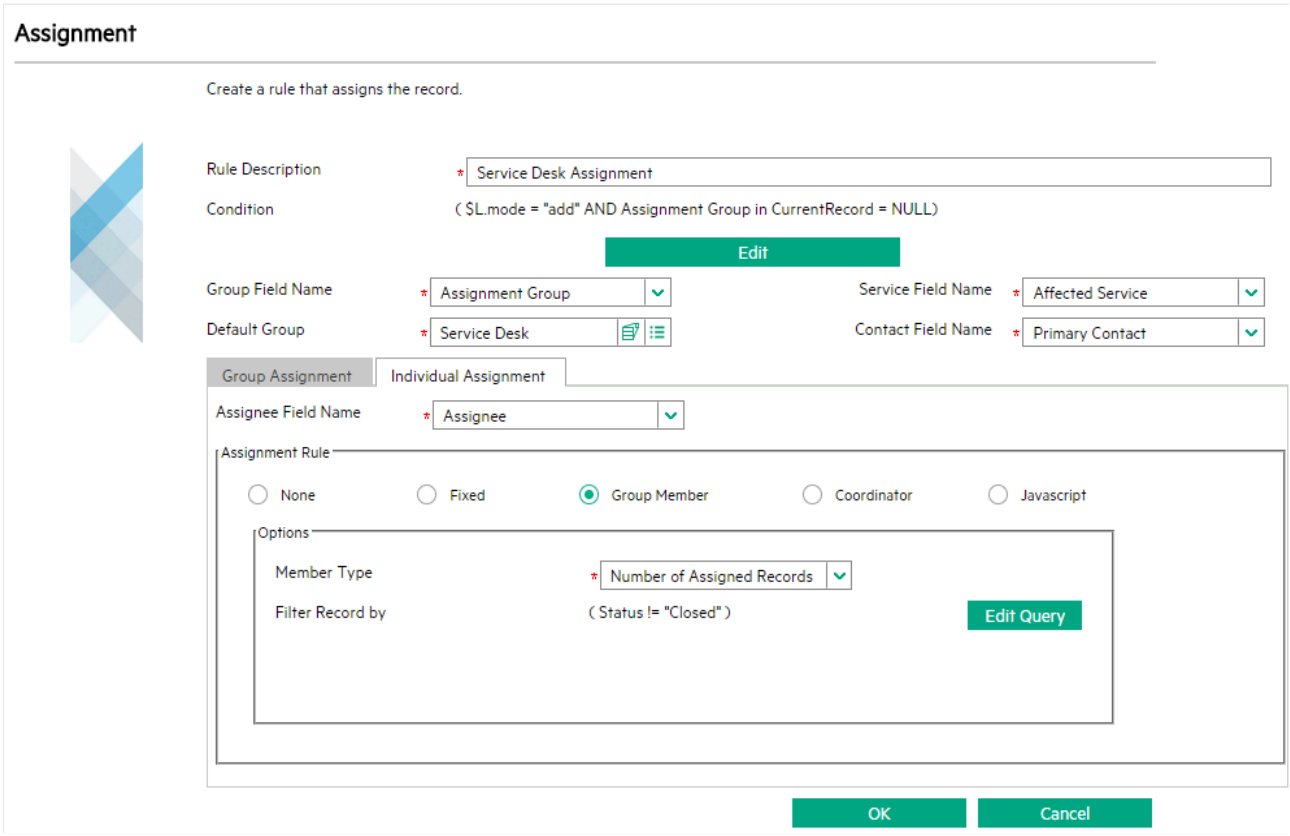
Assign based on workload[edit]

You can use the "Number of Assigned Records" member type to configure the assignment rules so that a record can be assigned based on people's workload. The system checks the number of working records of all the members in a group and then assigns the record to the member who has least working records.

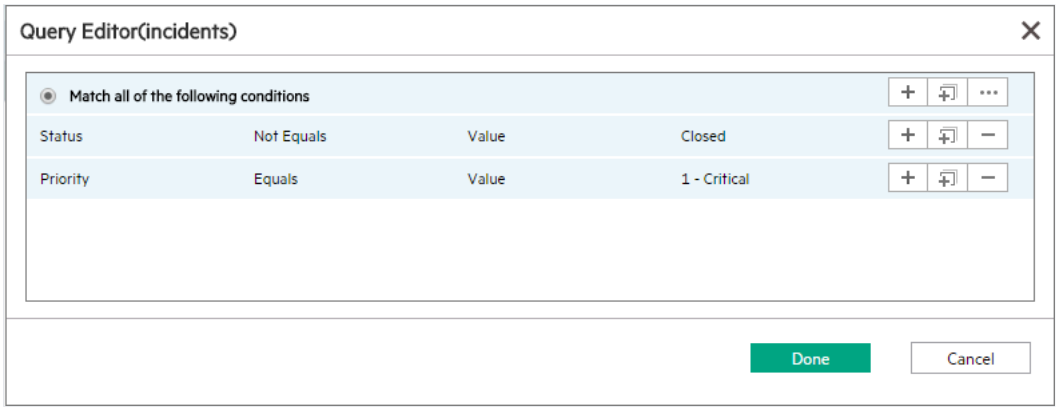
To avoid possible performance issues for this assignment type, follow these guidelines:

- **Only take open records into account.** Usually you need to append the query that filters out records other than the open records. You can use the Query Editor to edit the query string. In the example below, the query string Status != "Closed" is appended.





In addition, you can further narrow down the filter result. For example, if you plan to take only the open and critical records into account, you can append to the query string as shown below:



- Create index on the assignee field and the fields of the appended query. This can avoid full table scans and can thus increase the query speed. At run time, the calculation SQL is as follows:

```
select [assignee field], count(*) ncnt from [ticket file] where [assignee field] isin [members in group] and [the appended query string] group by [assignee field] order by ncnt
```

Use Javascript to configure assignment rules for individuals[edit]

In addition to using the configuration forms to configure assignment rules for individuals, you have the flexibility to use JavaScript to implement your assignment rules. In JavaScript, you can use the `assigneeValue` variable to specify an assignee.

Contents

- 1 Configure assignment rules for groups
  - 1.1 Assign to a predefined or fixed group
  - 1.2 Assign to a group that handles the associated service
  - 1.3 Use JavaScript to configure assignment rules for groups
- 2 Configure assignment rules for individuals
  - 2.1 Assign in a round robin manner
  - 2.2 Assign based on workload
  - 2.3 Use Javascript to configure assignment rules for individuals

7.1.2. Run action rules

You can use the Run Action rules to enable Service Manager to perform automatic cross table or module operations. For example, you can set the rules so that the solution of an incident can be copied to the interaction from which the incident is escalated.

Service Manager supports three types of the Run Action rules:

- Run Action rules on related records
- Run Action rules on other records
- Run Action rules on the current record

7.1.2.1. Configure Run Action rules on related records

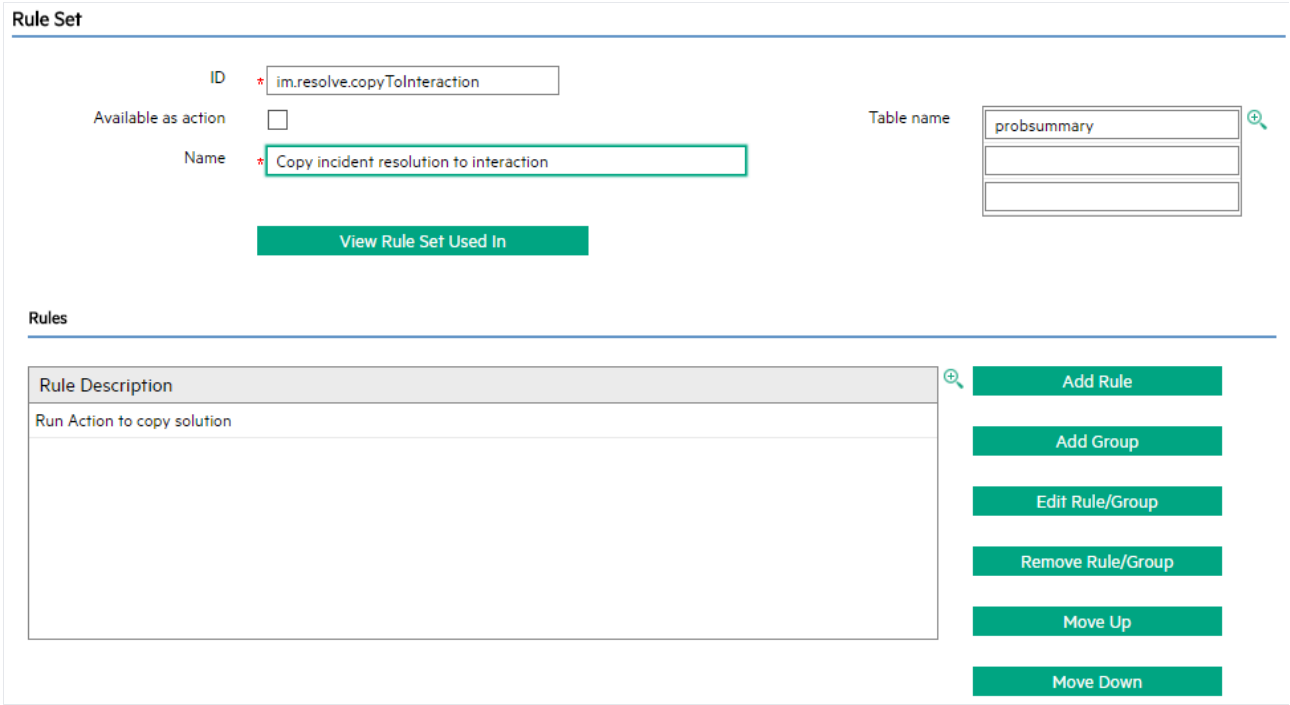
You can configure the Run Action rules to enable Service Manager to perform automatic operations on related records. The relationship of the related records is usually maintained by the Related Records functionality.

For example, when an incident is resolved, you might want to copy the incident resolution to the related interaction from which the incident is escalated. In this case, you can configure the Run Action rules with these steps:

1. Define a rule set on the source table
2. Define a rule set on the target table
3. Configure the rule set on the source table into an appropriate workflow

Define a rule set on the source table

To copy the incident resolution, you can define a rule set on the source table “probsummary” that stores incident records. The rule set definition form is as follows:



In addition to the above configuration, you also need to perform the following tasks:

- Select **Related Records** for the "Run Action on" field
- Select **Escalate From Interaction** for the "Relation Type" field. This means the target related record is the interaction from which the current incident is escalated.
- Select **sd.copyIncidentSolution** for the "Run Rule Set" field. This rule set is executed against the related interaction.
- Select **Save** for the "Action after Rule Set" field. The rule set is saved for the related interaction.

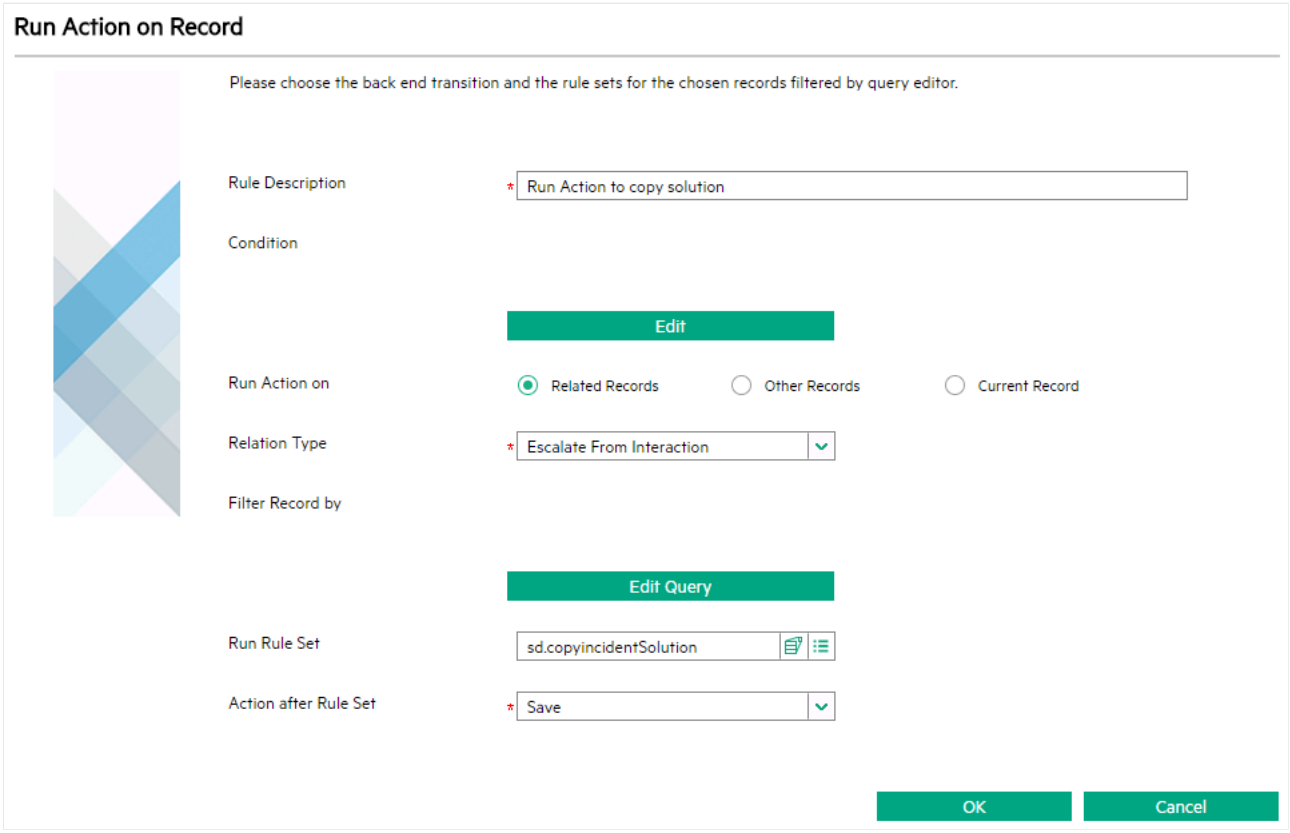
Note

Note: You can select "Do nothing", "save" or one of the back end transitions for this field.

- The back end transition list is retrieved from all the workflows of the current related record file based on your selected relation type. At run time, if some of the workflows don't have your selected back end transition, the corresponding transition is not run.
  For example, suppose you have a Change file "cm3r" and it meets the following conditions:
  - Relation type: You have selected "Caused Changes" as the relation type.
  - Workflows: Based on the selected relation type, there are two workflows for the Change file: "Normal Change" and "Standard Change".
  - Back end transitions: The "Normal Change" workflow has the back end transition "event.close", whereas the "Standard Change" workflow doesn't.

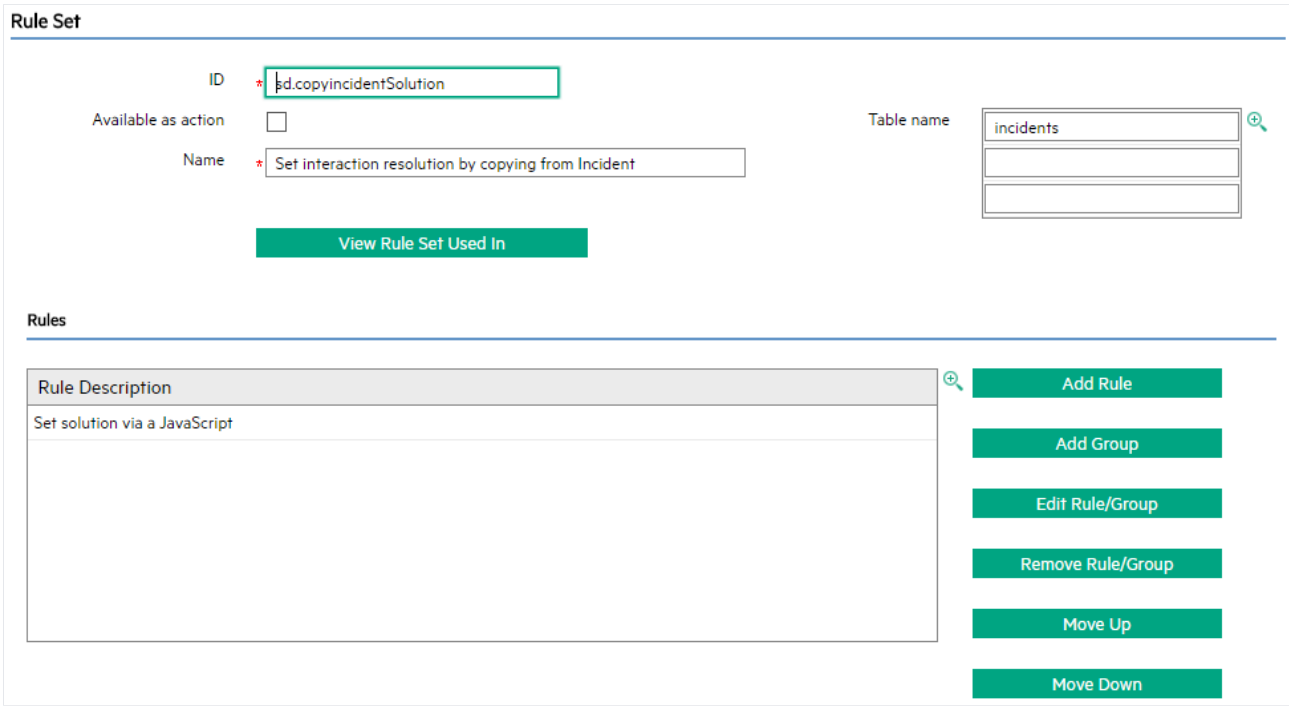
  In this case, if you select the action "Backend Transition: event.close", and if at run time there are two related caused changes: One for "Normal Change" and the other for "Standard Change", the back end transition is only run on the "Normal Change" workflow, not on the "Standard Change" workflow.
- You are suggested to use the same back end transition name in different workflows if these backend transitions follow the same business logic.
- While a back end transition is selected from the list, the interface is updated to indicate to which workflows the transition applies and to which it doesn't apply.

The form on which you can perform these tasks is shown below:



Define a rule set on the target table

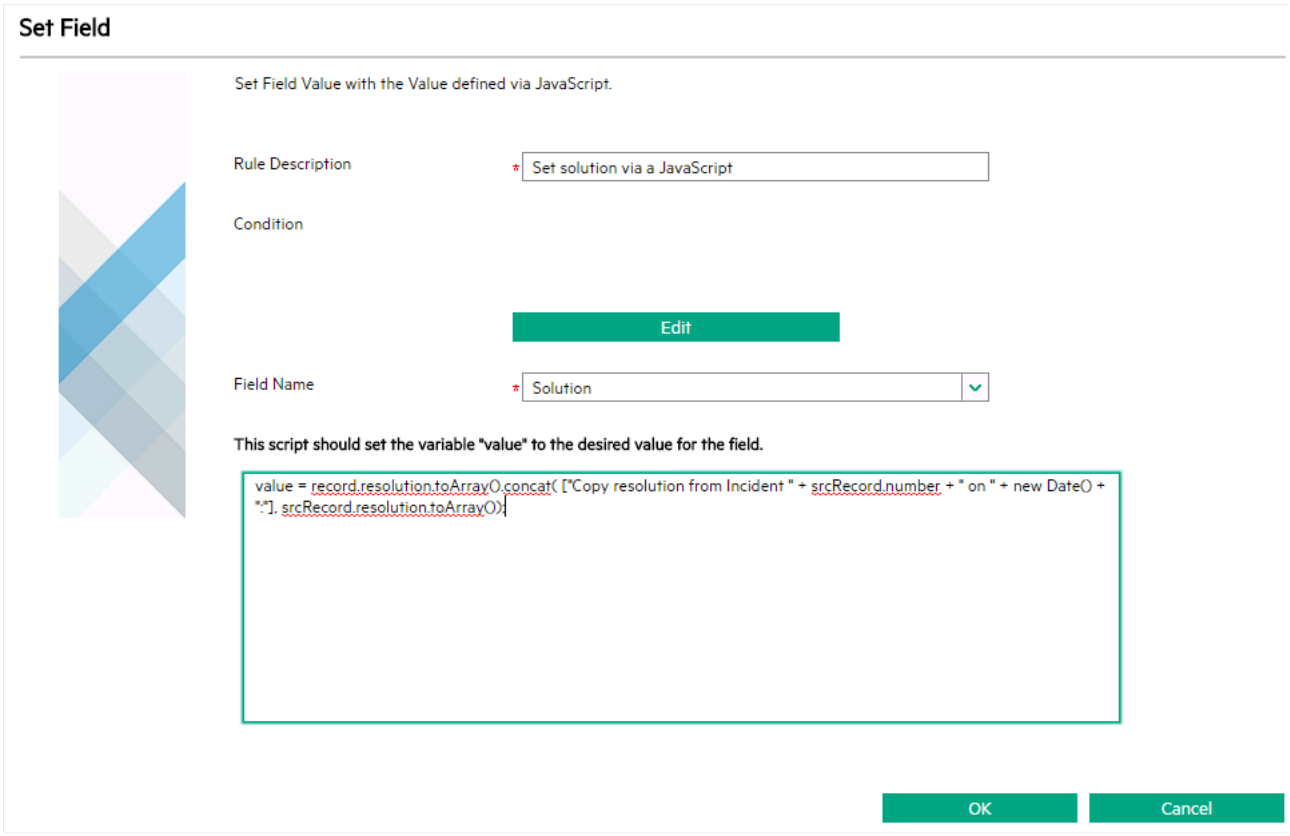
You need to define a rule set on the target table. This rule set is used as the value of the "Run Rule Set" field when you define the rule set on the source table. In this example, you define the rule set “sd.copyIncidentSolution” on the target table “incidents” that stores the interaction records.



You need to configure a Set Field rule in this rule set to append the resolution of the incident to the resolution field of the interaction. To do this, you can use JavaScript, in which you can refer to the current source record as `srcRecord` and the original copy of the source record (before any changes are made by the user) as `oldSrcRecord`. The system only supports invoking the current rule from the Run Action rule at run time. The JavaScript of this example is as follows:

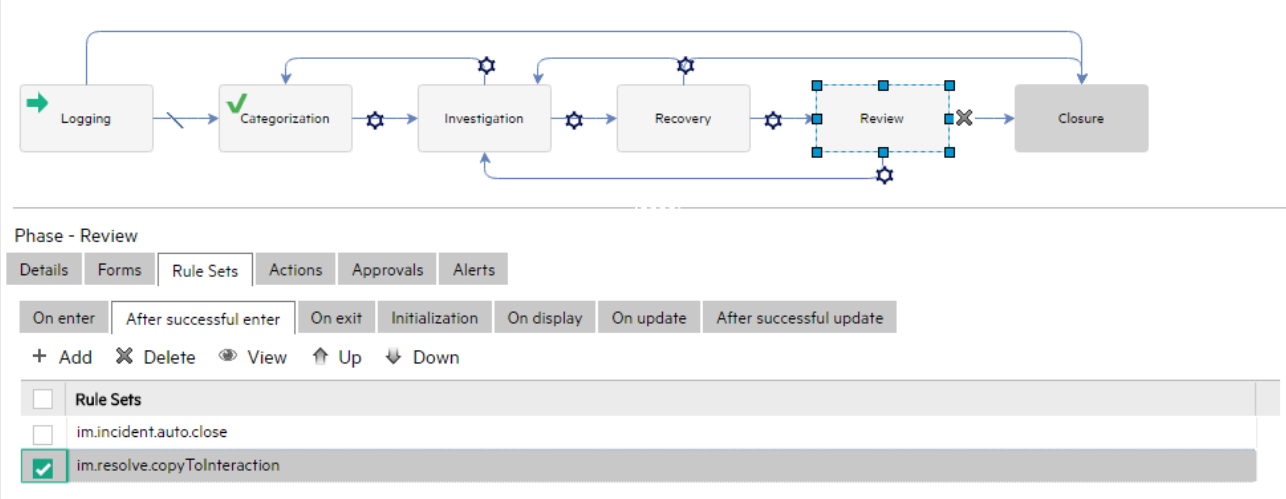
```
value = record.resolution.toArray().concat( ["Copy resolution from Incident " + srcRecord.number + " on " + new Date() + ":"], srcRecord.resolution.toArray());
```

In JavaScript, the current record and the original copy of the record are also referred as `record` and `oldRecord`. In this example, they are the current interaction record and original interaction record.



Configure the rule set on the source table into an appropriate workflow

In this example, you can configure the rule set into the **After successful enter** event of the Review phase of the Incident workflow. This means only after the incident record is resolved, the rule set is triggered.



Note

Note: The Run Action rule is executed in a sequential manner, which means the configured rule sets and actions of each related record are called one after another. If one of the rule sets and actions fails, for example, one of the related record is locked by another user, the whole execution stops and returns with a failure result. If you want the system to handle the failed execution on related records caused by record locking, you can combine the Run Action rules and the Run Scheduled Action rules. For details, see Combine the Run Action rules and the Run Scheduled Action rules.

7.1.2.2. Configure Run Action rules on other records

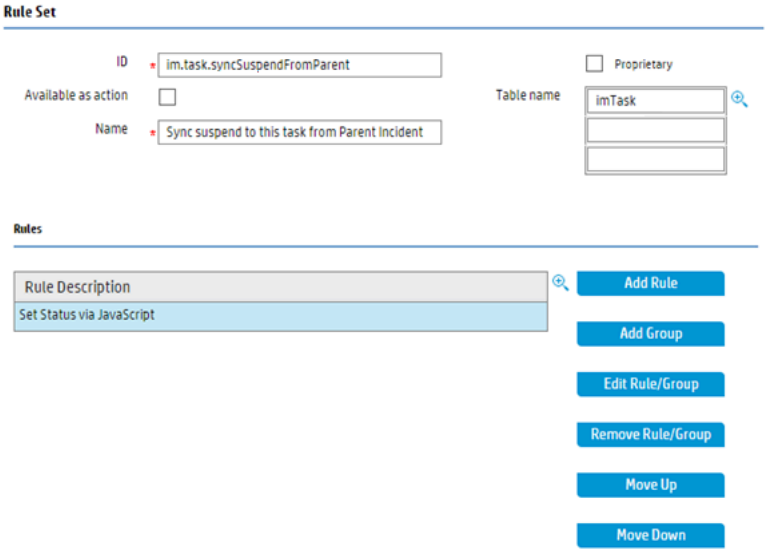
You can configure the Run Action rules to enable Service Manager to perform automatic operations on other records. The relationship of the records is built by manually defined query string. For example, you might want to sync up the status of an incident to its relevant incident tasks, that's to say, if the incident is suspended, the relevant incident tasks should be suspended as well.

To configure Run Action rules on other records, follow these steps:

1. Define a rule set on the source table
2. Define a rule set on the target table
3. Configure the rule set on the source table into an appropriate workflow

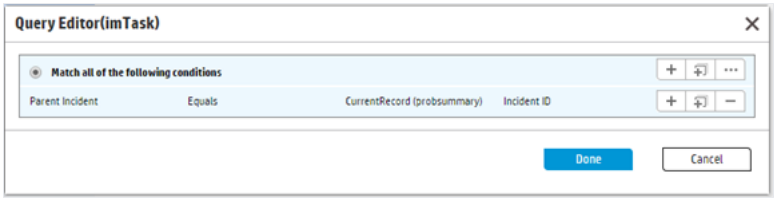
Define a rule set on the source table

In this example, you can define the rule set on the source table “probsummary” that stores incident records:



In addition to the above configuration, you also need to configure the Run Action rules in the rule set as follows:

- Select **Other Records** for the "Run Action on" field.
- Select **imTask** for the "Table Name" field. This means the target records are incident tasks.
- Use the Query Editor to configure the Filter Record as shown below. The only incident task that belongs to current incident record is included.



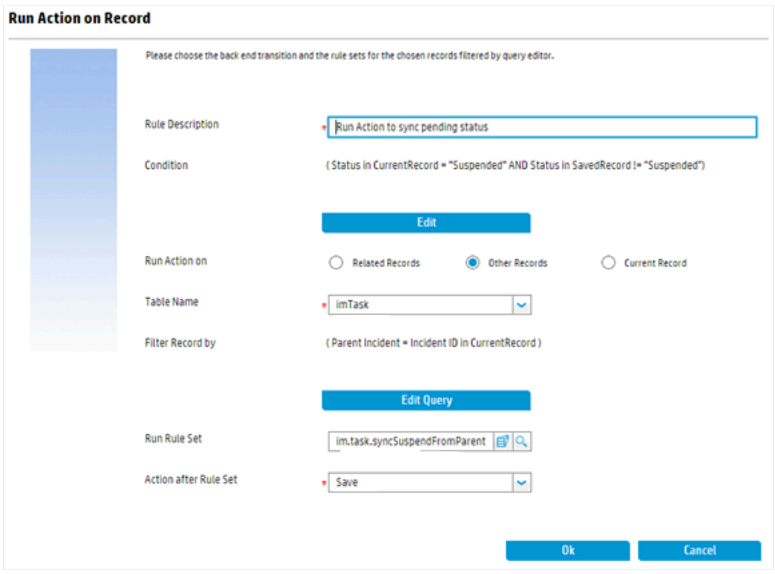
- Select **im.task.syncSuspendFromParent** for the Run Rule Set field. This rule set is executed against the related incident task.
- Select Save for the "Action after Rule Set" field. The save action is executed against the related incident task.

Note

NoteYou can select **Do nothing**, **Save**, or one of the back-end transitions for this field.

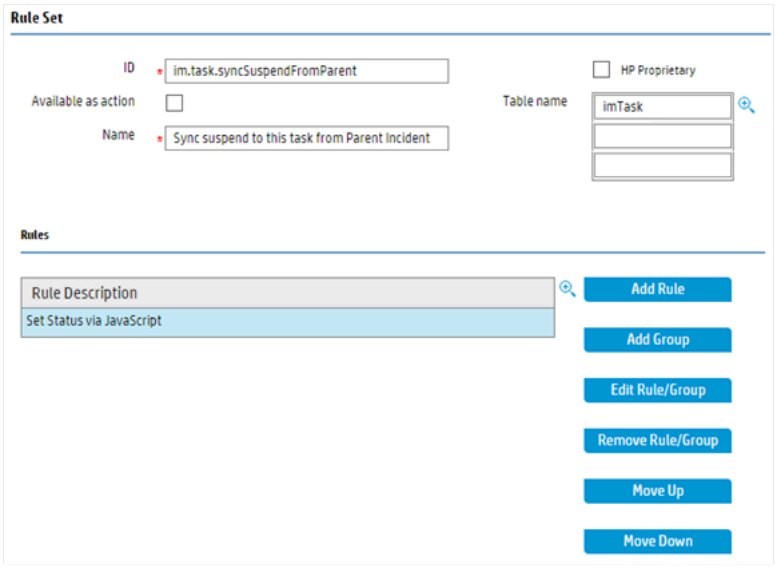
- The back-end transition list is retrieved from all the workflows of the selected record file. Similar to Configure Run Action rules on related records, at run time, if some of the other records don't have your selected back-end transition, the corresponding transition is not run.
- You are suggested to use the same back-end transition name in different workflows if these back-end transitions follow the same business logic.
- While a back-end transition is selected from the list, the interface is updated to indicate to which workflows the transition applies and to which it doesn't apply.

The form on which you can perform these tasks is shown below:

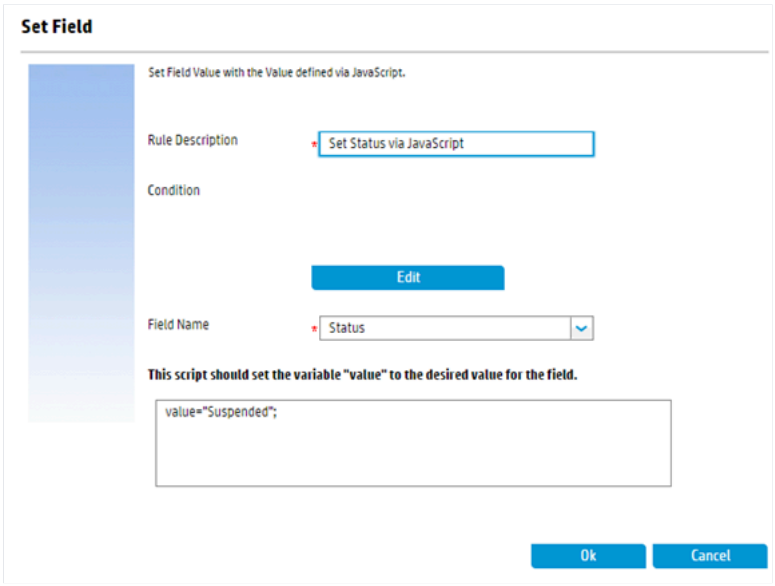


Define a rule set on the target table

You need to define a rule set on the target table. This rule set is used as the value of the "Run Rule Set" field when you define the rule set on the source table. In this example, you define the rule set “im.task.syncSuspendFromParent” on the target table “imTask” that stores the incident task records.



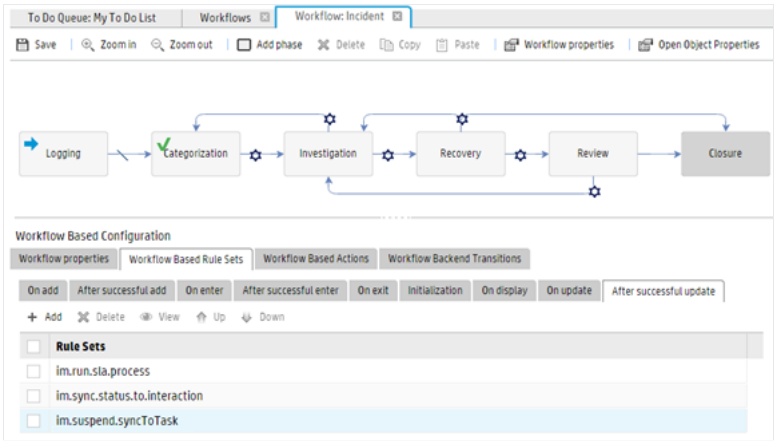
You need to configure a Set Field rule to change the status of the incident task to the value of “Suspended”. To do this, you can use JavaScript, in which you can refer to the current source record as `srcRecord` and the original copy of the source record (before any changes are made by the user) as `oldSrcRecord`. The system only supports invoking the current rule from the Run Action rule at run time.



In JavaScript, the current record and the original copy of the record are also referred as `record` and `oldRecord`. In this example, they are the current incident task record and original incident task record.

Configure the rule set on the source table into an appropriate workflow

In this example, you can configure the rule set into the **After successful update** event of the incident workflow, which means when the status of the incident record is changed to “Suspended”, the rule set is triggered.



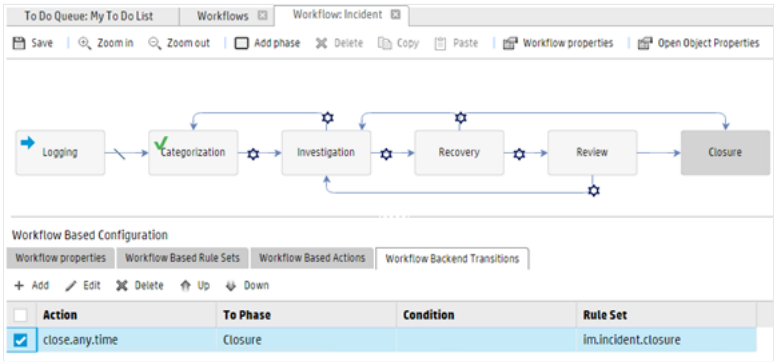
Note

Note: The Run Action rule is executed in a sequential manner, which means the configured rule sets and actions of each related record are called one after another. If one of the rule sets and actions fails, for example, one of the related record is locked by another user, the whole execution stops and returns with a failure result. If you want the system to handle the failed execution on related records caused by record locking, you can combine the Run Action rules and the Run Scheduled Action rules. For details, see Combine the Run Action rules and the Run Scheduled Action rules.

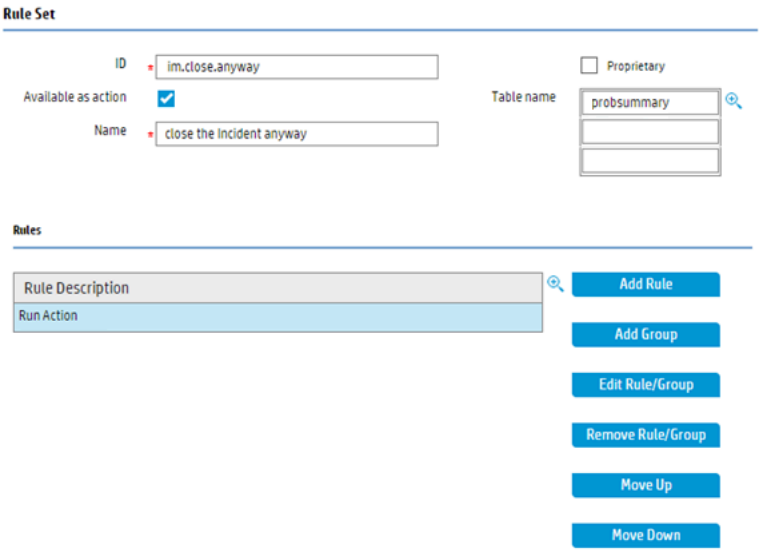
7.1.2.3. Configure Run Action rules on the current record

You can configure the Run Action rules to enable Service Manager to perform automatic operations on the current record. Usually this rule type can be used to easily expose a back end transition call as an action. To expose a back end transition configured in a workflow, you can use JavaScript or RAD calls as well. However, that approach is not quite user-friendly. By using the Run Action rules, you can easily expose the back end transitions. For details about calling back end transitions, see the Workflow back end transitions section in Configure a workflow.

For example, in the out-of-box incident workflow, there is a back end transition “close.any.time” that can move the incident to the Closure phase whatever current phase is. The back end transition is shown as follows:



To be able to invoke the transition, you need to first add a rule set that contains the Run Action rule, and then mark this rule set as **Available as action**, as shown below:



In addition, you also need to configure the Run Action rules in rule set as follows:

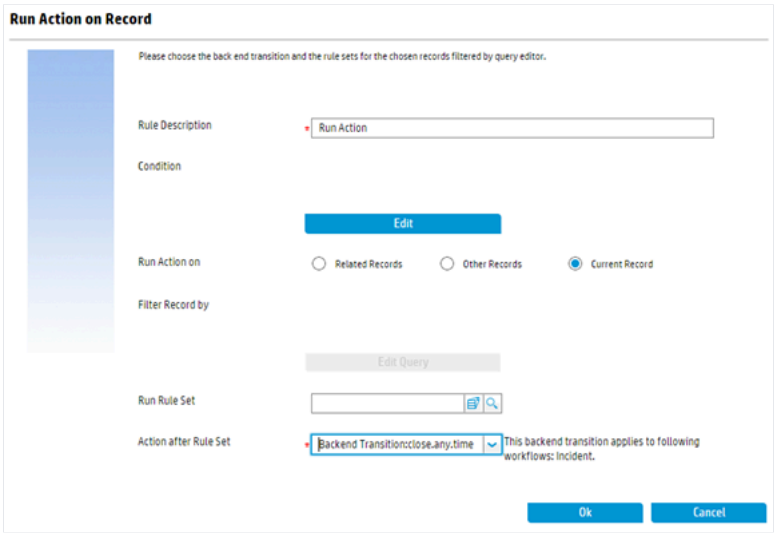
- Select **Current Record** for the "Run Action on" field
- Keep the "Run Rule Set" field empty
- Select **Backend Transition:close.any.time** for the "Action after Rule Set" field. The save action is executed against the current incident record.

Note

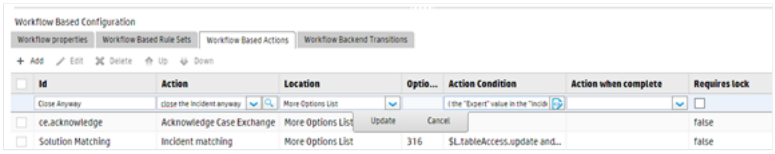
Note: You can select **Do nothing**, **Save**, or one of the back-end transitions.

- The back-end transition list is retrieved from all the workflows of the selected record file. Similar to Configure Run Action rules on related records, at run time, if some of the other records don't have your selected back-end transition, the corresponding transition is not run.
- You are suggested to use the same back-end transition name in different workflows if these back-end transitions follow the same business logic.
- While a back-end transition is selected from the list, the interface is updated to indicate to which workflows the transition applies and to which it doesn't apply.

The form on which you can perform these tasks is shown below:



Upon completing all the above configurations, you can expose the rule set as an action by adding it to the workflow level actions:



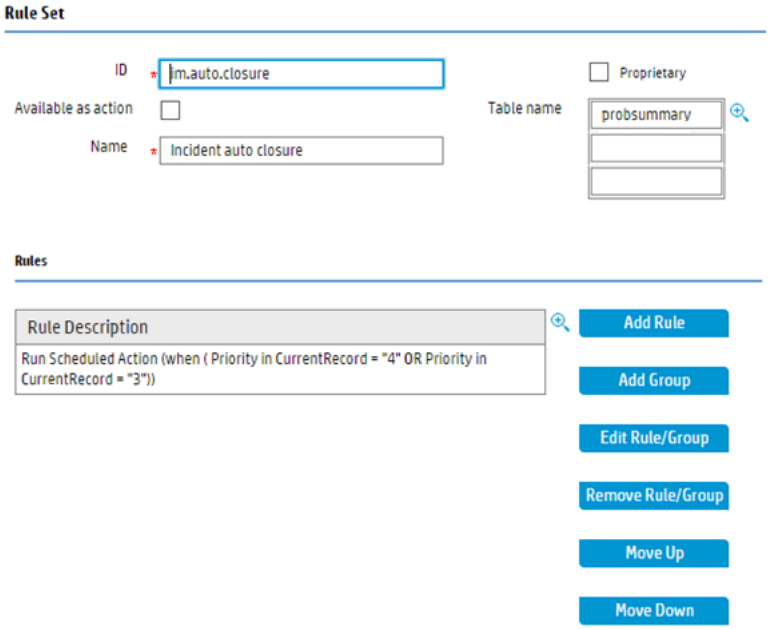
Note

Note: Only expose such actions to specific persons or operations. For example, you can expose it to the person who has the Expert permission on the Incident area by setting the security level with the Action Condition. If you want to make it available only to a Web Service call, set your Action Condition accordingly.

7.1.2.4. Configure Run Scheduled Action rules

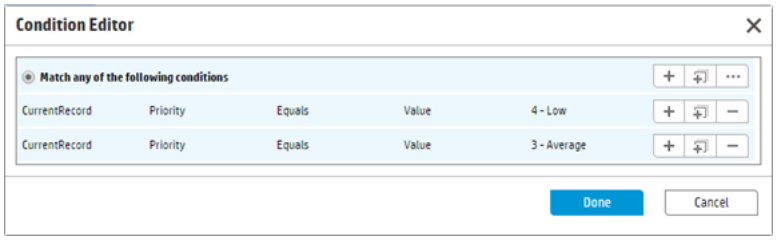
You can configure the Run Scheduled Action rules to enable Service Manager to perform scheduled automatic operations on the current record. This rule can be used in situations like automatic record closure.

For example, you might want the system to automatically close the incidents that were resolved seven days ago. To do that, first define a rule set with the Run Scheduled Action rules. You can define the rule set on the table "probsummary" that stores incident records, as shown below:



In addition to the above configurations, you also need to configure the Run Scheduled Action rules in this rule set as follows:

- Set the conditions as follows. This only takes the low priority incidents into account.

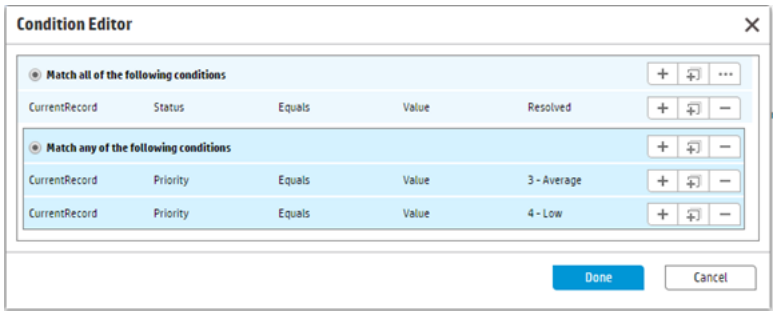


  At run time, only when the condition is matched, this rule is executed to generate a scheduled record with the class of "scheduledAction". Note that this rule type introduces a new back end scheduler process named "Scheduled Action processor", which checks for the schedule records and executes them.
- Select **Use field in record + interval** for the Calculation Type field
- Select **Resolved Time** for the Calc Field field

Note

Note: Make sure the resolve time of the incident is set correctly while the incident is resolved, because the schedule execution time is calculated based on the value of that field.

- Set **7 00:00:00** for the Calc Interval field. This means the scheduled action is executed after seven days.
- Set the Action Condition as follows. The condition is checked when the scheduled time arrives. In this example, it is seven days after the incident is resolved. Only when the status of the incident is still Resolved, and the priority is still low at the time, the action is executed; otherwise the action is ignored.



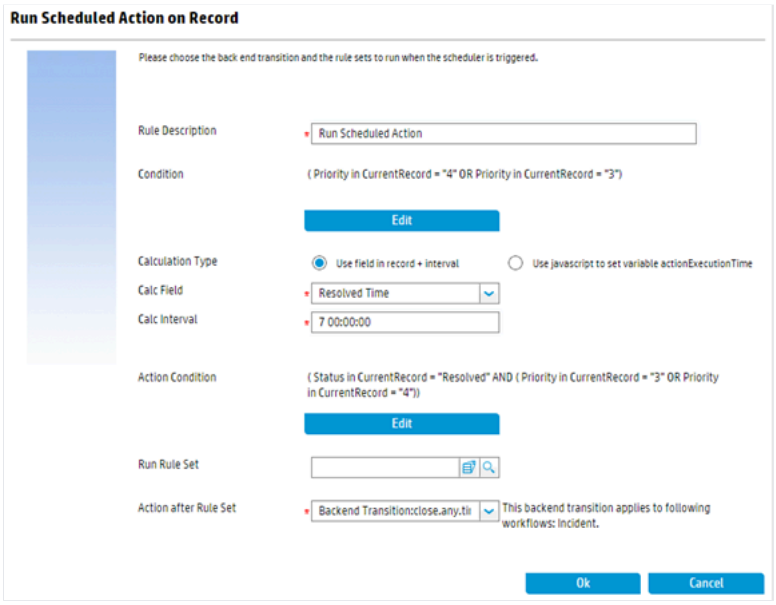
- Keep the Run Rule Set field empty
- Select **Backend Transition:close.any.time** for the "Action after Rule Set" field. The save action is executed against the current record by the back end scheduler process when the scheduled time arrives.

Note

Note: You can select **Do nothing**, **Save**, or one of the back-end transitions.

- The back-end transition list is retrieved from all the workflows of the selected record file. Similar to Configure Run Action rules on related records, at run time, if some of the other records don't have your selected back-end transition, the corresponding transition is not run.
- You are suggested to use the same back-end transition name in different workflows if these back-end transitions follow the same business logic.
- While a back-end transition is selected from the list, the interface is updated to indicate to which workflows the transition applies and to which it doesn't apply.

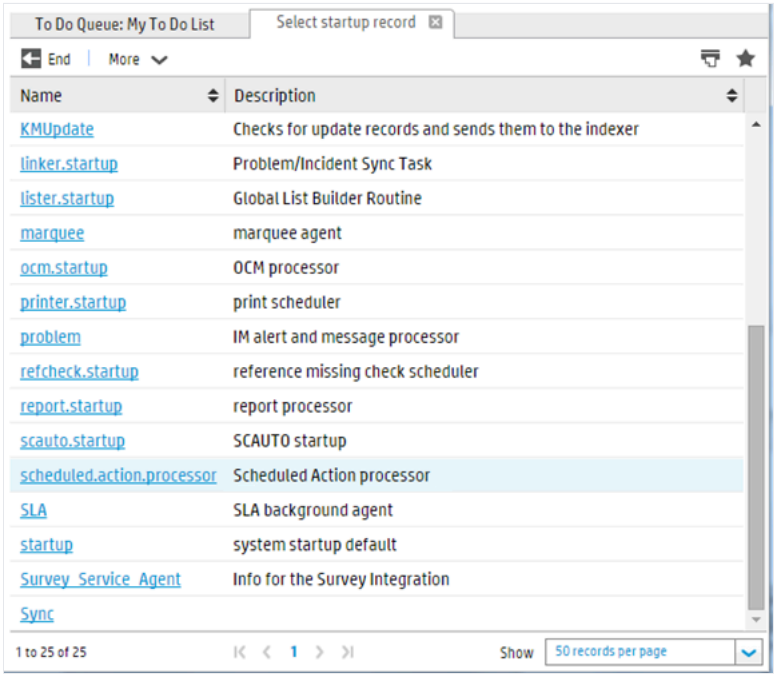
The form on which you can perform these tasks is shown below:



Note

Note: The Run Scheduled Action rule doesn't support selecting work schedule and time zone in this release.

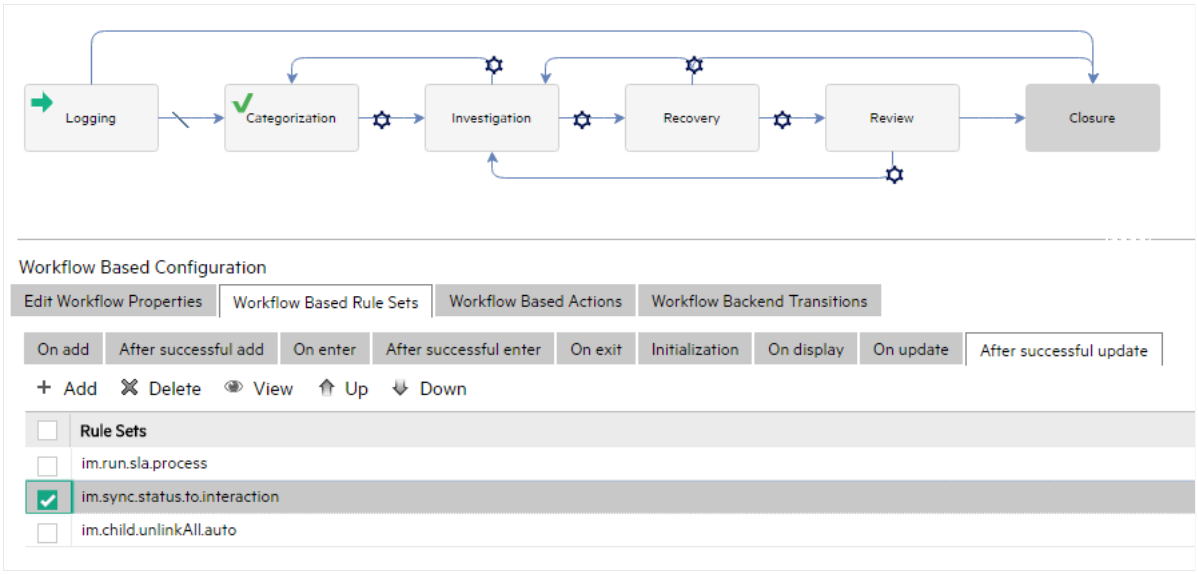
To make the Run Scheduled Action rules work at run time, make sure the back end processor "Scheduled Action processor" is started, as shown below:



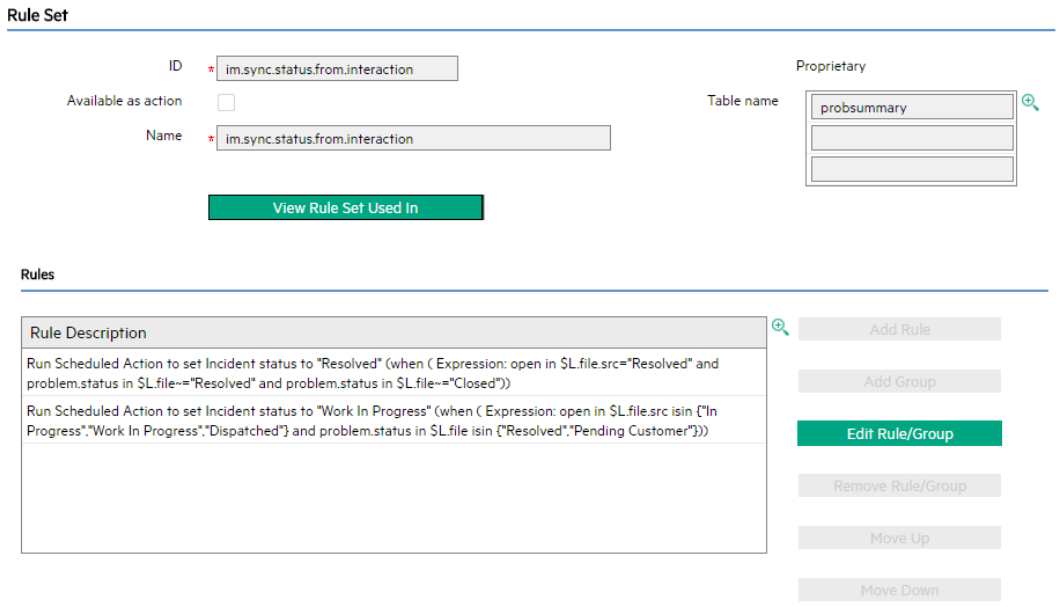
7.1.2.5. Combine the Run Action rules and the Run Scheduled Action rules

In Service Manager, the Run Action rules are executed in a sequential manner. Take the related records as an example, if one of the rules sets or actions against the related records fails, the whole execution stops and returns a failure result. To ensure successful execution, you can combine the Run Action rules and the Run Scheduled Action rules, so that the execution can be scheduled if it fails due to record locking.

In the out-of-box configuration, when the status of an incident changes, the status of the related interaction also changes accordingly. To achieve this, a rule set **im.sync.status.to.interaction** is configured in the **After successful update** event of the Incident workflow, as shown below.



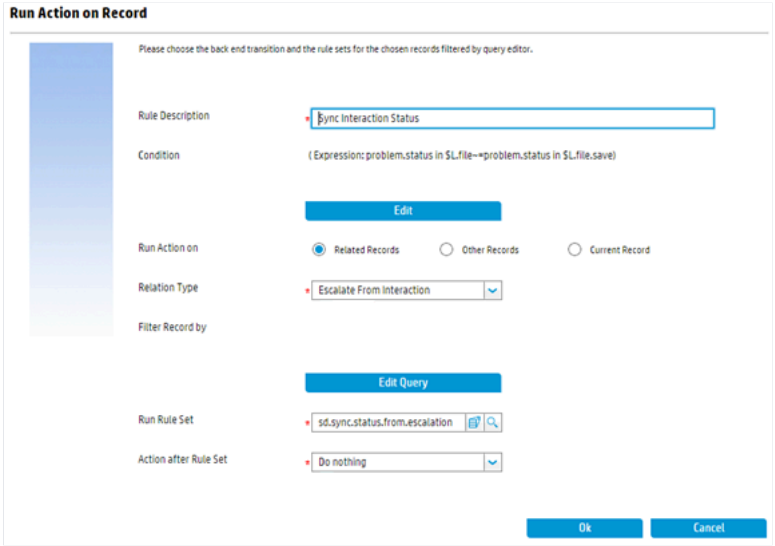
This rule set is configured on the "probsummary" table that stores incident records:



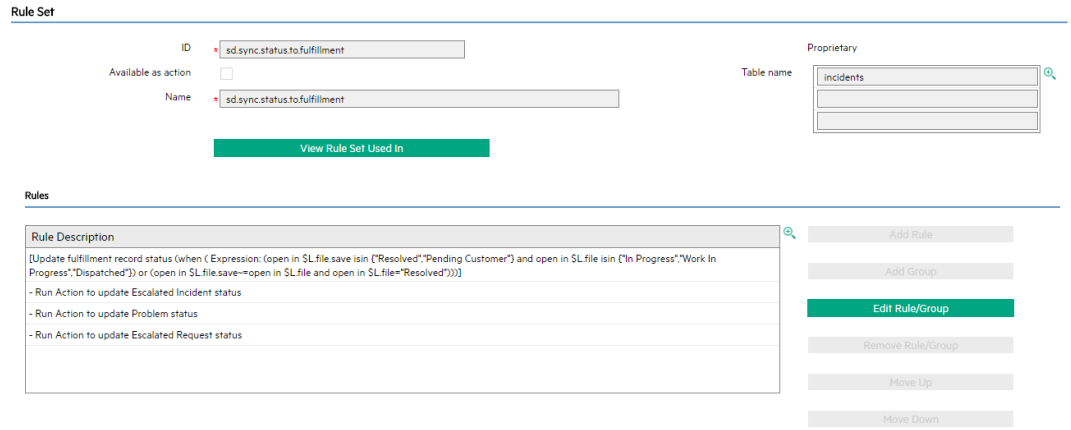
In addition, you also need to configure the Run Action rules in the rule set as follows:

- Name the condition something like "Sync Interaction Status", which makes it easy for you to understand the purpose of the condition
- Select **Related Records** for the "Run Action on" field
- Select **Escalate From Interaction** for the "Relation Type" field. This means the target related record is the interaction from which incident is escalated.
- Select **sd.sync.status.from.escalation** for the "Run Rule Set" field. This rule set is executed against the related interaction. To handle the operation failure on locked interaction, this rule set must contain the Run Scheduled Action rule.
- Select **Do nothing** for the "Action after Rule Set" field. Actions like the save operation on target records are configured in the **sd.sync.status.from.escalation** rule set.

The form on which you can perform these tasks is shown below:



After defining the rule set on the source table, you need to define the rule set on the target table. The rule set on the target table is used as the value of the "Run Rule Set" field when you define the rule set for the source table. In this example, you define the rule set **sd.sync.status.from.escalation** on the target table "incidents" that stores the interaction records:

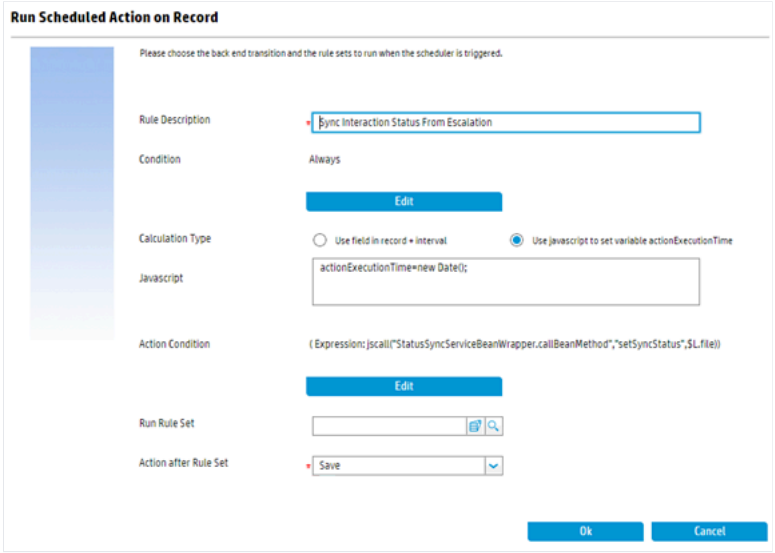


In this rule set, a Scheduled Action rule is configured to sync up the status of the incident and the status of the interaction. The configuration is as follows:

- Specify the scheduled execution time as the current time by using JavaScript. This ensures the schedule is executed as early as possible, that is, it is executed the next time the back end scheduler "Scheduled Action processor" is activated.
- Select **Escalate From Interaction** as the Relation Type. This means the target-related record is the interaction from which the incident is escalated.
- Specify the Action condition to call a JavaScript method `StatusSyncServiceBeanWrapper.callBeanMethod("setSyncStatus",$L.file)` . All the main logic is included in this JavaScript method.
  • If status synchronization is needed and successful, this method returns true, and then the Save action is executed.
  • If status synchronization is not needed, this method returns false, and then the Save action is ignored automatically.
- Keep the Run Rule Set field empty.
- Select **Save** for the "Action after Rule Set" field. While the record is saved, if it is locked, the saving operation fails, and a schedule record is rescheduled instead of being deleted.

Note

Note: Variables, which are referred to as `srcRecord` for the current source record and as `oldSrcRecord` for the original source record in JavaScript, aren't supported if the Run Scheduled Action rules are used.



7.2. Best practices for configuring task planner

Task Planner enables you to schedule tasks in Service Manager modules such as Change Management or Request Management. This section provides information about the best practices for using the Task Planner. For example, you can configure shared information across multiple tasks, or you can use record context to drive workflows.

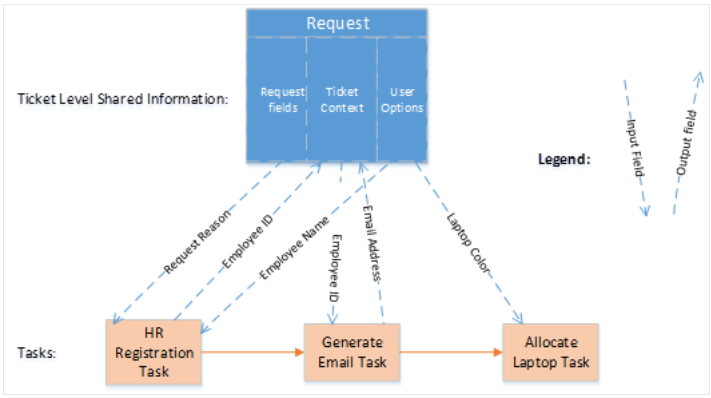
Configure shared information across tasks

You can use Task Planner to configure information shared by multiple tasks.

The following three types of information from a record can be shared by the tasks of a record:

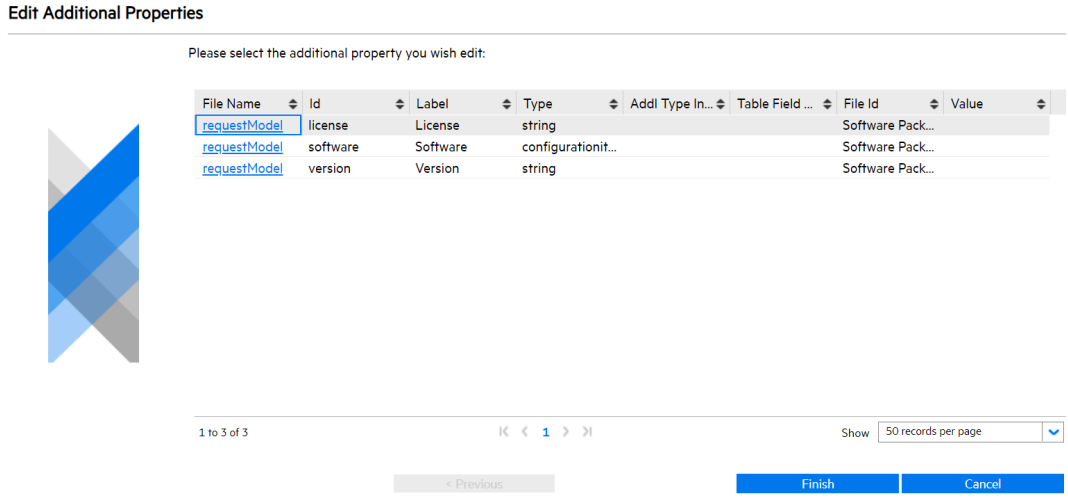
- Record fields, for example, the request fields or the change fields
- User selections, which are input by the end user when they use Service Manager
- Record context, which are additional properties defined in Task Planner. The values of these properties are populated during task implementation and then shared across the tasks of a record.

The following diagram demonstrates how these three types of information of a Request are shared among its tasks.

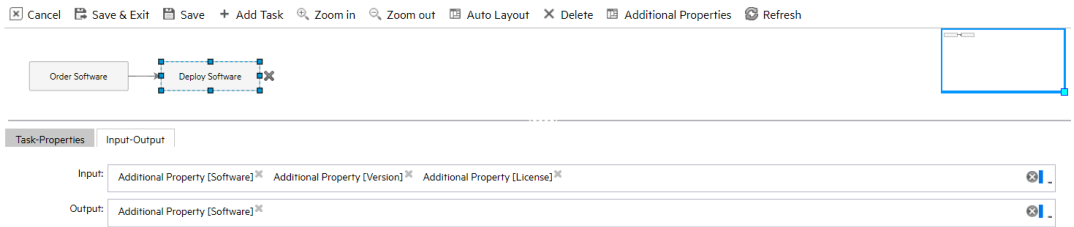


Each task can have its own input and output data. The input fields are used to receive data from other tasks, and can trigger the execution of the current task. The output fields are used to populate shared data to the record during task execution, and the data can then be shared by other tasks.

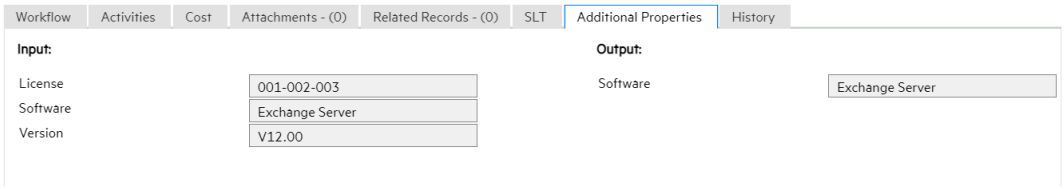
You can launch the Record Context (Ticket Context or Additional Properties) definition page from the Task Planner. The page is shown as follows:



On the definition page of the Task Planner, you can configure the input and output fields. The Record Context fields of a record can be used as the input or output fields of a task, and the output fields can be used in subsequent tasks as the input fields.



The following Task Context page shows the run-time result of a task, which contains the input and output fields/values. The input fields are read-only, while the output fields can be edited with appropriate values that can be populated to the next task later.



Use record context fields in Condition Editor

The Condition Editor does not support using the value of a record context field directly as part of the condition, so you have to manually retrieve the value and put it into a Service Manager variable, and then you can use this variable in Condition Editor.

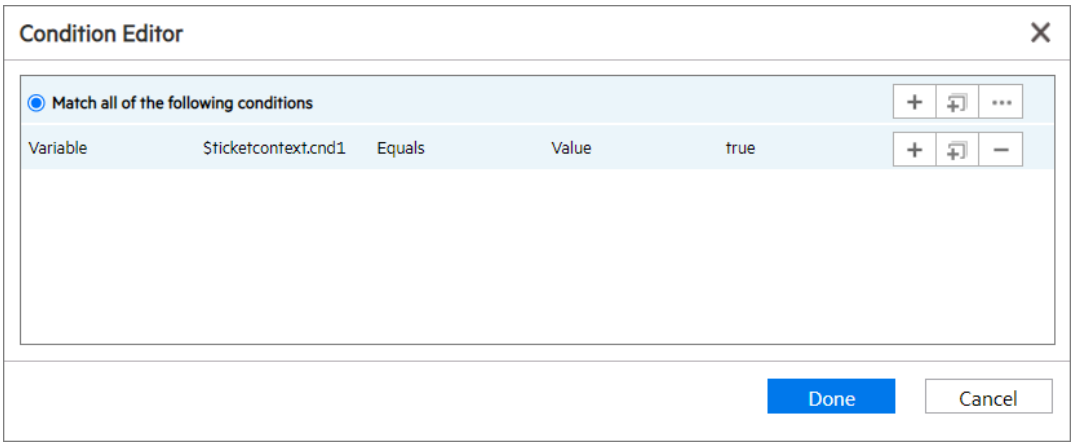
For example, if you want to use the record context field “employeeType” as part of the condition in the Request workflow, you need to execute the following JavaScript code before your condition is evaluated:

```
vars[“$ticketcontext.cnd1”] = lib.c.$("#taskPlannerService").getTicketContextFieldValue("request", record.number,
"employeeType");
```

Or you can use the JavaScript below in the Request Task workflow:

```
vars[“$ticketcontext.cnd1”] = lib.c.$("#taskPlannerService").getTicketContextFieldValue("request",
record.parent_request, "employeeType");
```

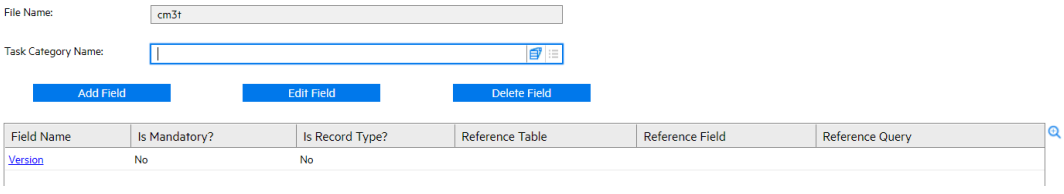
Finally, you can configure the condition to something in the Condition Editor as shown below:



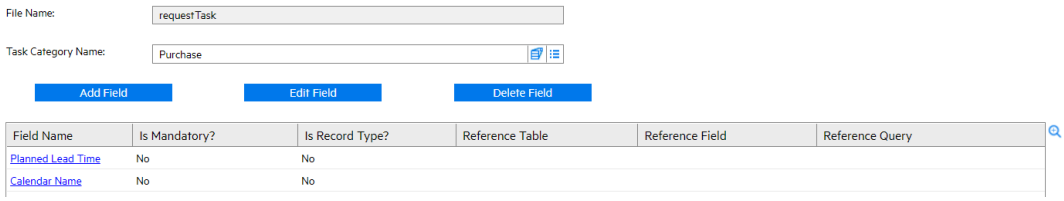
Use predefined additional task fields

When you plan a task in Task Planner, you might want to use fields that are not available in the out-of-box configuration. In this case, you can define new fields for Task Planner at the object level or category level.

For example, in the following screenshot you can see that a field is defined at the object level – if you do not specify any value for the Task Category Name field, the field is defined at the object level, which applies to all category tasks.

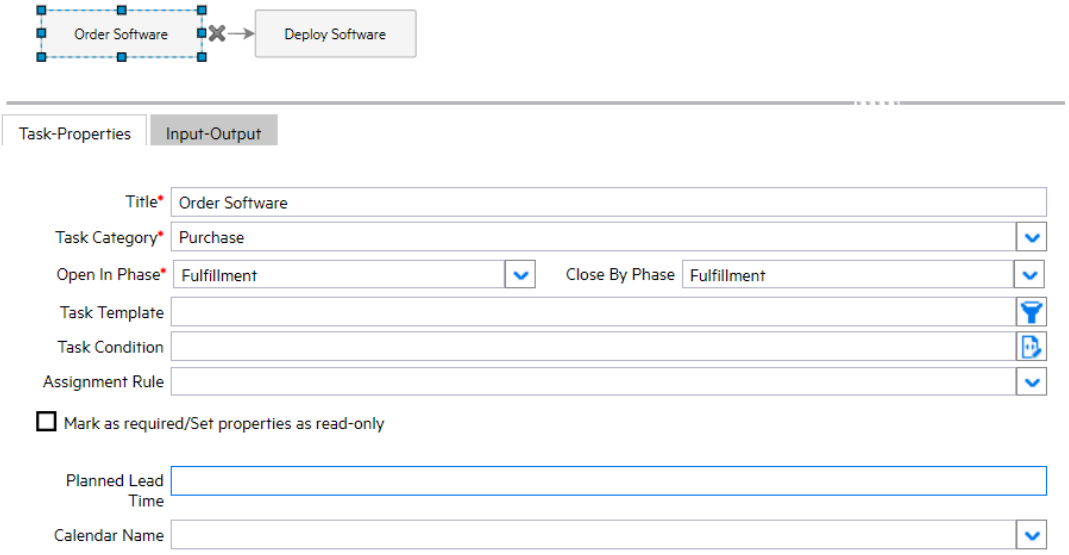


And as shown in the following screenshot, if you specify a value for the Task Category Name field, the field is defined at the category level, which applies only to this task category.



At run time, the predefined fields at both the object level and the category level are merged together and displayed in Task Planner. If a field is defined at both levels, the definition at the category level is used.

After being defined, the predefined additional task fields are displayed at the bottom of the Task-Properties tab. Your entered data is inserted into the task that is created in Task Planner.



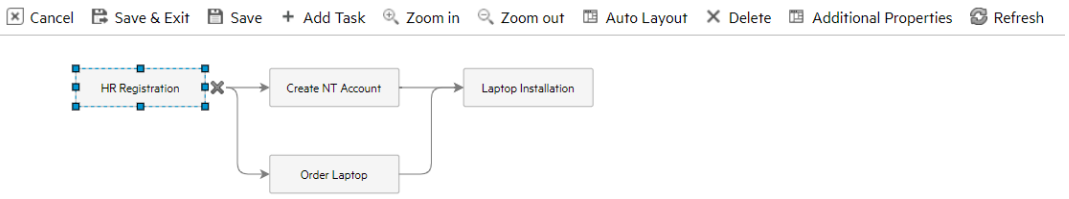
Plan appropriate number of tasks in a same open phase

When you define a task plan, you can specify the "Open in Phase" field of a task, which means this task is created at run time when the workflow of the record moves to the phase. Currently all the tasks you plan with the same "Open in Phase" value are created immediately when the record moves to the corresponding phase.

Too many tasks for a phase may result in decreased performance. For example, if you plan more than 10 tasks, the operator might feel obvious performance downgrade when the record is moved to that phase. To ensure the best performance, it is recommended that no more than 10 tasks are automatically started once a phase is entered.

Note

Note: The performance of generating tasks in Task Planner is not only affected by the number of planned tasks, but also by the business logic you plan for task creation. The more complex and time-consuming the logic is, the more performance downgrade you might encounter.



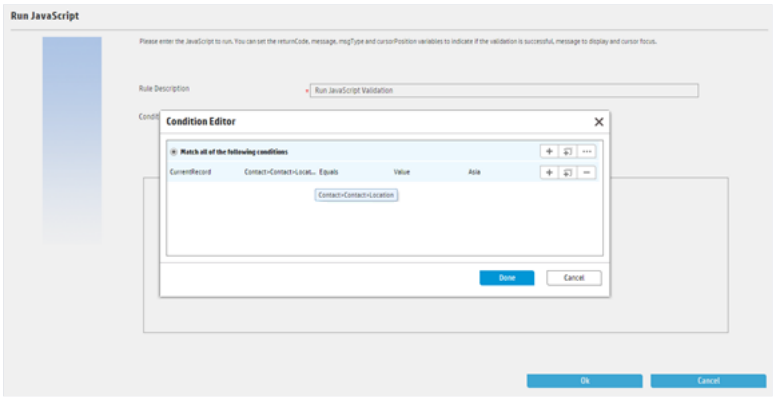
7.3. Best practices for using the condition editor

The Condition Editor enables you to build a condition without any knowledge of programming languages. Conditions always evaluate to True or False. When a condition evaluates to True, the system runs the rule or applies an action that the condition controls. This section provides information about the best practices of using the Condition Editor.

Use cross-table fields in Condition Editor[edit]

Cross-table fields, which are defined by a reference record, are the fields of related tables. The table relationship is maintained by the Relation Manager in Service Manager. You can use cross-table fields in Condition Editor to configure various conditions.

To configure the cross-table fields as part of a condition, you can select the cross-table field names in Condition Editor. However, using cross-table fields might be time-consuming because the system needs to prepare the variable of the reference record at run time. For example, an incident record has the field “Contact”, which refers to the Contact record. You can configure the Rule condition to use the Location of the Contact record directly, as shown below:



At run time, the expression is evaluated to something as follows:

`location in $L.file.contact.name.contacts.contact.name="Asia"`

The variable `$L.file.contact.name.contacts.contact.name` is prepared by the rule engine.

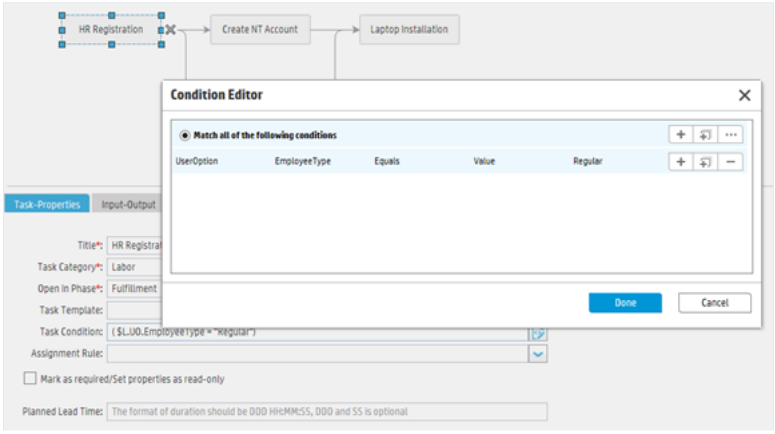
If you use the Condition in your own scenario, be sure to invoke the JavaScript method below at run time before evaluating the condition expression, which automatically prepares the cross-table reference record for you:

`lib.Workflow.initVarForCondition(conditionXml);`

Use user option fields in Condition Editor[edit]

You can configure the User Option fields as part of a condition by specifying the names and values of the user option fields in Condition Editor.

However, using the user option fields might be time-consuming because the system needs to prepare variables for the user option fields at run time. For example, you configure to use the User Option as part of a task condition in Task Planner as follows:



At run time, the expression is evaluated to something as follows:

`$L.UO.EmployeeType="Regular"`

The variable `$L.UO.EmployeeType` is prepared by the system.

If you use the condition in your own scenario, make sure that you invoke the JavaScript method below at run time before evaluating the condition expression, which automatically prepares the user option variable for you:

`lib.Workflow.initVarForCondition(conditionXml);`

Note

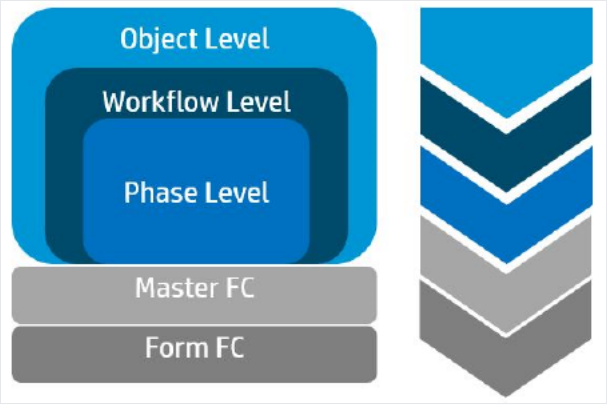
Note: The instance data of a user option is stored as a string value in the file userOption, and thus at run time, when the user option value is evaluated, it is also treated as a string value.

Contents

- 1 Use cross-table fields in Condition Editor
- 2 Use user option fields in Condition Editor

7.4. Consider rule set execution sequence

A rule set contains a list of rules that you may run against a record. Rules implement business logic to drive a workflow or a process. In Service Manager, you can define rule sets at the object level, workflow level, or phase level. The rule sets defined at different levels are executed in order. For example, rule sets defined at the object level are executed first. The following diagram illustrates this execution order (from top to bottom):



For each action in Service Manager, such as creating a record, rule sets defined at different levels might be executed. The following table provides a high level view of the execution order of these rule sets:

Action	Execution order of the rule sets (from left to right)
Create a record	On add, On enter, Format control (add), After successful add, After successful enter, Format control (subroutine after add), Initialization, Format control (initial), On display, Format Control (display)
Update a record	On update, Format control (update), After successful update, Format control (subroutine after update), Initialization, Format control (initial), On display, Format control (display)
Search and access a record	Initialization, Format control (initial), On display, Format control (display)
Fill, find, screen-redraw	On display, Format control (display)
• Move from one phase to another • Close record (inactive)	On exit (global + old phase), Transition, On enter (global + new phase), Format control (update), Format control (subroutine after update), Initialization, Format control (initial), On display, Format control (display) **Note** **Note**: The term "global" means rule sets defined at the object level or the workflow level.

When you define your own rule sets in Service Manager, you might need to consider the execution order of the rule sets, so that you can optimize the definition of your own rule sets accordingly.

The following tables provides a more detailed view of the execution order of the rule sets that are executed for each action. The events and their corresponding rule sets are executed from top to bottom.

Open an existing record

Events	Rule sets
Initialization	• Initialization rule sets at the object level • Initialization rule sets at the workflow level • Initialization rule sets at the phase level • Primary format control initialization • Format control initialization for the form of the phase
Display	• On-display rule sets at the object level • On-display rule sets at the workflow level • On-display rule sets at the phase level • Primary format control display • Format control display for the form of the phase

Create a new record (launched from document.new)

Events	Rule sets
Initialization	• Initialization rule sets at the object level • Initialization rule sets at the workflow level • Initialization rule sets at the phase level • Primary format control initialization • Format control initialization for the form of the phase
Display	• On-display rule sets at the object level • On-display rule sets at the workflow level • On-display rule sets at the phase level • Primary format control display • Format control display for the form of the phase

Add a record and stay in the logging phase

Events	Rule sets
On-add	• On-add rule sets at the object level • On-add rule sets at the workflow level • On-enter rule sets at the object level • On-enter rule sets at the workflow level • On-enter rule sets at the phase level • Primary format control on-add • Format control on-add for the form of the phase
Add the record to the database **Note:** This is a user action, not an event.	N/A
After-add	• After-successful-add rule sets at the object level • After-successful-add rule sets at the workflow level • After-successful-enter rule sets at the object level • After-successful-enter rule sets at the workflow level • After-successful-enter rule sets at the phase level • Primary format control subroutine after-add • Format control subroutine after-add for the form of the phase
Initialization	• Initialization rule sets at the object level • Initialization rule sets at the workflow level • Initialization rule sets at the phase level • Primary format control initialization • Format control initialization for the form of the phase
Display	• On-display rule sets at the object level • On-display rule sets at the workflow level • On-display rule sets at the phase level • Primary format control display • Format control display for the form of the phase

Add a record and automatically move to next phase

Events	Rule sets
Old phase on-add	• On-add rule sets at the object level • On-add rule sets at the workflow level • On-enter rule sets at the object level • On-enter rule sets at the workflow level • On-enter rule sets at the phase level • Primary format control on-add • Format control on-add for the form of the phase
To phase on-enter	• On-exit rule sets at the object level • On-exit rule sets at the workflow level • On-exit rule sets in the "from phase" • Transition rule sets • On-enter rule sets at the object level • On-enter rule sets at the workflow level • On-enter rule sets in the "to phase" **Note** Note In this event, if you need to use the current phase of the record in the rule sets or format control, the value of the current.phase field in the "current record" is the name of the "to phase," and the value of the current.phase field in the "saved record" is the name of the "from phase."
Add the record to the database **Note:** This is a user action, not an event.	N/A
From phase after-add	• After-successful-add rule sets at the object level • After-successful-add rule sets at the workflow level • After-successful-enter rule sets at the object level • After-successful-enter rule sets at the workflow level • After-successful-enter rule sets at the phase level • Primary format control subroutine after-add • Phase form's format control subroutine after-add
To phase post	• After-successful-enter rule sets at the object level • After-successful-enter rule sets at the workflow level • After-successful-enter rule sets at the phase level • Primary format control subroutine after- add • Phase form's format control subroutine after-add
To phase initialization	• Initialization rule sets at the object level • Initialization rule sets at the workflow level • Initialization rule sets at the phase level • Primary format control initialization • Format control initialization to the form of the phase

To phase display	• On-display rule sets at the object level • On-display rule sets at the workflow level • On-display rule sets at the phase level • Primary format control display • Format control display to the form of the phase

Save a record and stay in current phase

Events	Rule sets
On-update	• On-update rule sets at the object level • On-update rule sets at the workflow level • On-update rule sets at the phase level • Primary format control on-update • Format control on-update for the form of the phase
Save the record to the database **Note:** This is a user action, not an event.	N/A
After-update	• After-successful-update rule sets at the object level • After-successful- update rule sets at the workflow level • After-successful-update rule sets at the phase level • Primary format control subroutine on-update • Format control subroutine on-update for the form of the phase
Initialization	• Initialization rule sets at the object level • Initialization rule sets at the workflow level • Initialization rule sets at the phase level • Primary format control initialization • Format control initialization for the form of the phase
Display	• On-display rule sets at the object level • On-display rule sets at the workflow level • On-display rule sets at the phase level • Primary format control display • Format control display for the form of the phase

Save a record and automatically move to the next phase

Events	Rule sets
From phase on-update	• On-update rule sets at the object level • On-update rule sets at the workflow level • On-update rule sets at the phase level • Primary format control on-update • Format control on-update for the form of the phase
To phase on-enter	• On-exit rule sets at the object level • On-exit rule sets at the workflow level • On-exit rule sets in the "from phase" • Transition rule sets • On-enter rule sets at the object level • On-enter rule sets at the workflow level • On-enter rule sets in the "to phase" **Note** Note In this event, if you need to use the current phase of the record in the rule sets or format control, the value of the current.phase field in the "current record" is the name of the"to phase", and the value of the current.phase field in the "saved record" is the name of the "from phase".
Update the record to the database **Note:** This is a user action, not an event.	N/A
From phase after-update	• After-successful-update rule sets at the object level • After-successful-update rule sets at the workflow level • After-successful-update rule sets at the phase level • Primary format control subroutine after-update • Format control subroutine after-update for the form of the phase
To phase post	• After-successful-enter rule sets at the object level • After-successful-enter rule sets at the workflow level • After-successful-enter rule sets in the "to phase" • Primary format control subroutine after-update • Format control subroutine after-update from the form of the phase
To phase initialization	• Initialization rule sets at the object level • Initialization rule sets at the workflow level • Initialization rule sets at the phase level • Primary format control initialization • Format control initialization to the form of the phase

To phase display	• On-display rule sets at the object level • On-display rule sets at the workflow level • On-display rule sets at the phase level • Primary format control display • Format control display to the form of the phase

Manual transition without selecting "Save record prior to executing the transition"

Events	Rule sets
To phase on-enter	• On-exit rule sets at the object level • On-exit rule sets at the workflow level • On-exit rule sets in the "from phase" • Transition rule sets • On-enter rule sets at the object level • On-enter rule sets at the workflow level • On-enter rule sets in the "to phase" • Primary format control On-update • Format control on-update from the form of the phase  **Note** Note In this event, if you need to use the current phase of the record in the rule sets or format control, the value of the current.phase field in the "current record" is the name of the"to phase", and the value of the current.phase field in the "saved record" is the name of the "from phase".
Save the record to the database **Note:** This is a user action, not an event.	N/A
To phase after-enter	• After-successful-enter rule sets at the object level • After-successful-enter rule sets at the workflow level • After-successful-enter rule sets in the "to phase" • Primary format control subroutine after-update • Format control subroutine after-update from the form of the phase

Manual transition with "Save record prior executing the transition" selected

Events	Rule sets
From phase on-update	• On-update rule sets at the object level • Workflow level on-update rule sets • On-update rule sets in the "from phase" • Primary format control On-update • Format control on-update from the form of the phase
Save the record to the database **Note:** This is a user action, not an event.	N/A
From phase after-update	• After-successful-update rule sets at the object level • After-successful-update rule sets at the workflow level • After-successful-update rule sets in the "from phase" • Primary format control subroutine after-update • Format control subroutine after-update from the form of the phase
To phase on-enter	• On-exit rule sets at the object level • On-exit rule sets at the workflow level • On-exit rule sets in the "from phase" • Transition rule sets • On-enter rule sets at the object level • On-enter rule sets at the workflow level • On-enter rule sets in the "to phase" • Primary format control on-update • Format control on-update from the form of the phase  **Note** Note In this event, if you need to use the current phase of the record in the rule sets or format control, the value of the current.phase field in the "current record" is the name of the"to phase", and the value of the current.phase field in the "saved record" is the name of the "from phase".
Save the record to the database **Note:** This is a user action, not an event.	N/A
To phase after-enter	• After-successful-enter rule sets at the object level • After-successful-enter rule sets at the workflow level • After-successful-enter rule sets in the "to phase" • Primary format control subroutine after-update • Format control subroutine after-update from the form of the phase

Back end transition

Events	Rule sets
From phase initialization	• Initialization rule sets at the object level • Initialization rule sets at the workflow level • Initialization rule sets in the "from phase" • Primary format control initialization • Format control initialization from the form of the phase
To phase on-enter	• On-exit rule sets at the object level • On-exit rule sets at the workflow level • On-exit rule sets in the "from phase" • Transition rule sets • On-enter rule sets at the object level • On-enter rule sets at the workflow level • On-enter rule sets in the "to phase" • Primary format control On-update • Format control on-update from the form of the phase
Save the record to the database **Note:** This is a user action, not an event.	N/A
To phase after-enter	• After-successful-enter rule sets at the object level • After-successful-enter rule sets at the workflow level • After-successful-enter rule sets in the "to phase" • Primary format control subroutine after-update • Format control subroutine after-update from the form of the phase

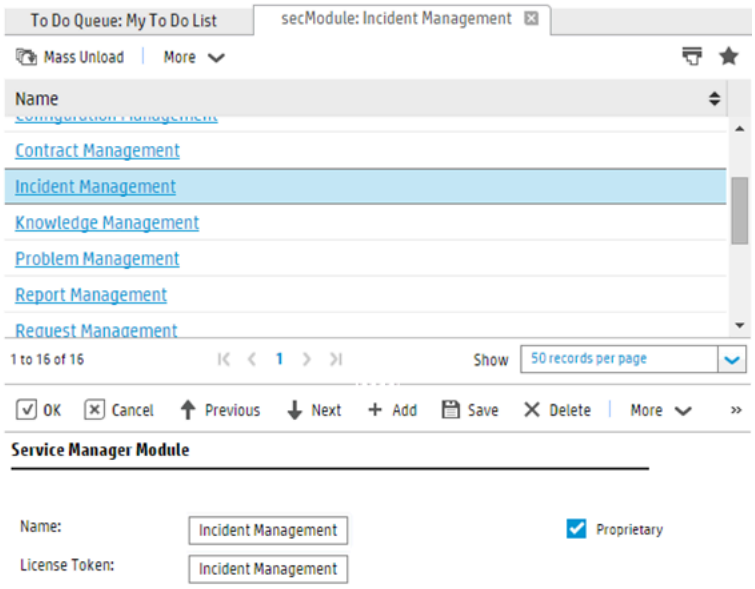
7.5. Best practices for tailoring Service Manager Codeless module

This chapter describes the best practices and recommendations for creating Service Manager Codeless module from scratch or for tailoring an out-of-box Service Manager Codeless module.

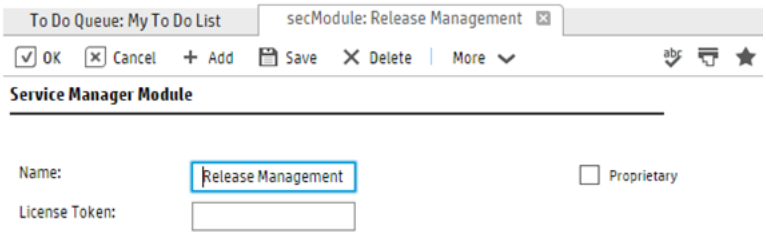
7.5.1. Create a Security Module

Note: If you are going to use an existing Service Manager module rather than adding a new module, ignore this section.

Service Manager doesn't include a menu entry for the Security module. To access the Security module, run the **db** command in Service Manager, and then select the `secModule` table to access the Security module.



If you want to add a new module to Service Manager, you must first add a new Security module. Usually, each Service Manager module maps to a Security module. For example, if you want to add a new Service Manager Release Management module, you must also add a Release Management security module.



7.5.2. Configure Security Areas for the module

A security area defines a specific functional area within Service Manager, such as Incident, Incident Task, or Incident Management Configuration. Each security area definition includes default rights that are copied to the role whenever a new role is created.

Usually, you must create the following Security Areas for Service Manager Codeless module:

- A Security Area that contains the default security rights and settings for tickets
- A Security Area that contains the default security rights and settings for the module's Management configuration. For example, the security rights for maintaining the Categories, Approval Definitions, and so on
- A Security Area that contains the default security rights and settings for request tasks (only if your Service Manager Codeless module includes tasks)

The default rights and settings in the Security Area are copied to new roles that are created for this area. However, the values of specific settings are inherited only if no value is specified for those settings in the role.

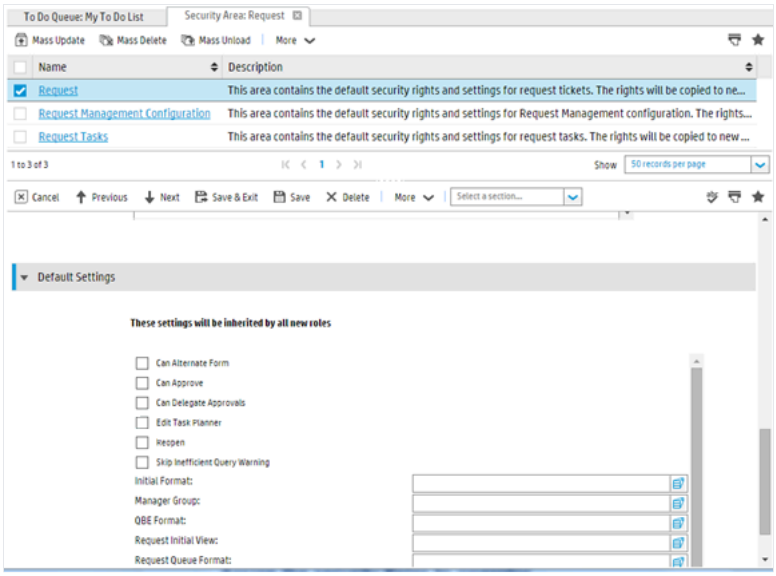
For example, an out-of-box Request module contains the following Security Areas:

- Request
- Request Management Configuration
- Request Tasks

Configure the Security Area settings[edit]

If you need to configure additional security rights in addition to the default rights (such as View, New, Update, Delete/Close, and Modify Template) to control a Security Area, you can modify the Security Area settings.

For example, out-of-box Request modules include a security control that determines whether users have the right to edit the task planner for a record. You can add settings such as this to the Request Area.



Contents

- 1 Configure the Security Area settings

7.5.3. Configure the Security Roles

Security Roles are groups of rights in Security Areas.

> **Note**
>
> Note Security Roles replace the profiles in Service Manager Classic modules.

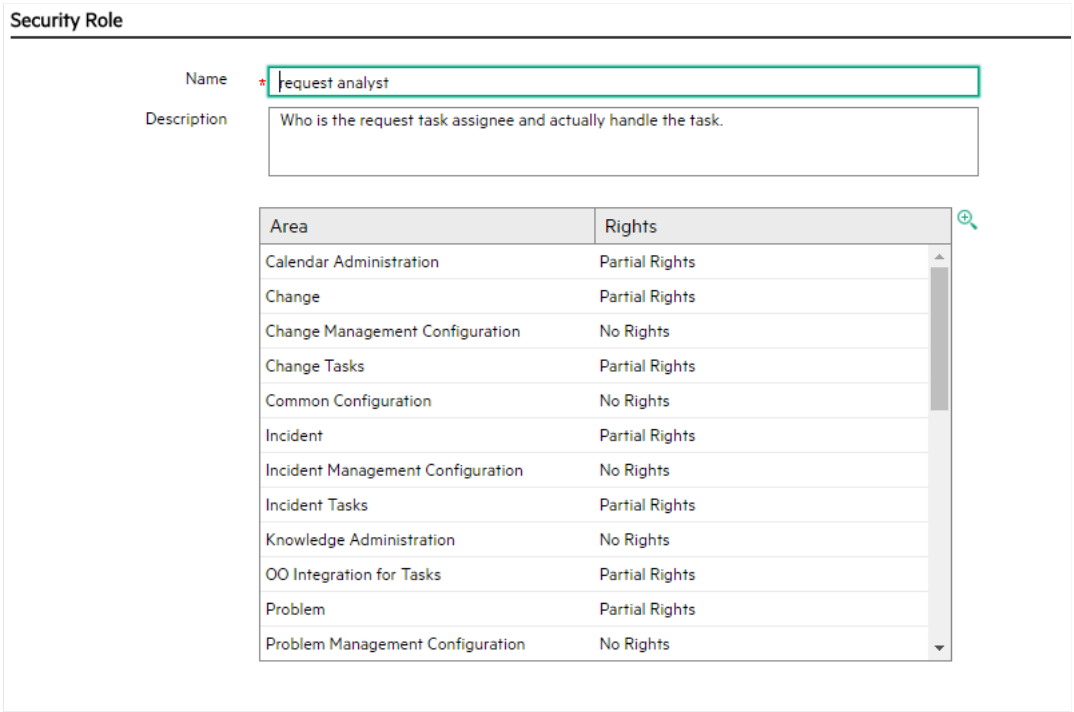
Out-of-box Request modules include the following Security Roles:

- Request Analyst
- Request Approver
- Request Coordinator
- Request Manager
- Request Process Owner
- Requestor

7.5.4. Assign Security Roles to an operator

To grant an operator the rights associated with a specific role, you must assign the Security Role to the operator.

The following screenshot demonstrates the Request module Security Roles that are assigned to Request Analysts in an out-of-box Request module.



7.5.5. Configure the dbdict and the data policy

Process designer framework requires the following fields in dbdict and datadict.

Name	Type	Comments
category	character	If your workflow is based on a category, this field stores the category of the record. Ignore this field if your workflow is based on an object. This field is automatically filled by the Process Designer framework in the following situations: • A record is created by the `document.new` RAD script. • A record's category is changed by the `document.chgCat` RAD script.
current.phase	character	This field indicates the current workflow phase of the record. This field is automatically filled by the Process Designer framework when the workflow phase changes.
record.active	logic	This field indicates whether this record is active or not. Usually, a record is set to inactive after it is closed, canceled, or withdrawn. This field is automatically filled by the Process Designer framework based on the value of the "Records in this phase are active" phase property on the target phase when the workflow phase changes. For more information about the "Records in this phase are active" workflow phase property, see the TODO, ...” section.

Note

Note If you are already using other field names in your dbdict to achieve the same purposes, add alias fields to them with the above field names.

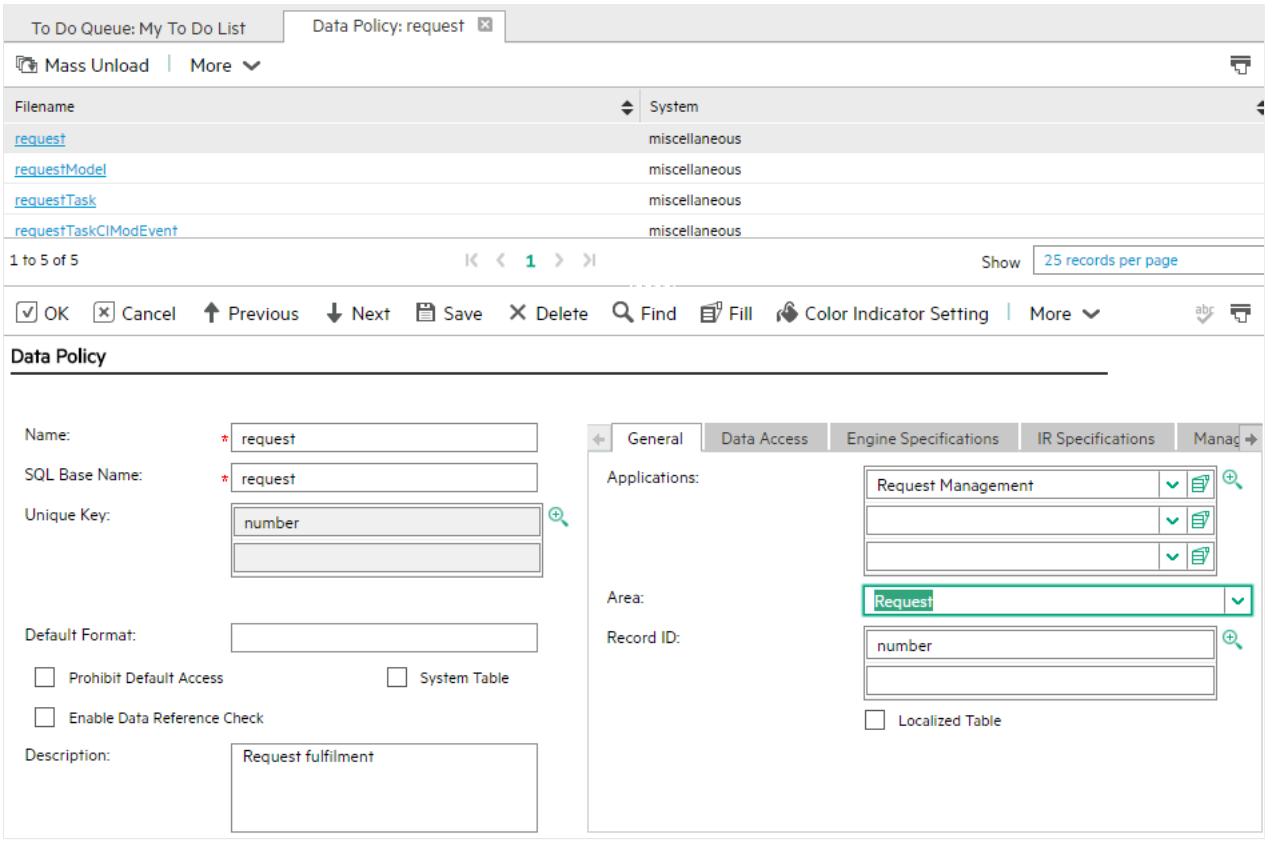
In the data policy, you must set the Area for the current file. This enables you to use the following variables to check whether a user can access a record. The variables are calculated based on the new, view, update, and delete folder settings from this Area, according to the Security Roles assigned to the current user:

- $L.tableAccess.new
- $L.tableAccess.view
- $L.tableAccess.update
- $L.tableAccess.delete
- $L.tableAccess.expert
- $L.tableAccess.admin

For example, out-of-box Request modules include the following mapping between files and Security Areas:

- Request (the Request record table) maps to the "Request" Security Area

As soon as the area is set, every access against this object is controlled by the security settings of that Security Area.



7.5.6. Configure a Process Designer object

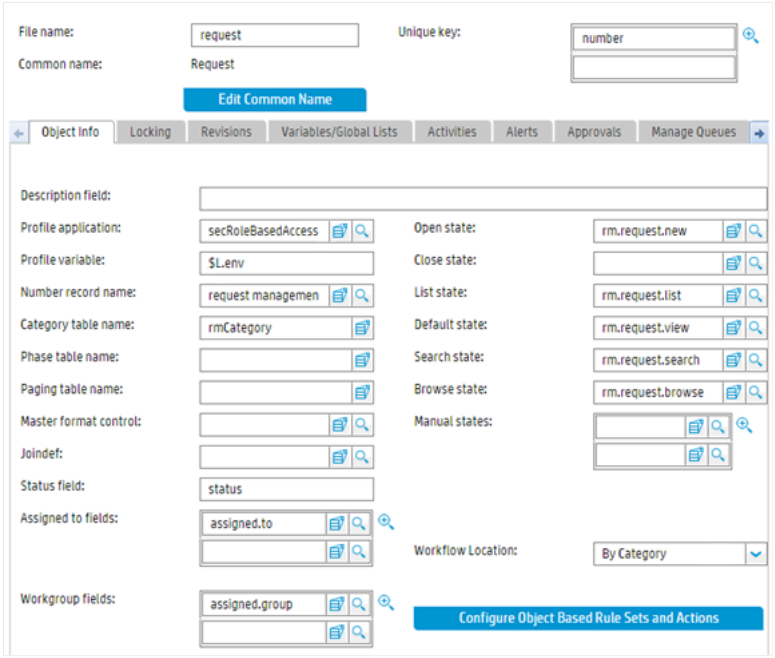
To enable Document Engine objects to use Process Designer, you must configure the following fields:

- **Profile application**
  Set this field to **secRoleBasedAccess** to replace the profile-based security application with the new role-based security application.
- **Profile variable**
  Set this field to **$L.env**.
- **Category table name**
  Specify the category table name if your module uses this concept.
- **Phase table name**
  If you store additional phase information (other than the Workflow Phase) in a table, specify the table name. Otherwise, leave this field empty.
- **Primary format control**
  Leave this field empty if you don't use primary format control anymore.

  Note
  Note Although format control is still supported with the Process Designer framework, we recommend that you convert all business logic into rule sets instead of format control.

- **Workflow Location**
  An object in Service Manager Codeless module is always associated with a workflow or workflows. Select one of the following options to configure the workflow location:
  - By Object (if the object has only one workflow)
  - By Category (if the object has several workflows, and each category is associated with a workflow)

  Note
  Note If you use categories, your category table must contain a field called "workflow" in the dbdict. If the workflow is stored in another field in this category table, the workflow will not work.



Object-based rule sets and actions

Object-based rules and actions help to reduce duplicated definitions. Instead of defining them for each workflow, you can define them at the object level, and they will apply to all workflows that are available for an object.

For more information, see Configure rule sets and actions.

7.5.7. Configure a workflow

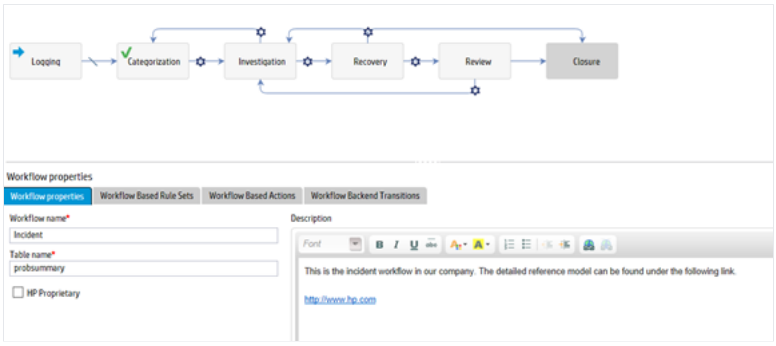
To create a new workflow, you can create one from scratch or copy an existing workflow.

If you copy an existing workflow, you must enter the information that's described in the following table.

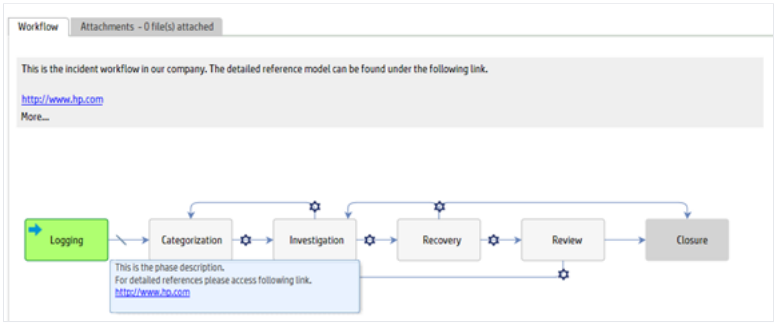
Field	Description
New workflow name	The name of the new workflow
Copy rule sets?	Determines whether to copy the rule set that is used by the workflow. Out-of-box rule sets are marked as **Proprietary** and are read-only. However, copied rule sets are editable. • Select this option if you want to customize rule sets. • Do not select this option if you want to use the out-of-box logic. **Note** Note We recommend that you use the out-of-box rule sets as far as possible, as this facilitates the upgrade process and enables you to benefit from any future enhancements to out-of-box rule sets.
Rule set prefix	If you copy the rule sets, you must specify a prefix for the copied rule sets. The format new rule set name is *<prefix>.<the original rule set name>*

7.5.7.1. Workflow Properties

We recommend that you add a description of the workflow in the Workflow properties tab of the workflow editor, as this provides guidance to operators when they work on records that follow the workflow.



When a description is entered into this field, that description is displayed when operators hover the mouse over the workflow in the workflow viewer of a record.



7.5.7.2. Workflow-based rule sets and actions

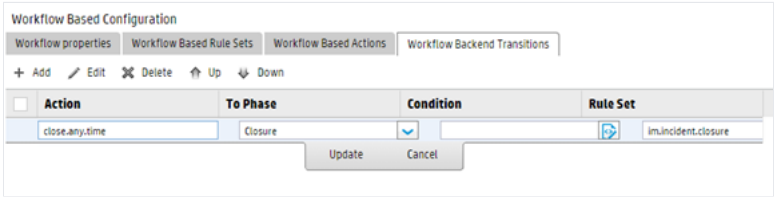
Workflow-based rules and actions help to reduce duplicated definitions. Instead of defining them for each workflow phase, you can define them at the workflow level, and they will apply to all workflow phases.

For more information, see Configure rule sets and actions.

7.5.7.3. Workflow backend transitions

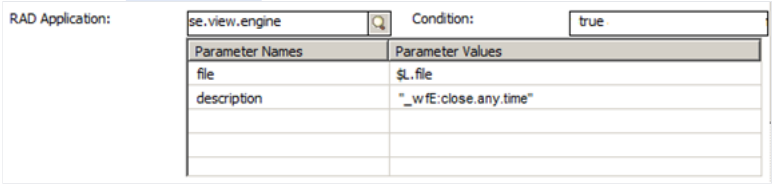
Workflow phases are connected by transitions to move from one phase to another phase, however if you want to move to one phase from whatever current phase is, you can use backend transition to achieve it.

For example, the Incident workflow in out-of-box systems includes a backend transition that moves an Incident record to the Closure phase.



You can invoke backend transitions by using the following methods:

- By using the Run Action rule (for more information, see Perform automatic operations with Run Action rules)
- By using the Run Scheduled Action rule (for more information, see Configure Run Scheduled Action rules)
- By using the `se.view.engine`, RAD application, as demonstrated by the following image.



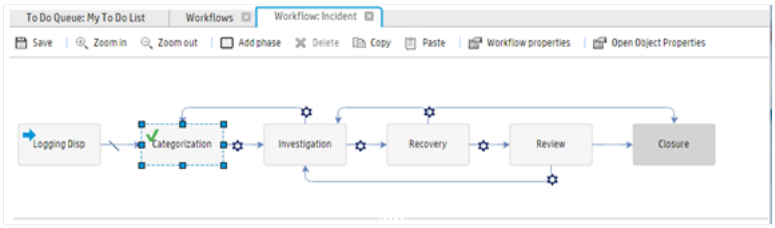
- By using a JavaScript, as demonstrated by the following image.

```
// with PD(Incident), do backend WF transition close
rc = this.scFile.doAction("_wfE:close.any.time");
```

7.5.7.4. Configure workflow phases

Workflow phases show the state of a record in the workflow and enforce the business logic that must occur in order for the record to move to the next phase. Transitions are used to move from one phase to another phase.

The workflow editor graphical interface enables you to add a phase to an existing workflow. You can add a phase and configure its properties from scratch, or you can clone an existing phase if you want to create a phase that has similar information (such as same form name, rule sets, or other attributes) to an existing phase.



Phase properties

When you create or configure a phase, you must configure the following properties.

Field	Description
Phase Order	The Phase Order field provides each phase with a numerical order within a workflow. These numbers are used in calculations for Response Service Level Targets (SLTs) and similar metrics, so that the number and timestamps of entries and exits from specific phases can be tracked. For example, a Process Target calculation can determine the time of entry to the final phase, and therefore determine whether a breach has occurred. As a best practice, you should specify your starting phase as 1, and your closing phase as the highest number. We also recommend that these numbers should be roughly sequential from phase to phase. However, some workflows may loop multiple times through a sequence or take divergent paths.
Name	The name of the phase. You can't modify it after you save current workflow.
Display name	The display name of the phase. **Note** Note When you create or update a phase's display name, the display name is also updated in the Description field of the corresponding record in the cm3rcatphase table. However, only one entry for the phase is stored in the cm3rcatphase table. Therefore, if you assign the same phase different display names in different workflows, only the display name that was modified last is recorded in the cm3rcatphase table. Similarly, only one entry for the phase is stored in the globallist of localized phase names. Therefore, if you assign the same phase different localized names in different workflows, only one localized phase name string is recorded in the globallist. In this situation, functionality that uses the globallist (for example, search forms) will not work as expected.
Table name	The selected table name during workflow creation. You can't modify it.
Form Edit condition	If the condition evaluates to true for a user, that user can edit the form. If it doesn't, the form is read-only.
Records in this phase are active	Select the checkbox if you want the records in this phase to be active. The "record.active" record field is set to "true" when the record is moved to this phase. Usually, this option is not selected for end phases (such as "Closure," "Cancel," or "Withdraw," and is selected for all other phases.
Make this the first phase	Select this option if you want this phase to be the first phase. Usually you set the "Logging" phase as the first phase.

Make this the default phase	Select this option if you want this phase to be the default phase. **Note** Note If the current phase of a record is set to a phase that doesn't exist in the current workflow, it will be moved to the default phase. This may occur if a phase is removed from a workflow or if data is imported from another source that did not share the same workflow. Or, if you change the category of a record, this record will be moved to the default phase.
Additional Phase Information	Select this option to open the Extended Phase Information page to modify the phase information. **Note** Note Only the Change Management module supports this feature. You can't edit or delete a phase name from the Extended Phase Information page or the cm3rcatphase.main form. Change Management workflows have unique workflow phases but they will share change phases if the workflow phases have the same name. For example, if Workflow 1 and Workflow 2 each have a phase named "Build and Test," they will share the same change phase record.

Phase-based rule sets and actions

Phase-based rules and actions apply only to the current phase. For more information, see

Configure rule sets and actions.

Alerts

You can use alerts to configure phases, however we recommend that you use the Run Scheduled Action rule instead.

For more information about the Run Scheduled Action rule, see Configure Run Scheduled Action rules.

7.5.7.5. Configure transitions between workflow phases

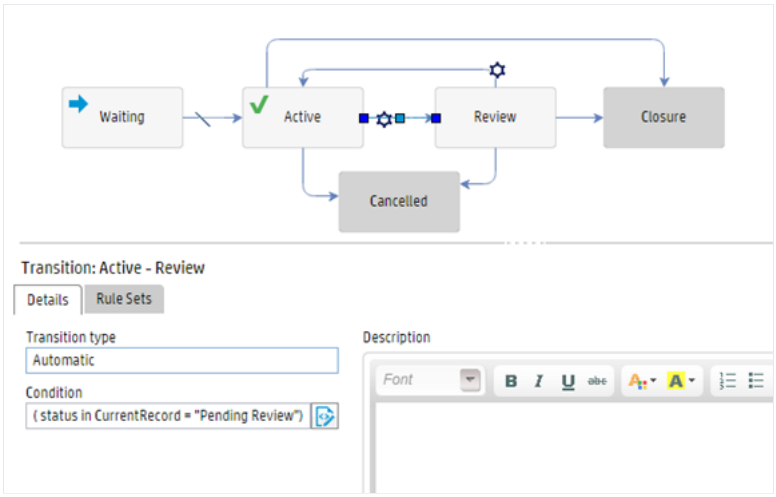
Process Designer workflow transitions occur when a record moves from one phase to another phase. Transitions can happen manually, automatically, or by default.

Automatic transitions[edit]

An automatic transition moves the workflow to another phase based on data in the workflow record. The transition occurs when the configured condition is met.

Usually, if your workflow is status driven, you can use automatic transitions and configure the status as the condition of this transition.

In the following example, when the record is in the "Active" phase and its status is set to "Pending Review," the record moves automatically to the "Review" phase when the record is saved.



We recommend that you add descriptions to automatic transitions. These are displayed to the operator to help guide their work.

Manual transitions[edit]

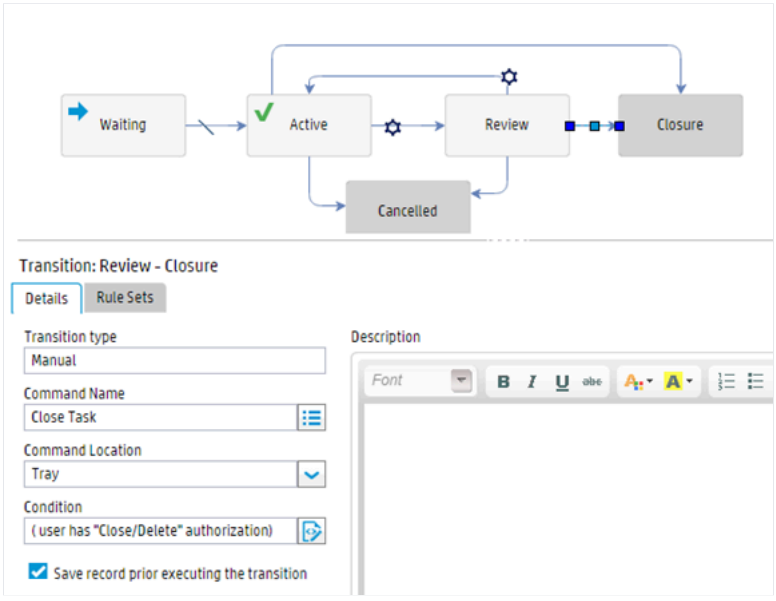
A manual transition requires the operator to perform an action to move a record from one phase to another. This type of transition, in which an operator must press a button or otherwise trigger an action, is a manual transition.

Usually, if your workflow is actions driven, you can use manual transitions so that operators move the workflow phase manually. You can also use manual transitions in a status-driven workflow if you still want to move to a specific phase manually.

We recommend that you add descriptions to manual transitions. These are displayed to the operator to help guide their work.

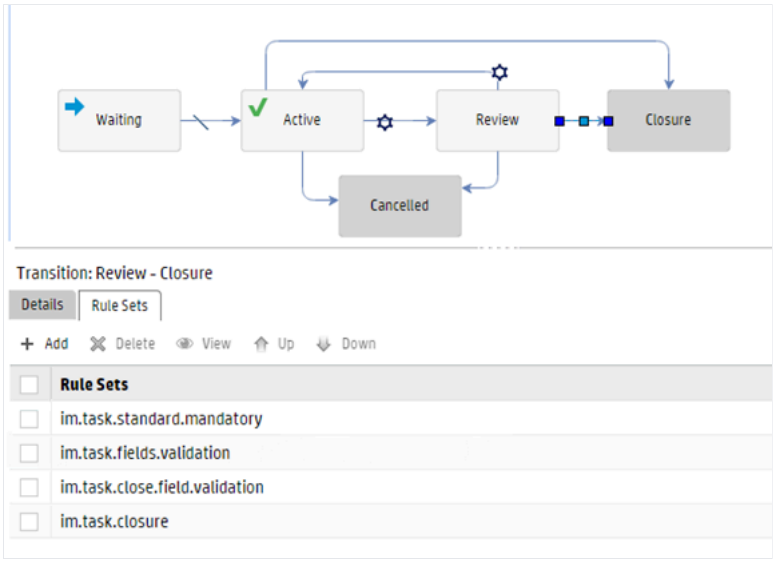
You can configure whether the record is saved before the transition occurs by selecting the "Save record prior executing the transition" option. Usually, if a manual transition needs to trigger the same events as a save operation (for example, to perform the same validation of a save operation), and if you don't want to configure duplicate validation Rule Set on this manual transition, you select this option.

For example, the following image shows a typical manual transition in which the target phase is the "Closure" phase. Therefore, the "Save record prior executing the transition" option is selected.

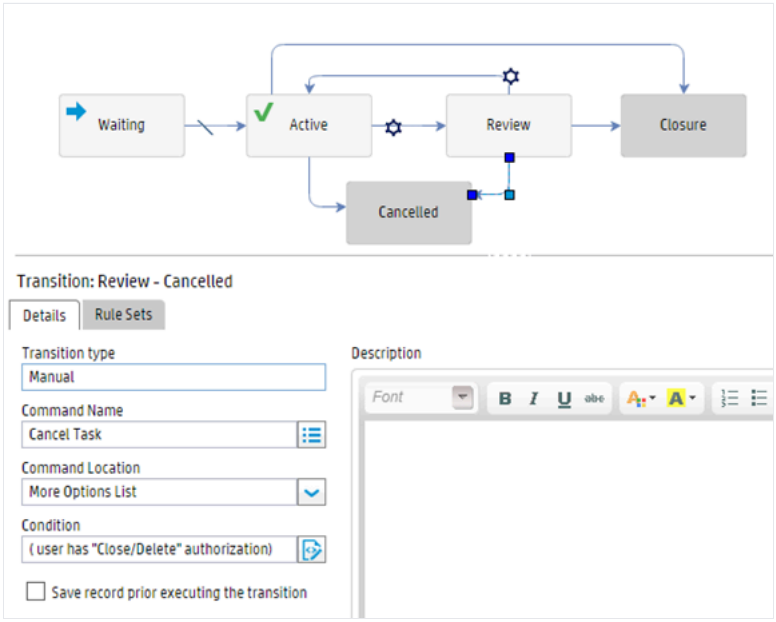


This transition uses the following rule set:

- The "im.task.close.field.validation" rule set is needed specifically for this close operation.
- These common mandatory and validation rules are triggered when the record is saved before the transition occurs.
- Only configure rule sets in transitions if they are absolutely necessary to move from one phase to another. Common mandatory rules and validation rules are already configured for the beginning phase (the "Review" phase in the following image), and don't need to be configured in the transition. We don't recommened that you configure rules that are triggered in the different "events."

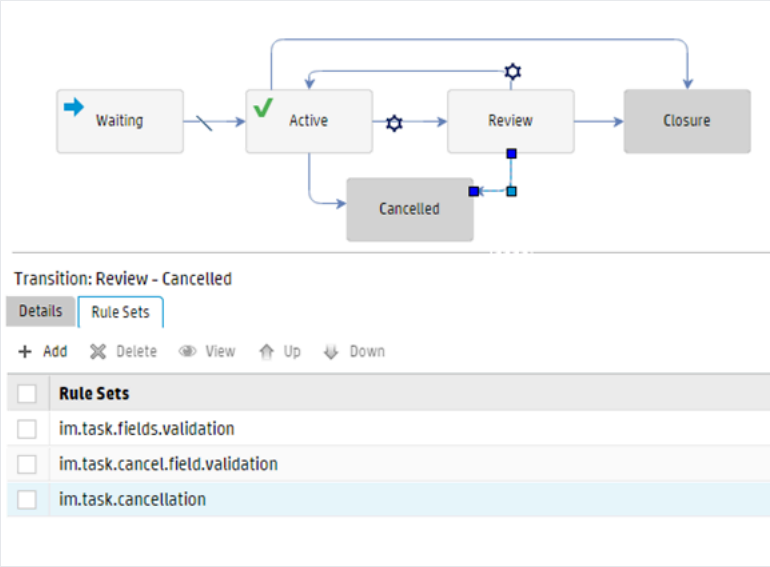


The following image shows an example manual transition in which the target phase is the "Canceled" phase. Therefore, the "Save record prior executing the transition" option is cleared.



This transition uses the following rule sets:

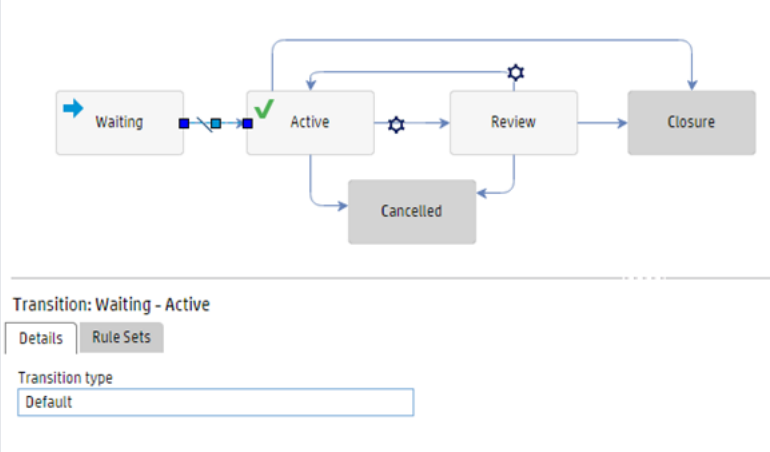
- The only common field validation used is the "im.task.fields.validation" rule set.
- The "im.task.cancel.field.validation" rule set is needed specifically for this close operation.



Use Default Transition[edit]

A default transition is a special case that moves the workflow automatically only when no other automatic transition conditions are satisfied.

If your "Logging" (or first) and default phases aren't the same phase, you can use a default transition between the "Logging" phase and the default phase.

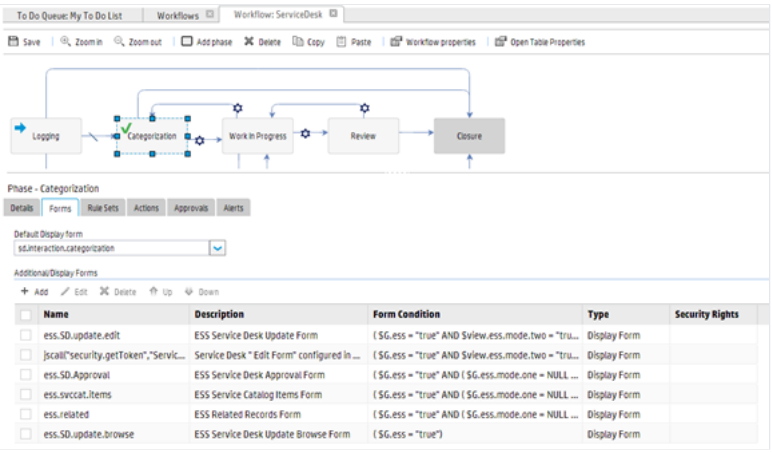


Forms

You can configure workflows so that a specified form is displayed when that a record moves to a specified phase.

You can user the following methods to specify the form that's displayed:

- In the **Forms** tab in the workflow editor, set the form that's displayed by default in the **Default Display form** field. This form is used when no conditional display forms are displayed.
- In the **Forms** tab in the workflow editor, configure forms that are displayed when certain conditions are met by clicking **Add** in the **Conditional/Additional Forms** section, and selecting **Display Form** in the **Type** drop-down list.

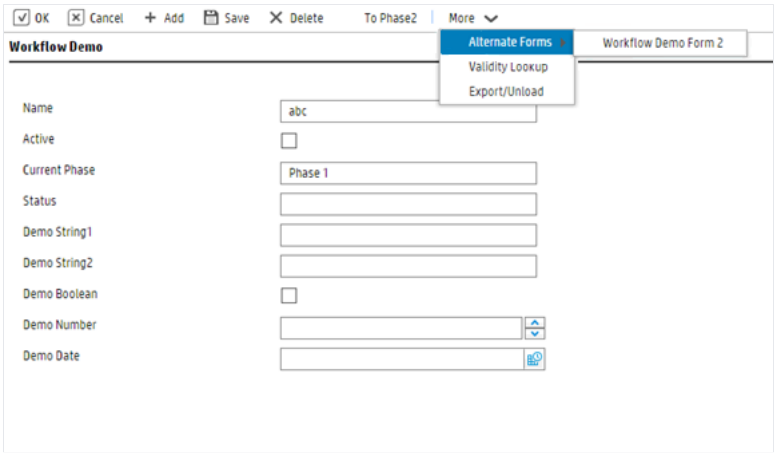


Note

Note

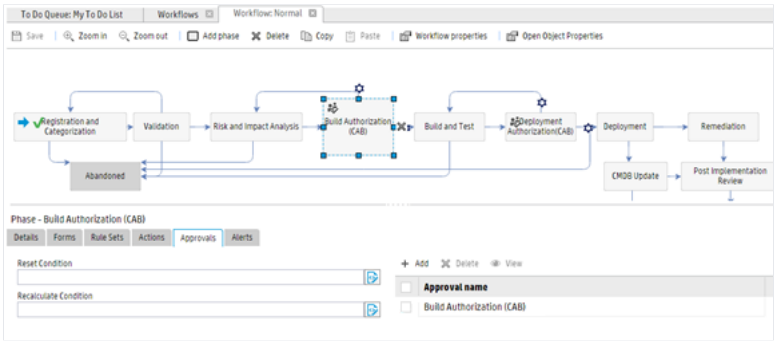
- Display forms that you specify for phases by using this method take precedence over the format setting in State and Display Screen. We recommend that you set the display form in the phase settings in the workflow editor, and that you use different forms for different phases to allow different information to be displayed and captured at various stages of the workflow.
- The name field supports JavaScripts, the `jscall` RAD expression, and RAD variables.

- In the **Forms** tab in the workflow editor, configure additional forms by clicking **Add** in the **Conditional/Additional Forms** section, and selecting **Additional Form** in the **Type** drop-down list. Additional forms are available as alternate forms when users view a record.



Approvals

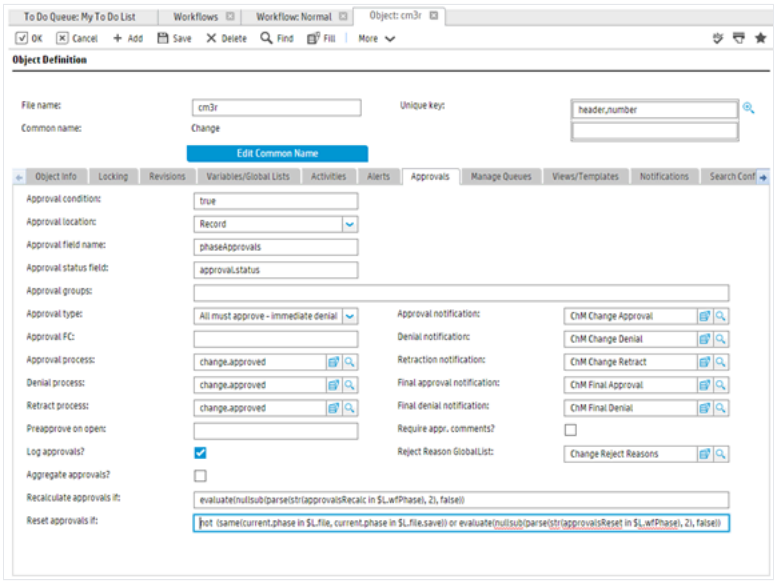
You can define approvals for a specific phase in the **Approvals** tab of the workflow editor if you require an approval when a record moves to this phase.



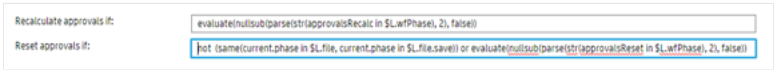
To do this, make the following configurations in the **Approvals** tab of the object definition record:

- To determine whether the document engine triggers the approval mechanism, set the value of the **Approval condition** field to "true," or set a specific condition.
- To set the file that contains the approval status, type the file name in the **Approval status** field.
- To set the conditions under which all existing approvals for a record are deleted and regenerated, enter a condition in the **Reset condition** field.
- To set the conditions under which the approvals are recalculated without first deleting the existing approvals, enter a condition in the **Recalculate condition** field.

> **Note**
>
> Note The reset condition takes priority over the recalculate condition.



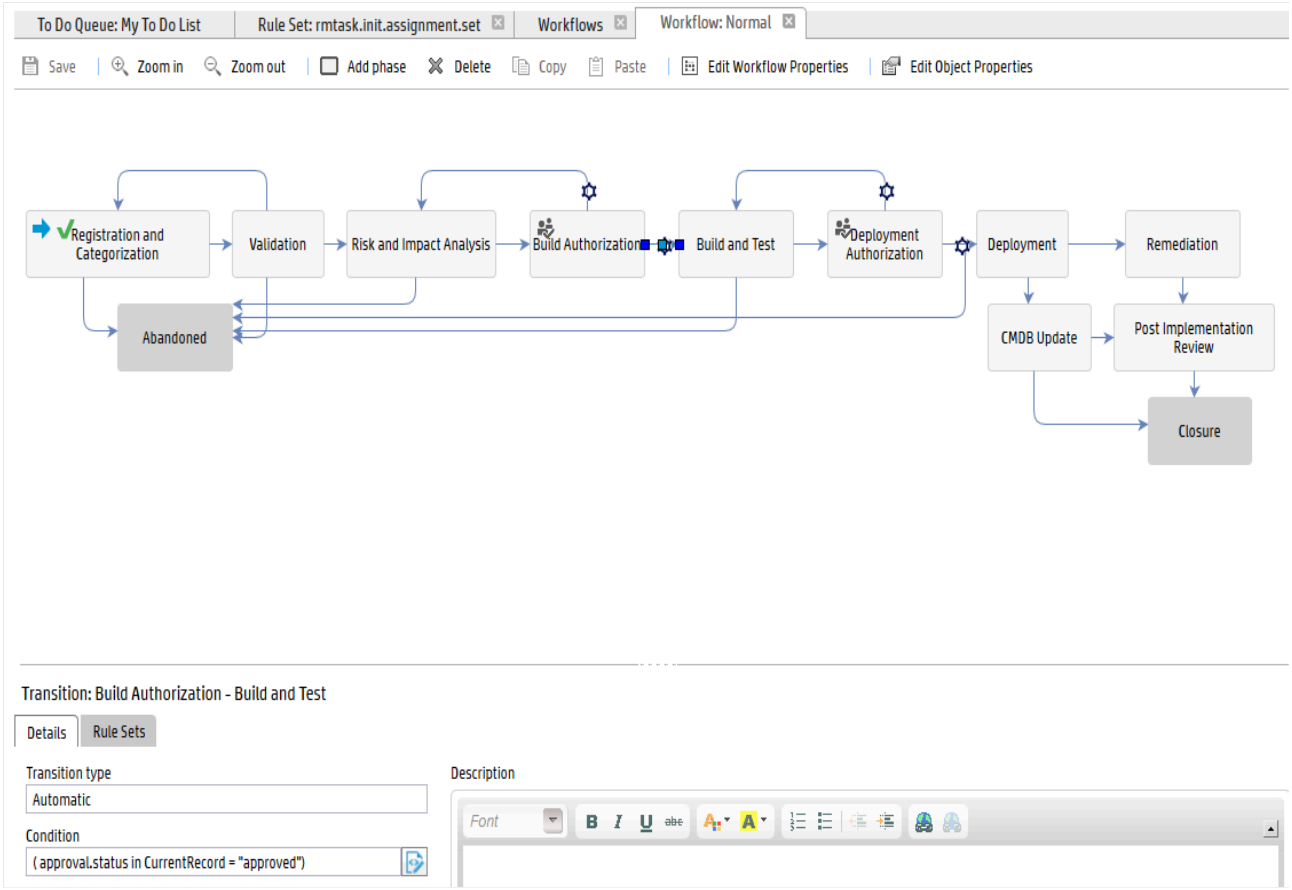
> **Note**
>
> Note Usually, the recalculate and reset conditions in the object record are configured as follows:



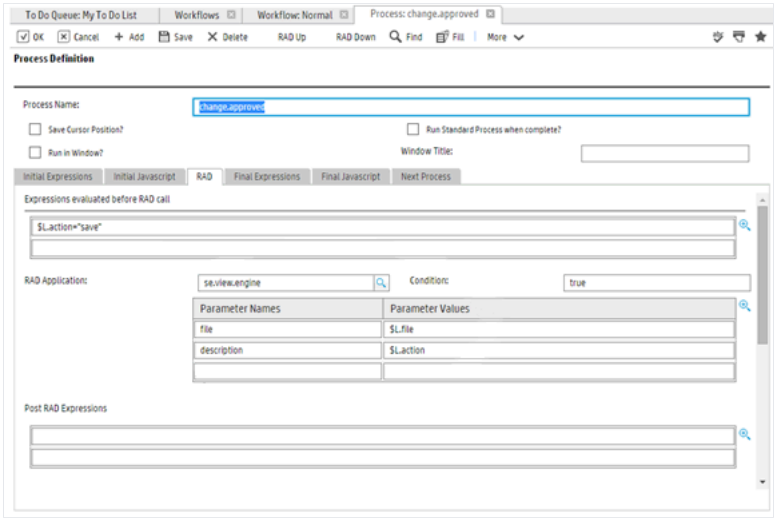
For more information about making further changes to approvals, see the Service Manager Help.

Configure the Approval/Denial process:[edit]

Usually, transitions from phases that require approvals to move to the next phase use automatic transitions. For example, you can configure a condition that the **approval.status** field must equal "approved" for an approved record and "denied" for denied record.



In order to trigger the automatic transition after the record is approved or denied, you must configure the approval/denial process to call the `se.view.engine` RAD script together with the "save" action. This triggers the Process Designer record save operation, and then triggers the automatic transition which matches the condition.



Contents

- 1 Automatic transitions
- 2 Manual transitions
- 3 Use Default Transition
- 4 Configure the Approval/Denial process:

7.5.8. Configure rule sets and actions

Rule sets enforce business logic in elements such as phases or transitions, for example checking if the user has filled in the required data or has the proper security level to perform a transition.

Actions perform a task for a phase, and refer to rule sets that are marked as "Available as Action."

7.5.8.1. Configure rule sets at various levels

You can configure rule sets in workflows at the following levels:

- At the phase level (only applies to the current phase)
- At the workflow level (applies to all phases in the workflow)
- At the object level (applies to all workflows of the object)

If you expect that a rule set will only be triggered at a specific phase, configure the rule set at the phase level. If you expect that a rule set will be triggered in all the phases of a workflow, configure the rule set at the workflow level. If you expect that a rule set will be triggered in all workflows that are associated with a specific object, configure the workflow at the object level. For example, you can move business logic that you have implemented in triggers to object-based rule sets.



Note

Note If a call bypasses the Document Engine, the triggers are still invoked, but the rule sets at various levels aren't invoked.

Rule set are executed in the following order:

1. Object level
2. Workflow level
3. Phase level

For more information, see Consider rule set execution order.

7.5.8.2. Configure rule sets at various triggering events

Following is the typical usages:

Triggering event	Triggering action	Typical usage	Typical configured rule type	Phase level	Workflow level	Object level

On add	Runs immediately before the record is added to the database	• Set the record ID if delaying assigning record number • Set default field values if they are empty • Automatically assign to the best team or assignee • Check the mandatory fields and variables for logging • Check whether the completed fields are valid	• Set Field from Number rule • Set Field rule • Assignment rule • Mandatory check: ○ Set Mandatory Fields rule ○ Mandatory Variables rule • Validation check: ○ Validate Date rule ○ Validate Text/Number rule ○ Validate against List rule ○ Validate		Yes	Yes

			against Table rule			
After successful add	Runs immediately after the record is added to the database	• Send email to notify relevant people • Send notification to notify operator with a customized message • Start or stop the elapsed time of the record	• Send HTML Email rule • Send Notifications rule • Start or Stop Clock		Yes	Yes

On enter	Runs when the record tries to move from another phase into this phase	• Check the mandatory fields and variables for Logging • Check whether the completed fields are valid • Set default field values	• Mandatory check: ○ Set Mandatory Fields rule ○ Mandatory Variables rule • Validation check: ○ Validate Date rule ○ Validate Text/Number rule ○ Validate against List rule ○ Validate against Table rule	Yes	Yes	Yes

			• Set Field rule			
After successful enter	Runs after the record successfully moves from another phase into this phase	• Send email to notify relevant people • Send notification to notify operator with customized message • Start or stop the elapsed time for a situation of the record • Scheduled automatic actions, forexample, automatic closure • Cross-module actions	• Send HTML Email rule • Send Notifications rule • Start or Stop Clock • Run Scheduled Action • Run Action	Yes	Yes	Yes
On exit	Runs when the record tries to move out of this phase			Yes	Yes	Yes
Initialization	Runs once just before the record is displayed to the user	• Set the record ID if not delaying assigning record number • Initialize the list that the current phase uses	• Set Field from Number rule • Run JavaScript rule	Yes	Yes	Yes

On display	Runs each time the record is displayed after a user action	Initialize the variables that are used on the display form	Run JavaScript rule	Yes	Yes	Yes

On update	Runs immediately before the record is updated in the database	• Check the mandatory fields and variables for logging • Check whether the completed fields are valid • Wizard to suspend or resume a process	• Mandatory check: ○ Set Mandatory Fields rule ○ Mandatory Variables rule • Validation check: ○ Validate Date rule ○ Validate Text/Number rule ○ Validate against List rule ○ Validate against Table rule	Yes	Yes	Yes

			• Run a Wizard			
After successful update	Runs immediately after the record is updated successfully in the database	• Send email to notify relevant people • Send notification to notify operator with customized message • Start or stop the elapsed time of the record • Scheduled automatic actions, for example, automatic closure • Cross-module actions	• Send HTML Email rule • Send Notifications rule • Start or Stop Clock • Run Scheduled Action • Run Action	Yes	Yes	Yes

Rule sets on Transitions	Runs when the transition occurs	• Check the mandatory fields and variables for logging • Check whether the completed fields are valid • Wizard to close or cancel the process for manual transition	• Mandatory check: ○ Set Mandatory Fields rule ○ Mandatory Variables rule • Validation check: ○ Validate Date rule ○ Validate Text/Number rule ○ Validate against List rule ○ Validate against Table rule			

			• Run a Wizard			

For more information, see Consider rule set execution order.

7.5.8.3. Configure actions at various levels

Actions can be applied to individual phases.

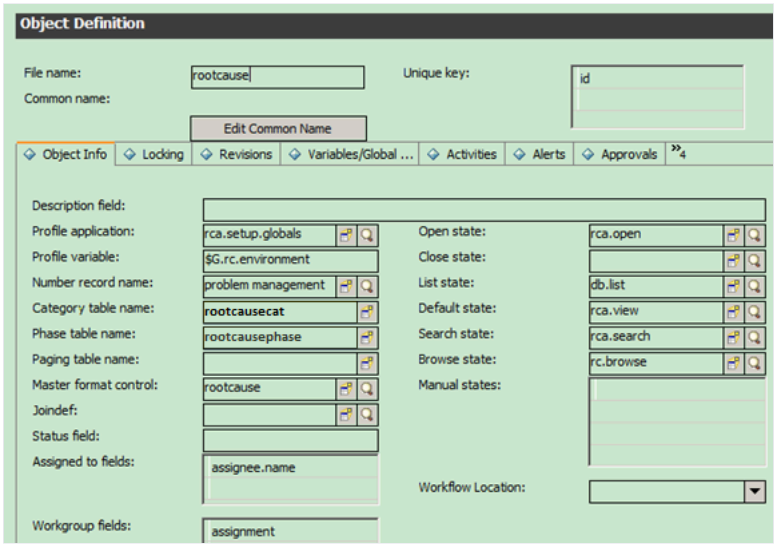
In certain circumstances, you may want to define actions at the workflow level. For example, in cases where a certain button needed to be present in each phase of a workflow. In other cases, you may want to define actions at the object level (for example, if a certain button needs to be present in each phase of all workflows).

7.5.9. Migrate legacy code to Process Designer

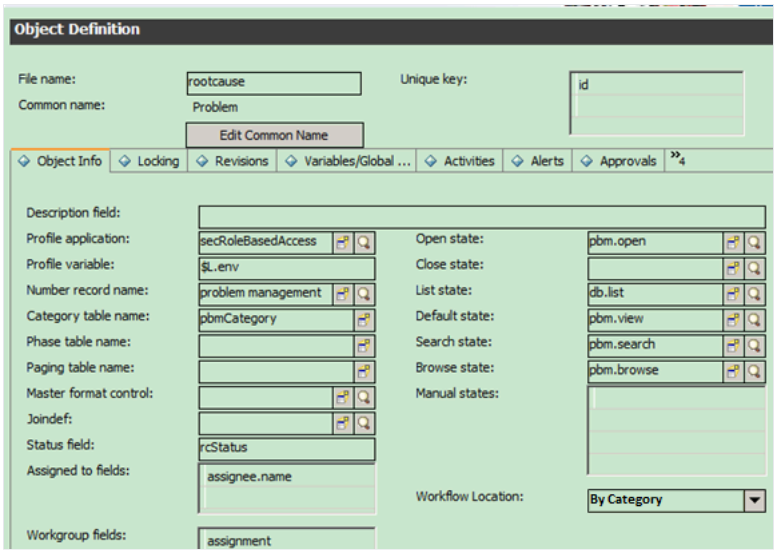
If you are upgrading to Service Manager Codeless from Service Manager Classic, all the legacy coding technologies (format control, display option, and so on) are supported by the Process Designer framework, and you can run your system in a hybrid mode. You can also migrate your legacy technology code to Process Designer technology, which is easier to maintain.

7.5.9.1. Enable a workflow

In a Service Manager Classic module, a workflow is usually configured in the category, and the category table name and phase table name are configured in the object.

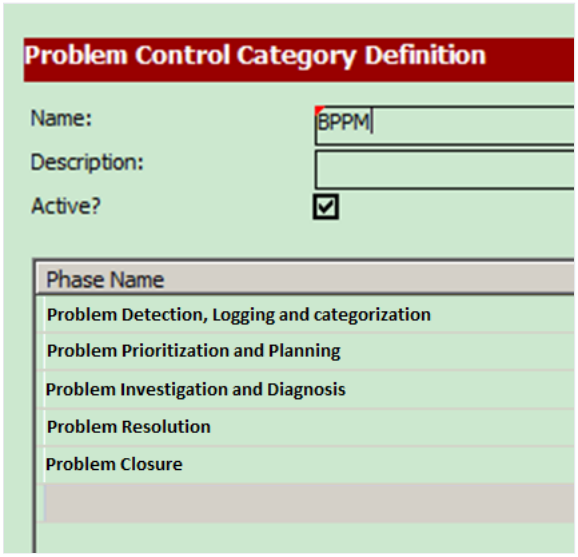


To enable the workflow, you must configure the workflow location in the object definition. If you set the location to "By Category," the workflow defined in the category record is used. If you set the location to "In Object," you must set the workflow in the object definition.

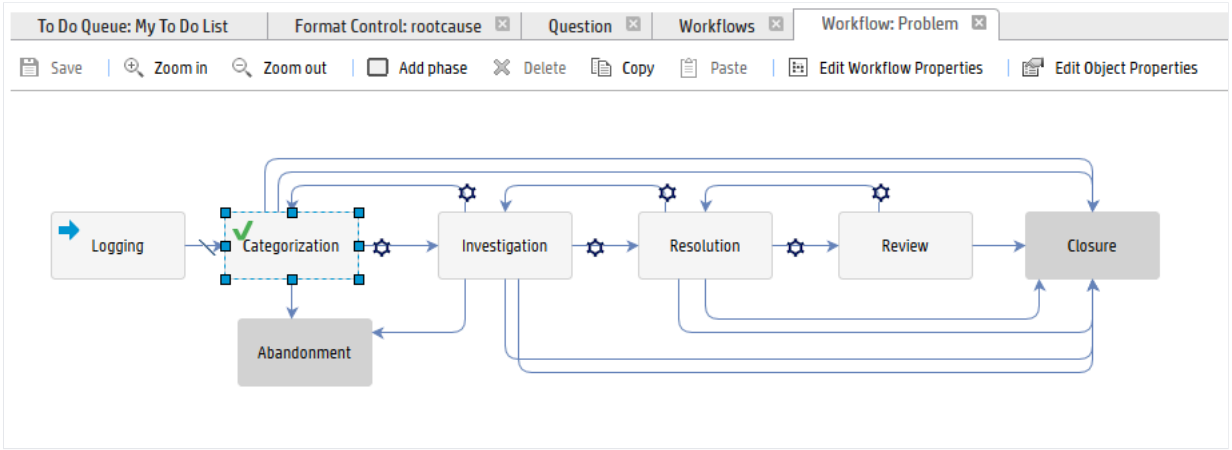


7.5.9.2. Configure the workflow

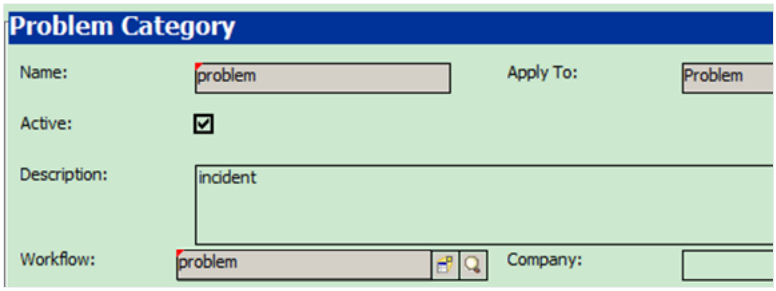
In a Service Manager Classic module, workflow phases are stored in the object-specific phase table, and the workflow logic is defined in the category by using the phases in sequence.



In Service Manager Codeless module, the workflow is configured graphically by using the Workflow Designer in the web UI. You can use this to define more complex logic between phases.

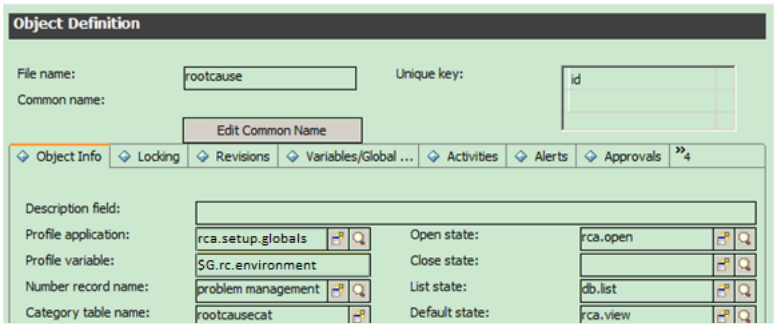


If your workflow location is set by category, the workflow name is specified in the category record:

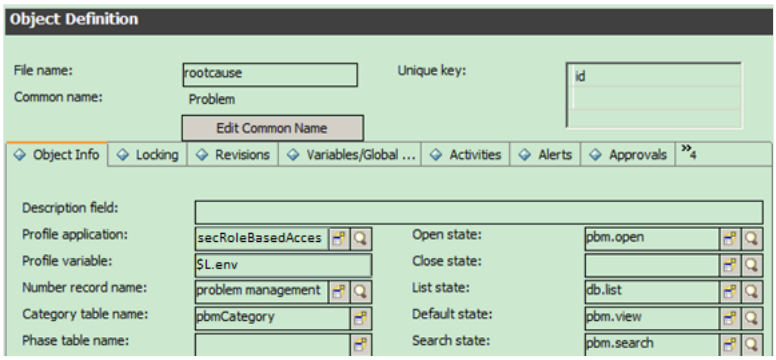


7.5.9.3. Enable Process Designer role-based security

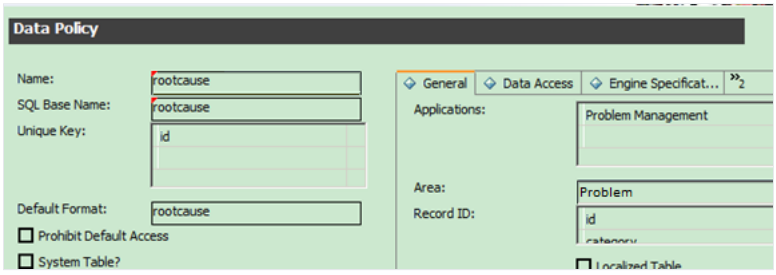
Service Manager Classic modules use the profile-based security model. This is enabled by using the module-specific application (for example, rca.setup.globals for Problem Management) in the object record, and set by the user in the operator table. The settings are stored in module-specific tables (for example, rcenv for Problem Management).



Process Designer provides a common role-base security model. This is enabled by using the secRoleBasedAccess application in the object record, and set by the user in the operator table.



The settings are stored in the secArea, secRights, and secRole tables. The security area is specified in the datadict record.



7.5.9.4. Migrate the Format Control to rule sets

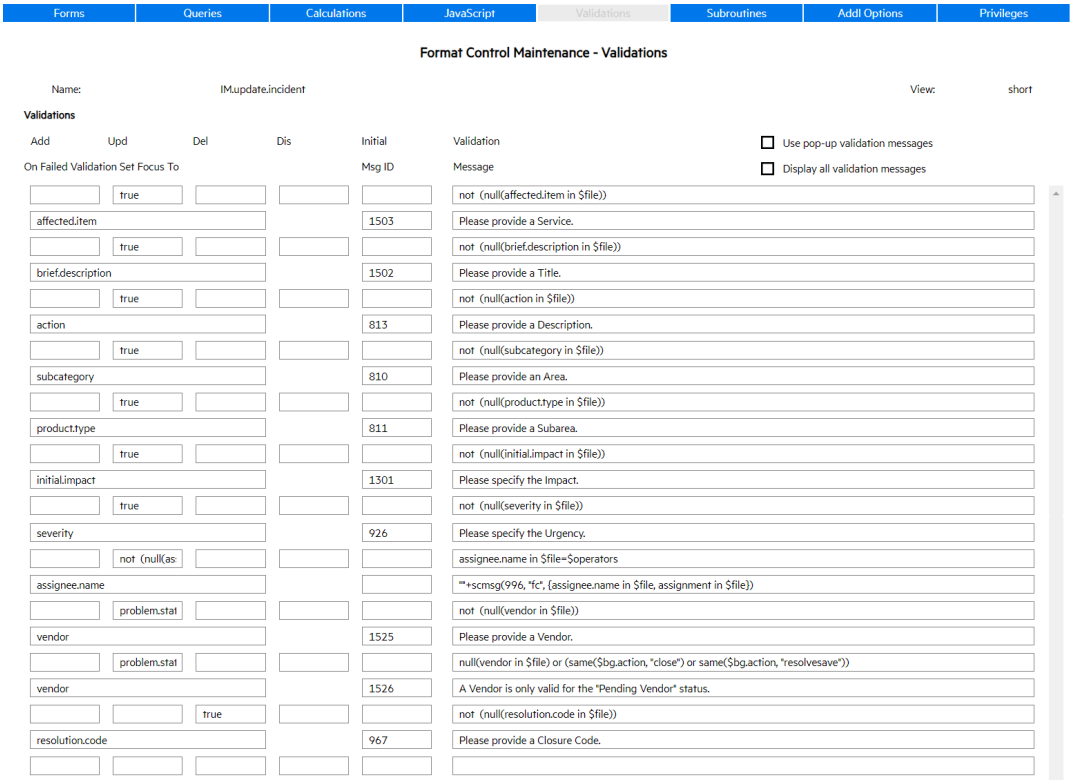
In Service Manager Classic, business logic is mainly configured in Format Control. Process Designer uses rule sets.

Format Control still works in Service Manager Codeless, but we suggest that you move all logic from Format Control to the workflow rule sets, as follows.

Format Control	Rule set
Primary Format Control "Add"	Object or workflow-based rule sets "On add"
Primary Format Control "Update"	Object or workflow-based rule sets "On update"
Primary Format Control "Display"	Object or workflow-based rule sets "On display"
Primary Format Control "Initial"	Object or workflow-based rule sets "Initialization"
Primary Format Control "Delete"	Object or workflow-based rule sets "On update"
Form level Format Control "Add"	Workflow phase-based rule sets "On Enter"
Form level Format Control "Update"	Workflow phase-based rule sets "On update"
Form level Format Control "Display"	Workflow phase-based rule sets "On display"
Form level Format Control "Initial"	Workflow phase-based rule sets "Initialization"
Form level Format Control "Delete"	Workflow phase-based rule sets "On update"

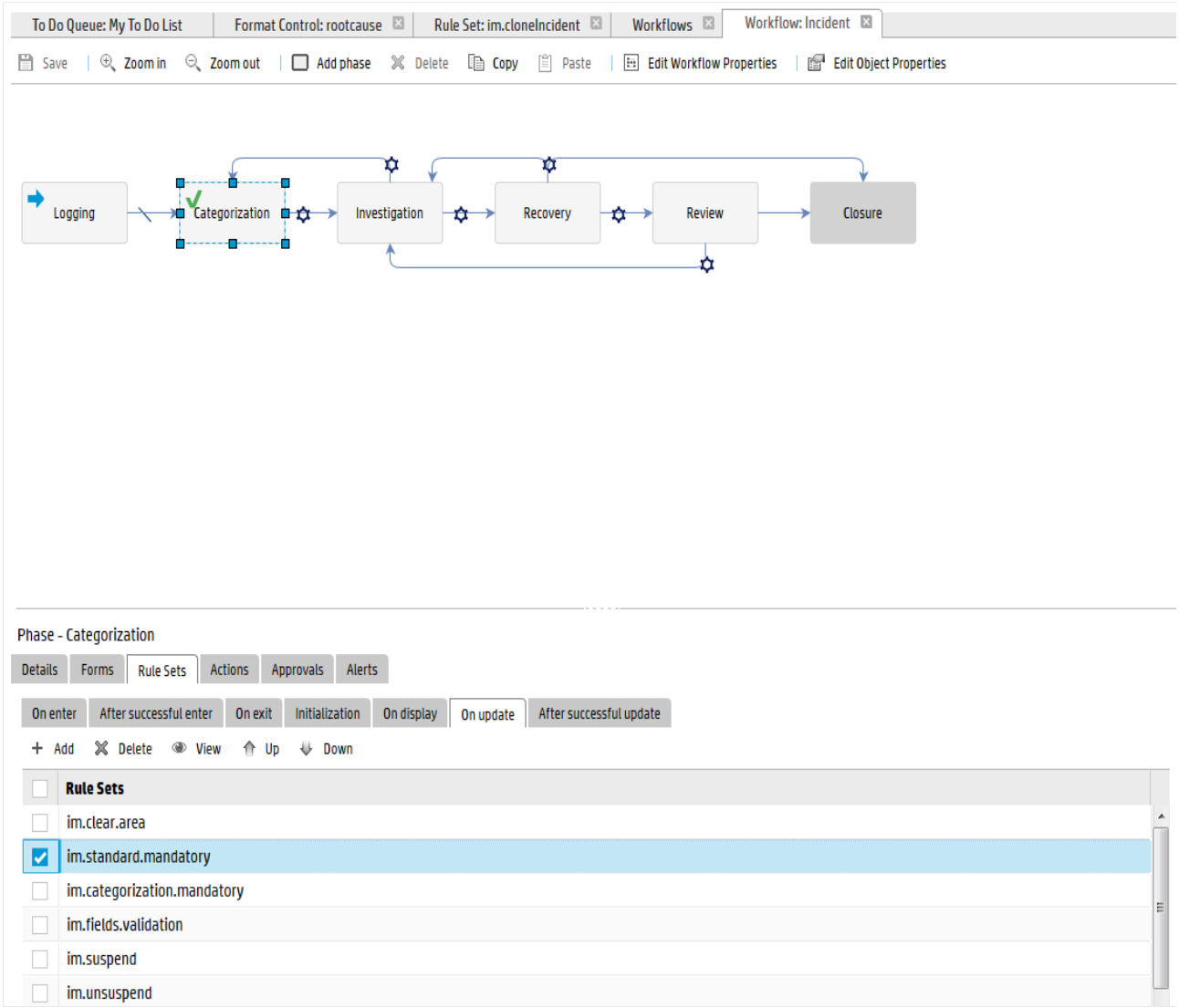
7.5.9.4.1. Mandatory validation

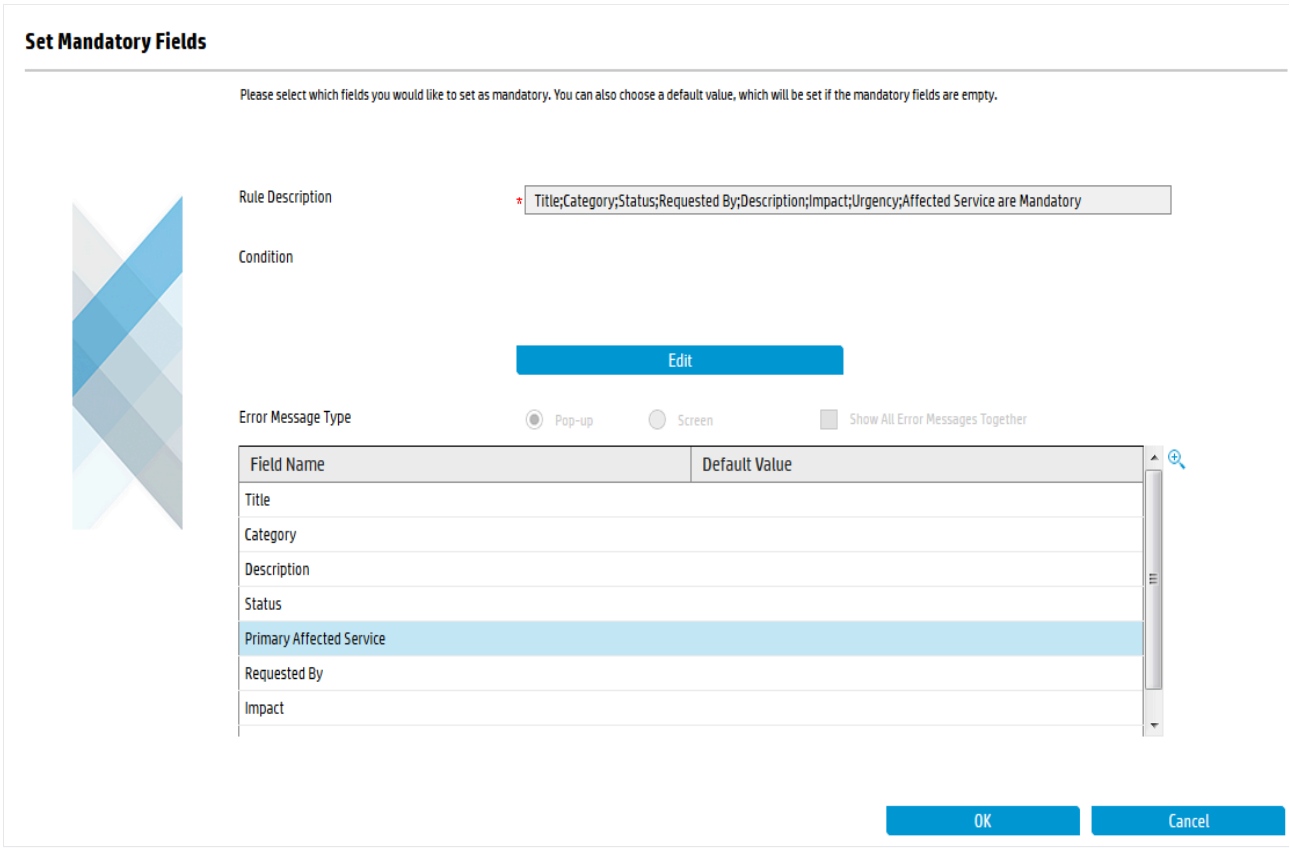
You can define Format Control validations on the master format control or the format level format control.



To migrate validation to rule sets, you must:

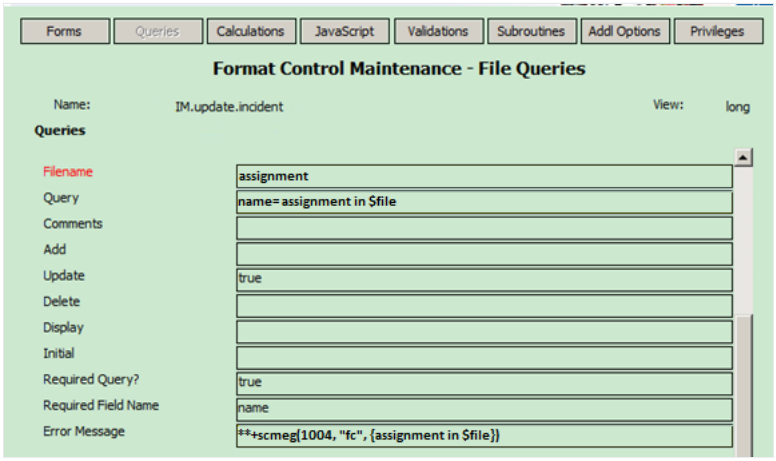
1. Define a rule set.
2. Set the rule set to the corresponding workflow or phase rule sets.
3. Add the Set Mandatory Fields rule to this rule set.


To Do Queue: My To Do List
Format Control: rootcause
Rule Set: im.cloneIncident
Workflows
Workflow: Incident
Save
Zoom in
Zoom out
Add phase
Delete
Copy
Paste
Edit Workflow Properties
Edit Object Properties
Logging
Categorization
Investigation
Recovery
Review
Closure
Phase - Categorization
Details
Forms
Rule Sets
Actions
Approvals
Alerts
On enter
After successful enter
On exit
Initialization
On display
On update
After successful update
Add
Delete
View
Up
Down
Rule Sets
im.clear.area
im.standard.mandatory
im.categorization.mandatory
im.fields.validation
im.suspend
im.unsuspend
To Do Queue: My To Do List
Format Control: rootcause
Rule Set: im.standard.mandatory
Workflows
Workflow: Incident
Mass Update
Mass Delete
Mass Unload
More
Id
Name
Tablename
im.standard.mandatory
Incident standard mandatory fields validation
probsummary
im.standard.set.default.values
Set the default incident values
probsummary
1 to 2 of 2
Show
100 records per page
Back
Previous
Next
More
Rule Set
ID
im.standard.mandatory
Available as action
Name
Incident standard mandatory fields validation
Proprietary
Table name
probsummary
Rules
CurrentRecord != "Suspended")))
Title;Category;Status;Requested By;Description;Impact;Urgency;Affected Service are Mandatory
Incident Manager are Mandatory (when (major.incident in CurrentRecord = true))
Inciden Manager are Mandatory (when (escalated in CurrentRecord = true))
Folder are Mandatory (when ($G.folderEntitlement = "true"))
Subcategory are Mandatory (when (problem.status in CurrentRecord != "Suspended" AND problem.status in CurrentRecord != "Categorize"))
Assignment Group are Mandatory (when ((current.phase in CurrentRecord = "Categorization" AND problem.status in CurrentRecord = "Assign") OR (current.phase in CurrentRecord != "Categorization" AND problem.status in CurrentRecord != "Suspended")))
Add Rule
Add Group
View Rule/Group
Remove Rule/Group
Move Up
Move Down

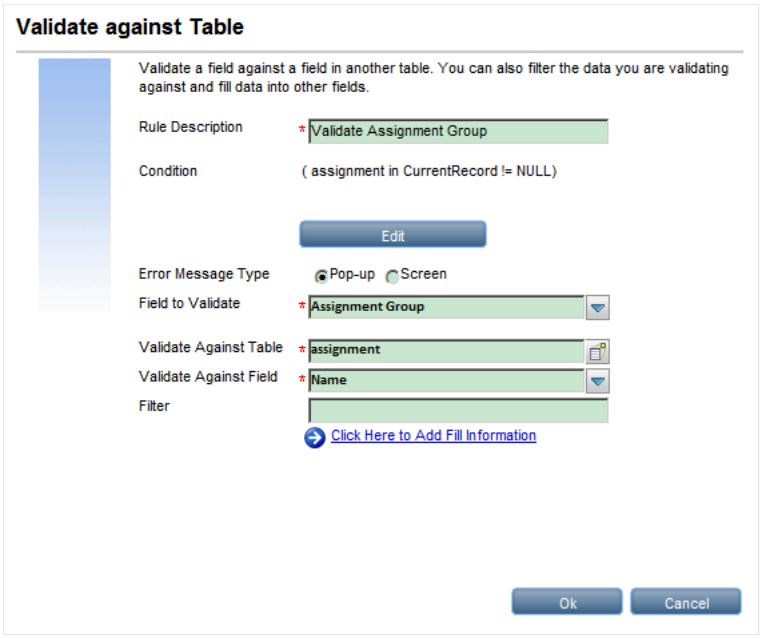


7.5.9.4.2. Validation against a table

To validate a field against another field in a table, you must define queries in master format control or in format level format control. You must write the code for the queries, manually create the scmessage, and indicate the ID if the message needs to be localized.

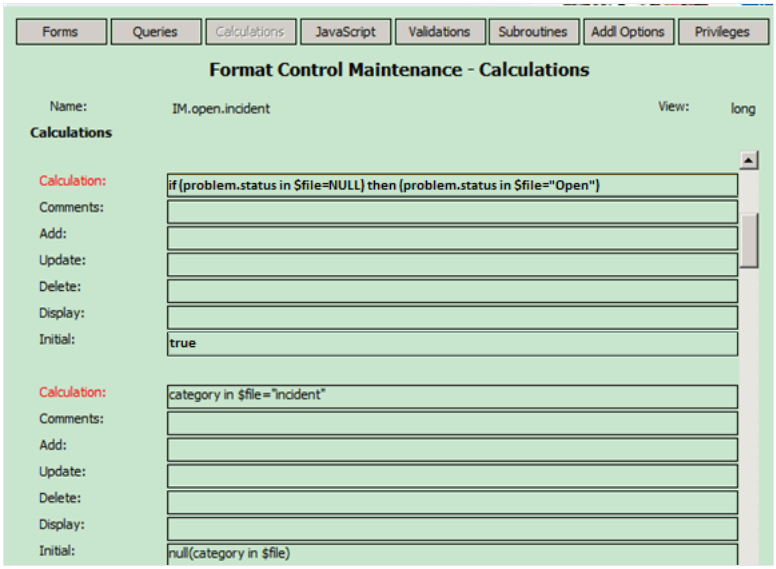


To do this in Service Manager Codeless, you can define a Validate Against Table rule.

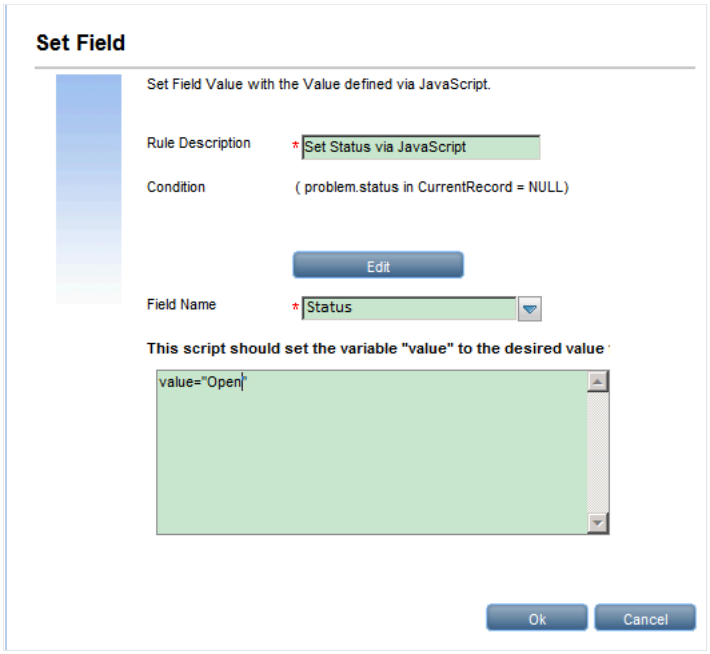


7.5.9.4.3. Set a field value

To set a field value by using Format Control, you must define the calculations in master format control or in format level format control. Additionally, you must use a RAD expression to write the field set statement (and the condition, if necessary).

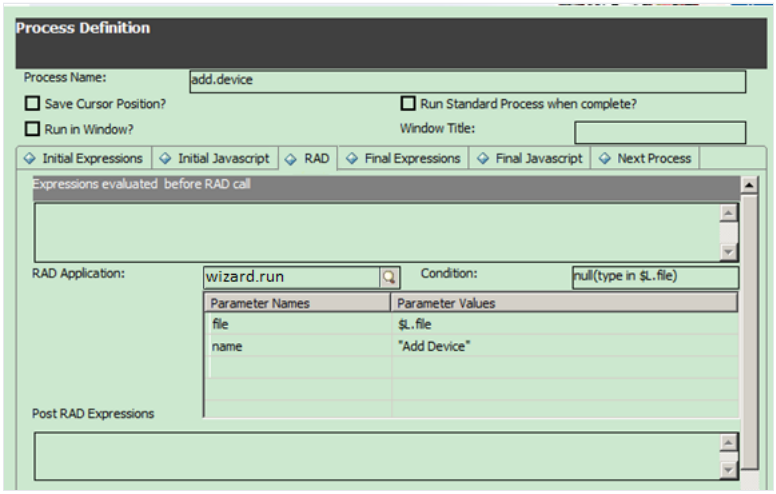


To achieve this by using a Process Designer rule set, you can define a rule set and assign it to the desired workflow or phase, and then add a Set Field rule to this rule set.

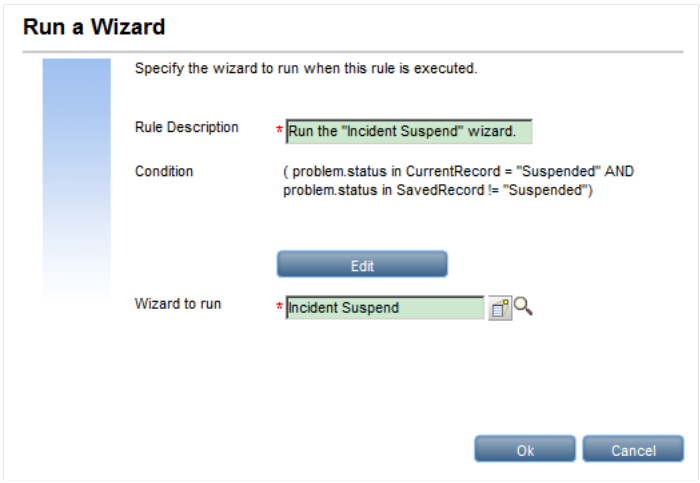


7.5.9.4.4. Run a Wizard

In Service Manager Classic, wizards are run by calling the wizard.run RAD application.

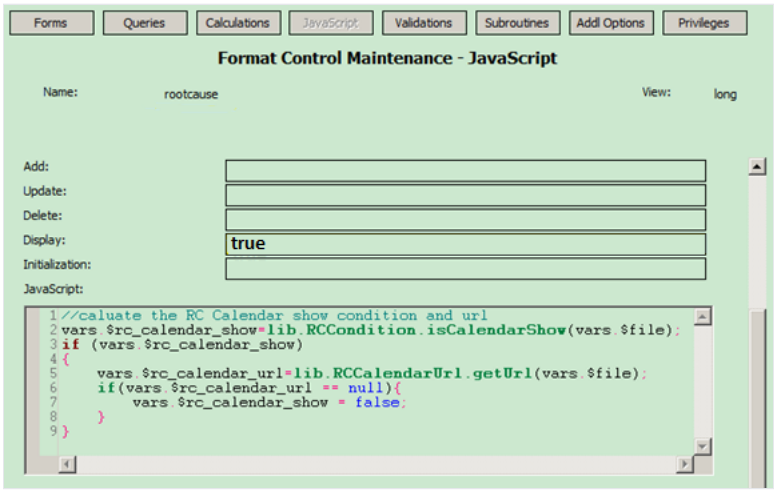


To do this in Service Manager Codeless, you can simply use the Run Wizard rule.

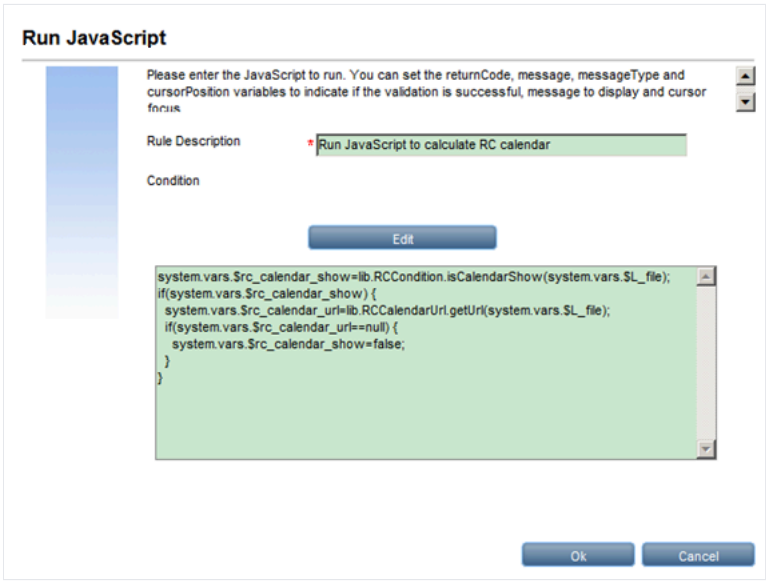


7.5.9.4.5. Run Javascript

To run a JavaScript by using Format Control, you must define the JavaScript in master format control or format level format control.



To do this in Service Manager Codeless, you can define a Run Java Script rule.



7.5.9.4.6. Additional supported rule types

The following rule types are available in out-of-box Service Manager deployments.

Rule type	Description
Launch a URL	Call a URL to launch a web page
Call a process	Call a Service Manager process record
Case Exchange	Trigger certain activities for the Case Exchange integration
Run a wizard	Run a Service Manager wizard
Clear Fields	Clear the specified field and related fields
JavaScript Validation	Use JavaScript to perform actions and validations
Run JavaScript	Use JavaScript to perform actions and validations
Mandatory Fields	Set fields as mandatory
Mandatory Variables	Set variables as mandatory
Send Notifications	Send Service Manager notifications
Launch a Script	Launch a Service Manager script
Send HTML Email	Send an HTML Email to users or a group
Start or Stop Clock	Start and stop a Service Manager clock
Set Field	Set a field value using JavaScript
Set Field from Number	Set field based on a number record
Validate Date	Validate a date against a date range
Field Validation Against a List	Validate a field against a list (global or defined)
Validate against Table	Validate a field against a field in another table and fill data into other fields
Validate Text/Number	Validate a field against a range of text or a number in another field of same table
Field Validation Against a Table	Validate a field against a different table
Popup Message Box	Create and configure popup message boxes that appear to end users
Assignment	Automatically distribute records (such as tasks or records) to the groups and individuals who are most able to process them
Run Action	Run actions (defined by rule sets) on records that have a specified relationship to the record that triggers the rule
Run Scheduled Action	Run actions (defined by rule sets) on records after a specified length of time has passed
Group rules	Group multiple rules into a rule group with an overall condition.

7.5.9.5. Migrate the display options to actions

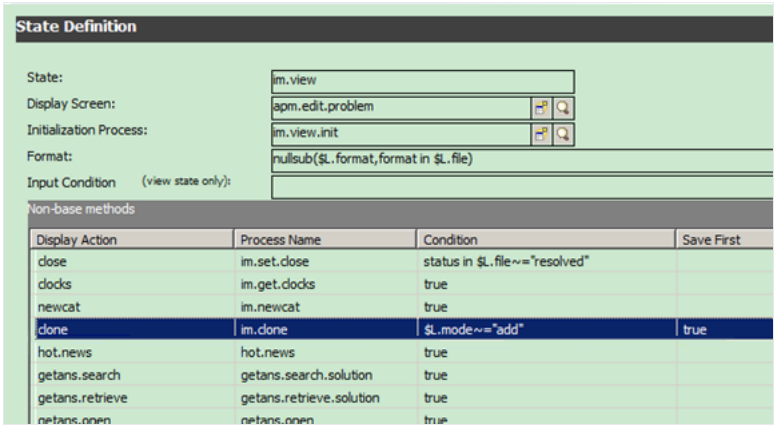
In Service Manager Classic, you must use display options to configure the buttons in a tray, more options list, or form.

In Service Manager Codeless, you can use actions to do this. Workflow actions and Manual workflow transitions appear in trays and more options lists as "virtual" display options, or on forms as buttons.

Display options still work in Service Manager Codeless, but we recommend that you move all display options to workflow actions. To do this, follow these steps:

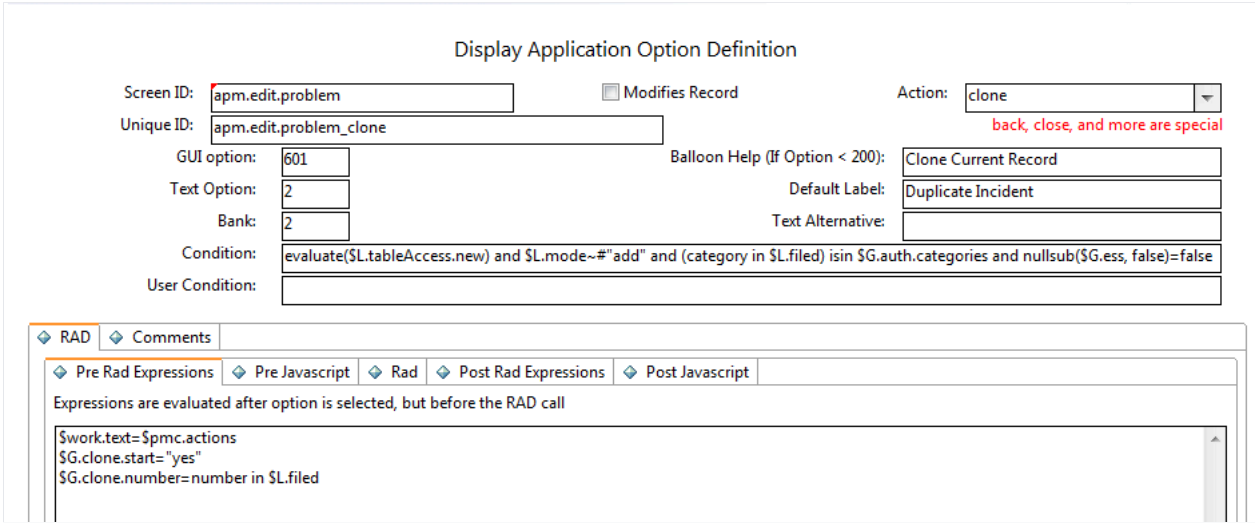
1. Identify your custom actions in the legacy states.
2. Examine the Display Screen field of each state to identify how the action is used in display options.
3. For each of your new custom actions, create a new rule set that calls the process (mapped to the action in the state definition) directly. The "Available as action" option in the rule sets must be selected.
4. Add the new actions to the workflows.

The mapping between the display action and the process is set in the state definition.

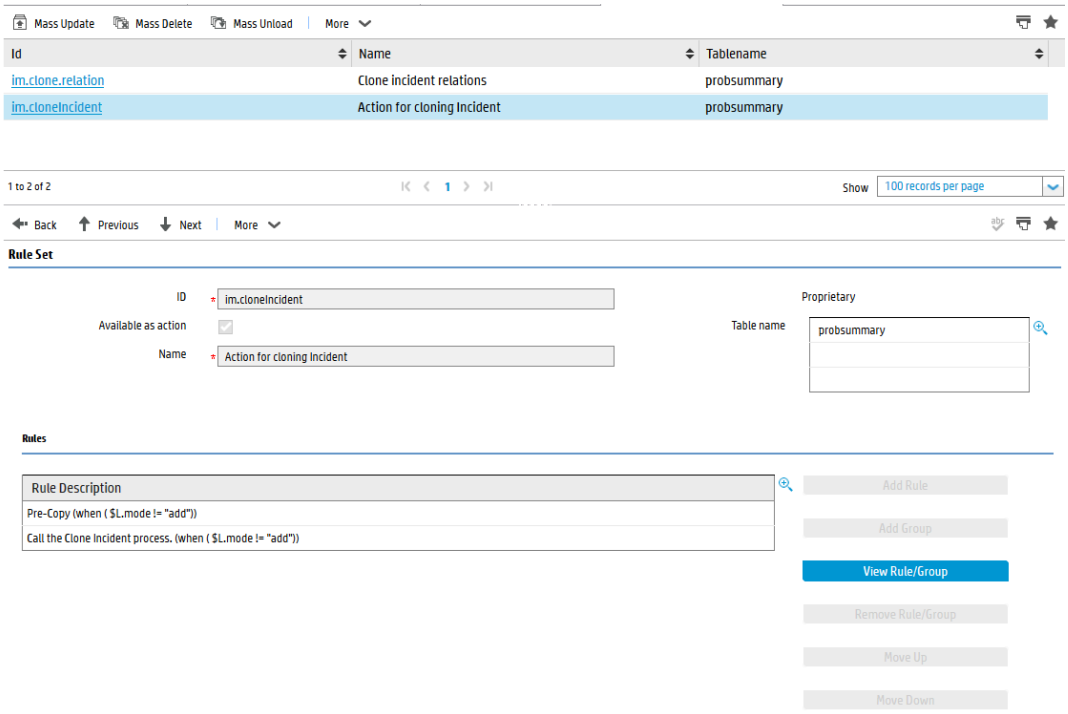


The definition of the display option that uses the action contains the following settings:

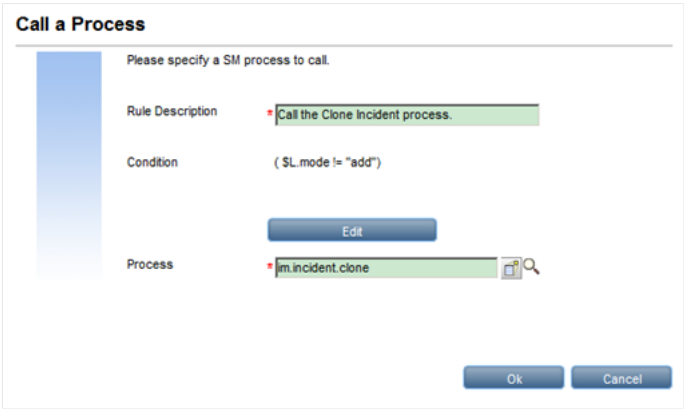
- The value in the **GUI option** field indicates where the display option is located (tray, more option list, or button).
- The **Default Label** field defines the display name.
- The **Condition** field contains a RAD Expression that definesthe security control.



To do this by using a Process Designer action, you must first define a rule set in which the "Available as action" option is selected.

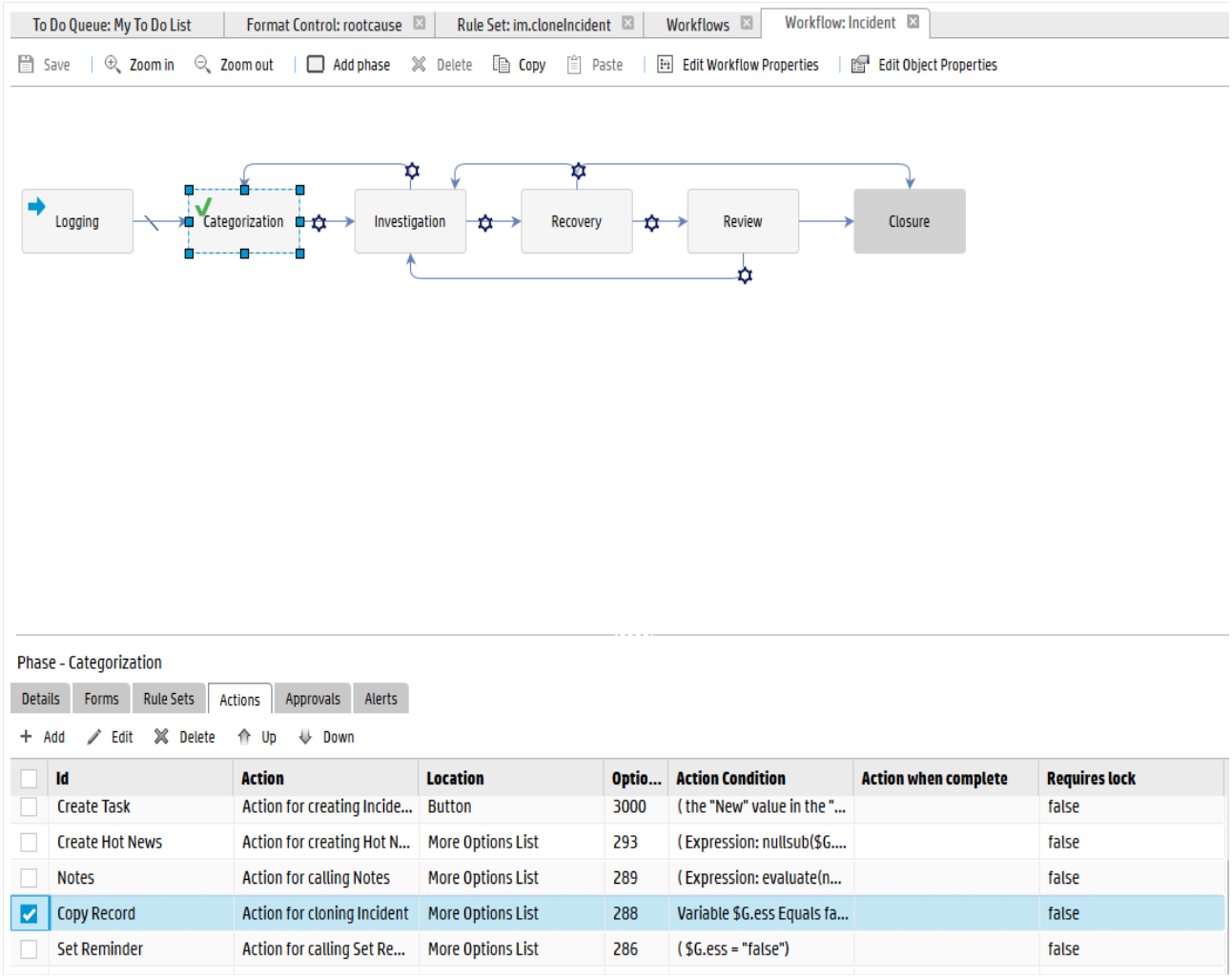


Then, you must add a Call a Process rule to this rule set.



You can use this "action-type" rule set at the workflow or phase levels.

- The Location column indicates where the display option is located (tray, more option list, or button)
- The ID column is set to the display name
- You can use the condition editor to set security controls



8. Wizards

This document describes the Wizard tool that you use to create custom wizards, and defines and describes each of the fields the tool includes. A detailed example walks you through all of the steps involved in planning and building a sample wizard that creates a new Service Level Agreement. For more troubleshooting information and common errors, see Troubleshoot Service Manager Wizard.

8.1. Wizards Overview

Wizards provide a powerful way to gather user input and help your users quickly and efficiently accomplish many different tasks within Service Manager. The Service Manager applications include many out-of-box wizards, and the Service Manager tailoring tools include a Wizard creation tool that enables you to design and build your own custom wizards that can be called from the various Service Manager applications and modules.

Prerequisites[edit]

Using the Wizard tool requires a solid understanding of the following areas, which are covered in other parts of the Service Manager tailoring documentation:

- Designing and creating forms
- Using the syntax of the Service Manager system language
- Performing traces for debugging purposes
- Creating processes and using the Document Engine

About creating a wizard[edit]

The Wizard tool enables implementers and administrators to add wizards within Service Manager that assist users with certain tasks, such as adding contacts to a database, escalating a Service Desk record to the applicable Service Manager module, and creating a new

configuration item type for Configuration Management.

Functionality in a wizard record can include any or all of the following functionality:

- Creating records
- Selecting records from a database
- Modifying the current record
- Executing expressions (RAD or JavaScript), processes, and format control
- Asking for user input
- Stacking wizards to run in succession

You may find it helpful to review the embedded, out-of-box wizards. These wizards offer a variety of functionality, such as:

- Create a User (the User Quick Add Utility)
- Escalate an Interaction
- Reject a Change
- Add a CI Type
- Add a Contract

You can view these wizards by clicking **Tailoring > Wizards**, and then clicking the **Search** button.

In addition to simplifying repetitive user tasks, Wizards are very useful for processes that require the user to supply prerequisite information. For example, during the interaction creation cycle you could present a wizard panel flow for the operator to enter data. Based on how the caller replies to questions regarding the problem, the wizard determines which panels the operator sees next. While the wizard is executing, the operator-entered data may be accumulated in a file variable that's returned to the calling application when the wizard is complete.

The wizard can display a form and execute a Format Control record or process. The instructions in the Format Control record where there is a condition on “initial” that evaluates to true will be executed before the form opens. The instructions where there is a condition on “add” that evaluates to true will be executed after the user clicks OK.

Wizards are also very helpful when multiple complex decisions must be made in order to reach a conclusion. For example, Change Management approval requirement conditions are normally based on the data contents of one field in the change record. An example of such a condition is `risk.assessment in $L.file="1."` However, there may be circumstances where the approval requirement condition is based on the values of several different fields. For instance, there may be three fields that affect the approval requirements: division, area, and department. Hard coding all the possible combinations of these field values into condition statements requires a great deal of work and would be difficult to maintain. You can define these conditions in wizard records that don't display forms but that allow you to call a subroutine or execute standard Service Manager processing statements (similar to Format Control calculation statements). These options allow for the manipulation of record data. At runtime, the wizard acts as a decision tree that results in significant processing reductions compared to that involved in the original method of coding.

Wizard flow[edit]

Wizard flow defines the order in which wizard panels execute. Wizard panels can move in a straight line from start to finish, or branch into several different process flows.

The more complex wizard flows use condition statements that must evaluate to true before certain wizard panels can execute. If none of the specified conditions evaluate to true, the wizard flow is considered complete, and control passes back to the calling application.

It is very helpful to develop a map of the entire flow showing the name of the form (if any) displayed by each wizard panel and the conditions controlling the flow from panel to panel. You can follow the map throughout the wizard creation process to avoid simple errors that could prevent your wizard from operating properly.

Wizards versus scripts[edit]

Service Manager wizards have much in common with the Service Manager scripts utility, a legacy tool that has been part of the system tailoring utilities for many releases. Wizards go beyond scripting, enabling much greater functionality and flexibility.

A major advantage of wizards over scripts is security. Wizards are more robust and ensure data integrity, which scripts can't do.

Unlike scripts, wizards allow you to specify the format control and display screen; you're not limited to the format control for a particular form, or to a particular display screen.

In addition, wizards themselves can call Format Control records and processes. Wizards don't allow direct calls to RAD applications.

Contents

- 1 Prerequisites
- 2 About creating a wizard

- 3 Wizard flow
- 4 Wizards versus scripts

8.2. Wizard fields

Before beginning to create a wizard, you should have a basic plan for the wizard flow, including the function that each wizard panel must perform. This requirement is demonstrated for the example of planning the SLA wizard panels for a new SLA wizard.

When you are ready to create the wizard, you use the Wizard tool to create each successive panel of the complete wizard. Generally each panel represents a screen or step of the wizard that the end user will see, although some panels aren't visible to the user and instead perform background activities, such as calling a Document Engine process.

To access the Wizard tool, click **Tailoring > Wizards** or type `wizards` in the Service Manager command line field.

The Wizard tool displays ten tabs:

- **Wizard Info**
- **File Selection**
- **Usage**
- **Actions**
- **Messaging**
- **Variables**
- **Next Wizard**
- **Comments**
- **Cancel Expressions**
- **Previous Wizard** (only visible after you click **Add**)

8.2.1. Wizard order of execution

When a wizard is run, it executes in the order the tabs are displayed in the wizard tool interface (Wizard Info, File Selection, Usage, and so on.)

On the Usage tab, the developer determines whether the wizard panel is displayed to the end user. If the panel is displayed, the order of execution then depends on which button the end user clicks, Next, Previous, Cancel, or Finish.

If the end user clicks the Previous button, the previous wizard panel is displayed, or some other panel as set by the developer using the Previous Wizard tab in the wizard tool.

The Finish button is intended to be used at the end of the workflow. If the user clicks it in the middle of the workflow, the wizard will proceed through the rest of the sequential wizard panels without prompting the user for further input.

The Comments tab is used for internal development comments, and has no effect on the wizard processing flow.

Within the File Selection tab, the Initial Expressions and Javascript evaluate *after* the **Select $L.file by** tab. The means that you can't populate a query variable in Initial Expressions and then use it in "Select $L.file by."

8.2.2. About variables

It is best to use local variables in a wizard in order to eliminate the chance of a variable collision between threads (or even in the same thread if the wizard calls an external application). However, variables local to the wizard application can't be used by the display application or in Process records called by the wizard.

Specifying a local variable as a wizard variable increases the scope of the variable to include the display application and called Processes. The variable will automatically be cleaned up by the server when the entire wizard flow is complete, so there is no danger of namespace collisions with variables from other threads or processes.

8.2.3. Variables to avoid using in wizards

When creating a wizard, you should not use local variables that are used by the **wizard.run** RAD application. Using those variables could cause errors and unexpected results.

The variables to avoid using when creating a wizard are listed below.

$L.action	$L.action.file	$L.action.file.save
$L.action.save	$L.addition	$L.allow.finish
$L.allow.skip	$L.base.format	$L.bitmap
$L.category	$L.disable.next	$L.disable.next.previous
$L.disable.previous	$L.display.screen	$L.ds.i
$L.ds.junk	$L.ds.nme	$L.ds.var
$L.env	$L.eval	$L.exit
$L.fc	$L.field.line.is	$L.field.name.is
$L.file	$L.file.save	$L.filename
$L.finish	$L.first.parent	$L.format
$L.format.name	$L.gen.ms	$L.i
$L.link	$L.list.field.name	$L.lng
$L.main.title	$L.messages	$L.mode
$L.ms	$L.msg	$L.msg.cond
$L.msg.lvl	$L.msg.type	$L.mult.sub
$L.next.wizard	$L.no.recs.msg	$L.object
$L.object.type	$L.old.format.name	$L.old.subformat
$L.option	$L.override.link	$L.par.lng
$L.parent	$L.parent.file	$L.parent.id
$L.parent.object	$L.pass.names	$L.pass.values
$L.passed.file	$L.phase	$L.prev.wizard
$L.prompt	$L.qbe	$L.qbe.format
$L.same	$L.save.name	$L.selected
$L.selection	$L.selection.filename	$L.selection.save
$L.selection.sort	$L.selection.sql	$L.sql
$L.start	$L.sub.format	$L.title
$L.var	$L.window.title	$L.wizard
$L.wizard.bitmap	$L.wizard.error.msg	$L.wizard.format
$L.wizard.name	$L.wizard.names	$L.wizard.prompt
$L.wizard.sql	$L.wizard.title	$L.wizard.values
$L.x	$exit	$normal
$error		

8.2.4. Wizard Info tab field definitions

Use this tab to enter basic information such as the wizard name, description, window title, and images to be included on the panels displayed to the end user.

Field	Description
Wizard Name name	This field forms the unique key for the wizard in the wizard dbdict. Enter a brief descriptive name for the wizard as the unique identifier for this wizard record. As a best practice, include the table (or module) for the wizard, the action the wizard panel performs, and other detail such as whether the wizard is the Start Node. For example, your first wizard panel name for creating a wizard panel for SLA could be something like "SLA Create 1 - Start - Type." This is a required field.
Start Node? start	Selecting this checkbox (setting it to true), indicates that this is the first in a series of wizard panels. When a wizard set as the Start Node also requests user input, the Previous button is automatically disabled when the wizard panel is displayed to the user. **Note** Note If the Start Node wizard is not displayed to the user, be sure to disable the Previous button on the first wizard panel that's displayed.
Brief Description brief.description	Enter a brief, meaningful description of the wizard, including any important identifying information.
Window Title window.title	The window title is displayed on the tab that holds the wizard thread when displaying the wizard panel to the user. This field also takes a message number from the scmessage database. For example, scmsg(18,"wizard"). **Note** Note If the wizard you are creating will be localized (translated to another language or languages), then using the scmessage database is the preferred method.
Title title	The title of the wizard entered in this field appears in bold in the upper left of the wizard panel when displayed. The title can be entered as a message number from the scmessage database such as scmsg(18,"wizard"). Enter a title that makes indicates the purpose of the panel to the user, such as "Create SLA." If more detail is needed, you can enter something like "Create SLA - Enter Description."

Prompt prompt	Enter a prompt that instructs the end user if this is a wizard panels that will be displayed to the user. Service Manager displays this information in the center of the panel, between the title and the wizard subformat, to give the user more detailed instructions on what kind of information should be entered in the panel. The prompt should be in large type. **Note** Note Localizable messages, such as scmsg(18,"wizard"), can be used for the titles and prompt rather than hard-coding the wizard with text. This allows for localized languages in a multi-lingual environment.
Bitmap bitmap	The image to display on the left side of the wizard panel. The default image is located in the Service Manager Install Directory at: `Client\plugins\com.hp.ov.sm.client.common_<SM version and build number>\src\resources\icons\obj16\wizard<1,2,3>.gif` If you want to use a different image, add it to the installation path above and enter the image file name in this field.
Global Lists global.list	Click Fill to select the global lists to be built or updated so that they are available when the wizard is run.

8.2.5. File Selection tab field definitions

Use this tab to designate system language or JavaScript code to be run before file selection and determine the content of $L.file. The tab includes three subtabs, described below. The **Select $L.file by** subtab includes four main options, listed from top to bottom as they appear on the screen.

Field	Description
Initial Expressions subtab init.expressions	Enter expressions using system language syntax that will be evaluated after the $L.file is initialized. Use these expressions to initialize variables.
JavaScript subtab javascript.init	Enter the JavaScript code that will run after the $L.file is initialized. Use this section to initialize variables or select values from related tables.
Select $L.file by subtab	You must choose one of the four main options listed below to designate how the wizard will select the $L.file variable.
No $L.file (use typecheck) file.selection	This option means that the wizard should not expect a file variable to be passed in from wherever the wizard is called; in fact, if one is passed in, it will be ignored. Instead it creates $L.file by initializing it as a record in the "typecheck" table, which is an internal table that contains fields corresponding to different field types supported by the system. This is often used in simple wizards that require only a small amount of input that doesn't need to be saved into the database.
$L.file passed in file.selection	Selecting this option indicates that the file variable ($L.file) was passed into this wizard as a parameter from the calling application or wizard. If a wizard is called as a next wizard, the file variable is automatically passed on. This option should be selected if you plan to continue to work on the current record stored in $L.file.
Create a record file.selection	Selecting this option indicates that the wizard will create a new record. You must enter or select the dbdict name in the record of type field. This initializes $L.file with the structure of the record from the table selected in the "of type" option.
of type (for Create a record) create.record.filename	Click Fill to select the dbdict that defines the record to be created.
Select records file.selection	Select this option to use a query statement to select one or many records.
of type (for Select records) select.record.filename	This field determines the record type for a query. Enter the dbdict name of the table from which the records should be selected, such as location. Click Fill to select a name from a list of all dbdict names.
using query select.query	Enter a query, using system language syntax, to search for records that will be used to select records. For example: "agreement.id="+$G.new.sla **Note** Note The quotes (") are required.
Resolve Variables *resolve.variables*	Select this box (set to true) to ensure that any variables entered in the Select Records field are evaluated before running the query.

No Records Message select.no.records	The text to display to the user if the query returns no records. This field also takes a message number from the message database. For example: scmsg(111,"wizard")

8.2.6. Usage tab field definitions

Use this tab to set wizard usage options, such as whether and which subformat will be displayed to the user and which buttons will be available on the panel displayed.

Field	Description
Wizard Usage **section**	Determines the action that Service Manager should take when the wizard is run. You must select one of the five main options presented on the screen.
Select one record from list wizard.type	Choose from the **Selection Criteria** listed below.
Selection Criteria	When you choose to **Select one record from list**, selecting one of the following options allows you to select a record from a list. The list is formed based on the options selected.
Use $L.file as $L.selection use.file.as.selection	This option uses the $L.file variable to form the list of records where the selection is to be performed.
Query for records query.for.records	This option allows you to specify a query to form the list of records. The fields that are needed for the query aren'ted below.
of type query.for.records.filename	Enter the name of the table (dbdict name) that the query should run against. Check that the dbdict for field names is valid for the query.
using query query.for.records.query	This option is used in conjunction with the options "Query for Records" and "of type." Enter the query using Service Manager system language syntax.
sort by query.for.records.sort	This option allows you to present the list, sorted in order, based on the fields specified in this option. Enter field names that are valid for the selected dbdict and sorted in ascending order.
If no records query.select.no.records	This option allows you to specify an action if there are no records on the list. The actions are as follows: **return**: Returns to the previous wizard panel if there is one, or to the place from which the wizard was called (if this is the first panel). **continue**: Shows the empty list and allows you to continue. **skip display**: Skips the wizard panel and goes to the following panel, if applicable.
No records message query.no.records.msg	This option allows you to display a message to present if there are no records in the list. An scmessage record can be used to construct the message.
If one record query.select.one.record	This option allows you to specify an action if there is just one record in the list. The one record can be displayed in a list, or it can be used as the current file variable without being presented to the user for selection.
Allow "Skip" option? allow.skip	If selected (set to true), enables users to skip the panel.
Request user input wizard.type	This field indicates that the user will be prompted for information. When this option is selected, the **Sub Format to display** entry becomes required and a prompt with instructions needs to be entered in the **Wizard Info** tab.

Skip Display wizard.type	Skip Display should be used when the current wizard manipulates the current record without the need for user input. Typically, a wizard that's not displayed to the user is part of a complex wizard workflow.
Cancel Immediately wizard.type	This option is used to cancel a process immediately if a wizard is started by a user type who is not intended to use the wizard. For example: A company creates a wizard to gather information from top-level management customers. Let us say that a user who doesn't fit the criteria chooses to start the wizard and navigate through the wizard panels. The Cancel Immediately option causes the process to immediately end when it becomes clear that the information doesn't need to be gathered from that user.
Select multiple records from list wizard.type	Selecting this option displays a list and allows the user to select multiple records from the list. The list is formed based on the options selected in the "Selection Criteria" section.
Multiple Selection field mult.field.name	This option determines which field in the list becomes available for the user to select multiple records from.
Sub Format to Display sub.format	Enter the name of the format for user input to display within the wizard format.

Main Format (Defaults to Medium) form.Name	The wizard creation tool offers several format choices that effect the size of the wizard and the placement of buttons. Service Manager offers the following wizard format options. If no format is selected, the default is "Medium." **Small (wizard.small)** - This wizard offers the least amount of space for text and options, but also requires the least amount of screen space. This wizard is ideal for clients running in low resolution environments. It allows only a small space for the subformat, so the subformats need to be kept small. Buttons available are **Previous**, **Next**, **Finish**, and **Cancel**, and placement is horizontally-aligned along the bottom. **Medium (wizard)** - This wizard offers sufficient space for most wizard tasks. Buttons available are **Previous**, **Next**, **Finish**, and **Cancel**, and placement is horizontally-aligned along the bottom. **Large (wizard.large)** - This wizard offers the most amount of space for text and options, but also requires the most amount of screen space. This wizard is ideal for clients running with high screen resoltuions. It offers the most amount of space for the subformat to display, allowing for more complex subforms. Buttons available are **Previous**, **Next**, **Finish**, and **Cancel**, and placement is horizontally-aligned along the bottom. **OK Only - Small (wizard.okonly.small**) - Uses the small wizard size and limits the buttons available to navigate through the task. Buttons available are **OK** and **Cancel**, and placement is horizontally-aligned along the bottom. **OK Only - Medium (wizard.okonly)** - Uses the medium wizard size and limits the buttons available to navigate through the task. Buttons available are **OK** and **Cancel**, and placement is horizontally-aligned along the bottom. **Buttons on Top (wizard.buttonsOnTop)** - Uses the medium wizard size and places the buttons along the top of the wizard. Buttons available are **Previous**, **Next**, **Finish**, and **Cancel**, and placement is horizontally-aligned along the top. **Buttons on Side (wizard.buttonsOnSide)** - Uses the medium wizard size and places the buttons along the right edge of the wizard. Buttons available are **Previous**, **Next**, and **Finish**, and placement is vertically-aligned along the right side.
Display Screen (Defaults to wizard.display) display.screen	Enter a display screen, if you want to customize the button and menu options available within the wizard. The default display screen is wizard.display.
Activate "Finish" option? allow.finish	Use this option for the final wizard in the series. Selecting this checkbox makes a Finish button appear on the wizard panel. **Note** Note It is possible to have the Finish button available prior to the last wizard in the workflow, if the wizard panels displayed after this wizard only provide optional information. The Finish button can't be activated before all required information can be collected from the user.

Turn off Next and Previous buttons? disable.next.previous	Selecting this checkbox makes the Next and Previous buttons unavailable in the wizard, so the user can't click "Next" or "Previous" to move forward or backward in the wizard. **Note** Note This selection should only be made on the very last wizard panel in the workflow. Also, the Finsih button needs to be activated if the Next and Previous buttons are disabled.
Turn off Next button only? disable.next	Selecting this checkbox makes the Next button unavailable in the wizard, so the user can't click "Next" to move forward in the wizard.
Turn off Previous button only? disable.previous	Selecting this checkbox makes the Previous button unavailable in the wizard, so the user can't click "Previous" to move backward in the wizard. **Note** Note This option should be used if the first displayed panel of the wizard is not the Start Node. Selecting Previous in such a case would exit the wizard without going through proper cancel processing.
Use Conditional Previous Exits? previous.cond	When you select this option, the Previous Wizard tab appears on the form. In the Previous Wizard tab, you define the different possible exits when the Previous button is clicked.
Never prompt on cancel?	Determines whether the user is prompted when canceling out of the wizard. The Return prompt can send the user back to the panel.

8.2.7. Actions tab field definitions

Use this tab to set wizard actions, including defining which Process or Format Control will be executed and on which file variable or record they will be performed.

Field	Description
Initial Process initial.process	This option lets you specify a process definition to be executed prior to the actions to be performed. For information on process definitions, see the *Document Engine* documentation.
Perform Actions On section	You must select one of the three options to specify which record(s) will be affected by the action you are going to perform. The file is either the current file, the whole list selected, or an action to be performed on every single record selected. Actions are defined in the wizard, and define what will happen when a user clicks **Next** on a wizard.  **Note** Note Depending on what you selected in the **File Selection** tab, you can choose whether the actions performed when a user clicks **Next** will be performed on the current file or a record selection. Record selection ($L.selection) must only be chosen if the $L.selection file variable was filled when you chose Select Records on the **File Selection** tab, and if you chose either "select one" or "select multiple records" from the list on the **Usage** tab.
Current File ($L.file) perform.action.on	The actions will be performed on the single record represented by the $L.file file variable. The current file may have been passed in, created, or selected with a query returning only a single record.  **Note** Note The actions defined in the Actions to Perform section apply to the current file selected in the **File Selection** tab and/or **Usage** tab.
Selection ($L.selection) perform.action.on	The actions defined in the Actions to Perform section apply to the group of records selected in the **File Selection** tab.
Each record in selection ($L.selection) perform.action.on	The actions defined in the Actions to Perform block apply to each record selected.
Actions to Perform section	Use the three subtabs in this section to indicate the actions to perform on the selected file.

Expressions subtab expressions	Enter any expressions that should be run as part of the wizard after the initial process, before the JavaScript and Format Control (or process). These expressions use Service Manager system language syntax, such as: `name in $L.file="Test"` These expressions are evaluated before the Reset Current File to Selection action is performed. Therefore, actions performed here should use $L.selection rather than $L.file if they are intended to manipulate the user selected record. **Note** Note These expressions can use the record(s) selected, such as storing information in a field.
JavaScript subtab javascript.actions	The JavaScript expressions will run as part of the wizard and are executed after the initial process and expressions, but before the Format Control or process entered on the next subtab. These JavaScript language expressions can be used to manipulate the current file variable or initialize variables.
Format Control/Process Name subtab	
Format Control format.control	The Format Control record to use. You can enter a Format Control record to execute queries, calculations, validations, or subroutines after running through the expressions and JavaScript, and before calling the process.
of type format.control.type	The Format Control type to use for this wizard to evaluate the applicable conditions, so that Format Control can be executed on the action (**Add**, **Update**, or **Delete**) that's evaluated in the Format Control record.
on bad validation bad.validation.action	When data validation in Format Control fails, choose to return the user to the current wizard panel to enter the correct information, or choose to let the wizard continue to the next panel, ignoring the failed validation. • **Return** sends the user back to the panel. Should only be used if the user has the opportunity to fix the failed validation. If the wizard doesn't request information from the user by displaying a subformat to enter information, selecting Return will end up in an infinite loop. • **Continue** lets the user continue.
Process Name process.name	Enter the name of a process to execute on this wizard panel. For more information, refer to the Document Engine documentation.

Reset Current File to Selections? reset.to.selections	If selected (set to true), replaces the current file variable ($L.file) with the value selected by the user ($L.selection). **Note** Note This field depends on the selections in the Usage tab. If "Select one record from list" or "Select multiple records from list" are chosen, those options generate $L.selection.
Restart Panel if restart.condition	Under some circumstances it may be necessary to run the wizard panel again. For example, an expected field may not have been filled in correctly, or an error code may have been returned from Format Control or the process. Enter a condition that evaluates to true or false to determine if this panel needs to be rerun.
Display Record(s) when complete? display.when.complete	Determines whether to display the record to the user when the wizard completes. If the condition is true, Service Manager displays the record to the user when complete. If the condition is false, no record is displayed.
Mode display.mode	This field determines how a record will be displayed to the user. • **Browse**: (This is the default.) Browse mode doesn't have any buttons to store a modified record. • **Add**: The Add button becomes available. • **Update**: The Save button becomes available.
Return Current File to calling application? reset.current.file	This option returns the current record to the application that called the wizard for further processing.

8.2.8. Messaging tab field definitions

Use this tab to set any messages to be displayed to the end user when specified conditions occur.

Field	Description
Message message	Message to be displayed when a certain condition occurs. This field will also take a message number from the scmessage database. For example: `scmsg(106,"fc").`
Condition message.cond	Enter a condition for the message that evaluates to true or false to determine if the message should be issued. For example: `not nullsub($L.finish,false)`
Type message.type	Select the format of the message to display to users (pop-up or on- screen). The default is on-screen.
Level message.level	Select the level of severity of the message: • **Info:** Informational only *(default)* • **Action needed** • **Error message:** The error message issued to the user when a serious error is encountered.

8.2.9. Variables tab field definitions

Use this tab to document the variables used in the wizard workflow.

Field	Description
Wizard variables **wizard.variables**	Use this tab to define variables for use within the wizard. Variables can be passed to applications or formats being created by the wizard, but must be assigned. For example, a variable such as $L.return.action must be defined here, if it will be used within the wizard. The data type and value of a variable can be different at different times, and can have a primitive or compound data type as its value. Service Manager contains three types of variables: 1) local, 2) global, and 3) thread. Local variables begin with $L and persist only within the currently executing RAD application. The server automatically cleans up local variables when it exits a RAD application.

8.2.10. Next Wizard tab field definitions

Use this tab to specify the next wizard panel (if any) to be called.

Field	Description
Wizard Name **next.wizard**	Name of the next wizard to execute within the workflow (series of wizard panels).
Condition **next.wizard.cond**	Enter a condition that evaluates to true or false. The next wizard in the workflow is determined by the first condition that evaluates to true, regardless of the conditions that follow (even if they evaluate to true as well). If all conditions evaluate to false, the wizard workflow will exit and finish.

8.2.11. Previous Wizard tab field definitions

This tab is displayed when you check the **Use Conditional Previous Exits?** checkbox in the Usage tab.

Field	Description
Wizard Name **prev.wizard**	Specify the name of the wizard to which the user should exit.
Condition **prev.wizard.cond**	Set the condition for when the user should be taken to this previous wizard. The condition must evaluate to true or false.

8.2.12. Comments tab field definitions

Use this tab to enter internal development comments about the wizard. These comments have no effect on the wizard workflow itself.

Field	Description
Comments **comments**	Enter any developer comments concerning the wizard here. As a best practice, use the Comments tab to describe details, such as the purpose of the wizard panel, the settings selected to implement that purpose, variables used, wizard workflow, and exit conditions.

8.2.13. Cancel Expressions tab field definitions

Enter system language expressions or JavaScript code that will execute if the end user clicks the Cancel button. This cleans up variables that were used in the wizard to ensure correct behavior the next time the wizard is executed.

Field	Description
Expressions Executed on Cancel subtab **cancel.expressions**	Enter any expressions that will execute when a user cancels the wizard process. This provides the wizard creator an opportunity to reset values or clean up variables initialized in the wizard.
JavaScript Executed on Cancel subtab **javascript.cancel**	Enter any JavaScript code that should be used to reset values and clean up variables that were initialized in the wizard.

8.3. Calling Wizards

The RAD application in charge of executing wizards is called **wizard.run**. You can call this application from:

- Menus
- Display Options
- Format Control
- Process Records

The sections following this one explain the steps required in calling wizards from these areas.

Wizards themselves can call:

- Format Control records
- Processes
- Other wizards.

Wizards don't allow direct calls to RAD applications.



Note

Warning: It is possible to send a wizard into an infinite loop. For example, this would happen if you called `wizard.run` from a process to execute a wizard that in turn calls the originating process.

8.3.1. Call a wizard from menus

To set up a wizard to be called from menus, follow these steps:

1. Click **Tailoring** > **Tailoring Tools** > **Menus**.
2. Select a menu record.
3. Add an option number; a group, if necessary; and a description with the name that you want to display on the menu.
4. Enter wizard.run for the application.
5. In the Parameter Name field, specify the name of the parameters to pass to the application. The name of the parameters to pass to the application should be enclosed with quotes "" and curly brackets {}. The available parameters for the wizard.run application are: **file**, **name**, **text**, and **prompt**.

6. In the Parameter Value field, specify the value of the parameter being passed, enclosed with quotes "" and curly brackets {}. For example, {"createUser Pre"}.

Example:

In this example, a wizard is set up to be called from the **SYSTEM ADMINISTRATION** menu through the option **Quick Add Utility**. Once the wizard is set up, the user clicks the following:

System Administration > **Ongoing Maintenance** > **User Quick Add Utility**

When the user clicks **User Quick Add Utility** on the menu, a wizard opens with a predefined set of steps that assist the user in populating information and completing the task of adding an operator.

To see how this wizard has been set up to be called from menus, follow these steps:

1. Click **Tailoring** > **Tailoring Tools** > **Menus**.
2. Open the SYSTEM ADMINISTRATION menu record.
3. The Description field displays **User Quick Add Utility**.
4. The Application field specifies **wizard.run**, the RAD application called from the menu.
5. The Parameter Name field specifies the name of a single parameter to pass to the menu item. In this example, {"name"} represents the name of the wizard. The name is enclosed with quotes "" and curly brackets {}.
6. The Parameter Value field specifies the value of the parameter being passed. In this example, {"createUser Pre"}. The name is enclosed with quotes "" and curly brackets {}.

8.3.2. Call a wizard from display options

To set up a wizard to be called from display options, follow these steps:

1. Click **Tailoring** > **Tailoring Tools** > **Display Options**.
2. Select the display option that you want to use to call a wizard.
3. Select the **Rad** tab.
4. In the RAD Application field, enter the RAD Application **wizard.run**.
5. The Names field specifies the parameters to pass to the RAD application. The available parameters are: file, name, text, and prompt. The **name** parameter is required.
6. The Values field specifies the values of the parameters being passed to the RAD application.
7. Click **Save**.
8. Click **OK**.

Example: Set up a wizard to be called from display options

In this example, a wizard is set up to be called from the display application **wizard.display.kmsearch**, so that when an operator selects the "Search Again" option, the wizard "KM choose shared content" is called.

To set up a wizard to be called from display options, follow these steps:

1. Click **Tailoring** > **Tailoring Tools** > **Display Options**.
2. Select the **wizard.display.kmsearch**, Text Option definition 7500.
3. Select the **Rad** tab.
4. In the RAD Application field, enter the RAD Application wizard.run.
5. The Names field specifies the parameters to pass to the RAD application. The available parameters are: file, name, text, and prompt. The **name** parameter is required.
6. In this example, the following values are specified for each parameter named.

Name of parameter	Value of the parameter
file	$L.file
name	KM choose shared content

7. Click **Save**.
8. Click **OK**.

8.3.3. Call a wizard from Format Control

To call a wizard from Format Control, follow these steps:

1. Click **Tailoring** > **Format Control**.
2. Select the Format Control where you want to call a wizard.
3. Click **Subroutines**.
   The Format Control Maintenance - Subroutines form opens.
4. In the Application field, enter the application wizard.run.
5. In the Names field, enter the name of the parameter(s) to pass to the subroutine application (name, file, text, and prompt).
6. In the Values field, enter the parameters to pass to the subroutine application. The type of parameters in this field must match the type of parameters in the Names field.
7. Specify the condition for each parameter. When a condition is set to "true" and a user clicks Save, Format Control runs every section or expression where the Update field is set to true before it updates the record.

8.3.4. Call a wizard from process records

To set up a wizard to be called from process records, follow these steps:

1. Click **Tailoring** > **Document Engine** > **Processes**.
2. Select a process record.
3. Select the **RAD** tab.
4. In the RAD Application field, enter the RAD Application wizard.run to associate the wizard with this record.
5. In the Condition field, enter the condition associated with the RAD Application field. For example, "true".
6. In the Parameter Names field, enter the parameter names to pass to the RAD application.
7. In the Values field, enter the parameter values to pass to the RAD application.
8. Click **Save**.
9. Click **OK**.

Example:

In this example, a wizard is set up to be called from the add.device Process record. The add.device Process record is opened to evaluate the RAD call.

To set up a wizard to be called from process records, follow these steps:

1. Click **Tailoring** > **Document Engine** > **Processes**.
2. Open the **add.device** Process record.
3. Select the **RAD** tab.
4. In the RAD Application field, enter the RAD Application wizard.run to associate the wizard with this record.
5. The Condition field contains the name of the parameters to be used (file and name).
6. In the Parameter Names field, enter the parameter names to pass to the RAD application.
7. The Values field specifies the values of the parameters being passed to the RAD application. In this example, the following values are specified for each parameter named.

Name of parameter	Value of the parameter
file	$L.file
name	"Add Device"

8. Click **Save**.
9. Click **OK**.

8.4. Example: Creating a Wizard that creates a new SLA

In this example, you will create a wizard that defines a new SLA record by prompting the user to enter the necessary data for a new SLA and then saving that data to the database.

Out of box, a Service Manager user creates a new SLA by clicking **Service Level Management > Service Level Agreements > Create New SLA**. The user then fills in the required fields and any relevant optional fields in the Service Level Agreement entry form.

The new wizard will guide users through supplying the necessary SLA record information by presenting a series of wizard panels. When all the information has been supplied by the user, the wizard process saves the SLA record to the database.

8.4.1. Adding Service Level Targets to the SLA

SLAs generally have Service Level Targets (SLTs) associated with them. Service Manager includes two out-of-box wizards for adding SLTs: Response SLT and Availability SLT.

After creating an SLA by filling out the Service Level Agreement form, the user can then go to the SLA record and click the **Add SLT** buttons in the **Response** and **Availability** tabs to launch the out-of-box Add Response and Add Availability SLT wizards. The Create SLA wizard will call the out-of-box Add SLT wizards so that the user can go seamlessly from creating a new SLA to adding any SLTs related to it.

8.4.2. Designing the SLA wizard

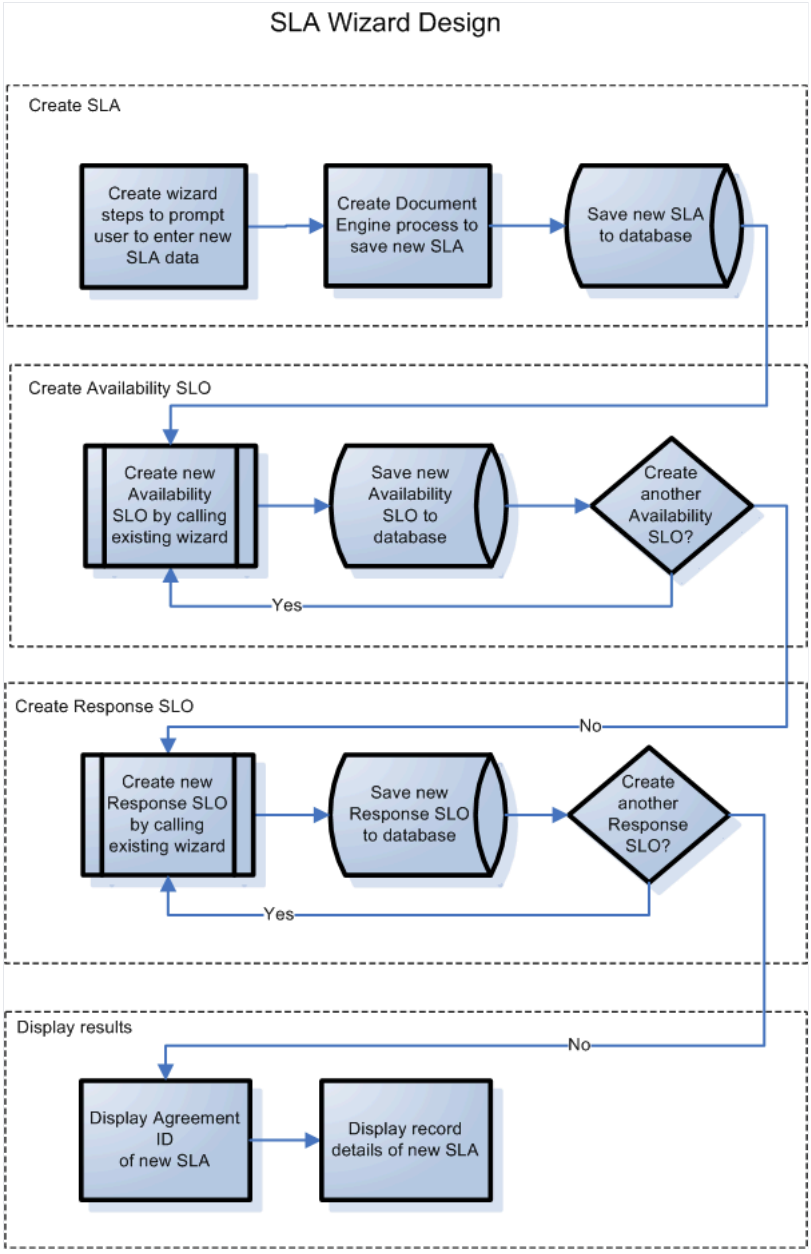
It is helpful to design a new wizard by considering the overall process flow that must occur, including both what the end user needs to do, and any background processes that need to take place.

In this case, we will need to create a series of consecutive wizard steps or panels to display to users to prompt them to enter the basic information needed for the new SLA record. When the required information has been gathered, the new SLA record must be saved by a Document Engine process to the database.

The wizard must then ask users if they want to add any availability or response SLTs. If they do, it will call the out-of-box SLT wizards, first for availability SLTs, and then for response SLTs.

Finally the wizard should display the record number and title of the newly-created SLA, and then provide a button for the user to view the record details.

The overall design of the new Create SLA wizard is shown in the following flowchart.



8.4.3. Planning the SLA wizard panels

In addition to the high-level wizard design, it is helpful to plan the function of each of the wizard panels.The general purpose of the wizard panels will be to prompt the user to enter the required fields for an SLA record:

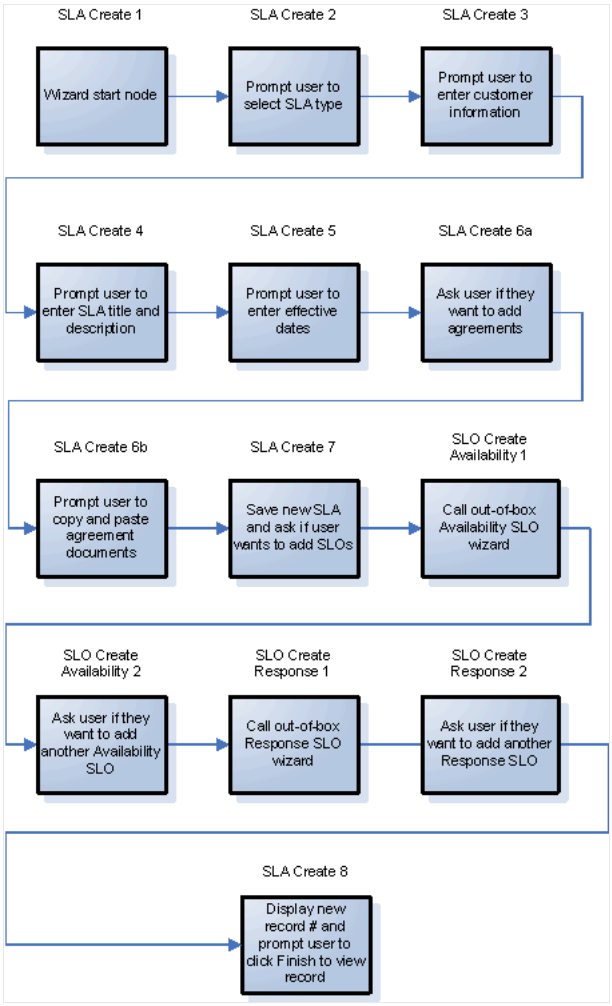
- SLA type (service or customer)
- Customer
- Title and description
- SLA effective dates

In addition, although it's not required, many users will want to copy and paste from the Agreement documents into the SLA record , so the wizard will also present panels to prompt the user for that information.

The names of the wizard panels will be consecutive (SLA Create 1, SLA Create 2, and so on) so that it is clear that these individual wizard panels connect together to form the Create SLA wizard that will be displayed to the end user.

The flowchart below shows the planned wizard panels and the function that each will perform. Each of the boxes in the flowchart also corresponds to the steps required to build the SLA wizard, described in the Building the SLA wizard section.

SLA Wizard Panel Names and their Associated Functions



8.4.4. Building the Create SLA wizard

This example describes how to use the Wizard tool to design each panel of the Create SLA wizard. Most of the panels involve prompting the user to provide information, so they also require us to use the Forms Designer to design the form controls for the subformat that's displayed within the wizard panel.

Each of the overview steps listed below corresponds to a box on the flowchart in the previous section. These steps, along with the specific tasks that need to be done using the Wizard Tool and Forms Designer, are listed below.

Step 1: Define the start node of the wizard

Step 2: Prompt the user to select SLA type

Step 3: Prompt for customer information

Step 4: Prompt for SLA title and description

Step 5: Prompt for effective dates

Step 6: Ask about agreement documents

Step 7: Prompt to add agreements

Step 8: Save new SLA and prompt for SLTs

Step 9: Call the Availability SLT wizard

Step 10: Prompt for another Availability SLT

Step 11: Call the Response SLT wizard

Step 12: Prompt for another Response SLT

Step 13: Display new SLA number and title

Step 14: Set menu to call the new SLA wizard

8.4.4.1. Step 1: Define the start node of the wizard

This first task creates the first wizard panel the user sees when executing the Create New SLA wizard. You use the Wizard Tool to define this first wizard panel.

To create a wizard to prompt for SLA type, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 1, and then click **Add**.

> **Caution**
>
> Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Select this checkbox to set to **true**, as this wizard is the first in a series of wizard panels.
Brief Description	Type: This wizard will create an SLA record and guide the user through creating associated SLTs.
Window Title	Type: Create New SLA
Title	Type: Create New SLA

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following fields:

Field	Value
Create a record	Select this option to indicate that the wizard will create a record. You must enter or select the record type in the record "of type" field. This becomes $L.file.
of type (for Create a record)	Click **Fill** to select the type of record to be created, and select **sla**.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.type

7. Select the **Next Wizard** tab and complete the following fields:

Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLACreate 2
Condition	The condition determines what happens next. In this case, you can enter the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizards, based on user actions or selections. Type: true

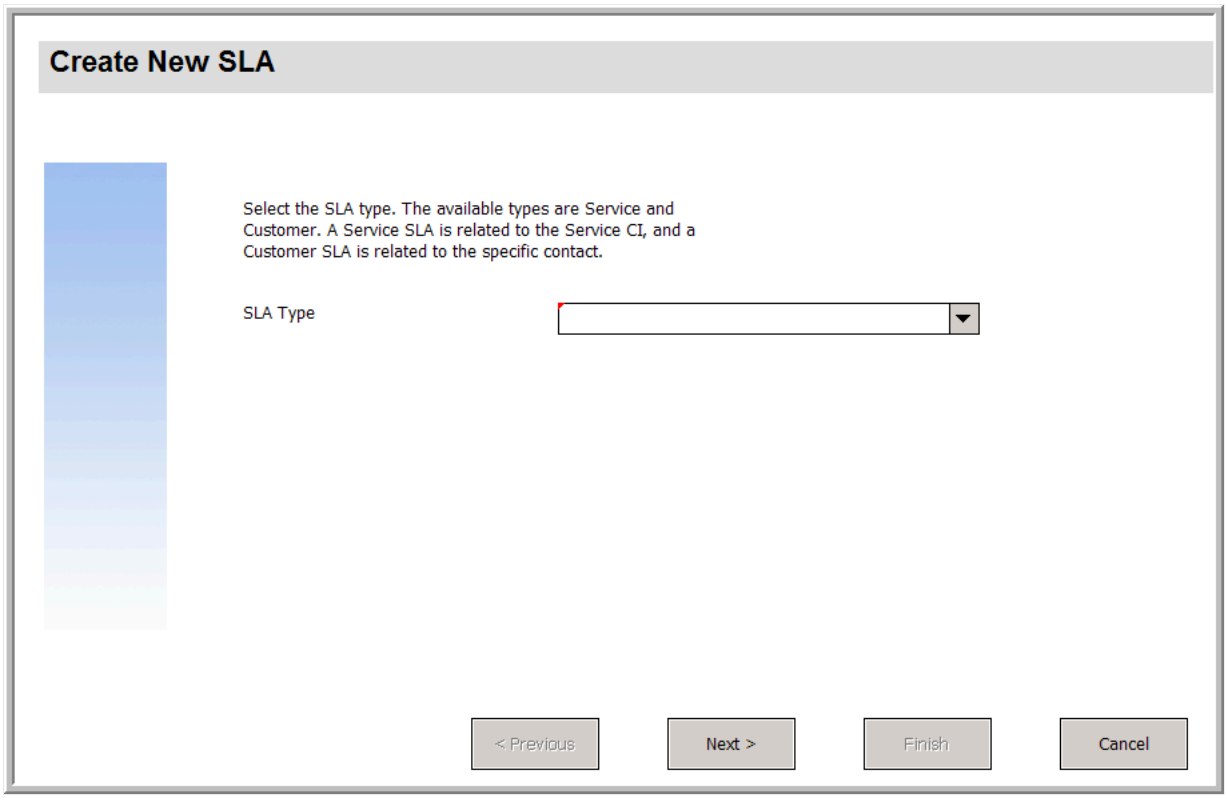
8. When you are finished, click **Save**.

You have created the first wizard panel. However to improve the appearance and behavior of the panel you will use Forms Designer to create the subformat to display in this wizard panel.

8.4.4.2. Step 2: Prompt the user to select SLA type

The purpose of this step is to refine the first wizard panel to prompt the user to select the SLA type: Service or Customer.

Use the Forms Designer to design the subformat that's displayed in the wizard panel:



For this task, you will use Forms Designer to create the form controls that prompt the user to select the SLA type. These form controls, together with the window title, prompt, buttons, and other choices you made in the wizard record, will be displayed to the end user as the screen shown above.

To create a new form for the SLA type wizard, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.type.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This helps avoid spelling errors.
3. Click **New**.

4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Wrap Label	
Caption	Type: Select the SLA type. The available types are Service and Customer. A Service SLA is related to the Service CI, and a Customer SLA is related to the specific contract.
Visible	Select.
Label	
Caption	Type: SLA Type :
Visible	Select.
Combo Box	The Combo Box should be place to the right of the Regular label box and contains a list of choices that are associated with the label.
Display List	Type two entries (each with an initial capital letter): ○ **Service** ○ **Customer**
Input	Type: type
Mandatory	Select.
Select Only	Select.
Value List	Type two entries (all lowercase letters): ○ service ○ customer
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have completed creating the form for the first panel of the Create New SLA wizard.

8.4.4.3. Step 3: Prompt for customer information

The purpose of this step is to create the panel prompting the user to enter customer, service contract, and service hours.



Note

Note You can use the record you just created, SLA Create 1, as a starting point, and revise the fields to reflect the information below. It is very important to remember to click the **Add** button after you start to make changes so that you don't overwrite the SLA Create 1 record.

Perform the following two tasks to design a wizard panel that gathers customer information.

Task 1: Create a wizard to gather customer information[edit]

For this task, you will create the next wizard panel the users will see, prompting them to enter customer, service contract, and service hours.

To create a wizard to gather customer information, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 2, and then click **Add**.

   Caution

   Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard panel prompts for the customer name, service contract, and service hours.
Window Title	Type: Create New SLA - Customer Information
Title	Type: Create New SLA - Customer Information

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following field:

Field	Value
$L.file passed in	Select this option to indicate that the $L.file variable should be passed to the wizard from a previous wizard.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run.
7. Select the **Actions** tab and the **Format Control/Process Name** subtab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that's being performed. This can either be the current file, the list selected (as a whole) or an action to be performed on every single record selected. Select **Current File ($L.file)**.

8. Select the **Next Wizard** tab and complete the following fields:

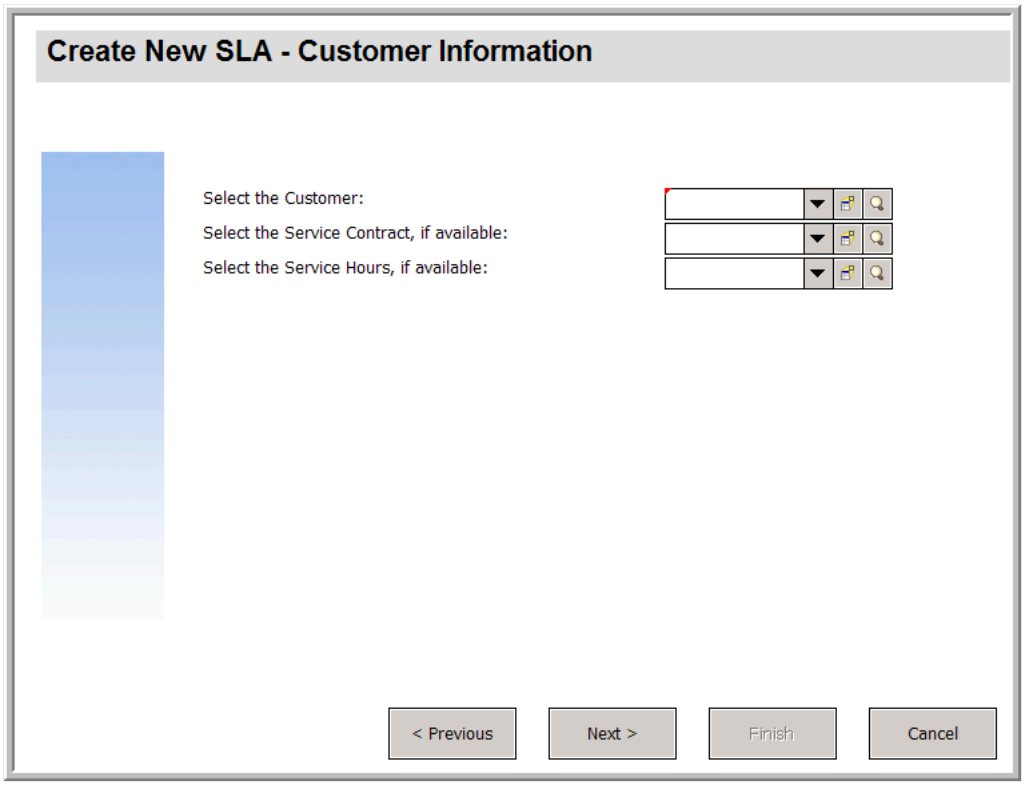
Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLACreate 3
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizards, based on user actions or selections. Type: true

9. When you are finished, click **Save**.

You have created the customer information wizard panel. Next, you will create the new wizard form in Forms Designer.

Task 2: Create a form to gather customer information[edit]

For this task, you will use Forms Designer to create controls that prompt the user to supply basic customer information. These form controls, together with the window title, prompt, buttons, and other choices you made in the wizard record, will be displayed to the end user as the screen shown below.



To create a new SLA form to gather customer information,follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.customer.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This will help you to avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Label	
Caption	Type: Select the customer:
Visible	Select.
Comfill	
Combo Button Visible	Select.
Input	customer
Mandatory	Select.
Select Only	Select.
Third Button Visible	Select.
Value List	Type: $lo.colist
Visible	Select.
Label	
Caption	Type: Select the Service Contract, if available:
Visible	Select.
Comfill	
Combo Button Visible	Select.
Fill Button Visible	Select.
Input	Type: service.contract
Select Only	Select.
Third Button Visible	Select.
Value List Condition	Type: select("contract.id","servicecontract","provider",[customer])
Visible	Select.
Label	
Caption	Type: Select the Service Hours, if available:
Visible	Select.
Comfill	
Combo Button Visible	Select.
Fill Button Visible	Select.
Input	Type: service.hours

Select Only	Select.
Third Button Visible	Select.
Value List	Type: $G.calendars
Visible	Select.
Display List Condition	select("name","servicecontract","provider",[customer])

6. Click **OK** twice to save the new form and exit.

You have completed creating one of the forms needed for the SLA wizard.

Contents

- 1 Task 1: Create a wizard to gather customer information
- 2 Task 2: Create a form to gather customer information

8.4.4.4. Step 4: Prompt for SLA title and description

The purpose of this step is to create the next wizard step, prompting the user to enter a title and description for the SLA.

Perform the following two tasks to design a wizard panel for the user to enter an SLA title and description.

Task 1: Create a wizard for SLA description[edit]

For this task, you will create a wizard panel that prompts the user to enter a title and description for the new SLA.

To create a wizard panel for the SLA title and description, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 3, and then click **Add**.

> **Caution**
> Caution If you use an existing SLA record to create a new SLA wizard panel, click "Add" rather than save. If you click Save, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard panel will prompt the user to enter a title and description for the SLA.
Window Title	Type: Create New SLA - Description
Title	Type: Create New SLA - Description

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following field:

Field	Value
$L.file passed in	Select this option to indicate that the $L.file variable should be passed to the wizard from a previous wizard.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard runs.

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.description

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that's going to be performed. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.

8. Select the **Next Wizard** tab and complete the following fields:

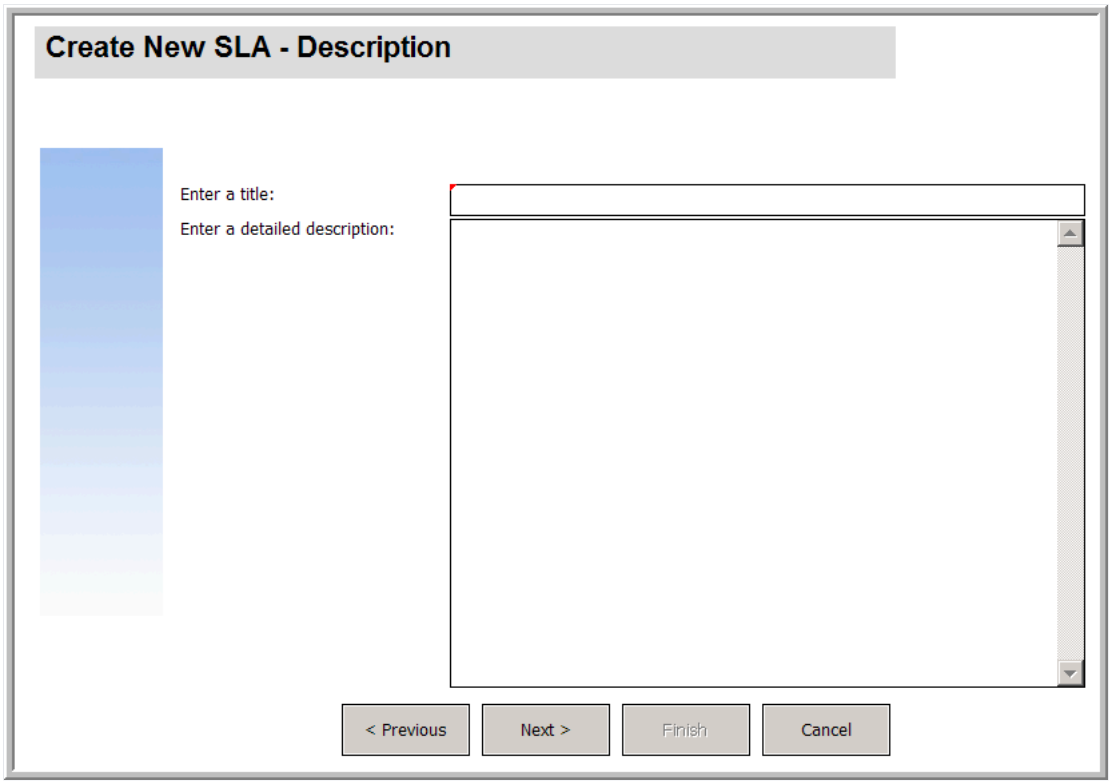
Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLACreate 4
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizards, based on user actions or selections. Type: true

9. When you are finished, click **Save**.

You have created the new wizard panel. Next, you will create the new wizard form in Forms Designer.

Task 2: Create a form for SLA description[edit]

For this task, you will use Forms Designer to create controls that prompt the user to enter a title and description. These form controls, together with the window title, prompt, buttons, and other choices you made in the wizard record, will be displayed to the end user as the screen shown below.



To create a new form to display the SLA description wizard, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.description.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This will help you to avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Label	
Caption	Type: Enter a title.
Visible	Select.
Text	
Input	Type: title
Visible	Select.
Label	
Caption	Type: Enter a detailed description.
Visible	Select.
Text Area	
Input	Type: description
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have created one of the forms needed for the SLA wizard.

Contents

- 1 Task 1: Create a wizard for SLA description
- 2 Task 2: Create a form for SLA description

8.4.4.5. Step 5: Prompt for effective dates

The purpose of this step is to create the next wizard step, prompting the user to enter the starting and ending dates the SLA.

Perform the following two tasks to design a wizard panel for the user to indicate the effective dates of the SLA:

Task 1: Create a wizard for SLA effective dates[edit]

For this task, you will create a wizard that prompts the user to enter valid start and end dates.

To create a wizard for valid start and end dates, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 4, and then click **Add**.

> **Caution**
> Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard panel will prompt the user to enter the beginning and ending dates of the SLA.
Window Title	Type: Create New SLA - Effective Dates
Title	Type: Create New SLA - Effective Dates

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following fields:

Field	Value
$L.file passed in	Select this option to indicate that the $L.file variable should be passed to the wizard from a previous wizard.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.validfromto

7. Select the **Actions** tab and the **Format Control/Process Name** subtab to define the actions when a user clicks Next, Cancel, or Previous within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that's going to be performed. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.

8. Select the **Next Wizard** tab and complete the following fields:

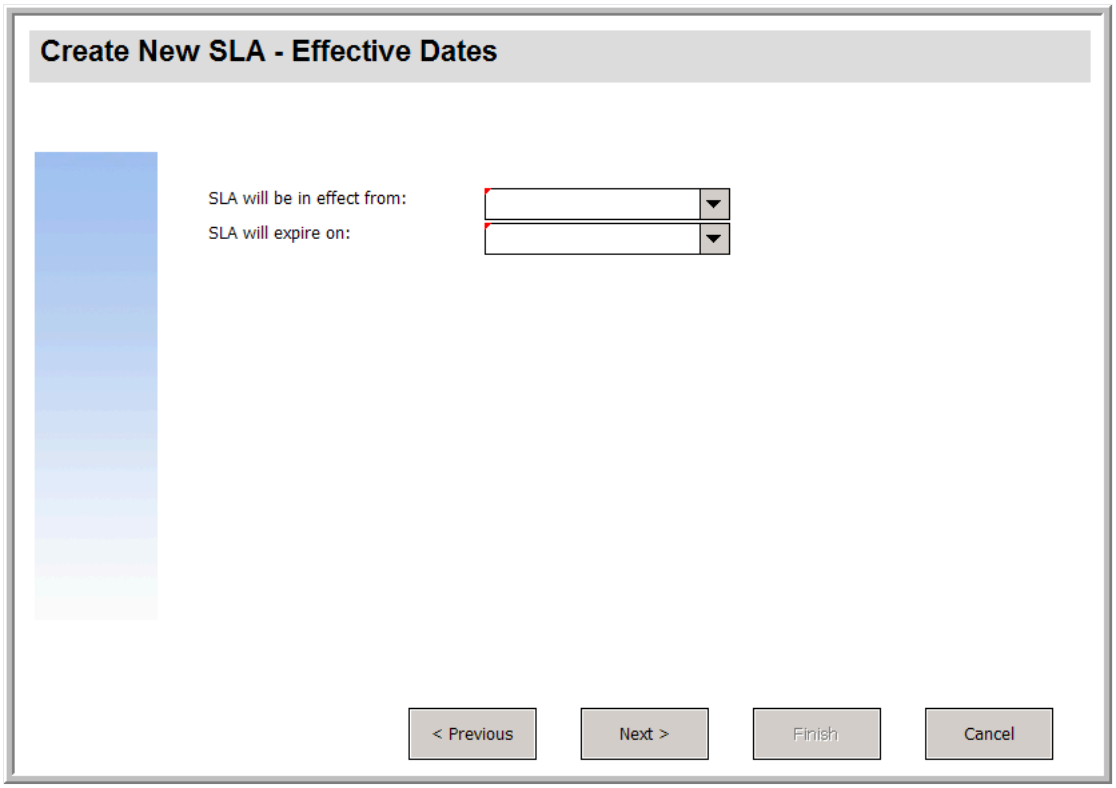
Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLACreate 5
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizards, based on user actions or selections. Type: true

9. When you are finished, click **Save**.

You have created the valid From and To dates wizard panel. Next, you will create the new wizard form in Forms Designer.

Task 2: Create a form for SLA effective dates[edit]

For this task, you will use Forms Designer to create the form controls that prompt the user to indicate the effective dates of the SLA. These form controls, together with the window title, prompt, buttons, and other choices you made in the wizard record, will be displayed to the end user as the screen shown below.



To create a new form for the SLA effective dates wizard, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.validfromto.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This will help you to avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Label	
Caption	Type: SLA will be in effect from:
Visible	Select.
Comfill	
Fill Button Visible	Select.
Input	start
Mandatory	Select.
Visible	Select.
Label	
Caption	Type: SLA will expire on:
Visible	Select.
Comfill	
Fill Button Visible	Select.
Input	Type: expiration
Mandatory	Select.
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have finished creating one of the forms needed for the SLA wizard.

Contents

- 1 Task 1: Create a wizard for SLA effective dates
- 2 Task 2: Create a form for SLA effective dates

8.4.4.6. Step 6: Ask about agreement documents

The purpose of this step is to create a wizard panel that asks the user whether they want to copy text from an agreement document.

Perform the following two tasks to design a wizard panel asks users about agreements and other documents related to the SLA.

>

Task 1: Create a wizard to ask about agreements[edit]

The purpose of this task is to create a wizard panel that asks users if they want to copy text from agreement documents to this SLA.

To create the panel, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 5, and then click **Add**.

> **Caution**
> Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard will ask the user whether they want to copy text from Agreement documents or attach other documents to the SLA.
Window Title	Type: Create New SLA - Copy Agreements?
Title	Type: Create New SLA - Copy Agreements?

5. Select the **File Selection** tab to complete the following fields:

Field/Subtab	Value
Initial Expressions subtab	Type: $L.agreement.docs=false
Select $L.file by subtab	Select the **$L.file passed in** option to indicate that the $L.file variable should be passed to the wizard from a previous wizard.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.ask.agreements

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.

8. Select the **Next Wizard** tab and complete the following fields:

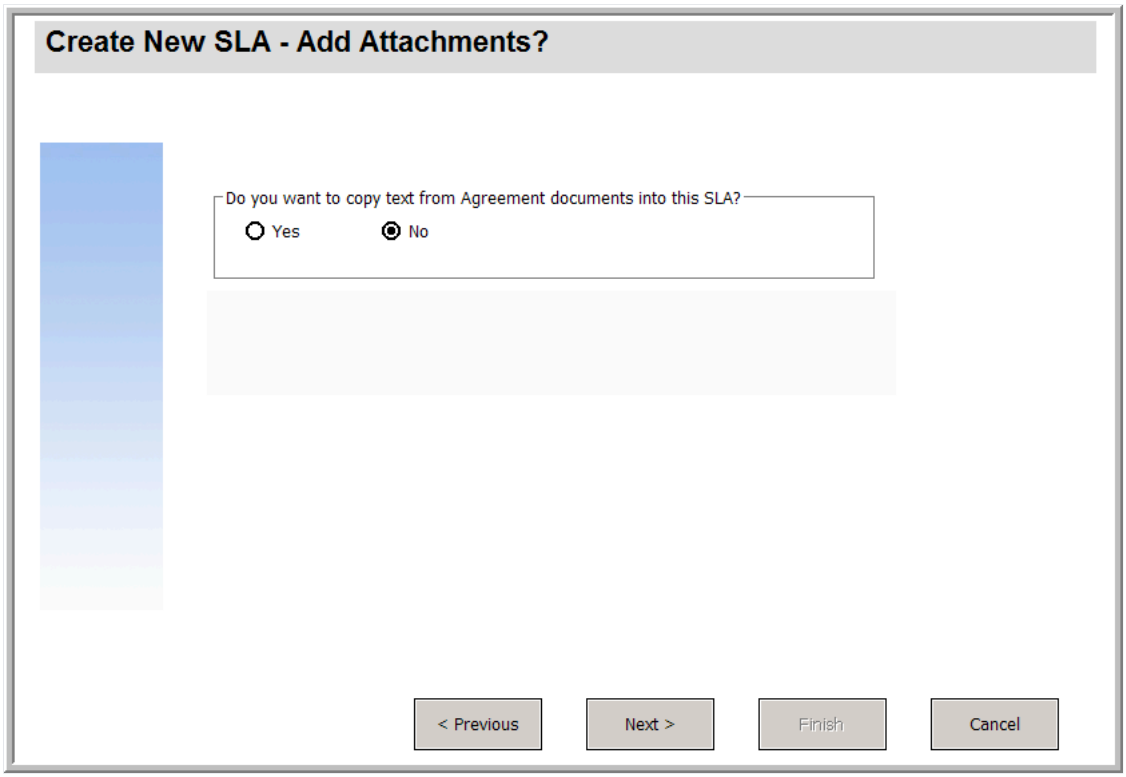
Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create 6a
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizazrds, based on user actions or selections. Type: $L.agreement.docs=true
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create 7
Condition	Type: true

9. When you are finished, click **Save**.

This completes the wizard panel that asks users about attachments. The next step creates the corresponding new wizard form in Forms Designer.

Task 2: Create a form to ask about agreement[edit]

For this task, you will use Forms Designer to create the form controls that ask users to whether they want top copy text from agreements to the SLA. These form controls, together with the window title, prompt, buttons, and other choices you made in the wizard record, will be displayed to the end user as the screen shown below.



To create a new form for the SLA wizard that asks about agreements, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.ask.agreements.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This will help you to avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Group	
Caption	Type: Do you want to copy text from Agreement documents into this SLA?
Visible	Select.
Radio Button	
Caption	Type: Yes
Input	Type: $L.agreement.docs
Value	Type: true
Visible	Select.
Radio Button	
Caption	Type: No
Input	Type: $L.agreement.docs
Value	Type: false
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have finished creating one of the forms needed for the SLA wizard.

Contents

- 1 Task 1: Create a wizard to ask about agreements
- 2 Task 2: Create a form to ask about agreement

8.4.4.7. Step 7: Prompt to add agreements

The purpose of this step is to create the wizard panel that prompts the user to add agreements related to the SLA if they answered Yes on the previous panel. The user does this by copying and pasting text into the entry field.

Perform the following two tasks to design a wizard panel to prompt the user to add agreements.

Task 1: Create a wizard to add agreements

For this task, you will create a wizard panel that prompts users to copy and paste text from agreements related to the SLA, if they answered Yes in the previous panel.

To create a wizard panel that prompts users to attach agreement documents, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 6a, and then click **Add**.

> **Caution**
> Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard will prompt the user to copy and paste text from Agreement documents.
Window Title	Type: Create New SLA - Agreement Documents
Title	Type: Create New SLA - Agreement Documents

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following field:

Field	Value
$L.file passed in	Select this option to indicate that the $L.file variable should be passed to the wizard from a previous wizard.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.add.agreements

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.

8. Select the **Next Wizard** tab and complete the following fields:

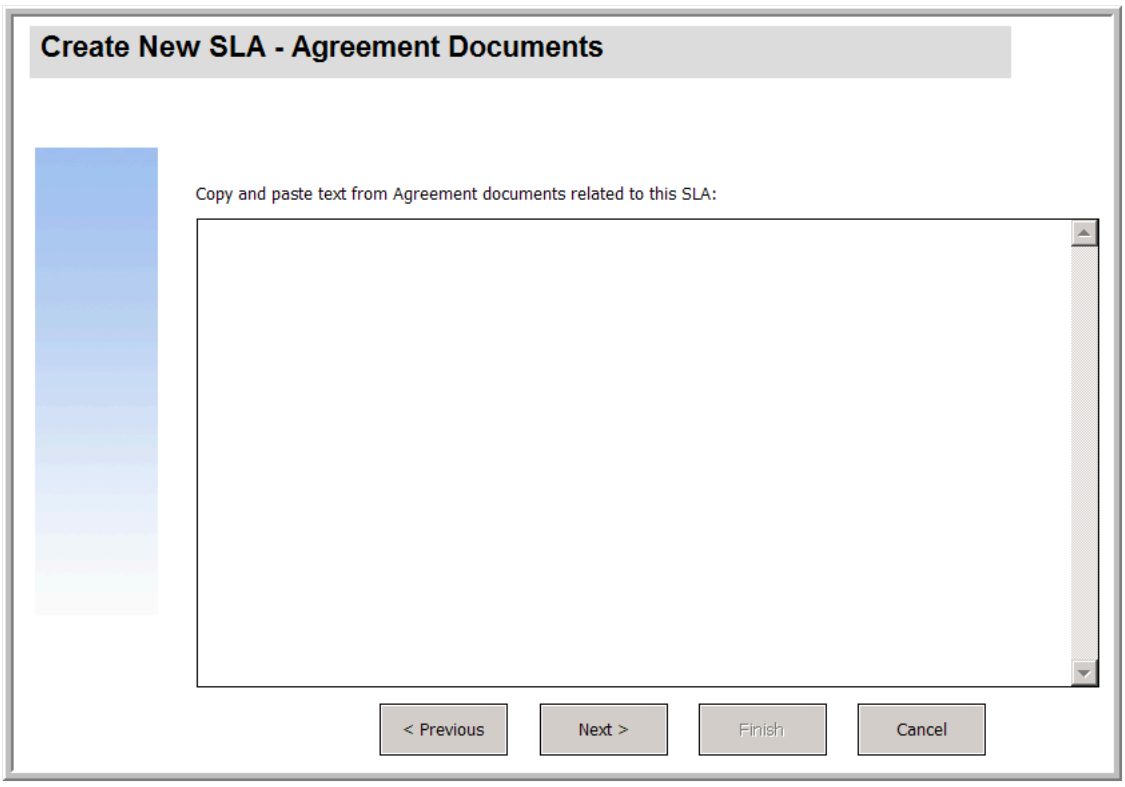
Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create 7
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizards, based on user actions or selections. Type: true

9. When you are finished, click **Save**.

You have created the wizard panel that prompts users to attach agreement documents. Next, you will create the new wizard form in Forms Designer.

Task 2: Create a form to add agreements

For this task, you will use Forms Designer to create the form controls that prompt the user to copy and paste text from agreement documents related to the SLA. These form controls, together with the window title, prompt, buttons, and other choices you made in the wizard record, will be displayed to the end user as the screen shown below.



To create a new form to display the wizard that prompts users to add agreement document text, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.add.agreements.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This will help you to avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Label	
Caption	Type: Copy and paste text from Agreement documents related to this SLA:
Visible	Select.
Text Area	
Input	Type: agreements
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have finished creating one of the forms needed for the SLA wizard.

8.4.4.8. Step 8: Save new SLA and prompt for SLOs

At this point, all of the information needed for the new SLA has been collected. The next task is create a process in Document Engine to save the new SLA to the database.

Additionally this task asks the user if they want to add SLTs to the SLA.

Perform the following three tasks to complete this step.

Task 1: Create a process to save the new SLA[edit]

For this task, you will create a process to save the new SLA record that you just created.

To create a process to save the new SLA record, follow these steps:

1. Click **Tailoring** > **Document Engine** > **Processes**. The Process Definition form opens.
2. In the Process Name field, type sla.save.from.wizard.
3. In the RAD tab, type the following expressions in the first two rows of the **Expressions evaluated before RAD call** section:
   $L.action="add"
4. In the **RAD Application** section of the RAD tab, type se.view.engine as the RAD application to be called, and then type true as the condition to execute it. Then enter the following parameter names and values:

Parameter Name	Parameter Value
file	Type: $L.file
description	$L.action

5. In the **Post RAD Expressions** section of the RAD tab, type $L.new.sla=agreement.id in $L.file.
6. Click **Add**.
7. When you are finished, click **OK**.

You have created the process to save the new SLA record to the database.

Task 2: Create a wizard to check SLT requirements[edit]

For this task, you will create a wizard to call the save process, and then create a subformat that asks users whether they want to add SLTs and check the SLT requirements.

To create a wizard to call the save process and check SLT requirements, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 7, and then click **Add**.

> **Caution**
> Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard panel will ask the user if they want to create Availability or Response SLTs.
Window Title	Type: Create New SLA - Check SLT Requirements
Title	Type: Create New SLA - Check SLT Requirements

5. Select the **File Selection** tab to complete the following fields:

Field/Subtab	Value
Initial Expressions subtab	Type: $L.create.response.slo=false $L.create.avail.slo=false
Select $L.file by subtab	Select the **$L.file passed in** option to indicate that the $L.file variable should be passed to the wizard from a previous wizard.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.ask.slos

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.
Format Control/Process Name subtab	Define the actions when a user clicks Next, Cancel, or Previous within the wizard. Complete the following field:
Process Name	To specify that this wizard can run a process after completing, type: sla.save.from.wizard
Wizard Variables	**$L.create.response.slo** $L.crate.avail.slo $L.new.sla

8. Select the **Next Wizard** tab and complete the following fields:

Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create Response 1
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizazrds, based on user actions or selections. Type: $L.create.response.slo=true
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create Availability 1
Condition	Type: $L.create.avail.slo=true
Wizard Name	Type: SLA Create 8
Condition	Type: true

9. When you are finished, click **Save**.

You have created the wizard panel that calls the save process and checks SLT requirements. Next, you will create the new wizard form in Forms Designer.

Task 3: Create a form to check SLT requirements[edit]

For this task, you will use Forms Designer to create form controls that ask users if they want to create Service Level Targets related to the SLA.

To create a new form for the wizard that calls the save process and checks SLT requirements:

1. Start the Forms Designer.
2. In the Form field, type sla.create.ask.slos.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This helps avoid spelling errors.

3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Group	
Caption	Type: Do you want to create Service Level Targets based on Help Desk response times?
Visible	Select.
Radio Button	
Caption	Type: Yes
Input	Type: $L.create.response.slo
Value	Type: true
Visible	Select.
Radio Button	
Caption	Type: No
Input	Type: $L.create.response.slo
Value	Type: false
Visible	Select.
Group	
Caption	Type: Do you want to create Service Level Targets based on service availability?
Visible	Select.
Radio Button	
Caption	Type: Yes
Input	Type: $L.create.avail.slo
Value	Type: true
Visible	Select.
Radio Button	
Caption	Type: No
Input	Type: $L.create.avail.slo

Value	Type: false
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have finished creating one of the forms needed for the SLA wizard.

Contents

- 1 Task 1: Create a process to save the new SLA
- 2 Task 2: Create a wizard to check SLT requirements
- 3 Task 3: Create a form to check SLT requirements

8.4.4.9. Step 9: Call the Availability SLT wizard

The next step is to create a wizard panel to call the existing out-of-box Availability Wizard for those users who answered that they did want an availability SLT.

Because this wizard panel only performs the background task of calling another wizard, there is no screen to show the end user. Thus there is no task requiring the use of Forms Designer in this step.

Task: Create a wizard to call an Availability SLT[edit]

For this task, you will create a wizard that will call the existing out-of-box Availability SLT wizard.

To create a wizard that calls the existing out-of-box Availability SLT wizard, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create Availability 1, and then click **Add**.

   Caution

   Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard calls the existing wizard to create an Availability SLT.
Window Title	Type: Create Availability SLT
Title	Type: Create Availability SLT

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following field:

Field	Value
$L.file passed in	checked

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Skip Display	Select this option as this panel doesn't include a form that needs to be displayed to the user.

7. Select the **Actions** tab to define the actions when a user clicks Next, Cancel, or Previous within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.
Format Control/Process Name subtab	Define the actions when a user clicks Next, Cancel, or Previous within the wizard. Complete the following field:
Process Name	To specify that this wizard can run a process after completing, type: slo.add.avail.sla

8. Select the **Next Wizard** tab and complete the following fields:

Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create Availability 2
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizazrds, based on user actions or selections. Type: true

9. When you are finished, click **Save**.

You have created a wizard panel that calls the existing out-of-box Availability SLT. A new form is not needed for this wizard.

Contents

- 1 Task: Create a wizard to call an Availability SLT

8.4.4.10. Step 10: Prompt for another Availability SLT

The next step is to ask the user if they want to add another availability SLT. If they do, this panel must call the out-of-box Availability SLT wizard again. If they don't, the wizard must call the out-of-box Response SLT if the user indicated they wanted to add a Response SLT, or else go to the final wizard screen.

Perform the following two tasks to design a wizard panel to prompt for another Availability SLT.

Task 1: Create a wizard for more Availability SLTs[edit]

For this task, you will create a wizard so users can create more Availability SLTs.

To create a wizard so users can create more Availability SLTs, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create Availability 2, and then click **Add**.

> **Caution**
> Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard panel asks the user if they want to create another Availability SLT.
Window Title	Type: Create another Availability SLT
Title	Type: Create another Availability SLT

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following field:

Field	Value
Initial Expressions:	$L.create.avail.slo=false
No $L.file (use typecheck)	Select this option to initialize a typecheck file, which acts as a holding file.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.ask.other.avail
Turn off Previous button only?	Checked

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.
Wizard Variables:	$L.create.avail.slo

8. Select the **Next Wizard** tab and complete the following fields:

Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create Availability 1
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizazrds, based on user actions or selections. Type: $L.create.avail.slo=true
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create 8
Condition	true

9. When you are finished, click **Save**.

You have created the wizard so users can create more Availability SLTs. Next, you will create the new wizard form in Forms Designer.

Task 2: Create a form for more Availability SLTs[edit]

For this task, you will use Forms Designer to create form controls that ask users if they want to create an additional Service Level Target record.

To create a new form for the wizard that creates more Availability SLTs, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.ask.other.avail.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This will help you to avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Group	
Caption	Type: Would you like to create another Availability SLT record?
Visible	Select.
Radio Button	
Caption	Type: Yes
Input	Type: $L.create.avail.slo
Value	Type: true
Visible	Select.
Radio Button	
Caption	Type: No
Input	Type: $L.create.avail.slo
Value	Type: false
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have finished creating one of the forms needed for the SLA wizard.

Contents

- 1 Task 1: Create a wizard for more Availability SLTs
- 2 Task 2: Create a form for more Availability SLTs

8.4.4.11. Step 11: Call the Response SLT wizard

The next step is to create a wizard panel to call the existing out-of-box Response Wizard for those users who answered that they did want to add a Response SLT.

Because this wizard panel only performs the background task of calling another wizard, there is no screen to show the end user. Thus there is no task requiring the use of Forms Designer in this step.

Task: Create a wizard to call a Response SLT

For this task, you will create a wizard that will call the existing out-of-box Response SLT wizard.

To create a wizard to call the existing out-of-box Response SLT wizard, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create Response 1, and then click **Add**.

   Caution

   Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard will call the out-of-box wizard that creates a Response SLT.
Window Title	Type: Create Response SLT
Title	Type: Create Response SLT

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following field:

Field	Value
$L.file passed in	checked

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Skip Display	Select this option as this panel doesn't include a form that needs to be displayed to the user.

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.
Format Control/Process Name subtab	Define the actions when a user clicks Next, Cancel, or Previous within the wizard. Complete the following field:
Process Name	To specify that this wizard can run a process after completing, type: slo.add.resp.sla

8. Select the **Next Wizard** tab and complete the following fields:

Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create Response 2
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizazrds, based on user actions or selections. Type: true

9. When you are finished, click **Save**.

You have created the wizard that will call the existing out-of-box Response SLT wizard. A new form is not needed for this wizard.

8.4.4.12. Step 12: Prompt for another Response SLT

The next step is to ask the user if they want to add another response SLT. If they do, this panel must call the out-of-box Reponse SLT wizard again. If they don't want to add any more response SLTs, they should be shown the final wizard screen.

Perform the following two tasks to design a wizard panel to prompt for another Response SLT.

Task 1: Create a wizard for more Response SLTs[edit]

For this task, you will create a wizard so that users can create more Response SLTs.

To create a wizard so that users can create more Response SLTs, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create Response 2, and then click **Add**.

> **Caution**
> Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard panel will ask the user if they want to add another Response SLT.
Window Title	Type: Create another Response SLT
Title	Type: Create another Response SLT

5. Select the **File Selection** tab and the **Select $L.file by** subtab to complete the following field:

Field	Value
Initial Expression:	$L.create.response.slo=false
No $L.file (use typecheck)	Select this option to initialize a typecheck file, which acts as a holding file.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.ask.other.response
Request User Input	checked.
Turn Off Previous Button Only	checked

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.
Wizard Variables	$L.create.response.slo $L.crate.avail.slo

8. Select the **Next Wizard** tab and complete the following fields:

Field	Value
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create Response 1
Condition	The condition determines what happens next. In this case, you entered the next wizard in sequence and an expression that evaluates to "true." Alternatively, you can specify different wizazrds, based on user actions or selections. Type: $L.create.response.slo=true
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create Availability 1
Condition	Type: $L.create.avail.slo=true
Wizard Name	This is the name of the next wizard in a series of wizards. Type: SLA Create 8
Condition	Type: true

9. When you are finished, click **Save**.

You have created the wizard so that users can create more Response SLTs. Next, you will create the new wizard form in Forms Designer.

Task 2: Create a form for more Response SLTs[edit]

For this task, you will use Forms Designer to create form controls that ask users if they want to create an additional Service Level Target record.

To create a new form for a wizard that creates more Response SLTs, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.ask.other.response.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This will help you to avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Group	
Caption	Type: Would you like to create another Response SLT?
Visible	Select.
Radio Button	
Caption	Type: Yes
Input	Type: $L.create.response.slo
Value	Type: true
Visible	Select.
Radio Button	
Caption	Type: No
Input	Type: $L.create.response.slo
Value	Type: false
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have created one of the forms needed for the SLA wizard.

Contents

- 1 Task 1: Create a wizard for more Response SLTs
- 2 Task 2: Create a form for more Response SLTs

8.4.4.13. Step 13: Display new SLA number and title

In this final wizard step, the panel will display the record number and title of the newly created SLA, and have the record details displayed when the user clicks the **Finish** button.

Complete the following tasks to perform this step.

Task 1: Create a wizard for the completed SLA[edit]

For this task, you will create a wizard panel that displays the SLA record number and title of the newly-created SLA record to the user.

To create a wizard for the completed SLA, follow these steps:

1. Click **Tailoring** > **Wizards**. The Wizard information form opens.
2. Select the **Wizard Info** tab.
3. In the Wizard Name field, type SLA Create 8, and then click **Add**.

   Caution

   Caution If you use an existing SLA record to create a new SLA wizard panel, click **Add** rather than **Save**. If you click **Save**, you will replace the existing record with the new record you are attempting to add.

4. In the **Wizard Info** tab, complete the following fields.

Field	Value
Start Node?	Leave this option unchecked.
Brief Description	Type: This wizard is the final panel the user sees, telling them the record number and name of the new SLA created.
Window Title	Type: Create new SLA - Done
Title	Type: Create new SLA - Done

5. Select the **File Selection** tab to complete the following fields:

Field	Value
Initial Expressions subtab	Type:
Select $L.file by subtab	
Select records "of type"	Select this option and click **Fill** to select **sla** as the record type.
using query	Type: "agreement.id="+$L.new.sla **Note** Note Be sure to include the double quotes (") at each end of "agreement.id" as these quotes are part of the query.

6. Select the **Usage** tab and complete the following fields to determine the action that Service Manager should take when the wizard is run:

Field	Value
Request user input	Select this field to prompt the user for information.
Sub Format to Display	Enter a subformat name of the format to display. Type: sla.create.finished
Activate "Finish" option?	Since this is the final wizard in a series of wizard panels, select this option to make a "Finish" button appear on this wizard panel.
Turn off Next and Previous buttons ?	Select this checkbox.

7. Select the **Actions** tab to define the actions when a user clicks **Next**, **Cancel**, or **Previous** within the wizard. Complete the following field:

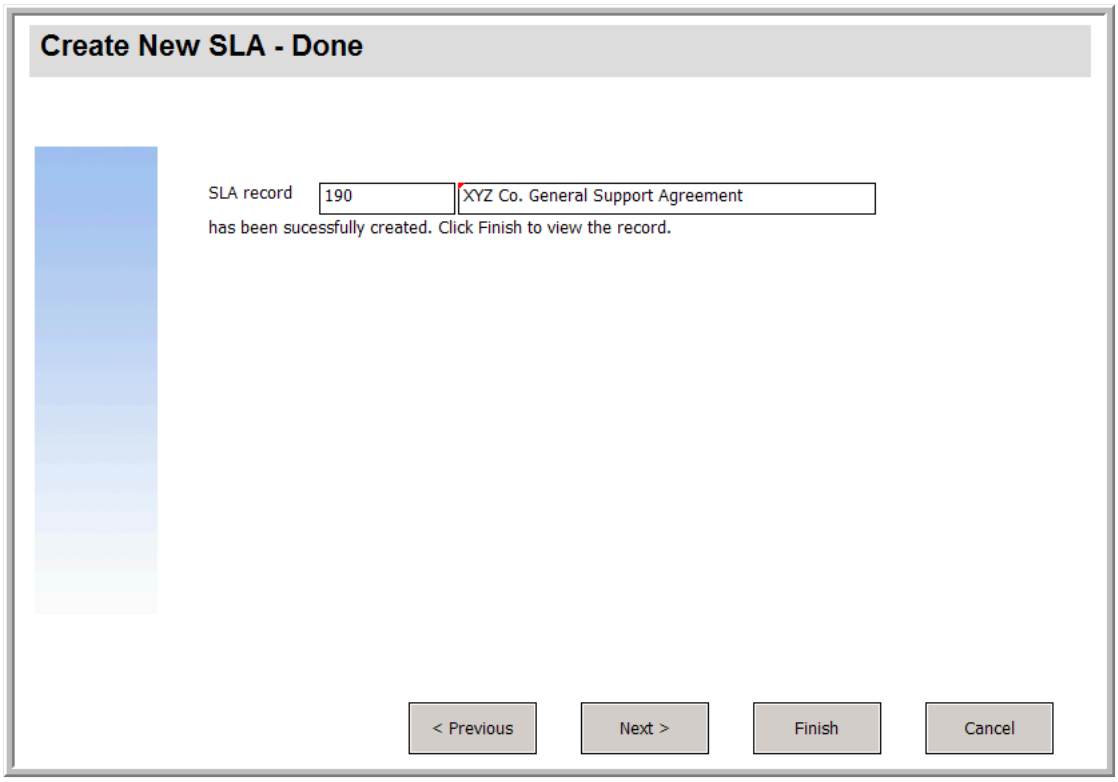
Field	Value
Perform Actions On	Here you can specify which record(s) will be affected by the action that you are going to perform. This can either be the current file, the list selected (as a whole), or an action to be performed on every single record selected. Select **Current File ($L.file)**.
Display Record(s) when complete?	Select this option (to set to true) to display the record to the user when the wizard completes.

8. When you are finished, click **Save**.

You have created the last wizard panel that displays the SLA record number and title of the newly-created SLA record. Next, you will create format for this wizard step in Forms Designer.

Task 2: Create a form for the completed SLA[edit]

For this task, you will use Forms Designer to create form controls display the record number and title of the new SLA record created. These form controls, together with the window title, prompt, buttons, and other choices you made in the wizard record, will be displayed to the end user as the wizard screen shown below.



To create a new form for the completed SLA, follow these steps:

1. Start the Forms Designer.
2. In the Form field, type sla.create.finished.
   Best Practice: Because the form name must match the subformat display entry exactly, you can copy the subformat display entry in the Usage tab and paste it into the Form field. This helps avoid spelling errors.
3. Click **New**.
4. Click **No** for Forms wizard.
5. Create the following for this form. When you start to design the new form, make sure the X and Y coordinates are at 0 (zero).

Property	Value
Label	
Caption	Type: SLA record
Visible	Select.
Text	
Input	Type: agreement.id
Visible	Select.
Text	
Input	Type: title
Visible	Select.
Label	
Caption	Type: has been successfully created. Click Finish to view the record.
Visible	Select.

6. Click **OK** twice to save the new form and exit.

You have finished creating the final wizard screen that's displayed for the SLA wizard.

Contents

- 1 Task 1: Create a wizard for the completed SLA
- 2 Task 2: Create a form for the completed SLA

8.4.4.14. Step 14: Set menu to call the new SLA wizard

The purpose of this step is to associate the new wizard with the menu command: **Service Level Management** > **Service Level Agreements** > **Create New SLA**. That is, the first panel of the new wizard is to be displayed rather than the Service Level Agreement entry form that's displayed out-of-box.

To set the menu to call the new wizard, follow these steps:

1. Click **Tailoring** > **Tailoring Tools** > **Menus**.
2. Type SLA in the Menu Name field, and then click **Search**.
3. Click in the row for **Option 5 – Service Level Agreements**.
4. In the Application column, replace thread.start by typing wizard.run.
5. In the Parameter Value column, replace sla.add.object by typing SLA Create 1.
6. Click **OK** to save the changes.

Now you can click **Service Level Management** > **Service Level Agreements** > **Create New SLA** to confirm that the menu changes were effected and the wizard opens and executes as expected.

9. Service Manager processes and best practices

Service Manager enables organizations to manage their IT infrastructures efficiently and effectively. This document describes the best practice Process Designer workflows that are standard with out-of-box Service Manager applications. It includes high-level workflow diagrams and step-by-step guidelines for the following Process Designer based modules: Change Management, Service Desk, Incident Management, Problem Management, Request Fulfillment, Knowledge Management, and Service Level Management.

The Service Manager best practice workflows are aligned with the Information Technology Infrastructure Library (ITIL) standard, a widely recognized source of guidelines for Information Technology Service Management (ITSM).

This document is based on the out-of-box implementation of Service Manager Codeless. For customers on Service Manager Hybrid, we recommend that you also implement the Codeless workflows described in this document by using the Process Designer technology.

9.1. Service Desk overview

The Service Manager Service Desk application, referred to as Service Desk throughout this chapter, supports the service desk function of the Information Technology Infrastructure Library (ITIL) with its User Interaction Management processes for the IT service and the customer base. The Service Desk application provides a single point of entry to the other Service Manager applications and enables you to document and track all calls received by the service desk.

Service Desk incorporates the essential concepts of ITIL to ensure that the best practices of IT service management are applied to the service desk to aid end customers, ensure data integrity, and streamline communication channels in the organization.

This section describes how Service Desk implements the best practice guidelines for the Service Desk Interaction Management processes.



Note

Note

If you enable the Streamlined Interaction solution for Service Desk, see Service Desk (Streamlined Interaction).

Topics in this section include:

- Service Desk within the ITIL framework
- Service Desk application
- Service Desk process overview
- Input and output for Service Desk Interaction Management
- Key performance indicators for Service Desk Interaction Management
- RACI matrix for Service Desk Interaction Management

9.1.1. Service Desk within the ITIL framework

Service Operation is one of five core publications from ITIL that covers the service lifecycle. The purpose of service operation is to deliver agreed-on levels of service to users and customers, and to manage the applications, technology, and infrastructure that support delivery

of the services.

The *service desk* is a key function of service operation. It provides a single, central point of contact for all users of IT. The service desk's goal is to restore normal service to users as quickly as possible. Restoring normal service could involve fixing a technical fault, fulfilling a service request, or answering a query — whatever is needed to enable users to return to their work. The service desk logs and manages customer interactions and provides an interface to other service operation processes and activities.

ITIL 2011 notes these specific responsibilities of a service desk:

- Logging, categorizing, and prioritizing all calls
- Providing first-line investigation and problem diagnosis
- Resolving incidents or service requests to be handled at the service desk level
- Escalating incidents and service requests that can't be resolved within agreed-on time limits
- Closing resolved incidents, requests, and other calls
- Communicating with users to keep them informed of progress, impending changes, agreed-on outages, and other such notifications

9.1.2. Service Desk application

The Service Manager Service Desk application incorporates the ITIL best practices that are used by organizations worldwide to establish and improve their capabilities in service management.

It provides a central *Service Operation* function, coordinating the efficient and effective delivery of services to end users and enabling various improvements, including the following:

- Improved customer service and satisfaction
- Increased accessibility through a single point of contact and information
- Better quality and faster turnaround of customer or user requests
- Improved teamwork and communication
- Enhanced focus and a proactive approach to service provision
- Improved usage of IT resources and increased productivity of all users

The Service Desk application enables a Service Desk agent to document and track user interactions. Service Desk provides one-click access to other Service Manager applications to automatically enter information received.

The Service Desk application covers:

- Direct interactions between a user and the service desk by phone or by email
- User activities that occur from use of the self-service Web portal (for example, searching the knowledge base, checking for status updates, or logging an interaction).

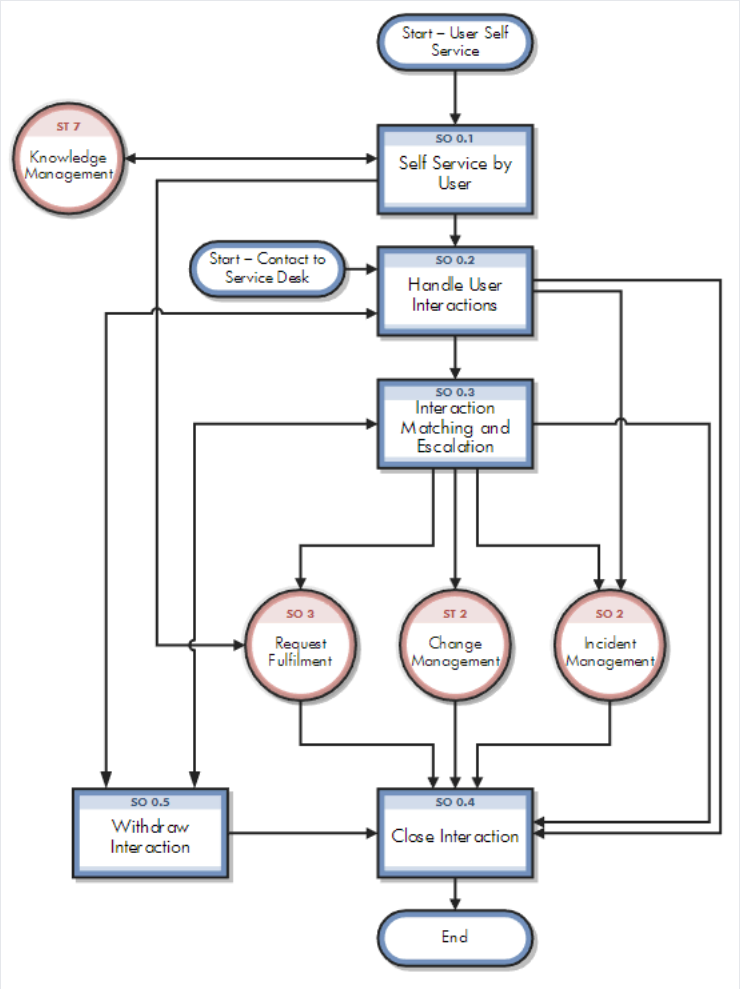
One of the best practices that derives from ITIL's service desk function is that user interactions should not be saved and updated later. Therefore, the Service Desk application requires that any new interaction either be resolved within the agreed upon time limits and then closed or, if it can't be resolved, escalated. The information gathered during the customer interaction can be used to open an incident if a reported issue requires further action. It can also be added to a record in another Service Manager application, such as Change Management.

9.1.3. Service Desk process overview

Every user contact with the service desk is logged as an interaction. User Interaction Management is the process for handling all interactions with the service desk that are received from self-service Web pages or directly by service desk personnel. These interactions can include service disruptions, service requests, requests for information (RFI), or complaints reported by users who communicate with the service desk by using instant messages, phone, E-mail, or by self-service Web pages. The User Interaction Management process enables you to easily log and resolve simple user requests and to escalate others into incidents requiring further action.

Multiple user interactions can be linked to a single incident in the tool. User Interaction Management describes all the activities a Service Desk agent needs to follow when registering a new incident or change. The Service Desk agent follows the necessary steps and searches for related knowledge records, known error records, and existing incidents or changes. This process streamlines service desk activities, thereby decreasing the workload for second line support teams.

A general overview of the User Interaction Management processes and workflows is depicted below. They are described in detail in Service Desk Workflows.



When a user contacts the service desk, the Service Desk agent uses the Service Desk application to create an interaction record. The Service Desk agent records the user name, the name of the component that the user is calling about, and a description of the service request. After collecting this information, the Service Desk agent performs the actions required to resolve the user request.

- If the service request is resolved without escalating it to an incident, the Service Desk agent can close the interaction record.
- If the service request can't be resolved without escalating it to an incident, the Service Desk agent searches for existing incidents, problems, or known errors that affect the same component or one of the parent assets of that component.
  • If an existing incident is found, the Service Desk agent can associate the current interaction with the existing incident.
  • If an existing incident is not found, the Service Desk agent can register a new incident based on the Service Desk interaction. Service Desk copies information from the interaction record into the newly-created incident.

For example, consider a user who can't print to a network printer:

- The user contacts the service desk for assistance.
- The Service Desk agent populates an interaction record with the relevant information.
- Because the issue can't be resolved immediately, the Service Desk agent opens an incident, and the incident is assigned to a technician.
- The technician discovers that the printer network connection is broken.
- The technician fixes the connection and closes the incident.
- The Service Desk agent contacts the user and instructs the user to attempt printing to the network printer.
- If the user can successfully print, the Service Desk agent can close the interaction. If the user still can't print, the Service Desk agent may create a new incident and then relate the unsolved interaction.
- If the user wishes to report a related or new issue, the Service Desk agent closes the interaction (as the original issue was resolved) and opens a new interaction detailing the new issue the user needs to report.

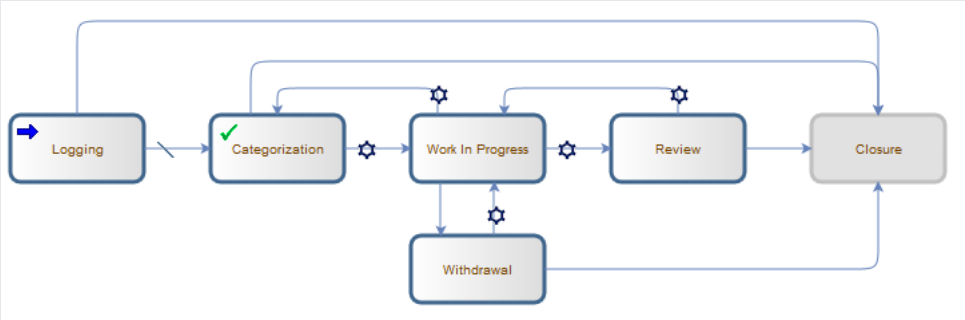
Interaction categories

Service Manager Categories classify and define the type of interaction. Each category could have its own workflow process. The steps of the workflow are represented by the phases. Service Manager requires that every interaction has an interaction category and phases.

Interaction phases

Service Manager uses phases to describe the steps needed to dispose an interaction. The phase also determines the forms users see, the actions users can manually trigger.

The following figure shows the workflow phases for an Interaction.



Service Desk user roles

The following table describes the responsibilities of the User Interaction Management user roles.

User Interaction Management user roles

Role	Responsibilities
User	• Report all IT-related requests to the service desk or use the self-service Web pages. • Validate solutions and answers provided by the IT department to a registered service request.
Service Desk Agent	• Register interactions based on contact with user. • Match user interaction to incidents, problems, known errors, or knowledge document. • Solve and close interactions. • Provide status updates to users on request. • Register incident based on a user interaction and assign to the correct support group. • Register Request for Change, based on a user interaction. • Register Service Request, based on a user interaction. • Validate a solution provided by a support group. • Report and verify a solution to a user. • Monitor Service Level Agreement (SLA) targets of all incidents registered and escalate, if required. • Communicate about service outages to all users. • Withdraw Interaction on behalf of a user.
Service Desk Manager	• Appoint people to the required roles. • Manage resources assigned to the service desk. • Manage service desk activities. • Attend CAB meetings • Report to senior managers on any issue that could significantly impact the business. • Take overall responsibility for incident and service request handling on the service desk. • Monitor and report on service desk performance. • Make improvements to the service desk.

9.1.4. Input and output for Service Desk Interaction Management

Interactions can be triggered and resolved in several ways. The following table outlines the inputs and outputs for the User Interaction Management process.

Input and output for Service Desk Interaction Management

Input to Service Desk Interaction Management	Output from Service Desk Interaction Management
A user can contact the service desk and give input by using instant messages, phone, email, self-service web pages, or other means.	Service desk personnel can handle an interaction in the following ways: • If the interaction is related to a new or existing incident, the interaction is handled by using the Incident Management process. • If the interaction involves a request, the interaction is sent to the Request Fulfillment process. • If the interaction requires a change, the interaction is sent to the Change Management process.

9.1.5. Key performance indicators for Service Desk Interaction Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your indicators for Service Desk Interaction Management processes. To visualize trend information, it is useful to graph KPI data periodically. In addition to the data provided by Service Manager, you may need additional tools to report on all of your KPI requirements.

Key Performance Indicators for Service Desk Interaction Management

Title	Description
First time fix	Percentage of interactions closed by the Service Desk agent at first contact without reference to other levels of support and going through the whole service desk workflow
First line fix	Percentage of interactions closed by the service desk without reference to other levels of support
Customer satisfaction	Customer satisfaction measured by surveys completed by customers

For completeness, the ITIL 2011 and COBIT 4.1 KPIs are included below.

ITIL 2011 Key Performance Indicators[edit]

The following are ITIL 2011 KPIs for User Interaction Management:

- Percentage of incidents closed by the service desk without reference to other levels of support (that is, closed by first point of contact).
- Number and percentage of incidents processed by each Service Desk agent.

COBIT 4.1 Key Performance Indicators[edit]

The following are the COBIT 4.1 KPIs for User Interaction Management:

- Amount of user satisfaction with first-line support (service desk or knowledgebase)
- Percent of first-line resolutions based on total number of requests
- Call-abandonment rate
- Average speed to respond to telephone and email or Web requests
- Percent of incidents and service requests reported and logged using automated tools
- Number of days of training per service desk staff member per year
- Number of calls handled per service staff member per hour
- Number of unresolved queries

Contents

- 1 ITIL 2011 Key Performance Indicators
- 2 COBIT 4.1 Key Performance Indicators

9.1.6. RACI matrix for Service Desk Interaction Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. It is especially useful in clarifying roles and responsibilities in cross-functional/departmental projects and processes. The RACI matrix for Service Desk Interaction Management is shown in the following table.

RACI matrix for Service Desk Interaction Management

Process ID	Activity	User	Service Desk Agent	Service Desk Manager
SO 0.1	Self-Service by User	R	I	A
SO 0.2	Interaction Handling	R	R	A
SO 0.3	Interaction Matching and Escalation	R	R	A
SO 0.4	Interaction Closure	R/I	R	A
SO 0.5	Withdraw Interaction	R/I	R	A

9.2. Service Desk workflows

Every time a user contacts the service desk it is logged as an interaction. User interaction management is the process of handling all interactions with the service desk that are received from self-service Web pages or directly by service desk personnel. These interactions can include service disruptions, service requests, requests for information (RFI), and complaints reported by users who communicate with the service desk by using instant messages, phone, email, or self-service web pages.

The Service Desk agent follows the necessary steps and searches for related knowledge records, known error records, problem records, and existing incidents or changes. The process enables Service Desk agents to easily log and resolve simple user requests and to escalate others into incidents requiring further action. The process streamlines service desk activities and decreases the workload for second-line support teams.

The User Interaction Management process consists of the following processes, which are included in this chapter:

- Self-Service by User (process SO 0.1)
- Interaction Handling (process SO 0.2)
- Interaction Matching and Escalation (process SO 0.3)
- Interaction Closure (process SO 0.4)
- Withdraw Interaction (process SO 0.5)
- Complaint Handling (process SO 0.6)

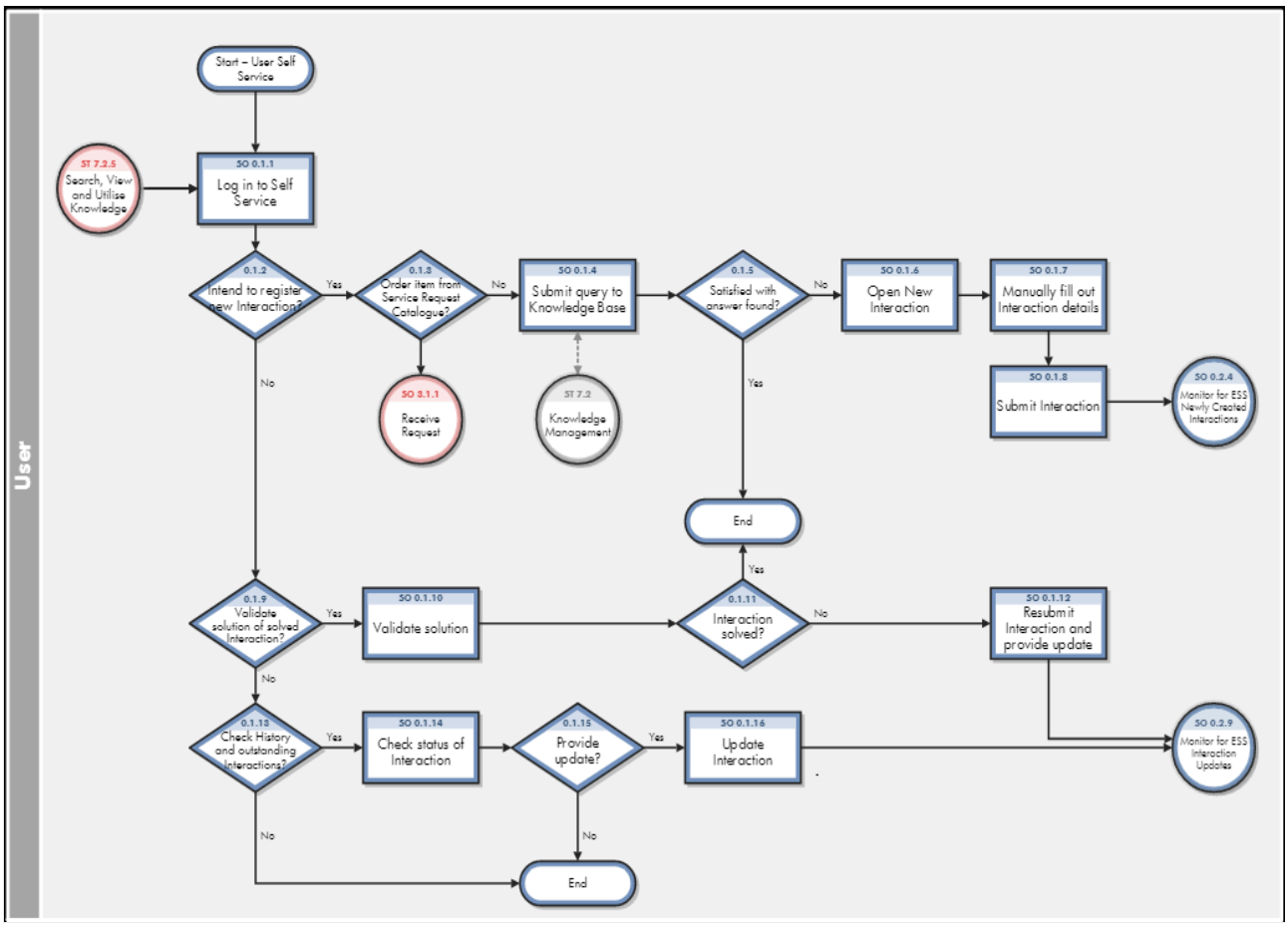
9.2.1. Self-Service by User (process SO 0.1)

By using the self-service web environment, users can perform the following activities without contacting the service desk:

- Search the knowledgebase to find an answer to a question or issue
- Monitor the status of previously reported interactions
- Log new interactions
- Order items from the service catalog

You can see the details of this process in the following figure and table.

Self-Service by User (SO 0.1) is illustrated in the following figure:



Self-Service by User (SO 0.1) process

Process ID	Procedure or Decision	Description	Role
SO 0.1.1	Log in to Self-Service	To gain access to the Self-Service Web interface, users must log on by using their login credentials.	User
SO 0.1.2	Intend to register new interaction?	If yes, continue with SO 0.1.3. If no, go to SO 0.1.9.	User
SO 0.1.3	Order item from Service Request Catalog?	If yes, log a service request. If no, go to SO 0.1.4.	User
SO 0.1.4	Submit query to Knowledge Base	To search for a knowledge document, users must complete a search.	User
SO 0.1.5	Satisfied with answer found?	If yes, stop. If not, go to SO 0.1.6.	User
SO 0.1.6	Open new interaction	To open a new interaction from the knowledge search screen, users must create a New Interaction.	User
SO 0.1.7	Manually fill out interaction details	To register a new interaction, users must provide a title and a description of the request; select the urgency, and preferred contact method; and can optionally provide affected Service and add an attachment.	User
SO 0.1.8	Submit interaction	When all mandatory fields are completed, submit the form to send the request to the service desk.	User
SO 0.1.9	Validate solution of solved interaction?	To validate the solution to a previously reported interaction, go to SO 0.1.10. If no, go to SO 0.1.13.	User
SO 0.1.10	Validate solution	Use View Closed Requests to get an overview of all solved interactions. Select the applicable interaction and validate the solution provided.	User
SO 0.1.11	Interaction solved?	If yes, stop. If not, go to SO 0.1.12.	User
SO 0.1.12	Resubmit interaction and provide update	When a user disagrees with the proposed solution, the user can resubmit the interaction and provide a reason for the disagreement. The newly-created interaction is sent to the service desk for further diagnosis.	User

SO 0.1.13	Check history and outstanding Interactions?	If a user wants to check the status or history of previously registered interactions, go to SO 0.1.14. If no, stop.	User
SO 0.1.14	Check status of Interaction	Use View Open Requests to get an overview of all open interactions. Select the interaction and view the status with last updates.	User
SO 0.1.15	Provide update?	If a user has additional details to add to the previously-logged interaction that may be useful to know for the specialist, go to SO 0.1.16. If no, stop.	User
SO 0.1.16	Update Interaction	There are two scenarios to update an interaction and have a Save button to save the updated information. • The Save button appears when a self-service user selects the option View Open Requests, selects an interaction, and clicks the Update button. Once the information is updated, the self-service user clicks Save & Exit to save the updated information in the request. • When you escalate an interaction, you can go back to the interaction to add more information or perform changes to it. You then have an Update button when you select an existing interaction. The interaction is also a status of Dispatched or Callback. Once you have added more information to the request or performed the changes, you can click Save & Exit.	User

9.2.2. Interaction Handling (process SO 0.2)

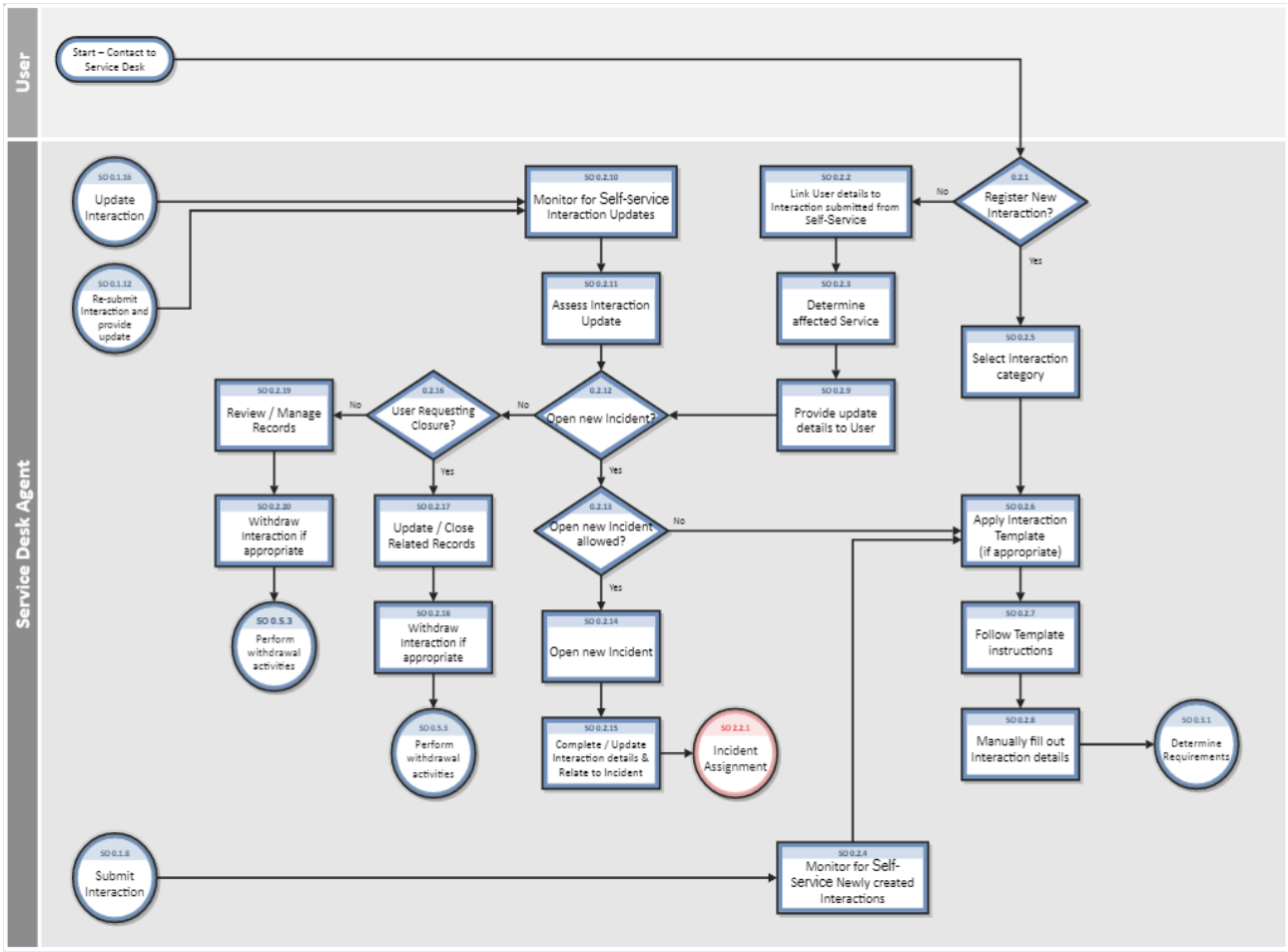
The service desk is responsible for handling all user interactions received by the self-service Web portal, email, or phone. The service desk attempts to resolve an interaction when the user makes first contact with the service desk. Interaction Handling includes the

registration and preliminary investigation of interactions including the matching against open incidents, problems, known errors, and the knowledgebase to maximize the first-line solving ratio.

When the service desk can't close an interaction on first contact, the Service Desk agent escalates it to Incident Management, Change Management, or Request Fulfillment.

You can see the details of this process in the following figure and table.

Interaction Handling (SO 0.2) is illustrated in the following figure:



Interaction Handling (SO 0.2) process

Process ID	Procedure or Decision	Description	Role
SO 0.2.1	Register New Interaction?	If the interaction is new, go to SO 0.2.5. If not, go to SO 0.2.2.	Service Desk Agent
SO 0.2.2	Link User details to Interaction submitted from Self-Service	Fill in the name of the caller in the Contact Person field and the name of the user in the Service Recipient field (if different).	Service Desk Agent
SO 0.2.3	Determine affected service	In the Affected Service field, select the service that matches the user request. Then, go to SO 0.2.9.	Service Desk Agent
SO 0.2.4	Monitor for Self-Service newly created Interactions	If there are new Interactions, follow the same Interaction registration process.	Service Desk Agent
SO 0.2.5	Select Interaction category	Select the appropriate Interaction category from the available category list, and then go to SO 0.2.6.	Service Desk Agent
SO 0.2.6	Apply Interaction Template (if appropriate)	If there is an interaction model available, apply the model to quickly define the interaction. If no model exists, the default interaction settings are shown.	Service Desk Agent
SO 0.2.7	Follow Template instructions	The predefined fields are filled in from the model.	Service Desk Agent
SO 0.2.8	Manually fill out Interaction details	Fill out the required interaction details such as short title, a full description, Contact, Service Recipient, and Affected Service. In addition, select the applicable impact and urgency. The assignment group is automatically filled in, based on the Service Desk Group configured in the Service Desk environment record.	Service Desk Agent
SO 0.2.9	Provide update details to User	Inform the user of recent updates made by Analysts, and then update the interaction by stating that the user requested an update.	Service Desk Agent
SO 0.2.10	Monitor for Self-Service Interaction Updates	If Interactions are updated, they must be reassessed and a new incident may need to be opened.	Service Desk Agent
SO 0.2.11	Assess Interaction Update	Evaluate Interactions that have been updated or resubmitted.	Service Desk Agent

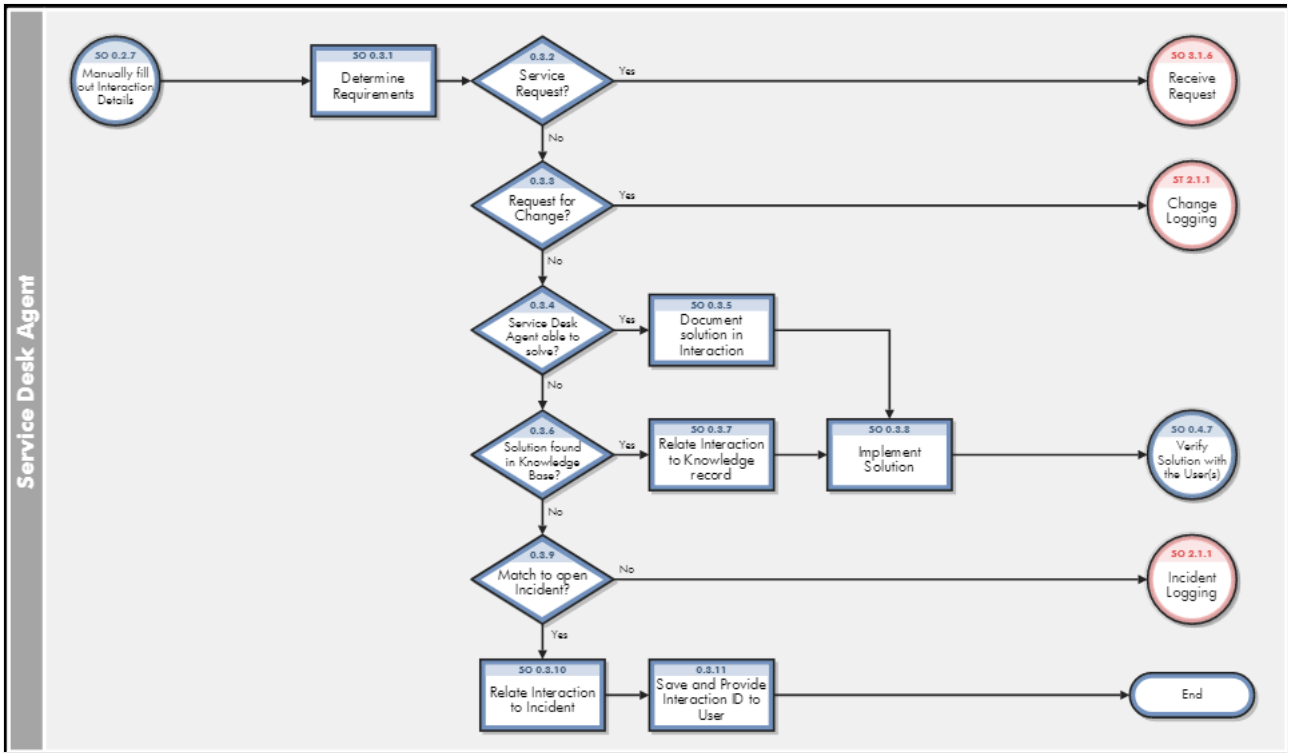
SO 0.2.12	Open new Incident?	If the user is unhappy with a solution provided and a new incident must be opened, go to SO 0.2.13. If not, go to SO 0.2.16.	Service Desk Agent
SO 0.2.13	Open new Incident allowed?	If opening a new incident is allowed due to a user request during a specific period after solution notification, go to SO 0.2.14. If not, go to SO 0.2.6.	Service Desk Agent
SO 0.2.14	Open new Incident	Because the previously registered incident was solved incorrectly, open a new incident and provide an update that states the reason that the incident was created.	Service Desk Agent
SO 0.2.15	Complete/Update Interaction details & Relate to Incident	Relate the interaction to the open incident.	Service Desk Agent
SO 0.2.16	User requesting closure?	If the user is requesting the incident be closed, go to SO 0.2.17. If not, go to SO 0.2.19.	Service Desk Agent
SO 0.2.17	Update/Close related records	Update the related record as needed and close it.	Service Desk Agent
SO 0.2.18	Withdraw Interaction if appropriate	Withdraw the interaction if this registration is not needed anymore.	Service Desk Agent
SO 0.2.19	Review/Manage records	Review the records and take action accordingly.	Service Desk Agent
SO 0.2.20	Withdraw Interaction if appropriate	Withdraw the interaction if this registration is not needed anymore.	Service Desk Agent

9.2.3. Interaction Matching and Escalation (process SO 0.3)

When an Interaction is received, the Service Desk Agent first determines if the Interaction is a service request or a request for change, and if so, logs the request. If the Service Desk Agent is not able to resolve the issue, the incident can either be related to an existing incident or logged as a new incident.

You can see the details of this process in the following figure and table.

Interaction Matching and Escalation (SO 0.3) is illustrated in the following figure:



Interaction Matching and Escalation (SO 0.3) process

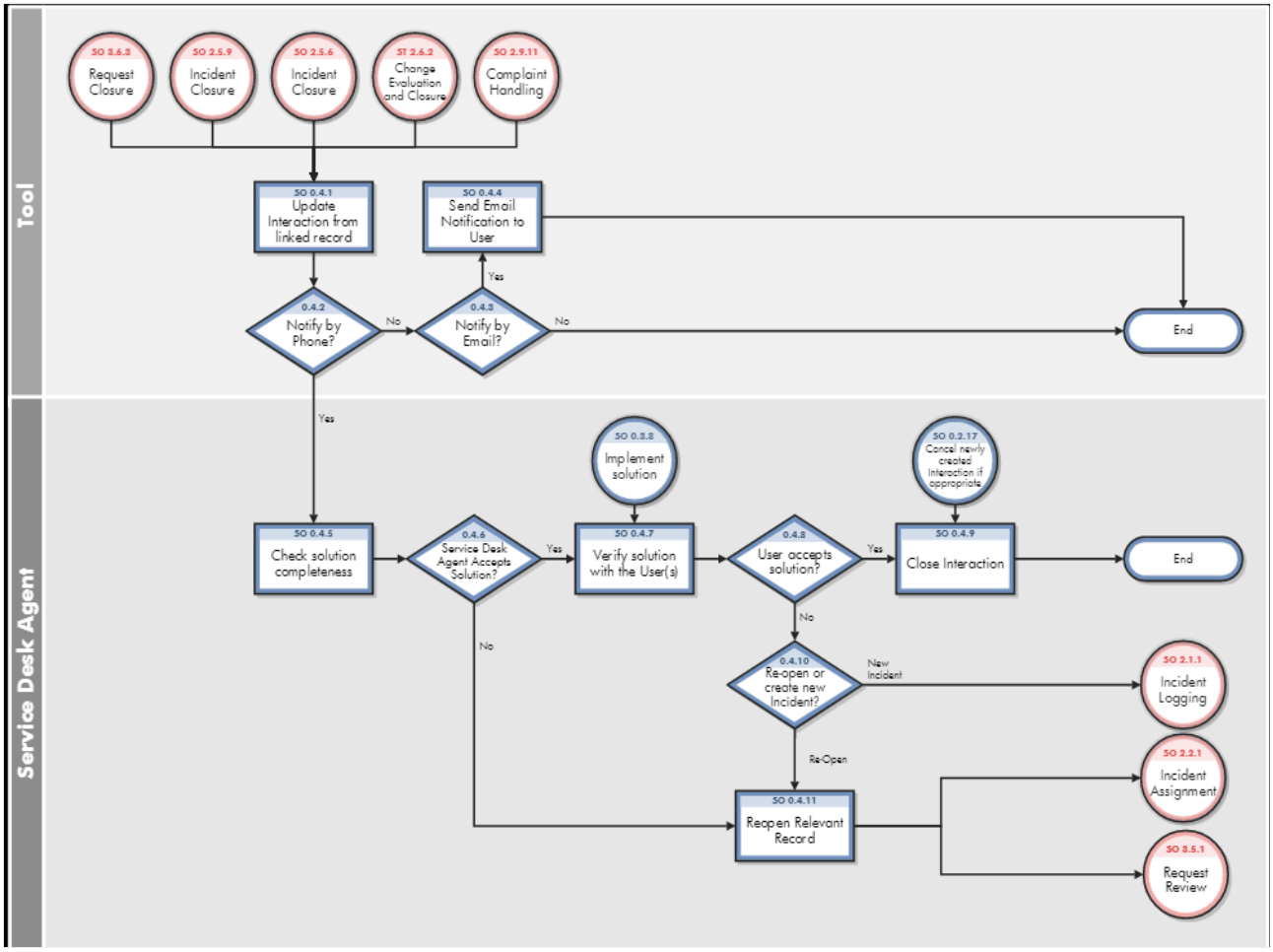
Process ID	Procedure or Decision	Description	Role
SO 0.3.1	Determine Requirements	After filling out the Interaction details, the Service Desk Agent determines what requirements of the request are.	Service Desk Agent
SO 0.3.2	Service Request?	If a service request is needed, the Service Desk agent logs the request. If not, proceed to SO 0.3.3.	Service Desk Agent
SO 0.3.3	Request for Change?	If a change is required, log the change request. If not, proceed to SO 0.3.4.	Service Desk Agent
SO 0.3.4	Service Desk Agent able to solve?	If the Service Desk agent is able to solve the request, proceed to SO 0.3.5. If not, proceed to SO 0.3.6.	Service Desk Agent
SO 0.3.5	Document solution in Interaction	The Service Desk agent documents the solution implemented.	Service Desk Agent
SO 0.3.6	Solution found in Knowledge Base?	If the solution is already documented in the Knowledge Base, proceed to SO 0.3.7. If not, proceed to SO 0.3.9.	Service Desk Agent
SO 0.3.7	Relate Interaction to Knowledge record	The Service Desk Agent selects “use solution” in the knowledge record to record this as the knowledge source and auto populate details of the solution in the Interaction record solution field.	Service Desk Agent
SO 0.3.8	Implement Solution	The Service Desk Agent then implements the solution for the User.	Service Desk Agent
SO 0.3.9	Match to open Incident?	The Service Desk Agents checks to see whether another open incident is similar to the new request, and if a match can be made. If a match can be made, proceed to SO 0.3.10. If not, log the incident.	Service Desk Agent
SO 0.3.10	Relate Interaction to Incident	If an open incident matches the new request, the Service Desk Agent relates the two.	Service Desk Agent
SO 0.3.11	Save and Provide Interaction ID to User	The Service Desk Agent saves the incident and provides the Interaction ID to the User.	Service Desk Agent

9.2.4. Interaction Closure (process SO 0.4)

When an interaction is resolved by the Service Desk on first intake, or solved by a related incident, change, or request that's resolved, the interaction is closed. Based on user preferences, the Service Desk communicates the solution to the user by phone or email.

You can see the details of this process in the following figure and table.

Interaction Closure (SO 0.4) is illustrated in the following figure:



Interaction Closure (SO 0.4) process

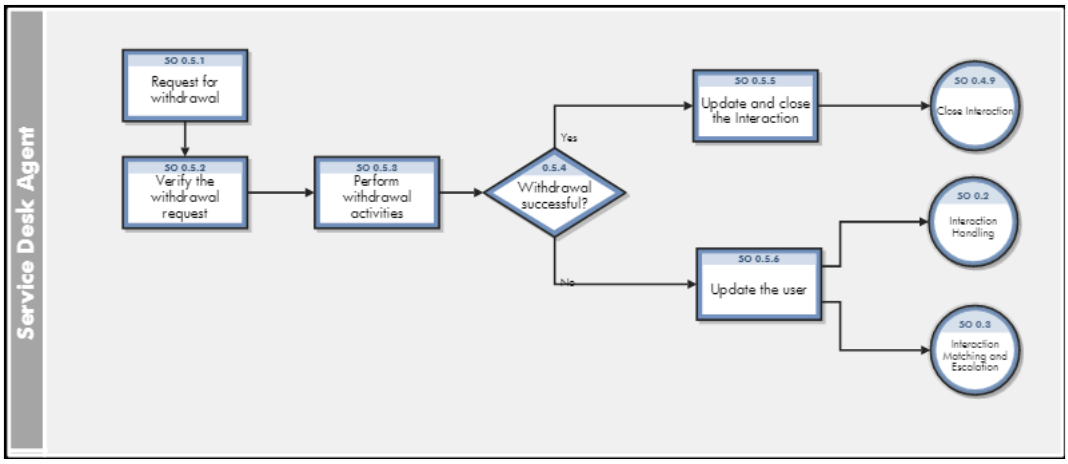
Process ID	Procedure or Decision	Description	Role
SO 0.4.1	Update Interaction from linked record	The Interaction could involve the closure of an incident, change request, or service request, or the submission of a complaint.	Service Desk Agent
SO 0.4.2	Notify by phone?	If the Notify By method states that the user wants to be notified by phone, go to SO 0.4.5. If not, go to SO 0.4.3.	Service Desk Agent
SO 0.4.3	Notify by email?	If the Notify By method states that the user wants to be notified by email, go to SO 0.4.4. If not, the User doesn't need to be notified.	Service Desk Agent
SO 0.4.4	Send email notification to User	Send the email notification.	Service Desk Agent
SO 0.4.5	Check solution completeness	The Service Desk agent checks the solution provided for all Callback interactions.	Service Desk Agent
SO 0.4.6	Service Desk Agent accepts solution?	If yes, go to SO 0.4.7. If not, go to SO 0.4.11.	Service Desk Agent
SO 0.4.7	Verify solution with the User(s)	The Service Desk Agent contacts the user and communicates the resolution. The user should verify the solution and confirm that the incident is solved and that the question or complaint is answered, or the Service Request is fulfilled.	Service Desk Agent
SO 0.4.8	User accepts solution?	If yes, go to SO 0.4.9. If no, go to SO 0.4.10.	Service Desk Agent
SO 0.4.9	Close interaction	The Service Desk Agent closes the interaction.	Service Desk Agent
SO 0.4.10	Reopen or Create new Incident	The solution provided may not solve the issue for all users. If the solution doesn't solve the issue for all users, the Service Desk Agent must either reopen the existing record or log the incident.	Service Desk Agent
SO 0.4.11	Reopen relevant record	The Service Desk agent reopens the incident for further investigation and diagnosis.	Service Desk Agent

9.2.5. Withdraw Interaction (process SO 0.5)

The Withdraw Interaction process identifies the steps to withdraw an interaction.

You can see the details of this process in the following figure and table.

Withdraw Interaction (process SO 0.5) is illustrated in the following figure:



Withdraw Interaction (SO 0.5) process

Process ID	Procedure or Decision	Description	Role
SO 0.5.1	Request for withdrawal	The user contact the Service Desk Agent and ask for the interaction to be withdrawn on their behalf by providing appropriate information.	Service Desk Agent
SO 0.5.2	Verify the withdrawal request	The Service Desk Agent verifies the request to check whether the Interaction can be withdrawn or not.	Service Desk Agent
SO 0.5.3	Perform withdrawal activities	The Service Desk Agent performs withdrawal activities for the request.	Service Desk Agent
SO 0.5.4	Withdrawal successful?	If the withdrawal is successful, go to SO 0.5.5. If not, go to SO 0.5.6.	Service Desk Agent
SO 0.5.5	Update and close the Interaction	The user is informed about the Interaction withdrawal and go to SO 0.4.9 to close the Interaction.	Service Desk Agent
SO 0.5.6	Update the user	The user is communicated on the reason why the Interaction can't be withdrawn. The Interaction moves to the previous status and the provisioning continues.	Service Desk Agent

9.3. Service Desk details

Service Manager uses its Service Desk application to enable the Service Desk Interaction Management process. The main function of Service Manager is to monitor, track, and record calls and open incidents, as necessary.

In User Interaction Management, a Service Desk Agent receives a call and opens a new interaction. The Service Desk Agent fills in the required fields, and then chooses to close the interaction or escalate it to an incident.

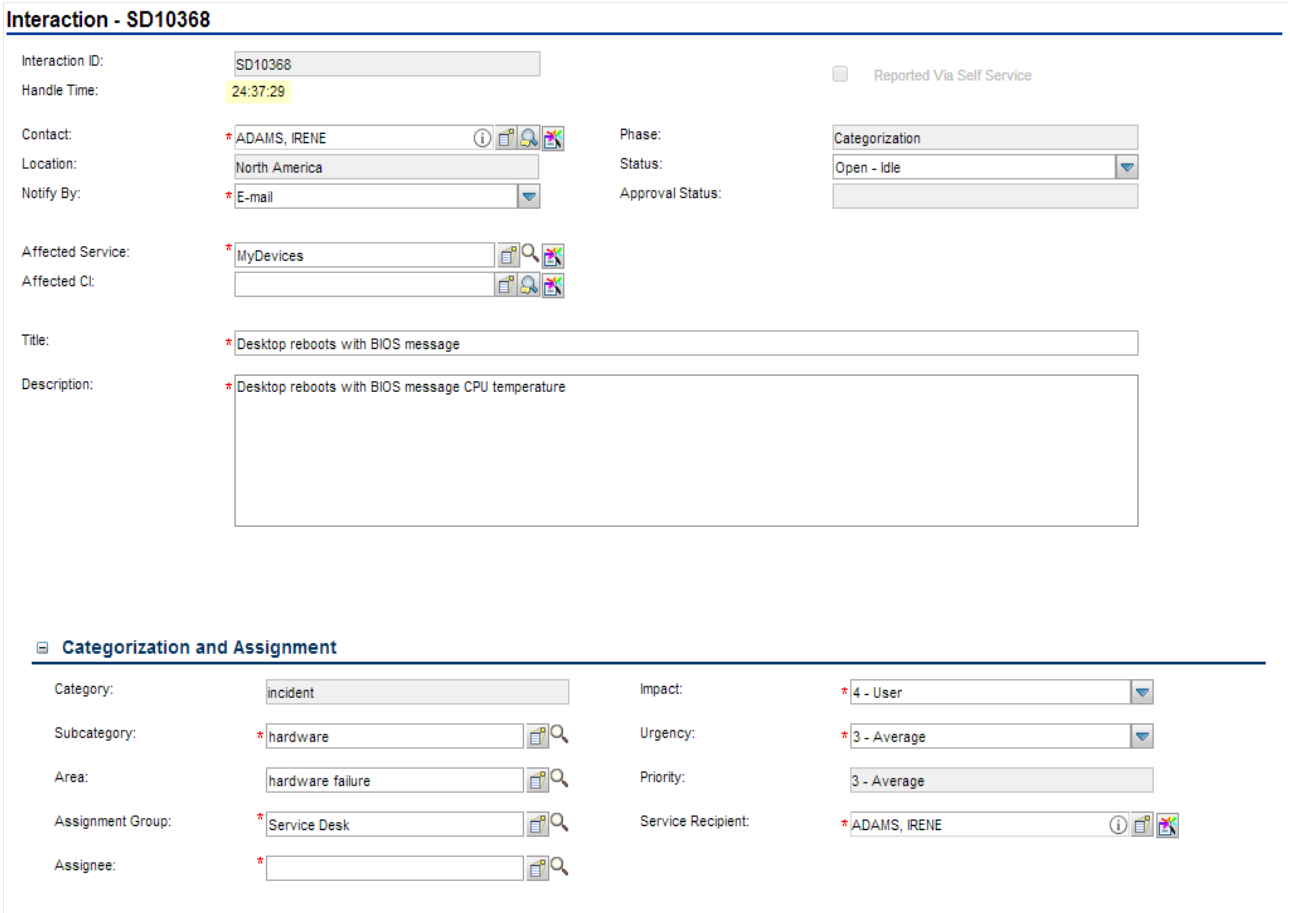
This section describes selected User Interaction Management fields in the out-of-box Service Manager system. Topics in this section include:

- New interaction form
- Interaction form after escalation
- Service Desk Interaction Management form details
- Interaction categories

9.3.1. New interaction form

When a Service Desk Agent clicks Create a New Interaction, Service Desk displays the new interaction form. The required fields in this form must be populated to register the new interaction. Service Desk fills in some of the fields automatically. The Service Desk Agent must fill in the others.

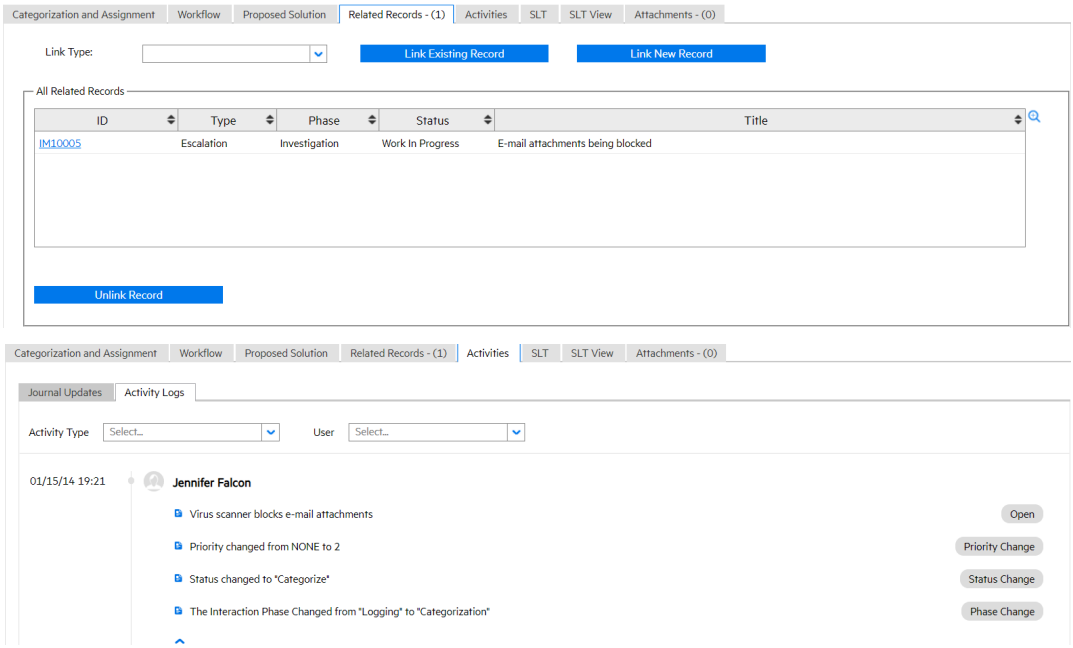
A new interaction that has been filled in is illustrated in the following screenshot:



9.3.2. Interaction form after escalation

After the Service Desk Agent escalates the interaction, Service Desk displays new sections and fields.

The following screenshots illustrate the same interaction after escalation.



9.3.3. Service Desk Interaction Management form details

The following table identifies and describes some of the features on Service Desk's Interaction Management forms.

Service Desk Interaction Management form details

Label	Description
Interaction ID	Service Manager populates this field with a unique ID when a Service Desk Agent registers a new interaction.
Status	The options in this field have been revised to align with our new best practices. **Tip** Tip You may want to tailor these options to match your business needs. These statuses are available out-of-box: • Open — The interaction has been initially logged. For example, when the Service Desk Agent is still on the phone with the customer. • Categorize — The interaction has been logged and saved by Service Desk Agent, or a request has been created by a self-service user. The categorization details will be provided with this status. • Assign — The status is manually set by Service Desk Agent, or a service catalog request will stay in this status when it is created by self-service user or Service Desk Agent. The assignment will be made with this status. • Suspended — The interaction can be suspended for a predefined duration at categorization or fulfillment. It will be worked upon only after it is unsuspended either manually or automatically. • In Progress — The interaction has no incidents, changes, or other records related to it. It is fulfilled by Service Desk. • Dispatched — The interaction has been escalated or the catalog request approved and the interaction is now related to another record, such as an incident, change, or request. • Resolved — The interaction has been resolved with solution filled. • Callback — There is an action pending for the interaction. The Service Desk Agent must now call the contact. If the Notify By field for that user is set to Telephone, the interaction is automatically set to Callback when the related record is closed. • Closed — The interaction was closed by the help desk or automatically after the related record was closed. • Withdrawal Requested — The user contacts the Service Desk Agent and ask for the interaction to be withdrawn on their behalf by providing appropriate information.

Phase	• Logging - The interaction has been initially logged. For example, when the Service Desk Agent is still on the phone with the customer. • Categorization – The interaction is categorized and prioritized in this phase. Assignment is also made in this phase. • Work In Progress – The interaction is under investigation, either by Service Desk or by other process, such as incident, change, or request. • Review – The proposed solution of the interaction is reviewed in this phase. • Closure – The interaction was closed by the help desk or automatically after the related record was closed. • Withdrawal – The interaction is in this phase where the user contacts the Service Desk Agent and ask for the interaction to be withdrawn on their behalf by providing appropriate information.
Contact	The Service Desk Agent populates this field with the person from which the call was received. The contact person is not necessarily the same person as the service recipient. This field ensures that the correct person will be notified about updates to the interaction. After filling in the contact name, the Service Desk Agent can use the Smart Indicator positioned at the end of the field to view open or closed interactions for this contact. This field includes a hover-over form that displays full name, telephone, and email address if available for the contact. This is a required field.
Service Recipient	The person who has the problem and needs it resolved. It's not necessarily the person who is calling to report the problem. Filling in this field automatically fills in the contact name from the contact record of who should be notified of the resolution. The Service Desk Agent populates this field with the person this issue is registered for. This field includes a hover-over form that displays full name, telephone, and email address if available for the service recipient. After filling in the service recipient, the Service Desk Agent can use the Smart Indicator positioned at the end of the field to view open or closed interactions for the service recipient. This is a required field.
Location	The location for which the interaction has been reported. The field is for informational purposes only. Location data is customer and implementation specific.

Source	The source from which the record is created. The following out-of-box sources are available: • 1 - User • 2 - Group • 3 - Event • 4 - Chat • 5 - Email • 6 - Telephone
Notify By	To notify the customer when the issue has been resolved, Service Manager prepopulates this field with Email for user. User or the Service Desk Agent can change it to None or Telephone, if applicable. When the related incident or change is closed: • Selecting Email sends email to the contact and closes the interaction • Selecting None closes the interaction without notifying the contact • Selecting Telephone sets the interaction to the status Callback, which tells the Service Desk Agent to call the contact. The Service Desk Agent asks the contact whether the solution is satisfactory and indicates the answer on the Required Actions tab. If the solution works for the customer, you close the interaction. If it doesn't work, then you must open a new incident. This is a required field.

Affected Service	The Service Desk Agent populates this field with the business service affected by the registered issue. Only business services the service recipient has a subscription for can be selected. As a best practice, users should select the affected service before selecting the Affected CI because the Affected CI selection is limited by the service selected by a user. Selecting the service first prevents a mismatch between the service and the CI. ITIL 2011 is centered around services, so a service construct should always be defined for best practices. If you have not yet created a service construct, start with a catch-all service, such as My Devices. **Note** Note The out-of-box options in this field are based on past Service Manager implementations. You should tailor these options to match your business needs. These business services are available out-of-box: • Applications • E-mail/Webmail • Handheld PDA & Telephony • Intranet • Internet • My Devices (The My Devices service represents all personal devices that the user would use.) • Printing Selecting the service: • May limit the list of affected CIs. • Validates that it is a valid service. An end user is more likely to know that the e-mail service doesn't work than what part of the e-mail service doesn't work. This is a required field. **Tip** Tip You can use the Smart Indicator, positioned at the end of the field, to search for related incidents or problems.

Affected CI	The Service Desk Agent populates this field with the configuration item (CI). Click Fill to select from a list of the physical CIs that relate to the service. Other CIs can be entered manually. If the business service doesn't contain any CIs, then the list shows only the CIs that the service recipient is subscribed to and the CIs that are assigned to the service recipient. If you choose an Applications service, you are presented with a list of CIs in the service, as well as those that you own. This field includes a hover-over form that displays Critical CI and Pending Change checkboxes to indicate whether or not these attributes apply to the CI. After filling in the affected CI, the Service Desk Agent can use the Smart Indicator positioned at the end of the field to search for open and closed incidents for this CI, and to view the details.
Title	The Service Desk Agent populates this field with a brief description that identifies the interaction. **Note** Note Service Manager searches this field when you do an advanced or expert text search. This is a required field.
Tags	Enter one or more free-form tags (keywords or terms) of this interaction. The tags you enter are searchable and can be used as keywords to highlight important information. For more information, see Working with tags.
Description	The Service Desk Agent populates this field with a detailed description of the interaction. When the location and telephone number differ from the contact details, the Service Desk Agent can record the correct information in the description field. Clicking Search Knowledge searches description fields across multiple Service Manager knowledgebases for the text entered. Depending on the permissions of the user, Service Manager may look in interactions, incidents, problems, known errors, and knowledge documents. The Service Desk Agent can use the solution from any returned document as the solution for the interaction. **Note** Note Service Managersearches this field when you do an advanced or expert text search. This is a required field.

Completion Code	This field contains a predefined completion code, describing the way this issue has been solved. The out-of-box options in this field are based on Service Manager customer reference data. **Note** Note You may want to tailor these options to match your business needs. These completion codes are available out-of-box: • Automatically Closed • Cancelled • Fulfilled • Invalid • Not Reproducible • Out of Scope • Request Rejected • Solved by Change/Service Request • Solved by User Instruction • Solved by Workaround • Unable to solve • Withdrawn by User
Knowledge Source	This field contains the reference number of the document from the knowledge base document used to solve the issue. If you find a knowledge article by using Search Knowledge, and then click Use Solution in that article to provide the solution to your customer, this field is populated with the Document ID of the document you used. If you don't use a knowledge document or if you don't click Use Solution in the knowledge document, this field is left blank.
Solution	This field contains a description of the solution used for this interaction. **Note** Note Service Managersearches this field when you do an advanced or expert text search.

Category	This field describes the type of interaction. The interaction type determines the process to escalate to when the interaction can't be solved on first intake. The categories are based on ITIL service-centric processes, and therefore focus on enabling assignment, reporting, and operational analysis for knowledge management purposes. From the category list: • Complaint/Incident/Request for Information/Request for Administration > Escalate — You can relate the interaction to an existing incident, an existing known error, an existing problem, or create a new incident. • Request for Change > Escalate — Service Manager creates a new change request. • **More** or More Actions icon > Order From Catalog — Service Catalog opens, allowing you to place an order. The interaction is given the category service catalog. Service Catalog interactions aren't escalated. When you approve the interaction, it opens the related record as defined in the service catalog connector. For more information on categories and the subcategories and areas associated with them, see Interaction categories. This is a required field.
Subcategory	The Service Desk Agent populates this field with the subcategory of concern. Service Manager displays different lists of subcategories, depending on the category you selected. For more information on categories and the subcategories and areas associated with them, see Interaction categories. This is a required field.
Area	The third level of classifying an interaction, mainly used for reporting purposes. Service Manager displays different lists of areas, depending on the subcategory you selected. For more information on categories and the subcategories and areas associated with them, see Interaction categories. This is a required field.

Assignment Group	The group assigned to work on the interaction. For a description of this field see the Assignment Group field description in the Incident Management form details section as this field functions similarly. The out-of-box data consists of default assignment groups for use as examples of types of assignment groups. You may want to change the sample assignment groups to meet your own needs. These assignment groups are available out-of-box: • Application • Email / Webmail • Field Support • Hardware • Incident Managers • Intranet / Internet Support • Network • Office Supplies • Office Support • Operating System Support • Problem Coordinators • Problem Managers • SAP Support • Service Desk • Service Desk Analysts • Service Manager
Assignee	The name of the person assigned to work on this interaction. This person is a member of the assigned support group. Assignees may belong to one or multiple assignment groups, based on the needs of your company.
Impact	The Service Desk Agent populates this field with the impact the interaction has on the business. The impact and the urgency are used to calculate the priority. The impact is based on how much of the business is affected by the issue. The stored value can be 1-4, as follows. • 1 - Enterprise • 2 - Site/Dept • 3 - Multiple Users • 4 - User This is a required field.
Urgency	The urgency indicates how pressing the issue is for the service recipient. The urgency and the impact are used to calculate the priority. The stored value can be 1-4, as follows. • 1 - Critical • 2 - High • 3 - Average • 4 - Low This is a required field.

Priority	This field describes the order in which to address this interaction in comparison to others. It contains a priority value calculated by (impact + urgency)/2. Decimals are truncated. The stored value based on that calculation can be 1-4, as follows: • 1 - Critical • 2 - High • 3 - Average • 4 - Low
Approval Status	This field is only used when you request something from the catalog. When you submit an order from the catalog, Service Manager automatically creates an interaction which, based on approval requirements, may have to be approved before it can be fulfilled. Service Manager populates this field with the current approval status for this interaction. These approval statuses are available out-of-box: • pending — The request has not been approved or a prior approval or denial has been retracted. • approved — All approval requirements are approved, or no approval necessary. • denied — The request has been denied.
Activities	The Activities section records information that the Service Desk Agent enters during the lifecycle of the Interaction. Every time you update an interaction, you can fill in an update on the Activities section (New Update). A log of all the updates is stored on the Journal Updates and activities list. Activities from related records that are flagged as customer visible also display here.
Related Records	The Related Records section contains a list of all related records for the interaction. These may include related incidents, known errors, problems, changes, and quotes.
SLT	The SLT (Service Level Target) section displays SLAs related to the interaction. SLAs in interactions are customer-related and selected, based on the customer contact or department and service related to the issue. The Service Level Objective (SLO) defines the details, such as beginning and ending state, and time allowed between these states. SLA selection takes place when a Service Desk Agent escalates the interaction. The best practice is that the Service Desk Agent should communicate the time of the next breach to the customer at this point. If SLAs are configured to be handled in the background, the information in this section may not display immediately. **Note** Note The out-of-box system is set up to run SLAs in the foreground. Tailoring the system to run SLAs in the background complicates communicating with the customer and should be avoided.

SLT View	If you set the View Type property of the Calendar control to **Calendar** or leave this property blank, this tab displays a normal view of the embedded Service Manager calendar. You can view the time period records and associated business records in a graphic and intuitive user interface. For more information about the Service Manager Calendar, see Calendar control and Calendar Administration. If you set the View Type property of the Calendar control to **SLM**, this tab visualizes the operation status for all related SLA, OLA, and UC of the current record. For more information about SLM visualization, see Visualize Service Level Management.
Escalate button	The Service Desk Agent clicks this button to create an incident or change from this interaction. The customer's issue could not be solved immediately. When research time is required, the Interaction should be escalated to an incident or a change, not saved as an interaction. There is no monitoring on saved interactions, other than self-service interactions. If the Service Desk has a role in the Incident Management process, this incident may be assigned to the Service Desk, and the Service Desk Agent can still work on it. Clicking Escalate starts the Escalate Interaction solution matching. For more information on the Escalate Interaction solution matching, see Escalate Interaction solution matching.
Image to Text button	The Service Desk Agent clicks this button to extract texts from attached images and then put the text into the description field on the interaction records. **Note** Note The condition for the action in the Service Desk workflow is `(Expression: lioption("Smart Analytics") and $lo.idol.img.enabled and not ($G.ess) and jscall("IDOL_Utilities.getAttachmentLength", $L.file)>0)` .
Withdraw button	The Service Desk Agent clicks this button to withdraw the interaction on behalf of the user.
Close Interaction button	The Service Desk Agent clicks this button to close the interaction. The customer's issue was resolved and requires no further action.
Close Invalid Request button	If the interaction is invalid, click this button to close it as an invalid interaction in the Categorization phase.

9.3.4. Interaction categories

The category hierarchy was designed to support the ITIL 2011 model of service-centric support. It is a natural-language-based hierarchy meant to enable the Service Desk Agent to easily classify the Interaction. The three-level hierarchy (category, subcategory, and area) creates a “sentence” that clearly and uniquely defines the issue without ambiguity.

The category determines which process the record belongs to. Combined with the subcategory and area, it also is used for to report results and to determine the knowledgebase assignment for the event.

Note

Note Since the category values represent best practices, customizing this data is not expected. The subcategory and area fields can be customized; however, they should cover the scope of the IT Service provisioning in natural language definition and should remain unmodified. If you choose to customize the subcategories and areas, be sure to set them up in a natural easy-to-follow hierarchy.

The categories, subcategories, and areas that come with Service Desk out-of-box are captured in this table.

Categories, subcategories, and areas

Category	Subcategory	Area
complaint	service delivery	availability
complaint	service delivery	functionality
complaint	service delivery	performance
complaint	support	incident resolution quality
complaint	support	incident resolution time
complaint	support	person
incident	access	authorization error
incident	access	login failure
incident	data	data or file corrupted
incident	data	data or file incorrect
incident	data	data or file missing
incident	data	storage limit exceeded
incident	facilities	hardware failure
incident	facilities	miscellaneous
incident	facilities	supplies
incident	failure	error message
incident	failure	function or feature not working
incident	failure	job failed
incident	failure	system down
incident	hardware	hardware failure
incident	hardware	missing or stolen
incident	performance	performance degradation
incident	performance	system or application hangs
incident	security	security breach
incident	security	security event/message
incident	security	virus alert
request for administration	grant access	grant access
request for administration	other	other
request for administration	password reset	password reset
request for change	service portfolio	new service
request for change	service portfolio	upgrade / new release
request for information	general information	general information
request for information	how to	how to

request for information	status	status
service catalog	service catalog	service catalog

9.3.5. Escalate Interaction solution matching

Depending on your selection, the Escalate Interaction wizard opens one of the following wizards:

- Escalate Interaction - Incident
- Escalate Interaction - RFC
- Escalate Interaction - Request
- Escalate Interaction - Problem

Escalate Interaction - Incident

You can escalate Interactions in the following categories to Incident:

When you escalate an interaction, you can use queries to search for known errors, incidents, or problems matching the title, CI or service value of the interaction. If you find matching records, you can link the records to the interaction.

- compliant
- incident
- request for information
- request for administration

Escalate Interaction - RFC

You can escalate Interactions in the **request for change** category to RFC:

Option 1: If change is open on Category in settings, when escalating an interaction to an RFC, a list of change categories is displayed from which the category for the new change request can be selected.

Option 2: If change is open on Change Model in settings, when escalating an interaction to an RFC, a list of Change Models is displayed from which the Change Model for the new change request can be selected.

Escalate Interaction - Request

You can escalate Interactions in the following categories to Request:

- Request for information
- Request for administration

Escalate Interaction - Problem

You can escalate Interactions in the Problem category to Problem.

9.4. Incident Management overview

The Service Manager Incident Management application, referred to as Incident Management throughout this chapter, supports the Incident Management process. It provides comprehensive Incident Management that allows you to restore normal service operations as quickly as possible and minimize the adverse impact on business operations.

Incident Management enables you to categorize and track various types of incidents (such as service unavailability or performance issues and hardware or software failures) and to ensure that incidents are resolved within agreed-on service level targets.

This section describes how Incident Management implements the best practice guidelines for the Incident Management processes. Topics in this section include:

- Incident Management within the ITIL framework
- Incident Management application
- Incident Management process overview
- Input and output for Incident Management
- Key performance indicators for Incident Management
- RACI matrix for Incident Management

9.4.1. Incident Management within the ITIL framework

Service Operation publication. The document describes Incident Management as the process responsible for restoring normal service operation as quickly as possible.

The ITIL publication points out that Incident Management is highly visible to the business, and therefore it is often easier to demonstrate its value in comparison to other areas of Service Operation. These values include:

- the ability to detect and resolve incidents, resulting in lower downtime and higher service availability
- the ability to align IT activity to real-time business priorities
- the ability to identify potential improvements to services, and additional service or training requirements

9.4.2. Incident Management application

The Incident Management application automates reporting and tracking of a single incident or a group of incidents associated with a business enterprise. It enables you to categorize types of incidents, and keep track of their resolution.

With Incident Management, the appropriate people can escalate and reassign incidents. Incident Management can also escalate an incident to properly meet the agreed-upon terms of the service contract. For example, if a network printer is disabled, a technician or manager can escalate the incident to a higher priority to ensure that the incident is fixed quickly.

Incident Management restores normal service operation as quickly as possible and minimizes the adverse impact on business operations, thus ensuring that the best possible levels of service quality and availability are maintained. It includes events that are communicated directly by Users, either through the Service Desk or through an automated interface between Event Management and Incident Management tools.

Incident Management defines normal service operation as service performance to meet Service Level Agreement (SLA), Operation Level Agreement (OLA), and Underpinning Contract (UC) targets.

Incidents can be reported and logged by support staff, who may notify the Service Desk if they notice an issue. Not all events are logged as incidents. Many classes of events aren't related to disruptions at all, but are indicators of normal operation or are simply informational.

The new Incident Management best practices make some changes you may want to take into consideration when implementing your updated system.

Incident Closure process[edit]

Service Manager includes the Service Desk application to perform user interaction activities. Service Manager is configured out-of-box to use a one-step Incident Closure process. Therefore, incident personnel can close the incident directly after resolving it. The Service Desk takes care of notifying the end user and closing the interaction that initiated the incident.

Legacy Service Manager customers who did not activate Service Desk and used a two-step incident close will find that this is no longer necessary, because the Service Desk application is now included.

Contents

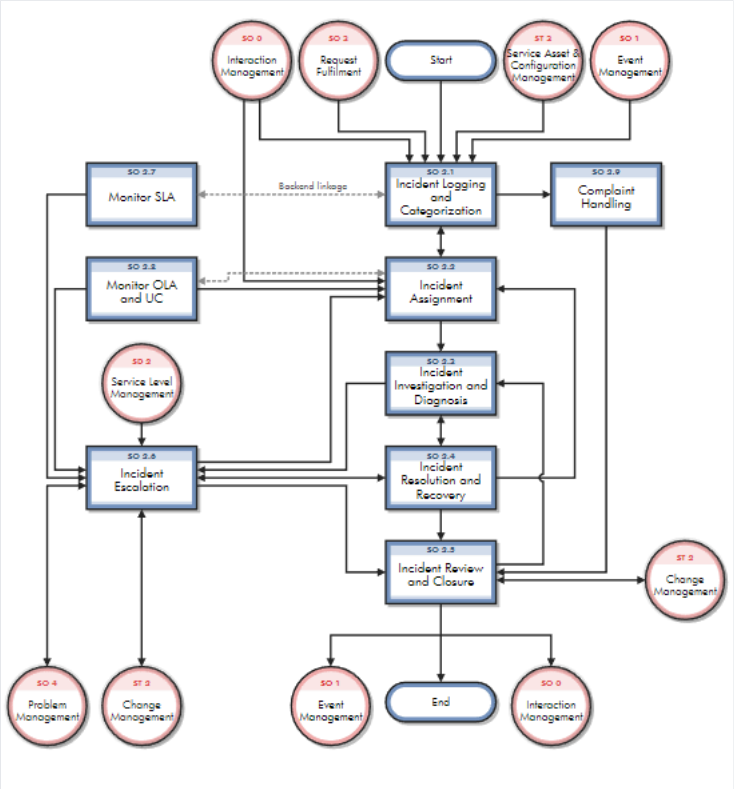
- 1 Incident Closure process

9.4.3. Incident Management process overview

The Incident Management process includes all necessary steps to log and resolve an incident, including any necessary escalations or reassignments. Monitoring of Service Level Agreements (SLAs), Operation Level Agreements (OLAs), and Underpinning Contracts (UCs) are also part of the overall process.

When an incident is opened, the associated SLA starts tracking the time that elapses. The Incident Coordinator assigns the incident to an Incident Analyst for investigation and diagnosis. If necessary, the incident can be reassigned to a different assignment group.

A general overview of the Incident Management processes and workflows is depicted in the figure below. They are described in detail in Incident Management Workflows.



9.4.3.1. Incident Management categories

Service Manager Categories classify and define the type of incident. Each category could have its own workflow process. The steps of the workflow are represented by the phases and tasks within the phase. Service Manager requires that every incident has an incident category and phases, but tasks are optional.

9.4.3.2. Incident Management phases

Service Manager uses phases to describe the steps needed to resolve an incident. The phase also determines the forms users see, the actions users can manually trigger.

The following figure shows the workflow phases for an Incident.



9.4.3.3. Incident Management tasks

Service Manager supports incident tasks to resolve an incident. Incident can't be closed if there are open Incident tasks underneath it. Except Incident Logging and Closure, each phase can optionally have one or multiple tasks, or no tasks. Each task includes a description, and information about urgency, priority, and assignment.

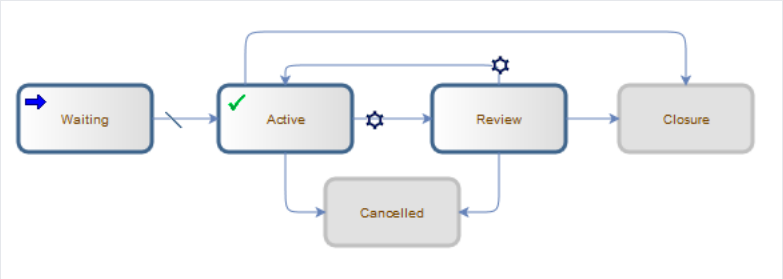
Incident Management tasks include:

- Opening, assigning a task from an incident.
- Searching for a task.
- Managing task categories, status and phases.
- Using the task queue.

9.4.3.4. Incident Task phases

This section describes the flow of an incident task as it progresses from the 'Waiting' phase to the 'Closed' phase in the Generic Task workflow.

The following figure shows the Incident Task workflow in Process Designer.



9.4.3.5. Incident Management user roles

The following table describes the responsibilities of the Incident Management user roles.

Incident Management User Roles and Responsibilities

Role	Responsibilities
Operator	Registers incidents based on an event and assigns them to the correct support group.
Service Desk Agent	• Register interactions based on contact with user. • Match user interaction to incidents, problems, known errors, or knowledge document. • Solve and close interactions. • Provide status updates to users on request. • Register incident based on a user interaction and assign to the correct support group. • Register Request for Change, based on a user interaction. • Register Service Request, based on a user interaction. • Validate a solution provided by a support group. • Report and verify a solution to a user. • Monitor Service Level Agreement (SLA) targets of all incidents registered and escalate, if required. • Communicate about service outages to all users.
Incident Analyst	• Reviews assigned incidents. • Investigates and diagnoses incidents. • Create incident tasks to investigate and diagnose incidents. • Documents incident resolutions or workarounds in the Service Management application. • Implements incident resolutions. • Verifies that incidents are resolved and closes them. • Create Incident tasks to implement incident resolutions.
Incident Coordinator	• Reviews incidents assigned to the support group. • Handles incidents escalated by an Incident Analyst of the support group. • Monitors Operational Level Agreements (OLA) and Underpinning Contracts (UC) targets of the support group.
Incident Manager	• Handles incidents escalated by the Incident Coordinator or by the Service Desk Agent. • Determines and executes the appropriate escalation actions. • Requests an Emergency Change, if required.

9.4.4. Input and output for Incident Management

Incidents can be triggered and resolved in several ways. The following table outlines the inputs and outputs for the Incident Management process.

Input and output for Incident Management

Input to Incident Management	Output from Incident Management
• Customer interactions with the Service Desk, which can be triggered to incidents • Event management tool, which automatically opens incidents • Support staff	• Resolved incidents • Documented workarounds, solutions, or knowledge articles • New problems, changes, or incidents Incidents can also trigger several other Service Manager processes, as described in the next section.

9.4.5. Key performance indicators for Incident Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your Incident Management processes. To visualize trend information, it is useful to graph KPI data periodically. In addition to the data provided by Service Manager, you may need additional tools to report on all of your KPI requirements.

Key Performance Indicators for Incident Management

Title	Description
% of incidents closed within SLA target time	The number of incidents closed within the SLA target time, relative to the number of all incidents closed, in a given time period.
Backlog of incidents	The number of incidents that aren't yet closed, in a given time period.
Total number of incidents	Total number of new reported incidents, in a given time period.

For completeness, the ITIL 2011 and COBIT 4.1 KPIs are included below.

ITIL 2011 Key Performance Indicators[edit]

The following are ITIL 2011 KPIs for Incident Management:

- Total number of incidents (as a control measure)
- Breakdown of incidents at each stage (for example, logged, work in progress, and closed)
- Size of current incident backlog
- Number and percentage of major incidents
- Mean elapsed time to achieve incident resolution or circumvention, separated by impact code
- Percentage of incidents handled within target response time; incident response-time targets may be specified in SLAs, for example, by impact and urgency codes
- Average cost per incident
- Number of incidents reopened and as a percentage of the total
- Number and percentage of incidents incorrectly assigned
- Number and percentage of incidents incorrectly categorized
- Number and percentage of incidents resolved remotely, without the need for a visit
- Number of incidents handled by each incident model
- Breakdown of incidents by time of day, which helps pinpoint peaks and ensure matching of resources

COBIT 4.1 Key Performance Indicators[edit]

The following are the COBIT 4.1 KPIs for Incident Management:

- Percent of incidents resolved within the time period specified
- Percent of incidents reopened
- Average duration of incidents by severity
- Percent of incidents that require local support (that is, field support or a personal visit)

Contents

- 1 ITIL 2011 Key Performance Indicators
- 2 COBIT 4.1 Key Performance Indicators

9.4.6. RACI matrix for Incident Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. It is especially useful in clarifying roles and responsibilities in cross-functional/departmental projects and processes. The RACI matrix for Incident Management is shown in the following table.

RACI Matrix for Incident Management

Process ID	Activity	Incident Manager	Incident Coordinator	Incident Analyst	Incident Operator	Service Desk Agent	Service Desk Manager	User
SO 2.1	Incident Logging and Categorization	A	I		R	R		
SO 2.2	Incident Assignment	A	R	R				
SO 2.3	Incident Investigation and Diagnosis	A	C/I	R				C/I
SO 2.4	Incident Resolution and Recovery	A	C/I	R				C/I
SO 2.5	Incident Review and Closure	A	C/I	R	I	I		I
SO 2.6	Incident Escalation	R/A	R	I				
SO 2.7	SLA Monitoring	A/I	I	I		R		
SO 2.8	OLA and UC Monitoring	A/I	R	I				
SO 2.9	Complaint Handling	A/I					R	C/I

9.5. Incident Management workflows

The Incident Management process logs, investigates, diagnoses, and resolves incidents. Incidents can be initiated by the escalation of Service Desk interactions or automatically detected and reported by event monitoring tools. The process includes all necessary steps to log and resolve an incident, including any necessary escalations or reassignments.

The Incident Management process consists of the following processes, which are included in this chapter:

- Incident Logging and Categorization (process SO 2.1)
- Incident Assignment (process SO 2.2)
- Incident Investigation and Diagnosis (process SO 2.3)
- Incident Resolution and Recovery (process SO 2.4)
- Incident Review and Closure (process SO 2.5)

- Incident Escalation (process SO 2.6)
- SLA Monitoring (process SO 2.7)
- OLA and UC Monitoring (process SO 2.8)

9.5.1. Incident Logging and Categorization (process SO 2.1)

Incidents are initiated and logged as part of the Interaction Management or the Event Management process, depending on the source and nature of the incident. All relevant information relating to incidents must be logged so that a full historical record is maintained. By maintaining accurate and complete incidents, future assigned support group personnel are better able to resolve recorded incidents.

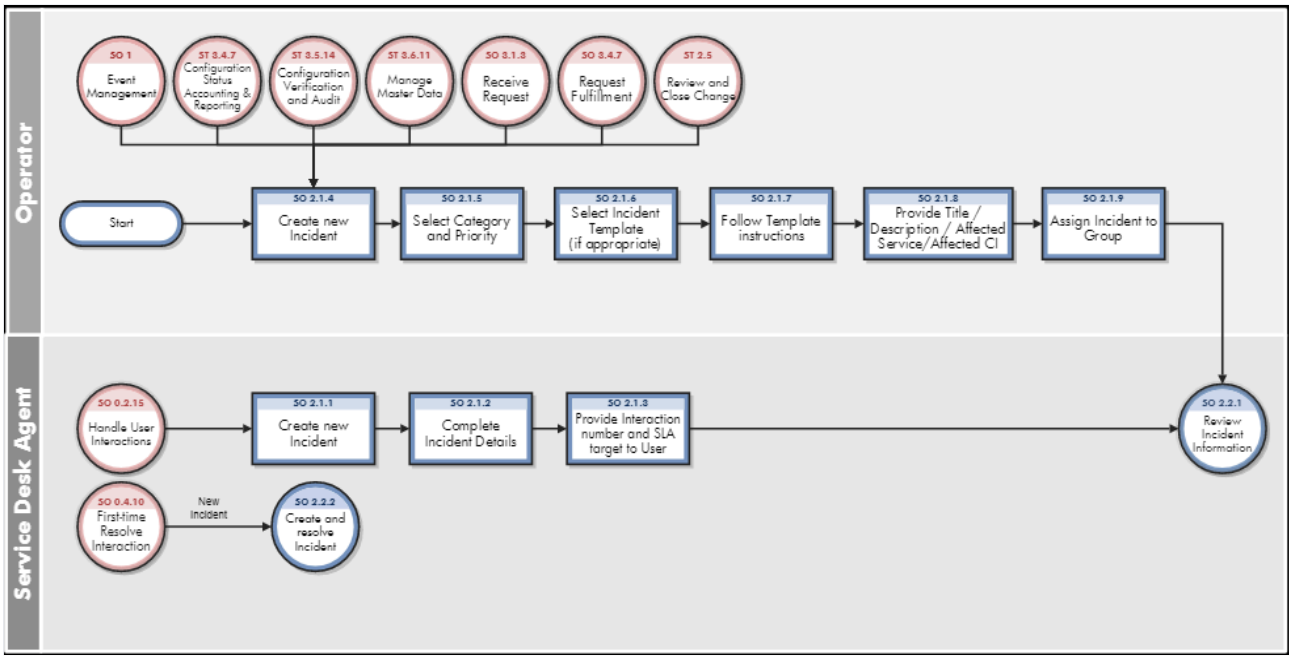
- If the incident is logged by the Service Desk Agent, most incident details are already provided by the interaction record. The Service Desk Agent verifies the Assignment Group to make sure the selected group is the most suitable group to solve the incident.
- If an incident is logged by an Operator, usually by using a system management tool, the incident must be based on the applicable incident model.

Operators and Service Desk Agents can perform the following Incident Logging tasks:

- Create new incident from monitoring system notification (Operator)
- Create new incident from user interaction (Service Desk Agent)
- Review and update incident information (Service Desk Agent)

You can see the details of this process in the following figure and table.

The Incident Logging and Categorization workflow is illustrated in the following figure:



Incident Logging and Categorization process

Process ID	Procedure or Decision	Description	Role
SO 2.1.1	Create new incident	A User interaction can't be solved on first intake and is escalated to the Service Manager process. The interaction is automatically related to the newly created incident. The Service Desk Analyst creates an incident from an interaction.	Service Desk Agent
SO 2.1.2	Complete Incident Details	Based on the categorization and the affected services, the incident is automatically assigned to the responsible support group. The Service Desk Analyst verifies that the assignment is correct.	Service Desk Agent
SO 2.1.3	Provide interaction number and SLA target to User	The Service Desk Analyst provides the interaction number to the User. The User keeps the interaction number as a reference to the incident. The Service Desk Analyst also provides a target solution date based on the SLA.	Service Desk Agent
SO 2.1.4	Create new Incident	An Incident is detected when monitoring the IT infrastructure. The Operator (or Initiator) decides to create an Incident manually or an Incident is generated automatically, depending on tool settings. If a new service request is identified as an incident or an incident occurs related to fulfillment activities, the Operator creates an incident from a service request. Go to SO 2.1.10 to select an Incident template (if appropriate).	Operator
SO 2.1.5	Select Category and Priority	Select the suitable Category and Priority by selecting the applicable impact level and urgency.	Operator
SO 2.1.6	Select Incident template (if appropriate)	The Operator (or Initiator) selects an incident template from a list, or a template is selected automatically, depending on the settings.	Operator

SO 2.1.7	Follow template instructions	The Operator (or Initiator) provides and records the incident details based on the instructions provided by the incident template. The template instructions may filled in by predefined scripts.	Operator
SO 2.1.8	Provide Title/Description/Affected Service/Affected CI	Provide a suitable title and description for the incident. This might be based on the event text. If possible, the affected Service, affected Configuration Item should be selected.	Operator
SO 2.1.9	Assign incident to group	The incident is manually assigned to the responsible support group, based on the incident categorization and the associated affected services.	Operator

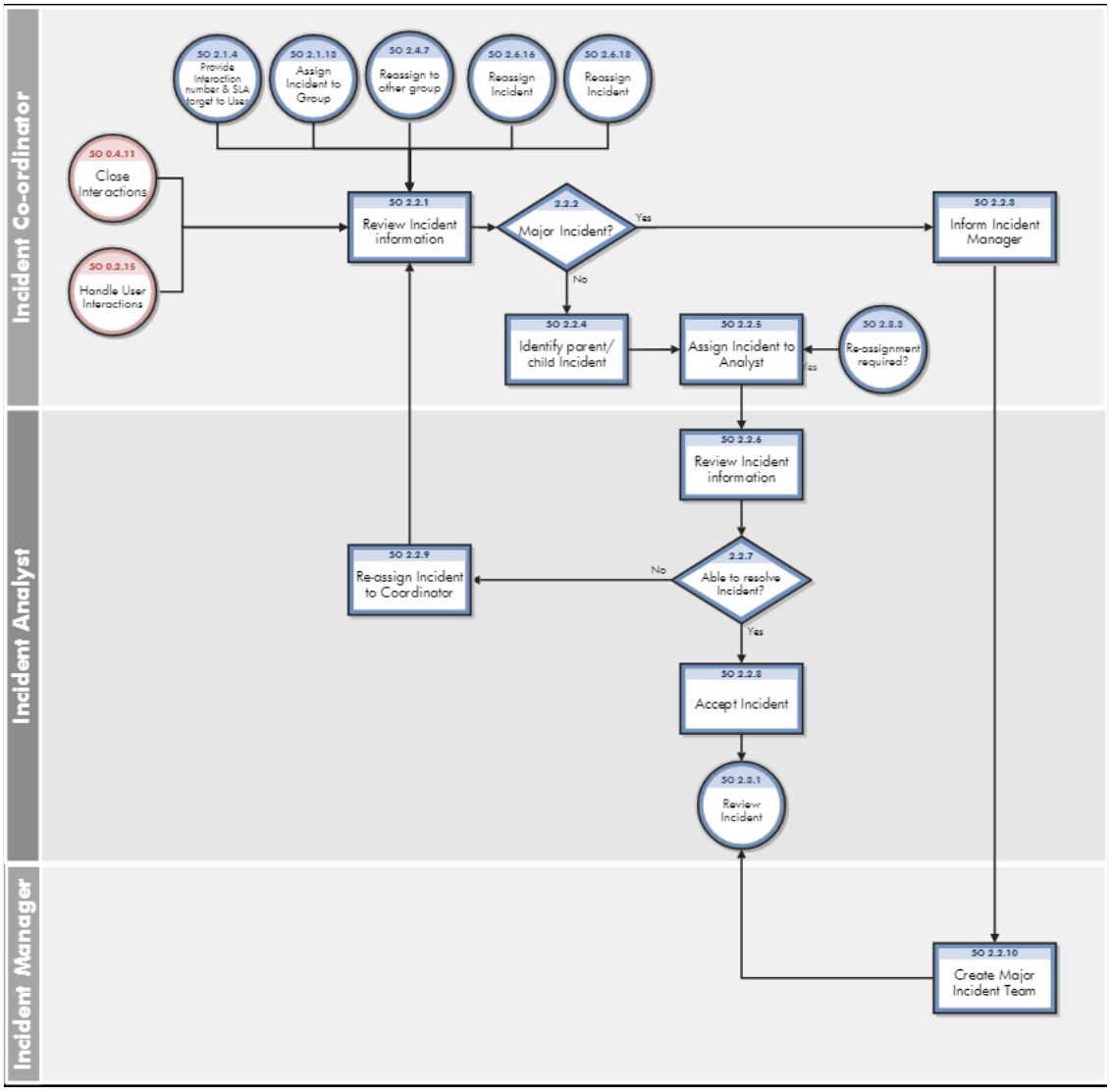
9.5.2. Incident Assignment (process SO 2.2)

Incidents are logged from an interaction by a Service Desk Agent or from an event by an Incident Coordinator monitors the incident queue and reviews open status incidents. The Incident Coordinator verifies whether an incident is a major incident using predefined criteria. If it is, the Incident Manager is informed about the incident arrival; otherwise, it is assigned to an Incident Analyst for further investigation and diagnosis.

The Incident Analyst receives an assigned incident and determines whether the incident can be resolved with the tools and knowledge available. If the incident can't be resolved, the Incident Analyst reassigns it to the Incident Coordinator.

You can see the details of this process in the following figure and table.

The Incident Assignment workflow is illustrated in the following figure:



Incident Assignment process

Process ID	Procedure or Decision	Description	Role
SO 2.2.1	Review Incident information	The Incident Coordinator monitors the incident queue and reviews all incoming Incidents.	Incident Coordinator
SO 2.2.2	Major Incident?	The Incident Coordinator verifies whether this incident is a major incident using predefined criteria. If yes, continue with SO 2.2.3. If no, go to SO 2.2.4	Incident Coordinator
SO 2.2.3	Inform Incident Manager	The Incident Coordinator informs the Incident Manager about the major incident. The Major Incident checkbox is checked and the ticket is assigned to the manager. Automatic notifications about the Major Incident arrival are sent to Incident Manager. Then go to SO 2.2.9.	Incident Coordinator
SO 2.2.4	Identify parent/child Incident	The operator identifies whether there is a need to create a parent-child record relationship to link multiple similar incidents together.	Operator
SO 2.2.5	Assign Incident to analyst	The Incident Coordinator assigns it to an Incident Analyst from the Incident Coordinator's group for further investigation and diagnosis.	Incident Coordinator
SO 2.2.6	Review Incident information	The Incident Analyst monitors the queue of incidents assigned to him/her and reviews the incoming incidents.	Incident Analyst
SO 2.2.7	Able to resolve Incident?	The Incident Analyst reviews the assigned incident to see if he/she can resolve it. If yes, continue with SO 2.2.7. If no, go to SO 2.2.8.	Incident Analyst
SO 2.2.8	Accept Incident	The Incident Analyst accepts the incident.	Incident Analyst

SO 2.2.9	Reassign Incident to coordinator	The Incident Analyst reassigns the incident to the Incident Coordinator if no resolution can be found. The analyst also provides the information on the current status, work performed on the incident, and information on reassignment. The Incident Coordinator can decide whether to escalate the incident, reassign the incident, or close the incident.	Incident Analyst
SO 2.2.10	Create Major Incident Team	The Incident Manager dynamically establishes a separate major incident team under the Incident Manager's direct leadership. The team is tasked to concentrate on this incident exclusively to ensure that adequate resources and focus are provided to find a swift resolution. The Incident Manager forms separate technical teams. Throughout, the Incident Coordinator and Service desk ensures that all activities are recorded and Users are kept updated and fully informed of the progress. A separate procedure with shorter timescales and greater urgency must be used for major incidents.	Incident Manager

9.5.3. Incident Investigation and Diagnosis (process SO 2.3)

Each support group involved with handling incidents must perform investigation and diagnosis tasks to determine the categorization of and solution to the incident. All actions performed by support group personnel are documented in the incident, so that a complete historical record of all activities is maintained at all times.

Incident Investigation and Diagnosis includes the following actions:

- Establishing the exact cause of the incident
- Documenting user requests for information or for particular actions or outcomes
- Understanding the chronological order of events
- Confirming the full impact of the incident, including the number and range of users affected
- Identifying any events that could have triggered the incident (for example, a recent change or user action)
- Searching known errors or the knowledgebase for a workaround or resolution
- Discovering any previous occurrences, including previously logged incident or problems and known errors, the knowledgebase, and error logs and knowledgebases of associated manufacturers and suppliers
- Identifying and registering a possible resolution for the incident

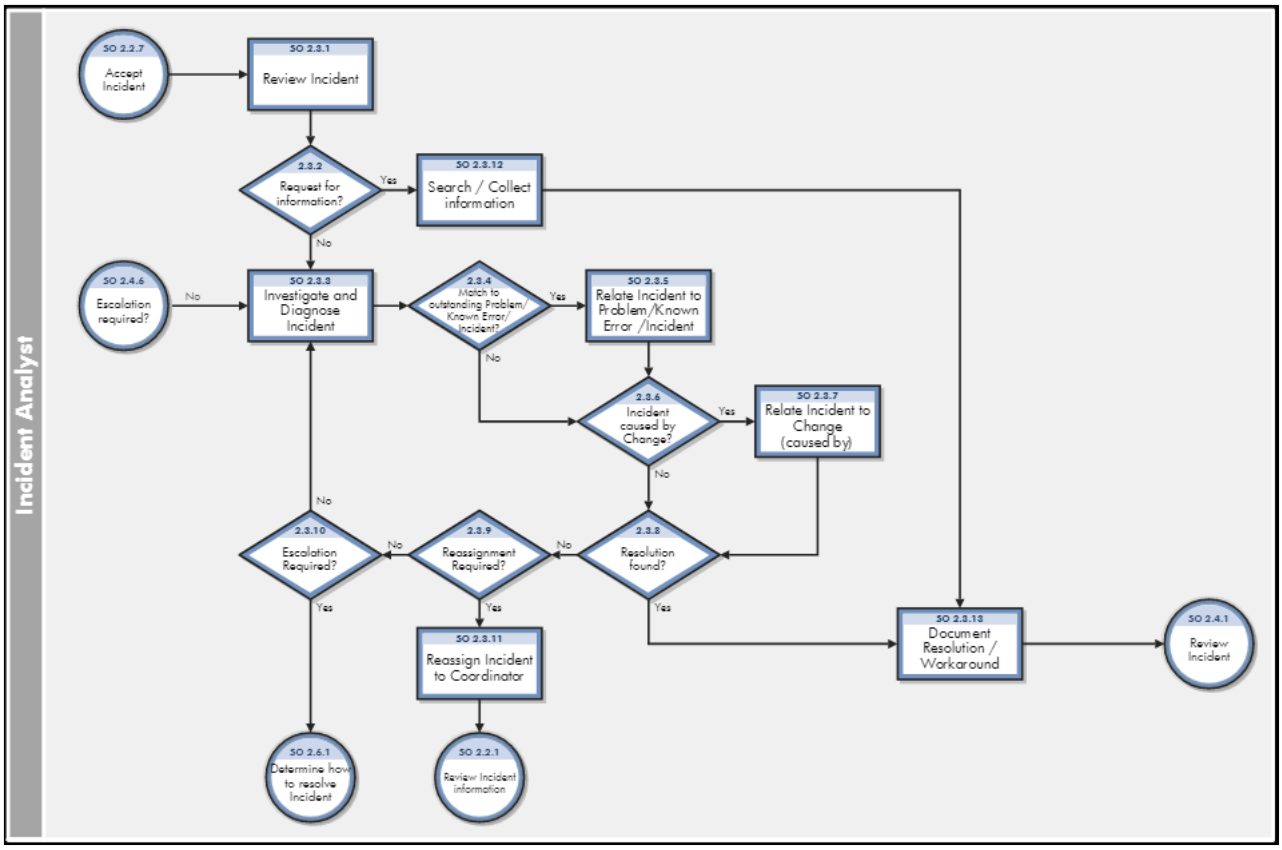
The Incident Analyst asks the following questions to determine how to resolve an incident:

- Is there a problem?
- Do I have the knowledge and tools to solve this problem?
- Can the incident be reproduced?

- Can the incident be related to an open problem or known error?
- Was the incident caused by the implementation of a change?
- Can a solution be found for this incident?

You can see the details of this process in the following figure and table.

The Incident Investigation and Diagnosis workflow is illustrated in the following figure:



Incident Investigation and Diagnosis process

Process ID	Procedure or Decision	Description	Role
SO 2.3.1	Review Incident	The Incident Analyst monitors the queue of incidents assigned to him/her and reviews the incoming incidents.	Incident Analyst
SO 2.3.2	Request for information?	The Incident Analyst evaluates the incident to see if it is categorized as a Request for Information (RFI) or if it is a service disruption. If it is a RFI, continue with SO 2.3.12. If it is a service disruption, go to SO 2.3.3.	Incident Analyst
SO 2.3.3	Investigate and Diagnose Incident	The Incident Analyst starts to investigate and diagnose the cause of the incident. The status of the incident is set to Work in Progress. Tasks could be created for carrying out the investigation activities.	Incident Analyst
SO 2.3.4	Match to outstanding Problem/ Known Error/ Incident?	The Incident Analyst searches the problem database to see if there is already a problem or known error defined for this incident. If yes, continue with SO 2.3.5. If no, go to SO 2.3.6.	Incident Analyst
SO 2.3.5	Relate incident to Problem/ Known Error/ Incident	When an incident matches an outstanding problem or known error, the incident is related to the problem or known error record.	Incident Analyst
SO 2.3.6	Incident caused by change?	The Incident Analyst searches the changes database to see if a recent change may have caused the service disruption. If the configuration item associated with the incident is listed, the Incident Analyst can also look at any changes that have recently been performed against this configuration item. The Incident Analyst can also view the configuration item tree to discover if related configuration items could have caused the incident. If yes, continue with SO 2.3.7. If no, go to SO 2.3.8.	Incident Analyst

SO 2.3.7	Relate incident to change (caused by)	When the incident is caused by a previous change, the incident is related to the change request. A solution still needs to be found to solve the incident.	Incident Analyst
SO 2.3.8	Resolution found?	The Incident Analyst checks the known error/knowledgebase for a workaround or resolution to this incident, or tries to find a solution. If yes, continue with SO 2.3.13. If no, go to SO 2.3.9.	Incident Analyst
SO 2.3.9	Reassignment Required?	If reassignment is required, go to SO 2.3.11. Otherwise, go to SO 2.3.10.	Incident Analyst
SO 2.3.10	Escalation Required	If a solution has not been identified review whether to escalate the Incident to the Incident Coordinator. If yes, go to SO 2.6.1 to determine how to resolve the Incident. If not, go to SO 2.3.3.to continue investigation and diagnosis of the Incident.	Incident Analyst
SO 2.3.11	Reassign Incident to Coordinator	The Incident Analyst reassigns the incident to the Incident Coordinator if no resolution can be found. The analyst also provides information on the current status, work performed on the Incident, and information on reassignment. The Incident Coordinator can decide whether to escalate the incident, reassign the incident, or close the incident.	Incident Analyst
SO 2.3.12	Search Collect information	The Incident Analyst searches for information to provide the requested information to the User.	Incident Analyst
SO 2.3.13	Document Resolution/Workaround	The Incident Analyst documents the solution or workaround in the incident.	Incident Analyst

9.5.4. Incident Resolution and Recovery (process SO 2.4)

As part of the Incident Resolution and Recovery process, the Incident Analyst identifies and evaluates potential resolutions before those resolutions are applied and escalates incidents as necessary. The Incident Analyst may escalate an incident to the Incident Coordinator, including those incidents that require a change. If the Incident Analyst doesn't have the required level of permissions to implement a change, the Incident Analyst reassigns the incident to another group that can implement the resolution. As soon as it becomes clear that

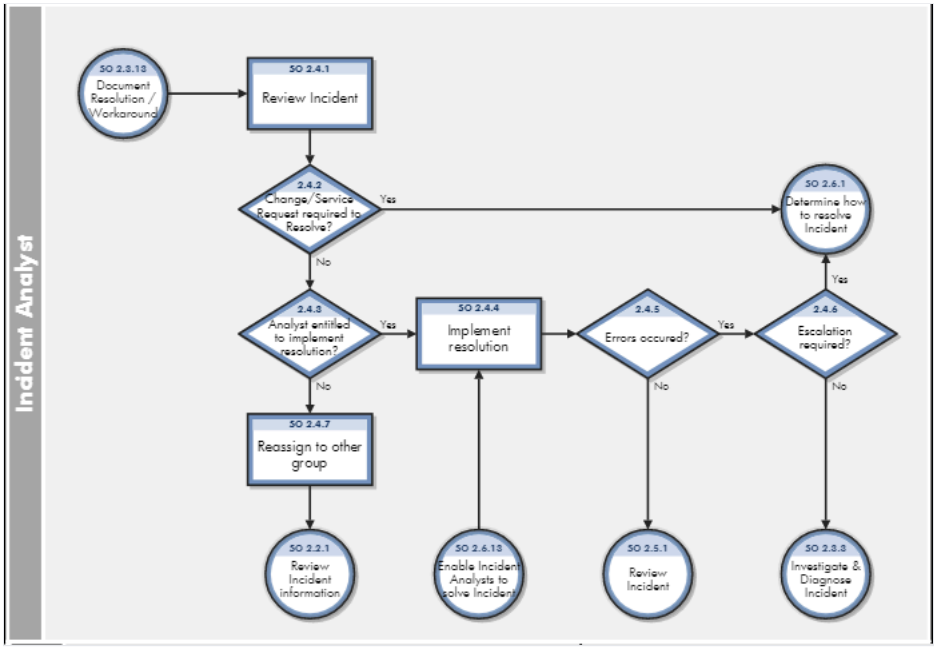
the assigned support group is unable to resolve the incident or if the target time period for first-point resolution is exceeded, the incident must be immediately escalated.

The objectives of the Incident Resolution and Recovery process are to ensure that:

- Recorded incidents include a resolution or workaround and information is complete.
- Incidents that require a change are escalated to the Incident Coordinator.
- Incidents for which the Incident Analyst has the required level of permissions are tested and implemented by the Incident Analyst in a production environment.
- Any incidents that the Incident Analyst doesn't have permissions to implement are reassigned to the applicable group for resolution implementation.
- Any implementation errors that occur during incident resolution correctly trigger resolution reversal and reinvestigation and diagnosis of the incident.
- The Incident Analyst initiates all required escalations.

You can see the details of this process in the following figure and table.

The Incident Resolution and Recovery workflow is illustrated in the following figure:



Incident Resolution and Recovery process

Process ID	Procedure or Decision	Description	Role
SO 2.4.1	Review Incident	The Incident Analyst reviews the incident information for the supplied resolution or workaround.	Incident Analyst
SO 2.4.2	Change/ Service Request required to Resolve?	The Incident Analyst determines whether the resolution provided needs to be implemented by using a Change or Service Request. If yes, go to SO 2.6.1 for the Incident Coordinator to determine how to resolve the Incident. If not, go to SO 2.4.3 to determine whether the Analyst is entitled to implement the resolution.	Incident Analyst
SO 2.4.3	Analyst entitled to implement resolution?	The Incident Analyst must judge if he/she has the permissions to implement the resolution. If yes, continue with SO 2.4.4. If no, go to SO 2.4.7.	Incident Analyst
SO 2.4.4	Implement resolution	The Incident Analyst tests the resolution and implements it in the production environment. Incident tasks can be created for resolution implementation if required.	Incident Analyst
SO 2.4.5	Errors occurred?	When there are errors during the implementation of a resolution, the Incident Analyst reverses the solution and the incident is returned to the investigation and diagnosis phase. If yes, go to SO 2.4.6. If no, continue with SO 2.5.1.	Incident Analyst
SO 2.4.6	Escalation required?	Determine if escalation to the Incident Coordinator is required at this point in the resolution process. If yes, go to the Incident Escalation process. If no, go to SO 2.3.3.	Incident Analyst
SO 2.4.7	Reassign to other group	When the Incident Analyst is not entitled to implement the solution, the analyst must reassign the incident to a support group or applicable vendor that can implement the solution.	Incident Analyst

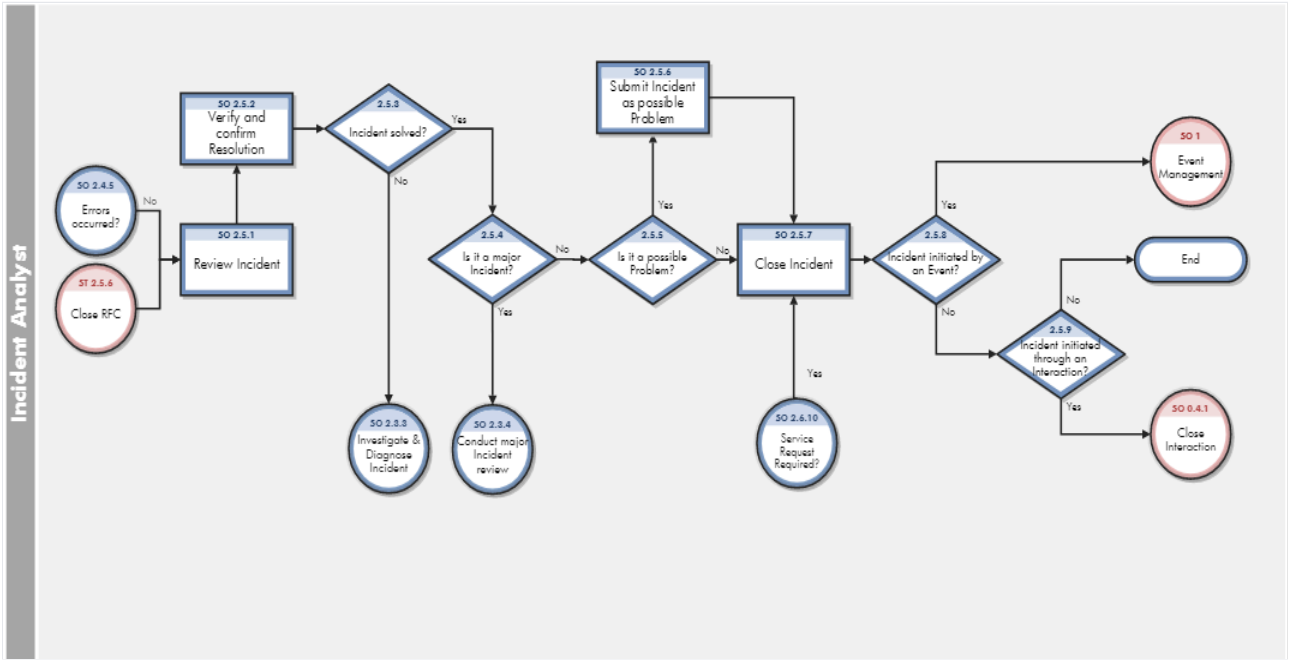
9.5.5. Incident Review and Closure (process SO 2.5)

The Incident Review and Closure process includes many steps to verify the success of implemented solutions and to verify that incidents are accurate and complete.

After a solution is implemented for an incident, the solution must be verified, typically by the group that implemented the solution. If necessary, the user can be contacted to verify the solution. The resolving group closes the incident and notifies the Service Desk to close the related interaction. When closing an incident, the likelihood of the incident recurring is determined and Problem Candidate is selected accordingly.

You can see the details of this process in the following figure and table.

The Incident Review and Closure workflow is illustrated in the following figure:



Incident Review and Closure process

Process ID	Procedure or Decision	Description	Role
SO 2.5.1	Review incident	The Incident Analyst reviews the incident resolution description.	Incident Analyst
SO 2.5.2	Verify and confirm Resolution	The Incident Analyst verifies that the resolution is correct and complete and confirms the resolution. If required, the Incident Analyst is entitled to contact the User (see SO 2.7.3) to validate the resolution.	Incident Analyst
SO 2.5.3	Incident solved?	Is the incident solved with the offered resolution? If yes, continue with SO 2.5.4. If no, go to SO 2.3.3.	Incident Analyst
SO 2.5.4	Is it a major Incident?	Major incident requires additional review at the stage. If it is a major incident, problem management activities may need to be undertaken to identify the root cause and take appropriate actions and plans to prevent future major incidents from occurring.	Incident Analyst
SO 2.5.5	Is it a possible Problem?	If the incident can be a possible problem, it is marked as possible problem and closed. Else the incident is closed without being marked as a possible problem.	Incident Analyst
SO 2.5.6	Submit Incident as possible Problem	The Incident Analyst submits the incident to Problem Management as a possible problem by checking the option to mark the Incident as a problem candidate.	Incident Analyst
SO 2.5.7	Close Incident	The Incident Analyst closes the incident and selects the applicable resolution code.	Incident Analyst
SO 2.5.8	Incident initiated by an Event?	Was the incident initiated by an event? If yes, then the event must be confirmed by using the event management process. If no, go to SO 2.5.8.	Incident Analyst
SO 2.5.9	Incident initiated through an Interaction?	Was the incident initiated by an interaction? If yes, continue with the Interaction Closure process. If no, then stop.	Incident Analyst

9.5.6. Incident Escalation (process SO 2.6)

When an Incident Analyst is unable to solve an assigned incident within the target time, the analyst escalates the incident to the Incident Coordinator. The Incident Coordinator determines how the incident can best be resolved by consulting the Incident Analyst and, if needed, other Incident Analysts. If an incident is severe (for example, designated as Priority 1), the appropriate IT managers must be notified so that they can anticipate and prepare for an escalation.

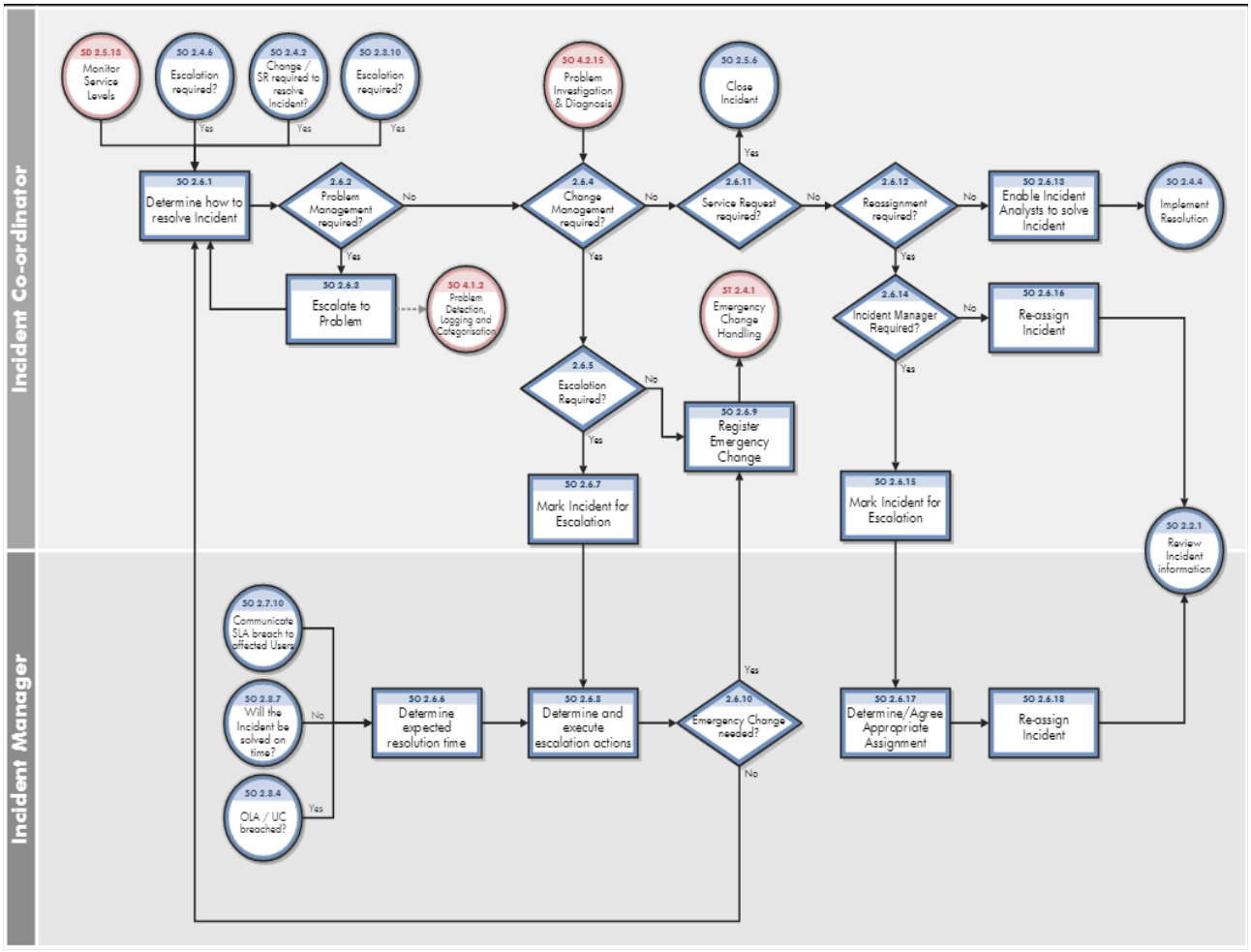
Incidents are escalated when the Incident Investigation and Diagnosis process or Incident Resolution and Recovery process exceeds SLA targets or if these targets are likely not to be met. If the steps to resolve an incident are taking too long or proving too difficult, the Incident Coordinator determines the following:

- Whether an Incident Analyst can be given the necessary resources to solve the incident
- Whether a change needs to be implemented
- Whether a request for service is needed

When an incident is escalated, the escalation should continue up the management chain. Senior managers are notified of the situation so that they can prepare to take any necessary actions, such as allocating additional resources or involving suppliers.

You can see the details of this process in the following figure and table.

The Incident Escalation workflow is illustrated in the following figure:



Incident Escalation process

Process ID	Procedure or Decision	Description	Role
SO 2.6.1	Determine how to resolve Incident	The Incident Coordinator gathers information from the Incident Analyst(s) about the status of the incident resolution and determines how the incident can best be resolved. The Incident Coordinator verifies that the expected resolution time matches any agreed on level, such as that specified in an SLA.	Incident Coordinator
SO 2.6.2	Problem Management required?	Is problem management required to solve the incident? If yes, continue with SO 2.6.3. If no, go to SO 2.6.4.	Incident Coordinator
SO 2.6.3	Escalate to Problem	Go to SO 2.6.1 to determine how to resolve the Incident.	Incident Coordinator
SO 2.6.4	Change Management required?	Is a change is required to solve the incident? If yes, continue with SO 2.6.5. If no, go to SO 2.6.11.	Incident Coordinator
SO 2.6.5	Escalation required?	Determine whether escalation is required to the Incident Manager to review what action to take with the Change Request. If yes go to SO 2.6.7 to mark the Incident for escalation. If not, go to SO 2.6.9 to Register Emergency Change	Incident Coordinator
SO 2.6.6	Determine expected resolution time	The Incident Manager verifies that the expected resolution time meets SLA targets.	Incident Manager
SO 2.6.7	Mark Incident for Escalation	Mark Incident for Escalation The Incident Coordinator checks the Escalation checkbox in the incident record and marks the Incident for Escalation. A notification is sent to the Incident Manager informing him/her of escalation.	Incident Coordinator

SO 2.6.8	Determine and execute escalation actions	The Incident Manager determines the actions to be performed to solve the incident within target times and designates escalation personnel to contact in the event of an escalation. This can include determining that the Service Desk is required to send an information bulletin to the affected users and stakeholders.	Incident Manager
SO 2.6.9	Register emergency change	The Incident Coordinator registers an emergency change request and contacts the Change Manager to inform the manager about the request, thereby starting the Emergency Change Handling process.	Incident Coordinator
SO 2.6.10	Emergency change needed?	If yes, go to SO 2.6.9. If no, go to SO 2.6.1.	Incident Manager
SO 2.6.11	Service Request required?	If yes, close the Incident. If not, go to SO 2.6.12.	Incident Coordinator
SO 2.6.12	Reassignment required?	Is it necessary to reassign the incident to a different support group with more knowledge (that is, a functional escalation)? If yes, continue with SO 2.6.14. If no, go to SO 2.6.13.	Incident Coordinator
SO 2.6.13	Enable Incident Analysts to solve incident	The Incident Coordinator enables the Incident Analyst(s) to focus solely on the resolution of the incident and provides the Incident Analyst(s) with all means necessary to speed up the resolution. Go to SO 2.4.4.	Incident Coordinator
SO 2.6.14	Incident Manager required?	Escalation may be required for the Incident Manager to agree the appropriate assignment for the Incident. This may be required where there is a dispute over which group should take ownership of the Incident. If the Incident Manager must get involved, go to SO 2.6.15. If not, go to SO 2.6.16.	Incident Coordinator

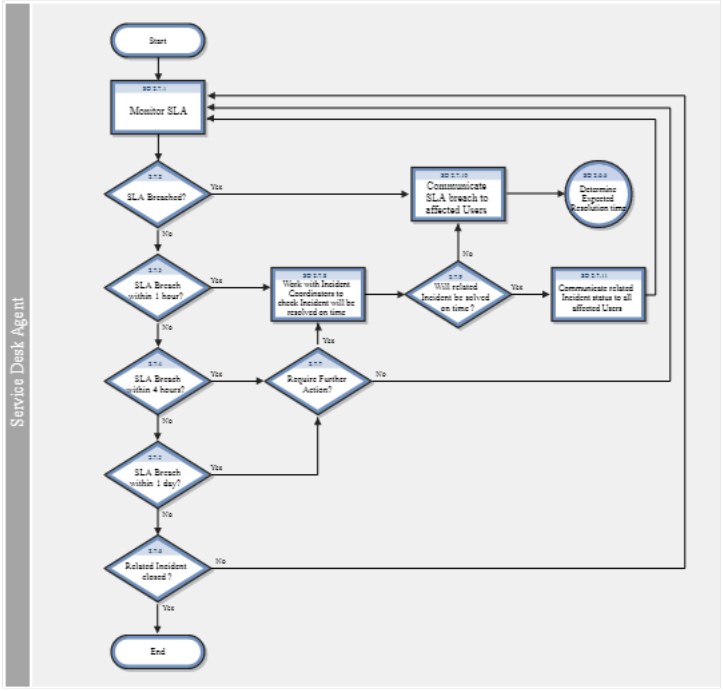
SO 2.6.15	Mark Incident for Escalation	The Incident Coordinator checks the Escalation checkbox in the incident record and marks the Incident for Escalation. A notification is sent to the Incident Manager informing him/her of escalation. Then go to SO 2.6.17.	Incident Coordinator
SO 2.6.16	Reassign incident	The Incident Coordinator reassigns the incident to another 2nd-line or 3rd-line support group.	Incident Coordinator
SO 2.6.17	Determine/ Agree appropriate assignment	The Incident Manager reviews the Incident to determine the appropriate Assignment Group based on the skills/ knowledge or permissions required to resolve the Incident.	Incident Manager
SO 2.6.18	Reassign incident	The Incident Manager reassigns the incident to another 2nd-line or 3rd-line support group.	Incident Manager

9.5.7. SLA Monitoring (process SO 2.7)

Service level agreements (SLAs) contain standards for incident resolution performance. This process describes the activities to monitor all interactions related to incidents from initialization to resolution. SLA Monitoring also determines whether time targets for incident resolution are met, and indicates whether escalation is required to meet the target resolution date according to the associated SLA. SLA Monitoring is an ongoing process performed by the Service Desk.

You can see the details of this process in the following figure and table.

The SLA Monitoring workflow is illustrated in the following figure:



SLA Monitoring process

Process ID	Procedure or Decision	Description	Role
SO 2.7.1	Monitor SLA	The Service Desk Agent monitors the SLA.	Service Desk Agent
SO 2.7.2	SLA breached?	Has the SLA target date/time been exceeded for this interaction? If yes, go to SO 2.7.10, and then start the Incident Escalation process. If no, go to SO 2.7.3.	Service Desk Agent
SO 2.7.3	SLA breach within 1 hour	Does the interaction need to be solved within 1 hour to reach the SLA target date/time? If yes, go to SO 2.7.8. If no, go to SO 2.7.4.	Service Desk Agent
SO 2.7.4	SLA breach within 4 hours?	Does the interaction need to be solved within 4 hours to reach the SLA target date/time? If yes, go to SO 2.7.7. If no, go to SO 2.7.5.	Service Desk Agent
SO 2.7.5	SLA breach within 1 day?	Does the interaction need to be solved within 1 day to reach the SLA target date/time? If yes, go to SO 2.7.7. If no, go to SO 2.7.6.	Service Desk Agent
SO 2.7.6	Related incident closed?	If yes, no further action is required. If no, go to SO 2.7.1.	Service Desk Agent
SO 2.7.7	Request further action?	Review the Incident and determine whether further action is required to ensure that it will be resolved within the SLA target date/time. If yes, go to SO 2.7.8 to work with the Incident Coordinators) to check the Incident will be resolved on time. If not, go to SO 2.7.1 to continue to monitor the SLA.	Service Desk Agent
SO 2.7.8	Work with Incident Coordinator(s) to see if incident can still be solved on time	Contact the Incident Coordinator with the related incident assigned to his/her group. Determine whether the group is able to solve the incident on time without further support.	Service Desk Agent
SO 2.7.9	Will related incident be solved on time?	If yes, the Incident Coordinator of the assigned group estimates that the related incident can still be solved on time, go to SO 2.7.11. If no, go to SO 2.7.10, and then escalate the incident.	Service Desk Agent

SO 2.7.10	Communicate SLA breach to affected Users	Identify which Users or user groups are affected by the SLA breach. Send a communication bulletin to inform all affected Users.	Service Desk Agent
SO 2.7.11	Communicate related incident status to all affected Users	Identify which Users or user groups are affected by the related incident. Send a communication bulletin to inform all affected Users of the incident status and expected resolution time.	Service Desk Agent

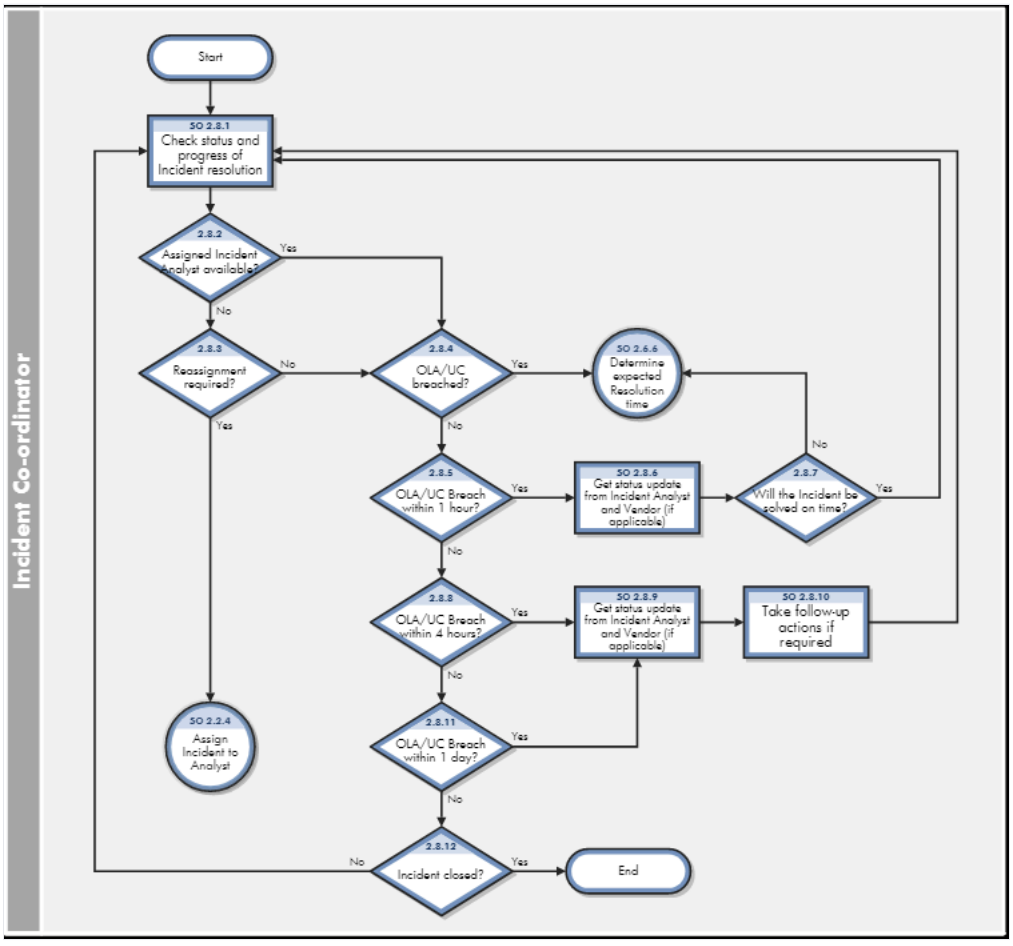
9.5.8. OLA and UC Monitoring (process SO 2.8)

One measure of the successful resolution of incidents is the performance of the individual support groups and applicable vendors. The performance of support groups is measured by targets set up within Operation Level Agreements (OLAs). The performance of vendors is measured by targets set up in the Underpinning Contracts (UCs).

The Incident Coordinator monitors all incidents assigned to the support group and applicable vendors. Performance is tracked until incidents are resolved or escalated to meet targeted agreement dates and times. The target date of an OLA and UC usually depends on the priority and category of the incident. The Incident Coordinator can escalate an incident to the Incident Manager if the target time has been or is about to be exceeded.

You can see the details of this process in the following figure and table.

The OLA and UC Monitoring workflow is illustrated in the following figure:



OLA and UC Monitoring process

Process ID	Procedure or Decision	Description	Role
SO 2.8.1	Check status and progress of Incident resolution	Check status and progress of incident resolution. Verify that the incident will be resolved before the target date and time specified in applicable Operation Level Agreement (OLA) and Underpinning Contract (UC).	Incident Coordinator
SO 2.8.2	Assigned Incident Analyst available?	External circumstances (for example, end of work shift, illness, or holiday) could cause an assigned Incident Analyst to become unavailable. If the Incident need to be assigned, SO 2.8.3. If not, go to SO 2.8.4.	Incident Coordinator
SO 2.8.3	Reassignment required?	If yes, go to SO 2.2.4. If no, go to SO 2.8.4.	Incident Coordinator
SO 2.8.4	OLA or UC breached?	If yes, start the Incident Escalation process (SO 2.6.6). If no, go to SO 2.8.5.	Incident Coordinator
SO 2.8.5	OLA/UC breach within 1 hour?	If yes, go to SO 2.8.6. If no, go to SO 2.8.8.	Incident Coordinator
SO 2.8.6	Get status update from Incident Analyst and Vendor (if applicable)	Contact the assigned Incident Analyst to receive a status update of the incident. If the incident is reported to a vendor, contact the vendor for a status update.	Incident Coordinator
SO 2.8.7	Will the incident be solved on time?	The Incident Coordinator estimates whether or not the incident can still be resolved on time. If yes, go to SO 2.8.1. If no, go to SO 2.6.6 to determine the expected resolution time.	Incident Coordinator
SO 2.8.8	OLA/UC breach within 4 hours?	Does the incident need to be resolved within 4 hours to reach the OLA/UC target date/time? If yes, go to SO 2.8.9. If no, go to SO 2.8.11.	Incident Coordinator
SO 2.8.9	Get status update from Incident Analyst and vendor (if applicable)	Contact the assigned Incident Analyst to receive a status update of the incident. If the incident is reported to a vendor, contact the vendor for a status update.	Incident Coordinator

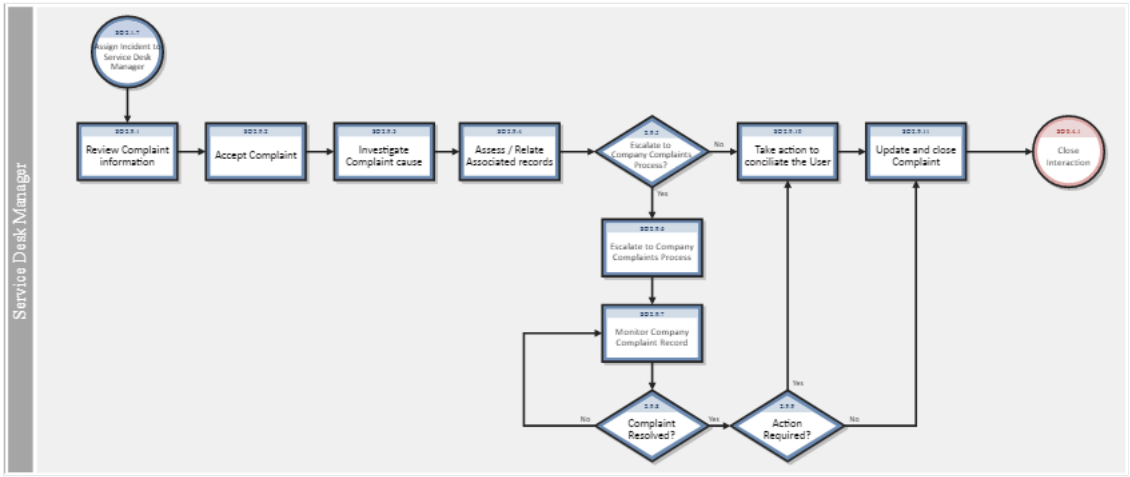
SO 2.8.10	Take follow-up actions if required	The Incident Coordinator determines whether follow-up actions are required to resolve the incident according to the OLA/UC. If required, the Incident Coordinator performs the required actions.	Incident Coordinator
SO 2.8.11	OLA/UC breach within 1 day?	If yes, go to SO 2.8.9. If no, go to SO 2.8.12.	Incident Coordinator
SO 2.8.12	Incident closed?	If yes, no further action is required. If no, go to SO 2.8.1.	Incident Coordinator

9.5.9. Complaint Handling (process SO 2.9)

Complaint Handling is the process by which the Service Desk Manager handles complaints. The Complaint category is typically used to indicate less than satisfactory service received by a user in the support or service delivery categories.

When the Service Desk Manager receives assigned incidents in the Incident or To Do queue, the manager accepts the incident. The manager investigates the cause of the complaint by evaluating the relevant information and talking to the people involved. The manager searches for an answer or solution to satisfy the user who filed the complaint, updates the incident with the agreed on details, and then closes the incident. You can see the details of this process in the following figure and table.

The Complaint Handling workflow is illustrated in the following figure:



Complaint Handling process

Process ID	Procedure or Decision	Description	Role
SO 2.9.1	Review complaint information	The Service Desk Manager monitors the incident queue and reviews assigned incidents. The Service Desk Manager checks the contents of the complaint.	Service Desk Manager
SO 2.9.2	Accept complaint	The Service Desk Manager accepts the incident to investigate the cause of the complaint.	Service Desk Manager
SO 2.9.3	Investigate complaint cause	The Service Desk Manager investigates the cause of the complaint by looking at the relevant information and talking to the people involved. The Service Desk Manager also searches for an answer or solution to satisfy the user who filed the complaint.	Service Desk Manager
SO 2.9.4	Assess/ Relate associated records	The Service Desk Manager assesses the associated records and relates them to existing records if necessary.	Service Desk Manager
SO 2.9.5	Escalate to company complaints process?	The Service Desk Manager assesses the complaint and determines whether it is within the scope of the Company Complaints Process. If escalation is necessary, go to SO 2.9.6. If not, go to SO 2.9.10.	Service Desk Manager
SO 2.9.6	Company complaints process	The Service Desk Manager escalates to have the complaint registered in the Company Complaints process and updates the Incident record.	Service Desk Manager
SO 2.9.7	Monitor company complain record	The Service Desk Manager monitors the complaint through the company complaint process.	Service Desk Manager
SO 2.9.8	Complaint resolved?	If the complaint is resolved, continue to SO 2.9.9. If not, go to SO 2.9.7.	Service Desk Manager
SO 2.9.9	Action required?	If the complaint has been resolved but further action must be taken, go to SO 2.9.10. If no further action is required, go to SO 2.9.11.	Service Desk Manager

SO 2.9.10	Take action to conciliate the user	The Service Desk Manager contacts the user to solve the user's issue and tries to reach an agreement.	Service Desk Manager
SO 2.9.11	Update and close complaint	The Service Desk Manager updates the incident with the agreed on details and closes the incident.	Service Desk Manager

9.6. Incident Management details

Service Manager uses the Incident Management application to enable the Incident Management process. The main function of Incident Management is to monitor, track, and record calls and open incidents as necessary.

In Incident Management, an Incident Analyst investigates, diagnoses, and proposes solutions for incidents. The Incident Analyst escalates those incidents requiring a change to the Incident Coordinator.

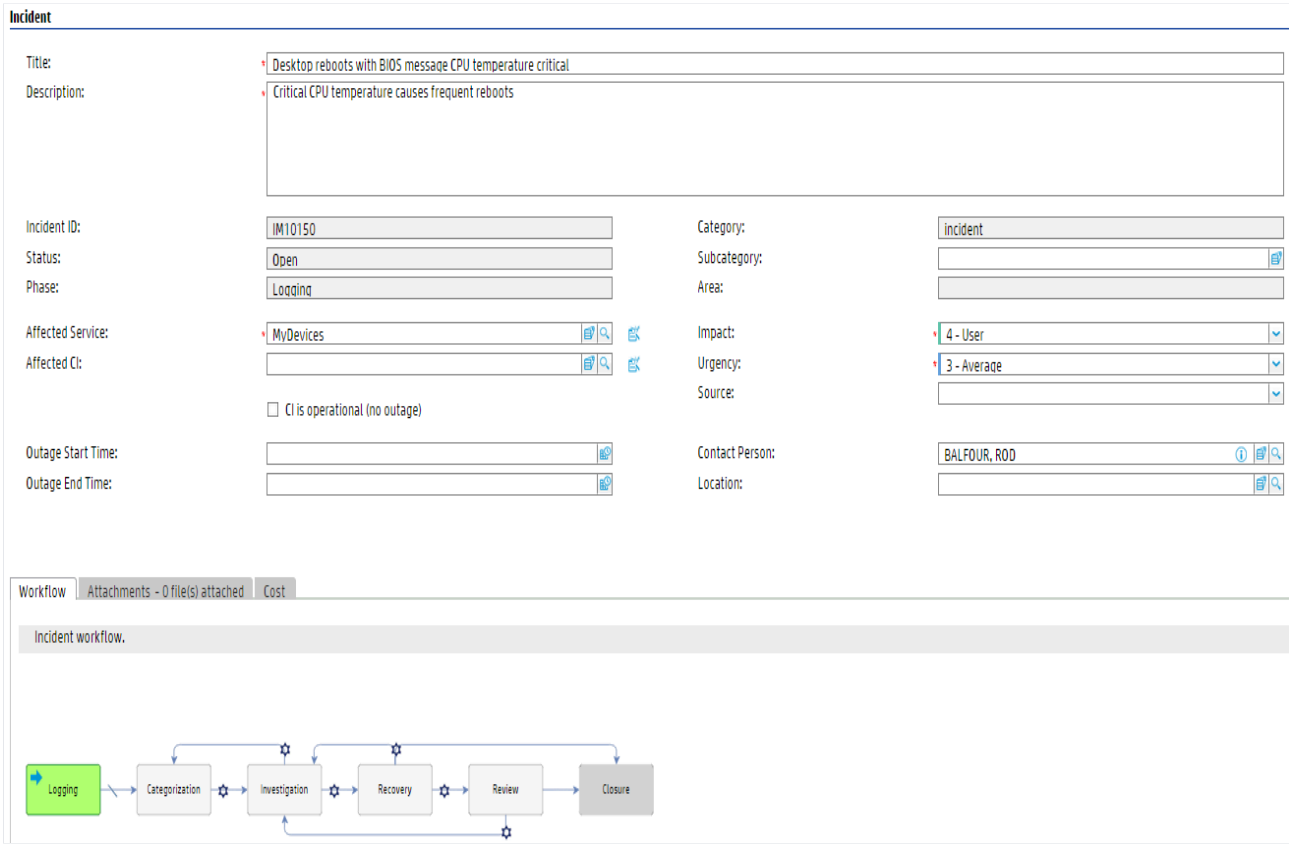
This section describes selected Incident Management fields in the out-of-box Service Manager system. Topics in this section include:

- Incident form after escalation from Service Desk
- Categorize incident form
- Investigate incident form
- Recover incident form
- Review incident form
- Incident Management form details

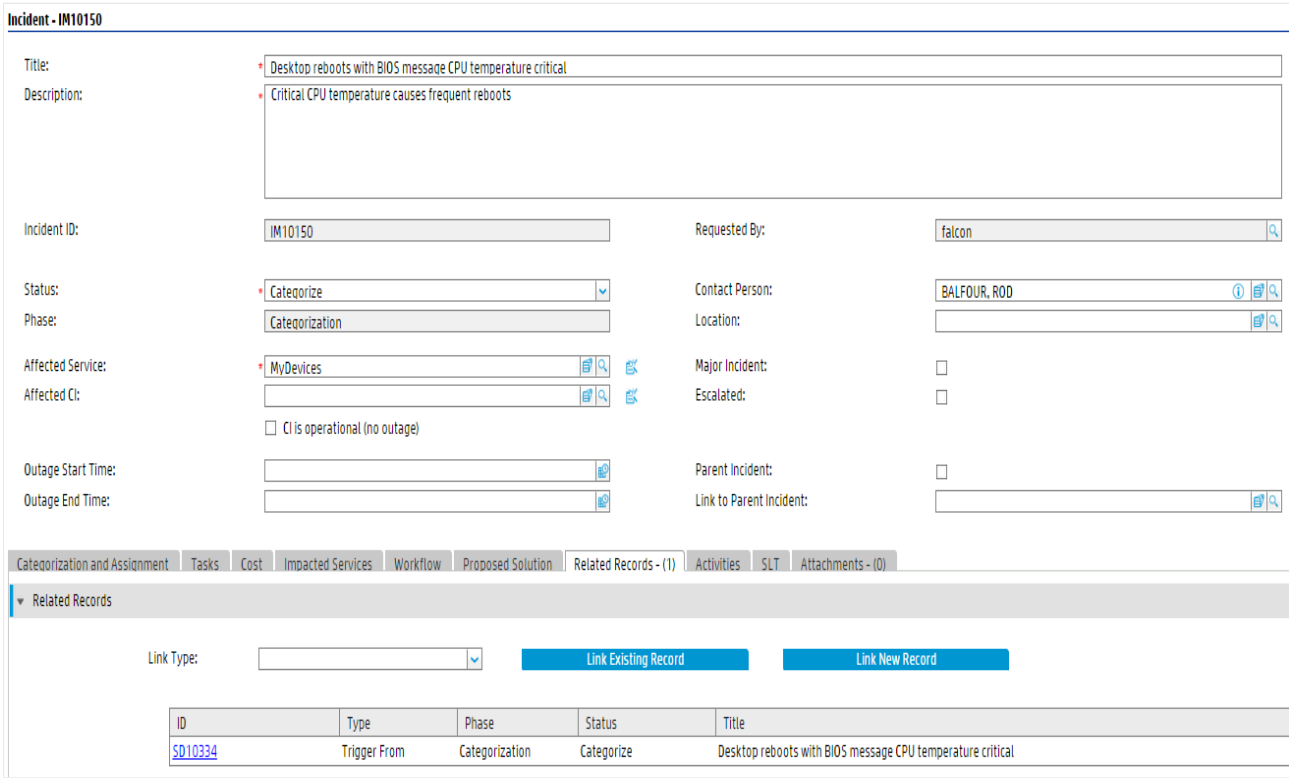
9.6.1. Incident form after escalation from Service Desk

The Incident Coordinator reviews incidents opened from the Service Desk and accepts or rejects each incident. The Incident Coordinator then assigns the incident to an Incident Analyst for investigation and diagnosis.

Incident opened from Service Desk is illustrated in the following screenshot:

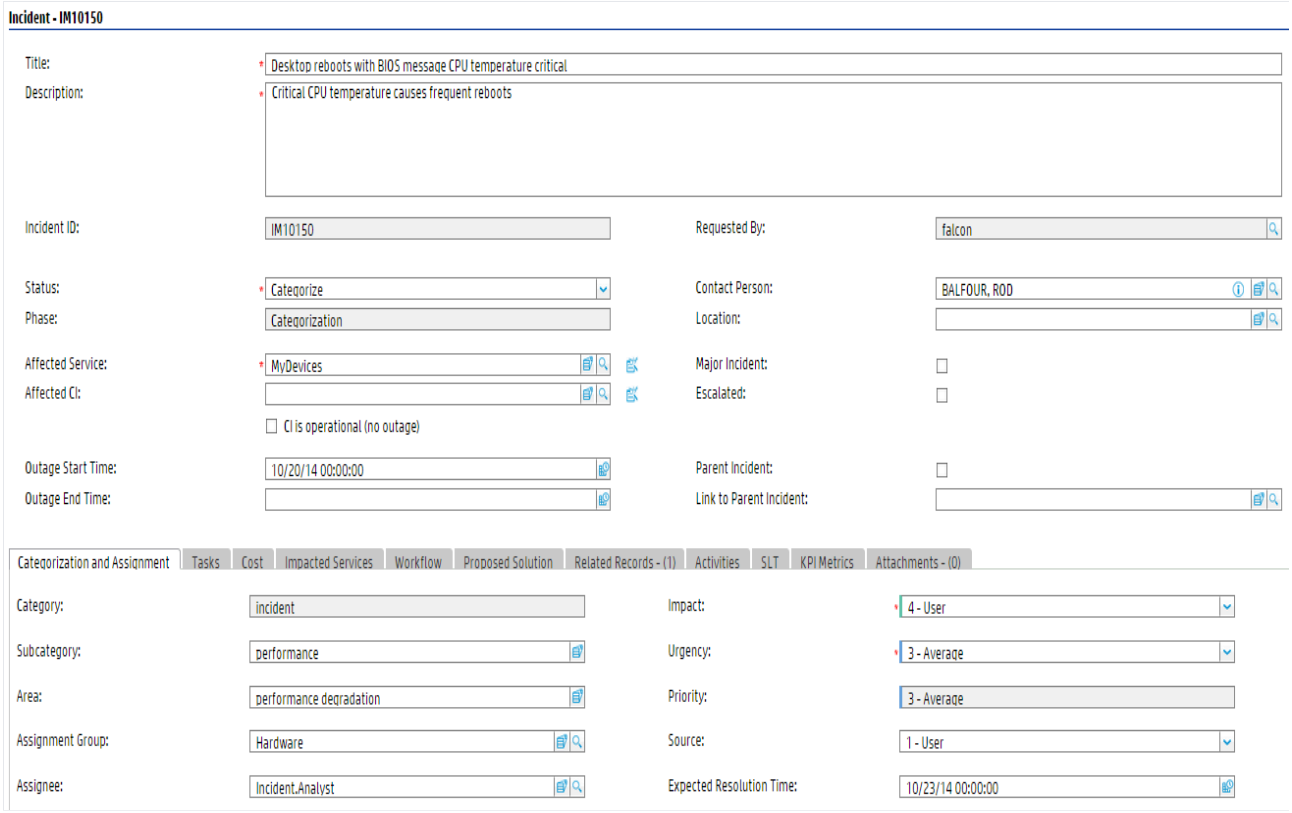


After saved, the incident enters Categorization phase, which is illustrated in the following screenshot:



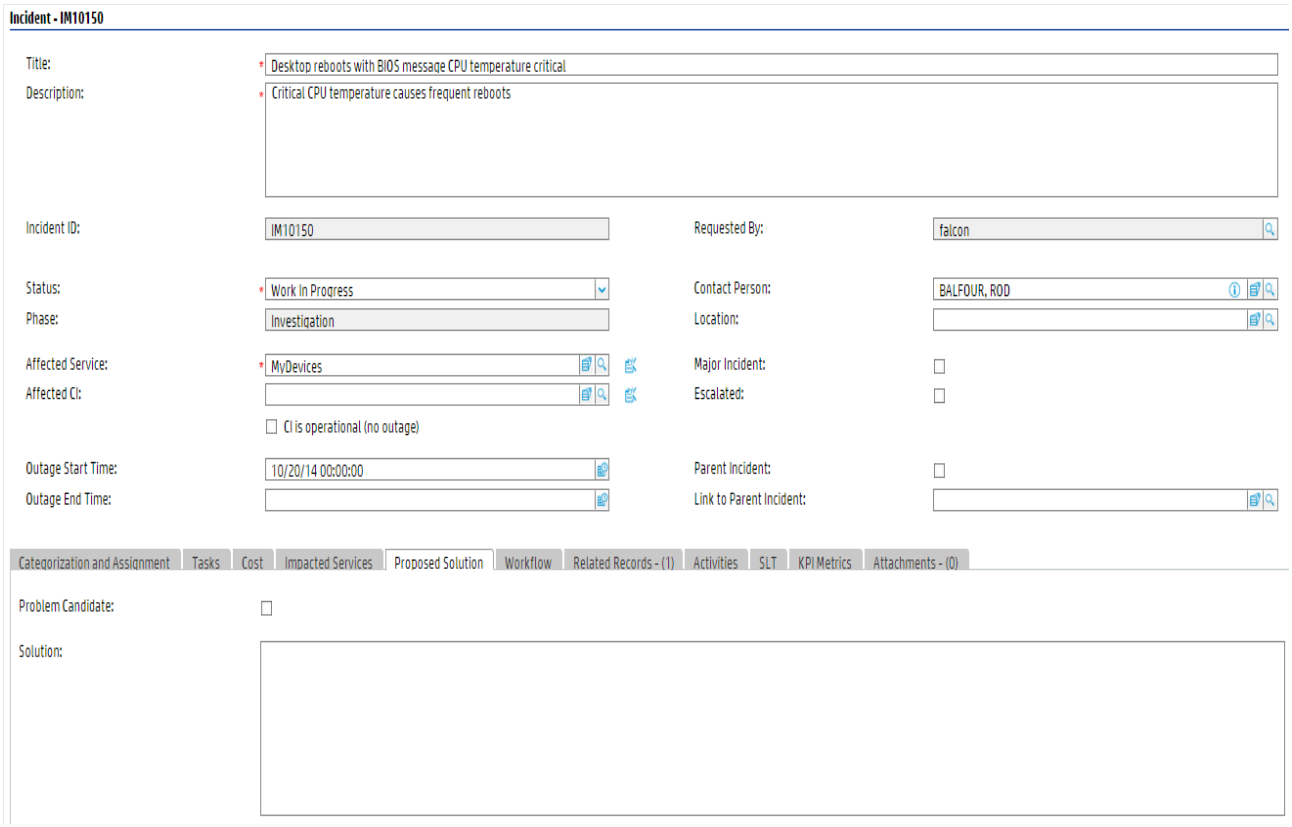
9.6.2. Categorize incident form

The Incident Coordinator uses the incident categorization form to review the information, and then categorize the incident, set expected resolution time, and assign the incident to an Incident Analyst in the appropriate support group.



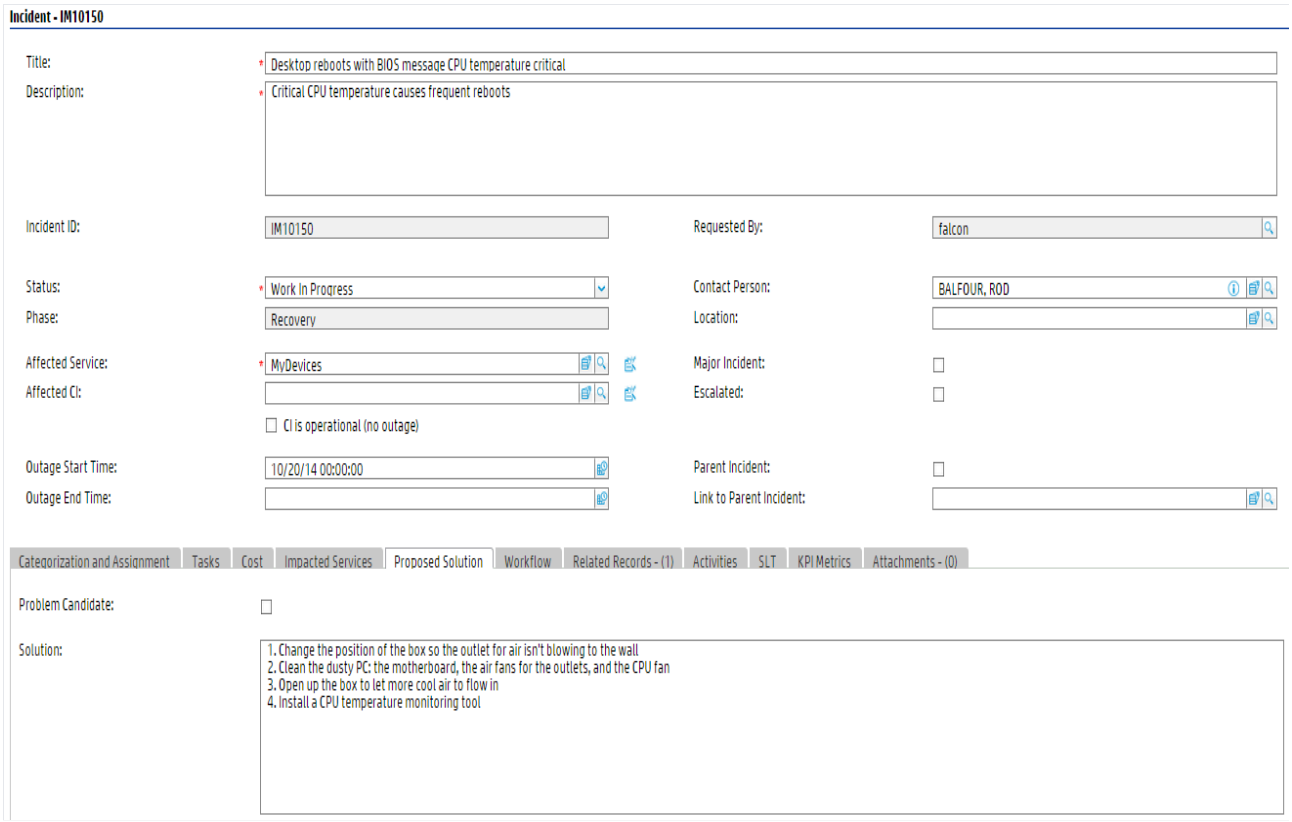
9.6.3. Investigate incident form

The Incident Analyst uses the incident investigation form to analyze the issue and determine if the incident can be resolved, and then updates the form accordingly. The Incident Manager uses the update incident form to monitor Service Level Agreement (SLA) compliance, to initiate escalation actions, or to register an emergency change request. The fields and tabs available for updating depend upon the assigned user role, assignment group, and the status of the incident.



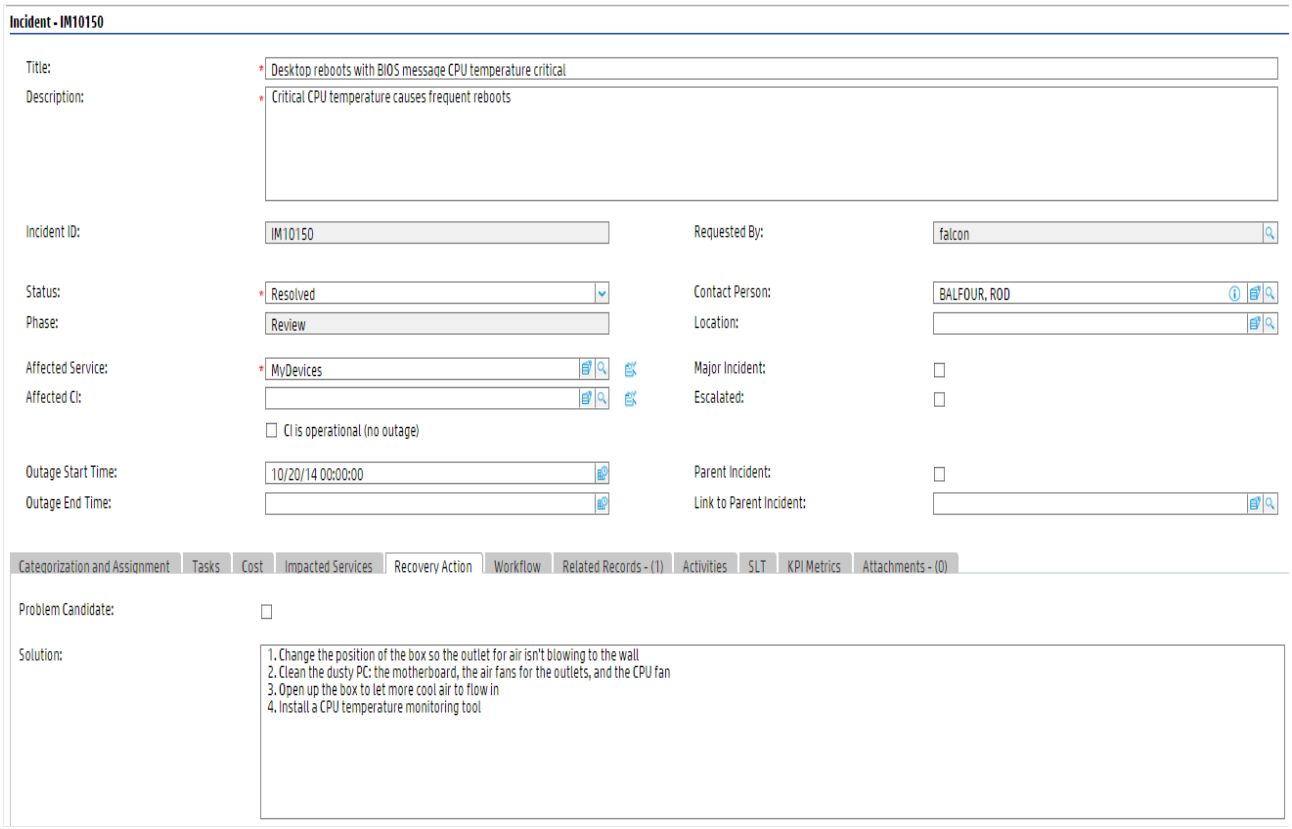
9.6.4. Recover incident form

The Incident Analyst tries to apply the resolution to the incident after diagnosis. Based on the nature of the resolution, the Incident Task, or the Change Management, Problem Management or vendor support is requested for assistance for the resolution.



9.6.5. Review incident form

The Incident Coordinator verifies the incident resolution with the requester. If the requester is not satisfied with the resolution, the Incident Coordinator can reassign, escalate or close the incident. If the requester is satisfied with the resolution, the Incident Coordinator then close the Incident ticket after reviewing whether it is a problem candidate.



9.6.6. Incident Management form details

The following table identifies and describes some of the features on the Incident Management forms.

Note

Note When setting up events or web services to create incidents automatically, you must be sure to include all required fields for the incident.

Incident Management form details

Label	Description
Incident ID	The system-generated unique ID for this incident.
Title	A short description that summarizes the incident. This field is prepopulated with data from an escalated interaction. This is a required field.
Tags	Enter one or more free-form tags (keywords or terms) of this incident. The tags you enter are searchable and can be used as keywords to highlight important information. For more information, see Working with tags.
Description	A detailed description of the incident. This field is prepopulated with data from an escalated interaction. This is a required field.
Phase	This is a system-generated field. These phases are available out-of-box: • Logging • Categorization • Investigation • Recovery • Review • Closure
Status	Displays the status of the incident. These statuses are available out-of-box: • Open — The incident has been opened but it's not currently being worked on. • Categorize – The incident has been categorized • Assign – The incident has been assigned to appropriate resource • Work In Progress — The incident is being addressed. • Pending Customer — You need more information from the customer • Pending Vendor — You need something from the vendor • Pending Evidence — You need evidence from the customer or vendor • Pending Parent Incident — The incident has been linked to a parent incident and waiting for the resolution of the parent incident. This status is automatically set when an incident is linked to a parent incident. • Pending Other — You need something from an outside source other than customer or vendor. • Suspended — Customer has agreed to suspend the incident for a time; the incident will not appear in your Inbox for that period. • Resolved — There is a resolution, but it has not yet been verified by the customer. • Closed — The incident has been resolved and the customer agrees.

Affected Service	The service affected by this incident. This field is populated with data from the interaction record. See Service Desk Interaction Management form details for additional information. This is a required field.
Affected CI	The configuration item (CI) that's affecting the service negatively. This field is populated with data from the interaction record. See Service Desk Interaction Management form details for additional information. This field includes a hover-over form that displays Critical CI and Pending Change checkboxes to indicate whether or not these attributes apply to the CI.
CI is operational (no outage)	This field indicates that the item is currently operational and that there is no outage if selected (set to true). By default when you open an incident against a CI, the CI is flagged as down. If the CI is still working, you should mark this field.
Outage Start	The date and time when the outage started. The outage start and outage end times are used to measure the availability for the Service Level Agreements (SLAs). If the CI is flagged as down, availability SLAs start counting against the CI. The default availability value is the incident open and close times, but you should change this value to report the actual outage start and end times because it may be several minutes or hours before the incident is opened or closed. For example, the device may have gone down in the night and the incident is not opened until someone reports the problem. In this case, the default open time doesn't accurately reflect the outage time.
Outage End	The date and time when the outage ended. The outage start and outage end times are used to measure the availability for the SLAs. If the CI is flagged as down, availability SLAs start counting against the CI. The default availability value is the incident open and close times, but you should change this value to report the actual outage start and end times because it may be several minutes or hours before the incident is opened or closed. For example, the device may have gone down in the night and the incident is not opened until someone reports the problem. In this case, the default open time doesn't accurately reflect the outage time.

Category	This field describes the type of incident, based on ITIL service-centric processes. This field is prepopulated with data from the escalated interaction. This is a required field. A workflow is bound to a category. You must select a category before opening an incident form if the incident is not opened from an interaction and there is no default category specified. If required, the Incident Coordinator, Incident Manager, and Incident Analyst can update this field and the related subcategory and area fields for incidents assigned to them. This field is read-only, to update the category, they need to click **More** > **Change Category** menu option. The following categories are available for the out-of-box incidents: • Complaint • Incident • Request for administration • Request for information
Subcategory	This field is prepopulated with the data from the interaction from which the incident is opened. The subcategory selections depend on the category. This is a required field when phase starts from Categorization. The out-of-box data is the same as in Interaction Management, but can be different. For additional information, see Service Desk Interaction Management form details and Interaction categories.
Area	The third level of classifying an interaction, mainly used for reporting purposes. This field is prepopulated with the data from the interaction from which the incident is opened. Service Manager displays different lists of areas, depending on the subcategory selected. For more information on categories and the areas and subareas associated with them, see Interaction categories. The out-of-box data is the same as in Interaction Management, but can be different. For additional information, see Service Desk Interaction Management form details.
Impact	This field is prepopulated with data from an escalated interaction. It specifies the impact the incident has on the business. The impact and the urgency are used to calculate the priority. These impacts are available out-of-box: • Enterprise • Site/Dept • Multiple Users • User
Urgency	This field is prepopulated with data from an escalated interaction. The urgency indicates how pressing the incident is for the organization. The urgency and the impact are used to calculate the priority. For additional information, seeService Desk Interaction Management form details.

Priority	The order in which to address this incident in comparison to others. The priority value is calculated using initial impact and urgency. This field only appears for incidents being updated or escalated from interactions.
Major Incident	If selected (set to true), this field indicates that the issue is a major issue, which requires informing the specified Incident Manager, and imputing review details under the Major Incident Review section.
Escalated	If selected (set to true), this field indicates that the incident needs to be escalated to Incident Manager, and possibly additional escalation teams also need to be aware of.
Requested By	The name of operator who opened the Incident ticket. This field is populated with the current operator.
Contact Person	This field contains the contact name related to the company for this incident. This field ensures that the correct person will be notified about updates to the incident. This field is prepopulated with data from an interaction when a user opens an incident from an interaction.
Location	The location for which the incident has been reported. This field is for informational purposes only. Location data is customer and implementation specific.
Source	The source from which the record is created. The following out-of-box sources are available: • 1 - User • 2 - Group • 3 - Event • 4 - Chat • 5 - Email • 6 - Telephone
Parent Incident	If selected (set to true), this field indicates that the incident is a parent incident, that allows other incidents link to it for issue classification, and the child incidents can be viewed under Child Incidents section. If unselected later after being selected, all the linked child incidents are unlinked. Once an incident is set as a parent incident, it can't be linked to a parent incident any longer.
Link to Parent Incident	The ID of the parent incident. This field can be populated with incident ID based on predefined query for parent incidents. One incident can only be linked to one parent incident. Once linked to a parent incident, this incident can't be set as parent incident any longer.

Categorization and Assignment section > Assignment Group	The support group assigned to work on this incident. The affected service or CI specified in the interaction form determines which default assignment group the system assigns to incidents that were escalated from interactions. An administrator assigns the default assignment group for a service on the Configuration Item (CI) detail form for the CI. When you search for the service in Configuration Management (Configuration Management > Resources > Search CIs), you see the default assignment group for the service or CI specified in the Config admin group field. When you escalate an interaction to an incident, the assignment group is prepopulated, based on the CI or service (if no CI selected, then based on selected service) selected in the interaction. You can change the assignment group, if necessary. The out-of-box data consists of default assignment groups for use as examples of types of assignment groups.  **Tip** Tip You may want to adapt the sample assignment groups to meet your own needs. These assignment groups are available out-of-box: • Application • Email / Webmail • Field Support • Hardware • Intranet / Internet Support • Network • Office Supplies • Office Support • Operating System Support • SAP Support • Service Desk • Service Manager This is a required field.
Categorization and Assignment section > Assignee	The name of the person assigned to work on this incident. This person is a member of the assigned support group. Assignees may belong to one or multiple assignment groups, based on the needs of your company.
Categorization and Assignment section > Expected Resolution Time	The Incident Coordinator can use this field to set an expectation for incident resolution time.
Workflow section	Workflow section displays a figure of incident workflow. It also indicates the current phase which the incident is in, and traces the phase transition history.

Tasks section	The user can add tasks whenever an incident is in a phase. Every task has to be finished before close the incident. To add a new task, click the **Tasks** section, and then click the **Link New Task** button. Service Manager provides a quick view of some of the most important fields in the task in the **Tasks** section. The data displayed includes the following information: • Task ID • Status • Phase • Priority • Title
Cost section	The user can add cost details of parts and labor for incident handling. For parts cost, the user needs to add expense lines with the part number from product catalog and the quantity that have been used during the incident handling. For labor cost, the user needs to add technicians who have been working on the incident with the working hours. Select a currency for cost calculation, then the total cost of the incident will be calculated by the system.
Activities > Vendor/Supplier	The name of the vendor/supplier the incident is assigned to. Used when a vendor/supplier needs to be involved in fixing the incident.
Activities > Vendor/Supplier Record	This number refers to the incident number from the vendor/supplier logging system. This is an informational field for reference only. This field is only visible when incident status is Pending Vendor/Supplier.
Related Records section > Link Type	Specify a relation type between problem and target ticket. These link type groups are available out-of-box: • Caused By Incidents • Caused Incidents • Caused By Changes • Related Interactions • Related Problems • Related Known Errors • Related Requests
Related Records section > Link Existing/New Record button	After select a link type, uses these two buttons to associate the incident with target ticket, or create a new target ticket and associate with this incident.
Related Records section > All Related Records table	Information of all related records of this problem is displayed in this table. The data displayed includes the following information: • ID • (Relation) Type • Phase • Status • Title

Related Records section > Unlink Record button	If you want to disassociate the incident with another ticket, select the related ticket from All Related Records table, and click this button to unlink these two tickets.
Proposed Solution/Recovery Action section > Solution	Provides a description of the solution for the incident.
Proposed Solution/Recovery Action section > Problem Candidate	If selected (set to true), this field indicates that the issue that caused the incident is most likely a problem. When selected, either a problem should have been created, or the incident should have been associated with other problems. This field is only enabled for users who have Expert rights. This capability is specified on the Incident Management Security Role area. When the Problem Management Candidate field is checked for the incident, the incident appears in the Problem Manager default view for incidents. The Problem Manager can then review the incident to decide whether or not to open a related problem. Examples of problem candidates include cases where several customers report the same issue or where an issue recurs repeatedly.
Closure Code	Specifies a predefined closure code to describe how the incident has been resolved. The out-of-box options in this field are based on customer reference data. **Tip** Tip You may want to tailor these options to match your business needs. These closure codes are available out-of-box: • Not Reproducible • Out of Scope • Request Rejected • Solved by Change/Service Request • Solved by User Instruction • Solved by Workaround • Unable to Solve • Withdrawn by User • No Fault Found • No User Response • Resolved Successfully • Diagnosed Successfully
Completion Comments	This field is to document additional comments to close the incident.
Affected Services	This section provides a list of affected services for the incident. When a configuration item for the incident is added or updated, a schedule record is created that runs a routine to update the list of affected services. If the incident is locked, the routine reschedules the schedule record for 5 minutes later.
SLT > Process Targets	This subsection provides a list of process SLTs related to the incident. The information includes Agreement name, status, SLT name, From and To specifications for the Agreement, and Expiration. Similar information is available for interactions, problems, and changes.

SLT > Uptime Targets	This subsection displays uptime availability data for the SLTs related to the incident. The data displayed includes the following information: • Status • SLT name • Required Monthly Uptime (%) • Withdrawn by User • Current Uptime this Month (%) • Next Expiration • Affected CI • SLT ID Similar information is available for interactions, problems, and changes.
SLT > Max Duration Targets	This subsection displays duration availability data for the SLTs related to the incident. The data displayed includes the following information: • Status • SLT name • Total outages this month • Average outage duration • Next expiration • Affected CI • SLT ID Similar information is available for interactions, problems, and changes.
SLT > Upcoming Alerts	This subsection displays all upcoming Agreement alerts to help users prioritize the incidents needing attention. The data displayed includes the following information: • Alert name • SLT name • Alert time **Note** Note For additional information, see Service Level Agreement alerts.
SLT View	If you set the View Type property of the Calendar control to **Calendar** or leave this property blank, this tab displays a normal view of the embedded Service Manager calendar. You can view the time period records and associated business records in a graphic and intuitive user interface. For more information about the Service Manager Calendar, see Calendar control and Calendar Administration. If you set the View Type property of the Calendar control to **SLM**, this tab visualizes the operation status for all related SLA, OLA, and UC of the current record. For more information about SLM visualization, see Visualize Service Level Management.

9.7. Problem Management overview

The Service Manager Problem Management application (Problem Management) supports the entire problem management process. Problem Management provides comprehensive problem management that enables you to find, fix, and prevent problems in the IT infrastructure, processes, and services.

Problem Management prevents problems and their resulting incidents, eliminates recurring incidents, and minimizes the impact of those incidents that can't be prevented. It maximizes system availability, improves service levels, reduces costs, and improves customer convenience and satisfaction.

This section describes how to implement the best practice guidelines for the problem management process in Problem Management. This section includes the following topics:

- Problem Management within the ITIL framework
- The Problem Management application
- Problem management process overview
- Input and output of Problem Management
- Key Performance Indicators for Problem Management
- RACI matrix for Problem Management

9.7.1. Problem Management within the ITIL framework

Service Operation document. The document describes problem management as the process by which the lifecycle of all problems is managed.

The main benefits of problem management are improved service quality and reliability. As incidents are resolved, information about their resolution is captured. This information is used to identify and quickly resolve similar incidents in the future, and then to identify and fix the root cause of those incidents.

Problem management functions both reactively and proactively.

- Reactive problem management resolves situations related to incidents. Reactive problem management is generally executed as part of the service operation process, and is based on incident history.
- Proactive problem management identifies and solves issues and known errors before incidents occur. Reactive problem management is generally driven as part of the continual service improvement process.

By actively preventing incidents, instead of reacting to them, an organization provides better service and operates more efficiently.

Differences between Problem Management and Incident Management[edit]

Incident Management and Problem Management are separate, but closely-related, processes. Incident Management enables you to restore service to users, whereas Problem Management manages the lifecycle of all problems and enables you to identify and remove the underlying causes of incidents.

Contents

- 1 Differences between Problem Management and Incident Management

9.7.2. Problem Management application

Problem Management helps you to minimize the effects of incidents caused by errors in the IT infrastructure. Problem Management helps you to prevent these errors from recurring. With Problem Management, the appropriate people can identify known errors, implement workarounds, and provide permanent solutions. Additionally, Problem Management enables you to identify errors in IT infrastructure, record them, track the history, find resolutions for them, and prevent their recurrence.

Problem Management helps your personnel to record resolutions and make them easily available to affected user groups, to react more quickly to issues related to incidents, and to proactively resolve issues before incidents occur. Over the long term, the use of Problem Management reduces the volume of incidents, and saves time and money.

Problem Management workflows and categories[edit]

Problem Management comes with out-of-box workflows for problems (the “Problem” workflow) and known errors (the "Known Error" workflow). These workflows are associated respectively with the out-of-box "problem" and "known error" categories. The workflows ensure that the problem workflow automatically conforms to the ITIL workflow.

If your business needs require changes to the out-of-box workflow, you can define new workflows. To do this, copy the out-of-box “Problem” workflow, add your own phases and transitions, and then associate the new workflow with the out-of-box “problem” category (or a new category). Each new category that you define enables you to associate a different workflow with a problem. If you define new categories, you can select one to be the default category for problems.

Problem tasks[edit]

Problem tasks have a single out-of-box task workflow (the “Problem Task” workflow) and four out-of-box task categories (the “Categorization,” “Investigation,” “Resolution,” and “Review” task categories). The four problem task categories are associated with that single workflow. You can define new workflows for problem tasks by copying and modifying the out-of-box workflow. You can also change the task categories or add other task categories. You can define unique task categories for the tasks that you assign to a problem.



Note

Note There are no out-of-box tasks for Known Error records.

Problem Management alerts[edit]

Problem Management creates automatic notifications. For example, it creates notifications when a problem or task opens, when the owner changes, or when the status changes. It also escalates problems automatically when they're not addressed on pre-agreed schedules. The expected resolution date is based on several elements, including discussion with the stakeholders.

Contents

- 1 Problem Management workflows and categories
- 2 Problem tasks
- 3 Problem Management alerts

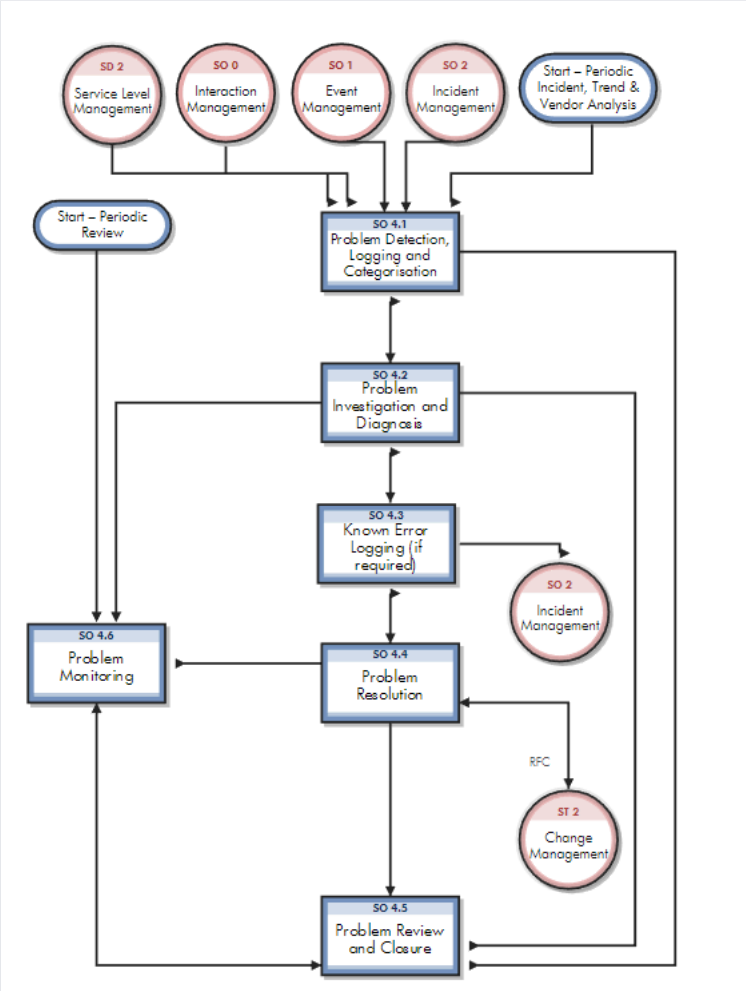
9.7.3. Problem Management process overview

The problem management process includes the activities that are required to identify and classify problems, diagnose the root cause of incidents, and determine resolutions to related problems. The process ensures that the resolution is implemented through the appropriate control processes, such as change management.

Problem Management includes the activities that are required to prevent the recurrence or replication of incidents. It enables you to form recommendations for improvement, maintain problems, and review the status of corrective actions.

Proactive problem management encompasses problem prevention, ranging from the prevention of individual incidents (for example, repeated difficulties with a particular system feature) to the formation of higher-level strategic decisions. The latter may require major expenditures to implement, such as investment in a better network. At this level, proactive problem management merges into availability management. Problem prevention also includes the information that's given to customers for future use. This information reduces future information requests and helps to prevent incidents caused by lack of user knowledge and training.

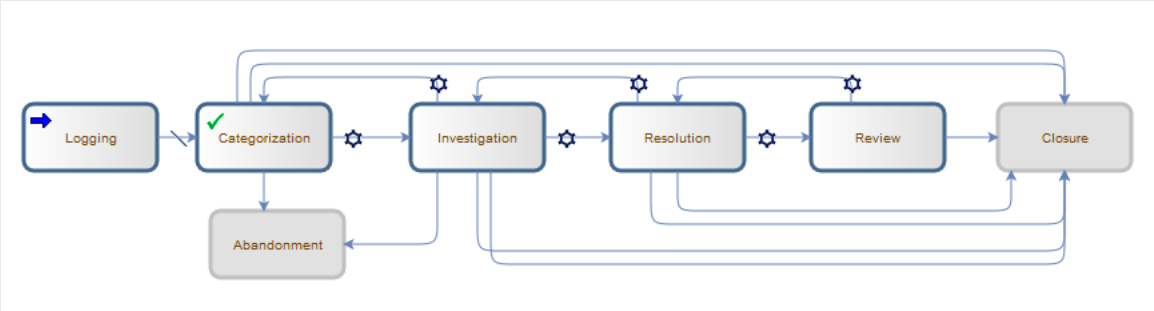
The following figure provides a general overview of the problem management processes and workflows. These workflows are described in detail in Problem Management Workflows.



Problem management phases

Service Manager uses phases to describe the steps needed to resolve a problem. The phase also determines the forms users see, the actions users can manually trigger. In an out-of-box system, most of the phase transitions are triggered by change to the problem status.

The following figure shows the workflow phases for a problem.



The out-of-box known error workflow uses only two phases to differentiate the open and closed known error records. A known error is opened with a documented root cause and workaround, and closed when a permanent solution is found.

The following figure shows the workflow phases for a known error:



Problem Management user roles

The following table describes the responsibilities of the Problem Management user roles.

Problem Management user roles

Role	Responsibilities
Problem Manager	• Communicate with stakeholders if required • Inform the Change Manager if required • Defer problems if needed • Decide on investigation of problems • Register Request for Changes or Service Requests to solve problems • Validate proposed solutions to problems • Validate the outcome of closed changes and close problem • Validate that a problem is solved • Conduct problem review and document lessons learned • Close problem and inform stakeholders • Monitor the problem resolution progress and perform the required action
Problem Coordinator	• Periodically perform analysis to see if new problems need to be registered • Register problems • Categorize and prioritize problems • Assign work to the Problem Analysts • Schedule the problem resolution • Coordinate root cause analysis and diagnosis Identify and raise known errors
Problem Analyst	• Investigate and diagnose assigned problems for workarounds and/or root causes • Review and accept or reject assigned errors problems or problem tasks • Investigate and diagnose assigned problems and propose solutions and workarounds • Identify major problems and ensure problem manager is notified • Implement corrective actions

9.7.4. Input and output for Problem Management

Problems can be triggered and resolved in several ways. The following table outlines the input and output of the Problem Management process.

Input and output for Problem Management

Input to Problem Management	Output from Problem Management
• Incidents for which the cause is not known and/or incidents that are likely to recur (from incident management) • Incidents that reveal that an underlying problem exists (for example, an application error or bug) • Notification from a supplier or a product manager that a problem exists (for example, from a development team or supplier known error database) • Potential security breaches of products deployed in the IT environment (for example, from suppliers or security analysts) • Analysis of incident trends and history (that is, proactive problem management) • Incident Management ○ Incidents classified as problem candidates ○ Trend analysis and review of closed incidents (for which a workaround has been used to resolve the incident) ○ Incident reports (trends, summary) ○ Suspicion or detection of a cause of one or more incidents by the Service Desk, resulting in the creation of a problem record. • Event management ○ Trend analysis and review of events (for example, performance events) ○ Error logs • Configuration management ○ Configuration details and relationships (service model) • Change management ○ RFC and change request status, approval and closure. • Security management ○ Notification of potential security breaches that require resolution • Suppliers (external providers) • Notification of problems from suppliers/vendors	• Problems • Known errors • Workarounds • Problem reports (for example, status updates, trends, and performance) • RFCs (to remove infrastructure errors) **Note** Note Information on workarounds, permanent fixes, or the progress of problems should be communicated to those who are affected and those who are required in order to support the affected services.

9.7.5. Key performance indicators for Problem Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your Problem Management processes. In addition to the data provided by Service Manager, you may need additional tools to report all of your KPIs. To visualize trend information, it is useful to display KPI data in graph form.

Problem Management KPIs

Title	Description
Average time to diagnose	The average time to diagnose problems and to pinpoint the root cause in a given time period.
Average time to fix	The average time to fix problem(s).
Number of new problems	The total number of problems recorded in a given time period.
Number of solved problems	The total number of problems solved in a given time period.
Incidents caused by problems	The number of incidents occurring before the problem is resolved in a given time period.

For completeness, the ITIL 2011 and COBIT 4.1 KPIs are included below.

ITIL 2011 Key Performance Indicators[edit]

The following are ITIL 2011 KPIs for Problem Management:

- The total number of problems recorded in a given period (as a control measure)
- The percentage of problems resolved within the SLA targets; also the percentage not resolved within the SLA targets
- The number and percentage of problems that exceed target resolution times
- The backlog of existing problems, and the growth trend of the backlog (that is, static, reducing, or increasing)
- The average cost of handling a problem
- The number of major problems, including opened, closed, and backlogged
- The percentage of major problem reviews successfully performed
- The number of known errors added to the known error Database (KEDB)
- The percentage accuracy of the KEDB (from audits of the database)
- The percentage of major problem reviews that were completed successfully and on time

COBIT 4.1 Key Performance Indicators[edit]

The following are the COBIT 4.1 KPIs for Problem Management:

- Number of recurring problems that have a business impact
- Number of business disruptions caused by operational problems
- Percentage of problems recorded and tracked
- Percentage of problems that recur (within a time period), ranked by severity
- Percentage of problems resolved within the required time period
- Number of open, new, and closed problems, ranked by severity
- Average and standard deviation of the time lag between problem identification and resolution
- Average and standard deviation of the time lag between problem resolution and closure
- Average duration between the logging of a problem and the identification of the root cause
- Percentage of problems for which root cause analysis was completed
- Frequency of reports or updates to an ongoing problem, based on the problem severity

Contents

- 1 ITIL 2011 Key Performance Indicators
- 2 COBIT 4.1 Key Performance Indicators

9.7.6. RACI matrix for Problem Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram (or RACI matrix) is used to describe the roles and responsibilities of the various teams or people that are responsible for delivering a project or operating a process.The matrix is especially useful for clarifying roles and responsibilities in cross-functional/departmental projects and processes.The following table displays the RACI matrix for Problem Management.

RACI matrix for Problem Management

Process ID	Activity	Problem Manager	Problem Coordinator	Problem Analyst	Change Coordinator
SO 4.1	Problem Detection, Logging, and Categorization	A/I	R	I	
SO 4.2	Problem Investigation and Diagnosis	A	C	R	
SO 4.3	Problem Resolution	A	C	R	R
SO 4.4	Problem Review and Closure	A/R	C		
SO 4.5	Problem Monitoring	A/R	C		

9.8. Problem Management workflows

The problem management process includes the activities that are required to identify and classify problems, diagnose the root cause of incidents, and determine resolutions to related problems. Problem management is responsible for ensuring that the resolution is implemented through the appropriate control processes, such as change management.

Problem management includes the activities that are required to prevent the recurrence or replication of incidents. It enables you to form recommendations for improvement, maintain problems, and review the status of corrective actions.

The problem management process consists of the following processes, which are included in this chapter:

- Problem Detection, Logging, and Categorization (process SO 4.1)
- Problem Investigation and Diagnosis (process SO 4.2)
- Problem Resolution (process SO 4.3)
- Problem Review and Closure (process SO 4.4)
- Problem and Known Error Monitoring (process SO 4.5)

9.8.1. Problem Detection, Logging, and Categorization (process SO 4.1)

Problem Coordinator determines that a problem needs to be opened in order to investigate an existing or potential problem. This process can be started in response to a single incident, a series of related incidents, or a single interaction. The process may also result from the proactive investigation of a potential problem.

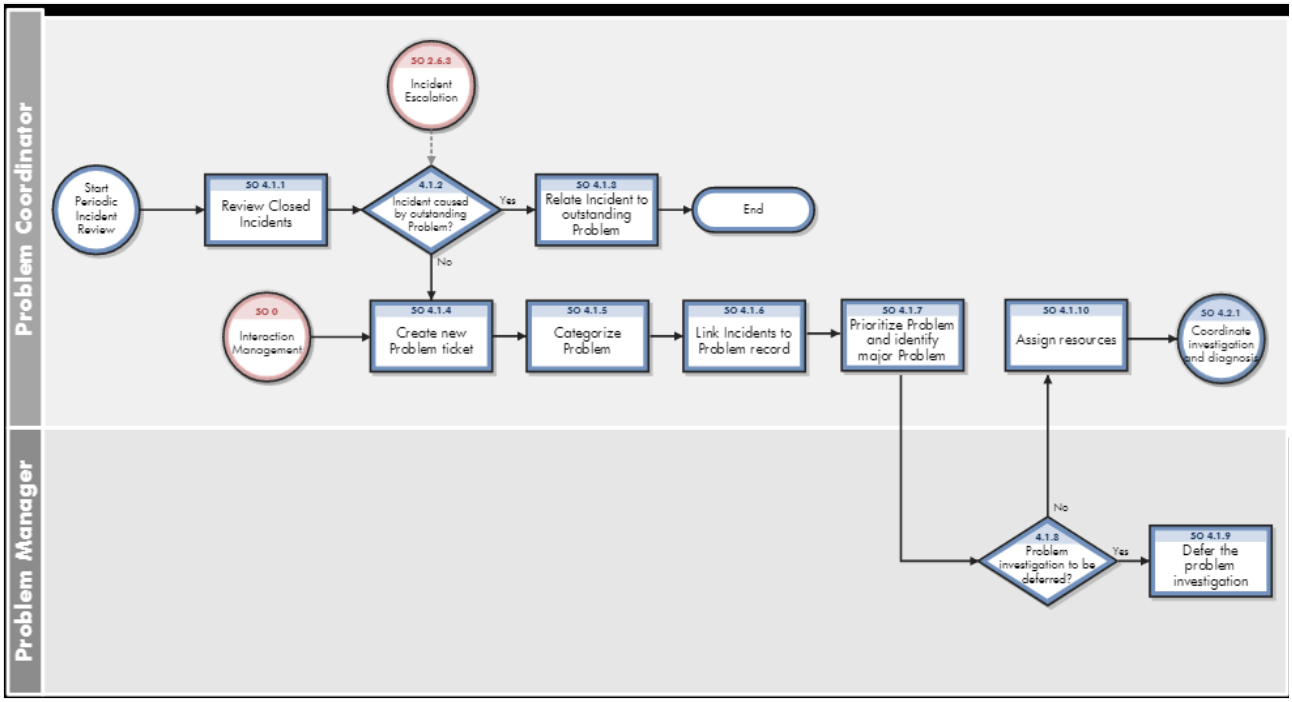
The Problem Detection, Logging, and Categorization process should include reference to information that assists analysis, such as:

- Asset and configuration
- Change management
- Published workaround information from suppliers
- Historical information about similar problems
- Monitoring event logs and other data collected by system management tools

The incident(s) or interaction(s) that initiated the problem should be referenced, and relevant details copied from the incident(s) or interaction(s) to the problem. If the Incident Analyst has identified a workaround or temporary fix, this should be included as well.

A problem ticket is created. All relevant details of the problem must be recorded so that an accurate historic record exists. Other details like impact and category of the problem are also identified.

The following figure illustrates the Problem Detection, Logging, and Categorization workflow:



Problem Detection, Logging, and Categorization process

Process ID	Procedure or Decision	Description	Role
SO 4.1.1	Review closed incidents	Periodically, the Problem Coordinator must review the closed incidents to detect new problems or to match incidents to existing problems that have not been resolved. Analysis of incident data may reveal similar or reoccurring incidents, which means that a permanent fix must be found. Select incidents since the last review by using the following criteria: • Major incidents (high impact) • Incidents resolved through a workaround or a temporary fix that's not matched to a problem. • Suspected problems (as identified by stakeholders) • Candidates for problems All closed incidents that aren't resolved through a permanent fix, temporary fix, or workaround must be matched to existing problems. Or, a new problem must be created. Incident management staff may have linked incidents to existing problems already (for example, if a workaround has been applied).	Problem Coordinator

SO 4.1.2	Incident caused by outstanding problem?	If the incident is caused by an outstanding problem, the workflow moves to SO 4.1.3. If the incident is not caused by an outstanding problem, the workflow moves to SO 4.1.4. It is important to link incidents to existing problems to monitor the number of reoccurring incidents. This helps you to identify problems that aren't resolved. The incident count is the number of times that this particular problem has resulted in an incident, and is updated in the problem. The incident count influences the prioritization of problems by indicating the frequency of occurrence and thus the business impact of this issue.	Problem Coordinator
SO 4.1.3	Relate incident to outstanding problem	If the incident is caused by an outstanding problem, the incident must be linked to the problem. If required, the problem is updated and the Problem Analyst is notified (for example, when a workaround has been applied).	Problem Coordinator
SO 4.1.4	Create new problem ticket	Create a new problem ticket that captures all the relevant data, such as: • User details • Configuration Item (CI) details • Date and time the problem was initially logged • Description • Details of all diagnostic or attempted recovery actions taken so far The Problem Coordinator can estimate the resources and costs that are required to resolve a problem during any stage of the Problem lifecycle. These details are entered in the problem record, and are used to decide the next course of action for the ticket.	Problem Coordinator

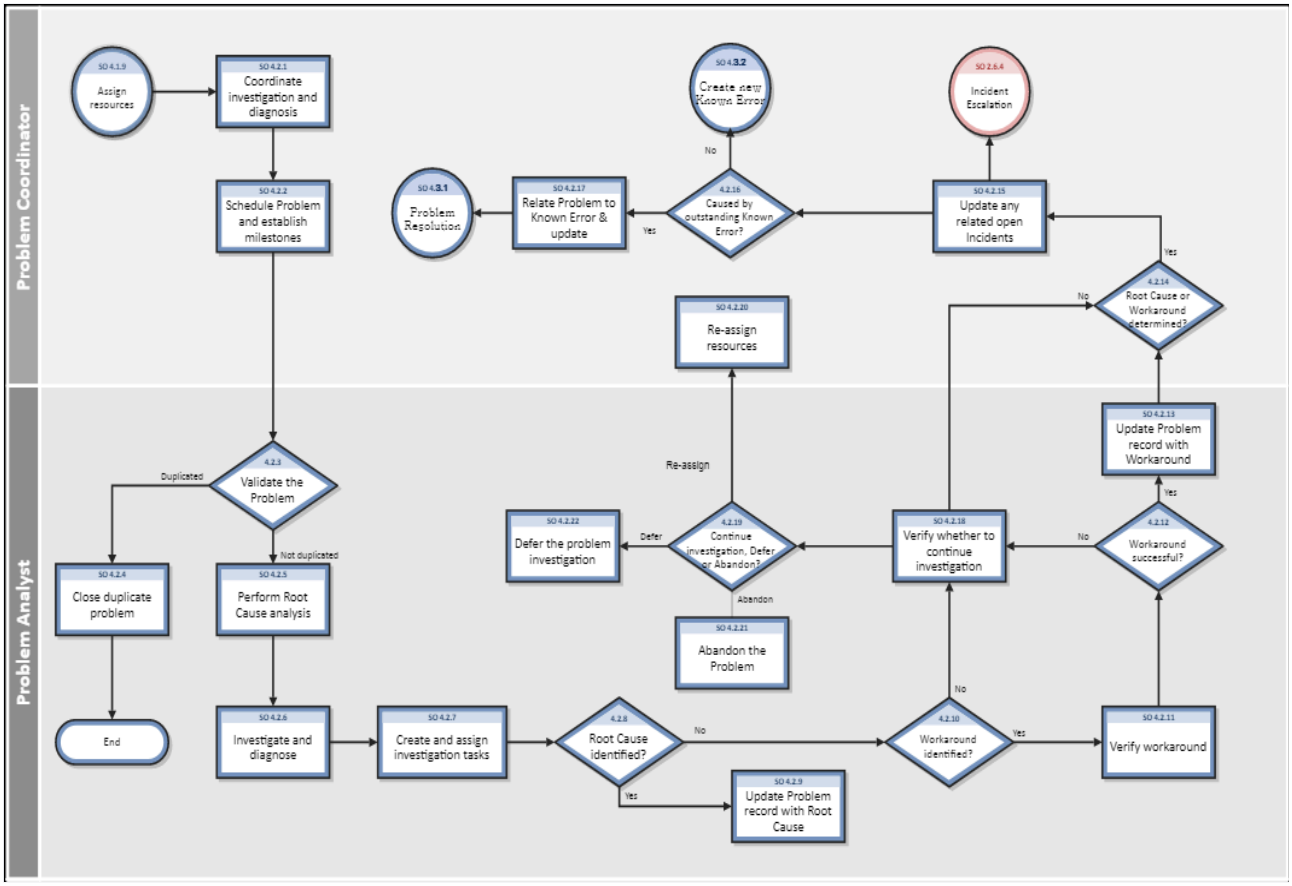
SO 4.1.5	Categorize problem	The Problem Coordinator categorizes the problem into a specific domain (for example, hardware, software, or security). Problems can be categorized in the same way as incidents, so that the true nature of problems can be easily traced in the future, and meaningful management information can be obtained. Other information (such as the estimated cost and estimated effort) is entered, if it is available at this stage. These fields can be updated at a later stage if new information becomes available. If the problem is not categorized in the appropriate category, the Problem Coordinator can change the category, which initiates a new workflow.	Problem Coordinator
SO 4.1.6	Link incidents to problem record	The Problem Coordinator links all related incident records to the problem record. The Problem Coordinator also captures other information, such as the impact (from SLM), urgency, and subcategory of the problem.	Problem Coordinator
SO 4.1.7	Prioritize problem and identify major problem	The Problem Coordinator identifies whether it is a major problem based on the impact and the organization's priority.	Problem Coordinator
SO 4.1.8	Problem investigation to be Deferred?	If the problem investigation must be deferred, move the problem to the appropriate status. If the problem investigation doesn't need to be deferred, go to SO 4.1.9.	Problem Manager

SO 4.1.9	Defer the problem investigation	The Problem Manager defers the problem investigation for a specific period of time. The reason for deferring the problem is detailed in the ticket. Periodically, the Problem Manager reviews the deferred problems to determine the appropriate action. Reasons for deferring problem include the following: • The likelihood of recurrence is low • The cost of resolving the problem is very high • There is currently no plan to investigate the problem	Problem Manager
SO 4.1.10	Assign Resources	The Problem Coordinator determines the skills and personnel that are required to resolve the problem, and assigns personnel to resolve the problem.	Problem Coordinator

9.8.2. Problem Investigation and Diagnosis (process SO 4.2)

The Problem Investigation and Diagnosis process helps identify the root cause of the problem. Where appropriate, the problem management process should develop and maintain workarounds that enable the incident management process to help service restoration. Different specialists can be involved in this root cause analysis. If necessary, refer to external resources to verify whether the problem has already been identified and published by vendors. Decide the target dates for the problem investigation.

The following figure illustrates the Problem Investigation and Diagnosis workflow:



Problem Investigation and Diagnosis process

Process ID	Procedure or Decision	Description	Role
SO 4.2.1	Coordinate investigation and diagnosis	The Problem Coordinator verifies the schedules of resources, and assigns the resources to the problem resolution. The assigned resource starts their investigation. The Problem Coordinator coordinates the tasks that required to resolve the problem, and maintains communication with all stakeholders.	Problem Coordinator
SO 4.2.2	Schedule the problem and establish milestones	The Problem Coordinator estimates the cost and effort required to resolve the problem, and determines the target dates for the problem resolution milestones. Target dates are determined by the priority of the problem and by the impact of the problem on affected services. Additionally, this phase of planning considers whether an effective workaround or fix is available.	Problem Coordinator
SO 4.2.3	Validate the problem	The Problem Analyst ensures that the problem record is valid. The Problem Analyst determines whether the problem record is a duplicate or new problem. Then, the Problem Analyst continues with root cause analysis. If the problem is a duplicate, it is linked to the problem that it is a duplicated of, and the workflow moves to SO 4.2.5. If the problem is not a duplicate, the workflow moves to SO 4.2.4.	Problem Analyst
SO 4.2.4	Close duplicate problem	The Problem Analyst closes the problem as a duplicate, and enters the necessary closure comments into the ticket.	Problem Analyst

SO 4.2.5	Perform root cause analysis	The Problem Analyst performs root cause analysis of the problem. Root cause analysis can include the following methods: • Chronological Analysis • Pain Value Analysis • Kepner and Tregoe • Brainstorming • Ishikawa Diagrams • Pareto Analysis	Problem Analyst
SO 4.2.6	Investigate and diagnose	The Problem Analyst analyzes the known data to identify and isolate the root cause of the problem. If a potential root cause is identified, it is verified. If more resources are required to do this, they are requested through the Problem Coordinator, who requests them from the Problem Manager and Problem Coordinator. Additionally, the Problem Analyst tries to identify a workaround. Potential workarounds are tested to verify that they work. If successful, a workarounds is documented in the problem record. The same is intimated to Service Desk and Incident Analysts	Problem Analyst
SO 4.2.7	Create and assign investigation tasks	The Problem Analyst creates and assigns problem tasks to the resource who is responsible for root cause analysis. The Problem Analyst enters the due date for the assigned task. Additional resources (for example, suppliers and other specialists) can be used for this analysis. The Problem Analyst monitors the outstanding problem tasks.	Problem Analyst

SO 4.2.8	Root cause identified?	If the root cause is not identified, the Problem Analyst must determine whether there is a workaround for the problem. If a root cause is identified, the Problem Analyst updates the problem record with the details.	Problem Analyst
SO 4.2.9	Update problem record with root cause	The Problem Analyst updates the problem record to indicate that a root cause has been found. The problem record is updated with any affected CIs.	Problem Analyst
SO 4.2.10	Workaround identified?	If a workaround is identified, the workflow moves to SO 4.2.11. If no workaround is identified, the workflow moves to SO 4.2.16.	Problem Analyst
SO 4.2.11	Verify workaround	The Problem Analyst creates a problem task and assigns it to the Investigation category in order to test the suitability of the identified workaround for resolving related incidents.	Problem Analyst
SO 4.2.12	Workaround successful?	If the workaround is successful, the workflow moves to SO 4.2.13. If the workaround is not successful, the workflow moves to SO 4.2.16.	Problem Analyst
SO 4.2.13	Update problem record with workaround	Update the workaround (in the known error and the problem) and inform stakeholders.	Problem Analyst
SO 4.2.14	Root cause or workaround determined?	The Problem Coordinator validates the results of the problem task. If the root cause is determined, the workflow moves to SO 4.2.15. If the root cause is not determined, the workflow moves to SO 4.2.16, and then determine whether additional resources are needed or whether escalation is required.	Problem Coordinator

SO 4.2.15	Update any related open Incidents	Review any related open incidents and advise the assigned Incident Analyst that a root cause and/or workaround has been identified. (An update will be made to the Activity Log in the incident record when the problem record is saved with an updated workaround).	Problem Coordinator
SO 4.2.16	Caused by outstanding known error?	Determine whether the root cause for this problem is related to an outstanding known error. If yes, continue with SO 4.2.xx. If no, forward the problem to the Problem Resolution phase, and then create a new known error record .	Problem Coordinator
SO 4.2.17	Relate problem to outstanding known error	The problem is moved to the Problem Resolution phase and linked to the existing known error record. The resolution of the problem is dependent on the resolution of this known error.	Problem Coordinator
SO 4.2.18	Verify whether to continue investigation	The Problem Analyst determines whether to continue with the investigation, start problem resolution, or recommend abandonment.	Problem Analyst
SO 4.2.19	Continue, defer, or abandon investigation?	If the Problem Analyst decides to continue the investigation, the workflow moves to SO 4.2.6. If the Problem Analyst determines they don't have the capabilities to investigate and determine the root cause of the problem (that is, they don't have the skill level or the available time), the Problem Analyst documents the reason that a root cause is not found, the Problem Coordinator is informed, and the workflow moves to SO 4.2.18. If the problem can be abandoned, the workflow moves to SO 4.2.19. If the problem can be deferred, the workflow moves to SO 4.2.20.	Problem Analyst

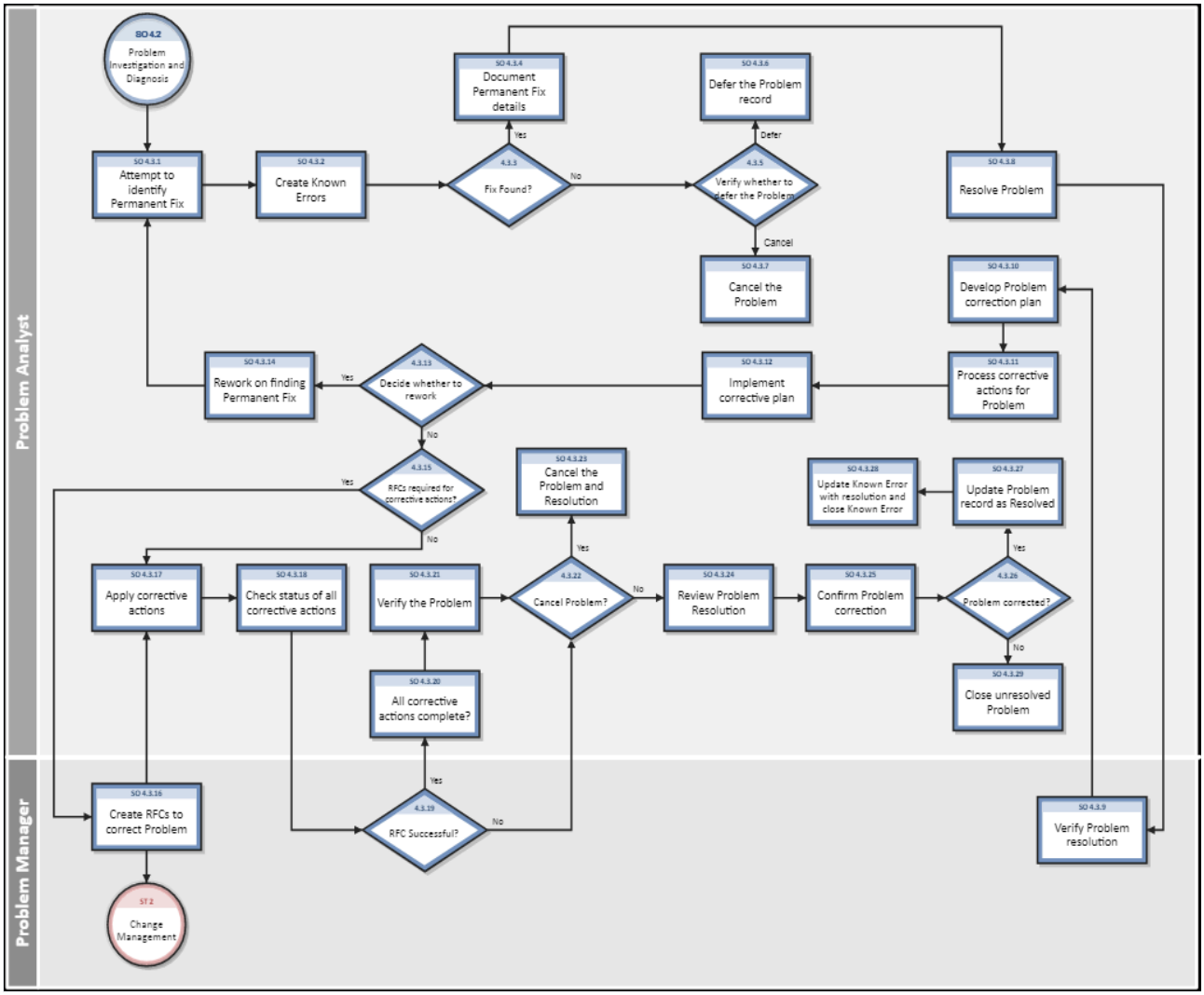
SO 4.2.20	Re-assign resources	The Problem Coordinator needs to re-assigns the problem to other resource to continue the problem investigation. The problem is moved back to Categorization phase with the Assign status again, and the workflow moves to SO 4.1.10.	Problem Coordinator
SO 4.2.21	Abandon the problem	The Problem Analyst abandons the problem ticket.	Problem Analyst
SO 4.2.22	Defer the problem investigation	The Problem Analyst defers the problem investigation for a specific period of time. The reason for deferring the problem is detailed in the ticket. Periodically, the Problem Manager reviews the deferred problems to determine the appropriate action. Reasons for deferring problem include the following: • The likelihood of recurrence is low • The cost of resolving the problem is very high • There is currently no plan to investigate the problem	Problem Analyst

9.8.3. Problem Resolution (process SO 4.3)

After the Problem Management Investigation phase has identified the root cause of an incident, the Problem Resolution phase starts. In collaboration with specialist staff, the Problem Analyst assesses the means of resolving the problem. If necessary, the Problem Analyst requests for an RFC according to change management procedures, and links the RFC to the problem record.

The Problem Resolution phase comprises activities that identify and apply a solution to a problem.

The following figure illustrates the Problem Resolution workflow:



Problem Resolution process

Process ID	Procedure or Decision	Description	Role
SO 4.3.1	Attempt to identify permanent fix	The Problem Analyst attempts to identify a permanent fix for the problem record. Workarounds may be found at this point, if they were not found in the previous phase. The workarounds are tested and documented, if successful. Sometimes, other teams or vendors are involved in identifying the fix for the problem. The problem ticket may incorporate many tasks that are assigned to various analysts or teams who are working towards the problem resolution.	Problem Analyst
SO 4.3.2	Create Known Error	If needed, create a known error record from the problem record for reference and to contribute to the known error knowledge base.	Problem Analyst
SO 4.3.3	Fix found?	If a permanent fix is identified, it is documented. If a permanent fix is not identified, it is verified for closure or deferral.	Problem Analyst
SO 4.3.4	Document permanent fix details	The Problem Analyst documents the permanent fix details in the problem record and updates the Knowledge Management Database with the permanent fix details. Then, the workflow moves to SO 4.3.8.	Problem Analyst
SO 4.3.5	Verify whether to defer the problem	The Problem Analyst decides whether to defer or cancel the problem. If the problem is waiting for vendor resolution or for resource or budget approval, the problem is deferred, and the workflow moves to SO 4.3.6. If the problem is canceled, the workflow moves to SO 4.3.7.	Problem Analyst
SO 4.3.6	Defer the problem record	The Problem Analyst defers the problem record for a period of time.	Problem Analyst

SO 4.3.7	Cancel the problem	The Problem Analyst closes the problem ticket and marks it as canceled.	Problem Analyst
SO 4.3.8	Resolve problem	In collaboration with specialist staff, the Problem Analyst assesses the means of resolving the problem. If necessary, they complete an RFC according to Change Management procedures, and link the RFC to the problem record.	Problem Analyst
SO 4.3.9	Verify problem resolution	The Problem Manager reviews the history of the problem and its resolution, and then makes a final decision as to whether or not the problem is corrected. If the problem is corrected, the workflow moves to SO 4.3.10.	Problem Manager
SO 4.3.10	Develop problem correction plan	The Problem Analyst develops a plan that details all the corrective actions that must be performed to fix the problem. The Corrective Action Plan must include documenting the results of any monitoring that may have been implemented to monitor the problem resolution.	Problem Analyst
SO 4.3.11	Process corrective actions for problem	The Problem Analyst processes the problem record and prepares to implement the corrective actions. Problem tasks may be created and assigned to the Resolution category in order to execute the corrective actions.	Problem Analyst
SO 4.3.12	Implement corrective plan	The schedule to implement the fix is identified and updated in the correction plan. The correction plan is executed.	Problem Analyst
SO 4.3.13	Decide whether to rework	The Problem Analyst checks the implementation result and decides whether to rework the problem correction or abandon the problem fix.	Problem Analyst
SO 4.3.14	Rework on finding permanent fix	If the plan must be reworked, the Problem Analyst updates the problem status to Work In Progress. Then, the workflow moves to SO 4.3.1.	Problem Analyst

SO 4.3.15	RFCs required for corrective actions?	The Problem Analyst determines if any CIs must be modified in order to implement the resolution. If RFCs are required to resolve the problem, the Problem Manager is informed and the workflow moves to SO 4.3.16. If RFCs aren't required to resolve the problem, the Problem Analyst can apply non-CI or pre-approved changes, and the workflow moves to SO 4.3.17.	Problem Analyst
SO 4.3.16	Create RFCs to correct problem	The Problem Manager creates RFCs, links them to the problem, and inform Change Manager if required. The Problem Manager monitors the RFCs that are required to correct the problem.	Problem Manager
SO 4.3.17	Apply corrective actions	Some Problem resolutions require both non-CI-related activities and activities that affect CIs. The Problem Analyst applies non-CI corrective actions to fix the error.	Problem Analyst
SO 4.3.18	Check status of all corrective actions	The Problem Analyst checks the status of all the corrective actions that are applied to the problem record.	Problem Analyst
SO 4.3.19	RFC successful?	The Problem Manager checks the RFCs from the Change Management process to verify that they are successful. If the RFCs are successful, the status of all corrective actions are checked. If the RFCs aren't successful, abandonment of the RFCs is recommended.	Problem Manager
SO 4.3.20	All corrective actions complete?	If all corrective actions are complete, the Problem Analyst begins to monitor the resolution.	Problem Analyst
SO 4.3.21	Verify the problem	The Problem Analyst verifies the resolution and the corrective actions to determine whether the problem can be abandoned or deferred.	Problem Analyst

SO 4.3.22	Cancel problem?	If the resolution to the problem needs to be canceled, the process moves to SO 4.3.23. If the resolution doesn't need to be canceled, the corrective action continues.	Problem Analyst
SO 4.3.23	Cancel the problem and resolution	The Problem Analyst closes the problem ticket and sets it to the "canceled" state.	Problem Analyst
SO 4.3.24	Review problem resolution	The Problem Analyst reviews the history of the problem and its resolution, and then determines whether the problem is corrected.	Problem Analyst
SO 4.3.25	Confirm problem correction	The Problem Analyst reviews the problem history and resolution data, and then confirms that the affected personnel no longer experience the problem.	Problem Analyst
SO 4.3.26	Problem corrected?	If the problem is not corrected, the problem is set to the "unresolved" state and closed. Then, the workflow moves to SO 4.3.28. If the error is corrected, the workflow moves to SO 4.3.27.	Problem Analyst
SO 4.3.27	Update problem record as resolved	The Problem Analyst sets the problem record to the “resolved” state. The Problem Analyst closes the problem ticket, or sends it to Problem Manager for review and closure.	Problem Analyst
SO 4.3.28	Update known error with resolution and close known error	The Problem Analyst updates the related open known error record with the problem resolution, and then closes the known error.	Problem Analyst
SO 4.3.29	Close unresolved problem	The Problem Analyst sets the problem record to the “unresolved” state and closes it. Sometimes, the problem record may be reworked. In such cases, the ticket is assigned to other teams, or the ticket's status is modified.	Problem Analyst

9.8.4. Problem Review and Closure (process SO 4.4)

After a problem has been resolved, it is automatically forwarded from the Problem Resolution phase to the Problem Review phase. In this phase, the problem(s) must be reviewed to determine and validate whether it has been resolved.

After a problem has been reviewed and closed, it is forwarded from the Problem Review phase to the Problem Closure phase. The problem record must be formally closed when any change has been completed and successfully reviewed, and the resolution has been applied.

A problem review should be scheduled whenever an investigation into unresolved, unusual, or high-impact problems justifies it. The purpose of the problem review is to seek improvements to the process, and to prevent the recurrence of incidents or mistakes.

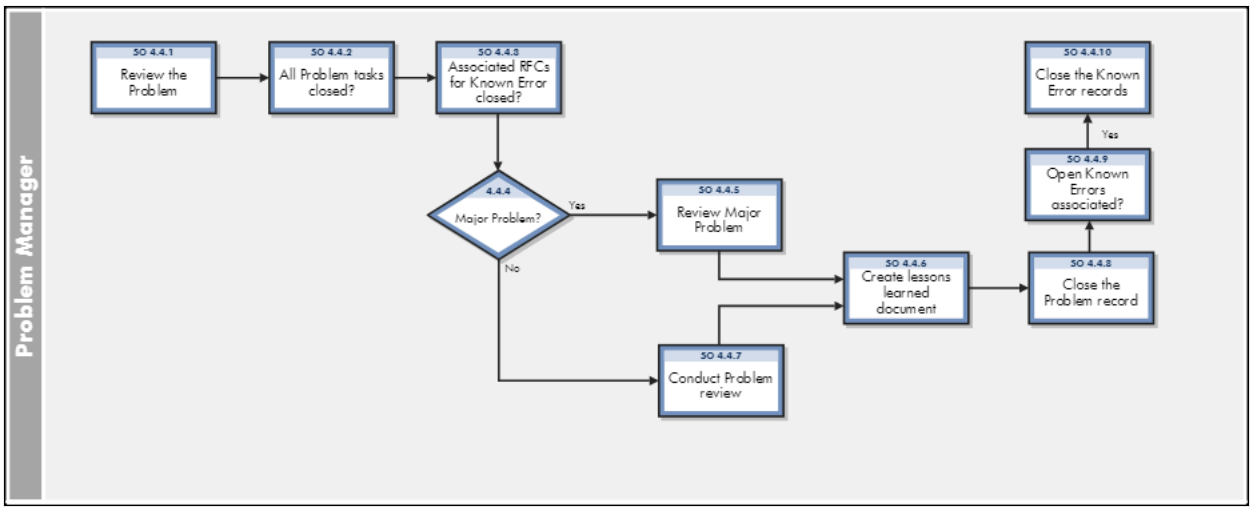
Problem reviews typically include the following elements:

- Reviews of individual incident levels and problem status against service levels
- Management reviews to highlight those problems that require immediate action
- Management reviews to determine and analyze trends, and to provide input for other processes, such as user education and training

Problem reviews should include identifying the following elements:

- Trends (for example, recurring problems and incidents)
- Recurring problems of a particular classification component or location
- Deficiencies caused by lack of resources, training, or documentation
- Non-conformance (for example, against standards, policies, and legislation)
- Problems identified as known errors in planned releases
- Staff resource commitment in resolving incidents and problems
- Recurrence of resolved incidents or problems
- Improvements to the service or to the problem management process should be recorded and entered into a service improvement plan. This information should be added to the problem management knowledge base. All relevant documentation should be updated (for example, user guides and system documentation).

The following figure illustrates the Problem Review and Closure workflow:



Problem Review and Closure process

Process ID	Procedure or Decision	Description	Role
SO 4.4.1	Review the problem	The Problem Manager reviews the problem to determine whether it can be closed and to determine the reason for the closure.	Problem Manager
SO 4.4.2	All problem tasks closed?	The Problem Manager checks whether there any problem tasks aren't closed. If there are open tasks, the task owner is requested to close or to cancel it.	Problem Manager
SO 4.4.3	Associated RFCs for known error closed?	The Problem Manager checks whether there any associated RFCs aren't closed. If there are open RFCs, the RFC owner is requested to close or to cancel it.	Problem Manager
SO 4.4.5	Major problem?	If the problem is major, a formal major review is conducted. If the problem is not major, a regular review is conducted.	Problem Manager
SO 4.4.5	Review major problem	After every major problem (as determined by the organization's priority system), a review must be conducted to determine the lessons learned. Specifically, the review should examine the following items: • Actions correctly performed • Actions incorrectly performed • What can be done better in the future • How to prevent recurrence of the problem • Whether there has been any third-party responsibility • Whether any follow-up actions are needed.	Problem Manager

SO 4.4.6	Create lessons learned document	A “lessons learned” document is created and placed in SKMS, and all stakeholders are informed. The Problem Manager sends necessary details for service or process improvement to the Service Improvement Process, if required.	Problem Manager
SO 4.4.7	Conduct problem review	The Problem Manager initiates problem review activities and coordinates the formal review process. All parties involved in the problem resolution are included in the review, which summarizes what went well, what could be done better next time, what went wrong, and why some things went wrong.	Problem Manager
SO 4.4.8	Close the problem record	The Problem Manager closes the problem record by providing appropriate the closure code and comments.	Problem Manager
SO 4.4.9	Open Known Errors associated?	If there is an open known error associated with the problem, choose whether to close the known error as well as closing the problem.	Problem Manager
SO 4.4.10	Close the Known Error records	The Problem Manager closes the associated known error.	Problem Manager

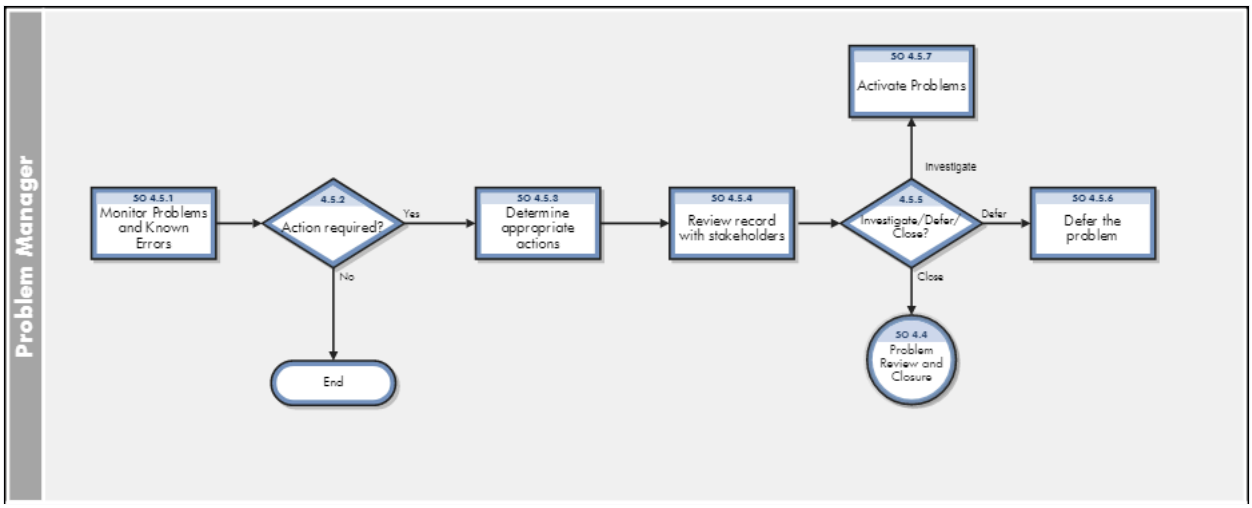
9.8.5. Problem and Known Error Monitoring (process SO 4.5)

Problem management monitors the continuing impact of problems and known errors on user services. In the Problem and Known Error Monitoring process, the Problem Manager periodically reviews the problem and known error records and monitors the progress of activities in those records against the target dates that are agreed with stakeholders.

Service Manager tracks individual problems and their associated known error activities. The Problem Manager evaluates the progress of those activities against the plans and associated budget. In the event that the impact of a problem becomes severe, the Problem Manager escalates the problem. In some cases, the Problem Manager may refer the escalated problem to an appropriate board to increase the priority of the request for change or to implement an urgent change.

The Problem Manager monitors the progress of each problem resolution against service level agreements, and periodically informs the stakeholders of that progress.

The following figure illustrates the Problem Monitoring workflow:



Problem Monitoring process

Process ID	Procedure or Decision	Description	Role
SO 4.5.1	Monitor Problems and Known Errors	The Problem Manager reviews the problem and known error records periodically, and compiles a list or report of the problem records for review. This list or report includes the following items: • Active problem records (to evaluate progress against the planned schedule and associated budget) • Deferred problem records (to evaluate whether they should remain in deferred status) The review may also be triggered by new releases or by changes being implemented. The Problem Manager identifies the appropriate action for the records. Deferred records are activated or closed. Active records may be deferred or abandoned for various reasons.	Problem Manager
SO 4.5.2	Action required?	If any action is required, it is performed. If no action is required, the monitoring process continues.	Problem Manager
SO 4.5.3	Determine appropriate actions	The Problem Manager identifies the appropriate action for the record. Possible actions include the following: • Record is closed as the problem is not relevant anymore • Record is investigated • Record is deferred	Problem Manager
SO 4.5.4	Review record with stakeholders	The actions for the records are reviewed with the stakeholders.	Problem Manager

SO 4.5.5	Investigate/Defer/Close?	Determine whether the problem is still relevant. If a deferred problem record needs to be worked on, it is activated. If the problem is not relevant, the problem record is checked for closure and deferring.	Problem Manager
SO 4.5.6	Defer the problem	The problem record is deferred. The tentative activation date is entered in the problem record. If a problem must be deferred, the activation dates may need to be modified.	Problem Manager
SO 4.5.7	Activate problems	The Problem Manager updates a deferred problem record with scheduling and resource information. The manager also moves the problem record to the appropriate state for the resumption of work. The record is activated, coordinated by the Problem Coordinator, and worked upon by the assigned Problem Analyst.	Problem Manager

9.9. Problem Management details

Service Manager uses the Problem Management application to enable the Problem Management process. The main function of Problem Management is to identify and resolve problems.

In Problem Management, the Problem Manager categorizes and prioritizes problems. The Problem Coordinator manages root cause analysis and resolution, and the Problem Analyst diagnoses the root cause of the problem and proposes and implements solutions for them.

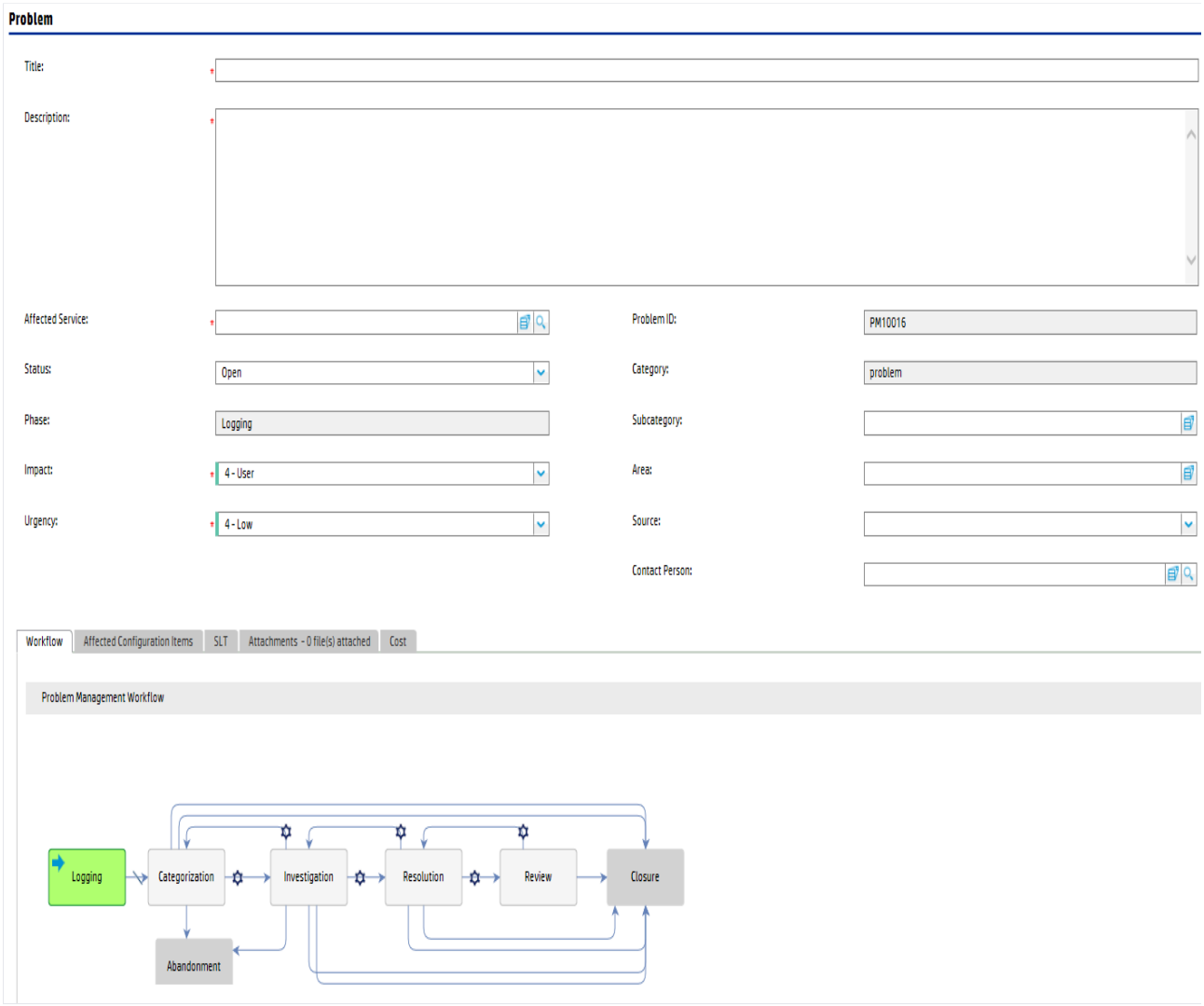
This section describes selected Problem Management fields in the out-of-box Service Manager system. This section includes the following topics:

- Problem form for problems opened from an incident
- Problem form details

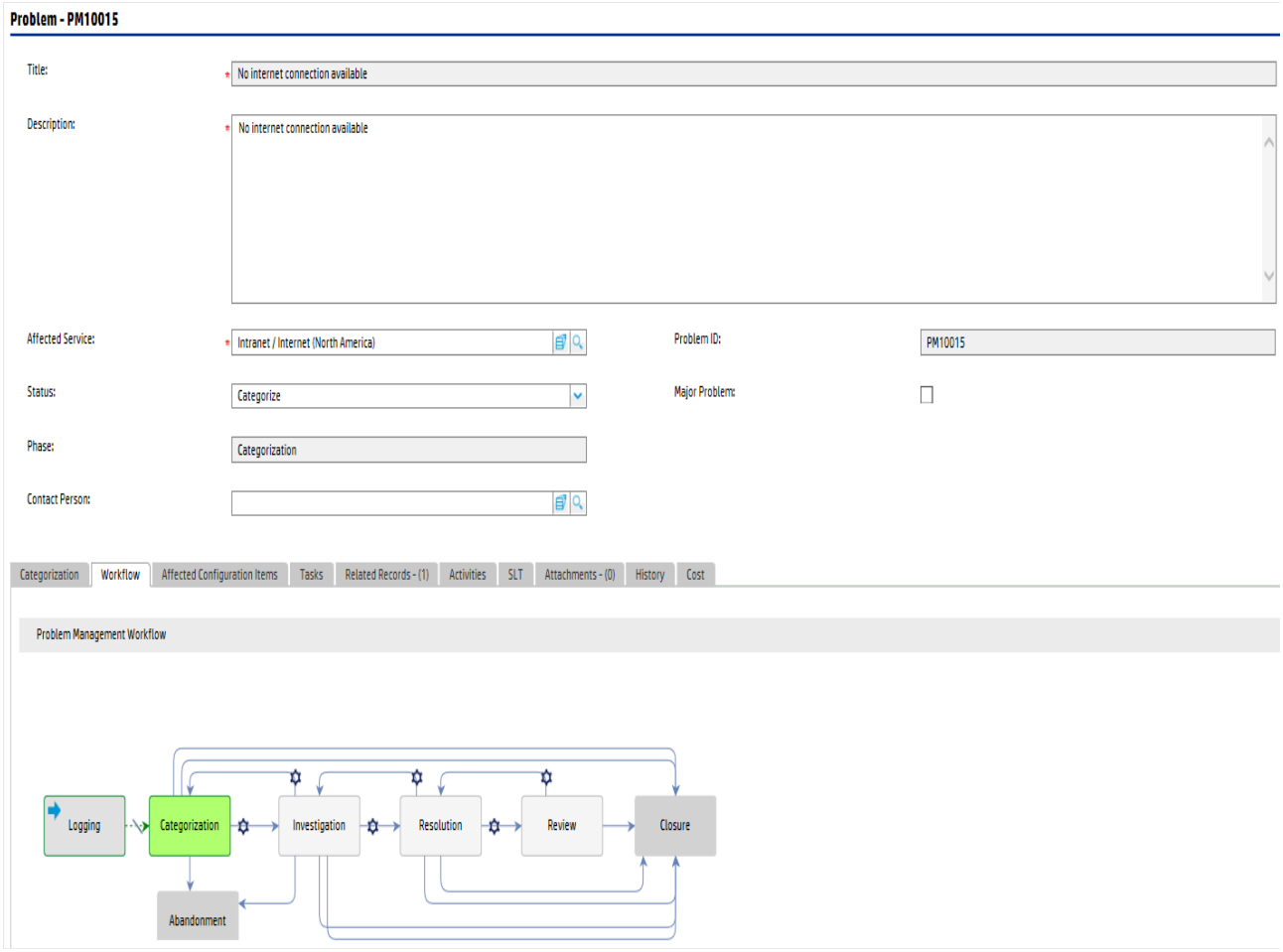
9.9.1. Problem form after escalation from incident

After a problem is opened from an incident, the problem enters the Logging phase.

The following screenshot illustrates a new problem form:

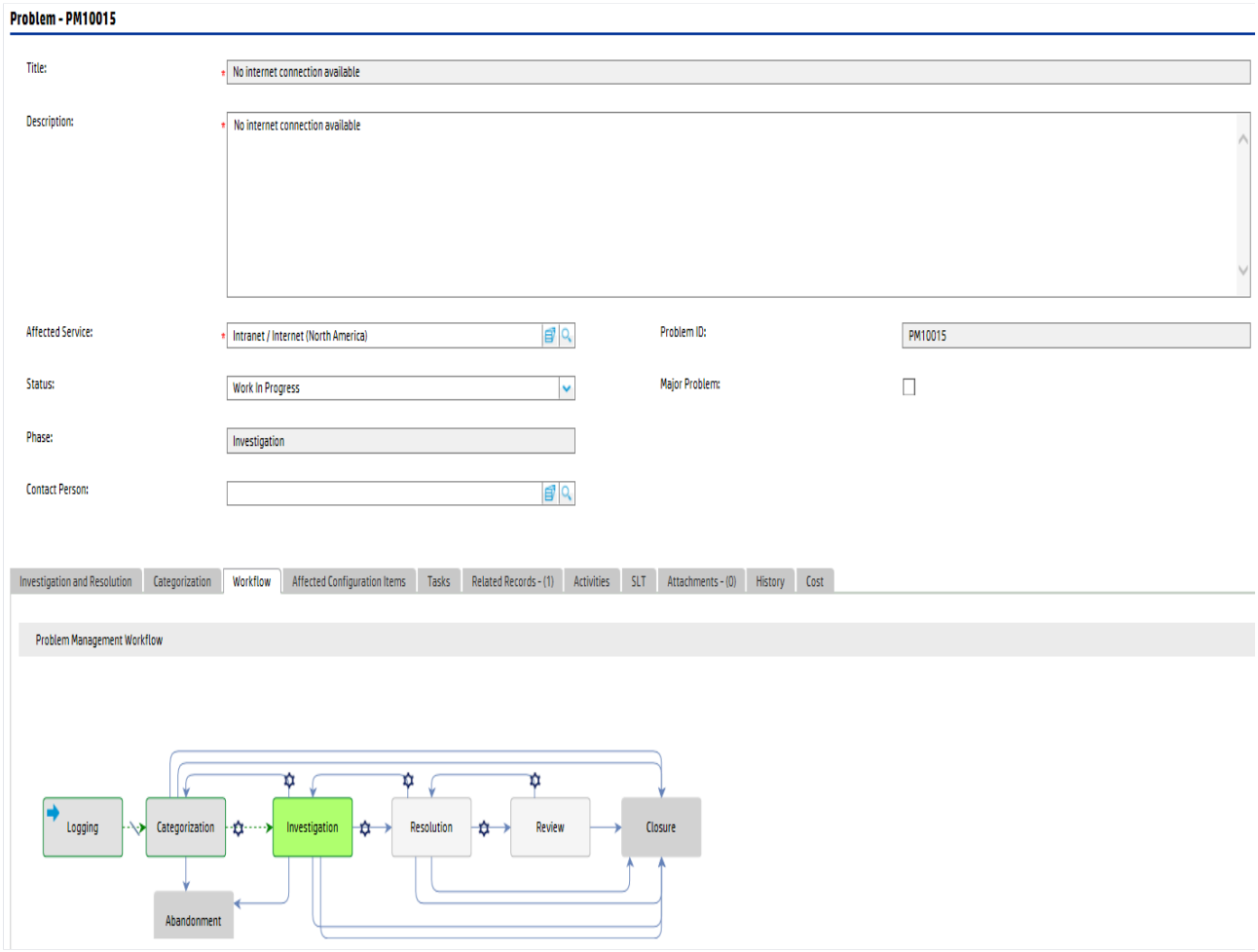


After you save the new problem form, the problem moves to the Problem Categorization phase. The Problem Manager categorizes and prioritizes problems. The following screenshot illustrates this change:



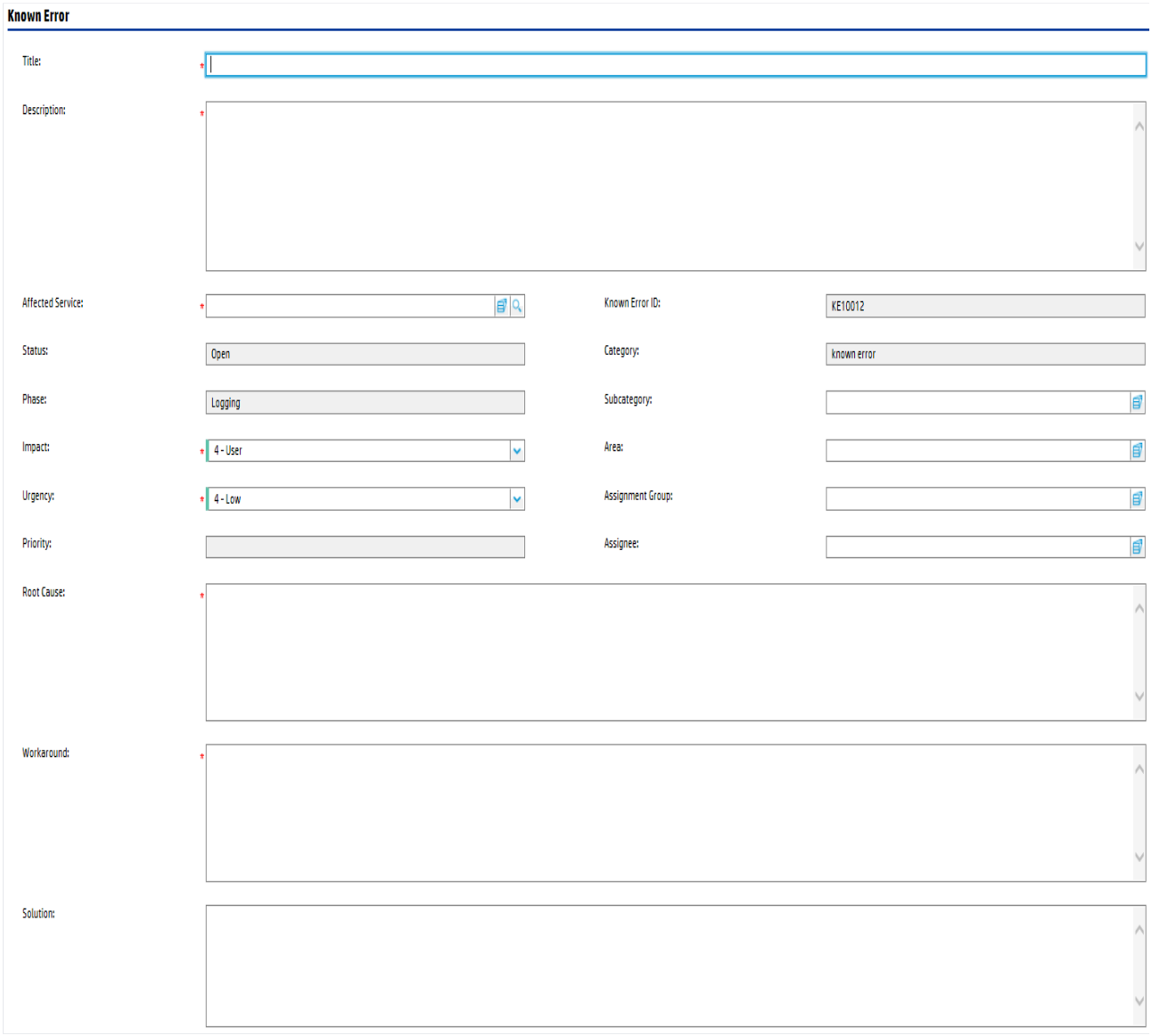
The Problem Analyst uses the problem investigation form to diagnoses the root cause of the problem and proposes solution.

The following screenshot illustrates a problem investigation form:



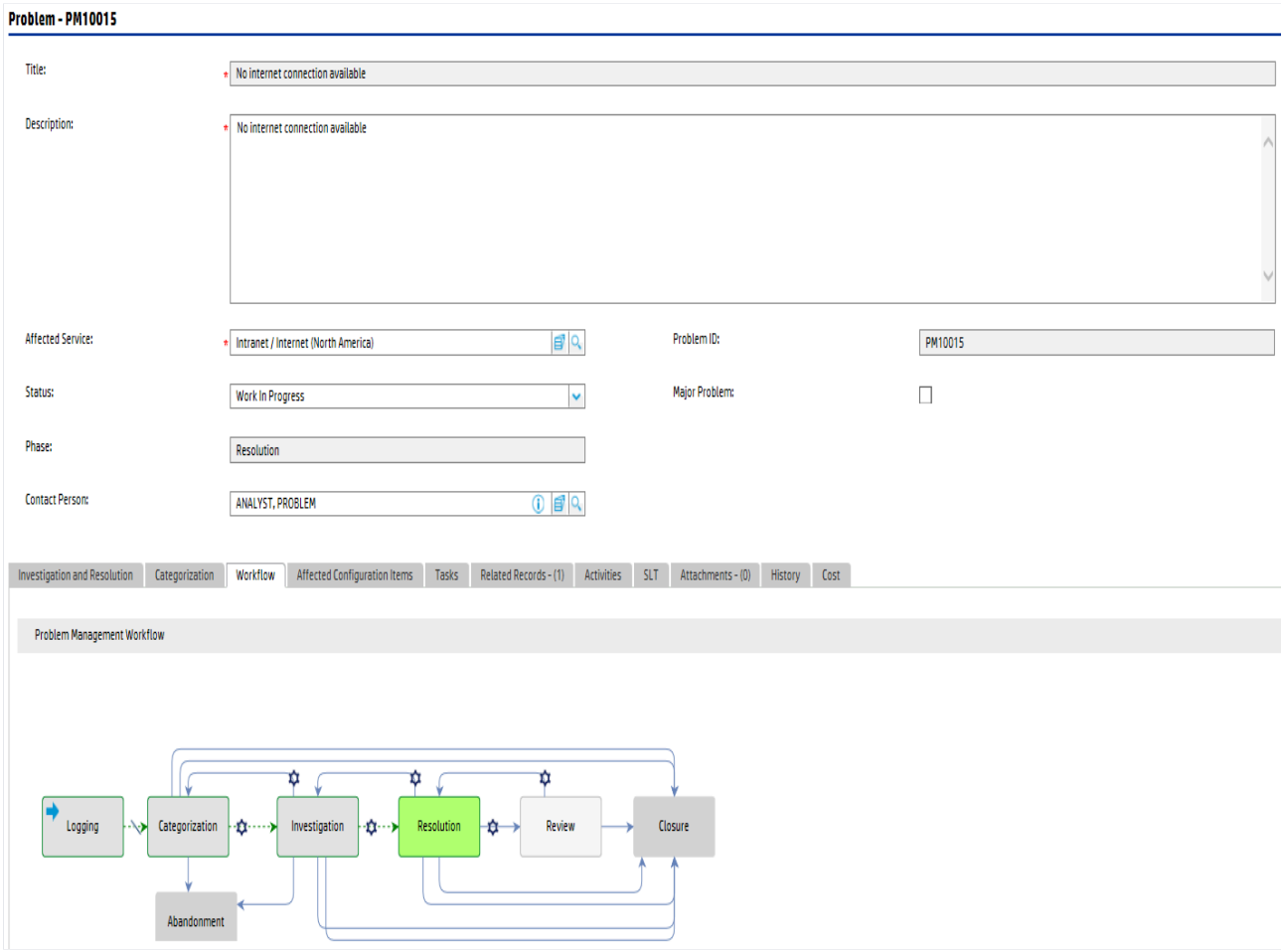
The Problem Coordinator uses the form to create known error records once the root cause and workaround of the problem has been found.

The following screenshot illustrates an open known error form:



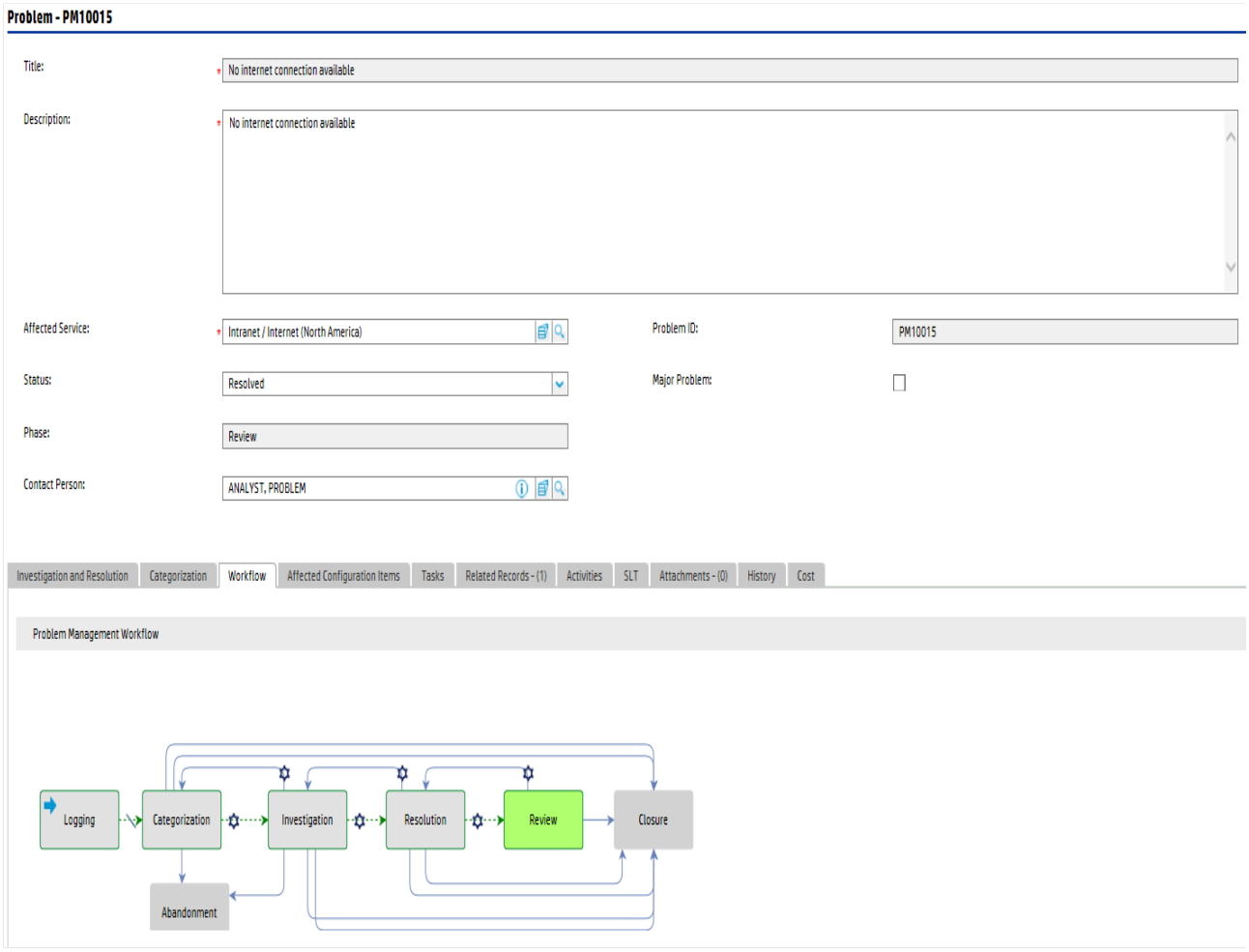
The Problem Analyst uses the problem resolution form to implement the proposes solution to resolve the problems.

The following screenshot illustrates a problem resolution form:



The Problem Manager uses the problem review form to conduct problem reviews.

The following screenshot illustrates a problem reviewform:



9.9.2. Problem form details

The following table identifies and describes some of the features of problem forms:

Problem form details

Label	Description
Problem ID	Specifies the unique ID of the problem. This is a system-generated field.
Title	A short description that summarizes the problem. This field is pre-populated with data from an incident when a user opens a problem from the incident. This is a required field.
Description	A detailed description of the problem. This field is pre-populated with data from an incident when a user creates a problem from the incident. This is a required field.
Affected Service	Specifies the service that is affected by the problem. This field is pre-populated with data from an incident when a user opens a problem from the incident. For more information about this field, see the "Affected service" section of the table in Service Desk Interaction Management form details. This is a required field.
Phase	This is a system-generated field. The following phases are available out-of-box: • Problem Logging • Problem Categorization • Problem Investigation • Problem Resolution • Problem Review • Problem Closure • Problem Abandonment

Status	Specifies the status of the problem. This field may affect the phase of the problem. All status changes must be performed manually. When the status changes, the problem phase may change automatically. There are several reasons to change the status of a problem (for example, when you are waiting for information from a vendor). The following statuses are available out-of-box: • Open — The problem has been opened, but it's not currently being worked on. • Categorize — The problem is being categorized. • Assign — The problem is being assigned to the appropriate resource. • Work In Progress — The problem is being addressed. • Pending — The problem is pending for a period of time. The Problem Coordinator has contacted the vendor for information or for a part, or the Problem Coordinator has contacted the user for more information. • Deferred — Because of several possible constraints, the resolution of this problem must be postponed. • Resolved — A permanent fix is identified, and the problem is resolved. • Closed — The problem is closed or canceled. • Abandoned — The problem is abandoned (this only occurs in the Abandonment phase).
Category	This field is pre-populated with the default category. If a default category is undefined, you must select a category before open a new problem form. The out-of-box data is only “problem”. The problem category can be shared by Service Desk or Incident categories. You can also define new categories according to your needs.
Subcategory	The second level of categorization. This field is pre-populated with data from an escalated incident. Service Manager displays different lists of subcategories, depending on the category that you selected. The subcategories are defined on a category.
Area	The third level of classification, mainly used for reporting purposes. This field is pre-populated with data from an escalated incident. Service Manager displays different lists of areas, depending on the category and subcategory that you selected. The areas are defined on a subcategory.

Impact	This field is pre-populated with data from an incident. It specifies the impact that the problem has on the business. The impact and the urgency are used to calculate the priority. The following impacts are available out-of-box: • 1 - Enterprise • 2 - Site/Dept • 3 - Multiple Users • 4 - User The out-of-box data is the same as Interaction Management and Incident Management.
Urgency	This field is pre-populated with data from the incident. The urgency indicates how pressing the problem is for the organization. The urgency and the impact are used to calculate the priority. For more information about this field, see Service Desk Interaction Management form details.
Major Problem	If selected (set to true), it indicates the problem is a major problem, then the Problem Manager needs to specified and notified.
Major Problem Review section	This section is only visible when Major Problem is selected. Review details for this major problem can be documented under this section.
Source	The source from which the record is created. The following out-of-box sources are available: • 1 - User • 2 - Group • 3 - Event • 4 - Chat • 5 - Email • 6 - Telephone • 7 - Incident
Contact Person	This field is pre-populated with data from the interaction or incident when the problem is opened from an interaction or incident. It specified the contact person for the reported problem.
Categorization > Priority	The order in which to address this problem in relation to other problems. The priority value is calculated by using the initial impact and urgency. This field only appears when problems that are being updated or escalated from incidents.

Categorization > Assignment Group	The group that is assigned to work on the problem. For more information about this field, see Incident Management form details. The out-of-box data consists of default assignment groups for use as examples of types of assignment groups. **Tip** Tip You may want to change the sample assignment groups to meet your own needs. The following assignment groups are available out-of-box: • Application • Email / Webmail • Field Support • Hardware • Intranet / Internet Support • Network • Office Supplies • Office Support • Operating System Support • SAP Support • Service Desk • Incident Manager • Problem Coordinators • Problem Managers • Service Manager This is a required field when status starts from Assign and phase starts from Categorization.
Categorization > Assignee	The name of the person who is assigned to work on this problem. If the Assignment Group field is filled in, the system will populate this field with the pre-defined Problem Coordinator for that group. This person can be changed to any other member of that group using the Fill function. The operator that you select should be a member of the Assignment Group.
Workflow section	Displays a figure of the problem workflow. Also indicates the phase that the problem is currently in, and traces the phase transition history.
Affected Configuration Items section > Primary CI	Specifies the name of the failing Configuration Item (CI). The primary CI identifies the CI that causes the service to go down or become unavailable. The affected CIs in the related incidents and interactions are all of the CIs affected by the service. It is the primary CI that must be fixed to restore the service. For example, if a mail service goes down because of a disk error on the server, the mail server is the primary CI. Every CI that connects to the mail service (that is, that has Microsoft Outlook installed) is an affected CI.
Affected Configuration Items section > Affected CIs' table	The affected CIs are CIs that will have an issue when the primary CI has an issue. These fields must be filled in manually and are for information only. This data doesn't drive any action and is not required.

Affected Configuration Items section > Affected CI Count	A system-generated count of the number of CIs that are affected by the outage. The count doesn't include the primary CI. The affected CI count is based on the number of items entered in theAffected Configuration Items section.The affected CI count is calculated based on the Assessment section in the Affected CIs table.
Tasks section	The user can add tasks during any phase of a problem. Every task must be finished before the problem can be closed. To add a new task, click the Tasks section, and then click the Link New Task button. Service Manager provides users with a quick view of some of the most important fields in the task in the Tasks section. The data displayed includes the following information: • Task ID • Status • Phase • Priority • Title
Related Records section > Link Type	Specifies a relationship type between the problem and the target ticket. The following groups of link types are available out-of-box: • Related Incidents • Caused Changes • Solved By Changes • Related Problems • Related Known Errors
Related Records section > Link Existing/New Record button	After you select a link type, use these two buttons to associate the problem with a target ticket, or to create a new target ticket and associate it with this problem.
Related Records section > All Related Records table	Displays information on all the records that are related to this problem. The data displayed includes the following information: • ID • (Relation) Type • Phase • Status • Title
Related Records section > Duplicate of Problem	Verifies whether this problem is a duplicate of another problem. If the problem is a duplicate, enter the duplicate problem ID in this field. Then, manually close the problem by clicking **More** > **Close Duplicate Problem**.
Related Records section > Duplicate Problems	Identifies problems that are duplicates of this problem. This field can be populated manually, or automatically, when other problems are verified as duplicates of this problem.
Related Records section > Related Incident Count	This is a system-generated field. The related incident count is the number of incidents that are related to the problem, as recorded in the scrlation table. To relate an incident to a problem, click the **Related Records** section, select **Link Type as Related Records**, and then click the **Link Existing Record** button.

Investigation and Resolution section > Expected Resolution Date	The expected problem resolution date should be approximately the same as the SLT Target date. The expected problem resolution date is the date when you plan to close the record. This should be done before the SLT Target date. This field has the Problem Management past due alert attached to it. This field appears when the problem enters Investigation phase. It becomes a required field in the “Work In Progress” and “Investigation” phases.
Investigation and Resolution section > Expected Root Cause Identified Date	Specifies the date by which the identification of the root cause of the problem is expected. You should base the date on the target date and on the identified dates in the SLT. This field appears in the “Investigation” phase to assist prioritization and planning in problem management processing. The field becomes a required field when the status changes to “Work In Progress” and the phase changes to “Investigation”.
Investigation and Resolution section > Solution Identified Date	The date when you identify the solution. This field appears when the problem enters the “Investigation” phase, and becomes required when the status changes to “Resolved”.
Investigation and Resolution > Root Cause	A detailed description of the cause of the problem. You can't move on from the Problem Investigation phase until you have entered a description in this field. That phase is incomplete until the cause of the problem is known.
Investigation and Resolution > Workaround	Describes a temporary solution or workaround.
Investigation and Resolution section > Estimated Man Days	Specifies a resource estimate to diagnose and resolve the problem. This data doesn't drive any action and is not required.
Investigation and Resolution > Estimated Cost	Provides a resource (cost) estimate to diagnose and resolve the problem. This data doesn't drive any action and is not required.
Investigation and Resolution > Solution	Describes a permanent solution to the problem. This field is required when the problem status changes to “Resolved”.
Closure Code	Uses a pre-defined closure code to specify the way in which the problem was closed. The out-of-box data is defined in the probcause table. This field is required when a problem is closed or abandoned. This field is populated in the closure wizard, and then automatically populated in the Summary section of the problem form.
Closure Comments	Comments to close the problem. This field is populated in the closure wizard, and then automatically populated in the Summary section of the problem form.
Cost section	The user can add cost details of parts and labor for this incident handling. For more information about this field, see Incident Management form details.

9.10. Change Management overview

The Service Manager Change Management application, referred to as Change Management throughout this chapter, supports the Change Management process. It controls the process to request, manage, approve, and control changes that modify your organization’s infrastructure. This includes assets such as network environment, facilities, telephony, and resources. Change Management enables you to control the changes to baseline service assets and configuration items across the entire service lifecycle.

This section describes how Change Management implements the best practice guidelines for the Change Management processes.

Topics in this section include:

- Change Management within the ITIL framework

- Change Management application
- Change Management process overview
- Input and output for Change Management
- Key performance indicators for Change Management
- RACI matrix for Change Management

9.10.1. Change Management within the ITIL framework

Change Management is addressed in ITIL's Service Transition publication. The document describes Change Management as the process responsible for ensuring that changes are recorded, evaluated, planned, tested, implemented, and reviewed in a controlled manner.

Change Management enables you to meet the following business objectives:

- Use standardized methods and procedures to ensure efficient and prompt handling of all changes.
- Record all changes to service assets and configuration items (CIs) in the Configuration Management System (CMS).
- Minimize overall business risk.
- Respond to customers' changing business requirements, maximize value and reduce the number of incidents, disruptions, and rework.
- Respond to business and IT requests for changes, aligns services with business needs.

The ITIL Change Management process model includes

- The steps to follow to handle a change
- The order in which the steps are to be followed
- Roles and responsibility of the stakeholders
- Scheduling and planning
- When and how to escalate a change

9.10.2. Change Management application

The primary objective of Change Management is to enable beneficial changes to be made with minimal disruption to IT Services. Changes are recorded, evaluated, authorized, prioritized, planned, tested, implemented, documented, and reviewed in a controlled manner. Change Management objectives are achieved by rigorous adherence to the process steps.

The Change Management application incorporates the essential change management concepts of ITIL to ensure that the best practices of IT service management are applied to the application.

Differences between Change Management and Request Fulfillment[edit]

Change Management and Request Fulfillment are separate processes, but they are closely related. Change Management handles any change to your business that modifies or disrupts the current state of that environment. Usually these modifications or disruptions affect multiple users or business units. Request Fulfillment handles common user requests for products and services. These requests usually affect only the person making the request, or a subordinate group of employees.

- Change Management
  - Manages changes (implementations) that modify a business environment.
  - Affects many users.
  - Scope is often large, including large groups or multiple business units.
- Request Fulfillment
  - Handles common requests for products and services.
  - Affects a small or limited number of users.
  - Scope is limited.

Contents

- 1 Differences between Change Management and Request Fulfillment

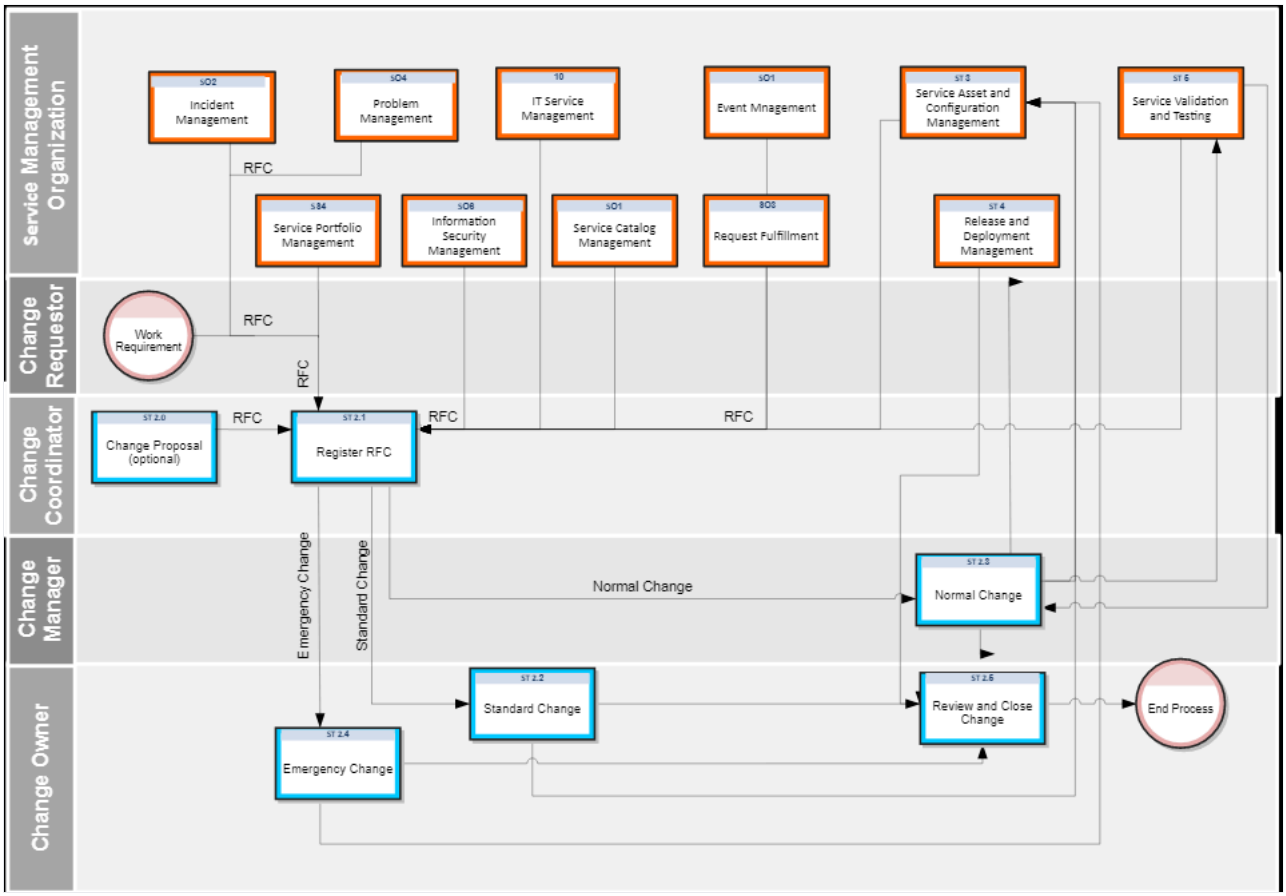
9.10.3. Change Management process overview

The Change Management process includes the activities necessary to control changes to service assets and configuration items across the entire service lifecycle. It provides standard methods and procedures to use when implementing all changes.

The purpose of Change Management is to ensure that:

- Changes follow a set process.
- Appropriate users are notified at key points in the process.
- Progress of a change is monitored and notifications are issued if deadlines are missed.
- Changes are supported throughout a simple or complex lifecycle.

The following figure is the process diagram for Change Management. For more information, see section Change Management Workflows.



9.10.3.1. Change categories and phases

Change Management uses categories to classify the type of change requested. Out-of-box, each change type has its own category that defines the workflow and phases needed to satisfy the change request. They are described in detail in the following sections.

The best practice process flows shipped with the Process Designer framework introduce four processes: Change Proposal, Standard, Normal and Emergency changes, which correspond to the "Change Proposal", "Standard Change", "Normal Change"and "Emergency Change" categories. This is a change with previous releases of Service Manager where at the category-level more specific changes were classified, such as Hardware or Software. They are added to any existing categories including any previous out-of-box categories that may still exist in the system.

As an administrator of the Service Manager application, you can use the default categories shipped with the product, or create new categories to match your business requirements.

9.10.3.2. Change Management categories

Service Manager Categories classify and define the type of change requested. Each category has its own workflow process. The steps of the workflow are represented by the phases and tasks within the phase. Service Manager requires that every change has a change category and phase, but tasks are optional.

Service Manager provides three out-of-box categories you can use to classify the changes in your business. The following table describes the out-of-box Change Management categories.

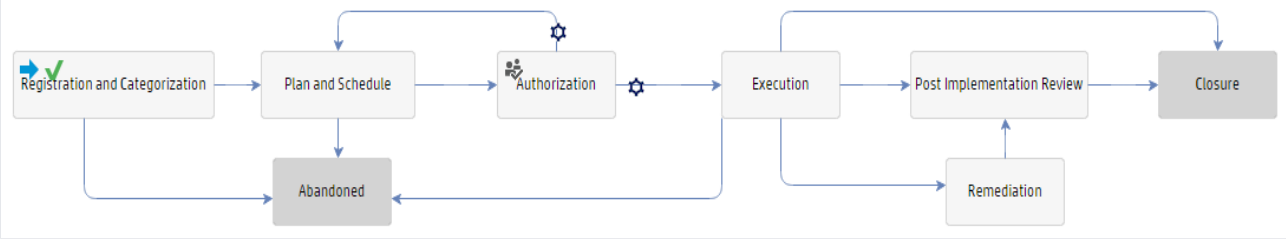
Change Management categories

Category	Description
Change Proposal	A Change Proposal is a high-level description of the suggestion or request for a new or changed service, in order to ensure that potential conflicts for resources or other issues are identified. For example, major changes that involve significant cost, risk or organizational impact are usually initiated by a change proposal.
Emergency Change	Emergency changes are the change processes to be applied in the production environment during emergency situations like service outage.
Normal Change	A Normal Change is a change that's categorized, prioritized, planned and that follows all approvals before deployment. Normal Changes can be further categorized as Major RFC, Significant RFC, and Minor RFC.
Standard Change	A Standard Change is a pre-authorized Change that's of low risk, relatively common and follows a standard procedure.

9.10.3.3. Change Management phases

Service Manager uses phases to describe the steps needed to complete a change request. The phase also determines the change screens users see, the approvals required to advance to the next phase, and the conditions that cause the system to issue alerts.

For example, the following figure shows the workflow phases for a Standard Change.



9.10.3.4. Change Management tasks

Service Manager lists the Change tasks necessary to complete a particular phase. Workflow can't proceed to the next phase until all the associated tasks of the current phase are completed. Tasks can be either sequential or parallel. For example, suppose you are in the Deployment phase of a normal Change, to replace a hard drive. The Change tasks listed may be to take a backup of the old hard drive, remove the old drive, install a new hard drive, test the new hard drive, and restore the data on to the new hard drive. In this example, the tasks are sequential because you can't restore data onto a new drive until you take a backup of the data and install the new hard drive. Parallel tasks might include determining the backup software to be used, the hard drive vendor to purchase from, and the effort and risk the hard drive Change might bring forth. Each phase can optionally have one task, multiple tasks, or no tasks. Tasks include a description, the urgency and priority of the task, task scheduling, and assignment information.

Change Management tasks include:

- Opening, assigning, and associating a task with a Change.
- Searching for a task.
- Managing task categories, environments, and phases.
- Using the task queue.

Change Task phases

This section describes the flow of a Change task as it progresses from the **Waiting** phase to the **Closed** phase in the Generic Task workflow. This workflow is required to implement task dependencies.

The following figure shows the Generic Task workflow in Process Designer.



To Change the phases of a Change task in the Generic Task workflow:

1. Sign in as a Change Coordinator, and then search for an open Change request.
2. Click **More** > **Open New Task** from the menu to create a new Change task.
   Or
   Click the task number in the **Tasks** section to open an existing Change task.
3. When a task is opened, it is in the **Waiting** phase. When the task is being executed, it is in the **Active** phase. The Change Implementer (assignee) Changes the status of task from **Ready** to **Assigned** and then to **In Progress**. Task status is Changed to **Completed** when the task is accomplished.
4. If the Change task is not planned to be implemented or if it's not successful, select the task status **Cancelled** and the task will be moved to **Cancelled** phase.
5. Change task can be closed directly from **Active** phase bypassing the **Review** Phase.
   • If the Risk Assessment value of the Change task is less than or equal to 2, then the **Review** phase is bypassed.
   • If the Risk Assessment value of the Change task is more than 2 (for example 3 or more), the task enters the **Review** phase.
6. Sign in as Change Reviewer, retrieve the task, and review it. Type your comments in the **Review Comments** field.
7. If the task completed is acceptable, close the task by selecting Successful from the **Closure Code** dropdown list and add closure comments, the task is now in the **Closed** phase. If the task is incompleted and needs to be reworked, click **Reopen** after adding review comments. The task moves back to the previous **Active** phase and to the **Assigned** status.
8. Repeat the same steps from 2 to 7 if you want to create, implement and close the tasks at any phase of the Change request.
9. A notification is sent to Change Owner on the successful completion of a task.

Change Task Status value and display list mapping is as follows:

Value	Status
0	Planned
1	Ready
2	Assigned
3	In Progress
4	Blocked
20	Completed
21	Completed with Problems
30	Cancelled
31	Withdrawn
32	Failed

9.10.3.5. Change Management user roles

The following table describes the responsibilities of the Change Management roles.

Change Management user roles

Role	Responsibilities
Change Approver	• Uses the Service Management tool or Change Advisory Board to approve or deny Change when requested. • Facilitates Emergency Change Advisory Board (E-CAB) meetings.
Change Coordinator	• Registers changes and applies the correct change model and change detail. • Schedules changes according to the plan created previously. • Creates the change tasks for building, testing, and implementing a change. • Coordinates the Risk and Impact Analysis phase of the change and creates change plan based on the assessment information. • Verifies if the change has passed the test criteria. • Verifies if the change is implemented successfully in the production environment. • After implementation, evaluates the change and closes the request. • If a change implementation fails, the coordinator activates a back-out plan to return the system to its original state.
Change Owner	• Assesses the validity of the RFC and involves in risk assessment. • Performs post implementation review and responsible for closure of change. • Responsible for convening the ECAB members for Emergency Change approval. • Coordinates with the experts within change process for Build and Test activities. Reviews, revises, and updates schedule for Emergency and Standard Changes. • Plans, receives, and reviews RFC for approvals. • Generates CMDB updates to be submitted to Service Asset and Configuration Management for processing.
Change Manager	• Reviews all changes after the Plan and Schedule phases and forwards them to the right Change Approver. • Organizes Change Advisory Board meeting if necessary. • Updates the change after approval. • Periodically reviews changes in a Post Implementation Review; determines and executes follow-up actions. • Coordinates all activities in case the Emergency Change Handling process is triggered.

ECAB	• Validates that the Emergency Change is truly an emergency (based on emergency change criteria). • Ensures the RFC for Emergency Change is complete. • Ensures the RFC for Emergency Change receives appropriate approval (based on the Change Management Policy). • Makes the final decision that the resolution being implemented to correct the production issue is the best option for the situation. • Ensures the Emergency Change is reviewed by the CAB, post implementation.
Technical Change Advisory Board (TCAB)	• Responsible for analyzing and reviewing the risk and impact of change request. • Authorizes, disapproves, or requests more information for each Normal Major change request. • Ensures all Normal Major changes are adequately assessed and prioritized. • When requested, participates in Change Post Implementation Reviews.
Deployment Change Advisory Board (DCAB)	• Reviews the Build and Test results and approves, abandons, or rejects them. If the Build and Test results are approved, the DCAB reviews and updates the implementation schedule and authorizes the Normal Change implementation to begin. • Participates in scheduling and coordination of the change. • When requested, participates in Change Post Implementation Reviews.

9.10.4. Input and output for Change Management

Changes can be triggered and resolved in several ways. The following table outlines the input and output for the Change Management process.

Input and output for Change Management process

Input to Change Management	Output from Change
• Policy and strategies for change and release • Request for change • Change proposal • Plans (change, transition, release, deployment, test, evaluation, and rendition) • Current change schedule and projected service outage (PSO) • Current assets or configuration items • As-planned configuration baseline • Test results, test report, and evaluation report	• Rejected Request for Changes (RFCs) • Approved RFCs • Change to a service or infrastructure • New, changed, or disposed assets or CIs • Change schedule • Revised PSO • Authorized change plans • Change decisions and actions • Change documents and records • Change Management reports

Key performance indicators for Change Management[edit]

The Key Performance Indicators (KPIs) in the following table are useful for evaluating a Change.

Key Performance Indicators for Change Management

Title	Description
% of unauthorized changes	Percentage of unauthorized implemented changes in a given period. A change in the infrastructure without a registered change request is considered unauthorized.
% of incidents caused by changes	Percentage of incidents caused by the implementation of a change in a given period.
% of emergency changes	Percentage of the total number of closed emergency changes in a given period.
% of successful changes	Percentage of the total number of closed changes successfully implemented in a given period.
% of backed out changes	Percentage of the total number of closed changes for which a remedy plan is activated in a given period.
% of rejected changes	Percentage of the total number of closed changes rejected in a given period.
Average time per phase	Average amount of time spent on each of the distinct change phases in a given period. Validation, Risk and Impact Analysis, TCAB Approval, Build and Test, DCAB Approval, Deployment, Post Implementation Review, CMDB Update, and Closure.

For completeness, the ITIL 2011 and COBIT 4.1 KPIs are included in the following sections.

ITIL 2011 Key Performance Indicators[edit]

The following are ITIL 2011 KPIs for Change Management:

- Number of changes implemented to services that met customer requirements (for example, quality/cost/time expressed as a percentage of all changes).
- Benefits of change expressed as the value of improvements made added to the value of negative impacts prevented or terminated as compared to the costs of the change process.
- Reduction in the number of disruptions to services, defects, rework caused by inaccurate specification, and poor or incomplete impact assessment.
- Reduction in the number of unauthorized changes.
- Reduction in the backlog of change requests.
- Reduction in the number and percentage of unplanned changes and emergency fixes.
- Change success rate (percentage of changes deemed successful at review, that is, the number of RFCs approved).
- Reduction in the number of changes in which remediation is required.
- Reduction in the number of failed changes.
- Average time to implement based on urgency/priority/change type.
- Incidents attributable to changes.
- Percentage accuracy in change estimate.

COBIT 4.1 Key Performance Indicators[edit]

The following are the COBIT 4.1 KPIs for Change Management:

- Number of disruptions or data errors caused by inaccurate specifications or incomplete impact assessment.
- Extent of application rework caused by inadequate change specifications.
- Minimum time and effort required to make changes.
- Percentage of emergency fixes.
- Percentage of unsuccessful changes to the infrastructure due to inadequate change specifications.
- Number of changes not formally tracked, reported, or authorized.
- Number of backlogged change requests.
- Percentage of changes recorded and tracked with automated tools.

- Percentage of changes that follow formal change control processes.
- Ratio of accepted and refused change requests.
- Number of different versions of each business application or infrastructure being maintained.
- Number and type of emergency changes to the infrastructure components.
- Number and type of patches to the infrastructure components.

Contents

- 1 Key performance indicators for Change Management
- 2 ITIL 2011 Key Performance Indicators
- 3 COBIT 4.1 Key Performance Indicators

9.10.5. RACI matrix for Change Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. It is especially useful in clarifying roles and responsibilities in cross-functional/departmental projects and processes. The RACI matrix for Change Management is shown in the following table.

RACI matrix for Change Management

Process ID	Activity	Change Manager	Change Requestor	Change Coordinator	Change Advisory Board(CAB)	Change Approver (or ECAB)	Change Owner
ST 2.1	Register RFC	A	R	R			
ST 2.1.3	Perform RFC Assessment	A	I	R		C/I	
ST 2.1.5	Reject RFC	C/I	R	R	R		
ST 2.1.6	Assign Change Owner	R	R	R			
ST 2.2.1	Identify Standard Change Model and Change Owner	R/A	R	I			I
ST 2.2.2	Prioritize Standard Change	R/A	C/I	I			R
ST 2.2.3	Plan and Schedule Standard Change	R/A	I	R			R
ST 2.2.4	Execute Standard Change	R/A	I	R			R
ST 2.2.6	Remove Standard Change Model	R	I	I			C/I
ST 2.3.1	Assess Change	R/C	I	R			R
ST 2.3.3	Determine Approval Requirements	R	C/I	C/I			A
ST 2.3.5	TCAB Approval	R/A	I	I	R		
ST 2.3.8	Coordinate Build and Test	R.A	I	R			R
ST 2.3.10	Schedule for Normal Change	R/A	I	R			R
ST 2.3.12	Create and Submit Deployment Plan	A	I	R			R

ST 2.3.13	DCAB Approval	R/A	I	I	R		
ST 2.3.15	Decision on Rebuild	R/A			R		
ST 2.3.17	Review RFC after Rebuild Decision	A	I	R			R
ST 2.3.19	Review RFC after DCAB Approval	A	I	R			R
ST 2.3.21	Coordinate and Monitor Change Implementation	R/A	I	R/C			R
ST 2.3.23	Provide CMDB Updates	R/A	I	R			R
ST 2.3.26	Assess Change Success	R/A	I	R			R
ST 2.4	Emergency Change	R/A	C/I		R	R	R
ST 2.4.3	Convene ECAB	R/A				C/I	R
ST 2.4.4	Approve Emergency Change	A/C				R	
ST 2.4.6	Plan and Design Solution	R/A		I			R
ST 2.4.8	Coordinate Emergency Change Build and Test	R/A	I	R			R
ST 2.4.10	Coordinate Emergency Change Implementation	R/A					R
ST 2.4.11	Abandon the Change	R	C	I			R
ST 2.5	Review and Close Change	R/A	C	R			R/C
ST 2.5.3	Conduct Formal Turnover to Support	C/I	C	A/C			R

ST 2.5.4	Perform PIR	A	C/I	R			R
ST 2.5.5	Notify Requestor of Change Results	A/C	I	R			R
ST 2.5.6	Close RFC	R/A	C	R			R

9.11. Change Management workflows

Change Management controls the process to request, manage, approve, and control changes that modify your organization's infrastructure. This managed infrastructure includes assets such as, network environments, facilities, telephony, and resources. For user requests for products and services, refer to Request Fulfillment.

Change Management automates the approval process and eliminates the need for memos, E-mail, and phone calls.

Note

Note The three change workflows (Standard, Normal, and Emergency) have an implementation phase but each are named differently: Execution, Deployment, and Implementation respectively. The names are by design and selected to match the types of activities for the flows as specified in ITIL. Because the activities aren't same, the implementation phase in each workflow is named differently.

The Change Management process consists of the following activities, which are included in this chapter:

Note

Note The following processes are depicted with a light blue border in each of the workflow diagrams in the following section.

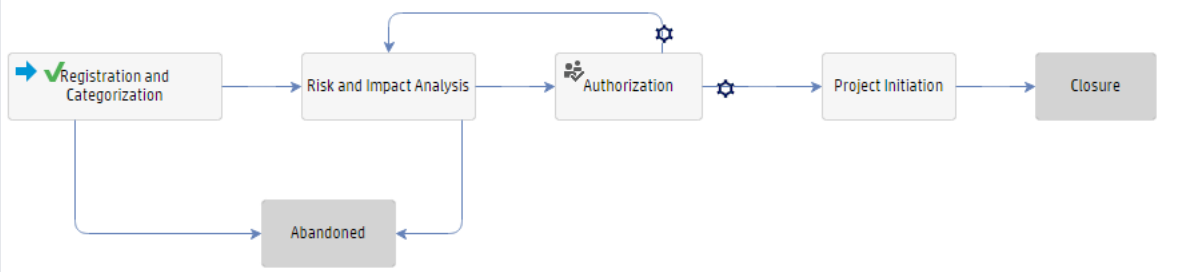
- Change Proposal (process ST 2.0)
- Register RFC (process ST 2.1)
- Standard Change (process ST 2.2)
- Normal Change (process ST 2.3)
- Emergency Change (process ST 2.4)
- Review and Close Change (process ST 2.5)

9.11.1. Change Proposal (process ST 2.0)

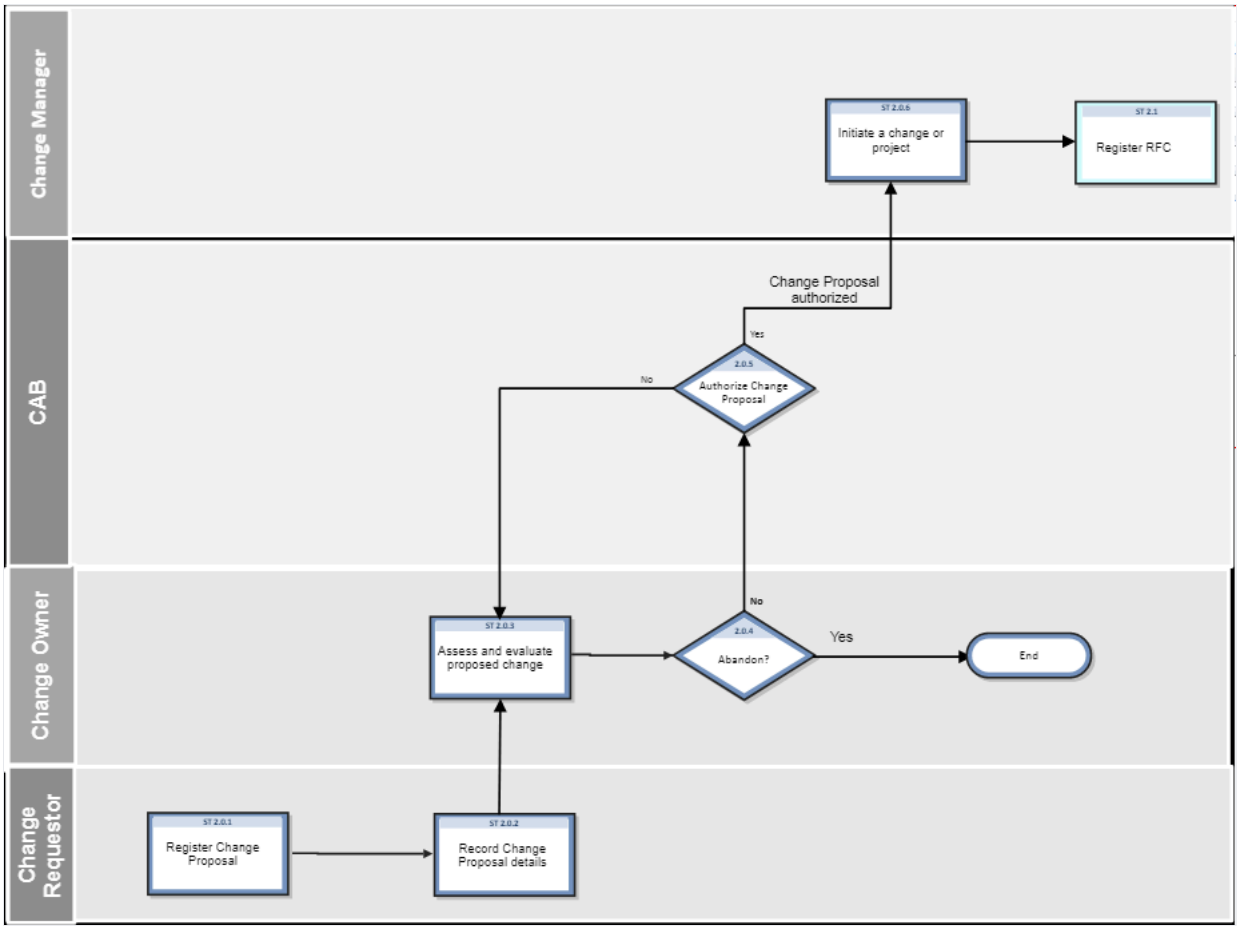
Change proposals are submitted to change management before chartering new or changed services in order to ensure that potential conflicts for resources or other issues are identified. Authorization of the change proposal doesn't authorize implementation of the change but simply allows the service to be chartered so that service design activities can commence.

A change proposal is used to communicate a high-level description of a change. This change proposal is normally created by the service portfolio management process and is passed to change management for authorization. In some organizations, change proposals may be created by a programme management office or by individual projects.

The following figure depicts the Change Proposal workflow in Process Designer.



For details of the Change Proposal process, see the following figure and table.


Change Manager
CAB
Change Owner
Change Requestor
ST 2.0.6
Initiate a change or project
ST 2.1
Register RFC
Change Proposal authorized
Yes
2.0.5
Authorize Change Proposal
No
No
ST 2.0.3
Assess and evaluate proposed change
2.0.4
Abandon?
Yes
End
ST 2.0.1
Register Change Proposal
ST 2.0.2
Record Change Proposal details

Change Proposal (process ST 2.0)

Process ID	Procedure or Decision	Description	Role
ST 2.0.1	Register Change proposal	The change requester logs a change proposal with a high-level description of the proposed change.	Change Requester
ST 2.0.2	Records Change proposal details	The change owner records details of the change proposal. The change proposal should include the following items: • A high-level description of the new, changed or retired service, including business outcomes to be supported, and utility and warranty to be provided • A full business case including risks, issues and alternatives, as well as budget and financial expectations • An outline schedule for design and implementation of the change	Change Requester
ST 2.0.3	Assess and evaluate proposed Change	The change owner assesses and evaluates the change proposal, including risks, issues and alternatives, as well as budget and financial expectations.	Change Owner
ST 2.0.4	Abandon?	The change owner decides whether to abandon the proposal or not.	Change Owner
ST 2.0.5	Authorize Change proposal	CAB reviews the change proposal and the current change schedule, identifies any potential conflicts or issues and responds to the change proposal by either authorizing it or documenting the issues that need to be resolved.	CAB

ST 2.0.6	Initiate a change or project	After the new or changed service is chartered, RFCs will be used in the normal way to request authorization for specific changes. These RFCs will be associated with the change proposal so that change management has a view of the overall strategic intent and can prioritize and review these RFCs appropriately.	Change Manager

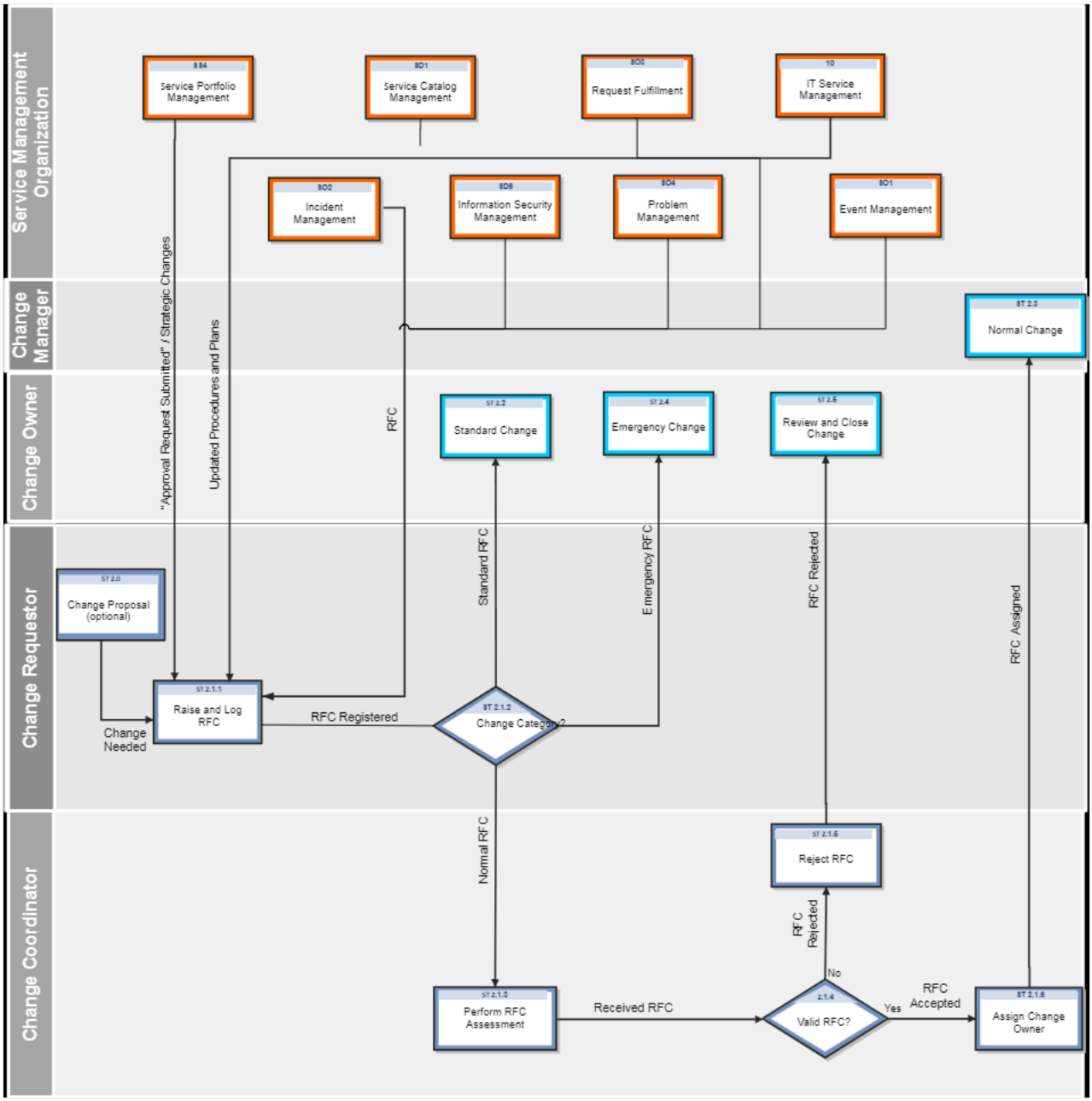
9.11.2. Register RFC (process ST 2.1)

An individual or organizational group that requires a change can initiate a Request for Change (RFC). Change requests can be initiated as part of a variety of management processes, including User Interaction Management, Incident Management, and Problem Management. Each RFC must be registered in an identifiable way. Service Manager provides change templates that standardize and speed up the Change Registration process. RFCs are received from requestors. In some cases, RFCs are logged on the requestor's behalf. For example, a business unit may require additional facilities. Another scenario may be that the Problem Management staff initiates solution for an error from several other sources. All RFCs are reviewed for completeness and accuracy. There may be additional information that must be entered into the RFC log prior to further processing.

The following user roles can perform Change Registration:

- Change Requestor
- Change Coordinator

Details for this process can be seen in the following figure and table.


Service Management Organization
Change Manager
Change Owner
Change Requestor
Change Coordinator
Service Portfolio Management
Service Catalog Management
Request Fulfillment
IT Service Management
Incident Management
Information Security Management
Problem Management
Event Management
Normal Change
Standard Change
Emergency Change
Review and Close Change
Change Proposal (optional)
Raise and Log RFC
Change Category?
Reject RFC
Perform RFC Assessment
Valid RFC?
Assign Change Owner
"Approval Request Submitted" / Strategic Changes
Updated Procedures and Plans
RFC
Standard RFC
Emergency RFC
RFC Rejected
RFC Assigned
Change Needed
RFC Registered
Normal RFC
RFC Rejected
No
Yes
Received RFC
RFC Accepted

Register RFC process

Process ID	Procedure or Decision	Description	Role
ST 2.1.1	Raise and Log RFC	All RFCs received are logged, provided with a unique identification number. Any other relevant information about the change request is also recorded. If the change request is in response to a trigger, i.e., a resolution to a Problem Record (PR), then the reference number of the triggering document is retained for traceability. The change is categorized (Standard, Normal, or Emergency) at the time of logging.	Change Requestor
ST 2.1.2	Change Category?	If this is a Standard Change, it will follow the Standard Change processes. If this is a Normal Change, it will follow the Normal Change processes. If this is an Emergency Change, it will follow the Emergency Change processes.	Change Requestor
ST 2.1.3	Perform RFC Assessment	The Change Coordinator receives the RFC and assesses it to determine whether it is valid. The RFC is rejected if: • The RFC is impractical • The RFC is a duplicate **Note** Note The activities of logging and categorizing an RFC may be delegated to a Change Coordinator if desired.	Change Coordinator
ST 2.1.4	Valid RFC?	The Change Coordinator determines whether the RFC is valid.	Change Coordinator

ST 2.1.5	Reject RFC	The Change Coordinator updates the RFC with an explanation of the rejection, and notifies the requestor.	Change Coordinator
ST 2.1.6	Assign Change Owner	The Change Coordinator looks for resources and assigns a Change Owner.	Change Coordinator

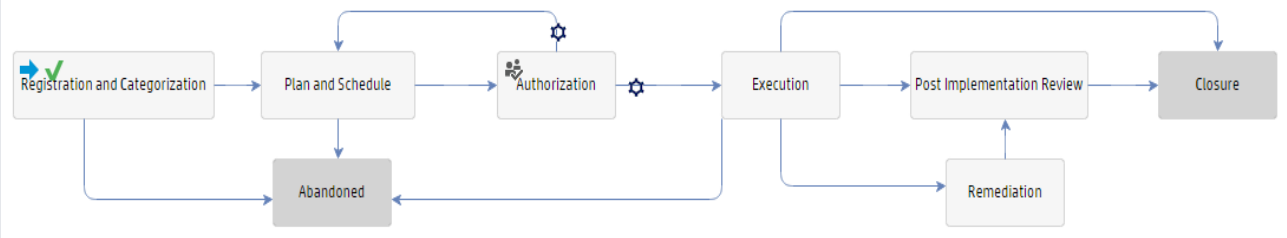
9.11.3. Standard Change (process ST 2.2)

A Standard Change is a pre-authorized change that follows a standard procedure; for example, a password reset or the provision of standard equipment to a new employee. Standard Change uses the Change Model configured in the system which can pre-populate the information in the Change ticket on registering the Change.

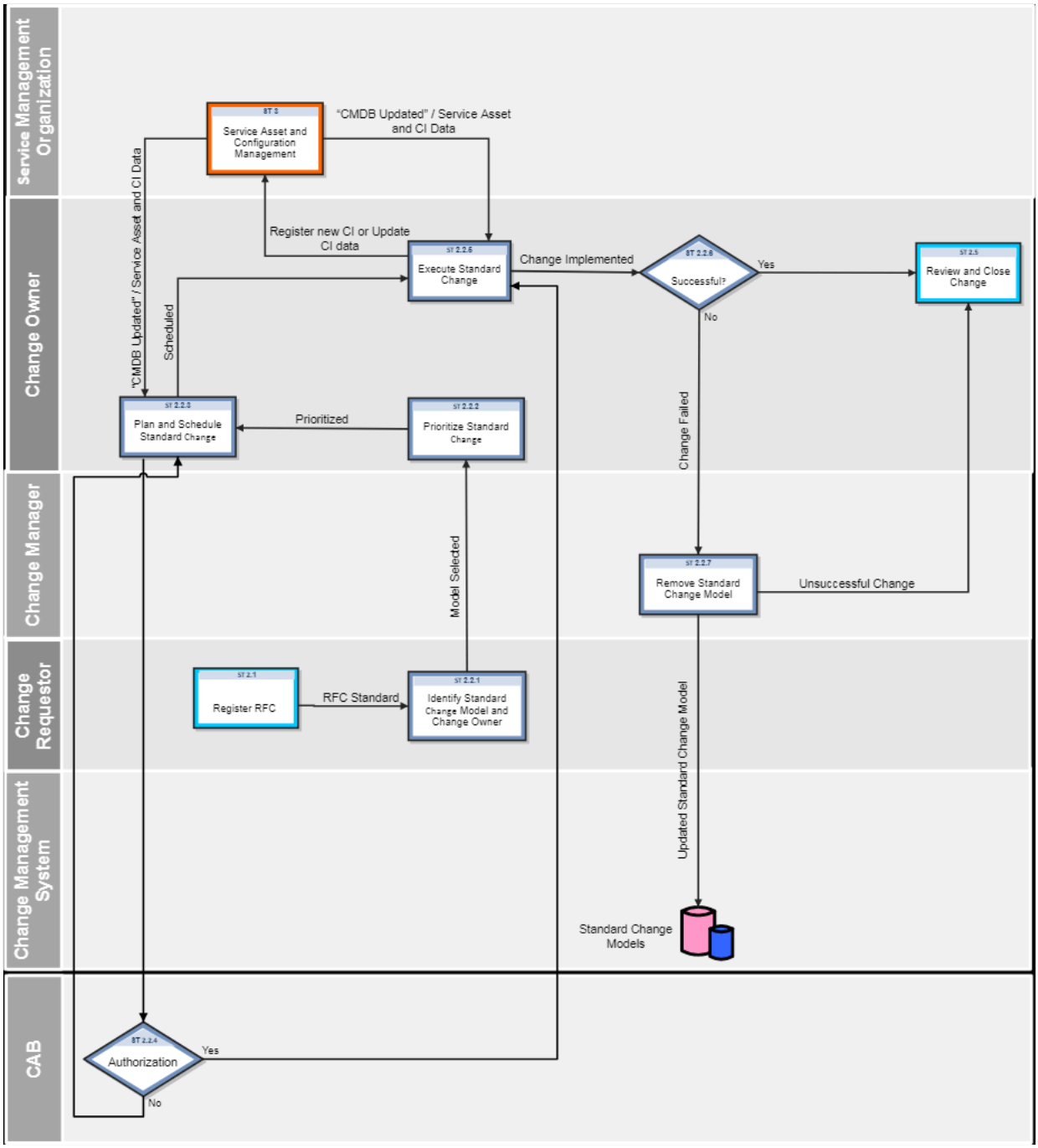
For an activity to be accepted as a Standard Change, the following requirements must be met:

- The documented tasks must be commonly known and proven
- Authority will be given in advance based on predetermined criteria
- The chain of events can be initiated by a functional Service Desk
- Budgetary approval will typically be predetermined or within the control of the Change Requester

The following figure depicts the Standard Change workflow in Process Designer.



For details of the Standard Change process, see the following figure and table.


Service Management Organization
Change Owner
Change Manager
Change Requestor
Change Management System
CAB
ST 0
Service Asset and Configuration Management
"CMDB Updated" / Service Asset and CI Data
Register new CI or Update CI data
"CMDB Updated" / Service Asset and CI Data
ST 2.2.6
Execute Standard Change
Change Implemented
BT 2.2.8
Successful?
Yes
No
ST 2.5
Review and Close Change
Scheduled
ST 2.2.3
Plan and Schedule Standard Change
Prioritized
ST 2.2.2
Prioritize Standard Change
Change Failed
Model Selected
ST 2.2.7
Remove Standard Change Model
Unsuccessful Change
ST 2.1
Register RFC
RFC Standard
ST 2.2.1
Identify Standard Change Model and Change Owner
Updated Standard Change Model
Standard Change Models
BT 2.2.4
Authorization
Yes
No

Standard Change process

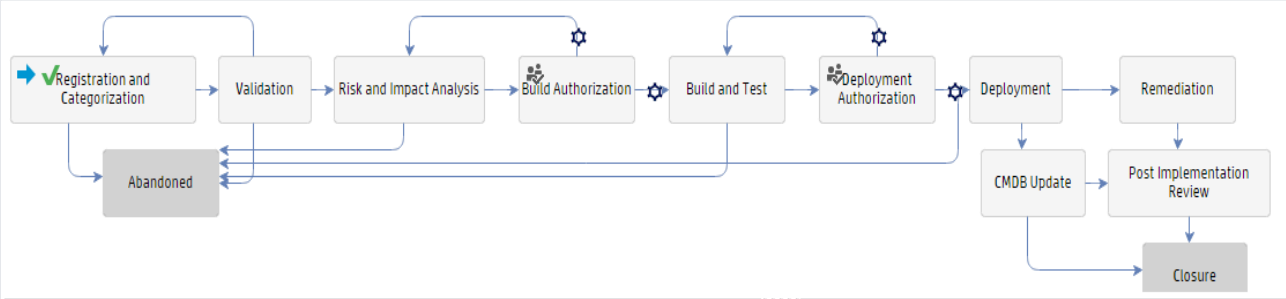
Process ID	Procedure or Decision	Description	Role
ST 2.1	Register RFC	The change requester logs an RFC and reviews the RFCs for completeness and accuracy.	Change Requester
ST 2.2.1	Identify Standard Change Model and Change Owner	The Standard change model that has to be requested is first identified and then the Change Owner is decided. In many cases the change owner is documented with the change model. If this is not the case, a Change Owner is decided manually.	Change Requester
ST 2.2.2	Prioritize Standard Change	Standard change is prioritized as High, Medium, or Low depending on the impact and urgency.	Change Owner
ST 2.2.3	Plan and Schedule Standard Change	The Change Owner plans the resources required, and also reviews revises and updates the schedule for Standard Changes as needed.	Change Owner
ST 2.2.4	Authorize Plan and Schedule	The Change Advisory Board (CAB) reviews and authorizes the resource plan and change schedule. If not authorized, the change returns to process ST 2.2.3.	CAB
ST 2.2.5	Execute Standard Change	The Change Owner executes the Standard Change. The CMDB is updated according to the changes performed.	Change Owner
ST 2.2.6	Successful?	The Change Owner checks whether the Change is successful.	Change Owner
ST 2.2.7	Remove Standard Change Model	If a Change Implementation is unsuccessful, the Standard Change Model may need to be slightly modified or removed from the database.	Change Manager
ST 2.5	Review and Close Change	The Change Owner reviews the change implementation and closes it if successful.	Change Owner

9.11.4. Normal Change (process ST 2.3)

Normal Change is a change that's categorized, prioritized, planned and that follows all approvals before deployment. The Normal Change activities describe the steps necessary to process a Normal Change by coordinating work effort. A Normal Change can be further categorized as Major, Significant, and Minor. All Major, Significant, and Minor changes go through the same workflow except that, for a Normal Major RFC or a Normal Significant RFC the Build Authorization phase is a manual transition and the change has to be approved

manually by Change Advisory Board (CAB) members. In a Normal Minor RFC the Build Authorization phase is auto approved. The Build Authorization phase is optional in a Minor RFC but it is still configured in the out-of-box system. The Change must be appropriately reviewed, approved, and executed successfully through the Normal Change process at least once prior to acceptance.

The following figure depicts the Normal Change workflow in Process Designer.



For details of the Normal Change process, see the following figure and table.

Error creating thumbnail: File missing

Normal Change process

Process ID	Procedure or Decision	Description	Role
ST 2.1	Register RFC	RFCs are logged by the Change Requestors. All RFCs are reviewed for completeness and accuracy.	Change Coordinator
ST 2.3.1	Assess Change	Assessing a change includes determining the risk and impact and identifying if the change requires a release to be generated. If a release is required, a Release and Deployment Manager is identified.	Change Owner
ST 2.3.2	Is a Release Required?	The Change Owner determines whether a release is required to implement the change.	Change Owner
ST 2.3.3	Determine Approval Requirements	The Change Manager reviews the change and decides whether a TCAB needs to be convened.	Change Manager
ST 2.3.4	Build Authorization Needed?	If Build Authorization is not needed, the Change Manager determines whether the change should be approved or rejected. If Build Authorization is needed, members are identified and change is submitted for approval.	Change Manager
ST 2.3.5	Build Authorization	Change Manager works with CAB and approves the Change.	CAB
ST 2.3.6	Review RFC	The Change Owner reviews the RFC for approvals.	Change Owner
ST 2.3.7	Approval Received?	Upon approvals, build and test is performed either by Release and Deployment process or Change Management process. If unapproved, the RFC is closed.	Change Owner

ST 2.3.8	Coordinate Build and Test	The Change Owner initiates the Release and Deployment Management process to perform build and test. The Service Validation Testing process is involved in validating and testing the build created by release. In case the build and test is unsuccessful, it is recommended to abandon the change to RDM which in turn is informed to the Change Owner. The Change Owner coordinates between the experts within Change process for building and testing activities. The change is built and tested thoroughly and validated.	Change Owner
ST 2.3.9	Build and Test Result Acceptable?	If the Build and Test result is acceptable, deployment is scheduled; else the RFC is sent for closure.	Change Owner
ST 2.3.10	Schedule for Normal Change	Change Owner involves the RDM and prepares the schedule for normal change. This schedule is based on the draft deployment plan obtained from the Release and Deployment Management process received.	Change Owner
ST 2.3.11	Release Related?	If release is related, the detailed deployment plan is created by the RDM process; else, Change Owner creates the same.	Change Owner
ST 2.3.12	Create and Submit Deployment Plan	The Change Owner creates the Deployment plan and submits it to CAB approval. The same is also received from RDM and is sent for CAB's review.	Change Owner
ST 2.3.13	Deployment Authorization	CAB reviews the Build and Test results and approves / abandons / rejects as deemed fit. If the Build and Test results are approved, CAB reviews and updates the implementation schedule and authorizes the Normal Change implementation to commence.	CAB

ST 2.3.14	Rejected?	If the deployment plan is rejected, the change moves back to Build and Test.	CAB
ST 2.3.15	Decision on Rebuild	If CAB decides on rebuilding, the change is rebuilt and tested again.	CAB
ST 2.3.16	Rebuild?	If a rebuild is required, the Change Owner is informed; else, the change is deployed.	CAB
ST 2.3.17	Review RFC after Rebuild Decision	The Change Owner reviews the RFC to check whether the rebuild will be performed by RDM or within change. Also a decision is made on what modules to be rebuilt and the timeline.	Change Owner
ST 2.3.18	Release Related?	If the rebuild is related to RDM, the request is sent to RDM for rebuilding and testing; if not, these are performed within the Change Management process.	Change Owner
ST 2.3.19	Review RFC after Deployment Authorization	The Change Owner reviews the RFC to go ahead with the deployment.	Change Owner
ST 2.3.20	Release Related?	If RDM is involved for deployment, RDM carries out the deployment activities; else it is carried out by the Change Management.	Change Owner
ST 2.3.21	Coordinate and Monitor Change Implementation	The Change Owner coordinates and monitors the Normal Change implementation. Various tasks are created for carrying out the deployment activities as mentioned in the deployment plan. After the deployment, the Change owner verifies whether the deployment was successful or not.	Change Owner
ST 2.3.22	Remediation Required?	If the change is unsuccessful it is backed out and moved to closure; else CMDB is updated for successful changes.	Change Owner
ST 2.3.23	Provide CMDB Updates	Change Owner generates CMDB updates that are to be submitted to Service Asset and Configuration Management for processing.	Change Owner

ST 2.3.24	Determine if implemented RFC Is a Release	If the implemented change involves a release, control is returned to Release and Deployment Management for further review and closure.	Change Owner
ST 2.3.25	Implemented RFC for a Release?	If the RFC was for a release, the post-deployment activities will take place in the Release and Deployment Management process; else move on to PIR.	Change Owner
ST 2.3.26	Assess Change Success	The Change Manager determines whether the change implementation is successful.	Change Manager
ST 2.3.27	Remediation Required?	If a release needs to be backed out, this is done in the Release and Deployment Management process.	Change Manager

9.11.5. Emergency Change (process ST 2.4)

Emergency changes can also be initiated in the Incident Management process. They should be used only to repair an IT service error that's negatively impacting the business at a high level of severity. Changes that are intended to make an immediately required business improvement are handled as normal changes, although they may be assigned a high priority based on the urgency of the required business improvement.

The emergency change process follows the normal change process, except for the following:

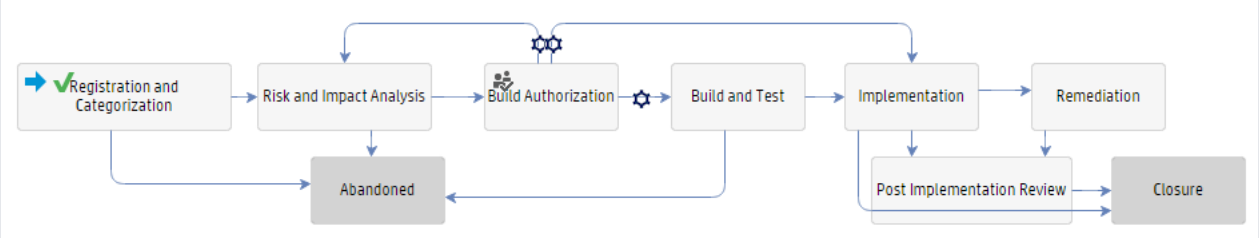
- Approval is given by the Emergency Change Approval Board (E-CAB) instead of waiting for a regular CAB meeting.
- Testing may be reduced, or in extreme cases eliminated, if doing so is considered necessary to deliver the change immediately.
- Updating of the change request and configuration data may be deferred, till normal working hours.

If the E-CAB decides to handle an emergency change as a normal change, the emergency change is recategorized and implemented using the normal change process.

The following user roles are involved in Emergency Change Handling:

- Change Manager
- E-CAB
- Change Owner

The following figure depicts the Emergency Change workflow in Process Designer.



For details of the Emergency Change process, see the following figure and table.

SMA 2018.02 Helpcenter BestPracticesGuide PD NormalChange Workflow.png

Emergency Change process

Process ID	Procedure or Decision	Description	Role
ST 2.1	Register RFC	Emergency Change activities provide the expedited Change Management process, which implements changes to the production environment due to an emergency occurring by a service outage. RFCs are logged by the Change Requestors.	Change Requestor
ST 2.4.1	Assign a Change Owner	The Change Coordinator assigns the Emergency change to the Change Owner.	Change Coordinator
ST 2.4.2	Assess the RFC	The Change Owner receives the RFC and assesses it to determine whether it is valid. It is recategorized as Normal if it doesn't qualify as Emergency Change. Risk and Impact Analysis is done at this stage.	Change Owner
ST 2.4.3	Convene ECAB	The Change Owner convenes Emergency Change Advisory Board (ECAB) members to authorize the change. The E-CAB members are authorized to make decisions about high impact emergency changes.	Change Owner
ST 2.4.4	Approve Emergency Change	ECAB reviews the Emergency Change and assesses its urgency, impact, and risk. Based on their review the ECAB members either approve or deny the change.	Emergency CAB (ECAB)
ST 2.4.5	Emergency Change Approved?	If ECAB approves, the change is implemented. Else the change is denied by the Change Approver and re-categorized as a Normal Change.	Emergency CAB (ECAB)
ST 2.4.6	Plan and Design Solution	The Change Owner plans the resources required, designs the solution to implement Emergency Change.	Change Owner

ST 2.4.7	Build and Test Required?	If Build and Test is required, Change Owner performs the building and testing activities by creating change tasks at this phase of the change. If build and test is not required the change is scheduled for implementation.	Change Owner
ST 2.4.8	Coordinate Emergency Change Build and Test	The Change Owner coordinates the Build and Test activities for Emergency Change.	Change Owner
ST 2.4.9	Schedule Emergency Change	The Change Owner reviews, revises and updates the schedule as needed for Emergency Change.	Change Owner
ST 2.4.10	Coordinate Emergency Change Implementation	The Change Owner coordinates the activities for implementing the Emergency Change. Successful change implementation is followed by update of the CMDB and formal turnover to support.	Change Owner
ST 2.4.11	Abandon the Change	If an Emergency change is denied by the ECAB, the RFC is abandoned. Go to 2.5 to close the change.	Change Owner
ST 2.5	Review and Close Change	The Change Owner reviews the change implemented and closes if it is successful. Unsuccessful change is backed out and the emergency RFC is closed after PIR (Post Implementation Review). Change tasks are created to roll back the change and to bring the environment to agreed stable state.	Change Owner

9.11.6. Review and Close Change (process ST 2.5)

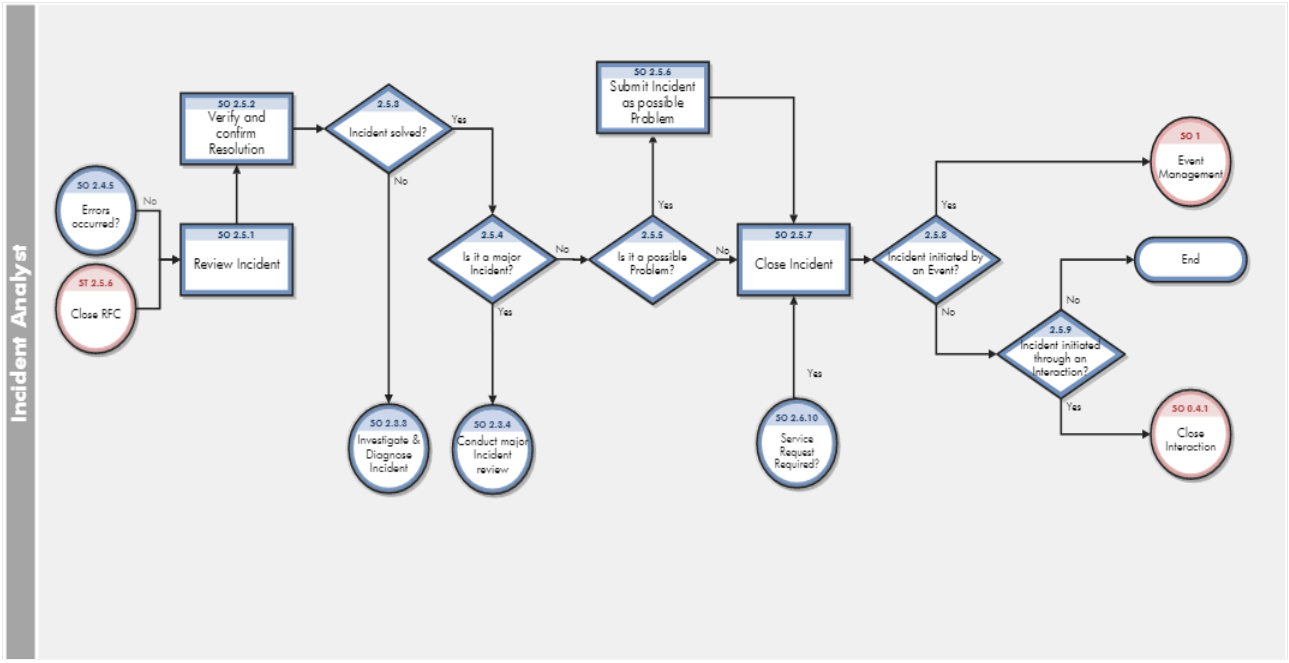
After a change is completed, the results must be reported for evaluation to those responsible for managing changes, and then presented for stakeholder agreement. This process includes the closing of related user interactions, incidents, and known errors. The Change Owner and Change Manager review the change implementation and close it if successful.

Post-implementation review of the change (or PIR) is performed to confirm the following:

- The change meets its objectives
- The change Requester and stakeholders are satisfied with the results
- Unanticipated effects have been avoided.
- Lessons learned are incorporated in future changes.

By default, an Emergency Change or a Normal Change requires PIR, while a Standard Change doesn't.

Details for this process can be seen in the following figure and table.


Incident Analyst
SO 2.4.5
Errors occurred?
ST 2.5.6
Close RFC
No
SO 2.5.1
Review Incident
SO 2.5.2
Verify and confirm Resolution
2.5.3
Incident solved?
Yes
No
SO 2.3.3
Investigate & Diagnose Incident
2.5.4
Is it a major Incident?
No
Yes
SO 2.3.4
Conduct major Incident review
SO 2.5.6
Submit Incident as possible Problem
Yes
2.5.5
Is it a possible Problem?
No
SO 2.5.7
Close Incident
Yes
SO 2.6.10
Service Request Required?
2.5.8
Incident initiated by an Event?
Yes
No
SO 1
Event Management
End
2.5.9
Incident initiated through an Interaction?
No
Yes
SO 0.4.1
Close Interaction

Review and Close Change process

Process ID	Procedure or Decision	Description	Role
ST 2.5.1	Review RFC for Success	The Change Owner checks whether the RFC deployed by Release and Deployment Management process is successful.	Change Owner
ST 2.5.2	RFC Successful?	If the release is successful, it is handed over to the support staff. If the release is unsuccessful, it is subject to a Post-Implementation Review (PIR).	Change Owner
ST 2.5.3	Conduct Formal Turnover to Support	The Change Owner ensures that all the functionality changes are documented and performs the tasks required to hand over the change to the support staff.	Change Owner
ST 2.5.4	Perform PIR	The Change Owner and Change Manager perform a Post-Implementation Review (PIR) to ensure that the change is reviewed and lessons learned from the change are implemented.	Change Owner
ST 2.5.5	Notify Requestor of Change Results	The Change Requestor is notified of the success or failure of the change.	Change Owner
ST 2.5.6	Reject RFC	The Change Coordinator rejects the RFC.	Change Coordinator
ST 2.5.7	Close RFC	The Change Requestor is notified of the success or failure of the change.	Change Owner

9.12. Change Management details

Service Manager uses the Change Management application to enable the Change Management process. The main function of Change Management is to standardize the methods and processes a business organization uses to plan and implement changes. Change Management records all changes to service assets and configuration items in the Configuration Management System (CMS).

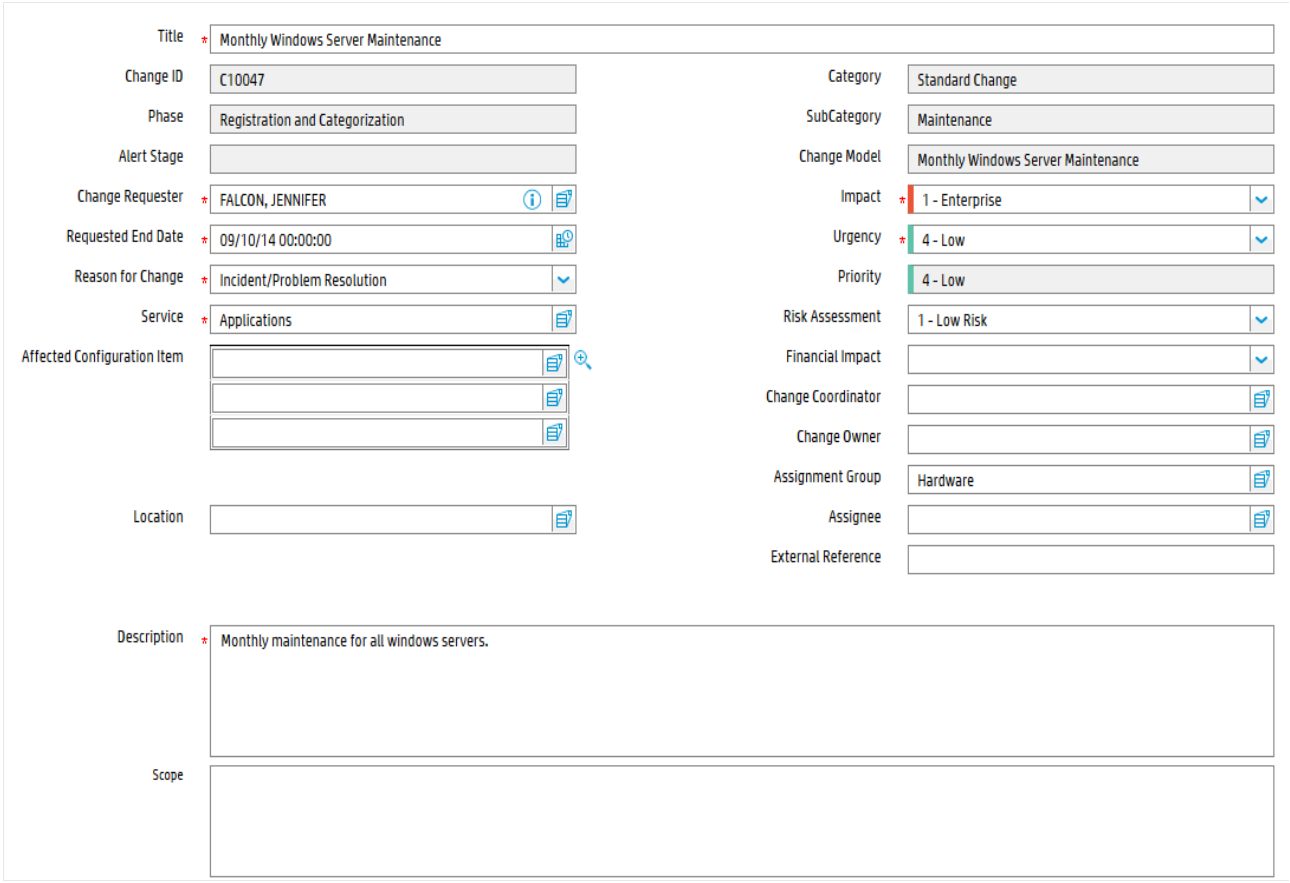
In Change Management, the Change Manager sends the change requests to the appropriate approvers and coordinates Emergency Change handling. The Change Approver and approves or denies the change request. The Change Coordinator plans the change implementation and verifies if the change has been completed satisfactorily, and the Change Analyst implements the change.

This section describes selected Change Management fields in the out-of-box Service Manager system. Topics in this section include:

- Change Management form after escalation from a problem
- Change Management form details

9.12.1. Change Management form after escalation from a problem

The following figure shows an example new change request that's created from a problem record in Problem Management. As with any new change, you must provide the required fields before you can save it. See Change Management form details for a list and description of the fields on this form.



9.12.2. Change Management form details

The following table identifies and describes some of the features on the Change Management forms.

Change Management field descriptions

Label	Description
Change ID	This is a system-generated field assigned when the change is opened.
Change Model	A change model is a record that's used to predefine the contents of a specific type of Request for Change (RFC), including the information used to populate the RFC and the tasks that are needed to complete the change. When you open a change request using a change model, most of the necessary information is added to the change automatically.
Phase	This is a system-generated field that specifies the name of the current phase of the change.
Approval Status	This is a system-generated field that defines the global approval status for the change, not for a single approval. The system sets this field depending on current approvals and the approval type defined for the module. These approval statuses are available out-of-box: • Pending • Approved • Denied
Change Requester	The name of the user requesting the change. This is a required field. This field includes a hover-over form that displays full name, telephone, and email address if available for the user requesting the change.
Assignment Group	The group assigned to work on the change. For a description of this field see the **Assignment Group** field description in the Incident Management form details section in *Service Manager Processes and Best Practices* as this field functions similarly. The out-of-box data consists of default assignment groups for use as examples of types of assignment groups. You may want to change the sample assignment groups to meet your own needs. These assignment groups are available out-of-box: • Application • Email / Webmail • Field Support • Hardware • Intranet / Internet Support • Network • Office Supplies • Office Support • Operating System Support • SAP Support • Service Desk • Service Manager

Change Coordinator	The person responsible for coordinating the change implementation. Each Change Coordinator may belong to several assignment groups. Each group must have only one Change Coordinator.
Service	Specifies the service affected by the change. This is a system-generated field and is prepopulated when a change request is created from an interaction. This is a required field.
Affected CI	The list of Configuration Items (CIs) affected by the change. The system prepopulates this field when a change request is created from an incident or problem. Users can add additional CIs. This field includes a hover-over form that displays checkboxes for Critical CI and Pending Change.
Location	Specifies the location for the change. The system prepopulates this field when the change is created by escalating an interaction.
Title	Provides a short description or a gist of the change. This is a required field.
Description	Provides a detailed description of the change. This is a required field.
Scope	Provides a detailed description of the change scope.
Category	This is a system-generated field that classifies the type of change.
Impact	This field is prepopulated with data from an incident when a change is created from an incident. It specifies the impact the problem has on the business. The impact and the urgency are used to calculate the priority. These impacts are available out-of-box: • 1 - Enterprise • 2 - Site/Dept • 3 - Multiple Users • 4 - User The out-of-box data is the same as Interaction Management, Problem Management, and Incident Management. This is a required field.
Urgency	The urgency indicates how pressing the change is for the organization. The urgency and the impact are used to calculate the priority. This field functions similarly to the same field for interaction, incident, and problem tickets. For more information, see User Interaction Management form details in *Service Manager Processes and Best Practices*. This is a required field.
Priority	This is a system-generated field using the urgency and impact of the change. This field functions similarly to the same field for interaction, incident, and problem tickets. For additional information, see User Interaction Management form details in *Service Manager Processes Processes and Best Practices*.

Risk Assessment	Specifies a code that indicates the risk incurred with the implementation of the change. This field becomes required in the Change Plan and Schedule phase. These risk assessments are available out-of-box: • 0 - No Risk • 1 - Low Risk • 2 - Some Risk • 3 - Moderate Risk • 4 - High Risk • 5 - Very High Risk After a user selects this field, the change may require additional approvals based on the risk. The approval is based on the risk number in the assessment approval record. This is a required field. **Note** Note This field becomes read-only if you have selected the **Enable Risk Assessment** checkbox. You need to click the **Calculate Risk** button to calculate the risk.
Financial Impact	Specifies a code that indicates the financial impact of the change. These financial impact codes are available out-of-box: • 1 - High • 2 - Medium • 3 - Low
Review Required	This option specifies whether post-implementation review is required or not. By default, it is set to true for Emergency and Normal changes in the Risk and Impact Analysis phase, while false for Standard changes in the Plan and Schedule phase. This field can't be changed in subsequent phases.
Requested End Date	The system prepopulates this field if the change request is triggered from an interaction. This is the date the change initiator requests the change implementation. This is a required field if not prepopulated.
Alert Stage	This is a system-generated field that lists the current Alert Stage of this request. Change Management updates this field automatically when processing alerts against this change. Do not update it manually. The alerts are processed against a change by using the phase definition. This field is not active in an out-of-box system and must be manually enabled.
Scheduled Implementation Start	This field specifies the date and time that the work to implement the change should start. This field becomes required in the Plan and Schedule phase.
Scheduled Implementation End	This field specifies the date and time that the work to implement the change should end. This field becomes required in the Plan and Schedule phase.

Scheduled Downtime Start	The date and time when the change is scheduled to begin. Scheduled downtime only needs to be filled when the service is down, while implementing the change.
Scheduled Downtime End	The date and time when the change is scheduled to end. Scheduled downtime only needs to be filled when the service is down, while implementing the change.
Configuration Item(s) Down	If selected (set to true), indicates that the Configuration Items (CIs) are currently not operational and the downtime is scheduled. The fields Scheduled Downtime Start and Scheduled Downtime End are used along with the field Configuration Item(s) Down to indicate the scheduled time to bring the CI down. These fields are never required and should only be populated if you plan to bring down the CIs as part of the change. The interval selected applies to all the CIs of the change and can't be specified by individual CI. When the change is closed, you may get the form confirming the outage times, and when you close the change, the CIs will be set as Up in Configuration Management.
Ex. Project Ref.	This field references an external project number.
Implementation Plan	An assessment of the change, often generated by the Change Implementer, that the Change Coordinator uses to assess the impact of the change to services.
Effect of not Implementing	The impact if not implementing the change. This is a required field.
Change Owner	The name of the user owning the change. This is a required field.
Subcategory	The subcategory is a breakdown of the category and describes the type of change in more detail.
Actual Implementation Start	The time when the implementation actually began.
Actual Implementation End	The time when the implementation actually ended.
Review Results	Results of the review after Post Implementation Review, this is a required field.

Closure Code	The completion code indicates the way a change is closed. VALID VALUES • 1 - Successful Indicates the Change is completed successfully without any issues. • 2 - Successful (with problems) Indicates the Change is closed but with some issues. • 3 - Failed Indicates the Change implementation is failed. The Change is closed with a reason for the failure. • 4 - Rejected Indicates the Change is rejected by a Change Coordinator or Change Owner. • 5 - Withdrawn Indicates the Change is withdrawn by the Change Requester. • 6 - Cancelled Indicates the Change is cancelled by a Change Coordinator or Change Owner.
Associated CIs section > CMDB attributes need to be changed for CIs in the list Completed/Cancelled CMDB attributes modifications CMDB relationships need to be changed for CIs in the list Completed/Cancelled CMDB relationships modifications	The data in this section is used by the UCMDB integration whenever there are past changes to the values registered for the CI.
Affected Services section > Affected Services	This provides a list of affected services. When a configuration item for an incident is added or updated, a schedule record is created that runs a routine to update the list of affected services.
Approvals section> Current Approvals	This section provides an overview of the current approvals related to any changes for the CI, and important information such as approval status, and approvers as well. This includes a list of groups or operators who must acknowledge or accept the risk, cost, and so on associated with the implementation of a Change request or task. Approvals give controlling authorities the ability to stop work and to control when certain work activities can proceed. The data displayed includes the following information: • Approval Type • Approval Status • # Approved • # Denied • # Pending

Approvals section> Approval Log	This subsection provides an overview of past approvals related to the changes for the CI, as well as important information such as approval status and approvers. The data displayed includes the following information: • Action • Approver/Operator • By • Date/Time • Phase
Reason for Change	A code that indicates the primary reason for implementing the request. Examples of reason codes are Incident/Problem Resolution and Business Requirement.
Approval section > Pending Reviews	The name(s) of the groups or operator IDs that should review the change for the CI after it has been approved.
Cost section > Total Cost	This field indicates the system-calculated total cost of the Change, which is accumulated by the parts and labor cost of the Change itself and its related Change tasks.
Cost section > Currency	Specifies a currency code used by the system to calculate the total cost.
Cost section > Parts table	Specifies the parts used by the Change with their part numbers from the product catalog and the quantity used.
Cost section > Labor table	Specifies the technician who worked on the Change and the actual working hours.
Tasks	Whenever a change is in a phase where the user can generate tasks, Service Manager allows user a quick view of some of the most important fields in the task in the **Tasks** section. The data displayed includes the following information: • Task No • Phase • Status • Description • Category
Remediation Method	Indicates the remediation method: Full, or Partial.
Remediation Comments	Provides a detailed method for backing out the change if there is a problem implementing the change. This is a required entry for all changes while backing out a change. It is also required in the "Release back out" phase and for the Release Management category in order to close the "Release plan and design" phase.

9.13. Knowledge Management overview

The Service Manager Knowledge Management application, referred to as Knowledge Management throughout this chapter, supports the Knowledge Management process.

Knowledge Management provides the framework to help you manage information throughout your IT Service Management life cycle. The primary purpose of Knowledge Management is to improve efficiency by reducing the need to rediscover knowledge.

Knowledge Management processes can interact with other Service Manager processes (in particular, Interaction Management, Incident management, and Problem management). The scope of this document is limited to the Knowledge Library and its contents

Topics in this section include:

- Knowledge Management Within The ITIL Framework
- Knowledge Management Application
- Knowledge Management Process Overview
- Key Interfaces with Other Processes
- Key performance indicators for Knowledge Management
- RACI Matrix For Knowledge Management

9.13.1. Knowledge Management Within The ITIL Framework

Knowledge Management (KM) was added to ITIL v3 as part of Service Transition – the ITIL process that addresses the development and deployment of new or changed services. Prior to ITIL v3, the Incident and Problem Management processes were responsible for addressing the management of knowledge. But with the development of a specific Knowledge Management process, ITIL v3 now provides a detailed set of guidelines and workflows for the management of all knowledge in the Service Management life cycle. ITIL v3 also calls for the integration of the Knowledge Management process with Service Desk (Interaction Management), Incident Management, and Problem Management.

The goal of Knowledge Management is to enable your organization to efficiently access, update, and share all knowledge that pertains to the Service Management life cycle. Benefits of Knowledge Management include, but aren't limited to, the following:

- more efficient handling of knowledge
- the reduced likelihood that multiple stakeholders will attempt to solve the same problem in isolation without first sharing knowledge
- the ability to control the access of sensitive information to certain people

Central to Knowledge Management is a repository known as the Service Knowledge Management System (SKMS). A typical SKMS stores all knowledge in a Knowledge Base. The items in the Knowledge Base are referred to as Knowledge Documents, which can also include a variety of attachment types such as text files and graphics files. It also provides tools for handling important tasks such as generating reports.

Service Manager not only acts as the SKMS, but it also goes one step further by providing Knowledge Center Support (KCS) methodologies out-of-the-box. KCS methodologies specify ways to capture information from incidents to be used by analysts in problem-solving. It also specifies that knowledge must by definition evolve over time as additional stakeholders review and modify knowledge content.

Processes and Best Practices describes Knowledge Management as the process responsible for providing knowledge to all other IT Service Management processes. Because the number of processes involved is very large, the scope of this document is restricted to those processes related to storing new KM documents and updating, archiving, and retiring existing KM documents.

9.13.2. Knowledge Management Application

The Service Manager Knowledge Management application helps you manage all knowledge related to Interactions, Incidents, and Problems.

9.13.2.1. Types of Knowledge Documents in Service Manager

Document types are templates for documents in Knowledge Management. Knowledge Management includes several default document types:

- Question/Answer
- Problem-Solution
- Reference
- Error Message/Cause
- External

9.13.2.2. Knowledge Management Categories

All documents in the knowledge documents knowledge base are assigned to a document category. Categories for knowledge documents are ordered into hierarchies. The document categories order documents into top-level categories and subcategories. Each

subcategory has a parent category that's the category immediately above it in the ordered list of categories. A top-level category is also a parent category of the subcategory immediately below it in the ordered list of categories.

9.13.2.3. Knowledge Groups

Knowledge groups enable you to collect users into groups that have access to work with the same set of documents. A knowledge group might be the user for a department, or a group of document authors, or a special group of users within your organization such as subject matter experts.

You associate a knowledge group with at least one document category or subcategory to give the members of a knowledge group access to documents in the category or subcategory and all subcategories below it. Access privileges can range from search and view to publishing privileges depending upon what knowledge profile(s) you assign to the category.

9.13.2.4. Knowledge Management Profiles

The Knowledge Management profiles control access to knowledge documents as well as rights for creating, editing, and administering documents.

Within each security profile, there is a direct association between a defined (named) profile and the categories and sub-categories for which the selected capabilities apply. Each security profile maps to a document category or sub-category. A user with that profile has access to the documents specified by the profile-to-category mapping and all subcategories in that branch of the category tree.

Profiles can be modified to select only some of the available capabilities for a profile, effectively creating sub-profiles. For example, some users may not be allowed to publish knowledge documents externally as well as internally.

The out-of-box KM security profiles are briefly described in the following table:

Contribute And Approve Knowledge/Change Request Process

KM Profile	Privileges
DEFAULT	With this profile, a user can search and view externally published knowledge documents for those documents in categories to which this profile has access. They can also submit feedback on these documents.
INTERNAL USER	With this profile, a user can search and view externally published knowledge documents for those documents in categories to which this profile has access. They can also search and view internally approved knowledge documents for those documents in categories to which this profile has access. They can also submit feedback on these documents.
KCS I	Author and contribute knowledge documents.
KCS EDITOR	Editor Author knowledge documents and edit working copy documents in the workflow for those categories to which this profile has access.
KCS II	Author knowledge documents, contribute knowledge documents, edit published documents without placing them in workflow (edit in place), and publish documents internally only. Also, view adaptive learning data when adaptive learning is enabled in the system.
KCS III	Author knowledge documents, contribute knowledge documents, edit published documents without placing them in workflow (edit in place), and publish documents internally and externally. Also, view adaptive learning data when adaptive learning is enabled in the system.
KM ADMIN	Author knowledge documents, contribute knowledge documents, edit published documents without placing them in workflow (edit in place), and publish documents internally and externally. Also, view adaptive learning data when adaptive learning is enabled in the system.

9.13.2.5. Generic OOB Knowledge Management Document Phases

KM documents pass through the following generic out-of-box phases:

1. Draft
2. Triage
3. Revise
4. Review
5. Conclude

Draft

A Knowledge Contributor creates a draft document by first selecting one of the following document types:

- **Error Message/Cause**: a specific error message a user encountered and what the user was doing when the error occurred
- **External**: files, such as text or graphic files, that can be associated with incidents and problems
- **Question/Answer**: a general question and its corresponding answer
- **Problem/Solution**: a specific problem, including its probable cause, and a corresponding solution
- **Reference**: general information
  Once a draft is created, the Knowledge Document is said to be a **Working copy**. Note that in the Service Manager Knowledge Management application, a working copy of a published document has an "R" appended to the document ID. A new document

submitted to be published is also considered a working copy while it is in workflow waiting to be published, but there is no "R" appended to the document ID.

Triage

After the contributor enters the relevant details and submits the draft document, it enters the Triage state. The purpose of the Triage state is to allow a KM analyst to review the submission and determine if it would make an appropriate document. The analyst can also perform minor modifications of the content to ensure stylistic quality. If the document is acceptable, the analyst submits it to a specific KM expert for revision.

Revise

The purpose of the Revise state is to give a subject matter expert the chance to add or modify content that would improve the document. After revising the document, the KM expert submits the document for review to the KM team.

Review

The purpose of the Review state is to give the KM team a chance to finally decide if the document is ready for publication. The KM team can send the document back for further revision or to accept the document.

Conclude

> **Note**
>
> Note This is the Publish phase in the out-of-box KM workflow.

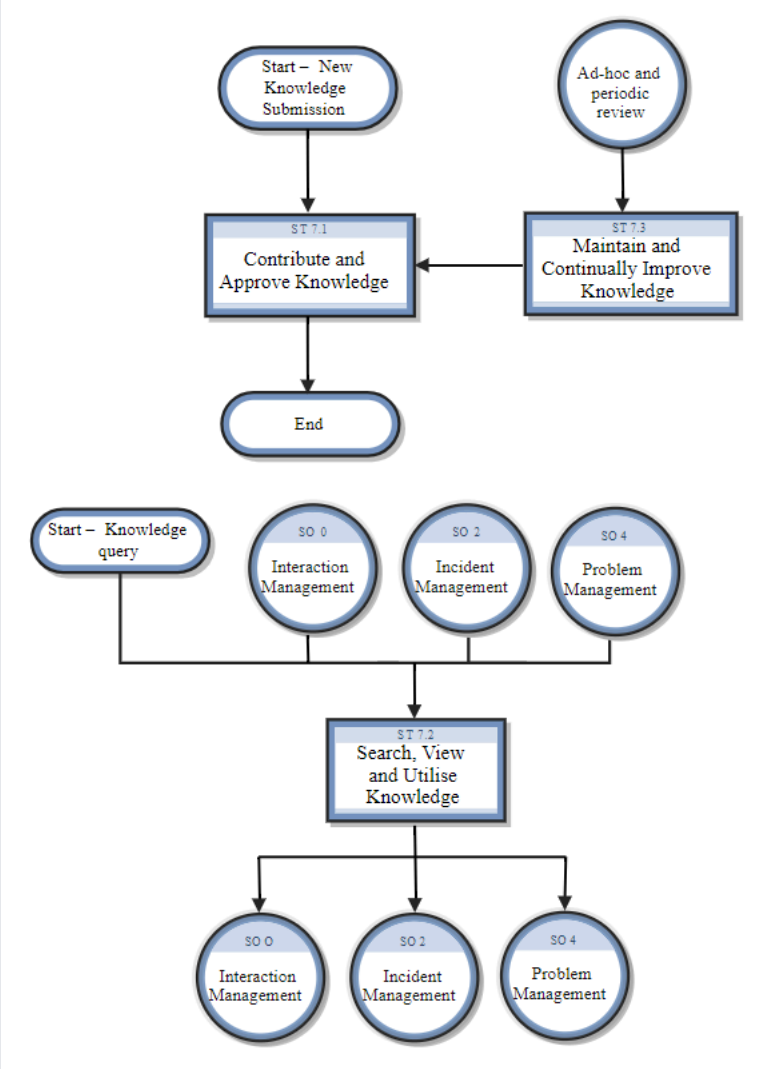
The purpose of the Conclude stage is to allow the assigned owner on the KM team to determine what to do with the document. The owner has the following options:

- If the document is new, the owner can approve and publish the submission, or terminate the KM record to prevent publication of the document.
- If the document already exists and was resubmitted for revision and review, the owner can publish the revised version, revert to the original version, or retire the document.

Documents can also be retired when they are no longer useful. A retired document is a knowledge document that's no longer searched during a knowledge base search.

9.13.3. Knowledge Management Process Overview

A general overview of the Knowledge Management processes and workflows is depicted in the following figure. Workflows are described in detail in Knowledge Management Workflows.



Knowledge Management User Roles

The following table describes the responsibilities of the Knowledge Management roles.

Knowledge Management User Roles

Role	Responsibilities
Knowledge Management Process Owner	• Accountable for the definition, management, governance and improvement of the KM Process • Ensures that the KM process and working practices are effective and efficient • Ensures that all stakeholders are sufficiently involved in the KM process • Ensures that (business) management is sufficiently informed as to the volume, impact and cost of Knowledge • Ensures tight linkage between the KM process and other related processes
Knowledge Manager	• Implementation and ongoing management of the Knowledge Management process • Championing the Knowledge Management process with people at all levels • Management of Knowledge Analysts • Ensuring process efficiency and consistency • Final QA and approval of all Knowledge Submissions • Deciding the scope of publishing (external and / or internal) • Continual improvement of the Knowledge Library and process • Fast tracking urgent 'Hot News' notices through the KM process • Reviewing KM reports • Identifying, allocating and tracking continual improvement actions and maintenance activities
Knowledge Expert	• Reviewing and amending Knowledge Submissions • Rejecting Knowledge Submissions based on technical content • Assisting with reviews of Knowledge Documents for currency and relevance
Knowledge Analyst	• Administering the KM process • Reviewing Knowledge Candidates for basic content, spelling, format, readability, duplication and editing to achieve a consistent quality level • Rejecting inappropriate or duplicated Knowledge Submissions • Co-ordinating Knowledge Expert reviews and reviewing Knowledge Document feedback and identifying improvements • Producing and distributing KM reports • Driving periodic reviews of Knowledge and co-ordinating and / or performing improvement actions • Retiring Knowledge Documents no longer deemed relevant

Knowledge Contributor	• Identifying Knowledge Submissions • Identifying changes required to existing Knowledge Documents • Submitting new or revised Knowledge Submissions • Assisting with reviews of Knowledge Documents for currency and relevance
Knowledge User	• Maintaining awareness of the KM facilities • Searching the Knowledge Base for solutions • Providing feedback on Knowledge Documents **Note** Note • Self-Service will only access the Knowledge Library • Operator record search will access Interaction, Incident, Known Error and Knowledge libraries

9.13.4. Key Interfaces with Other Processes

The following table shows how Knowledge Management is linked with other Service Manager processes.

Input and output for Knowledge Management

Process	Description
SO 0 – Interaction Management	Knowledge documents may be used in the resolution of Incidents, either via the Service Desk or self-service. Feedback on the knowledge documents will feed in to the Knowledge Management process.
SO 2 – Incident Management	During the Incident resolution process, KM documents can be submitted that specify how an analyst found a resolution or workaround.
SO 4 – Problem Management	Information on Known Errors and workarounds may be used to create Knowledge Documents.

9.13.5. Key performance indicators for Knowledge Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your Knowledge Management processes. To visualize trend information, it is useful to graph KPI data periodically. In addition to the data provided by Knowledge Management, you may need additional tools to report on all of your KPI requirements.

Key Performance Indicators for Knowledge Management

Title	Description
Number of KM documents created	The total number of KM documents in your organization will grow over time; however, older KM documents will gradually become outdated and should be retired. Monitor the number of newly created documents to determine the optimal rate of growth over a period of time.
Number of times a KM document is accessed	A useful document is accessed frequently. Use this indicator to figure out which documents are the most and least useful.
Number of KM documents used to resolve Interactions	KM documents should ultimately lead to resolutions. Monitor the number of KM documents that lead to resolutions of Interactions to help determine how successfully your Service Desk is able to access and reuse knowledge.
Number of KM documents used to resolve Incidents	Monitor the number of KM documents that lead to resolutions of Incidents to help determine how successfully KM documents help resolve serious issues that have been escalated to Incidents.
Number of KM documents with an expired review date	To ensure the efficiency of the document publication process, monitor the number of documents that have not been reviewed in a timely manner.

9.13.6. RACI Matrix For Knowledge Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. It is especially useful in clarifying roles and responsibilities in cross-functional/departmental projects and processes. The RACI matrix for Knowledge Management is shown in the following table.

RACI Matrix For Knowledge Management

Process ID	Activity	Knowledge Contributor	Knowledge Analyst	Knowledge Expert	Knowledge Manager	Knowledge User
ST 7.1	Contribute and Approve Knowledge	R	R	C	A/R	
ST 7.2	Search, View and Utilise Knowledge				A	R
ST 7.3	Maintain and Continually Improve Knowledge	R	R	R	A/R	

9.14. Knowledge Management workflows

The Knowledge Management process collects, organizes, structures, and distributes knowledge for ongoing use. It is responsible for ensuring that an organization can archive and retrieve knowledge in an efficient manner.

The Knowledge Management process consists of the following processes that are included in this chapter.

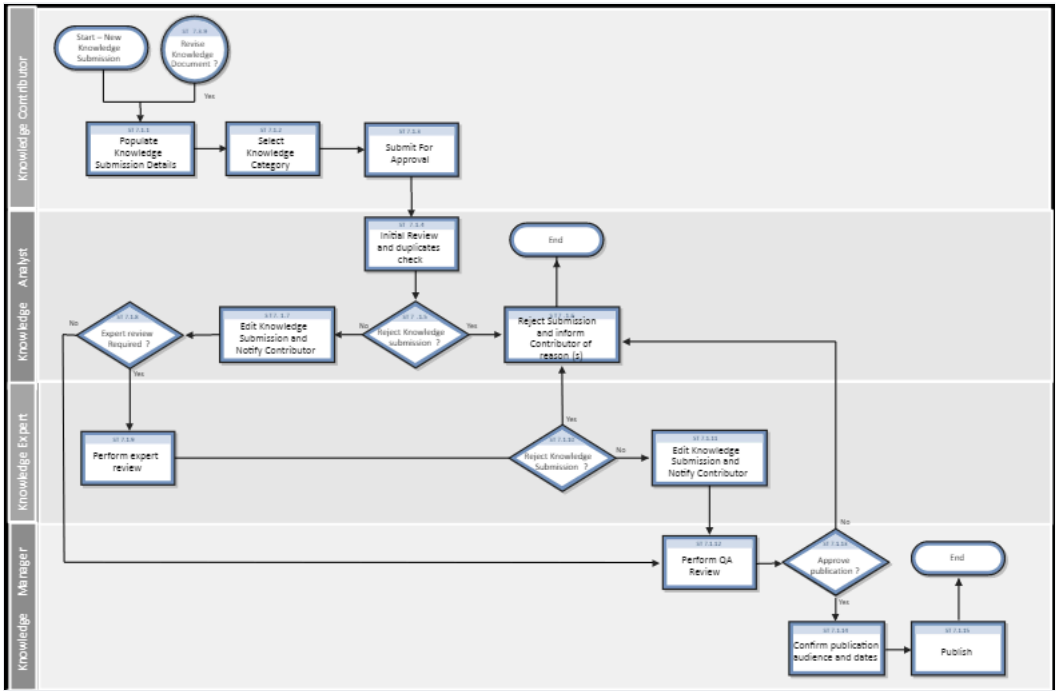
- Contribute and Approve Knowledge Document (process ST 7.1)
- Search, View and Utilize Knowledge Document (Process ST 7.2)
- Maintain and Continually Improve Knowledge (Process ST 7.3)

9.14.1. Contribute and Approve Knowledge Document (process ST 7.1)

The Contribute and Approve Knowledge Document process starts with the submission of a KM document. You can contribute a knowledge document by authoring knowledge articles or using external documents that are uploaded into a knowledge base. You can use the rich-text editor to author documents, and you can add documents as attachments that can include images, text files, Word files, or PDFs.

- The process begins when a contributor creates a new submission or revising an existing knowledge document. The contributor selects a documentation type, enters basic details, selects a category type, and submits the draft for review by a Knowledge Analyst.
- The analyst can edit, accept, or reject the document. If necessary, the document can be passed to a Knowledge Expert, who can also edit, accept, or reject it. The Knowledge Expert is typically a subject-matter expert with more in depth understand of a give topic and the ability to determine the accuracy and currency of the KM document.
- The Knowledge Manager makes the final decision about whether to accept the document or return it for further revision. If the document meets all KM publication criteria, the Knowledge Manager accepts and publishes the document.

The Contribute And Approve KM Doc/Change Request Workflow is illustrated in the following figure:



Contribute And Approve Knowledge/Change Request Process

Process ID	Procedure or Decision	Description	Role
ST 7.1.1	Populate Knowledge details	Enter details of the Knowledge Submission into Service Manager. Suggestions for new or improved KM documents can be received from a number of areas including Incident and Problem Management. Go to ST 7.1.2 to select the appropriate Knowledge category.	Knowledge Contributor
ST 7.1.2	Select Knowledge category	Select the appropriate category from those available within Service Manager. Go to ST 7.1.3 to submit the Knowledge Submission.	Knowledge Contributor
ST 7.1.3	Submit for approval	Submit the Knowledge Submission for approval by the Knowledge team. Go to ST 7.1.4 in order for the Knowledge Analyst to perform an initial review and check for duplicates.	Knowledge Contributor
ST 7.1.4	Initial review and duplicates check	The Knowledge Submission is reviewed to ensure all required information has been provided and in the correct format. A check is also done against other recent Knowledge Submissions to avoid duplication of effort. Go to ST 7.1.5 to determine whether to reject the Knowledge Submission.	Knowledge Analyst
ST 7.1.5	Reject Knowledge Submission?	The Knowledge Submission will be rejected if the Knowledge Analyst feels it's not valid or if it is a duplication of another existing proposal. If yes, go to ST 7.1.6 to inform the Knowledge Contributor of the reasons for rejection. If no, go to ST 7.1.7 to edit the Knowledge Submission.	Knowledge Analyst
ST 7.1.6	Retire Knowledge Submission and inform Contributor of reason(s)	Inform the Knowledge Contributor of the reasons for the Knowledge Submission being rejected. The 'Contribute and Approve Knowledge' process ends.	Knowledge Analyst

ST 7.1.7	Edit Knowledge Submission and notify Contributor	Add any comments / additional information to the Knowledge Submission ready for further review and approval, and notify the Knowledge Contributor. Go to ST 7.1.8 to determine whether an expert review is required.	Knowledge Analyst
ST 7.1.8	Expert Review Required?	Some Knowledge Submissions can be managed by the Knowledge team only. If the Knowledge Submission refers to a specialised area, an expert will be asked to review and approve it. If yes, go to ST 7.1.9 for the Knowledge Expert to review the Knowledge Submission. If no, go to ST 7.1.12 for the Knowledge Manager to perform a QA review.	Knowledge Analyst
ST 7.1.9	Perform Expert review	Review the Knowledge Submission. Go to ST 7.1.10 to determine whether to reject the Knowledge Submission.	Knowledge Expert
ST 7.1.10	Reject Knowledge Submission?	If the Knowledge Expert believes the Knowledge Submission is invalid, they can decide to reject it and provide reasons which will be provided to the Knowledge Contributor. If yes, go to ST 7.1.6 to inform the Knowledge Contributor of the rejection reason. If no, go to ST 7.1.11 to edit the Knowledge Submission.	Knowledge Expert
ST 7.1.11	Edit Knowledge Submission and notify Contributor	Add comments / additional information to the Knowledge Submission ready for further review and approval. Notify the Knowledge Contributor if any amendments have been made. Go to ST 7.1.12 in order for the Knowledge Manager to perform a QA review.	Knowledge Expert
ST 7.1.12	Perform QA Review	The Knowledge Manager performs a final review of the Knowledge Submission to ensure that it is consistent with the quality and format of the rest of the Knowledge Documents. Go to ST 7.1.13 to decide whether or not to approve the Knowledge Submission for publication.	Knowledge Manager

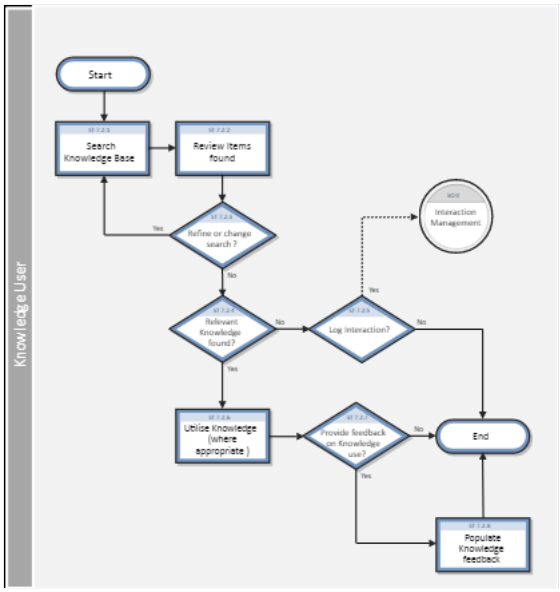
ST 7.1.13	Approve Publication?	Once the Knowledge Manager is sure that the Knowledge Submission has had the necessary checks and approvals, they will decide whether or not to publish it. If yes, go to ST 7.1.14 to confirm the correct audience & dates for publication. If no, go to ST 7.1.6 for the Knowledge Analyst to inform the Knowledge Contributor of the rejection reason.	Knowledge Manager
ST 7.1.14	Confirm publication audience and dates	Confirm the appropriate audience for the Knowledge Submission and if there are any restrictions or deadlines associated with its publication. Go to ST 7.1.15 to publish.	Knowledge Manager
ST 7.1.15	Publish	Publish the Knowledge Submission to the Knowledge Base. The 'Contribute and Approve Knowledge' process ends.	Knowledge Manager

9.14.2. Search, View and Utilize Knowledge Document (Process ST 7.2)

The Knowledge Base is the central repository of the Knowledge Management system. Service Manager provides powerful search and retrieval functionality for accessing, retrieving, and displaying knowledge documents. Searches can be performed using Filters and even Boolean searches are allowed.

Each type of Knowledge Bases has different fields that are indexed for searching, so specific search parameters that match the fields in the knowledge base must be provided. For example, the knowledge articles have a title and author field. When you view an incident, the out-of-box system displays the incident number, incident description, and solution for closed incidents.

Search, View and Utilize Knowledge process is illustrated in the following figure:



Search, View And Utilize Knowledge Process

Process ID	Procedure or Decision	Description	Role
ST 7.2.1	Search Knowledge Base	Enter search criteria into the tool. Search queries can be entered by users through the Self Service functionality or by Operators from the Interaction, Incident and Problem Management modules (this is shown in grey on the process flow as they're not always present when this process is followed.). Go to ST 7.2.2 to review items found.	Knowledge User
ST 7.2.2	Review Items Found	Review search results and determine whether any are appropriate. Go to ST 7.2.3 to refine or amend the search, if necessary.	Knowledge User
ST 7.2.3	Refine or change search	If appropriate Knowledge Documents have not been returned, the search criteria can be amended to obtain better results. If yes, go to ST 7.2.1 to search the Knowledge Base again. If no, go to ST 7.2.4 to determine whether relevant Knowledge has been found.	Knowledge User
ST 7.2.4	Relevant Knowledge Found?	Has a Knowledge Document been returned that's helpful and relevant to the query? If yes, go to ST 7.2.6 to Utilize Knowledge where appropriate. If no, go to ST 7.2.5 to determine whether to log an Interaction.	Knowledge User
ST 7.2.5	Log Interaction?	If a suitable Knowledge Document was not found, an Interaction may need to be raised to deal with the issue that was being researched. If yes, go to Interaction Management (SO 0.1.1). If no, the 'Search, View and Utilize Knowledge' process ends.	Knowledge User
ST 7.2.6	Utilize Knowledge (where appropriate)	Apply the Knowledge where appropriate. Go to ST 7.2.7 to determine whether to provide feedback on the Knowledge Document.	Knowledge User

ST 7.2.7	Provide feedback on Knowledge Use?	Feedback is monitored for key aspects such as quality and availability of the Knowledge Documents. If yes, go to ST 7.2.8 to populate Knowledge feedback. If no, the 'Search, View and Utilize Knowledge' process ends.	Knowledge User
ST 7.2.8	Populate Knowledge feedback	Populate feedback on the Knowledge Document used. The 'Search, View and Utilize Knowledge' process ends.	Knowledge User

9.14.3. Maintain and Continually Improve Knowledge (Process ST 7.3)

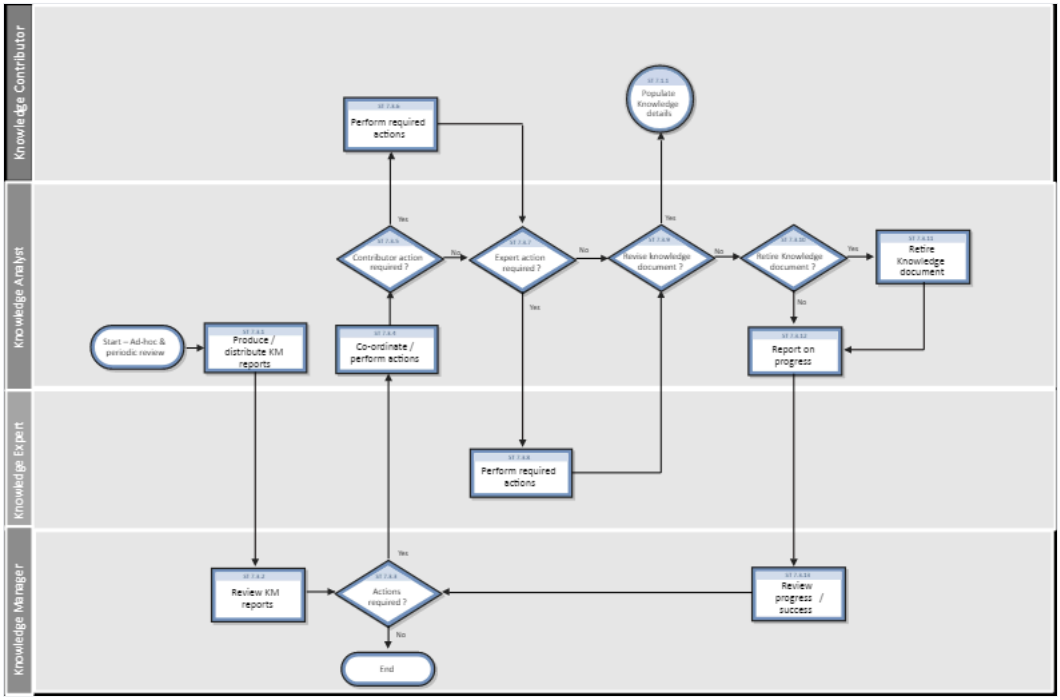
The Maintain And Continually Improve Knowledge process:

- Records the details of new knowledge (documents)
- Updates existing knowledge (documents) where there is any inaccuracy or incompleteness
- Removes obsolete knowledge (documents)

The process is performed by the Knowledge Contributor, Knowledge Analyst, Knowledge Expert, or Knowledge Manager.

Details for this process can be seen in the following figure and table.

The Maintain and Continually Improve Knowledge Workflow is illustrated in the following figure:



Maintain and Continually Approve Knowledge

Process ID	Procedure or Decision	Description	Role
ST 7.3.1	Produce / distribute KM reports	Produce and distribute KM reports to the Knowledge Manager. Go to ST 7.3.2 for the Knowledge Manager to review the KM reports.	Knowledge Analyst
ST 7.3.2	Review KM reports	Review the reports for any issues with the tool or process. Go to ST 7.3.3 to determine required actions.	Knowledge Manager
ST 7.3.3	Actions required?	Identify whether any additional actions need to be taken. If yes, go to ST 7.3.4 to co-ordinate / perform actions. If no, the 'Maintain and Continually Improve Knowledge' process ends.	Knowledge Manager
ST 7.3.4	Co-ordinate / perform actions	Review the identified actions and resolve any that don't need Knowledge Contributor or Knowledge Expert involvement. Go to ST 7.3.5 to determine whether action is required by the Knowledge Contributor.	Knowledge Analyst
ST 7.3.5	Contributor action required?	If yes, go to ST 7.3.6 for the Knowledge Contributor to perform the required actions. If no, go to ST 7.3.7 to determine whether action is required by a Knowledge Expert.	Knowledge Analyst
ST 7.3.6	Perform required actions	Implement appropriate actions to resolve the issue identified in the KM report. Go to ST 7.3.7 to determine whether action is required by a Knowledge Expert.	Knowledge Contributor
ST 7.3.7	Expert action required?	If yes, go to ST 7.3.8 for the Knowledge Expert to perform the required actions. If no, go to ST 7.3.9 to determine whether to revise the Knowledge Document.	Knowledge Analyst
ST 7.3.8	Perform required actions	Implement appropriate actions to resolve the issue identified in the KM report. Go to ST 7.3.9 to determine whether to revise the Knowledge Document.	Knowledge Expert

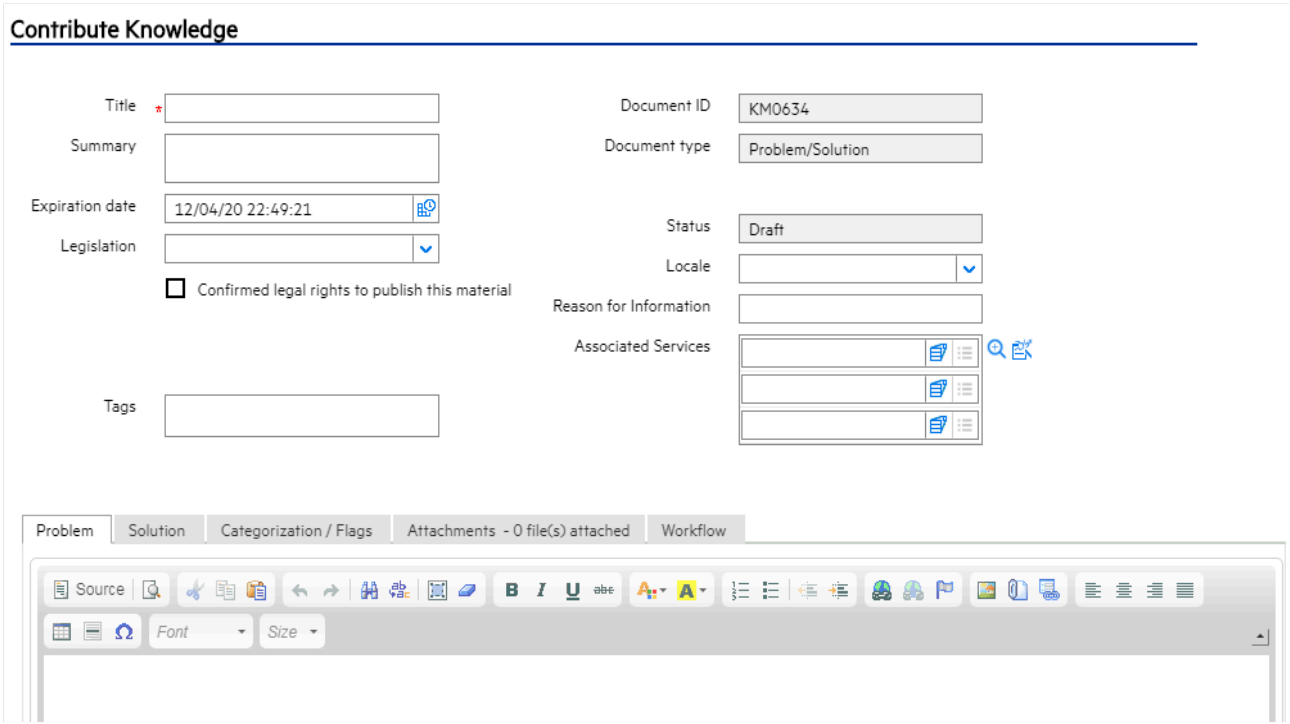
ST 7.3.9	Revise Knowledge Document?	If yes, go to ST 7.1.1 for the Knowledge Contributor to populate Knowledge details. Note: The Knowledge Analyst is the Knowledge Contributors role in this scenario. If no, go to ST 7.3.10 to determine whether to retire the Knowledge Document.	Knowledge Analyst
ST 7.3.10	Archive Knowledge Document?	If yes, go to ST 7.3.11 to retire the Knowledge Document. If no, go to ST 7.3.12 to report on progress against actions.	Knowledge Analyst
ST 7.3.11	Retire Knowledge Document	Archive the Knowledge Document in order for it to be no longer available in searches of the Knowledge Base. Go to ST 7.3.12 to report on progress.	Knowledge Analyst
ST 7.3.12	Report on progress	Provide the Knowledge Manager with a progress report on any outstanding actions. Go to ST 7.3.13 in order for the Knowledge Manager to review progress/success.	Knowledge Analyst
ST 7.3.13	Review progress/success	Review progress on actions identified and determine whether any additional actions are required. Go to ST 7.3.3 determine whether additional actions are required.	Knowledge Manager

9.15. Knowledge Management details

Service Manager uses the Knowledge Management application to enable the Knowledge Management process. The main function of Knowledge Management is to record, store, update, and change Knowledge Documents for use throughout the organization.

The following figure shows the Service Manager KM Contribute Knowledge form for the Problem/Solution document type. The form is used to contribute knowledge (a new KM Document request). As with any new KM document, you must complete the required fields before you can save the record. See the following table for a list and description of the fields on this form.

The Contribute Knowledge form is illustrated in the following screenshot:



The following table describes the Contribute Knowledge form for the Problem/Solution document type.

Contribute Knowledge form field descriptions

Label	Description
Title	The title of the document. The title should accurately describe the content of the document.
Summary	A summary of the document.
Expiration date	The date that the KM document will automatically expire.
Legislation	A drop-down field that you can modify to include specific legislation and regional requirements. Out-of-box options are listed as HIPAA, ISO/IEC 20,000, and SOX.
Confirmed legal rights to publish this material	A checkbox that you must select to confirm that you have the legal rights to use the data contained within the document. If this option is not selected, when you attempt to publish the document, the system prompts you to confirm your legal rights to publish this document.
Tags	Enter one or more free-form hashtags (keywords or terms) of this document. The hashtags you enter are searchable and can be used as keywords to highlight important knowledge information. For more information, see Working with tags.
Document ID	Service Manager creates the ID of the document automatically.
Document type	There are five Document Types: Error Message/Cause; External; Question/Answer; Problem/Solution; and Reference.
Status	The KM document's current state in the KM workflow.
Locale	The language used by the search engine when searching and indexing. The default locale is English.
Reason for Information	This text field can be used for describing the reason for this document or for additional copyright information.
Associated Services	Click the **Fill Field Associated Services** icon to select one or multiple business services that are associated with this document. Specifying an associated service will cause this knowledge document and the service record to be linked to each other in their **Related Records** tab and **Related Documents** respectively. Once this knowledge document is published, it is listed in the **Related Documents** section of the service records. After you select the services, the **Notify Service Subscribers on Publish** option is displayed. Select this option if you want to notify users who have subscribed to this service once the document is published. To view subscribers to the services, click the **View Subscribers** icon. A list of users and departments actively subscribed to the selected services is displayed. Click the subscription ID link of a department to see users in that department.

9.16. Configuration Management overview

The Service Manager Configuration Management application, referred to as Configuration Management throughout this chapter, supports the Configuration Management process. It enables you to define and control the components of services and infrastructure, and to maintain accurate configuration information about the historical, planned, and current state of services and infrastructure.

Configuration Management ensures that you identify, baseline, and maintain selected components of a complete IT service, system, or product as Configuration Items and that you control changes to them by requiring formal approvals. Configuration Management also ensures that you control releases into your business environments.

This section describes how Configuration Management implements the best practice guidelines for the Configuration Management processes. Topics in this section include:

- Configuration Management within the ITIL framework
- Configuration Management application
- Universal Configuration Management Database
- Configuration Management process overview
- Input and output for Configuration Management
- Key performance indicators for Configuration Management
- RACI matrix for Configuration Management

9.16.1. Configuration Management within the ITIL framework

Service Transition publication. The document describes Configuration Management as the process responsible for managing services and assets to support the other Service Management processes.

Configuration Management is planned and implemented in conjunction with Change Management and Release Management to ensure that the service provider can manage its IT assets and configurations effectively. Configuration Management enables enterprises to efficiently identify, control, maintain, and verify the versions of CIs that exist in their infrastructure. Planning is an important part of Configuration Management, because planning ahead enables you to understand the impact that an incident or change could have on your infrastructure.

Responsibility for implementing controls can be delegated, but accountability remains with the responsible manager. Those authorizing the change should provide the manager with information on the cost, risks, and impact of a proposed change and a list of resources required for its implementation.

Configuration Management defines and controls the components of services and infrastructure and maintains accurate configuration information about the historical, planned, and current state of services and infrastructure.

Effective Configuration Management provides the following benefits:

- Accommodates changes to and reuse of standards and best practices.
- Significantly reduces incident resolution time by using a central repository for critical infrastructure data that can be accessed by other applications.
- Includes configuration grouping and business relationships.
- Enables you to meet business and customer control objectives and requirements.
- Provides accurate configuration information to enable people to make decisions at the right time. For example, to authorize changes and releases or to resolve incidents and problems faster.
- Minimizes the number of quality and compliance issues caused by improper configuration of services and assets.
- Optimizes the use of service assets, IT configurations, capabilities, and resources.

9.16.2. Configuration Management application

The Configuration Management application identifies, defines, and tracks an organization's CIs by creating and managing records for those items. Other Service Manager applications can then access these records from a central repository. For example, when you create an incident, you can access the hardware component details from Configuration Management and populate the new incident with that information. Access to Configuration Management significantly reduces the time spent to resolve the incident, as well as alerts you to other potential incidents due to component relationships and dependencies defined in the database.

Configuration Management assures you that releases into controlled environments and operational use are performed on the basis of formal approvals. Configuration Management also provides a configuration model of services, assets, and infrastructure by recording relationships between service assets and configuration items.

All CIs are defined in the device file, the foundation of Configuration Management. Each CI record can include contact, location, vendor, and outage history. Other Service Manager applications, such as Incident Management and Change Management, access Configuration Management to populate fields on forms through the use of link records.

Configuration Management enables you to do the following:

- Identify, control, record, report, audit, and verify service assets and CIs, including versions, baselines, constituent components, and their attributes and relationships.
- Account for, manage, and protect the integrity of service assets and CIs throughout the service lifecycle by ensuring that only authorized components are used and only authorized changes are made.

As new and updated services and systems are released and distributed, accurate configuration information must be available to support the planning and control of changes. Service Manager's out-of-box Configuration Management workflow tracks the IT assets and configurations that make up the infrastructure. These assets can be hardware, software, and associated documentation. The inter-relationships between these components are also monitored. Effective results integrate the service provider's configuration information processes and those of its customers and suppliers. All major assets and configurations must be accounted for and have a responsible manager who ensures that protection and control is maintained.

User profiles determine the access level within Configuration Management. Depending on your access level, you can do the following:

- Add, edit, and save CI records.
- Manage CIs using predefined views to find CIs quickly.
- View and modify software installation information.
- View the maintenance schedule for a CI.
- View and modify SLA information.
- Add CIs to a contract and manage existing contracts.

9.16.3. Universal Configuration Management Database

An integration between Universal CMDB (UCMDB) and Service Manager enables you to share information about the actual state of a configuration item (CI) between your UCMDB system and Service Manager. Any organization that wants to implement the best practices Configuration Management and Change Management ITIL processes can use this integration to verify that CIs actually have the attribute values the organization has agreed to support.

Service Manager allows you to programmatically define what actions you want to take whenever a CI's actual state doesn't match the expected state as defined in the CI record. For example, you can use this integration to automate the creation of Service Manager change or incidents to update or rollback CIs that have unexpected attribute values.

The integration offers several different ways for users to view CI actual state information:

- By default, the integration automatically updates the managed fields of Service Manager CI records as part of the regular UCMDB synchronization schedule. You can choose the option to configure the integration to automatically create change or incidents instead.
- You can view the current actual state of a CI by looking at the Actual State section in the Service Manager CI record. For more information see Baselines, Managed state, and Actual state.
- You can use the Service Manager **View in UCMDB UI** option to view the current CI attributes from UCMDB.

You can specify CI relationships directly in Service Manager or define them in UCMDB and push them to Service Manager like any other asset, by using web services. You can also create UCMDB CI relationships from Service Manager CIs.

9.16.3.1. Baselines

Baselines are an optional feature of Configuration Management that allow you to define a set of attributes that all instances of a configuration item (CI) should have. A baseline is a template CI that defines the expected or authorized attributes of a CI. Typically, a baseline only describes the attributes that you expect CIs to share in common and doesn't include attributes that you expect to vary. For example, a baseline describing PCs might require that all PC CIs be assigned the same model number and operating system version but not the same owner or serial number. In this example, the model number and the operating system would be authorized attributes of the baseline, while the owner and the serial number would be individually-managed attributes.



Note

Note Baseline records replace baseline configuration item groups from previous versions of Service Manager. The upgrade process converts existing baseline configuration item groups to query groups.

Baseline records are separate from the CI records they manage. You must first create a baseline record before you can associate it with one or more CIs. All baseline records must have a name, a list of authorized attributes, and a state. Baseline records can optionally have a

version number, which administrators can configure from the Configuration Management environment record. A baseline record's status determines whether you can add or edit attributes, and whether you can associate CIs to the baseline. After you authorize a baseline record, its attributes are locked and you can only associate or remove CIs from the baseline.

It is up to a Configuration Management manager to determine whether a CI that's out of compliance with its baseline is acceptable or requires a change. Keep in mind that both the CI record and the baseline record describe the expected or managed state of a CI. A baseline record is intended to describe the expected state across many similar items. A CI record describes the expected state of an individual item.

There may be cases where it is acceptable for an individual CI to have a different managed state than other CIs in the same baseline. For example, you might have a baseline requiring that all application servers have 8 GB of RAM. However, you may also want one of your application servers, the Web server, to have 16 GB of RAM. You may want to authorize this exception to the baseline rather than creating a new baseline record to describe just one CI.

Baselines only check for compliance against the managed state of the CI. The actual state of the CI is irrelevant to a baseline compliance check. Continuing the example above, the Web server CI record might list 16 GB of RAM as the managed state. This makes it out of compliance with the baseline that requires all application servers to have 8 GB of RAM. If a discovery process later reveals that the Web server actually only has 12 GB of RAM, this might cause Service Manager to open an unplanned change, but it will not cause a new violation of the baseline. Only differences between the CI's managed state (16 GB of RAM) and the baseline (8 GB of RAM) matter.

Baseline section[edit]

Each CI record has a baseline section that lists details about the baseline, if any, that's currently managing the CI. The baseline section lists the name of the managing baseline, its version, and a list of the attribute names and attribute values the baseline expects. If the CI has a value other than the baseline value, Service Manager displays a warning message stating that the Configuration Item is out of compliance with Baseline.

Contents

- 1 Baseline section

9.16.3.2. Managed state

In Service Manager, the managed state is the subset of CI attributes that have been defined as critical enough to be closely managed by a formal change process and have been approved by that process. You may add managed state information for a CI in several ways:

- Automatically add CI attributes from an integration to Universal CMDB
- Automatically add CI attributes from an integration to Connect-It and Universal CMDB
- Manually add CI attributes

After you add the managed state information to a CI, any changes to the CI attributes must go through a Change Management process.

Service Manager owns the managed state of a CI and acts as the definitive source of what the CI attributes should be. The actual state of the CI may differ from the managed state and may trigger actions in Service Manager such as an out of compliance with baseline warning message or the opening of an unplanned change.

Managed State section[edit]

The Managed State section uses subsections to display data about each CI. There are three subsections for this purpose, The Network subsection and the Additional subsection are used for all CI types. The third subsection depends upon the CI and CI type selected. For example, the Adobe Reader is an application CI type and therefore includes the Application subsection in the Managed State section.

Contents

- 1 Managed State section

9.16.3.3. Actual state

The actual state of a CI is the current list of CI attributes. By default, Service Manager only stores and displays the expected or managed state of CIs. Service Manager can only receive actual state information if you set up an integration to Universal CMDB. Service Manager uses the actual state to determine if a CI is in compliance with its managed state. Service Manager compares the managed attribute values listed in the CI record to the attributes values listed in Universal CMDB. If any of the managed attribute values differ from the managed state, Service Manager takes action as defined in the Discovery Event Manager (DEM) settings. By default, Service Manager opens an unplanned change whenever the actual state of a CI attribute differs from the managed state.

Actual State section[edit]

The Actual State section displays the list of CI attributes passed from an Universal CMDB integration. The list of CI attributes varies from CI to CI and may not match your list of managed attributes. That is, the Actual State section displays all the CI attributes it receives from the Universal CMDB integration whether they are managed fields in Service Manager or not.

To view the actual state of the CI, you must first create an integration to an Universal CMDB server. The Universal CMDB server periodically discovers the actual state of CIs and records the actual state in the Configuration Management database. Service Manager accesses the actual state information by using a Web services connection. Service Manager sends the CI ID to the Universal CMDB server and receives a full list of the attributes for that CI. Service Manager displays the CI attributes in the Actual State section of the Configuration Management form.

If a Service Manager CI doesn't have a matching CI in the Universal CMDB server, then Service Manager doesn't display the Actual State section. For example, you may track office furnishing CIs in Service Manager that can't be discovered and tracked in the Universal CMDB.

Contents

- 1 Actual State section

9.16.3.4. CI relationships

Service Manager tracks upstream and downstream relationships between CIs. A relationship between CIs means that there is some dependency between CIs. If an upstream CI has an interruption of service, Service Manager assumes that all CIs with a downstream relationship to the affected CI also have an interruption of service. For example, if a network router has an interruption of service, then all servers and PCs that connect to that router also have an interruption of service.

Any given CI typically has one upstream relationship and one or more downstream relationships. CIs can have logical or physical relationships based on the logical name of the configuration item. CI relationships are independent of baseline, actual or managed states.

CI Relationship section (CI visualization)[edit]

Each CI record has a section that graphically displays relationships between CIs and the current state of each item in the configuration. (UCMDB has a similar relationship diagram.) Service Manager gathers information from all available applications to determine the current state of a CI. You can view, add, or update relationships using the graphical interface. Service Manager uses smart indicators to tell you if there are any current issues, related records, or breaches to availability SLAs for the CI.

Contents

- 1 CI Relationship section (CI visualization)

9.16.4. Configuration Management process overview

The Configuration Management process ensures that selected components of a complete IT service, system, or product (the Configuration Item) are identified, baselined, and maintained and that changes to them are controlled. It provides a Configuration model of the services, assets, and infrastructure by recording the relationships between service assets and Configuration Items. It also ensures that releases into controlled environments and operational use are completed on the basis of formal approvals. It provides a configuration model of the services, assets, and infrastructure by recording the relationships between service assets and Configuration Items (CIs).

Configuration Management may cover non-IT assets, work products used to develop the services, and Configuration Items required to support the services that aren't formally classified as assets. Any component that requires management to deliver an IT Service is considered part of the scope of Configuration Management.

The asset management portion of this process manages service assets across the whole service life cycle, from acquisition to disposal. It also provides a complete inventory of assets and the associated owners responsible for their control.

The Configuration Management portion of this process maintains information about any CI required to deliver an IT service, including its relationships. This information is managed throughout the life cycle of the CI. The objective of Configuration Management is to define and control the components of an IT service and its infrastructure, and to maintain accurate configuration information.

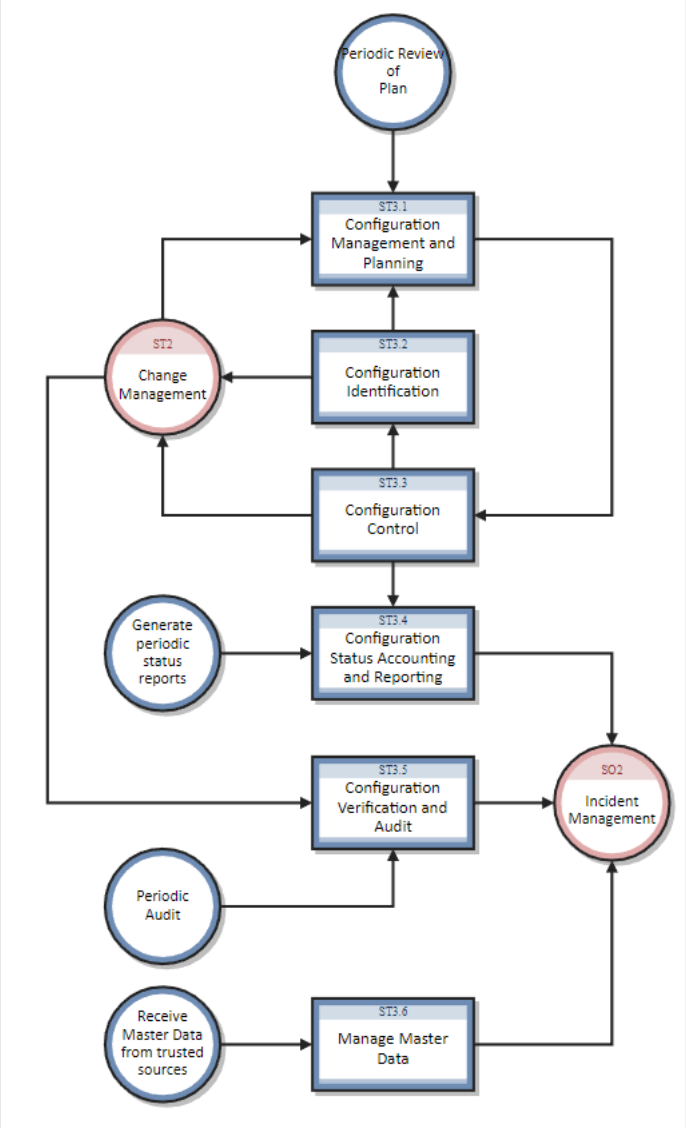
The Configuration Management process manages service assets to support other Service Management processes. Effective Configuration Management facilitates greater system availability, minimizes production issues, and resolves issues more efficiently.

The Configuration Management process ensures that selected components of a complete IT service, system, or product (the Configuration Item) are identified, baselined, and maintained and that changes to them are controlled. It also ensures that releases into controlled environments and operational use are completed on the basis of formal approvals.

Configuration Management comprises five basic activities. The Configuration Management process encompasses all of these activities and ensures that assets are tracked and monitored effectively. The basic activities within the scope of Configuration Management are:

- Configuration Management Planning (process ST 3.1) - includes the activities that enable you to plan the function, scope, and objectives of Service Manager for your organization.
- Configuration Identification (process ST 3.2) - includes the activities that enable you to identify and label all of your company's existing IT components. The information you track includes asset identification, contact, asset network relationship, and model or version data. Enter this information into the database.
- *Inventory maintenance*
  • Configuration Control (process ST 3.3) - includes the activities that enable you to ensure that all information regarding your IT components is kept up to date and accurate. Components can be added, modified, or removed only through controlling documentation, such as an approved Request for Change (RFC).
  • Master data management (process ST 3.6) - includes the activities that enable you to reconcile master reference data managed in other administrations.
- Configuration Status Accounting and Reporting (process ST 3.4) - includes the activities that enable you to run reports of the current and historical data that's concerned with each IT component throughout its life cycle. Status accounting makes changes to components that can be tracked.
- Configuration Verification and Audit (process ST 3.5) — includes the activities that enable you to check and verify physical existence of IT components and ensure that they are correctly recorded in the database.

A general overview of the Configuration Management processes and workflows is depicted in the figure below. They are described in detail in Configuration Management Workflows.



Configuration Management user roles

The following figure describes the responsibilities of the Configuration Management user roles.

Configuration Management user roles

Role	Responsibilities
Configuration Administrator	• Reviews proposed updates to the Configuration Management system (CMS) • Evaluates the pre-modification and post-modification configuration states. • Verifies that CI information is correct and complete and contains a description of attributes to be modified. • Verifies that proposed modifications comply with Configuration Management policies. • Verifies that Configuration details are updated in the Configuration Management database.
Configuration Auditor	• Reviews and validates CMS updates and creates exception reports, if needed. • Conducts configuration audits and performs appropriate actions, if an unregistered component is detected or if a component is missing. • Ensures that information in Configuration Management is correct and that all CIs are accurately and completely recorded.
Configuration Manager	• Manages the Configuration Management plan and policies. • Evaluates any task that requests a change to the CMS data model before the manager releases the task for implementation. For example, the introduction of a new CI into the IT infrastructure would require a request for change and a review of that request prior to implementation of the change. • Verifies that there is no existing CI type that meets the needs of the change and that the proposed data model change doesn't conflict with other parts of the model.
CMS/Tools Administrator	• Configures the data model, policies, and CI types in Configuration Management.

9.16.5. Input and output for Configuration Management

Configuration activities can be triggered and resolved in several ways. The following table outlines the inputs and outputs for the Configuration Management process.

Input and output for Configuration Management

Input to Configuration Management	Output from Configuration Management
• Changes required in the Configuration Management System (CMS) • Tasks initiated from changes or service requests to create or modify Configurations Items (CIs) and relationships	• Configuration Management plan • Configuration Management policies • Configuration Management data model (defining CI types and attributes) • Configuration reports (for example, overview of CIs, subscriptions, license reports, stock reports, or configuration utilization reports) ○ Configuration audit report • Incidents reported due to discrepancies or unauthorized changes detected • Creation and modification of CIs and configuration data

9.16.6. Key performance indicators for Configuration Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your Configuration Management processes. To visualize trend information, it is useful to graph KPI data periodically. Note that some KPIs can't be reported by using only the data from Service Manager.

Key Performance Indicators for Configuration Management

Title	Description
% of CIs related to Services	Number of CIs that are related to one or more IT services as a percentage of the total number of registered CIs that can be related to IT services, in a given time period.
% of CIs related to other CIs	Number of CIs related to one or more other CIs as a percentage of the total number of registered CIs that can be related to other CIs, in a given time period.
% of inaccurate CIs	Number of CIs in the CMS that are registered with inaccurate information as a percentage of the total number of registered CIs, in a given time period.

For completeness, the ITIL V3 and COBIT 4.1 KPIs are included below.

ITIL V3 key performance indicators[edit]

The following are ITIL V3 KPIs for Configuration Management:

- Percentage improvement in maintenance scheduling over the life of an asset
- Degree of alignment between provided maintenance and business support
- Assets identified as the cause of service failures
- Improved speed for Incident Management to identify faulty CIs and restore service
- Impact of incidents and errors affecting particular CI types, for example, from particular suppliers or development groups, for use in improving the IT service
- Percentage reuse and redistribution of under-utilized resources and assets
- Degree of alignment of insurance premiums with business needs
- Ratio of used licenses against paid for licenses (should be close to 100%)
- Average cost per user for licenses (that is, more effective charging options achieved)
- Achieved accuracy in budgets and charges for the assets utilized by each customer or business unit
- Percentage reduction in business impact of outages and incidents caused by Configuration Management
- Improved audit compliance

COBIT 4.1 key performance indicators[edit]

The following are the COBIT 4.1 KPIs for Configuration Management:

- Number of business compliance issues caused by improper configuration of assets
- Number of deviations identified between the configuration repository and actual asset configurations
- Percent of licenses purchased and not accounted for in the repository
- Average lag time period between identifying a discrepancy and rectifying it
- Number of discrepancies relating to incomplete or missing configuration information
- Percent of configuration items meeting specified service levels for performance, security, and availability

Contents

- 1 ITIL V3 key performance indicators
- 2 COBIT 4.1 key performance indicators

9.16.7. RACI matrix for Configuration Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. It is especially useful in clarifying roles and responsibilities in cross-functional/departmental projects and processes. The RACI matrix for Configuration Management is shown in the following table.

RACI matrix for Configuration Management

Process ID	Activity	Configuration Manager	CMS/Tools Administrator	Configuration Administrator	Configuration Auditor	Change Coordinator
ST 3.1	Configuration Management Planning	A/R	R			
ST 3.2	Configuration Identification	A/C		R		C/I
ST 3.3	Configuration Control	A/C		R		C/I
ST 3.4	Configuration Status Accounting and Reporting	A/I		R	R	
ST 3.5	Configuration Verification and Audit	A/C		R	R	
ST 3.6	Manage Master Data	A		R		

9.17. Configuration Management workflows

The Configuration Management process manages service assets to support other Service Management processes. Effective Configuration Management facilitates greater system availability, minimizes production issues, and resolves issues more efficiently.

The Configuration Management process consists of the following processes, which are included in this chapter:

- Configuration Management Planning (process ST 3.1)
- Configuration Identification (process ST 3.2)
- Configuration Control (process ST 3.3)
- Configuration Status Accounting and Reporting (process ST 3.4)
- Configuration Verification and Audit (process ST 3.5)
- Master data management (process ST 3.6)

9.17.1. Configuration Management Planning (process ST 3.1)

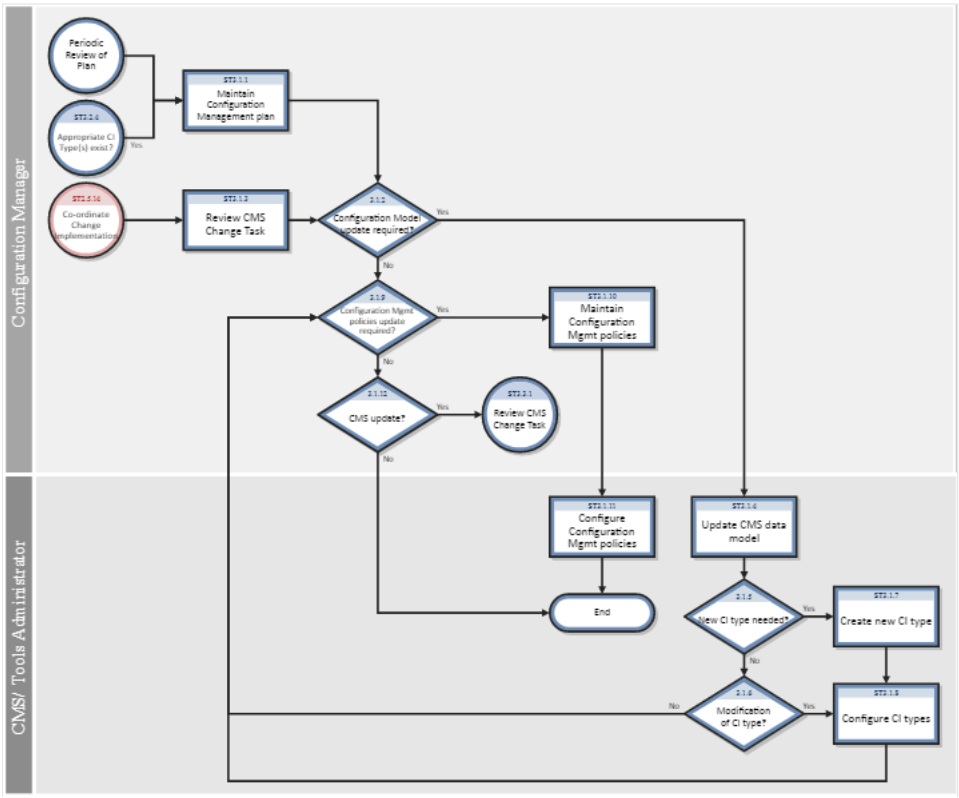
Infrastructure and services should have an up-to-date Configuration Management plan, which can stand alone or form part of other planning documents. The Configuration Management plan should include or describe the following:

- Scope, objectives, policies, standards, roles, and responsibilities
- Configuration Management processes to provide the following services:
- ** Define the Configuration Items that comprise related service(s) and infrastructure
    - Control changes to configurations
    - Record and report the status of Configuration Items
    - Verify the completeness and correctness of Configuration Items according to the requirements for accountability, traceability, and auditability
- Configuration Control (access, protection, version, build, and release controls)

- Interface control process for identifying, recording, and managing CIs and information at the common boundary of two or more organizations (for example, system interfaces and releases)
- Planning and establishing the resources to bring assets and configurations under control and maintain the Configuration Management system (for example, training)
- Management of suppliers and subcontractors performing Configuration Management

Details for this process can be seen in the following figure and table.

The Configuration Management Planning workflow is illustrated in the following figure:



Configuration Management Planning process

Process ID	Procedure or Decision	Description	Role
ST 3.1.1	Maintain Configuration Management plan	The Configuration Manager maintains the Configuration Management policies, objectives, scope, and principles. Periodically, this plan is reviewed to determine improvements. The Configuration Management plan (ACM plan) also defines the scope and level of detail of Configuration Item (CI) data to be maintained in the CMS. A Configuration Management plan provides the guidelines for documenting and modeling IT services in the CMS (identification of CIs).	Configuration Manager
ST 3.1.2	Configuration model update required?	Determine whether the Configuration model should be updated. If yes, go to ST 3.1.4. If no, go to ST 3.1.9.	Configuration Manager
ST 3.1.3	Review CMS change task	The Configuration Manager receives a task from Configuration Management to update the CMS data model (for example, when a new type of CI is introduced in the IT infrastructure as a result of a release).	Configuration Manager
ST 3.1.4	Update CMS data model	The data model defines the structure and information model of the CMS. This includes: • Model of IT services (breakdown of services into service components) • CI relationships types • Definition of CI types • Definition of CI attributes • Identification of data sources (such as HR-system or ERP) The Configuration Manager determines the type of modification that's required for the CMS model.	CMS/Tools Administrator
ST 3.1.5	New CI type needed?	If a new CI type is needed, go to ST 3.1.7. If not, continue with ST 3.1.6.	CMS/Tools Administrator

ST 3.1.6	Modification of CI type required?	If a modification of the CI type is required, go to ST 3.1.8. If not, continue with ST 3.1.9.	CMS/Tools Administrator
ST 3.1.7	Create new CI type	The CMS/Tools Administrator adds a new CI type (device type). This includes the definition of CI attributes and screen design.	CMS/Tools Administrator
ST 3.1.8	Configure CI types	Create or modify the definition of the CI type. This includes: • CI subtypes • Attribute definitions • Screen design • CI relationships types • Naming conventions • Business rules on required fields	CMS/Tools Administrator
ST 3.1.9	Configuration Management policies update required?	The Configuration Administrator determines whether the Configuration Management policies must be updated (to reflect the SCAM plan). If so, go to ST 3.1.12.	Configuration Manager
ST 3.1.10	Maintain Configuration Management policies	The Configuration Manager maintains the Configuration Management policies. Policies may be applicable for specific asset types (or CI Types) or services. Policies may include business rules and requirements for specific information to be maintained in the CMS (for example, for compliance purposes or to monitor contracts). Policies determine how often a configuration audit is required. Policies also designate which data in a CI may be updated by inventory tools, as well as what actions must be performed if unauthorized software is detected. Other items covered by policies and business rules include: • Naming conventions • Labeling rules • Asset capitalization rules (for example, to set the depreciation start date) • Procedures for lost or stolen items	Configuration Manager

ST 3.1.11	Configure Configuration Management policies	Configuration Management policies and requirements are translated into tool settings (for example, required fields, schedule for automated inventory and discovery, and reconciliation rules).	
ST 3.1.12	CMS update?	If yes, go to ST 3.3.1. If not, the process is finished.	Configuration Manager

9.17.2. Configuration Identification (process ST 3.2)

In the Configuration Identification process, the Configuration Administrator selects Configuration Items (CIs), records their identifying characteristics, and assigns unique identifiers to the selected items. This process helps to ensure efficient data storage and retrieval.

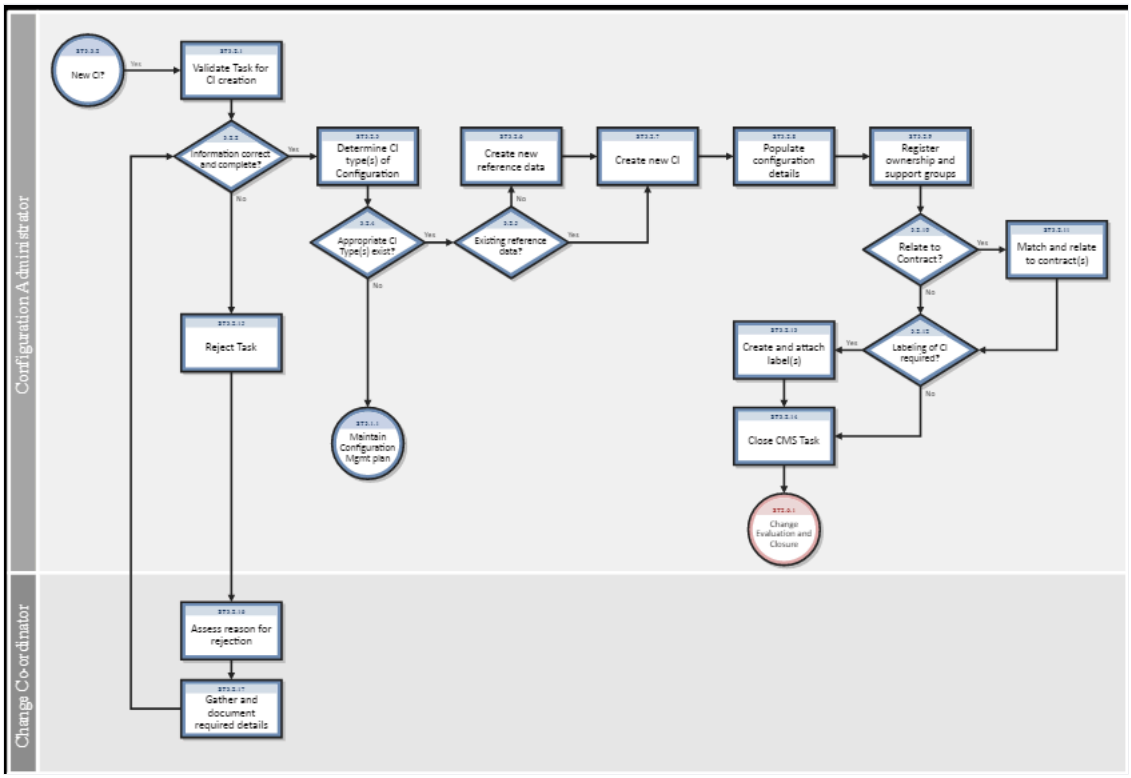
Configuration Identification process is enables you to do the following:

- Identify and register CIs
- Assign unique labels
- Record relationship information

Configuration Identification is responsible for collecting information about CIs and their relationships, and for loading this information into Configuration Management. Configuration Identification is also responsible for labeling the CIs, which enables the corresponding configuration records to be found.

Details for this process can be seen in the following figure and table.

The Configuration Identification workflow is illustrated in the following figure:



Configuration Identification process

Process ID	Procedure or Decision	Description	Role
ST 3.2.1	Validate task for CI creation	The Configuration Administrator reviews the task to verify that all required information to create a new configuration item is complete and correct. Configuration describes a group of CIs that work together to deliver an IT Service, or a recognizable part of an IT Service. The term configuration can also refer to the parameter settings for one or more CIs.	Configuration Administrator
ST 3.2.2	Information correct and complete?	If the information is correct and complete, continue with ST 3.2.3. If not, go to ST 3.2.15 (reject task).	Configuration Administrator
ST 3.2.3	Determine CI type(s) of configuration	Determine the CI type(s) needed to register the CIs. A CI type is used as a template to document the CI, including, attributes and required fields.	Configuration Administrator
ST 3.2.4	Appropriate CI type(s) exist?	A CI can only be registered if the CI type is known and a Configuration Management policy is available for these types. Existing types must match the attributes that need to be managed and allow designation of a person who is responsible for maintaining the CI. CIs of a registered type can be used as templates for new CIs. If there are existing CI types, continue with ST 3.2.5. If not, go to ST 3.2.11.	Configuration Administrator
ST 3.2.5	Existing reference data?	Verify that the reference data (the product definition from the manufacturer or supplier) for the configuration exist. If there is no reference data, go to ST 3.2.6. If yes, continue with ST 3.2.7.	Configuration Administrator
ST 3.2.6	Create new reference data	Create a reference data.	Configuration Administrator
ST 3.2.7	Create new CI	Create the CIs part of the configuration. One or more CIs can be created. Select the CI type (template). Select the model.	Configuration Administrator

ST 3.2.8	Populate configuration details	Enter the required CI attributes, according to the Configuration Management policies. Capture relationships and dependencies between the CIs. Depending upon the CI type and business rules, examples of details include: • Serial number location (for example, on stock) • Purchase order number • Receipt date warranty conditions and warranty end date • CI specific attributes	Configuration Administrator
ST 3.2.9	Register ownership and support groups	All CIs must be assigned to an owner (that is, a reference to an organizational entity such as a cost center) and an administrator (the group responsible for managing the CI during its life cycle). Activities include: • Assign owner • Assign Configuration Administrator (group) • Assign support group for incident assignment (for example, if needed for automated assignment in case of events detected on the device)	Configuration Administrator
ST 3.2.10	Relate to contract?	Determine related contracts for the components, such as: • Maintenance or support contracts • Financial contracts (for example, lease or rental) • License contract or service contracts (for example, SLA, UC, and OLA) If no contracts are relevant for this Configuration, go to ST 3.2.12. If yes, continue with ST 3.2.11 to link the items to the contract.	Configuration Administrator

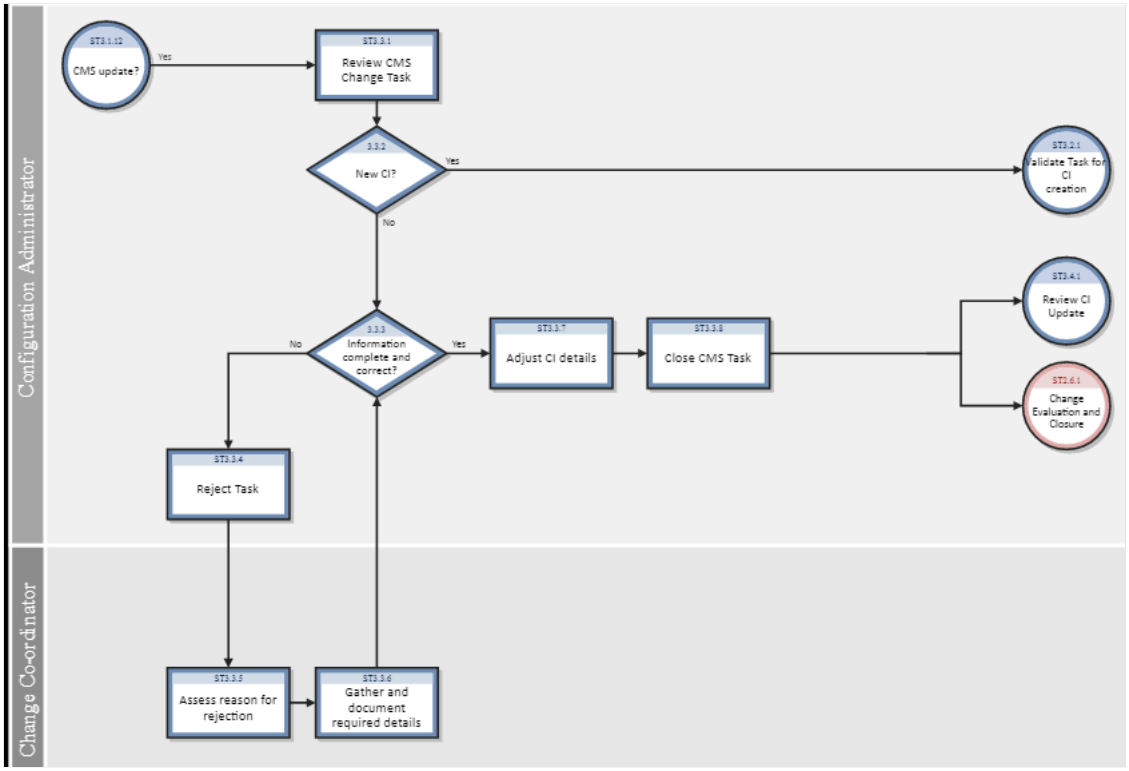
ST 3.2.11	Match and relate to contract(s)	Link the CIs to one or more contracts. Capture the inclusion date of the CI to the contract. If needed, inform the Contract Manager of the new items attached to the contract.	Configuration Administrator
ST 3.2.12	Labeling of CI required?	Determine whether CIs need to be labeled according to the Configuration Management policies. If not, go to ST 3.2.14. If yes, continue with ST 3.2.13.	Configuration Administrator
ST 3.2.13	Create and attach label	Create and print a label. Physically attach the label to the CI.	Configuration Administrator
ST 3.2.14	Close Configuration Management task	After completion, the task can be closed. Update closure code.	Configuration Administrator
ST 3.2.15	Reject task	If the task can't be completed, reject the task. Update the task with reasons and details of any issues found.	Configuration Administrator
ST 3.2.16	Assess reason for rejection	The Change Coordinator assesses the reason for the rejection.	Change Coordinator
ST 3.2.17	Gather and document required details	The Change Coordinator documents the details related to the rejected task.	Change Coordinator

9.17.3. Configuration Control (process ST 3.3)

In the Configuration Control process, the Configuration Administrator reviews the Configuration Management task for updating the Configuration Management system (CMS) and evaluates the configuration in its premodification and postmodification state. The Configuration Administrator verifies the information is correct and complete, and contains a description of attributes to be modified; the proposed modifications comply with Configuration Management policies; and that the configuration details are updated in the Configuration Management database.

Details for this process can be seen in the following figure and table.

The Configuration Control workflow is illustrated in the following figure:



Configuration Control process

Process ID	Procedure or Decision	Description	Role
ST 3.3.1	Review CMS Change task	The Configuration Administrator reviews the task for updating the Configuration Management System (CMS).	Configuration Administrator
ST 3.3.2	New CI?	If the task refers to the creation of one or more new CIs, go to ST 3.2.1 and follow the procedure to validate a task for CI creation. If the task is related to the modification of an existing CI, continue with ST 3.3.3.	Configuration Administrator
ST 3.3.3	Information complete and correct?	Verify that all information to update the CIs is available and correct. The task should refer to at least one CI that must be updated. The task contains a description of the attributes to be modified. If not all information is complete and correct, go to ST 3.3.4 (reject task). If yes, continue with ST 3.3.7.	Configuration Administrator
ST 3.3.4	Reject task	If the configuration update can't be completed, the task is rejected. A reason and recommended actions must be provided.	Configuration Administrator
ST 3.3.5	Assess reason for rejection	The Change Coordinator assesses the reason for the rejection.	Change Coordinator
ST 3.3.6	Gather and document required details	The Change Coordinator documents the details related to the rejected task.	Change Coordinator
ST 3.3.7	Adjust CI details	Modify the configuration details in the Configuration Management database. Configuration modifications can include: • Status (items transferred from test to production or to retirement) • Location (moves) • Relationships and dependencies • Installation of software on the item • Transfer of ownership • Assign contract to a CI	Configuration Administrator

ST 3.3.8	Close CMS task	After completion of the configuration updates, the task can be closed.	Configuration Administrator

9.17.4. Configuration Status Accounting and Reporting (process ST 3.4)

Configuration Status Accounting and Reporting ensures that all configuration data and documentation are recorded as each CI progresses through its life cycle from test to production to retirement. Configuration information should be kept current and made available for planning, decision making, and managing changes to the defined configurations.

Configuration Status Accounting and Reporting keeps track of the following CI status changes:

- New items received (as evidenced by a goods receipt procedure or from development)
- Installation of items
- Transition from test to production
- System down (based upon events)
- Retired or disposed items
- Lost or stolen items
- Unauthorized CIs and Version changes of CIs

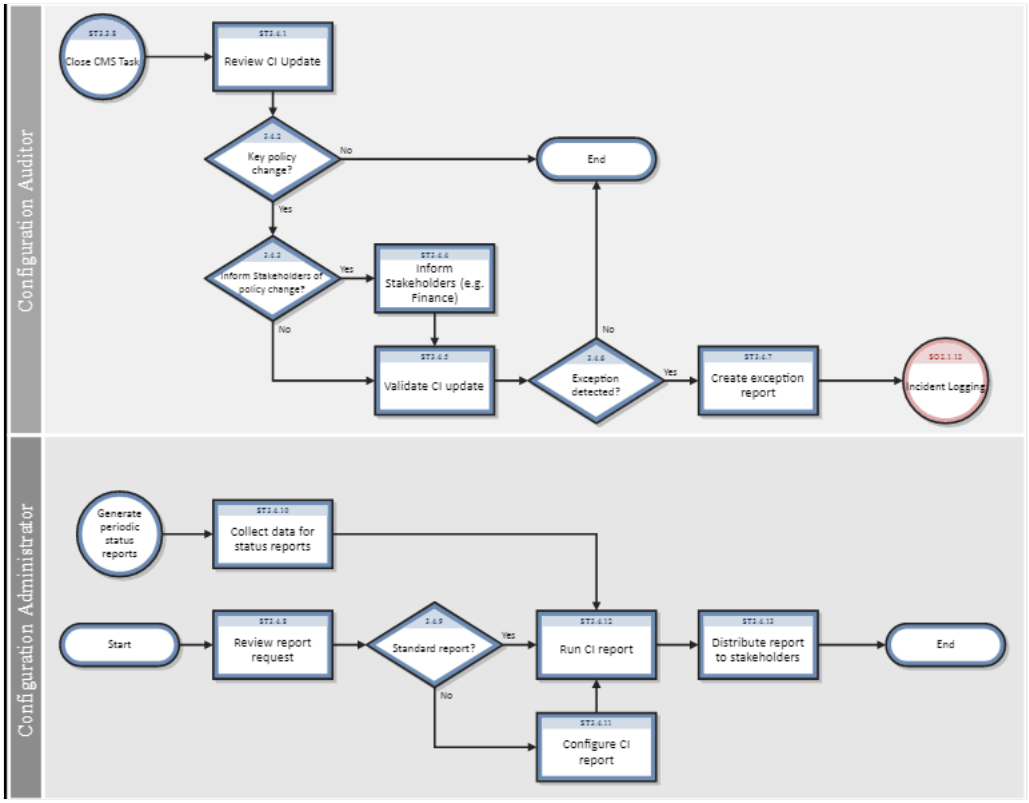
Current and accurate configuration records should be maintained to reflect changes in the status, location, and versions of CIs. The history of each CI must be maintained. Changes to CIs are tracked through various states, such as ordered, received, in acceptance test, live, under change, withdrawn, or disposed.

Where required, configuration information should be accessible to users, customers, suppliers, and partners to assist them in their planning and decision making. For example, an external service provider may make configuration information accessible to the customer and other parties to support the other service management processes in an end-to-end service. Archiving procedures should be defined for data related to retired or disposed CIs.

Configuration Management reports should be available to all relevant parties. The reports should cover the identification and status of the CIs, including their versions and associated documentation. A large set of different reports are needed for the different stakeholders (for example, audit reports, software compliance reports, and charge back reports).

Details for this process can be seen in the following figure and table.

The Configuration Status Accounting and Reporting workflow is illustrated in the following figure:



Configuration Status Accounting and Reporting process

Process ID	Procedure or Decision	Description	Role
ST 3.4.1	Review CI update	Modifications of key attributes of the CI are logged in the history log and verified. During Configuration Identification and control activities, configuration status records are created. These records enable key changes to be visible and traceable. CI attributes that can be logged include: • status (for example, system down) • version number • serial number • installation date • audit status (for example, missing or lost) • removed from a contract Critical CI changes are logged with entries for reason, date stamp, time stamp, and person recording the status change.	Configuration Auditor
ST 3.4.2	Key policy change?	Determine whether the policy must be reviewed or validated, based on the documented Configuration Management policies (and policies related to finance, procurement, Contract Management, and security).	Configuration Auditor
ST 3.4.3	Inform stakeholders of policy change?	Specific changes must be reported to the stakeholders. These include: • Procurement • Finance (for example, by linking to the general ledger) • Contract Manager Verify that the event must be reported. If not, go to ST 3.4.5. If yes, continue with ST 3.4.4.	Configuration Auditor

ST 3.4.4	Inform stakeholders	Inform stakeholders of the event (for example, the Contract Manager when an asset is included in the contract, or procurement when an item is received). Examples of events that should trigger stakeholder notification include: • Received and accepted items • Installation of the asset (for example, for depreciation start date) • Lost or stolen item • Retirement or disposal of an item (for finance)	Configuration Auditor
ST 3.4.5	Validate CI update	Confirm that all relevant status data documented in the CI is complete and correct, according to Configuration Management policies derived from agreements, relevant legislation, and standards. Ensure that the status change or version update is a result of an authorized change.	Configuration Auditor
ST 3.4.6	Exception detected?	If the CI update or CI details aren't correct or complete according to the Configuration policies, continue with SO3.4.7.	Configuration Auditor
ST 3.4.7	Create exception report	Create a new incident (see SO 2.1.11).	Configuration Auditor
ST 3.4.8	Review report request	The Configuration Administrator reviews the request for Configuration Management information.	Configuration Administrator
ST 3.4.9	Standard report?	Configuration Management has defined a number of standard reports (for example, overview of CIs in stock or by status). If this is a standard report, continue with ST 3.4.12. If not, go to ST 3.4.11.	Configuration Administrator
ST 3.4.10	Collect data for status reports	Periodically, Configuration Management procedures provide reports for the different stakeholders, such as financial asset managers, contract managers, or procurement.	Configuration Administrator

ST 3.4.11	Configure CI report	If a standard report doesn't exist, the Configuration Administrator creates a query to select the data to display from the CMS.	Configuration Administrator
ST 3.4.12	Run CI report	The report or query is run against the database. The data is collected in a standard format.	Configuration Administrator
ST 3.4.13	Distribute report to stakeholders	Provide the requested data to the stakeholders. Close the request (if applicable).	Configuration Administrator

9.17.5. Configuration Verification and Audit (process ST 3.5)

Verification and auditing is responsible for ensuring that information in Configuration Management is accurate and that all Configuration Items (CIs) are identified and recorded in Configuration Management. The process can be conducted manually, or by using automated inventory and discovery tools.

Verification includes routine checks that are part of other processes (for example, verifying the serial number of a desktop PC when a user logs an incident). Audit is a periodic, formal check. You should verify and audit your configurations regularly to ensure proper functioning of the entire Configuration Management process, and for related IT service management processes.

The objective of verification and auditing for Configuration Management is to detect and manage all exceptions to configuration policies, processes, and procedures, including security and license use rights. The verification process ensures that configuration records are accurate and complete, and that any recorded changes are approved. Configuration audits help to maintain the integrity of the Configuration Management System (CMS).

Also included in the configuration and audit process is the periodic review of installed software against the policy for software usage to identify personal or unlicensed software or any software instances in excess of current license agreements.

Configuration Verification and Audit activities include:

- Make sure that baselines and standards match the actual components in the IT environment
- Verify that services and products are built and documented, according to documented requirements, standards, or contractual agreements
- Verify that the correct and authorized versions of any CI exists and is correctly identified and described
- Verify the physical existence of CIs (for example, in the organization, in the Definitive Media Library, or in stock)
- Check that release documentation and configuration administration are present before making a release
- Confirm that the current environment is as expected and documented in the CMS, and that any Change requests are resolved
- Check that configuration modifications are implemented through authorized changes
- Validate the existence of a SLA against each CI
- Verify that CI specifications are compliant with defined configuration policies and baselines
- Validate that all required documentation for each CI is available (for example, maintenance contracts, license records, or warranties)
- Check data quality for accuracy and completeness
- Initiate an incident for discovered unauthorized changes

The following are examples of discrepancies:

- Unauthorized software installed
- Unauthorized access to resources and services (for example, access rights not reflected in subscriptions)
- Discrepancy of status or configuration details, as registered in the CMS, compared with the actual status.

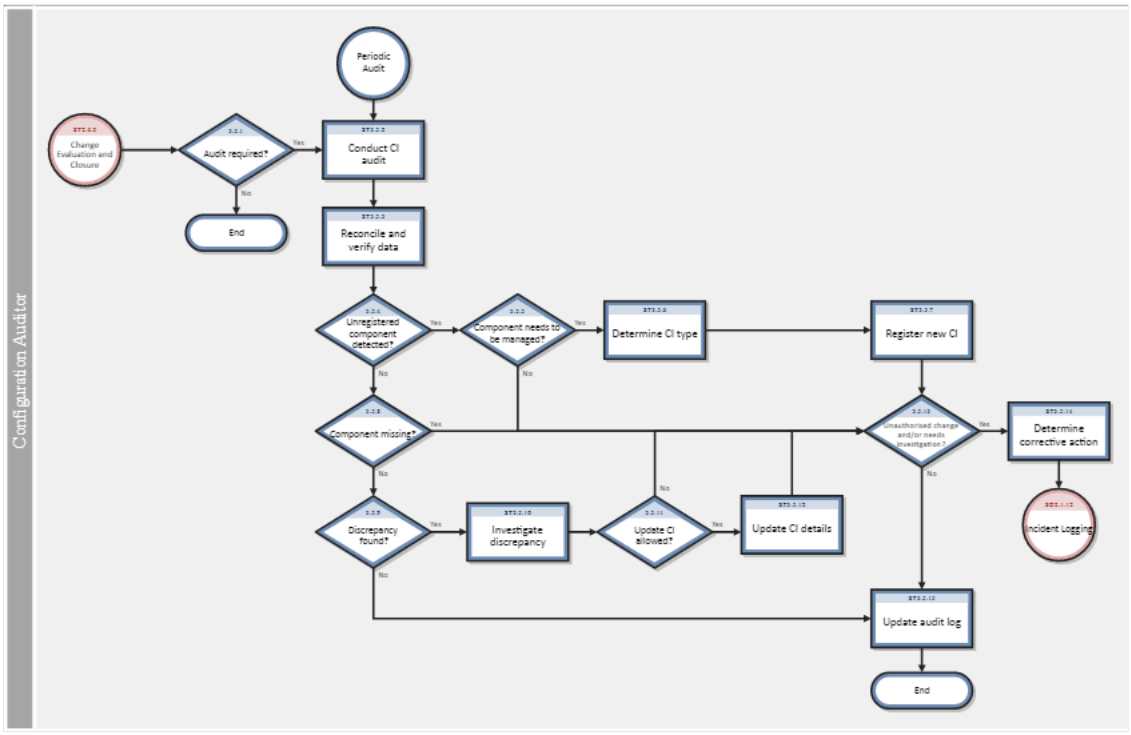
Configuration Verification and Audit processes, both physical and functional, should be scheduled and a check performed to ensure that adequate processes and resources are in place. Benefits of this process include:

- Protection of the physical configurations and the intellectual capital of the organization
- Verification that the service provider is in control of its configurations, master copies, and licenses

- Confidence that configuration information is accurate, controlled, and visible
- Conformance of changes, releases, systems, and IT environments to contracted or specified requirements.
- Accuracy and completeness of configuration records

Configuration audits should be carried out regularly, before and after a major change (or release), after a disaster, and at random intervals. Deficiencies and nonconformities should be recorded, assessed and corrective action initiated, acted on, and reported back to the relevant parties and plan for improving the service. Unauthorized and unregistered items that are discovered during the audit should be investigated and corrective action taken to address possible issues with procedures and the behavior of personnel. All exceptions are logged and reported as incidents. Details for this process can be seen in the following figure and table.

The Configuration Verification and Audit workflow is illustrated in the following figure:



Configuration Verification and Audit process

Process ID	Procedure or Decision	Description	Role
ST 3.5.1	Audit required?	Configuration audits should be considered before and after a major change or release.	Configuration Auditor
ST 3.5.2	Conduct CI audit	Configuration audits (manual or automated) are scheduled periodically. The audit verifies each individual CI. It uses an automated inventory tool that scans the system. Another method is to scan the IT environment and discover the component connected to the enterprise. New components may be discovered, requiring management in the CMS.	Configuration Auditor
ST 3.5.3	Reconcile and verify data	Collected data from the audit must be reconciled and compared with the data already stored in the CMS. Different reconciliation keys and rules can be applied to match the discovered item with the CI in the CMS.	Configuration Auditor
ST 3.5.4	Unregistered component detected?	An unregistered component may be detected in cases where the item can't be matched and found in the CMS. If an unregistered component is detected, go to ST 3.5.5. If not, continue with ST 3.5.8.	Configuration Auditor
ST 3.5.5	Component needs to be managed?	Determine whether the new component needs to be registered in the CMS, based on the scope of the CMS. If yes, continue with ST 3.5.6. If no, go to ST 3.5.13.	Configuration Auditor
ST 3.5.6	Determine CI type	The CI type is selected, based on the properties of the discovered component (for example, model name or type of device).	Configuration Auditor
ST 3.5.7	Register new CI	Create a new CI. Enter the additional attributes of the CI, based on the audit data. Go to ST 3.5.13.	Configuration Auditor

ST 3.5.8	Component missing?	If a component can't be discovered during an audit, it may be lost or stolen (for example, the CI has not been connected to the network for some period of time). The audit status is updated to Lost. If yes, continue with ST 3.5.13. If no, continue with ST 3.5.9.	Configuration Auditor
ST 3.5.9	Discrepancy found?	Based upon the comparison between the CMS administration and the actual data from the audit, one or more discrepancies may be detected. If yes, continue with ST 3.5.10. If not, continue with ST 3.5.15.	Configuration Auditor
ST 3.5.10	Investigate discrepancy	The mismatch between the CMS administration and the actual configuration is investigated in more detail. For each discrepancy, attribute differences and relationships are investigated.	Configuration Auditor
ST 3.5.11	Update CI allowed?	To reduce the number of manual activities, some fields are populated by discovery and auditing tools. These attributes will not be maintained manually. Determine whether the differences can be updated directly without a formal change procedure. If yes, continue with ST 3.6.12. If no, go to ST 3.5.13.	Configuration Auditor
ST 3.5.12	Update CI details	The configuration details are updated, based on the audit date to ensure that the administration is correctly reflecting the actual situation.	Configuration Auditor
ST 3.5.13	Unauthorized change and/or needs investigation?	Determine whether the mismatch between the audit and the CMS administration requires further investigation (for example, detection of unauthorized software). If yes, go to ST 3.5.14. If no, continue with ST 3.5.15.	Configuration Auditor

ST 3.5.14	Determine corrective action	Document the discrepancy and determine the appropriate actions (for example, additional investigation is needed). An incident must be created and assigned to the person responsible for executing the actions. Follow SO 2.1.11 to create a new incident.	Configuration Auditor
ST 3.5.15	Update audit log	The CI is updated with the audit status and last audit date.	

9.17.6. Master data management (process ST 3.6)

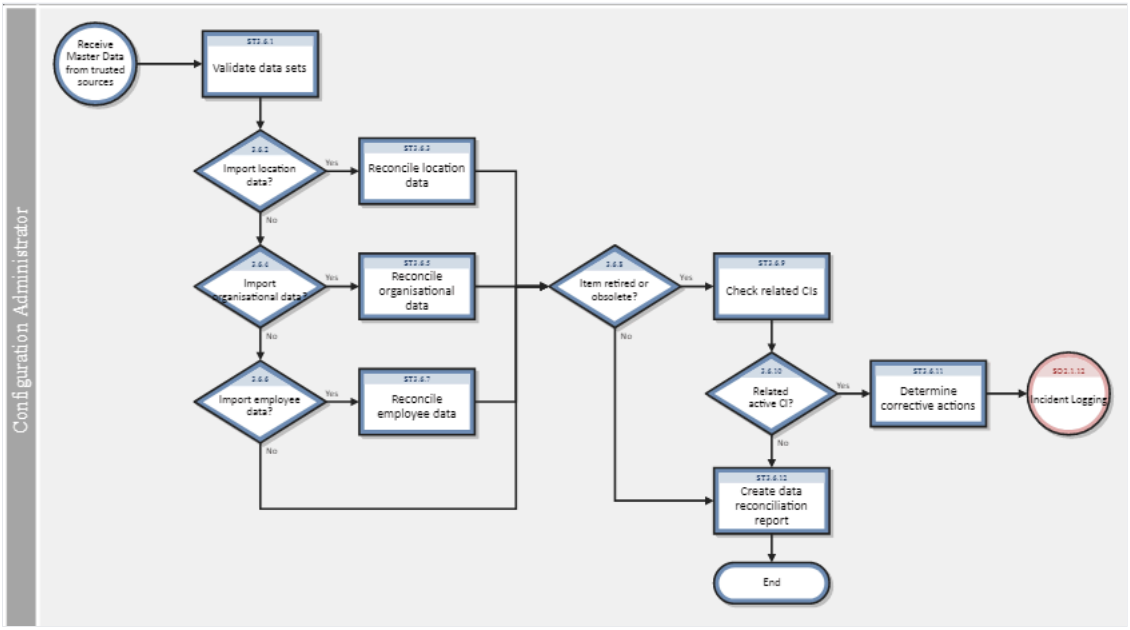
Master reference data is key data that the Configuration Management System (CMS) depends on and is often provided by different organizational functions, such as human resources management, finance, and facilities. For example, master data can include details about organization units, cost centers, employee data, and locations.

The objective of the Master data management process is to reconcile master reference data managed in other administrations. Modification of this reference data is processed in CMS.

Changes in organizational structures, locations, and employee data might result in exceptions or incidents, because existing Configuration Items (CIs) and contracts remain associated with these entities (for example, the retirement of an employee who still has a laptop or mobile phone assigned). Modification of this data must be reviewed and appropriate actions should be initiated.

Details for this process can be seen in the following figure and table.

Master data management workflows is illustrated in the following figure:



Master data management process

Process ID	Procedure or Decision	Description	Role
ST 3.6.1	Validate data sets	Periodically data sets are received from trusted sources. The Configuration Administrator checks the format and content against the defined specifications.	System Administrator Configuration Administrator
ST 3.6.2	Import location data?	If you want to import location data, continue with ST 3.6.3. If not, go to ST 3.6.4.	System Administrator Configuration Administrator
ST 3.6.3	Reconcile location data	Import and load location data into the CMS.	System Administrator Configuration Administrator
ST 3.6.4	Import organizational data?	If you want to import organizational data, continue with ST 3.6.5. If not, go to ST 3.6.6.	System Administrator Configuration Administrator
ST 3.6.5	Reconcile organizational data	Import and load organizational data into the CMS.	System Administrator Configuration Administrator
ST 3.6.6	Import employee data?	If you want to import employee data, continue with ST 3.6.7. If not, stop.	System Administrator Configuration Administrator
ST 3.6.7	Reconcile employee data	Import and load employee data into the CMS.	System Administrator Configuration Administrator
ST 3.6.8	Item retired or obsolete?	Verify that one or more items in the data set are retired or no longer present. Be sure to update the status of items in the CMS.	System Administrator Configuration Administrator

ST 3.6.9	Check related CIs	Verify that one or more CIs are still related to retired items in the modified master data record. For example, a retired user may still have one or more subscriptions or CIs for which that user is responsible. Updates of interest include: • Status updates (for example, retirement) • Job profile changes (for validating access rights and related current subscriptions) • Reorganizations (for example, merge or split of departments) • Cost center changes Master data modifications must be verified to ensure that these updates don't conflict with configuration administration.	System Administrator Configuration Administrator
ST 3.6.10	Related active CI?	If there is a related active CI, continue with ST 3.6.11. If not, go to ST 3.6.12.	System Administrator Configuration Administrator
ST 3.6.11	Determine corrective actions	Follow the procedure to create a new incident (see SO 2.1.11).	System Administrator Configuration Administrator
ST 3.6.12	Create data reconciliation report	Create a report with a summary of data modifications and reconciliation errors, which includes statistics of the number of modifications (for example, new items and retired items).	System Administrator

9.18. Configuration Management Details

Service Manager uses the Configuration Management application to enable the Configuration Management process. The main function of Configuration Management is to identify, baseline, and maintain the Configuration Items (CIs) and to control changes to them. It also ensures that formal approvals guide releases into controlled environments and operational uses.

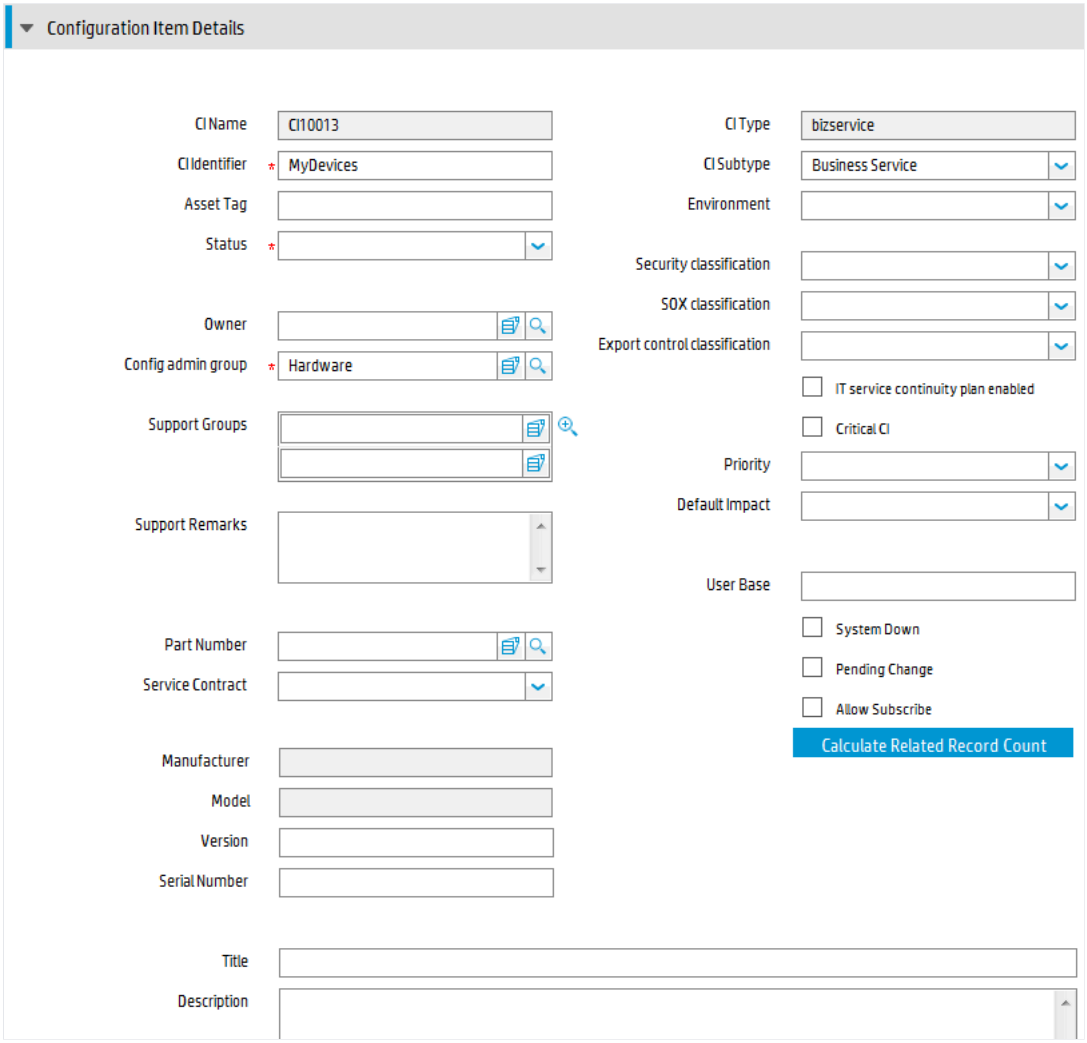
This section explains to the administrator or developer how selected Configuration Management fields are implemented in the out-of-box Service Manager system. Topics in this section include:

- MyDevices configuration item form
- Configuration Management form details

9.18.1. MyDevices configuration item form

The Configuration Manager can view and edit details about a CI on the Configuration item form.

The MyDevices configuration item form is illustrated in the following screenshot:



9.18.2. Configuration Management form details

The following table identifies and describes the fields on the Configuration Management forms.

Configuration Management field descriptions

Label	Description
CI Identifier	The name of the CI. This is a required field.
CI Name	System-generated field that specifies the unique ID of the configuration item (CI).
Asset Tag	This is a legacy field intended for customers migrating from previous versions of Service Manager to track the label or tag placed on physical assets, such as for example, a bar code.
Status	This field specifies the status of the CI. The out-of-box data is: • Available • Planned/On order • Received • In Stock • Reserved • In use • Maintenance • Disposed/Retired • Installed The field is updated manually to reflect the current status of the CI. This is a required field. The Installed status is the default status.
Owner	This field identifies the department that owns the CI, for example, the HR department could own the laptops that its employees use.
Config admin group	This field identifies the group responsible for supporting the CI while the Owner identifies the department that owns the CI. For example, a PC is owned by the HR department, but IT is the Config admin group responsible for supporting the CI. It is the assignment group responsible for handling interactions or incidents for the CI. This is a required field.
Support Groups	This field identifies what assignment groups receive when this CI is part of an interaction as well as when escalating to an incident.
Support Remarks	This field is a comment field intended to describe or provide notes for the support groups.
Part Number	This field specifies the inventory component number for the CI as defined by the company-defined CI inventory number in the model table. The system uses this number to provide data on the Manufacturer, Model, and Version fields if available.
Service Contract	This field specifies the service contract covering the CI.
Manufacturer	This is a system-generated field that specifies the manufacturer of the CI if one is associated with the Part Number. This field along with model and serial number uniquely identify the CI.
Model	This is a system-generated field that specifies the manufacturer's model if one is associated with the Part Number. This field along with manufacturer and serial number uniquely identify the item.
Version	This field specifies the manufacturer's version number for the CI.
Serial Number	This field specifies the manufacturer's serial number for the CI.

Title	This field specifies the title of the CI owner; for example Mr. or Mrs.
Description	This field is a free-form text field to add additional information about the CI.
CI Type	This field identifies the type of CI. The out-of-box data is: • Application • Business Service • CI Group • Computer • Display Device • Example • Furnishings • Hand Held Devices • Mainframe • Network Components • Office Electronics • Software License • Storage • Telecommunications The Managed State section displays different fields depending on the CI type selected.
CI Subtype	This field identifies the subtype of CI. The list of available subtypes depends upon the CI Type the user selected. For more information, see the Configuration Item types and subtypes table.
Environment	This field specifies if a CI belongs to a particular environment. The out-of-box data is: • Development • Test • Production • Failover • None
Security classification	This field specifies if the CI has any security restrictions. The out-of-box data is: • Unrestricted • Restricted • Confidential • Most Confidential
SOX classification	This field specifies if the CI has a Sarbanes Oxley (SOX) classification that applies to the CI. The out-of-box data is: • Critical • Non Critical

Export control classification	This field specifies if the CI has an Export Control classification. The out-of-box data is: • EAR99 (Non Controlled) • 4D994 • 5D991 • 5D002 • 5D992
IT service continuity plan enabled	This field specifies if the CI has an IT service continuity plan enabled for it.
Critical CI	This field specifies if the CI is critical for day-to-day operation, such as an E-mail server or RDBMS server. If you open an incident on a critical CI, the incident indicates that this is a critical CI.
Priority	This field specifies the default priority of any related records opened against the CI. The information in this field is used to prepopulate the priority in an incident or interaction. When a user selects the CI in an incident or interaction, it populates the incident or interaction priority based on the CI priority field. The out-of-box data is: • 1 - Critical • 2 - High • 3 - Average • 4 - Low For additional information, see the Incident Management form details table.
Default Impact	This field specifies the default impact of any related record opened against the CI. The information in this field is used to prepopulate the impact in an incident or interaction. When a user selects the CI in an incident or the interaction, it populates the incident or interaction impact based on the CI Default Impact field. The out-of-box data is: • 1 - Enterprise • 2 - Site/Dept • 3 - Multiple Users • 4 - User For additional information, see the Incident Management form details table.
Calculate Related Record Counts	Clicking this button displays a count of related incidents, problems, known errors, and changes that were opened against this CI.
User Base	This field displays a count of the number of users who use the CI.
System Down	This field indicates whether the CI is currently operational or has an open incident related to it causing it to be non-operational. When you close the incident for the CI, this action clears the flag. The CI is no longer marked as down.
Pending Change	This field indicates whether or not there are any changes pending against this CI.

Allow Subscribe	This field determines if the CI is available for subscriptions from the Service Catalog.
Baseline > Baseline	This field indicates if the CI has an associated baseline and if the CI is in compliance.
Baseline > Baseline Version	This field indicates the baseline version that the CI is tracked against. Baseline Versions enable you to have CIs with the same baseline configuration but slight differences. You can have several versions of that baseline, or if you have updates for a new version of a software installed, then you can select a specific version of a baseline for a CI.
Managed State	This section lists the expected values of CI attributes. All changes to fields in the Managed State section require a Change Management record. See the Managed State subsections table for the Managed State subsection field descriptions.
Actual State	This section lists the actual values of CI attributes if the Service Manager system has an integration to Universal CMDB. It shows the latest discovered information from the UCMDB or its sources.
CI Changes > Pending Attribute Changes	This field lists the attributes that are waiting to be changed through a Change Management record or changes requested through an Unplanned Change (requires an Universal CMDB integration). The data in this field can only be modified through Change Management. Each CI has a set of managed attributes that can be changed through Change Management.
CI Changes > Historic Attribute Changes	This field lists the attributes that are have already been changed through a Change Management record or changes requested through an Unplanned Change (requires an Universal CMDB integration).
Relationships > Upstream Relationships > Upstream Configuration Item, Relationship Name, Relationship Type, Relationship Subtype	This field shows information about which upstream CIs are dependent on the selected CI. Upstream CIs depend on the current CI. For example, the upstream E-mail service depends on the downstream E-mail server, the network, and your E-mail program.
Relationships > Upstream Relationships > Add	This option links to the add a new CI relationship record that enables you to add a new upstream relationship to this CI.

Relationships > Upstream Relationships > View Relationship Type (All, Logical, Physical)	This option provides different views of upstream CI relationships for the specified CI. • All: displays all upstream CI relationships for this CI that are either physical or logical. • Logical: displays all upstream logical CI relationships for the specified CI. A logical connection means that you can access the CI but there is no direct physical connections to other CIs. For example, a network printer that you use. • Physical: displays all upstream physical CI relationships for the specified CI. A physical connection is when a CI is directly attached to another device. For example, a PC connected to a dedicated printer with printer cable. To view ALL/Logical/Physical upstream relationships of the specified CI, select an option in the View Relationship Type field and then click **Filter**. A list of CI relationship records displays. Click **Cancel** in a CI relationship record to return to the specified CI.
Relationships > Downstream Relationships > Relationship Name, Relationship Type, Relationship Subtype	This option shows the CIs that have a downstream dependency on this CI. For example, the upstream E-mail service depends on the downstream E-mail server, the network, and your E-mail program.
Relationships > Downstream Relationships > Add	This option links to the add a new CI relationship record that enables you to add a new downstream relationship to this CI.
Relationships > Downstream Relationships > View Relationship Type (All, Logical, Physical)	This option provides different views of downstream CI relationships for the specified CI. • All: displays all downstream CI relationships for this CI that are either physical or logical. • Logical: displays all downstream logical CI relationships for the specified CI. A logical connection means that you can access the CI but there is no direct physical connections to other CIs. For example, a network printer that you use. • Physical: displays all downstream physical CI relationships for the specified CI. A physical connection is when a CI is directly attached to another device. For example, a PC connected to a dedicated printer with printer cable. To view ALL/Logical/Physical downstream relationships of the specified CI, select an option in the View Relationship Type field and then click **Filter**. A list of CI relationship records displays. Click **Cancel** in a CI relationship record to return to the specified CI.
Relationship Graph	This section displays a graphical representation of the upstream and downstream relationships for the CI.
Software > Applications & Drivers	This section displays information about the software and drivers installed on the CI. For example, a PC might list Microsoft Office and Adobe Reader along with the version, install date, and license ID for each. An Administrator enters this data using the Managed Software menu.

CI Owner > Primary Contact & Support Contacts	This field displays the CI owner who is the person assigned the CI and uses it on a day-to-day basis. Support contacts are secondary contacts who may have access to the CI. For example, a subscriber would be a department for a printer, but the users would be all the people who use the printer to print. The CI owner is the person who is responsible for the printer, such as the department manager.
Subscribers > Subscriber, Type, Status	This is a system-generated section that shows all the subscriptions (people or departments) made against the CI, and the status of the subscription. Example: People and departments can subscribe to Services or CIs. When looking at an interaction, the Service Desk Agent views a list of all the CIs the caller is subscribed to, and their current status.
Location > Location Information & Location Comments	This section describes the physical location of the CI and may include information such as special access requirements (for example, you may require badge access or you may need to be accompanied by authorized personnel in some locations). For example, the location information might contain, Australia, Home Site, main building, second floor, room 3.
Vendor > Vendor Information & Contract and Response Information	This section provides Vendor Information and Contract and Response Information about the CI for support and maintenance. When the user enters the vendor name, the system automatically provides the additional details.
Audit > Audit Policy, Audit Status, Audit Discrepancy, Last Audit Date, Next Scheduled Audit, Last Audited By	These fields display auditing information and are only enabled for those users who have the capability to audit CIs. The user role is Configuration Auditor.
Metrics > Outage History, Uptime Objectives, Max Duration Objectives	This section displays information related to the SLA and SLO availability data for the CI.
Financial > Contracts, Expense Lines, Labor, Parts	This section displays information for the service contracts, parts, labor, and expenses for the CI.
Attachments	This section displays the Filename and Size of each attachment of the CI record. Users can add new attachments using the **Add File** button and remove any existing attachments by clicking the remove links.

9.18.2.1. Configuration Item types and subtypes

The following table lists the types and subtypes available for the out-of box Configuration Item (CI) Names.

Configuration Item types and subtypes

CI Name	CI Type	CI Subtype
Application	application	Anti-Virus / Security Back-up Business Development Tools Entertainment Graphics Internet/Web Networking Operating System Reference Other
Business Service	bizservice	Business Service Application Service Infrastructure Service
CI Group	cigroup	Ad Hoc Baseline
Computer	computer	Desktop Dumb Terminal Laptop Tower MAC Server Host VAX Windows Unix Mainframe Logical Partition Terminal Server
Display Device	displaydevice	Monitor Projector
Example	example	
Furnishings	furnishings	Artwork Armoire Bookcase Chair Computer Desk Desk Collection File Cabinet Meeting Table
Hand Held Devices	handhelds	PDA Cell Phone Pager Blackberry Device GPS Device

Mainframe	mainframe	Controller Host CPU FEP NCP LPAR
Network Components	networkcomponents	Router Hub Switch Modem Network Interface Card Gateway Firewall Network Component ATM Switch RAS LB Concentrator Net Device Switch Router
Office Electronics	officeelectronics	Copy Machine Printer Fax Machine Paper Shredder Camera Speaker Calculator Multifunction Word Processor Typewriter VCR Television UPS Net Printer
Software License	softwarelicense	DBMS License Development Tool License Enterprise Management License Operating System License Outlook Productivity Tools License Project Management License Utility Software License
Storage	storage	CDRW Direct Attached Storage (DAS) HDD Network Attached Storage (NAS) Storage Area Network (SAN) ZIP CD Burner

Telecommunications	telcom	Desk Phone Flush Wall Mount Headsets & Accessories NBX PBX Paging Solution Surface Mount

9.18.2.2. Managed State subsections

The Managed State section uses subsections to display data about each CI. There are three subsections for this purpose. The Network subsection and the Additional subsection are used for all CI types. The third subsection depends upon the CI and CI type selected. For example, the Adobe Reader is an application CI type and therefore includes the Application subsection in the Managed State section.

The following table outlines the subsections and fields available for the different CI types.

Managed State subsections

Sub-Tab	Visible Condition	Field Label	Field Name
Hardware	Type: computer or Type: networkcomponents or Type: officeelectronics	Machine Name Primary MAC Address Additional MAC Addresses OS Name OS Manufacturer OS Version Bios ID Bios Manufacturer Bios Model Physical Memory (Kb)	machine.name mac.address addlMacAddress operating.system os.manufacturer os.version bios.id bios.manufacturer bios.model physical.mem.total
Network	true	Network Name Primary IP Address Subnet Mask Default Gateway Configuration File Addl IP Address Addl Subnet Mask	network.name ip.address subnet.mask default.gateway config.file addlIPAddress addlSubnet
Application	Type: application	Application Name Administration URL/Port Business Import Level Disaster/Recovery Coverage Disaster/Recovery Tier Primary Directory Path Data Classification Product Version License Type Service Hours Notification Group	ci.name admin.urlport business.import. level disaster.coverage recorvery.tier primary.path data.classification product.version license.type service.hours notification.groups
Database	Type: database	Data Privacy Data Classification Port Number Disaster/Recovery Coverage Disaster/Recovery Tier Administration URL/Port Product Version Listener Access Port Notification Group	data.privacy recorvery.tier port.number NULL recorvery.tier admin.urlport product.version listener.port notification.group
Telecom	Type: telecom	Admin ID Admin Password Remote Access Phone Remote Access IP Voice type Disaster/Recovery Coverage Disaster/Recovery Tier Grid Login Server Name Monitored	admin.id admin.password remote.phone remote.ip NULL disaster.recovery recorvery.tier grid login.server.name monitored

Service	Type: bizservice	Service Name Service Type Service Status Allow Subscriptions Administration URL/Port Business Import Level Disaster/Recovery Coverage Disaster/Recovery Tier Primary Directory Path	ci.name subtype service.status allowSubscription admin.urlport NULL NULL recorvery.tier primary.path
Additional	true	Manufacturer Name Type Description	addl.manufacturer addl.name addl.type addl.description

9.19. Request Fulfillment overview

The Service Manager Request Fulfillment application, referred to as Request Fulfillment throughout this chapter, supports the Request Fulfillment process. It enables you to route and support all requests for non-standard operational services in an effective way, and ensure that requests will not compromise day-to-day operational activities.

This section describes how Request Fulfillment implements the best practice guidelines for the Request Fulfillment processes. Topics in this section include:

- Request Fulfillment within the ITIL framework
- Request Fulfillment application
- Request Fulfillment process overview
- Input and output for Request Fulfillment
- Key performance indicators for Request Fulfillment
- RACI matrix for Request Fulfillment

9.19.1. Request Fulfillment within the ITIL framework

Request Fulfillment is addressed in ITIL's *Service Operation* publication. The document describes Request Fulfillment as the process responsible for dealing with Service Requests. Many of these are actually small-sized, low cost, frequently performed, and low-risk, which means they are better handled by a separate process rather than being allowed to congest and obstruct the normal incident and change management processes.

Request Fulfillment enables you to meet the following business objectives:

- Maintain user and customer satisfaction through efficient and professional handling of all service requests.
- Provide a channel for users to request and receive standard services for which a pre-defined authorization and qualification process exists.
- Provide information to users and customers about the availability of services and the procedures for obtaining them.
- Source and deliver the components for requested standard services such as licenses and software media.
- Assist with some general information, complaints or comments.

Request Fulfillment includes the following key features:

- Request model, which defines the prerequisites, required authorizations, and sequenced or parallel standard tasks to fulfill the service request
- A detailed, customizable catalog of products
- Scheduling of service requests and tasks
- Automated request fulfillment
- Order and stock management
- Interaction with other Service Manager applications, such as Service Catalog, Configuration Management, Service Desk, Incident Management, Change Management, and Service Level Management
- Integration with other Service Manager applications, such as Configuration Management and Change Management.

- Integration with other products, including:
    - Providing a common web service interface so that other products are able to access service requests and tasks
    - Integration with Asset Manager for request fulfillment billing

9.19.2. Request Fulfillment application

Service Manager Request Fulfillment is an application used to manage user requests for products and services. Requests affect only the person making the request, or a subordinate group of employees. Examples include password resets, individual PC upgrades, and new employee setup.

The Request Fulfillment application enables business staff to improve their productivity or the quality of business services and products. It can also help reduce the cost of providing services and reduce people effort involved in requesting and receiving access to services. Moreover, the use of Request Fulfillment application can increase the control level of an organization's services and the number of fulfilled requests.

Request Fulfillment includes the following key features:

- Request model, which defines the prerequisites, required authorizations, and sequenced or parallel standard tasks to fulfill the service request
- A detailed catalog of products that you can customize
- Scheduling of service requests and tasks
- Automated request fulfillment
- Order and stock management
- Interaction with other Service Manager applications, such as Service Catalog, Configuration Management, Service Desk, Incident Management, Change Management, and Service Level Management
- Integration with other products, including:
  • Providing a common web service interface so that other products are able to access service requests and tasks
  • Integration with Asset Manager for request fulfillment billing

9.19.2.1. Differences between Request Fulfillment and Change Management

Request Fulfillment and Change Management are separate processes, but they are closely related. Request Fulfillment handles common user requests for products and services. These requests usually affect only the person making the request, or a subordinate group of employees. Change Management handles any change to your business that modifies or disrupts the current state of that environment. Usually these modifications or disruptions affect multiple users or business units.

- Request Fulfillment
    - Handles common requests for products and services.
    - Affects a small or limited number of users.
    - Scope is limited.
- Change Management
    - Manages changes (implementations) that modify a business environment.
    - Affects many users.
    - Scope is often large, including large groups or multiple business units.

9.19.2.2. Key elements of Request Fulfillment

Request Fulfillment includes the following key elements.

Request Model

A repeatable model of handling a particular category of service request. A request model defines specific agreed tasks that need to be followed to fulfill the service request of this category. Request models can be very simple, with no requirement for authorization (for example, password reset), or can be complex with many sequential or parallel tasks that require authorization (for example, provision of an existing IT service).

Product Catalog

The product catalog is a predefined catalog of products (hardware, software, or called parts). The product catalog defines the models of independent items that may be requested or ordered.

The product catalog supports general information of parts, including part number, description, cost, and manufacturer. The inventory of the specific item is also available in the product catalog portal. The receiving details of the purchased items can be pre-defined in the

product catalog.

Catalog items are represented as records in the `productCatalog` table.

Vendors

Vendors are internal or external providers of parts. Vendors have a many-to-many relationship with catalog items, and may or may not directly interact with Service Manager.

Vendors are represented as records in the `vendor` table. The terms under which a specific vendor will provide a specific catalog item are stored in the `modelvendor` table.

Requests

A request is a high level record that defines the basic request information such as requester, required dates, coordinator, and description. Request records are the "tickets" that trace the workflow of a request from the user perspective, data entry and request task addition, through authorizations, fulfillment, and follow-up.

Request records are stored in the `request` table.

Request Tasks

A request task is a low level record that defines one of the steps to fulfill the service request. Request tasks can be created automatically according to the request model definition or manually by the authorized users. After creation, request tasks are assigned to internal workgroups or external vendors.

Request tasks are stored in the `requestTask` table.

Orders

Order records are the "tickets" that trace the workflow of an actual order of a part item or several part items from the ordering and receiving perspective. Orders are created manually by authorized users or by automated background processes when the reorder rules defined for stock management are satisfied.

Order records are stored in the `request` table, with a particular "Order" category.

Authorization processing

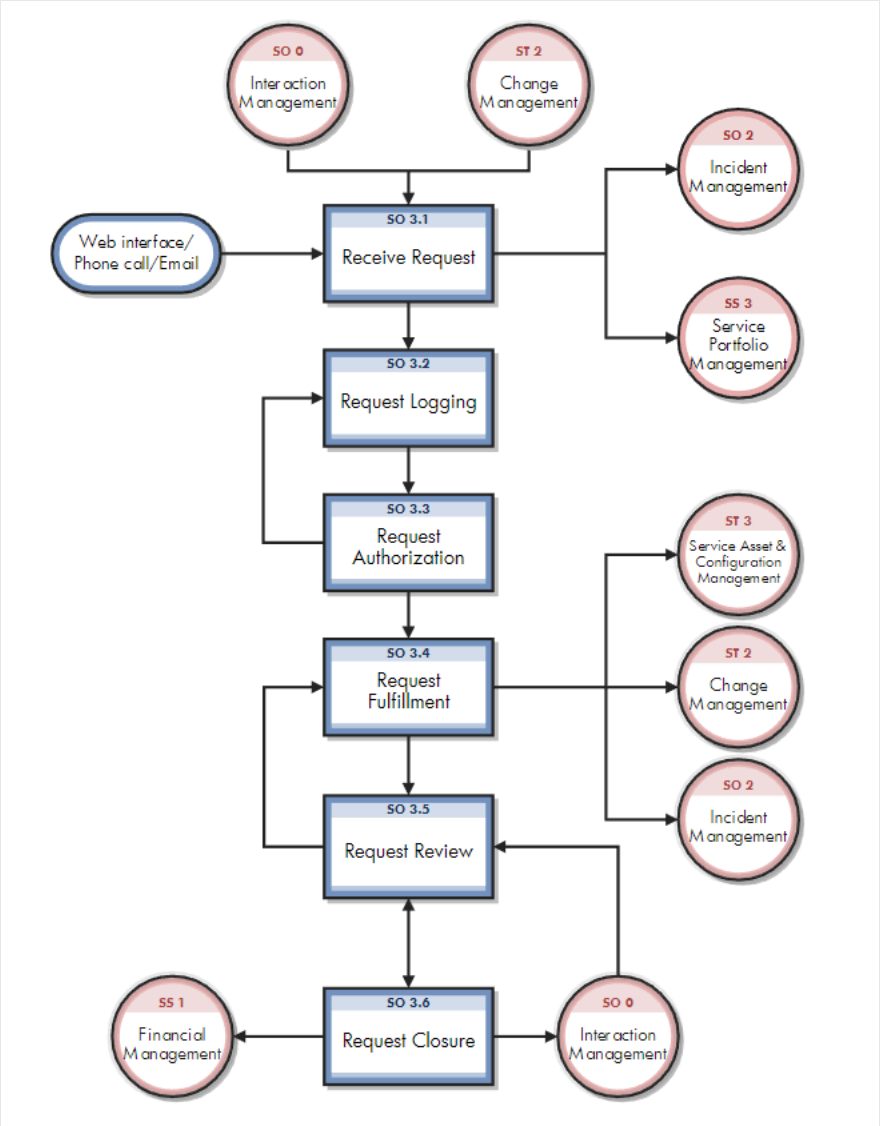
The authorization process automates and formalizes the technical and business evaluation by the appropriate levels of management of requests. Authorizations control accepts risk, cost, and responsibility of a request and its tasks. Authorizations create "chains" of groups who may be required to approve requests before they can advance within their lifecycle. Authorizations can have conditions attached, such as total cost, lead time requirements, and impact.

Authorizations can be defined either in request model or in the dedicated Authorization phase of the request fulfillment workflow.

9.19.3. Request Fulfillment process overview

The Request Fulfillment process includes the activities required to select items from service catalog and submit a service request, to give financial and business approvals, to provision, and to fulfill service requests. It is responsible for ensuring that an IT support is offered for self-help practices and requests can be effectively fulfilled after needed authorizations.

A general overview of the Request Fulfillment processes and workflows is depicted in the figure below. These workflows are described in detail in Request Fulfillment Workflows.



Request Fulfillment user roles

The following table describes the responsibilities of Request Fulfillment user roles.

Request Fulfillment user roles

Role	Responsibilities
Request Process Owner	• Designs request fulfillment models and workflows. • Ensures process technicians have the required knowledge and the required technical and business understanding to deliver the process, and understand their role in the process. • Communicates process information or changes as appropriate to ensure awareness. • Reviews opportunities for process enhancements and for improving the efficiency and effectiveness of the process. • Works with other process owners to ensure there is an integrated approach to the design and implementation of request fulfillment, incident management, event management, access management, and problem management.
User	• Uses Self Service or the Service Desk to log appropriate Service Requests.
Requester	• Initiates a Service Request through the Request Fulfillment portal on behalf of users.
Request Coordinator	• Provides a single point of contact and end-to-end responsibility to ensure submitted Service Requests have been processed. • Provides initial triage of Service Requests to determine which IT resources should be engaged to fulfill them. • Assigns a Service Request to the correct group. • Communicates Service Requests to other IT resources that will be involved in fulfilling them. • Escalates Service Requests in line with established Service Level Targets.
Request Analyst	• Carries out one or more activities in the fulfillment of Service Requests. • Updates the records to show that activities have been carried out correctly. • Responsible for the provisioning of Service Requests within the agreed SLA.
Request Approver	• Reviews Service Request details. • Confirms Service Request details are correct. • Approves/Rejects Service Requests.
Request Manager	• Plans and manages support for request fulfillment tools and processes. • Handles staff, customer and management concerns, requests, issues and enquiries. • Involved in Service Request escalation. • Ensures request fulfillment activities operate in line with service level targets.

Stock Manager	• Responsible for managing stock and defining reorder rules for different stockrooms.

9.19.4. Input and output for Request Fulfillment

Requests can be triggered and resolved in several ways. The following table outlines the inputs and outputs for the Request Fulfillment process.

Input and output for Request Fulfillment

Input to Request Fulfillment	Output from Request Fulfillment
• Service Requests • RFCs • Requests from various sources such as phone calls, web interfaces, or emails • Request for information	• Authorized/rejected service requests • Request fulfillment status reports • Fulfilled service requests • Incidents (rerouted) • RFCs/standard changes • Asset/CI updates • Updated request records • Closed service requests

9.19.5. Key performance indicators for Request Fulfillment

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your Request Fulfillment processes. To visualize trend information, it is useful to graph KPI data periodically. In addition to the data provided by Service Manager, you may need additional tools to report on all of your KPI requirements.

Key Performance Indicators for Request Fulfillment

Title	Description
Number of service requests	The total number of Service Requests. The indicator is used as a control measure.
Size of backlog	The size of current backlog of outstanding service.
Elapsed time	The mean elapsed time for handling each type of Service Requests.
Average cost	The average cost per type of Service Request.
Customer satisfaction	The level of user satisfaction with the handling of Service Requests (as measured in some form of satisfaction survey).

For completeness, the ITIL 2011 CSFs and KPIs are included below.

ITIL 2011 Critical Success Factors and Key Performance Indicators[edit]

The following are ITIL 2011 CSFs and KPIs for Request Fulfillment:

- **CSF** - Requests must be fulfilled in an efficient and timely manner that's aligned to agreed service level targets for each type of request
  - **KPI** - The mean elapsed time for handling each type of service request
  - **KPI** - The number and percentage of service requests completed within agreed target times
  - **KPI** - Breakdown of service requests at each stage (e.g. logged, work in progress, closed etc.)
  - **KPI** - Percentage of service requests closed by the service desk without reference to other levels of support (often referred to as "first point of contact")
  - **KPI** - Number and percentage of service requests resolved remotely or through automation, without the need for a visit

    - **KPI** - Total numbers of requests (as a control measure)
    - **KPI** - The average cost per type of service request
- **CSF** - Only authorized requests should be fulfilled
    - **KPI** - Percentage of service requests fulfilled that were appropriately authorized
    - **KPI** - Number of incidents related to security threats from request fulfillment activities
- **CSF** - User satisfaction must be maintained
    - **KPI** - Level of user satisfaction with the handling of service requests (as measured in some form of satisfaction survey)
    - **KPI** - Total number of incidents related to request fulfillment activities
    - **KPI** - The size of current backlog of outstanding service requests.

Contents

- 1 ITIL 2011 Critical Success Factors and Key Performance Indicators

9.19.6. RACI matrix for Request Fulfillment

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. It is especially useful in clarifying roles and responsibilities in cross-functional/departmental projects and processes. The RACI matrix for Request Fulfillment is shown in the following table.

RACI matrix for Request Fulfillmen

Process ID	Activity	User/Requester	Request Coordinator	Request Analyst	Request Approver	Request Manager
SO 3.1	Receive Request	R	R			A
SO 3.2	Request Logging		R			A
SO 3.3	Request Authorization	I	C		R	A
SO 3.4	Request Fulfillment	I	R	R		A/C
SO 3.5	Request Review		C			A/R
SO 3.6	Request Closure	I	R			A

9.20. Request Fulfillment workflows

The Request Fulfillment process includes the activities required to select items from service catalog and submit a service request, to give financial and business approvals, to provision, and to fulfill service requests. It is responsible for ensuring that an IT support is offered for self-help practices and requests can be effectively fulfilled after needed authorizations.

The Request Fulfillment process consists of the following processes, which are included in this chapter:

- Receive Request (process SO 3.1)
- Request Logging (process SO 3.2)
- Request Authorization (process SO 3.3)
- Request Fulfillment (process SO 3.4)
- Request Review (process SO 3.5)
- Request Closure (process SO 3.6)

9.20.1. Receive Request (process SO 3.1)

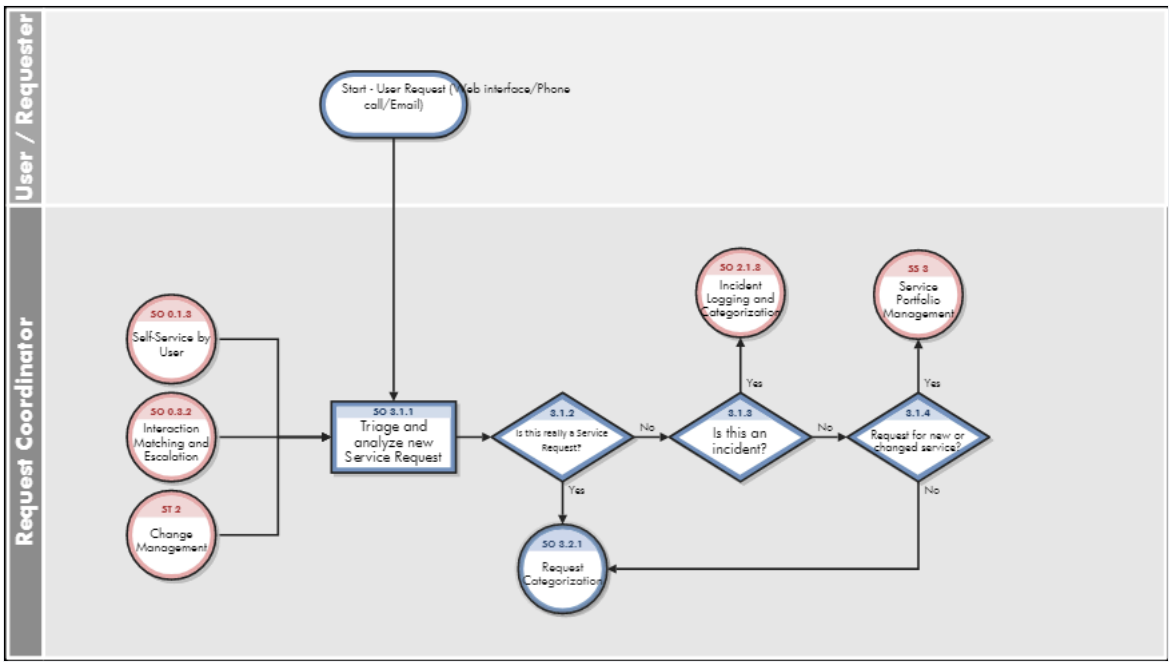
Fulfillment work on service requests should not begin until a formalized request is received.The Receive Request process starts when a User or Requester uses Self Service or the Service Desk to log appropriate Service Requests. It is also usual to have requests that come in from other sources such as RFCs, emails, web interfaces, or phone calls. A Service Request submitted by the User or Requester can be a request for existing Service Catalog, a request for a new service, or an amendment to the Service Catalog. The Request Coordinator needs to triage and analyze the request, and then decide what to do next. As a result of the Receive Request process, a Service Request will be logged.

The following user roles can perform Receive Request:

- User or Requester
- Request Coordinator

Details for this process can be seen in the following figure and table.

The Receive Request workflow is illustrated in the following figure:



Receive Request process

Process ID	Procedure or Decision	Description	Role
SO 3.1.1	Triage and analyze the new Service Request	New Service Requests mostly come from the Service Desk or Service Catalog. It is also usual to have requests that come in from RFCs, emails, web interfaces or phone calls. Based on the information provided by User or Requester, Request Coordinator will triage and analyze the new Service Request from different sources.	Request Coordinator
SO 3.1.2	Is this really a Service Request?	Initially, it must be determined whether the request is really a Service Request or not. If yes, go to SO 3.2.1 to log the Service Request. If no, go to SO 3.1.3.	Request Coordinator
SO 3.1.3	Is this an Incident?	In some cases, an Incident has taken place, but a Service Request is reported. If this is the case, go to SO 2.1.8 to create a new Incident. The Service Request and any related information should be forwarded to Incident Management. If not, go to SO 3.1.4.	Request Coordinator
SO 3.1.4	Request for new or changed service?	In other cases, the request may actually be a request for new or changed service features. If this is the case, the Service Request will be passed to the Service Portfolio Management processes to properly review the need and business case for the service change. If not, go to SO 3.2.1 to log the Service Request.	Request Coordinator

9.20.2. Request Logging (process SO 3.2)

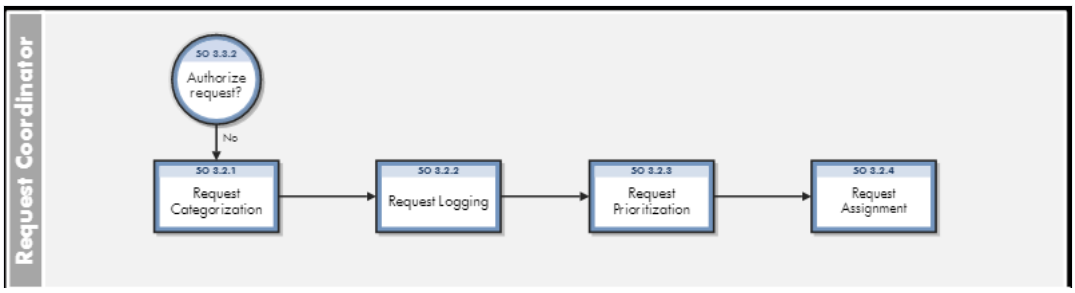
All Service Requests must be fully logged and date/time stamped, regardless of whether they are raised through a Service Desk, RFC, telephone call, or email. Allocating suitable request categorization code should take place at the very beginning of the logging process, so that the exact type of the request is recorded. Besides the categorization code, allocating appropriate prioritization code and determining assignment group/person to fulfill the request are also the important steps in the Request Logging process.

The following user roles can perform Request Logging:

- Request Coordinator

Details for this process can be seen in the following figure and table.

The Request Logging workflow is illustrated in the following figure:



Request Logging process

Process ID	Procedure or Decision	Description	Role
SO 3.2.1	Request Categorization	Before request details are recorded, the appropriate request model (grouped by request category/sub-category) or request category should be chosen based on the type of request being fulfilled.	Request Coordinator
SO 3.2.2	Request Logging	All relevant information of the request must be logged so that a full historical record is maintained. The information needed for a Service Request is listed in the "Request Fulfillment Details" section.	Request Coordinator
SO 3.2.3	Request Prioritization	Appropriate prioritization code helps to determine how the Service Request is handled by support tools and support staff. Prioritization can normally be determined by both the urgency of the request (how quickly the business needs to have it fulfilled) and the level of impact it is causing.	Request Coordinator
SO 3.2.4	Request Assignment	In the request logging stage, the assignment group/person of the Service Request is determined either by a pre-defined assignment rule or Request Coordinator.	Request Coordinator

9.20.3. Request Authorization (process SO 3.3)

No work should take place to fulfill a request until it has been properly authorized. Simple requests can be pre-authorized, while a more rigorous authorization may be needed for other requests before any work can proceed.

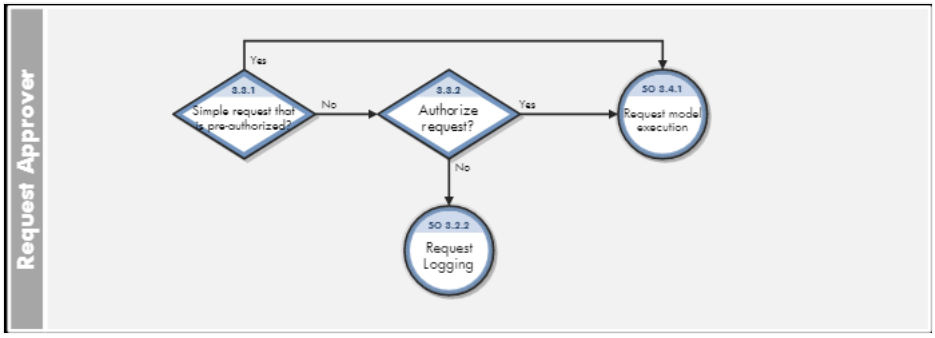
A Service Request that can't be properly authorized should be returned to the Request Logging phase.

The following user roles can perform Request Authorization:

- Request Approver

Details for this process can be seen in the following figure and table.

The Request Authorization workflow is illustrated in the following figure:



Request Authorization process

Process ID	Procedure or Decision	Description	Role
SO 3.3.1	Simple request that's pre-authorized?	If it is a simple request that doesn't require a dedicated authorization, go to SO 3.4.1 to execute the request model. If a more rigorous authorization is required, go to SO 3.3.2 for request authorization.	Request Approver
SO 3.3.2	Authorize request?	Request Approver reviews the Service Request details and decide to approve or deny the request. If request is approved, go to SO 3.4.1 to execute the request model. If request is denied, go to SO 3.3.3.	Request Approver

9.20.4. Request Fulfillment (process SO 3.4)

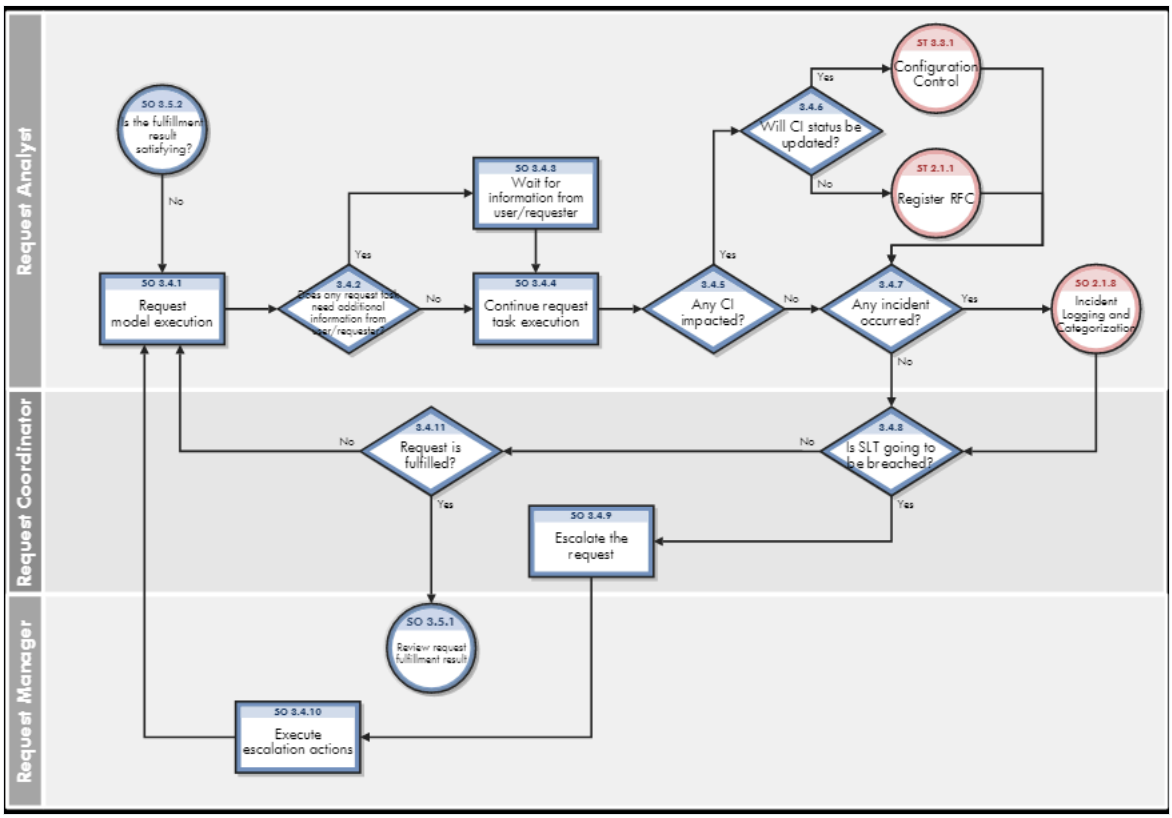
In the Request Fulfillment process, the process steps and activities indicated in the request model are executed by the assignment group/person to fulfill the request. The process steps and activities are created as request tasks.

The following user roles can perform Request Fulfillment:

- Request Coordinator
- Request Analyst
- Request Manager

Details for this process can be seen in the following figure and table.

The Request Fulfillment workflow is illustrated in the following figure:



Request Fulfillment process

Process ID	Procedure or Decision	Description	Role
SO 3.4.1	Request Model Execution	The request tasks defined in the request model are executed by the Request Analyst who is the task assignee. In some cases, additional tasks out of the request model may be added and executed in a request record if necessary.	Request Analyst
SO 3.4.2	Does any request task need additional information from user/requester?	During request fulfillment, Request Analyst checks whether the information provided to fulfill the request is enough or not. If yes, go to SO 3.4.3. If no, go to SO 3.4.4.	Request Analyst
SO 3.4.3	Waiting for information provided from user/requester	Request task status is set as "Pending Customer" and a "Visible to Customer" update is added, so that User or Requester will be informed.	Request Analyst
SO 3.4.4	Continue request task execution	Request Analyst continues task execution with sufficient information provided by User or Requester.	Request Analyst
SO 3.4.5	Any CI impacted?	Request Analyst checks whether CIs are impacted during request fulfillment process. If yes, go to SO 3.4.6. If no, go to SO 3.4.7.	Request Analyst
SO 3.4.6	Will CI status be updated?	If only CI status needs to be updated or new CIs are going to be created, go to the Service Asset & Configuration Management process (ST 3.3.1) to review and update the CIs. If no, go to the Change Management process (ST 2.1.1) to raise and log RFC.	Request Analyst
SO 3.4.7	Any incident occurred?	If an incident occurs during request fulfillment process, the new incident will be created by the Incident Management process (SO 2.1.8) and related to the request. If no incident occurs, go to SO 3.4.8.	Request Analyst

SO 3.4.8	Is SLT going to be breached?	In the whole request fulfillment process, Request Coordinator keeps monitoring the Service Level Target. If SLT is going to be breached, Request Coordinator will escalate the request in time (SO 3.4.9). If no, go to SO 3.4.11.	Request Coordinator
SO 3.4.9	Escalate the request	Request Coordinator indicates the responsible Request Manager whom the Service Request will be escalated to.	Request Coordinator
SO 3.4.10	Execute escalation actions	The Request Manager determines the actions to be performed to fulfill the request within the target time and designates the persons to execute the actions.	Request Manager
SO 3.4.11	Request is fulfilled?	Request Coordinator looks up the execution progress of request tasks and if all tasks are finished, go to SO 3.5.1 to review the fulfillment result. If some tasks aren't closed, go to SO 3.4.1 to continue task execution in request model.	Request Coordinator

9.20.5. Request Review (process SO 3.5)

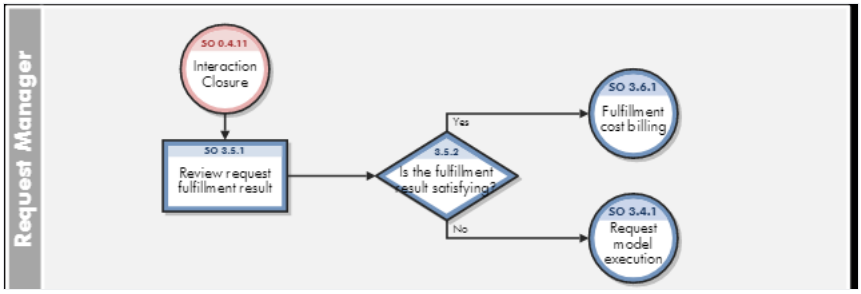
After a request is fulfilled, the accountable person should review the fulfillment result including the fulfillment costs, whether the fulfillment activities are in line with Service Level Target, and so on.

The following user role can perform Request Review:

- Request Manager

Details for this process can be seen in the following figure and table.

The Request Review workflow is illustrated in the following figure:



Request Review process

Process ID	Procedure or Decision	Description	Role
SO 3.5.1	Review request fulfillment result	The Request Manager reviews the request fulfillment result, including the costs for the fulfillment activities, whether Service Level Target has been achieved, and so on.	Request Manager
SO 3.5.2	Is the fulfillment result satisfying?	Request Manager decides whether the fulfillment result is satisfying. If yes, go to SO 3.6.1. If no, go to SO 3.4.1 to re-fulfill the request.	Request Manager

9.20.6. Request Closure (process SO 3.6)

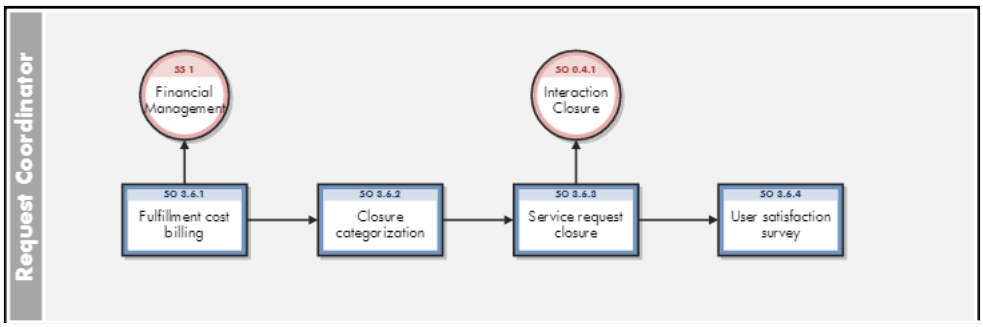
Once the Service Request activities are completed, the Financial Management process might be involved for fulfillment cost billing. When the closure action happens, request categorization should be checked and confirmed. When a request is closed, Service Desk will be notified of the completion status, and the Service Desk will communicate with User or Requester on the fulfillment result. User satisfaction survey can also be carried out for some requests.

The following user role can perform Request Closure:

- Request Coordinator

Details for this process can be seen in the following figure and table.

The Request Closure workflow is illustrated in the following figure:



Request Closure process

Process ID	Procedure or Decision	Description	Role
SO 3.6.1	Fulfillment cost billing	The Financial Management process will need to be notified of any costs incurred by the fulfillment activities.	Request Coordinator
SO 3.6.2	Closure categorization	When the closure action happens, request categorization will be checked and confirmed. If the categorization subsequently turns out to be incorrect, Request Coordinator will update the request record so that a correct closure categorization is recorded for the request. Request Coordinator can ask for advice or guidance from the fulfillment groups as necessary.	Request Coordinator
SO 3.6.3	Service request closure	When the Service Request is formally closed, Service Desk will be notified of the completion status and the related Interaction will be updated accordingly (SO 0.4.1). Service Desk then communicates with User or Requester on the fulfillment result. In case User or Requester is unsatisfied with the fulfillment result, the request record may be re-opened.	Request Coordinator
SO 3.6.4	User satisfaction survey	A user satisfaction survey can be carried out for the agreed percentage of closed requests.	Request Coordinator

9.21. Request Fulfillment details

Service Manager uses the Request Fulfillment application to enable the Request Fulfillment process. The main function of Request Fulfillment is to standardize the methods and processes a business organization uses to log, approve, validate, monitor, and escalate service requests as necessary.

This section describes selected Request Fulfillment fields in the out-of-box Service Manager system. Topics in this section include:

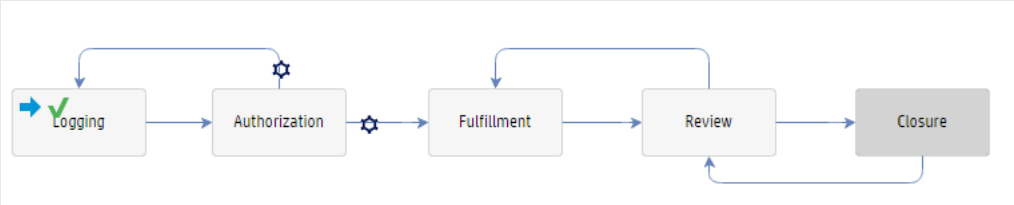
- Request Fulfillment process flow
- Request Fulfillment forms
- Request Fulfillment form details
- Request task forms
- Request task form details

9.21.1. Request Fulfillment process flow

The Request Fulfillment process flow in Service Manager is as follows.

Generic Request workflow

This workflow addresses the request fulfillment process. Its associated request category is “Generic Request.”



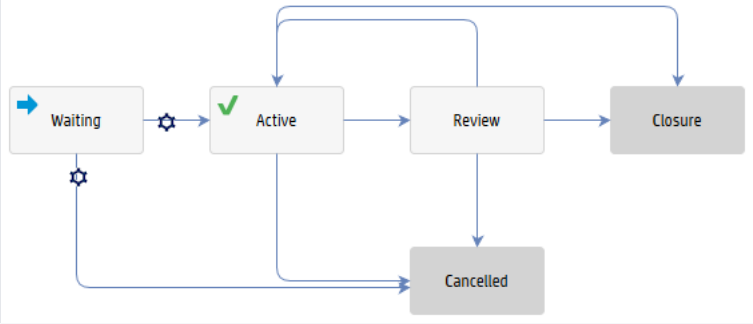
Order workflow

The order workflow is used for purchase and stock management in Request Fulfillment application. Its associated request category is “Order.”

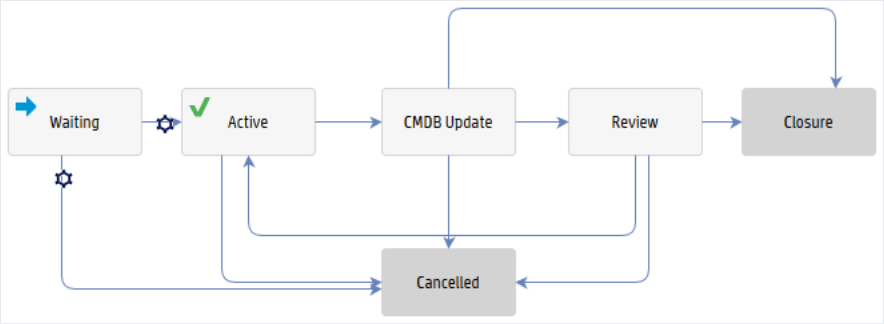


Request task workflows

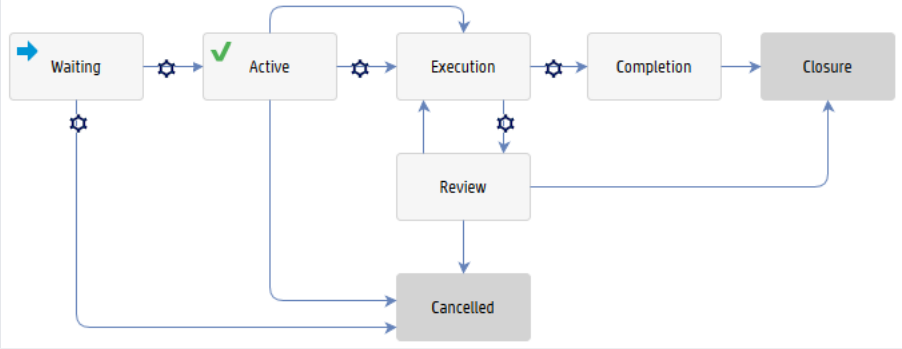
The following is the request task workflow without CMDB update. Its associated request task category is “Labor.”



The following is the request task workflow with CMDB update. Its associated request task categories are “Purchase,” “Reservation,” “Installation,” and “Uninstallation.”



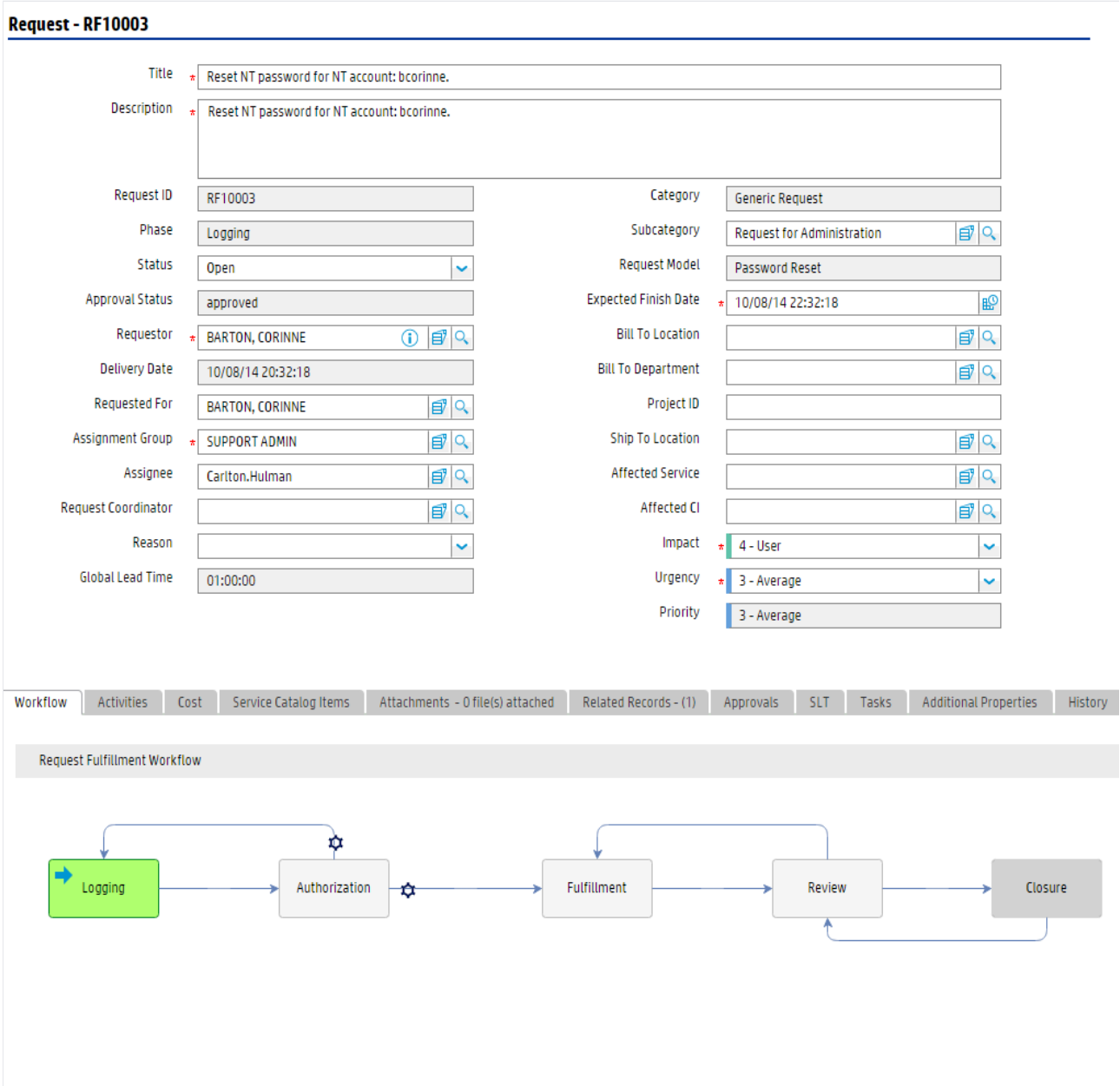
The following is the request task workflow for automation (OO flow execution). Its associated request task category is “Automation.”



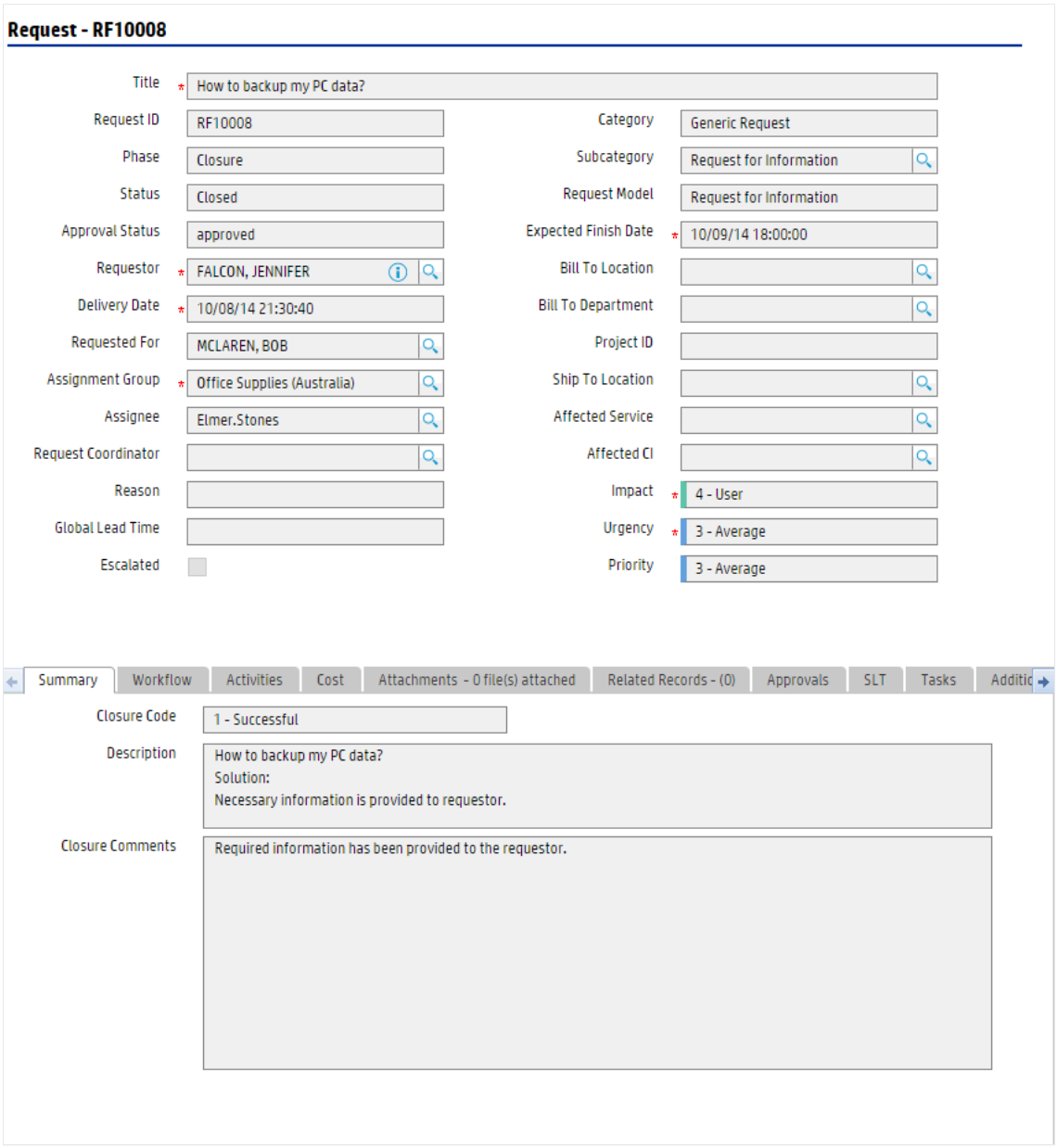
9.21.2. Request Fulfillment forms

Generic Request forms

The following figure shows the open request form that associated with the generic request workflow.

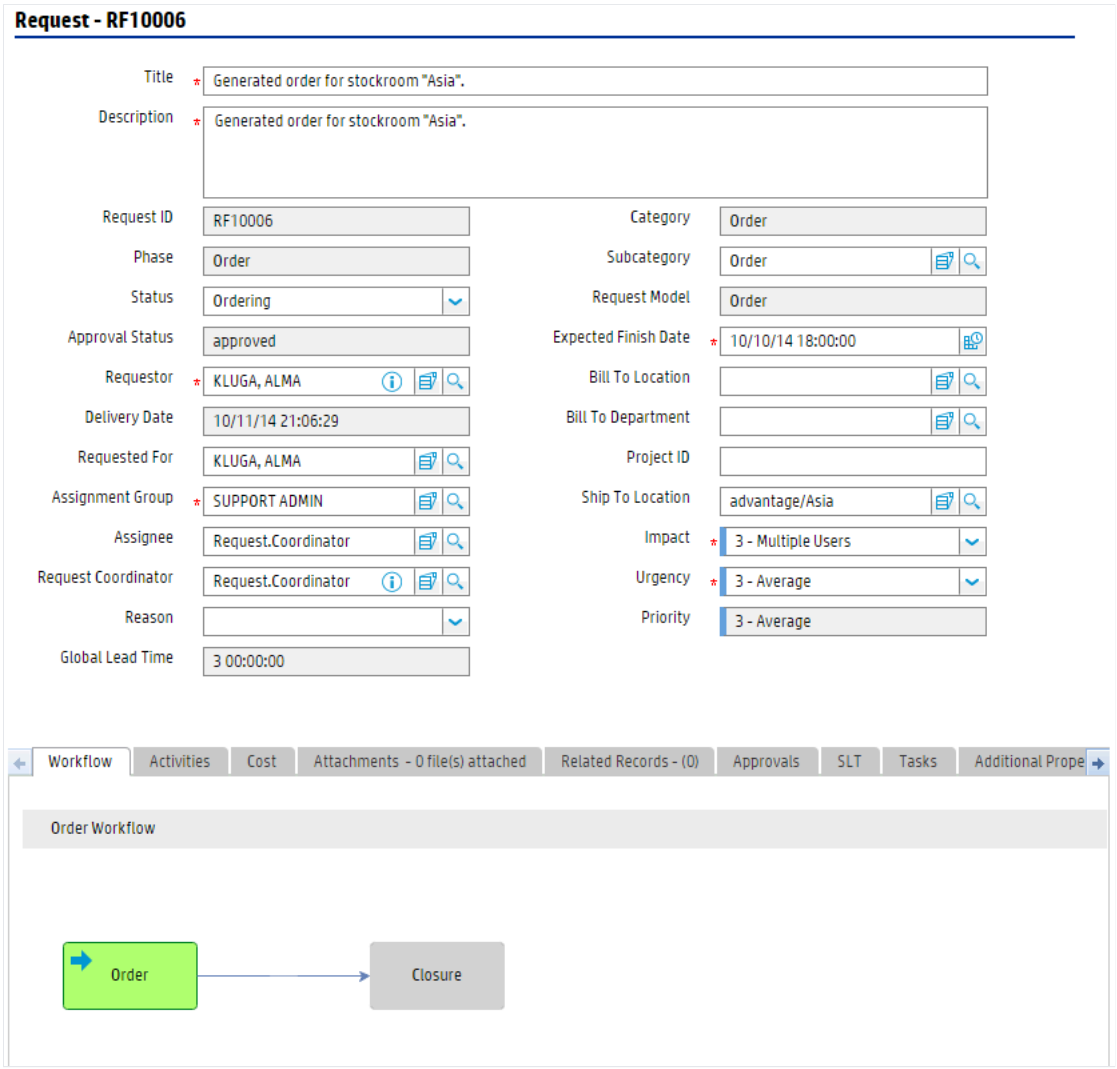


The following figure shows the closed request form that associated with the generic request workflow.

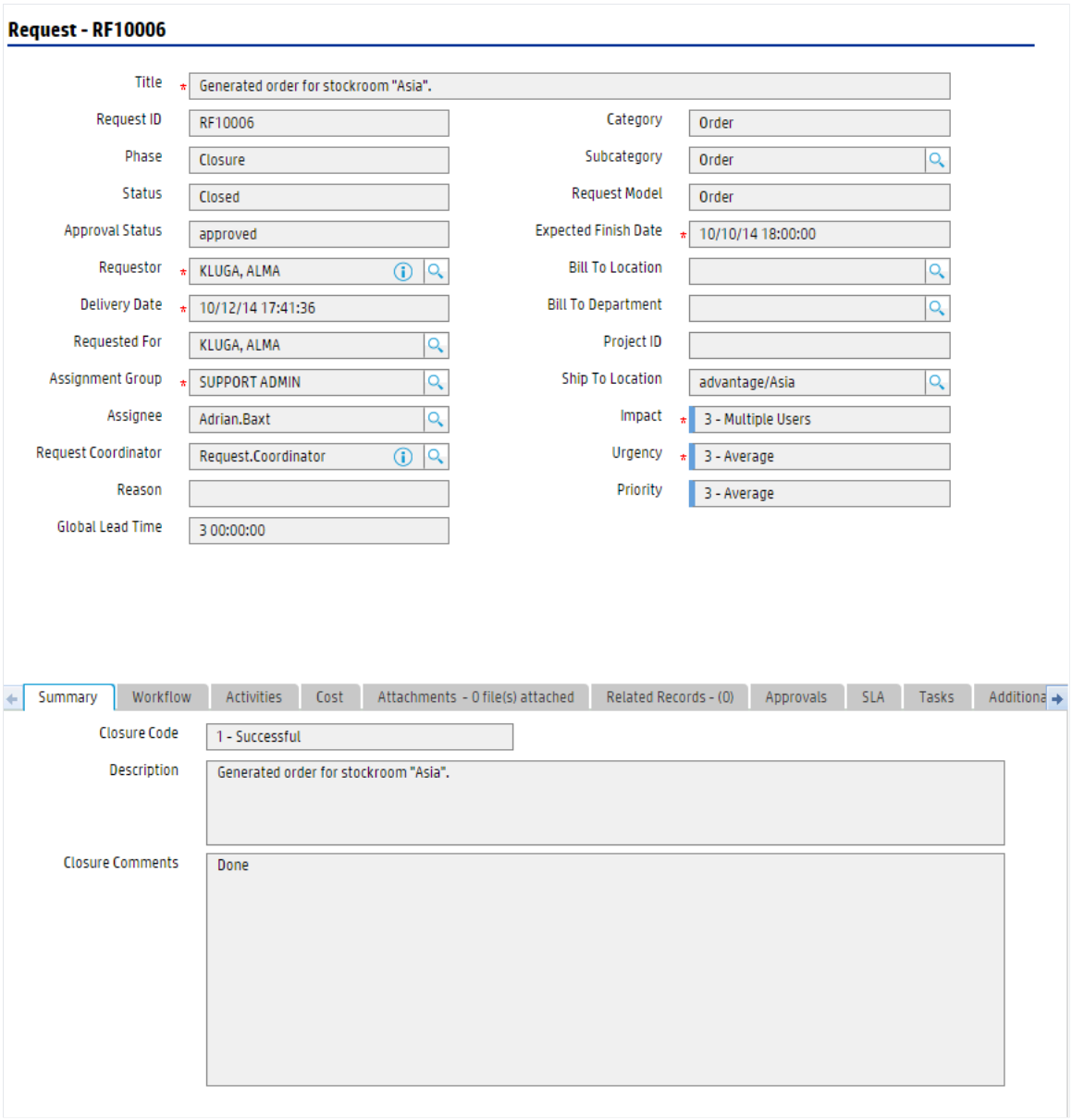


Order forms

The following figure shows the open order form that associated with the order workflow.



The following figure shows the closed order form that associated with the order workflow.



9.21.3. Request Fulfillment form details

The following table identifies and describes some of the features on the Request Fulfillment form.

Request/Order field descriptions

Label	Description
Request ID	The system-generated unique ID for this request/order.
Title	A short description that summarizes the request/order. It is a mandatory field.
Description	A detailed description of the request/order. It is a mandatory field.
Category	This field describes the type of Service Request. In out-of-box, the following categories are available: • Generic Request • Order
Subcategory	Specifies the second level of classifying a Service Request. In out-of-box, the following subcategories are available for the “Generic Request” category : • Employee Off-boarding • Employee On-boarding • Hardware • Others • Request for Administration • Request for Access • Request for Information • Software In out-of-box, the following subcategory is available for the “Order” category: • Order
Request Model	A Request Model is a record that's used to predefine the contents of a specific type of Service Request, including the information used to populate the request/order and the tasks that are needed to complete the request/order. When you open a request/order using a Request Model, basic information is added to the request/order automatically.
Phase	This is a system-generated field that specifies the name of the current phase of the request/order. The following phases are available in out-of-box “Generic Request” workflow: • Logging • Authorization • Fulfillment • Review • Closure The following phases are available in out-of-box "Order" workflow: • Order • Closure

Status	Displays the status of the request/order. The following statuses are available in out-of-box “Generic Request” workflow: • Open - Request is logged and waiting for authorization. • In Progress - Request is being fulfilled. • Pending Customer - Additional information is required from user/requester. • Suspended - Fulfillment activities for the request have been suspended for a pre-defined time period. • Fulfilled - Request is fulfilled and being reviewed. • Closed - Request is closed. The following statuses are available in out-of-box "Order" workflow: • Ordering - Order is being handled. • Closed - Order is closed.
Approval Status	Defines the approval status for the request/order. The system sets this field depending on current approvals and the approval type defined for the record. These approval statuses are available out-of-box: • pending • approved • denied
Requester	The name of the person who submits the Service Request. It is a mandatory field.
Requested For	The name of the user for whom the requester submits this request/order.
Delivery Date	Indicates the target fulfillment time of request. It equals request creation time plus “Global Lead Time.”
Expected Finish Date	Indicates the expected finish date of request from user/requester perspective. If Delivery Target is selected when ordering is made from Service Catalog, the field value will be calculated by request creation time plus Delivery Target. If Delivery Target is unavailable for the request, the field value will leave blank when request is logged. In this case, Request Coordinator will manually input the expected date on behalf of user/requester. It is a mandatory field.
Assignment Group	The support group assigned to work on this request/order. This field can be prepopulated by the data defined in assignment rule or manually set by Request Coordinator. It is a mandatory field.
Assignee	The person assigned to work on this request/order. This person is a member of the assignment group. This field can be prepopulated by the data defined in assignment rule or manually set by Request Coordinator.
Request Coordinator	The name of the person responsible for coordinating the implementation of the request/order. Each Coordinator may belong to several assignment groups. Each group can have just one Request Coordinator.

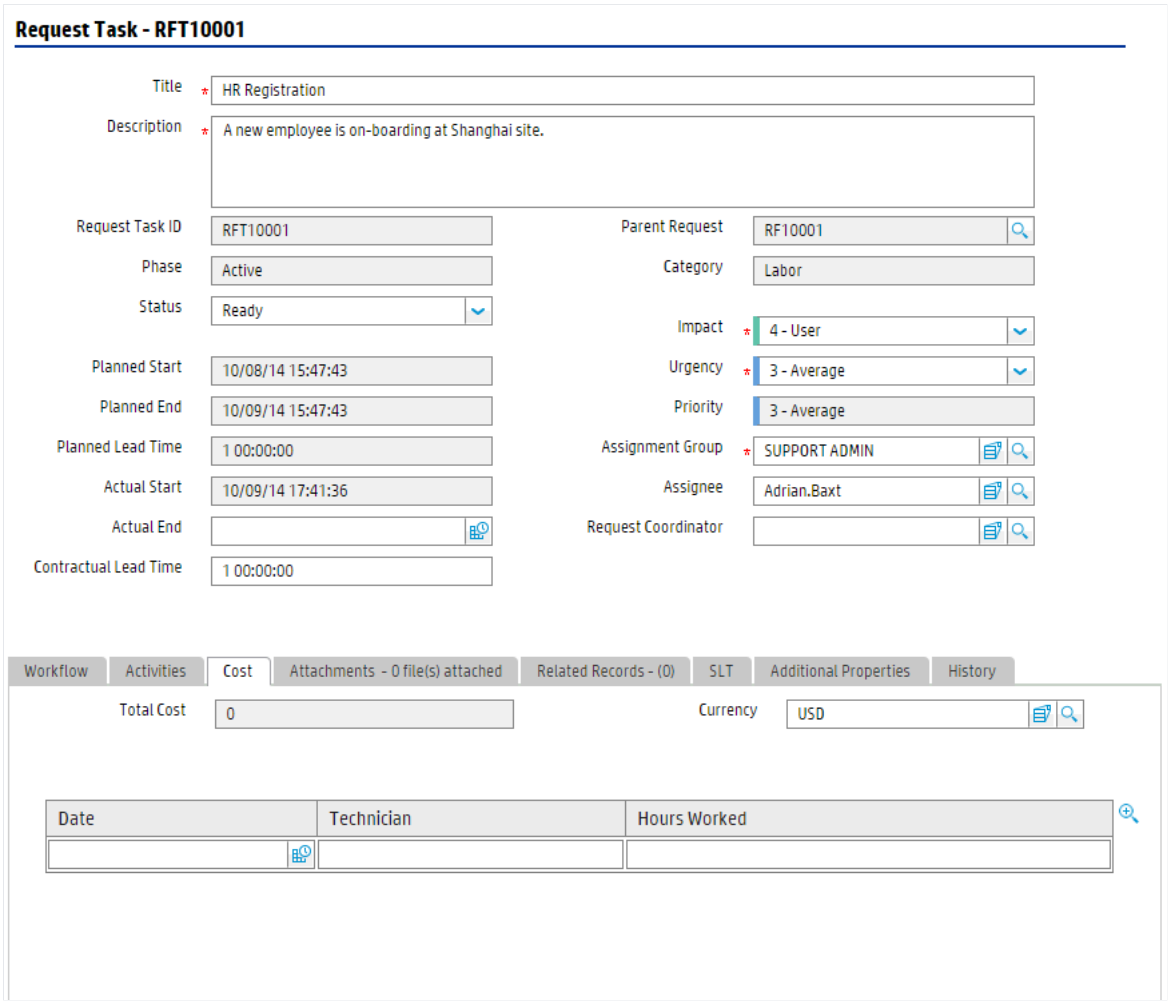
Global Lead Time	This field is the sum of the longest “Planned Lead Time” of all request tasks based on task plan definition in Request Model. It is read-only and copied from Request Model. The out-of-box request fulfillment SLT is measured based on the value of this field.
Impact	A measure of the effect of the request/order on business processes. Impact and urgency are used to assign priority. It is a mandatory field. These impacts are available out-of-box: • Enterprise • Site/Dept • Multiple Users • User
Urgency	A measure of how long it will be until a request/order has a significant impact on the business. Impact and urgency are used to assign priority. It is a mandatory field. These urgencies are available out-of-box: • Critical • High • Average • Low
Priority	The order in which to address the request/order in comparison to others. The priority value is calculated using impact and urgency.
Reason	Select the reason for requesting the request/order: • Conversion • Customer Request • Legal • Maintenance • New • Problem Resolution
Ship To Location	The destination location the requested items should be shipped to.
Affected Service	Specifies the service that's affected by the request/order.
Affected CI	Specifies the CI that's affected by the request/order. Affected CI is one of the children of affected service.
Bill To Location	The location where the invoice should be mailed for the items shipped. Available locations are defined in System Administration > Base System Configuration > Locations.
Bill To Department	The department where the invoice should be sent for the items shipped. The departments available for selection are defined in **System Administration** > **Base System Configuration** > **Departments**.
Project ID	The identification number given to the project.
Escalated	If selected (set to true), this field indicates that the request needs to be escalated to Request Manager.

Request Manager	This field is only visible when “Escalated” is selected. It indicates the person who will handle the request escalation.
Source	The source from which the record is created. The following out-of-box sources are available: • 1 - User • 2 - Group • 3 - Event • 4 - Chat • 5 - Email • 6 - Telephone
Cost > Total Cost	Total cost incurred by fulfillment activities, including cost of part items and labor cost.
Cost > Currency	Specifies the currency of the total cost.
Cost > Date/Technician/Hours Worked	The table specifies when and who will work on the request/order for how many hours. The labor cost is calculated based on the hourly rate of the technician and the hours that he/she worked.
Summary > Closure Code	Specifies a predefined closure code to describe how the request/order has been fulfilled. These closure codes are available out-of-box: • Successful • Successful (with problems) • Failed • Rejected (financial) • Rejected (technical) • Rejected (security) • Withdrawn • Withdrawal requested by customer • Cancelled • Denied request fulfillment
Summary > Closure Comments	This field is to document additional comments to close the request/order.
Workflow section	Displays a figure of generic request workflow/order workflow. It indicates the current phase which the request/order is in, and traces the phase transition history.
Activities section	Records information that the operator enters during the lifecycle of the request/order. Every time you update a request/order, you can fill in an update on the Activities section (New Update) with or without “Visible to Customer” flagged. A log of all the updates is stored on the Journal Updates and activities list. Updates from user/requester also display here with activity type “Update from customer.”
Attachments section	You can use the Attachments section to attach documents to request/order.
Related Records section	Contains a list of all related records for the request/order. The related records in request form include interactions, changes, incidents and requests. The related records in order form include interactions.

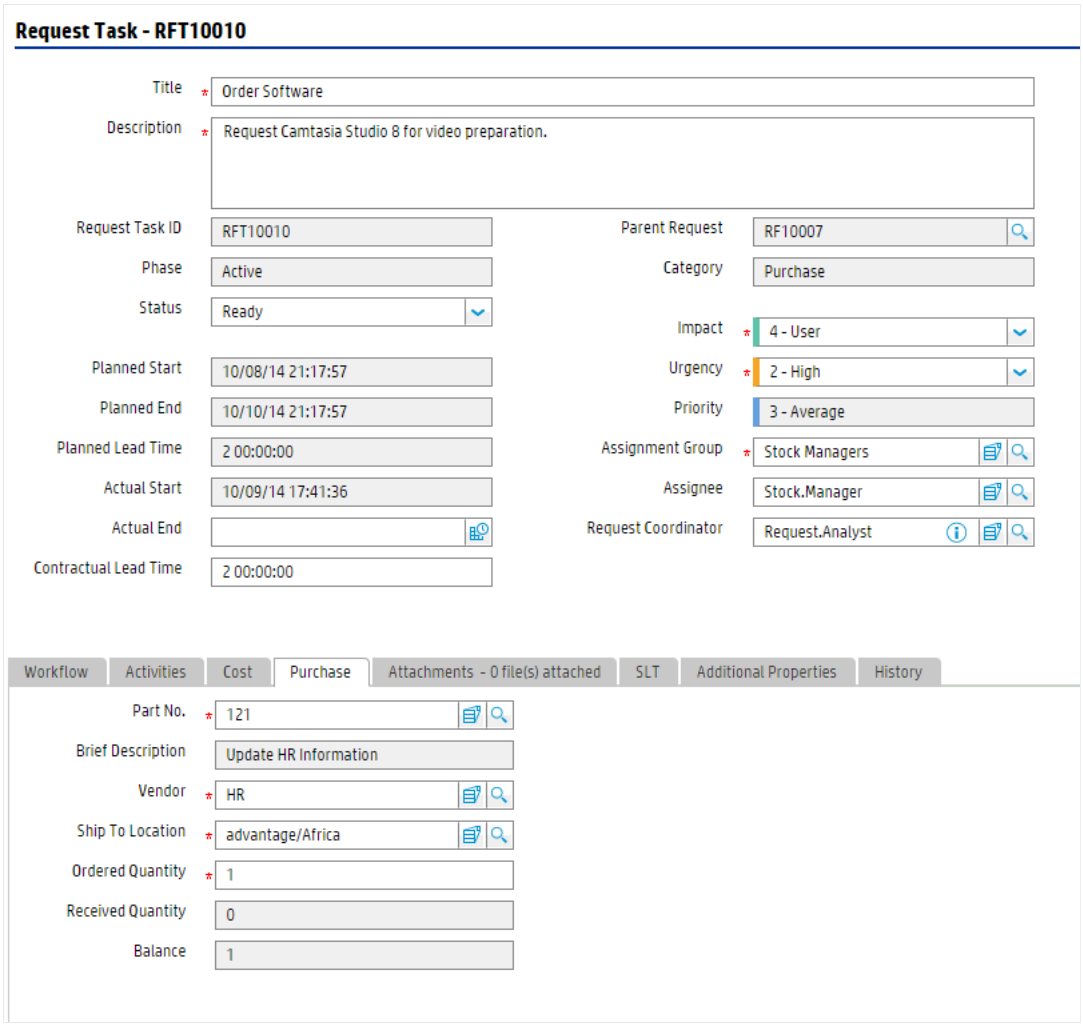
Approvals > Current Approvals	This subsection provides an overview of the current approvals related to request/order. The data displayed includes the following information: • Approval Type • Approval Status • # Approved • # Denied • # Pending
Approvals > Approval Log	This subsection provides an overview of the past approvals related to request/order. The data displayed includes the following information: • Action • Approver/Operator • By • Date/Time • Phase • ID
SLT section	The SLT (Service Level Target) section displays SLAs related to the request/order. The Service Level Targets subsection defines the Process Target details, such as beginning and ending state, and time allowed between these states. The Upcoming Alerts subsection displays the alerts to generate when processing the Process Target.
Tasks section	The planned/created request tasks are displayed in this section. Most of the tasks come from Request Model. However authorized operator (with "Edit Task Planner" right in "Request" area) is allowed to plan additional tasks via "Edit" button in the tasks section. To plan a new task, click the "Edit" button in tasks section, task editor is displayed for planning new tasks.
Task Context section	Displays the input and output parameters of all request tasks defined for the request/order.
History section	Displays the timestamps and operators that the request/order record is opened or updated by.

9.21.4. Request task forms

The following figure shows the common task form.

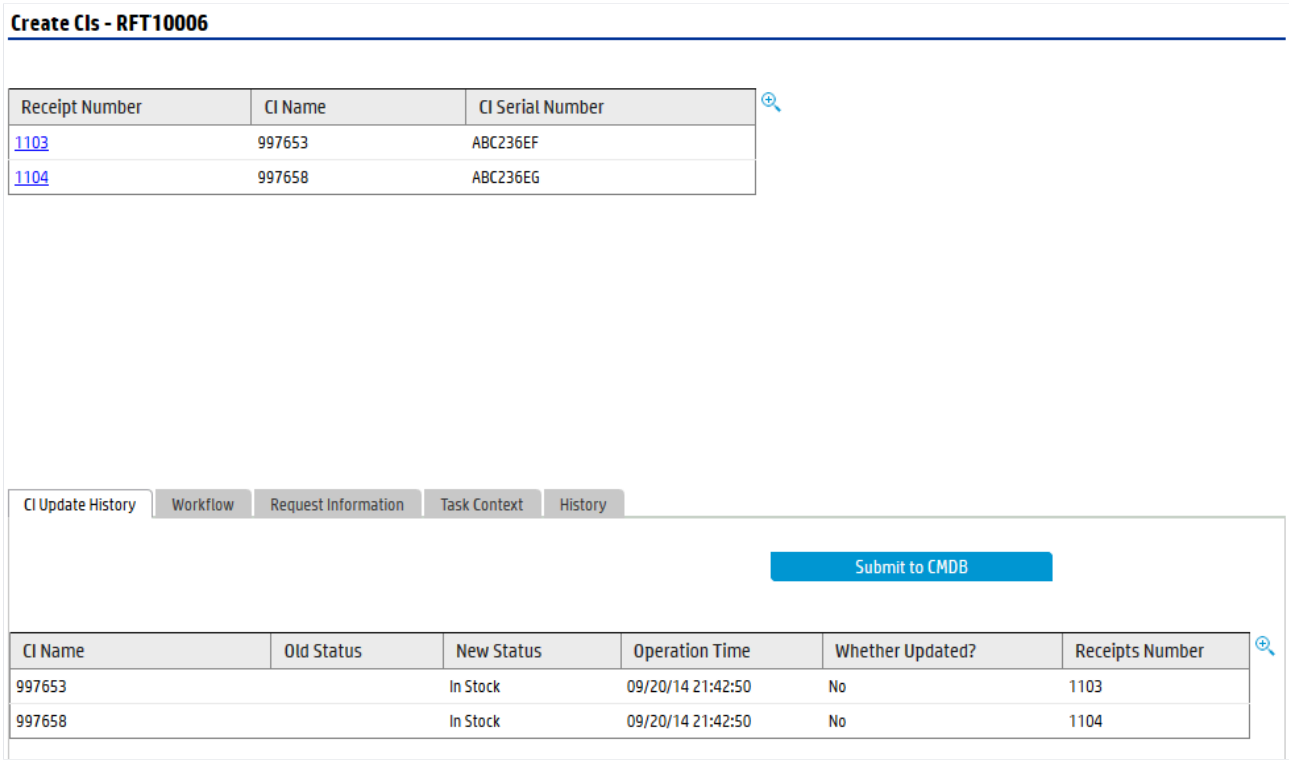


The following figure shows the purchase task form.

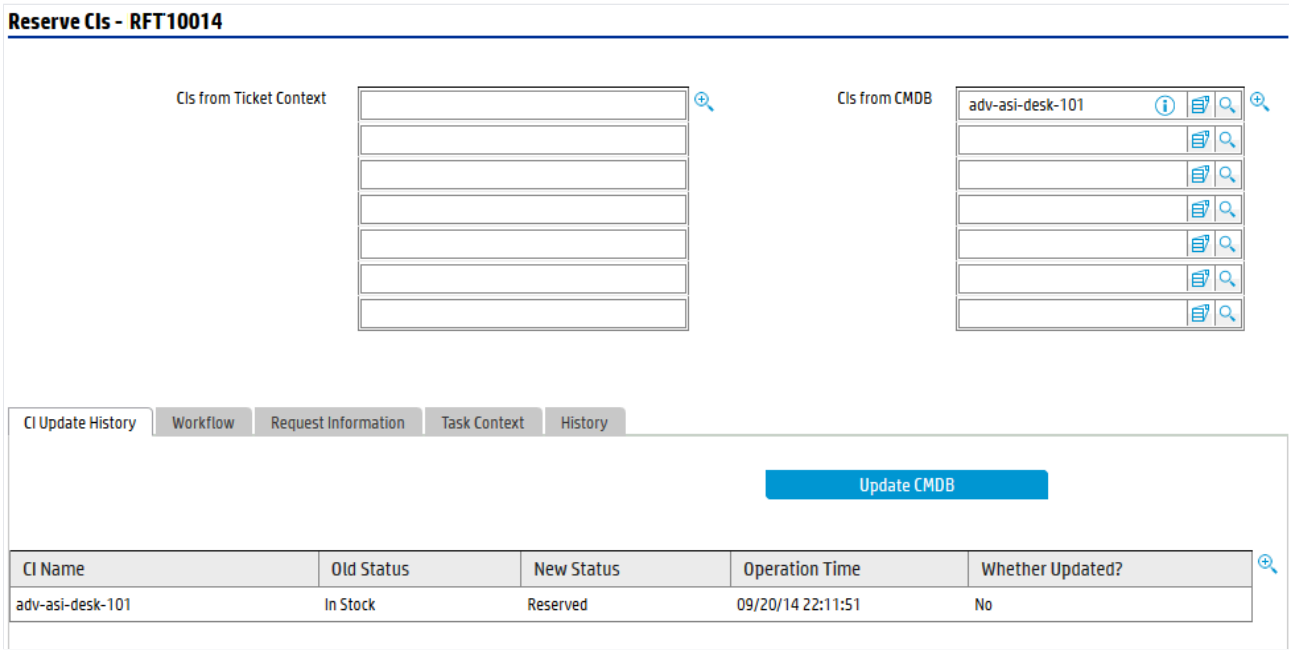


The following figures show different CMDB update forms for different request task categories.

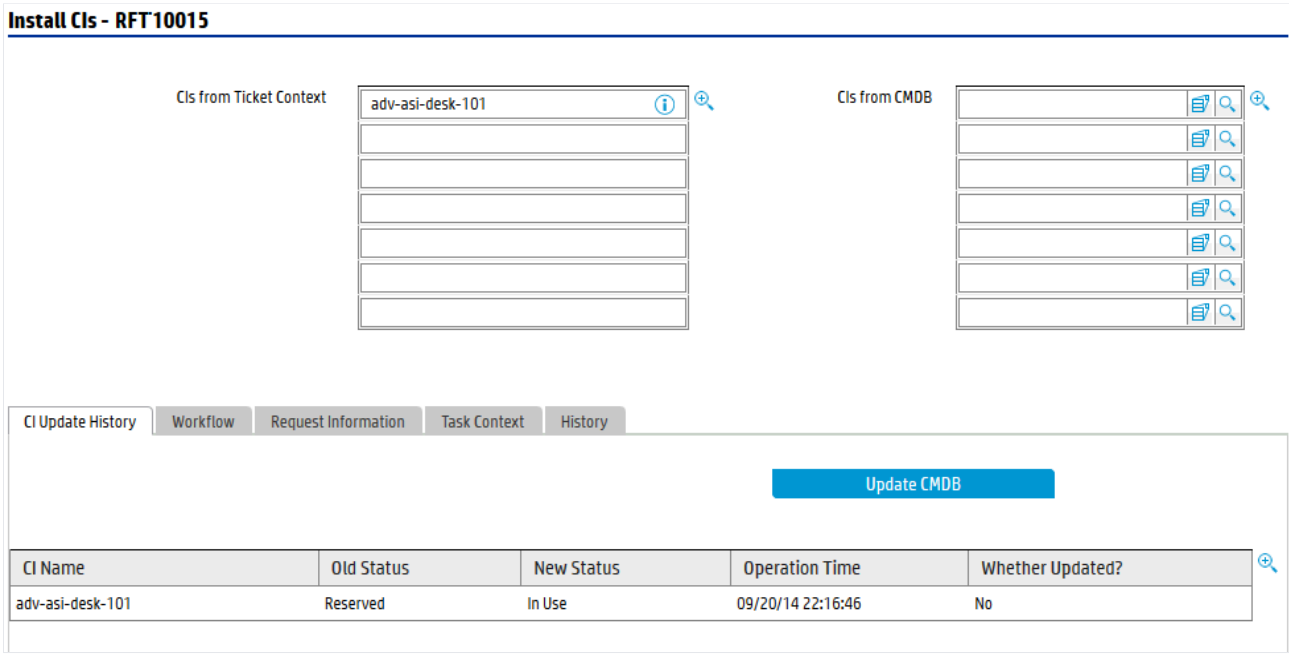
CMDB update form in “Purchase” task:



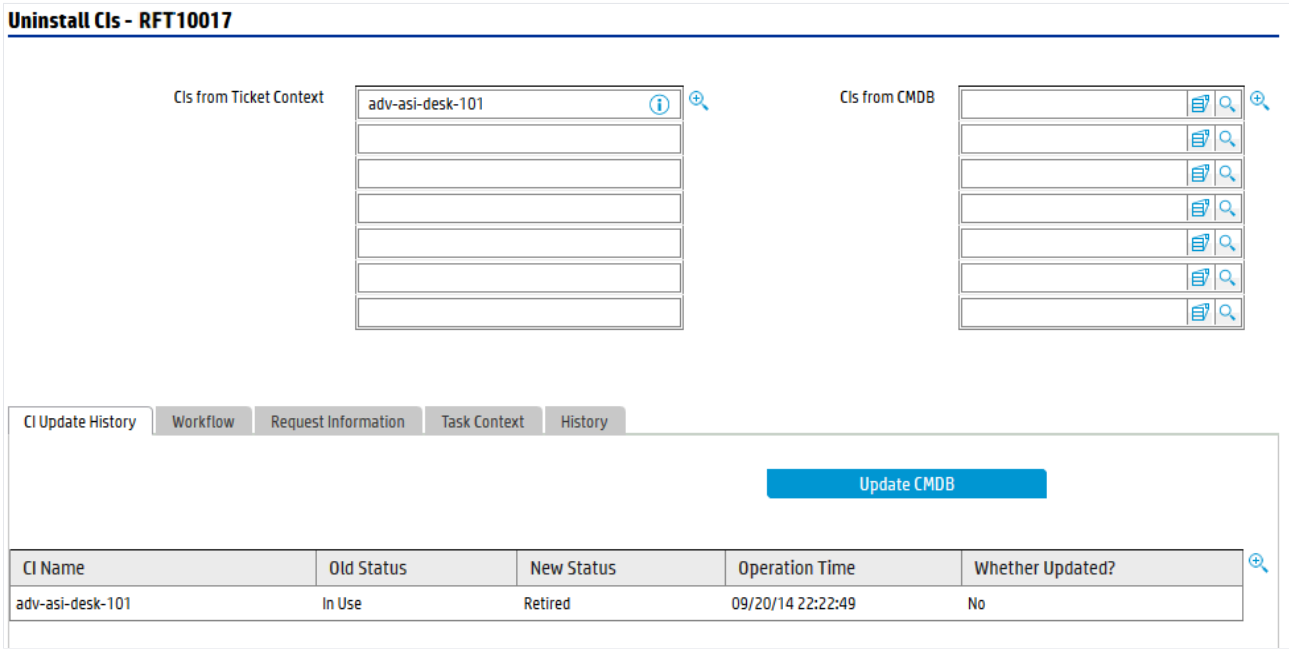
CMDB update form in “Reservation” task:



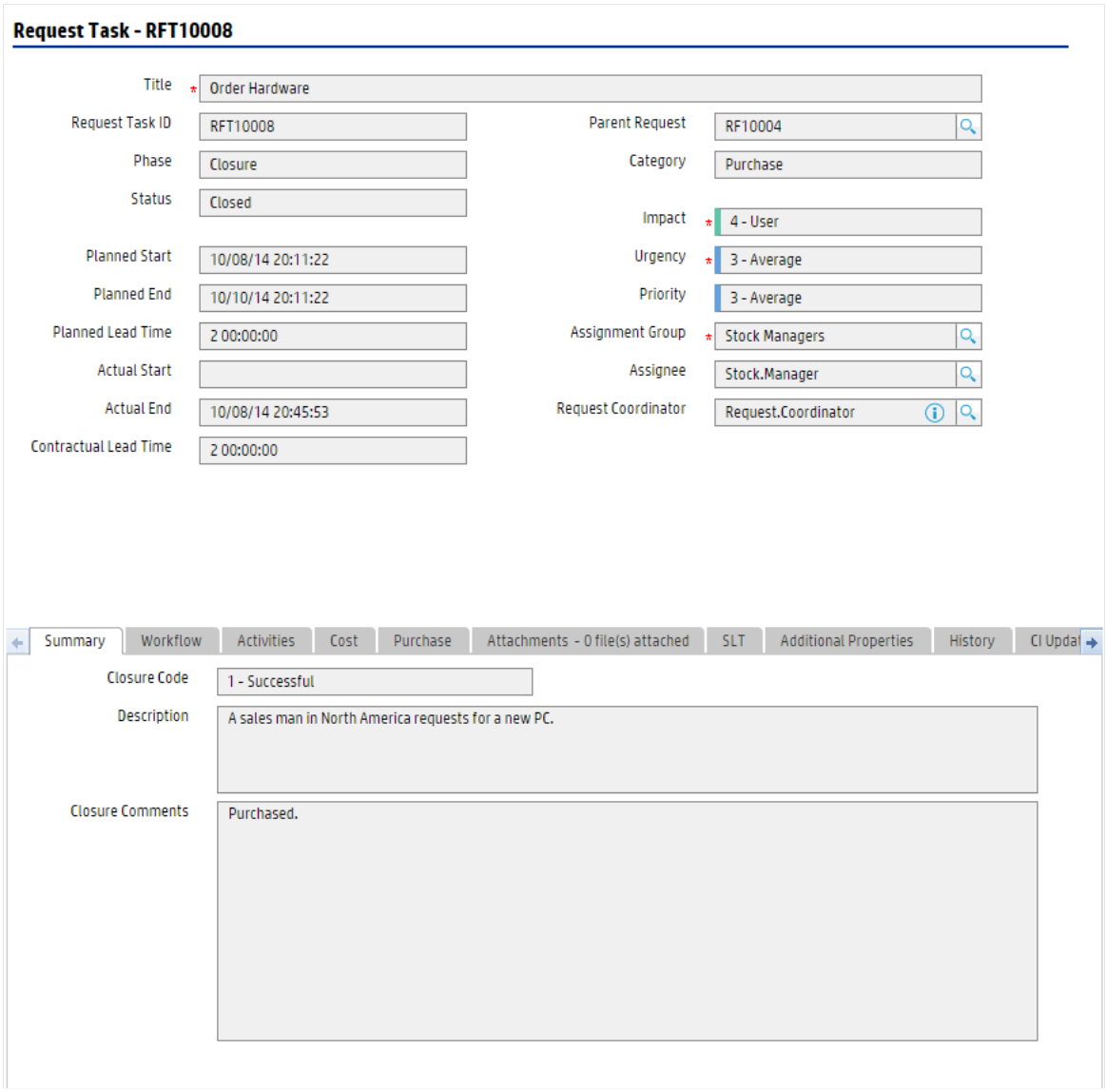
CMDB update form in “Installation” task:



CMDB update form in “Uninstallation” task:



The following figure shows the closed request task form.



9.21.5. Request task form details

The following table identifies and describes some of the features on the request task form.

Request task field descriptions

Label	Description
Request Task ID	The system-generated unique ID for this request task.
Title	A short description that summarizes the request task. It is a mandatory field.
Description	A detailed description of the request task. This field is prepopulated with data from parent request/order. It is a mandatory field.
Parent Request	Displays the unique ID of parent request/order.
Category	This field describes the type of request task. In out-of-box, the following categories are available: • Automation • Installation • Labor • Purchase • Reservation • Uninstallation
Phase	This is a system-generated field that specifies the name of the current phase of the request task. These phases are available in out-of-box workflows: • Waiting • Active • Review • Closure • Cancelled • CMDB Update (only available in "Purchase"/"Reservation"/"Installation"/"Uninstallation" task workflow) • Execution (only available in "Automation" task workflow) • Completion (only available in "Automation" task workflow)
Status	Displays the status of the request task. These statuses are available in out-of-box workflows: • Planned – Request task is opened but not activated. • Ready – Request task is activated. • In Progress – Request task is being handled by Request Analyst. • Pending Customer – Additional information is required from user/requester. • Pending Vendor/Supplier – Vendor/Supplier will handle this request task. • CMDB Update (Only available in "Purchase"/"Reservation"/"Installation"/"Uninstallation" task workflow) – CMDB is being updated. • Pending Review – High priority request task will be reviewed. • Cancelled – Request task is cancelled. • Closed – Request task is closed.

Impact	A measure of the effect of the request task on business processes. Impact and urgency are used to assign priority. These impacts are available out-of-box: • Enterprise • Site/Dept • Multiple Users • User This field is prepopulated with data from parent request/order. It is a mandatory field.
Urgency	A measure of how long it will be until a request task has a significant impact on the business. Impact and urgency are used to assign priority. These urgencies are available out-of-box: • Critical • High • Average • Low This field is prepopulated with data from parent request/order. It is a mandatory field.
Priority	The order in which to address the request task in comparison to others. The priority value is calculated using impact and urgency.
Assignment Group	The support group assigned to work on this request task. This field can be prepopulated by the data defined in assignment rule or manually set by Request Coordinator. It is a mandatory field.
Assignee	The person assigned to work on this request task. This person is a member of the assignment group. This field can be prepopulated by the data defined in assignment rule or manually set by Request Coordinator.
Request Coordinator	The name of the person responsible for coordinating the implementation of the parent request/order. Each Coordinator may belong to several assignment groups. Each group can have just one Request Coordinator.
Planned Start	If “Global Lead Time” is not zero in request or order, this field will be calculated starting from request “Delivery Date” and upstream from the last task to the first one in task planner, task dependency and planned lead time will be considered. In case “Global Lead Time” is zero in request or order, this field will be calculated from the “Expected Finish Date” instead of “Delivery Date”. This field is read-only in the request task form.
Planned End	If “Global Lead Time” is not zero in request or order, this field will be calculated starting from request “Delivery Date” and upstream from the last task to the first one in task planner, task dependency and planned lead time will be considered. In case “Global Lead Time” is zero in request or order, this field will be calculated from the “Expected Finish Date” instead of “Delivery Date”. This field is read-only in the request task form.
Planned Lead Time	Defines the duration time that the request task will spend. Data is prepopulated by the value defined in task planner of Request Model. This field is read-only.

Actual Start	It will be auto-populated when task is activated.
Actual End	It will be manually filled by Request Analyst when request task is done.
Contractual Lead Time	By default it equals to “Planned Lead Time”. However it is allowed to be adjusted by authorized operator according to real situation. Once it is changed, the “Planned Start” and “Planned End” will be re-calculated accordingly. The out-of-box request task SLT is still measured based on “Planned Lead Time”.
Cost > Total Cost	Total cost incurred by fulfillment activities. This field in “Purchase” task indicates the cost of received part items. This field in other tasks indicates the labor cost.
Cost > Currency	Specifies the currency of the total cost.
Cost > Date/Technician/Hours Worked	The table specifies when and who will work on the request task for how many hours. The labor cost is calculated based on the hourly rate of the technician and the hours spent by the technician.
Purchase > Part No.	The part no. of the purchased item listed in product catalog. This is a required field.
Purchase > Vendor	The vendor of the specified part item which is defined in the modelvendor table. This is a required field.
Purchase > Ship To Location	The destination location the purchased items should be shipped to. This is a required field.
Purchase > Ordered Quantity	The number of part items requested or ordered. This is a required field.
Purchase > Received Quantity	The number of received part items.
Purchase > Balance	Equals “Ordered Quantity” minus “Received Quantity”.
CI Update History > button Submit to CMDB/Update CMDB	The Request Analyst clicks this button to execute actual CI creation or CI status update. A confirmation message box will be displayed once this button is clicked. If “Yes” is selected in the message box, CI will be created or CI status will be updated. Otherwise, CMDB update action is abandoned.
CI Update History > CI Name/Old Status/New Status/Operation Time/Whether Updated?/Receipts Number	Shows the CMDB update result. New created CIs will be shown in “Purchase” task. Status updated CIs will be shown in “Reservation”/“Installation”/“Uninstallation” task.
Request Information > CartItem Id/Cart Item/Requested For/Quantity/Status/User Selection	Displays the request information provided by user/requester. The request information includes the items ordered from Service Catalog, the user selection details of the ordered items. The information in this table will be referenced by the operator who performs CMDB update.

Summary > Closure Code	Specifies a predefined closure code to describe how the request task has been fulfilled. These closure codes are available out-of-box: • Successful • Successful (with problems) • Failed • Rejected (financial) • Rejected (technical) • Rejected (security) • Withdrawn • Withdrawal requested by customer • Cancelled • Denied request fulfillment
Summary > Closure Comments	This field is to document additional comments to close the request task.
Workflow section	Displays a figure of request task workflow. It indicates the current phase which the request task is in, and traces the phase transition history.
Activities section	Records information that the operator enters during the lifecycle of the request task. Every time you update a request task, you can fill in an update on the Activities section (New Update) with or without "Visible to Customer" flagged. A log of all the updates is stored on the Journal Updates and activities list.
Attachments section	You can use the Attachments section to attach documents to request task.
Related Records section	Contains a list of all related records for the request task. The related records in request task form only include changes.
SLT section	The SLT (Service Level Target) section displays SLAs related to the request task. The Service Level Targets subsection defines the Process Target details, such as beginning and ending state, and time allowed between these states. The Upcoming Alerts subsection displays the alerts to generate when processing the Process Target.
Task Context section	Displays the input and output parameters of the request task.
History section	Displays the timestamps and operators that the request task is opened or updated by.

9.22. Service Level Management overview

You can use Service Level Management (SLM) to improve the quality of services that you provide to customers. You can also use Service Level Management to quantify the financial benefits in reduced incidents, outages, and time invested in system failures and downtime. Service Level Management collects performance information automatically to track service guarantees. Service Level Management enables you to achieve the following results:

- Ensure compliance with service and process targets set in the Service Level Agreement
- Report performance information to track the effectiveness and efficiency of managed services
- Detect and track failures of service guarantees
- Quantify costs associated with planned and unplanned service outages

Service Level Management uses the Service Level Targets (SLTs) to measure operational activities in other Service Manager applications.

This section describes how Service Level Management implements the best practice guidelines for the Service Level Management processes.

Topics in this section include:

Service Level Management within the ITIL Framework

Service Level Management Application

Input and output for Service Level Management

Key performance indicators for Service Level Management

Service Level Management Roles

RACI matrix for Service Level Management

9.22.1. Service Level Management within the ITIL Framework

The Service Level Management process is described in ITIL's Service Design publication.

The document describes Service Level Management as a vital process for every IT Service to document Service Level Targets and responsibilities within Service Agreements. The purpose of the SLM process is to ensure that all IT Services are delivered to agreed and achievable targets.

By using Service Level Management in Service Manager each IT Service Provider has a consistent interface to the business for all service level related data. The business gets up to date information about performance against the agreed service level targets. The operational teams are aligned to the agreed service targets, which will help to reduce breached targets. For breached targets, SLM provides the underlying data to facilitate the analysis of the cause of breaches. SLM provides the required tools to facilitate the prevention of breaches from re-occurrence. Furthermore, SLM provides reliable documentation features to efficiently communicate with the business .

9.22.2. Service Level Management Application

The impetus for a robust Service Level Management system is delivering value to the business by improving the quality of service , reduced number of incidents, and increased customer satisfaction. Over time, you can quantify the financial benefits in reduced incidents, outages, and time invested in system failures and downtime. The time and money recovered can be invested in improved customer relationships, more sophisticated monitoring, better training, and improved business efficiency.

Service management best practices describe the goals for Service Level Management (SLM) as follows:

The goal for SLM is to maintain and improve IT Service quality, through a constant cycle of agreeing, monitoring and reporting upon IT Service achievements and instigation of actions to eradicate poor service–in line with business or cost justification. Through these methods, a better relationship between IT and its customers can be developed.

Service Manager supports these goals by providing a service management best practices compliant application framework with a built-in workflow. The primary Service Level Management goals are the following:

- Tracking of Service Targets for Service and CIs, which includes planned and unplanned outages
- Proactive tracking of Process Targets, which tracks the amount of time it takes for the incident, service desk interaction, or change request to advance to the next state (For example, the amount of time required for an incident record status to change from Open to Work in Progress.)

Configuring Service Level Targets within SLA, OLAs or UCs, are the supporting elements which help Service Level Management to accomplish these goals.

9.22.3. Input and output for Service Level Management

A number of sources are relevant for the Service Level Management process.

Input	Output
Business information: from the organization's business strategy, plans, and financial plans and information on their current and future requirements	Service reports: providing details of the service levels achieved in relation to the targets contained within SLAs. These reports should contain details of all aspects of the service and its delivery, including current and historical performance, breaches and weaknesses, major events, changes planned, current and predicted workloads, customer feedback, and improvement plans and activities
Business Impact Analysis: providing information on the impact, priority, risk and number of users associated with each service	Service Improvement Plan (SIP): an overall program or plan of prioritized improvement actions, encompassing all services and all processes, together with associated impacts and risks
Business requirements: details of any agreed, new or changed business requirements	The Service Quality Plan (SQP): documenting and planning the overall improvement of service quality
The strategies, policies and constraints from Service Strategy	Service Level Agreements (SLAs): a set of targets and responsibilities should be documented and agreed within an SLA for each operational service
The Service Portfolio and Service Catalog	Service Level Requirements (SLRs): a set of targets and responsibilities should be documented and agreed within an SLR for each proposed new or changed service.
Change information: from the Change Management process with a forward schedule of changes and a need to assess all changes for their impact on all services	Operational Level Agreements (OLAs): a set of targets and responsibilities should be documented and agreed within an OLA for each internal support team
CMS: containing information on the relationships between the business services, the supporting services and the technology	Reports on OLAs and underpinning contracts
Customer and user feedback, complaints and compliments	Service review meeting minutes and actions: all meetings should be scheduled on a regular basis, with planned agendas and their discussions and actions recorded and progressed
Information and input from any of the other processes (for example, Incident Management, Capacity Management, and Availability Management), together with the existing SLAs, SLRs, and OLAs and past service reports on the quality of service delivered.	SLA review and service scope review meeting minutes: summarizing agreed actions and revisions to SLAs and service scope Revised contracts: changes to SLAs or new SLRs may require existing underpinning contracts to be changed, or new contracts to be negotiated and agreed.

9.22.4. Key performance indicators for Service Level Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your indicators for Service Level Management. To visualize trend information, it is useful to graph KPI data periodically. In addition to the data provided by Service Manager, you may need additional tools to report on all of your KPI requirements.

Title	Description
% of SLAs / OLAs / UCs that meet expectations	This KPI gives the user the ability to review the SLAs, OLAs and UCs, which met their target.
% of breached SLAs / OLAs / UCs	This KPI gives the user the ability to review the SLAs, OLAs and UCs, which did not met their target.

ITIL recommended KPIs[edit]

- Percentage reduction in SLA targets missed

- Percentage reduction in SLA targets threatened
- Percentage increase in customer perception and Satisfaction of SLA achievements, via service reviews and Customer Satisfaction Survey responses
- Percentage reduction in SLA breaches caused because of third-party support contracts (underpinning contracts)
- Percentage reduction in SLA breaches caused because of internal Operational Level Agreements (OLAs).

Deliver service as previously agreed at affordable costs:

- Total number and percentage increase in fully documented SLAs in place
- Percentage increase in SLAs agreed against operational services being run
- Percentage reduction in the costs associated with service provision
- Percentage reduction in the cost of monitoring and reporting of SLAs
- Percentage increase in the speed of developing and agreeing appropriate SLAs
- Frequency of service review meetings.

Manage business interface:

- Increased percentage of services covered by SLAs
- Documented and agreed SLM processes and procedures are in place
- Reduction in the time taken to respond to and implement SLA requests
- Increased percentage of SLA reviews completed on time
- Reduction in the percentage of outstanding SLAs for annual renegotiation
- Reduction in the percentage of SLAs requiring corrective changes (for example, targets not attainable; changes in usage levels). Care needs to be taken when using this KPI
- Percentage increase in the coverage of OLAs and third-party contracts in place, whilst possibly reducing the actual number of agreements (consolidation and centralization)
- Documentary evidence that'ssues raised at service and SLA reviews are being followed up and resolved
- Reduction in the number and severity of SLA breaches
- Effective review and follow-up of all SLA, OLA and underpinning contract breaches.

Contents

- 1 ITIL recommended KPIs

9.22.5. Service Level Management Roles

Role	Responsibilities
Service Level Management Process Owner	• Accountable for the definition, management, governance and improvement of the Service Level Management Process • Ensures that the Service Level Management process and working practices are effective and efficient • Ensures that all stakeholders are sufficiently involved in the Service Level Management process • Ensures that (business) management is sufficiently informed as to the volume, impact and cost of SLAs, OLAs and UCs • Ensures tight linkage between the Service Level Management process and other related processes
Service Level Manager	• Ensures that the customer's current and future Service requirements are identified, understood and documented in SLAs and SLRs • Drafts and ensures sign off of the SDD • Negotiates and agrees on the levels of Service to be delivered with the customer (either internal or external); formally documenting these levels of Service in SLAs • Negotiates and agrees to OLAs and, in some cases, other SLAs and agreements that underpin the SLAs • Ensures that targets agreed to within Underpinning Contracts are aligned with SLA and SLR targets • Ensures that Service Performance Reviews are regularly performed, and any required actions are performed • Ensures that improvement initiatives identified in Service Reviews are acted on and progress reports are provided to customers
Service Owner	• Provides the correct relationship information for the Services and CIs in the Service Catalog • Reports any discrepancies in the Service Catalog details and the actual environment into the Service Catalog maintenance process • Provides input for Service Catalog maintenance process improvement
Supplier Manager	• Produces requests for proposal in cooperation with all the stakeholders (Service Level Manager, Business Manager, etc) • Selects suppliers based on the evaluation of proposal they supply and feedback from stakeholders • Drafts and negotiates SLAs, Contracts, Agreements or any other documents for Third-Party Suppliers • Ensures the targets agreed within contracts are aligned with SLR targets • Ensures that SLA breaches of each supplier are highlighted, investigated and action(s) taken to prevent their recurrence

Service Level Analyst	• Ensures that Service reports are produced for each customer Service • Performs gap analysis to highlight breaches of SLA targets, investigating reasons and recommending actions to prevent their recurrence • Plans and schedules the Service Performance Review meetings and document the results and actions
Support Groups	Implement improvement initiatives identified in Service Reviews
Customer	Responsible for the purchase of goods or Services
User	Consumes goods or services

9.22.6. RACI matrix for Service Level Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. The RACI matrix for Service Level Management is shown in the table below.

Process ID	Activity	Service Level Manager	Supplier Manager	Service Owner	Customer	Service Level Analyst	Resolver Groups
SD 2.1	Maintain the Service Catalog	A/R		C	C		
SD 2.2	Create or Amend IT Services	A/R	R	R	C		
SD 2.3	Draft Supporting Agreements	A/R	R	R			
SD 2.4	Finalise SLAs / OLAs / UCs	A/R	R	R	C/I		
SD 2.5	Monitor Service Levels	A/R	I	I	C		
SD 2.6	Service Level Reporting	A/R				R	
SD 2.7	Perform Service Reviews	A/R			C	R	
SD 2.8	Service Improvement Programme	A/R		R	C		R
SD 2.9	Review / Revise SLA / OLA / UC	A/R				R	

9.23. Service Level Management workflows

The Service Level Management process includes all necessary steps to create and maintain Service offerings including the management of the following items:

- Service Level Agreements between business and IT
- Operational Level Agreements between IT and IT
- Underpinning contracts between IT and external providers.

In addition, Service Level Management process includes the activities such as:

- Service Level Reporting, including measurement of the service performance
- Production of service reports
- Conduction of service reviews with identification of improvement opportunities documented in the SIP or SQP
- The assessment of customer satisfaction with logging of complaints and compliments

The Service Level Management process can consist of the following activities:

Updating the Service Catalog (SD 2.1)

Create or Amend IT Services (SD 2.2)

SLRs and draft OLAs and UCs (SD 2.3)

Finalize SLRs/OLAs/UCs (SD 2.4)

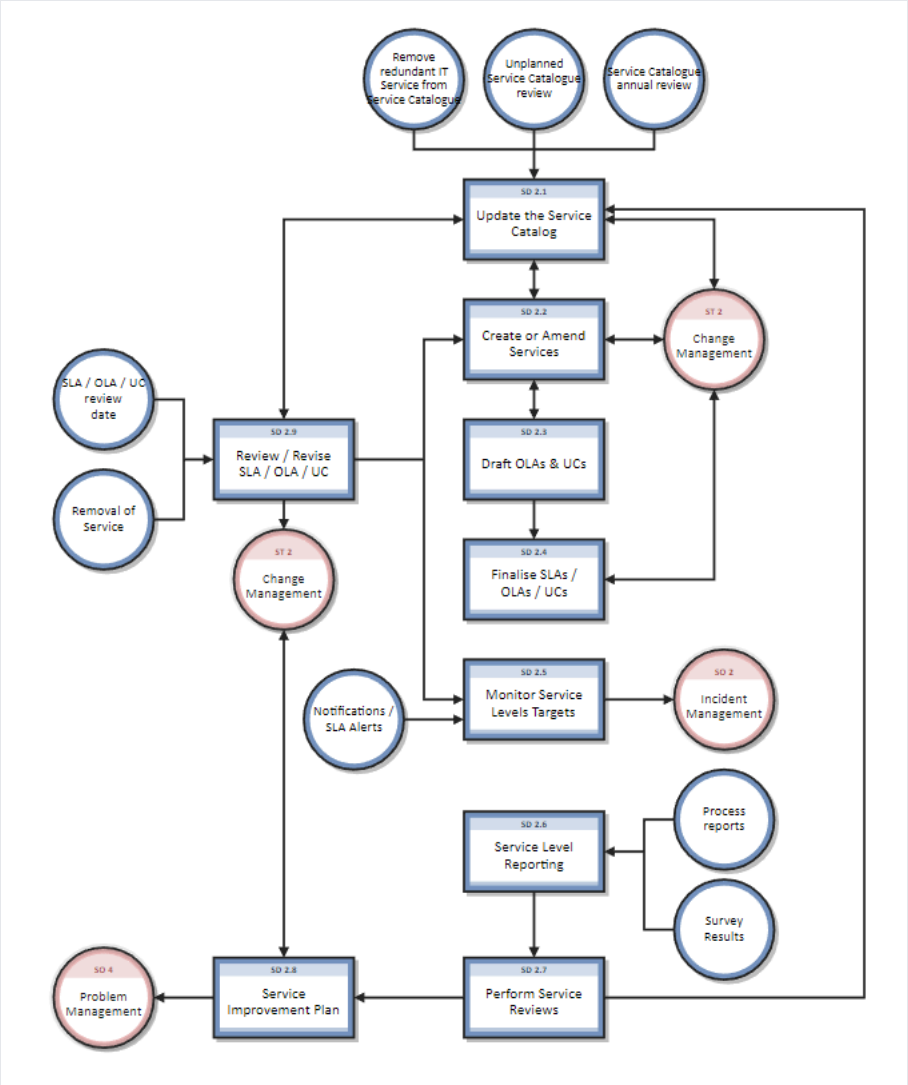
Monitor Service Levels (SD 2.5)

Service Level Reporting (SD 2.6)

Perform Service Reviews (SD 2.7)

Service Improvement Plan (SD 2.8)

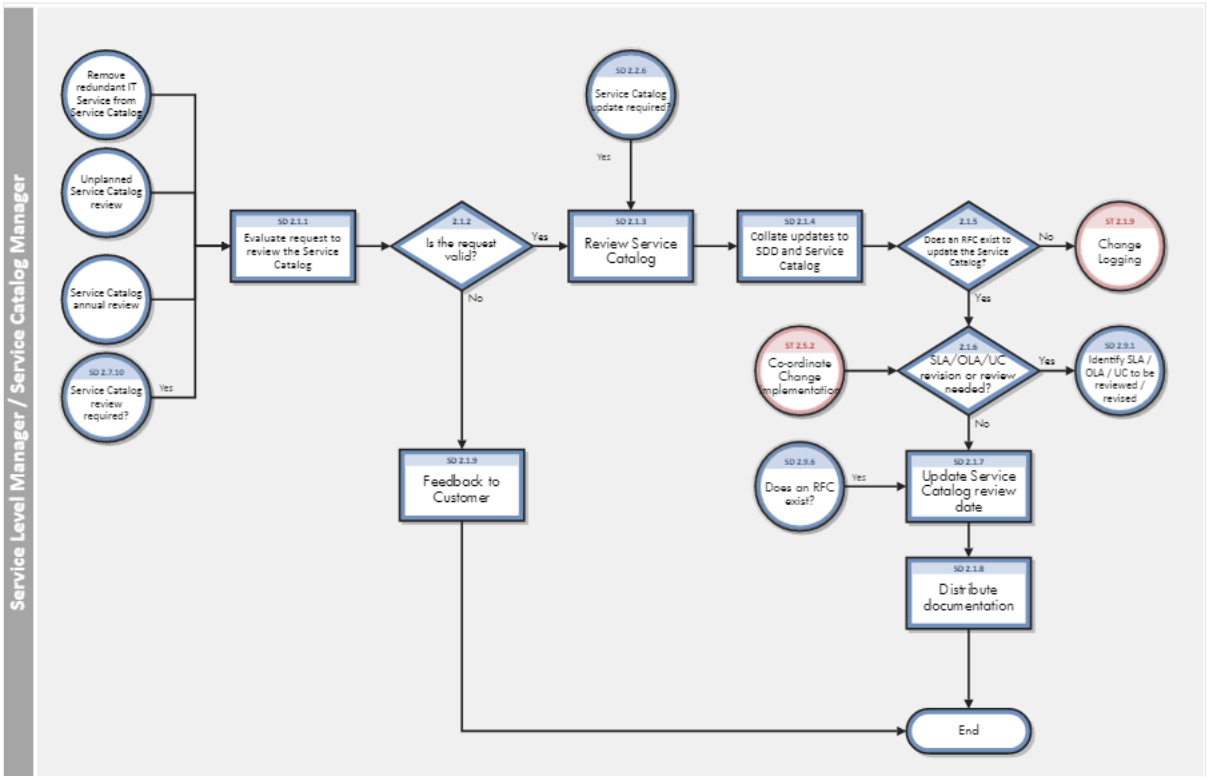
Review and Revise SLAs/OLAs/UCs (SD 2.9)



9.23.1. Updating the Service Catalog (SD 2.1)

Process ID	Procedure or Decision	Description	Role
SD 2.1.1	Evaluate request to review the Service Catalog	The Service Level Manager reviews the request and any relevant information to determine whether or not the request is valid. Go to SD 2.1.2 to determine whether the request is valid.	Service Level Manager
SD 2.1.2	Is the request valid?	If yes, go to SD 2.1.3 to review the Service Catalog. If no, go to SD 2.1.9 to feedback to the Customer.	Service Level Manager
SD 2.1.3	Review Service Catalog	Determine what changes may be required to the Service Catalog and SDD. The Service Owner and Customer may be involved in the review. Go to SD 2.1.4 to collate updates to the SDD and Service Catalog.	Service Level Manager
SD 2.1.4	Collate updates to SDD and Service Catalog	Compile updates to the SDD and Service Catalog. Go to SD 2.1.5 to determine whether an RFC exists.	Service Level Manager
SD 2.1.5	Does an RFC exist to update the Service Catalog?	If yes, go to SD 2.1.6 to determine whether the SLA / OLA / UC need to be revised or reviewed. If no, go to Change Logging (ST 2.1.9) to create a new Change record.	Service Level Manager
SD 2.1.6	SLA / OLA / UC Revision or review needed?	If yes, go to SD 2.9.1 to identify any agreements to be reviewed and/or revised. If no, go to SD 2.1.7 to update the Service Catalog review date.	Service Level Manager
SD 2.1.7	Update Service Catalog review date	Update the Service Catalog to show that it has been reviewed and record the next scheduled review date. Go to SD 2.1.8 to distribute documentation.	Service Level Manager

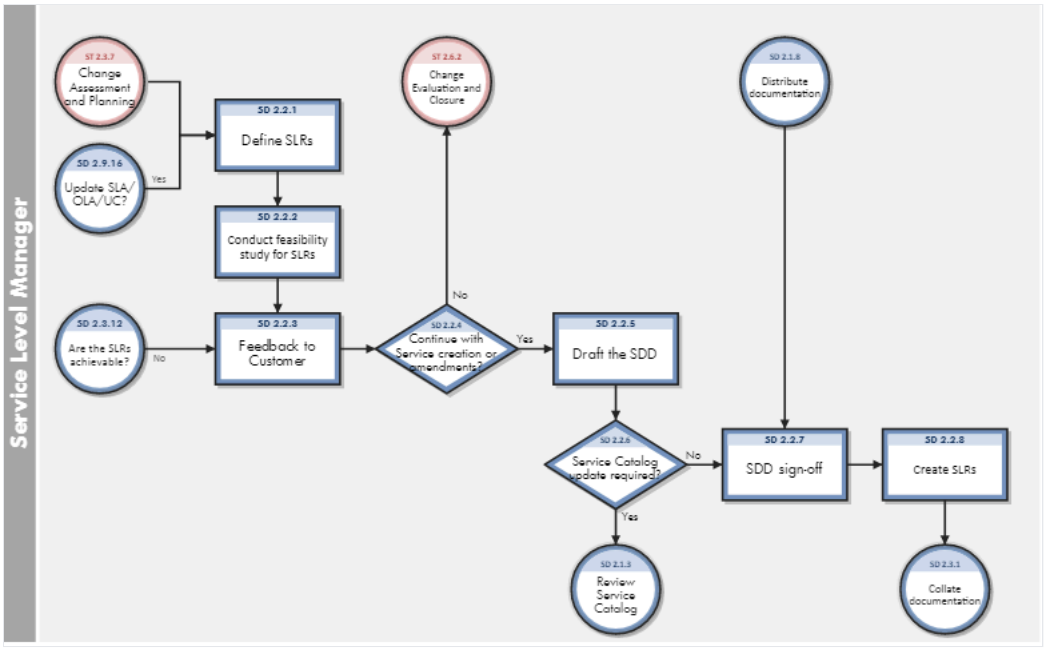
SD 2.1.8	Distribute documentation	Once the RFC to change the Service Catalog has been confirmed as successful, the new documents (including the SDD and Service Catalog) are circulated to the relevant stakeholders. The Maintain the Service Catalog process ends.	Service Level Manager
SD 2.1.9	Feedback to Customer	The customer is advised that the request to amend the Service Catalog is invalid and provided with the reason. The Maintain the Service Catalog process ends.	Service Level Manager



9.23.2. Create or Amend IT Services (SD 2.2)

Process ID	Procedure or Decision	Description	Role
SD 2.2.1	Define SLRs	Once a request is received for a new or amended IT Service, determine and agree the main requirements of the business. Go to SD 2.2.2 to conduct a feasibility study for the Service Level Requirements.	Service Level Manager
SD 2.2.2	Conduct feasibility study for SLRs	Initiate a feasibility study, ensuring that any existing OLAs and UCs are taken into consideration. The Supplier Manager may be involved with Service Providers if necessary. The study must consider all elements including technology, cost and Serviceability. Go to SD 2.2.3 to feedback to the Customer.	Service Level Manager
SD 2.2.3	Feedback to Customer	The results of the feasibility study are fed back to the Customer. Go to SD 2.2.4 to determine whether to continue with the Service creation or amendments.	Service Level Manager
SD 2.2.4	Continue with Service creation or amendments?	If yes, go to SD 2.2.5 to draft the SDD. If no, go to Change Evaluation and Closure (ST 2.6.2) to close the Change.	Service Level Manager

SD 2.2.5	Draft the SDD	Draft the SDD based on the data gathered and validated in the requirements definition and feasibility study. The SDD covers: • a description of the business process concerned • a detailed description of the technology and support platforms • links to related policies and Service Management procedures • plans to monitor and measure the Service Go to SD 2.2.6 to determine whether a Service Catalog update is required.	Service Level Manager
SD 2.2.6	Service Catalog update required?	If yes, go to SD 2.1.10 to review the Service Catalog. If no, go to SD 2.2.7 to sign-off the SDD.	Service Level Manager
SD 2.2.7	SDD sign-off	Validate the SDD with the Customer and then sign-off with the Service Owner and Service Level Manager. Once signed off, update the Service Catalog to reflect the new or amended Service. Go to SD 2.2.8 to finalize the SLRs.	Service Level Manager
SD 2.2.8	Create SLRs	The SLRs are created by the Service Level Manager and the Customer and are then collated into the SLR document.	Service Level Manager

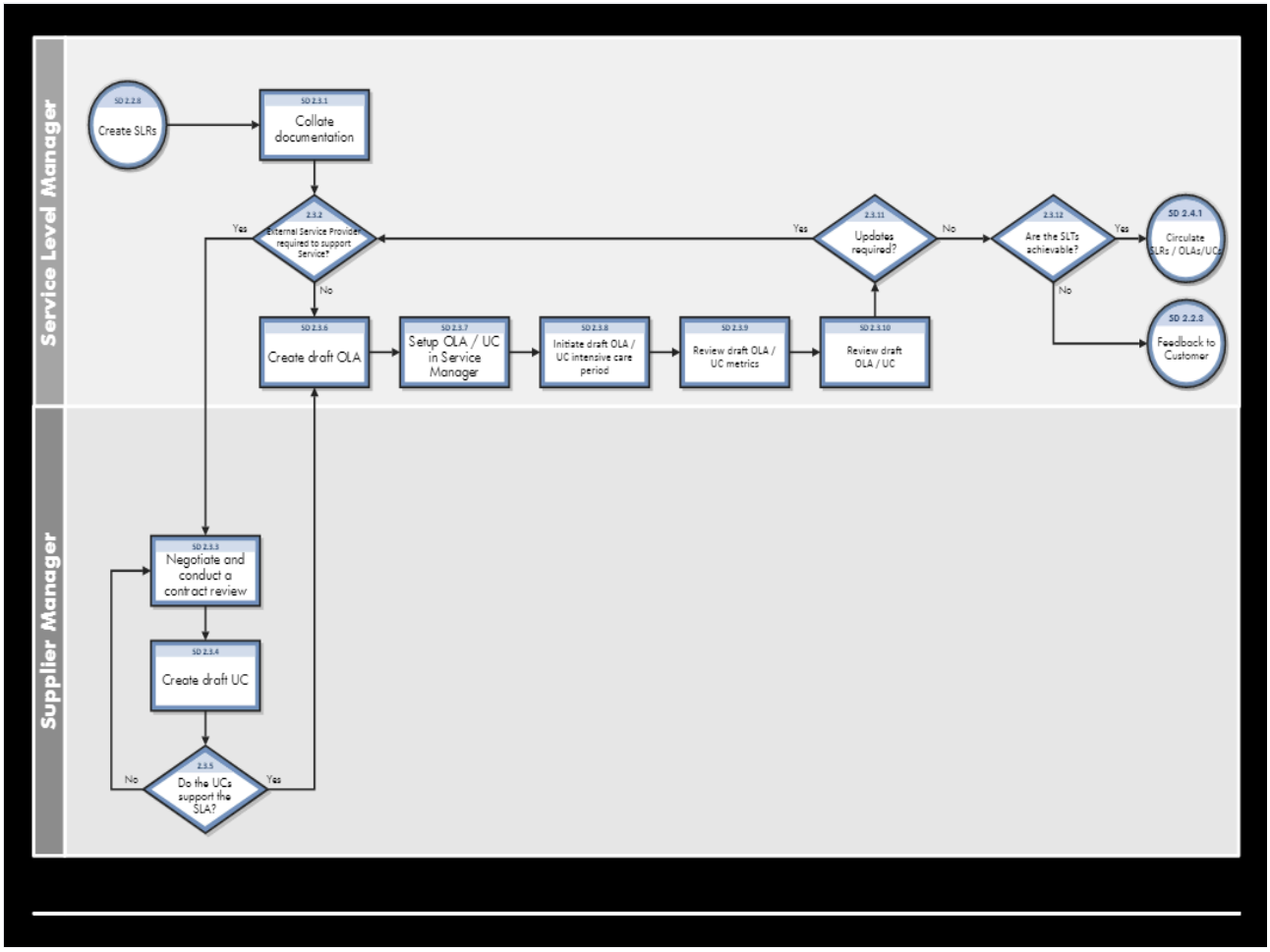


9.23.3. SLRs and draft OLAs and UCs (SD 2.3)

Process ID	Procedure or Decision	Description	Role
SD 2.3.1	Collate documentation	Compile supporting documentation such as existing OLAs and UCs, the Service Catalog, the SDD and SLRs documents. Go to SD 2.3.2 to determine whether the Service Provider is required to support the Service.	Service Level Manager
SD 2.3.2	External Service Provider required to support Service?	If yes, go to SD 2.3.3 to negotiate with the Service Provider and conduct a contract review. If no, go to SD 2.3.6 to create the draft OLA.	Service Level Manager
SD 2.3.3	Negotiate and conduct a contract review.	Contact the Service Provider(s) to discuss how the SLRs will be supported. If contracts are already in place it may not be possible to renegotiate, and this will need to be taken into account when the SLA is drafted. Go to SD 2.3.4 to create the draft UC.	Supplier Manager
SD 2.3.4	Create draft UC	The UC is drafted following the contract review and negotiation. Go to SD 2.3.5 to determine whether the UCs support the SLA.	Supplier Manager
SD 2.3.5	Do the UCs support the SLA?	If the UCs do support the SLRs, go to SD 2.3.6 to create the draft OLA. If no, go to SD 2.3.3 to continue negotiation and contract review.	Supplier Manager
SD 2.3.6	Create draft OLA	Review the SLRs, the SDD and the Service Catalog to determine the requirements for the OLA with the Service Owner or Supplier Manager. Determine the OLA intensive care start date and duration. Go to SD 2.3.7 to setup the OLA/UC in Service Manager.	Service Level Manager

SD 2.3.7	Setup OLA / UC in Service Manager	The Service Level Manager will ensure that the OLA / UC is set up in Service Manager by liaising with the Service Manager Admin team. Go to SD 2.3.8 to initiate the draft OLA / UC intensive care period.	Service Level Manager
SD 2.3.8	Initiate draft OLA / UC intensive care period	Monitor the performance of the OLA / UC against the draft OLA / UC metrics using data from other processes, such as Incident and Problem Management. Go to SD 2.3.9 to review the draft OLA / UC metrics.	Service Level Manager
SD 2.3.9	Review draft OLA / UC metrics	Once the OLA intensive care period is complete, conduct a Service Review meeting to: • determine if achievement meets expectations of the draft OLA / UC • determine whether the SLTs in the SLRs are achievable • determine whether the draft OLA / UC are 'fit for purpose' • identify amendments to the draft OLAs / UCs prior to converting these into formal OLA / UC metrics • identify gaps between the required Service and current Service capabilities and consider additional resource requirements to bridge them Go to SD 2.3.10 to review the draft OLA / UC.	Service Level Manager
SD 2.3.10	Review draft OLA / UC	The drafts OLAs / UCs are reviewed to ensure they meet the requirements and support the new SLRs. Go to SD 2.3.11 to determine whether updates are required.	Service Level Manager

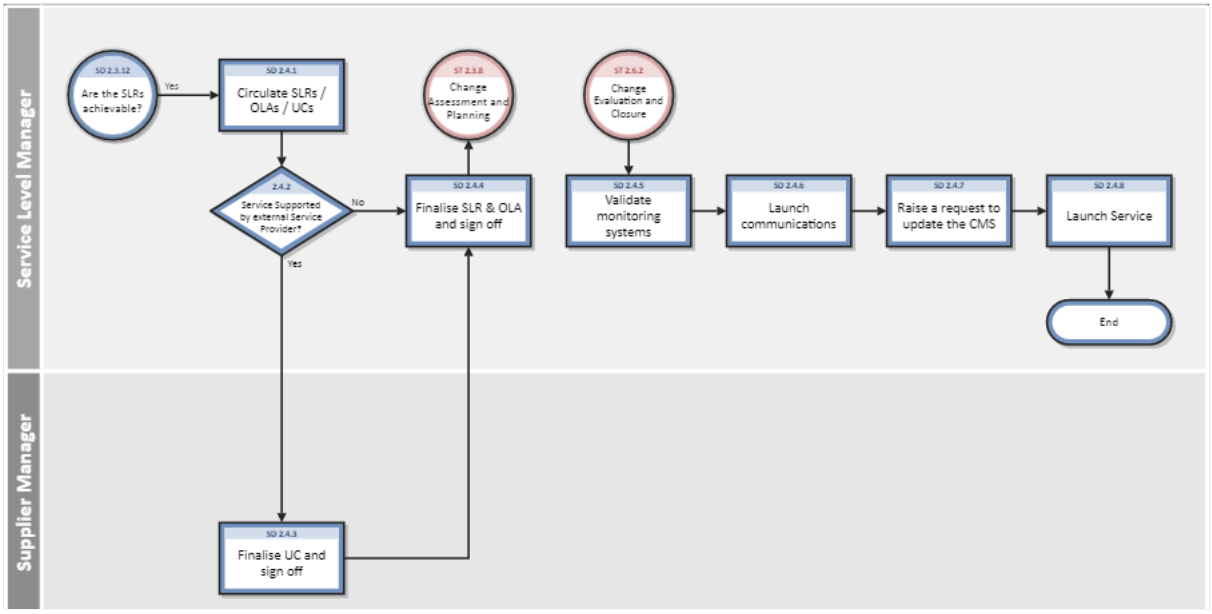
SD 2.3.11	Updates required?	If updates are required to the draft agreements based on the review, go to SD 2.3.2 to repeat the process If no, go to SD 2.3.12 to determine whether the SLTs in the SLRs are achievable.	Service Level Manager
SD 2.3.12	Are the SLTs achievable?	If yes, go SD 2.4.1 to circulate the draft agreements. If no, go to SD 2.2.3 to feedback to the customer.	Service Level Manager



9.23.4. Finalize SLRs/OLAs/UCs (SD 2.4)

Process ID	Procedure or Decision	Description	Role
SD 2.4.1	Circulate SLRs / OLAs / UCs	The SLRs, OLAs and UCs are circulated to the relevant audience as pre-reading before the final confirmation and sign off meeting. Go to SD 2.4.2 to determine whether the Service will be supported by a Service Provider.	Service Level Manager
SD 2.4.2	Service supported by an external Service Provider?	If yes, go to SD 2.4.3 finalize and sign off the UC. If no, go to SD 2.4.4 to finalize the SLR and OLA, and sign off.	Service Level Manager
SD 2.4.3	Finalize UC and sign off	The UCs are finalized to ensure they can support the SLR and are signed off by the Supplier Manager and the Service Provider. Go to SD 2.4.4 to finalize and sign-off the SLR and OLA.	Supplier Manager
SD 2.4.4	Finalize SLR & OLA and sign-off	The Service Level Manager finalizes and signs off the SLR and OLA with the Customer, Service Owner and Supplier Manager. As part of the SLR discussion a launch date for the Service is agreed which takes into account how much time is needed to update the Service and the Service Manager. Go to Change Assessment and Planning (ST 2.3.8) to plan and schedule the SLA and OLA Change. The Finalize SLRs / OLAs / UCs process is temporarily stopped pending implementation of the Change.	Service Level Manager
SD 2.4.5	Validate monitoring systems	Once the Change has been confirmed as successful, the monitoring systems are re-validated to ensure they accurately monitor and manage the service level targets. Go to SD 2.4.6 to launch communications.	Service Level Manager

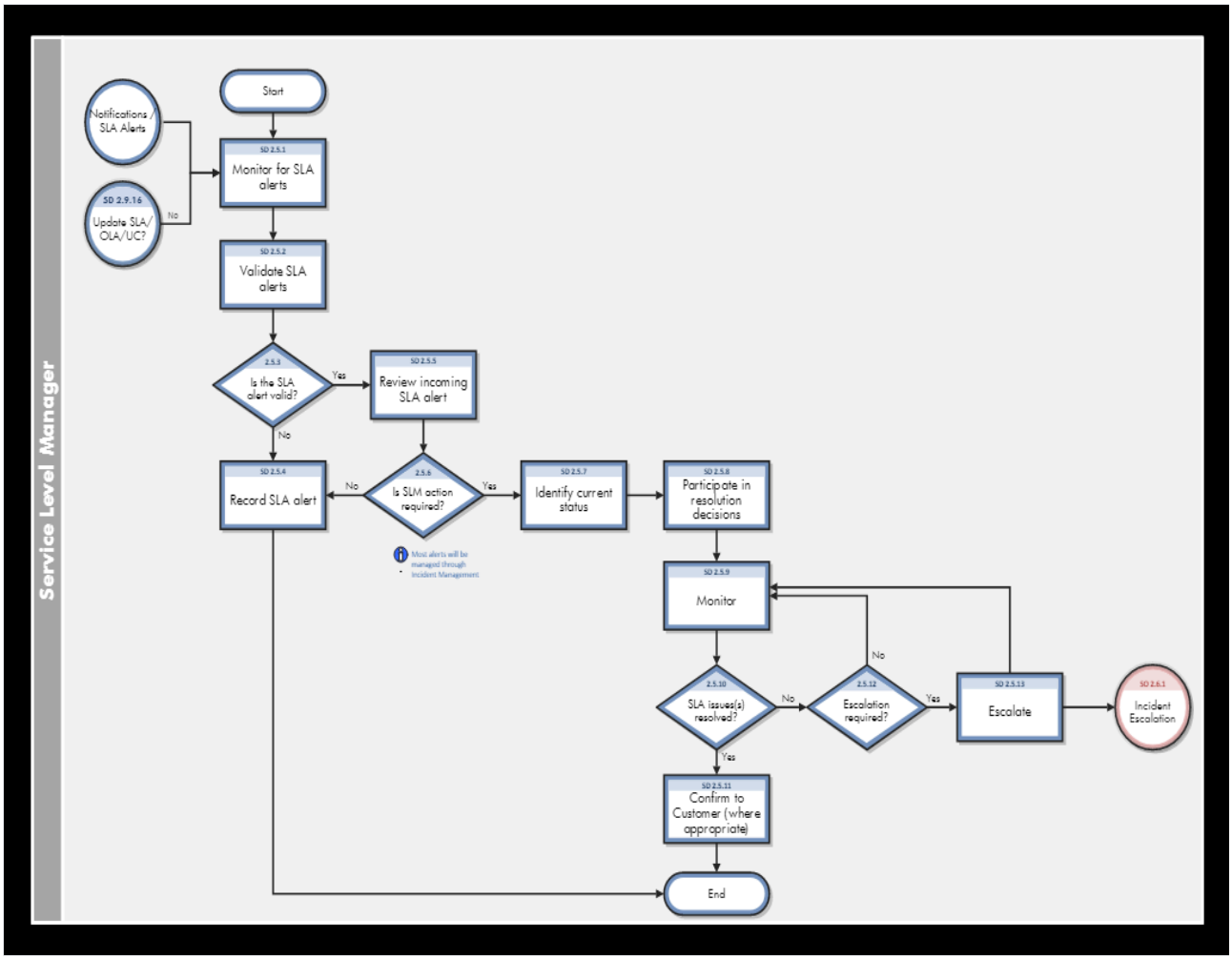
SD 2.4.6	Launch communications	Initiate communications to the end-user community and the delivery teams, including the Service Desk. Go to SD 2.4.7 to raise a request to update the CMS.	Service Level Manager
SD 2.4.7	Raise a request to update the CMS	Raise a request to update the CMS for the new or amended Service. Go to SD 2.4.8 to launch the Service.	Service Level Manager
SD 2.4.8	Launch Service	The Service is launched and the following elements become "Live": • Agreed Service Level parameters automated in the Service Management system • Monitoring of Service delivery through the Service Management system The Finalize SLAs/OLAs/UCs process ends.	Service Level Manager



9.23.5. Monitor Service Levels (SD 2.5)

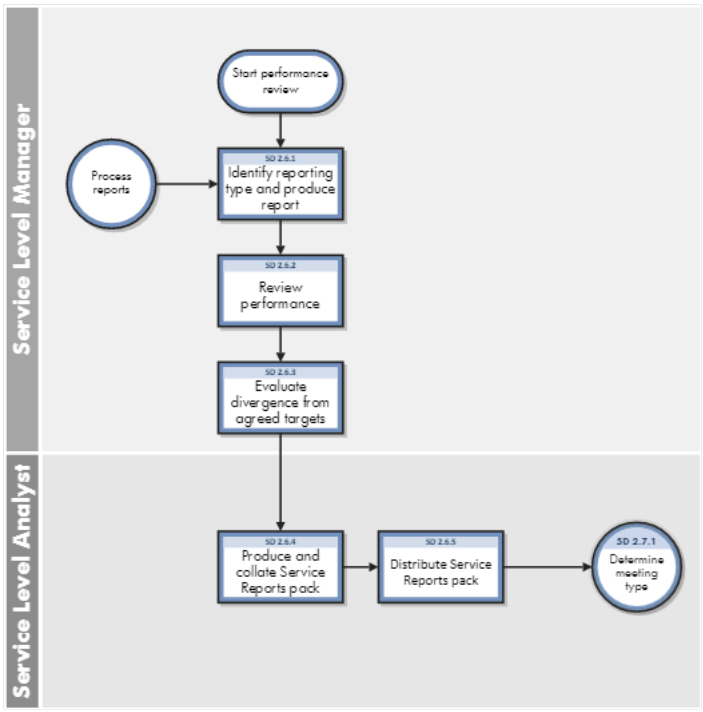
Process ID	Procedure or Decision	Description	Role
SD 2.5.1	Monitor for SLA alerts	The majority of day-to-day issues will be handled by the Service Desk and those Service Providers involved in delivering the Service/s. The SLM team will be notified of significant breaches of Service Level targets,that is, urgent P1 and major Incidents. Go to SD 2.5.2 to validate SLA alerts.	Service Level Manager
SD 2.5.2	Validate SLA alerts	Once an automated SLA alert is received its validity / relevance is assessed with agreed escalation points contained within the SLA. Go to SD 2.5.3 to determine whether the SLA alert is valid.	Service Level Manager
SD 2.5.3	Is the SLA alert valid?	If yes, go to SD 2.5.5 to review the incoming SLA alert. If no, go to SD 2.5.4 to record the SLA alert.	Service Level Manager
SD 2.5.4	Record SLA alert	The Service Level Manager records the invalid SLA Alert for future review of alert criteria. The Monitor Service Levels process ends.	Service Level Manager
SD 2.5.5	Review incoming SLA alert	The SLA alert is reviewed to determine: • associated Service and SLA • importance level of escalation • Incident/Problem/Major Incident record number • Support groups Go to SD 2.5.6 to determine whether SLA action is required.	Service Level Manager
SD 2.5.6	Is SLM action required?	If yes, go to SD 2.5.7 to identify the current SLA alert status. If no, go to SD 2.5.4 to record the SLA alert.	Service Level Manager

SD 2.5.7	Identify current status	Gather additional information about the current status of the Incident generating the alert. Go to SD 2.5.8 to participate in resolution decisions.	Service Level Manager
SD 2.5.8	Participate in resolution decisions	The Service Level Manager participates in decisions regarding the ongoing management of the Incident and possible resolution activities. This ensures that the Customer is kept informed of progress against the SLA and that the Service Owners and Service Providers are kept aware of the ongoing impact to the business. Go to SD 2.5.9 to monitor the Incident.	Service Level Manager
SD 2.5.9	Monitor	Periodically review the Incident to ensure that progress is being made and that agreed actions have been completed. Go to SD 2.5.10 to determine whether the SLA issue(s) is resolved.	Service Level Manager
SD 2.5.10	SLA Issue(s) resolved?	If the reason for the SLA alert has been resolved, go to SD 2.5.11 to confirm with the Customer (where appropriate). If no, go to SD 2.5.12 to determine whether escalation is required.	Service Level Manager
SD 2.5.11	Confirm to Customer (where appropriate)	Confirm the resolution with the Customer. The Monitor Service Levels process ends.	Service Level Manager
SD 2.5.12	Escalation required?	If yes, go to SD 2.5.13 to perform Incident escalation. If no, go to SD 2.5.9 to monitor the Incident.	Service Level Manager
SD 2.5.13	Escalation	If the performance is nearing its target, the Incident should be escalated via Incident Management. Go to Incident Escalation (SO 2.6.1) to determine how to resolve the Incident.	Service Level Manager



9.23.6. Service Level Reporting (SD 2.6)

Process ID	Procedure or Decision	Description	Role
SD 2.6.1	Identify reporting type and produce report	Reporting requirements are identified and reports produced based on: • Service metrics • SLAs • Service Catalog • Service Improvement updates Go to SD 2.6.2 to review	Service Level Manager
SD 2.6.2	Review performance	Review metrics available for Service Performance, including Incident resolution and response times. The metrics are validated for accuracy. Go to SD 2.6.3 to evaluate divergence from agreed targets.	Service Level Manager
SD 2.6.3	Evaluate divergence from agreed targets	The metrics are compared to the service targets for each live SLA. Go to SD 2.6.4 in order for the Service Level Analyst to produce and collate the Service Reports pack.	Service Level Manager
SD 2.6.4	Produce and collate Service Reports pack	Record findings and document within the Service Reports pack according to the structure agreed within the SLA. Go to SD 2.6.5 to distribute the Service Reports pack.	Service Level Analyst
SD 2.6.5	Distribute Service Reports pack	The completed Service Report packs are distributed to the Service Level Managers and Customers. Go to SD 2.7.1 in order for the Service Level Manager to determine the meeting type for the Service Review.	Service Level Analyst

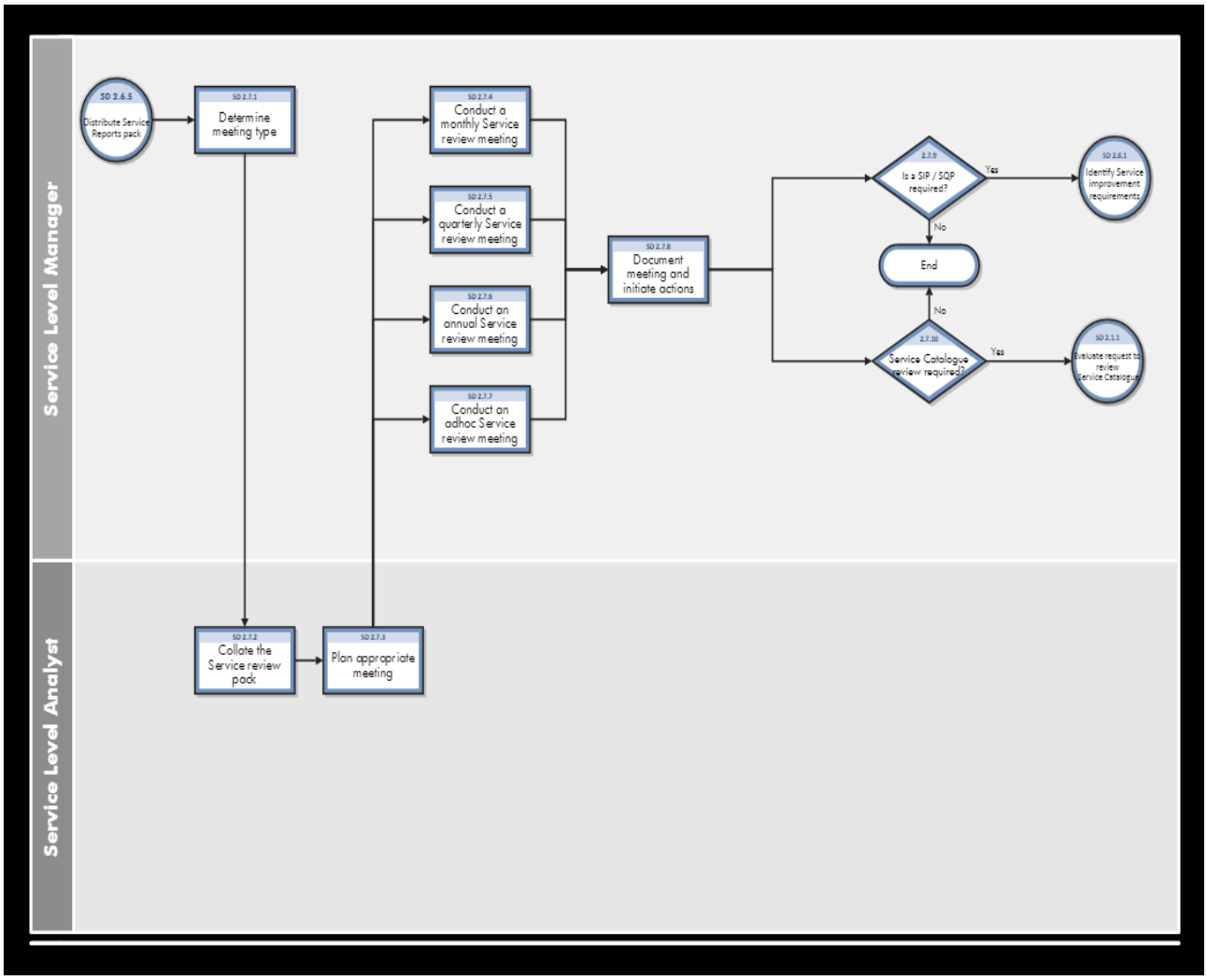


9.23.7. Perform Service Reviews (SD 2.7)

Process ID	Procedure or Decision	Description	Role
SD 2.7.1	Determine meeting type	Determine the appropriate meeting type from the agreed meeting schedule. Go to SD 2.7.2 to collate the Service review pack.	Service Level Manager
SD 2.7.2	Collate the Service review pack	Collate the Service review pack, including: • Meeting agenda • Previous minutes and outstanding actions • Service metrics • SIP progress • Informal and ad-hoc Customer feedback Key inputs should also be obtained from other Service Management processes and related documents. Go to SD 2.7.3 to plan the appropriate meeting.	Service Level Analyst
SD 2.7.3	Plan appropriate meeting	Determine whether the Service review is a monthly, quarterly, annual or ad-hoc meeting. Set the date and location of meeting and circulate the agenda to the appropriate audience. Go to one of the appropriate steps: • SD 2.7.4 to conduct a monthly Service Review meeting • SD 2.7.5 to conduct a quarterly Service Review meeting • SD 2.7.6 to conduct an annual Service Review meeting • SD 2.7.7 to conduct an ad-hoc Service Review meeting	Service Level Analyst

SD 2.7.4	Conduct a monthly Service review meeting	The purpose of this meeting is to understand and manage operational issues. The Customer and Service Level Manager will discuss: • requests for new and amended Services • updates to existing Service provision • progress on ongoing SIPs • Changing business requirements which may impact the current IT Services provided Go to SD 2.7.8 to document the meeting and initiate actions.	Service Level Manager
SD 2.7.5	Conduct a quarterly Service review meeting	This meeting follows the same agenda as the monthly Service Review meeting but with an additional item to cover more strategic issues. Go to SD 2.7.8 to document the meeting and initiate actions.	Service Level Manager
SD 2.7.6	Conduct an annual Service review meeting	This meeting is specifically designed to review the overall operation of the IT Service Management process in conjunction with the Customer. This will be an opportunity to review the SLAs that are included in the Service review pack allowing all parties to re-confirm that the IT Services are meeting the Customer's requirements and to discuss any future SLRs. Go to SD 2.7.8 to document the meeting and initiate actions.	Service Level Manager

SD 2.7.7	Conduct an adhoc Service review meeting	Both the Service Level Manager and Customer are entitled to call an ad-hoc Service Review meeting without having to wait for the next scheduled Service Review, in order to cover specific issues that may have arisen. Go to SD 2.7.8 to document the meeting and initiate actions.	Service Level Manager
SD 2.7.8	Document meeting & initiate actions	Document and issue minutes. There are 2 parallel activities from this activity. Go to SD 2.7.9 to determine whether a Service Improvement Plan (SIP) is required. Go to SD 2.7.10 to determine whether a Service Catalog review is required.	Service Level Manager
SD 2.7.9	Is an SIP/SQP required?	If yes, go to SD 2.8.1 to identify Service improvement requirements. If no, the Perform Service Reviews process ends.	Service Level Manager
SD 2.7.10	Service Catalog review required?	If yes, go to SD 2.1.1 to evaluate the request to review the Service Catalog. If no, the Perform Service Reviews process ends.	Service Level Manager



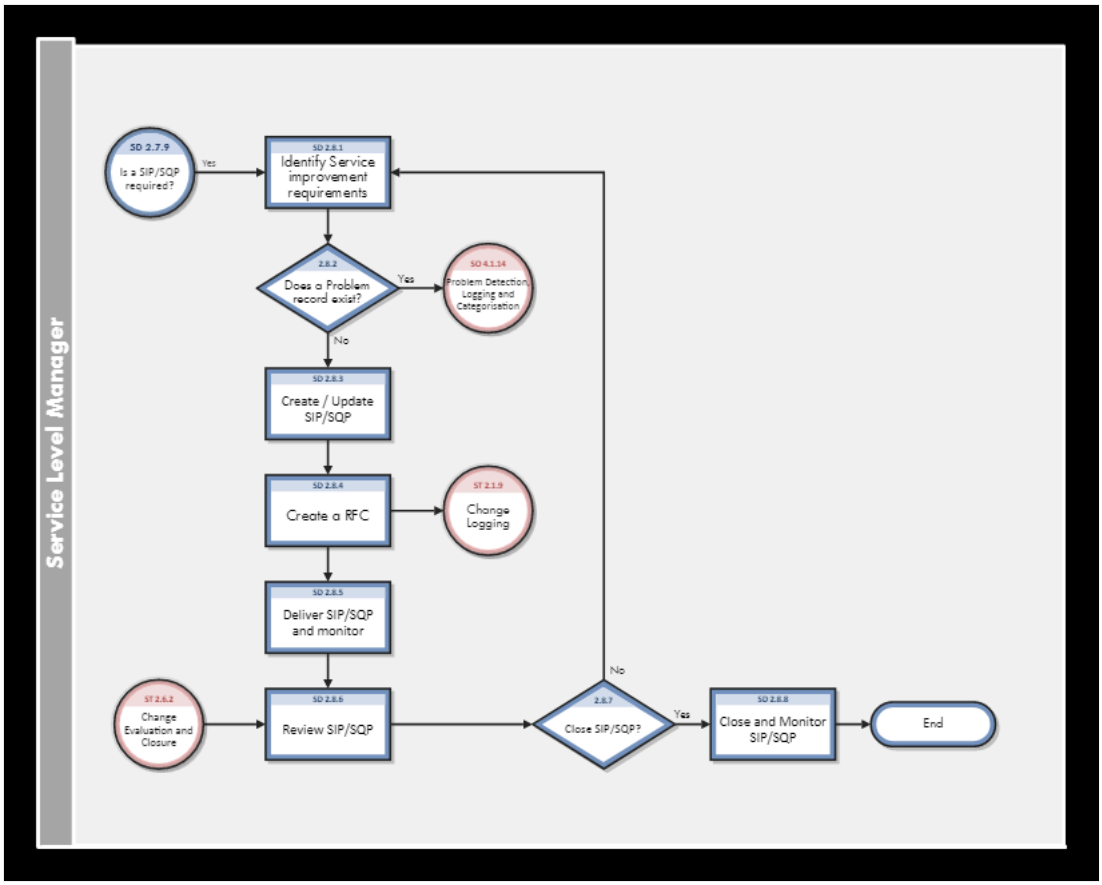
9.23.8. Service Improvement Plan (SD 2.8)

Process ID	Procedure or Decision	Description	Role
SD 2.8.1	Identify Service improvement requirements	Various inputs are used to establish the underlying requirement, such as: • Service Review data • Process Review data • Service and process reports • SLA monitoring • Problem Management • Customer Satisfaction The Service Level Manager will create a team to assist with requirement identification which includes Service Level Management, Problem Management, Customer satisfaction results and any relevant technical or process personnel. The Complaints and Compliments, Problem and Known Error database is checked to ensure the issues aren't being dealt with already. Go to SD 2.8.2 to determine whether a Problem record exists.	Service Level Manager
SD 2.8.2	Does a Problem record exist?	If yes, go to Problem Detection, Logging and Categorization (SO 4.1.14) to determine the related outstanding Problem. If no, go to SD 2.8.3 to develop an SIP/SQP plan.	Service Level Manager

SD 2.8.3	Create / Update an SIP/SQP	The creation of an SIP/SQP must include CSFs and measurement KPIs to assess whether targets are being met. The following people should be involved in discussion and agreement of the SIP/SQP: • Customer • Service Owner • Service Level Manager • Relevant support groups An SIP/SQP Program Board may be created and includes the following people: • Service Owner • Service Level Manager • Delivery team senior management The Program Board agrees on the composition of the team and appoints an SIP/SQP Program Manager. Go to SD 2.8.4 to create an RFC.	Service Level Manager
SD 2.8.4	Create an RFC	There are 2 parallel activities following this process step. An RFC should be submitted to support the new SIP. Go to Change Logging (ST 2.1.8) to create a new Change and go to SD 2.8.5 to deliver the SIP and monitor progress.	Service Level Manager

SD 2.8.5	Deliver SIP/SQP & monitor	SIP/SQP Delivery is the responsibility of the SIP Program Manager and follows Change Management and project management processes. While Service Level Management is a key stakeholder in the SIP/SQP process, the team is not responsible for actual delivery. The SIP/SQP Program Board is responsible for monitoring the progress of the SIP/SQP through regular progress reports. Service Level Management monitors all ongoing SIPs/SQPs ensuring they remain focused on business objectives and to provide input to Service Review meetings. Go to SD 2.8.6 to review the SIP/SQP.	Service Level Manager
SD 2.8.6	Review SIP/SQP	Change Management feeds into this activity advising that the Change record has been closed. When the SIP/SQP is nearing completion, the SIP/SQP Program Board assesses the likely success against the CSFs and accompanying KPIs. A short evaluation report is produced which provides recommendations for closure or further actions Go to SD 2.8.7 to determine whether to close the SIP/SQP.	Service Level Manager
SD 2.8.7	Close SIP/SQP?	If yes, go to SD 2.8.8 to monitor SIP/SQP requirements. If no, go to SD 2.8.1 to identify Service improvement requirements.	Service Level Manager

SD 2.8.8	Close & Monitor SIP/SQP	Once the SIP/SQP is closed, monitor for a period of time to allow the benefits of the project to take effect. The SIP/SQP Program Board retains responsibility for the monitoring period and will determine when it can end. The Service Improvement Program process ends.	Service Level Manager

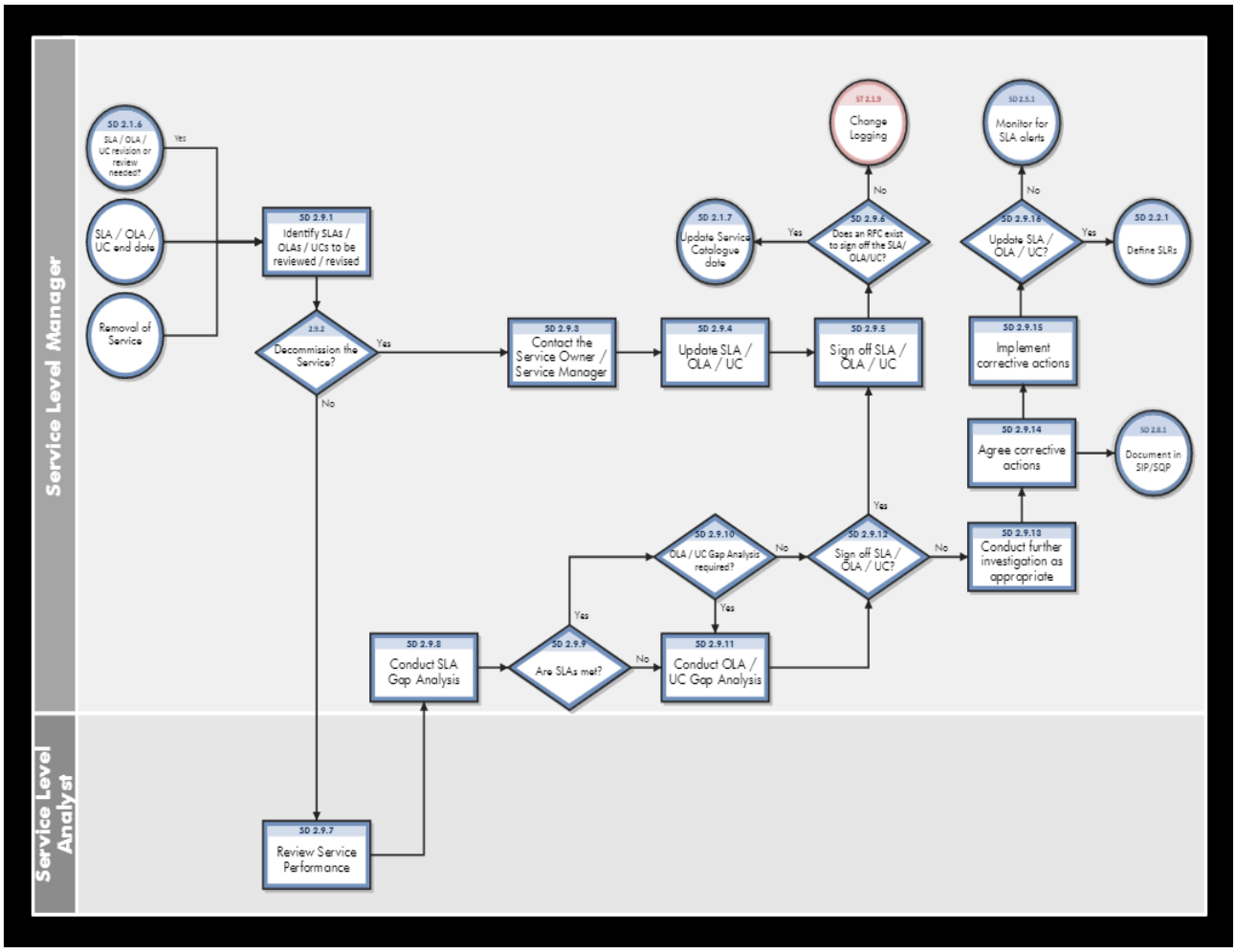


9.23.9. Review and Revise SLAs/OLAs/UCs (SD 2.9)

Process ID	Procedure or Decision	Description	Role
SD 2.9.1	Identify SLAs /OLAs/UCs to be reviewed / revised	Conduct an investigation of SLAs /OLAs/UCs to identify all agreements that require review. Go to SD 2.9.2 to determine whether to decommission the Service.	Service Level Manager
SD 2.9.2	Decommission the Service?	If yes, go to SD 2.9.3 to contact the Service Owner or Service Manager. If no, go to SD 2.9.7 to review the Service performance.	Service Level Manager
SD 2.9.3	Contact the Service Owner / Service Manager	The Service Level Manager works with the Service Owners and Service Managers to review the SLAs/OLAs/UCs. Go to SD 2.9.4 to update the SLA /OLA/UC.	Service Level Manager
SD 2.9.4	Update SLA /OLA/UC	The Service Level Manager updates the SLA with the Customer to reflect the decommissioning of the Service. The Service Level Manager updates the OLA/UC with the Service Owner and Supplier Manager to reflect the decommissioning of the Service. Go to SD 2.9.5 to sign-off the SLA /OLA/UC.	Service Level Manager
SD 2.9.5	Sign off SLA /OLA/UC	The SLAs /OLAs/UCs are signed off following the review. Go to SD 2.9.6 to determine whether an RFC exists.	Service Level Manager
SD 2.9.6	Does an RFC exist to sign off the SLA /OLA/UC?	If yes, go to Maintain the Service Catalog (SD 2.1.7) to update the Service Catalog. If no, go to Change Logging (ST 2.1.9) to create a new Change.	Service Level Manager

SD 2.9.7	Review Service performance	Obtain data from other Service management processes to assess the performance of actual results against agreed targets. Go to SD 2.9.8 to conduct an SLA gap analysis.	Service Level Analyst
SD 2.9.8	Conduct SLA Gap Analysis	Prepare an SLA Gap Analysis to identify the shortfalls in SLA results against the business requirements and document in a report. Shortfalls will need to be addressed within a SIP. Go to SD 2.9.9 to determine whether the agreed SLAs are being met.	Service Level Manager
SD 2.9.9	Are SLAs met?	If yes, go to SD 2.9.10 to determine whether an OLA/UC Gap Analysis is required. If no, go to SD 2.9.11 to conduct an OLA/UC Gap Analysis.	Service Level Manager
SD 2.9.10	OLA/UC Gap Analysis required?	If yes, go to SD 2.9.11 to conduct an OLA/UC Gap Analysis. If no, go to SD 2.9.12 to determine whether to sign-off the SLA /OLA/UC.	Service Level Manager
SD 2.9.11	Conduct OLA/UC Gap Analysis	Review the Service delivery chain and identify all OLAs/UCs which support delivery of the Service. Conduct an appropriate Gap Analysis to identify shortfalls against agreed targets and document. Go to SD 2.9.12 to determine whether to sign-off the SLA /OLA/UC.	Service Level Manager
SD 2.9.12	Sign off SLA /OLA/UC?	If yes, go to SD 2.9.5 to sign-off the SLA /OLA/UC. If no, go to SD 2.9.13 to conduct further investigation as appropriate.	Service Level Manager

SD 2.9.13	Conduct further investigation as appropriate	Conduct further investigation to ensure issues are removed and the SLA / OLA / UC can be signed off. Go to SD 2.9.14 to agree corrective actions-	Service Level Manager
SD 2.9.14	Agree corrective actions	Identify and agree corrective actions with the Service Owner and Supplier Manager and document them in the SIP/SQP. Go to SD 2.9.15 to implement corrective actions.	Service Level Manager
SD 2.9.15	Implement corrective actions	Implement corrective actions. Go to SD 2.9.16 to determine whether to update the SLA/OLA/UC.	Service Level Manager
SD 2.9.16	Update SLA/OLA/ UC?	If yes, go to Create or Amend Services (SD 2.2.1) to define requirements. If no, go to Monitor Service Levels (SD 2.5.1) to monitor for SLA alerts.	Service Level Manager



9.24. Service Level Management details

Service Manager uses the Service Level Management module to enable the Service Level Management process. The main function of Service Level Management is to improve the quality of services that you provide to customers. Service Level Managers, Supplier Managers and customers work together to ensure the various agreements are correctly documented and monitored.

This section describes selected Service Level Management details in the out-of-box Service Manager system.

SLA

A service-level agreement (SLA) is an agreement between two or more parties, where one is the customer (business) and the others are service providers. This can be a legally binding formal or an informal "contract".

Service level agreements are also defined at different levels:

- Customer-based SLA:
  An agreement with an individual customer group, covering all the services they use.
- Service-based SLA:
  An agreement for all customers using the services being delivered by the service provider.

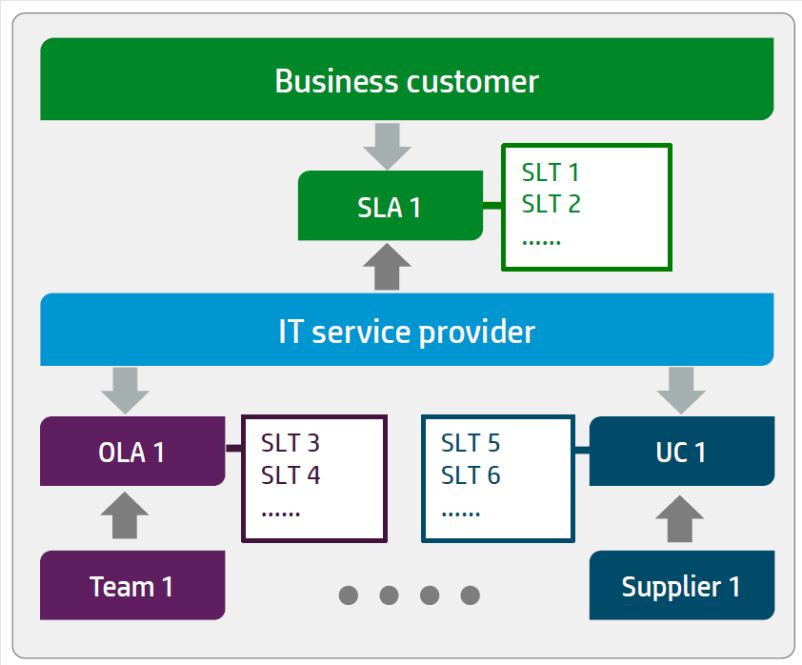
OLA

An operational-level agreement (OLA) defines the interdependent relationships among the internal support groups of an IT organization working to support a service-level agreement (SLA). The agreement describes the responsibilities of each internal support group toward other support groups, including the process and timeframe for delivery of their services. The objective of the OLA is to present a clear, concise and measurable description of the service provider's internal support relationships.

UC

An Underpinning Contract (UC) is a legally binding contract between IT service provider and supplier or third party to deliver agreed level of service quality or goods at specified time.

The relation between these three elements is defined in the graphic below.



Topics in this section include:

The Workflow for Agreements, Underpinning Contracts

Creating a new SLA, OLA, or UC

The Draft phase

The Review phase

The Agreed phase

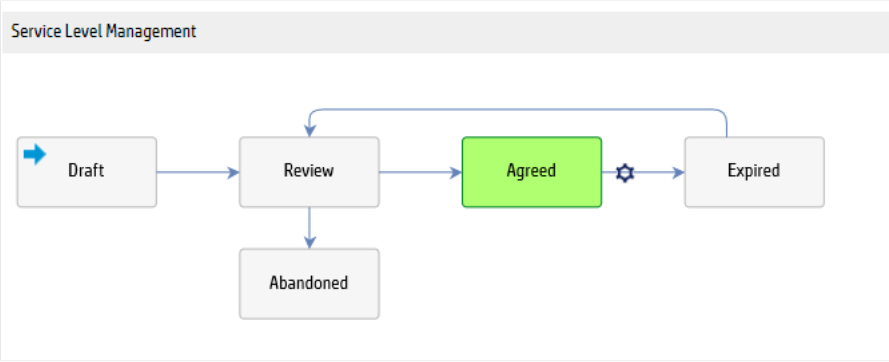
The Expired phase

Service Reviews and usages of SIPs/SQPs

Service Level Management form details

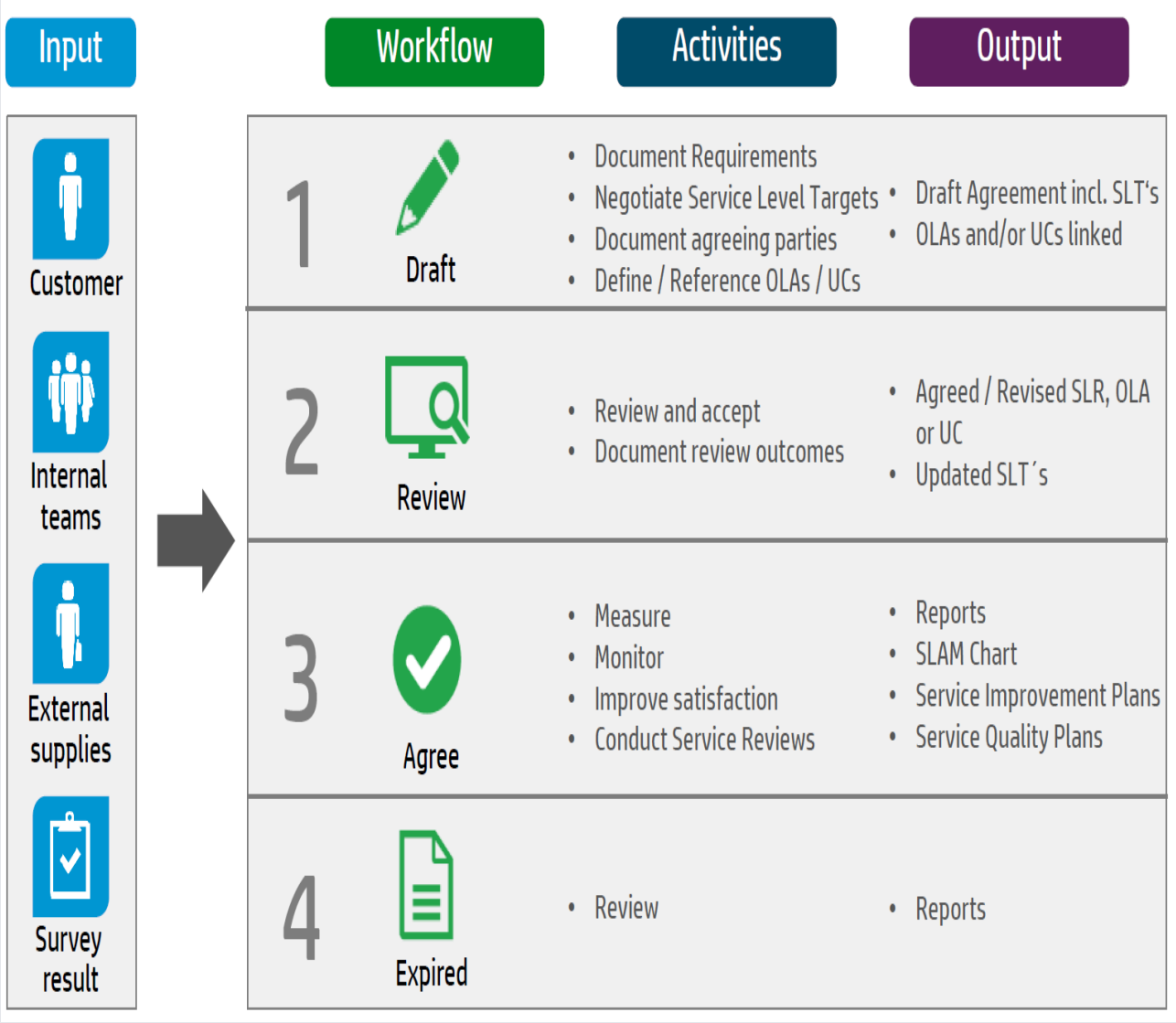
9.24.1. The Workflow for Agreements, Underpinning Contracts

The full Service Level Management module was moved towards Process Designer. As part of this step, the following workflow was introduced to support the several activities in the Service Level Management process.



Phase	Description
Draft	Initialize a new SLR, OLA or UC in draft phase to document all required details
Review	Review with the various stakeholders the documented details prior activating the SLA, OLA or UC
Agreed	The SLA, OLA and UC are agreed and active.
Expired	The SLA, OLA and UC are expired and inactive.
Abandoned	The SLA, OLA and UC have been abandoned.

The different activities in the workflow phases are summarized as follow.

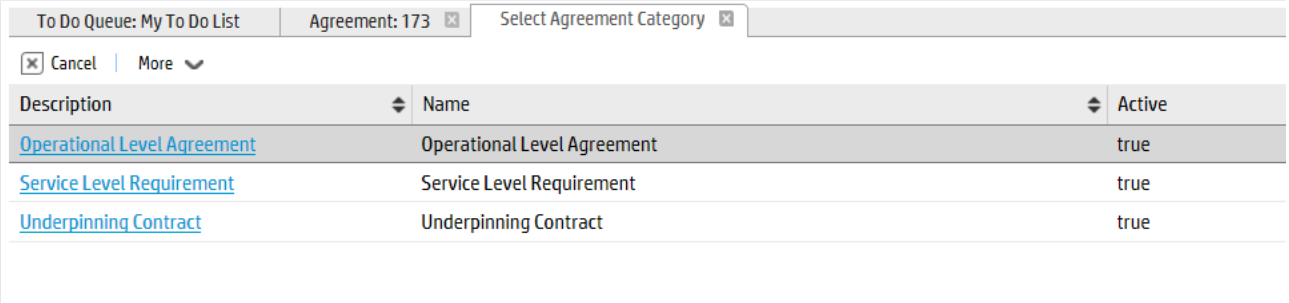


9.24.2. Creating a new SLA, OLA or UC

To create a new SLA, OLA or UC, click **Service Level Management** > **Agreements** > **New Agreement**, and then select the agreement category.

- To start a new SLA, select **Service Level Requirement**, since every SLA starts with documented SLRs. Once an SLR is agreed it becomes an SLA.

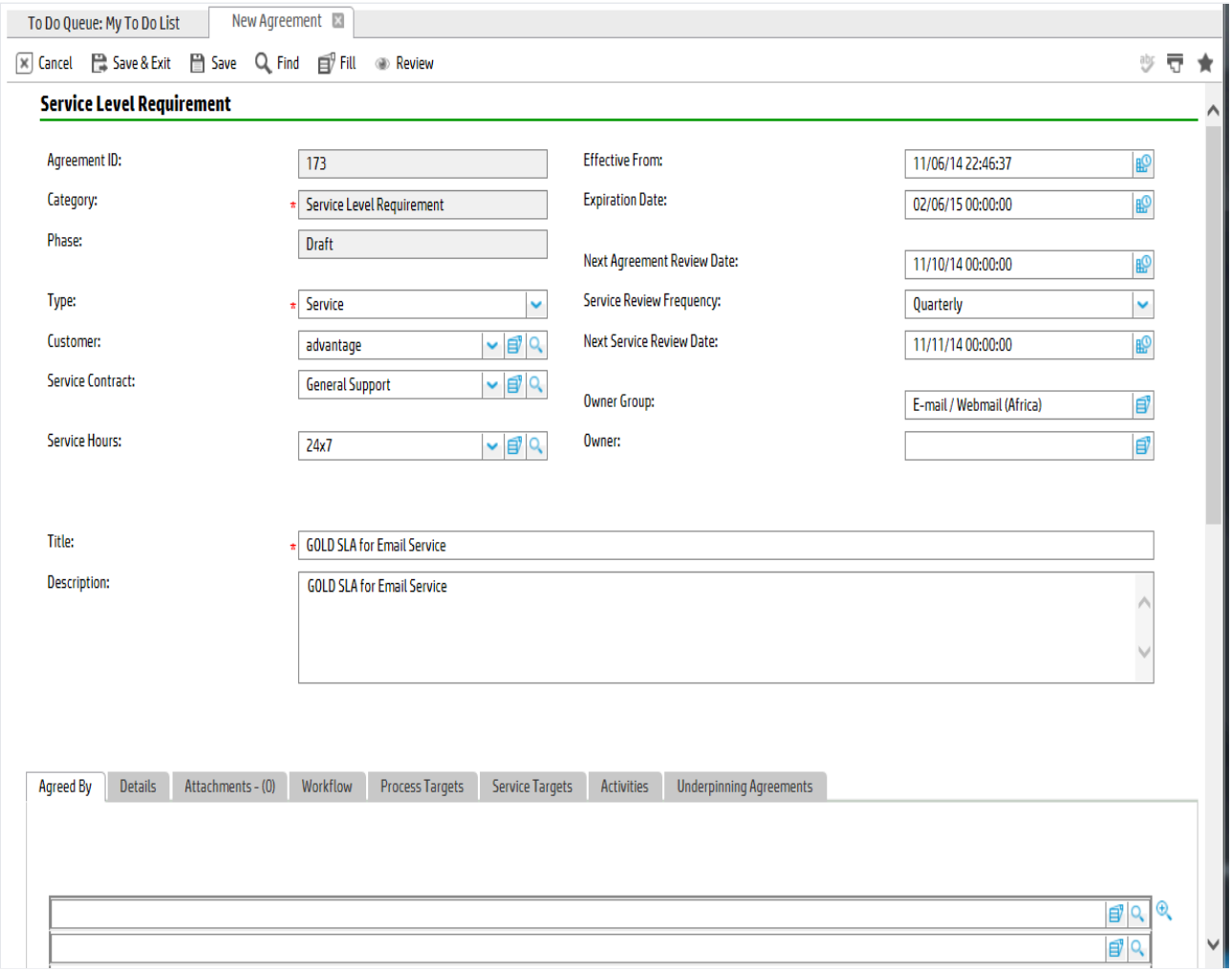
- To create a new OLA or UC, select the correct category.



9.24.3. The Draft phase

Every record type in Service Level Management module starts from draft phase. In draft phase, you can document some general requirements and the requested Service Level Targets. Process Targets represent response targets, whereas Service Targets measure the availability.

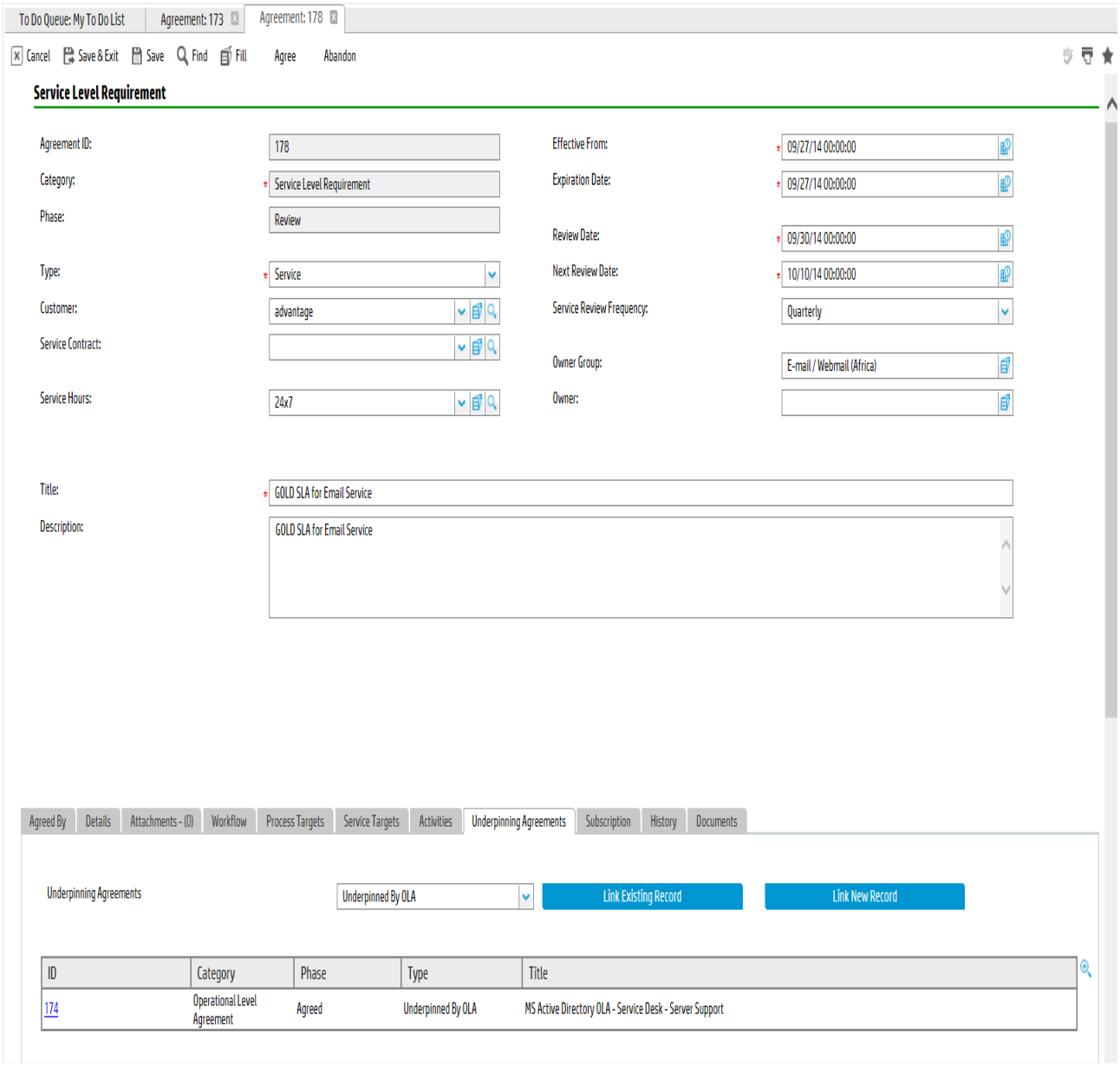
Ensure that you document all required fields. Click the **Review** button to move the record into the next phase.



9.24.4. The Review phase

In Review phase, all stakeholders have the chance to review the agreed details and adjust in case requirements have changed or can't be met. The review phase is the last prior to activating the agreement or contract.

The review phase should be used to double-check whether all underpinning OLAs and contracts meet the agreed targets of the SLA. By looking at the **Underpinning Agreement** tab the Service Level Manager can easily verify the associated OLAs/UCs.

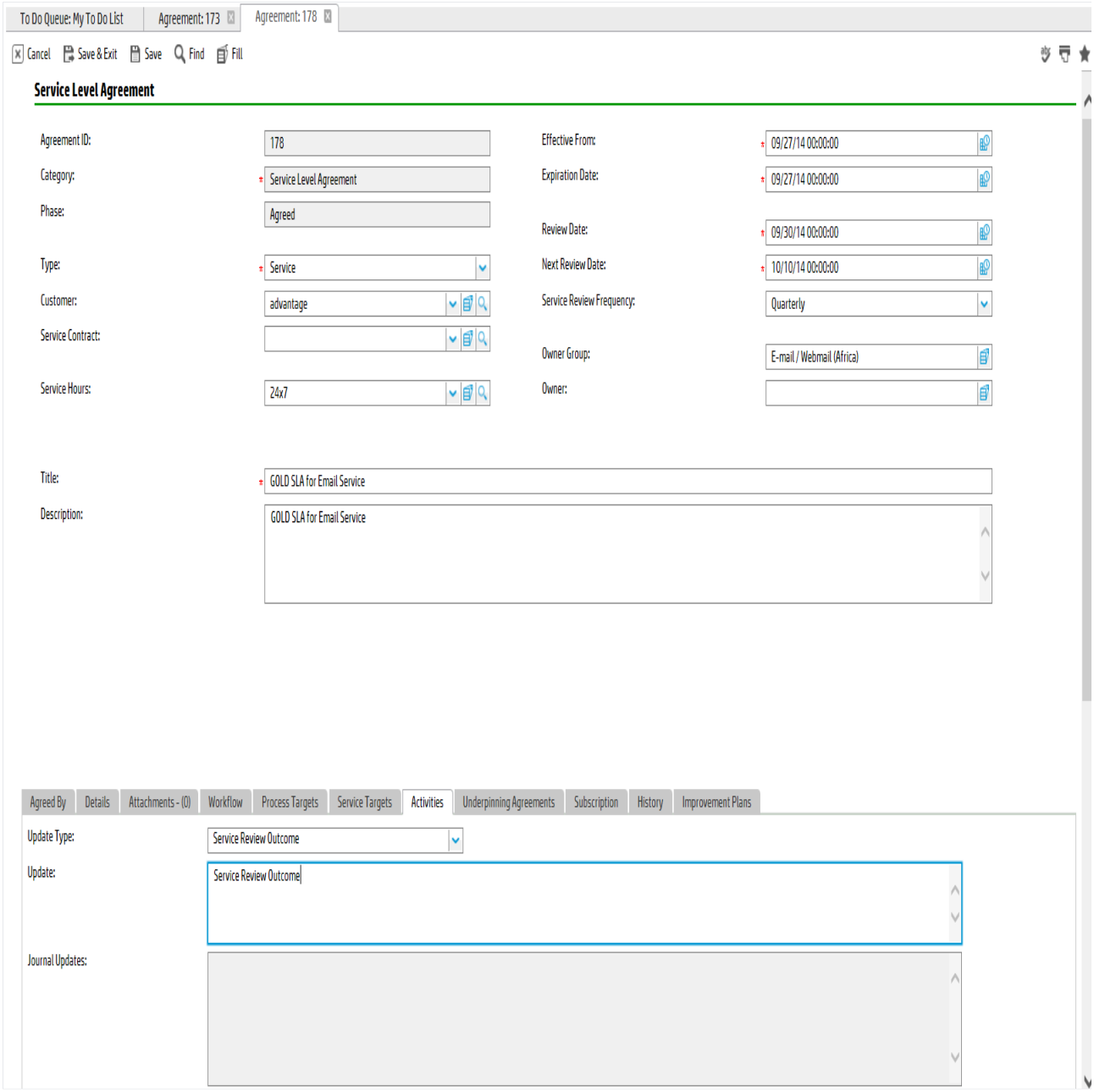


In accordance to ITIL guidance, all OLAs/UCs should support the defined SLAs within the business. Where specific clauses and targets are required for an individual service, the conditions in the SLTs of the OLA/UC should be leveraged. By using the conditioned SLTs of the OLA/UC, the Service Level Manager can ensure that the agreed targets of the SLA are correctly covered.

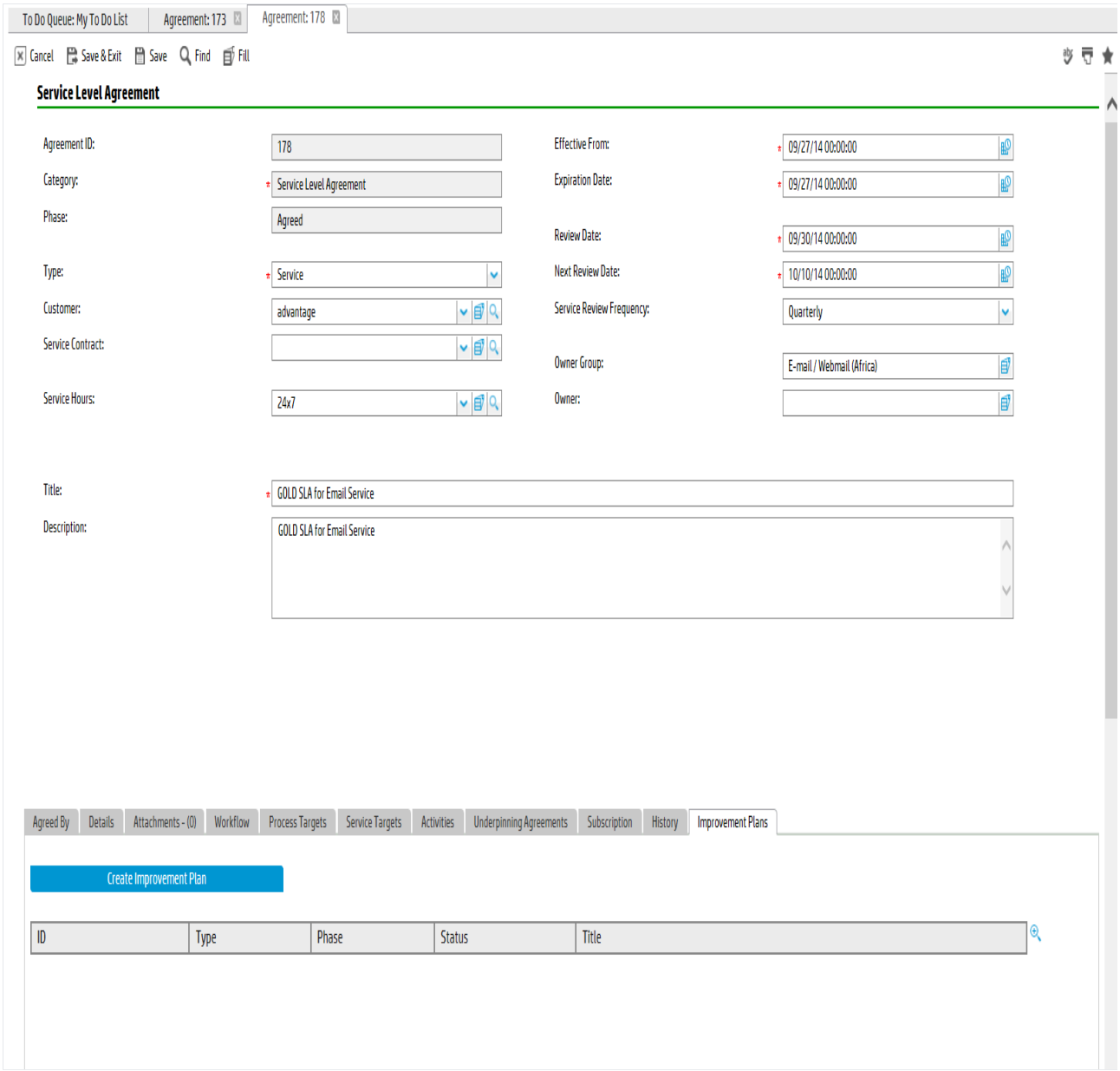
9.24.5. The Agreed phase

In the Agreed phase, the SLM records are active and used. In case modifications are required, the SLA, OLA,or UC is directly updated in this phase. Service Reviews help to keep the records up to date. All changes have to go through Change Management and shall be tracked in the activity log.

The system allows to document SIPs and SQP directly from the agreement. This helps to validate the actions directly in each service review. To document service reviews activities, you can use the activity log.

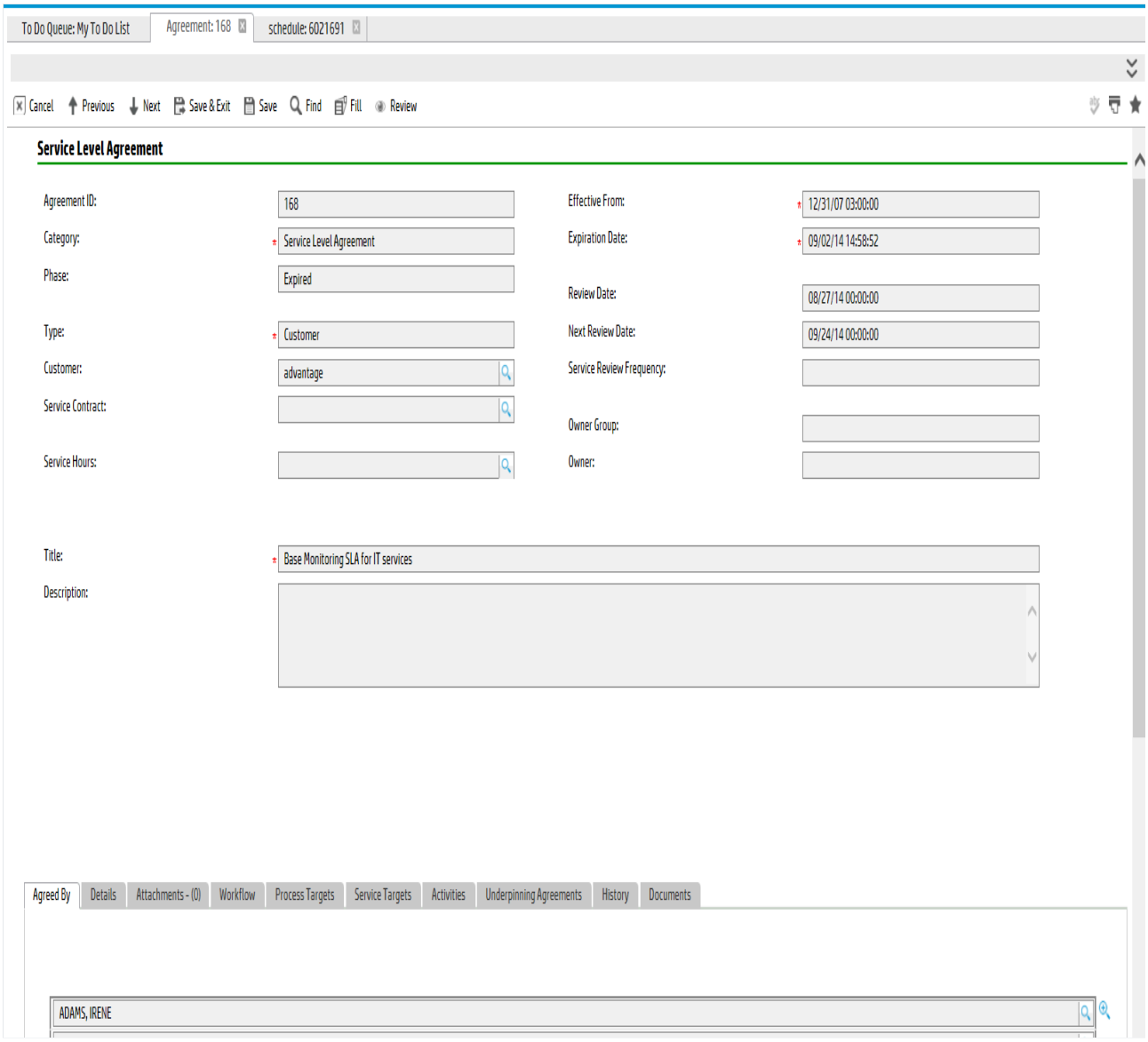


To document SIP/ SQP, you can use the Improvement Plan feature.



9.24.6. The Expired phase

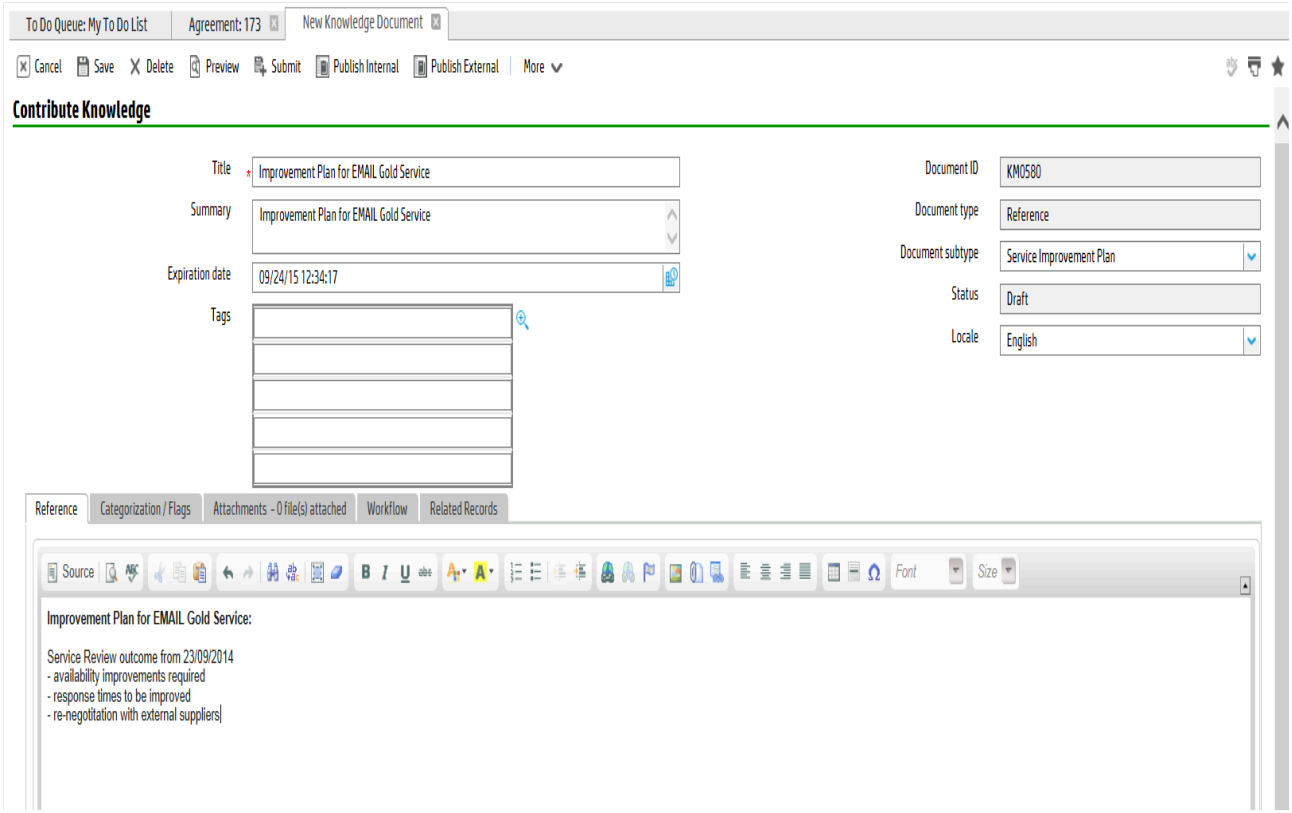
The system automatically moves the record into the expired phase once it reaches the expiration date. In expired phase, the records are inactive and read-only. In case the record needs to be activate again, click the **Review** button to move it back to Review phase.



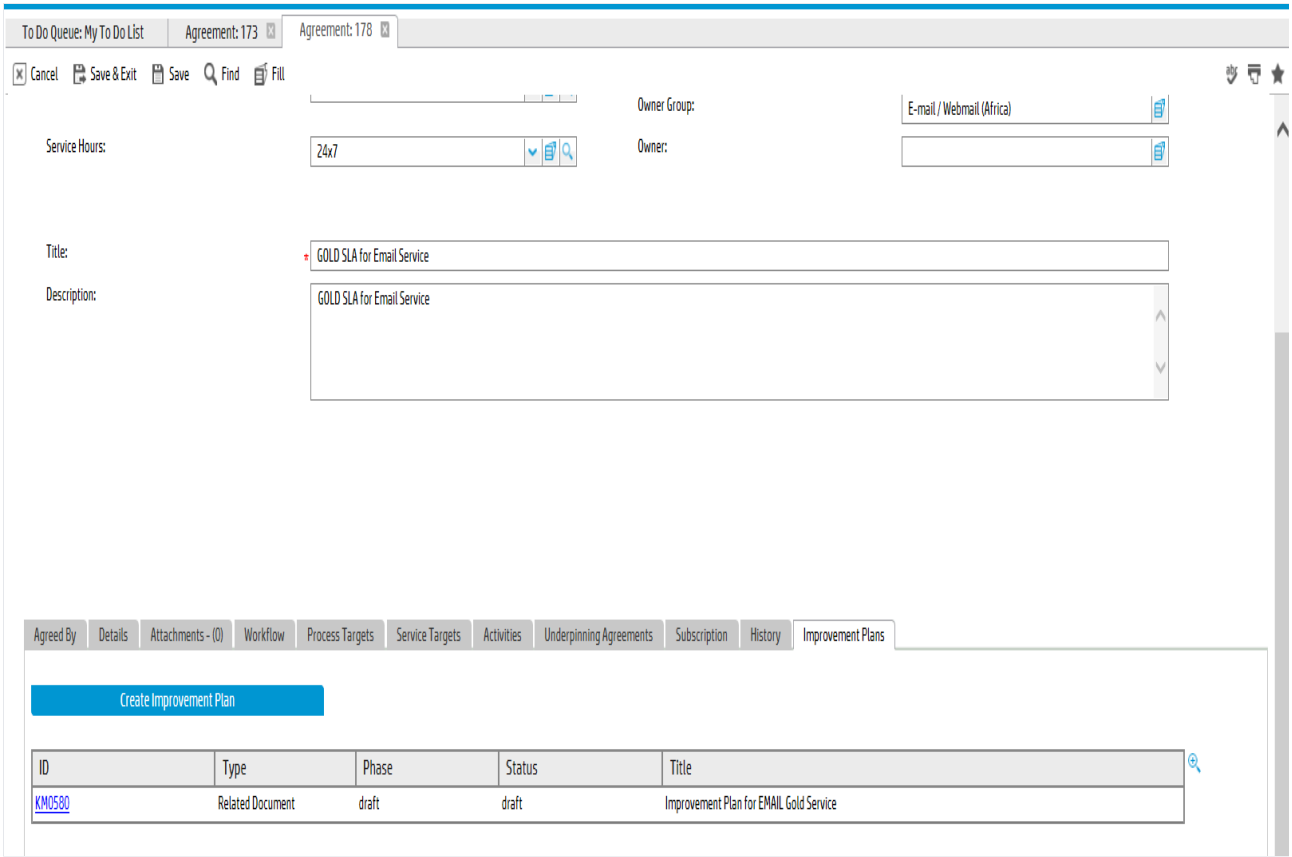
9.24.7. Service Reviews and usages of SIPs/SQPs

Service Reviews meetings should be conducted frequently. The outcome of these meetings are documented in either Service Improvement or Service Quality Plans. Service Manager allows to document the details of the service reviews by leveraging the knowledge management module.

To create an SIP or SQP, click the **Create Improvement Plan** button.



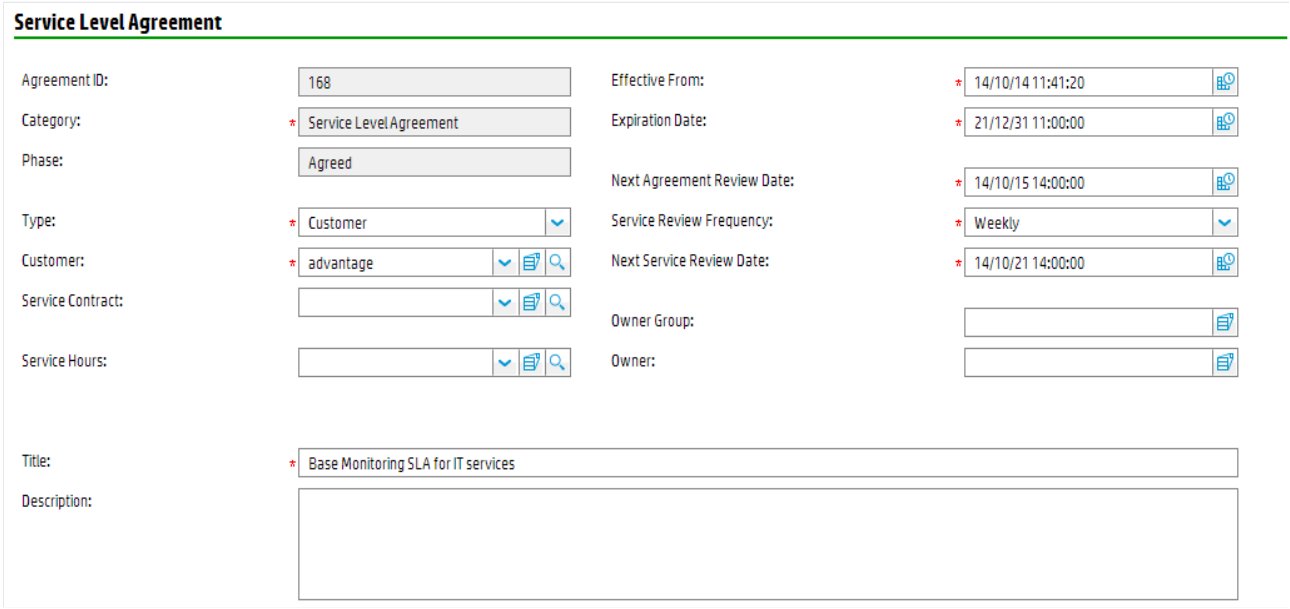
All SIPs/SQPs are listed directly in the Service Level Management record.



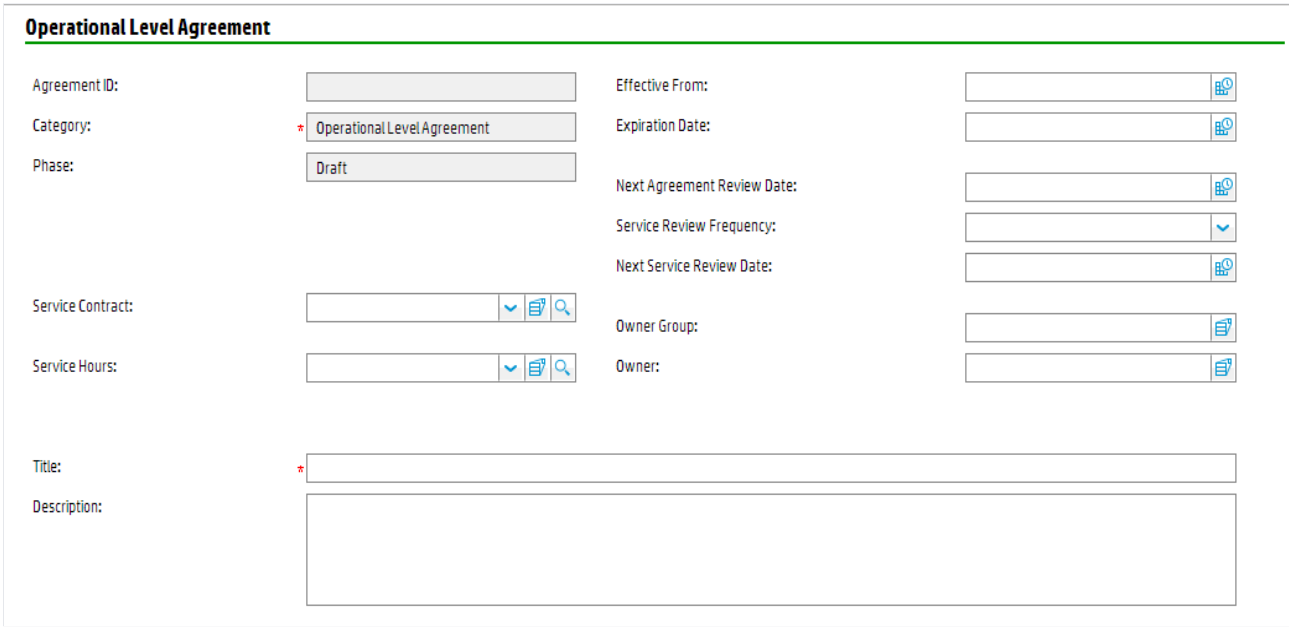
9.24.8. Service Level Management forms

The following forms are Service Level Management forms.

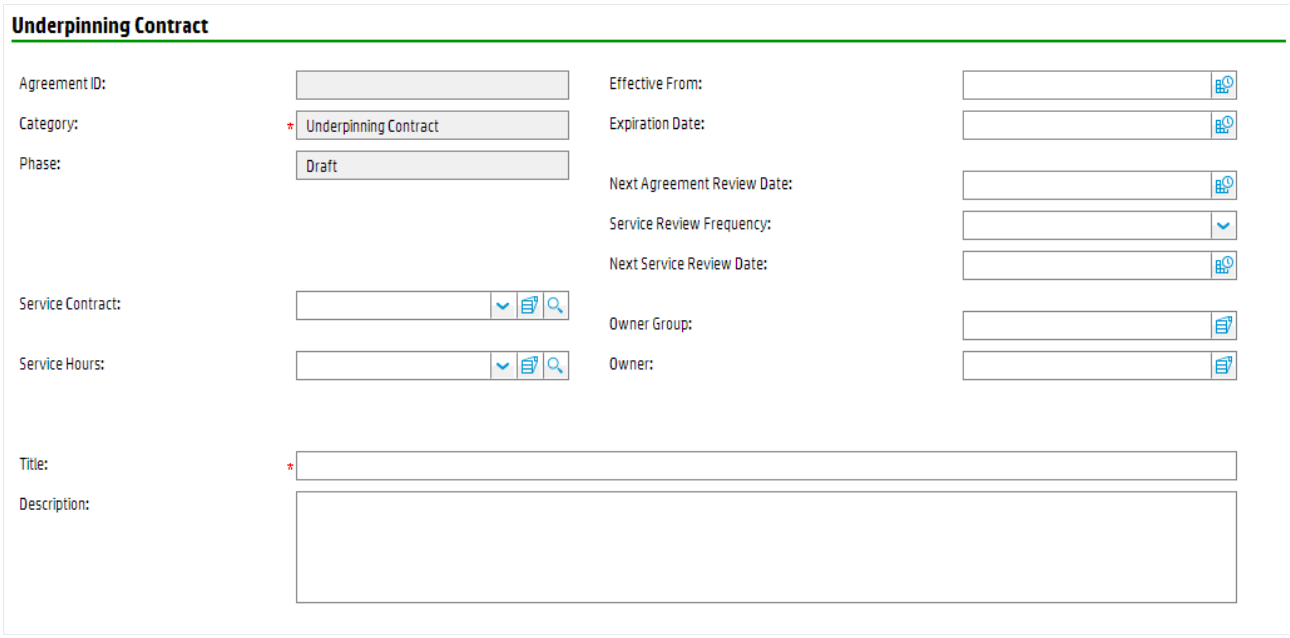
Service Level Agreement



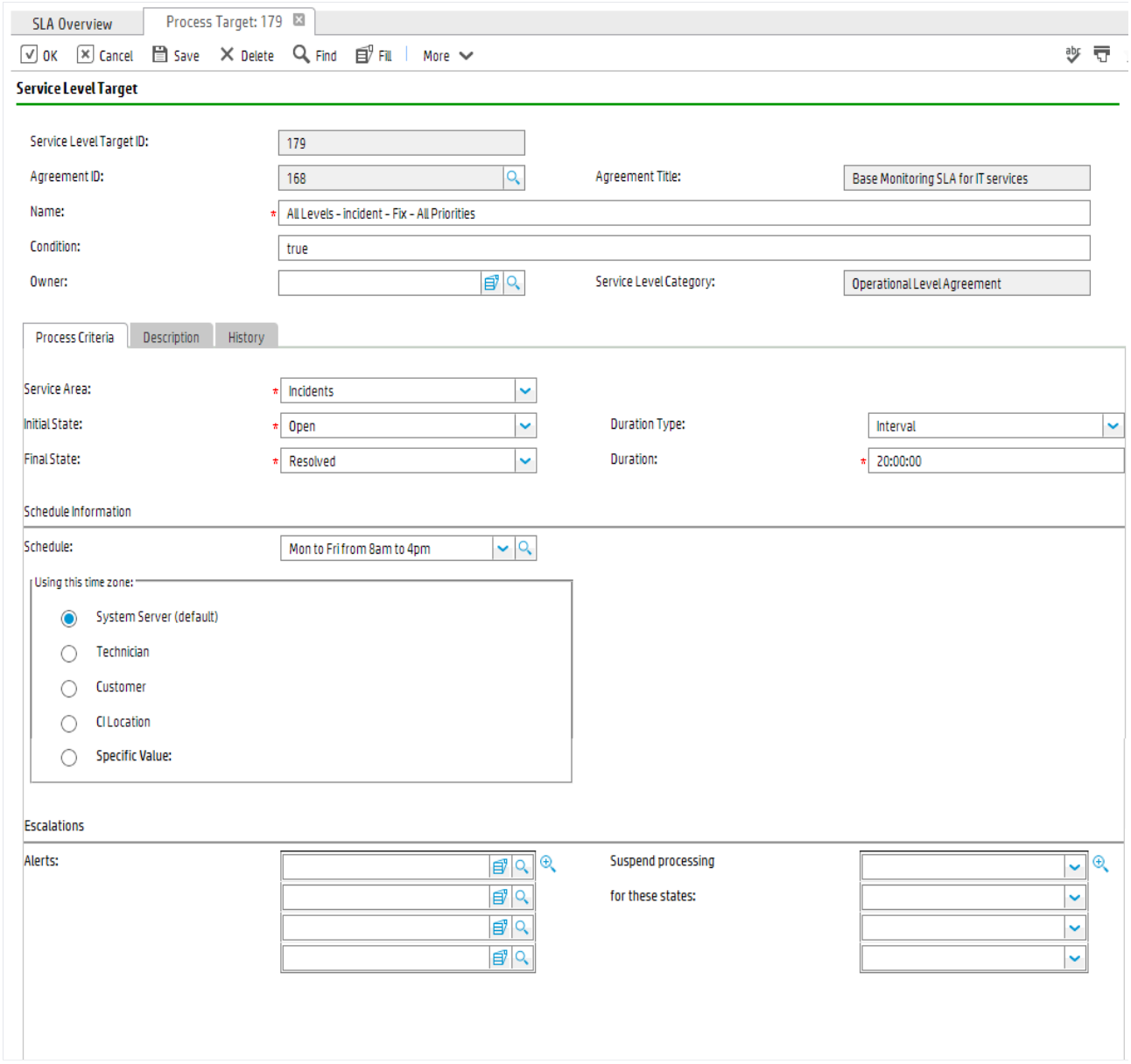
Operational Level Agreement



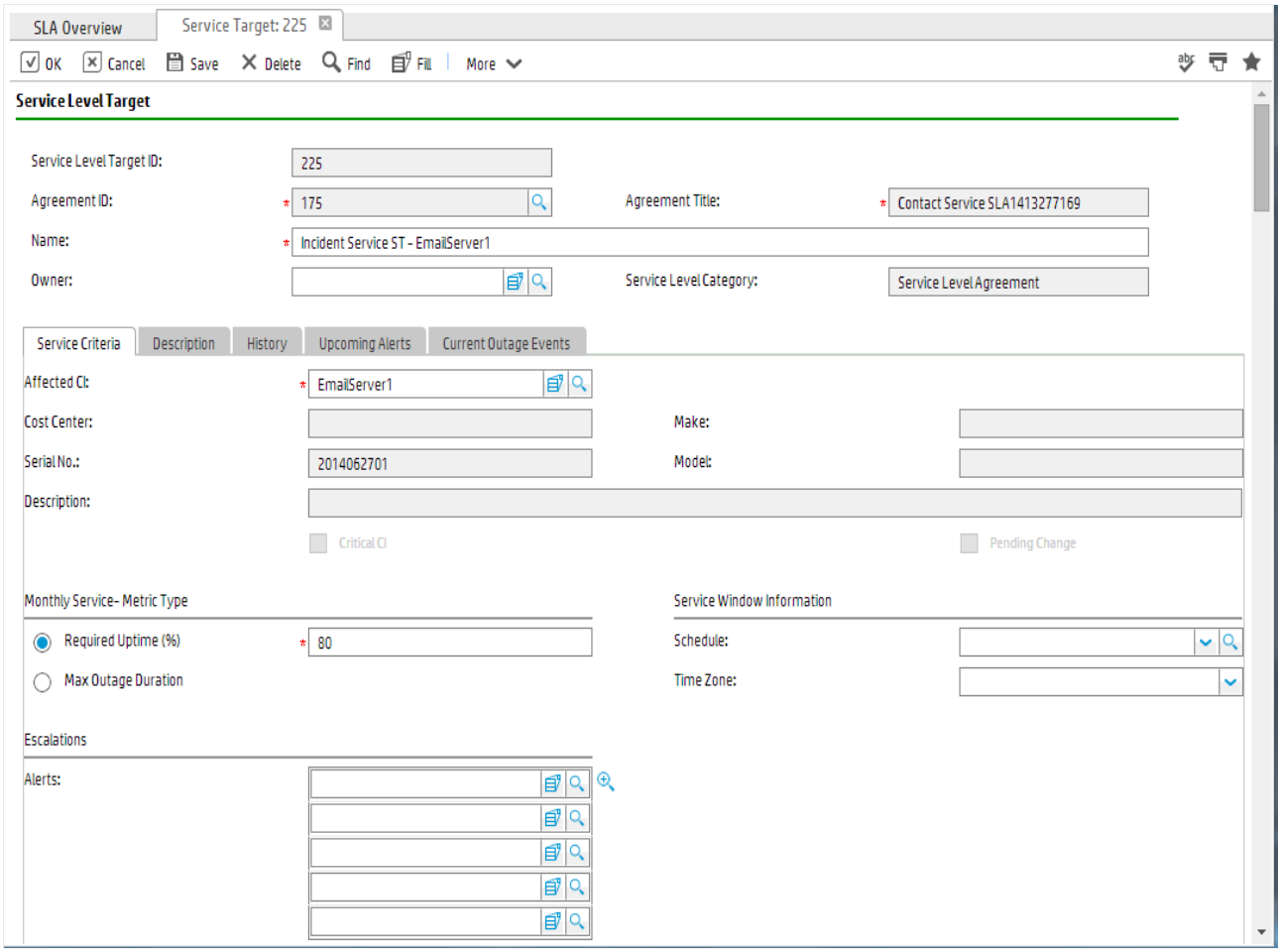
Underpinning Contract



Service Level Target - Process Target



Service Level Target - Service Target



9.24.9. Service Level Management form details

The following table identifies and describes some of the features of Service Level Management forms.

SLA/OLA/UC field descriptions

Label	Description
MAIN	
Agreement ID	The unique ID of the Agreement. Service Manager populates this field when the Agreement is first created. Service Manager uses this number to track relationships between Agreements and their supporting data.
Category	The category of agreement that best describes the Agreement.
Phase	The current phase of the record.
Type	The type field identifies an Agreement as a Service or Customer Agreement. This field has two possible values: Service or Customer.
Customer	The name of the customer who is directly impacted by the service levels measured by the Agreement.
Service Contract	The ID of the contract (entitlement) that covers the service specified in the Agreement. The contracts available in the drop-down list are dependent on the selected customer for the Agreement.
Service Hours	The schedule that customers are entitled to the service guaranteed by the Service Level Agreement (SLA). When you register Interactions or open Incidents, Service Manager checks the Service Hours schedule, to determine if the customer submitting the Interaction or Incident is entitled to the service.
Title	A brief descriptive title for the Agreement.
Description	A description of the service levels covered by the Agreement.
Effective From	The date and time when the Agreement coverage begins.
Expiration Date	The date and time when the Agreement coverage expires.
Next Agreement Review Date	The date when the agreement is supposed to be reviewed next time.
Service Review Frequency	This field identifies how often the agreement is to be reviewed.
Next Service Review Date	The date when the relevant service in the agreement is supposed to be reviewed next time.
Owner Group	The group who is responsible for the agreement.
Owner	The operator who is responsible for the agreement.
Agreed By	
The individuals in the contacts table who reach an agreement on the Agreement.	
Details	

Notes, verbal agreements, or comments that are relevant to the Agreement. For example, an additional agreement that was not part of the original agreement.
Attachments
You can use the Attachments section to attach documents to the Agreement.
Workflow
Displays a figure of request task workflow. It indicates the current phase which the request task is in, and traces the phase transition history.
Process Targets
Displays the Service Level Targets which define the Process Target details.
Service Targets
Displays the Service Level Targets which define the Service Target details.
Activities
Records information that the operator enters during the lifecycle of the Agreement. Every time you update the agreement, you can fill in an update on the Activities section. A log of all the updates is stored on the Journal Updates and activities list.
Underpinning Agreements
Displays a list of Underpinning Agreements related to this Agreement.
Subscriptions
Displays a list of subscription information related to this Agreement.
History
Displays the monthly overall performance metrics of this Agreement, as well as the specific performance metrics of its different types of service level targets (process targets and service targets).
Improvement Plans
Displays a list of Improvement Plans related to this Agreement.
Assignment Groups
The group in the assignment table associated with agreement for Operational Level Agreement.
Related SLA
Displays a list of SLAs related to this Operational Level Agreement.
External Assignment Groups
The external group in the assignment table associated with agreement for Underpinning Contract.

Service Level Target field descriptions

Label	Description
MAIN	
Service Level Target ID	The unique ID of the Process or Service Target. Service Manager creates this field automatically when adding the SLT. This field is read-only unless you are searching for specific SLTs.
Agreement ID	The unique ID of the Agreement associated to the Service Target. Service Manager populates this field when you create a new SLT from the Agreement screen using the wizard. Service Manager uses this number to track relationships between Agreements and their supporting data.
Name	The name of the Process or Service Target.
Condition	Determines whether to process the process Target. You can enter the condition manually, or Service Manager can create it automatically based on field parameters that you select when you add the SLT by using the wizard.
Owner	The owner of the process for the Process or Service Target.
Agreement Title	The title of the Agreement associated to the Service Target.
Service Level Category	The service level category of agreement that best describes the Service Target.
Process Criteria	
Service Area	The application area in Service Manager that the Process Target applies to. The value displayed in the drop-down list reflect the Service Manager areas or modules that are available for use in Service Level Agreement.
Initial State	A state when the Process Target (SLT) should start measuring the process time.
Final State	A state when the Process Target (SLT) should stop measuring the process time.
Duration Type	The type of response time to measure when processing the Process Target.
Duration	The maximum interval of time allowed between the Initial State and the Final State.
Schedule Information - Schedule	The schedule used to determine the valid times for processing the Process Target.
Schedule Information - Using this time zone	The time zone to use when processing the Process Target.
Escalation - Alerts	The alert(s) to generate when processing the Process Target.
Escalation - Suspend processing for these states	States when processing for the Service Level Target (SLT) should be suspended.

Description	
Displays a list of subscription information related to this Service Level Target.	
History	
Displays the timestamps and operators that the Service Level Target is opened or updated by.	
Service Criteria	
Affected CI	The unique ID of the affected configuration item (CI) measured by the Service Target. The CI is required to have a valid identifier and must have a corresponding entry in the device table.
Cost Center:	The cost center information for this configuration item (CI). Service Manager populates this field when you create a new SLT from the Agreement screen using the wizard.
Serial No.	The serial number for this configuration item (CI). Service Manager populates this field when you create a new SLT from the Agreement screen using the wizard.
Description	The manufacturer's model identification for this configuration item (CI). Service Manager populates this field when you create a new SLT from the Agreement screen using the wizard.
Make	The manufacturer of this configuration item (CI). Service Manager populates this field when you create a new SLT from the Agreement screen using the wizard.
Model	The manufacturer's model identification for this configuration item (CI). Service Manager populates this field when you create a new SLT from the Agreement screen using the wizard.
Monthly Service- Metric Type - Required Uptime	The percentage of time that the affected configuration item (CI) should be available per month.
Monthly Service- Metric Type - Max Outage Duration	The maximum amount of time that the affected configuration item (CI) may be unavailable for a single outage.
Service Window Information - Schedule	The schedule used to determine the valid times for processing the Service Target.
Service Window Information - Time Zone	The time zone to use when processing the Service Target.
Escalation - Alerts	The alert(s) to generate when processing the Service Target.
Upcoming Alerts	
Displays a list of Upcoming Alerts related to this Service Level Target.	

Current Outage Events
Displays a list of Current Outage Events related to this Service Level Target.

9.25. Release and Deployment Management

Service Manager uses the Release and Deployment Management application to enable the Release and Deployment Management process.

The following sections shows how you can tailor Service Manager to support Release and Deployment Management as a subset of Change Management.

9.25.1. Release and Deployment Management Overview

The Service Manager Release and Deployment Management application, referred to as Release and Deployment Management throughout this chapter, supports the Release and Deployment Management process. It is implemented as a separate category inside the Change Management application and supports and relies on the Change Management process. It ensures that the Configuration Management Database (CMDB) is kept up to date, that changes are appropriately managed, and that all new software and hardware is stored in the Definitive Software Library (DSL) and Definitive Hardware Store (DHS). After one or more changes are developed, tested, and packaged into releases for deployment, Release and Deployment Management is responsible for introducing these changes and managing their release. Release and Deployment Management also contributes to the efficient introduction of changes by combining them into one release and deploying them together.

This section describes how Release and Deployment Management implements the best practice guidelines for the Release and Deployment Management processes.

Topics in this section include:

- Release and Deployment Management within the ITIL framework
- Release and Deployment Management application
- Release and Deployment Management process overview
- Key performance indicators for Release and Deployment Management

9.25.1.1. Release and Deployment Management within the ITIL framework

Release and Deployment Management is addressed in ITIL's Service Transition publication. The goal of Release and Deployment Management is to plan, schedule and control the build, test and deployment of releases to deliver new or changed functionality, its enabling systems, technology and organization while protecting the integrity of existing IT and Business Services.

Release and Deployment Management includes the assembly and implementation of new or changed services, from release planning through to early life support transition for operational use. It includes, physical and virtual assets, applications and software, training and communication

The purpose of Release and Deployment Management is to:

- Define and agree release and deployment plans with customers and stakeholders
- Ensure that each release package consists of a set of related assets and service components that are compatible with each other
- Ensure that integrity of a release package and its constituent components is maintained throughout the transition activities and recorded accurately in the CMS and stored in the DML
- Deploy release packages from the DML to the production environment following an agreed plan and schedule
- Ensure that all release and deployment packages can be tracked, installed, tested, verified, and/or uninstalled or backed out if appropriate
- Record and manage deviations, risks, issues related to the new or changed service and take necessary corrective action
- Ensure that there is knowledge transfer to enable the customers and users to optimise their use of the service to support their business activities
- Ensure that skills and knowledge are transferred to operations and support staff to enable them to effectively and efficiently deliver, support and maintain the service according to required warranties and service levels

The objective of Release and Deployment Management is to ensure that:

- There are clear and comprehensive release and deployment plans that enable the customer and business change projects to align their activities with these plans

- A release package can be built, installed, tested and deployed efficiently to a deployment group or target environment successfully and on schedule
- All aspects that are related within an IT service are considered when creating, building and implementing a new or subsequent release of that service.
- Release activities are repeatable, controllable, scalable and sustainable
- There is minimal unpredicted impact on the production services, operations and support organization

9.25.1.2. Release and Deployment Management application

The Release and Deployment Management application ensures that the Configuration Management Database (CMDB) is kept up to date, that changes are appropriately managed, and that all new software and hardware is stored in the Definitive Software Library (DSL) and Definitive Hardware Store (DHS). After one or more changes are developed, tested, and packaged into releases for deployment, Release and Deployment Management is responsible for introducing these changes and managing their release. Release and Deployment Management also contributes to the efficient introduction of changes by combining them into one release and deploying them together.

The purpose of Release and Deployment Management is to ensure that all changes are deployed successfully in the least disruptive manner. Release and Deployment Management is responsible for the following functionality:

- Driving the release strategy, which is the overarching design, plan, and approach for deployment of a change into production in collaboration with the Change Advisory Board (CAB).
- Determining the readiness of each release based on release criteria (such as quality of release, release package and production environment readiness, training and support plans, rollout and back out plans, and risk management plan).

Release and Deployment Management offers these benefits for users:

- Provides a packaged release for all changes deployed into production and only deploys changes approved by change management.
- Provides two different types of releases: hardware and software. Once the release type is determined, the application provides the appropriate tasks.
- Ability to manage changes of groups such as Configuration Item (CI) groups and business services.
- Ability to terminate a release at any time except in the Training phase or once it has been installed or verified. At that point you have the ability to back-out the change.
- Availability of optional training phase.

9.25.1.3. Release and Deployment Management process overview

The Release and Deployment Management process includes activities necessary to control releases to service assets and configuration items across the entire service lifecycle. It provides standard methods and procedures to use when implementing all releases.

The purpose of Release and Deployment Management is to ensure that:

- Releases follow a set process
- Appropriate users are notified at key points in the process
- Progress of a release is monitored and notification are issued if deadlines are missed
- Releases are supported throughout a simple or complex lifecycle

Release types[edit]

Release and Deployment Management includes three release types: Emergency, Major, and Minor. All three release types utilize the Release Management category and workflow in the Change Management module. Additional categories and workflows can be created if required. The steps of the workflow are represented by the phases and tasks within the phase. Service Manager requires that every release has a category and phase, but tasks are optional.

Release phases and transitions[edit]

Service Manager uses phases to describe the steps necessary to complete a release. The phase also determine the release screens users see, the approvals required to advance to the next phase, and the conditions that cause the system to issue notification and alerts. Each phase has workflow transitions that control how the release record will move through the workflow. When the work for a given phase has been completed the user will advance it to the next phase. If the release has encountered a failure condition then the release may be move back to an earlier phase for rework or abandoned.

Release tasks[edit]

Release Management utilizes tasks to capture work that's necessary for the overall release. This is work in addition to the Change specific tasks that are required to complete the changes that are associated to the release. Examples of required release activities are: determining if funding is available to support the release, arranging for required contracts and licenses, building and delivering training, evaluating the financial impact of the planned release, and performing knowledge transfer after the release has been deployed. Release Models can be used to pre-define task plans for Releases.

Integration with Change Management[edit]

In Service Manager the Release record and workflow control the overall process of the release, including approvals for the release and the release specific activities (tasks) that need to be complete. The release is related to multiple change requests (RFCs) that have their own workflow, approvals, and tasks. The planning, assessment, build, and deployment of each change is tracked and worked in the Change Management module. The Release provides the higher level oversight and provides visibility into the Change Management process through the Related Records section in the release record.

Release roles[edit]

The following table describes the responsibilities of the Release and Deployment Management user roles.

Release and Deployment Management user roles

Role	Responsibilities
Release Manager	• Manages all aspects of the end-to-end release process • Reviews and updates the release policy • Facilitate the bundling of releases • Coordinate planning and preparation for Deployment • Coordinate creation of Deployment Plans • Ensures coordination between the build and test environment team and release teams • Ensures the teams follow the organization's established policies and procedures • Provides management reports on release progress • Updates the Service Knowledge Management System
Release Coordinator	• Registers the release and applies the correct release model and release detail. • Schedules the release according to the plan created previously. • Creates the release tasks required for the release. • Coordinates the Risk and Impact Analysis phase of the release and creates release plan based on the assessment information. • Verifies if the release has passed the test criteria. • Verifies if the release is implemented successfully in the production environment. • After implementation, evaluates the change and closes the request. • If a change implementation fails, the coordinator activates a back-out plan to return the system to its original state.
Release Approver	• Uses the Service Manager tool to Change Advisory Board to approve or deny Releases when requested

Contents

- 1 Release types
- 2 Release phases and transitions
- 3 Release tasks
- 4 Integration with Change Management

- 5 Release roles

9.25.1.4. Key performance indicators for Release and Deployment Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your Release and Deployment Management processes. To visualize trend information, it is useful to graph KPI data periodically. In addition to the data provided by Service Manager, you may need additional tools to report on all of your KPI requirements.

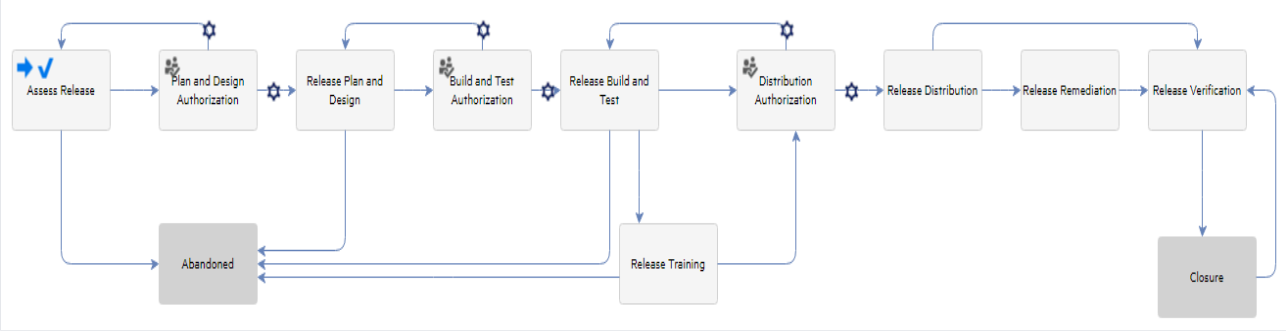
Key performance indicators for Release and Deployment Management

Title	Description
% of release success rate	Percentage of the total number of closed releases successfully implemented in a given period.
% of release defect rate	Percentage of releases resulting in Incidents in a given period.
Number of known release errors	The number of known errors by category by release over time.
% of approved releases that don't result in an incident	Percentage of approved releases that don't result in an Incident in a given period.
% of releases implemented in the approved release implementation window	Percentage of releases that are completed within the approved implementation window.

9.25.2. Release and Deployment Management Workflows

The Release Management category includes twelve phases, which follows the recommended Information Technology Infrastructure Library (ITIL) process by providing you with a set of approvals and tasks that you can expand.

The Release and Deployment Management workflow is illustrated in the following figure:



Release Registration and Assess[edit]

Releases may be raised either as the result of advanced planning activities, so they are coordinated with planned maintenance windows, or in response to a change request or group of change requests. The release should contain changes that are planned to be executed together, either because they all affect the same service or because they will be executed during the same release window. During the Assess phase the importance and value of the release should be determined and documented in the release record.

Plan and Design Authorization[edit]

In this phase the Release Manager works with the relevant stakeholders, such as service owners or project managers to approve the Plan and Design phase of the release. If unapproved the release is abandoned.

Release Plan and Design[edit]

Plan releases in line with requirements resulting from approved changes. Analyze all the affected Configuration Items (CIs) and CI Groups, review what software or hardware is needed to accomplish the plan, and determine the cost. This phase requires the plan and design approval before going to the next step.

Build and Test Authorization[edit]

The Release Manager reviews the Planning and Design activities and discusses the results with all relevant stakeholders to decide whether to approve the activities required to build and test the release. In case the release is rejected the Release Manager either initializes a rework of the plan and design steps or is abandoning the release.

Release Build and Test[edit]

Build effective release packages for the deployment of one or many changes into production. Test release mechanisms to ensure minimum disruption to the production environment. During this phase the individual changes that make up the release will be built and tested in isolation as part of the change management process and the combined release package will be built and tested as well. This phase requires that the build and testing of the release has been approved before going to the next phase.

Release Training[edit]

Identify and fulfill training requirements for various users. This phase is optional and is activated when you select high Impact Assessment and high Urgency for the release change.

Release Distribution Authorization[edit]

All stakeholders of the release review the Build and Test results and approving / abandoning / rejecting the planned activities. If the Build and Test results are approved, the release manager updates the implementation schedule and authorizes the release distribution to commence. In case the release is rejected the Release Manager either initializes a rework of the build and test / training activities or is abandoning the release.

Release Distribution[edit]

During the distribution phase of the release the individual changes that make up the release will be implemented in a coordinated fashion in order to maximize the likelihood of success while minimizing the required service disruption. The implementation of each individual change is executed and tracked in the Change Management process.

Release Remediation[edit]

The remediation phase is an optional phase that's entered if the release encounters an issue during the deployment phase and the work done for the release must be undone. The work required for back out will have been determined and recorded during the build and test phase.

Release Verification and Closure[edit]

After the release is completed, the results must be report for evaluation to those responsible for managing releases and then presented for stakeholder agreement. This process involves analyzing the results for the changes that make up the release and the closing of related user interactions, incidents, and known errors. The verification phase is performed to confirm that:

- The release met it objectives
- The release requestor and stakeholders are satisfied with the results
- Lessons learnt are incorporated in future releases

Abandoned[edit]

You can abandon a release in any phase before the distribution phase was approved. Prior distribution the release is still in planning and design steps hence not widely used by an audience. In these phases the release might be abandoned due to various reasons. When the release is abandoned, the release process should ensure that the reasons for this decision are properly documented. Once abandoned, the release is closed. It's not foreseen to reopen an abandoned release.

Contents

- 1 Release Registration and Assess
- 2 Plan and Design Authorization
- 3 Release Plan and Design
- 4 Build and Test Authorization
- 5 Release Build and Test
- 6 Release Training
- 7 Release Distribution Authorization
- 8 Release Distribution
- 9 Release Remediation
- 10 Release Verification and Closure
- 11 Abandoned

9.25.3. Release and Deployment Management Details

Service Manager uses the Release and Deployment Management application to enable the Release and Deployment Management process.

The following table identifies and describes some of the features on the Release and Deployment Management forms.

Release and Deployment Management field descriptions

Label	Description
Release ID	This is a system-generated unique value assigned when the release is opened.
Phase	This is a system-generated field that specifies the name of the current phase of the change. For a list of the phases, see Release and Deployment Management Workflow.
Status	The status of the release within the release phase. The status is either initial (the phase has started) or closed (the phase has ended)
Approval Status	This is a system-generated field that defines the global approval status for the change, not for a single approval. The system sets this field depending on current approvals and the approval type defined for the module. These approval statuses are available out-of-box: • Pending • Approved • Denied
Reason for Release	A code that indicates the primary reason for implementing the release. Examples of reason codes are Incident/Problem Resolution and Business Requirement.
Assigned To	The name of the person the release is assigned to.
Release Type	This field indicated the type of release. The release types available out-of-box are: • Major • Minor • Emergency This is a required field.
Release Coordinator	The person responsible for coordinating the release implementation. Each Release Coordinator may belong to several assignment groups. Each group must have only one Release Coordinator.
Initiated By	The name of the user requesting the release. This is a required field. This field includes a hover-over form that displays full name, telephone, and email address if available for the user requesting the release.
Alert Stage	This is a system-generated field that lists the current Alert Stage of this request. This field is updated automatically when alerts are processed against the release. Do not update it manually. The alerts are processed against a release by using the phase definition.
Planned Start	This field specifies the date and time that the work to implement the release should start.

Planned End	This field specifies the date and time that the work to implement the release should end.
Downtime Start	The date and time when the release is scheduled to begin. Scheduled downtime only needs to be filled when the service is down, while implementing the release.
Downtime End	The date and time when the release is scheduled to end. Scheduled downtime only needs to be filled when the service is down, while implementing the release.
Configuration Item(s) Down	If selected (set to true), indicates that the Configuration Items (CIs) are currently not operational and the downtime is scheduled. The fields Downtime Start and Downtime End are used along with the field Configuration Item(s) Down to indicate the scheduled time to bring the CI down. These fields are never required and should only be populated if you plan to bring down the CIs as part of the change. The interval selected applies to all the CIs of the change and can't be specified by individual CI. When the change is closed, you may get the form confirming the outage times, and when you close the change, the CIs will be set as Up in Configuration Management.
Risk Assessment	Specifies a code that indicates the risk incurred with the implementation of the change. These risk assessments are available out-of-box: • 0 - No Risk • 1 - Low Risk • 2 - Some Risk • 3 - Moderate Risk • 4 - High Risk • 5 - Very High Risk After a user selects this field, the release may require additional approvals based on the risk. The approval is based on the risk number in the assessment approval record.
Impact Assessment	This field specifies the impact the release has on the business. The impact and the urgency are used to calculate the priority. These impacts are available out-of-box: • 1 - Enterprise • 2 - Site/Dept • 3 - Multiple Users • 4 - User The out-of-box data is the same as Interaction Management, Problem Management, Incident Management, and Change Management. This is a required field.
Urgency	The urgency indicates how pressing the release is for the organization. The urgency and the impact are used to calculate the priority. This field functions similarly to the same field for interaction, incident, problem, and change tickets. For more information, see Service Desk Interaction Management form details. This is a required field.

Priority	This is a system-generated field using the urgency and impact of the release. This field functions similarly to the same field for interaction, incident, problem, and change tickets. For additional information, see Service Desk Interaction Management form details.
Policy / Regulation	This field is used to indicate if the release must follow specific policies or regulations.
Description	Provides a detailed description of the release. This is a required field.
Overall Assessment	A detailed description of the reasons to implement the release. It should be as detailed as possible.
Closure Code	The completion code indicates the way a release is closed. VALID VALUES • 1 - Successful • 2 - Successful (with problems) • 3 - Failed • 4 - Rejected • 5 - Withdrawn • 6 - Cancelled
Closing Comments	This field contains comments about the status of the release when it is closed.
Associated CIs	The list of Configuration Items (CIs) affected by the release.
Release Information	The Release Information section records the Products being affected by the Release along with the Package Version of the release.
Backout Method	Provides a detailed method for backing out the release if there is a problem during the deployment phase. This is a required entry during the Plan and Design phase.

Approvals	The Current Approvals section provides an overview of the current approvals related to release and important information such as approval status, and approvers as well. This includes a list of groups or operators who must acknowledge or accept the risk, cost, and so on associated with the implementation of the release. Approvals give controlling authorities the ability to stop work and to control when certain work activities can proceed. The data displayed includes the following information: • Approval Type • Approval Status • # Approved • # Denied • # Pending The Approval Log section provides an overview of past approvals related to the release as well as important information such as approval status and approvers. The data displayed includes the following information: • Action • Approver/Operator • By • Date/Time The Pending Reviews section provides the name(s) of the groups or operator IDs that should review the release after it has been approved.
Tasks	Whenever a release is in a phase where the user can generate tasks, Service Manager allows user a quick view of some of the most important fields in the task in the Tasks section. The data displayed includes the following information: • Task No • Status • Approval Status • Assigned To • Description • Category

Activities	The Activities section allows users to enter new updates for the record or view journal updates and historic activities for the record. **Journal Updates** tab • **New Update Type** Specifies or categorizes the activity update; for example, communication with the customer. • **Visible to Customer?** A checkbox that a user can select to make the update visible to a customer. • **New Update** This field is used to enter notes to explain and describe updates made for the record. If Journaling is enabled, the text entered here displays in Journal Updates. If Activities is enabled, the text entered in this field displays as an activity record for the selected activity type. **Note** Note The System Administrator is responsible for enabling Journaling and Activities. • **Journal Updates** Journal Updates shows text entered in the New Update field along with the operator's name and a timestamp for the update. This field displays information when Journaling is enabled. **Activity Logs** tab Select an activity type from the **Activity Type** drop-down list or select a user from the **User** drop-down list to filter the activity logs. The activity log lists activities for the current record in order of occurrence, with the newest at the beginning of the list. Each activity log displays the following information: • Date/Time • Operator • Description • Type • Visible to Customer icon (if you have selected the **Visible to Customer** checkbox)
Test Results	The Test Results section allows you to record the planned release testing activities along with an indicator of whether the test passed or failed and a description of the results of the test.

Attachments	You can use the Attachments section to attach documents to releases. To attach a document to a release: 1. Click **Add File...** in the Attachments section. 2. Use the Specify File Location window to browse for the file. 3. Click **OK**. The selected file appears in the Attachments section. Additional methods to attach files to an incident record with a Windows client include: You can cut or copy a file from a file management utility, or your desktop, and paste it into the Attachments section of a record. You can also cut or copy an attachment from one Service Manager record into the Attachments section of another record. Use a file management utility to open the folder containing the document you want to attach, or select a file from your desktop. Make sure that the Attachments section is open on your screen. Drag the selected file to the Attachments section.

9.26. Service Desk (Streamlined Interaction)

The following sections are based on the Streamlined Interaction solution, which is introduced as of Service Manager 9.41. The Streamlined Interaction feature is optional and is disabled by default. You need to manually enable it before you read the topics in this section. To enable the Streamlined Interaction solution, see Service Manager9.61 Help Center.



Note

Note For information about the previous Service Desk application (Codeless mode) that's not based on the new Streamlined Interaction solution, see Service Desk Overview.

9.26.1. Service Desk Overview (Streamlined Interaction)

The Service Manager Service Desk application, referred to as Service Desk throughout this chapter, supports the service desk function of the Information Technology Infrastructure Library (ITIL) with its User Interaction Management processes for the IT service and the customer base. The Service Desk application provides a single point of entry to the other Service Manager applications and enables you to document and track all calls received by the service desk.

Service Desk incorporates the essential concepts of ITIL to ensure that the best practices of IT service management are applied to the service desk to aid end customers, ensure data integrity, and streamline communication channels in the organization.

This section describes how Service Desk implements the best practice guidelines for the Service Desk Interaction Management processes.

Topics in this section include:

- Service Desk within the ITIL framework
- Service Desk application
- Service Desk process overview
- Input and output for Service Desk Interaction Management
- Key performance indicators for Service Desk Interaction Management
- RACI matrix for Service Desk Interaction Management

9.26.1.1. Service Desk within the ITIL framework

Service Operation is one of five core publications from ITIL that covers the service lifecycle. The purpose of service operation is to deliver agreed-on levels of service to users and customers, and to manage the applications, technology, and infrastructure that support delivery of the services.

The service desk is a key function of service operation. It provides a single, central point of contact for all users of IT. Service desk staff execute the incident management and request fulfillment processes to restore normal service to users as quickly as possible. Restoring normal service could involve fixing a technical fault, fulfilling a service request, or answering a query — whatever is needed to enable

users to return to their work. The service desk logs and manages customer interactions and provides an interface to other service operation processes and activities.

ITIL 2011 notes these specific responsibilities of a service desk:

- Logging, categorizing, and prioritizing all calls
- Providing first-line investigation and problem diagnosis
- Resolving incidents or service requests when first contacted or whenever possible
- Escalating incidents and service requests that can't be resolved within agreed-on time limits
- Closing resolved incidents, requests, and other calls
- Communicating with users to keep them informed of progress, impending changes, agreed-on outages, and other such notifications

9.26.1.2. Service Desk application

The Service Manager Service Desk application incorporates the ITIL best practices that are used by organizations worldwide to establish and improve their capabilities in service management.

It provides a central Service Operation function, coordinating the efficient and effective delivery of services to end users and enabling various improvements, including the following:

- Improved customer service and satisfaction
- Increased accessibility through a single point of contact and information
- Better quality and faster turnaround of customer or user requests
- Improved teamwork and communication
- Enhanced focus and a proactive approach to service provision
- Improved usage of IT resources and increased productivity of all users

The Service Desk application enables a Service Desk agent to document and track user interactions. Service Desk provides one-click access to other Service Manager applications to automatically enter information received.

The Service Desk application covers:

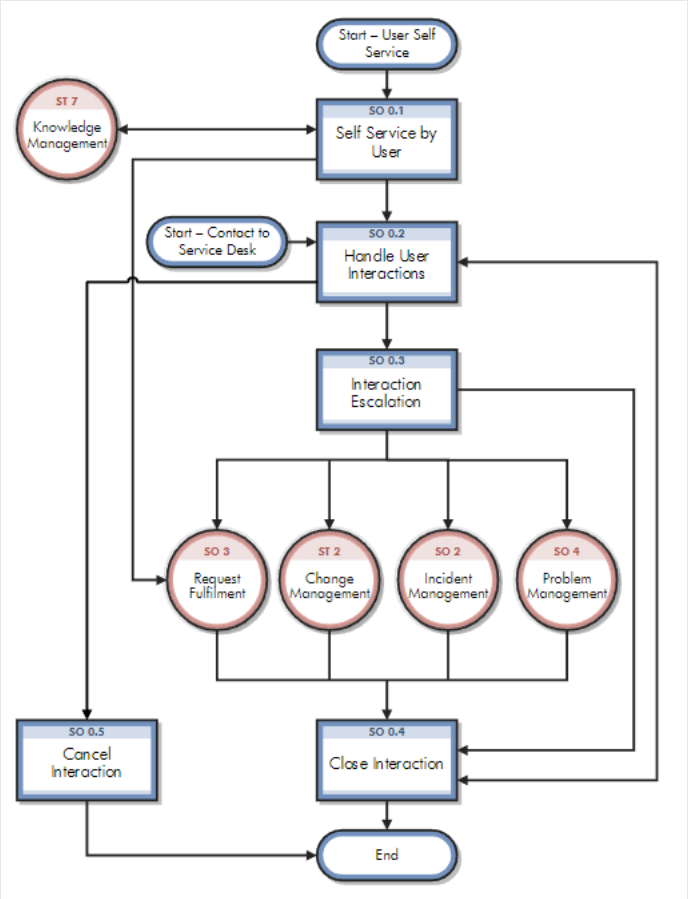
- Direct interactions between a user and the service desk by phone or by email
- User activities that occur from use of the self-service web portal (for example, searching the knowledge base, checking for status updates, or logging an interaction).

One of the best practices that derives from ITIL's service desk function is that user interactions should not be saved and updated later. Therefore, the Service Desk application requires resolving any new complaints or compliments within the agreed upon time limits and then closed, or triggering fulfillment processes for the other types of interactions. The information gathered during the customer interaction can be used to open an incident, service request, or other fulfillment records. It can also be added to a record in another Service Manager application such as Incident Management.

9.26.1.3. Service Desk process overview

Every user contact with the service desk is logged as an interaction. User Interaction Management is the process for handling all interactions with the service desk that are received from self-service Web pages or directly by service desk personnel. These interactions can include service disruptions, service requests, requests for information (RFI), complaints, or compliments reported by users who communicate with the service desk by using instant messages, phone, E-mail, or by self-service Web pages. The User Interaction Management process enables you to easily log and resolve simple user requests (such as complaints and compliments) and to escalate others into fulfillment processes (such as Incident Management) requiring further action.

A general overview of the User Interaction Management processes and workflows is depicted below. They are described in detail in Service Desk Workflows (Streamlined Interaction).



When a user contacts the service desk, the Service Desk agent uses the Service Desk application to create an interaction record. The Service Desk agent records the user name, the name of the component that the user is calling about, and a description of the service request. After collecting this information, the Service Desk agent performs the actions required to resolve the user request.

- If the user request is a complaint or compliment, the Service Desk manager can handle the interaction and close it.
- If the user request can be resolved by the Service Desk agent on first contact, the Service Desk agent will use the interaction to trigger an incident or a service request, and then close the incident or service request directly with the solution provided. Then, the interaction will be closed automatically.
- If the user request can't be resolved without escalation, the Service Desk agent can register a new record in a fulfillment process (such as the incident, service request, problem, or change fulfillment process) based on the Service Desk interaction. Service Desk copies information from the interaction record into the newly created fulfillment record.

For example, consider a user who can't print to a network printer:

- The user contacts the service desk for assistance.
- The Service Desk agent populates an interaction record with the relevant information.
- Because the issue can't be resolved immediately, the Service Desk agent opens an incident, and the incident is assigned to a technician.
- The technician discovers that the printer network connection is broken.
- The technician fixes the connection and closes the incident. The interaction is automatically closed accordingly.
- The Service Desk agent contacts the user and instructs the user to attempt printing to the network printer.
- If the user can successfully print, the Service Desk agent can send out survey based on the closed interaction. If the user still can't print, the Service Desk agent may register a new interaction and then escalate the interaction to a new incident.
- If the user wishes to report a related or new issue, the Service Desk agent opens a new interaction detailing the new issue that the user needs to report.

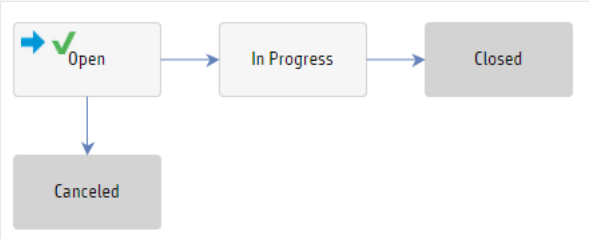
Interaction categories

Service Manager categories classify and define the type of interaction. Each category could have its own workflow process. The steps of the workflow are represented by the phases. Service Manager requires that every interaction has an interaction category and phases.

Interaction phases

Service Manager uses phases to describe the steps needed to complete an interaction. The phase also determines the forms that users see, and the actions that users can trigger manually.

The following figure shows the workflow phases for an interaction (complaint or compliment).



Service Desk user roles

The following table describes the responsibilities of the User Interaction Management user roles.

User Interaction Management user roles

Role	Responsibilities
User	• Report all IT-related requests to the service desk or use the self-service web pages. • Validate solutions and answers provided by the IT department to a registered service request.
Service Desk Agent	• Register interactions based on contact with users. • Solve and close interactions. • Provide status updates to users on request. • Register incidents based on user interactions and assign them to the correct support groups. • Register requests for change, based on user interactions. • Register service requests, based on user interactions. • Register problems, based on user interactions. • Validate solutions provided by support groups. • Report and verify solutions to users. • Monitor Service Level Agreement (SLA) targets of all interactions. • Communicate about service outages to all users.
Service Desk Manager	• Appoint people to the required roles. • Manage resources assigned to the service desk. • Manage service desk activities. • Attend CAB meetings. • Report any issue that could significantly impact the business to senior managers. • Take overall responsibility for incident and service request handling on the service desk. • Monitor and report on service desk performance. • Make improvements to the service desk.

9.26.1.4. Input and output for Service Desk Interaction Management

Interactions can be triggered and resolved in several ways. The following table outlines the input and output for the User Interaction Management process.

Input and output for Service Desk Interaction Management

Input to Service Desk Interaction Management	Output from Service Desk Interaction Management
A user can contact Service Desk and give input by using instant messages, phone, email, self-service web pages, or other means.	Service Desk personnel can handle an interaction in the following ways: • If the interaction is a user complaint or compliment, the interaction is handled by Service Desk. • If the interaction can be resolved by Service Desk when first contacted, the interaction will be triggered to either an incident or a service request and closed by Service Desk directly with a solution provided. • If the interaction requires an incident, service request, problem, or change, the interaction is sent to different fulfillment processes.

9.26.1.5. Key performance indicators for Service Desk Interaction Management

The Key Performance Indicators (KPIs) in the following table are useful for evaluating your indicators for Service Desk Interaction Management processes. To visualize trend information, it is useful to graph KPI data periodically. In addition to the data provided by Service Manager, you may need additional tools to report on all of your KPI requirements.

Key Performance Indicators for Service Desk Interaction Management

Title	Description
First time fix	Percentage of interactions closed by the Service Desk agent at first contact without reference to other levels of support
First line fix	Percentage of records closed by the service desk without reference to other levels of support
Customer satisfaction	Customer satisfaction measured by surveys completed by customers

For completeness, the ITIL 2011 and COBIT 4.1 KPIs are included below.

ITIL 2011 Key Performance Indicators[edit]

The following are ITIL 2011 KPIs for User Interaction Management:

- Percentage of incidents closed by the service desk without reference to other levels of support (that is, closed by first point of contact).
- Number and percentage of incidents processed by each Service Desk agent.

COBIT 4.1 Key Performance Indicators[edit]

The following are the COBIT 4.1 KPIs for User Interaction Management:

- Amount of user satisfaction with first-line support (service desk or knowledgebase)
- Percent of first-line resolutions based on total number of requests
- Call-abandonment rate
- Average speed to respond to telephone and email or Web requests
- Percent of incidents and service requests reported and logged using automated tools
- Number of days of training per service desk staff member per year
- Number of calls handled per service desk staff member per hour
- Number of unresolved queries

Contents

- 1 ITIL 2011 Key Performance Indicators
- 2 COBIT 4.1 Key Performance Indicators

9.26.1.6. RACI matrix for Service Desk Interaction Management

A Responsible, Accountable, Consulted, and Informed (RACI) diagram or RACI matrix is used to describe the roles and responsibilities of various teams or people in delivering a project or operating a process. It is especially useful for clarifying roles and responsibilities in cross-functional/departmental projects and processes. The RACI matrix for Service Desk Interaction Management is shown in the following table.

RACI matrix for Service Desk Interaction Management

Process ID	Activity	User	Service Desk Agent	Service Desk Manager
SO 0.1	Self-Service by User	R	I	A
SO 0.2	Interaction Handling	R	R/I	A/R
SO 0.3	Interaction Escalation	R	R	A
SO 0.4	Close Interaction	R/I	R	A
SO 0.5	Cancel Interaction	R/I	R	A

9.26.2. Service Desk Workflows (Streamlined Interaction)

Every time a user contacts the service desk, as an interaction is logged. User interaction management is the process of handling all interactions with the service desk that are received from self-service web pages or directly by service desk personnel. These interactions can include service disruptions, service requests, requests for information (RFI), complaints, or compliments reported by users who communicate with the service desk by using instant messages, phone, email, or self-service web pages.

The process enables Service Desk agents to easily log and resolve simple user requests and to escalate others into fulfillment processes requiring further action. The process streamlines service desk activities and decreases the workload for second-line support teams.

The User Interaction Management process consists of the following processes, which are included in this chapter:

- Self-Service by User (process SO 0.1)
- Interaction Handling (process SO 0.2)
- Interaction Escalation (process SO 0.3)
- Close Interaction (process SO 0.4)
- Cancel Interaction (process SO 0.5)

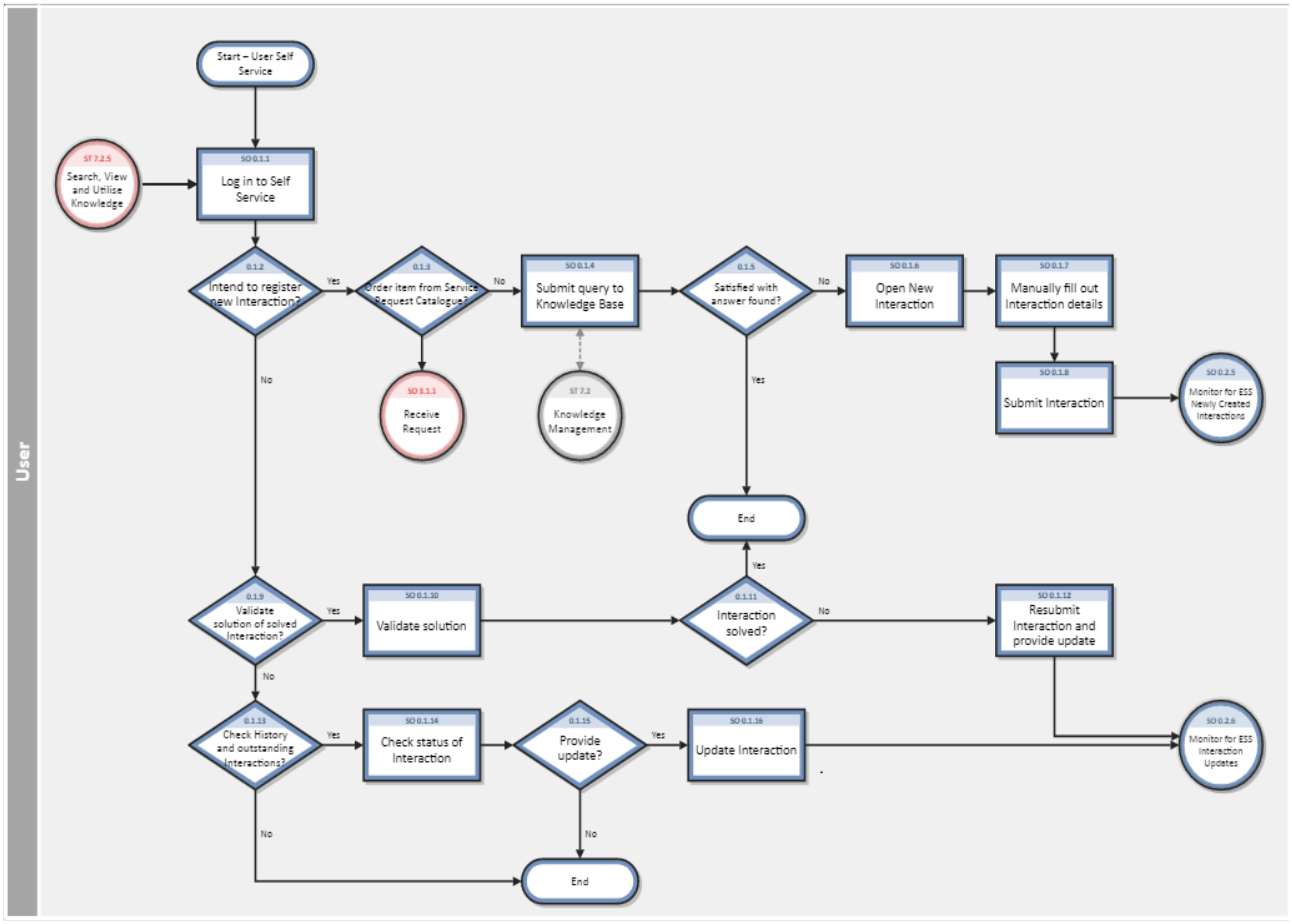
9.26.2.1. Self-Service by User (process SO 0.1)

By using the self-service web environment, users can perform the following activities without contacting the service desk:

- Search the knowledgebase to find an answer to a question or issue
- Monitor the status of previously reported interactions
- Log new interactions
- Order items from the service catalog

You can see the details of this process in the following figure and table.

Self-Service by User (SO 0.1) is illustrated in the following figure:



Self-Service by User (SO 0.1) process

Process ID	Procedure or Decision	Description	Role
SO 0.1.1	Log in to Self-Service	To gain access to the Self-Service Web interface, users must log on by using their login credentials.	User
SO 0.1.2	Intend to register new interaction?	If yes, continue with SO 0.1.3. If no, go to SO 0.1.9.	User
SO 0.1.3	Order item from Service Request Catalog?	If yes, log a service request. If no, go to SO 0.1.4.	User
SO 0.1.4	Submit query to Knowledge Base	To search for a knowledge document, users must complete a search.	User
SO 0.1.5	Satisfied with answer found?	If yes, stop. If not, go to SO 0.1.6.	User
SO 0.1.6	Open new interaction	Create a new interaction.	User
SO 0.1.7	Manually fill out interaction details	To register a new interaction, users must provide a title and a description of the request; select the urgency, and preferred contact method; and can optionally provide affected Service and add an attachment.	User
SO 0.1.8	Submit interaction	When all mandatory fields are completed, submit the form to send the request to the service desk.	User
SO 0.1.9	Validate solution of solved interaction?	To validate the solution to a previously reported interaction, go to SO 0.1.10. If no, go to SO 0.1.13.	User
SO 0.1.10	Validate solution	Use View Closed Requests to get an overview of all solved interactions. Select the applicable interaction and validate the solution provided.	User
SO 0.1.11	Interaction solved?	If yes, stop. If not, go to SO 0.1.12.	User
SO 0.1.12	Resubmit interaction and provide update	When a user disagrees with the proposed solution, the user can resubmit the interaction and provide a reason for the disagreement. The newly-created interaction is sent to the service desk for further diagnosis.	User

SO 0.1.13	Check history and outstanding Interactions?	If a user wants to check the status or history of previously registered interactions, go to SO 0.1.14. If no, stop.	User
SO 0.1.14	Check status of Interaction	Use View Open Requests to get an overview of all open interactions. Select the interaction and view the status with last updates.	User
SO 0.1.15	Provide update?	If a user has additional details to add to the previously-logged interaction that may be useful to know for the specialist, go to SO 0.1.16. If no, stop.	User
SO 0.1.16	Update Interaction	There are two scenarios in which you may update an interaction and then require a Save button to save the updated information. • The Save button appears when a self-service user selects the option View Open Requests, selects an interaction, and clicks the Update button. Once the information is updated, the self-service user clicks **Save & Exit** to save the updated information in the request. • When you escalate an interaction, you can go back to the interaction to add more information or perform changes to it. You then have an Update button when you select an existing interaction. The interaction is also a status of Dispatched. Once you have added more information to the request or performed the changes, you can click **Save & Exit**.	User

9.26.2.2. Interaction Handling (process SO 0.2)

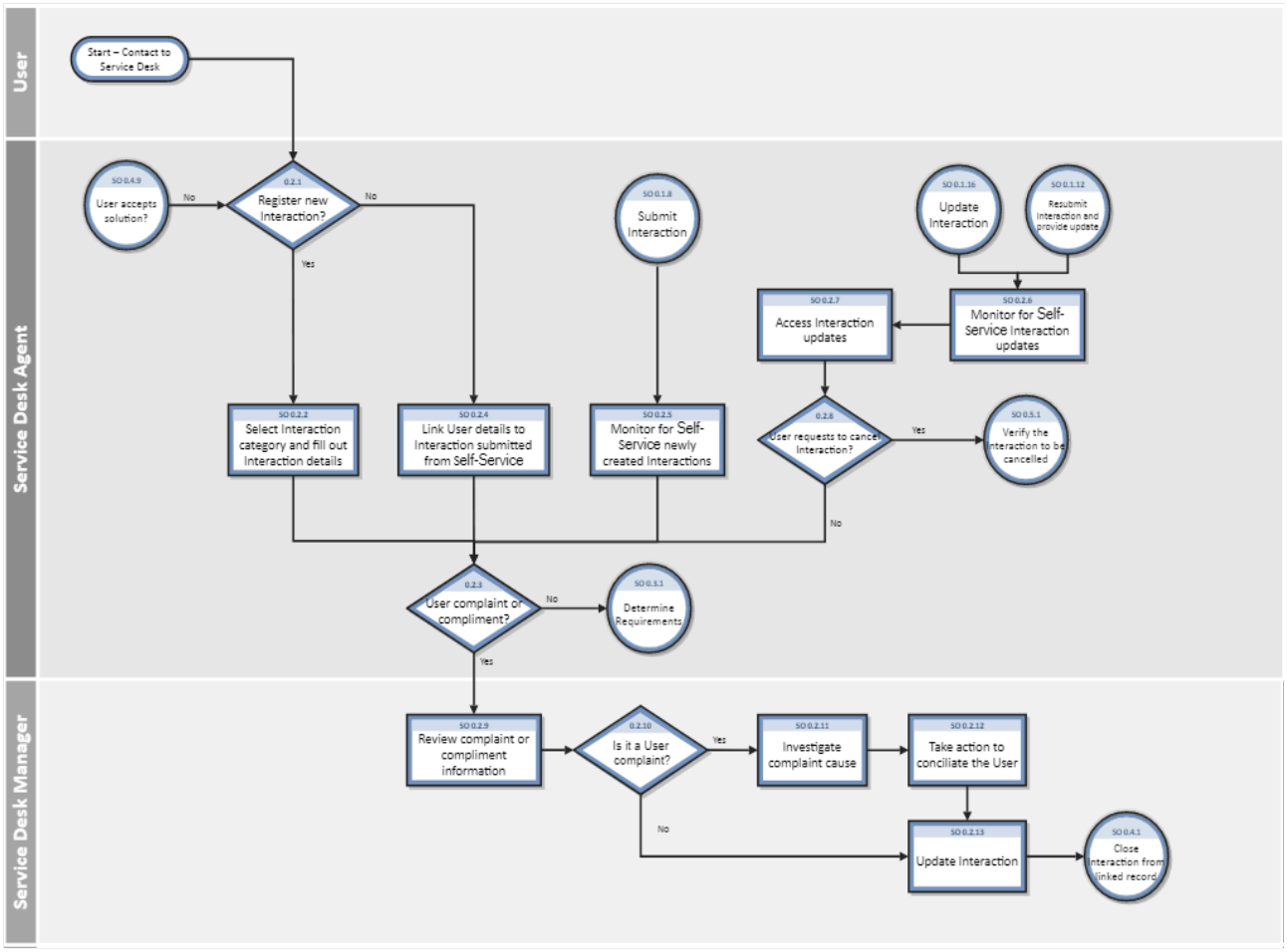
The service desk is responsible for handling all user interactions received by the self-service Web portal, email, or phone. The service desk attempts to resolve an interaction of user complaints or compliments.

Handling user complaints or compliments is the process that the Service Desk Manager is responsible for.

When the Service Desk Manager receives assigned interactions, the manager investigates the cause of the complaint or compliment by evaluating the relevant information and talking to the people involved. The manager searches for an answer or solution to satisfy the user who filed the complaint, updates the interaction with the agreed details, and then closes the interaction.

You can see the details of this process in the following figure and table.

Interaction Handling (SO 0.2) is illustrated in the following figure:



Interaction Handling (SO 0.2) process

Process ID	Procedure or Decision	Description	Role
SO 0.2.1	Register new Interaction?	If a new interaction is required, go to SO 0.2.2. If not, go to SO 0.2.4.	Service Desk Agent
SO 0.2.2	Select Interaction category and fill out Interaction details	Select the appropriate Interaction category from the available category list and fill out the required Interaction details such as a full Description, Contact, Service Recipient, and so on.	Service Desk Agent
SO 0.2.3	User complaint or compliment?	If the Interaction is a user complaint or compliment, go to SO 0.2.9. If not, go to SO 0.3.1 to determine what requirement of the user request is.	Service Desk Agent
SO 0.2.4	Link User details to Interaction submitted from Self-Service	In Self-Service Interaction, fill in the name of the caller in the Contact field and the name of the User in the Service Recipient field (if different).	Service Desk Agent
SO 0.2.5	Monitor for Self-Service newly created Interactions	If there are new Interactions, follow the same Interaction registration process.	Service Desk Agent
SO 0.2.6	Monitor for Self-Service Interaction updates	If Interactions are updated, they must be reassessed.	Service Desk Agent
SO 0.2.7	Assess Interaction updates	Evaluate Interactions that have been updated or resubmitted.	Service Desk Agent
SO 0.2.8	User requests to cancel Interaction?	If the User requests to cancel Interaction, go to SO 0.5.1 to verify the Interaction to be cancelled. If not, go to SO 0.2.3.	Service Desk Agent
SO 0.2.9	Review complaint or compliment information	The Service Desk Manager reviews the detailed information.	Service Desk Manager
SO 0.2.10	Is it a User complaint?	If it is a user complaint, go to SO 0.2.11. If not, go to SO 0.2.13.	Service Desk Manager
SO 0.2.11	Investigate complaint cause	The Service Desk Manager investigates the cause of the complaint by looking at the relevant information and talking to the people involved. The Service Desk Manager also searches for an answer or solution to satisfy the User who filed the complaint.	Service Desk Manager

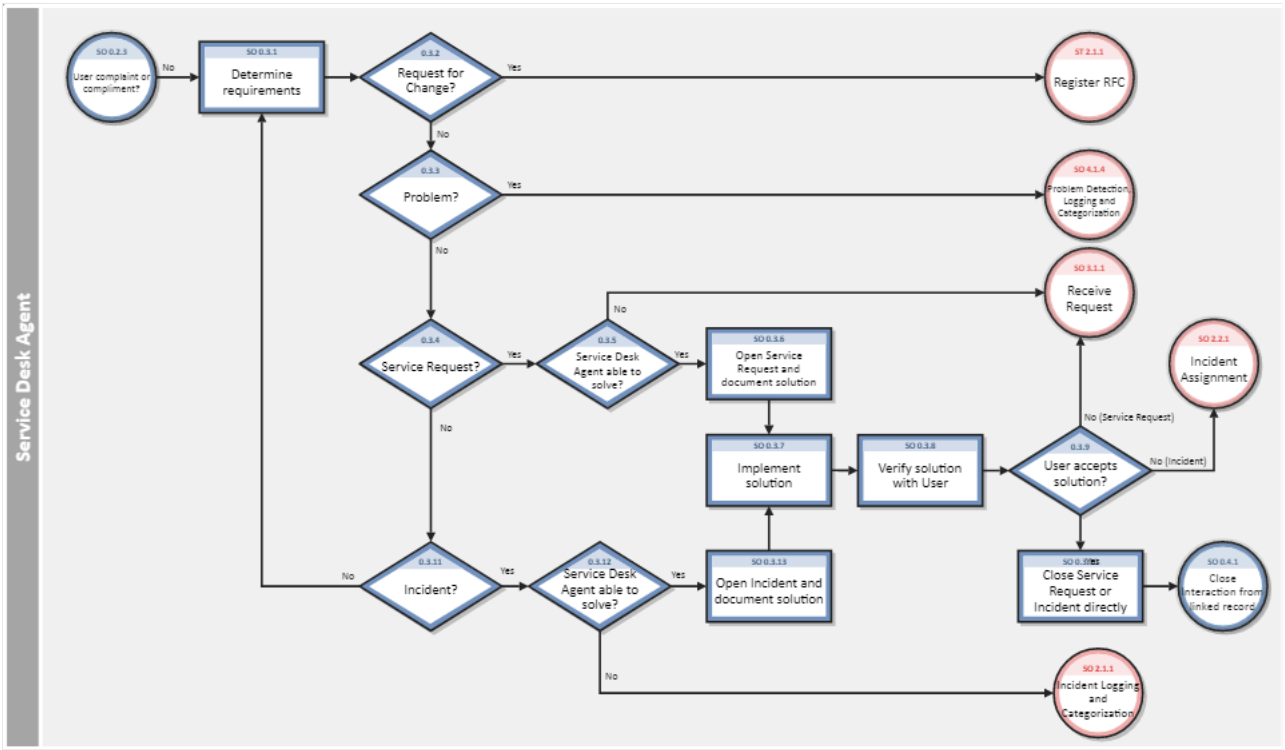
SO 0.2.12	Take action to conciliate the User	The Service Desk Manager contacts the User to solve the User's issue and tries to reach an agreement.	Service Desk Manager
SO 0.2.13	Update Interaction	The Service Desk Manager updates the Interaction with the agreed solution, and then go to SO 0.4.1 to close the Interaction.	Service Desk Manager

9.26.2.3. Interaction Escalation (process SO 0.3)

When an Interaction is received, the Service Desk Agent first determines if the Interaction is a service request, a request for change, a problem or an incident, and if so, logs the request.

You can see the details of this process in the following figure and table.

Interaction Escalation (SO 0.3) is illustrated in the following figure:



Interaction Escalation (SO 0.3) process

Process ID	Procedure or Decision	Description	Role
SO 0.3.1	Determine requirements	After filling out the Interaction details, the Service Desk Agent determines what requirement of the user request is.	Service Desk Agent
SO 0.3.2	Request for Change?	If a Change is required, go to ST 2.1.1 to log the RFC. If not, go to SO 0.3.3.	Service Desk Agent
SO 0.3.3	Problem?	If a Problem is needed, go to SO 4.1.4 to create a new Problem. If not, go to SO 0.3.4.	Service Desk Agent
SO 0.3.4	Service Request?	If a Service Request is required, go to SO 0.3.5. If not, go to SO 0.3.11.	Service Desk Agent
SO 0.3.5	Service Desk Agent able to solve?	If the Service Desk Agent is able to solve the Service Request, go to SO 0.3.6. If not, go to Request Fulfillment process SO 3.1.1.	Service Desk Agent
SO 0.3.6	Open Service Request and document solution	The Service Desk Agent creates a new Service Request and documents the solution to be implemented.	Service Desk Agent
SO 0.3.7	Implement solution	The Service Desk Agent then implements the solution for the User.	Service Desk Agent
SO 0.3.8	Verify solution with User	The Service Desk Agent contacts the User and communicates the solution. The user should verify the solution and confirm that the Service Request is fulfilled or the Incident is resolved.	Service Desk Agent
SO 0.3.9	User accepts solution?	If the User accepts the solution, go to SO 0.3.10. If not, go to Request Fulfillment process SO 3.1.1 for Service Request or Incident Management process SO 2.2.1 for Incident assignment.	Service Desk Agent
SO 0.3.10	Close Service Request or Incident directly	The Service Desk Agent closes the Service Request or Incident on first intake.	Service Desk Agent
SO 0.3.11	Incident?	If it is an Incident, go to SO 0.3.12. If not, go to SO 0.3.1.	Service Desk Agent

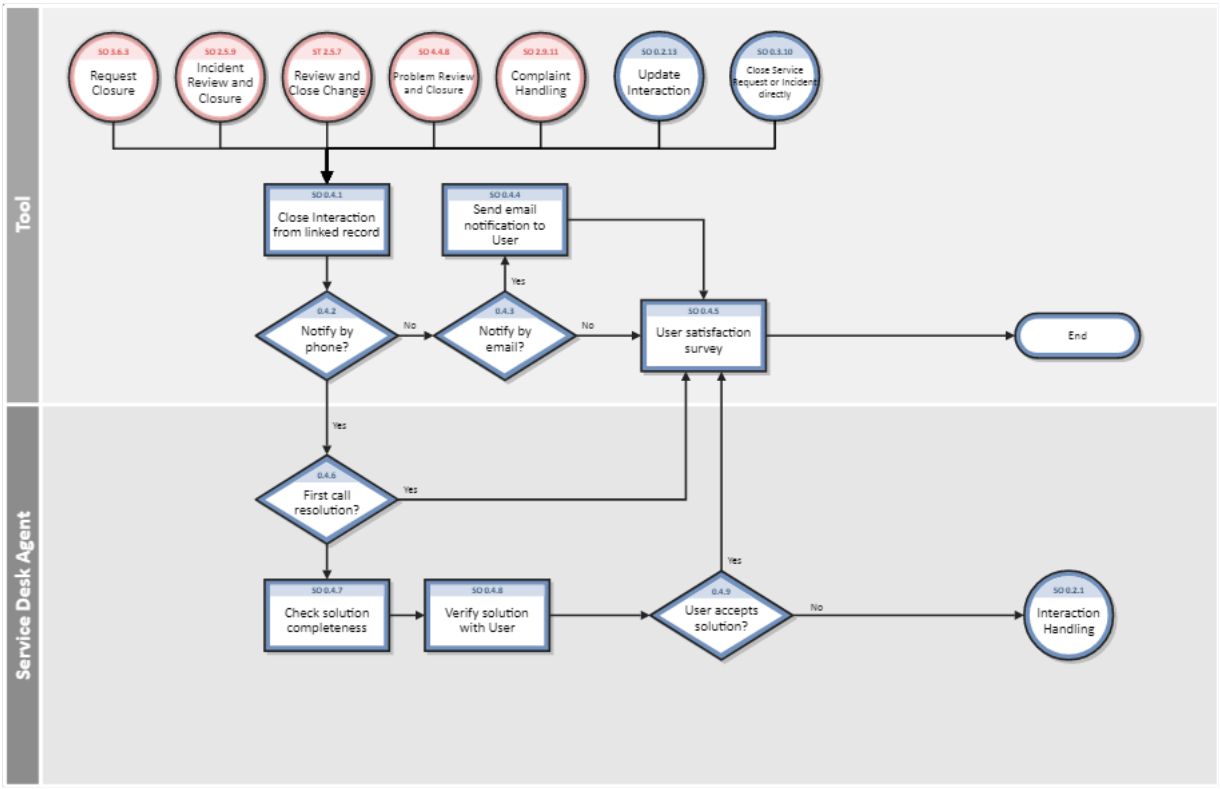
SO 0.3.12	Service Desk Agent able to solve?	If the Service Desk Agent is able to solve the Incident, go to SO 0.3.13. If not, go to Incident Management process SO 2.1.1.	Service Desk Agent
SO 0.3.13	Open Incident and document solution	The Service Desk Agent creates a new Incident and documents the solution to be implemented. Then go to SO 0.3.7.	Service Desk Agent

9.26.2.4. Close Interaction (process SO 0.4)

When an interaction is solved by a related incident, change, request, or problem that's resolved, the interaction is closed. Based on user preferences, the Service Desk communicates the solution to the user by phone or email.

You can see the details of this process in the following figure and table.

Close Interaction (SO 0.4) is illustrated in the following figure:



Interaction Closure (SO 0.4) process

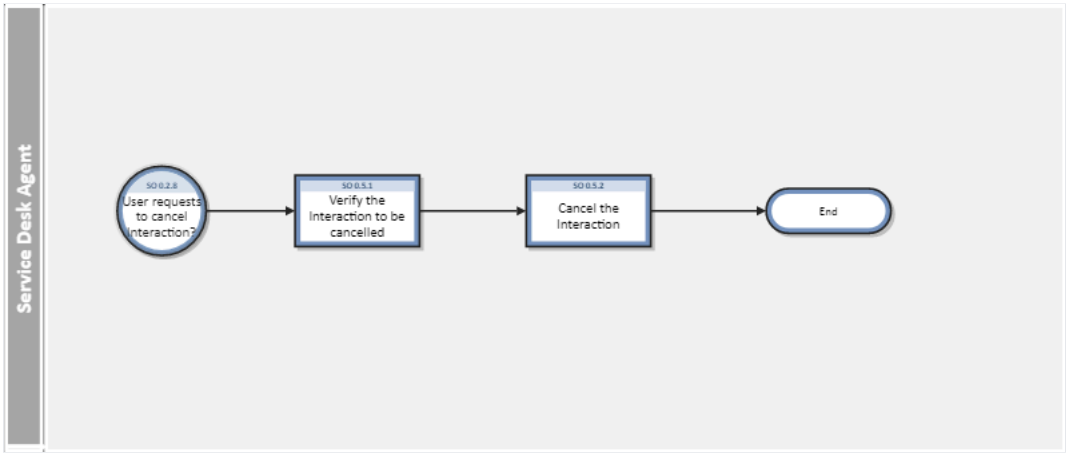
Process ID	Procedure or Decision	Description	Role
SO 0.4.1	Close Interaction from linked record	The Interaction closure is caused by the closure of an Incident, Change, Problem or Service Request, or the submission of a User complaint or compliment.	Tool
SO 0.4.2	Notify by phone?	If the Notify By method states that the User wants to be notified by phone, go to SO 0.4.6. If not, go to SO 0.4.3.	Tool
SO 0.4.3	Notify by email?	If the Notify By method states that the User wants to be notified by email, go to SO 0.4.4. If not, go to SO 0.4.5.	Tool
SO 0.4.4	Send email notification to User	Email notification is sent to User.	Tool
SO 0.4.5	User satisfaction survey	A user satisfaction survey that's automatically generated by the system can be sent out.	Tool
SO 0.4.6	First call resolution?	If the related Service Request or Incident is closed by Service Desk Agent on first contact, go to SO 0.4.5. If not, go to SO 0.4.7.	Service Desk Agent
SO 0.4.7	Check solution completeness	The Service Desk Agent checks the solution provided by fulfillment team.	Service Desk Agent
SO 0.4.8	Verify solution with User	The Service Desk Agent contacts the User and communicates the solution. The user should verify the solution and confirm that the incident is solved, the question or complaint is answered, or the Service Request is fulfilled.	Service Desk Agent
SO 0.4.9	User accepts solution?	If the User accepts the solution, go to SO 0.4.5. If not, go to SO 0.2.1 to register new Interaction.	Service Desk Agent

9.26.2.5. Cancel Interaction (process SO 0.5)

The Cancel Interaction process identifies the steps to cancel an interaction.

You can see the details of this process in the following figure and table.

Cancel Interaction (process SO 0.5) is illustrated in the following figure:



Cancel Interaction (SO 0.5) process

Process ID	Procedure or Decision	Description	Role
SO 0.5.1	Verify the Interaction to be cancelled	The Service Desk Agent verifies the User updates to check whether the Interaction can be cancelled or not.	Service Desk Agent
SO 0.5.2	Cancel the Interaction	The Service Desk Agent cancels the Interaction based on User updates.	Service Desk Agent

9.26.3. Service Desk Details (Streamlined Interaction)

Service Manager uses its Service Desk application to enable the Service Desk Interaction Management process. The main function of Service Desk is to monitor, track, and record calls and toopen fulfillment records, as necessary.

In User Interaction Management, a Service Desk Agent receives a call and opens a new interaction. The Service Desk Agent fills in the required fields, and then chooses to close the interaction or escalate it to a fulfillment process.

This section describes selected User Interaction Management fields in the out-of-box Service Manager system.

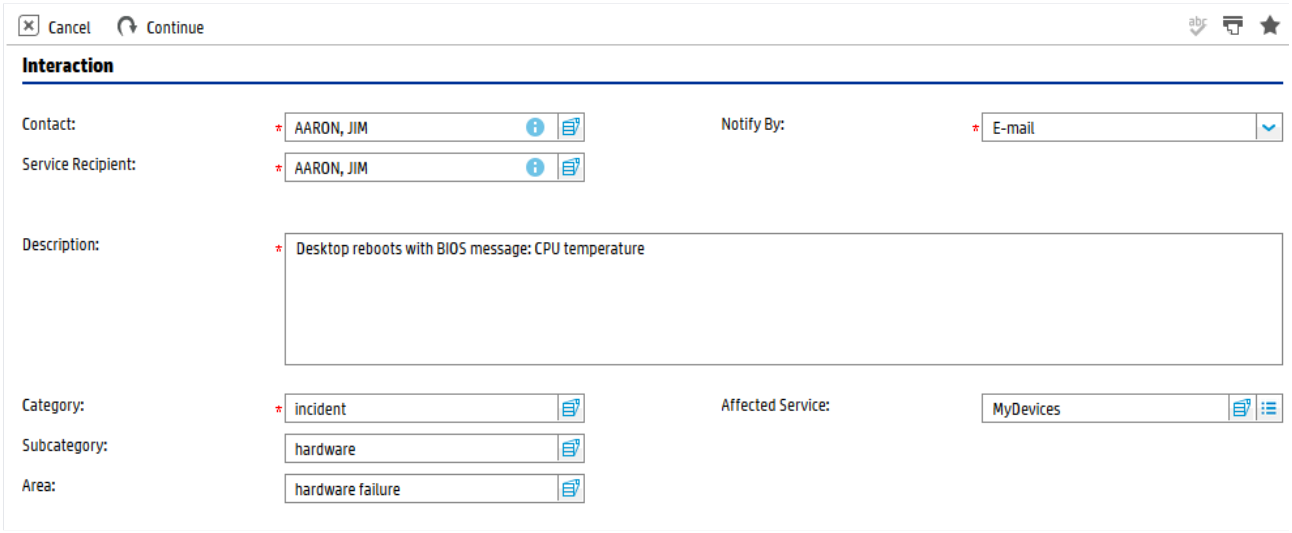
Topics in this section include:

- New interaction form
- Interaction form after escalation
- Service Desk Interaction Management form details
- Interaction categories

9.26.3.1. New interaction form

When a Service Desk Agent clicks **Create Streamlined Interaction**, Service Desk displays the new interaction form. The required fields in this form must be populated to register the new interaction. Service Desk fills in some of the fields automatically. The Service Desk Agent must fill in the others.

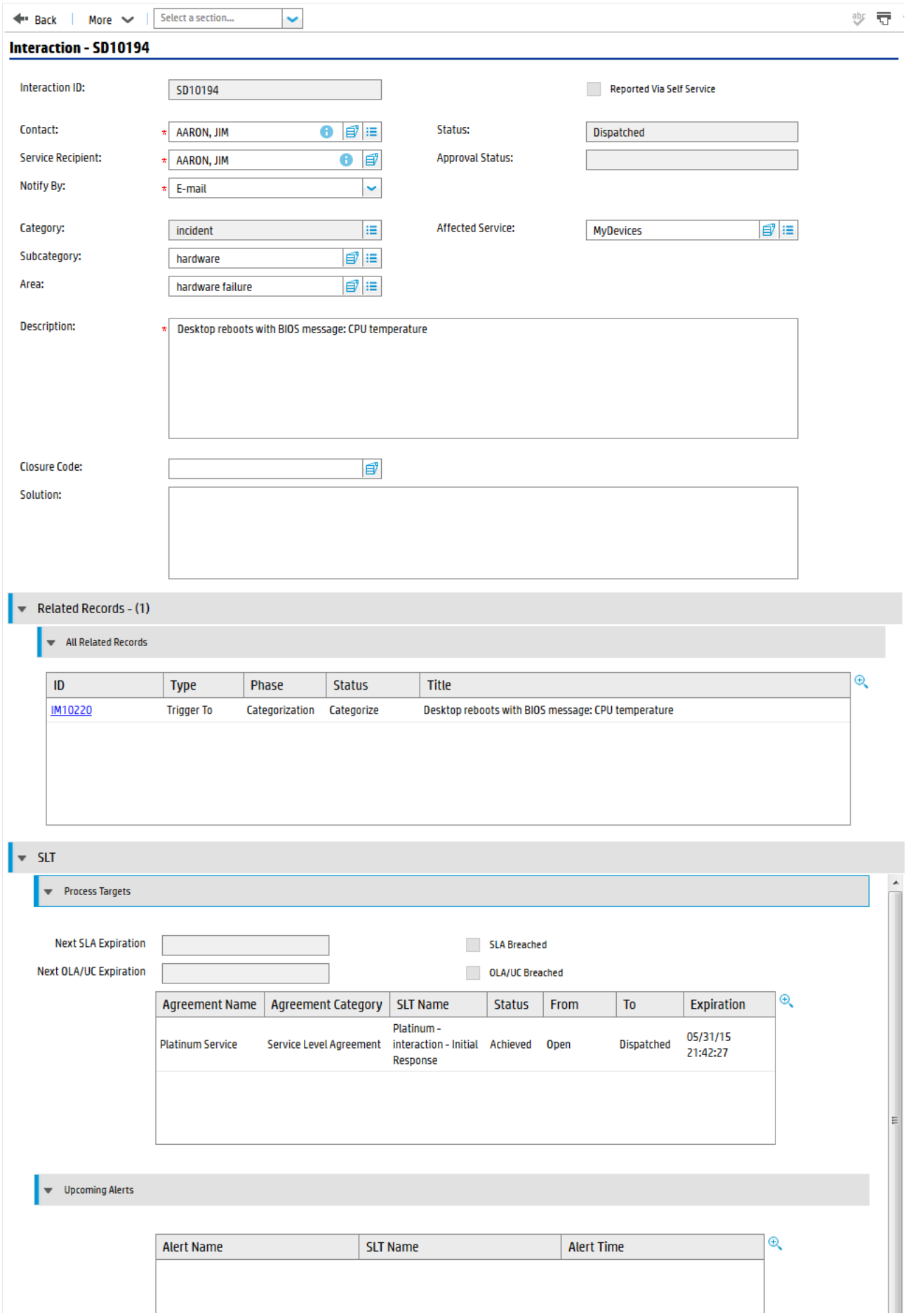
A new interaction from that has been completed is illustrated in the following screenshot:

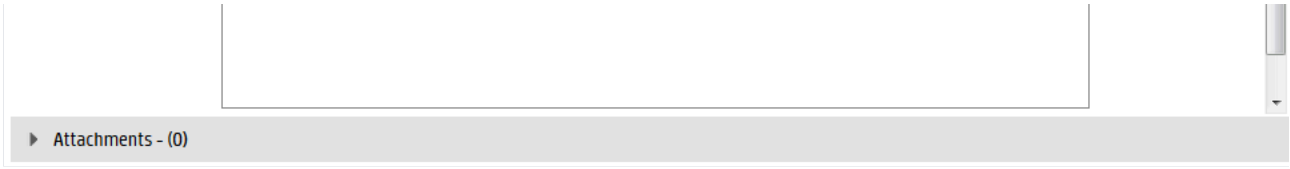


9.26.3.2. Interaction form after escalation

After the Service Desk Agent escalates the interaction, Service Desk displays new sections and fields.

The same interaction after escalation is illustrated in the following screenshot:


Back
More
Select a section...
Interaction - SD10194
Interaction ID:
SD10194
Reported Via Self Service
Contact:
AARON, JIM
Status:
Dispatched
Service Recipient:
AARON, JIM
Approval Status:
Notify By:
E-mail
Category:
incident
Affected Service:
MyDevices
Subcategory:
hardware
Area:
hardware failure
Description:
Desktop reboots with BIOS message: CPU temperature
Closure Code:
Solution:
Related Records - (1)
All Related Records
ID
Type
Phase
Status
Title
IM10220
Trigger To
Categorization
Categorize
Desktop reboots with BIOS message: CPU temperature
SLT
Process Targets
Next SLA Expiration
SLA Breached
Next OLA/UC Expiration
OLA/UC Breached
Agreement Name
Agreement Category
SLT Name
Status
From
To
Expiration
Platinum Service
Service Level Agreement
Platinum - interaction - Initial Response
Achieved
Open
Dispatched
05/31/15 21:42:27
Upcoming Alerts
Alert Name
SLT Name
Alert Time



9.26.3.3. Service Desk Interaction Management form details

The following table identifies and describes some of the features on Service Desk's Interaction Management forms.

Service Desk Interaction Management form details

Label	Description
Interaction ID	Service Manager populates this field with a unique ID when a Service Desk Agent registers a new interaction.
Status	The options in this field have been revised to align with our new best practices. **Tip** Tip You may want to tailor these options to match your business needs. These statuses are available out-of-box: • Open — The interaction has been initially logged. For example, when the Service Desk Agent is still on the phone with the customer. • In Progress — The interaction has no incidents, changes, or other records related to it. It is fulfilled by Service Desk. • Dispatched — The interaction has been escalated or the catalog request approved and the interaction is now related to another record, such as an incident, change, problem, or request. • Pending Customer - The interaction is pending for User input to proceed its fulfillment process. • Pending Approval - The service catalog request is pending for approval. • Closed — The interaction is closed by the help desk or automatically after the related record is closed. • Canceled — The interaction is canceled by Service Desk Agent based on the user updates. • Resolved — The interaction is resolved when the linked incident is move to the "Resolved" status.
Contact	The Service Desk Agent populates this field with the person from whom the call was received. The contact person is not necessarily the same person as the service recipient. This field ensures that the correct person will be notified about updates to the interaction. This field includes a hover-over form that displays full name, telephone, and email address for the contact, if available. This is a required field.
Service Recipient	The person who has the problem and needs it resolved. It's not necessarily the person who is calling to report the problem. Filling in this field automatically fills in the contact name from the contact record of who should be notified of the resolution. The Service Desk Agent populates this field with the person this issue is registered for. This field includes a hover-over form that displays full name, telephone, and email address if available for the service recipient. This is a required field.

Notify By	To notify the customer when the issue has been resolved, Service Manager prepopulates this field as E-mail. The user or the Service Desk Agent can change it to None or Telephone, if applicable. When the related fulfillment record is closed: • Selecting E-mail closes the interaction and sends an email to the contact • Selecting None closes the interaction without notifying the contact • Selecting Telephone tells the Service Desk Agent to ask the contact whether the solution is satisfactory. If the solution works for the customer, the Service Desk Agent closes the interaction. If it doesn't work, then the Service Desk Agent must open a new interaction. This is a required field.
Affected Service	The Service Desk Agent populates this field with the business service affected by the registered issue. Only business services the service recipient has a subscription for can be selected. ITIL 2011 is centered around services, so a service construct should always be defined for best practices. If you have not yet created a service construct, start with a catch-all service, such as My Devices. **Note** Note The out-of-box options in this field are based on past Service Manager implementations. You should tailor these options to match your business needs. These business services are available out-of-box: • Applications • Email/Webmail • Handheld PDA & Telephony • Intranet • Internet • My Devices (The My Devices service represents all personal devices that the user would use.) • Printing Selecting the service: • Validates that it is a valid service. An end user is more likely to know that the email service doesn't work than what part of the email service doesn't work. This is a required field.
Title	A short description that summarizes the interaction. This is a required field.
Tags	Enter one or more free-form tags (keywords or terms) of this interaction. The tags you enter are searchable and can be used as keywords to highlight important information. For more information, see Working with tags.

Description	The Service Desk Agent populates this field with a detailed description of the interaction. When the location and telephone number differ from the contact details, the Service Desk Agent can record the correct information in the description field. **Note** Note Service Manager searches this field when you perform an advanced or expert text search. This is a required field.
Closure Code	This field contains a predefined closure code, describing the way this issue has been solved. The out-of-box options in this field are based on Service Manager customer reference data. **Note** Note You may want to tailor these options to match your business needs. These closure codes are available out-of-box: • Automatically Closed • Cancelled • Denied Service Catalog Request • Fulfilled • Invalid • Not Reproducible • Out of Scope • Request Rejected • Solved by Change/Service Request • Solved by User Instruction • Solved by Workaround • Unable to solve • Withdrawn by User
Solution	This field contains a description of the solution used for this interaction. **Note** Note Service Manager searches this field when you perform an advanced or expert text search.

Category	This field describes the type of interaction. The interaction type determines the process to escalate to when the interaction is not a user complaint or compliment. The categories are based on ITIL service-centric processes, and therefore focus on enabling assignment, reporting, and operational analysis for knowledge management purposes. From the category list: • Incident > Continue — You can relate the interaction to a new incident. • Request for Change > Continue — Service Manager creates a new change request. • Service Request/Request for Administration/Request for Information > Continue - You can relate the interaction to a new service request. • Problem > Continue - Service Manager creates a new problem. • Support Catalog > Continue - Support Catalog opens, allowing you to place an order. After an order is submitted, the interaction is given the category configured in the ordered support catalog item and a related record is created as defined in the ordered support catalog item as well. • Service Catalog > Continue — Service Catalog opens, allowing you to place an order. The interaction is given the category service catalog. Service Catalog interactions aren't escalated. When you approve the interaction, it opens the related record as defined in the service catalog connector. • Complaint/Compliment > Continue - No related record is created. Service Desk handles such types of interactions. **Note** Note The Service Desk Agent needs to have proper permissions to open corresponding fulfillment records. For more information on categories and the subcategories and areas associated with them, see Interaction categories. This is a required field.
Subcategory	The Service Desk Agent populates this field with the subcategory of concern. Service Manager displays different lists of subcategories, depending on the category you selected. For more information on categories and the subcategories and areas associated with them, see Interaction categories.
Area	The third level of classifying an interaction, mainly used for reporting purposes. Service Manager displays different lists of areas, depending on the subcategory you selected. For more information on categories and the subcategories and areas associated with them, see Interaction categories.

Assignment Group	The group assigned to work on the user complaint or compliment. For a description of this field see the Assignment Group field description in the Incident Management form details section as this field functions similarly. The out-of-box data consists of default assignment groups for use as examples of types of assignment groups. You may want to change the sample assignment groups to meet your own needs. These assignment groups are available out-of-box: • Application • Email / Webmail • Field Support • Hardware • Incident Managers • Intranet / Internet Support • Network • Office Supplies • Office Support • Operating System Support • Problem Coordinators • Problem Managers • SAP Support • Service Desk • Service Desk Analysts • Service Manager
Assignee	The name of the person assigned to work on the user complaint or compliment. This person is a member of the assigned support group. Assignees may belong to one or multiple assignment groups, based on the needs of your company.
Approval Status	This field is only used when you request something from the catalog. When you submit an order from the catalog, Service Manager automatically creates an interaction which, based on approval requirements, may have to be approved before it can be fulfilled. Service Manager populates this field with the current approval status for this interaction. These approval statuses are available out-of-box: • pending — The request has not been approved or a prior approval or denial has been retracted. • approved — All approval requirements are approved, or no approval necessary. • denied — The request has been denied.
Activities	The Activities section records information that Service Desk enters during the lifecycle of a user complaint or compliment. Every time you update a user complaint or compliment, you can fill in an update on the Activities section (New Update). A log of all the updates is stored on the Journal Updates and activities list. Self-Service updates from users are also displayed here.

Related Records	The Related Records section contains a list of all related records for the interaction. These may include related incidents, problems, changes, and requests.
SLT	The SLT (Service Level Target) section displays SLAs related to the interaction. SLAs in interactions are customer-related and selected, based on the customer contact or department and service related to the issue. The Process Targets define the details, such as beginning and ending state, and time allowed between these states. SLA selection takes place when a Service Desk Agent escalates the interaction. The best practice is that the Service Desk Agent should communicate the time of the next breach to the customer at this point. If SLAs are configured to be handled in the background, the information in this section may not display immediately. **Note** Note The out-of-box system is set up to run SLAs in the foreground. Tailoring the system to run SLAs in the background complicates communicating with the customer and should be avoided.
SLT View	If you set the View Type property of the Calendar control to **Calendar** or leave this property blank, this tab displays a normal view of the embedded Service Manager calendar. You can view the time period records and associated business records in a graphic and intuitive user interface. For more information about the Service Manager Calendar, see Calendar control and Calendar Administration. If you set the View Type property of the Calendar control to **SLM**, this tab visualizes the operation status for all related SLA, OLA, and UC of the current record. For more information about SLM visualization, see Visualize Service Level Management.
Continue button	The Service Desk Agent clicks this button to create an incident, problem, request, or change from this interaction. If the Service Desk has a role in the Incident Management process, this incident may be assigned to the Service Desk, and the Service Desk Agent can still work on it.
Cancel Interaction button	The Service Desk Agent clicks this button to cancel the interaction on behalf of the user.
Close button	The Service Desk clicks this button to close the user complaint or compliment. The customer's issue was resolved and requires no further action.
Fulfill button	The Service Desk handles the user complaint or compliment.
More > Send Survey	After interaction is closed, the Service Desk Agent can carry out a user satisfaction survey.

9.26.3.4. Interaction categories

The category hierarchy was designed to support the ITIL 2011 model of service-centric support. It is a natural-language-based hierarchy meant to enable the Service Desk Agent to easily classify the Interaction. The three-level hierarchy (category, subcategory, and area) creates a “sentence” that clearly and uniquely defines the issue without ambiguity.

The category determines which process the record belongs to. Combined with the subcategory and area, it also is used for to report results and to determine the knowledgebase assignment for the event.

Note

Note Since the category values represent best practices, customizing this data is not expected. The subcategory and area fields can be customized; however, they should cover the scope of the IT Service provisioning in natural language definition and should remain unmodified. If you choose to customize the subcategories and areas, we recommend that you set them up in a natural easy-to-follow hierarchy.

The categories, subcategories, and areas that come with Service Desk out-of-box are captured in this table.

Categories, subcategories, and areas

Category	Subcategory	Area
complaint	service delivery	availability
complaint	service delivery	functionality
complaint	service delivery	performance
complaint	support	incident resolution quality
complaint	support	incident resolution time
complaint	support	person
compliment	service delivery	availability
compliment	service delivery	functionality
compliment	service delivery	performance
compliment	support	incident resolution quality
compliment	support	incident resolution time
compliment	support	person
incident	access	authorization error
incident	access	login failure
incident	data	data or file corrupted
incident	data	data or file incorrect
incident	data	data or file missing
incident	data	storage limit exceeded
incident	facilities	hardware failure
incident	facilities	miscellaneous
incident	facilities	supplies
incident	failure	error message
incident	failure	function or feature not working
incident	failure	job failed
incident	failure	system down
incident	hardware	hardware failure
incident	hardware	missing or stolen
incident	performance	performance degradation
incident	performance	system or application hangs
incident	security	security breach
incident	security	security event/message
incident	security	virus alert
problem	access	authorization error

problem	access	login failure
problem	data	data or file corrupted
problem	data	data or file incorrect
problem	data	data or file missing
problem	data	storage limit exceeded
problem	facilities	hardware failure
problem	facilities	miscellaneous
problem	facilities	supplies
problem	failure	error message
problem	failure	function or feature not working
problem	failure	job failed
problem	failure	system down
problem	hardware	hardware failure
problem	hardware	missing or stolen
problem	performance	performance degradation
problem	performance	system or application hangs
problem	security	security breach
problem	security	security event/message
problem	security	virus alert
request for administration	grant access	grant access
request for administration	other	other
request for administration	password reset	password reset
request for change	service portfolio	new service
request for change	service portfolio	upgrade / new release
request for information	general information	general information
request for information	how to	how to
request for information	status	status
service catalog	service catalog	service catalog
service request	Employee Off-boarding	
service request	Employee On-boarding	
service request	Hardware	
service request	Request for Administration	
service request	Request for Information	
service request	Software	

support catalog	support catalog	support catalog

9.26.3.5. Escalate Interaction

Depending on your selection, the **Continue** button opens different fulfillment records as follows:

- Escalate Interaction - Incident
  You can escalate Interactions in the following category to Incident:
  - incident
- Escalate Interaction - RFC
  You can escalate Interactions in the following category to RFC:
  - request for change

  Option 1: If change is open on Category in settings, when escalating an interaction to an RFC, a list of change categories is displayed from which the category for the new change request can be selected.
  Option 2: If change is open on Change Model in settings, when escalating an interaction to an RFC, a list of Change Models is displayed from which the Change Model for the new change request can be selected.
- Escalate Interaction - Request
  You can escalate Interactions in the following categories to Request:
  - service request
  - request for information
  - request for administration

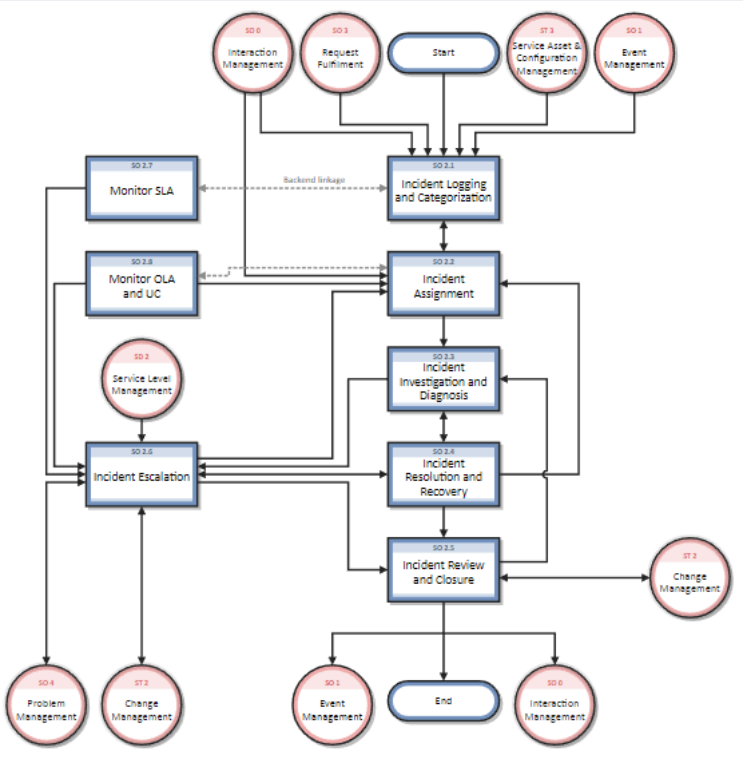
  Option 1: If request is open on Category in settings, when escalating an interaction to a request, a list of request categories is displayed from which the category for the new request can be selected.
  Option 2: If request is open on Request Model in settings, when escalating an interaction to a request, a list of Request Models is displayed from which the Request Model for the new request can be selected.
- Escalate Interaction - Problem
  You can escalate Interactions in the problem category to Problem.

9.26.4. Changes in other modules

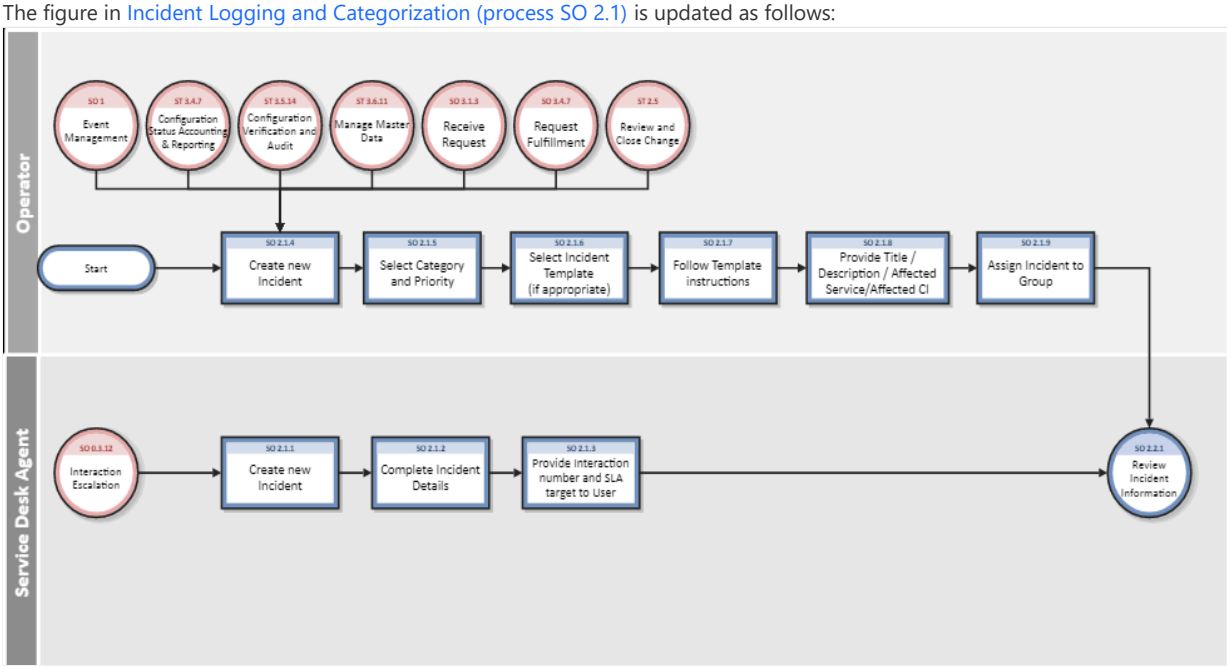
The section describes the process changes in other modules if you use the Streamlined Interaction solution.

SO 2 Incident Management

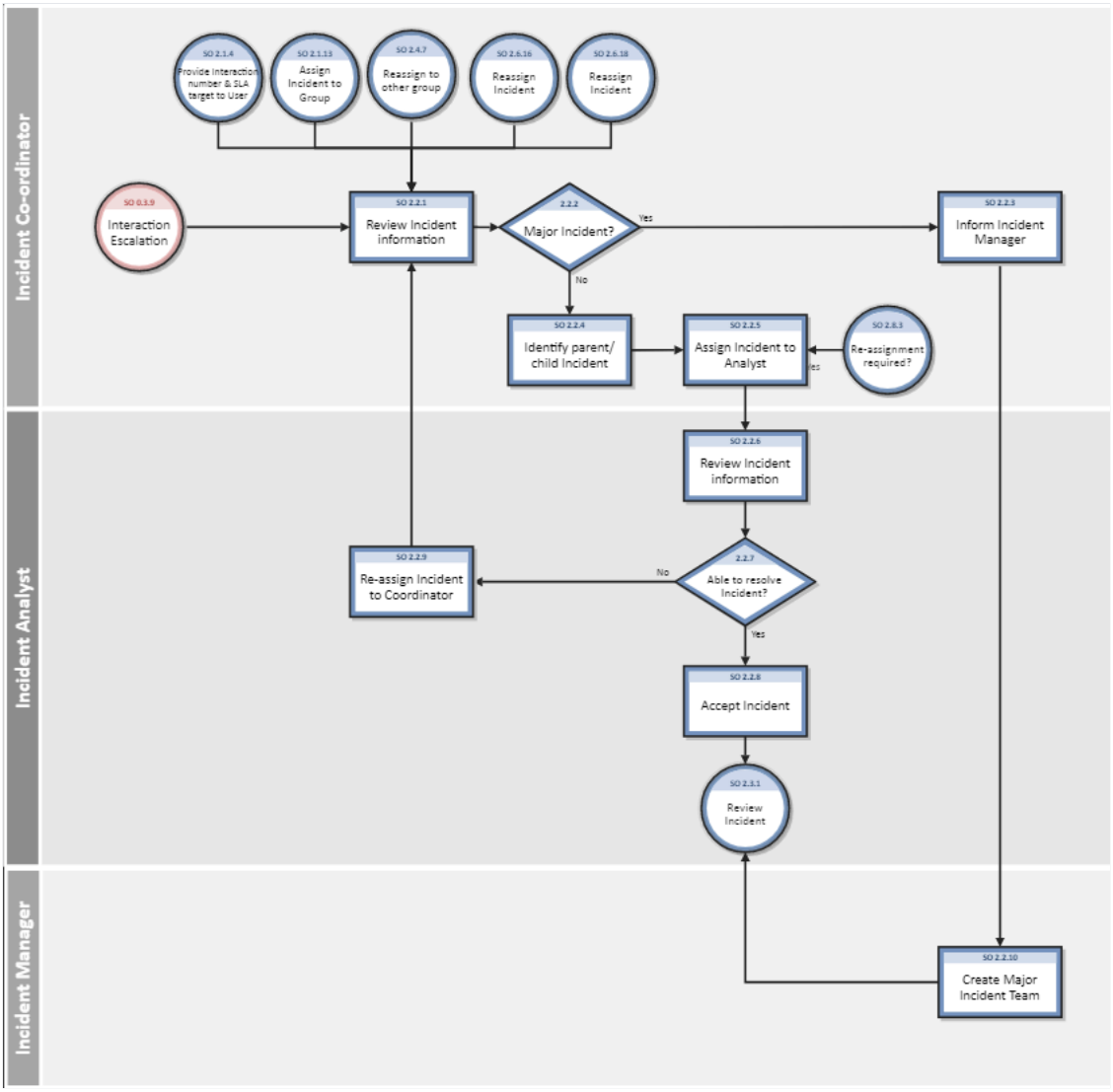
- The figure in Incident Management Overview is updated as follows:



- The figure in Incident Logging and Categorization (process SO 2.1) is updated as follows:



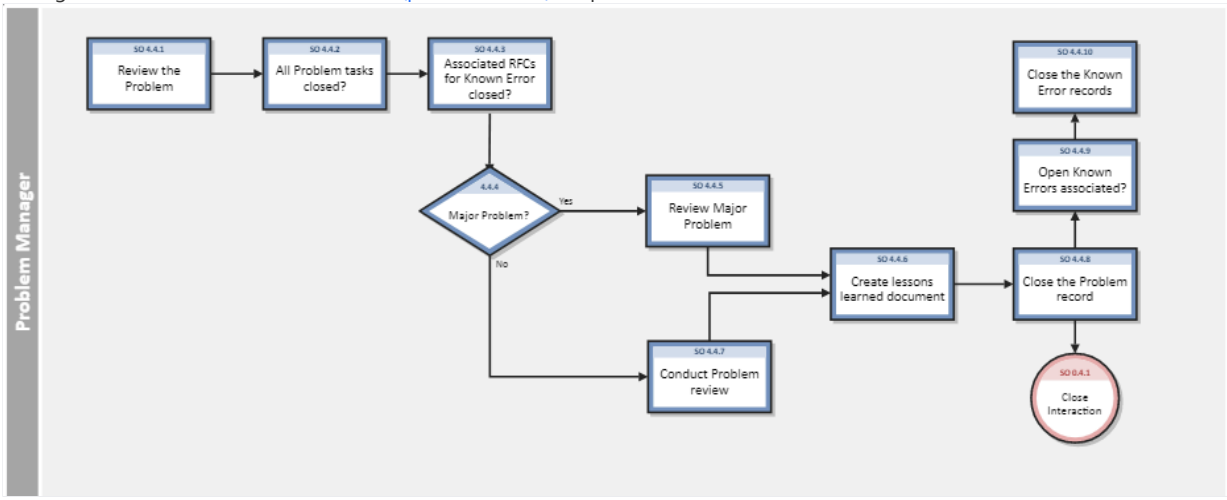
- The figure in Incident Assignment (process SO 2.2) is updated as follows:



- The Complaint Handling (process SO 2.9) section doesn't apply to the Streamlined Interaction solution.

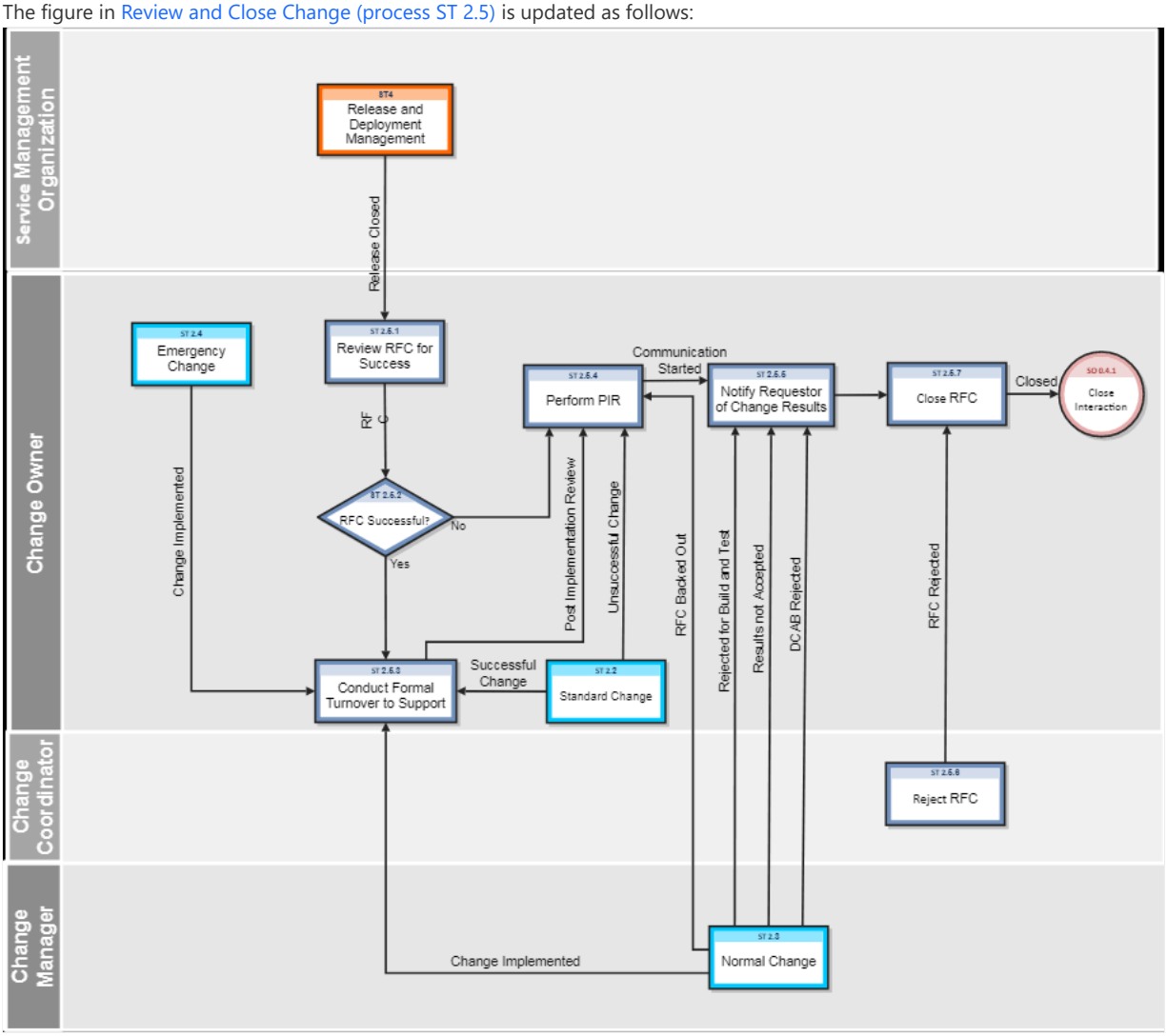
SO 4 Problem Management

- The figure in Problem Review and Closure (process SO 4.4) is updated as follows:



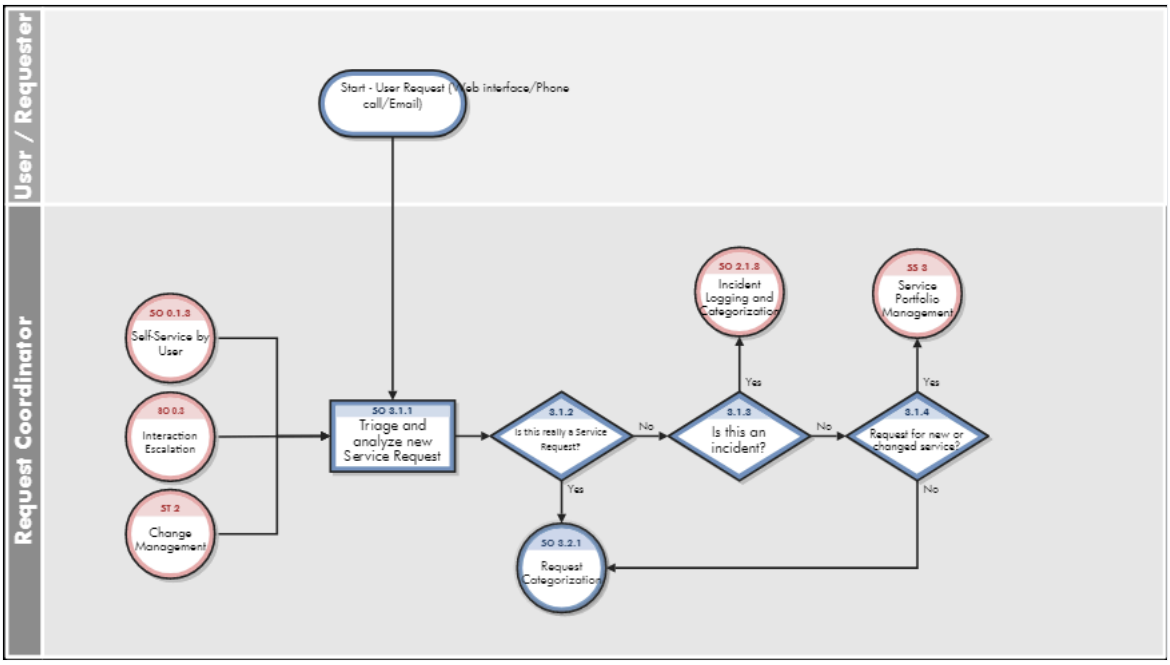
ST 2 Change Management

- The figure in Review and Close Change (process ST 2.5) is updated as follows:

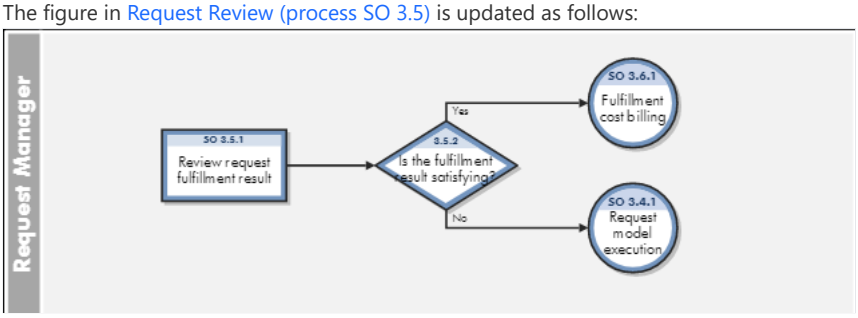


SO 3 Request Fulfillment

- The figure in Receive Request (process SO 3.1) is updated as follows:



- The figure in Request Review (process SO 3.5) is updated as follows:



10. Database reports

Service Manager database report works with third-party reporting tools (such as Microsoft Power BI) to provide visualized reports generated from your queried database view for the purposes of analysis, data discovery, and so on. To create reports from the database view, you need a working knowledge of your DBMS or RDBMS, the third-party reporting tools, and a general understanding of Service Manager applications and utilities.

For additional information about any of these concepts, see the following:

- For information about a particular third-party reporting tool, refer to the vendor documentation for your reporting tool.
- For information about a particular RDBMS, refer to the vendor documentation for your database type.
- For database configuration, see Prepare your RDBMS.
- For information about Service Manager administration and configuration, and customizing the Service Manager product, see this help center.

Note

Note

- Database report is accessible to the users with the SysAdmin capability word.
- Database report works with Service Manager applications version 9.7x. However, if you perform a fresh installation of version 9.7x binaries and keep using an earlier applications version but still hope to use Crystal Report to generate operational reports, you need to keep the old binaries and then connect to the legacy integration of the earlier version. Refer to the Service Manager Help Center for detailed instructions.

Caution

Caution

Running complex SQL statements as typical for the reporting purpose against the Service Manager production database may result in serious performance issues like table locks and deadlocks. Therefore we strongly recommend setting up a replica of the Service Manager production database synchronized by the RDBMS mechanism and connecting the reporting application to this replica database. In this configuration, reporting SQL statements no longer cause locks that will block Service Manager production.

Add a new database report

To add a new database report, follow these steps:

1. From the System Navigator, click **Tailoring** > **Database Report**.
2. Type a unique name for the new report in the **Report Name** field.
3. Type a unique name for the new report database view in the **Database View Name** field. Only English characters, numbers, and underscores are supported.
   If you leave the **Database View Name** field empty, the system automatically generates a name by adding the **V_** prefix to the report name. Moreover, if you include other characters in the report name, the system replaces these characters with underscores.
4. Specify one or more files on the **File Names** tab.
   To add a file to the report, either type the file name directly or select a file from the drop-down list.
   To remove a file from the report, either click the file name and then press Backspace or Delete.
5. Click **Add** to add this new report.
6. Configure the required information for the report.

Field/Tab	Description
Field Names	Displays only the fields that have column names in the files. If there are duplicated field column names in different files, the alternate column name automatically combines the file name and the column name. To include a field in the database view, select **Yes** in the **Field Selected** drop-down list of this field.
Relations	Defines the relations between different source and target files and fields. You can combine the data from multiple files and fields into a single report. **Note:** If you have selected multiple files on the **File Names** tab, you must define the corresponding relations for each selected file on the Relations tab. **Note:** You can't use the fields in the alias table to define the relations.
Filters	Add one or more conditions to filter the data in the database view. Click **Add New Filter Criteria**. Populate all the fields as required, and then click **Finish** to add the criterion to the Conditions list. Click **Add New Compound** to add a compound rule to a filter that has more than one criterion. **Note:** You can't use the fields in the alias table to define the filter criteria.
SQL	Displays the SQL scripts to create a database view. When you add or update a report, the system generates the SQL scripts automatically.
Select All Fields	Select this checkbox to include all fields in the database view. After you select this checkbox, your setting in the **Field Selected** drop-down list of a field on the **Field Names** tab will be overridden.
Use Alias Fields	After you select **Yes** in the **Field Selected** drop-down list of a field, select this checkbox to include its related alias fields in the database view.

7. (Optional) Click **Refresh from DBDICT** to fetch the latest data update from the database dictionary to the current report.
8. Click **Save** and **OK**.
9. Click **Create Database View** to execute the SQL scripts and create a database view.

> **Note**
> Note A database view will not be created automatically after you add or update a database report. You must click **Create Database View** to create a database view.

Modify an existing database report

To modify an existing database report, follow these steps:

1. From the System Navigator, click **Tailoring** > **Database Report**. Click **Search** to display the current report list.
2. Click a report to display the detailed configuration.
3. Configure the required report view information.
4. Click **Save** to save your modifications, and then click **Create Database View**. The existing database view with the same name will be overwritten.

> **Note**
> Note Removing a report will not remove its relevant database view.

Example: Generating an Incident by CI Types report with Microsoft Power BI

This section provides an example tutorial for creating reports using Microsoft Power BI. You will need a Power BI license or a trial version to apply the steps in this section. This type of report can also be created using other BI tools, which may also suit your business needs.

Suppose you might want to create a report that displays the details of all incident records by CI types. Follow these steps to create a report with data extracted from the probsummary table and the device table:

1. Download and install Power BI Desktop from Microsoft website.
2. Sign in to Service Manager as a system administrator.
3. From the System Navigator, click **Tailoring** > **Database Report**. Click **Search** to display the current report list.
4. Type Incident CI Used in the Report Name field, type probsummary and device in the File Names fields respectively, and then click **Add**.
5. Click the **Field Names** tab, and then review the fields to be used to generate the database report view. If a field is unnecessary, select **No** in the Field Selected drop-down list of this field.
6. Click the **Relations** tab, and then specify the values as follows:

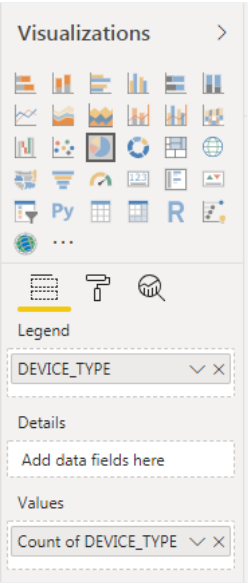
Source File Name	Source Field Name	Target File Name	Target Field Name
probsummary	logical.name	device	logical.name

7. Click the **Filters** tab, and then click **Add New Filter Criteria**. Specify the values as follows:

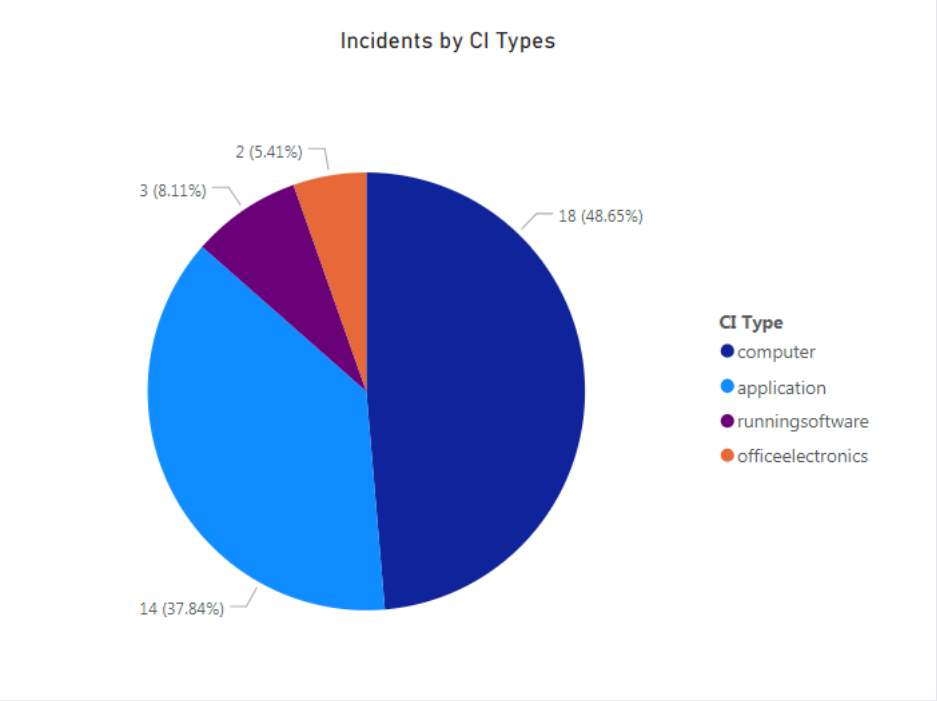
File Name	Field Name	Comparison	Value
device	istatus	Equals	In Use

8. Click **Finish** and **Save**.
9. Click **Create Database View**. The system opens a dialog that may resemble the following:
   The database view V_INCIDENT_CI_USED will be replaced if it exists. Do you want to process?
   Click **Yes**.
   Note: The SQL scripts in the demo data is generated based on the SQL server. If you are working with the Oracle or PostgreSQL database, you can click **Create database view** directly and the system will automatically convert the SQL scripts to the SQL execution of the corresponding database.
10. Launch Power BI, and then connect to the Service Manager database.
11. Select the **V_INCIDENT_CI_USED** view, and then click **Load**.

12. Click **Pie chart**, set the value for the Legend section to **DEVICE_TYPE**, and then set the value for the Values section to **Count of DEVICE_TYPE**. See the following screenshot as an example:



13. The report will be generated. See the following screenshot as an example:



11. Service Manager third-party components

This topic lists the third-party components in Service Manager and their versions.

Server

Module	Name	Version
Server	asm-3.2.jar	3.2
Server	bcmail-fips-1.0.4.jar	1.0.4
Server	bcpkix-fips-1.0.6.jar	1.0.6
Server	bc-fips-1.0.2.4.jar	1.0.2.4
Server	boost	1.80
Server	cglib-2.2.jar	2.2
Server	commons-beanutils-1.9.4.jar	1.9.4
Server	commons-codec-1.15.jar	1.15
Server	commons-collections-3.2.2.jar	3.2.2
Server	commons-collections4-4.1.jar	4.1
Server	commons-email-1.5.jar	1.5
Server	commons-email-1.3.jar	1.3
Server	commons-io-2.11.0.jar	2.11.0
Server	commons-lang-2.6.jar	2.6
Server	commons-lang3-3.4.jar	3.4
Server	commons-logging-1.2.jar	1.2
Server	coreutils	5.3.0
Server	diffutils	2.8.7-1
Server	ecj-4.4.jar	4.4
Server	ews-java-api-2.0.jar	2.0
Server	ezmorph-1.0.6.jar	1.0.6
Server	FastInfoset-1.1.1.jar	1.1.1
Server	google-collections-1.0.jar	1.0
Server	gson-2.9.0.jar	2.9.0
Server	htmlparser-1.6.jar	1.6
Server	httpclient-4.5.13.jar	4.5.13
Server	httpcore-4.4.4.jar	4.4.4
Server	idm-client-core-1.10.37-4.jar	1.10.37-4
Server	icu4c	70.1
Server	itext-2.1.7.js2.jar	2.1.7.js2
Server	jackson-annotations-2.15.0.jar	2.15.0
Server	jackson-core-2.15.0.jar	2.15.0

Server	jackson-databind-2.15.0.jar	2.15.0
Server	jackson-module-jaxb-annotations-2.15.0.jar	2.15.0
Server	jasperreports-6.13.0.jar	6.13.0
Server	javax.mail-1.6.2.jar	1.6.2
Server	jaxb-api-2.3.1.jar	2.3.1
Server	jaxb-impl-2.3.3.jar	2.3.3
Server	jcommon-1.0.21.jar	1.0.21
Server	jfreechart-1.0.17.jar	1.0.17
Server	jgroups-5.2.2.jar	5.2.2
Server	JRE	Zulu11.54+25-CA (Win/Linux)
Server	joda-time-2.8.jar	2.8
Server	json-20231013.jar	20231013
Server	json-lib-2.4.jar	2.4
Server	jsonic-1.3.0.jar	1.3.0
Server	jsoup-1.15.4.jar	1.15.4
Server	jsr311-api-1.0.jar	1.0
Server	kuromoji-ipadic-0.9.0.jar	0.9.0
Server	langdetect-1.1-20120112.jar	1.1-20120112
Server	log4j-api-2.20.0.jar	2.20.0
Server	log4j-core-2.20.0.jar	2.20.0
Server	log4j-slf4j-impl-2.20.0.jar	2.20.0
Server	log4j-jcl-2.20.0.jar	2.20.0
Server	libjpeg	9d
Server	libpng	1.6.37
Server	libtiff	4.1.0
Server	libZip	1.9.2
Server	log4j-2.17.1.jar	2.17.1
Server	lucene-analyzers-common-7.3.0.jar	7.3.0
Server	lucene-analyzers-kuromoji-7.3.0.jar	7.3.0
Server	lucene-core-7.3.0.jar	7.3.0
Server	lucene-demo-7.3.0.jar	7.3.0
Server	mimepull-1.9.3.jar	1.9.3
Server	mmseg4j-core-1.10.0.jar	1.10.0
Server	Mozilla Nspr	Mozjs-50.0.2

Server	openldap	2.4.59
Server	opennlp-tools-1.8.4.jar	1.8.4
Server	opennlp-uima-1.8.4.jar	1.8.4
Server	opencv	3.4.3
Server	openssl	1.0.2zf
Server	openldap	2.4.42
Server	oro-2.0.8.jar	2.0.8
Server	pcre2	10.40
Server	PostgreSQL	14.5
Server	saaj-api-1.3.5.jar	1.3.5
Server	saaj-impl-1.5.0.jar	1.5.0
Server	securityContext-05.00.001.jar	05.00.001
Server	slf4j-api-2.0.7.jar	2.0.7
Server	SpiderMonkey	50.0.2
Server	spring-2.5.jar	2.5
Server	spring-aop-5.2.22.jar	5.2.22
Server	spring-beans-5.2.22.jar	5.2.22
Server	spring-context-5.2.22.jar	5.2.22
Server	spring-core-5.2.22.jar	5.2.22
Server	spring-expression-5.2.22.jar	5.2.22
Server	spring-web-5.2.22.jar	5.2.22
Server	tika-core-2.7.0.jar	2.7.0
Server	Tomcat	9.0.63
Server	wink-common-1.4.jar	1.4
Server	wink-json-provider-1.4.jar	1.4
Server	wink-server-1.4.jar	1.4
Server	wink-spring-support-1.4.jar	1.4
Server	XercesC	3.2.3
Server	Zlib	1.2.12

Web tier

Module	Name	Version
Web tier	bc-fips-1.0.2.4.jar	1.0.2.4
Web tier	btf-1.3.jar	1.3
Web tier	castor-core-1.4.1.jar	1.4.1
Web tier	castor-xml-1.4.1.jar	1.4.1
Web tier	commons-beanutils-1.9.4.jar	1.9.4
Web tier	commons-codec-1.4.jar	1.4
Web tier	commons-collections-3.2.2.jar	3.2.2
Web tier	commons-collections4-4.2.jar	4.2
Web tier	commons-compress-1.21.jar	1.21
Web tier	commons-configuration-1.10.jar	1.10
Web tier	commons-digester-2.1.jar	2.1
Web tier	commons-fileupload-1.3.3.jar	1.3.3
Web tier	commons-io-2.7.jar	2.7
Web tier	commons-lang-2.6.jar	2.6
Web tier	commons-lang3-3.4.jar	3.4
Web tier	commons-logging-1.1.jar	1.1
Web tier	commons-math3-3.6.1.jar	3.6.1
Web tier	concurrent-1.3.4.jar	1.3.4
Web tier	curvesapi-1.06.jar	1.06
Web tier	FastInfoset-1.2.15.jar	1.2.15
Web tier	ecj-4.4.jar	4.4
Web tier	genericHandlersFramework-2.0.jar	2
Web tier	httpclient-4.5.13.jar	4.5.13
Web tier	httpcore-4.4.3.jar	4.4.3
Web tier	istack-commons-runtime-3.0.7.jar	3.0.7
Web tier	idm-client-core-1.10.37-4.jar	1.10.37-4
Web tier	itext-2.1.7.js7.jar	2.1.7
Web tier	jackson-annotations-2.15.0.jar	2.15.0
Web tier	jackson-core-2.15.0.jar	2.15.0
Web tier	jackson-coreutils-2.0.jar	2.0
Web tier	jackson-databind-2.15.0.jar	2.15.0
Web tier	jackson-jaxrs-base-2.12.7.jar	2.12.7

Web tier	jackson-jaxrs-json-provider-2.12.7.jar	2.12.7
Web tier	jackson-module-jaxb-annotations-2.15.0.jar	2.15.0
Web tier	jai_codec-1.1.2.jar	1.1.2
Web tier	jai_core-1.1.2.jar	1.1.2
Web tier	jai_imageio-1.0.jar	1
Web tier	jakarta.xml.soap-api-1.4.2.jar	1.4.2
Web tier	jakarta.activation-1.2.2.jar	1.2.2
Web tier	jakarta.activation-api-1.2.1.jar	1.2.1
Web tier	jakarta.annotation-api-1.3.5.jar	1.3.5
Web tier	jakarta.xml.bind-api-2.3.5.jar	2.3.5
Web tier	jasperreports-6.13.0.jar	6.13.0
Web tier	javahelp-2.0.02.jar	2.0.02
Web tier	javax.activation-api-1.2.0.jar	1.2.0
Web tier	javax.inject-1.jar	1
Web tier	jaxb-api-2.3.1.jar	2.3.1
Web tier	jaxb-runtime-2.3.1.jar	2.3.1
Web tier	jaxen-1.1.3.jar	1.1.3
Web tier	jboss-jaxrs-api_2.1_spec-2.0.2.Final.jar	2.0.2.Final
Web tier	jboss-logging-3.4.3.Final.jar	3.4.3.Final
Web tier	jcl-over-slf4j-1.7.30.jar	1.7.30
Web tier	jcommon-1.0.21.jar	1.0.21
Web tier	jcs-1.3.jar	1.3
Web tier	jdom2-2.0.6.jar	2.0.6
Web tier	jettison-1.5.3.jar	1.5.3
Web tier	jfreechart-1.0.17.jar	1.0.17
Web tier	js-1.7R4.jar	1.7
Web tier	json-patch-1.13.jar	1.13
Web tier	jsoup-1.15.4.jar	1.15.4
Web tier	jstl-api-1.2.1.jar	1.2.1
Web tier	jstl-impl-1.2.4.jar	1.2.4
Web tier	less-compile-1.2.jar	1.2
Web tier	log4j-api-2.17.1.jar	2.17.1
Web tier	log4j-core-2.17.1.jar	2.17.1
Web tier	log4j-slf4j2-impl-2.17.1.jar	2.17.1

Web tier	log4j-web-2.17.1.jar	2.17.1
Web tier	mimepull-1.9.3.jar	1.9.3
Web tier	mlibwrapper_jai-1.1.2.jar	1.1.2
Web tier	msg-simple-1.2.jar	1.2
Web tier	mxgraph-core.jar	4.2.0
Web tier	neva-unknown.jar	unknown
Web tier	opencsv-1.5.jar	1.5
Web tier	poi-4.1.2.jar	4.1.2
Web tier	poi-ooxml-4.1.2.jar	4.1.2
Web tier	poi-ooxml-schemas-4.1.2.jar	4.1.2
Web tier	reactive-streams-1.0.4.jar	1.0.4
Web tier	resteasy-core-spi-5.0.7.Final.jar	5.0.7.Final
Web tier	resteasy-core-5.0.7.Final.jar	5.0.7.Final
Web tier	resteasy-jackson2-provider-5.0.7.Final.jar	5.0.7.Final
Web tier	resteasy-jaxb-provider-5.0.7.Final.jar	5.0.7.Final
Web tier	resteasy-servlet-initializer-5.0.7.Final.jar	5.0.7.Final
Web tier	saaj-api-1.3.5.jar	1.3.5
Web tier	saaj-impl-1.5.2.jar	1.5.2
Web tier	securityContext-05.00.001.jar	05.00.001
Web tier	sitemesh-2.3.jar	2.3
Web tier	slf4j-api-1.7.30.jar	1.7.30
Web tier	SparseBitSet-1.2.jar	1.2
Web tier	spring-aop-5.3.27.jar	5.3.27
Web tier	spring-beans-5.3.27.jar	5.3.27
Web tier	spring-context-5.3.27.jar	5.3.27
Web tier	spring-core-5.3.21.jar	5.3.21
Web tier	spring-expression-5.3.27.jar	5.3.27
Web tier	spring-jcl-5.3.27.jar	5.3.27
Web tier	spring-security-config-5.8.3.jar	5.8.3
Web tier	spring-security-core-5.8.3.jar	5.8.3
Web tier	spring-security-crypto-5.8.3.jar	5.8.3
Web tier	spring-security-web-5.8.3.jar	5.8.3
Web tier	spring-web-5.3.27.jar	5.3.27
Web tier	ssce-5.10.jar	5.10

Web tier	stax-ex-1.8.jar	1.8
Web tier	thumbnailator-0.4.8.jar	0.4.8
Web tier	tika-core-2.7.0.jar	2.7.0
Web tier	txw2-2.3.1.jar	2.3.1
Web tier	EXT JS	4.1
Web tier	EXT JS	3.4
Web tier	amplifyjs	1.1.0
Web tier	jquery.selectBoxIt	v3.8.1
Web tier	jquery	v3.6.3
Web tier	jqueryui	v1.13.2
Web tier	com.github.taylorhakes:promise-polyfill	8.1.0
Web tier	com.github.indutny:elliptic	6.4.1
Web tier	mxgraph	4.2.0
Web tier	CKEditor	4.21.0
Web tier	codemirror	5.56.0M1
Web tier	react	17.0.2

Windows client

Module	Name	Version
Windows client	Eclipse RCP	4.24-202206070700
Windows client	bc-fips-1.0.2.4.jar	1.0.2.4
Windows client	CKEditor	4.21.0
Windows client	commons-lang-2.2.jar	2.2
Windows client	commons-logging-1.1.jar	1.1
Windows client	concurrent-1.3.4.jar	1.3.4
Windows client	FastInfoset-1.2.15.jar	1.2.15
Windows client	jackson-annotations-2.15.0.jar	2.15.0
Windows client	jackson-core-2.15.0.jar	2.15.0
Windows client	jackson-databind-2.15.0.jar	2.15.0
Windows client	jackson-module-jaxb-annotations-2.15.0.jar	2.15.0
Windows client	jai_codec-1.1.2.jar	1.1.2
Windows client	jai_core-1.1.2.jar	1.1.2
Windows client	jai_imageio-1.0.jar	1
Windows client	jakarta.activation-1.2.2.jar	1.2.2
Windows client	jcommon-1.0.21.jar	1.0.21
Windows client	jcs-1.3.jar	1.3
Windows client	jdom2-2.0.6.jar	2.0.6
Windows client	jettison-1.5.3.jar	1.5.3
Windows client	jfreechart-1.0.17.jar	1.0.17
Windows client	js-1.6R4.jar	1.6R4
Windows client	jsoup-1.15.4.jar	1.15.4
Windows client	ktable.jar	unknown
Windows client	log4j-api	2.17.1
Windows client	log4j-core	2.17.1
Windows client	mimepull-1.9.3.jar	1.9.3
Windows client	mlibwrapper_jai-1.1.2.jar	1.1.2
Windows client	saaj-api-1.3.5.jar	1.3.5
Windows client	saaj-impl-1.5.2.jar	1.5.2
Windows client	stax-ex-1.8.jar	1.8
Windows client	serializer-2.7.2.jar	2.7.2
Windows client	servlet-4.0.6.jar	4.0.6

Windows client	wintertreesc-5.10.jar	5.1
Windows client	xalan-2.7.2.jar	2.7.2
Windows client	jre	zulu17.42.19-17.0.7-win_x64

Chat server

Module	Name	Version
Chat server	Openfire	4.7.5

Mobility

Module	Name	Version
Mobility	activation-1.1.1.jar	1.1.1
Mobility	bc-fips-1.0.2.4.jar	1.0.2.4
Mobility	bsf-springsecurity204-authn-05.10.310.jar	05.10.310
Mobility	checker-qual-3.12.0.jar	3.12.0
Mobility	commons-codec-1.13.jar	1.13
Mobility	commons-collections4-4.4.jar	4.4
Mobility	commons-configuration-1.8.jar	1.8
Mobility	commons-fileupload-1.3.3.jar	1.3.3
Mobility	commons-io-2.11.0.jar	2.11.0
Mobility	commons-lang-2.6.jar	2.6
Mobility	commons-lang3-3.1.jar	3.1
Mobility	commons-logging-api-1.0.4.jar	1.0.4
Mobility	commons-math3-3.6.1.jar	3.6.1
Mobility	error_prone_annotations-2.11.0.jar	2.11.0
Mobility	failureaccess-1.0.1.jar	1.0.1
Mobility	FastInfoset-1.2.18.jar	1.2.18
Mobility	genericHandlersFramework-2.0.1.jar	2.0.1
Mobility	guava-31.1-jre.jar	31.1
Mobility	idm-client-core-1.10.37-4.jar	1.10.37-4
Mobility	istack-commons-runtime-3.0.11.jar	3.0.11
Mobility	j2objc-annotations-1.3.jar	1.3
Mobility	jackson-annotations-2.15.0.jar	2.15.0
Mobility	jackson-core-2.15.0.jar	2.15.0
Mobility	jackson-databind-2.15.0.jar	2.15.0
Mobility	jackson-module-jaxb-annotations-2.15.0.jar	2.15.0
Mobility	jakarta.activation-1.2.2.jar	1.2.2
Mobility	jakarta.xml.bind-api-2.3.3.jar	2.3.3
Mobility	jakarta.xml.soap-api-1.4.2.jar	1.4.2
Mobility	javax.activation-api-1.2.0.jar	1.2.0
Mobility	jaxb-api-2.3.1.jar	2.3.1
Mobility	jaxb-runtime-2.3.3.jar	2.3.3
Mobility	jcl-over-slf4j-1.7.30.jar	1.7.30

Mobility	jempbox-1.8.16.jar	1.8.16
Mobility	jsoup	1.16.1
Mobility	jsr305-3.0.2.jar	3.0.2
Mobility	jstl-api-1.2.1.jar	1.2.1
Mobility	jstl-impl-1.2.4.jar	1.2.4
Mobility	listenablefuture-9999.0-empty-to-avoid-conflict-with-guava.jar	9999.0
Mobility	log4j-api-2.17.1.jar	2.17.1
Mobility	log4j-core-2.17.1.jar	2.17.1
Mobility	log4j-slf4j-impl-2.17.1.jar	2.17.1
Mobility	log4j-web-2.17.1.jar	2.17.1
Mobility	metadata-extractor-2.18.0.jar	2.18.0
Mobility	poi-4.1.2.jar	4.1.2
Mobility	saaj-api-1.3.5.jar	1.3.5
Mobility	saaj-impl-1.5.2.jar	1.5.2
Mobility	securityContext-05.00.001.jar	05.00.001
Mobility	slf4j-api-1.7.30.jar	1.7.30
Mobility	SparseBitSet-1.2.jar	1.2
Mobility	spring-aop-5.3.27.jar	5.3.27
Mobility	spring-beans-5.3.27.jar	5.3.27
Mobility	spring-context-5.3.27.jar	5.3.27
Mobility	spring-context-support-5.3.27.jar	5.3.27
Mobility	spring-core-5.3.27.jar	5.3.27
Mobility	spring-expression-5.3.27.jar	5.3.27
Mobility	spring-jcl-5.3.27.jar	5.3.27
Mobility	spring-mobile-device-1.1.2.RELEASE.jar	1.1.2
Mobility	spring-oxm-5.3.27.jar	5.3.27
Mobility	spring-security-config-5.8.3.jar	5.8.3
Mobility	spring-security-core-5.8.3.jar	5.8.3
Mobility	spring-security-crypto-5.8.3.jar	5.8.3
Mobility	spring-security-web-5.8.3.jar	5.8.3
Mobility	spring-web-5.3.27.jar	5.3.27
Mobility	spring-webmvc-5.3.27.jar	5.3.27
Mobility	stax-ex-1.8.3.jar	1.8.3
Mobility	txw2-2.3.3.jar	2.3.3

Mobility	tika-core-1.28.5.jar	1.28.5
Mobility	tika-parsers-1.28.5.jar	1.28.5
Mobility	xmpcore-6.1.11.jar	6.1.11
Mobility	Sencha Touch	2.4.2



© Copyright 2026 Open Text

For more info, visit https://docs.microfocus.com